ARCH LINUX 分册教材

第六册

桌面环境与图形栈

X/Wayland、窗口管理器、桌面环境、字体与输入法

依据 Arch Linux 中文维基全量条目编写

共 9 册 · 997 篇 · 安装 / 系统 / 网络 / 硬件 / 桌面 / 软件 / 开发 / 安全

从「能开机」到「能干活」

装完系统，命令行已能用，屏幕却常停在黑屏或陌生的 TTY 上。本册补齐这一层：把内核、显卡驱动、显示协议、窗口管理与桌面会话串成一条能自我诊断的链路。读完应能做到：面对黑屏、分辨率异常、撕裂、缩放模糊、外接显示器不亮，知道先怀疑哪一层、按什么顺序验证；挑一套合适的会话组合——完整桌面环境，或窗口管理器加零散组件——让配置可复现、可迁移；在中文环境里收尾，字体回退正确，输入法在各类应用里都能出字。

技术重点有三处：显卡与驱动的选择（不同厂商、混合图形与 PRIME 类方案、硬件视频加速）；显示协议的分岔（X 服务器与 Wayland 各自的模型、配置入口与能力边界）；会话之上的组织层（桌面环境、合成器、显示管理器、Freedesktop.org 规范、字体与输入法）。

在九册中的位置

第一册讲发行版哲学与社区，第五册讲硬件与外设，本册正处在两者交汇：借前者「自己选择并为选择负责」的态度挑桌面，用后者关于显卡、显示器与输入设备的结论定驱动和缩放。第二册的包管理与启动知识是前提，显示管理器本就是启动流程的下游；第三册的内核模块、日志与电源管理是排查图形问题的工具箱；第四册的网络决定装驱动、取字体是否顺手。往后看，第七册的软件与应用、第八册的开发工具、第九册的安全与加密都以本册为地基：没有图形栈，浏览器、播放器与游戏无从谈起；没有可靠的会话与锁屏，安全只是纸面。

怎么读

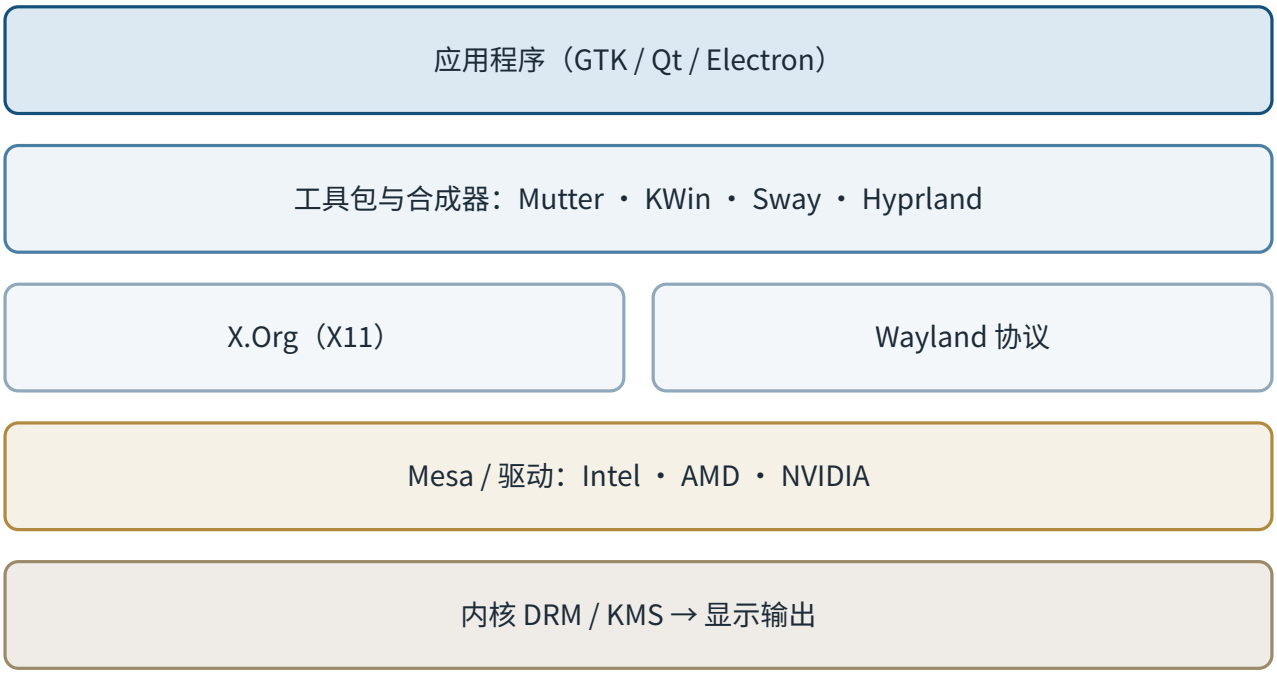
主线是「先驱动、再会话、后打磨」：先读图形与显卡章节，确认驱动与混合图形的搭配；再在 X 与 Wayland 之间走通一条路；然后进入桌面环境，或窗口管理器加显示管理器的组合；最后收尾字体与输入法。GNOME、KDE、Xfce、LXDE 这几章按需取一两个即可，其余桌面环境宜当资料检索。偏爱键盘与自建配置的读者，可把堆叠式、平铺式、动态窗口管理器与启动器几章连读，它们共享「组件自己拼」的思路。Freedesktop.org 的几节建议「用到再查」——桌面集成与门户的规范总要落到具体组件上才有意义。Xorg 命令、通知、无障碍等零散章节浏览标题后跳读即可。

前置知识与常见误区

前置只有两条：能在终端里工作并读懂系统日志，改配置前先备份。误区有五个：其一，把「装桌面环境」当成「有图形界面」，忽略驱动与会话，装完仍进不去；其二，把 Wayland 当作 X 的直系替代，忽略截图、全局快捷键、屏幕共享与输入法的差异，指望兼容层包办；其三，一出问题就归咎发行版，实际多是驱动版本组合与配置相互作用；其四，字体与输入法拖到最后才配，而框架在会话启动时已定，改完常要重启会话；其五，照抄他人配置而不看显示器、DPI 与显卡差异，或同时用手写配置和桌面设置面板改同一项。

图形栈

X11 与 Wayland 的分岔



X11 是外部服务端协议；Wayland 由合成器直接管理客户端，路径更短

从应用到显示输出的完整图形链路

目　录

第 1 章　桌面环境（23 节）

1.1 Common Desktop Environment　1.2 Lumina　1.3 TheShell　1.4 Moksha　1.5 Liri　1.6 Cutefish　1.7 Equinox 桌面环境　1.8 Sugar　1.9 Trinity　1.10 Polybar　1.11 深度桌面环境　1.12 Budgie　1.13 切换深色模式　1.14 Mako　1.15 COSMIC　1.16 Tint2　1.17 LXQt　1.18 Cinnamon　1.19 桌面环境比较　1.20 MATE　1.21 桌面环境　1.22 Enlightenment　1.23 Pantheon

第 2 章　图形（21 节）

2.1 Intel GMA 3600　2.2 Nvidia-xrun　2.3 AMDGPU PRO　2.4 Uvesafb　2.5 混合图形技术　2.6 图形处理器　2.7 虚拟现实(VR)　2.8 外接显卡　2.9 Nouveau　2.10 NVIDIA Optimus　2.11 PRIME　2.12 ATI　2.13 硬件视频加速　2.14 HiDPI　2.15 NVIDIA/提示和技巧　2.16 Bumblebee　2.17 AMDGPU　2.18 NVIDIA　2.19 NVIDIA/故障排除　2.20 Intel 图形处理器　2.21 AMD Catalyst

第 3 章　X 服务器（13 节）

3.1 Xprofile　3.2 Xhost　3.3 Idesk　3.4 Xephyr　3.5 Xcompmgr　3.6 光标主题　3.7 Dunst　3.8 Redshift　3.9 XScreenSaver　3.10 Picom　3.11 多显示器　3.12 FreeNX　3.13 Xorg

第 4 章　GNOME（8 节）

4.1 GNOME/Gedit　4.2 GNOME/Web　4.3 GNOME/Flashback　4.4 GNOME/文件　4.5 GNOME/Keyring　4.6 GNOME/提示与技巧　4.7 GDM　4.8 GNOME

第 5 章　Wayland 合成器（8 节）

5.1 Wayfire　5.2 Hyprlock　5.3 MangoWM　5.4 Niri　5.5 Dwl　5.6 Weston　5.7 Hyprland　5.8 Sway

第 6 章　堆叠式窗口管理器（8 节）

6.1 Blackbox　6.2 Twm　6.3 2bwm　6.4 IceWM　6.5 Fluxbox　6.6 Compiz　6.7 FVWM　6.8 Openbox

第 7 章　平铺式窗口管理器（8 节）

7.1 Notion　7.2 Herbstluftwm　7.3 平铺式窗口管理器比较　7.4 Ratpoison　7.5 Qtile　7.6 Bspwm　7.7 Ion3　7.8 Xmonad

第 8 章　显示管理器（7 节）

8.1 Ly　8.2 Console TDM　8.3 Plasma Login Manager　8.4 XDM　8.5 SLiM　8.6 Greetd　8.7 SDDM

第 9 章　输入法（7 节）

9.1 Gcin　9.2 Smart Common Input Method　9.3 Rime　9.4 IBus　9.5 输入法　9.6 Fcitx　9.7 Fcitx 5

第 10 章　Freedesktop.org（6 节）

10.1 XDG Autostart　10.2 XDG 用户目录　10.3 Xdg-utils　10.4 Xdg-menu　10.5 XDG 桌面门户　10.6 XDG MIME 应用程序

第 11 章　字体（6 节）

11.1 Java 运行环境字体配置　11.2 微软字体　11.3 字体配置/中文　11.4 字体配置　11.5 字体配置/示例　11.6 字体

第 12 章　KDE（3 节）

12.1 Amarok　12.2 KDE Wallet　12.3 KDE

第 13 章　LXDE（3 节）

13.1 LXDM　13.2 LXDE　13.3 PCManFM

第 14 章　Suckless（3 节）

14.1 Slock　14.2 Dmenu　14.3 St

第 15 章　Wayland（3 节）

15.1 Kanshi　15.2 通用Wayland会话管理器　15.3 Wayland

第 16 章　Xfce（3 节）

16.1 Xfwm　16.2 Thunar　16.3 Xfce

第 17 章　动态窗口管理器（3 节）

17.1 Dwm　17.2 Awesome　17.3 I3

第 18 章　应用程序启动器（3 节）

18.1 KRunner　18.2 Gmrun　18.3 Rofi

第 19 章　Xorg 命令（2 节）

19.1 Xinit　19.2 Xrandr

第 20 章　图形用户界面（2 节）

20.1 显示管理器　20.2 Gamescope

第 21 章　Xorg commands（1 节）

21.1 Xcalc

第 22 章　无障碍（1 节）

22.1 使用无障碍功能安装 Arch Linux

第 23 章　深度（1 节）

23.1 Deepin-wine

第 24 章　窗口管理器（1 节）

24.1 窗口管理器

第 25 章　通知（1 节）

25.1 桌面通知程序

桌面环境

桌面环境把窗口管理、面板、通知、主题与应用打包成一个会话，日常手感由此决定。先读《桌面环境》弄清会话构成与「自己打造」的路径，再用《桌面环境比较》圈定候选，最后精读一款主流实现：Cinnamon、MATE、LXQt、Pantheon 篇幅最长、排错最全；CDE、Lumina、Trinity 等小型或历史项目尝鲜即可。

常见误解是把 Polybar、Tint2、Mako 当作桌面环境——它们只是面板与通知组件，需在已有会话里自行拼接；《切换深色模式》也说明主题能否跟随取决于工具包与应用本身。

1.1　Common Desktop Environment

难度 入门 | **篇幅** 1.7 KB | **所属** 桌面环境

本节将讲到

- 安装
- 用法
- 参考

Common Desktop Environment 是基于 Motif 小部件工具箱的 Unix 和 OpenVMS 桌面环境。 它是 UNIX98 工作站产品标准的一部分，并且长期以来就是与商用 Unix 工作站相关联的“经典” Unix 桌面。 尽管是旧版环境，但它仍然对 Linux 系统提供支持。

安装

基本 CDE 系统是通过 cdesktopenv-gitAUR AUR 包安装的。

用法

dtlogin

cdesktopenv-gitAUR 软件包提供了 `dtlogin` systemd 服务，该服务在启动时将启动 CDE 登录管理器。

xinit

可以使用 `startx` 直接启动 CDE（安装 xorg-xinit包）：

```
$ export PATH=$PATH:/usr/dt/bin
$ export LANG=C
$ startx /usr/dt/bin/Xsession
```

参考

- Common Desktop Environment Wiki

1.2 Lumina

难度 入门 | **篇幅** 2.1 KB | **所属** 桌面环境

本节将讲到

- 安装
- 启动 Lumina 桌面环境
- 报告错误

警告：Lumina DE 主要针对 FreeBSD，而不是 Linux。

Lumina Desktop Environment（简称 Lumina）是一种轻型，符合 XDG，BSD 许可的桌面环境，其重点在于简化完成工作的能力，同时最大程度地减少系统开销。 从版本 0.8.0+ 开始，它需要 Qt 5，Fluxbox 窗口管理器，并使用少量 X 实用程序来执行各种任务，例如 numlockx 和 xscreensaver。

Lumina 的功能包括：

- 很少的系统开销。
- 智能的“favorites”系统，用于创建应用程序，文件和目录的快捷方式。
- 通过“ Insight”文件管理器进行 ZFS 文件还原功能。
- 桌面系统是基于插件的，类似于 Android 或其他现代操作系统。
- 可以轻松访问特定于操作系统的功能，例如屏幕亮度，音量和电池状态。

安装

从 AUR 安装 lumina-desktopAUR 或 lumina-desktop-gitAUR。

启动 Lumina 桌面环境

通过 .xinitrc 启动

```
 ~/.xinitrc
exec start-lumina-desktop
```

有关信息，请访问 xinitrc wiki 页面。

报告错误

错误应报告到 GitHub 上的 Lumina 存储库。

1.3 TheShell

难度 入门 | **篇幅** 2.3 KB | **所属** 桌面环境

本节将讲到

- 安装
- 开始

- 问题解决
- 提示技巧

theShell 是一个使用 Qt 写成的桌面环境。

安装

你可以安装稳定版本： theshellAUR。

theShell 有一个不稳定且未经充分测试的版本，叫做 “blueprint”，它可以用 theshell-blueprintAUR 安装。

开始

在 X Session 中用 `ts-startsession` 启动 theShell。或使用显示管理器并使用“theShell”作为会话。

问题解决

按下 “Adjust Screen Resolution” 却没有反应……

请确保 kscreen包 和 kde-cli-tools包 工具已经安装。

更多问题，参见 theshell issues。

提示技巧

安装稳定版本和 “blueprint” 版本

theShell 和 theShell Blueprint 可以被一起安装。只需重命名二进制和初始化脚本（`theshell-b` 和 `init-theshell-b` 是一对好名字），并将它们放在二进制文件夹中，重命名 `.desktop` 文件，将其更改为启动新的初始化脚本并将其放在 xsessions 文件夹中。

使用另一个窗口管理器

在 theShell 设置中，*Danger* 类别下，在 *Window Manager Command* 下输入用于启动窗口管理器的命令。

1.4 Moksha

难度 入门 | **篇幅** 2.6 KB | **所属** 桌面环境

本节将讲到

- 安装
- 其他应用
- 参见

Moksha 是 2014 年发布的桌面环境。它是 Enlightenment 17 的分支，目前在基于 Ubuntu 的 Bodhi Linux 中用作默认桌面环境。与 *extra* 存储库中可用的更高版本的 Enlightenment 版本相比，它的主要优点是它更稳定，并且内存占用显着减少。

安装

您可以使用软件包 moksha-gitAUR 安装 Moksha。

其他应用

对于其他功能，可能需要安装以下组件：

- **moksha-module-deskshow** — Deskshow Moksha 模块

: https://github.com/JeffHoogland/moksha-modules-extra || moksha-module-deskshow-gitAUR

- **moksha-module-emprint** — Emprint Moksha 模块

: https://github.com/JeffHoogland/moksha-modules-extra || moksha-module-emprint-gitAUR

- **moksha-modules-extra** — 额外的杂项 Moksha 模块

: https://github.com/JeffHoogland/moksha-modules-extra || moksha-modules-extra-gitAUR

- **moksha-radiance-theme** — 来自 git 的 Moksha Radiance

: https://github.com/JeffHoogland/MokshaRadiance || moksha-radiance-theme-gitAUR

参见

- 官方网站
- 源代码
- 如何在 Arch Linux 上安装 Moksha - 论坛

1.5 Liri

难度 入门 | **篇幅** 2.7 KB | **所属** 桌面环境

本节将讲到

- 安装
- 启动桌面
- 另请参见

Liri 是具有现代设计和功能的桌面环境。 Liri 是 Hawaii，Papyros 和 Liri Project 之间的合并。

警告：Liri 目前处于 Alpha 阶段，可能存在问题。

安装

安装元软件包 liri-git-metaAUR 以获取所有 LiriOS 生态系统。

或者，可以使用 flatpak包 安装它。 请参阅 LiriOS 下载页面中的 Flatpak 说明。

启动桌面

使用图形登录管理器运行

支持 Wayland 的登录管理器（例如 SDDM 和 GDM）可以运行 Liri 会话。

从 tty 运行

登录到 tty 并输入：

```
$ liri-session
```

会话管理器会自动检测硬件，并相应地运行合成器。

但是，可以强制使用一种模式，例如，强制使用嵌套模式并在 Weston 内部运行：

```
$ liri-session -platform wayland
```

注意：不要使用 `~/.xinitrc` 启动 Liri。Xinit 通常用于启动 Xorg ，但是 Liri 使用 Wayland，这是一种更新的图形协议。

使用 systemd 用户会话运行

首先，您需要按照 systemd/用户中的说明使用 systemd 用户会话设置 D-Bus。然后启用 `liri.service` 用户单元

每次您要启动 Liri 会话时：

```
$ systemctl --user isolate liri.target
```

目前已知 *logind* 集成不适用于 systemd 用户会话，因此某些功能可能无法正常工作。systemd 用户会话是相当新的，开发人员正在使用它测试 Liri。

另请参见

- 官方网站
- Liri 的 GitHub 页面

1.6 Cutefish

难度 入门 | **篇幅** 2.9 KB | **所属** 桌面环境

本节将讲到

- 安装
- 启动
- 配置
- 参见

Cutefish 是一个桌面环境，最初是为CutefishOS 项目编写的，它注重简洁、美观和实用。它使用Qt5框架编写，提供简单、通用的外观和感觉。

安装

可通过安装 cutefish包组 来安装Cutefish。

启动

Cutefish 可以通过显示管理器启动，也可以从控制台手动启动。Cutefish 附带的显示管理器是SDDM。

图形化

启用 `sddm.service`.

手动

可以通过xinitrc或直接在startx命令行中调用 `cutefish-session` 。

配置

Cutefish 的配置主要可以在 Dock 上的设置程序中完成。

键绑定

Cutefish 本身不支持按键绑定，但使用sxhkd守护进程可以添加自定义按键绑定。

它可以通过自定义会话启动：

```
/usr/bin/cutefish-session-sxhkd
#!/bin/bash
nohup sxhkd &
cutefish-session
/usr/share/xsessions/cutefish-session-sxhkd.desktop
[Desktop Entry]
Type=Application
Exec=cutefish-session-sxhkd
TryExec=cutefish-session-sxhkd
Name=Cutefish Desktop (with sxhkd)
Keywords=session-sxhkd
Comment=session-sxhkd
```

如果你是从archinstall安装的cutefish,修改这个之后有可能进不去，譬如sddm,可以不用修改

```
$HOME/.config/sxhkd/sxhkdrc
ctrl + alt + t
    cutefish-terminal
```

```
super + d
    cutefish-launcher
```

终端颜色方案

Cutefish Terminal 本身没有内置的配色方案切换器。不过，我们可以编辑源代码 加入我们自己的配色方案。

首先使用ABS获取 cutefish-terminal包 的PKGBUILD。

在构建步骤中或手动地编辑主题，即 `Cutefish-Dark.colorcheme` 或 `Cutefish-Light.colorcheme` 中的 `*/path/to/the/sources/*qmltermwidget/lib/color-schemes/`。

为避免每次更新都重复这一步骤，最好创建一个补丁，以便更新后重复使用。

参见

- CutefishOS
- CutefishOS Github

1.7 Equinox 桌面环境

难度 入门 | **篇幅** 3.0 KB | **所属** 桌面环境

本节将讲到

- 安装
- 启动桌面环境
- 应用

Equinox Desktop Environment（EDE）是一种桌面环境，旨在简单，轻巧和快速。

它主要提供最基本的功能：窗口管理器（默认情况下使用 PekWM），包含面板的简单 GUI，监视可移动媒体的守护程序和通知守护程序。除此之外，只剩下一些配置程序，计算器等。从 2.0 版开始，EDE 遵循 FreeDesktop.org 准则。

与其他桌面环境不同，EDE 基于 FLTK 工具包。它特别适合于 RAM 很少的系统，或者想要完全自定义系统并需要功能和应用程序尚未膨胀的 GUI 的用户。

安装

可以使用 edeAUR 包从 AUR 安装 EDE。

或者，您可以从#自定义存储库中获取它。

自定义存储库

要启用 EDE 的非官方用户存储库，请将以下行添加到 `/etc/pacman.conf` 中：

```
[ede]
SigLevel = Optional
Server = http://ede.elderlinux.org/repos/archlinux/$arch
```

接下来刷新软件包列表，并通过 pacman 安装 EDE 软件包：

- *edelib*
- *ede-common*
- *ede*
- *ede-wallpapers* (optional)

启动桌面环境

要启动 EDE，可以使用显示管理器或使用 startx。如果选择后者，只需将以下内容写入用户的 `.xinitrc`：

```
exec startede
```

应用

由于 EDE 是基本的桌面环境，因此您甚至必须自己添加最常见的应用程序，例如文件管理器或编辑器。您有选择的自由。

由于这个桌面环境的性质，安装轻量级软件显然很有意义。但是，没有太多的 FLTK 应用程序可用，因此您可能不得不依靠第二个工具包，例如 GTK。

一些建议

- 文件管理器：PCManFM
- 浏览器：Dillo 或 Midori
- 编辑器：Leafpad
- 终端模拟器：Xterm

1.8 Sugar

难度 入门 | **篇幅** 3.1 KB | **所属** 桌面环境

本节将讲到

- 安装
- 启动 Sugar
- 另请参见

作为 OLPC 倡议的产物，Sugar) 是一个类似于 KDE 和 GNOME 的桌面环境，但面向儿童和教育。

Sugar 有一个特殊的分类法来命名其系统的各个部分。图形界面本身构成了 **Glucose**（葡萄糖）组。这是安装 Sugar 时理应符合预期的核心系统。但要真正使用这个桌面环境，您需要 activities（一些应用程序）。基础的 activities 是

Fructose（果糖）的一部分。然后，**Sucrose** 由 Glucose 和 Fructose 组成，代表应该作为基本 Sugar 桌面环境分发的内容。额外的 activities 是 **Honey**（蜜糖）的一部分。请注意，Ribose（底层操作系统）在这里被 Arch 取代。

安装

- 对于核心系统 (*Glucose*)，安装 sugar包。它提供了图形界面和桌面会话，但它本身并不是很有用。
- sugar-fructose包组 软件包组包含了基本 activities (*Fructose*)，包括网络浏览器、文本编辑器、媒体播放器和终端模拟器。
- sugar-runner包 软件包提供了一个帮助脚本，可以在另一个桌面环境中启动 Sugar，或者直接从命令行启动。

从 Activity 库安装

Sugar Activity Library 提供了许多 Activity Bundles，打包为带有“.xo”扩展名的 zip 文件。这些包可以从 Sugar 下载并安装到用户的目录中，但是安装并不能确保满足依赖关系。因此，不推荐此安装 activities 的方法，因为它们可能由于缺少依赖项而无法启动。常用依赖：

- 对于web activities，从官方存储库安装 webkit2gtk包。
- 对于基于 GTK 2 的 activities，从 AUR 安装 sugar-toolkit-gtk2AUR。

为了检查 activities 无法启动的原因，请查看位于 `~/.sugar/default/logs/[app_id]-1.log` 的log文件。

启动 Sugar

Sugar 可以使用 display manager 以图形方式启动，也可以从控制台手动启动。

图形方式

从显示管理器的会话菜单中选择 *Sugar* 。

手动

如果 sugar-runner包 已安装，Sugar 可以使用 `sugar-runner` 命令启动。

另一种方法是将 `exec sugar` 添加到 `~/.xinitrc` 文件中。然后，可以使用 `startx` 命令启动 Sugar（有关详细信息，请参阅 xinitrc）。设置好`~/.xinitrc`文件后，也可以安排为 Start X at login。

另请参见

- Sugar 官网
- Activities for Sugar

1.9 Trinity

难度 入门 | **篇幅** 3.8 KB | **所属** 桌面环境

本节将讲到

- 安装
- 启动
- 锦囊妙计
- 问题解决
- 参见

翻译状态：

- 本文（或部分内容）译自 Trinity，最近一次同步于 2024-6-21，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

来自 Trinity Desktop Environment (TDE) 项目页面：

: TDE 是一个完整的软件桌面环境，设计用于类 Unix 操作系统，面向喜欢传统桌面模式的计算机用户，是免费/自由软件。TDE 诞生于 2010 年，是 KDE 3.5 的一个分叉。TDE 是一个完全独立的项目，有自己的个性和开发团队，适用于各种 Linux 发行版、BSD 和 DilOS。

TDE 仍然依赖于旧版本的 Qt，由于该 Qt 版本已经过时，他们现在自己维护该版本。

Trinity 的应用程序和小程序也可用于其他桌面环境。

安装

二进制软件包

从 trinity 资源库中安装 *tde-tdebase* 软件包以获得基本的 Trinity 环境，或安装 *tde-meta* 软件包以获得更完整的 Trinity 桌面环境。

如果在升级过程中出现任何错误，请按照pacman/软件包签名#导入非官方密钥的方法添加 `0x8685AD8B` 密钥。

从源代码构建

Trinity 软件包存储库包含大多数 Trinity 软件包的 PKGBUILD 文件，这些文件位于 "arch "文件夹中。

源代码在 git 仓库中。有关克隆的更多信息，请访问 GIT 信息页面。

如何构建 TDE页面中说明了建议的构建顺序。

注意：请参阅DeveloperWiki:Building in a clean chroot，在 chroot 中构建软件包。

启动

手动

从 Linux 控制台启动 Trinity

```
$ startx /opt/trinity/bin/starttde
```

更多信息请参阅xinit。

图形化

*tde-tdebase*自带 TDE 显示管理器。要在启动时启动它，启用 `tdm.service`。

锦囊妙计

Trinity "Kicker" 面板和其他桌面环境

要在其他桌面环境中使用 Trinity "kicker "桌面面板和小程序，请创建此脚本并使其可执行。对于 Plasma5，使用"'系统设置 > 启动和关机 > 自动启动 > 添加脚本'"。

```
#!/bin/bash
/opt/trinity/bin/tdeinit
/opt/trinity/bin/kicker
/opt/trinity/bin/tdebuildsycoca --noincremental
```

问题解决

TDE 显示管理器

如果遇到任何问题，可能需要手动配置 `default.target`。有关解决方法，请参见显示管理器#加载显示管理器。

参见

- TDE Git 仓库
- TDE bug追踪
- 邮件列表
- Developers Web
- QT and TQT Tutorials and Documentation
- 怎样构建

1.10 Polybar

难度 入门 | **篇幅** 4.0 KB | **所属** 桌面环境

本节将讲到

- 安装
- 配置
- 参见

polybar 是一种能简单快速地创建状态栏的工具。它可以自定义许多功能模块，比如工作区、日期和音量等等。Polybar 对于一些没有状态栏或者状态栏的功能有限的窗口管理器格外有用，例如 awesome 和 i3。Polybar 也可以用于 Plasma 这样的桌面环境。

安装

安装 polybar包 包，或者安装它的开发版本 polybar-gitAUR。

配置

将配置文件样例 `/etc/polybar/config.ini` 复制到 `$XDG_CONFIG_HOME/polybar/config`。polybar 默认按 `~/.config/polybar/config.ini`、`/etc/xdg/polybar/config.ini` 和 `/etc/polybar/config.ini` 的顺序查找文件，并将最先找到的作为配置文件。

运行 Polybar

Polybar 可以使用以下参数运行：

```
Usage: polybar [OPTION]... [BAR]

  -h, --help                    显示帮助信息并退出
  -v, --version                 显示构建详情并退出
  -l, --log=LEVEL               设置日志的详细级别（默认：notice）
                                LEVEL 可以是：error, warning, notice, info, trace
  -q, --quiet                   保持安静，不显示日志（会覆盖 -l）
  -c, --config=FILE             配置文件的路径
  -r, --reload                  当配置文件被修改后自动重新加载
  -d, --dump=PARAM              输出参数 PARAM 的值并退出
  -m, --list-monitors           输出可用的显示器列表并退出（不含克隆的显示器）
  -M, --list-all-monitors       输出可用的显示器列表并退出（包含克隆的显示器）
  -w, --print-wmname            输出生成的 WM_NAME 并退出
  -s, --stdout                  将数据打印到标准输出，而不是在图形界面中展示
  -p, --png=FILE                运行3秒后将 png 截图保存到 FILE
```

你可能想要在你的窗口管理器的引导程序中启动 Polybar，详见#在窗口管理器中运行。

配置样例

以下是一份基础的配置：

```
[bar/mybar]
modules-right = date

[module/date]
type = internal/date
date = %Y-%m-%d%
```

它定义了一个 bar `mybar` 并在其中加入了一个模块 `date`。

Polybar 默认生成了一个包含很多配置好的模块的配置文件 `/usr/share/doc/polybar/config`。

在窗口管理器中运行

创建一个包含启动逻辑的executable，例如 `$HOME/.config/polybar/launch.sh`：

```
#!/bin/bash

# 终止正在运行的 bar 实例
killall -q polybar
# 如果你所有的 bar 都启用了 ipc，你也可以使用
# polybar-msg cmd quit

# 运行 Polybar，使用默认的配置文件路径 ~/.config/polybar/config.ini
polybar mybar 2>&1 | tee -a /tmp/polybar.log & disown

echo "Polybar launched..."
```

这个脚本会在窗口管理器重启后重新运行 Polybar。

bspwm

如果使用的是 bspwm，将下面内容添加到 `bspwmrc`：

```
$HOME/.config/polybar/launch.sh
```

i3

如果使用的是i3，将下面内容添加到你的 i3 的配置文件：

```
exec_always --no-startup-id $HOME/.config/polybar/launch.sh
```

参见

- Polybar Github Wiki

1.11 深度桌面环境

难度 入门 | **篇幅** 4.4 KB | **所属** 桌面环境

本节将讲到

- 安装
- 启动

- 配置
- 故障排除
- 报告 Bug

深度桌面环境 (Deepin Desktop Environment, DDE) 是 Linux 发行版 Deepin 的桌面环境。

安装

如果你想安装一个最小化的 DDE，安装 deepin包组 组即可。这将安装所有基础组件。

deepin-extra包组 组包含了一些额外的应用程序来提供一个更完整的桌面环境。

要能够使用内置的网络管理，需要安装 networkmanager包 包，并且 `NetworkManager.service` 需要被激活并设为开机自启。

启动

通过显示管理器

无需修改此配置文件，DDE会自己完成更改操作，只需启用 `lightdm.service` 服务即可。

要使用 DDE 默认的 lightdm greeter，你必须修改 `[Seat:*]` 部分下的配置文件以声明：

```
/etc/lightdm/lightdm.conf
[Seat:*]
...
greeter-session=lightdm-deepin-greeter
```

需要注意的是，非 root 用户需要存在有效的主目录才能使 greeter 工作。

通过 xinit

要通过 xinit 使用 DDE，你需要添加以下内容到你的 .xinitrc 文件。

```
~/.xinitrc
exec startdde
```

Execute `startx` or `xinit` to start DDE.

注意：如果你想在开机时自动启动xorg，请参阅 Start X at login .

配置

自定义触摸板手势

deepin官方没有提供自定义触摸板手势，但我们可以通过修改配置文件的方式来自定义触摸板。

配置文件目录：

```
/usr/share/dde-daemon/gesture.json
```

如果想要禁止某项手势，把Action设置为none。例如：

```
{
        "Name": "tap",
        "Direction": "none",
        "Fingers": 3,
        "Action": {
            "Type": "built-in",
            "Action": "none"
        }
}
```

设置完后注销或重启使其生效。

也可以尝试第三方的编辑工具 debuggerx01/dde_gesture_manager。

故障排除

从待机状态恢复后没有背景

由于 NVIDIA 驱动存储其 FBO 的方式，从待机状态下恢复后背景突然消失，仅留下一个可能带有一些颜色噪音的白色屏幕。这个 bug 似乎在 GNOME 上游被修复，但在 DDE 中仍然存在。

一个可能的解决方法是在每次计算机从待机中恢复时重启窗口管理器。完成这项任务的一个方式是创建下列的 systemd 服务

```
/etc/systemd/system/resume@.service
[Unit]
Description=User resume actions
After=suspend.target

[Service]
User=%I
Type=simple
ExecStart=/usr/bin/deepin-wm-restart.sh

[Install]
WantedBy=suspend.target
```

来运行下列的脚本

```
/usr/bin/deepin-wm-restart.sh
#!/bin/bash
export DISPLAY=:0
deepin-wm --replace
```

一旦在正确的目录中创建了这两个文件，要启用这个脚本，只需要运行这些命令：

```
# chmod +x /usr/bin/deepin-wm-restart.sh
# systemctl enable resume@user
# systemctl start resume@user
```

第一个命令使你创建的脚本可执行，第二个命令确保服务始终在开机时启动，最后一个命令使服务立即启动，因此你可以测试解决方法而无需重启。

无线网络无法连接

网络管理器会随即生成并设置MAC地址。这是默认选项，如果要禁止它，将下面的几行加入到网络管理器的配置文件中。

```
/etc/NetworkManager/NetworkManager.conf
[device]
wifi.scan-rand-mac-address=no
```

报告 Bug

任何上游或 Arch 打包相关 bug 应在这里报告。所有的深度开发人员将看见 bug 报告并且尽可能快地解决它们。

1.12 Budgie

难度 入门 | **篇幅** 4.9 KB | **所属** 桌面环境

本节将讲到

- 安装
- 启动
- 使用方法
- 主题
- 配置

Budgie 是一个桌面环境，以前是 Solus 里的一个项目，2022 年 1 月在新成立的 Buddies of Budgie 组织下独立。它使用 GTK 3 作为窗口部件，并使用 C 和 Vala 编写。从 Budgie 10.10 开始，Budgie 作出了一个从 Xorg 改用 wayland 的巨大的跳跃，对 Xorg 的支持被立即放弃了。

安装

安装 budgie包组 软件包组以获取本桌面环境的所有组件，其中包括budgie-desktop包 软件包及用于显示配置的 budgie-desktop-services包 ，用于系统设置的 budgie-control-center包 等运行时依赖。下列软件包是可选安装的，可以为桌面提供更多功能：

budgie-desktop-view包： 桌面图标支持

network-manager-applet包： 任务栏集成的网络管理

blueman包： 蓝牙支持

由 Ubuntu Budgie 开发的其他附属程序包含于 budgie-extras包 软件包中（请注意，此软件包还会修改现有功能，并可能导致问题）。

配置用户目录

Budgie 根据 XDG 用户目录指令创建众所周知的用户目录，如桌面、下载等。登出再登录就可以让 Budgie 菜单检测到配置的变化。

文件管理器

Budgie 没有自带文件管理器，也没有默认安装其他文件管理器。 GNOME文件 (Nautilus) 工作得很好，这些其他的文件管理器也可用。

启动

从显示管理器桌面菜单中选择 *Budgie Desktop* 会话，或者修改 xinitrc 使其包括 Budgie 桌面：

```
~/.xinitrc
export XDG_CURRENT_DESKTOP=Budgie:GNOME
exec budgie-desktop
```

使用方法

Budgie 的侧栏被称作 "Raven"，可以在上面查看通知消息，设置系统和应用音量，查看日历，以及查看正在播放的视频或音乐等。通知部分可以通过按下 `Super+N` 键或点击面板中的通知小程序来访问它。小程序部分可以通过 `Super+A` 快速打开。侧栏也可以通过单击面板中的 "Raven Trigger" 小程序打开，并出现在之前选中的窗格中。

主题

Budgie 使用 GTK 3 作为用户界面，因此被许多 GTK 主题支持。Budgie 还提供了一个内置主题，该主题仅适用于其自身的元素，如面板和 Raven，可以在 Budgie Desktop Setting 中切换。图标主题和光标主题也可以在 Budgie Desktop Setting 中设置。

配置

Budgie 桌面的配置可以通过内置的 Budgie Desktop Setting 来完成，改变系统设置则通过 budgie-control-center包 来完成。

改变按钮布局

使用 dconf包、dconf-editor包 或者 gsettings 可改变窗口按钮布局。

比如：

```
gsettings set com.solus-project.budgie-wm button-layout
'close,minimize,maximize:appmenu'
gsettings set com.solus-project.budgie-helper.workarounds fix-button-layout
'close,minimize,maximize:menu'
```

使用不同的窗口管理器

Budgie 不支持使用不同的窗口管理器。

参见

- Wikipedia:Budgie (desktop environment))
- Budgie 桌面官方网站 (英文)
- Budgie git 官方开发 git 仓库

1.13 切换深色模式

难度 入门 | **篇幅** 5.1 KB | **所属** 桌面环境

本节将讲到

- 工具包
- 应用程序

在明亮模式和黑暗模式/主题之间切换非常有趣。它允许您在日落时切换到黑暗模式，或使用键盘快捷键切换模式。

主题之间的切换可以应用于当前正在运行的程序（可能需要守护程序），或者仅应用于新启动的程序。本文重点介绍在运行时进行切换，因此在使用过程中进行切换会影响当前正在运行的程序。

在明暗模式之间切换需要来自应用程序或应用程序工具包（例如 GTK 和 Qt）的支持。

若你只是想要快速简便地将整个显示风格调暗，可以尝试以下操作： 安装 gnome-themes-extra包， gnome-themes-extra-gtk2AUR， adwaita-qt5-gitAUR adwaita-qt6-gitAUR 以及 qt6ct包。在 qt6ct包 中选择主题为 Adwaita-Dark，并且设置后面这些环境变量：`GTK_THEME=Adwaita:dark`, `GTK2_RC_FILES=/usr/share/themes/Adwaita-dark/gtk-2.0/gtkrc` 以及 `QT_QPA_PLATFORMTHEME=qt5ct`。最后，在“终端”输入这行命令：`gsettings set org.gnome.desktop.interface gtk-theme Adwaita-dark`

工具包

GTK

要更改明/暗模式，您必须更改使用的主题。

大多数主题的确有深色变体，按照惯例，后缀为 `-dark`。例如，默认的 GTK 主题 `Adwaita` 具有变体 `Adwaita-dark`。

要永久更改为深色变体，请参见 GTK#深色版主题。

要立即切换主题以运行程序，需要提供 xsettings 规范的守护程序。

Xfce 中的一个是 xfsettingsd，由 xfce4-settings包 包提供。

要查询当前的 GTK 主题：

```
$ xfconf-query -c xsettings -p /Net/ThemeName
```

设置 GTK 主题：

```
$ xfconf-query -c xsettings -p /Net/ThemeName -s "new-theme"
```

对该条目的更改是即时的，并且会影响所有 GTK 应用程序。

Qt

Qt 具有类似于 GTK 的主题支持。

Qt 应用程序主题化的一种方法是使用 GTK 进行样式设置。GTK 主题的更改也会影响 Qt 应用程序。

应用程序

Firefox

Firefox 自动使用当前的 GTK 主题模式，并相应地调整浏览器的外观。有关更多设置和注意事项，请参见 Firefox#Dark themes。

要聪明地更改 Web 内容，建议使用 Dark Reader 附加组件。

通过将 `Automation` 设定为 `Use system color scheme`，深色阅读器会自动使用深色 GTK 主题激活。

Thunderbird

Thunderbird 符合当前的 GTK 主题，但建议进行一些更改。

请参阅 Thunderbird#Theming tweaks。

Visual Studio Code

要在 Visual Studio Code 中更改主题，此脚本可能会有所帮助。

Alacritty

Alacritty 支持多种自定义配色方案。可以在此处找到配置语法和已发布的配色方案。

要快速更改主题，应声明一个指向每种配色方案的指针，例如 `&black`。然后，您只需设置 `colors: *black` 就可以切换到配色方案。对配置文件的更改是即时的，并且会影响所有当前正在运行的程序。

边框和标题栏以 GTK 为主题。为了遵守 GTK 主题，您应该将 `gtk_theme_variant` 设置为默认值 `None`。

1.14 Mako

难度 入门 | **篇幅** 5.8 KB | **所属** 桌面环境

本节将讲到

- 介绍
- 使用方法

介绍

Mako 是一个轻量级的通知守护进程，您可以阅读 mako(5) 以获取详细信息。 其配置文件位于 `~/.config/mako/config`，用于屏幕显示（OSD）的图标存储在 `~/.config/mako/icons/` 并且应该是 PNG 格式。

使用方法

在本节的其余部分，脚本使用的所有图像都可以从此 GitHub 文件夹获取。

注：由于本页面是从Hyprland页面的Mako一节迁移过来的，所以脚本路径均为 `~/.config/hypr`。

媒体键通知

脚本代码（请按右侧按钮展开阅读）

```
~/.config/hypr/scripts/volume
#!/usr/bin/env bash

iDIR="$HOME/.config/mako/icons"

# Get Volume
get_volume() {
	volume=$(pamixer --get-volume)
	echo "$volume"
}

# Get icons
get_icon() {
	current=$(get_volume)
	if [[ "$current" -eq "0" ]]; then
		echo "$iDIR/volume-mute.png"
	elif [[ ("$current" -ge "0") && ("$current" -le "30") ]]; then
		echo "$iDIR/volume-low.png"
	elif [[ ("$current" -ge "30") && ("$current" -le "60") ]]; then
		echo "$iDIR/volume-mid.png"
	elif [[ ("$current" -ge "60") && ("$current" -le "100") ]]; then
		echo "$iDIR/volume-high.png"
	fi
}

# Notify
notify_user() {
```

```
	notify-send -h string:x-canonical-private-synchronous:sys-notify -u low -i
"$(get_icon)" "Volume : $(get_volume) %"
}

# Increase Volume
inc_volume() {
	pamixer -i 5 && notify_user
}

# Decrease Volume
dec_volume() {
	pamixer -d 5 && notify_user
}

# Toggle Mute
toggle_mute() {
	if [ "$(pamixer --get-mute)" == "false" ]; then
		pamixer -m && notify-send -h string:x-canonical-private-synchronous:sys-
notify -u low -i "$iDIR/volume-mute.png" "Volume Switched OFF"
	elif [ "$(pamixer --get-mute)" == "true" ]; then
		pamixer -u && notify-send -h string:x-canonical-private-synchronous:sys-
notify -u low -i "$(get_icon)" "Volume Switched ON"
	fi
}

# Toggle Mic
toggle_mic() {
	if [ "$(pamixer --default-source --get-mute)" == "false" ]; then
		pamixer --default-source -m && notify-send -h string:x-canonical-private-
synchronous:sys-notify -u low -i "$iDIR/microphone-mute.png" "Microphone Switched OFF"
	elif [ "$(pamixer --default-source --get-mute)" == "true" ]; then
		pamixer -u --default-source u && notify-send -h string:x-canonical-
private-synchronous:sys-notify -u low -i "$iDIR/microphone.png" "Microphone Switched
ON"
	fi
}
# Get icons
get_mic_icon() {
	current=$(pamixer --default-source --get-volume)
	if [[ "$current" -eq "0" ]]; then
		echo "$iDIR/microphone.png"
	elif [[ ("$current" -ge "0") && ("$current" -le "30") ]]; then
		echo "$iDIR/microphone.png"
	elif [[ ("$current" -ge "30") && ("$current" -le "60") ]]; then
		echo "$iDIR/microphone.png"
	elif [[ ("$current" -ge "60") && ("$current" -le "100") ]]; then
		echo "$iDIR/microphone.png"
	fi
}
# Notify
notify_mic_user() {
	notify-send -h string:x-canonical-private-synchronous:sys-notify -u low -i
"$(get_mic_icon)" "Mic-Level : $(pamixer --default-source --get-volume) %"
}

# Increase MIC Volume
inc_mic_volume() {
	pamixer --default-source -i 5 && notify_mic_user
}

# Decrease MIC Volume
dec_mic_volume() {
	pamixer --default-source -d 5 && notify_mic_user
}

# Execute accordingly
if [[ "$1" == "--get" ]]; then
	get_volume
elif [[ "$1" == "--inc" ]]; then
	inc_volume
```

```
elif [[ "$1" == "--dec" ]]; then
	dec_volume
elif [[ "$1" == "--toggle" ]]; then
	toggle_mute
elif [[ "$1" == "--toggle-mic" ]]; then
	toggle_mic
elif [[ "$1" == "--get-icon" ]]; then
	get_icon
elif [[ "$1" == "--get-mic-icon" ]]; then
	get_mic_icon
elif [[ "$1" == "--mic-inc" ]]; then
	inc_mic_volume
elif [[ "$1" == "--mic-dec" ]]; then
	dec_mic_volume
else
	get_volume
fi
```

屏幕背光通知

脚本代码（请按右侧按钮展开阅读）

```
~/.config/hypr/scripts/backlight
#!/usr/bin/env bash

iDIR="$HOME/.config/mako/icons"

# Get brightness
get_backlight() {
	LIGHT="$(cat /sys/class/leds/*::kbd_backlight/brightness)"
	echo "${LIGHT}"
}

# Get icons
get_icon() {
	current="$(cat /sys/class/leds/*::kbd_backlight/brightness)"

	if [[ ("$current" -ge "0") && ("$current" -le "1") ]]; then
		icon="$iDIR/brightness-20.png"
	elif [[ ("$current" -ge "1") && ("$current" -le "2") ]]; then
		icon="$iDIR/brightness-60.png"
	elif [[ ("$current" -ge "2") && ("$current" -le "3") ]]; then
		icon="$iDIR/brightness-100.png"
	fi
}

# Notify
notify_user() {
	notify-send -h string:x-canonical-private-synchronous:sys-notify -u low -i
"$icon" "Keyboard Brightness : $(brightnessctl -d '*::kbd_backlight' g)"
}

# Increase brightness
inc_backlight() {
	brightnessctl -d *::kbd_backlight set 33%+ && get_icon && notify_user
}

# Decrease brightness
dec_backlight() {
	brightnessctl -d *::kbd_backlight set 33%- && get_icon && notify_user
}

# Zero brightness
zero_backlight() {
	brightnessctl -d *::kbd_backlight s 0%
}

# Full brightness
full_backlight() {
```

```
        brightnessctl -d *::kbd_backlight s 100%
}

# Execute accordingly
if [[ "$1" == "--get" ]]; then
        brightnessctl -d '*::kbd_backlight' g
elif [[ "$1" == "--inc" ]]; then
        inc_backlight
elif [[ "$1" == "--dec" ]]; then
        dec_backlight
elif [[ "$1" == "--zero" ]]; then
        zero_backlight
elif [[ "$1" == "--full" ]]; then
        full_backlight

else
        get_backlight
fi
```

键盘（Keyboard）语言通知

脚本代码（请按右侧按钮展开阅读）

```
~/.config/hypr/scripts/lang
#!/usr/bin/env bash

icon="$HOME/.config/mako/icons/language.png"

# Get language
get_lang() {
        lang=$(hyprctl devices -j | gojq -r '.keyboards[] | select(.name == "at-
translated-set-2-keyboard") | .active_keymap' | cut -c 1-2 | tr 'A-Z' 'a-z')
        case $lang in
                en)
                        lang="English language"
                        ;;
                ru)
                        lang="Русский язык"
                        ;;
                uk)
                        lang="Українська мова"
                        ;;
        esac
        echo $lang
}

# Notify
notify-send -h string:x-canonical-private-synchronous:sys-notify -u low -i "$icon"
"$(get_lang)"
```

1.15 COSMIC

难度 进阶 | **篇幅** 6.3 KB | **所属** 桌面环境

本节将讲到

- 安装
- 启动
- 配置
- 参见

翻译状态：

- 本文（或部分内容）译自 COSMIC，最近一次同步于 2026-04-27，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

COSMIC 是一个使用 Rust 编程语言开发的桌面环境，使用**iced** 跨平台 Rust GUI库和 Smithay 作为其合成器 Cosmic-comp 的构建模块。Cosmic-comp 相当于 smithay 自己的 anvil 合成器参考实现，就像 Wayland 项目使用 Weston 作为合成器参考实现一样。它的第一个版本被称为 **Epoch**。

安装

安装 cosmic-session包 软件包或 cosmic包组 组，开发版本可以安装 cosmic-session-gitAUR 获得，它依赖于所有的组件包

当您通过源代码进行构建时， 请注意，依赖项 cosmic-applets-gitAUR 需要至少 8GB RAM。限制连接到单任务是一个构建它的办法。如果你需要重复构建它，设置环境变量 `CARGO_TARGET_DIR` 有助于减少下载次数：

```
MOLD_JOBS=1 CARGO_TARGET_DIR=/tmp/mytarget
```

作为 cosmic-session 的依赖项被引入的组件

COSMIC 由一个合成器（compositor）、一个库（library）和一些小程序（applets）组成，这些组件可以分别作为独立的部分进行安装：cosmic-comp包，cosmic-applets包，cosmic-app-library包，cosmic-bg包，cosmic-icon-theme包，cosmic-launcher包，cosmic-notifications包，用于在屏幕上叠加显示通知或提示信息的 cosmic-osd包，提供底部或顶部的面板（panel）以及 dock 栏的 cosmic-panel包，cosmic-settings包，以及其他的一些包.

独立组件

文本编辑器, cosmic-text-editor包, 文件管理器 cosmic-files包, 终端模拟器, cosmic-terminal包, 媒体播放器 cosmic-player包, 壁纸选择器, cosmic-wallpapers包。

COSMIC文件管理器中的网络共享

为了在COSMIC文件管理器中连接网络共享，需要安装这些相关的GVFS包：

协议	软件包
AppleTalk	gvfs包
File Transfer Protocol (FTP)	gvfs包
Network File System (NFS)	gvfs-nfs包
Server Message Block (SMB)	gvfs-smb包
SSH	gvfs包
WebDAV	gvfs-dnssd包

COSMIC并不提供秘密存储或密钥功能，所以要记住密码，需要安装诸如gnome-keyring之类的密码存储元件。查看GNOME/Keyring获得更多信息。

启动

最简单的方法是通过已安装的Display Manager启动它，如GNOME、KDE Plasma默认使用的GDM、SDDM 等，将会在登录界面发现一个额外的cosmic启动选项。

如果要仅从tty启动，只需运行：

```
$ start-cosmic
```

使用 Cosmic 会话管理器

COSMIC包含了一个基于greetd的显示会话管理器cosmic-greeter包。如果要使用这个dm，请激活`cosmic-greeter.service`服务。

配置

The panel can be used to configure besides using the settings applet, examples of applets are provided.

SSH_AUTH_SOCK 未设置

如果你在使用 Cosmic 登录管理器时看到了 `$SSH_AUTH_SOCK not set` 消息， 启用由依赖项 gcr-4包 提供的 `gcr-ssh-agent.socket` systemd 用户单元。

夜光

因为 COSMIC DE 当前并不包含 `wlr-gamma-control-unstable-v1` 协议（参见 pop-os/cosmic-comp#2059）， wlsunset 和 gammastep 之类的工具并不能在 COSMIC 会话中控制背光和色温。

为了解决这个问题，你可以使用 redshift包 或 gammastep包 之类的工具，在 COSMIC DE 混成器启动前通过 TTY 上的 libdrm 直接调节 DRM 伽马表。另一个可选项是用依赖于 systemd 服务的 drm-colortemp 或 nebula-drm-gitAUR 以自动化这一过程。

参见

- COSMIC Welcome Page
- System76 博客
- Pop!_OS 欢迎界面

1.16 Tint2

难度 进阶 | **篇幅** 8.6 KB | **所属** 桌面环境

本节将讲到

- 安装
- 配置
- 运行tint2
- 启用透明度
- 全屏/覆盖

Tint2 是一个 Linux 下的面板程序，被开发者称为“简洁轻量的面板/任务栏”。虽然不依赖什么软件包，它也可以配置成系统托盘、任务列表、电源监视器甚至时钟；外观配置也很简单。因此，对于默认的 WM 没有面板（比如 Openbox ）的用户来说，Tinit2 是个不错的选择。

安装

tint2 可以通过安装软件包tint2包获得，软件包位于官方软件仓库。

配置

tint2 有个配置文件在`~/.config/tint2/tint2rc`。一个结构化的具有默认设定的配置文件将在tint2第一次运行时被创建。你可以按你的偏好来修改这个文件。在这里能找到tint2的完整的配置文档。可以配置字体，颜色，外观，位置，以及配置文件中更多的其他条目。tint2包现在包含一个GUI的配置工具可以通过下列命令访问：

```
$ tint2conf
```

另外，你可以通过图形界面 tintwizard 编辑你的tint2rc配置文件。

tint2-git(AUR)的面板启动程序

在AUR的tint2-gitAUR的版本中，tint2-git已经可以在面板中添加启动项。你需要手动修改配置文件来实现这个功能，因为 tintwizard 不提供添加启动器支持。

注意：当你手动修改添加启动器后又使用tintwizard来编辑你的tint2的配置文件，tintwizard将删除一切它无法识别的选项。比方说你的启动器。

把如下配置添加到你的tint2配置文件： 找到`# Panel`：

```
# Panel
panel_items = LTSBC
```

找到新的一节`# Launchers`：

```
# Launchers
launcher_icon_theme = LinuxLex-8
launcher_padding = 5 0 10
launcher_background_id = 9
launcher_icon_size = 85
launcher_item_app = /some/where/application.desktop
launcher_item_app = /some/where/anotherapplication.desktop
```

`panel_items`是一个新的配置选项，它定义了tint2用下面的方式以便显示：

L : 显示启动器 **T** : 显示任务栏 **S** : 显示系统托盘 **B** : 显示电池状态 **C** : 显示时钟 **F** : 添加一个可扩展的空白间隙（freespace）。你可以指定多个。如果同时存在“T”则不会生效。（自从0.12版本起） **E** : 添加一个执行器插件。你可以指定多个。（自0.12.4版本起） **P** : 添加一个推按钮。你可以指定多个。（自0.14版本起） **:** : 添加一个分割符。你可以指定多个。（自0.13.0版本起）

OpenBox3的应用菜单

如果运行AUR里的tint2-gitAUR，你可以在tint2面板上创建程序启动器。然而tint2还不支持嵌套菜单，所以没法原生使用程序启动菜单。接下来介绍如何为OpenBox3创建这样一个菜单。

首先，你需要安装openbox包，tint2-gitAUR和xdotool包。然后，为Openbox的右键菜单创建一个键绑定：

```
~/.config/openbox/rc.xml
<keyboard>
 <!-- Keybinding for opening OpenBox menu -->
 <keybind key="C-A-space">
   <action name="ShowMenu"><menu>root-menu</menu></action>
 </keybind>
</keyboard>
```

这就设置了启动OpenBox右键菜单的快捷键`Ctrl+Alt+Space`。测试快捷键是否生效：

```
$ xdotool key ctrl+alt+space
```

如果右键菜单顺利弹出，说明生效了。现在，在`/usr/share/applications/`目录下创建一个`open-openbox-menu.desktop`文件。把这行添加到文件里`Exec=xdotool key ctrl+alt+space`,其中，ctrl+alt+space是你选择的组合键。然后从文件管理器重新打开你的新的`open-openbox-menu.desktop`文件，你将看到右键菜单弹出。最后，将这个文件作为启动器添加到tint2。这样就得到了tint2上的程序启动菜单。如果你需要将菜单固定到某个位置，你可以使用 `xdotool mousemove x y`。你可以创建一个脚本在 `open-openbox-menu.desktop` 中引用它，因为它涉及两个命令。

更多的帮助请参考Openbox Menus来创建你的个性菜单，menumaker可以为大多数（几乎所有）你安装的程序生成一个完满的menu.xml文件。

从0.14版本起，你可以创建按钮。只需将上面实例中的字符串“xdotool key ctrl+alt+space”添加到你想成为开始菜单o操作的按钮动作即可。

音量控制

tint2不附带音量控制小程序，请参见应用程序列表/多媒体#音量控制l

运行tint2

Openbox

你可以简单的通过这条命令运行tint2:

```
$ tint2
```

如果你想在Openbox启动的时候启动tint2，修改~/.config/openbox/autostart添加如下:

```
tint2 &
```

注意：如果在~/.config/openbox没有autostart文件，你可以从/etc/xdg/openbox/autostart复制一份。

OpenBox autostart选项的更多信息参见Openbox help。

Gnome

Gnome 中，底部面板和任务栏被Activities视图所替代。要在原底部面板位置使用tint2，运行

```
# gnome-session-properties
```

并添加

```
# /usr/bin/tint2
```

使tint2作为gnome启动时运行的程序。gnome下次启动时，tint2将自动运行。

i3

在 i3 中，要使用 tinit2 替代 i3status ，请在 i3 配置文件的末尾添加以下行：

```
~/.config/i3/config
exec --no-startup-id tint2
```

并注释或删除同一文件中的任何类似 `bar{status_command i3status}` 的部分。

多个面板

通过使用不同的配置文件执行tint2,可以同时运行多个tint2面板：

```
tint2 -c path/to/first/config/file
tint2 -c path/to/second/config/file
```

启用透明度

tint2支持假透明和真透明。使用哪种透明度由`tint2rc`配置文件中的`disable_transparency`选项控制。

如果您想完全禁用透明度，需要使用`disable_transparency = 1`并将面板背景不透明度设置为100。例如：

```
background_color = #000000 100
```

假透明

对于假透明，您需要设置`disable_transparency = 1`。

假透明会捕获桌面背景的一部分，并将其用作面板背景。因此，在激活tint2之前设置背景图像是很重要的。一个用于Openbox的启动脚本示例可能是（使用Feh设置背景）：

```
...
feh --randomize --no-fehbg --bg-fill ~/Pictures/wallpapers/
(sleep 1 && tint2) &
...
```

真透明

对于真透明，您需要先激活像picom这样的合成器，并设置 `disable_transparency = 0`。

不透明度由tint2配置文件中`background_color`属性的第二个参数控制configuration file。

如果您在运行中进行更改，可能需要重启tint2以使透明度生效。

全屏/覆盖

要强制tint2保持在应用程序（覆盖层）之上，您需要适当地设置panel_layer选项。当您使用 `Alt+Tab` 从全屏窗口切换到普通应用程序时，这可能会很有帮助。在Crunchbang论坛上有关于此的讨论。

```
#Panel
panel_layer = top
strut_policy = follow_size
```

第三方扩展

也可以通过其他应用程序扩展tint2。要添加第三方扩展，请检查官方Wiki的Applets部分。

另请参阅

- Tint2 upstream wiki

1.17 LXQt

难度 进阶 | **篇幅** 10.2 KB | **所属** 桌面环境

本节将讲到

- 安装
- 启动桌面
- 配置
- 提示与技巧
- 故障排除

2013 伊始，洪任諭（“PCMan”）启动了将 LXDE 移植到 Qt 的项目。LXDE-Qt 的首个预览版于 2013 年 7 月 3 日发布。2013 年 7 月 21 日，Razor-qt（一个与 LXDE 设计相似的桌面）与 LXDE 宣布合并。

于是 LXQt （the Lightweight Qt Desktop Environment）诞生了。这个桌面集合了 Razor-qt 和 LXDE 的组件。尽管 LXDE 目前的精力已经集中到 LXQt 的开发上，GTK2 版本的 LXDE 依然在维护。 目前，LXQT已经初步支持Wayland。

安装

首先，安装并配置 Xorg。随后安装 lxqt包组 包组和一个图标主题（如 breeze-icons包 或 oxygen-icons包）。

你还可以安装以下附加功能包：

- **nm-tray** — 一个简单的基于Qt的NetworkManager前端。

: https://github.com/palinek/nm-tray || nm-trayAUR

- **SDDM** — LXQt 推荐的显示管理器。

: https://github.com/sddm/sddm || sddm包

- 如果需要，可安装锁屏组件，比如 slock包 或 xscreensaver包。二者均已包含在 LXQt 安装包中，其它锁屏程序也可能会集成其中。
- 如果您使用 LightDM 作为显示管理器，则可以使用 light-locker。
- 如果要在挂起/休眠时禁用屏幕锁定，请在 *LXQt> 首选项 > LXQt设置 > 会话设置 > 挂起/休眠之前锁定屏幕*下勾选相应复选框。

提示：LXQt 在会话中使用 xdg-utils包 中的 *xdg-screensaver* 作为屏幕锁，它只能与 XScreenSaver 和 xautolock包 配合工作。可以使用它或自行选用其他屏幕锁。例如，要使用 *slock*，可以遵照 Slock#Lock on suspend 并安装打过补丁的 xdg-utils-slockAUR 来获得符合 LXQt 设计期望的感觉。

- 你可以编辑 `~/.config/lxqt/lxqt.conf` 并添加你的锁屏选项而不需要 xautolock包 ， 比如使用i3lock需要添加以下内容：

```
[Screensaver]
lock_command=i3lock
```

- 对于音频，请参阅 General recommendations#声音。
- 某些 LXQt 面板插件需要安装额外的包来发挥功能，请查阅 lxqt-panel包 的可选依赖。

启动桌面

使用 xinit

在 Xinitrc 中添加:

```
~/.xinitrc
exec startlxqt
```

图形界面登入

在显示管理器的桌面菜单中选择 *LXQt Desktop*.

配置

LXQt 通常能通过尝试使用 GUI 应用程序修改其设置。它的配置文件位于 `~/.config/lxqt` 目录中。这个目录会被自动初始化，新用户的默认配置可在 `/usr/share/lxqt` 中找到。

屏幕亮度

如果你发现 LXQt 的亮度快捷键调整屏幕对比度而非亮度，你可以在 LXQt 配置中心 > 快捷键修改来让亮度快捷键调用 xbacklight 命令。

```
xbacklight -inc 10
xbacklight -dec 10
```

如果使用英特尔内核模式设置，xbacklight 将无法正常工作，但是可以使用下列命令代替实现功能

```
pkexec lxqt-backlight_backend --inc
pkexec lxqt-backlight_backend --dec
```

要让快捷键正常工作，可能需要创建两个脚本来上下调节屏幕亮度，然后将快捷键指向脚本路径。

另一个改变屏幕亮度的方法是使用 brightnessctl包。

面板部件

如果无法将“CPU 统计信息”和“系统统计信息”部件添加到面板中，请确保已安装 libstatgrab包 和 libsysstat包。

使用不同的窗口管理器

LXQt 显示一个对话框，用于在首次登录时选择首选的窗口管理器。之后，您可以通过 *会话设置*， `lxqt-config-session` 指定用于 LXQt 的不同窗口管理器;或通过编辑 `~/.config/lxqt/session.conf`。更改以下行：

```
window_manager=current_window_manager
```

改为想选择的某个窗口管理器：

```
window_manager=想选择的窗口管理器
```

自动启动

要在登录的时候启动 X 应用程序，在 LXQt 主菜单中依次点击*首选项* > *LXQt 设置* > *LXQt 会话设置*。此外，也可以通过下面命令启动：

```
$ lxqt-config-session
```

从这个窗口中，点击左侧的*自动启动*。将程序添加到全局自动启动 (程序会在所有实现了 XDG Autostart 规范的会话中启动) 或本地自动启动 (标记为 LXQt 自动启动) (参见 issue 746 获取此选项的 Bug 信息)。对于每个添加的项目，`lxqt-config-session` 会在相应的 XDG Autostart 目录中创建一个 Desktop 条目 (即 *.desktop* 文件)。

*全局自动启动*和 *LXQt 自动启动*间的区别不取决于相应的 .desktop 文件位于什么目录，而是取决于 `OnlyShowIn` 设置。如果 `OnlyShowIn=true`，则将项目视为 *LXQt 自动启动*。此外，如果 `X-LXQt-Module=true`，则项目不会显示在 `lxqt-config-session` 中。

设置环境变量

LXQt 会话的环境变量在*会话设置*中定义。

编辑应用程序菜单

可以通过编辑 `/usr/share/applications/lxqt-*.desktop` 中的 *.desktop* 文件修改菜单，参阅桌面配置项。

主题

摘自官方wiki：

LXQt 主题是 qss样式表。它们要么位于 `/usr/share/lxqt/themes/`，要么位于 `~/.local/share/lxqt/themes`。到目前为止，可以使用以下主题：

- Ambiance
- Arch Colors
- Clearlooks
- Dark
- Frost
- KDE-Plasma
- Kvantum
- Leech
- Light
- Silver
- System (普通小部件主题)
- Valendas

可以在这里关注一些关于主题的讨论。请注意，将“系统”主题与Qt小部件样式Breeze，Oxygen或Qtcurve一起使用会导致任务管理器按钮中出现未剪切的文本。 为了提高某些主题的透明度，需要合成器。只有 Breeze、Oxygen 和 Kvantum 小部件样式支持所有面板菜单的透明度。 如果有人对没有自定义主题的环境外观感兴趣，请创建以下文件（如果<prefix>是 /usr）：

```
mkdir /usr/share/lxqt/themes/notheme
touch /usr/share/lxqt/themes/notheme/lxqt-panel.qss
```

然后在 中 `lxqt-config-appearance` 选择 Notheme。

提示与技巧

屏幕混成器

可以用下列命令将 picom包 等混成器应用添加到自动启动应用当中：

```
picom --vsync -r 12 --no-fading-openclose -b
```

自定义离开菜单

只需将软件各自提供的 `.desktop` 文件复制到 `~/.local/share/applications` 并将其修改为包含 `NoDisplay=true` 指令，就可以自定义“离开”下可用的选项。

参考：#876。

要考虑屏蔽的文件的完整列表包括：

```
lxqt-hibernate.desktop
lxqt-leave.desktop
lxqt-lockscreen.desktop
lxqt-logout.desktop
lxqt-reboot.desktop
lxqt-shutdown.desktop
lxqt-suspend.desktop
```

例子：要移去“休眠”选项：

```
$ mkdir -p ~/.local/share/applications
$ sed '/OnlyShowIn/aNoDisplay=true' </usr/share/applications/lxqt-hibernate.desktop
>~/.local/share/applications/lxqt-hibernate.desktop
```

故障排除

桌面图标聚拢在一块

在桌面上移动图标时，可能会将它们彼此放得太近而粘连在一起。如果实在没办法把它们分开，请在会话设置中停用桌面，移除 `~/.config/pcmanfm-qt/lxqt/desktop-items-0.conf` 后再启动桌面。

用 Xrdp 运行LXQt

使用 Xrdp 运行 LXQt 进行远程登录的好处是快速方便，同时最大限度地减少服务器上的资源消耗。设置 Xrdp 相当轻松，只需要用户调整 `~/.xinitrc` 。由于 LXQt 似乎依赖于某些 D-Bus 服务功能，因此该文件的末尾应该有以下行：

```
exec dbus-run-session -- startlxqt
```

参阅

- LXQt 主页
- LXQt 开发相关信息
- 在deviantART上的LXQt相关内容
- LXQt在GitHub上的维基页面信息

1.18 Cinnamon

难度 进阶 | **篇幅** 14.1 KB | **所属** 桌面环境

本节将讲到

- 安装
- 启动
- 配置
- 提示与技巧
- 故障排除

Cinnamon 是一个提供先进创新的特点和传统的用户体验的 Linux 桌面。不过，其底层技术实质是基于 GNOME 的分支。 截至 2.0 版本，Cinnamon 就已经是一个完整的桌面环境，不仅仅是 GNOME Shell 和 Unity 的前端了。

安装

Cinnamon 可通过 cinnamon包 软件包进行安装。

注意：如果您有英特尔的 GPU，请确保 不要在 xf86-video-intel 驱动下使用 Cinnamon，因为这会导致随机冻屏的问题 (但是仍可移动鼠标)。可使用 modesetting(4) 驱动代替，而不必移除 xf86-video-intel包 (KDE 也 建议这样做)。

Cinnamon 应用

Cinnamon 引入了基于 GNOME 核心应用程序的 X-App，但是这些应用已经进行了修改，可在 Cinnamon、MATE 和 XFCE 上使用；这些应用具有传统的用户界面 (UI)。

应用	GNOME	Cinnamon
文字编辑器	Gedit/Pluma	xed包
图片查看器	Eye of GNOME	xviewerAUR
文档阅读器	Evince/Atril	xreader包

应用	GNOME	Cinnamon
媒体播放器	Totem	xplayerAUR
图像管理器	gThumb	pixAUR

后备模式

当 Cinnamon 发生崩溃事件时, 会激活其*后备*模式。要控制此模式下打开的窗口，需要安装 metacity包 软件包以及 gnome-shell包 以显示任务栏。

启动

图形化登录

在喜欢的显示管理器中选择 *Cinnamon* 或 *Cinnamon (Software Rendering)* 。*Cinnamon* 选项是启用了 3D 加速的会话，一般情况下请使用这个。如果显卡驱动出现问题，可以试试 *Cinnamon (Software Rendering)*，它禁用了 3D 加速功能。

手动启动 Cinnamon

如果您喜欢从控制台启动 Cinnamon，可添加以下行到 Xinitrc:

```
~/.xinitrc
 exec cinnamon-session
```

如果想用 *Cinnamon (Software Rendering)*，请用 `cinnamon-session-cinnamon2d` 代替 `cinnamon-session`。

重启 Cinnamon

从命令行中执行以下行:

```
$ nohup cinnamon --replace > /dev/null 2>&1 &
```

配置

Cinnamon 很容易配置，大部分的配置都可在图形化界面下完成。更多详情可查看以下网站 applets，extensions 和 theming.

Cinnamon 系统设置

cinnamon-settings 可在命令行中启动指定的设置模块。如果后面没有跟随 (正确的) 参数，它将启动 *System Settings* (系统设置)。例如，要启动面板 (panel) 设置：

```
$ cinnamon-settings panel
```

列出所有可用的模块：

```
 $ pacman -Ql cinnamon | awk -F'[_.]' '/cs_.+\.py/ {print $2}'
```

打印机 : 安装 system-config-printer包 进行打印机配置。 **网络** : 要添加网络模块的支持, 请启用 Network Manager。要在 Network Manager 里面保存 Wifi 密码，需要安装 GNOME Keyring。 **蓝牙** : 要添加蓝牙设备支持，请安装软件包 blueberry包。

应用程序和扩展

许多 Cinnamon 的应用程序和扩展可以在 AUR (package search) 中找到，也可以在 Cinnamon 的“小程序”和“拓展”中找到 (*在线获取更多*选项卡中):

```
$ cinnamon-settings applets
$ cinnamon-settings extensions
```

也可以从 Cinnamon spices 下载并手动安装。

注意：如果你安装后没有发现这些拓展或者是应用程序, 按下 `Alt+F2` 并在对话框键入 `r` 重启 Cinnamon。

按下电源按钮睡眠

这是电源按钮的默认行为。如需更改，打开 `cinnamon-settings` 面板进入**系统设置**，点击**电源管理**。更改**按下电源按钮时**选项，选择你所希望使用的操作。

管理 Cinnamon 使用的语言

注意：Cinnamon 控制面板从 2.2 版本开始删除了语言配置模块 \[失效链接 2022-09-17 ⓘ\]

- 要添加删除语言，请查看 Locale。
- 要在启用的语言间切换，请安装软件包 mintlocaleAUR。
- 要在 Cinnamon 中正确显示另外的语言，请安装 cinnamon-translations包 软件包。
- 要修改键盘布局: **系统设置 > 硬件 > 键盘 > 布局**。

使用不同的窗口管理器

Cinnamon 目前不支持这样做。

提示与技巧

创建自定义应用程序

关于创建自定义应用程序，可以在这里找到教程。

默认的桌面背景壁纸路径

当在 Cinnamon 设置自定义的路径的壁纸时，Cinnamon 会将其复制到 `~/.cinnamon/backgrounds`. 因此，每次改变你的壁纸时，你都得再次在设置菜单添加你的墙纸到/或将其复制到 `~/.cinnamon/backgrounds`。

此外， Mint 的官方壁纸在每次大版本发行时都会放出供自由使用。查看 AUR。

显示 "家目录" 和文件系统桌面图标

默认情况下 Cinnamon 在启动时就会启用桌面图标，但是不会在屏幕上显示它们。要显示家目录、文件系统、回收站、已挂载的卷和网络服务的图标，请打开 Cinnamon 的设置，然后点击 桌面。点上要在屏幕上看到的图标的复选框。

菜单编辑器

菜单小程序支持自定义命令。右键单击"菜单"小程序，然后点击"配置..."，然后点击“打开菜单编辑器”。选择一个子菜单（或者创建一个新的子菜单），然后选择“新建项目”。填好名称、命令和备注。如果需要在终端运行，选中“在终端运行”复选框，图形化应用程序不选中“在终端运行”复选框。然后点击”确定“并关闭菜单编辑器。启动器就添加到了菜单。

工作空间

可以将工作空间调度器添加到面板。右键单击面板，然后选择*将小程序添加到面板*选项。将*工作空间切换*小程序添加到面板中。要更改其位置，请在面板上单击鼠标右键，然后将*面板编辑模式*的开关更改为打开 (On)。点击将切换器拖动到所需位置，并在完成后关闭面板编辑模式。

默认情况下，有两个工作空间。要添加更多，请按 `Ctrl+ Alt +`上键 显示所有的工作空间，然后点击右边的加号按钮在屏幕添加更多的工作空间。

此外，还可以在命令行中选择数字进行切换：

```
$ gsettings set org.cinnamon.desktop.wm.preferences num-workspaces 4
```

将 4 替换为想要切换到的工作空间编号。

隐藏桌面图标

默认情况下，Nemo 中启用了桌面图标渲染功能。要禁用这个功能，用下面命令行改变设置：

```
$ gsettings set org.nemo.desktop show-desktop-icons false
```

主题、图标和背景

Linux Mint 风格的主题、图标和背景可通过安装 mint-themesAUR, mint-x-iconsAUR, mint-y-iconsAUR 和 mint-backgroundsAUR 软件包获得。 因而，最后者是所有 Linux Mint 版本中包含的所有背景 (图片) 的集合。AUR 上还提供了各个 Linux Mint 版本的背景。

主题和图标可在 设置 → 主题 中编辑。背景则在 设置 → 背景 中。

官方 Linxu Mint Cinnamon 主题也包含在了 mint-themesAUR 软件包内。

可以如下这样在 Shell 中设置桌面主题：

```
$ gsettings set org.cinnamon.theme name "Theme-Name"
```

声音效果

Cinnamon 不附带用于桌面启动之类事件的声音 (它们是 Linux Mint 默认使用的声音效果)。这些声音效果可与安装 mint-artworkAUR。声音事件可以在 设置 → 声音 → 声音效果 中编辑。

调整窗口的大小

用 Alt+右键 调整窗口的大小，使用 gsettings：

```
$ gsettings set org.cinnamon.desktop.wm.preferences resize-with-right-button true
```

截图

截一个图这篇文章中有讲到，安装 gnome-screenshot包 就能有截图功能了。 默认快捷键为 `Prt Sc`, 可以在小程序的*快捷键* > *系统* > *截图与录像*下的*菜单* > *偏好设置* > *键盘*中更改。默认保存目录为 `$HOME/Pictures`, 不过可以像下面这样自定义路径：

```
$ gsettings set org.gnome.gnome-screenshot auto-save-directory
file:///home/USER/some_path
```

阻止 Cinnamon 覆盖 xrandr/xinput 配置

cinnamon-settings-daemon 提供了一些插件，它们可以管理显示、键盘和鼠标。这些插件会覆盖用户设置配置 (例如 xinitrc 文件中的 *xrandr* 命令)。要防止这种情况发生，有必要防止这些*设置守护程序插件* (settings daemon plugins) 启动。

可以将有关的插件的 *.desktop* 条目 (位于 `/etc/xdg/autostart/`) 复制到 `$HOME/.config/autostart` 处。之后在复制好的文件当中加一行 `Hidden=true`。

提示：用 `cinnamon-session --debug` 命令启动会话，以查看哪些插件被报告已被启动。

要保留显示、键盘和鼠标设置，请考虑禁用以下功能：

```
cinnamon-settings-daemon-a11y-keyboard.desktop
cinnamon-settings-daemon-a11y-settings.desktop
cinnamon-settings-daemon-keyboard.desktop
cinnamon-settings-daemon-mouse.desktop
cinnamon-settings-daemon-xrandr.desktop
```

故障排除

调试

可以使用`cinnamon-looking-glass` 工具 (杂项 - Cinnamon 调试器) 来检查多种 Cinnamon 环境中的各种事务：

- a list of currently-open windows - 当前打开的窗口的列表

- a list of currently-loaded extensions (applets, desklets, etc.) - 当前加载的扩展 (小程序、桌板等)
- logs - 日志

如果遇到桌面崩溃等事情 (通常是因为扩展不兼容或者有 Bug)，*logs (日志)* 功能就显得尤为有用。

出现 cinnamon-settings: No module named Image 错误

如果 *cinnamon-settings* 不能启动，同时还伴有这样无法找到某个模块的错误信息（例如 Image module，图像模块)，可能是使用了已过期的已编译文件，这些文件指向了不再存在的文件位置。这种情况下，请移除 `/usr/lib/cinnamon-settings` 和其子文件夹下的所有 `*.pyc` 文件。请参阅上游 Bug 报告。

崩溃后从 tty 启动 Cinnamon

如果 Cinnamon 已经完全没反应了，可以在 tty (按 Alt+F2 进入) 中用如下命令重启它：

```
$ export DISPLAY=:0; cinnamon --replace &
```

视频画面撕裂

因为 muffin包 基于 mutter包, GNOME 能用的视频撕裂修复方法应该对 Cinnamon 也有用。参见 GNOME/Troubleshooting#Tear-free video with Intel HD Graphics 获取更多信息。

禁用 网络管理 小程序

即使你不使用 NetworkManager 并从从默认面板删除*网络管理*小程序，Cinnamon 依然会载入 *nm-applet* 并显示在系统托盘上。你不能卸载 NetworkManager，因为 NetworkManager 被 cinnamon包 和 cinnamon-control-center包 依赖，但是可以很容易地将其禁用。要达成目的，首先应该把自启动文件从 `/etc/xdg/autostart/nm-applet.desktop` 复制到 `~/.config/autostart/nm-applet.desktop`，然后用喜欢的文本编辑器打开，并在尾部加上 `X-GNOME-Autostart-enabled=false`。

此外，你也可以通过创建以下符号链接来禁用：

```
$ ln -s /bin/true /usr/local/bin/nm-applet
```

从系统托盘中将特定图标 (例如 *nm-applet*图标) 列入黑名单的功能已被请求上游。

1.19 桌面环境比较

难度 进阶 | **篇幅** 14.5 KB | **所属** 桌面环境

有关主要文章，请参见桌面环境。

另请参见维基百科：X 视窗系统桌面环境的比较。

桌面环境	小组件工具包	X11窗口管理器	wayland混成器	任务栏	终端模拟器	文件管理器	计算器	文本编辑器	图片浏览器	媒体播放器	文件检视器	显示管理器
Budgie	gtk3包 gtk4包 libadwaita包	- （已删除）										

（已删除）

（开发中）

lightdm-pantheon-greeter包

1.20 MATE

难度 进阶 | **篇幅** 18.0 KB | **所属** 桌面环境

本节将讲到

- 安装
- 启动
- 配置
- 提示与技巧
- 故障排除

摘自 MATE 主页:

: The MATE 桌面环境是 GNOME 2 的延续，为 Linux 及其他类 Unix 系统提供直观和有吸引力的桌面。MATE 开发社区非常活跃，添加很多新的功能并保留传统使用习惯。

安装

可以通过如下方式安装安装 MATE:

- mate包组 包含标准 MATE 需要的核心桌面环境，包含 marco包, mate-panel包 and mate-session-manager包 等。
- mate-extra包组 包含额外的工具和程序，这些程序可以和 MATE 桌面协同工作。仅安装 mate-extra包组 时并不会通过依赖引入全部 mate包组 。要安装全部 MATE 软件包，请同时安装两个软件包组。

MATE 应用程序

MATE 大部分由 GNOME 2 软件和更加组成，这些软件会被重命名以避免和 GNOME3 版本冲突，下面是名称对比：

应用程序	GNOME 2	MATE
菜单编辑器	Alacarte	mozo包
文件管理器	Nautilus	caja包
窗口管理器	Metacity	marco包
文本编辑器	Gedit	pluma包
图片	Eye of GNOME	Eye of MATE (eom包)
Document viewer	Evince	atril包
归档管理器	File Roller	engrampa包

其他以 GNOME 开头的核心程序 (比如 GNOME Terminal, GNOME Panel, GNOME Menus 等) 会将前缀改为 MATE, 变成 MATE Panel, MATE Menus 等。

额外的 MATE 软件包

有些非官方 MATE 程序是由 MATE 社区开发维护，并没有包含在 mate包组 或 mate-extra包组 中。

- **Dock Applet** — MATE面板的应用程序坞。

: https://github.com/robint99/dock-applet || mate-applet-dock包

- **Online Radio Applet** — 一个用于MATE面板的小程序，可以让您通过单击一次即可播放您最喜欢的在线广播电台。

: http://www.zavedil.com/online-radio-applet/ || mate-applet-streamer包

- **MATE Menu** — MATE面板的高级菜单，是MintMenu的一个分支。

: https://github.com/ubuntu-mate/mate-menu || mate-menuAUR

- **MATE Tweak** — MATE的调整工具，是mintDesktop的一个分支。

: https://github.com/ubuntu-mate/mate-tweak || mate-tweakAUR

- **BriskMenu** — SolusOS分发的MATE桌面环境的现代高效菜单。

: https://github.com/getsolus/brisk-menu || brisk-menuAUR

要使用 Caja 的高级功能，需要安装额外的软件包，请参考 File manager functionality.

启动

在显示管理器中选择 *MATE* 即可,

或者是手动方式，要用 startx 启动 MATE,将 `exec mate-session` 加入 `~/.xinitrc` 文件.

配置

MATE可以通过其提供的 mate-control-center包 包中的 *Control Center* 应用程序 (*mate-control-center*) 进行配置。要管理某些硬件，您可能需要安装其他工具。

音频 : mate-media包 包支持 ALSA 和 PulseAudio 后端. **蓝牙** : 要支持 Bluetooth 设备，请安装 blueman包 。 请参见 Blueman 的相关文档。 **网络** : 要配置网络，请安装 network-manager-applet包 包。请参见 NetworkManager。 **电源** : UPower 后端受到 mate-power-manager包 包的支持。 **打印机** : 要配置打印机，请安装 system-config-printer包 包。

无障碍

MATE非常适合视力或行动受限的个人使用。Install orca包, espeakAUR (为盲人或视力受损者设计的屏幕阅读器) 和 onboard包 (对于行动受限用户而言非常有用的屏幕键盘)

在首次启动MATE之前，以需要辅助功能的用户身份输入以下命令：

```
$ gsettings set org.mate.interface accessibility true
```

一旦你启动MATE，你可以通过 *System* > *Preferences* > *Assistive Technologies* 配置无障碍应用。 不过，如果你需要使用Orca，你需要在 `Alt-F2` 运行窗口中运行它，以开始获得语音功能。

提醒

电池放电

要禁用电池放电的通知，请运行以下命令：

```
$ gsettings set org.mate.power-manager notify-discharging false
```

亮度

请参见 Backlight#Kernel command-line options。

提示与技巧

禁用合成

默认情况下，合成功能已启用。要禁用它，您可以在系统偏好设置的 *Look and Feel* > *Windows* > *General* 中勾选 *Enable software compositing window manager*。 另外，您也可以从终端运行以下命令来禁用合成功能：

```
$ gsettings set org.mate.Marco.general compositing-manager false
```

禁用新窗口居中

默认情况下，新窗口将居中显示。要禁用新窗口居中，请在系统偏好设置的 *Windows* > *Placement* 中勾选 *Center new windows*。您也可以从终端运行以下命令来禁用新窗口居中：

```
$ gsettings set org.mate.Marco.general center-new-windows false
```

禁用窗口吸附

窗口吸附功能默认已启用。要禁用它，请在系统偏好设置的 *Windows* > *Placement* 勾选 *Enable window tiling*。另外，您也可以从终端运行以下命令来禁用窗口吸附功能：

```
$ gsettings set org.mate.Marco.general allow-tiling false
```

取消最大化窗口的装饰

使用 mate-tweakAUR 工具可以隐藏最大化窗口的装饰;安装后，在系统偏好设置的 *Look and Feel* > *MATE Tweak* > *Windows* 并在 *Window Behaviour* 部分中启用 *Undecorate maximized windows*。

提示：如果此选项变为灰色且不可用，请确保已安装 mate-netbook包 包。

显示或隐藏桌面图标

默认情况下，MATE在桌面上显示多个图标： 桌面目录的内容、计算机、主目录和网络目录、废纸篓和已挂载的驱动器。您可以使用 `gsettings` 逐个或全部显示或隐藏它们。

隐藏所有桌面图标

```
$ gsettings set org.mate.background show-desktop-icons false
```

这样做可能会导致辅助显示器上出现一些图形异常。

隐藏个别图标

隐藏电脑图标：

```
$ gsettings set org.mate.caja.desktop computer-icon-visible false
```

隐藏用户目录图标：

```
$ gsettings set org.mate.caja.desktop home-icon-visible false
```

隐藏网络图标：

```
$ gsettings set org.mate.caja.desktop network-icon-visible false
```

隐藏回收站图标

```
$ gsettings set org.mate.caja.desktop trash-icon-visible false
```

隐藏已挂载的卷：

```
$ gsettings set org.mate.caja.desktop volumes-visible false
```

将 `false` 替换成 `true` ，图标将重新出现。

使用不同的窗口管理器

marco 窗口管理器可以通过以下任一方法被另一个窗口管理器取代：

使用gsettings（推荐）

执行以下操作以指定MATE的不同窗口管理器：

```
$ gsettings set org.mate.session.required-components windowmanager wm-name
```

使用MATE会话自动启动

您可以使用 *mate-session-properties* 自动启动所选择的窗口管理器。这意味着在登录时，自动启动的窗口管理器将替代默认的窗口管理器。 在系统偏好设置中导航到 *Startup Applications*。在对话框中点击 *Add*.。命令应采用以下语法： `*wm-name* --replace`。

阻止Caja管理桌面

执行以下操作以防止Caja管理桌面：

```
$ gsettings set org.mate.background show-desktop-icons false
$ killall caja  # Caja will be restarted by session manager
```

更改窗口装饰按钮顺序

您可以使用图形化的dconf-editor或命令行工具gsettings来更改按钮顺序：

```
$ gsettings set org.mate.Marco.general button-layout 'close,maximize,minimize:'
```

将 **menu**, **close**, **minimize** 和 **maximize** 以您希望的顺序使用逗号分隔开。冒号用于指定窗口按钮在标题栏的哪一侧显示，并且必须在更改时使用。

在驱动器挂载后自动打开文件管理器

默认情况下，MATE 在挂载驱动器时会自动打开一个新的文件管理器窗口。要禁用此功能：

```
$ gsettings set org.mate.media-handling automount-open false
```

要禁用自动挂载：

```
$ gsettings set org.mate.media-handling automount false
```

在Caja中的空间视图

为了确保每个新文件夹以新窗口的形式打开（也称为空间视图），打开Caja的首选项对话框，点击行为标签，并选中“在单独窗口中打开每个文件夹”的选项。或者，执行以下命令以达到相同的效果：

```
$ gsettings set org.mate.caja.preferences always-use-browser false
```

更改字体DPI设置

您可以通过右键单击桌面，选择 *Change desktop background* > *Fonts* > *Details* > *Resolution* 来更改MATE中字体的DPI（每英寸点数）。在弹出的窗口中，您可以调整字体的分辨率以改变其大小和清晰度。

更改应用程序菜单图标

默认情况下，应用程序菜单图标设置为 `start-here`。要使用其他图标，请将图标复制到 `/usr/local/share/pixmaps` 等文件夹中，并执行以下操作：

```
$ gsettings set org.mate.panel.menubar icon-name icon
```

其中，*icon* 是你的图标的名字。在图标名字中不要包括文件扩展名。最后，重新启动MATE面板。

面板速度设置

隐藏/显示延迟

要调整面板在启用自动隐藏时消失或重新出现所需的时间，请执行以下操作：

```
$ dconf write /org/mate/panel/toplevels/panel/(un)hide-delay time
```

其中，*panel* 是 *top* 或 *bottom*，*time* 是以毫秒为单位的值，例如300。

动画速度

要设置面板动画发生的速度，请执行以下操作：

```
$ dconf write /org/mate/panel/toplevels/panel/animation-speed value
```

其中 *panel* 可以是 *top* 或 *bottom*，*value* 可以是 `"'fast'"`, `"'medium'"` 或 `"'slow'"` 之一。

将终端设置为 caja-open-terminal

`caja-open-terminal` 扩展使用GSettings确定使用哪个终端 - *mate-terminal* 是默认值。要更改将使用的终端，请运行以下命令。

```
$ gsettings set org.mate.applications-terminal exec my-terminal
```

其中 *my-terminal* 是要启动的终端可执行文件的名称，例如：*xterm*。

故障排除

切换合成特效

在使用NVIDIA专有驱动程序和合成窗口管理器的环境中工作时，某些软件可能会出现渲染图形的问题。

为了方便切换合成功能，请将以下脚本保存在Home目录中的某个位置：

```
~/.scripts/compositing.sh
```

```
#!/bin/sh
if [ "$(gsettings get org.mate.Marco.general compositing-manager)" = "true" ]
then
  gsettings set org.mate.Marco.general compositing-manager false
else
  gsettings set org.mate.Marco.general compositing-manager true
fi
```

然后创建一个自定义的键盘快捷键，用于执行该文件，例如 `Ctrl+Alt+C`，将其设置为 `sh ~/.scripts/compositing.sh`。

为合成特效启用垂直同步

MATE的窗口管理器marco通过DRI3/Xpresent支持无撕裂的软件合成。

如果您的显卡驱动程序不支持DRI3（例如Nvidia专有驱动程序），*marco* 不支持通过 *OpenGL* 进行垂直同步， 这可能会导致启用合成时出现视频撕裂。 在这种情况下，考虑使用 picom 等支持OpenGL的不同的 composite manager。

保持光标主题一致

请参见 Cursor themes#Desktop environments.

在 LightDM 中使用渐变背景

如果您希望将默认的 MATE (1.8) *Stripes* 背景同时用作 LightDM 的背景，以实现从LightDM到MATE的平滑过渡，您会发现默认背景是根据灰度PNG图像动态生成的，上面再添加了一个从蓝色到绿色的垂直渐变色，而这是LightDM当前不支持的。如果您坚持要做这个操作，可以通过临时设置 `/org/mate/desktop/background/show-desktop-icons` 为 `false` 来解决，可以通过`System Tools` 菜单中的 `dconf-editor` 工具或者通过运行命令来进行设置。

```
$ gsettings set org.mate.background show-desktop-icons false
```

从 Alt-F2 `Run Application` 对话框中运行 `killall mate-panel`，并在面板重新出现之前按下 `Print Screen` 键。 然后，您会看到一个 `Save As` 对话框，以保存您所需的完整渲染、屏幕大小的PNG图像，以供LightDM使用。运行

```
$ gsettings set org.mate.background show-desktop-icons true
```

会使您的桌面图标重新出现，如果有需要的话。

启用面板阴影

由于竞争条件，在登录到MATE桌面后，即使启用了合成，面板阴影也不会显示。

复制到 `/usr/share/applications/marco.desktop` 并添加延迟:

```
~/.local/share/applications/marco.desktop
X-MATE-Autostart-Phase=Applications
X-MATE-Autostart-Delay=2
X-MATE-Provides=windowmanager
X-MATE-Autostart-Notify=true
```

注意：Delays are only allowed in the applications phase, hence `X-MATE-Autostart-Phase` must be set to `Applications`.

如果没有效果，请增加延迟时间的长度。

登录注销/关机延迟由 at-spi-registryd 引起

在登出或关机时，您可能会遇到一个弹出窗口，显示 *A program is still running: at-spi-registryd.desktop*。作为解决方法，您可以阻止 *at-spi-registryd* 启动 - 请参见 GTK#Suppress warning about accessibility bus - 但这可能会影响某些辅助功能。

Caja的文本文件预览

由于迁移到GTK 3，该功能不再起作用。

GTK 2 应用似乎忽略了默认的 MATE 主题

mate-themes包 包附带的主题需要可选依赖项 gtk-enginesAUR 包和 gtk-engine-murrineAUR 包，以使GTK 2主题正常工作。

CSD 应用程序上的额外装饰

当CSD应用程序（如Firefox、Visual Studio Code等）取消最大化时，会出现额外的装饰。

卸载 mate-netbook包 包可以解决这个问题。

键盘布局选择器

当启用多个键盘布局时，系统托盘中会显示一个布局选择图标。但是由于一个错误 (), 根据当前使用的主题，有时会显示为白色字体在明亮的背景上 (或者在其他几乎无法辨认的配置中，例如绿色字体)。

可以通过手动设置字体颜色来解决这个问题 (例如，将其设置为黑色的 `'0 0 0'`):

```
$ gsettings set org.mate.peripherals-keyboard-xkb.indicator foreground-color '0 0 0'
```

参见

- MATE 主页
- MATE Arch Linux维基
- MATE 桌面截图
- MATE 桌面环境 - 关于MATE的Arch Linux论坛讨论（该帖子在2017年1月5日关闭）

1.21 桌面环境

难度 进阶 | **篇幅** 19.2 KB | **所属** 桌面环境

本节将讲到

- 概况
- 桌面环境列表
- 自己打造桌面环境

桌面环境（DE）是桌面比拟的一个实现，由一捆程序组成，它们共享一个共同的图形用户界面（GUI）。

概况

桌面环境将各种组件捆绑在一起，以提供常见的图形用户界面元素，如图标、工具栏、壁纸和桌面小部件。此外，大多数桌面环境包括一套集成的应用程序和实用程序。最重要的是，桌面环境提供了他们自己的窗口管理器，然而通常可以用另一个兼容的窗口管理器来代替。

用户可以自由地以任何方式配置他们的GUI环境。桌面环境只是提供了一个完成这一任务的完整而方便的手段。请注意，用户可以自由地混合和匹配来自多个桌面环境的应用程序。例如，一个 KDE 用户可以安装和运行 GNOME 应用程序，如 Epiphany 网络浏览器，如果他们喜欢它而不是 KDE 的 Konqueror 网络浏览器。这种方法的一个缺点是，许多由桌面环境项目提供的应用程序在很大程度上依赖于各自桌面环境的基础库。因此，从一系列的桌面环境中安装应用程序将需要安装更多的依赖项。寻求节约磁盘空间的用户通常会避免这种混合环境，或者选择只依赖少数外部库的替代品。

此外，由桌面环境提供的应用程序往往能更好地与他们的本地环境整合。从表面上看，使用不同部件工具包的混合环境会导致视觉上的差异（也就是说，界面会使用不同的图标和部件样式）。在可用性方面，混合环境的行为可能并不相似（例如，单击与双击图标；拖放功能），因此可能会造成混乱或意外的行为。

桌面环境列表

注意：桌面环境需要添加适量对应的图片以便用户挑选。

官方支持

- **Budgie** — Budgie 是一款专为现代用户设计的桌面环境，它专注于简单和优雅。

: https://blog.buddiesofbudgie.org || budgie包组

Budgie桌面环境预览，版本10.9.2

- **Cinnamon** — Cinnamon 致力于提供传统的用户体验。Cinnamon 是一个 fork GNOME 3 的项目。

: https://github.com/linuxmint/Cinnamon || cinnamon包

Cinnamon桌面环境预览，版本6.2.2

- **Cutefish** — Cutefish 是一个全新的现代桌面环境。

: https://cutefish-ubuntu.github.io || cutefish包组

Cutefish桌面环境预览，版本0.8

- **COSMIC** — COSMIC 是由 System76 使用 Rust 为 Pop! OS 开发的桌面环境，灵感来自于定制版的 GNOME。仍处于 beta 状态。

: https://github.com/pop-os/cosmic-epoch || cosmic包组

- **Deepin** — Deepin 桌面界面和应用程序功能的设计直观且优雅。四处移动、共享和搜索等成为简单、愉悦的体验。

: https://www.deepin.org/ || deepin包组

Deepin桌面环境预览，版本未知

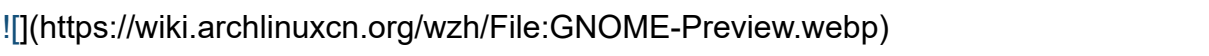
- **Enlightenment** — Enlightenment desktop shell 基于 Enlightenment Foundation Libraries提供了高效的窗口管理器以及其他基本桌面组件，如文件管理器、桌面图标和小部件。它支持主题，并能够在较旧的硬件和嵌入式设备上运行。

: https://www.enlightenment.org/ || enlightenment包

Enlightenment桌面环境预览，版本未知

- **GNOME** — GNOME 桌面环境是一个具有现代（“GNOME”）和经典（“GNOME Classic”）会话的迷人而直观的桌面。

: https://www.gnome.org/ || gnome包组



GNOME桌面环境预览，版本46

- **GNOME Flashback** — GNOME Flashback 是 GNOME 3 的一个 shell，最初称为 GNOME 后备模式。桌面布局和底层技术与 GNOME 2 类似。

: https://wiki.gnome.org/Projects/GnomeFlashback || gnome-flashback包



GNOME-Flashback桌面环境预览，版本未知

- **KDE Plasma** — KDE Plasma 桌面环境是一个熟悉的工作环境。Plasma 提供了现代桌面计算体验所需的所有工具，因此可以从一开始就提高生产力。

: https://www.kde.org/plasma-desktop || plasma包组



KDE桌面环境预览，版本6.1.2

- **LXDE** — 轻量级 X11 桌面环境是一个快速且节能的桌面环境。它配备了现代界面、多语言支持、标准键盘快捷键和附加功能，如标签式文件浏览。以轻量级为基本设计，其努力比其他环境更少地占用 CPU 和内存。

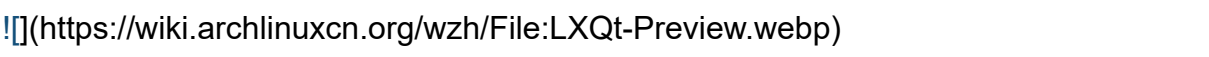
: https://lxde.org/ || lxde包组



LXDE桌面环境预览，版本未知。GTK3版本界面与该图片类似

- **LXQt** — LXQt 是轻量级桌面环境 LXDE 的 Qt 移植，也是其下一代产品。它是 LXDE-Qt 和 Razor-qt 项目之间合并的产品：一个轻量级、模块化、速度极快且用户友好的桌面环境。

: https://lxqt.github.io/ || lxqt包组



LXQt桌面环境预览，版本2.0

- **MATE** — Mate 为使用传统隐喻的 Linux 用户提供了一个直观而有吸引力的桌面。MATE 最初是一个 fork GNOME 2 的项目，但现在使用 GTK 3。

: https://mate-desktop.org/ || mate包组

Mate桌面环境预览，版本1.28.2

- **Pantheon** — Pantheon 是为 elementary OS 开发的默认桌面环境。 它使用 Vala 和 GTK3 工具开发。在可用性和外观方面，该桌面环境与 GNOME Shell 和 macOS 有诸多相似之处

: https://elementary.io/ || pantheon包组

- **Phosh** — Phosh 是一个基于 GNOME 的 Wayland shell 的用户界面，适用于移动设备。

: https://gitlab.gnome.org/World/Phosh/phosh || phosh包

Phosh桌面环境预览，版本未知

- **Sugar** — Sugar 学习平台是一个由活动组成的计算机环境，旨在帮助 5 至 12 岁的儿童通过富媒体表达方式共同学习。Sugar 是全球努力为每个儿童提供优质教育机会的核心组成部分——目前全世界有近 100 万儿童在使用它，他们分布在 40 多个国家，使用 25 种语言。Sugar 提供了帮助人们通过获得素质教育过上充实生活的手段，而目前有很多人错过了这种教育。

: https://sugarlabs.org/ || sugar包 + sugar-fructose包组

Sugar桌面环境预览，版本未知

- **UKUI** — UKUI 是一款轻量级的 Linux 桌面环境，基于 GTK 和 Qt 进行开发。UKUI 是优麒麟默认的桌面环境。

: https://www.ukui.org/ || ukui包组

UKUI桌面环境预览，版本10

- **Xfce** — Xfce 体现了传统的 UNIX 哲学的模块化和可重复使用性。它由一些组件组成，提供了人们对现代桌面环境所期望的全部功能，同时保持了相对的轻便。它们被分别打包，可以在可用的包中挑选，以创建最佳的个人工作环境。

: https://xfce.org/ || xfce4包组

Xfce桌面环境预览，版本4.18

非官方支持

- **CDE** — CDE是 Unix 和 OpenVMS 的桌面环境，基于 Motif 小部件工具包。它是 UNIX98 工作站产品标准的一部分，长期以来一直是与商业 Unix 工作站相关的“经典”Unix 桌面。尽管它是一个遗留环境，但它仍然通过对 Linux 系统的支持而保持活力。

: https://sourceforge.net/projects/cdesktopenv/ || cdesktopenvAUR

- **COSMIC** — COSMIC 是由 System76 为 Pop! OS 创建的基于 Rust 桌面环境，它受到高度定制的 GNOME 的启发。还未正式发布，可能体验不佳。

: https://github.com/pop-os/cosmic-epoch || cosmic-epoch-gitAUR

- **EDE** — “ Equinox 桌面环境”是一个设计简单、重量极轻、速度极快的桌面环境。

: https://edeproject.org/ || edeAUR

- **KDE 1** — “前向移植”以在现代系统上工作，主要是为了怀旧，现在移植工作确实有效，可以用作日常桌面使用。在 AUR 中也有越来越多的 KDE1 应用程序。

: https://invent.kde.org/historical/kde1-kdebase || kde1-kdebase-gitAUR

- **Liri** — Liri 是一个具有现代设计和功能的桌面环境， Liri是基于 Hawaii, Papyros 和 Liri Project功能的合并，具备高度的实验性.

: https://liri.io/ || liri-shell-gitAUR

- **Lumina** — Lumina 是一个用 Qt 5 为FreeBSD编写的轻量级桌面环境，使用 Fluxbox 作为窗口管理器。

: https://lumina-desktop.org/ || lumina-desktopAUR

- **Maui Shell** — Maui Shell是一个融合桌面、平板和移动设备的shell环境。

: https://github.com/Nitrux/maui-shell || maui-shell-gitAUR

- **Moksha** — Enlightenment的桌面分支，目前用作基于 Ubuntu 的 Bodhi Linux 的默认桌面环境。

: https://www.bodhilinux.com/moksha-desktop/ || moksha-gitAUR

- **Pantheon** — Pantheon 最初为基本操作系统发行版创建的默认桌面环境，它是使用 Vala 和 GTK3 工具包从头开始编写的，就可用性和外观而言，Pantheon与 GNOME Shell 和 macOS 有一些相似之处。

: https://elementary.io/ || pantheon-session-gitAUR

- **PaperDE** — PaperDE 桌面环境基于 Qt/Wayland 和 wayfire构建.

: https://gitlab.com/cubocore/paper/paperde || paperdeAUR

- **plainDE** — 一个用 Qt 编写的轻量级 GNU/Linux 桌面环境。目前包含面板和控制中心。

: https://plainDE.github.io/ || plainde-metaAUR

- **Plasma Mobile** — Plasma Mobile 是一个基于 KDE Plasma 堆栈和 Wayland 的移动桌面环境.

: https://invent.kde.org/plasma/plasma-mobile || plasma-mobileAUR

- **theDesk** — theDesk一种尽可能透明的桌面环境，它使用 Qt5 作为其小部件工具包，使用 KWin 作为其窗口管理器。

: https://github.com/vicr123/thedesk || thedeskAUR

- **Trinity** — Trinity Desktop Environment（TDE）项目是一个适用于类 Unix 操作系统的计算机桌面环境，其主要目标是保留整个 KDE 3.5 计算风格。

: https://www.trinitydesktop.org/ || See Trinity

自己打造桌面环境

桌面环境代表了安装*完整*图形环境的最简单方法。然而，如果主流桌面环境都不能满足用户的要求，那么用户也可以自由地构建和定制自己的图形环境。一般来说，构建一个自定义环境需要选择一个合适的窗口管理器或混成器，一个任务栏和一些应用程序（一个最小的方案通常包括终端模拟器，文件管理器和文本编辑器）。

通常由桌面环境提供的其它应用程序有：

- 桌面合成器（Xorg）
- 桌面通知程序
- 显示管理器
- 屏幕锁定器

- 应用程序启动器
- 音频控制
- 背光控制
- 媒体控制
- Polkit 身份认证组件
- 电源管理
- 屏幕捕获
- 色温
- 壁纸设置工具
- 登出对话框
- 默认应用程序

使用其它窗口管理器

如果桌面环境有对应的文章，请查看其*使用不同的窗口管理器*一节，否则请查阅官方文档。

- Budgie#使用不同的窗口管理器
- Cinnamon#使用不同的窗口管理器
- GNOME#使用不同的窗口管理器
- KDE#使用不同的窗口管理器
- LXDE#使用不同的窗口管理器
- LXQt#使用不同的窗口管理器
- MATE#使用不同的窗口管理器
- Xfce#使用不同的窗口管理器

1.22 Enlightenment

难度 高级 | **篇幅** 20.7 KB | **所属** 桌面环境

本节将讲到

- Enlightenment
- Enlightenment DR16
- 参考资料
- 配置输入法

Enlightenment

这个软件包提供了 Enlightenment 的窗口管理器及其构建库 EFL（Enlightenment Foundation Libraries）。后者提供了额外的桌面环境特性，如工具包、对象画布和抽象对象。Enlightenment 从 2005 年开始开发，2011 年 2 月发布 1.0 稳定版本。

安装

可安装同名软件包 enlightenment包 。

还可以安装 terminology包，这是一个基于 EFL 的终端仿真器。

从 AUR 安装

警告：其中某些软件包使用不稳定的开发版代码，请风险自担。

开发版的软件包源码及其依赖的包构建文件可以通过安装 enlightenment-gitAUR 获得。 以下是基于 EFL 的应用，大部份是开发版本，尚未正式发布：

- ecrire-gitAUR – Ecrire 文本编辑器
- ediAUR – 基于 EFL 的集成开发环境(IDE)
- eluminance-gitAUR – Eluminance 图片浏览器
- enjoy-gitAUR – Enjoy 音乐播放器
- eperiodiqueAUR – Eperiodique periodic 表格查看器
- ephoto包 and ephoto-gitAUR – Ephoto图片查看器
- epourAUR – 基于 EFL 的种子(torrent)客户端
- epymc-gitAUR – E Python 多媒体中心
- equate-gitAUR – Equate 计算器
- eruler-gitAUR – Eruler 屏幕尺和测量工具
- efbb-gitAUR – Escape from Booty Bay 类似《愤怒的小鸟》的游戏
- elemines-gitAUR – Elemines 《扫雷》类型的游戏
- rage包 和 rage-gitAUR – Rage 视频播放器
- terminology-gitAUR – terminology包 的当前 git 主分支版本

启动 Enlightenment

只需从喜欢的显示管理器中选择*Enlightenment*， 或配置好 xinitrc 即可从控制台启动。

Entrance

警告：Entrance 仍旧处于高度实验性阶段，尚未被 systemd 完全支持 ，若要使用请自担风险。

Enlightenment 提供了一个名为 Entrance 的新显示管理器，由 entrance-gitAUR 提供。Entrance 十分精巧， 它用 `/etc/entrance.conf` 管理配置。可以通过 systemd 启用 `entrance.service` 服务来使用它。

手动启动

如果更偏好手动启动 Enlightenment，请在控制台里输入并执行 `startx /usr/bin/enlightenment_start`。详见 xinitrc。

要尝试使用 Wayland 混成器，请改为输入`enlightenment_start`。可能为此需要安装 efl-gitAUR 和 enlightenment-gitAUR，因为这两个软件包虽然仍处于实验开发阶段，但功能和体验相对更完整。

配置

Enlightenment 提供了一个精巧的配置系统，可以从主菜单中选择“设置”子菜单进入。

网络

ConnMan

Enlightenment 首选的网络管理器是 ConnMan， 包名： connman包 。配置方法参见：ConnMan。

为实现更多的配置，还可以安装 Econnman （AUR 中的 econnmanAUR 或 econnman-gitAUR）及其相关依赖包。

把 ConnMan 添加到书架

1. 设置 -> 扩展 -> 模块
2. 系统
3. 连接管理器（Connection Manager）
4. 加载（选中并点击*加载*）
5. 在屏幕底部书架点击右键
6. 进入书架 -> 内容
7. 滚动项目列表找到 *ConnMan*
8. 点击*添加*

NetworkManager

你也可以使用 networkmanager包 来管理网络连接。参见 NetworkManager。

注意：这个小程序需要Appindicator支持才能在Enlightenment的系统托盘中显示。可参考NetworkManager#Appindicator.作为使用该程序的一种选择，NetworkManager包含了CLI and TUI两种网络配置界面--参见NetworkManager#nmcli examples。

Polkit 代理

Enlightenment 没有提供图形化的 Polkit 认证代理。如果要执行某些需授权的操作（例如安装系统设备上的文件系统），你要安装一个认证代理并且使它自动启动。后者可以导航至***设置面板 > 应用 > 启动应用程序 > 系统***设置项并激活它。AUR 中提供了一个基于 EFL 的认证代理，名为 polkit-efl-gitAUR。

集成 GNOME 密钥环

在Enlightenment中可以使用gnome-keyring. 然而你需要做一点小的更改才能让它完全地工作。首先你需要设置Enlightenment去自动启动gnome-keyring，定位到*Settings Panel* > *Apps* > *Startup Applications* > *System* 并激活 *Certificate and Key Storage*、*GPG Password Agent*、*SSH Key Agent* 以及 "Secret Storage Service"。 然后, 你应该编辑 `~/.pam_environment` 并添加下面的代码:

```
#Set gnome-keyring as the ssh authentication agent
SSH_AUTH_SOCK=/run/user/${UID}/keyring/ssh
```

上述代码会覆盖 "enlightenment-start" 变量的自动启动配置，从 "ssh-agent" 切换到 gnome-keyring。

更多信息请参考 GNOME Keyring 一文。

系统托盘

注意：从 Enlightenment 20 版开始，对 Xembed 的支持已被移除 。这意味着许多“传统的”托盘部件将无法显示在托盘。要使用这些托盘部件，需要另外安装一个独立的系统托盘程序（例如 stalonetray包）

Enlightenment 支持系统托盘，但默认未启用。若要启用系统托盘，请打开 Enlightenment 主菜单，导航至***设置***子菜单，点击***模块***选项，向下滚动至***系统托盘***选项并聚焦，点击***加载***按钮。这样就加载了模块，可将其添加到书架中。在待添加系统托盘的书架上右击，聚焦于***书架***子菜单，点击*内容* 选项，向下滚动到***系统托盘***并聚焦，然后点击*添加*按钮。

通知

Enlightenment 的“通知”扩展模块提供了一个通知服务器。

- 通知可以按下述定义显示在屏幕任一角落
- 可用的屏幕策略有：主屏幕、当前屏幕、所有屏幕和 Xinerama
- 通知可以按紧急程度过滤（低、普通、紧急，及各种组合形式）
- 可以设置默认通知消隐时间，也可以设置是否强制不自动消隐
- 通知服务器可以设置是否忽略替换 ID 的请求

主题

下列更多主题用于定制 Enlightenment 外观：

- enlightenment-themes.org
- relighted.c0n.de 200 种不同颜色组合的默认主题
- git.enlightenment.org（用 git 抓取喜欢的主题，运行 'make' 生成 .edj 后缀的主题文件）
- packages.bodhilinux.com 这里有一堆不错的主题（需从 .deb 包中释放出 .edj 文件，可以用 Arch Linux 的基础组件 bsdtar 释放）。在他们的维基上还提供了一个很不错的分类
- exchange.enlightenment.org (已封存)

你可以在主题设置对话框中安装这些 .edj 文件格式的主题，或者把它们放在 `~/.e/e/themes` 目录中。

注意：Enlightenment 未提供稳定的主题 API，多年以来，甚至在 E17 发布后，很多主题 API 也已改变。未及时更新的主题很可能无法正常工作。

提示：若要使 GTK 和 Qt 应用程序与 Enlightenment 默认主题相匹配，你可以下载一个类似 E17 GTK 主题 这样的主题包，放在 `~/.themes/` 中或者安装 gtk-theme-e17gtk-gitAUR 包，并在 Enlightenment 的设置中选中它，然后对其进行配置。这样可以使所有 GTK2 和 GTK3 应用匹配默认 Enlightenment 主题，然后，你可以配置 Qt 应用程序（或者配置 Qt 的默认设置），让其使用 Gtk+ 主题。这样 Qt 应用程序将模拟当前使用的 GTK 应用程序。这样就可以让绝大部分应用程序都使用 Enlightenment 的主题。参阅 Qt 与 GTK 应用程序外观一致化。

GTK+

替换 GTK+ 主题的选项在***设置 > 全部 > 外观 > 应用程序主题***。

模块和小部件

模块 : 小工具的后端支持代码在 Enlightenment 中使用的名称。 **小工具** : 前端或用户界面，应该有助于 Enlightenment 的用户完成某项任务。

很多模块提供了可以添加到桌面或面板上的小工具。某些模块 (如 CPUFreq) 只提供了单个的小工具；而一些模块 (如 Composite) 虽然不提供小工具，但提供了额外的功能。 注意某些小工具（如 Systray）只能被添加到面板上，而另一些小工具（如 Moon）只能在桌面上加载。

"外部" 模块

警告：这些第三方模块不被官方开发者支持。它们直接来自 git，因而可能随时出现工作异常。若要使用请自担风险。

除了这里列举的模块, 更多的模块可以从 e-modules-extra-gitAUR 找到。

Scale Windows

*Scale Windows*模块添加了额外的功能，但需要开启 compositing. 缩放窗口特效（Scale Windows）可以缩小所有打开的窗口并使它们全部进入预览视图。 这项功能与macOS中的"Mission Control"功能相类似. scale pager特效缩放所有桌面并将它们如壁纸一样显示，类似于插件 expo. 这两项功能都可以添加到桌面，或者与快捷键、鼠标以及屏幕边缘绑定起来。

某些用户喜欢将标准的窗口选择快捷键 `ALT + Tab` 改变为使用缩放窗口特效（Scale Windows）去选择窗口。为了达到上述目的, 需要依次定位到*Menu*> *Settings* > *Settings Panel* > *Input*> *Keys*. 在这里你可以设置任何你喜欢的快捷键。

若需要将窗口选择键绑定功能替换为缩放窗口特效（Scale Windows）,滚动做面板直到 *ALT* 节然后找到并选择 `ALT + Tab`。然后滚动右面版寻找 "Scale Windows" 并选择 *Select Next* 或者 *Select Next (All)* 并点击 *Apply* 保存设置，*Select Next*选项仅能看到当前桌面的窗口，*Select Next (All)* 选项可看到所有桌面上的窗口。

这些可从上游 git 包获得。

默认快捷键绑定

<table class="wikitable"><caption>Enlightenment 的一些默认快捷键绑定</caption><tbody><tr><td>Shift + F10</td><td>垂直方向最大化</td></tr><tr><td>Ctrl + Menu</td><td>显示 "客户端"(窗口)菜单</td></tr><tr><td>Alt + Escape</td><td>显示 "Everything 启动器" (应用、窗口等)</td></tr><tr><td>Win + Left</td><td>左分屏最大化</td></tr><tr><td>Win + Right</td><td>右分屏最大化</td></tr><tr><td>Alt + Shift + F10</td><td>水平方向最大化</td></tr><tr><td>Alt + Shift + Left</td><td>转到左侧的桌面</td></tr><tr><td>Alt + Shift + Right</td><td>转到右侧的桌面</td></tr><tr><td>Ctrl + Alt + D</td><td>显示桌面</td></tr><tr><td>Ctrl + Alt + F</td><td>切换全屏</td></tr><tr><td>Ctrl + Alt + I</td><td>切换图标模式</td></tr><tr><td>Ctrl + Alt + K</td><td>杀死窗口</td></tr><tr><td>Ctrl + Alt + L</td><td>锁定桌面</td></tr><tr><td>Ctrl + Alt + N</td><td>最大化窗口</td></tr><tr><td>Ctrl + Alt + R</td><td>向上卷起窗口</td></tr><tr><td>Ctrl + Alt + W</td><td>窗口菜单</td></tr><tr><td>Ctrl + Alt + X</td><td>关闭窗口</td></tr><tr><td>Ctrl + Alt + Down</td><td>下移一层</td></tr><tr><td>Ctrl + Alt + Up</td><td>上移一层</td></tr><tr><td>Ctrl + Alt + Left</td><td>转到左侧的桌面</td></tr><tr><td>Ctrl + Alt + Right</td><td>转到右侧的桌面</td></tr><tr><td>Ctrl + Alt + Delete</td><td>结束会话对话框

</td></tr><tr><td>Ctrl + Alt + Insert</td><td>启动默认终端
</td></tr></tbody></table>

故障排除

如果 Enlightenment 出现了一些奇怪的行为, 可以尝试下面的步骤:

1. 尝试使用默认主题，看这些行为是否会出现；
2. 禁用你安装的所有第三方模块；
3. 备份 `~/.e` 文档并移除 `~/.e` (使用命令`mv ~/.e ~/.e.back`)；

若你确定自己发现了一个 Bug，请将它提交到 directly 上游页面。

混成

当需要在无法打开设置窗口的情况下重置设置的时候, 可以通过硬编码（hardcoded）快捷键 `Ctrl + Alt + Shift + Home` 重置混成器的设置。

字体看不清楚

如果字体太小而无法阅读,首先确定你已经安装了正确的字体包。 安装ttf-dejavu包 和 ttf-bitstream-vera包 字体包是一个不错的选择。

你可以在 *Settings* > *Settings Panel* > *Look* > *Scaling*选项中设置缩放。

背光总是较暗

你或许会发现在登出的情况下 Enlightenment 会常规性的将背光调低为30%，却只能在你登录到另一个新的Enlightenment session时才能恢复到100%。当使用Enlightenment和另一个桌面环境时，该问题特别明显；因为当使用该桌面环境时，背光不会自动恢复到正常水平。要修复该问题， 打开 *Settings Panel*, 在*Look* 标签下, 勾选 *Composite* 选项。勾选*Don't fade backlight* 前的方框并点击*OK*.

光标主题不一致

有时可能会发现桌面的光标主题与应用（如 Firefox）中的光标主题不一致。 这是因为应用使用的是 X 光标主题，而 Enlightenment 有自己的光标主题设置。为了两者一致, 你可以设置 Enlightenment 总是使用 X 光标主题： 打开 Enlightenment的 *Settings Panel*，然后点击 *Input* 标签。之后点击 *Mouse* 选项。将其主题从*Enlightenment* 切换到 *X* 然后点击 *OK* 保存即可。 此时应该可以看到光标主题在每一个地方都是一致的了。 如果 X 光标主题并不总是一致的，可参考光标主题

背景图片

你需要将想要设置为背景的图片拷贝到 `~/.e/e/backgrounds/` 目录下。

在桌面的任意地方点击鼠标中键（MMB）或右键（RMB）访问“设置”选项，选择`/Desktop/Backgrounds/`

任何新拷贝到 `~/.e/e/backgrounds/` 文件夹下的图片都会使可供选择的背景列表自动更新。从下拉菜单中选择你想要设置的图片。在全局设置中的 *appropriate* 选项卡内，可以调整背景图像的平铺、填充屏幕等。

Enlightenment DR16

Enlightenment, 开发版 16 第一次发布于2000年,在2009年到达了 1.0版. 初始情况下，DR16表示Enlightenment项目的0.16版。它就是现在Arch源的"Enlightenment16", 直到今天还在开发维护中, 通常由它的维护者 Kim 'kwo' Woelders提供更新。With compositing, shadows and transparencies, E16 kept all of the speed that presided over its foundation by original author Carsten "Rasterman" Haitzler but with up to date refinement.

安装 E16

安装enlightenment16AUR包.

如需要深入的文档，可以参考`/usr/share/doc/e16/e16.html`.

基本设置

E16的大多数设置文件保存在`~/.e16`目录中，并且是基于文本、可以编辑的。其中也包含了菜单（menus）。

快捷键可以通过手动修改,也可以使用e16keyedit 软件修改；该软件可以在sourceforge页面找到。注意：默认情况下不会在`~/.e16`目录下创建键盘快捷键绑定文件。如果你想做修改的话，可以通过下面的命令将安装包自带的键盘快捷键绑定文件复制到你的home目录下：

```
$ cp /usr/share/e16/config/bindings.cfg ~/.e16
```

启动、重启、停止脚本

在你的`~/.e16`文件夹中创建 Init, a Start and a Stop 文件夹: 任何在这些文件夹中的 .sh 脚本 将会在启动时(位于Init文件夹)、每次重启时(位于Start文件夹)或者关机时(位于Stop文件夹)被执行; 假如你允许它们通过 MMB / settings / session / <enable scripts> button并 通过`chmod +x **yourscript.sh**`命令赋予它们可执行权限。经典的例子是启动pulseaudio或者你喜欢的网络管理程序。

合成器（Compositor）

阴影、透明等等特效位于 Composite 项下的 MMB 或 RMB / 设置。

参考资料

- Enlightenment 主页
- Enlightenment 开发者文档
- E17-Stuff
- DR16 下载资源
- Enlightenment 用户邮件列表
- Enlightenment 开发者邮件列表
- ircs://irc.libera.chat/e

配置输入法

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： 这篇文章或章节的内容已经过期。（在 Talk:Enlightenment# 中讨论）

注意：英文版本节文字已删除。为方便中文用户，本节内容暂予保留

E17 内置了输入法支持的模块，支持的输入法有 iiimf 、scim 和 uim 。使用这些输入法的配置在

```
Settings -> Settings Panel -> Language -> Input Method Settings -> Advanced
```

System 配置中，使用者只需选择即可。使用其他输入法的用户可以在 Personal 配置中添加。

ibus

ibus 的配置参数为：

```
Input Method Parameters:
 Name               ibus
 Execute Command    /usr/bin/ibus-daemon --xim
 Setup Command      /usr/bin/ibus-setup
Exported Environment Variables:
 GTK_IM_MODULE      ibus
 QT_IM_MODULE       ibus
 XMODIFIERS         @im=ibus
```

1.23 Pantheon

难度 高级 | **篇幅** 22.7 KB | **所属** 桌面环境

本节将讲到

- 安装
- 启用Pantheon桌面
- Configuration
- Known Issues
- Troubleshooting

Pantheon 是linux发行版 elementary os 的默认桌面环境。由开发者使用vala语言和gtk3工具包编写完成，高效并且易于使用。用户界面上，与GNOME-shell和Mac OS X多有相似之处。

安装

Pantheon桌面环境目前不由archlinux官方维护，要在archlinux安装使用Pantheon桌面环境可以向系统添加三方维护的软件源，并从该软件源安装相关软件包，将以下代码添加到软件源列表（需要管理员权限） `/etc/pacman.conf`:（每日构建版本）

```
[pantheon]
SigLevel = Optional
Server = http://pkgbuild.com/~alucryd/$repo/$arch
```

提示：对这些软件包的编译脚本感兴趣的可以访问Alucryd's GitHub repository.

当然，如果喜欢也可以选择自己编译这些软件包，相关的软件包源码都可以直接从 AUR 系统找到.

如果只是向体验基础的Pantheon用户界面.可以安装pantheon-session-bzrAUR\[损坏的链接：package not found\]，这将自动安装以下软件包：

- cerbere-bzrAUR\[损坏的链接：package not found\]: 进程守护程序
- gala-bzrAUR\[损坏的链接：package not found\]: 窗口管理器
- wingpanel-bzrAUR\[损坏的链接：package not found\]: 顶部面板组件
- slingshot-launcher-bzrAUR\[损坏的链接：package not found\]: 顶部面板应用程序菜单
- plank-bzrAUR\[损坏的链接：package not found\]: 地步Dock程序启动器

当然，要获取更完美的Pantheon桌面体验，还请同时安装以下软件包：

提示：请确保系统安装的以下程序包都是带-bzr的版本，已避免可能导致的错误。

- audience-bzrAUR\[损坏的链接：package not found\]: Video player
- contractor-bzrAUR\[损坏的链接：package not found\]: Service for sharing data between apps
- dexter-contacts-bzrAUR\[损坏的链接：package not found\]: Contacts manager (does not build)
- eidete-bzrAUR: Simple screencaster
- elementary-icon-theme-bzrAUR\[损坏的链接：package not found\]: elementary icons
- elementary-scan-bzrAUR\[损坏的链接：package not found\]: Simple scan utility (does not build)

- elementary-wallpapers-bzrAUR\[损坏的链接：package not found\]: elementary wallpaper collection
- gtk-theme-elementary-bzrAUR\[损坏的链接：package not found\]: elementary GTK theme
- feedler-bzrAUR\[损坏的链接：package not found\]: RSS feeds reader (does not build)
- footnote-bzrAUR\[损坏的链接：package not found\]: Note taking app
- geary包: Email client
- indicator-pantheon-session-bzrAUR\[损坏的链接：package not found\]: Session indicator
- lightdm-pantheon-greeter-bzrAUR\[损坏的链接：package not found\]: LightDM greeter
- maya-calendar-bzrAUR\[损坏的链接：package not found\]: Calendar
- midori-granite-bzrAUR: Web browser
- noise-bzrAUR\[损坏的链接：package not found\]: Audio player
- pantheon-backgrounds-bzrAUR\[损坏的链接：package not found\]: Wallpaper collection
- pantheon-calculator-bzrAUR\[损坏的链接：package not found\]: Calculator
- pantheon-default-settings-bzrAUR\[损坏的链接：package not found\]: Pantheon default's settings (appearance, etc.)
- pantheon-files-bzrAUR\[损坏的链接：package not found\]: File explorer
- pantheon-notify-bzrAUR\[损坏的链接：package not found\]: Notification daemon
- pantheon-print-bzrAUR\[损坏的链接：package not found\]: Print settings
- pantheon-terminal-bzrAUR\[损坏的链接：package not found\]: Terminal emulator
- plank-theme-pantheon-bzrAUR\[损坏的链接：package not found\]: Pantheon theme for plank
- scratch-text-editor-bzrAUR\[损坏的链接：package not found\]: Text editor
- snap-photobooth-bzrAUR\[损坏的链接：package not found\]: Webcam app
- switchboard-bzrAUR\[损坏的链接：package not found\]: Settings manager

注意：You will also need to install plugs, look for "switchboard-plug-*" in the AUR or in Alucryd's GitHub repository.

推荐安装以下字体包来获取最佳桌面体验:

- ttf-opensans包: Open Sans Fonts

- ttf-raleway-font-familyAUR\[损坏的链接：package not found\]: Raleway Font Family
- ttf-dejavu包: Font family based on the Bitstream Vera Fonts
- ttf-droid包: General-purpose fonts released by Google as part of Android
- gnu-free-fonts包: Set of free outline fonts covering the Unicode character set
- ttf-liberation包: Red Hats Liberation fonts

补充信息

Packages based on older evolution-data-server

dexter-contacts-bzrAUR\[损坏的链接：package not found\] and feedler-bzrAUR\[损坏的链接：package not found\] do not build because they are based on evolution-data-server 3.2. Arch Linux provides version 3.10 which uses a different Vala API.

启用Pantheon桌面

使用显示服务器启动

pantheon-session-bzrAUR\[损坏的链接：package not found\] provides a session entry for display managers such as gdm包 or lightdm包.

注意：Either use the bzr version of *cerbere* or add 'gala' to the monitored processes for this to work.

通过 .xinitrc 启动

你也可以用 `~/.xinitrc` 启动Pantheon。这段代码将能够成功启动Pantheon:

```
#!/bin/sh

if [ -d /etc/X11/xinit/xinitrc.d ]; then
  for f in /etc/X11/xinit/xinitrc.d/*; do
    [ -x "$f" ] && . "$f"
  done
  unset f
fi

gsettings-data-convert &
xdg-user-dirs-gtk-update &
/usr/lib/polkit-gnome/polkit-gnome-authentication-agent-1 &
/usr/lib/gnome-settings-daemon/gnome-settings-daemon &
/usr/lib/gnome-user-share/gnome-user-share &
eval $(gnome-keyring-daemon --start --components=pkcs11,secrets,ssh,gpg)
export GNOME_KEYRING_CONTROL GNOME_KEYRING_PID GPG_AGENT_INFO SSH_AUTH_SOCK
exec cerbere
```

注意：Pantheon may refuse to start correctly, resulting in errors such as no visible mouse cursor and others. In this case you have to add the window manager 'gala' to the list of monitored processes of cebere. This can be done in `dconf-editor` and should look like this. Remember do not run dconf-editor as root, use your local username.

Autostart applications

Pantheon, when launched via `~/.xinitrc`, does not support XDG autostart. However, there are 3 other ways to achieve this for applications which do not provide a systemd unit:

- You may add any program to your `~/.xinitrc`, preferably right before the *exec cerbere* line. This is the better choice for one-shot programs.
- Or you may edit the `org.pantheon.cerbere.monitored-processes` key using *dconf-editor* and add the programs of your choice. This method is best for applications which keep running in the background.
- Or you may use a program like dapperAUR, dex-gitAUR, or fbautostartAUR to add support for XDG autostart in your `~/.xinitrc`.

注意：Keep in mind that applications started via *cerbere* cannot be terminated, they will keep respawning.

Configuration

Configuring Pantheon is done via switchboard-bzrAUR\[损坏的链接：package not found\] and its plugs, most are available in the AUR and the custom repo. All pantheon settings can also be altered via *dconf*, they are located in the `org.pantheon` key. Use *dconf-editor* for easy editing.

Part of the configuration is handled by gnome-control-center包 via a dedicated plug, which unfortunately only supports GNOME up to 3.6. Use gnome-control-center包 itself and gnome-tweak-tool包\[损坏的链接：replaced by gnome-tweaks包\] instead.

Pantheon Files

Enable context menu entries

If you want to enable context menu entries such as for file-roller包 to extract/compress archives, then you have to additionally install contractor-bzrAUR\[损坏的链接：package not found\].

Terminal

Opacity (transparency)

You can set a certain opacity to make Pantheon Terminal (semi-)transparent. Open `dconf-editor` and go to `org.pantheon.terminal.settings.opacity` to set your desired opacity.

Known Issues

Indicators not working in wingpanel

Make sure the /etc/xdg/autostart/indicator-\[name\].desktop file contains Pantheon inside OnlyShowIn=

```
OnlyShowIn=Unity;XFCE;GNOME;Pantheon;
```

注意：

- Indicator support itself is a complex issue, due to standards discrepancies between Ubuntu and Gnome's implementation of KDE's status notification indicators.
- The pantheon devs are working on a number of their own indicators for wingpanel.

Indicator-session menus not working

- indicator-session-bzrAUR\[损坏的链接：package not found\]

This version of indicator-session relies on dbus methods native to Unity for most of its functions; it can be made to work by patching out the use of Unity dialogs, such as in indicator-session-pantheon-bzr\[失效链接 2020-08-04 ⓘ\]

- indicator-sessionAUR

This version of indicator-session fails to interact with the session manager somehow; a (crudely hacked) version that uses systemd/logind instead is available indicator-session-systemdAUR\[损坏的链接：package not found\]

About This Computer, *Lock* and *Sound Settings* (indicator-soundAUR or indicator-sound-pantheon-bzr\[失效链接 2020-08-04 ⓘ\]) rely on gnome components that may not be installed, such as gnome-control-center and gnome-screensaver.

For *Lock* functionality (including "Ctrl+L" hotkey), replace gnome-screensaver with light-locker or XScreenSaver and a script that emulates the gnome-screensaver dbus.

No transparency in pantheon-terminal

Transparency in pantheon-terminal is not yet fully functional with GTK themes other than the elmentaryOS theme. Either use gtk-theme-elementary包 or add this code to your theme.

White icons in pantheon-files

Currently there seems to be a bug which displays the view icons in the top location in a white colour instead of black. This can be fixed by installing gtk-theme-elementary-bzrAUR\[损坏的链接：package not found\] or adding the following line to `gtk-widgets.css` or `gtk-widgets.css` of your gtk-theme-elementary包 theme:

```
GtkToolItem { color: @text_color; }
```

Wingpanel is transparent

Wingpanel is transparent by design when using the elementary GTK theme. It becomes black when a maximized window occupies your screen. However, using other GTK themes will produce a solid panel most of the time.

Corrupted graphics in canonical indicators

Indicators behave incorrectly with every theme I have tried. They are very ancient, all of them date back to 2012 because the newer indicators depend on Ubuntu patches,

and they should be killed with fire anyway. Wingpanel is doing just that and I hope the next major release will ship their new plugin system.

Cannot interact with the LightDM Pantheon greeter

You need to delete `/var/lib/lightdm/.pam_environment`. Do note however that this file is a workaround for the following LightDM bug: https://bugs.launchpad.net/ubuntu/+source/unity-greeter/+bug/1024482

Troubleshooting

Gala crashes on start

It appears that unconfigured gala tries to use default gnome wallpaper as a background. However, the corresponding file is absent unless you have gnome-themes-standard包\[损坏的链接：package not found\] installed. Thus, install gnome-themes-standard包\[损坏的链接：package not found\] to workaround the crash. It is safe to remove this package after you configure pantheon in a way you want.

How can I add new applications to the dock?

Either drag and drop a desktop file on it, or right click on a running application and select "Keep in dock". You can then reorder icons by drag and drop.

How can I change the default appearance such as GTK theme, font size, etc?

Use gnome-tweak-tool包\[损坏的链接：replaced by gnome-tweaks包\] or see GTK+.

I do not have any mouse cursor

The 'gala' window manager is most likely not running. #通过 .xinitrc 启动 Add 'gala' to the list of cerbere's monitored processes.

Wingpanel is empty except for Applications

The indicators that are displayed in the wingpanel are split into separate packages. #安装 Install additional indicators such as wingpanel-indicator-datetime-bzrAUR\[损坏的链接：package not found\], indicator-powerAUR\[损坏的链接：package not found\] or indicator-soundAUR.

图形

本章解决显卡从被识别到可用的问题：选对驱动，走通安装、加载与 Xorg、Wayland 配置，再处理混合图形与硬件视频加速。

先读《图形处理器》判断卡型，再按品牌读《Intel 图形处理器》《AMDGPU》《NVIDIA》中对应的一节，属必读；双显卡笔记本接读《PRIME》，要切换再看 NVIDIA Optimus 与 Bumblebee。其余偏旧章节可先跳过。最易误解的是把装驱动当成一步：同一块卡常有开源与专有两套互斥方案，装错或混装就启动失败、黑屏，故各节都把“选择正确的驱动”放在最前。

2.1　Intel GMA 3600

难度 入门 | **篇幅** 2.9 KB | **所属** 图形

本节将讲到

- Xorg 驱动
- 问题解决
- 参阅

Intel GMA3600 是基于 PowerVR SGX 545 显示核心的集成显卡，搭载于 Atom N2600,N2800,D2550 中。

Linux 内核从 3.5 版本开始支持此设备。

Xorg 驱动

目前没有单独的 Xorg 驱动，请使用 xorg-server包 提供的 modesetting 驱动:

```
/etc/X11/xorg.conf.d/20-gpudriver.conf
Section "Device"
    Identifier "Intel GMA3600"
    Driver     "modesetting"
EndSection
```

modesetting 驱动可以使用 xrandr 禁用/启用 LVDS, VGA 等端口，修改分辨率.

下面设置可以禁用 LVDS 并强制启用 VGA：

```
/etc/X11/xorg.conf.d/20-gpudriver.conf
Section "Device"
    Identifier "Intel GMA3600"
    Driver     "modesetting"
    Option     "Monitor-LVDS-0" "Ignore"
    Option     "Monitor-VGA-0" "Monitor"
EndSection

Section "Monitor"
    Identifier "Ignore"
    Option     "Ignore"
EndSection

Section "Monitor"
```

```
    Identifier "Monitor"
    Option     "Enable"
EndSection
```

问题解决

恢复后黑屏

如果恢复系统后黑屏，请执行：

```
# touch /etc/pm/sleep.d/99video
```

播放视频

目前没有办法使用视频加速，可以用 Atom CPU 用多线程解码:

```
mplayer -lavdopts threads=4 -fs myvideo.avi
```

参阅

- https://www.change.org/en-GB/petitions/intel-listen-to-the-community-and-develop-gma3600-drivers-for-linux
- https://ubuntuforums.org/showthread.php?t=1953734
- https://communities.intel.com/message/158477

2.2 Nvidia-xrun

难度 入门 | **篇幅** 4.0 KB | **所属** 图形

本节将讲到

- 安装
- 配置
- 使用
- 问题

nvidia-xrun是一个实用的独立显卡与独立的NVIDIA图形完全性能。bumblebee目前的状态提供了非常糟糕的表现，nvidia-xrun这个解决方案提供了一个更复杂的程序，拥有更好的GPU利用效率。

安装

安装：

- nvidia包
- bbswitch包
- nvidia-xrunAUR 或者 nvidia-xrun-gitAUR
- 一个窗口管理器, 例如 openbox包 可选，仅用来运行需要使用nvidia的程序，因为直接使用nvidia-xrun运行一些程序（如steam）表现较差，使用一些像Openbox窗口管理器会好一些。

配置

设置nvidia的bus id

如果安装nvidia-xrun完毕后，`/etc/X11/nvidia-xorg.conf.d/30-nvidia.conf`文件中已经设置好bus id，可直接跳过本步。 如果你从\[nvidia-xrun github repo\]下载安装的nvidia-xrun，你应该需要进行手动设置bus id。

获取ID：一般的设备的总线ID是1:0:0，为了确保正确，使用一下命令获取ID:

```
lspci | grep NVIDIA
```

在输出内容中第行首即可看到ID。

新增文件`/etc/X11/nvidia-xorg.conf.d/30-nvidia.conf`， 添加类似如下内容：

```
/etc/X11/nvidia-xorg.conf.d/30-nvidia.conf
Section "Device"
    Identifier "nvidia"
    Driver "nvidia"
    BusID "PCI:1:0:0"
EndSection
```

同样的，如果遇到问题你可以调整一些NVIDIA设置：

```
/etc/X11/nvidia-xorg.conf.d/30-nvidia.conf
Section "Screen"
    Identifier "nvidia"
    Device "nvidia"
    #  Option "AllowEmptyInitialConfiguration" "Yes"
    #  Option "UseDisplayDevice" "none"
EndSection
```

自动y运行窗口管理器

编辑~/.nvidia-xinitrc，例如使用openbox，在其中添加：

```
openbox-session
```

在tty登录后，通过以下命令启动openbox桌面环境，在openbox中运行程序即可使用NVIDIA渲染：

```
nvidia-xrun
```

使用bbswitch在管理nvidia显卡

平时使用bbswitch关闭nvidia显卡，在需要使用nvidia运行程序时，运行`nvidia-xrun`就会唤醒nvidia显卡，并自动打开设定好的窗口管理器。

- 在启动时载入bbswitch模块

```
# echo 'bbswitch ' > /etc/modules-load.d/bbswitch.conf
```

- 关闭nvidia显卡的选项

```
# echo 'options bbswitch load_state=0 unload_state=1' > /etc/modprobe.d/bbswitch.conf
```

- 将nvidia相关模块加入黑名单

```
$ lsmod | grep nvidia | cut -d ' ' -f 1 > /tmp/nvidia
```

```
$ lsmod | grep  nouveau | cut -d ' ' -f 1 >> /tmp/nvidia
$ sort -n /tmp/nvidia | uniq >  /tmp/nvidia.conf#去重
$ sed -i 's/^\w*$/blacklist &/g' /tmp/nvidia.conf  #添加blacklist
# cp /tmp/nvidia.conf /etc/modprobe.d/nvidia.conf  #移动
```

重启系统即可。 查看状态：

```
cat /proc/acpi/bbswitch
```

开关显卡可以使用bbswitch相关命令

```
# tee /proc/acpi/bbswitch <<<OFF
# tee /proc/acpi/bbswitch <<<ON
```

更多bbswitch信息查看 Bumblebee-Project/bbswitch

使用

1. 切换到tty
2. 登录
3. 运行`nvidia-xrun [app]`

问题

直接使用nvidia-xrun运行steam表现较差，使用一些像Openbox窗口管理器会更好。

2.3 AMDGPU PRO

难度 进阶 | **篇幅** 7.0 KB | **所属** 图形

本节将讲到

- 专有组件的用途
- 安装
- 用法
- 故障排除
- 参考链接

本页介绍了 AMD GPU 的闭源驱动程序。

提示：大多数用户不需要这些专有驱动程序。

专有组件的用途

AMD 通过标准分发渠道发布其开源驱动程序。他们还定期发布*Radeon Software for Linux*套件，其中包括开放组件和专有组件。那里不需要开源组件，专有组件是从最新的 ubuntu lts 版本重新打包的。它们发布在 AUR 的 amdgpu-pro-installer 包组中。

该评论解释了为什么 AMD 仍然打包闭源驱动程序:

```
如今，我们打包的驱动程序主要用于：
* 软件迁移速度较慢的企业/LTS 发行版的客户不会自动获取最新的图形驱动程序 - 我们为他们提供开源和专有/工作站选项
```

* 使用工作站应用程序的客户需要来自面向工作站的驱动程序的额外性能/认证（尽管 Marek 在去年做了很多出色的工作来提高工作站应用程序上的 Mesa 性能）
* 第三个目标受众是寻找现成 OpenCL 的客户，它们可以与打包的开放/封闭驱动程序一起使用，也可以与最近发行版中基于上游的堆栈一起使用。

有几个专有组件：OpenGL、OpenCL、Vulkan 和 AMF。有时，由于开源的组件可能缺少特定功能，可能需要使用到这些（专有）组件。

AMDGPU PRO OpenGL 是一种专有的用户态二进制驱动程序，它基于开源 amdgpu 内核驱动程序运行。从 Radeon Software 18.50 与 Mesa 19 基准测试该文章可以看出：在 OpenGL 游戏方面，RadeonSI Gallium3D 驱动程序能全方位替代专有的 AMD OpenGL 驱动程序。AMD 建议购买 Radeon Pro 以外的非专业显卡用户使用 amdgpu 图形堆栈。使用该组件主要是因为对应的开源组件缺少某些软件所依赖的兼容层。请参阅下面的 gentoo wiki 链接。

AMDGPU PRO Vulkan - 目前唯一具有光线追踪的实现（但是被用户报告为故障状态）。它也是 AMF 的必需依赖项。

AMDGPU PRO OpenCL - 使用该组件主要是因为 Mesa OpenCL 不完全完整。仅适用于 Polaris GPU 的专有组件。未来的 GPU 会使用开放的 ROCm OpenCL。

AMDGPU AMF - 用于 GPU 硬件编解码。

安装

对于专有的 OpenGL 实现，安装 amdgpu-pro-oglpAUR 和可选的 lib32-amdgpu-pro-oglpAUR 以支持 32 位应用程序。

有关可用的 OpenCL 实现，请参阅 GPGPU#AMD/ATI。

对于专有的 Vulkan 实现，安装 vulkan-amdgpu-proAUR 和可选的 lib32-vulkan-amdgpu-proAUR 以获得 32 位应用程序支持。

对于高级媒体框架（AMF）实现，安装 amf-amdgpu-proAUR。

要让 OBS 使用它，请使用 obs-studio-amfAUR。

注意：自 2022 年 10 月起，由于 AMD 驱动程序的一个错误，您需要 amdgpu-pro-installer-fix 包(或者 https://github.com/HannesMann/archlinux-amdgpu-pro - 如果-fix包不起作用) 和 20220815.8413c63-1 版本的 linux-firmware包 来让 H264 和 HEVC 工作。

用法

使用专有的 OpenGL

使用 progl 命令运行应用程序，例如：

```
$ progl glmark2
```

如何确保程序使用的是 AMDGPU-PRO 驱动程序

执行下面的命令：

```
$ glxinfo | grep "OpenGL vendor string" | cut -f2 -d":" | xargs
```

如果返回的是 `AMD`, 那就表明运行的是开源驱动程序。如果返回 `Advanced Micro Devices, Inc.`，则表明运行了专有驱动程序。

或者, 也可以运行 glmark2。 当用的是开源驱动，在 OpenGL 信息里面就可以看到：

```
    GL_VENDOR:     AMD
    GL_RENDERER:   Radeon RX 580 Series (POLARIS10, DRM 3.40.0, 5.10.7-arch1-1, LLVM 11.0.1)
    GL_VERSION:    4.6 (Compatibility Profile) Mesa 20.3.3
```

如果用的是闭源驱动，看到的就应该是：

```
    GL_VENDOR:     ATI Technologies Inc.
    GL_RENDERER:   Radeon RX 580 Series
    GL_VERSION:    4.6.14756 Compatibility Profile Context
```

使用专有的 Vulkan

用 `vk_pro` 命令（脚本）启动应用程序，例如：

```
$ vk_pro vkmark
```

更过信息可以参考 Vulkan#通过_AMD_Vulkan_Prefixes_选择。

使用 Advanced Multimedia Framework

参考 FFmpeg#AMD AMF

故障排除

Intel + AMD 混合显示

对于使用 Intel GPU 和 AMD GPU 混合配置的用户，由于不同的 MESA 实现，专有 AMDGPU Pro 工作站驱动程序的使用可能无法按预期工作。

症状如下：开机时黑屏，但鼠标光标移动正常。

不幸的是，反向 PRIME 不能作为解决方案。请参考开发者回复。

卸载软件包

如果遇到问题，例如由于黑屏而无法登录系统，可以通过卸载与 AMDGPU PRO 相关的所有软件包来恢复所有状态。

切换虚拟控制台（例如 `Ctrl+Alt+F2`），登录并运行：

```
# pacman -R $(pacman -Qg Radeon_Software_for_Linux | cut -f2 -d" "
```

再重启系统。

Southern Islands (SI) 或者 Sea Islands (CIK) 显卡

如果使用 Southern Islands (SI) 或 Sea Islands (CIK) 显卡，遇到在运行 `clinfo` 时打印：

```
amdgpu_device_initialize: DRM version is 2.50.0 but this driver is only compatible with 3.x.x.
```

要保证使用的是 amdgpu 驱动程序，而不是 radeon。

用下面的命令检查当前正在使用的驱动程序：

```
lspci -k
03:00.0 Display controller: Advanced Micro Devices, Inc. [AMD/ATI] Opal XT [Radeon R7 M265/M365X/M465]
        Subsystem: Acer Incorporated [ALI] Aspire V5 Radeon R7 M265
        Kernel driver in use: radeon
        Kernel modules: radeon, amdgpu
```

更多信息请参考 AMDGPU#开启_Southern_Islands_(SI)_和_Sea_Islands_(CIK)_支持_和_Sea_Islands_(CIK)_支持)。

参考链接

- Gentoo:AMDGPU-PRO

2.4 Uvesafb

难度 进阶 | **篇幅** 8.2 KB | **所属** 图形

本节将讲到

- 安装
- 准备系统
- 配置 uvesafb
- Uvesafb 内核参数
- Uvesafb 和 915resolution

翻译状态：

- 本文（或部分内容）译自 Uvesafb，最近一次同步于 2015-02-28，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

**这篇文章或章节的翻译质量不佳。**

原因：Last updated in 2015（在 Talk:Uvesafb# 中讨论）

与其他 framebuffer 驱动不同，uvesafb 需要一个叫做 v86d 的用户空间虚拟化守护进程。在 x86 机器上模拟 x86 代码看起来很愚蠢，但若是想在其他架构上使用

framebuffer 代码这就显得很重要了 (尤其是非 x86 架构). 一个新的 framebuffer 驱动已添加到 2.6.24 版的内核中。它比标准的 vesafb 有更多的特性，包括:

1. 延迟后适当间隔和硬件悬停。
2. 支持自定义分辨率，就像使用系统 BIOS 一样。

vesafb 支持的硬件它应该都支持。

安装

从 AUR 安装 v86dAUR.

准备系统

记得从引导器配置中删除一切与 framebuffer 有关的内核启动参数以防止 vesafb framebuffer 加载。

```
$ grep vga /proc/cmdline
$ grep -ir vga /etc/modprobe.d/
```

应该不会返回错误。如果你在别处还有 `vga=` 选项，记得移除。

GRUB

注意：这可能不起作用。

编辑 `/etc/default/grub`, 注释掉 `GRUB_GFXPAYLOAD_LINUX=keep` 行。

然后用标准脚本生成 `grub.cfg`:

```
# grub-mkconfig -o /boot/grub/grub.cfg
```

GRUB legacy

从 `/boot/grub/menu.lst` 的 kernel 行移除所有对 `vga=xxx` 的引用以便 uvesafb 正常工作。

Systemd

使用 systemd 的话，往 `/etc/mkinitcpio.conf` 的 HOOKS 部分添加 v86d. 这能让 uvesafb 在启动时接管工作。

```
HOOKS="base udev v86d ..."
```

配置 uvesafb

规定分辨率

uvesafb 的配置在 `/usr/lib/modprobe.d/uvesafb.conf`:

```
# This file sets the parameters for uvesafb module.
# The following format should be used:
# options uvesafb mode_option=<xres>x<yres>[-<bpp>][@<refresh>] scroll=<ywrap|ypan|
redraw> ...
#
```

```
# For more details see:
# https://www.kernel.org/doc/html/latest/fb/uvesafb.html
#
options uvesafb mode_option=1280x800-32 scroll=ywrap
```

mode_option 的文献可见于 。

为避免你的设置在升级时被覆盖，把下面的文件复制到 /etc/modprobe.d/uvesafb.conf:

```
# cp /usr/lib/modprobe.d/uvesafb.conf /etc/modprobe.d/uvesafb.conf
```

然后往 /etc/mkinitcpio.conf 的 FILES 部分添加你的配置文件，比如:

```
FILES="/etc/modprobe.d/uvesafb.conf"
```

为使变更生效，你需要重新生成 *initramfs* 镜像。

```
# mkinitcpio -p linux
```

重启后变更生效。

调整分辨率

通过以下命令可得到可用分辨率:

```
$ cat /sys/bus/platform/drivers/uvesafb/uvesafb.0/vbe_modes
```

可根据得到的条目来修改 /usr/lib/modprobe.d/uvesafb.conf.

检查当前分辨率

下面的任一命令都可以用来检查当前 framebuffer 分辨率:

```
$ cat /sys/devices/virtual/graphics/fbcon/subsystem/fb0/virtual_size
$ cat /sys/class/graphics/fb0/virtual_size
```

Uvesafb 内核参数

如果你编译了个人定制内核，你也可以把 uvesafb 编译进内核，之后比如通过 /etc/rc.local 运行 v86d。这种情况下，该选项能够以 video=uvesafb:<options> 格式传递内核参数。请注意当你如下把 uvesafb 和 915resolution 结合起来时，这个方案是无效的。

Uvesafb 和 915resolution

接下来我们要处理一个更复杂的情形。许多为笔记本设计的 Intel 图形芯片组被报告说有一个易出 bug 的 BIOS，它甚至无法支持宽屏的原生主分辨率！为此创造了 915resolution, 用于在启动时给 BIOS 打补丁以让 X server 支持宽屏的分辨率。 目前，X server 能在没有 915resolution 的情况下完成这些。然而，915resolution 能与 uvesafb 结合来获得一个支持宽屏的 framebuffer, 根本就不需要启动 X 了。这种情况下，运行完 915resolution 之后我们需要加载 uvesafb 来让它工作在正常的分辨率之下。

915resolution-static

在这种情形中，915resolution 需要被静态编译 (因为它要包含在 initramfs 中，因此不能有任何到外部库文件的连接). 所以你不能使用 \[community\] 源中的 915resolution 软件包，应该转而使用 AUR 中的 915resolution-staticAUR\[损坏的链接：package not found\]. 它静态编译了 915resolution 并提供了一个它的钩子，所以你在加载 uvesafb 之前就能运行 915resolution 并获得正确的分辨率。所以通过 makepkg 和 pacman 安装 915resolution-static .

分辨率

你需要编辑 915resolution 钩子以确定你要替换的 BIOS 模式和你想要的分辨率。你可以用如下命令得到 915resolution 所有选项的信息:

```
$ 915resolution -h
```

编辑 `/lib/initcpio/hooks/915resolution` 并修改 915resolution 的选项:

```
run_hook ()
{
   msg -n ":: Patching the VBIOS..."
   /usr/sbin/915resolution 5c 1280 800
   msg "done."
}
```

默认的 5c 用来替换成你要的 BIOS 模式的代码。使用命令 `915resolution -l` 来得到所有可用 BIOS 图形模式的代码。

注意：你可能会选择你*并不需要*的代码 (framebuffer 或是 X 也都不需要), 因为 915resolution 会把它替换成用户选定的模式。上例中，`1280 800` 是新的所需分辨率。

钩子阵列

添加 915resolution 钩子并在其后添加 v86d 钩子到 `/etc/mkinitcpio.conf` 的 HOOKS 部分。务必位于 keymap 之前，以便从悬停和文件系统中恢复。

```
HOOKS="base udev 915resolution v86d ..."
```

之后用 mkinitcpio 重新生成 initramfs (替换成你自己的预设文件):

```
mkinitcpio -p linux
```

疑难问题

Uvesafb 无法保留内存

检查你是否忘记删除了 `vga=xxx` 内核参数 -- 它把 UVESA framebuffer 覆盖为标准 VESA.

Error: "pci_root PNP0A08:00 address space collision + Uvesafb cannot reserve memory"

这发生在 Acer Aspire One 751h 上使用 2.6.34-ARCH 内核时; 这是否在其他系统上也有我们不得而知。即使用另一份 uvesafb 设置调制 framebuffer, uvesafb 也不能保留必要的内存区域。

你可以通过在引导器配置里加上如下内核参数来解决。

```
pci=nocrs
```

另见

- Uvesafb Kernel Page
- Gentoo's uvesafb Page
- VESA mode numbers

2.5 混合图形技术

难度 进阶 | **篇幅** 11.5 KB | **所属** 图形

本节将讲到

- 动态切换
- 疑难解答

混合图形技术是指在同一计算机上包含两张显卡。笔记本电脑制造商将两张性能和功耗不同的显卡置入一台笔记本电脑，使用混合图形技术来同时支持高性能和节能的使用场景，即在集成显卡的 3D 渲染性能不够用时使用专用/独立显卡。

有许多技术用于应对该问题，许多制造商也开发了自己的解决方案。这项技术在 Windows 上支持完善，但在 Linux 上依然不够完善。本文将简要解释一些方法和社区解决方案，以应对供应商对 GNU/Linux 缺乏支持的现况。

注意：除非您的设备生产于 2010 年前，否则其应当可以使用动态切换方法。以前的混合图形解决方案要么需要简单粗暴地重启电脑，要么需要重启整个图形栈（还得重新登录）才能生效。

动态切换

大部分较新的混合图形技术使用两张显卡：一张专用/独立显卡，一张集成显卡，通过帧缓冲区进行协调，没有硬件多路复用器。集成显卡一直开启，而专用显卡仅在有高性能渲染需求时启用，节电时停用。大部分情况下无法仅使用专用显卡，且所有切换和渲染都由软件控制。 启动时，Linux 内核开始使用视频模式并设定低级图形驱动（low-level graphic drivers），这会被应用程序使用。大部分 Linux 发行版使用 X.org 创建图形环境。最后，一些其他软件被启动，先是登录管理器，然后是窗口管理器，再往后是其它软件。这个等级制的系统被设计为在单显卡的系统上使用。

注意：请参阅 NVIDIA Optimus 和 Bumblebee 以了解使用 NVIDIA 专有驱动的混合图形技术的详细信息。请参阅 PRIME 以了解更为全面的（包括 AMDGPU、nouveau 和 NVIDIA 专有驱动）方案。

完全关闭独立显卡

您可能想完全关闭高性能显卡以节省电池电量。

使用 BIOS/UEFI

一些笔记本电脑制造商在 BIOS 或 UEFI 添加了切换开关，用以禁用独立显卡。

使用 udev 规则

确保移除了所有与 NVIDIA 有关的显示管理器配置。

将 nouveau 驱动加入禁用列表：

```
/etc/modprobe.d/blacklist-nouveau.conf
blacklist nouveau
options nouveau modeset=0
```

然后创建：

```
/etc/udev/rules.d/00-remove-nvidia.rules
# 如果 NVIDIA USB xHCI Host Controller 设备存在，则将其移除
ACTION=="add", SUBSYSTEM=="pci", ATTR{vendor}=="0x10de", ATTR{class}=="0x0c0330",
ATTR{power/control}="auto", ATTR{remove}="1"

# 如果 NVIDIA USB Type-C UCSI 设备存在，则将其移除
ACTION=="add", SUBSYSTEM=="pci", ATTR{vendor}=="0x10de", ATTR{class}=="0x0c8000",
ATTR{power/control}="auto", ATTR{remove}="1"

# 如果 NVIDIA Audio 设备存在，则将其移除
ACTION=="add", SUBSYSTEM=="pci", ATTR{vendor}=="0x10de", ATTR{class}=="0x040300",
ATTR{power/control}="auto", ATTR{remove}="1"

# 移除NVIDIA VGA/3D controller 设备
ACTION=="add", SUBSYSTEM=="pci", ATTR{vendor}=="0x10de", ATTR{class}=="0x03[0-9]*",
ATTR{power/control}="auto", ATTR{remove}="1"
```

重启并执行 `lspci` 以查看您的 NVIDIA GPU 是否还在。

检查电量消耗以确保 GPU 没有偷偷耗电。如果依然耗电，或许可以使用 acpi_call 将其完全关闭。

使用 bbswitch

使用 NVIDIA GPU 时，这可以由 bbswitch 更安全地完成，其由一个内核包构成，会自动发出正确的 ACPI 信号以在不需要时禁用专用 GPU，或开机自动禁用。

注意：bbswitch 从内核版本 4.8 开始不使用 PCI-E 接口电源管理方法。请参阅 Bumblebee#Broken power management with kernel 4.8 以了解详细信息。

: 译者注：由于 Bumblebee 页面年久失修，因此上述链接的段落不存在于本维基，您需要前往英文 ArchWiki 的相应页面查看。

使用 acpi_call

另外，对于 bbswitch 不支持的 GPU，相同的效果可以由手动安装的 acpi_call包 来完成。

提示：对于不在官方仓库内的内核，可以改为安装 acpi_call-dkms包，另见 DKMS。

安装后加载内核模块：

```
# modprobe acpi_call
```

内核模块加载后，执行位于 `/usr/share/acpi_call/examples/turn_off_gpu.sh` 的脚本。

该脚本会遍历并尝试关闭所有的数据总线，您会看到类似如下的输出：

```
# /usr/share/acpi_call/examples/turn_off_gpu.sh
Trying _SB.PCI0.P0P1.VGA._OFF: failed
Trying _SB.PCI0.P0P2.VGA._OFF: failed
Trying _SB_.PCI0.OVGA.ATPX: failed
Trying _SB_.PCI0.OVGA.XTPX: failed
Trying _SB.PCI0.P0P3.PEGP._OFF: failed
Trying _SB.PCI0.P0P2.PEGP._OFF: failed
Trying _SB.PCI0.P0P1.PEGP._OFF: failed
Trying _SB.PCI0.MXR0.MXM0._OFF: failed
Trying _SB.PCI0.PEG1.GFX0._OFF: failed
Trying _SB.PCI0.PEG0.GFX0.DOFF: failed
Trying _SB.PCI0.PEG1.GFX0.DOFF: failed
Trying _SB.PCI0.PEG0.PEGP._OFF: works!
Trying _SB.PCI0.XVR0.Z01I.DGOF: failed
Trying _SB.PCI0.PEGR.GFX0._OFF: failed
Trying _SB.PCI0.PEG.VID._OFF: failed
Trying _SB.PCI0.PEG0.VID._OFF: failed
Trying _SB.PCI0.P0P2.DGPU._OFF: failed
Trying _SB.PCI0.P0P4.DGPU.DOFF: failed
Trying _SB.PCI0.IXVE.IGPU.DGOF: failed
Trying _SB.PCI0.RP00.VGA._PS3: failed
Trying _SB.PCI0.RP00.VGA.P3MO: failed
Trying _SB.PCI0.GFX0.DSM._T_0: failed
Trying _SB.PCI0.LPC.EC.PUBS._OFF: failed
Trying _SB.PCI0.P0P2.NVID._OFF: failed
Trying _SB.PCI0.P0P2.VGA.PX02: failed
Trying _SB_.PCI0.PEGP.DGFX._OFF: failed
Trying _SB_.PCI0.VGA.PX02: failed
```

看到 "works" 了么？这说明脚本找到了一个 GPU 所在的总线并已将该 GPU 关闭。您的电池续航延长应当能够证明这一点。

提示：若您在禁用 GPU 后遇到系统休眠和挂起问题，请尝试通过向其发送相应的 acpi_call 以重新启用它，另见电源管理/挂起与休眠#自定义 systemd 单元。

自动关闭 GPU

目前，GPU 会在下次启动时重新启用，欲解决，请在启动时加载模块：

```
/etc/modules-load.d/acpi_call.conf
#Load 'acpi_call.ko' at boot.
acpi_call
```

在启动时加载

可以使用 systemd-tmpfiles 在启动时关闭 GPU：

```
/etc/tmpfiles.d/acpi_call.conf
w /proc/acpi/call - - - - \_SB.PCI0.PEG0.PEGP._OFF
```

以上配置会在启动时被 systemd 加载。其将特定的 OFF 信号写入 `/proc/acpi/call` 文件。显然您需要将配置中的 `_SB.PCI0.PEG0.PEGP._OFF` 修改为适用于您的系统的配置（注意您需要转义反斜杠）。

在 X 服务器初始化后加载

在部分系统上，在 X 服务器初始化前禁用专用 GPU 可能导致系统挂起。这种情况下，在 X 服务器初始化后可能是个更好的选择，可以由显示管理器来完成。在 LightDM 上，一个 *display-setup-script* seat 配置参数能够以 root 权限执行禁用 GPU 的脚本。如果使用 SDDM 则可以将 `echo "_SB.PCI0.PEG0.PEGP._OFF" > /proc/acpi/call` 添加到 `/usr/share/sddm/scripts/wayland-session` 或 `/usr/share/sddm/scripts/Xsession`，分别对应使用 Wayland 或 Xorg，将 `_SB.PCI0.PEG0.PEGP._OFF` 修改为适用于您的系统的配置。

System76

一些 System76 笔记本（如 Oryx Pro）有其自己的混合图形选项。欲使用其自己的混合图形选项，请安装 system76-powerAUR，启用 `system76-power.service` 并运行 `system76-power graphics hybrid`。

完全关闭专用 GPU

首先运行 `system76-power graphics integrated` 以确保您正在使用集成图形模式，然后重启电脑。在集成模式内，执行 `system76-power graphics power off` 以关闭专用 GPU，该命令并非永久生效，需要每次启动都执行一次。

疑难解答

部分应用程序启动时间延迟 30 秒

**本文或本章节可能需要合并到Vulkan。**

附注： 和 Vulkan#AMDGPU - Vulkan 应用程序启动过慢 及其相似，只是本文完全没有为相应的环境变量设定适当的参数。（在 Talk:混合图形技术 中讨论）

Vulkan 在被调用时会尝试初始化 `/usr/share/vulkan/icd.d/nvidia_icd.json` 中指定的可安装客户端驱动程序（Installable Client Driver，ICD），nvidia-utils包 配置该文件引用 `libGLX_nvidia` 驱动，以向 Vulkan 提供 GPU 驱动的路径。然而，若 GPU 被禁用，该驱动会初始化失败，导致部分应用程序（如基于 Chromium/Electron 的应用程

序）启动需要等待 30 秒的超时。欲阻止 Vulkan 加载驱动以减缓超时，您可以使用 `VK_DRIVER_FILES` 环境变量覆盖 ICD JSON 文件的路径：

```
$ export VK_DRIVER_FILES=
```

禁用 NVIDIA 专用 GPU 后依然高功耗

如果使用 acpi_call 禁用专用 GPU 后功耗依然居高不下，请使用 `lsmod` 检查 nouveau 内核模块是否被加载，若没有则确保其已被安装，`/etc/modprobe.d/` 中所有 `.conf` 文件中屏蔽 Nouveau 的条目被移除，并且 Nouveau 内核模块在启动时被自动加载。重启后功耗应该就降低了。

提示：请参阅内核模块以了解更多关于内核模块加载和屏蔽的详细信息。

注意：如果重启后亮度控制出现问题，`/sys/class/backlight` 有多个目录，请将 `acpi_backlight=native` 添加到内核参数。

2.6 图形处理器

难度 进阶 | **篇幅** 13.2 KB | **所属** 图形

本节将讲到

- 安装
- 加载
- 监控

图形处理器 (graphics processing unit, GPU) 是计算机中负责生成显示器所显示画面的硬件，也经常（不严谨地）称为显卡 (graphics card)。存在两种类型的设备：集成显卡 (integrated GPU, iGPU) 和独立显卡 (discrete GPU, dGPU)。在中文语境下，有时集成显卡也会不严谨地称为“核显”。

由于 GPU 擅长并行计算，因此催生了通用图形处理器（英语：general-purpose computing on graphics processing units） (GPGPU)。

安装

Linux 内核中已经包含了用于硬件加速帧缓冲的显卡驱动。然而，OpenGL、Vulkan、硬件视频加速和 Xorg 的 2D 加速仍然需要用户空间驱动支持。

首先，识别设备型号（输出中的第一行以及 *Subsystem* 一项显示了具体型号）：

```
$ lspci -v -nn -d ::03xx
```

提示：这里的 `::03` 表示“显示控制器 PCI 设备类”，并且 `xx` 表示“该类的任何子类”。您可以在安装 hwdata包 后，在 `/usr/share/hwdata/pci.ids` 文件中搜索 `Display controller` 查看。

然后，安装所需的显卡驱动。参阅下方三个品牌的表格。

对于 32 位软件，需要启用 multilib 仓库并安装以 *lib32-* 为前缀的用户空间驱动，例如 lib32-mesa包、lib32-vulkan-radeon包、lib32-nvidia-utils包 等等。

对于 X11，其它 Device Dependent X (DDX) 驱动可以在 xorg-drivers包组 中找到，或者可以搜索 xf86-video。大部分情况下，您不需要安装任何 DDX 驱动；详情参阅 Xorg#驱动安装。

AMD

AMD 支持开源驱动。曾经提供过一个专有驱动，但已经不再打包了。

提示：要寻找从型号名（如 *Radeon RX 6600*）到 GPU 家族名（如 *RDNA 2*）的对照表，参阅 Wikipedia:List of AMD graphics processing units#Features overview.

有关各个 GPU 家族对 VA-API 的支持情况，参阅硬件视频加速#对照表了解详情。

文档	GPU 家族	DRM 驱动	OpenGL	Vulkan	VA-API
AMDGPU	GCN 3 及更新（如 RDNA 1-4）	包含在 Linux 中	mesa包	vulkan-radeon包	mesa包
AMDGPU[1] / ATI	GCN 1&2	取决于选择的驱动			
ATI	TeraScale 1-3	无			
R300 到 R500	无				
R100 & R200	mesa-amber包				
Rage 4 及更旧	无可用驱动				

1. 自从 linux包≥6.19 以来默认启用，但其余版本也可以手动选择驱动

Intel

Intel 提供并支持开源驱动，除了旧的基于 PowerVR 的显卡（GMA 3600 系列）不支持以外。

提示：Intel 所说的 Gen *N* 硬件并不是指 CPU 的代数，它指的是 GPU 的代数，和 CPU 的不一样。

有关各个 GPU 家族对 VA-API 的支持情况，参阅硬件视频加速#对照表了解详情，下面只列出了软件包。

文档	GPU 家族	DRM 驱动	OpenGL	Vulkan	VA-API
Intel 图形处理器	Gen 12.1 及更新	包含在 Linux 中	mesa包	vulkan-intel包[1]	intel-media-driver包
Gen 8 到 11	intel-media-driver包 或旧版 libva-intel-driver包				
Gen 7 & 7.5	libva-intel-driver包				
Gen 5 & 6	无				
Gen 3 到 4.5	无				
Gen 2	mesa-amber包				
Gen 1	无可用驱动				

1. Gen 7 和 7.5 只有不完整的支持，Gen 8 被限制在 Vulkan 1.3。

NVIDIA

NVIDIA 不支持完全开源的驱动。他们从 2022 年起改为使用一种混合方案（开源内核 DRM 驱动和闭源用户空间驱动）。

提示：要寻找从型号名（如 *RTX 4060*）到 GPU 家族名（如 *NV190*）的对照表，参阅 nouveau wiki 上的代号页面。

参阅硬件视频加速#NVIDIA以了解支持哪些 API，并参阅硬件视频加速#VDPAU 驱动了解各个 GPU 家族的支持详情。

许可	文档	GPU 家族	DRM 驱动	OpenGL	Vulkan
开源	Nouveau1	Kepler (NVE0/GKXXX) 及更新	包含在 Linux 中	mesa包	vulkan-nouveau包
Rankine (NV30) 到 Fermi (NVC0/GF1XX)	无				
Fahrenheit (NV04/05) 到 Kelvin (NV20)	mesa-amber包				
开源 DRM 驱动、专有用户空间驱动	NVIDIA1	Turing (NV160/TUXXX) 及更新	nvidia-open包	nvidia-utils包	
专有	Maxwell (NV110/GMXXX) 到 Ada Lovelace (NV190/ADXXX)	nvidia-580xx-dkmsAUR	nvidia-580xx-utilsAUR		
Kepler (NVE0/GKXXX)	nvidia-470xx-dkmsAUR	nvidia-470xx-utilsAUR			
Fermi (NVC0/GF1XX)	nvidia-390xx-dkmsAUR	nvidia-390xx-utilsAUR	无		
Tesla (NV50/G80-90-GT2XX)	nvidia-340xx-dkmsAUR	nvidia-340xx-utilsAUR			
Curie (NV40/G70) 及更旧	不再打包				

1. 对于同时使用集成显卡和独立显卡、且启用了 NVIDIA Optimus 的笔记本电脑，参阅 NVIDIA Optimus。

加载

大部分驱动的内核模块会在系统启动时自动加载。

如果没有自动加载，那么：

- 确保您**没有**在内核参数中设置 `nomodeset` 或 `vga=`，因为加载内核显卡驱动需要内核级显示模式设置。
- 以及，检查您是否使用内核模块#黑名单禁用了显卡驱动。

监控

监控 GPU 的工具经常用于查看 GPU 的温度、核心和显存 (VRAM) 使用率、以及 P-state。

CLI

- **amdgpu_top** — 显示 AMDGPU 使用率的工具

: https://github.com/Umio-Yasuno/amdgpu_top || amdgpu_top包

- **nvtop** — 支持 AMD、Intel 和 NVIDIA 的 GPU 进程监视器

: https://github.com/Syllo/nvtop || nvtop包

- **radeontop** — GPU 使用率查看器，支持显示总体的活动百分比和各个部分的使用情况。支持 ATI R600 或更新的显卡，以及使用 AMDGPU 驱动的显卡。

: https://github.com/clbr/radeontop || radeontop包

- **nvidia-smi** — NVIDIA 专有驱动捆绑的命令行监控工具，支持 Fermi 或更新的 NVIDIA GPU

: https://docs.nvidia.com/deploy/nvidia-smi/index.html || nvidia-utils包

GUI

- **amdgpu_top** — 显示 AMDGPU 使用率的工具

: https://github.com/Umio-Yasuno/amdgpu_top || amdgpu_top包

- **AmdGuid** — 一个基础的风扇控制 GUI，完全使用 Rust 编写

: https://github.com/Eraden/amdgpud || amdguid-wayland-binAUR, amdguid-glow-binAUR

- **Radeon Profile** — 用于读取和更改 AMD Radeon 显卡当前时钟频率的 Qt5 工具

: https://github.com/emerge-e-world/radeon-profile || radeon-profile-gitAUR

- **TuxClocker** — 基于 Qt5 的监控和超频工具

: https://github.com/Lurkki14/tuxclocker || tuxclockerAUR

- **Plasma System Monitor** — 一个显示系统传感器、进程信息和其它系统资源的界面。捆绑在 KDE Plasma 中。

: https://apps.kde.org/plasma-systemmonitor/ || plasma-systemmonitor包

- **LACT** — 在 Linux 系统上控制您的 AMD、Nvidia 或 Intel GPU

: https://github.com/ilya-zlobintsev/LACT || lact包

2.7 虚拟现实(VR)

难度 进阶 | **篇幅** 13.5 KB | **所属** 图形

本节将讲到

- 环境兼容列表
- 支持的设备和工具集
- 过时的设备和工具集
- 支持的软件
- 其他注意事项

虚拟现实是使用各种外围设备，头戴式显示器或CAVEs和跟踪器为用户模拟环境的一系列过程。虚拟显示技术不会从屏幕上向您显示静态视点，而是将您的视点相对于您站立的位置（在贴头或投影的表面上）进行渲染，以提供与您自己的眼睛观看视角相同的效果。

许多虚拟现实的设备已经发布或即将发布，这为人们带来了不是那么昂贵，身临其境的虚拟现实环境。这些设备大多数都具有全部或部分Linux支持，并且许多具有AUR软件包。

环境兼容列表

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：Use Template:Yes and question marks.（在Talk:虚拟现实(VR))讨论）

Legend:

- 绿色: 原生支持
- 黄色: 通过工具集支持或部分支持
- 红色: 不完整支持
- 无色: 未知/未完成/计划支持

	Oculus Rift	OSVR	OpenVR	Leap Motion	Razer Hydra
Dolphin (从源仓库 fork 的 VR 版本)			部分完成		
Dolphin (官方 OSVR 支持)	通过 OSVR			通过 OSVR	通过 OSVR
Minecrift (Minecraft VR)		(计划) 通过 OSVR-SteamVR	(计划)	(计划) 通过 OSVR	

	Oculus Rift	OSVR	OpenVR	Leap Motion	Razer Hydra
Janus VR					通过 OSVR
Team Fortress 2	通过 OpenVR	通过 OSVR-SteamVR	不完整支持，随机修复状态	通过 OSVR	通过 OSVR
Half Life 2	显示处于一只眼黑，一只眼无色的状态	通过 OSVR-SteamVR	显示处于一只眼黑，一只眼无色的状态	通过 OSVR	通过 OSVR
VRUI VR 工具集和演示					
4089: The Ghost Within	通过 OpenVR	通过 OSVR-SteamVR	直到V社修复问题之前都不完善	通过 OSVR	通过 OSVR
Games/Programs in Wine	在 OVRSDK 版本 <=0.5.0.0, 且 oculus-wine-wrapper-gitAUR\[损坏的链接：package not found\] 和 wine-unity3d-gitAUR\[损坏的链接：package not found\]上支持	跟踪器工作完美。 但是在Unity演示没有Render Manager的工作时有问题，使用Render Manager的Unity演示会出现白屏或者黑屏。			

支持的设备和工具集

OpenXR

OpenXR是一种开放的，不收费用的标准，用于访问虚拟现实以及增强现实的平台和设备。它由Khronos集团维护，并被大多数行业所采用，大多数运行时都支持OpenXR。

Monado

Monado是由Collabora开发的开源OpenXR运行时。它正在开发中，旨在提供支持大多数耳机的通用运行时。你可以在 https://monado.freedesktop.org/ 上查看进度

OpenVR / SteamVR

Valve致力于通过OpenVR创建用于VR开发的开放API。不幸的是，当API开放时，实际的默认实现（SteamVR）却没有进行开放。它提供了一个OpenXR标准的运行时。

配置

安装 Steam, 通过Steam里的下载按钮下载SteamVR。

故障排除

配置和启动错误

SteamVR / OpenVR创建一个配置目录~/.openvr，该目录可能在不同的版本上出现配置错误。删除该目录，然后完全卸载/重新安装SteamVR可以解决。

在某些配置下，访问Rift显然也很麻烦。一种替代方法是使用OSVR-SteamVR驱动程序和OSVR-Oculus-Rift插件。

OpenHMD

OpenHMD 目标是为沉浸式技术提供免费和开源的API和驱动程序，例如具有内置头部跟踪功能的头戴式显示器。目标是在便携式跨平台封装中实现尽可能多设备的支持。

OpenHMD支持多种设备，例如Oculus Rift，HTC Vive，Sony PSVR，Deepoon E2等。

可以从第三方获得 .NET，Java，Perl，Python和Rust的编程语言绑定。

安装

安装 openhmd-gitAUR.

SteamVR 支持

可以将OpenHMD与SteamVR一起使用。为此，您需要安装 steamvr-openhmd-gitAUR 并创建符号链接，该链接指向SteamVR驱动程序目录中的OpenHMD SteamVR驱动程序，例如：

```
ln -s /usr/lib/steamvr/openhmd ~/.steam/steam/steamapps/common/SteamVR/drivers/openhmd
```

过时的设备和工具集

OSVR

OSVR是由Sensics，Inc（一家历史悠久的VR公司）和Razer共同努力创建的VR的完全或几乎完全开放的软件API，最终开发人员只需要将各自的头戴式耳机挂接一些功能即可获得一流的支持。它支持最广泛的外围设备，并通过JSON配置文件具有极其灵活的配置。

它还提供了一个插件，使其可以用作OpenVR实施，使您可以使用它与支持的任何设备一起玩OpenVR / SteamVR游戏。

配置

安装 osvr-core-gitAUR, 还有一些其他需要支持的独立设备，现在支持的插件有：osvr-oculus-rift-gitAUR\[损坏的链接：package not found\]

osvr-leap-motion-gitAUR

如果希望将OSVR与SteamVR / OpenVR游戏和应用程序一起使用，安装 osvr-steamvr-gitAUR链接驱动到SteamVR， 运行OSVR服务器，并通过以下命令使其在后台运行:

```
osvr_server /usr/share/osvrcore/sample-configs/your_device_config.json
```

您可能希望自定义配置以适合您的个人需求。

要测试您的安装是否正常并且跟踪器可用，请安装osvr-tracker-viewer-gitAUR 和执行 `OSVRTrackerView`. 您应该看到OSVR可以拾取的每个跟踪器的一组轴。如果不这样做，请运行 `osvr_print_tree` 以查看可用的跟踪器或是否存在配置问题。

Oculus Rift

Oculus Rift 已经停止支持Linux了，更多请见

Oculus Rift.

Leap Motion

Leap Motion是一款价格实惠的手持跟踪器，可以轻松地安装在HMD的面板上，以允许您与虚拟对象进行交互。不幸的是，最新的Orion软件不适用于Linux，因此当前可用的跟踪功能正常但存在很多错误，它仅真正适用于Linux上的社交互动。但由于它的成本不到HMD或等效跟踪系统的1/10，因此它仍然是相当有用的设备。

配置

安装 leap-motion-driverAUR, osvr-leap-motion-gitAUR 可选择安装 leap-motion-sdkAUR.

要配置， 启用 `leapd.service` 需执行 `LeapControlPanel`. 要测试跟踪是否有效，请运行安装随附的 `Playground`。

支持的软件

当前，有一些应用程序可以在Rift和Linux上正常运行，其中一些应用程序位于AUR中。

Dolphin (VR fork 版本)

dolphin-emu-vr-gitAUR\[损坏的链接：package not found\] 是Gamecube的仿真器，具有一些修补程序，可使其具有完整的头部追踪立体渲染效果，并带有许多自定义功能，使游戏在VR中开箱即用（例如，禁用剔除功能，让您查看整个世界）。

鉴于Dolphin项目已开始正式支持上游OSVR，因此大部分支持已停止。

注意：当将此应用程序与Rift一起使用时，它可以在纵向（直接）模式下正常工作，并且应在不旋转Rift的情况下运行，这样可以最小化延迟。

Dolphin (官方 OSVR 支持)

olphin项目已开始致力于使用OSVR正式增加对VR的支持，可通过 dolphin-emu-osvr-gitAUR获得支持。 它甚至可以使用OSVR的路径树输入作为控制器输入，这样您就可以使用收缩或六轴控制器输入作为Wiimote输入。但是，在某些地方支持有限，因为和原始的Oculus-only 的fork版本差别较大。

wine中的游戏和程序

使用Wine时，许多应用程序具有一定程度的兼容性问题，但通常需要进行一些程度的调整才能使其按预期运行。

oculus-wine-wrapper-gitAUR\[损坏的链接：package not found\] 是一个实用工具，用于在运行Wine时修补Oculus SDK的Linux和Windows版本之间的差异。它为Wine应用程序创建要使用的共享内存上下文，从而允许该应用程序访问本机Oculus SDK。但似乎没有必要将SDK安装到wineprefix。

Unity 游戏

为了在基于Unity的游戏中获得最佳性能，理想情况下，您应使用`-force-opengl`使其强制为opengl模式。 但是，对于未打补丁的Wine上，目前尚不可能做到这一点，因为它试图强制执行的WGL上下文与典型的GLX上下文有一些差异有相关描述\[失效链接 2021-11-19 ⓘ\]。 使用 wine-unity3d-gitAUR\[损坏的链接：package not found\] 包将使您可以使用本机OpenGL运行这些游戏，从而可以在计算机上以不错的性能进行游戏。但是，玩家尝试更改视频模式或将设置弄乱，因此需要提供默认的视频设置。此外，由于它使用的是本机OpenGL，因此nvidia的__GL_THREADED_OPTIMIZATIONS可以显着提高性能。总体而言，该命令应如下所示：

```
env __GL_THREADED_OPTIMIZATIONS=1 wine UnityGame.exe -screen-height 1080 -screen-width 1920 -popupwindow -force-opengl
```

或者使用 oculus-wine-wrapper-gitAUR\[损坏的链接：package not found\]:

```
env __GL_THREADED_OPTIMIZATIONS=1 oculus-wine-wrapper UnityGame.exe -screen-height 1080 -screen-width 1920 -popupwindow -force-opengl
```

Minecrift (Minecraft VR)

可在这里找到使用最新的JRift（本机Java rift运行时）版本运行现有Minecrift版本的指南。 您可能会希望在使用Rift时安装较旧的版本，因为JRift已不断更新以匹配Oculus SDK。

另外，许多用户报告说使用JRE8的性能要好于JRE7。

注意：使用Rift时，此应用程序可在纵向（直接）模式下正常工作，并且应在不旋转Rift的情况下运行。但是，从15年5月12日开始，游戏中的GUI将显示为拉长的16：9矩形，而不是停留在屏幕比例上。这虽然是可用的，但是表现不理想。

JanusVR

“janusVR：一个沉浸式，协作的多维互联网。” JanusVR是一款应用程序，可让您在多人游戏体验中浏览3D网站。janusvr的AUR包可以使用: janusvrAUR

当JanusVR升级时，AUR软件包不会自动更新，但是当有新版本可用时，应用程序会告诉您，发生这种情况时，只需重新构建新软件包即可。

注意：此应用程序可在纵向（直接）模式下正常使用（从42.3开始），并且应在不旋转Rift的情况下运行。

Leap Motion 支持

Leap Motion使您可以通过手势输入，并且上世界上的其他人看到。您将需要将Leap安装到HMD的前面，并确保使用的是默认的头像。

其他注意事项

启用OpenAL双耳模式支持

请查看 Gaming#Binaural audio with OpenAL.

2.8 外接显卡

难度 进阶 | **篇幅** 16.4 KB | **所属** 图形

本节将讲到

- 安装
- 显卡计算
- Xorg
- Wayland

翻译状态：

- 本文（或部分内容）译自 External GPU，最近一次同步于 2024-07-20，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

在搭载了雷电 3+#Thunderbolt_3) 或 USB4的设备上，可以通过显卡拓展坞连接一张桌面级的外接显卡（eGPU）。eGPU.io 是一个专门讨论外接显卡的社区论坛。尽管大多数时候仍需一些手动配置（如下），Linux 对外接显卡的支持相当不错。

注意：对于搭载了 USB4 的笔记本，其传输速率一般被设置为最小值（20 Gbit/s）。可以手动将其设置得更高（40，80 或更高）来获取更佳的体验。如果您的拓展坞使用雷电接口，可以启用笔记本的 USB4 模式来达到相同效果。

安装

雷电

使用雷电接口的拓展坞在插入前可能首先需要经过设备认证（取决于 BIOS/UEFI 配置）。详见 Thunderbolt#User device authorization。如果成功插入，外接显卡将在 `lspci` 中显示。

```
$ lspci | grep -E 'VGA|3D'
00:02.0 VGA compatible controller: Intel Corporation UHD Graphics 620 (rev 07)
# 集成显卡
1a:10.3 VGA compatible controller: NVIDIA Corporation GP107 [GeForce GTX 1050] (rev a1)
# 外接显卡
```

根据电脑的硬件和拓展坞的硬件情况，雷电接口会限制主机与外接显卡间的带宽。限制程度取决于 PCIe 通道数和封装的DMI互联接口（OPI）模式：

```
# dmesg | grep PCIe
[19888.928225] pci 0000:1a:10.3: 8.000 Gb/s available PCIe bandwidth, limited by 2.5
GT/s PCIe x4 link at 0000:05:01.0 (capable of 126.016 Gb/s with 8.0 GT/s PCIe x16 link)
```

驱动

您应该安装与您的外接显卡兼容的驱动程序：

- AMDGPU AMD 驱动
- NVIDIA 专有 NVIDIA 驱动
- Nouveau 开源 NVIDIA 驱动

成功安装后，`lspci -k` 应当显示显卡与驱动已经成功绑定：

```
$ lspci -k
1a:10.3 VGA compatible controller: NVIDIA Corporation GP107 [GeForce GTX 1050] (rev a1)
        Subsystem: NVIDIA Corporation GP107 [GeForce GTX 1050]
        Kernel driver in use: nvidia
        Kernel modules: nouveau, nvidia_drm, nvidia
```

注意，在某些情况下，AMDGPU 驱动（无论使用雷电接口还是 USB4）可能会被设置为错误的 *pcie_gen_gap* 模式，然后回滚到 PCIe 1.1 的速率。这可能会导致性能严重降低。 可以通过调整模块参数（见内核模块#使用 /etc/modprobe.d/中的文件) 或传递内核参数来解决以上问题。

```
/etc/modprobe.d/amd-egpu-pcie-speed.conf
options amdgpu pcie_gen_cap=0x40000
```

这会将速率设置为 PCIe 3 模式。完整的参数选项见 amd_pcie.h。

显卡计算

在完成 #安装之后，像 GPGPU#CUDA 这样无需在屏幕上显示的纯计算工作，理论上无需额外配置即可使用。*nvidia-smi* 工具（由 nvidia-utils包 提供）在使用专有 NVIDIA 驱动时应该能正常运行。专有的 NVIDIA NVENC/NVDEC 也应该正常运行（不包括与 OpenGL 的互操作）。

在这种使用场景下，热插拔也应该受支持（对拔出支持取决于驱动程序）。不过，对于 NVIDIA 设备，使用 `nvidia-persistenced` 会导致热拔出时设备出现故障。

Xorg

通过一些额外配置，您可以将集成显卡（iGPU）和外接显卡（eGPU）结合起来使用。这样配置有其优点，但也会带来一些额外的问题。

Xorg 在外接显卡上渲染，PRIME 显示分载到集成显卡

- 大多数使用显卡的程序可以正常执行在外接显卡上：`glxinfo`/`glxgears`, `eglinfo`/`eglgears_x11`, `NVENC`/`NVDEC`（包括与 OpenGL 的互操作）。
- Xorg 仅能在外接显卡接入后启动。
- 连接到外接显卡上的显示器开箱即用，而 PRIME显示分载适用于连接到集成显卡的显示器（例如笔记本内屏）。

有关这些内容，见 PRIME#将独立显卡作为主 GPU 和 PRIME#反向 PRIME。 同时，NVIDIA 官方驱动文档也有教程： Chapter 33. Offloading Graphics Display with RandR 1.4.

Xorg 的配置文件应该像如下所示：

```
/etc/X11/xorg.conf.d/80-egpu-primary-igpu-offload.conf
Section "Device"
    Identifier "Device0"
    Driver     "nvidia"
    BusID      "PCI:26:16:3"                  # 根据 lspci 填写，将十六进制转换为十进制。
    Option     "AllowExternalGpus" "True"     # 专有 NVIDIA
 驱动所需。
EndSection

Section "Module"
    # 为集成显卡加载 modesetting 模块，其提供程序应当是 XrandR 1.4。
    Load "modesetting"
EndSection
```

注意：Xorg 使用十进制的总线 ID，但大多数工具使用十六进制。在 `xorg.conf` 中，必须将形如 `1a:10.3` 的总线 ID转换为 `26:16:3`。

提示：在更新的 Xorg 中，无需指定 `ServerLayout` 和 `Screen` 项，这些项将会被自动推断。 第一个定义的 `Device` 会被首先考虑。

要**确认**此次设置是否有效，使用 `xrandr --listproviders` 检查。输出应当如下：

```
Providers: number : 2
Provider 0: id: 0x1b8 cap: 0x1, Source Output crtcs: 4 outputs: 4 associated providers: 0 name:NVIDIA-0
Provider 1: id: 0x1f3 cap: 0xf, Source Output, Sink Output, Source Offload, Sink Offload crtcs: 3 outputs: 5 associated providers: 0 name:modesetting
```

要**输出到笔记本内屏及其他连接到集成显卡上的显示**，可以使用 RandR 1.4 PRIME显示分载。请使用 `xrandr --listproviders` 输出的设备名：

```
xrandr --setprovideroutputsource modesetting NVIDIA-0 && xrandr --auto
```

注意：`xrandr --auto` 是可选的命令，可以使用其他任何基于 RandR 的显示配置工具替代。这条命令只是防止出现屏幕全黑的情况。

要在显示管理器弹出登录页面或启动桌面环境之前执行这条命令，见 Xrandr#Configuration 和 Xinit。

Vulkan 可能会自行枚举 Xorg 上的所有显卡，所以如果要在本配置下执行如 `vkcube` 这样的程序，可能需要传递参数 `--gpu_number 1`。另外，也可以使用此参数以在枚举期间对显卡重新排序：`__NV_PRIME_RENDER_OFFLOAD=1 vkcube` 或者等效的 `prime-run vkcube`。

提示：在笔记本上使用 *optimus-manager* 时，仅需将外接显卡的 `BusID` 放入位于 `/etc/optimus-manager/xorg/` 下的管理模式与显卡的配置文件中，就可以在外接显卡上渲染。

Xorg 在集成显卡上渲染，PRIME 渲染分载到外接显卡

- 程序默认在集成显卡上渲染，但 PRIME 渲染分载可以使程序在外接显卡上渲染。
- Xorg 可以在未连接外接显卡时启动，但在连接外接显卡后，必须重启 Xorg 才能使用渲染与显示分载。
- 连接到集成显卡上的显示器（例如笔记本内屏）开箱即用，而 PRIME 显示分载适用于连接到外接显卡上的显示器。

详细内容见 PRIME#PRIME 显卡分载。NVIDIA 官方驱动文档中也有记录：Chapter 34. PRIME Render Offload。

对大多数独立显卡而言，此模式应当是默认模式，无需手动配置 Xorg。如果存在问题，或是在使用专有 NVIDIA 驱动，应当如下配置：

```
/etc/X11/xorg.conf.d/80-igpu-primary-egpu-offload.conf
Section "Device"
    Identifier "Device0"
    Driver     "modesetting"
EndSection

Section "Device"
```

```
    Identifier "Device1"
    Driver     "nvidia"
    BusID      "PCI:26:16:3"                # 根据 lspci 填写，将十六进制转换为十进制。
    Option     "AllowExternalGpus" "True"   # 专有 NVIDIA 驱动所需。
EndSection
```

要**确认**是否配置成功，执行 `xrandr --listproviders` 时，应当显示如下内容：

```
$ xrandr --listproviders
Providers: number : 2
Provider 0: id: 0x47 cap: 0xf, Source Output, Sink Output, Source Offload, Sink Offload crtcs: 3 outputs: 5 associated providers: 0 name:modesetting
Provider 1: id: 0x24a cap: 0x2, Sink Output crtcs: 4 outputs: 4 associated providers: 0 name:NVIDIA-G0
```

**本文或本章节可能需要合并到PRIME#配置应用程序使用 GPU 渲染。**

附注： 也许应该在专门的页面探讨这些问题，而不是在这里。（在 Talk:外接显卡 中讨论）

要让某些程序在外接显卡上渲染，可以用以下方式启用 PRIME 渲染分载：

- 对于专有 NVIDIA 驱动：

```
    $ __NV_PRIME_RENDER_OFFLOAD=1 __VK_LAYER_NV_optimus=NVIDIA_only __GLX_VENDOR_LIBRARY_NAME=nvidia some_program
```

- 对于专有 NVIDIA 驱动（封装版）：

```
    $ prime-run some_program
```

- 对于开源驱动：

```
    $ DRI_PRIME=1 some_program
```

要**输出到连接至外接显卡的显示器**，还可以这样使用 RandR 1.4 PRIME 显示分载：

```
$ xrandr --setprovideroutputsource NVIDIA-G0 modesetting && xrandr --auto
```

提示：提供程序的顺序并不相同，同时 NVIDIA 的命名也稍有不同。

NVIDIA 驱动版本 460.27.04+ 优化了在一些特定情形下对渲染和显示分载的结合支持：

: 添加了对 “反向 PRIME 绕行”（Reverse PRIME Bypass）的支持，这使得当某一渲染分载程序处于全屏，非重定向，且仅在给定的使用 NVIDIA 驱动的 PRIME 显示分载输出中显示时，可以绕过 PRIME 渲染分载和 PRIME 显示分载的带宽限制。当 X 服务器启用详细日志时，可以在 X 日志中找到此特性的使用情况。

在外接显卡上执行多个独立 Xorg 实例

详见 Nvidia-xrun#External GPU setup.

在 Xorg 上使用外接显卡的已知问题

- 大多数独立显卡的 Xorg 驱动不支持热插入：必须在启动 Xorg 前插入外接显卡。注销后重新登录即可重启 Xorg。

- 完全不支持热拔出：这样会导致系统不稳定或卡死（根据 NVIDIA 官方文档）。

Wayland

有关 Wayland 对外接显卡（或多显卡）支持程度的测试目前尚为数不多，不过理论上其相对于 Xorg 来说配置会更为便捷。

需要注意的是，Wayland 混成器尚无对显卡热插拔的明确支持，不过大多数混成器已经在某种程度上支持了：

- KDE's kwin: https://invent.kde.org/plasma/kwin/-/merge_requests/811
- GNOME's Mutter: https://gitlab.gnome.org/GNOME/mutter/-/issues/17, https://gitlab.gnome.org/GNOME/mutter/-/merge_requests/1562
- wl-roots: https://gitlab.freedesktop.org/wlroots/wlroots/-/issues/1278

使用开源驱动时，需要这样执行 DRI 分载：

```
$ DRI_PRIME=1 some_program
```

一些像 all-ways-egpu 这样的项目正致力于在 Wayland 下提供更为便捷的显卡切换方法。

热插拔 NVIDIA 外接显卡

Wayland 支持在使用 PRIME 时热插拔外接显卡。NVIDIA 独立显卡对 PRIME 的优秀支持同样适用于外接显卡。

首先，确保没有进程正在使用 NVIDIA 模块。每个 EGL 程序会占用 1MB 的独显显存，即使其运行在集成显卡上。可以通过 `nvidia-smi` 查看是否有此类进程。 为防止出现此类问题，请将 `__EGL_VENDOR_LIBRARY_FILENAMES=/usr/share/glvnd/egl_vendor.d/50_mesa.json` 设置为环境变量。 不过，最好将其放在 `/etc/environment.d/50_mesa.conf` 中。

然后，卸载 NVIDIA 模块：

```
# rmmod nvidia_uvm
# rmmod nvidia_drm
# rmmod nvidia_modeset
# rmmod nvidia
```

模块卸载完成后，就可以连接外接显卡。显卡初始化完毕后，使用 `modprobe nvidia-drm` 或 `modprobe nvidia-current-drm` 重新加载 NVIDIA 模块。使用哪条命令取决于模块的来源，例如从 NVIDIA 官网下载或是使用软件包管理器直接安装。在某些情况下（例如 GIGABYTE AORUS GAMING BOX）外接显卡可能不兼容

专有驱动，因此需要使用开源驱动的命令：`modprobe nvidia-current-open-drm`。

模块加载成功后，PRIME 就应该能正常使用，不过因为我们设置了 `__EGL_VENDOR_LIBRARY_FILENAMES` 变量来使用 MESA，所以在启动程序前需要添加 `__EGL_VENDOR_LIBRARY_FILENAMES=/usr/share/glvnd/egl_vendor.d/10_nvidia.json`。完整的环境变量列表如下：

```
__GLX_VENDOR_LIBRARY_NAME=nvidia __NV_PRIME_RENDER_OFFLOAD=1
__VK_LAYER_NV_optimus=NVIDIA_only
__EGL_VENDOR_LIBRARY_FILENAMES=/usr/share/glvnd/egl_vendor.d/10_nvidia.json %command%
```

对于 GNOME 用户，可能需要修补 switcheroo-control包 来将 `__EGL_VENDOR_LIBRARY_FILENAMES` 包含到环境变量中。这将允许程序原生运行在外接显卡上，不会出现鼠标错位，而且能“使用独立显卡启动”。不过，关于如何修补，就不在本篇文章的讨论范围之内了。

热插拔 NVIDIA 外接显卡，并临时停用 NVIDIA 独立显卡

如果您同时拥有集成显卡（iGPU）和 NVIDIA 独立显卡（dGPU），并且尝试连接 NVIDIA 外接显卡（eGPU），显卡间会产生冲突，这会导致只有独立显卡能够使用。要解决冲突，必须临时停用独立显卡，这样 NVIDIA 驱动就不会识别到它。最好的方法就是覆盖驱动。

首先，卸载 NVIDIA 驱动模块：

```
# rmmod nvidia_uvm
# rmmod nvidia_drm
# rmmod nvidia_modeset
# rmmod nvidia
```

然后，使用 driverctlAUR 工具覆盖独立显卡驱动。在下面的例子中，使用 0000:01:00.0 代表独立显卡的地址。您设备的独立显卡地址可以使用 `lspci` 查看。

```
# driverctl --nosave set-override 0000:01:00.0 vfio-pci
```

建议使用参数 `--nosave`，否则 driverctl 会在系统启动时覆盖驱动。这样，在出错的时候，仅需重启就可以恢复独立显卡。

独立显卡禁用后，就可以使用 `modprobe nvidia-drm` 加载内核模块，然后使用 `nvidia-smi` 检查其显示一张或两张显卡。

重新启动独立显卡有点麻烦，因为它很反直觉。首先，卸载 NVIDIA 模块，拔出外接显卡，然后执行以下命令：

```
# modprobe nvidia-current
# driverctl --nosave unset-override 0000:01:00.0
# modprobe nvidia-current
# driverctl --nosave unset-override 0000:01:00.0
# modprobe nvidia-current-modeset
```

```
# modprobe nvidia-current-drm
```

前两个命令需要执行两次，这看起来尽管很奇怪，不过只有这样才能重新启动独立显卡。第一次执行命令时可能会报错，不过这并不重要。

2.9 Nouveau

难度 进阶 | **篇幅** 17.3 KB | **所属** 图形

本节将讲到

- 安装
- 加载
- 提示与技巧
- 故障排除

本文包含安装和配置NVIDIA显卡开源驱动 Nouveau 的内容. 有关官方闭源驱动的信息请查看NVIDIA.

查找硬件的代号 (Wikipedia 包含更详细的列表), 然后和功能矩阵进行比较，以查看支持的功能。

安装

安装 mesa包 包，它提供了用于 3D 加速的 DRI 驱动程序。

- 为x86_32 提供支持，另外安装 multilib 源中的的 lib32-mesa包 包。
- 对于 DDX 驱动程序（提供 Xorg 中的 2D 加速），安装 xf86-video-nouveau包 包。

注意：有建议指出不安装 xf86-video-nouveau包 驱动，而是使用 modesetting 驱动来支持 NV50 (G80) 及更新版本的硬件，可能会更有利。可以参考一篇来自 2021 年的 用户报告。

详见硬件视频加速

使用 Mesa NVK Vulkan 驱动程序

警告： 这个驱动仍在开发中，因此可能会出现性能问题。一些在开源和专有驱动上正常运行的内容（主要是游戏）可能在使用 NVK 上表现不佳（甚至完全无法运行）。如果玩游戏是一个重要的使用场景，那么在 NVK 成熟之前，您应该避免使用它。

使用 NVK 需要内核版本在6.7及以上， mesa版本在24.1及以上。

在启用 NVK 之前，必须卸载以下所有包（以及它们的 `lib32` 和 DKMS 版本）：

- nvidia包, nvidia-open包, nvidia-lts包, nvidia-betaAUR
- nvidia-settings包, nvidia-utils包
- egl-wayland包

如果你使用的是多 GPU 设备，请确保没有在 `/etc/modprobe.d` 中将 Nouveau 列入黑名单。

注意：您可能需要卸载系统上的所有 GPU 管理器，它们大多数会将模块列入黑名单，这可能会干扰 NVK

然后安装 vulkan-nouveau包（如果需要，则安装 lib32-vulkan-nouveau包）。

使用 `nouveau.config=NvGspRm=1` 内核参数

最后重启系统

可以使用 vulkan-tools包 中的 `vulkaninfo` 来验证,它会显示系统中的 NVIDIA GPU 正在使用 NVK 驱动程序。

```
$ vulkaninfo
...
GPU id : 0 (NVIDIA GeForce RTX 3050 Ti Laptop GPU (NVK GA107)):
        Surface type = VK_KHR_wayland_surface
        Formats: count = 8
...
```

加载

Nouveau的内核模块应该在系统启动时就已自动加载，如果没有的话：

- 确保你的内核参数中没有`nomodeset` 或者 `vga=`， 因为Nouveau需要内核模式设置。
- 另外，确保你没有在 modprobe 配置文件 `/etc/modprobe.d/` 或 `/usr/lib/modprobe.d/` 中屏蔽 Nouveau。
- 检查 dmesg 中有没有 opcode 错误，如果有的话，将 `nouveau.config=NvBios=PRAMIN` 加入内核参数禁止模块卸载
- 检查 `/etc/X11/xorg.conf` 和 `/etc/X11/xorg.conf.d/` 中是否存在任何文件，并且该文件是否引用了 Template:Nvidia 驱动程序。最好将该文件重命名。

尽早启动 KMS

提示：如果你对这个问题的解决有问题的话，请访问这个页面.

Nouveau 驱动依赖Kernel mode setting (KMS)。当系统启动时，KMS 模块会在其它模块之后启用，所以显示的分辨率发生改变。查看Nouveau KernelModeSetting 页面获取更多细节。

可以设置将 KMS 尽早启动，在 initramfs 加载时就接管功能。

将 `nouveau` 加入 `/etc/mkinitcpio.conf` 的 `MODULES` 数组:

```
MODULES="... nouveau ..."
```

如果你使用了一个自定义的EDID文件，你应该像这样把它加入到initramfs 中：

```
/etc/mkinitcpio.conf
FILES="/lib/firmware/edid/your_edid.bin"
```

重新生成初始ramdisk映像：

```
# mkinitcpio -p <kernel preset; e.g. linux>
```

如果 Nouveau 出问题了，不得不多次重建 nouveau-drm 进行测试，请不要在 initramfs 中添加`nouveau`。因为这样会容易忘记重建 initramfs 而使测试更加困难。先使用“延迟启动”，直到系统已经稳定。如果需要自定义固件，使用 initrams 可能会有更多问题（一般不建议）

提示与技巧

保留已安装的Nvidia驱动

如果你想保留已经安装的官方驱动但又想要使用Nouveau驱动，像下面注释掉`/etc/modprobe.d/nouveau_blacklist.conf`
或 `/usr/lib/modprobe.d/nvidia-utils.conf`中的内容

```
#blacklist nouveau
```

你可能还需要注释掉其他优先使用专有驱动的配置文件，例如 systemd-modules-load 的 `/usr/lib/modules-load.d/nvidia-utils.conf` 和 Udev 的 `/usr/lib/udev/rules.d/60-nvidia.rules`。可以使用以下命令检查驱动程序安装了哪些文件：

```
# pacman -Ql nvidia-utils | grep conf
```

然后，确保禁用所有以 `nvidia-` 为前缀的服务，这些服务可能会调用 `nvidia-modprobe` 在启动时加载模块。例如：

```
 $ systemctl status nvidia-persistenced.service
```

如果你使用的是 Xorg，可以通过创建文件 `/etc/X11/xorg.conf.d/20-nouveau.conf` 并添加以下内容，告诉 Xorg 加载 nouveau 驱动，而不是 NVIDIA 驱动：

```
Section "Device"
    Identifier "Nvidia card"
    Driver "nouveau"
EndSection
```

重启以使更改生效。然后通过查看内核消息检查是否加载成功：

```
# dmesg
```

安装最新的开发包

你可以通过AUR安装最新的git包：

- 你可以通过mesa-gitAUR安装最新的Mesa（包含最新的DRI驱动）。

- 你可以通过xf86-video-nouveau-gitAUR 安装最新的DDX驱动。
- 你也可以尝试安装像 linux-mainlineAUR 这样比较新的内核版本，这可能会带来更好的性能。
- 要获得Nouveau最新的更新, 你应该使用AUR中的linux-gitAUR 包，并且编辑 PKGBUILD 以使用Nouveau自己的内核库，目前它位于：git://anongit.freedesktop.org/git/nouveau/xf86-video-nouveau.
- libdrm-gitAUR
- lib32-libdrm-gitAUR
- lib32-mesa-gitAUR

你可以在 Nouveau Source page找到上游驱动源.

注意：在 #安装中提到, xf86-video-nouveau不再是必需的，也不推荐安装，在大多数情况下，未安装该驱动时，您的 GPU 将表现得更好。

双输出

Nouveau 支持xrandr拓展和多显示器，教程详见RandR12

这是一个完整的例子 `/etc/X11/xorg.conf.d/20-nouveau.conf` 用来演示在双输出模式下运行两个显示器。当然，你可能更喜欢像GNOME显示控制中心 (`gnome-control-center display`)这样的图形化配置工具.

```
# the right one
Section "Monitor"
          Identifier    "NEC"
          Option "PreferredMode" "1280x1024_60.00"
EndSection

# the left one
Section "Monitor"
          Identifier    "FUS"
          Option "PreferredMode" "1280x1024_60.00"
          Option "LeftOf" "NEC"
EndSection

Section "Device"
    Identifier "nvidia card"
    Driver "nouveau"
    Option  "Monitor-DVI-I-1" "NEC"
    Option  "Monitor-DVI-I-2" "FUS"
EndSection

Section "Screen"
    Identifier "screen1"
   Monitor "NEC"
    DefaultDepth 24
      SubSection "Display"
       Depth      24
       Virtual 2560 2048
      EndSubSection
    Device "nvidia card"
EndSection

Section "ServerLayout"
    Identifier "layout1"
    Screen "screen1"
```

```
EndSection
```

设置控制台分辨率

使用fbset包工具调整控制台分辨率. 你也可以通过 video= kernel 这样的选项来调整控制台分辨率 (详见 KMS).

电源管理

Nouveau 驱动中缺乏合适的电源管理，大多数显卡在使用过程中会保持在较低的功率状态，频率也较低。对于某些显卡，已有实验性支持的 GPU 重新调频（详见 Nouveau PowerManagement page），并且从内核 4.5 开始，可以通过位于 /sys/kernel/debug/dri/*/pstate 的 debugfs 接口进行控制。

正如 upstream suggested 的那样，这个 debugfs 接口在 Turing 及更新版本的显卡上不可用，仅适用于 Kepler 及更早版本的显卡。

例如，要检查系统中第一块显卡的可用电源状态和当前设置，可以运行：

```
 # cat /sys/kernel/debug/dri/0/pstate
```

也可以通过向该接口写入来手动设置/强制某个电源状态：

```
# echo pstate > /sys/kernel/debug/dri/0/pstate
```

警告： 重新调频仍处于高度实验阶段。手动设置电源状态可能会导致系统挂起、数据损坏或显卡过热。

风扇控制

如果硬件支持，可以通过 `/sys` 控制风扇转速。

```
$ find /sys -name pwm1_enable
/sys/devices/pci0000:00/0000:00:01.0/0000:01:00.0/hwmon/hwmon1/pwm1_enable
$ readlink /sys/devices/pci0000:00/0000:00:01.0/0000:01:00.0/driver
../../../../bus/pci/drivers/nouveau
```

`pwm1_enable` 可以设置为 0, 1 或 2，意思是 NONE, MANUAL 和 AUTO 风扇控制。设置为手动时，可以手动设置 `pwm1`，例如设置为 40 表示 40% 的转速.

警告：风险自担，不要太热烧了显卡!

可以通过 udev 规则设置:

```
$ cat /etc/udev/rules.d/50-nouveau-hwmon.rules
ACTION=="add", SUBSYSTEM=="hwmon", DRIVERS=="nouveau", ATTR{pwm1_enable}="2"
```

参考:

- https://floppym.blogspot.de/2013/07/fan-control-with-nouveau.html
- https://web.archive.org/web/20141031191559/https://kalgan.cc/blog/posts/Controlling_nVidia_cards_fans_with_nouveau_in_Debian/

Optimus

要在笔记本上使用 Optimus(使用两个 GPUs)，请阅读 bumblebee 和 PRIME

垂直同步

Xorg 合成器在使用 Nouveau 时容易出现问题。与大多数合成器不同，Picom 提供了许多选项，可以进行调整以获得更流畅、无撕裂的效果。预计能够提供良好效果的配置如下：

```
$ picom -b --unredir-if-possible --backend xr_glx_hybrid --vsync --use-damage --glx-no-stencil
```

提示： 使用其他合成器时，别忘了关闭窗口管理器（如 KWin）的合成功能。

故障排除

将 `drm.debug=14` 和 `log_buf_len=16M` 添加到您的内核参数中，以启用视频调试：

建立详细的Xorg日志：

```
startx -- -logverbose 9 -verbose 9
```

查看加载的视频模块的参数和值：

```
modinfo -p video
```

禁用 MSI

如果出现模块加载错误或 X 服务器无法启用，请尝试将 `nouveau.config=NvMSI=0` 加入内核参数.

Source: https://bugs.freedesktop.org/show_bug.cgi?id=78441

虚拟输出问题

Nouveau 驱动可能会检测到“虚拟”输出。例如，VGA-1 和 LVDS-1 都显示为连接状态，但实际上只有 LVDS-1 存在。

这会导致显示问题和/或在关闭笔记本盖时无法进入挂起状态。

内核参数

可以通过在内核参数中禁用虚拟输出（如上述示例中的 VGA-1）来解决此问题：

```
video=VGA-1:d
```

此处 **d** 表示禁用(disable)

Nouveau 内核模块还有一个选项，可以禁用 TV 输出检测：

```
 tv_disable=1
```

Xorg 配置

可以通过将以下内容添加到 `/etc/X11/xorg.conf.d/20-nouveau.conf` 来在 Xorg 中禁用虚拟输出：

```
Section "Monitor"
```

```
Identifier "VGA-1"
Option "Ignore" "1"
EndSection
```
Source: http://gentoo-en.vfose.ru/wiki/Nouveau#Phantom_and_unpopulated_output_connector_issues

Xrandr

Xrandr can disable the output:

```
 $ xrandr --output VGA-1 --off
```
This can be added to the xinit configuration.

Random lockups with kernel error messages

Specific Nvidia chips with Nouveau may give random system lockups and more commonly throw many kernel messages, seen with *dmesg*. Try adding the `nouveau.noaccel=1` kernel parameter. See Fedora:Common kernel problems#Systems with nVidia adapters using the nouveau driver lock up randomly for more information.

Note that using `nouveau.noaccel=1` kernel parameter might cause ~%100 CPU usage on Wayland when there is no iGPU or disabled iGPU by factory. You can switch to X11 session or prefer adding `LIBGL_ALWAYS_SOFTWARE=1` environment variable for wayland to disable OpenGL hardware acceleration completely.

As an alternative, you can also use the `QT_XCB_FORCE_SOFTWARE_OPENGL=1` environment variable to disable OpenGL acceleration in Qt applications.

Flat Panel Table Invalid

NVIDIA graphics cards with recent chipsets can cause startup issues - this includes X11 being unable to start and lspci freezing indefinitely.

This can break live distributions/installation media. This can be detected either by running lspci, or checking the systemd journal for the error:

```
nouveau E[     DRM]Pointer to flat panel table invalid
```
The system may start if the Nouveau driver is disabled by passing the following kernel parameters:

```
modprobe.blacklist=nouveau
```
The Nouveau driver can then be loaded using

```
# modprobe nouveau
```
The system should then function correctly. If you have another Nvidia graphics card, or just want to be safe, you can disable the offending card using:

```
$ echo 1 > /sys/bus/pci/devices/[card device id]/remove
```

2.10 NVIDIA Optimus

难度 进阶 | **篇幅** 17.4 KB | **所属** 图形

本节将讲到

- 可用方法
- 仅使用集成显卡
- 仅使用英伟达显卡
- 使用可切换显卡
- 疑难解答

NVIDIA Optimus 是一项允许集成图形处理器（GPU）和英伟达（NVIDIA）独立图形处理器置入并通过一台笔记本电脑访问的技术。作为前提条件，请为两张显卡安装相关的 GPU 驱动。

可用方法

这里有几种可行的方法：

- #仅使用集成显卡 - 能节约用电，因为此时英伟达 GPU 是完全关闭（不通电）的。
- #仅使用英伟达显卡 - 能比集成显卡提供更强的性能表现，但是更耗电（这对移动设备不太友好）。此方法和 optimus-manager 以及 nvidia-xrun 两者一样采用了相同的底层处理，所以在选用一种更自动化的方法之前，应该先利用这个方法来排除故障和验证一些常规功能。
- 两者兼用 (在需要的时候使用英伟达 GPU，其他时候关闭它以节省电量):
- #使用 PRIME 渲染分载（render offload） - 英伟达官方支持方法。
- #使用 optimus-manager - 用一个简单的命令切换显卡（需要注销重新登录来生效）。还支持与 PRIME 渲染分载的混合模式。这可实现需要时使用英伟达 GPU 达到最大性能，不用时则将后者关闭。自 1.4 版本起，还支持 AMD+NVIDIA 组合。
- #使用 nvidia-xrun - 使用英伟达显卡在另外的 TTY 上运行单独的 X 会话。这可实现需要时使用英伟达 GPU 达到最大性能，不用时则将后者关闭。
- #使用大黄蜂（Bumblebee） - 提供了与 Windows 下相似的功能，可以选择需要的程序使用英伟达显卡，而其他的程序则用集成显卡运行。不过大黄蜂有重大的性能问题。
- #使用 switcheroo-control - 类似于大黄蜂，但专为 GNOME 用户设计。允许应用程序在其桌面项文件中指定是否偏好使用独立显卡，并允许你从右键菜单中手动选择在英伟达 GPU 上运行任意应用程序。
- #使用 nouveau - （与英伟达官方提供的专有驱动相比）提供的性能较差，而且可能导致睡眠和休眠问题。不适用于最新的英伟达 GPU。
- #使用 EnvyControl - 类似于 optimus-manager，但不需要复杂的配置或后台运行守护进程，如果你是 GNOME 用户，也不需要安装修补版的 GDM。

- #使用 NVidia-eXec - 类似于大黄蜂，但没有性能影响。它适用于 Xorg 和 Wayland。这个包是实验性的，目前仅在 GNOME/GDM 下进行测试。
- #使用 nvidia-switch - 类似于 nvidia-xrun，但不需要切换 TTY，切换将通过登录和注销在显示管理器中完成。这个包在基于 Debian 的系统上进行测试，但像 nvidia-xrun 一样，它应该可以在所有 Linux 系统上工作。

注意：所有这些方法选择都是互斥的，如果测试了一种方法后又决定使用另一种方法，则必须确保在尝试另一种方法之前，还原通过遵循前一种方法所做的所有配置更改，否则可能会发生文件冲突和不定行为。

仅使用集成显卡

如果你只想使用某个特定的GPU而不进行切换，请检查你系统BIOS中的选项。应该有一个选项可以禁用其中一个显卡。有些笔记本电脑只允许禁用独立显卡，或者反之，但如果你只计划使用其中一个显卡，值得检查一下。

如果你的BIOS不允许禁用Nvidia显卡，你可以从Linux内核本身禁用它。参见Hybrid graphics#完全关闭独立GPU。

在不切换渲染提供程序的情况下使用CUDA

你可以在不将渲染切换到Nvidia显卡的情况下使用CUDA。你只需要确保在启动CUDA应用程序之前Nvidia显卡已通电，详见Hybrid graphics#完全关闭独立GPU。

现在，当你启动CUDA应用程序时，它将自动加载所有必要的内核模块。在使用CUDA后关闭Nvidia显卡之前，必须先卸载`nvidia`内核模块：

```
# rmmod nvidia_uvm
# rmmod nvidia
```

仅使用英伟达显卡

专有的英伟达驱动可配置为主渲染提供程序。它也有明显的画面撕裂问题——除非通过启动 NVIDIA#DRM 内核级显示模式设置启用 PRIME Sync，更多信息请参见 。专有驱动确实允许使用独立 GPU，而且（截至 2017 年一月）它相比 nouveau 驱动有更明显的性能优势。

首先，安装英伟达驱动和 xorg-xrandr包 软件包。然后，配置 `/etc/X11/xorg.conf.d/10-nvidia-drm-outputclass.conf`，其中的选项将与软件包提供的 `/usr/share/X11/xorg.conf.d/10-nvidia-drm-outputclass.conf` 文件结合提供与此配置的兼容性。

注意：在一些配置上，此配置会使英伟达驱动无法通过 EDID 文件自动探测显示参数值。解决方案可参考 #分辨率和屏幕扫描错误“EDID errors in Xorg.log”。

`/etc/X11/xorg.conf.d/10-nvidia-drm-outputclass.conf`

```
Section "OutputClass"
    Identifier "intel"
    MatchDriver "i915"
    Driver "modesetting"
EndSection

Section "OutputClass"
    Identifier "nvidia"
    MatchDriver "nvidia-drm"
    Driver "nvidia"
    Option "AllowEmptyInitialConfiguration"
    Option "PrimaryGPU" "yes"
    ModulePath "/usr/lib/nvidia/xorg"
    ModulePath "/usr/lib/xorg/modules"
EndSection
```

接下来，依所使用的登录图形的方式不同而有所不同，请按实际情况选择阅读。

startx

请在 `~/.xinitrc` 文件的开头处添加下列两行内容：

```
~/.xinitrc
xrandr --setprovideroutputsource modesetting NVIDIA-0
xrandr --auto
```

现在重启以加载驱动，X 服务器也应该启动了。

如果显示 DPI 不正确，请再添加下列一行：

```
xrandr --dpi 96
```

如果在启动 X 服务器时出现黑屏，请确保 `~/.xinitrc` 文件中的两个 `xrandr` 命令后面没有“&”符号。如果有“&”号，窗口管理器可能在 `xrandr` 命令执行完成前就运行了，从而导致了黑屏。

显示管理器

如果打算使用显示管理器，就不是使用 `~/.xinitrc` 文件了，而是需要为显示管理器创建或编辑显示配置脚本。

LightDM

为 LightDM 显示管理器创建/编辑脚本：

```
/etc/lightdm/display_setup.sh
#!/bin/sh
xrandr --setprovideroutputsource modesetting NVIDIA-0
xrandr --auto
```

然后赋予脚本可执行权限。

通过编辑 `/etc/lightdm/lightdm.conf` 的 `[Seat:*]` 部分配置 LightDM 来运行这个脚本：

```
# nano /etc/lightdm/lightdm.conf
[Seat:*]
display-setup-script=/etc/lightdm/display_setup.sh
```

现在重启，然后显示管理器应该能启动了。

SDDM

为 SDDM 显示管理器创建/编辑脚本（SDDM 是 KDE 的默认显示管理器）：

```
/usr/share/sddm/scripts/Xsetup
xrandr --setprovideroutputsource modesetting NVIDIA-0
xrandr --auto
```

GDM

要为 GDM 创建启动脚本，请创建两个新的 .desktop 文件，如下：

```
/usr/share/gdm/greeter/autostart/optimus.desktop
/etc/xdg/autostart/optimus.desktop
[Desktop Entry]
Type=Application
Name=Optimus
Exec=sh -c "xrandr --setprovideroutputsource modesetting NVIDIA-0; xrandr --auto"
NoDisplay=true
X-GNOME-Autostart-Phase=DisplayServer
```

并且请确保 GDM 使用 Xorg 作为后端。

检查 3D 设置

可通过安装 mesa-utils包 并运行以下命令来检查英伟达显卡是否被使用：

```
$ glxinfo | grep NVIDIA
```

更多信息

更多信息请参见英伟达官方网页上的这个主题文章 。

使用可切换显卡

使用 PRIME 渲染分载（render offload）

这是 NVIDIA 官方支持可切换显卡的方法。

参见 PRIME#PRIME 渲染分载。

使用 nouveau

图形切换参见 PRIME，开源 NVIDIA驱动程序见 nouveau。

使用大黄蜂（Bumblebee）

参见 Bumblebee。

使用 switcheroo-control

参见 PRIME#GNOME 集成。

使用 nvidia-xrun

参见 nvidia-xrun。

使用 optimus-manager

参见 Optimus-manager 上游文档。它涵盖了在 Arch Linux 系统中的安装和配置。

使用 EnvyControl

参见 EnvyControl 上游文档。它涵盖了安装和使用说明。

使用 NVidia-eXec

参见 NVidia-eXec 上游文档。它涵盖了安装和使用说明。

使用 nvidia-switch

参见 nvidia-switch 上游文档。它涵盖了安装和使用说明。

疑难解答

垂直同步撕裂

开启 DRM 内核级显示模式设置，这将启用 PRIME 同步并修复撕裂问题。

官方论坛查看详细信息：forum thread。

Failed to initialize the NVIDIA GPU at PCI:1:0:0 (GPU fallen off the bus / RmInitAdapter failed!)

添加 `rcutree.gp_init_delay=1` 到内核参数。原始话题见 和 。

分辨率和屏幕扫描错误“EDID errors in Xorg.log”

这是由于 NVIDIA 的驱动程序没有检测显示器的 EDID。你需要手动指定路径的 EDID 文件或以类似的方式提供相同的信息。

增加这些线路和变化部分反映你自己的系统：

```
/etc/X11/xorg.conf
Section "Device"
              Option         "ConnectedMonitor" "CRT-0"
              Option         "CustomEDID" "CRT-0:/sys/class/drm/card0-LVDS-1/edid"
       Option         "IgnoreEDID" "false"
       Option         "UseEDID" "true"
EndSection
```

如果 Xorg 不会启动，尝试将所有 CRT 替换为 DFB。`card0` 是标识为英特尔卡，显示器通过 LVDS 连接。EDID 二进制文件位于此目录。如果硬件配置不同，CustomEDID 的值可能有所不同，但这已得到证实。不管怎样，路径都将从 `/sys/class/drm` 开始。

或者你可以使用工具如 read-edid包 生成你的 EDID，并将驱动指向此文件。也可以使用 modelines，但是务必要修改 `UseEDID` 和 `IgnoreEDID`。

无 EDID 错误的分辨率问题

使用 *nvidia-xconfig* 时，可能会在 `xorg.conf` 中生成错误的信息，特别是错误的显示器刷新率，限制了可能的分辨率。尝试注释掉 `HorizSync`/`VertRefresh` 行。如果这有帮助，你可能还可以删除本文中未提及的其他内容。

锁定问题(lspci 挂起)

问题：lspci 挂起，系统暂停失败，关机时挂起，optirun 挂起。多出现在新的笔记本电脑或使用了类似 bbswitch GTX 的 965m 时（例如 bumblebee）以及 nouveau 的情况。

当独立显卡接通电源，可能出现这种情况，参见 (kernel bug 156341)。

具体解决方法参见 this issue。 你可以添加 `acpi_osi="!Windows 2015"` 或 `acpi_osi=! acpi_osi="Windows 2009"` 到内核参数中。

笔记本电脑未发现屏幕/NVIDIA Optimus

检查输出是否类似：

```
$ lspci | grep VGA
00:02.0 VGA compatible controller: Intel Corporation Core Processor Integrated Graphics Controller (rev 02)
01:00.0 VGA compatible controller: nVidia Corporation Device 0df4 (rev a1)
```

NVIDIA 驱动自 319.12 Beta 起已经包含在内核（版本 3.9 级以上）中。

另一个解决方案是安装 Intel 驱动进行显示，如果需要运行 3D 软件，可以使用 Bumblebee 来使用 NVIDIA 显卡。

随机冻结 "(EE) NVIDIA(GPU-0): WAIT"

在使用集成 AMD 卡和专用 NVIDIA 卡的设置上使用专有驱动时，用户报告冻结长达 10 秒，Xorg 日志中出现以下错误：

```
[   219.796] (EE) NVIDIA(GPU-0): WAIT (2, 8, 0x8000, 0x0002e1c4, 0x0002e1cc)
[   226.796] (EE) NVIDIA(GPU-0): WAIT (1, 8, 0x8000, 0x0002e1c4, 0x0002e1cc)
```

虽然尚未找到根本原因，但似乎与集成卡和专用卡与 Xorg 的交互方式冲突有关。

解决方法是使用可切换显卡，详见 PRIME#PRIME render offload。

使用 optimus-manager 时出现 "No Devices detected"

在某些情况下，*lspci* 会将 PCI 域作为第一列输出，导致 *optimus-manager* 生成的文件在尝试映射 `BusID` 时在多款笔记本电脑上出现问题。

如果你遇到黑屏、GUI 部分加载或 Xorg 崩溃并显示 `(EE) - No Devices detected`，解决方法和错误报告可在上游 GitHub 找到。

Xorg：外部显示器仅在鼠标移动时更新

解决方法是卸载 iGPU 的 Xorg 驱动（例如 xf86-video-amdgpu包 或 xf86-video-intel 包）。只要外部显示器端口（HDMI/DP/USB-C）直接连接到 NVIDIA dGPU，这应该有效。

提示：桌面行为可能会变得不稳定（例如切换虚拟桌面或 Alt-Tab 时外部显示器图像冻结），也可以通过禁用笔记本电脑的显示器来解决。

低功耗（TDP）

自 530.41 驱动版本以来，出现了显卡锁定在低功耗限制的情况（参见 GitHub issue 483）。NVIDIA 驱动已禁用使用 `nvidia-smi` 命令手动设置功耗限制的功能，因此许多笔记本电脑被锁定在低功耗和性能不佳的状态。

要解决此问题（适用于 Ampere 代或更新），启动/启用 `nvidia-powerd.service`，这将启用 DynamicBoost。

NVIDIA GPU 无法关闭或保持停用状态

某些进程可能会由于它们与 GPU 的交互方式而保持你的 NVIDIA GPU 处于活动状态。这会导致功耗显著增加、电池寿命缩短和温度升高。

你可以通过运行以下命令检查你的 GPU 是否处于活动状态或已挂起：

```
$ cat /sys/bus/pci/devices/0000\:01\:00.0/power/runtime_status
```

如果状态为 `active`，你可能正在运行一个保持 GPU 活动的进程。

如果你使用一个探测 GPU 温度的温度监控器，它通常会调用 `nvidia-smi` 来获取此温度，这将唤醒你的 GPU 并保持其处于活动状态。

你可以使用 nvtop包 检查是否有进程（如 Xorg）正在使用 NVIDIA GPU，但此方法并非在所有情况下都有效。例如，如果你运行 ollama包 服务器，它将始终保持你的 GPU 处于活动状态，但不会显示在 `nvtop` 或调用 `nvidia-smi` 中。

记得检查与你选择的故障排除方法相关的文章。

2.11 PRIME

难度 进阶 | **篇幅** 19.1 KB | **所属** 图形

本节将讲到

- PRIME GPU 分载
- 反向 PRIME
- 疑难解答
- 参见

PRIME 是一种用于管理最新一些台式机和笔记本电脑上的混合图形的技术（NVIDIA 的 Optimus，Radeon 的 AMD 动态可切换图形）。PRIME GPU 分载（offloading）和反向 PRIME（reverse PRIME）是在 Linux 内核中支持无复用混合显示的尝试。

: 本文出现的一些不常用缩写的解释： : iGPU - integrated GPU，集成显卡 : dGPU - discrete/dedicated GPU，独立/专用显卡

PRIME GPU 分载

用户希望在更强大的显卡上渲染应用程序，并将结果发送到连接显示器的显卡。

命令 `xrandr --setprovideroffloadsink provider sink` 可用于让渲染分载提供程序（offload provider）将其输出发送到接收端（sink）提供程序（也就是连接了显示器的提供程序）。提供程序和接收端标识符可以是数字（0x7d、0x56）或区分大小写的名称（Intel、radeon）。

注意：当使用来自官方存储库的大多数默认 Xorg DDX（`xf86-video-*` 或内置内核级显示模式设置）驱动程序时，不再需要此设置，因为它们默认启用 DRI3，因此会自动地这样设置。不过，再次显式设置它们一次也不会有什么害处。

样例:

```
$ xrandr --setprovideroffloadsink radeon Intel
```

命令中的提供程序也可以用序号代替名字：

```
$ xrandr --setprovideroffloadsink 1 0
```

对于开源程序 - PRIME

要将独立显卡用于最需要它的应用程序（例如游戏、3D 建模器...），请在程序的启动命令前面加上环境变量 `DRI_PRIME=1`：

```
$ DRI_PRIME=1 glxinfo | grep "OpenGL renderer"
OpenGL renderer string: Gallium 0.4 on AMD TURKS
```

注意：除了数值外，您还可以指定 PCI 设备名称。格式类似于 `/sys/bus/pci/devices/`，但前缀为 `pci-` 并将分号和点替换为下划线，例如 `DRI_PRIME=pci-0000_01_00_0`。

其他应用程序仍将使用功耗较低的集成显卡。一旦 X 服务器重新启动，这些设置就会丢失，可能需要制作一个脚本并在桌面环境启动时自动运行（或者将其放在 `/etc/X11/xinit/xinitrc.d/` 目录下面）。不过，这样可能会缩短电池寿命并增加热量。

更多信息可以参考 Gentoo:AMDGPU#Identifying which graphics card is in use。

为了让 `DRI_PRIME` 在 Vulkan 应用程序上工作，需要安装 vulkan-mesa-layers包，以及用于 32 位应用程序的 lib32-vulkan-mesa-layers包。

PRIME 渲染分载

NVIDIA 驱动程序从版本 435.17 开始支持这个方案。iGPU 驱动程序 xf86-video-amdgpu包（450.57）和 xf86-video-intel包（455.38）官方支持内核级显示模式设置（modesetting）。

要在 NVIDIA 显卡上运行程序，您可以使用 nvidia-prime包 软件包提供的 `prime-run` 脚本：

```
$ prime-run glxinfo | grep "OpenGL renderer"
$ prime-run vulkaninfo
```

PCI-Express Runtime D3 (RTD3) 电源管理

开源驱动程序

在 GPU 不和 PRIME 分载或反向 PRIME 一起使用的时候，内核 PCI 电源管理就会关闭该 GPU。

内核级显示模式设置、xf86-video-amdgpu包、xf86-video-intel包、xf86-video-nouveau包 驱动程序都支持此功能。

以下命令可用于检查每个 GPU 当前的电源状态：

```
 $ cat /sys/class/drm/card*/device/power_state
```

NVIDIA

注意：

- 安培（Ampere）架构的显卡通常不需要配置，因为默认情况下已经启用。对于某些安培架构显卡的用户，可能需要 udev 规则。
- 一些使用混合显卡的用户报告称，在升级到较新的 NVIDIA 驱动程序（似乎 >525）后，他们的独立 NVIDIA 安培 GPU 无法保持在 D3Cold 电源状态 。
- 一些使用前安培显卡的用户报告称，在较新的驱动程序上 D3 支持已损坏，可以通过禁用 GSP 固件来解决，使用 `NVreg_EnableGpuFirmware=0`。

对于在 Intel Coffee Lake 或更高版本 CPU 以及某些 Ryzen CPU（如 5800H）平台上运行的图灵（Turning）架构显卡，可以在不使用的时候完全关闭 GPU。

注意：如果您计划使用挂起或休眠，请参见 NVIDIA/提示和技巧#保留挂起后的显存。

需要以下 udev 规则，如 NVIDIA 所建议的：

```
/etc/udev/rules.d/80-nvidia-pm.rules
# Enable runtime PM for NVIDIA VGA/3D controller devices on driver bind
ACTION=="bind", SUBSYSTEM=="pci", ATTR{vendor}=="0x10de", ATTR{class}=="0x030000",
TEST=="power/control", ATTR{power/control}="auto"
ACTION=="bind", SUBSYSTEM=="pci", ATTR{vendor}=="0x10de", ATTR{class}=="0x030200",
TEST=="power/control", ATTR{power/control}="auto"

# Disable runtime PM for NVIDIA VGA/3D controller devices on driver unbind
ACTION=="unbind", SUBSYSTEM=="pci", ATTR{vendor}=="0x10de", ATTR{class}=="0x030000",
TEST=="power/control", ATTR{power/control}="on"
```

```
ACTION=="unbind", SUBSYSTEM=="pci", ATTR{vendor}=="0x10de", ATTR{class}=="0x030200", TEST=="power/control", ATTR{power/control}="on"
```

一些用户还报告以下附加行也是必要的：

```
/etc/udev/rules.d/80-nvidia-pm.rules
# Enable runtime PM for NVIDIA VGA/3D controller devices on adding device
ACTION=="add", SUBSYSTEM=="pci", ATTR{vendor}=="0x10de", ATTR{class}=="0x030000", TEST=="power/control", ATTR{power/control}="auto"
ACTION=="add", SUBSYSTEM=="pci", ATTR{vendor}=="0x10de", ATTR{class}=="0x030200", TEST=="power/control", ATTR{power/control}="auto"
```

同时设置以下模块参数：

```
/etc/modprobe.d/nvidia-pm.conf
options nvidia "NVreg_DynamicPowerManagement=0x02"
```

此外，您可以安装 nvidia-prime-rtd3pmAUR，它提供了这两个配置文件。

在设置好 udev 规则和模块参数后，您需要重新启动笔记本电脑。

要检查 NVIDIA GPU 是否已关闭，可以使用以下命令：

```
$ cat /sys/bus/pci/devices/0000:01:00.0/power/runtime_status
```

您将看到 `suspended` 或 `running`。如果显示 `suspended`，则 GPU 已关闭，现在功耗将为 0 瓦，从而延长电池寿命。

在某些情况下，例如 NVIDIA RTX A1000，可能不会列出上述任何选项，而是显示 `active`。这并不一定意味着 GPU 处于 `running` 状态。在这种情况下，您可以使用以下命令检查状态：

```
$ cat /sys/bus/pci/devices/0000:01:00.0/power/runtime_suspended_time
```

当 GPU 处于 `suspended` 状态时，每次运行该命令时计数器都会递增。当 GPU 的状态变为 `running` 时，它将停止递增。

如果您注意到 `runtime_suspended_time` 没有递增，您可以使用以下命令检查您的 D3 状态。

```
$ cat /proc/driver/nvidia/gpus/0000:01:00.0/power
```

如果它显示 `Runtime D3 status: Not supported`，您可能需要按照此论坛帖子中的步骤禁用它。一位用户指出禁用 `GpuFirmware` 仅适用于闭源驱动程序，不适用于 nvidia-open包。

还需要启用 `nvidia-persistenced.service` 服务以避免内核在 NVIDIA 设备资源不再使用时清空设备状态。

配置应用程序使用 GPU 渲染

**本文或本章节可能需要合并到外接显卡#Xorg 在集成显卡上渲染，PRIME 渲染分载到外接显卡。**

附注： 此小节提到了这些变量。也许这可以与此部分合并以避免重复？（在 Talk:PRIME 中讨论）

即使没有启用动态电源管理，应用程序的渲染分载也要启用。

要在动态电源管理启用时将应用程序分载到 NVIDIA GPU，需添加以下环境变量：

```
__NV_PRIME_RENDER_OFFLOAD=1 __GLX_VENDOR_LIBRARY_NAME=nvidia command
```

在 Steam 游戏上使用时, 启动命令行选项可以设置为：

```
__NV_PRIME_RENDER_OFFLOAD=1 __GLX_VENDOR_LIBRARY_NAME=nvidia %command%
```

注意：`__NV_PRIME_RENDER_OFFLOAD` 的值可能需要根据系统设置为 `0`。建议检查哪个 GPU 是 `0`，哪个是 `1`，因为此变量指定将使用哪个 GPU。

GNOME 集成

安装 switcheroo-control包 并启用 `switcheroo-control.service` 以使用 GNOME 集成。

GNOME 尊重桌面项中的 `PrefersNonDefaultGPU` 属性。或者也可以通过右键单击图标并选择 *Launch using Discrete Graphics Card* 来使用 GPU 启动应用程序。

关于windows游戏

用 Wine 或者 Proton 运行 Windows DirectX 游戏时，你需要使用如下命令指示 DXVK 要使用的 GPU：

```
DXVK_FILTER_DEVICE_NAME "[your preferred card name]"
```

用 `vulkaninfo` 或许显卡名称; DXVK使用字字符串匹配。

疑难解答

如果安装了 bumblebee包，则需要将其删除，因为它将 `nvida_drm` 列入黑名单，该驱动程序由 X 服务器加载 NVIDIA 驱动程序来实现渲染分载。

PRIME 同步

当使用 PRIME 时，主 GPU 渲染屏幕内容/应用程序，并将其传递给备用 GPU 进行显示。参考an NVIDIA thread 此帖，"传统的 vsync 可以将应用程序的呈现与系统内存中的副本同步，但需要一种额外的机制来将系统内存中的副本与 iGPU 的显示引擎同步。与传统的垂直同步不同，这种机制必须涉及到 dGPU 和 iGPU 驱动之间的通信。"

这种同步是使用 PRIME 同步实现的。要检查显示系统是否启用了 PRIME 同步，可以查看 `xrandr --prop` 的输出。

使用下面的命令启用（PRIME 同步）：

```
$ xrandr --output <output-name> --set "PRIME Synchronization" 1
```

注意：

- 在 NVIDIA 驱动程序上使用 PRIME 同步的先决条件是启用 NVIDIA#DRM 内核级显示模式设置。
- PRIME 同步无法在 AMDGPU DDX 驱动上使用（xf86-video-amdgpu包）。

反向 PRIME

注意：

- 470 beta 之前的 NVIDIA 驱动程序上的 AMDGPU + NVIDIA 不支持反向 PRIME。更多详细信息，请参考 。
- 当前仅启用外部显示器时，将只能获得 1 FPS。 更多信息参考，一个解决方案是使用 `LIBGL_DRI3_DISABLE=true` 环境变量。

如果第二个 GPU 的输出无法被主 GPU 访问，可以使用反向 **PRIME**来使用它们。这将涉及使用主 GPU 渲染图像，然后将它们传递给第二个 GPU。

它可能开箱即用，但如果不能，则需要按照以下步骤进行配置。

配置

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：上半段Intel+NVIDIA，下半段Intel+Radeon。 识别 GPU 总线 ID 应放到通用部分。（在 Talk:PRIME讨论）

首先, 识别集成显卡的总线 ID（BusID）

```
lspci -d ::03xx
00:02.0 VGA compatible controller: Intel Corporation UHD Graphics 630 (Mobile)
01:00.0 VGA compatible controller: NVIDIA Corporation TU117M [GeForce GTX 1650 Mobile / Max-Q] (rev a1)
```

在上面的示例中，Intel 集成显卡的总线 ID 是 00:02.0 ，翻译到 pci 总线上为 PCI:0:2:0。

如下设置您的 `xorg.conf` 并调整总线 ID。

```
/etc/X11/xorg.conf
Section "ServerLayout"
        Identifier "layout"
        Screen 0 "intel"
        Inactive "nvidia"
        Option "AllowNVIDIAGPUScreens"
EndSection

Section "Device"
        Identifier "nvidia"
        Driver "nvidia"
EndSection

Section "Screen"
        Identifier "nvidia"
        Device "nvidia"
EndSection

Section "Device"
        Identifier "intel"
        Driver "modesetting"
        BusID "PCI:0:2:0"
EndSection

Section "Screen"
```

```
        Identifier "intel"
        Device "intel"
EndSection
```

命令 `xrandr --setprovideroutputsource provider source` 将提供程序设置为源输出。 例如：

```
$ xrandr --setprovideroutputsource radeon Intel
```

完成后，独立显卡的输出应该在 xrandr 中可用，可以执行以下操作来配置内部和外部显示器：

```
$ xrandr --output HDMI-1 --auto --above LVDS1
```

已知问题

如果重启后你只有一个提供程序，可能是因为当 Xorg 启动时`nvidia`模块还没加载。您需要启用早期模块加载。 有关详细信息，请参阅NVIDIA#早启动。

疑难解答

**本文或本章节的事实准确性存在争议。**

原因： 本节中的解决方法缺乏来源支持（在 Talk:PRIME 中讨论）

XRandR 只识别出一个显卡

删除或者移走 `/etc/X11/xorg.conf` 以及 `/etc/X11/xorg.conf.d/` 目录下任何与 GPU 有关的文件

当程序使用独立显卡渲染时，程序黑屏

某些情况下 PRIME 的正常工作需要一个混合窗口管理器。如果你的窗口管理器不进行混合渲染，你可以在其基础上使用混合窗口管理器。

如果你使用 Xfce，你可以进入*菜单* > *设置* > *窗口管理器调整* > *混合器*并启用混合渲染，然后再次尝试你的应用程序。

在基于 GL 的渲染合成器下黑屏

目前基于 GL 的渲染合成器和 PRIME 分载渲染还有一些问题。当基于 Xrender 的渲染合成器（xcompmgr、xfwm、compton的默认后端，cairo-compmgr 和一些其他的合成器）可以正常工作，基于 GL 的渲染合成器（Mutter/muffin、Compiz、使用 GLX 后端的 compton、Kwin 的 OpenGL 后端等）一开始会显示黑屏，就好像没有运行渲染合成器一样。虽然也可以通过调整使用渲染分载的程序窗口大小来强制显示图像，但这并不是一个实用的解决方案，因为它不适用于全屏的 Wine 应用程序。这意味着像 GNOME3 和 Cinnamon 这样的桌面环境在使用 PRIME 分载时存在问题。

此外，如果使用的是英特尔 IGP，也许可以通过把 IGP 当作 UXA 而不是 SNA 运行来修复 GL合成问题，然而这可能会导致渲染分载进程出现问题（例如，`xrandr --listproviders` 可能不会打印独立 GPU）。

某些程序在 Wayland 下打开时有延迟

如果你启用了 RTD3（来自#NVIDIA），在使用 Wayland 时，程序打开时会感到一些延迟，这是因为它会先尝试启动 GPU（这需要大约1秒或更长时间），然后才尝试打开程序，浪费了时间和电池寿命。这是 NVIDIA 驱动的问题。更多细节参见此帖。

欲解决该问题，请在您的 `/etc/environment`文件中添加以下两行：

```
/etc/environment
__EGL_VENDOR_LIBRARY_FILENAMES=/usr/share/glvnd/egl_vendor.d/50_mesa.json
__GLX_VENDOR_LIBRARY_NAME=mesa
```

使用 DXVK 运行 Wine 游戏时出错

当使用 PRIME 分载时，遇到 `Major opcode of failed request: 156 (NV-GLX)` 是一个已知问题。目前唯一的解决方法是启动X会话时完全使用 NVIDIA GPU。一个用户友好的方式是在 NVIDIA 独显和PRIME分载方法之间切换是使用 optimus-manager 工具或自己编写一些自动化脚本。

参见

- Nouveau Optimus

2.12 ATI

难度 高级 | **篇幅** 24.2 KB | **所属** 图形

本节将讲到

- 选择正确的驱动
- 安装
- 载入
- 配置
- 性能调整

本文主要介绍较旧的显卡使用的开源的 **ATI**/Radeon 驱动.

选择正确的驱动

要选择正确的驱动，请参考Xorg#AMD。本文主要介绍 **ATI**。

如果你不确定该用哪种，请先试一试开源版的。开源驱动能满足大多数的需要，而且，一般来说遇到的麻烦也更少些。查看现在功能开发进展情况可访问功能矩阵。

这个页面可以将市场名(例如 Radeon HD4330) 映射到芯片组名(例如 R700).

安装

安装软件包 mesa包，它提供 DRI 和 3D 加速。

- 若需要x86_64下的32位支持,可以从 multilib 安装 lib32-mesa包.
- 要使用 Xorg 2D 加速 DDX 驱动，请安装软件包 xf86-video-ati包.

加速视频解码方面，libva-mesa-driver包 和 lib32-libva-mesa-driver包提供了VA-API, mesa-vdpau包 和 lib32-mesa-vdpau包 提供 VDPAU。

载入

参考 Kernel mode setting#Early KMS start。

radeon模块应该在启动时被正常载入.

要是没有的话...

- 确保内核参数中**没有** `nomodeset` 或 `vga=`,因为现在 radeon 需要内核级显示模式设置。
- 另外，确保 radeon 模块不在内核模块黑名单中。

KMS早启动

参考 Kernel mode setting#KMS 早启动

配置

Xorg 会自动装入驱动并通过 EDID 获得显示器分辨率，只有性能优化时才需要额外配置。

如果要手动配置，请添加文件 `/etc/X11/xorg.conf.d/20-radeon.conf`, 并加入：

```
Section "OutputClass"
    Identifier "Radeon"
    MatchDriver "radeon"
    Driver "radeon"
EndSection
```

通过此段可以调整显卡的设置。

性能调整

启动视频加速

参见Hardware video acceleration。

图形工具

- **radeon-profile** — 显示 Radeon 显卡的 Qt 应用程序。

: https://github.com/marazmista/radeon-profile || radeon-profile-gitAUR

- **CoreCtrl** — 流行的通用 GPU/CPU 控制器，高级用法请参考 设置 页面。

: https://gitlab.com/corectrl/corectrl || corectrl包

驱动配置

下面这些选项属于`/etc/X11/xorg.conf.d/**20-radeon.conf**`.

请在应用驱动选项之前先阅读 radeon(4) 和 RadeonFeature。

Acceleration architecture; Glamor是一种使用OpenGL的 2D加速方式，适用于R300及以上显卡驱动。 自xf86-video-ati版本1:7.2.0-1后, 在radeonsi(南方群岛系列 和 superior GFX 显卡)上glamor默认启用; 在其他显卡默认使用 EXA.

```
Option "AccelMethod" "glamor"
```

使用 Glamor 加速方式时可以启用 **ShadowPrimary** 选项，它将启用一个被称为 "shadow primary" 的缓冲区来供CPU快速存取像素信息，并给每个显示控制器 (CRTC) 分离出单独的 scanout 缓冲区。这将提升某些 2D 工作的性能，但可能会降低其他（比如3D）工作的性能。

```
Option "ShadowPrimary" "on"
```

ColorTiling 和 **ColorTiling2D** 是绝对安全的,并且默认被启用. 大多数用户能注意到性能的提升,但是这个功能R200及更早的显卡不支持. 早的显卡虽可以启用,但是工作负担转移到了cpu上

```
Option "ColorTiling" "on"
Option "ColorTiling2D" "on"
```

DRI3 默认是启用的，老卡默认使用 DRI2， 要切换到 DRI3：

```
Option "DRI" "3"
```

TearFree 使用硬件的 flipping mechanism 机制来防止撕裂。当前启用这个选项会禁用 "EnablePageFlip" 选项。

```
Option "TearFree" "on"
```

EXAVSync 选项仅在使用 EXA 加速方式时有效，它通过stalling the engine until the display controller has passed the destination region来避免撕裂。这将导致性能下降，并已知在某些芯片上导致不稳定。

```
Option "EXAVSync" "yes"
```

下面是一个简单的 `/etc/X11/xorg.conf.d/**20-radeon.conf**` 配置文件示例：

```
Section "OutputClass"
        Identifier  "Radeon"
        Driver "radeon"
        MatchDriver "radeon"
        Option "AccelMethod" "glamor"
        Option "DRI" "3"
        Option "TearFree" "on"
        Option "ColorTiling" "on"
        Option "ColorTiling2D" "on"
EndSection
```

提示：driconfAUR 是一个可以修改诸多设置的小工具，如 vsync, anisotropic filtering, texture compression 等。它还有一些程序（比如Goole Earth）需要的"disable Low Impact fallback"功能。

内核参数

提示：你也许想用 `systool` 来调试新的参数，参见这里。

如果 **gartsize** 没有被自动检测到，请添加 `radeon.gartsize=32` 到内核参数来手动定义它。

注意：对于新的AMD显卡不再需要设置这个参数：

```
[drm] Detected VRAM RAM=2048M, BAR=256M
[drm] radeon: 2048M of VRAM memory ready
[drm] radeon: 2048M of GTT memory ready.
```

重启后生效。

关闭 PCIE 2.0

从3.6版内核开始，radeon里PCIE 2.0选项默认启用。

对一些主板可能不稳定，可以向内核参数添加 `radeon.pcie_gen2=0` 来关闭。

参考 Phoronix 文章

Gallium HUD

radeon 驱动支持激活一个HUD，来显示透明的图像及文字于正在渲染的程序（如游戏）之上。可以显示当前帧率，每个CPU核心负载或者CPU负载平均值。这个HUD受 GALLIUM_HUD 环境变量控制，可以添加一些参数如：

- "fps" - 显示当前帧率
- "cpu" - 显示平均CPU负载
- "cpu0" - 显示第一个CPU核心负载
- "cpu0+cpu1" - 显示前两个CPU核心负载
- "draw-calls" - 显示一个物体的每个材质被显示到屏幕上多少次(displays how many times each material in an object is drawn to the screen)
- "requested-VRAM" - 显示GPU的VRAM使用量
- "pixels-rendered" - 显示正在显示的像素计数

关于参数的完整列表以及操作GALLIUM_HUD的一些注意事项，你可以添加"help"参数运行一个简单程序（如glxgears）来查看相应的终端输出。

```
# GALLIUM_HUD="help" glxgears
```

更多信息参见邮件列表页面或这篇博客.

混合交火

这是一项通常在配备了双显卡——一块比较节能（比如Intel的核芯显卡），另一块为高性能、高能耗（Radeon或NVIDIA）的笔记本上启用的特性。有两种方法可以启用它：

- 如果不需要运行很吃GPU的程序，可以禁用独立显卡（参见Ubuntu wiki ）：

```
    # echo OFF > /sys/kernel/debug/vgaswitcheroo/switch
```

。

- PRIME是在Linux系统中使用双显卡切换的正确方式，但仍然需要用户进行大量手动设置。

节能

注意：所有vbios中包含有适当电源状态表的显卡芯片（R1xx 及更新的）都支持电源管理。"dpm"仅在 R6xx 以及更新的芯片上被支持。

开源驱动的节电功能默认关闭,需要可手动启用.

三种方法可供选择:

1. dpm(3.13内核后默认启用)
2. dynpm
3. profile

详见 https://www.x.org/wiki/RadeonFeature/#index3h2 .

动态电源管理

从3.13内核开始,在很多 AMD Radeon 设备上 DPM 默认启用。如果要禁用可加入参数 `radeon.dpm=0` 到内核参数。

提示：DPM 可以支持 R6xx，但是在内核里默认没有启用，仅 R7xx 及之后的显卡才默认启用. 在内核参数中加入 `radeon.dpm=1` 可以启用 dpm.

不像dynpm，“dpm"方式根据GPU负载情况动态调整时钟频率和电压，同时它会启用频率和电压门控.

dpm有3种模式可选:

- `battery` 最节能
- `balanced` 默认
- `performance` 最佳性能

可以通过sysfs来更改dpm的模式,如下:

```
# echo battery > /sys/class/drm/card0/device/power_dpm_state
```

你也许想要强制显卡工作在某一性能等级下:

- `auto` 默认; 使用当前dpm模式限定的所有性能等级
- `low` 强制在最低性能
- `high` 强制在最高性能

```
# echo low > /sys/class/drm/card0/device/power_dpm_force_performance_level
```

命令行工具

- radcard - 一个获取/调整 DPM 电源状态与级别的脚本。

老方法

动态频率调整

此方法根据GPU负载自动改变时钟频率，所以GPU忙碌时显示性能提高，空闲时降低。自动变频试图在垂直消隐期间进行，但由于变频函数可能在周期内无法完成，这种方法可能导致显示闪烁。因此，动态调整只能在单显示器下正常工作。

可以通过以下命令启用：

```
# echo dynpm > /sys/class/drm/card0/device/power_method
```

基于计划的频率调整

该方法允许你选择5种不同的计划。主要来说,不同的计划最终都改变GPU时钟频率和电压。这种方式不同于积极方式,但更稳定,少闪屏,而且可用于多显示器环境

要激活此方法，可运行以下命令：

```
# echo profile > /sys/class/drm/card0/device/power_method
```

可选的计划：

- `default` 使用默认时钟频率，不改变电源状态。这是默认启用的计划。
- `auto` 根据系统是否使用电池，在电源状态 `mid` 和 `high` 间自动切换
- `low` 强制GPU运行于`low`电源状态.注意`low`在某些笔记本上可能导致显示问题,如`auto` 计划在显示器关闭时只选中`low`.(Note that `low` can cause display problems on some laptops, which is why `auto` only uses `low` when monitors are off.) 在其他计划中,当显示器进入节能状态时,将自动切换为 `low` 计划
- `mid` 强制GPU运行于`mid`电源状态.

- `high` 强制GPU运行于`high`电源状态.

例如，我们可以激活`low` 计划(你可以根据需要替换为上述其他计划):

```
# echo low > /sys/class/drm/card0/device/power_profile
```

永久配置

上述方法不是永久性的，系统重启后将丢失。为了让它一直有效，可以使用udev规则, 例如设置基于计划的频率调整 :

```
/etc/udev/rules.d/30-radeon-pm.rules
KERNEL=="dri/card0", SUBSYSTEM=="drm", DRIVERS=="radeon", ATTR{device/power_method}
="profile", ATTR{device/power_profile}="low"
```

注意：如果上述规则失效，你可以试试删除 `dri/` 前辍.

KERNEL=="card0", SUBSYSTEM=="drm", DRIVERS=="radeon", ATTR{device/power_method}="profile",

As another example, dynamic power management can be permanently forced to a certain performance level:

```
/etc/udev/rules.d/30-radeon-pm.rules
KERNEL=="dri/card0", SUBSYSTEM=="drm", DRIVERS=="radeon",
ATTR{device/power_dpm_force_performance_level}="high"
```

To determine the `KERNEL` name, execute:

```
$ udevadm info --attribute-walk /sys/class/drm/card0 | grep "KERNEL="
```

其它

要查看当前GPU频率，可以运行如下命令，你可以看到类似如下输出：

```
# cat /sys/kernel/debug/dri/0/radeon_pm_info
  state: PM_STATE_ENABLED
  default engine clock: 300000 kHz
  current engine clock: 300720 kHz
  default memory clock: 200000 kHz
```

It depends on which GPU line yours is, however. Along with the radeon driver versions, kernel versions, etc. So it may not have much/any voltage regulation at all.

Thermal sensors are implemented via external i2c chips or via the internal thermal sensor (rv6xx-evergreen only). To get the temperature on asics that use i2c chips, you need to load the appropriate hwmon driver for the sensor used on your board (lm63, lm64, etc.). The drm will attempt to load the appropriate hwmon driver. On boards that use the internal thermal sensor, the drm will set up the hwmon interface automatically. When the appropriate driver is loaded, the temperatures can be accessed via lm_sensors tools or via sysfs in `/sys/class/hwmon`.

风扇速度

即使上述省电功能应该能很好调整风扇速度，一些显卡在空闲状态时仍然太吵了。这时，如果你的风扇支持的话，你可以尝试手动改变风扇速度。

注意：

- 请谨记以下方法会将风扇速度设置为固定值，它不会随着 gpu 的压力而调整，所以在高负荷工作时这有可能导致过热。
- 设置低于标准的数值时最好检查你的 gpu 温度。

要控制风扇速度，请参考 Fan speed control#AMDGPU sysfs fan control (amdgpu 和 radeon 使用相同的控制方式)。

如果要让此成为永久设置，使用 #永久配置。

如果固定值不符合你的期望，还可以自定义为按一个温度/风扇速度曲线来调整，比如写一个脚本，来根据当前温度 (/sys/class/drm/card0/device/hwmon/hwmon0/temp1_input) 设置风扇速度，最好还能设置为温度变化后延迟调整。这里有一个图形界面的工具：radeon-profile-gitAUR。

TV输出

首先，检查你有S-video输出：`xrandr`应该会给你类似如下的结果：

```
Screen 0: minimum 320x200, current 1024x768, maximum 1280x1200
...
S-video disconnected (normal left inverted right x axis y axis)
```

现在我们应该告诉Xorg它已经接上了 (本来就是,对吧)

```
xrandr --output S-video --set "load detection" 1
```

设定TV制式标准

```
xrandr --output S-video --set "tv standard" ntsc
```

为它添加一个分辨率（目前只支持800x600）

```
xrandr --addmode S-video 800x600
```

复制模式（clone mode)

```
xrandr --output S-video --same-as VGA-0
```

好了，让我们来看看效果吧

```
xrandr --output S-video --mode 800x600
```

这时，在电视上你应该能看到你的桌面，分辨率是800x600。

要关掉这一输出：

```
xrandr --output S-video --off
```

在KMS中强制TV输出

内核可识别下列格式的 `video=` 参数 （参见KMS） ：

```
video=<conn>:<xres>x<yres>[M][R][-<bpp>][@<refresh>][i][m][eDd]
```

例如：

```
video=DVI-I-1:1280x1024-24@60e
```

带有空格的参数必须用引号括起：

`"video=9-pin DIN-1:1024x768-24@60e"`
Current mkinitcpio implementation also requires `#` in front. For example:

```
root=/dev/disk/by-uuid/d950a14f-fc0c-451d-b0d4-f95c2adefee3 ro quiet radeon.modeset=1
security=none # video=DVI-I-1:1280x1024-24@60e "video=9-pin DIN-1:1024x768-24@60e"
```

- GRUB Legacy 可直接接受如上参数。
- LILO 需要在双引号前使用“\\”转义 (例如 `# \"video=9-pin DIN-1:1024x768-24@60e\"`)

You can get list of your video outputs with following command:

```
$ ls -1 /sys/class/drm/ | grep -E '^card[[:digit:]]+-' | cut -d- -f2-
```

HDMI 音频输出

HDMI 音频输出在 xf86-video-ati包 软件包中提供支持。要启禁用，在内核参数中添加 `radeon.audio=0`。

如果启动后没有视频输出，则请禁用这个参数。

注意：

- 如果在安装驱动后 HDMI 音频没有工作，请使用这里提供的方法进行检测。
- 如果在 PulseAudio 中声音出现问题，尝试设置 `tsched=0`（参见 PulseAudio/Troubleshooting#Glitches, skips or crackling）并确保 `rtkit` 守护进程正在运行。
- 因为 HDA 兼容硬件的相似性，你的声卡可能使用相同的模块。请使用推荐的方式改变默认声卡，比如修改 alsa 配置文件的 `defaults` 节点。

多显设置

使用 RandR 扩展

参见 Multihead#RandR。

独立的 X screen

独立的双显示器可以按正常方式配置，radeon 驱动有一个 `"ZaphodHeads"` 选项可以把显示的区域绑定到固定的设备，例如：

```
/etc/X11/xorg.conf.d/20-radeon.conf
Section "Device"
  Identifier "Device0"
  Driver "radeon"
  Option "ZaphodHeads" "VGA-0"
  VendorName "ATI"
  BusID "PCI:1:0:0"
  Screen 0
EndSection
```

有些显卡有多个输出（HDMI，DVI 和 VGA），而双屏显示的时候它只使用 HDMI+DVI，这时你可以通过 `"ZaphodHeads" "VGA-0"` 来更改输出。这在使用多输出显卡时很方便。

关闭垂直同步刷新

radeon驱动默认启用垂直同步刷新，除了跑分外各种情况下工作良好。要关闭它，可以设置 `vblank_mode=0` 环境变量或创建 `~/.drirc` （如果已存在请修改），加入以下部分：

```
~/.drirc
<driconf>
    <device screen="0" driver="dri2">
        <application name="Default">
            <option name="vblank_mode" value="0" />
        </application>
    </device>
</driconf>
```

请确保是 **dri2**，而不是你的显卡型号（如 r600 ）。

如果 vsync 还是启用状态，可以通过禁用`/etc/X11/xorg.conf.d/20-radeon.conf`. See #Driver options.

故障排除

使用 EXA 时性能低

注意：仅适用于使用 EXA 的老显卡(R600 或更老).新卡应该使用 Glamor 。

如果2D性能(比如在终端或浏览器的滚动滑块)有问题, 你可以将 `Option "MigrationHeuristic" "greedy"` 添加到你的 `xorg.conf` 文件的 `**Device**` 部分. 禁用 `EXAPixmaps` 也可能避免一些问题，但是可能带来别的问题，所以不建议使用。

这是一个样例 `/etc/X11/xorg.conf.d/**20-radeon.conf**`:

```
Section "OutputClass"
        Identifier  "My Graphics Card"
        MatchDriver "radeon"
        Option "AccelMethod" "exa"
        Option  "MigrationHeuristic"  "greedy"
        #Option "EXAPixmaps" "off"
EndSection
```

添加没有被侦测到的分辨率

参见Xrandr的文章.

电视屏幕显示黑边

我的Radeon HD 5770用HDMI连接到电视时, 电视显示图像模糊,周围有2-3cm黑边,用催化剂时不是这样. 这是对付过扫描(Overscan)的(参见Wikipedia:Overscan),使用xrandr关闭它:

```
xrandr --output HDMI-0 --set underscan off
```

KMS启用时,黑幕,没有控制台,但是 X 能够工作

当在同一台PC使用两张或以上的ATI显卡时可能会遇到此问题. 例如 Fujitsu Siemens Amilo PA 3553 笔记本就有这个问题. 这是因为fbcon控制台驱动程序将自己映射到错误显卡的错误 framebuffer 设备上. 在内核参数添加:

```
fbcon=map:1
```

这将告诉fbcon映射自己到 `/dev/fb1` 而不是 `/dev/fb0`.如果这并未解决你的问题，尝试如下配置启动:

```
fbcon=map:0
```

显示器旋转对光标起效却对窗口/内容不起效

启用EXA代替glamor的新显卡用户可能会发现,用xrandr旋转显示器将导致光标和显示器尺寸旋转了,但窗口与里面内容却保持原来方向. 另外移动鼠标时光标按照原来的方向移动.若有此问题,在你的 `/var/log/Xorg.0.log` 中查找下面这一行:

```
(EE) RADEON(0): Rotation requires acceleration!
```

新显卡上使用EXA时加速将被禁用(来源: 这里). 你必须从启用EXA (参见这里) 和旋转中二选一.

在ATI X1600 (RV530 series)上3D应用程序显示黑窗口

这三种方法可能有效:

- 将 `pci=nomsi` 添加到你的启动器的内核参数.
- 如果没用的话,试试用`noapic`代替`pci=nomsi`.
- 如果还是没用,你可以试试`vblank_mode=0 glxgears` 或者 `vblank_mode=1 glxgears`,看看哪个对你有用. 然后安装driconfAUR , 在 `~/.drirc`里设置此参数.

从休眠中唤醒后光标崩溃

如果显示器唤醒后光标垂直方向重复刷新，可以在配置文件 `20-radeon.conf` 中的 `"Device"` 部分里设置 `"SWCursor" "True"`。

多显示器模式下DisplayPort黑屏

尝试以内核参数 `radeon.audio=0` 启动。

R9-390 Poor Performance and/or Instability

Firmware issues with R9-390 series cards include poor performance and crashes (frequently caused by gaming or using Google Maps) possibly related DPM. Comment 115 of this bug report includes instructions for a fix.

QHD / UHD / 4k support over HDMI for older Radeon cards

Older cards have their pixel clock limited to 165MHz for HDMI. Hence, they do not support QHD or 4k only via dual-link DVI but not over HDMI.

One possibility to work around this is to use custom modes with lower refresh rate, e.g. 30Hz.

Another one is a kernel patch removing the pixel clock limit, but this may damage the card!

Official kernel bug ticket with patch for 4.8: https://bugzilla.kernel.org/show_bug.cgi?id=172421

The patch introduces a new kernel parameter `radeon.hdmimhz` which alters the pixel clock limit.

Be sure to use a high speed HDMI cable for this.

Horizontal flickering occasionally when using 4k DP output on 390X

If you use 390X (or perhaps similar models) and the 4k output from DP, you may experiencing occasional horizontal artifacts / flickering (i.e. every half an hour or so, a horizontal strip of pixels with a height of ~100 pixels across the whole screen's width shaking up and down for a few seconds). This might be a bug of the radeon driver. Changing to AMDGPU seems to fix it.

See also

Benchmark showing the open source driver is on par performance-wise with the proprietary driver for many cards.

2.13 硬件视频加速

难度 高级 | **篇幅** 26.4 KB | **所属** 图形

本节将讲到

- 安装
- 验证
- 配置
- 疑难解答
- 对照表

硬件视频加速让显卡进行视频的解码和编码，以减少CPU的负载并节省电力。

Linux 中有多种方式可以实现视频加速:

- 视频加速API (Video Acceleration API, 缩写为VA-API) 是一套 Intel 提供的视频硬件编解码的开源库和标准。

- Unix视频解码及演示 API （Video Decode and Presentation API for Unix，缩写为VDPAU） (VDPAU) 是一套 Nvidia 提供的把部分视频解码和后期处理工作转移到显卡的开源库和 API。
- Advanced Media Framework SDK (AMF) 是 AMD 开发的 GPU 开源框架，可以用 AMDGPU PRO 技术栈实现最优的处理。
- NVDEC/NVENC NVIDIA 的 费米、开普勒、Maxwell 和 Pascal 系列显卡所使用的专有 API。
- Vulkan Video is an extension of the Vulkan graphics API designed to support hardware-accelerated video encoding and decoding.

大致的总览对比和应用程序支持，参见页面下方的对照表。

安装

Intel

VA-API

Intel graphics 开源驱动支持 VA-API:

- Broadwell (~2014) 及之后的 HD Graphics 系列（例如英特尔 Arc 系列显卡），请使用 intel-media-driver包 。
- GMA 4500 (2008) 到 Coffee Lake (2017) 的 GPU, 请使用 libva-intel-driver 包 。
- Haswell Refresh#Haswell_Refresh) 到 Skylake 的 VP9 解码，以及 Broadwell 到 Skylake 的混合 VP8 编码，请使用 intel-hybrid-codec-driver-gitAUR.

注意：Skylake 及之后的硬件还需要 linux-firmware包.

参阅 VAAPI 支持的硬件和功能。

Vulkan

ANV 开源 vulkan 驱动通过 vulkan-intel包 提供 Vulkan 视频支持。

注意：启用 Vulkan 视频支持需要额外配置，参见 #Configuring Vulkan Video

Intel 视频处理库 (Intel VPL)

对于 Intel VPL ，请安装基础库 libvpl包 ，并确保安装下列之一的运行时环境：

- vpl-gpu-rt包 ： 为 Tiger Lake 和更新的GPU提供支持
- intel-media-sdk包 (已终止后续支持)：为旧 Intel GPU 提供支持

NVIDIA

Nouveau 开源驱动支持 VA-API 和 VDPAU:

- GeForce 8 到 GeForce GTX 750 的硬件，请使用 mesa包
- 还需要 nouveau-fwAUR 固件包（目前是从NVIDIA二进制驱动解压出来的）

NVIDIA 专有驱动通过 nvidia-utils包 支持以下硬件:

- GeForce 8 系列和新 GPU 上的 VDPAU；
- Fermi) 和更新的GPU 上的NVDEC;
- Kepler) 和更新的GPU上的 NVENC ；
- Pascal) 和更新的GPU 上的 Vulkan 视频。

AMD/ATI

AMDGPU 和 ATI 开源驱动通过 mesa包 支持 VA-API 和 VDPAU:

- VA-API on Radeon HD 2000 and newer GPUs.
- VDPAU on Radeon R600 and newer GPUs.

RADV 开源 vulkan 驱动通过 vulkan-radeon包 提供支持。

AMDGPU PRO 专有驱动是基于 AMDGPU 驱动的，所以支持 VA-API、VDPAU 和 AMF。

- Fiji 和更新的 GPUs 通过 amf-amdgpu-proAUR 支持 AMF。

注意：

- 你也许需要强制你的应用程序使用 AMDGPU PRO Vulkan 驱动。
- HEVC 可能在 Navi 之前的 GPU 上无法使用。

翻译层

- **libvdpau-va-gl** — OpenGL/VAAPI 做后端的 VDPAU 驱动，仅支持 H.264.

: https://github.com/i-rinat/libvdpau-va-gl || libvdpau-va-gl包

- **nvidia-vaapi-driver** — CUDA NVDEC 作为后端的 VA-API.

: https://github.com/elFarto/nvidia-vaapi-driver/ || libva-nvidia-driver包

注意：尽管 libva-nvidia-driver包 的实现确实启用了硬件视频解码，但由于当前 NVIDIA 的电源管理限制，它实际上 比使用 CPU 进行视频解码消耗更多的电力 。

验证

你的系统也许不需要任何配置就可以完美地开箱即用。因此，建议阅读本节，以确认是否确实如此。

提示：

- 有命令行支持的 mpv 是测试硬件加速的绝佳工具。查看 `mpv --hwdec=auto *video_filename*` 的日志并参见 hwdec 以获得更多信息。
- 对于 Intel GPU ，例如使用 intel-gpu-tools包 并以 root 运行 `intel_gpu_top` 以监控视频播放期间的 GPU 活动。若 “Video” 栏超过 0%，则表示正在使用 GPU 进行视频解码/编码。（如果你使用的是 新的 Xe 驱动 来运行 `intel_gpu_top` 会抛出错误，所以检查并运行 `gputop`，若 vcs 字段 超过 0%，则表示正在使用 GPU 进行视频解码/编码。）
- 对于 AMD，Intel 和 Nvidia ，在播放视频时使用 nvtop包 来查看(在顶部几行) "DEC"（解码器）的利用率。
- 对于 AMD GPU ，使用 radeontop包 查看 GPU 利用率。不同于 intel-gpu-tools包 ，当前没有任何方法在 radeontop包 上查看编码器/解码器的利用率。
- 无论是哪种 GPU，你都可以使用 htop包 等工具对比 CPU 使用率。特别是对于更高分辨率(4k+)的视频，如果启用了并正常使用 VA-API，CPU 占用应显著降低，尤其是在笔记本等低功耗设备上。

验证 VA-API

运行 libva-utils包 包提供的 `vainfo` 来检查 VA-API 的设置：

```
$ vainfo
libva info: VA-API version 0.39.4
libva info: va_getDriverName() returns 0
libva info: Trying to open /usr/lib/dri/i965_drv_video.so
libva info: Found init function __vaDriverInit_0_39
libva info: va_openDriver() returns 0
vainfo: VA-API version: 0.39 (libva 1.7.3)
vainfo: Driver version: Intel i965 driver for Intel(R) Skylake - 1.7.3
vainfo: Supported profile and entrypoints
      VAProfileMPEG2Simple            :   VAEntrypointVLD
      VAProfileMPEG2Simple            :   VAEntrypointEncSlice
      VAProfileMPEG2Main              :   VAEntrypointVLD
      VAProfileMPEG2Main              :   VAEntrypointEncSlice
      VAProfileH264ConstrainedBaseline:   VAEntrypointVLD
      VAProfileH264ConstrainedBaseline:   VAEntrypointEncSlice
      VAProfileH264ConstrainedBaseline:   VAEntrypointEncSliceLP
      VAProfileH264Main               :   VAEntrypointVLD
      VAProfileH264Main               :   VAEntrypointEncSlice
      VAProfileH264Main               :   VAEntrypointEncSliceLP
      VAProfileH264High               :   VAEntrypointVLD
      VAProfileH264High               :   VAEntrypointEncSlice
      VAProfileH264High               :   VAEntrypointEncSliceLP
      VAProfileH264MultiviewHigh      :   VAEntrypointVLD
      VAProfileH264MultiviewHigh      :   VAEntrypointEncSlice
      VAProfileH264StereoHigh         :   VAEntrypointVLD
      VAProfileH264StereoHigh         :   VAEntrypointEncSlice
      VAProfileVC1Simple              :   VAEntrypointVLD
      VAProfileVC1Main                :   VAEntrypointVLD
      VAProfileVC1Advanced            :   VAEntrypointVLD
      VAProfileNone                   :   VAEntrypointVideoProc
      VAProfileJPEGBaseline           :   VAEntrypointVLD
      VAProfileJPEGBaseline           :   VAEntrypointEncPicture
      VAProfileVP8Version0_3          :   VAEntrypointVLD
      VAProfileVP8Version0_3          :   VAEntrypointEncSlice
      VAProfileHEVCMain               :   VAEntrypointVLD
      VAProfileHEVCMain               :   VAEntrypointEncSlice
```

`VAEntrypointVLD` 指的是你的显卡能够解码这个格式，`VAEntrypointEncSlice` 指的是你的显卡可以编码这个格式。

上面的例子里，`i965` 驱动正在运行，所以你可以看见这一行：

```
libva info: Trying to open /usr/lib/dri/i965_drv_video.so
```

如果运行 `vainfo` 的时候出现了下面的错误提示：

```
libva info: va_openDriver() returns -1
vaInitialize failed with error code -1 (unknown libva error),exit
```

你要重新配置正确的驱动，详见Hardware video acceleration#Configuring VA-API.

验证 VDPAU

安装 vdpauinfo包 以验证 VDPAU 驱动是否正确加载，并获取完整的配置信息报告：

```
$ vdpauinfo
display: :0   screen: 0
API version: 1
Information string: G3DVL VDPAU Driver Shared Library version 1.0

Video surface:

name   width height types
-------------------------------------------
420    16384 16384  NV12 YV12
422    16384 16384  UYVY YUYV
444    16384 16384  Y8U8V8A8 V8U8Y8A8

Decoder capabilities:

name                        level macbs width height
----------------------------------------------------
MPEG1                         --- not supported ---
MPEG2_SIMPLE                    3  9216  2048  1152
MPEG2_MAIN                      3  9216  2048  1152
H264_BASELINE                  41  9216  2048  1152
H264_MAIN                      41  9216  2048  1152
H264_HIGH                      41  9216  2048  1152
VC1_SIMPLE                      1  9216  2048  1152
VC1_MAIN                        2  9216  2048  1152
VC1_ADVANCED                    4  9216  2048  1152
...
```

Verifying Vulkan Video

Install vulkan-tools包 and use *vulkaninfo* to verify if the video processing extensions are available:

```
$ vulkaninfo | grep VK_KHR_video_
        VK_KHR_video_decode_av1                       : extension revision 1

        VK_KHR_video_decode_h264                      : extension revision 9

        VK_KHR_video_decode_h265                      : extension revision 8

        VK_KHR_video_decode_queue                     : extension revision 8

        VK_KHR_video_encode_h264                      : extension revision 14

        VK_KHR_video_encode_h265                      : extension revision 14

        VK_KHR_video_encode_queue                     : extension revision 12

        VK_KHR_video_maintenance1                     : extension revision 1

        VK_KHR_video_queue                            : extension revision 8
```

配置

Although the video driver should automatically enable hardware video acceleration support for both VA-API and VDPAU, it may be needed to configure VA-API/VDPAU manually. Only continue to this section if you went through #Verification.

The default driver names, used if there is no other configuration present, are guessed by the system. However, they are often hacked together and may not work. The guessed value will be printed in the Xorg log file, which is `~/.local/share/xorg/Xorg.0.log` if rootless, or `/var/log/Xorg.0.log` if Xorg is running as root. To search the log file for the values of interest:

```
$ grep -iE 'vdpau | dri driver' xorg_log_file
(II) RADEON(0): [DRI2] DRI driver: radeonsi
(II) RADEON(0): [DRI2] VDPAU driver: radeonsi
```

In this case `radeonsi` is the default for both VA-API and VDPAU.

注意：如果你使用 GDM, 请以 root 权限运行 `journalctl -b --grep='vdpau | dri driver'` 。

This does not represent the *configuration* however. The values above will not change even if you override them.

配置 VA-API

You can override the driver for VA-API by using the `LIBVA_DRIVER_NAME` environment variable:

- Intel graphics:
- For libva-intel-driver包 use `i965`.
- For intel-media-driver包 use `iHD`.
- NVIDIA:
- For Nouveau use `nouveau`.
- For NVIDIA NVDEC use `nvidia`.
- AMD:
- For AMDGPU driver use `radeonsi`.

注意：

- You can find the installed drivers in `/usr/lib/dri/`. They are used as `/usr/lib/dri/**${LIBVA_DRIVER_NAME}**_drv_video.so`.
- Some drivers are installed several times under different names for compatibility reasons. You can see which by running `sha1sum /usr/lib/dri/* | sort`.
- `LIBVA_DRIVERS_PATH` can be used to overrule the VA-API drivers location.
- Since version 12.0.1 Mesa provides `radeonsi` instead of `gallium`.

配置 VDPAU

You can override the driver for VDPAU by using the `VDPAU_DRIVER` environment variable.

The correct driver name depends on your setup:

- For Intel graphics you need to set it to `va_gl`.
- For the open source AMD driver set it to `radeonsi`.
- For the open source Nouveau driver set it to `nouveau`.
- For NVIDIA's proprietary version set it to `nvidia`.

注意：

- You can find the installed drivers in `/usr/lib/vdpau/`. They are used as `/usr/lib/vdpau/libvdpau_**${VDPAU_DRIVER}**.so`.
- Some drivers are installed several times under different names for compatibility reasons. You can see which by running `sha1sum /usr/lib/vdpau/*`.
- For hybrid setups (both NVIDIA and AMD), it may be necessary to set the `DRI_PRIME` environment variable. For more information see PRIME.

Configuring Vulkan Video

- Intel graphics: Vulkan Video support in vulkan-intel包 can be enabled with the `ANV_DEBUG=video-decode,video-encode` environment variable.
- AMD: Vulkan Video support in vulkan-radeon包 is enabled by default for VCN 2, 3, and 4+ since Mesa 25. To force-enable support on older cards, set the `RADV_PERFTEST` environment variable to `video_decode,video_encode`.

配置应用程序

多媒体框架：

- FFmpeg#Hardware video acceleration
- GStreamer#Hardware video acceleration

视频播放器：

- Kodi#Hardware video acceleration
- MPlayer#Hardware video acceleration
- mpv#Hardware video acceleration
- VLC media player#Hardware video acceleration

网页浏览器：

- Chromium#Hardware video acceleration
- Firefox#Hardware video acceleration
- GNOME/Web#Video

多媒体录制/串流：

- Open Broadcaster Software#Hardware video acceleration

疑难解答

打开 VDPAU 后端失败

You need to set `VDPAU_DRIVER` variable to point to correct driver. See #配置 VDPAU.

VA-API 初始化失败

An error along the lines of `libva: /usr/lib/dri/i965_drv_video.so init failed` is encountered. This can happen because of improper detection of Wayland. One solution is to unset `$DISPLAY` so that mpv, MPlayer, VLC, etc. do not assume it is X11. Another mpv-specific solution is to add the parameter `--gpu-context=wayland`.

This error can also occur if you installed the wrong VA-API driver for your hardware.

使用 AMDGPU 驱动时视频解码损坏或失真

When experiencing video decoding corruption or distortion with AMDGPU driver, set `allow_rgb10_configs=false` as an environment variable.

对照表

VA-API 驱动

Codec	libva-intel-driver包	intel-media-driver包	mesa包	libva-nvidia-driver包 (NVDEC adapter)
Decoding				
MPEG-2	GMA 4500 and newer	Broadwell and newer	Radeon HD 6000 and newer GeForce 8 and newer1	See #NVIDIA driver only
H.263/MPEG-4 Visual3	否	否	Radeon HD 6000 and newer	
VC-1	Sandy Bridge and newer	Broadwell and newer	Radeon HD 2000 and newer GeForce 9300 and newer1	
H.264/MPEG-4 AVC	Ironlake and newer	Radeon HD 2000 and newer GeForce 8 and newer1		
H.265/HEVC 8bit	Cherryview/Braswell and newer	Skylake and newer	Radeon R9 Fury and newer	
H.265/HEVC 10bit	Broxton and newer	Broxton/Apollo Lake and newer	Radeon 400 and newer	
VP8	Broadwell and newer	Broadwell and newer	否	
VP9 8bit	Broxton and newer Hybrid: Haswell refresh to Skylake2	Broxton/Apollo Lake and newer	Raven Ridge + Radeon RX 5000 and newer	
VP9 10bit & 12bit	Kaby Lake and newer	Kaby Lake and newer		
AV1 8bit & 10bit	否	Tiger Lake and newer	Radeon RX 6600 and higher/newer	
Encoding				

Codec	libva-intel-driver包	intel-media-driver包	mesa包	libva-nvidia-driver包 (NVDEC adapter)
MPEG-2	Ivy Bridge and newer	Broadwell and newer except Broxton/Apollo Lake	否	否4
H.264/MPEG-4 AVC	Sandy Bridge and newer	Broadwell and newer	Radeon HD 7000 and newer	
H.265/HEVC 8bit	Skylake and newer	Skylake and newer	Radeon 400 and newer	
H.265/HEVC 10bit	Kaby Lake and newer	Kaby Lake and newer	Raven Ridge + Radeon RX 5000 and newer	
VP8	Cherryview/Braswell and newer Hybrid: Haswell to Skylake2	否		
VP9 8bit	Kaby Lake and newer	Ice Lake and newer		
VP9 10bit & 12bit	否			
AV1 8bit & 10bit	Alchemist and newer	Radeon RX 7900 and higher/newer		

1. Up until GeForce GTX 750.
2. Hybrid VP8 encoder and VP9 decoder supported by intel-hybrid-codec-driver-gitAUR.
3. MPEG-4 Part 2 is disabled by default due to VAAPI limitations. Set the environment variable `VAAPI_MPEG4_ENABLED=true` to try to use it anyway.
4. NVIDIA CUDA adapter codec support is in active development and susceptible to change .

VDPAU 驱动

Codec	Color depth	mesa包	nvidia-utils包	libvdpau-va-gl包 (VA-API adapter)
Decoding				
MPEG-2	8bit	Radeon R600 and newer GeForce 8 and newer1	GeForce 8 and newer	否
H.263/MPEG-4 Visual	8bit	Radeon HD 6000 and newer GeForce 200 and newer1	GeForce 200 and newer	
VC-1	8bit	Radeon HD 2000 and newer GeForce 9300 and newer1	GeForce 8 and newer2	
H.264/MPEG-4 AVC	8bit	Radeon HD 2000 and newer GeForce 8 and newer1	GeForce 8 and newer	See #VA-API drivers
H.265/HEVC	8bit	Radeon R9 Fury and newer	GeForce 900 and newer3	否
10bit	Radeon 400 and newer	否4		
VP9	8bit	Raven Ridge + Radeon RX 5000 and newer	GeForce 900 and newer3	
10bit/12bit	否4			
AV1	8bit			

Codec	Color depth	mesa包	nvidia-utils包	libvdpau-va-gl包 (VA-API adapter)
		Radeon RX 6600 and higher/newer	GeForce 30 and newer5	
10bit	否4			

1. Up until GeForce GTX 750.
2. Except GeForce 8800 Ultra, 8800 GTX, 8800 GTS (320/640 MB).
3. Except GeForce GTX 970 and GTX 980.
4. NVIDIA implementation is limited to 8bit streams .
5. Starting with driver version 510.

NVIDIA driver only

Codec	nvidia-utils包	
NVDEC	NVENC	
MPEG-2	Fermi and newer1	否
VC-1		
H.264/MPEG-4 AVC	Kepler and newer2	
H.265/HEVC 8bit	Maxwell (GM206) and newer	Maxwell (2nd Gen) and newer
H.265/HEVC 10bit	Pascal and newer	
VP8	Maxwell (2nd Gen) and newer	否
VP9 8bit	Maxwell (GM206) and newer	
VP9 10bit & 12bit	Pascal and newer	
AV1 8bit & 10bit	Ampere and newer3	Ada Lovelace and newer

1. Except GM108 (not supported)
2. Except GM108 and GP108 (not supported)
3. Except A100 (not supported)

应用程序支持

应用程序	解码		编码				文档
VA-API	VDPAU	NVDEC	Vulkan	VA-API	NVENC	Vulkan	
FFmpeg	是	是	是	是	是	是	是	FFmpeg#Hardware video acceleration
GStreamer	是	否	是	是	是	是	否	GStreamer#Hardware video acceleration
Kodi	是	是	否	否	–	–	–	Kodi#Hardware video acceleration
mpv	是	是	是	是	–	–	–	mpv#Hardware video acceleration
VLC media player	是	是	否	–	–	VLC media player#Hardware video acceleration		
MPlayer	否	是	否	否	–	–	–	MPlayer#Hardware video acceleration
Chromium	是	否	否	否	是	否	否	Chromium#Hardware video acceleration
Firefox	是	否	否	否	否	否		

应用程序	解码	编码	文档				
							Firefox#Hardware video acceleration
GNOME/Web	GStreamer	?	?	?	GNOME/Web#Video		

2.14 HiDPI

难度 高级 | **篇幅** 27.2 KB | **所属** 图形

本节将讲到

- 桌面环境
- X Server
- X Resources
- GUI toolkits
- 引导程序

HiDPI（High Dots Per Inch）显示器，指的是在较小尺寸下却拥有较高分辨率的显示器。Apple 将其称作“视网膜屏幕”，这项技术主要存在于高端笔记本电脑和显示器中。

并非所有的软件都在高分辨率屏幕下工作良好。此页列出了一些常见的调整方法，让您更好的使用 HiDPI 显示器。

桌面环境

GNOME

可在**设置 > 设备 > 显示 > 缩放**中设置合理的数值。也可以使用 gsettings 进行设置：

```
$ gsettings set org.gnome.settings-daemon.plugins.xsettings overrides
"[{'Gdk/WindowScalingFactor', <2>}]"
$ gsettings set org.gnome.desktop.interface scaling-factor 2
```

注意：`scaling-factor`仅能设置为整数。1 = 100%，2 = 200%，依此类推。非整数倍缩放请见下。

非整数倍缩放

在某些设备（例如小平板电脑）上使用 `scaling-factor`设置整数倍缩放的效果可能并不理想。wayland 和 Xorg 均支持非整数倍缩放，但步骤不同：

Wayland

启用实验性的非整数倍缩放功能：

```
$ gsettings set org.gnome.mutter experimental-features "['scale-monitor-framebuffer']"
```

之后再次打开**设置 > 设备 > 显示**。如果没有立即生效，请尝试重启。

注意：使用 Xwayland 的旧程序，因为渲染方法不同，在非 100% 缩放下都可能出现字体模糊的问题。

如要为所有用户启用，需创建如下三个文件：

```
/etc/dconf/profile/user
user-db:user
system-db:local
```

```
/etc/dconf/db/local.d/00-hidpi
[org/gnome/mutter]
experimental-features=['scale-monitor-framebuffer']
/etc/dconf/db/locks/hidpi
/org/gnome/mutter/experimental-features
```

运行`dconf update`并重启系统。这将永久锁定选项。

Xorg

您可以同时使用 `scaling-factor` 和 xrandr 来实现任意的非整数倍缩放。这可以使 TTF 字体被正确缩放，防止出现单独使用 `xrandr` 时出现的模糊现象。您可以使用 `gsettings` 来指定放大系数，并用 xrandr 指定缩小系数。

首先使将缩放设为“UI看起来太大”的最小值。通常 2 已经足够大，如果不够大就继续尝试 3 以及更大的数。之后使用 xrandr 设置缩小系数。首先获取相关的输出名称，下面的例子将使用 `eDP1` 。先试着将缩小系数设为1.25，如果 UI 看起来仍然太大，则增大缩放系数。反之则缩小缩放系数。

```
$ xrandr --output eDP1 --scale 1.25x1.25
```

注意：如果你的鼠标光标移动范围和屏幕显示并不匹配，你可能需要同时使用 `--panning`。参阅#Side display。

可考虑使用autorandr包让设置在重启时保持不变。参考这个网页获取更多信息。

在GNOME设置守护进程中的xsettings插件中，DPI设置是硬编码的。这将忽略Xorg的设置。这里有一篇重编译 Gnome Settings Daemon\[失效链接 2022-09-18 ⓘ\]的博客文章。在文档中还介绍了另一种方法来设置 xsettings 的DPI：

你可以使用dconf编辑器转到这个键值：

```
/org/gnome/settings-daemon/plugins/xsettings/overrides
```

加入这个条目：

```
'Xft/DPI': <153600>
```

来自 README.xsettings

注意必须使用这种方式指定（使用<>）。

上述DPI数值的单位是1024分之一英寸。

字体缩放

可以另外设置字体缩放。在 Gnome Tweak 中的**字体 > 缩放**，或者也可使用 gsettings。字体缩放倍率不需要是整数。

```
$ gsettings set org.gnome.desktop.interface text-scaling-factor 1.5
```

Xorg 下的 GTK+ 和 Gnome 外壳元素

在老版本的 Gnome 中（3.35 以前），Gnome 外壳 UI 元素中的字体缩放可能需要以单独编辑界面 CSS 文件的方式单独设置。文件是 `/usr/share/themes/`你的主题`/gnome-shell/gnome-shell.css`。单独设置其中所有的”font-size“属性，使其与缩放后的 UI 大小一致。设置完后，切换到另一个主题，再切换回来。

如果需要更改窗口标题栏的字体大小，可以使用 dconf 编辑器。（`org > gnome > desktop > wm > preferences :: titlebar-font`）这同时需要关闭`title-bar-uses-system-fonts`选项。 或者也可使用 gsettings：

```
$ gsettings set org.gnome.desktop.wm.preferences titlebar-font 'Cantarell Bold 22' ## 具体内容依需求更改
$ gsettings set org.gnome.desktop.wm.preferences titlebar-uses-system-font false
```

KDE

您可以使用KDE的设置来微调字体、图标和部件缩放，这些改动会同时影响 Qt 和 GTK+ 程序。

要调整整体缩放：

1. **系统设置→显卡与显示器→显示器配置→缩放率**
2. 将滑块调整至适合的位置
3. 重新启动以使设置生效

要仅调整字体缩放：

1. **系统设置→外观→字体**
2. 勾选**“固定字体DPI”**并调整DPI的值。调整之后重新启动应用程序即可生效。要在整个桌面上生效，您需要注销之后重新登录。

要仅调整图标缩放：

1. **系统设置→外观→图标→配置图标大小**
2. 为每一项选择合适的图标大小，更改将会立即生效。

非整数倍缩放下的Bug

当您使用非整数倍的缩放比例时，这可能导致一些 Qt 应用程序的字体渲染出现问题（例如 okular）。

有一个办法可以绕过这个问题：

1. 将缩放比例设为1

2. 用上面的方法调整字体和图标缩放（这会影响所有的应用程序，且不会导致字体问题）
3. 重新启动KDE
4. 如果需要缩放 GTK 程序，设置环境变量`GDK_SCALE/GDK_DPI_SCALE`（参见下文）。

托盘图标不缩放

在 Xorg 环境下，托盘图标不会跟随整体缩放。Plasma 默认会忽略 Qt 设置。要让 Plasma 使用 Qt 设置，要么使用 Wayland，要么将 `PLASMA_USE_QT_SCALING` 设为 1；但对于多台不同 DPI 显示器混用的环境，这会让 UI 显示变得更糟糕。也可以将底栏高度设置为原来的两倍以缩放底栏和底栏内的应用程序图标，此时右键菜单内的图标等其他元素将不会缩放。

Xfce

打开**设置管理器→外观→字体**，修改 DPI 的值。在 HiDPI 显示器上，通常可以设为180或者192。要获得更精确的数字，可以使用`xdpyinfo | grep resolution`，使用输出DPI两倍的值。

要增大托盘图标，右键点击托盘的空白处，选择属性，将最大图标大小设置为32,48或者64。

Cinnamon

应当开箱即用。

Enlightenment

对于E18，首先打开设置面板，在**外观→缩放**中，你可以调整缩放倍数。在MBPr 15上，你可以选择1.2。

X Server

某些程序使用 X Server 所提供的 DPI 值。比如 i3 （来源） 和 Chromium（来源）。

**本文或本章节的事实准确性存在争议。**

原因： Xorg always sets dpi to 96. See this, this and finally this.（在 Talk:HiDPI 中讨论）

要验证 X Server 是否正确检测到了您的显示器的DPI，使用 xorg-xdpyinfo包 中的 `xdpyinfo` 工具。

```
$ xdpyinfo | grep -B 2 resolution
screen #0:
  dimensions:    3200x1800 pixels (423x238 millimeters)
  resolution:    192x192 dots per inch
```

这个例子中 X Server 检测到的屏幕尺寸并不准确（423mm x 328mm,实际上Dell XPS 9530的屏幕尺寸是346mm x 194mm），但报告的 DPI 是96的整数倍。通常这往往比正确的DPI更好，可以保证字体渲染正确。

如果 `xdpyinfo` 显示的DPI不正确，参阅Xorg#Display size and DPI了解如何修复。

X Resources

如果你没有使用一个桌面环境，比如 KDE，Xfce，或是没有一个为您操作 Xorg 设置的程序，你可以通过在 Xresources 中的 `Xft.dpi` 变量手动修改DPI设置。

```
~/.Xresources
Xft.dpi: 180
Xft.autohint: 0
Xft.lcdfilter:  lcddefault
Xft.hintstyle:  hintfull
Xft.hinting: 1
Xft.antialias: 1
Xft.rgba: rgb
```

确保设置在X启动时已经被载入。例如在`~/.xinitrc`中使用`xrdb -merge ~/.Xresources`。有关详细信息，请参阅 Xresources。

通常这会让大多数地方的字体大小正确，但这并不会影响图标大小。

在设置`Xft.dpi`的同时设置 GUI toolkit 缩放（例如`GDK_SCALE`）可能导致某些程序界面元素过大（例如firefox）。

GUI toolkits

Qt 5

自从 Qt 5.6 开始，Qt 5 应用程序可以遵守屏幕DPI。设置环境变量`QT_AUTO_SCREEN_SCALE_FACTOR`以启用这项功能。

```
export QT_AUTO_SCREEN_SCALE_FACTOR=1
```

如果自动检测的 DPI 并不理想，你也可以按屏幕(`QT_SCREEN_SCALE_FACTORS`)或全局(`QT_SCALE_FACTOR`)手动设置缩放，有关详细信息，请参阅 Qt 博客文章。

注意：

- 如果您手动设置了缩放，则需要设置`QT_AUTO_SCREEN_SCALE_FACTOR=0`。否则某些明确启用 HiDPI 支持的程序会被缩放两次。
- `QT_SCALE_FACTOR`缩放字体，但`QT_SCREEN_SCALE_FACTORS`并不会。
- 如果您还在**xrdb**中手动设置过DPI以支持其他toolkits，同时使用`QT_SCALE_FACTORS`会使字体过大。
- 如果您有多个不同DPI的屏幕，即 #Side display，您可能需要设置`QT_SCREEN_SCALE_FACTORS="2;2"`。

还可以采取其他方式，例如固定DPI:

```
$ QT_FONT_DPI=96 clementine
```

一个启动WPS表格的实例：

```
$ QT_SCREEN_SCALE_FACTORS=1 QT_FONT_DPI=192 QT_IM_MODULE=fcitx /usr/bin/et %F
```

GDK 3 (GTK+ 3)

要将UI缩放为两倍大小：

```
export GDK_SCALE=2
```

并同时不影响字体：

```
export GDK_DPI_SCALE=0.5
```

GTK+ 2

GTK+ 2本身并不支持缩放UI。但您可以使用 themix-full-gitAUR 创建预缩放过的主题。

Elementary (EFL)

要将缩放倍数设为1.5：

```
 export ELM_SCALE=1.5
```

更多信息请查看 https://phab.enlightenment.org/w/elementary/

引导程序

GRUB

降低帧缓冲分辨率

参见GRUB/Tips and tricks#Setting the framebuffer resolution。

改变GRUB字体大小

在`/usr/share/fonts/`中找一个你喜欢的字体。

使用GRUB工具转换：

```
# grub-mkfont -s 30 -o /boot/grubfont.pf2 /usr/share/fonts/FontFamily/FontName.ttf
```

注意：`-s 30`是字体大小。

编辑`/etc/default/grub`来设置字体。参见GRUB/Tips and tricks#Background image and bitmap fonts:

```
GRUB_FONT="/boot/grubfont.pf2"
```

使用`grub-mkconfig -o /boot/grub/grub.cfg`重新生成 GRUB 配置。

应用程序

浏览器

Firefox

Firefox 应当使用 #GDK 3 (GTK+ 3)) 的设置。但是，建议的 `GDK_SCALE` 可能会使 Firefox 界面的缩放并不一致，并且不能使用小数.你可以使用`GDK_DPI_SCALE`来代替。

要覆盖这些设置，请打开 Firefox 的高级选项页面（`about:config`）并将 `layout.css.devPixelsPerPx` 设置为 `2`（或是其它需要的数值），但这个选项也可能导致 Firefox 界面的缩放不一致。如果 Firefox 没有缩放字体，您可能需要创建 `userChrome.css` 并添加适当的样式。有关 `userChrome.css`，请访问 mozillaZine。

```
~/.mozilla/firefox/<profile>/chrome/userChrome.css
@namespace url("https://www.mozilla.org/keymaster/gatekeeper/there.is.only.xul");

/* #tabbrowser-tabs, #navigator-toolbox, menuitem, menu, ... */
* {
    font-size: 15px !important;
}

/* exception for badge on adblocker */
.toolbarbutton-badge {
    font-size: 8px !important;
}
```

警告：下面的扩展不支持 Firefox Quantum （57版本及以上）。

如果你将 HiDPI 显示器与另一台显示器一起使用，你可以使用 AutoHiDPI 来自动调整。另外，从 Firefox 49 以来，它会自动根据您的屏幕分辨率调整缩放，从而更容易处理多个屏幕的情况。

Chromium / Google Chrome

Chromium 应当使用#GDK 3 (GTK+ 3))设置。

要覆盖这一选项，请使用`--force-device-scale-factor` flags。这将缩放所有内容。例如：`chromium --force-device-scale-factor=2`。

当此选项设为1时将会采用正常缩放，此选项也可以使用小数。要永久应用此更改，对于Chromium，可以将其添加到`~/.config/chromium-flags.conf`：

```
~/.config/chromium-flags.conf
--force-device-scale-factor=2
```

对于Chrome，使用 `~/.config/chrome-flags.conf`。

如果你将 HiDPI 显示器与另一台显示器一起使用，你可以使用 reszoom 扩展。这个扩展将会根据所在屏幕自动调整缩放。

Opera

Opera 应当使用 #GDK 3 (GTK+ 3)) 设置。

要覆盖这一选项，请使用 `--alt-high-dpi-setting=X` 命令行选项。其中 X 是所需的 DPI。例如，使用 `--alt-high-dpi-setting=144` Opera 将认为 DPI 是 144。在较新版本中，Opera 将使用字体DPI（在 KDE 中可以强制设置字体 DPI）。

Thunderbird

参见#Firefox。要访问 `about:config`，打开**首选项→高级→配置编辑器**。

Wine程序

运行

```
$ winecfg
```

并且更改**显示**选项卡中的 DPI。这仅影响字体大小。

Skype

Skype for Linux (skypeforlinux-stable-binAUR) 使用 #GDK 3 (GTK+ 3))。

Spotify

您可以简单的使用 `Ctrl++` 和 `Ctrl+-` 来调整缩放。使用 `Ctrl+0` 来恢复默认缩放。缩放设置保存在`~/.config/spotify/Users/YOUR-SPOTIFY-USER-NAME/prefs`，您可能需要自行创建该设置文件：

```
~/.config/spotify/Users/YOUR-SPOTIFY-USER-NAME/prefs
app.browser.zoom-level=100
```

Spotify 也可以使用自定义的缩放启动。这个系数和`~/.config/spotify/Users/YOUR-SPOTIFY-USER-NAME/prefs`中的系数相乘的积是实际的缩放倍数。

```
$ spotify --force-device-scale-factor=1.5
```

Zathura 文档查看器

文档缩放应当开箱即用。

UI缩放可以在配置文件中设置 （注意 "font" 是一个girara option）：

```
set font "monospace normal 20"
```

Sublime Text 4

Sublime Text 4 完整支持显示缩放。在**首选项→设置**中添加 `"ui_scale": 2.0` 。

IntelliJ IDEA

IntelliJ IDEA 15 及以上应有对 HiDPI 的支持， 如果这不能工作最简便的解决办法可能是覆盖默认的字体设置：

: *File -> Settings -> Behaviour & Appearance -> Appearance*

14版本中将 `-Dhidpi=true` 添加到 `$HOME/.IdeaC14/`
或 `/usr/share/intelligj-idea-ultimate-edition/bin/` 的方法应该不再需要了。

NetBeans

NetBeans 可以在启动时用`--fontsize`选项设置字体大小。要要使此设置永久生效，编辑`/usr/share/netbeans/etc/netbeans.conf`，添加`--fontsize`到`netbeans_default_options`。

编辑器字体大小可以在 Tools → Option → Fonts & Color 中设置。

输出窗口的字体大小可以在Tools → Options → Miscelaneous → Output中控制。

Gimp 2.8

使用 HiDPI主题，或是更改现有主题的 `gtkrc`。(Change all occurrences of the size `button` to `dialog`，例如`GimpToolPalette::tool-icon-size`。)

另外也有 gimp-hidpi。

Steam

官方 HiDPI支持

- 从2018年1月25日开始，测试版本已经支持HiDPI并能自动启用。
- **Steam →设置→界面→勾选 "按照显示器大小放大图标和文本（需要重启）"**
- 如果自动检测并不能正确工作，使用`GDK_SCALE=2`来指定缩放级别。
- 如果以上不工作,使用`steam -forcedesktopscaling 2`或者设置`STEAM_FORCE_DESKTOPUI_SCALING=2.0`。在2023年6月UI大改动之后,该参数还支持分数缩放级别,例如`1.25`。

非官方

使用 HiDPI-Steam-Skin 可以增大字体大小，虽然并不完美，但至少能用。

注意：皮肤的 README 列出了皮肤应当放置的位置。正确的位置下面应该有一个叫`skins_readme.txt`的文件。

MetroSkin Unofficial Patch 也可以使用。

Java 程序

可以通过设置 `sun.java2d.uiScale` 来缩放使用 AWT/Swing 框架的 Java 程序。例如：

```
java -Dsun.java2d.uiScale=2 -jar some_application.jar
```

从 Java 9 开始，`GDK_SCALE` 也可以缩放 Swing 程序。

Mono 程序

根据 ，Mono 程序应像 GTK3) 程序一样缩放。

MATLAB

MATLAB 的近期版本(R2017b)可以设置缩放：

```
>> s = settings;s.matlab.desktop.DisplayScaleFactor
>> s.matlab.desktop.DisplayScaleFactor.PersonalValue = 2
```

此设置将在重启 Matlab 后生效。

VirtualBox

注意：仅适用于启用了缩放的 KDE。

VirtualBox 按照系统的缩放级别缩放虚拟监视器，从而降低虚拟机的最大分辨率（参见）。

可以单独缩小 VirtualBox 来绕过这个问题：

```
$ QT_SCALE_FACTOR=0.5 VirtualBox --startvm vm-name
```

Zoom

可以指定`QT_SCALE_FACTOR`来启动 Zoom：

```
$ QT_SCALE_FACTOR=2 zoom
```

不支持的程序

run_scaled-gitAUR 可以缩放应用程序（内部使用xpra包）。

另一个办法是使每个程序在各自的 VNC 桌面上运行。要缩放 VNC 查看器，使用 vncdesk-gitAUR。你可以为每个程序设置单独的桌面，使用简单的命令启动客户端和服务端，例如`vncdesk 2`。

x11vnc 有一个实验性的选项 `-appshare`，可以为每个窗口单独打开一个 VNC 查看器。也许我们可以在上面做些文章。

多显示器

HIDPI 将会对整个桌面生效，所以如果你同时使用普通显示器，那么在普通显示器上显示元素会变得很大。但是 Wayland 已经支持为每个显示器设置单独的缩放。

侧边显示器

有一个解决方案是使用 xrandr 提供的缩放选项。比如你希望把一个非 HIDPI 的显示器（假设连接在 DP1）放置在内部 HIDPI 显示器（假设连接在 eDP 1）的右侧，你可以运行如下命令：

```
xrandr --output eDP-1 --auto --output DP-1 --auto --scale 2x2 --right-of eDP-1
```

当外部的显示器拓展到内部显示器上方的时候，你可能会看到外部显示器显示了一些内部显示器的内容，这种情况下你可以手动指定显示器的位置或者使用这个脚本.

你可能会遇到鼠标没办法移动到整个屏幕范围的问题，这是一个已知的问题，你可以通过应用 `xserver-org` 的补丁来解决（或者尝试使用平移选项 `panning`，但这可能会引发其他问题）。

这是一个 4K 笔记本电脑外接 1920x1080 显示器在右侧时使用平移语法的示例：

```
xrandr --output eDP-1 --auto --output HDMI-1 --auto --panning 3840x2160+3840+0 --scale 2x2 --right-of eDP-1
```

通常来说，如果你的HIDPI显示器分辨率为 AxB 像素，普通显示器分辨率为 CxD，并且你将其缩放比例设置为 ExF，那么将普通显示器放置在HIDPIP显示器右侧的命令行格式如下：

```
xrandr --output eDP-1 --auto --output HDMI-1 --auto --panning [C*E]x[D*F]+[A]+0 --scale [E]x[F] --right-of eDP-1
```

如果平移（panning）没有办法解决的话，那么手动设置显示器位置并调整主要显示屏幕可能是更好的选择。

以下是一个示例语法：一台分辨率为 2560x1440、DPI 为 210 的 WQHD 笔记本内置显示器（`eDP1`），使用原生分辨率放置在一台分辨率为 1920x1080、DPI 为 96 的 FHD 外接显示器（`HDMI`）下方，并将外接显示器缩放以匹配全局 DPI 设置。

```
xrandr --output eDP-1 --auto --pos 0x1458 --output HDMI-1 --scale 1.35x1.35 --auto --pos 0x0 --fb 2592x2898
```

总屏幕大小（`--fb`）和位置（`--pos`）需要根据缩放倍数进行计算。

在本例中：

- 笔记本显示器（`eDP1`）没有缩放，使用原生分辨率，因此其尺寸为 2560x1440。
- 外接显示器（`HDMI`）被缩放，需要将其视为更大的屏幕，其逻辑尺寸为 `(1920*1.35)x(1080*1.35)`。
- `eDP1` 的 Y 轴位置为 `1080*1.35=1458`。
- 总屏幕大小：
- X 方向：取 `eDP1` 和 `HDMI` 中较大的宽度，即 `1920*1.35=2592`。
- Y 方向：两者高度之和，即 `1440 + (1080*1.35)=2898`。

通用命令：

假设：

- 高分辨率显示器（HiDPI）分辨率为 AxB 像素。

- 普通显示器分辨率为 CxD 像素。
- 普通显示器的缩放比例为 ExF。
- 高分辨率显示器放置在普通显示器下方。

命令如下：

```
xrandr --output eDP-1 --auto --pos 0x(DxF) --output HDMI-1 --auto --scale [E]x[F] --pos 0x0 --fb [greater between A and (C*E)]x[B+(D*F)]
```

You may adjust the "sharpness" parameter on your monitor settings to adjust the blur level introduced with scaling.

注意：Above solution with `--scale 2x2` does not work on some Nvidia cards. No solution is currently available. A potential workaround exists with configuring `ForceFullCompositionPipeline=On` on the `CurrentMetaMode` via `nvidia-settings`. For more info see .

多外部监视器

如果你有多于个低于内置的 HiDPI 显示器 DPI 的显示器，那么可能会碰到一些问题。这种情况下您可以想要缩小 HiDPI 显示器的内容，例如：

```
xrandr --output eDP1 --scale 0.5x0.5 --output DP2 --right-of eDP1 --output HDMI1 --right-of DP2
```

另外，当您缩小内置 HIDPI 显示器中的内容时，内置显示器的字体可能会变得有些模糊。但这与扩大外置显示器内容时带来的模糊并不相同。您可以对比之后选择自己想要的方案。

镜像

如果你只想使用镜像的话，可以按照如下操作：

A×B是你 HiDPI 显示器的原生分辨率，CxD 是你的另一个显示器的原生分辨率。

```
xrandr --output HDMI --scale [A/C]x[B/D]
```

一个 QHD 的例子(3200/1920 = 1.66 and 1800/1200 = 1.5)：

```
xrandr --output HDMI --scale 1.66x1.5
```

UHD和1080p的例子 (3840/1920=2 2160/1080=2)：

```
xrandr --output HDMI --scale 2x2
```

您可以调整显示器上的清晰度选项以减少模糊感。

Linux 控制台

默认的 Linux 控制台在 HiDPI 显示器上太小了。kbd包中最大的字体是 `latarcyrheb-sun32`，其他软件包中包含一些更大的字体，例如terminus-font包中的`ter-132n`和`ter-132b`（粗体）。参阅Linux console#Fonts获取配置详情。

在改变字体之后，当您切换到其他控制台时，屏幕上的内容可能无法读取。要修复这一问题，你可以设置force specific mode，例如`video=2560x1600@60`（设为您显示器的原生分辨率）并重新启动。

另请参阅

- Ultra HD 4K Linux Graphics Card Testing (Nov 2013)
- Understanding pixel density

2.15 NVIDIA/提示和技巧

难度 高级 | **篇幅** 27.3 KB | **所属** 图形

本节将讲到

- 修复终端分辨率
- 使用电视输出
- 使用电视（DFP）作为唯一显示器的 X 配置
- 无显示器（Headless）分辨率
- 检查电源来源

修复终端分辨率

从 nouveau 切换可能会导致启动终端以较低分辨率显示。

对于 GRUB，请参阅 GRUB/Tips and tricks#设置帧缓冲分辨率获取详细信息。

对于 systemd-boot，请在 `*esp*/loader/loader.conf` 中设置 `console-mode`。详情请参阅 systemd-boot#启动选单配置。

对于 rEFInd，请在 `*esp*/EFI/refind/refind.conf` 中设置 `use_graphics_for +,linux`。一个小问题是，这会在启动时隐藏内核参数的显示。

提示：如果上述方法无法修复您的终端分辨率，可能需要在 UEFI 设置中完全禁用 Legacy BIOS 模式（通常称为兼容性支持模块，CSM 或 Legacy Boot）。在继续之前，请确保所有设备都配置为使用 UEFI 启动。

使用电视输出

参见 Wikibooks:NVIDIA/TV-OUT。

使用电视（DFP）作为唯一显示器的 X 配置

如果未自动检测到显示器，X 服务器会回退到 CRT-0。当使用 DVI 连接的电视作为主显示器，并且在电视关闭或断开连接时启动 X 时，这可能会成为一个问题。

要强制 NVIDIA 使用 DFP，请将 EDID 的副本存储在文件系统中的某个位置，以便 X 可以解析该文件，而不是从电视/DFP 读取 EDID。

要获取 EDID，请启动 `nvidia-settings`。它会以树形格式显示一些信息，暂时忽略其他设置，选择 GPU（相应的条目应标为 "GPU-0" 或类似名称），点击 `DFP` 部分（同样，`DFP-0` 或类似名称），点击 获取 `EDID` 按钮并将其存储在某个位置，例如 `/etc/X11/dfp0.edid`。

如果前端没有连接鼠标和键盘，可以仅使用命令行获取 EDID。运行一个足够详细的 X 服务器以打印出 EDID 块：

```
$ startx -- -logverbose 6
```

X 服务器完成初始化后，关闭它，您的日志文件可能会在 `/var/log/Xorg.0.log` 中。使用 `nvidia-xconfig` 提取 EDID 块：

```
$ nvidia-xconfig --extract-edids-from-file=/var/log/Xorg.0.log --extract-edids-output-file=/etc/X11/dfp0.bin
```

编辑 `xorg.conf`，在 `Device` 部分添加：

```
Option "ConnectedMonitor" "DFP"
Option "CustomEDID" "DFP-0:/etc/X11/dfp0.bin"
```

`ConnectedMonitor` 选项强制驱动程序将 DFP 识别为已连接。`CustomEDID` 为设备提供 EDID 数据，这意味着它将像在 X 进程期间电视/DFP 已连接一样启动。

这样，您可以在启动时自动启动显示管理器，并在电视开机时拥有一个正常工作和正确配置的 X 屏幕。

如果上述更改没有生效，您可以在 `xorg.conf` 的 `Device` 部分尝试删除 `Option "ConnectedMonitor" "DFP"` 并添加以下行：

```
Option "ModeValidation" "NoDFPNativeResolutionCheck"
Option "ConnectedMonitor" "DFP-0"
```

`NoDFPNativeResolutionCheck` 防止 NVIDIA 驱动程序禁用所有不符合原生分辨率的模式。

无显示器（Headless）分辨率

在无显示器模式下，分辨率会回退到 640x480，这是 VNC 或 Steam Link 使用的分辨率。要以更高的分辨率（例如 1920x1080）启动，请在 `xorg.conf` 的 `Screen` 子节中指定一个 `Virtual` 条目：

```
Section "Screen"
   [...]
   SubSection     "Display"
       Depth       24
       Virtual     1920 1080
   EndSubSection
EndSection
```

提示：使用无显示器模式可能会比较复杂且容易出错。例如，在无显示器模式下，桌面环境和 nvidia-utils 包 不提供图形化的方式来更改分辨率。为了便于设置分辨率，可以使用 DP 或 HDMI 虚拟适配器，模拟连接到该端口的显示器的存在。然后可以通过远程会话（如 VNC 或 Steam Link）正常更改分辨率。

检查电源来源

NVIDIA X.org 驱动也可以用来检测 GPU 当前的电源来源。要查看当前的电源来源，请检查 'GPUPowerSource' 只读参数（0 - 交流电，1 - 电池）：

```
$ nvidia-settings -q GPUPowerSource -t
1
```

监听 ACPI 事件

NVIDIA 驱动程序会自动尝试连接到 acpid 守护进程，并监听诸如电池电量、扩展坞、某些热键等 ACPI 事件。如果连接失败，X.org 将输出以下警告：

```
~/.local/share/xorg/Xorg.0.log
NVIDIA(0): ACPI: failed to connect to the ACPI event daemon; the daemon
NVIDIA(0):     may not be running or the "AcpidSocketPath" X
NVIDIA(0):     configuration option may not be set correctly.  When the
NVIDIA(0):     ACPI event daemon is available, the NVIDIA X driver will
NVIDIA(0):     try to use it to receive ACPI event notifications.  For
NVIDIA(0):     details, please see the "ConnectToAcpid" and
NVIDIA(0):     "AcpidSocketPath" X configuration options in Appendix B: X
NVIDIA(0):     Config Options in the README.
```

虽然完全无害，但你可以通过在 `/etc/X11/xorg.conf.d/20-nvidia.conf` 中禁用 `ConnectToAcpid` 选项来消除此消息：

```
Section "Device"
  ...
  Driver "nvidia"
  Option "ConnectToAcpid" "0"
  ...
EndSection
```

如果你使用的是笔记本电脑，安装并启用 acpid 守护进程可能是个好主意。

在 shell 中显示 GPU 温度

有三种方法可以查询 GPU 温度。*nvidia-settings* 要求你正在使用 X，而 *nvidia-smi* 或 *nvclock* 则不需要。另外请注意，*nvclock* 目前不适用于较新的 NVIDIA 显卡，如 GeForce 200 系列显卡以及嵌入式 GPU，如 Zotac IONITX 的 8800GS。

nvidia-settings

要在 shell 中显示 GPU 温度，请使用 *nvidia-settings*，如下所示：

```
$ nvidia-settings -q gpucoretemp
  Attribute 'GPUCoreTemp' (hostname:0[gpu:0]): 49.
    'GPUCoreTemp' is an integer attribute.
    'GPUCoreTemp' is a read-only attribute.
    'GPUCoreTemp' can use the following target types: GPU.
```

该主板的 GPU 温度为 49 °C。

为了仅获取温度以用于 *rrdtool* 或 *conky* 等工具：

```
$ nvidia-settings -q gpucoretemp -t
49
```

nvidia-smi

使用 *nvidia-smi*，它可以直接从 GPU 读取温度，而无需使用 X，例如在运行 Wayland 或无头服务器时。

要在 shell 中显示 GPU 温度，请使用 *nvidia-smi*：

```
$ nvidia-smi
Wed Feb 28 14:27:35 2024
+-----------------------------------------------------------------------------------------
---+
| NVIDIA-SMI 550.54.14              Driver Version: 550.54.14      CUDA Version: 12.4
|
|-----------------------------------------+------------------------
+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr.
ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute
M. |
|                                         |                        |               MIG
M. |
|
=========================================+========================+=====================
==|
|   0  NVIDIA GeForce GTX 1660 Ti     Off |   00000000:01:00.0  On |
N/A |
|  0%   49C    P8              9W /  120W |     138MiB /   6144MiB |      2%
Default |
|                                         |                        |
N/A |<br>+-----------------------------------------+------------------------
+----------------------+

+-----------------------------------------------------------------------------------------
---+
| Processes:
|
|  GPU   GI   CI        PID   Type   Process name                              GPU
Memory |
|        ID   ID                                                               Usage
|
|
=========================================================================================
==|
|    0   N/A  N/A    223179      G   weston                                     
120MiB |
<br>+-----------------------------------------------------------------------------------------
-------+<br>```
```

仅获取温度：

$ nvidia-smi -q -d TEMPERATURE

==========NVSMI LOG==========

Timestamp : Wed Feb 28 14:27:35 2024 Driver Version : 550.54.14 CUDA Version : 12.4

Attached GPUs : 1 GPU 00000000:01:00.0 Temperature GPU Current Temp : 49 C GPU T.Limit Temp : N/A GPU Shutdown Temp : 95 C GPU Slowdown Temp : 92 C GPU Max Operating Temp : 90 C GPU Target Temperature : 83 C Memory Current Temp : N/A Memory Max Operating Temp : N/A

为了仅获取温度以用于 *rrdtool* 或 *conky* 等工具：
$ nvidia-smi --query-gpu=temperature.gpu --format=csv,noheader,nounits

49

nvclock

安装 [nvclock](https://aur.archlinux.org/packages/nvclock/)AUR 包。

> **注意：***nvclock* 无法访问较新的 NVIDIA 显卡（如 GeForce 200 系列显卡）上的温度传感器。

nvclock 和 *nvidia-settings*/*nv-control* 报告的温度之间可能存在显著差异。根据 *nvclock* 的作者（thunderbird）在[这篇帖子](https://sourceforge.net/projects/nvclock/forums/forum/67426/topic/1906899)中的说法，*nvclock* 的值应该更准确。

超频与散热

> **警告：**超频可能会永久损坏您的硬件。您已被警告。

在 nvidia-settings 中启用超频

> **注意：**
>
> - 如果 Xorg 服务器以无根模式运行，则无法应用超频设置。考虑 [以 root 身份运行 Xorg](../X%20服务器/Xorg.md#Xorg_as_Root)。
> - 启用 DRM 内核模式设置可能会导致超频不可用，无论 Coolbits 值如何。

超频通过 `Device` 部分中的 *Coolbits* 选项控制，该选项启用了各种不受支持的功能：
Option "Coolbits" "value"

> **提示：**可以使用 *nvidia-xconfig* 轻松控制 *Coolbits* 选项，它会操作 Xorg 配置文件：
>
> ```
> # nvidia-xconfig --cool-bits=value
> ```

Coolbits 值是二进制数系统中其组成位的总和。组成位如下：

- `1`（位 0）- 在 *nvidia-settings* 的*时钟频率*页面上启用旧版（Fermi 之前）核心的超频。
- `2`（位 1）- 设置此位时，驱动程序将"尝试在使用具有不同显存量的 GPU 时初始化 SLI"。
- `4`（位 2）- 在 *nvidia-settings* 的*温度监控*页面上启用手动配置 GPU 风扇速度。
- `8`（位 3）- 在 *nvidia-settings* 的 *PowerMizer* 页面上启用超频。自 337.12 版本起适用于 Fermi 架构及更新架构。
- `16`（位 4）- 使用 *nvidia-settings* CLI 选项启用超压。自 346.16 版本起适用于 Fermi 架构及更新架构。

要启用多个功能，请将 *Coolbits* 值相加。例如，要启用 Fermi 核心的超频和超压，请设置 `Option "Coolbits" "24"`。

Coolbits 的文档可以在 `/usr/share/doc/nvidia/html/xconfigoptions.html` 和[这里](https://download.nvidia.com/XFree86/Linux-x86_64/565.77/README/xconfigoptions.html#Coolbits)找到。

> **注意：**另一种方法是在 DOS 下（首选）或在 Win32 环境中通过 [nvflash](https://www.techpowerup.com/download/nvidia-nvflash/) 和 [NiBiTor 6.0](https://www.guru3d.com/files-details/nvidia-bios-editor-download-nibitor.html) 编辑并重新刷写 GPU BIOS。刷写 BIOS 的优势在于不仅可以提高电压限制，而且通常比 Coolbits 等软件超频方法更稳

定。[Fermi BIOS 修改教程](https://ivanvojtko.blogspot.sk/2014/03/how-to-overclock-geforce-460gtx-fermi.html)

设置静态 2D/3D 时钟

使用 [kernel module parameter](../../02_系统管理与内核/内核/内核模块.md) 参数以最大性能级别启用 PowerMizer（没有此设置，VSync 将无法工作）：

/etc/modprobe.d/nvidia.conf

options nvidia NVreg_RegistryDwords="PerfLevelSrc=0x2222"

降低 GPU Boost 时钟

对于 [Ampere (NV170/GA*XXX*)](https://nouveau.freedesktop.org/CodeNames.html#NV170) GPU 及更高版本，时钟提升的工作方式不同，最大时钟在启动时设置为支持的最高限制。如果这是您想要的，则无需进一步配置。

缺点是电源效率较低。随着时钟速度的提高，需要增加电压以保持稳定性，从而导致功耗、发热和风扇噪音的非线性增加。因此，降低提升时钟限制将提高效率。

可以使用 nvidia-smi 更改提升时钟限制，以 root 身份运行：

- 列出支持的时钟速率：

$ nvidia-smi -q -d SUPPORTED_CLOCKS

- 将 GPU 提升时钟限制设置为 1695 MHz：
- 将内存提升时钟限制设置为 5001 MHz：

为了优化效率，请使用 nvidia-smi 检查运行您最喜欢的游戏时的 GPU 利用率。VSync 应开启。降低提升时钟限制将增加 GPU 利用率，因为较慢的 GPU 将使用更多时间来渲染每一帧。最佳效率是在不会导致卡顿的最低时钟下实现的，当利用率达到 100% 时会出现卡顿。然后，每一帧都可以渲染得足够快以跟上刷新率。

例如，在 RTX 3090 Ti 上使用上述设置而不是默认设置，在 4K@60 下玩 Hitman 3 时，功耗降低了 30%，温度从 75 度降至 63 度，风扇速度从 73% 降至 57%。

保存超频设置

通常，在 *nvidia-settings* 界面中插入的时钟和电压偏移不会保存，重启后会丢失。 幸运的是，有一些工具提供了在专有驱动程序下进行超频的界面，能够保存用户的超频偏好并在启动时自动应用它们。 其中一些工具是：

- [gwe](https://aur.archlinux.org/packages/gwe/)AUR - 图形界面，在桌面会话启动时应用设置
- [nvclock](https://aur.archlinux.org/packages/nvclock/)AUR 和 [systemd-nvclock-unit](https://aur.archlinux.org/packages/systemd-nvclock-unit/)AUR - 图形界面，在系统启动时应用设置
- [nvoc](https://aur.archlinux.org/packages/nvoc/)AUR - 基于文本，配置文件位于 `/etc/nvoc.d/`，在桌面会话启动时应用设置

否则，可以在[启动](../../02_系统管理与内核/系统管理/自动启动.md)时在 *nvidia-settings* 的命令行界面中设置 `GPUGraphicsClockOffset` 和 `GPUMemoryTransferRateOffset` 属性。例如：

$ nvidia-settings -a "GPUGraphicsClockOffset[performance_level]=offset" $ nvidia-settings -a "GPUMemoryTransferRateOffset[performance_level]=offset"

其中 `*performance_level*` 是最高性能级别的编号。如果机器上有多个 GPU，则应指定 GPU ID：`[gpu:*gpu_id*]GPUGraphicsClockOffset[*performance_level*]=*offset*`。

自定义 TDP 限制

现代 NVIDIA 显卡会限制频率以保持在 TDP 和温度限制内。为了提高性能，可以更改 TDP 限制，这将导致更高的温度和更高的功耗。

例如，将功耗限制设置为 160.30W：
要在启动时设置功耗限制（无需驱动程序持久性）：
/etc/systemd/system/nvidia-tdp.timer

[Unit] Description=Set NVIDIA power limit on boot

[Timer] OnBootSec=5

[Install] WantedBy=timers.target

/etc/systemd/system/nvidia-tdp.service

[Unit] Description=Set NVIDIA power limit

[Service] Type=oneshot ExecStart=/usr/bin/nvidia-smi -pl 160.30

现在启用 `nvidia-tdp.timer`。

在登录时设置风扇速度

> [](https://wiki.archlinuxcn.org/wzh/File:Tango-edit-clear.png)**本文或本章节的语言、语法或风格需要改进。参考：[帮助:风格](https://wiki.archlinuxcn.org/wzh/Help:%E9%A3%8E%E6%A0%BC)**
>
> **原因：**请参阅 [#在 nvidia-settings 中启用超频](#在_nvidia-settings_中启用超频) 以了解 *Coolbits* 的描述。（在[Talk:NVIDIA/提示和技巧](https://wiki.archlinuxcn.org/wiki/Talk:NVIDIA/%E6%8F%90%E7%A4%BA%E5%92%8C%E6%8A%80%E5%B7%A7)讨论）

您可以使用 *nvidia-settings* 控制台界面调整显卡的风扇速度。首先确保您的 Xorg 配置已在 [Coolbits](#在_nvidia-settings_中启用超频) 选项中启用了位 2。

> **注意：**GeForce 400/500 系列显卡目前无法使用此方法在登录时设置风扇速度。此方法仅允许通过 *nvidia-settings* 在当前 X 会话中设置风扇速度。

将以下行放入您的 [xinitrc](../Xorg%20命令/Xinit.md) 文件中，以在启动 Xorg 时调整风扇速度。将 `*n*` 替换为您要设置的风扇速度百分比。
nvidia-settings -a "[gpu:0]/GPUFanControlState=1" -a "[fan:0]/GPUTargetFanSpeed=n"

您还可以通过增加 GPU 和风扇编号来配置第二个 GPU。
nvidia-settings -a "[gpu:0]/GPUFanControlState=1" -a "[fan:0]/GPUTargetFanSpeed=n" \ -a "[gpu:1]/GPUFanControlState=1" -a [fan:1]/GPUTargetFanSpeed=n" &

如果您使用 [GDM](../GNOME/GDM.md) 或 [SDDM](../显示管理器/SDDM.md) 等登录管理器，可以创建一个桌面条目文件来处理此设置。创建 `~/.config/autostart/nvidia-fan-speed.desktop` 并将此文本放入其中。再次将 `*n*` 更改为您想要的速度百分比。
[Desktop Entry] Type=Application Exec=nvidia-settings -a "[gpu:0]/GPUFanControlState=1" -a "[fan:0]/GPUTargetFanSpeed=n" X-GNOME-Autostart-enabled=true Name=nvidia-fan-speed

> **注意：**在驱动程序版本 349.16 之前，使用 `GPUCurrentFanSpeed` 而不是 `GPUTargetFanSpeed`。

要调整多个显卡的风扇速度，请运行：

$ nvidia-xconfig --enable-all-gpus $ nvidia-xconfig --cool-bits=4

> **注意：**在某些笔记本电脑上（包括 ThinkPad [X1 Extreme](https://devtalk.nvidia.com/default/topic/1052110/linux/can-t-control-gtx-1050-ti-max-q-fan-on-thinkpad-x1-extreme-laptop/post/5340658/#5340658) 和 [P51/P52](https://devtalk.nvidia.com/default/topic/1048624/linux/how-to-set-gpu-fan-speed/post/5321818/#5321818)），有两个风扇，但都不由 nvidia 控制。

使用 NVML 的简单超频脚本

Nvidia 管理库 (NVML) 提供了一个 API，可以管理 GPU 的核心和内存时钟偏移以及功耗限制。要使用此功能，您可以安装 [python-nvidia-ml-py](https://aur.archlinux.org/packages/python-nvidia-ml-py/)AUR，然后使用以下 Python 脚本与您所需的设置。此脚本需要在每次重启后以 root 身份运行以重新应用超频/降压。

from pynvml import *

nvmlInit()

myGPU = nvmlDeviceGetHandleByIndex(0)

nvmlDeviceSetGpcClkVfOffset(myGPU, 000)

nvmlDeviceSetMemClkVfOffset(myGPU, 000)

nvmlDeviceSetPowerManagementLimit(myGPU, 000000)

内核模块参数

> [](https://wiki.archlinuxcn.org/wzh/File:Tango-edit-clear.png)**本文或本章节的语言、语法或风格需要改进。参考：[帮助:风格](https://wiki.archlinuxcn.org/wzh/Help:%E9%A3%8E%E6%A0%BC)**
>
> **原因：**提供高级示例而不解释其作用是没有意义的。（在[Talk:NVIDIA/提示和技巧](https://wiki.archlinuxcn.org/wiki/Talk:NVIDIA/%E6%8F%90%E7%A4%BA%E5%92%8C%E6%8A%80%E5%B7%A7)讨论）

一些选项可以设置为内核模块参数，完整的列表可以通过运行 `modinfo nvidia` 或查看 `nv-reg.h` 获得。另请参阅 [Gentoo:NVidia/nvidia-drivers#Kernel module parameters](https://wiki.gentoo.org/wiki/NVidia/nvidia-drivers#Kernel_module_parameters)。

例如，启用以下选项将启用 PAT 功能 ，这会影响内存的分配方式。PAT 最早在 Pentium III 中引入 ，并且大多数较新的 CPU 都支持此功能（参见 [wikipedia:Page attribute table#Processors](https://en.wikipedia.org/wiki/Page_attribute_table#Processors)）。如果您的系统支持此功能，它应该会提高性能。

/etc/modprobe.d/nvidia.conf

options nvidia NVreg_UsePageAttributeTable=1

在某些笔记本电脑上，要启用任何 NVIDIA 设置调整，您必须包含此选项，否则会返回"不支持设置应用程序时钟"等错误。

/etc/modprobe.d/nvidia.conf

options nvidia NVreg_RegistryDwords="OverrideMaxPerf=0x1"

> **注意：**根据 [Kernel module#Using modprobe.d](../../02_系统管理与内核/内核/内核模块.md#Using_modprobe.d)，如果您使用 [early KMS](NVIDIA.md#DRM_kernel_mode_setting)，则需要 [regenerate the initramfs](../../01_安装与包管理/Initramfs/Mkinitcpio.md)。

保留挂起后的显存

默认情况下，NVIDIA Linux 驱动程序在系统挂起和恢复时仅保存和恢复必要的显存分配。引用 NVIDIA 的话：

：由此导致的显存内容丢失部分由用户空间的 NVIDIA 驱动程序和某些应用程序进行补偿，但在退出电源管理周期时可能会导致渲染损坏和应用程序崩溃等故障。

"仍处于实验阶段"的接口允许保存所有显存（前提是磁盘或 RAM 上有足够的空间）。

要保存和恢复所有显存内容，需要为 `nvidia` 内核模块设置 `NVreg_PreserveVideoMemoryAllocations=1` [内核模块参数](../../02_系统管理与内核/内核/内核模块.md)。虽然 NVIDIA 默认不设置此参数，但 Arch Linux 为支持的驱动程序设置了此参数，使得保留功能开箱即用。

要验证 `NVreg_PreserveVideoMemoryAllocations` 是否已启用，请执行以下命令：
输出中应包含一行 `PreserveVideoMemoryAllocations: 1`，以及 `TemporaryFilePath: "/var/tmp"`，您可以在下面阅读有关此内容的更多信息。

根据上游要求，必要的服务 `nvidia-suspend.service`、`nvidia-hibernate.service` 和 `nvidia-resume.service` 在支持的驱动程序上默认已启用。

有关更多详细信息，请参阅 [NVIDIA 的文档](https://download.nvidia.com/XFree86/Linux-x86_64/565.77/README/powermanagement.html)。

> [](https://wiki.archlinuxcn.org/wzh/File:Tango-inaccurate.png)**本文或本章节的事实准确性存在争议。**
>
> **原因：** 当使用早期 KMS 时，即在 initramfs 中加载 `nvidia` 模块时，它无法访问存储先前显存的 `NVreg_TemporaryFilePath`。换句话说，如果需要休眠，则不应使用早期 KMS。（在 [Talk:NVIDIA/提示和技巧](https://wiki.archlinuxcn.org/wiki/Talk:NVIDIA/%E6%8F%90%E7%A4%BA%E5%92%8C%E6%8A%80%E5%B7%A7) 中讨论）

> **注意：**
>
> - 根据 [内核模块#使用 modprobe.d](../../02_系统管理与内核/内核/内核模块.md#使用_modprobe.d)，如果使用 [早期 KMS](NVIDIA.md#DRM_内核模式设置)，则需要[重新生成 initramfs](../../01_安装与包管理/Initramfs/Mkinitcpio.md)。
> - 显存内容默认保存到 `/tmp`，这是一个 [tmpfs](../../02_系统管理与内核/文件系统/Tmpfs.md)。[NVIDIA 建议](https://download.nvidia.com/XFree86/Linux-x86_64/565.77/README/powermanagement.html#PreserveAllVide719f0)使用其他文件系统以获得最佳性能。如果内存大小不足，这也是必需的。因此，Arch Linux 在支持的驱动程序上默认设置 `nvidia.NVreg_TemporaryFilePath=/var/tmp`。
> - 所选文件系统需要支持未命名的临时文件（例如 ext4 或 XFS），并且有足够的容量来存储显存分配（即至少比所有 NVIDIA GPU 的内存容量总和多 5%）。使用命令 `nvidia-smi --query-gpu=memory.total --format=csv,noheader,nounits` 列出系统中所有 GPU 的内存容量。
> - 虽然 `nvidia-resume.service` 被 NVIDIA 标记为必需，但它可以是可选的，因为它的功能也由 [systemd-sleep(8)](https://man.archlinux.org/man/systemd-sleep.8) 钩子（`/usr/lib/systemd/system-sleep/nvidia`）提供，并且后者会自动调用。请注意，[使用 Wayland 的 GDM](https://gitlab.gnome.org/GNOME/gdm/-/issues/784) 明确要求启用 `nvidia-resume.service`。

驱动持久化

NVIDIA 提供了一个可以在启动时选择运行的守护进程。在标准的单 GPU X 桌面环境中，持久化守护进程并不是必需的，实际上可能会引发一些问题 。更多详情请参阅 NVIDIA 文档中的[驱动持久化](https://docs.nvidia.com/deploy/driver-persistence/index.html#persistence-daemon)部分。

要在启动时启动持久化守护进程，请启用 `nvidia-persistenced.service`。有关手动使用的信息，请参阅[上游文档](https://docs.nvidia.com/deploy/driver-persistence/index.html#usage)。

Forcing YCbCr with 4:2:0 subsampling

If you are facing [limitations of older output standards](https://en.wikipedia.org/wiki/HDMI#Refresh_frequency_limits_for_standard_video) that can still be mitigated by using YUV 4:2:0, the NVIDIA driver has an undocumented X11 option to enforce that:

Option "ForceYUV420" "True"

This will allow higher resolutions or refresh rates but have detrimental impact on the image quality.

2.16 Bumblebee

难度 高级 | **篇幅** 28.8 KB | **所属** 图形

本节将讲到

- Bumblebee: Linux上的 Optimus
- 安装
- 用法
- 配置
- 如 Windows 一般在集显和独显间切换

引自 Bumblebee FAQ:

"*Bumblebee 致力于使 NVIDIA Optimus 在 GNU/Linux 系统上可用，实现两块不同的供电配置的显卡同时插入使用，共享同一个 framebuffer。*"

Bumblebee: Linux上的 Optimus

Optimus 技术是不依赖于硬件复杂结构的 *交火显卡* 实现。独立显卡按需渲染，并传输给集成显卡，集成显卡则负责显示功能。当笔记本通过电池供电时，独立显卡将关闭，以延长电池寿命。在使用 Intel 集成显卡和 NVIDIA 独立显卡的台式机上也能使用这项技术。

Bumblebee 通过软件来实现它的功能，包括两个部分：

- 利用独立显卡渲染程序，并通过集成显卡将图像显示在屏幕上。这是利用 VirtualGL 或 primus （见后面小节）实现的，相当于连接到了一个供独立显卡使用的 X 服务器。
- 独立显卡空闲的时候会被禁用。（参见 #电源管理）

Bumblebee 试图模拟 Optimus 技术的行为；当需要的时候，使用独立显卡进行渲染，不使用的时候则关闭。当前的版本仅支持按需渲染，高负荷程序自动调用独立显卡的功能仍然在开发之中。

安装

安装 Bumblebee 之前，检查你的 BIOS 并尽可能激活 Optimus (老式电脑称之为"可切换显卡"，BIOS有可能没有提供此项设置)。如果 "Optimus" 和 "switchable" 都没有在BIOS里，就保证两种GPU都已启用并且集成显卡是主要显示设备。显示应该连接在主板上的集成显卡，而不是独立显卡。如果集成显卡之前被禁用而安装了独立显卡的驱动，那就删除 `/etc/X11/xorg.conf` 或者有关独立显卡的 `/etc/X11/xorg.conf.d` 中的文件。

安装:

- bumblebee包 - 提供守护进程以及程序的主要安装包。
- mesa包 - 开源的 **OpenGL** 标准实现。
- 对于合适的NVIDIA驱动，参看NVIDIA#安装。
- xf86-video-intel包 - Intel 驱动（可选）。

对于32位程序 (必须启用Multilib）在64位机器上的支持，安装:

- lib32-virtualgl包 - 为32位应用提供的渲染/显示桥。
- lib32-nvidia-utils包 或者 lib32-nvidia-340xx-utilsAUR（和64位对应）。

要使用 Bumblebee，请确保添加你的用户到 `bumblebee` 组：

```
# gpasswd -a user bumblebee
```

并启用 `bumblebeed.service`。重启系统并参考#用法。

用法

测试

安装 mesa-utils包 并使用 `glxgears` 测试 Bumblebee 是否工作：

```
$ optirun glxgears -info
```

如果失败，尝试下列命令:

- 64位系统:

```
$ optirun glxspheres64
```

- 32位系统:

```
$ optirun glxspheres32
```

如果一个内有动画的窗口出现，那么 Optimus 和 Bumblebee 正在工作。

注意：如果 `glxgears` 失败但 `glxspheres*XX*` 有效，替换所有 "`glxgears`" 为 "`glxspheres*XX*`"。

一般用法

```
$ optirun [options] application [application-parameters]
```

例如，用 Optimus 启动 Windows 程序:

```
$ optirun wine application.exe
```

另外，用 Optimus 打开 NVIDIA 设置面板:

```
$ optirun -b none nvidia-settings -c :8
```

注意：一个打了补丁的 nvdockAUR 版本可用： nvdock-bumblebeeAUR

更多 `optirun` 的选项参见手册页:

```
$ man optirun
```

配置

你可以按需配置 Bumblebee 的行为，可以通过 `/etc/bumblebee/bumblebee.conf` 来调节诸如优化，电源管理，以及其他任务。

速度优化

使用VirtualGL作为桥接

Bumblebee 使用你的 Optimus NVIDIA 显卡来渲染一个配置了 VirtualGL 的不可见的 X 服务器，并且将结果传输到你当前的 X 服务器上。传输之前将压缩侦，这可以节省带宽并且能够用于加速 bumblebee 的优化。

要为单个应用程序指定不同的压缩方法：

```
$ optirun -c compress-method application
```

压缩方法会影响 GPU性能和GPU使用，压缩方法会最大限度的使用 CPU,并且尽可能少的使用 GPU；非压缩的方法最大限度的使用 GPU，而尽可能少的使用 CPU。

压缩方法

: - `jpeg`

- `rgb`
- `yuv`

非压缩方法

: - `proxy`

- `xv`

以下是用 Asus N550JV 笔记本和测试程序 unigine-heavenAUR的性能测试结果:

Command	FPS	Score	Min FPS	Max FPS
optirun unigine-heaven	25.0	630	16.4	36.1
optirun -c jpeg unigine-heaven	24.2	610	9.5	36.8
optirun -c rgb unigine-heaven	25.1	632	16.6	35.5
optirun -c yuv unigine-heaven	24.9	626	16.5	35.8
optirun -c proxy unigine-heaven	25.0	629	16.0	36.1
optirun -c xv unigine-heaven	22.9	577	15.4	32.2

注意：使用 `jpeg` 压缩方法时会有延迟。

要为所有应用程序使用一个标准的压缩方法，
在 `/etc/bumblebee/bumblebee.conf` 里把 `VGLTransport` 设为 `*compress-method*`:

```
/etc/bumblebee/bumblebee.conf
[...]
[optirun]
VGLTransport=proxy
[...]
```

你也可考虑 VirtualGL 从你的显卡读回像素的方式。设置 `VGL_READBACK` 环境变量为 `pbo` 应该能提升性能。比较以下两者:

```
# PBO should be faster.
VGL_READBACK=pbo optirun glxgears
# The default value is sync.
VGL_READBACK=sync optirun glxgears
```

注意：调整CPU频率将直接影响渲染性能

Primusrun

注意：因为合成会损害性能，所以不建议在合成窗口管理器（compositing WM）工作时使用primus。参照 #Primus issues under compositing window managers\[损坏的链接：无效的章节\]。

`primusrun` (来自 primus包) 正成为默认选项，因为其耗电量更低并且有时可以提供比 `optirun/virtualgl`更好的性能。它可以独立运行，但不能接受 `optirun` 的选项。将 `primus` 设置为 `optirun` 的桥接可提供更过的灵活性。

对于64位系统上32位程序的支持，安装 lib32-primus包（须启用multilib）。

用法（独立运行）：

```
$ primusrun glxgears
```

`optirun` 的桥接：

默认的配置将 `virtualgl` 作为桥接。在命令行上将其覆盖：

```
$ optirun -b primus glxgears
```

或者，在 `/etc/bumblebee/bumblebee.conf` 中设置 `Bridge=primus` 使其永久生效。

提示：如果要禁用 `VSYNC` 请参考 #Primusrun mouse delay (disable VSYNC))\[损坏的链接：无效的章节\]。这也可以取消鼠标延迟并轻微提高性能。

电源管理

电源管理的目的是为了自动关闭 bumblebee 不再使用的 NVIDIA 显卡。 如果已安装 bbswitch包 ，Bumblebee 守护进程启动时会自动检测到，不需要特别设置。

使用bbswitch时默认的 NVIDIA 电源状态

bbswitch的默认行为是保持显卡的电源状态。 `bumblebeed` 启动时并不禁用显卡，所以你没有bumblebeed时使用bbswitch需要以下内容。

根据需要设置 `load_state` 和 `unload_state` 模块选项 (见 bbswitch documentation).

```
/etc/modprobe.d/bbswitch.conf
options bbswitch load_state=0 unload_state=1
```

关机时启用NVIDIA显卡

如果上次关机时关闭了NVIDIA显卡的电源，NVIDIA显卡可能会在下一次启动时初始化异常。一个解决的办法是在 `/etc/bumblebee/bumblebee.conf` 里设置 `TurnCardOffAtExit=false`， 然而这么做会导致每次停止bumblebee守护进程时启用NVIDIA显卡，就算是手动停止也是一样。为确保NVIDIA显卡总是在关机时启用，添加如下 systemd 服务(如果使用 bbswitch包 的话):

```
/etc/systemd/system/nvidia-enable.service
[Unit]
Description=Enable NVIDIA card
DefaultDependencies=no

[Service]
Type=oneshot
ExecStart=/bin/sh -c 'echo ON > /proc/acpi/bbswitch'

[Install]
WantedBy=shutdown.target
```

然后以root权限运行 `systemctl enable nvidia-enable.service` 以启用服务。

多显示器

显示器连接在 Intel 显卡上

如果 (DisplayPort/HDMI/VGA) 接口连接到 Intel 显卡上，你可以通过 xorg.conf 设置多个显示器。设置他们使用 Intel 显卡，同时 Bumblebee 仍然可以使用 NVIDIA 显

卡。下面的配置文件示例配置了两个不同的 1080p 的显示器，并且使用了 HDMI 输出。

```
/etc/X11/xorg.conf
Section "Screen"
    Identifier     "Screen0"
    Device         "intelgpu0"
    Monitor        "Monitor0"
    DefaultDepth    24
    Option         "TwinView" "0"
    SubSection "Display"
        Depth          24
        Modes          "1980x1080_60.00"
    EndSubSection
EndSection

Section "Screen"
    Identifier     "Screen1"
    Device         "intelgpu1"
    Monitor        "Monitor1"
    DefaultDepth   24
    Option         "TwinView" "0"
    SubSection "Display"
        Depth          24
        Modes          "1980x1080_60.00"
    EndSubSection
EndSection

Section "Monitor"
    Identifier     "Monitor0"
    Option         "Enable" "true"
EndSection

Section "Monitor"
    Identifier     "Monitor1"
    Option         "Enable" "true"
EndSection

Section "Device"
    Identifier     "intelgpu0"
    Driver         "intel"
    Option         "XvMC" "true"
    Option         "UseEvents" "true"
    Option         "AccelMethod" "UXA"
    BusID          "PCI:0:2:0"
EndSection

Section "Device"
    Identifier     "intelgpu1"
    Driver         "intel"
    Option         "XvMC" "true"
    Option         "UseEvents" "true"
    Option         "AccelMethod" "UXA"
    BusID          "PCI:0:2:0"
EndSection

Section "Device"
    Identifier "nvidiagpu1"
    Driver "nvidia"
    BusID "PCI:0:1:0"
EndSection
```

你可能需要为Intel和NVIDIA显卡调整 BusID 字段的值:

```
$ lspci | grep VGA
00:02.0 VGA compatible controller: Intel Corporation 2nd Generation Core Processor
Family Integrated Graphics Controller (rev 09)
```

BusID 值为 0:2:0

显示器连接在 NVIDIA 显卡上

在一些笔记本上，数字视频输出(HDMI 或 DisplayPort) 是硬线连接到NVIDIA芯片上的。如果你想同时使用所有的显示器，你需要开两个 X Server.第一个使用Intel驱动用于笔记本面板和连接到VGA上的显示，第二个由optirun启用NVIDIA显卡用于数字显示。

目前网上有一些有关这个的设置教程。其中一个可在Bumblebee的 wiki 页面找到，另一个详见下文。

xf86-video-intel-virtual-crtc 和 hybrid-screenclone

本方法使用了一个打了补丁来扩展为含有一个虚拟显示器的Intel驱动，而hybrid-screenclone程序用于把虚拟显示器的内容复制到通过optirun运行于NVIDIA显卡的第二个X Server上的显示器上。有幸发现 Triple-head monitors on a Thinkpad T520 它是这一切在Ubuntu系统上如何完成的详细解说。

精简起见，下文DP指代 Digital Output (DisplayPort).如果笔记本是HDMI接口的话教程也是一样。

- 设置系统为只使用 NVIDIA 显卡，测试 DP/显示器组合，并生成 xorg.nvidia.conf.这步并非必须但推荐在支持设置为nvidia-only的BIOS上执行。为此，首先卸载 bumblebee 包，并只安装 NVIDIA 驱动。然后重启，进入BIOS设置为NVIDIA-only.重启进入Arch时，把显示器连接到DP并使用 startx 来检测这一切是否正常工作。使用 Xorg -configure 来为你的NVIDIA显卡生成 xorg.conf 文件。这在向下进一步深入时会派上用场。
- 重新安装 bumlbebee 和 bbswitch,重启并在BIOS里设置系统 Gfx 值回到 Hybrid.
- 安装 xf86-video-intel-virtual-crtcAUR\[损坏的链接：package not found\],并以它替换掉 xf86-video-intel.
- 安装 screenclone-gitAUR\[损坏的链接：package not found\]
- 改变以下 bumblebee.conf 设置:

```
/etc/bumblebee/bumblebee.conf
KeepUnusedXServer=true
Driver=nvidia
```

注意：把 PMMethod 保留不变为 "bumblebee".这与上面链接里教程的做法相反，但在Arch里需这么做以使 bbswitch 模块自动加载。

- 复制在第一步里生成的 xorg.conf 到 `/etc/X11` (例如 `/etc/X11/xorg.nvidia.conf`).在 `bumblebee.conf` 的\[driver-nvidia\]部分，修改 `XorgConfFile` 为它的路径。
- 测试你的 `/etc/X11/xorg.nvidia.conf` 是否有效 `startx -- -config /etc/X11/xorg.nvidia.conf`
- 为了你的DP显示器在虚拟显示器上以正常分辨率出现，你需要编辑 `/etc/X11/xorg.nvidia.conf` 里的Monitor section.由于这是附加步骤，你仍需要测试你自动生成的 xorg.conf.当发现 xrandr 显示的虚拟显示器分辨率不对后，再回来参考教程里的这条步骤。
- 首先要生成 Modeline.你需要工具 amlc,它用于输入一些基本参数后自动生成 Modeline.

: : 例子: 24" 1920x1080 Monitor : 以 `amlc -c` 参数启动工具

```
Monitor Identifier: Samsung 2494
Aspect Ratio: 2
physical size[cm]: 60
Ideal refresh rate, in Hz: 60
min HSync, kHz: 40
max HSync, kHz: 90
min VSync, Hz: 50
max VSync, Hz: 70
max pixel Clock, MHz: 400
```

这就是 `amlc` 生成的 Monitor Section:

```
Section "Monitor"
    Identifier     "Samsung 2494"
    ModelName      "Generated by Another Modeline Calculator"
    HorizSync      40-90
    VertRefresh    50-70
    DisplaySize    532 299  # Aspect ratio 1.778:1
    # Custom modes
    Modeline "1920x1080" 174.83 1920 2056 2248 2536 1080 1081 1084 1149             #
174.83 MHz,  68.94 kHz,  60.00 Hz
EndSection  # Samsung 2494
```

在 `xorg.nvidia.conf` 里加入这个 Monitor Section.你还可以编辑你的文件使其包含 ServerLayout, Monitor, Device 和 Screen 部分。以下是我的，仅供参考:

```
/etc/X11/xorg.nvidia.conf
Section "ServerLayout"
        Identifier     "X.org Nvidia DP"
        Screen      0  "Screen0" 0 0
        InputDevice    "Mouse0" "CorePointer"
        InputDevice    "Keyboard0" "CoreKeyboard"
EndSection

Section "Monitor"
    Identifier     "Samsung 2494"
    ModelName      "Generated by Another Modeline Calculator"
    HorizSync      40-90
    VertRefresh    50-70
    DisplaySize    532 299  # Aspect ratio 1.778:1
    # Custom modes
    Modeline "1920x1080" 174.83 1920 2056 2248 2536 1080 1081 1084 1149             #
174.83 MHz,  68.94 kHz,  60.00 Hz
EndSection  # Samsung 2494
```

```
Section "Device"
        Identifier  "DiscreteNvidia"
        Driver      "nvidia"
        BusID       "PCI:1:0:0"
EndSection

Section "Screen"
        Identifier "Screen0"
        Device     "DiscreteNvidia"
        Monitor    "Samsung 2494"
        SubSection "Display"
                Viewport   0 0
                Depth     24
        EndSubSection
EndSection
```

- 也要把额外显示器插上之后再 startx.查看 `/var/log/Xorg.0.log`.确保你的VGA显示器以正确的模式被检测出来。你也要看看虚拟输出出现的模式。
- 运行 `xrandr` 然后三个显示器以及它们支持的模式都会被列出来。
- 如果对于你的VIRTUAL显示器没有符合你原生分辨率的Modelines输出, 记下输出的名称。对我来说是 `VIRTUAL1`.然后再看看 Xorg.0.log 文件。你应该会看到一条消息: "Output VIRTUAL1 has no monitor section".我们需要在 `/etc/X11/xorg.conf.d` 里放入含有合适 Monitor Section 的文件，稍后退出并重启X.

```
/etc/X11/xorg.conf.d/20-monitor_samsung.conf
Section "Monitor"
    Identifier     "VIRTUAL1"
    ModelName      "Generated by Another Modeline Calculator"
    HorizSync      40-90
    VertRefresh    50-70
    DisplaySize    532 299  # Aspect ratio 1.778:1
    # Custom modes
    Modeline "1920x1080" 174.83 1920 2056 2248 2536 1080 1081 1084 1149             #
174.83 MHz,  68.94 kHz,  60.00 Hz
EndSection  # Samsung 2494
```

- 为开启NVIDIA显卡，运行: `sudo tee /proc/acpi/bbswitch <<< ON`
- 为 DisplayPort 显示器开启另一个X Server: `sudo optirun true`
- 检查位于 `/var/log/Xorg.8.log` 的第二个 X Server 的日志。
- 运行 xrandr 以把 VIRTUAL 显示器设置成正确的大小和位置，例如: `xrandr --output VGA1 --auto --rotate normal --pos 0x0 --output VIRTUAL1 --mode 1920x1080 --right-of VGA1 --output LVDS1 --auto --rotate normal --right-of VIRTUAL1`
- 请注意在 xrandr 输出列表中的 VIRTUAL 显示器的位置。计数从零开始，也就是说，如果它给出了第三个显示器，你会指定 `-x 2` 作为参数传递给 `screenclone`（注：这可能并不总是正确的。如果你看到内部的笔记本电脑显示屏克隆在监视器上，只管尝试 `-x 2`）。
- 克隆VIRTUAL显示器的内容到由Bumblebee创建的，经由NVIDIA的芯片连接到所述的DisplayPort显示器的X服务器:

`:screenclone -d :8 -x 2`

这就行了，三个显示器都应该配置好并正常工作。

如 Windows 一般在集显和独显间切换

在 Windows 里，Optimus 的工作方式是NVIDIA有一个请求 Optimus 的程序的白名单，并且你可以按需添加。当你启动程序时，会自动选择使用哪一张显卡。

为了在Linux里模仿这一行为，你可以使用 libgl-switcheroo-gitAUR\[损坏的链接：package not found\].安装后，往 .xprofile 里添加:

```
~/.xprofile
mkdir -p /tmp/libgl-switcheroo-$USER/fs
gtkglswitch &
libgl-switcheroo /tmp/libgl-switcheroo-$USER/fs &
```

为此，你需要往你希望启动程序的shell里添加 (我一般就加到 .xprofile 里)

```
export LD_LIBRARY_PATH=/tmp/libgl-switcheroo-$USER/fs/\
$LIB${LD_LIBRARY_PATH+:}$LD_LIBRARY_PATH
```

全部完成后，你从该shell运行的每一个程序都会弹出一个GTK+窗口询问该使用哪一张卡(你也可以往配置文件的白名单里添加程序)。配置文件位于 `$XDG_CONFIG_HOME/libgl-switcheroo.conf`, 通常是 `~/.config/libgl-switcheroo.conf`

注意：此工具通过创建FUSE文件系统，然后将其添加到动态库搜索路径来生效，启动软件时，可能导致速度慢甚至段错误。

疑难问题

注意：报告 Bug 的位置在 Bumblebee-Project，Bumblebee 的 Wiki 中描述。

\[VGL\] ERROR: Could not open display :8

一个关于部分wine程序的已知问题是它会fork并杀死父进程而不是跟踪它的动态(例如免费网络游戏 "Runes of Magic")

这是一个关于 VirtualGL 的已知问题。自从bumblebee 3.1,只要你安装了它就可以用 Primus 作为渲染桥接:

```
$ optirun -b primus wine windows program.exe
```

如果没用的话，一个替代方案如下:

```
$ optirun bash
$ optirun wine windows program.exe
```

如果使用NVIDIA驱动的话，一个该问题的解决方案是修改 `/etc/bumblebee/xorg.conf.nvidia` 把 Option `ConnectedMonitor` 改为 `CRT-0`.

Xlib: extension "GLX" missing on display ":0.0"

从官网下载安装的NVIDIA驱动不会工作。

1\. 卸载驱动：

```
# ./NVIDIA-Linux-*.run --uninstall
```

2\. 删除生成的配置文件：

```
# rm /etc/X11/xorg.conf
```

3\. 重新从源里安装： #安装

\[ERROR\]Cannot access secondary GPU: No devices detected

某些情况下，运行 `optirun` 会返回:

```
[ERROR]Cannot access secondary GPU - error: [XORG] (EE) No devices detected.
[ERROR]Aborting because fallback start is disabled.
```

这种情况下，你需要把 `/etc/X11/xorg.conf.d/20-intel.conf` 移动到别的地方， restart bumblebeed 守护进程，这样应该有效。如果你要改变一些 Intel 模块的特性，把 `/etc/X11/xorg.conf.d/20-intel.conf` 合并到 `/etc/X11/xorg.conf` 中。

也需要把 `/etc/X11/xorg.conf.d/10-monitor.conf` 里的driver行注释掉。

如果你正在用 `nouveau` 驱动你应该试着换成 `nvidia` 驱动。

你需要把 NVIDIA 显卡定位到别的地方 (例如文件 `/etc/X11/xorg.conf.d`),使用根据 `lspci` 的输出确定的 `BusID`:

```
Section "Device"
    Identifier "nvidiagpu1"
    Driver "nvidia"
    BusID "PCI:0:1:0"
EndSection
```

NVIDIA(0): Failed to assign any connected display devices to X screen 0

如果终端输出如下:

```
[ERROR]Cannot access secondary GPU - error: [XORG] (EE) NVIDIA(0): Failed to assign any connected display devices to X screen 0
[ERROR]Aborting because fallback start is disabled.
```

你要修改 `/etc/bumblebee/xorg.conf.nvidia` 里的这行:

```
Option "ConnectedMonitor" "DFP"
```

为:

```
Option "ConnectedMonitor" "CRT"
```

systemd-logind: failed to get session: PID XXX does not belong to any known session

如果终端输出如下 (*PID*会有所不同):

```
[ERROR]Cannot access secondary GPU - error: [XORG] (EE) systemd-logind: failed to get session: PID 753 does not belong to any known session
```

```
[ERROR]Aborting because fallback start is disabled.
```
在 /etc/mkinitcpio.conf 里改变 MODULES 为:

```
MODULES="i915 bbswitch"
```
或:

```
MODULES="i915 nouveau bbswitch"
```
然后运行:

```
# mkinitcpio -p linux
```
每次内核更新之后你都要执行一次这个命令。

另外，如下内核参数需要加到 boot loader配置中。

Failed to initialize the NVIDIA GPU at PCI:1:0:0 (GPU fallen off the bus / RmInitAdapter failed!)

把 `rcutree.rcu_idle_gp_delay=1` 加到boot loader 配置的内核参数中。 (参考设置另见BBS post)

Could not load GPU driver

如果终端输出如下:

```
[ERROR]Cannot access secondary GPU - error: Could not load GPU driver
```
并且尝试加载 nvidia 模块时得到:

```
modprobe nvidia
modprobe: ERROR: could not insert 'nvidia': Exec format error
```
你需要针对你的内核手动编译 nvidia 包，例如 nvidia-dkms包 或从 ABS nvidia包 编译。

NOUVEAU(0): \[drm\] failed to set drm interface version

考虑切换到 nvidia 官方驱动。在此的评论说 nouveau 驱动在某些显卡和bumblebee上有问题。

\[ERROR\]Cannot access secondary GPU - error: X did not start properly

在 `/etc/bumblebee/xorg.conf.nvidia` 把 `"AutoAddDevices"` 设置为 `"true"`（看这里）：

```
Section "ServerLayout"
    Identifier  "Layout0"
    Option      "AutoAddDevices" "true"
    Option      "AutoAddGPU" "false"
EndSection
```
/dev/dri/card0: failed to set DRM interface version 1.4: Permission denied

这可以通过在 `/etc/bumblebee/xorg.conf.nvidia` 里添加几行解决(见这里):

```
Section "Screen"
    Identifier "Default Screen"
    Device "DiscreteNvidia"
EndSection
```

ERROR: ld.so: object 'libdlfaker.so' from LD_PRELOAD cannot be preloaded: ignored

你也许是想在64位系统上用bumblebee运行32位程序。见 "Note" box Installation\[损坏的链接：无效的章节\].

Fatal IO error 11 (Resource temporarily unavailable) on X server

把 `/etc/bumblebee/bumblebee.conf` 里的 `KeepUnusedXServer` 从 `false` 改为 `true`.这是因为你的程序fork到了后台并且 bumblebee 对此一无所知。

视频撕裂

视频撕裂是 Bumblebee 上一个常见的问题，要修复这个问题，你需要启用 vsync。默认情况下，Intel 显卡已启用此设置，但是还是检查一下 Xorg 的日志，要检查 nvidia 是否启用了此设置，运行:

```
$ optirun nvidia-settings -c :8
```

`X Server XVideo Settings -> Sync to VBlank` 以及 `OpenGL Settings -> Sync to VBlank` 应该都是已经启用状态。 Intel 显卡通常有比较少的撕裂，所以应该作为视频回放设备。特别是使用 VA-API 编码视频的时候（比如：`mplayer-vaapi` 以及 `-vsync` 参数）。

参考Intel\[损坏的链接：无效的章节\]了解如何修复 Intel 显卡的视频撕裂。

如果仍然无效，尝试从桌面环境禁用 compositing。同时可尝试禁用 triple buffering.

Bumblebee cannot connect to socket

你可能会得到这样的输出:

```
$ optirun glxspheres64
```

或 (32位):

```
$ optirun glxspheres32
[ 1648.179533] [ERROR]You've no permission to communicate with the Bumblebee daemon. Try adding yourself to the 'bumblebee' group
[ 1648.179628] [ERROR]Could not connect to bumblebee daemon - is it running?
```

如果你已在 `bumblebee` 群组中 (`$ groups | grep bumblebee`),尝试 removing the socket `/var/run/bumblebeed.socket`.

Running X.org from console after login (rootless X.org)

见 Xorg#Rootless Xorg (v1.16))\[损坏的链接：无效的章节\].

Primusrun 鼠标延迟/禁用 VSYNC

对于 `primusrun`, `VSYNC` 默认启用，因此会导致鼠标输入延迟甚至轻微降低性能。不用 `VSYNC` 来测试 `primusrun`:

```
$ vblank_mode=0 primusrun glxgears
```

如果你想使用它替代 `primusrun`,新建文件:

```
/usr/bin/optiprime
#!/bin/sh
vblank_mode=0 primusrun "$@"
```

让它可执行:

```
# chmod +x /usr/bin/optiprime
```

用法:

```
$ optiprime glxgears
```

总之它不会有重大性能提升，但会解决上述的鼠标延迟。

Command	FPS	Score	Min FPS	Max FPS
optiprime unigine-heaven	31.5	793	22.3	54.8
primusrun unigine-heaven	31.4	792	18.7	54.2

Tested with Asus N550JV laptop and benchmark app unigine-heavenAUR.

另见

- Bumblebee project repository\[失效链接 2022-09-17 ⓘ\]
- Bumblebee project wiki
- Bumblebee project bbswitch repository

2.17 AMDGPU

难度 高级 | **篇幅** 33.3 KB | **所属** 图形

本节将讲到

- 选择正确的驱动
- 安装
- 加载
- Xorg 配置
- 特性

AMDGPU 是 Graphics Core Next 系列 AMD Radeon 图形卡的开源图形驱动程序。

选择正确的驱动

请根据您的显卡，参考Xorg#AMD选择合适的显卡驱动。目前驱动支持从 Southern Islands（GCN 1，发布于2012年） 到最新的显卡，而AMD没有计划支持 GCN 架构之前的显卡。如果您使用的是 AMD 不支持的显卡，可以选择使用开源的 ATI 驱动。

安装

安装 mesa包 软件包，它提供用于3D加速的DRI驱动程序和用于加速视频解码的 VA-API/VDPAU 驱动。

- 对于32位程序，请从 multilib 仓库中安装 lib32-mesa包 软件包以获得支持。
- 对于 DDX 驱动支持(可提供对 Xorg 的 2D 加速)，可以安装 xf86-video-amdgpu包 软件包。
- 对于 Vulkan 支持：
- **安装 vulkan-radeon包** 驱动：尽管*vulkan-driver* 提供的第一个选择不是它（因为可选按照字母顺序排序）。
- 对应安装的原生驱动，可选择性安装 lib32-vulkan-radeon包 软件包来获得32位应用程序支持。

实验版本

对于某些用户来说使用 mesa 的上游实验构建可能是值得的。

安装 mesa-gitAUR 软件包，它提供用于3D加速的DRI驱动程序。

- 对于32位程序支持，请从 AUR 或者 *mesa-git* 仓库安装 lib32-mesa-gitAUR
- 对于 DDX 驱动支持(可提供对 Xorg 的2D加速)，安装 xf86-video-amdgpu-gitAUR软件包。
- 如果您使用 *mesa-git* 仓库并且要启用 Vulkan 支持，请安装 amdonly-gaming-vulkan-radeon-gitAUR 软件包。可选安装 lib32-amdonly-gaming-vulkan-radeon-gitAUR来支持32位应用程序。如果您用的是来自 AUR 的 mesa-gitAUR 软件包，则不需要进行该操作。

提示：如果您不想用 mesa-gitAUR 编译安装驱动（很耗时），您也可以使用 mesa-git 这个非官方用户仓库

启用 Southern Islands (SI) 和 Sea Islands (CIK) 支持

官方支持的内核启用了 AMDGPU 驱动，支持 Southern Islands（GCN 1，发布于2012年）和 Sea Islands （GCN 2，发布于2013年）显卡。`amdgpu` 内核驱动程序需要在radeon之前加载。可以通过运行`lspci -k`来检查加载了哪个内核驱动程序。示例输出如下：

```
$ lspci -k -d ::03xx
01:00.0 VGA compatible controller: Advanced Micro Devices, Inc. [AMD/ATI] Curacao PRO [Radeon R7 370 / R9 270/370 OEM]
	Subsystem: Gigabyte Technology Co., Ltd Device 226c
	Kernel driver in use: amdgpu
	Kernel modules: radeon, amdgpu
```

如果 `amdgpu` 驱动没有被加载使用，可以跟着下面的内容进行处理。

加载 amdgpu 内核驱动

`amdgpu` 和 `radeon` 两个模块都需要设置的内核参数是 `cik_support=` 和 `si_support=`。

它们需要在内核参数或 *modprobe* 配置文件中进行设置，具体参数取决于显卡的 GCN 版本。

如果不确定您拥有的是哪个版本的显卡，那么也可以同时使用这两个参数。

提示：dmesg可能会指示应该使用的正确内核参数: `[..] amdgpu 0000:01:00.0: Use radeon.cik_support=0 amdgpu.cik_support=1 to override.`

在内核命令行中设置内核模块参数

设置以下内核参数之一:

- Southern Islands (SI): `radeon.si_support=0 amdgpu.si_support=1`
- Sea Islands (CIK): `radeon.cik_support=0 amdgpu.cik_support=1`

指定正确的模块加载顺序

当内核启用 AMDGPU 对 SI/CIK 显卡的支持时，radeon驱动可能会在`amdgpu`驱动之前加载。

确保`amdgpu`在Mkinitcpio#MODULES数组中被设置为第一个模块，例如 `MODULES=(amdgpu radeon)`。

设置内核参数

对于 Southern Islands (SI) 使用 `si_support=1` 内核参数，对于 Sea Islands (CIK) 使用 `cik_support=1`:

```
/etc/modprobe.d/amdgpu.conf
options amdgpu si_support=1
options amdgpu cik_support=1
/etc/modprobe.d/radeon.conf
options radeon si_support=0
options radeon cik_support=0
```

确保 `modconf` 添加到 `HOOKS` 数组中在 `/etc/mkinitcpio.conf` 和 Mkinitcpio#创建和启用镜像。

编译支持 amdgpu 驱动的内核

在构建或者编译内核的时候，需要在配置中设置 `CONFIG_DRM_AMDGPU_SI=Y` 和/或者 `CONFIG_DRM_AMDGPU_CIK=Y`。

在系统启动时候完全禁用 radeon 内核驱动模块

内核可能仍会根据涉及的具体图形芯片探测并加载 radeon 模块，但在确认 amdgpu 正常工作后，radeon 模块就不需要加载了。 在每个步骤之间重新启动以确认它在移动到下一步之前有效：

1. 在内核命令行添加模块参数以确保 `amdgpu` 按预期工作，

2. 使用 `MODULES=(amdgpu)` mkinitcpio 方法但**不要将** `radeon` 添加到配置中,
3. 在登录桌面后测试 `modprobe -r radeon` 命令确保能够在卸载 `radeon` 内核模块后不影响系统图形界面的运行，
4. 将 `radeon` 模块列入黑名单，以免在第二阶段启动期间被内核探测：

```
/etc/modprobe.d/radeon.conf
blacklist radeon
```

当上面的配置生效之后，命令 `lsmod` 和 `dmesg` 的输出现在应该只会显示 amdgpu 驱动模块被加载的消息，不会存在 radeon 驱动模块的消息。系统中的 `/sys/module/radeon` 目录应该也不存在。如果存在，则说明配置没有生效，radeon 驱动模块仍然被系统加载了。

ACO编译器

ACO编译器是一个由 Valve Corporation开发的开源着色器（shader）编译器，可直接与 LLVM编译器 以及 Windows 10 竞争。与 LLVM 相比，它提供更短的编译时间，并且在玩游戏时表现更好。

一些基准性能测试可以在 GitHub 和 Phoronix (1) (2) (3)中看到。

从 mesa包20.2 版本开始, ACO 是默认着色器编译器。

加载

应该会在系统启动时自动加载。

如果没有自动加载:

- 如果需要的话，确保 #开启 Southern Islands (SI) and Sea Islands (CIK) 支持_and_Sea_Islands_(CIK)_支持)。
- 确保你已经安装最新的 linux-firmware包 软件包。
- 确保你没有将 `nomodeset` 或 `vga=` 作为内核参数，因为`amdgpu` 需要内核级显示模式设置。
- 确保你没有禁用 `amdgpu` 通过使用内核模块黑名单。

尽管驱动有可能自动加载，但是加载可能会发生在 X 服务器需要它的时间点之后。这种情况下请参考内核级显示模式设置#KMS 早启动。

Xorg 配置

Xorg 会自动加载驱动程序，并且使用显示器的 EDID 来设置原始分辨率。只有在调整驱动程序时才需要配置。

如果你想手动配置，新建 `/etc/X11/xorg.conf.d/20-amdgpu.conf` 文件，并在其中添加以下内容：

```
/etc/X11/xorg.conf.d/20-amdgpu.conf
Section "OutputClass"
     Identifier "AMD"
     MatchDriver "amdgpu"
     Driver "amdgpu"
 EndSection
```

使用此部分配置，您可以启用功能并调整驱动程序设置，请在设置驱动程序选项之前先查看 amdgpu(4)。

无撕裂渲染

*TearFree*使用硬件翻页机制控制撕裂预防。默认情况下，如果旋转画面输出、使用RandR变换和RandR 1.4 从属输出时，TearFree会被启用，其他情况下默认关闭。你可以通过指定`true`或`false`来控制此配置保持启用/关闭：

```
Option "TearFree" "true"
```

DRI 级别

DRI 设置要启用的最大 DRI 级别。DRI2 的有效值为“2”，DRI3 的有效值为“3”。如果 Xorg 版本 >= 1.18.3，则 DRI3 的默认值为“3”，否则使用 DRI2：

```
Option "DRI" "3"
```

可变刷新率

参考可变刷新率。

10-bit 色深

警告：当启用 10 位深度颜色时，许多应用程序可能会出现图形瑕疵或崩溃。尤其是 Steam，它因 X 错误而崩溃。

较新的 AMD 显卡支持 10bpc 颜色，但默认为 24 位颜色，必须明确启用 30 位颜色。如果应用程序也支持，启用它可以减少渐变中的可见条带/伪影。 要检查您的显示器是否支持它，请在您的 Xorg 日志文件中搜索“EDID”（例如 `/var/log/Xorg.0.log` 或 `~/ .local/share/xorg/Xorg.0.log`):

```
[   336.695] (II) AMDGPU(0): EDID for output DisplayPort-0
[   336.695] (II) AMDGPU(0): EDID for output DisplayPort-1
[   336.695] (II) AMDGPU(0): Manufacturer: DEL  Model: a0ec  Serial#: 123456789
[   336.695] (II) AMDGPU(0): Year: 2018  Week: 23
[   336.695] (II) AMDGPU(0): EDID Version: 1.4
[   336.695] (II) AMDGPU(0): Digital Display Input
[   336.695] (II) AMDGPU(0): 10 bits per channel
```

要检查它当前是否启用可以通过搜索“Depth”:

```
[   336.618] (**) AMDGPU(0): Depth 30, (--) framebuffer bpp 32
[   336.618] (II) AMDGPU(0): Pixel depth = 30 bits stored in 4 bytes (32 bpp pixmaps)
```

使用默认配置，log 会显示 depth 为 24，并且 24 位色彩存储在 4 个字节中。

要检查 10 bits色深是否工作，请退出 Xorg（如果它正在运行）并运行 `Xorg -retro` 这将显示黑白网格，然后按 `Ctrl-Alt-F1` 和 { {ic|Ctrl-C}} 退出 X，然后运行 `Xorg -depth 30 -retro`。 如果这些命令能正常执行，那么就说明10位色深生效了。

要通过 `startx` 以 10 位色深启动，请使用 `startx -- -depth 30`。 要永久启用它，请添加到（如果没有指定文件则新建）：

```
/etc/X11/xorg.conf.d/20-amdgpu.conf
Section "Screen"
       Identifier "asdf"
       DefaultDepth 30
EndSection
```

降低输出延迟

如果您想最大程度地减少延迟，您可以禁用翻页（page fliping）和无撕裂（tear free）：

```
/etc/X11/xorg.conf.d/20-amdgpu.conf
Section "OutputClass"
     Identifier "AMD"
     MatchDriver "amdgpu"
     Driver "amdgpu"
     Option "EnablePageFlip" "off"
     Option "TearFree" "false"
EndSection
```

参考游戏#降低_DRI_延迟来降低显示延迟。

注意：设置这些选项可能会导致出现撕裂和短暂的瑕疵。

特性

视频加速

参考硬件视频加速#AMD/ATI.

性能监控

GPU性能监控通常用于检查温度以及 GPU 的 P-states。

命令行界面

- **amdgpu_top** — 显示AMDGPU使用率的工具

: https://github.com/Umio-Yasuno/amdgpu_top || amdgpu_top包

- **nvtop** — AMD、Intel 和 NVIDIA 的 GPU 进程监控

: https://github.com/Syllo/nvtop || nvtop包

- **radeontop** — GPU 利用率查看器，包括总活动百分比和单个块

: https://github.com/clbr/radeontop || radeontop包

图形用户界面

- **amdgpu_top** — 显示AMDGPU使用率的工具

: https://github.com/Umio-Yasuno/amdgpu_top || amdgpu_topAUR

- **AmdGuid** — 完全用Rust编写的一个基本的图形用户界面风扇控制工具

: https://github.com/Eraden/amdgpud || amdguid-wayland-binAUR, amdguid-glow-binAUR

- **TuxClocker** — Qt5监控和超频工具。

: https://github.com/Lurkki14/tuxclocker || tuxclockerAUR

手动

检查GPU的P-states，执行：

```
$ cat /sys/class/drm/card0/device/pp_od_clk_voltage
```

监控GPU，执行：

```
# watch -n 0.5 cat /sys/kernel/debug/dri/0/amdgpu_pm_info
```

检查GPU利用率，执行：

```
$ cat /sys/class/drm/card0/device/gpu_busy_percent
```

检查GPU频率，执行：

```
$ cat /sys/class/drm/card0/device/pp_dpm_sclk
```

检查GPU温度，执行：

```
$ cat /sys/class/drm/card0/device/hwmon/hwmon*/temp1_input
```

检查显存频率，执行：

```
$ cat /sys/class/drm/card0/device/pp_dpm_mclk
```

检查显存占用，执行：

```
$ cat /sys/class/drm/card0/device/mem_info_vram_used
```

检查显存大小，执行：

```
$ cat /sys/class/drm/card0/device/mem_info_vram_total
```

超频

从 Linux 4.17 开始, 只要开启了相关功能，就可以通过 `/sys/class/drm/card0/device/pp_od_clk_voltage` 调整显卡的频率和电压。

启动参数

需要通过附加内核参数 `amdgpu.ppfeaturemask=0xffffffff` 来解锁在 sysfs 中对其的访问限制。

并非所有比特位都有定义，随着时间的推移可能会添加新功能。设置所有 32 位可能会（在将来的更新中）启用不稳定的功能，这些功能可能会导致屏幕闪烁或无法从挂起中恢复等问题。结合默认的 ppfeaturemask 设置 PP_OVERDRIVE_MASK 位 0x4000 应该就足够了。 要为系统计算一个合理的参数，可以执行：

```
$ printf 'amdgpu.ppfeaturemask=0x%x\n'
"$(($(cat /sys/module/amdgpu/parameters/ppfeaturemask) | 0x4000))"
```

手动（默认）

注意：在 sysfs 中，`/sys/class/drm/...` 之类的路径只是符号链接，可能会在重新启动之间发生变化。永久位置可以在 `/sys/devices/` 中找到，例如 `/sys/devices/pci0000:00/0000:00:01.0/0000:01:00.0/`。想要得到可靠的结果可能要相应地调整命令。

要深入了解所有可能的选项，请参阅amdgpu功耗与发热控制内核文档。

可以为最大 P-state 7 设置 GPU 核心频率， 以 Polaris GPU 为例可以执行下面的命令设置为 1209MHz 频率和 900mV 电压：

```
# echo "s 7 1209 900" > /sys/class/drm/card0/device/pp_od_clk_voltage
```

同样的操作可以应用于 VRAM，例如 Polaris 5xx 系列卡上的最大 P-state 2：

```
# echo "m 2 1850 850" > /sys/class/drm/card0/device/pp_od_clk_voltage
```

警告：仔细检查输入的值，因为错误可能会导致硬件损坏！

执行下面的命令让修改生效：

```
# echo "c" > /sys/class/drm/card0/device/pp_od_clk_voltage
```

要检查超频是否成功，可以读取 3D 负载下的时钟和电压：

```
# watch -n 0.5  cat /sys/kernel/debug/dri/0/amdgpu_pm_info
```

可以使用下面的命令重置为默认设置：

```
# echo "r" > /sys/class/drm/card0/device/pp_od_clk_voltage
```

如果要避免深度节能 P-states 引起的闪烁、伪影和卡顿之类的问题，也可以禁止驱动程序切换到某些 P-states。要让显卡强制使用最高的 VRAM P-state，同时允许 GPU 本身以较低的频率运行，首先找到最高的可用P-state，然后设置：

```
# cat /sys/class/drm/card0/device/pp_dpm_mclk
0: 96Mhz *
1: 456Mhz
2: 675Mhz
3: 1000Mhz
# echo "manual" > /sys/class/drm/card0/device/power_dpm_force_performance_level
# echo "3" >  /sys/class/drm/card0/device/pp_dpm_mclk
```

仅允许三个最高的 GPU pstates：

```
# echo "5 6 7" >  /sys/class/drm/card0/device/pp_dpm_sclk
```

执行下面命令可以限制GPU的极限功耗（参数以 50 瓦为例）：

```
# echo 50000000 > /sys/class/drm/card0/device/hwmon/hwmon0/power1_cap
```

注意：上述程序已使用 Polaris RX 560 卡进行测试。不同的 GPU 可能会有不同的行为或错误。

辅助工具

如果不想（或不敢）完全通过手动超频 GPU，社区提供了一些超频工具来帮助超频和监控 AMD 显卡。

命令行工具

- **amdgpu-clocks** — 用于监控和设置 AMD GPU 自定义电源状态的脚本。它还提供了一个 Systemd 服务，可以在系统启动时自动置。

: https://github.com/sibradzic/amdgpu-clocks || amdgpu-clocks-gitAUR

图形界面工具

- **TuxClocker** — Qt5 监控和超频工具。

: https://github.com/Lurkki14/tuxclocker || tuxclockerAUR

- **CoreCtrl** — 一个 GUI 超频工具，具有类似 WattMan 的 UI，支持每个应用程序配置文件。

: https://gitlab.com/corectrl/corectrl || corectrl包

- **LACT** — 一个GTK工具，可显示信息和控制你的AMD GPU。

: https://github.com/ilya-zlobintsev/LACT || lactAUR

开机自动修改

一种办法是使用system的单元，如果希望设置在开机时自动应用修改，可以参考这个 Reddit 帖子来实现开机时自动配置和启用设定。

另一种办法是使用udev规则，例如要设置为低性能等级来延长续航：

```
/etc/udev/rules.d/30-amdgpu-low-power.rules
SUBSYSTEM=="pci", DRIVER=="amdgpu", ATTR{power_dpm_force_performance_level}="low"
```

性能等级

AMDGPU 提供了不同的性能等级，可以使用power_dpm_force_performance_level调节，有下列参数可供选择：

- **auto**: 驱动根据当前情况动态选择最佳电源配置。
- **low**: 时钟频率强制设置为最低功耗状态。
- **high**: 时钟频率强制设置为最高功耗状态。
- **manual**: 用户可以为不同的时钟频率区间手动调整功耗状态（用于#电源配置）。

- **profile_standard**, **profile_min_sclk**, **profile_min_mclk**, **profile_peak**: 时钟频率和功耗门控被关闭，根据用户需求时钟频率可设置为不同配置。该模式推荐用于特定工作负载。

要让AMDGPU设备工作在一个低性能等级，可以执行下面的命令：

```
# echo "low" > /sys/class/drm/card0/device/power_dpm_force_performance_level
```

注意：每次启动系统后应该重新设置性能等级，参见#开机自动修改使修改自动应用。

电源配置

AMDGPU 通过电源配置提供多种优化，最常用的一种是 OpenCL 密集型应用程序的计算模式。下面的命令可以列出所有的可用电源配置：

```
cat /sys/class/drm/card0/device/pp_power_profile_mode
NUM        MODE_NAME     SCLK_UP_HYST   SCLK_DOWN_HYST SCLK_ACTIVE_LEVEL
MCLK_UP_HYST   MCLK_DOWN_HYST MCLK_ACTIVE_LEVEL
  0    BOOTUP_DEFAULT:         -                -                -                -
-                -
  1   3D_FULL_SCREEN:         0              100               30                0
100               10
  2     POWER_SAVING:        10                0               30                -
-                -
  3            VIDEO:         -                -                -               10
16               31
  4               VR:         0               11               50                0
100               10
  5          COMPUTE *:        0                5               30               10
60               25
  6           CUSTOM:         -                -                -                -
-                -
```

注意：命令参数路径中用 `card0` 标识机器中的特定 GPU，如果有多个 GPU，要确保找到正确的路径。

要使用特定的电源配置文件，首先应该启用对它们的手动控制：

```
# echo "manual" > /sys/class/drm/card0/device/power_dpm_force_performance_level
```

然后通过写入与其关联的 NUM 字段来选择电源配置文件，例如下面的命令可以启用计算运行（COMPUTE）：

```
# echo "5" > /sys/class/drm/card0/device/pp_power_profile_mode
```

注意：电源配置改动要在每次启动时重新配置，参考 #开机启动 来自动化该过程

启用 GPU 显示缩放

**本文或本章节可能需要合并到xrandr。**

附注： 不特定于 AMDGPU。（在 Talk:AMDGPU#将 "启用 GPU 显示缩放" 移动到 xrandr 中讨论）

为了避免使用显示器内置的缩放器，而要使用 GPU 自己的缩放器，在不使用显示器的原始分辨率时，可以用下面的命令实现：

```
$ xrandr --output "<output>" --set "scaling mode" "<scaling mode>"
```

"scaling mode" 可以设置的值有: None, Full, Center, Full aspect

- 要显示可用的输出和设置，执行：

```
$ xrandr --prop
```

- 要为每个有效显示输出都设置 `scaling mode = Full aspect`，使用下面的命令:

```
$ for output in $(xrandr --prop | grep -E -o -i "^[A-Z\-]+-[0-9]+"); do xrandr --output "$output" --set "scaling mode" "Full aspect"; done
```

排除故障

模块参数

amdgpu模块含有一些使用掩码的配置参数(`modinfo amdgpu | grep mask`)，这些参数文档请参阅内核源码

Xorg 或应用程序无法启动

- "(EE) AMDGPU(0): \[DRI2\] DRI2SwapBuffers: drawable has no back or front?" error after opening glxgears, can open Xorg server but OpenGL apps crash.
- "(EE) AMDGPU(0): Given depth (32) is not supported by amdgpu driver" error, Xorg won't start.

在 Xorg 下将屏幕颜色深度设置为 16 或 32 会导致异常或崩溃。为避免这种情况，可以通过往配置的*Screen*区块添加设置来使用标准的 24 位屏幕颜色深度：

```
/etc/X11/xorg.conf.d/10-screen.conf
Section "Screen"
        Identifier     "Screen"
        DefaultDepth    24
        SubSection      "Display"
                Depth   24
        EndSubSection
EndSection
```

屏幕伪影和频率问题

由于 GPU 时钟速度的管理方式存在问题，动态电源管理可能会在显示器运行高刷新率(120+hz)时导致屏幕伪影 .org/show_bug.cgi?id=96868。

一个规避的办法 是手动

将 `/sys/class/drm/card0/device/power_dpm_force_performance_level` 设置为 `high` 或 `low`

要使设置持久化生效，可以创建一个 udev 规则：

```
/etc/udev/rules.d/30-amdgpu-pm.rules
KERNEL=="card0", SUBSYSTEM=="drm", DRIVERS=="amdgpu", ATTR{device/power_dpm_force_performance_level}="high"
```

执行下面的命令检查 KERNEL 参数名称：

```
 $ find /sys/class/drm/ -regextype awk -regex '.+/card[0-9]+' -printf '%f\n'
```

另一个图形界面的解决方案 是用 radeon-profile-gitAUR 和 radeon-profile-daemon-gitAUR 来管理 *power_dpm*。

更换内核版本也可能解决这个问题，该问题看起来已在6.12.9版本修复。

Chromium 伪影

如果在使用 Chromium 时看到伪影，可以尝试强制使用基于 vulkan 的后端。在 `chrome://flags` 中*启用* `#ignore-gpu-blocklist` 和 `#enable-vulkan`。

R9 390 系列性能不佳或不稳定

如果在使用 AMD R9 390 系列显卡时遇到问题 ，可以在内核参数中设置 `radeon.cik_support=0 radeon.si_support=0 amdgpu .cik_support=1 amdgpu.si_support=1 amdgpu.dpm=1 amdgpu.dc=1` 来强制使用 amdgpu 驱动模块而不是 radeon 驱动模块。

如果还不起作用，请尝试通过在内核参数中设置：`radeon.cik_support=0 radeon.si_support=0 amdgpu.cik_support=1 amdgpu.si_support=1` 来禁用动态电源管理（DPM）。

因内核错误"\[drm\] IP block:gmc_v8_0 is hung!"画面冻结

如果在执行 GPU 密集型任务期间遇到冻结和内核崩溃，并出现内核错误"\[drm\] IP block:gmc_v8_0 is hung!" 时，一个规避的方法是在内核参数中设置 `amdgpu.vm_update_mode=3` 来强制使用 CPU 完成 GPUVM 页面表更新。 这里列出了这个方法的缺点 。

屏幕闪烁或出现白/灰屏

当改变分辨率或连接外接显示器时屏幕出现闪烁或白屏，在内核参数中加入 `amdgpu.sg_display=0`。

在 Vega 显卡上玩游戏时系统冻结或崩溃

由于 GPU 核心频率的管理方式存在问题，使用显卡驱动的动态电源管理可能会导致系统在玩游戏时完全冻结。 一个规避方法是禁用动态电源管理, 更多细节可以参考 ATI#动态电源管理。

火狐浏览器 WebRenderer 损坏

当用户强制启用 WebRenderer 时，可能会出现伪影和其他异常（例如无法选择扩展选项）。解决方法是回退到 OpenGL 合成。

两倍速或花栗鼠（chipmunk）音频，或连接 4K@60Hz 设备时无音频

这有时是由 AMDGPU 设备和通过 HDMI 连接的 4K 显示器之间的通信问题引起的。一种可能的规避方法是通过显示器的内置设置启用 HDR 或“超高清深色”。在许多基于

Android 的电视上，这对应着将相关选项设置为“标准”（Standard）而不是“最佳”（Optimal）。

独立显卡的电源管理/动态重新激活问题

如果遇到内核驱动已经加载，但游戏时独立显卡不可用或在使用时独立显卡被禁用（类似于 ） 的问题，可以通过设置内核参数 `amdgpu.runpm=0` 来防止独显在运行时动态断电。

kfd: amdgpu: TOPAZ not supported in kfd

在系统日志或内核消息密钥环中打印危急级别的错误消息

```
kfd: amdgpu: TOPAZ not supported in kfd
```

如果没有使用 Radeon Open Compute 的计划，可以安全的忽略这个问题。因为旧显卡不支持 TOPAZ。

MCLK锁定在最大（1000MHz）或最小（96MHz）频率导致的高待机功耗或低游戏性能问题（6.4内核）

在高分辨率和刷新率的设置下，MCLK（vram，显存频率）可能会锁定在最高值（1000MHz） ，造成GPU待机功耗过高。使用Linux 6.4.x内核，MCLK锁定在最低频率（96MHz）影响游戏性能 。

这可能是因为显示器没有为受影响的分辨率和刷新率设置使用一个低V-Blank值的协调视频时序（Coordinated Video Timings， CVT），参考这篇内容来解决。

未能挂起（suspend）到内存

当系统进入S3状态后，`amdgpu`内核模块会尝试缓存显存数据到内存中，以避免显存不能充分刷新，显存中的数据老化导致数据丢失。

如果你使用了大量显存而空闲内存不足，这会导致挂起失败。因为IO子系统可能会在此之前挂起，不论是否有足够的swap空间该问题都可能发生。

你可能会看到类似的消息：

```
kernel: systemd-sleep: page allocation failure: order:0, mode:0x100c02(GFP_NOIO|
__GFP_HIGHMEM|__GFP_HARDWALL), nodemask=(null),cpuset=/,mems_allowed=0
kernel: Call Trace:
kernel:  <TASK>
kernel:  dump_stack_lvl+0x47/0x60
kernel:  warn_alloc+0x165/0x1e0
kernel:  __alloc_pages_slowpath.constprop.0+0xd7d/0xde0
kernel:  __alloc_pages+0x32d/0x350
kernel:  ttm_pool_alloc+0x19f/0x600 [ttm 0bd92a9d9dccc3a4f19554535860aaeda76eb4f4]
```

通过一个用户空间服务可解决该问题，该服务能够在系统挂起前将足够的内存交换到SWAP，确保能够申请到足够的内存用以显存缓存。

关机和挂起（suspend）失败

*hid_sensor_*_3d*这类内核模块能够造成系统在启动、关机和挂起时死锁。进程列表里将会显示了多个`udev-worker`实例，导致系统睡眠时冻结。

你会看到类似消息：

```
kernel: PM: suspend entry (deep)
kernel: Filesystems sync: 0.002 seconds
kernel: Freezing user space processes
kernel: Freezing user space processes failed after 20.004 seconds (1 tasks refusing to
freeze, wq_busy=0):
kernel: task:(udev-worker)    state:D stack:0     pid:479   tgid:479   ppid:422
flags:0x00004006
kernel: Call Trace:
kernel:  <TASK>
kernel:  __schedule+0x3db/0x1520
kernel:  ? srso_alias_return_thunk+0x5/0xfbef5
kernel:  ? __wake_up_common+0x78/0xa0
kernel:  ? srso_alias_return_thunk+0x5/0xfbef5
```

要解决该问题，将有问题的模块添加到黑名单中，创建配置文件如`/etc/modprobe.d/blacklist-hid_sensors.conf`：

```
blacklist hid_sensor_accel_3d
blacklist hid_sensor_gyro_3d
blacklist hid_sensor_magn_3d
```

参考资料

- Gentoo:AMDGPU

2.18 NVIDIA

难度 高级 | **篇幅** 37.9 KB | **所属** 图形

本节将讲到

- 安装
- Xorg 配置
- Wayland配置
- NVswitch
- 提示和技巧

翻译状态：

- 本文（或部分内容）译自 NVIDIA，最近一次同步于 2025-11-2，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

本文包含安装和配置 NVIDIA 官方显卡驱动的信息。想要了解社区开源驱动的信息，参见 Nouveau。如果您使用的是有混合显卡的笔记本（如Intel核显+NVIDIA显卡），请额外参阅 NVIDIA Optimus 页面。

安装

警告：请避免从 NVIDIA 官网下载驱动包进行安装。建议使用 Arch Linux 的 pacman 安装驱动程序，这样驱动程序能够在更新系统时与其他组件一同更新。

注意：在具有混合图形技术的机型上双重启动时，启用Windows或第三方应用中的 *集显模式/节能模式* 可能会完全禁用NVIDIA独立显卡(例如ASUS节能模式)，导致其无法被检测到。

首先，从 nouveau NVIDIA代号查询页 中查找您的显卡系列代号(例如：NV110, NVC0)。您也可以运行以下命令获知显卡型号/名称：

```
$ lspci -k -d ::03xx
```

随后为您的显卡安装合适的驱动：

: > **注意**：

- 使用 dkms包 安装时, 请参阅 DKMS#安装 部分
- 使用DKMS方式安装的驱动并不与特定内核绑定，其会为安装了头文件的每个内核重新编译NVIDIA内核模块。

GPU 家族	驱动	支持状态
Blackwell (GBXXX))系列或更新版本	nvidia-open包 适用于 linux包 nvidia-open-lts包 适用于 linux-lts包 nvidia-open-dkms包 适用于任意内核	上游推荐使用 支持中1
Turing (NV160/TUXXX)系列到 Ada Lovelace (NV190/ADXXX)系列	同时被以下包支持： - nvidia-open包 在 Turing 架构上可能存在电源管理问题2, 可能会在 Turing 架构的笔记本上崩溃2 - nvidia-580xx-dkmsAUR	

1. 如果以上驱动安装后都不能正常工作，您也许需要使用 nvidia-open-betaAUR 以获得更新版本的驱动。
2. NVIDIA 的开源内核模块无法在 Turing 架构的显卡上启用 D3 电源管理_电源管理)。这会降低搭载Turing 架构显卡且使用 NVIDIA Optimus 配置的笔记本电脑的续航时间。建议改用专有驱动模块（例如 nvidia-580xx-dkmsAUR），并在 内核参数 中设置`NVreg_EnableGpuFirmware=0`。有关此问题的更多信息。
3. 在安装了 Intel CPU 11 代或更新版本的处理器以及Linux 5.18 (或更高版本)的系统上可能无法正常工作，原因是与其与 Indirect Branch Tracking 这个安全功能不兼容。您可以在 Arch_的启动流程中设置 `ibt=off` 内核参数来禁用它。请注意，这项安全功能负责缓解一些攻击技术的影响。
4. NVIDIA不再积极为这些显卡提供驱动。这意味着这些驱动不正式支持现在的Xorg版本。因此使用 Nouveau 驱动可能会更方便，因为它依然为这些老显卡提供对当前Xorg的支持。但是，NVIDIA 提供的 legacy 驱动依旧可用，并且或许拥有更好的 3D 性能（稳定性）。

对于 32 位应用程序支持，请安装 multilib 仓库中对应的 *lib32* nvidia 软件包（例如 lib32-nvidia-utils包）。

nvidia-utils包 软件包包含一个文件，其将会在重启后屏蔽 *nouveau* 内核模块。另外，您还可以将`kms` 从 `/etc/mkinitcpio.conf` 里的`HOOKS` 数组中移除，并重新生成 initramfs。 这将防止 initramfs 包含 `nouveau` 模块，以确保内核在早启动阶段不会加载它。

注意：如果您正在使用 Wayland， 您需要首先配置 #DRM 内核级显示模式设置 后重启，否则您可能会遇到黑屏。

一旦驱动安装完毕，就可以进入下一步了：#Xorg配置或#Wayland配置

定制内核

安装 nvidia-open-dkms包 软件包以及你内核对应的头文件。

确保您的内核设置了 `CONFIG_DRM_SIMPLEDRM=y` 选项，如果使用 `CONFIG_DEBUG_INFO_BTF`，则 PKGBUILD 中需要以下选项（从内核 5.16 开始）：

```
install -Dt "$builddir/tools/bpf/resolve_btfids" tools/bpf/resolve_btfids/resolve_btfids
```

linux包 6.15 引入了新的 NVIDIA GPU驱动 Nova。nvidia-utils包 默认会将其加入黑名单。您可以运行systemd-analyze来验证这一点。 如果您还安装了其他版本的驱动程序，可能还需要手动将`nova_core`和`nova_drm`模块加入黑名单。

DRM 内核级显示模式设置

由于NVIDIA不支持自动 KMS 晚加载, 要正常运行Wayland 合成器或没有 root 权限的 Xorg，您必须启用 DRM (直接渲染管理器)内核级显示模式设置。

nvidia-utils包 560.35.03-5 版本后默认启用 DRM (Direct Rendering Manager) 内核级显示模式设置，如您使用更旧的驱动版本，请为`nvidia_drm`模块设置以下内核模块参数：`modeset=1`。

您可以使用以下命令验证DRM是否自动启用：

```
# cat /sys/module/nvidia_drm/parameters/modeset
```

其应当返回 `Y`。

警告：官方支持的内核 默认启用 `simpledrm`, 如您使用545或更旧版本的NVIDIA驱动，您需要设置内核模块参数 `nvidia_drm.fbdev`。因为当其不被启用时，NVIDIA 驱动程序尝试使用 `efifb` 或 `vesafb` 进行帧缓冲(其无法在`simpledrm`下工作)。

注意：版本 470 之前的 NVIDIA 驱动程序 (例如 nvidia-390xx-dkmsAUR) 不支持对 XWayland 进行硬件加速，导致非 Wayland 原生应用程序在 Wayland 会话中性能不佳。

早启动

对于基本功能，只需添加内核参数就足够了。如果您希望确保驱动能被最早加载，或者遇到了启动问题（例如 `nvidia` 内核模块在显示管理器启动后才被加载），可以将 `nvidia`、`nvidia_modeset`、`nvidia_uvm` 和 `nvidia_drm` 添加到 initramfs 中。请参阅内核模块#早期模块加载以了解如何配置您的 initramfs 生成器。mkinitcpio用户可能还需要在每次 nvidia-open包 驱动程序更新时重新生成 initramfs映像。请参阅 #Pacman 钩子以自动化这些步骤

mkinitcpio

如果使用 mkinitcpio initramfs，请按照 mkinitcpio#MODULES 添加模块。

如果你将驱动添加到了 initramfs 中，记得每次更新 nvidia-open包 驱动程序后都要运行 mkinitcpio。请参见 #pacman hook 来自动执行这些步骤。

Booster

如果您使用的是 Booster，请参阅 Booster#早期加载内核模块。

dracut

如果您使用的是 dracut，请参阅 dracut#早期内核模块加载。

Pacman 钩子

为了避免更新 NVIDIA 驱动之后忘了更新 initramfs，你可以使用 Pacman Hooks：

```
/etc/pacman.d/hooks/nvidia.hook
[Trigger]
Operation=Install
Operation=Upgrade
Operation=Remove
Type=Package
Target=nvidia-open
Target=linux
# Change the linux part above and in the Exec line if a different kernel is used
# 如果使用不同的内核，请更改上面的 linux 部分和 Exec 行中的内容
[Action]
Description=Updating NVIDIA module in initcpio
Depends=mkinitcpio
When=PostTransaction
NeedsTargets
Exec=/bin/sh -c 'while read -r trg; do case $trg in linux) exit 0; esac;
done; /usr/bin/mkinitcpio -P'
```

务必保证 `Target` 项所设置的软件包与你在前面的安装过程中所使用的相符（例如 `nvidia-open-dkms` 或 `nvidia-open-lts` 等）。

注意：`Exec` 那一行看起来非常复杂，是为了避免在 `nvidia-open` 和 `linux` 软件包都发生更新的时候重复运行 `mkinitcpio`。如果你觉得无所谓，可以删掉 `Target=linux` 以及 `NeedsTargets`，然后 `Exec` 就可以简化为 `Exec=/usr/bin/mkinitcpio -P`.

硬件加速视频解码

GeForce 8 系列及更新的显卡通过 VDPAU 进行视频硬件解码。Fermi 架构（400、500系列）及更新的显卡支持通过 NVDEC 进行硬件加速解码。参见硬件视频加速。

基于 NVENC 的硬件加速视频编码

想要使用 NVENC，则需要安装 `nvidia_uvm` 模块，并在 `/dev` 下创建相关设备节点。

最新的驱动程序包提供了一个 udev规则，它可以自动创建设备节点，因此不需要进一步的操作。

如果您使用旧的驱动程序 (例如 nvidia-340xx-dkmsAUR) ，则需要另外创建设备节点。调用 `nvidia-modprobe` 实用程序会自动创建它们。您可以自己创建 `/etc/udev/rules.d/70-nvidia.rules` 规则来自动运行它：

```
/etc/udev/rules.d/70-nvidia.rules
ACTION=="add", DEVPATH=="/bus/pci/drivers/nvidia", RUN+="/usr/bin/nvidia-modprobe -c 0 -u"
```

Xorg 配置

NVIDIA专有驱动不需要任何 Xorg 服务器配置文件。您可以启动 X 来检验没有配置文件的 Xorg 能否正确运行。但是，您可能需要创建配置文件（优先创建`/etc/X11/xorg.conf.d/20-nvidia.conf`而不是`/etc/X11/xorg.conf`）来调整 Xorg 运行时的一些设置。您可以用NVIDIA Xorg 配置工具来生成配置，也可以手动创建它。假如是手动创建的话，它可以是一个最小配置文件(也就是意味着它仅仅把一些基础的选项传给 Xorg 服务器)，也可以是包含大量的绕开Xorg 自动发现与预配置选项的设置的文件。

提示：更多配置项相关内容参见 NVIDIA/故障排除。

自动配置

NVIDIA的软件包已经包含一个自动配置的工具来帮助您创建Xorg服务器配置文件(`xorg.conf`)。您可以通过运行下面的命令来实现自动配置：

```
# nvidia-xconfig
```

该命令会自动检测并根据现有硬件的情况创建（或修改）`/etc/X11/xorg.conf`。

再一次检查您的配置文件`/etc/X11/xorg.conf`中的默认色深、水平同步、垂直刷新和分辨率是否正确。

nvidia-settings

nvidia-settings包可以让你通过 CLI 或者图形界面配置很多选项。无参数运行`nvidia-settings`会启动图形界面版本的配置工具，而 CLI 选项参见nvidia-settings(1)。

你可以以非 root 用户身份使用 CLI/图形界面配置工具，并把设置保存到`~/.nvidia-settings-rc`或使用*Save to X configuration File*按钮来保存到xorg.conf（用于多用户环境）。

通过以下命令来为当前用户加载 `~/.nvidia-settings-rc`：

```
$ nvidia-settings --load-config-only
```

参见 Autostarting 来在每次开机的时候自动执行此命令。

{{注意|

- 在保存了`nvidia-settings`的设置变更之后，Xorg 可能会无法启动或者启动时崩溃。修改或者删掉生成的`~/.nvidia-settings-rc`及（或）Xorg 文件可能能够恢复正常启动。

手动配置

有些调整项无法通过 #自动配置 或 #nvidia-settings 自动启用，但可以通过修改你的配置文件来进行配置。Xorg服务器需要重新启动以应用这些配置。

参见 NVIDIA Accelerated Linux Graphics Driver README and Installation Guide 来了解更多细节及选项。

最简配置

用 root 用户创建一个配置文件`20-nvidia.conf`(或 `/etc/X11/xorg.conf`)：

```
/etc/X11/xorg.conf.d/20-nvidia.conf
Section "Device"
   Identifier     "Device0"
   Driver         "nvidia"
   VendorName     "NVIDIA Corporation"
EndSection
```

关闭启动时的 Logo

如果您使用的是旧版驱动(nvidia-340xx-dkmsAUR)，您可能希望禁用在 X 启动时显示的 NVIDIA 徽标启动画面。添加`"NoLogo"`选项到`Device`节里：

```
Option "NoLogo" "1"
```

覆盖显示器侦测

Device节下面的"ConnectedMonitor"选项允许您覆盖X服务器在启动时的显示器侦测过程，这可能有助于在启动时节约大量时间。可用的选项包括："CRT"用于模拟信号连接，"DFP"用于数字显示器，"TV"用于电视。

下面的例子是强制NVIDIA的驱动绕开启动检测并且强制把显示器识别为DFP：

```
Option "ConnectedMonitor" "DFP"
```

注意： "CRT"适用于所有传输模拟信号的15 pin VGA连接器，包括平板显示器上的VGA接口。"DFP"仅适用于DVI、HDMI和DisplayPort等数字信号连接器。

启用亮度控制

添加内核参数：

```
nvidia.NVreg_RegistryDwords=EnableBrightnessControl=1
```

或者，在Device节下添加:

```
Option "RegistryDwords" "EnableBrightnessControl=1"
```

如果亮度控制依旧无法生效，请尝试安装nvidia-bl-dkmsAUR。

注意：安装 nvidia-bl-dkmsAUR 会提供 `/sys/class/backlight/nvidia_backlight/` 接口以用于背光亮度控制，但是你的系统可能会继续发送亮度调整信号到 `/sys/class/backlight/acpi_video0/`。 解决方案之一是监视文件变化，例如为 `acpi_video0/brightness` 添加 *inotifywait* 并将变化翻译并写入 `nvidia_backlight/brightness`。 参见 Backlight#sysfs modified but no brightness change.

启用 SLI

警告：自 GTX 10xx系列（1080、1070、1060等等）开始，仅 2 路 SLI 是受支持的。3 路或 4 路 SLI 或许能够在 CUDA/OpenCL 应用上使用，但是会让几乎所有 OpenGL 应用崩溃。

根据NVIDIA驱动的 README 附录 B: *这个选项可以在支持的配置中控制 SLI 渲染的相关配置。*一个*支持的配置*是指有一块 SLI 认证的主板以及 2 或者 3 个 SLI认 证的 GeForce GPU 的计算机。

您可以用lspci查找第一个 GPU 的 PCI 总线 ID：

```
# lspci -d ::03xx
00:02.0 VGA compatible controller: Intel Corporation Xeon E3-1200 v2/3rd Gen Core processor Graphics Controller (rev 09)
03:00.0 VGA compatible controller: NVIDIA Corporation GK107 [GeForce GTX 650] (rev a1)
04:00.0 VGA compatible controller: NVIDIA Corporation GK107 [GeForce GTX 650] (rev a1)
08:00.0 3D controller: NVIDIA Corporation GM108GLM [Quadro K620M / Quadro M500M] (rev a2)
```

在Device节下添加 BusID（例如前面例子里的那 3 个）：

```
BusID "PCI:3:0:0"
```

注意：这个格式很重要。BusID的值必须指定为"PCI:<BusID>:0:0"的格式。

根据需要的 SLI 渲染模式来添加值到Screen节下面：

```
Option "SLI" "AA"
```

下表列出了可用的渲染模式。

Value	Behavior
0, no, off, false, Single	渲染时仅使用单GPU。
1, yes, on, true, Auto	启用SLI并让驱动自动选择合适的渲染模式。
AFR	启用SLI并使用交替帧渲染模式。
SFR	启用SLI并使用分割帧渲染模式。
AA	启用SLI和使用SLI抗锯齿。与全场景反锯齿结合使用以改善视觉效果。

另外，您可以使用 `nvidia-xconfig` 实用工具来将这些变动写入`xorg.conf`：

```
# nvidia-xconfig --busid=PCI:3:0:0 --sli=AA
```

从shell来验证一下SLI是否被启用：

```
$ nvidia-settings -q all | grep SLIMode
   Attribute 'SLIMode' (arch:0.0): AA
     'SLIMode' is a string attribute.
     'SLIMode' is a read-only attribute.
     'SLIMode' can use the following target types: X Screen.
```

警告：启用 SLI 之后，你的系统可能会在启动 xorg 的时候假死、没有反应。建议你在重启之前禁用你的显示管理器。

如果这个配置没有效果，你可能需要使用 `nvidia-settings` 提供的PCI总线ID：

```
$ nvidia-settings -q all | grep -i pcibus
Attribute 'PCIBus' (host:0[gpu:0]): 101.
  'PCIBus' is an integer attribute.
  'PCIBus' is a read-only attribute.
  'PCIBus' can use the following target types: GPU, SDI Input Device.
Attribute 'PCIBus' (host:0[gpu:1]): 23.
  'PCIBus' is an integer attribute.
  'PCIBus' is a read-only attribute.
  'PCIBus' can use the following target types: GPU, SDI Input Device.
```

并将 xorg.d 配置中的 PrimaryGPU 选项注释掉：

```
/usr/share/X11/xorg.conf.d/10-nvidia-drm-outputclass.conf
...

Section "OutputClass"
...
    # Option "PrimaryGPU" "yes"
...
```

使用后面这个配置或许会顺便解决任何图形界面启动问题。

多台显示器

参见Multihead以获取更多信息。

使用 nvidia-settings

nvidia-settings 工具可以配置多个监视器。

对于 CLI 配置，首先运行以下命令获取 `CurrentMetaMode`:

```
$ nvidia-settings -q CurrentMetaMode
Attribute 'CurrentMetaMode' (hostnmae:0.0): id=50, switchable=no, source=nv-control ::
DPY-1: 2880x1620 @2880x1620 +0+0 {ViewPortIn=2880x1620, ViewPortOut=2880x1620+0+0}
```

将 `::` 之后的所有内容保存到属性末尾（在本例中为：`DPY-1: 2880x1620 @2880x1620 +0+0 {ViewPortIn=2880x1620, ViewPortOut=2880x1620+0+0}`），并使用 `nvidia-settings --assign "CurrentMetaMode=*your_meta_mode*"` 重新配置显示器。

提示：您可以为所使用的不同监视器和分辨率配置创建 shell alias。

ConnectedMonitor

如果驱动程序没有正确检测到第二个监视器，则可以使用 ConnectedMonitor 强制设置。

```
/etc/X11/xorg.conf
Section "Monitor"
    Identifier     "Monitor1"
    VendorName     "Panasonic"
    ModelName      "Panasonic MICRON 2100Ex"
    HorizSync       30.0 - 121.0 # this monitor has incorrect EDID, hence Option
"UseEDIDFreqs" "false"
    VertRefresh     50.0 - 160.0
    Option         "DPMS"
EndSection

Section "Monitor"
    Identifier     "Monitor2"
    VendorName     "Gateway"
    ModelName      "GatewayVX1120"
    HorizSync       30.0 - 121.0
    VertRefresh     50.0 - 160.0
    Option         "DPMS"
EndSection

Section "Device"
    Identifier     "Device1"
    Driver         "nvidia"
    Option         "NoLogo"
    Option         "UseEDIDFreqs" "false"
    Option         "ConnectedMonitor" "CRT,CRT"
    VendorName     "NVIDIA Corporation"
    BoardName      "GeForce 6200 LE"
    BusID          "PCI:3:0:0"
    Screen          0
EndSection

Section "Device"
    Identifier     "Device2"
    Driver         "nvidia"
    Option         "NoLogo"
    Option         "UseEDIDFreqs" "false"
    Option         "ConnectedMonitor" "CRT,CRT"
    VendorName     "NVIDIA Corporation"
    BoardName      "GeForce 6200 LE"
    BusID          "PCI:3:0:0"
    Screen          1
EndSection
```

通过添加带有 `Screen` 选项的重复设备配置，能让 X 在一张显卡上使用两台显示器而不需要 `TwinView`。注意，`nvidia-settings` 将去掉您添加的任何 `ConnectedMonitor` 选项。

TwinView

若你想用一个大屏幕,而不是两个屏幕.将`TwinView`设为`1`。只有所有显示器都接到同一张显卡上时，TwinView 才能工作。

```
Option "TwinView" "1"
```

示例配置：

```
/etc/X11/xorg.conf.d/10-monitor.conf
Section "ServerLayout"
    Identifier     "TwinLayout"
    Screen         0 "metaScreen" 0 0
EndSection

Section "Monitor"
    Identifier     "Monitor0"
    Option         "Enable" "true"
EndSection

Section "Monitor"
    Identifier     "Monitor1"
    Option         "Enable" "true"
EndSection

Section "Device"
    Identifier     "Card0"
    Driver         "nvidia"
    VendorName     "NVIDIA Corporation"

    #refer to the link below for more information on each of the following options.
    Option         "HorizSync"           "DFP-0: 28-33; DFP-1: 28-33"
    Option         "VertRefresh"         "DFP-0: 43-73; DFP-1: 43-73"
    Option         "MetaModes"           "1920x1080, 1920x1080"
    Option         "ConnectedMonitor"    "DFP-0, DFP-1"
    Option         "MetaModeOrientation" "DFP-1 LeftOf DFP-0"
EndSection

Section "Screen"
    Identifier     "metaScreen"
    Device         "Card0"
    Monitor        "Monitor0"
    DefaultDepth    24
    Option         "TwinView" "True"
    SubSection "Display"
        Modes          "1920x1080"
    EndSubSection
EndSection
```

另请参考设备选项信息

如果您有多个支持 SLI 的显卡，则可以把多个显示器接到不同的显卡上。“MetaModes”选项与 SLI Mosaic 模式配合使用可实现此功能。下面是一个适用于上述情况的配置，它可以完美运行 GNOME。

```
/etc/X11/xorg.conf.d/10-monitor.conf
Section "Device"
        Identifier      "Card A"
        Driver          "nvidia"
        BusID           "PCI:1:00:0"
EndSection

Section "Device"
        Identifier      "Card B"
        Driver          "nvidia"
        BusID           "PCI:2:00:0"
```

```
EndSection

Section "Monitor"
        Identifier      "Right Monitor"
EndSection

Section "Monitor"
        Identifier      "Left Monitor"
EndSection

Section "Screen"
        Identifier      "Right Screen"
        Device          "Card A"
        Monitor         "Right Monitor"
        DefaultDepth    24
        Option          "SLI" "Mosaic"
        Option          "Stereo" "0"
        Option          "BaseMosaic" "True"
        Option          "MetaModes" "GPU-0.DFP-0: 1920x1200+4480+0,
GPU-1.DFP-0:1920x1200+0+0"
        SubSection      "Display"
                        Depth           24
        EndSubSection
EndSection

Section "Screen"
        Identifier      "Left Screen"
        Device          "Card B"
        Monitor         "Left Monitor"
        DefaultDepth    24
        Option          "SLI" "Mosaic"
        Option          "Stereo" "0"
        Option          "BaseMosaic" "True"
        Option          "MetaModes" "GPU-0.DFP-0: 1920x1200+4480+0,
GPU-1.DFP-0:1920x1200+0+0"
        SubSection      "Display"
                        Depth           24
        EndSubSection
EndSection

Section "ServerLayout"
        Identifier      "Default"
        Screen 0        "Right Screen" 0 0
        Option          "Xinerama" "0"
EndSection
```

对于同时拥有AMD核显和NVIDIA独立显卡的用户

由于NVIDIA自动生成的xorg.conf配置默认不包含AMDGPU这会导致双屏用户其中一个屏幕不能显示 需要先进行自动配置，然后在`/etc/X11/xorg.conf.d/`下新建`20-amdgpu.conf`：

```
Section "Device"
     Identifier "AMD"
     Driver "amdgpu"
EndSection
```

TwinView 和垂直同步

如果您正在使用 TwinView 和垂直同步 (**nvidia-settings** 中的 *Sync to VBlank* 选项)，您将注意到只有一个屏幕能正常同步，除非您有两个相同的显示器。虽然 **nvidia-settings** 确实提供了一个选项来更改垂直同步的屏幕 (**Sync to this display**

device 选项），但这并不总是有效。一个解决方案是在启动时添加以下环境变量，例如在 `/etc/profile` 中附加：

```
export __GL_SYNC_TO_VBLANK=1
export __GL_SYNC_DISPLAY_DEVICE=DFP-0
export VDPAU_NVIDIA_SYNC_DISPLAY_DEVICE=DFP-0
```

你可以把 `DFP-0` 改为你首选要同步的屏幕 (`DFP-0` 是DVI接口， `CRT-0` 是VGA接口). 您可以从 "X Server XVideoSettings" 部分的 **nvidia-settings** 中找到显示器的标识符。

用 TwinView 玩游戏

如果你想在使用 TwinView 时玩全屏游戏，你会注意到游戏将两个屏幕识别为一个大屏幕。虽然这理论上没问题（因为虚拟 X 屏幕实际上是屏幕大小的总和），但您可能不希望同时在两个屏幕上显示。

要纠正 SDL 的这种行为，请尝试：

```
export SDL_VIDEO_FULLSCREEN_HEAD=1
```

对于 OpenGL，向 xorg.conf 中添加适当的 Metamode 并重新启动 X:

```
Option "Metamodes" "1680x1050,1680x1050; 1280x1024,1280x1024; 1680x1050,NULL; 1280x1024,NULL;"
```

另一种可以单独使用或与上述方法结合使用的方法是在单独的 X 服务器中启动游戏。

马赛克拼接模式

马赛克模式是使用多个显示器进行跨显卡合成的唯一方式。您的窗口管理器不一定能区分不同的显示器。马赛克模式需要一个有效的 SLI 配置。即使使用基本模式而不使用 SLI，GPU 仍然必须支持 / 兼容 SLI。

基本马赛克拼接模式

Base Mosaic 模式适用于任何 Geforce 8000 系列或更高版本的 GPU。无法从 nvidia-setting GUI 中启用它。您必须使用 *nvidia-xconfig* 命令行程序或手动编辑 `xorg.conf`。必须指定元模式。以下是 2x2 配置中四个 DFP 的示例，每个 DFP 以 1920x1024 运行，两个 DFP 连接到两个显卡：

```
# nvidia-xconfig --base-mosaic --metamodes="GPU-0.DFP-0: 1920x1024+0+0, GPU-0.DFP-1: 1920x1024+1920+0, GPU-1.DFP-0: 1920x1024+0+1024, GPU-1.DFP-1: 1920x1024+1920+1024"
```

注意：虽然文档列出了 2x2 的监视器配置，在基础马赛克模式下 GeForce 显卡被人为限制了只能使用3个显示器。Quadro 卡支持 3 个以上的显示器。截至 2014 年 9 月，Windows 驱动程序已取消了这一人为限制，但仍保留在 Linux 驱动程序中。

SLI 马赛克拼接模式

如果您有 SLI 配置，并且每个 GPU 都是 Quadro FX 5800、Quadro Fermi 或更新型号，那么您可以使用 SLI 马赛克模式。可以从 nvidia-settings GUI 内或从命令行通过以下方式启用：

```
# nvidia-xconfig --sli=Mosaic --metamodes="GPU-0.DFP-0: 1920x1024+0+0, GPU-0.DFP-1: 1920x1024+1920+0, GPU-1.DFP-0: 1920x1024+0+1024, GPU-1.DFP-1: 1920x1024+1920+1024"
```

Wayland配置

关于 XWayland，请参阅 Wayland#Xwayland。

有关进一步的配置选项，请查看相应混成器的 wiki 页面或文档。

注意：在驱动程序版本 555.xx 之前，或者在使用不支持通过`linux-drm-syncobj-v1`协议进行显式同步的混成器时，NVIDIA 驱动程序可能会出现严重问题，表现为闪烁、帧顺序错乱等，这在原生 Wayland 或 Xwayland 应用程序中均会发生。

配置

`nvidia_drm` 模块有如下两个可选的内核参数。在使用560.35.03-5或更高版本的nvidia-utils包时，这两个参数默认已启用

modeset

使用Wayland时，`modeset`必须被启用。560.35.03-5或更高版本的驱动已经默认启用了这一设置。

如您使用的是较旧的驱动，您可能需手动启用 `modeset`，请参阅 #DRM 内核级显示模式设置和 Wayland系统需求进行配置。

fbdev

启用`fbdev`对于某些 Wayland 配置是必要的。

在Linux 6.11以及更高版本上这是一个硬性需求，但目前尚不清楚这是预期行为还是一个错误

您可以参考 #DRM 内核级显示模式设置中的方法进行设置。其区别在于验证其是否打开时，您需要执行：

```
# cat /sys/module/nvidia_drm/parameters/fbdev
```

如果没有成功设置，其将返回缺少文件错误，而不是`N`。

休眠支持

Wayland 的休眠可能比 X 更容易受到默认设置的影响，详细信息请参见 Tips and tricks#Preserve video memory after suspend。

如您使用 GDM，请参考 GDM#Wayland 与 NVIDIA 专有驱动。

nvidia-application-profiles-rc.d

某些 Wayland 合成器在默认情况下会占用大量的 VRAM，您可以通过对它们的进程名称应用 GLVidHeapReuseRatio 应用程序配置键来解决这个问题。例如，niri 用户可通过以下方式释放 ~2.5GiB 的显存占用：

```
/etc/nvidia/nvidia-application-profiles-rc.d/50-limit-free-buffer-pool-in-wayland-compositors.json
{
    "rules": [
        {
            "pattern": {
                "feature": "procname",
                "matches": "niri"
            },
            "profile": "Limit free buffer pool on Wayland compositors"
        }
    ],
    "profiles": [
        {
            "name": "Limit free buffer pool on Wayland compositors",
            "settings": [
                {
                    "key": "GLVidHeapReuseRatio",
                    "value": 0
                }
            ]
        }
    ]
}
```

NVswitch

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：需要多个语法改进，以及关于 fabric manager 的部分应当制作为一个 AUR 包。（在Talk:NVIDIA讨论）

对于具有 NVswitch 的系统，例如 AWS 上的 H100x8：

- 安装 nvidia-fabricmanager
- 安装所有fabric manager所需要的内核模块

使用 fabricmanager 时，pytorch 会报告未找到 GPU。

安装 fabric manager:

1. 从此处下载tarball。
2. 版本 555.42.02 运作良好
3. 修改 sbin/fm_run_package_installer.sh 中的安装脚本以修复安装文件路径

要获取匹配的内核驱动程序：

1. 克隆 nvidia-beta-dkms 和 nvidia-utils-beta 的 AUR

2. 将 PKGBUILD 更改为使用版本 555.42.02
3. 构建安装并重启

随后运行 `systemctl enable nvidia-fabricmanager` 以及 `systemctl start nvidia-fabricmanager`，现在 pytorch 应当能正确识别到GPU。

提示和技巧

参考 NVIDIA/提示和技巧.

故障排除

参考 NVIDIA/故障排除.

另请参见

- Current graphics driver releases in official NVIDIA Forum
- NVIDIA Developers Forum - Linux Subforum

2.19 NVIDIA/故障排除

难度 高级 | **篇幅** 38.1 KB | **所属** 图形

本节将讲到

- 启动失败
- 崩溃与卡死
- 显示问题
- 性能问题
- 其他问题

启动失败

安装驱动后系统无法启动

如果安装 NVIDIA 驱动后，系统在进入显示管理器之前卡住，请尝试内核级显示模式设置#禁用 KMS。

Xorg 无法加载或出现红屏死机

如果出现红屏且使用 GRUB，请编辑 `/etc/default/grub`，取消 `GRUB_TERMINAL_OUTPUT=console` 的注释，以禁用 GRUB 帧缓冲。详情参见 GRUB/技巧和窍门#禁用 framebuffer。

启动 X 时黑屏／关闭 X 时计算机断电

如果更新 NVIDIA 驱动后启动 Xorg 时屏幕保持黑屏，或者关闭 Xorg 会导致计算机断电，请尝试以下解决方法：

- 在 xinitrc 的开头添加 `xrandr --auto`。
- 使用 `rcutree.gp_init_delay=1` 内核参数。
- 也可以尝试在 mkinitcpio.conf 中直接添加 `nvidia` 模块。

检测到屏幕，但没有可用的配置

有时 NVIDIA 驱动和 X 无法正确找到正在使用的屏幕。如果显卡具有多个输出端口，请尝试将显示器连接到其他端口。对于笔记本电脑，这可能是因为显卡还具有 VGA/TV 输出。`Xorg.0.log` 中会提供更多信息。

还可以尝试在 `Section "Device"` 中添加一个无效的 `Option "ConnectedMonitor"`，使 Xorg 报错并提示正确配置方式。有关 ConnectedMonitor 选项的更多信息，请参阅官方文档。

重新运行 X 后，查看 `Xorg.0.log`，获取有效的 CRT-x、DFP-x 和 TV-x 值。

`nvidia-xconfig --query-gpu-info` 可能有所帮助。

X 启动失败并显示“Failing initialization of X screen”

如果 `/var/log/Xorg.0.log` 显示 X 服务器未能初始化屏幕：

```
(EE) NVIDIA(G0): GPU screens are not yet supported by the NVIDIA driver
(EE) NVIDIA(G0): Failing initialization of X screen
```

并且 *nvidia-smi* 显示 `No running processes found`，那么请先更新到最新版 nvidia-utils包，然后将 `/usr/share/X11/xorg.conf.d/` 下的 `10-nvidia-drm-outputclass.conf` 复制到 `/etc/X11/xorg.conf.d/`。编辑复制后的文件，添加 `Option "PrimaryGPU" "yes"`，最后重启计算机。此时问题应该已经解决。

Xorg 在启动过程中失败，但手动启动正常

在启动速度非常快的系统上，systemd 可能会在 NVIDIA 驱动完全初始化之前尝试启动显示管理器。只有在开机阶段运行 Xorg 时，日志中才会出现类似以下内容：

```
/var/log/Xorg.0.log
[     1.807] (EE) NVIDIA(0): Failed to initialize the NVIDIA kernel module. Please see
the
[     1.807] (EE) NVIDIA(0):     system's kernel log for additional error messages and
[     1.808] (EE) NVIDIA(0):     consult the NVIDIA README for details.
[     1.808] (EE) NVIDIA(0):  *** Aborting ***
```

这种情况下，需要建立显示管理器对 DRI 设备的启动顺序依赖。首先创建一个新的 udev 规则文件，以便为 DRI 设备创建设备单元：

```
/etc/udev/rules.d/99-systemd-dri-devices.rules
ACTION=="add", KERNEL=="card*", SUBSYSTEM=="drm", TAG+="systemd"
```

然后让显示管理器依赖这些设备：

```
/etc/systemd/system/display-manager.service.d/10-wait-for-dri-devices.conf
[Unit]
Wants=dev-dri-card0.device
```

```
After=dev-dri-card0.device
```

如果桌面还需要其他显卡，请将相应设备以空格分隔，添加到 Wants 和 After 中。

带集成 GPU 的系统出现黑屏

如果系统具有集成 GPU（例如 Intel HD 4000、VIA VX820 Chrome 9 或 AMD Cezanne），并且安装了 nvidia-580xx-dkmsAUR 或更早版本的软件包，那么在开机、切换虚拟终端或退出 X 会话时可能出现黑屏。这可能由显卡模块间的冲突引起。解决方法是将相关 GPU 模块加入黑名单。创建 `/etc/modprobe.d/blacklist.conf`，阻止相关模块在启动时加载：

```
/etc/modprobe.d/blacklist.conf
install i915 /usr/bin/false
install intel_agp /usr/bin/false
install viafb /usr/bin/false
install radeon /usr/bin/false
install amdgpu /usr/bin/false
```

多 GPU 系统中 X 启动失败并显示“no screens found”

当系统中存在多块 GPU 时，X 可能启动失败，并显示：

```
[ 76.633] (EE) No devices detected.
[ 76.633] Fatal server error:
[ 76.633] no screens found
```

此时需要在 X 配置中添加独立显卡的 BusID。拥有 Intel CPU 和集成 GPU 的系统，或安装了多块 NVIDIA 显卡的系统，都可能遇到这种情况。首先查找 BusID：

```
# lspci -d ::03xx
00:02.0 VGA compatible controller: Intel Corporation Xeon E3-1200 v2/3rd Gen Core processor Graphics Controller (rev 09)
01:00.0 VGA compatible controller: NVIDIA Corporation GK107 [GeForce GTX 650] (rev a1)
08:00.0 3D controller: NVIDIA Corporation GM108GLM [Quadro K620M / Quadro M500M] (rev a2)
```

然后在 X 配置文件中显卡的 Device 段添加 BusID。下面展示了配置段的示例（请替换为适合自己系统的值）：

```
/etc/X11/xorg.conf.d/10-nvidia.conf
Section "Device"
    Identifier     "Device0"
    Driver         "nvidia"
    VendorName     "NVIDIA Corporation"
    BusID          "PCI:1:0:0"
EndSection
```

注意：BusID 的格式非常重要！

在上例中，`01:00.0` 要写成 `1:0:0`。不过，有时转换更为复杂。`lspci` 输出中的数字是十六进制，而配置文件中的 BusID 使用十进制！因此，BusID 大于 9 时需要转换成十进制。例如，`lspci` 中的 `5e:00.0` 应写为 `PCI:94:0:0`。

Linux 4.8 及更高版本中 modprobe 报错“Could not insert 'nvidia': No such device”

在 Linux 4.8 中，尝试使用独立显卡时可能出现以下错误：

```
# modprobe nvidia -vv
modprobe: INFO: custom logging function 0x409c10 registered
```

```
modprobe: INFO: Failed to insert module '/lib/modules/4.8.6-1-
ARCH/extramodules/nvidia.ko.gz': No such device
modprobe: ERROR: could not insert 'nvidia': No such device
modprobe: INFO: context 0x24481e0 released
insmod /lib/modules/4.8.6-1-ARCH/extramodules/nvidia.ko.gz
# dmesg
...
NVRM: The NVIDIA GPU 0000:01:00.0 (PCI ID: 10de:139b)
NVRM: installed in this system is not supported by the 370.28
NVRM: NVIDIA Linux driver release.  Please see 'Appendix
NVRM: A - Supported NVIDIA GPU Products' in this release's
NVRM: README, available on the Linux driver download page
NVRM: at www.nvidia.com.
...
```

此问题由 Linux 内核中与 PCIe 电源管理有关的不正确提交引起（参见此 NVIDIA DevTalk 讨论）。

解决方法是在内核参数中添加 `pcie_port_pm=off`。注意，这会禁用所有设备的 PCIe 电源管理。

系统无法从挂起状态恢复

日志中可能出现以下信息：

```
kernel: nvidia-modeset: ERROR: GPU:0: Failed detecting connected display devices
kernel: nvidia-modeset: ERROR: GPU:0: Failed detecting connected display devices
kernel: nvidia-modeset: WARNING: GPU:0: Failure processing EDID for display device DELL
U2412M (DP-0).
kernel: nvidia-modeset: WARNING: GPU:0: Unable to read EDID for display device DELL
U2412M (DP-0)
kernel: nvidia-modeset: ERROR: GPU:0: Failure reading maximum pixel clock value for
display device DELL U2412M (DP-0).
```

根据此讨论，一种可能的解决方法是：

运行以下命令获取 `*version*` 字符串：

```
# strings /sys/firmware/acpi/tables/DSDT | grep -i 'windows ' | sort | tail -1
```

在引导加载程序配置中添加 `acpi_osi=! "acpi_osi=*version*"` 内核参数。

另一个可能的原因是使用了 nvidia-open包 软件包，参见：

- https://bbs.archlinux.org/viewtopic.php?pid=2047692
- https://github.com/NVIDIA/open-gpu-kernel-modules/issues/450
- https://github.com/NVIDIA/open-gpu-kernel-modules/issues/223
- https://github.com/NVIDIA/open-gpu-kernel-modules/issues/94

从挂起状态恢复后黑屏

如果遇到黑屏问题，并且日志包含：

```
archlinux kernel: NVRM: GPU at PCI:0000:08:00: GPU-926ecdb0-adb1-6ee9-2fad-52e7214c5011
archlinux kernel: NVRM: Xid (PCI:0000:08:00): 13, pid='<unknown>', name=<unknown>,
Graphi>
archlinux kernel: NVRM: Xid (PCI:0000:08:00): 13, pid='<unknown>', name=<unknown>,
Graphi>
archlinux kernel: NVRM: Xid (PCI:0000:08:00): 13, pid='<unknown>', name=<unknown>,
Graphi>
archlinux kernel: NVRM: Xid (PCI:0000:08:00): 13, pid='<unknown>', name=<unknown>,
Graphi>
```

```
archlinux kernel: NVRM: Xid (PCI:0000:08:00): 13, pid='<unknown>', name=<unknown>, Graphi>
```

需要按照 NVIDIA/提示和技巧#保留挂起后的显存 中的说明启用 NVIDIA 挂起、休眠及睡眠服务。

Xorg 多显示器配置中一个或多个显示器无法工作

如果一个或多个显示器在 Xorg 中没有任何输出，并且在 `nvidia-settings` 控制面板中显示为“Disabled”，则当前 Xorg 配置可能不兼容。 NVIDIA 专有驱动在正常工作时不应强制要求 X 配置文件。参见 NVIDIA#Xorg 配置。

只需移动或重命名 `/etc/X11/xorg.conf` 以及 `xorg.conf.d` 中与 NVIDIA 驱动有关的配置文件。Xorg 只会读取扩展名为 `.conf` 的文件。

```
$ mv /etc/X11/xorg.conf /etc/X11/xorg.conf.old
```

禁用旧的 NVIDIA Xorg 配置后，可以使用 `nvidia-xconfig` 生成适合当前硬件的新配置。不过，通常无需这样做，驱动也应当能够工作。

```
$ nvidia-xconfig
```

注意：`nvidia-xconfig` 会生成名为 `/etc/X11/xorg.conf.backup` 的备份文件，但多次运行 `nvidia-xconfig` 可能覆盖该文件。

崩溃与卡死

一般性崩溃

- 尝试禁用 GSP 固件。
- 尝试在 xorg.conf 中禁用 `RenderAccel`。
- 如果 Xorg 报告 `"conflicting memory type"` 或 `"failed to allocate primary buffer: out of memory"`，或者使用 nvidia-96xx 驱动时发生“Signal 11”崩溃，请在内核参数中添加 `nopat`。
- 如果 NVIDIA 编译器提示当前 GCC 版本与用于编译内核的 GCC 版本不同，请在 `/etc/profile` 中添加：

```
export IGNORE_CC_MISMATCH=1
```

- 如果全屏应用程序冻结或崩溃，请尝试在桌面环境设置中启用 `Display Compositing` 和 `Direct fullscreen rendering` 选项。

网格着色器支持不佳

此问题只出现在依赖网格着色器的新游戏中，例如《最终幻想 VII 重生》。即使使用最新的 NVIDIA 测试版驱动，游戏环境仍可能无法显示。

不过，在等待 NVIDIA 修复期间，可以借助 pyroveil，通过 SPIR-V 绕过该问题。

按照 GitHub 上的教程编译并安装该工具，然后设置 `PYROVEIL=1` 和 `PYROVEIL_CONFIG=*/path/to/pyroveil*/hacks/*ffvii-rebirth-nvidia*/pyroveil.json` 环境变量启动游戏。

OpenGL 应用程序中的图形异常、卡死和错误

较新的 CPU（Intel Sandy Bridge〔2011 年〕及更新型号，或 AMD Zen〔2017 年〕及更新型号）具有微操作缓存。由于缓存别名（Cache Aliasing）问题，微操作缓存可能与 NVIDIA 驱动的 OpenGL 功能发生冲突。通常可以在 BIOS 中禁用微操作缓存，但这会损失性能。禁用微操作缓存也有助于缓解 Xwayland 应用程序中最严重的图形异常，但无法完全解决问题。

更新或重启系统时发生内核恐慌

这是 NVIDIA 550 系列驱动中的一个已知错误。 目前原因尚不明确，但似乎只影响笔记本电脑。详情参见 BBS#293400。

解决方法是：如果硬件支持，切换到 nvidia-open-dkms包；否则改用 nvidia-535xx-dkmsAUR。

GSP 固件

GSP 固件从 2024 年 6 月发布的 NVIDIA 555 版驱动开始默认启用，已知可能引起各种问题，包括 Vulkan 故障和系统崩溃。

要禁用 GSP 固件，请为 `nvidia` 内核模块设置 `NVreg_EnableGpuFirmware=0` 模块参数。此方法仅适用于 NVIDIA 专有驱动；如需从开源驱动切换，请参见 NVIDIA#安装。

如有必要，不要忘记重新生成 initramfs。重启后新的内核模块选项才会生效。

与 VRR 和 TTY 切换有关的崩溃

如果在桌面环境与 TTY 之间切换时驱动崩溃，且 journal 日志包含：

```
archlinux kernel: [drm:nv_drm_atomic_commit [nvidia_drm]] *ERROR* [nvidia-drm] [GPU ID [...]] Flip event timeout
archlinux kernel: nvidia-modeset: ERROR: GPU:0: Idling display engine timed out: [...]
```

尝试通过显示器的 OSD 菜单禁用可变刷新率（VRR）功能。

另一种解决方法是对显示子系统隐藏驱动的 VRR（G-Sync / FreeSync）能力，以阻止应用程序或合成器启用 VRR：在内核参数中设置 `nvidia_modeset.conceal_vrr_caps=1`，以强制使用固定刷新率。

Xid 79：GPU 已从总线上掉线

如果在 journal 中发现此类崩溃，可能有多种原因，包括电源故障、线缆问题或 PCI 连接问题。

不过，也有人将其报告为 NVIDIA 驱动问题，其中提到一种通过限制显存时钟频率来解决问题的方法。

首先运行以下命令，获取可用的显存时钟频率：

```
$ nvidia-smi -q -d SUPPORTED_CLOCKS | grep Memory
        Memory                                            : 9501 MHz
        Memory                                            : 9251 MHz
        Memory                                            : 5001 MHz
        Memory                                            : 810 MHz
        Memory                                            : 405 MHz
```

然后使用 `--lock-memory-clocks`（或 `-lmc`）选项，将最低频率设为支持的最小值，最高频率设为低于最大值的下一档，并按 `*min*,*max*` 的格式指定。例如：

```
# nvidia-smi --lock-memory-clocks 405,9251
```

这会设定允许的时钟频率范围，避免设备始终被强制运行在最高频率。也可以将范围上限设置为最大值，但即使显卡未使用，也会增加耗电量。

注意：根据官方文档，`--lock-memory-clocks` 不适用于采用 NVIDIA Hopper 架构的设备。如果使用的是该架构，请改用 `--lock-memory-clocks-deferred`（或 `-lmcd`），该选项只接受一个目标值，而不是一个范围。

显示问题

避免 Xorg 画面撕裂

注意：

- 据报告，这可能降低部分 OpenGL 应用程序的性能，并导致 WebGL 问题。它还会显著延长驱动在负载结束后降低时钟频率所需的时间（NVIDIA 支持讨论）。
- 已知 `ForceFullCompositionPipeline` 会导致一些使用 NVIDIA 535 驱动、通过 Proton 运行的 Vulkan 游戏无法正常工作。
- 已知 `ForceCompositionPipeline` 可能导致部分全屏应用立即冻结，使系统无响应，直到切换到其他窗口。一个解决方法是在 Gamescope 中以无边框窗口模式（`-b`）运行程序，并保持屏幕上至少还有一个其他元素（可以是任务栏或其他图形应用程序）。

NVIDIA 驱动会根据情况使用合成管线，为图像显示做准备。不要将其与合成管理器混淆：后者负责窗口呈现，也可以通过 VSync 缓解画面撕裂。默认情况下，只有当前屏幕变换需要时，才会启用合成管线。该管线会优先使用 GPU 的专用显示引擎，无法使用时则采用通用图形引擎。可以使用以下选项进行控制：

- `ForceCompositionPipeline` 强制使用合成管线（详见#多显示器）。

- `ForceFullCompositionPipeline` 强制使用合成管线，并强制由图形引擎完成全部处理。

设置 `ForceFullCompositionPipeline=On` 可以避免 Xorg 中的画面撕裂。要测试此选项是否有效，请运行：

```
$ nvidia-settings --assign CurrentMetaMode="nvidia-auto-select +0+0
{ ForceFullCompositionPipeline = On }"
```

或者，在 X Server Display Configuration 菜单中点击 Advanced 按钮，选择 Force Composition Pipeline 或 Force Full Composition Pipeline，然后点击 Apply。

要永久应用此更改，必须将选项添加到 Xorg 配置文件的 `"Screen"` 段。同时应在驱动配置中启用 `TripleBuffering` 并禁用 `AllowIndirectGLXProtocol`。配置示例如下：

```
/etc/X11/xorg.conf.d/20-nvidia.conf
Section "Device"
        Identifier "NVIDIA Card"
        Driver     "nvidia"
        VendorName "NVIDIA Corporation"
        BoardName  "GeForce GTX 1050 Ti"
EndSection

Section "Screen"
    Identifier     "Screen0"
    Device         "Device0"
    Monitor        "Monitor0"
    Option         "ForceFullCompositionPipeline" "on"
    Option         "AllowIndirectGLXProtocol" "off"
    Option         "TripleBuffer" "on"
EndSection
```

如果没有 Xorg 配置文件，可以使用 `nvidia-xconfig` 为当前硬件生成配置（参见 NVIDIA#自动配置），再将其从 `/etc/X11/xorg.conf` 移动到推荐位置 `/etc/X11/xorg.conf.d/20-nvidia.conf`。

注意：`nvidia-xconfig` 在 `20-nvidia.conf` 中生成的许多选项会由驱动自动设置，并非必需。如果仅希望启用合成管线，只需要保留包含 `Identifier` 和 `Option` 的 `"Screen"` 段，其他段可以删除。

多显示器

在多显示器配置中，需要为每个显示器单独指定 `ForceCompositionPipeline=On`。例如：

```
$ nvidia-settings --assign CurrentMetaMode="DP-2: nvidia-auto-select +0+0
{ForceCompositionPipeline=On}, DP-4: nvidia-auto-select +3840+0
{ForceCompositionPipeline=On}"
```

否则，`nvidia-settings` 命令将禁用第二台显示器。

以上命令适用于连接到 DP-2 和 DP-4 的两台 3840x2160 显示器。需要通过导出 `xorg.conf` 读取正确的 `CurrentMetaMode`，并为每个显示器附加

`ForceCompositionPipeline`。设置 `ForceCompositionPipeline` 只会影响目标显示器。

可以使用 `--query` 查询当前显示器名称及其偏移量：

```
$ nvidia-settings --query CurrentMetaMode
```

提示：在使用不同型号显示器的多显示器配置中，显示器的刷新率可能略有差异。如果驱动启用了 V-Sync，它只会同步到其中一个刷新率，从而可能导致其他显示器出现画面撕裂。因此最好同步到主要使用的显示器。可以在 `~/.nvidia-settings-rc` 中通过 `0/XVideoSyncToDisplayID=` 配置，或者安装 nvidia-settings包 并使用图形界面进行设置。

从挂起或休眠恢复后屏幕显示异常

如果外部显示器在挂起或休眠后无法唤醒，也适用本节。

参见 NVIDIA/提示和技巧#保留挂起后的显存。

使用 GDM 服务时从挂起恢复后屏幕损坏的问题，已于 515.43.04 版驱动中修复。

屏幕显示异常：“六屏”问题

部分使用 GeForce GT 100M 系列显卡的用户发现，启动 X 后画面被分成六块，且分辨率被限制为 640x480。Quadro 2000 和高分辨率显示器上也有人报告过同样的问题。

要解决此问题，请在 `Device` 段中启用 `NoTotalSizeCheck` 模式验证选项：

```
Section "Device"
 ...
 Option "ModeValidation" "NoTotalSizeCheck"
 ...
EndSection
```

使用 nvidia-470 时文本和图标不可见

GTK4 的一次更新给依赖 nvidia-470 旧显卡驱动的用户带来了问题：更新后文本和图标会随机消失，只有鼠标悬停到窗口上时才会重新出现。

解决方法是指定渲染器：

```
/etc/environment
GSK_RENDERER=vulkan
```

修复 GNOME Shell 从睡眠恢复后的图形异常

如果从睡眠恢复后 GNOME Shell 中出现异常字体或奇怪的图形故障，请尝试设置以下内核参数以启用电源管理：

```
nvidia.NVreg_DynamicPowerManagement=0x02
```

更多信息：https://download.nvidia.com/XFree86/Linux-x86_64/575.64/README/dynamicpowermanagement.html

显示器关闭或恢复时系统卡死

如果桌面环境通过 DPMS 关闭显示器后，或者系统从挂起恢复时立即卡死或崩溃，并且 dmesg/journal 中出现类似以下 GSP 超时记录：

```
NVRM: _kgspLogXid119: ******************************** GSP Timeout
********************************
NVRM: Xid (PCI:0000:01:00): 119, Timeout after 6s of waiting for RPC response from GPU0
GSP! Expected function 76 (GSP_RM_CONTROL) sequence 1321
```

在这些电源状态切换过程中，GPU 可能会降至不稳定的低时钟频率状态。

解决方法：使用 nvidia-smi 锁定更高的 GPU／显存最低时钟频率。

前提条件：

启用并启动 `nvidia-persistenced.service`。

查询支持的时钟频率（根据结果选择有效的最小值／最大值组合）：

```
nvidia-smi -q -d SUPPORTED_CLOCKS
nvidia-smi -q -d CLOCK
```

设置最低时钟频率（以下为示例值，请根据 GPU 支持的最高频率调整）：

```
nvidia-smi -lgc 800,2100
nvidia-smi -lmc 800,10000
```

测试 DPMS 和挂起／恢复，检查问题是否得到解决。若要还原设置：

```
nvidia-smi -rgc
nvidia-smi -rmc
```

如需通过 systemd 永久配置，可创建 `nvidia-clocks.service` 之类的单元：

```
/etc/systemd/system/nvidia-clocks.service
[Unit]
Description=Set NVIDIA GPU minimum clocks to avoid  GSP timeouts
Requires=nvidia-persistenced.service
After=nvidia-persistenced.service

[Service]
Type=oneshot
ExecStart=/usr/bin/nvidia-smi -lgc 500,2100
ExecStart=/usr/bin/nvidia-smi -lmc 500,10000
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
```

可以将最低时钟频率设置得比上面提到的 800 更低，以减少空闲功耗；但要注意，设置得太低可能导致问题再次出现。

然后启用并启动 `nvidia-clocks.service`。

性能问题

安装新版驱动后性能下降

**本文或本章节的事实准确性存在争议。**

原因： 应尝试找到根本原因（检查 dmesg、dkms 日志等），而不是盲目降级驱动。（在 Talk:NVIDIA/故障排除 中讨论）

如果 FPS 比旧版驱动低，请检查是否启用了直接渲染（`glxinfo` 包含在 mesa-utils 包 软件包中）：

```
$ glxinfo | grep direct
```

如果命令输出：

```
direct rendering: No
```

一种可能的解决方法是退回到先前安装的驱动版本，然后重新启动。

Xorg 严重卡顿

**本文或本章节的事实准确性存在争议。**

原因： 根据一位 NVIDIA 开发者的说法，该问题并非 GNOME 独有，而相关讨论中的其他评论也没有提到多显示器配置。（在 Talk:NVIDIA/故障排除 中讨论）

Mutter 的一个常见问题是：动画、视频播放和游戏可能导致 Xorg 桌面严重卡顿。

参见 NVIDIA/提示和技巧#保留挂起后的显存。

这应该能够解决问题。如果仍未解决，可能没有完善的解决办法。可以尝试添加以下选项来缓解问题：

```
/etc/environment
CLUTTER_DEFAULT_FPS=YOUR_MAIN_DISPLAY_REFRESHRATE
__GL_SYNC_DISPLAY_DEVICE=YOUR_MAIN_DISPLAY_OUTPUT_NAME
```

在 NVIDIA Settings 中关闭 Sync to VBlank 和 Allow flipping，并使用 `--load-config-only` 参数配置 NVIDIA Settings 在启动时自动运行。 即使这样，桌面仍可能存在卡顿，尤其是第二台（或第三台）显示器，但表现应当明显改善。

400 系列显卡出现 CPU 使用率峰值

如果使用 400 系列显卡时 CPU 使用率间歇性飙升，可能是由于 PowerMizer 不断切换 GPU 时钟频率。将 PowerMizer 设置从 Adaptive 切换到 Performance，并在 Xorg 配置的 `Device` 段中添加以下内容：

如果使用台式机：

```
 Option "RegistryDwords" "PowerMizerEnable=0x1; PerfLevelSrc=0x2222;
PowerMizerDefaultAC=0x1"
```

如果使用笔记本电脑：

```
 Option "RegistryDwords" "PowerMizerEnable=0x1; PerfLevelSrc=0x3322;
PowerMizerDefaultAC=0x1"
```

其他问题

应用程序启动时出现 Vulkan 错误

(https://wiki.archlinuxcn.org/wzh/File:Tango-inaccurate.png)**本文或本章节的事实准确性存在争议。**

原因： 需要其他用户确认（在 Talk:NVIDIA/故障排除 中讨论）

运行需要 Vulkan 加速的应用程序时，如果出现以下错误：

```
Vulkan call failed: -4
```

请尝试删除 `~/.nv` 或 `~/.cache/nvidia` 目录。

HDMI 没有音频

有时运行以下命令时，无法看到 NVIDIA HDMI 音频设备：

```
$ aplay -l
```

在一些新设备上，NVIDIA GPU 的音频芯片在启动时处于禁用状态。更多信息参见 NVIDIA 网站及论坛帖子。

需要重新加载 NVIDIA 设备并启用音频。首先确保 GPU 已开启（笔记本电脑／Bumblebee 用户尤其应注意），且 X 没有运行在该 GPU 上，因为下面的操作会重置显卡：

```
# setpci -s 01:00.0 0x488.l=0x2000000:0x2000000
# rmmod nvidia-drm nvidia-modeset nvidia
# echo 1 > /sys/bus/pci/devices/0000:01:00.0/remove
# echo 1 > /sys/bus/pci/devices/0000:00:01.0/rescan
# modprobe nvidia-drm
# xinit -- -retro
```

如果 TTY 运行在 NVIDIA 显卡上，请将这些命令写入脚本，以免操作过程中失去画面输出。

某些情况下背光无法关闭

默认情况下，DPMS 应根据超时设置或通过 xset 关闭背光。然而，可能由于 NVIDIA 专有驱动的错误，最终只会显示黑屏，并未真正进入节能状态。在该问题修复之前，可以尝试以 root 身份使用 `vbetool` 作为变通方法。

安装 vbetool包 软件包。

可以按需关闭显示器，按任意键后重新打开背光：

```
vbetool dpms off && read -n1; vbetool dpms on
```

也可以通过 xrandr 禁用并重新启用显示器输出，无需 root 权限。

```
xrandr --output DP-1 --off; read -n1; xrandr --output DP-1 --auto
```

HardDPMS

专有驱动 415 引入了一个名为 HardDPMS 的新功能。据部分用户报告，它解决了通过 DisplayPort 连接的显示器无法进入挂起状态的问题。 自 440.26 版起，此功能默认启用。如果使用更早的驱动，可以在 `Device` 或 `Screen` 段中设置 `HardDPMS` 选项。例如：

```
/etc/X11/xorg.conf.d/20-nvidia.conf
Section "Device"
    ...
    Option          "HardDPMS" "true"
    ...
EndSection

Section "Screen"
    ...
    Option          "HardDPMS" "true"
    ...
EndSection
```

`HardDPMS` 会受 `BlankTime` 等屏幕保护程序设置触发。以下 `ServerFlags` 配置会在显示器空闲 10 分钟后使其进入挂起状态：

```
/etc/X11/xorg.conf.d/20-nvidia.conf
Section "ServerFlags"
    Option      "BlankTime" "10"
EndSection
```

xrandr BadMatch 错误

如果尝试使用 xrandr 配置 DELL U2515H 等 WQHD 显示器，并且 `xrandr --addmode` 返回 `X Error of failed request: BadMatch`，可能是因为 NVIDIA 专有驱动将 HDMI 输出的最大像素时钟限制在 225 MHz 或更低。若要使用显示器的最高分辨率，需要安装 nouveau 驱动。可以通过设置内核参数 `nouveau.hdmimhz=297`（或 `330`），强制 nouveau 使用指定的像素时钟频率。

另一种可能是显示器的 EDID 信息有误。参见 #覆盖 EDID。

还有一种可能是，当前 NVIDIA 驱动默认只允许 EDID 明确报告的显示模式；但用户有时需要 EDID 未提供的刷新率或分辨率（即使 EDID 本身没有错误）。

在这种情况下，可以在 `xorg.conf` 中添加一个选项，以允许非 EDID 模式：

```
Section "Device"
    Identifier      "Device0"
    Driver          "nvidia"
    VendorName      "NVIDIA Corporation"
...
    Option          "ModeValidation" "AllowNonEdidModes"
...
EndSection
```

此选项可以针对各个输出单独设置。更多信息参见 README 附录 B：X 配置选项。

覆盖 EDID

参见 内核级显示模式设置#强制设置显示模式与 EDID、Xrandr#疑难排除 和 Qnix QX2710#Fixing X11 with Nvidia。

nvidia-settings 图形界面无法超频

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：内容重复，“无法工作”的描述不够明确（在Talk:NVIDIA/故障排除讨论）

解决方法是在启用超频后，通过 nvidia-settings 命令行界面查询和设置相关变量（参见 NVIDIA/提示和技巧#在 nvidia-settings 中启用超频，以及 nvidia-settings(1)）。

查询所有变量的示例：

```
nvidia-settings -q all
```

将 PowerMizerMode 设置为优先使用性能模式的示例：

```
nvidia-settings -a [gpu:0]/GPUPowerMizerMode=1
```

将风扇速度固定在 21% 的示例：

```
nvidia-settings -a [gpu:0]/GPUFanControlState=1 -a [fan:0]/GPUTargetFanSpeed=21
```

一次设置多个变量的示例（GPU 超频 50 MHz、显存超频 50 MHz、GPU 电压增加 100 mV）：

```
nvidia-settings -a GPUGraphicsClockOffsetAllPerformanceLevels=50 -a
GPUMemoryTransferRateOffsetAllPerformanceLevels=50 -a GPUOverVoltageOffset=100
```

超频失败并出现 Unknown Error

如果以非 root 用户身份运行 Xorg 并尝试对 NVIDIA GPU 超频，可能出现类似以下错误：

```
$ nvidia-settings -a "[gpu:0]/GPUGraphicsClockOffset=10"
ERROR: Error assigning value 10 to attribute 'GPUGraphicsClockOffset' (trinity-
zero:1[gpu:0]) as specified in assignment
        '[gpu:0]/GPUGraphicsClockOffset=10' (Unknown Error).
```

为避免此问题，必须以 root 身份运行 Xorg。详情参见 Xorg#没有 root 权限的 Xorg。

功耗

检查驱动的使用情况：

```
# lsof /dev/nvidia*
kwin_wayl  867      user    17u    CHR    195,0      0t0  418 /dev/nvidia
kwin_wayl  867      user    18u    CHR    195,0      0t0  418 /dev/nvidiactl
```

如果已在内核模块上配置节能功能：

```
$ grep . /sys/bus/pci/devices/0000:01:00.0/power/*
/sys/bus/pci/devices/0000:01:00.0/power/control:auto
/sys/bus/pci/devices/0000:01:00.0/power/runtime_active_time:445933
/sys/bus/pci/devices/0000:01:00.0/power/runtime_status:active
/sys/bus/pci/devices/0000:01:00.0/power/runtime_suspended_time:1266
/sys/bus/pci/devices/0000:01:00.0/power/wakeup:disabled
```

```
# rmmod nvidia_drm
$ grep . /sys/bus/pci/devices/0000:01:00.0/power/*
/sys/bus/pci/devices/0000:01:00.0/power/control:auto
/sys/bus/pci/devices/0000:01:00.0/power/runtime_active_time:461023
/sys/bus/pci/devices/0000:01:00.0/power/runtime_status:suspended
/sys/bus/pci/devices/0000:01:00.0/power/runtime_suspended_time:1064192
/sys/bus/pci/devices/0000:01:00.0/power/wakeup:disabled
```

测试软件 GL 渲染

NVIDIA 二进制驱动不会遵循 Mesa 环境变量 `LIBGL_ALWAYS_SOFTWARE=1`，但可以通过设置以下环境变量，使 libglvnd 和 EGL 使用 Mesa：

```
__GLX_VENDOR_LIBRARY_NAME=mesa
__EGL_VENDOR_LIBRARY_FILENAMES=/usr/share/glvnd/egl_vendor.d/50_mesa.json
```

这样就会使用 Mesa libgl 进行软件 GLX 和 EGL 渲染，有助于测试某个错误是否与 NVIDIA GL 库有关。

刷新率被限制为 120 Hz

较新版本的驱动（550xx 之后）似乎会在 8 bpc 输出时浪费带宽，可能使信号超出接口规范限制，导致原本在接口带宽范围内的更高刷新率模式无法使用。 在内核参数中添加 `nvidia-modeset.hdmi_deepcolor=0`，或通过 modprobe 设置该选项。 但请注意，HDR 显示器需要深色彩（Deep Color）功能。

Wayland 下 HDMI 60 Hz 输出的色彩空间错误

某些情况下（例如使用 HDMI 连接、刷新率为 60 Hz 的 GTX 1660 Super 显卡），驱动似乎会错误判断输出色彩空间，使颜色看起来比正常情况更暗。由于 Wayland 下没有简单的方法显式设置色彩空间，可以在内核参数中添加 `nvidia-modeset.debug_force_color_space=2`，或通过 modprobe 设置该选项，作为临时解决方法。

2.20 Intel 图形处理器

难度 高级 | **篇幅** 46.7 KB | **所属** 图形

本节将讲到

- 安装
- 加载
- Xorg 配置
- 基于内核模块参数的选项
- 技巧和窍门

由于 Intel 提供并维护开源驱动程序，因此 Intel 显卡基本上是即插即用的。

有关 Intel GPU 型号以及相应芯片组和 CPU 的全面列表，请参阅 Wikipedia:list of Intel graphics processing units 和 Gentoo:Intel#Feature support。

注意：

- 开源驱动不支持基于 PowerVR 架构的图形卡 (比如GMA 3600 系列)
- Intel 第 *N* 代硬件不代表 CPU 的代数，它表示不同于 CPU 代数的 GPU 的代数
- 参见 Xorg#驱动安装 来识别你的显卡

安装

- 安装以下任一软件包，它提供用于 3D 加速的 DRI 驱动程序。
- mesa包 是最新的 Mesa 软件包，其中包括用于 Intel 第 3 代及更高版本硬件的现代 Gallium3D 驱动程序。这是推荐选择。
- mesa-amber包 是旧版 Mesa 软件包，包括用于第 2 代至第 11 代硬件的经典（非 Gallium3D）驱动程序。此驱动程序对于第 7 代和较旧的硬件具有更好的性能和稳定性，但是不再维护。
- 为支持 32 位程序，请同时从 multilib 仓库中安装 lib32-mesa包 或 lib32-mesa-amber包 软件包。
- 对于 DDX 驱动支持（可提供对 Xorg 的 2D 加速），安装以下任一软件包：
- 包含在 xorg-server包 软件包内的 *modesetting* 驱动是对于第 4 代及以上硬件的推荐选择。它通过 *glamor* 使用 DRI 来实现加速。
- xf86-video-intel包 软件包为第 2 代至第 9 代硬件提供了旧的 Intel DDX 驱动。通常不建议使用此软件包，请参见下面的注释。
- 为支持 Vulkan （Haswell 及更新架构，对于老更老架构的支持是不完整或缺失的），安装 vulkan-intel包 软件包。对于 32 位的 Vulkan 支持，安装 lib32-vulkan-intel包 软件包。

另请参见硬件视频加速.

注意：

- 有的文档 (Debian & Ubuntu, Fedora, KDE) 推荐不安装 xf86-video-intel包 驱动程序，而是回落到给第四代及更新 GPU (2006年以来的GMA 3000及以上显卡) 使用的 modesetting 驱动程序。请参考 、, Xorg#安装 和 modesetting(4). 然而，modesetting 驱动程序可能导致比如 XFCE 上的屏幕撕裂和鼠标抖动、切换Chromium中虚拟的桌面产生伪影 和mpv中的vsync抖动/视频抖动 等问题
- xf86-video-intel包 软件包未正确支持第 11 代及更新硬件，会导致硬件加速的缺失和可能会导致 Plasma Desktop 几乎不可用的渲染问题。参见
- 有多起报告（参见 ）称当安装有 xf86-video-intel包 软件包时，整个图形栈会卡死，甚至切换至不同的虚拟控制台（通过按下 `Ctrl+Alt+F*n*`）都无法解决问题，只有通过使用 SysRq 杀死用户进程可以解决问题

加载

Intel 内核模块应会在系统引导时自动加载。

如果它没有自动加载，请尝试：

- 由于 Intel 需要内核模式设置，确保您**没有**在内核参数中添加 `nomodeset`
- 请同时确认您没有在 `/etc/modprobe.d/` 或 `/usr/lib/modprobe.d/` 中把 Intel 列入 modprobe 的黑名单导致禁用了 Intel。

启用 KMS 早启动

`i915` 驱动支持内核级显示模式设置 (KMS)，并且从 mkinitcpio v32 开始启用，因为 `kms` 钩子被默认包含。对于其他配置方式，请参考 Kernel_mode_setting#KMS_早启动获得有关如何在启动过程中第一时间启用KMS的说明。

启用 GuC / HuC 固件加载

**本文或本章节的事实准确性存在争议。**

原因： 尽管 Intel 的文档没有说，但是 Tiger Lake 和 Rocket Lake GPU 可能实际上支持 `enable_guc=3`, 并且默认设置了 `enable_guc=1`.（在 Talk:Intel graphics#TGL/RKL GuC Submission 中讨论）

从第 9 代硬件 (Skylake 及以后的架构) 开始, Intel GPU 都包含了一个用于提供以下功能的图形微控制器 「*Graphics micro (μ) Controller*」 (GuC) :

- 安装 intel-media-driver包 软件包来启用硬件视频加速后，能够把一些多媒体解码功能从 CPU 分载（offload）到 *HEVC/H.265 微控制器 （micro (μ) Controller）* (HuC) 上。该功能于第 9 代推出。
- 使用 GuC 进行调度、上下文提交（context submission）和电源管理。该功能通过第 12 代硬件于 Alder Lake-P（移动端）推出。

为了使用这些功能，您必须加载 GuC 固件。对于 HuC 支持，一些视频特性(如 SKL 低功耗编码模式下的 CBR 速率控制)也需要加载 HuC 固件才能实现。 GuC 和 HuC 的固件文件都包含在 linux-firmware包 软件包中。

GuC 功能由 `i915.enable_guc` 内核参数控制，用法如下：

enable_guc 的值	GuC Submission	HuC Firmware Loading	作为平台默认值	支持的平台
0	否	否	Tiger Lake, Rocket Lake, 和 Pre-Gen12	All
1	是	否	–	Alder Lake-P (Mobile) 及更新
2	否	是	Alder Lake-S (Desktop)	Gen9 及更新

enable_guc 的值	GuC Submission	HuC Firmware Loading	作为平台默认值	支持的平台
3	是	是	Alder Lake-P (Mobile) 及更新	第 9.5 代或更新（某些程度上更好）

如果 GuC submission 或者 HuC 固件加载并未在您的 GPU 上默认启用，您可以手动启用它。

警告：即使您的显卡不支持相关功能，手动启用 GuC/HuC 固件加载也可能会污染内核。此外，启用 GuC/HuC固件加载可能会导致某些系统出现问题；如果您遇到冻结（例如，从休眠恢复后），请禁用它。

首先确保您已经安装了 linux-firmware包 软件包。

配置 `i915.enable_guc` 内核参数，比如：

```
/etc/modprobe.d/i915.conf
options i915 enable_guc=2
```

然后重建 initramfs，在下一次启动时您可以通过 dmesg 命令来确认 GuC 和 HuC 都已被启用。

```
# dmesg
[30130.586970] i915 0000:00:02.0: [drm] GuC firmware i915/icl_guc_33.0.0.bin version 33.0 submission:disabled
[30130.586973] i915 0000:00:02.0: [drm] HuC firmware i915/icl_huc_9.0.0.bin version 9.0 authenticated:yes
```

如果您的显卡不支持它们，你会看到：

```
# dmesg
[    0.571339] i915 0000:00:02.0: [drm] Incompatible option enable_guc=2 - GuC is not supported!
[    0.571340] i915 0000:00:02.0: [drm] Incompatible option enable_guc=2 - HuC is not supported!
```

或者，使用以下方法进行检查：

```
# cat /sys/kernel/debug/dri/0/gt/uc/guc_info
# cat /sys/kernel/debug/dri/0/gt/uc/huc_info
```

注意：从 Linux 4.20.11 起，当 GuC/HuC 被启用时，不支持通过设置 `enable_gvt=1` 来使用 GVT-g 图形虚拟化 功能。i915模块将无法初始化，如系统日志所示：

```
# journalctl
... kernel: [drm:intel_gvt_init [i915]] *ERROR* i915 GVT-g loading failed due to Graphics virtualization is not yet supported with GuC submission
... kernel: i915 0000:00:02.0: [drm:i915_driver_load [i915]] Device initialization failed (-5)
... kernel: i915: probe of 0000:00:02.0 failed with error -5
... kernel: snd_hda_intel 0000:00:1f.3: failed to add i915 component master (-19)
```

请注意相关警告并非致命，正如 中解释的那样：

```
# journalctl -b
... kernel: i915 0000:00:02.0: Direct firmware load for i915/gvt/vid_0x8086_did_0x5916_rid_0x02.golden_hw_state failed with error -2
```

Xorg 配置

通常不需要为了运行 Xorg 去配置任何东西。

但是如果需要使用一些驱动选项或是 Xorg 未正常启动，您可以创建一个 Xorg 配置文件。

使用 modesetting 驱动

如果您已经安装了 xf86-video-intel包 软件包但是想要显式加载 modesetting 驱动而不是让 DDX 驱动有着更高的优先级，比如试着去比较它们：

```
/etc/X11/xorg.conf.d/20-intel.conf
Section "Device"
  Identifier "Intel Graphics"
  Driver "modesetting"
EndSection
```

使用 Intel 驱动

注意：以下操作需要安装 xf86-video-intel包 软件包。

创建一个与下述类似的 Xorg 配置文件：

```
/etc/X11/xorg.conf.d/20-intel.conf
Section "Device"
  Identifier "Intel Graphics"
  Driver "intel"
EndSection
```

您可以在 `Driver "intel"` 下方开新行添加额外的选项。请参考 intel(4) 手册页以获得完整的选项列表。

注意：您可能需要添加不止一段 `device` 配置，本文将在需要的地方指出。

AccelMethod

创建配置文件时，您可能需要指定 `Option "AccelMethod"` 。传统的选项包括 `UXA`, `SNA` (默认) 和 `BLT`.

如果您在使用默认选项 `SNA` 时遇到问题 (比如花屏、文字损坏等)，尝试改用`UXA` 。只需要在配置文件中添加如下一行即可：

```
Option      "AccelMethod"  "uxa"
```

另请参阅 intel(4) § CONFIGURATION DETAILS 手册文件中的 AccelMethod 选项。

对于较新的 GPU 使用 Intel DDX 驱动

从第 8 代 Intel GPU 开始（Broadwell），需要 Iris Mesa 驱动：

```
Option      "DRI"  "iris"
```

禁用 TearFree，TripleBuffer，SwapbuffersWait

如果你在使用组合器（在比如 GNOME，KDE Plasma，Xfce 等现代桌面环境中是默认的），那么 TearFree，TripleBuffer 和 SwapbuffersWait 通常可以禁用以提高性能和省电。

```
Option      "TearFree"        "false"
Option      "TripleBuffer"    "false"
```

```
Option      "SwapbuffersWait" "false"
```

基于内核模块参数的选项

`i915` 内核模块允许您通过内核模块参数来配置选项。其中一些选项会影响到省电。

您可以使用以下命令生成所有选项的列表以及简短说明和默认值：

```
$ modinfo -p i915
```

若要检查当前启用了哪些选项，请运行

```
# systool -m i915 -av
```

您将注意到，许多选项默认为 -1，说明芯片的省电状态都为默认值。然而，您可以通过修改内核模块参数来配置更积极的省电策略。

注意：Diverting from the defaults will mark the kernel as tainted from Linux 3.18 onwards. This basically implies using other options than the per-chip defaults is considered experimental and not supported by the developers.

Framebuffer 压缩 (enable_fbc)

启用 Framebuffer 压缩 (FBC) 功能能够减少电源消耗，同时减少刷新屏幕所需的内存带宽。

如果您的硬件支持，此功能将被自动启用。您可以通过该命令来检查是否启用：

```
$ modinfo i915 | grep enable_fbc
parm:           enable_fbc:Enable frame buffer compression for power savings (default:
-1 (use per-chip default)) (int)
```

如果 parm 被设为 `-1`，那您不需要做任何事。如果在其他情况下要强制启用 FBC，请在内核参数中添加 `i915.enable_fbc=1` 或在 `/etc/modprobe.d/i915.conf` 中进行如下配置：

```
/etc/modprobe.d/i915.conf
options i915 enable_fbc=1
```

注意：在 Sandy Bridge (第6代)之前的几代 Intel GPU 上，帧缓冲（Framebuffer）压缩可能不可靠或不可用。 这导致系统日志中可能会记录类似如下信息：

```
kernel: drm: not enough stolen space for compressed buffer, disabling.
```

在Sandy Bridge之前的CPU上启用帧缓冲区压缩会导致无休止的错误消息：

```
# dmesg
[ 2360.475430] [drm] not enough stolen space for compressed buffer (need 4325376
bytes), disabling
[ 2360.475437] [drm] hint: you may be able to increase stolen memory size in the BIOS
to avoid this
```

解决方案是禁用帧缓冲压缩，这将增加些微功耗（约0.06W）。为了禁用它，请在内核参数中添加 `i915.enable_fbc=0`。 请参阅此处 了解更多关于禁用帧缓冲的影响。

Fastboot

注意：Skylake 和更新的架构，会在默认情况下启用此参数。以及 Bay- 和 Cherry-Trail (VLV/CHV) 自 Linux 5.1 版本以来也会启用。

Intel Fastboot 的目标是在 Xorg 启动之前将帧缓冲区保留为 BIOS 或引导加载程序设置的内容，以避免出现屏闪。

要在未默认启用快速启动的平台上强制启用快速启动，请设置 `i915.fastboot=1` 内核参数，或在 `/etc/modprobe.d/i915.conf` 里作如下设置：

```
/etc/modprobe.d/i915.conf
options i915 fastboot=1
```

Intel GVT-g 图形虚拟化支持

请参考 Intel GVT-g 以了解详细信息。

启用性能支持

从第六代处理器 (Sandy Bridge 及更新的架构) 开始，Intel GPU 提供性能计数器，用于向驱动程序提供内部性能数据。 驱动程序和硬件寄存器将此基础结构称为 *Observation Architecture* (内部称为 "OA") ，但英特尔的文档一般普遍将此功能称为提供*可观测性能计数器（Observability Performance Counters）* .

默认情况下，只有使用 CAP_SYS_ADMIN（相当于root）或 CAP_PERFMON 能力（Capabilities）运行的程序才能使用 OA 。 大多数应用程序将在没有这两种情况下运行，从而导致以下警告：

```
MESA-INTEL: warning: Performance support disabled, consider sysctl
dev.i915.perf_stream_paranoid=0
```

若要在不使用能力(或 root)的情况下启用性能支持，请按 sysctl 中的描述设置内核参数。

警告：`perf_event_paranoid` 系列选项默认为限制访问，因为允许应用程序访问性能数据存在风险 。 话虽如此， `dev.i915.perf_stream_paranoid` 只会影响对 GPU 性能计数器的访问，这比访问 CPU 架构执行上下文寄存器之类的带来的风险要小。

技巧和窍门

设置缩放模式

这对于某些全屏应用程序很有用：

```
$ xrandr --output LVDS1 --set PANEL_FITTING param
```

其中 `*param*` 可以为：

- `center`: 分辨率将完全保持原设置，不会进行缩放
- `full`: 缩放分辨率，以便使用整个屏幕
- `full_aspect`: 将分辨率缩放到可能的最大值，但保持纵横比。

如果它不起作用，请尝试：

```
$ xrandr --output LVDS1 --set "scaling mode" param
```

其中 *param* 可以为 "Full"、 "Center" 或 "Full aspect"。

注意：此选项目前不适用于外接显示器 (比如 VGA, DVI, HDMI, DP).

GMA 4500上的硬件加速 H.264 解码

对于某些 GMA 4500 系列的 GPU，libva-intel-driver包 软件包只提供对 MPEG-2 解码的硬件加速，而并不包括 H.264 解码。要检查这是否对您的 GPU 有影响，请同时安装驱动和 libva-utils包 软件包，并检查 `vainfo` 命令的输出结果中是否包含以 `VAProfileH264` 开头的项目。

对于 H.264 的解码支持是在另一个 g45-h264 分支中独立维护的。您可以安装 libva-intel-driver-g45-h264AUR 软件包来使用它。然而，要注意该支持是实验性的，并且其开发工作已被放弃。在GMA 4500系列GPU上使用VA-API与此驱动程序能够帮助 CPU 分载（offload），但可能无法实现像非硬件加速那样平滑的播放。使用 mplayer 进行的测试表明，使用 vaapi 播放 H.264编码的1080p 视频可以将 CPU 负载减半(与 XV overlay相比)，但是播放非常卡顿，而720p 播放效果相当不错 。这与其他人的经历相印证 。在 BIOS 中将预先分配的显存大小设置得更高，可以获得更好的硬件解码播放效果。这样做了之后即使是1080p h264视频也可以流畅播放。同时使用 mpv-gitAUR 、ffmpeg-gitAUR 和 libva-intel-driver-g45-h264AUR 也可以流畅播放 (1080p/720p)视频。通过 MPV 和 Firefox 插件“发送到 MPV 播放器”，可以观看硬件加速的 YouTube 视频。

覆写报告的 OpenGL 版本

`MESA_GL_VERSION_OVERRIDE` 环境变量可以为任何应用程序覆写报告的 OpenGL 版本。比如，设置 `MESA_GL_VERSION_OVERRIDE=4.5` 环境变量将会把 OpenGL 版本报告为 4.5.

注意：您可以使用此变量报告任何已知的 OpenGL 版本，即使 GPU 不支持它。一些应用程序可能不再崩溃，而一些可能反而开始崩溃。所以您可能不希望全局设置这个变量。

监控

**本文或本章节可能需要合并到Hardware video acceleration#Verification。**

附注： This overlaps the content at the previously linked page and would probably be a better fit in a generic page instead of this one dedicated to Intel GPUs. Otherwise, Hardware video acceleration#Verification should be modified to link to each dedicated page instead of duplicating content.（在 Talk:Intel 图形处理器 中讨论）

- **intel_gpu_top** — 适用于 Intel GPU 的类似 top 命令的任务监视器。 （需要 root 权限）

: https://gitlab.freedesktop.org/drm/igt-gpu-tools || intel-gpu-tools包

- **nvtop** — 适用于 AMD、Intel 和 NVIDIA 的 GPU 进程监视器（目前对 Intel GPU 仅有非常基础的支持）

: https://github.com/Syllo/nvtop || nvtop包

设置亮度和伽玛值

参见背光。

测试新的实验性 Xe 驱动

为了尝试（实验性的）新的 Xe 驱动，你需要：

- linux包 6.8 或更高版本
- Tiger Lake 或更新的集成显卡，或者独立显卡
- 官方源的 mesa包 软件包。或者保证 mesa 使用 `-D intel-xe-kmd=enabled` 进行编译

运行此命令，然后记录下您的 PCI ID：

```
lspci -nn | grep VGA
00:02.0 VGA compatible controller : Intel Corporation TigerLake-LP GT2 [Iris Xe Graphics] [8086:9a49] (rev 01)
```

然后将带有合适 PCI ID 的以下内容添加到内核参数里：

```
... i915.force_probe=!9a49 xe.force_probe=9a49
```

确保您有在需要时可以撤销变更的备用启动方案。

故障排除

画面撕裂

使用了 Intel 驱动

SNA 加速方法在一些机器上引起撕裂。要解决这个问题，请在驱动程序中启用 `TearFree` 选项，方法是在配置文件中添加以下内容：

```
/etc/X11/xorg.conf.d/20-intel.conf
Section "Device"
  Identifier "Intel Graphics"
  Driver "intel"
  Option "TearFree" "true"
EndSection
```

参见原始错误报告以获得更多信息。

注意：

- 当 `SwapbuffersWait` 为 `false` 时，该选项可能不工作
- 此选项可能会显著增加内存分配并降低性能。
- 对于对vsync定时非常挑剔的应用程序来说，这个选项是有问题的，比如 Super Meat Boy.

- 此选项不适用于UXA加速方法，仅适用于SNA。
- 对于 Intel UHD 620 或 630 显卡，您需要添加 `Option "TripleBuffer" "true"` 选项才能使 `TearFree` 工作。
- 通过添加 `Option "DRI" "2"` 来禁用 DRI3 可能是必须的，不然任意全屏应用（比如视频播放）会使 TearFree 失效。

使用了 modesetting 驱动

modesetting 驱动在最近才支持 TearFree 。直到 2023 年 11 月，这个补丁还没有进入稳定版，所以现在你会需要 xorg-server-gitAUR。

```
/etc/X11/xorg.conf.d/20-intel.conf
Section "Device"
  Identifier "Intel Graphics"
  Driver "modesetting"
  Option "TearFree" "true"
EndSection
```

禁用垂直同步 (VSYNC)

适用于以下情景：

- 由于 GPU 的原因，Chomium/Chrome 有滞后和性能低下的问题，而在使用 --disable-gpu 选项启动时能流畅运行。
- glxgears 测试未显示预期性能

Intel 驱动程序使用三重缓冲进行垂直同步；这允许充分的性能释放并避免画面撕裂。若要关闭垂直同步（例如用于基准测试），请在主目录中使用如下 `.drirc` 文件：

```
~/.drirc
<device screen="0" driver="dri2">
       <application name="Default">
              <option name="vblank_mode" value="0"/>
       </application>
</device>
```

注意：不要使用 driconfAUR 来创建此文件。它有问题，会设置错误的驱动程序。

DRI3 问题

DRI3 是 xf86-video-intel包 中的默认 DRI 版本。在某些系统上，这可能会导致这样的问题。要切换回 DRI2，请在配置文件中添加以下行：

```
Option "DRI" "2"
```

对于 `modesetting` 驱动程序，这种禁用 DRI3 的方法不起作用。所以我们可以设置环境变量 `LIBGL_DRI3_DISABLE=1`。

GTK应用程序中的字体和屏显损坏（挂起/恢复后缺少字形）

如果您在 GTK 应用程序中遇到缺少字形的问题，以下解决方法可能会有所帮助。编辑 `/etc/environment` 并添加以下行：

```
/etc/environment
COGL_ATLAS_DEFAULT_BLIT_MODE=framebuffer
```

另请参考 FreeDesktop bug 88584.

启动过程中加载模块时显示空白屏幕

**这一章节正在考虑移除。**

原因: 从 mkinitcpio v32 开始，`kms` 钩子被默认包含，因此大部分配置会默认启动早启动。 (在 Talk:Intel 图形处理器 讨论)

如果您使用 KMS 晚启动，然后屏幕在加载模块（Loading modules）时显示为空白，您可以试着把 `i915` 和 `intel_agp` 添加到 initramfs 中。请参考内核级显示模式设置#KMS_早启动。

另外，附加以下内核参数似乎也有效：

```
video=SVIDEO-1:d
```

如果您需要输出到VGA，请尝试以下操作：

```
video=VGA-1:1280x800
```

X 窗口系统在使用 Intel 驱动程序时冻结或崩溃

一些关于 X 窗口系统崩溃、冻结或 GPU 挂起的问题，可以通过设置 NoAccel 选项禁用 GPU 来解决。请在配置文件中添加以下内容：

```
  Option "NoAccel" "True"
```

或者，尝试通过设置 DRI 选项来只禁用 3D 加速：

```
  Option "DRI" "False"
```

添加未检测到的分辨率

这个问题在 Xrandr 页面中有详细的说明。

无法调节背光

如果您的设备从挂起中恢复后，调整屏幕背光亮度的热键失效，请根据 Backlight 一文检查您的配置。

如果问题仍然存在，请尝试如下任一内核参数:

```
acpi_osi=Linux
acpi_osi="!Windows 2012"
acpi_osi=
```

还要确保您没有使用 fastboot 模式 (`i915.fastboot` 内核参数) ，因为它会破坏背光控制。

Chromium 和 Firefox 损坏或无响应

如果您在 Chromium 和/或 Firefox 中遇到了损坏、无响应、性能滞后或缓慢的问题，一些可能的解决方案包括：

- 把 AccelMethod 设置为 "uxa"
- 禁用垂直同步 (VSYNC))
- 启用 TearFree 选项
- 禁用 DRI 和加速法 (在 Intel 第十代处理器 Iris 显卡上测试过):

```
    Option "NoAccel" "True"
    Option "DRI" "False"
```

4.0+ 版本的内核会在使用 Broadwell 或 Core-M 芯片时崩溃

X/Wayland 加载后几秒钟，机器将冻结，journalctl 将记录内核崩溃并提到 Intel 显卡：

```
Jun 16 17:54:03 hostname kernel: BUG: unable to handle kernel NULL pointer dereference
at           (null)
Jun 16 17:54:03 hostname kernel: IP: [<          (null)>]           (null)
...
Jun 16 17:54:03 hostname kernel: CPU: 0 PID: 733 Comm: gnome-shell Tainted: G     U
O    4.0.5-1-ARCH #1
...
Jun 16 17:54:03 hostname kernel: Call Trace:
Jun 16 17:54:03 hostname kernel:  [<ffffffffa055cc27>] ?
i915_gem_object_sync+0xe7/0x190 [i915]
Jun 16 17:54:03 hostname kernel:  [<ffffffffa0579634>]
intel_execlists_submission+0x294/0x4c0 [i915]
Jun 16 17:54:03 hostname kernel:  [<ffffffffa05539fc>]
i915_gem_do_execbuffer.isra.12+0xabc/0x1230 [i915]
Jun 16 17:54:03 hostname kernel:  [<ffffffffa055d349>] ?
i915_gem_object_set_to_cpu_domain+0xa9/0x1f0 [i915]
Jun 16 17:54:03 hostname kernel:  [<ffffffff811ba2ae>] ? __kmalloc+0x2e/0x2a0
Jun 16 17:54:03 hostname kernel:  [<ffffffffa0555471>] i915_gem_execbuffer2+0x141/0x2b0
[i915]
Jun 16 17:54:03 hostname kernel:  [<ffffffffa042fcab>] drm_ioctl+0x1db/0x640 [drm]
Jun 16 17:54:03 hostname kernel:  [<ffffffffa0555330>] ?
i915_gem_execbuffer+0x450/0x450 [i915]
Jun 16 17:54:03 hostname kernel:  [<ffffffff8122339b>] ? eventfd_ctx_read+0x16b/0x200
Jun 16 17:54:03 hostname kernel:  [<ffffffff811ebc36>] do_vfs_ioctl+0x2c6/0x4d0
Jun 16 17:54:03 hostname kernel:  [<ffffffff811f6452>] ? __fget+0x72/0xb0
Jun 16 17:54:03 hostname kernel:  [<ffffffff811ebec1>] SyS_ioctl+0x81/0xa0
Jun 16 17:54:03 hostname kernel:  [<ffffffff8157a589>] system_call_fastpath+0x12/0x17
Jun 16 17:54:03 hostname kernel: Code:  Bad RIP value.
Jun 16 17:54:03 hostname kernel: RIP  [<          (null)>]           (null)
```

这可以通过禁用 execlist 支持来解决，该支持在 4.0 内核中被设为为默认值。添加以下内核参数：

```
i915.enable_execlists=0
```

目前已知该问题直到 4.0.5 内核上仍存在。

Windows 客户机运行迟缓

VirtualBox 中 Windows 客户机的视频输出有时会挂起，直到主机强制屏幕更新(例如移动鼠标光标)。删除 `enable_fbc=1` 选项可以解决这个问题。

屏幕闪烁

面板自刷新 (PSR), 一种 Intel iGPU （核显）使用的节能技术已知在某些情况下会导致闪烁 FS#49628 FS#49371 FS#50605. 临时的解决办法是通过设置 `i915.enable_psr=0` 内核参数来禁用该功能。

在 i915 驱动上使用 OpenGL 2.1

正如这篇文章所说，把 mesa包 从 13.x 版本更新到 17 可能会导致第三代 Intel GPU （如GMA3100，参阅这里）上的 OpenGL 2.1 支持失效，并使其降级到 OpenGL 1.4。然而您可以通过配置 `/etc/drirc` 或 `~/.drirc` 文件中的选项来手动恢复它：

```
/etc/drirc
<driconf>
...
    <device driver="i915">
        <application name="Default">
            <option name="stub_occlusion_query" value="true" />
            <option name="fragment_shader" value="true" />
        </application>
    </device>
...
</driconf>
```

注意：

- 作出该让步的原因是 Chromium 和其他应用程序的糟糕体验。如果有需要，您可以参考 此文 来对 drirc 文件作特定于应用程序的修改，比如专门针对 Chromium 禁用 gl2.1。
- 新的包含在 mesa包 软件包内的基于 Gallium 的 i915 驱动会一直报告 OpenGL 2.1，所以这些设置对于那个驱动来说不再需要了。

KMS 问题：终端只在很小的一块区域中显示

其中一个低分辨率视频端口会在引导时启用，这将导致终端仅能使用屏幕的一小块区域。 要修复这个问题，在引导加载程序的内核命令行参数中，使用 i915 模块参数 `video=SVIDEO-1:d` 来明确禁用该端口。有关详细信息，请参阅内核参数。

如果这不起作用，请尝试禁用 TV1 或 VGA1，而不是 SVIDEO-1。视频端口名称可以用 xrandr 列出。

Haswell CPU 使用 HDMI 输出时没有声音

根据一个 Linux 内核问题，在设置 `intel_iommu=on` 参数时声音不会通过 HDMI 端口输出。要修复该问题，使用如下内核参数：

```
intel_iommu=on,igfx_off
```

或者也可以禁用 IOMMU：

```
intel_iommu=off
```

低功耗 Intel CPU 会崩溃或冻结

**本文或本章节的事实准确性存在争议。**

原因： 在额外的声明里提到，`enable_dc=0` 可能不会影响到电源管理（在 Intel graphics#Incorrect statements regarding power usage penalty of enable_dc=0 中讨论）

由于低功耗 Intel CPU 上电源管理功能的问题，低功耗 Intel 处理器和/或笔记本电脑处理器有随机挂起或崩溃的倾向。如果发生此类崩溃，您将看不到任何报告此问题的日志。添加以下内核参数可能有助于解决问题。

注意：不建议同时使用以下三个内核参数

```
intel_idle.max_cstate=1 i915.enable_dc=0 ahci.mobile_lpm_policy=1
```

`ahci.mobile_lpm_policy=1` 能修复部分联想笔记本电脑和一些宏碁笔记本电脑因 SATA 控制器电源管理问题而出现的挂起故障。该解决方法严格来说与 Intel 显卡无关，但它确实解决了相关问题。添加此内核参数可以将链路电源管理(LPM)从固件默认值修改为最大性能，并且还可以解决当您在某些联想机器上更改显示器亮度时出现的挂起问题。但这么做在现代超级本上会增加 1 至 1.5 W 的闲置功耗。有关更多信息，特别是有关其他状态的信息，请参阅 Linux内核邮件列表和 Red Hat 文档。

`i915.enable_dc=0` 会禁用GPU电源管理。这确实解决了某些 Intel 系统上的随机挂起问题，特别是 Goldmount 和 Kaby Lake Refresh 芯片。但使用此参数会提高笔记本电脑的功耗并缩短电池寿命。

`intel_idle.max_cstate=1` 会限制处理器的睡眠状态，防止处理器进入深度睡眠状态。这绝对不是理想的做法，而且确实会导致更高的功率使用和更低的电池寿命。然而，它确实解决了许多 Intel 系统上的随机挂起问题。如果您使用的是 Intel Baytrail 或 Kaby Lake Refresh 芯片，请使用此参数。Intel Baytrail 芯片由于一个硬件缺陷，在没有这个内核参数的情况下会随机挂起。有关 max_cstate 参数的更多信息，请参见内核文档以及 GitHub 上关于 cstates 的一篇文章。

如果您尝试通过一次性添加 `intel_idle.max_cstate=1 i915.enable_dc=0 ahci.mobile_lpm_policy=1` 等三个参数来解决频繁挂起的问题，然后成功了，那您应该用排除法找出真正解决问题的那个参数。因为如果实际的问题与 SATA 电源管理有关，并且 `ahci.mobile_lpm_policy=1` 参数解决了这个问题，那么禁用 cstate 和显示电源管理是不可取的。

参考 Linux Reviews 网站以获取更多信息。

添加对 165Hz 显示器的支持

**本文或本章节可能需要合并到内核级显示模式设置#强制设置显示模式与_EDID。**

附注： 这种技术似乎并不特定于 i915。在合并之前，请验证安装脚本是必需的，并且没有更简单的方法来将 EDID BIN 添加到 initramfs。（在 Talk:Intel 图形处理器 中讨论）

对于某些 165Hz 显示器，*xrandr* 可能不会显示165Hz选项，且 #添加未检测到的分辨率中的方法并不能解决这一问题。在这种情况下，请参见 StackExchange 上的问答 i915-driver-stuck-at-40hz-on-165hz-screen。

注意：除了创建 `/etc/initramfs-tools/hooks/edid` 文件之外，应该通过以下方式创建 mkinitcpio 钩子：

```
/etc/initcpio/install/edid
#!/bin/bash

build() {
add_file /lib/firmware/edid/edid.bin
}

help() {
cat <<HELPEOF
This hook add support for 165Hz
HELPEOF
}
```

然后在 `/etc/mkinitcpio.conf` 文件的 HOOKS 中添加 *edid* ，就像下面这样

```
/etc/mkinitcpio.conf
HOOKS=(... fsck edid)
```

最后 重新生成initramfs.

Raptor Lake 和 Alder Lake-P 处理器从睡眠/挂起中恢复时冻结

来自不同供应商的 Raptor Lake 和 Alder Lake-P 第 12 代移动处理器的笔记本电脑的用户在从暂停状态中醒来后，遇到了了冻结和黑屏。这是因为根据 freedesktop issue 5531 和 6401 的描述，许多笔记本电脑供应商提供了错误的 VBT（Video BIOS Table，视频 BIOS 表），错误地描述了连接到 iGPU 的实际端口。在这种情况下，所有被记录的案例都怀疑是重复的 eDP 条目。

考虑到大多数供应商不会为运行正常的 Windows 操作系统的笔记本电脑发布 BIOS 更新，Linux 用户只能在内核端解决这个问题。有两个方法可以防止出现重复的 eDP 条目以影响到内核：修补内核或者加载一个 修改过的 VBT。

为了修补内核，重复的 eDP 条目需要通过分析以下命令的输出被辨别出来：

```
# intel_vbt_decode /sys/kernel/debug/dri/1/i915_vbt
Child device info:
        Device handle: 0x0008 (LFP 1 (eDP))
        Device type: 0x1806 (unknown)
 ...
 Child device info:
        Device handle: 0x0080 (LFP 2 (eDP))
        Device type: 0x1806 (unknown)
```

输出表明这里确实存在重复的 eDP，内核应该忽略掉第二个，但是用户还是被鼓励检查这个。然后可以使用下面的补丁修补内核，如果有需要的话重复条目的索引可以被替换为 `ignoreEntry = 1`。

```
--- a/drivers/gpu/drm/i915/display/intel_bios.c
+++ b/drivers/gpu/drm/i915/display/intel_bios.c
@@ -3688,6 +3688,14 @@
{
        struct intel_bios_encoder_data *devdata;

+ int ignoreEntry = 0;
+
        list_for_each_entry(devdata, &i915->display.vbt.display_devices, node)
- func(i915, devdata);
+ {
+ if (ignoreEntry != 1)
+ {
+ func(i915, devdata);
+ ignoreEntry++;
+ }
+ }
}
```

第二种解决这个问题的方法是通过直接清除 VBT 内重复条目来修改 VBT。

复制 VBT，使用十六进制编辑器编辑它然后修改重复的 device handle 的 device type 至 `00 00`：

```
$ cat /sys/kernel/debug/dri/0/i915_vbt > vbt
--- vbt
+++ modified_vbt
@@ -22,10 +22,10 @@
 00000150  00 08 00 20 00 08 00 10  00 08 00 02 00 08 00 01  |... ............|
 00000160  00 08 00 00 01 08 00 00  00 04 00 00 00 40 00 00  |.............@..|
 00000170  00 20 00 00 00 10 00 00  00 02 00 00 00 01 00 00  |. ..............|
-00000180  00 00 01 00 00 02 8b 01  02 04 00 00 27 08 00 06  |............'...|
-00000190  18 00 00 00 00 00 00 00  00 00 00 00 00 0a 00 00  |................|
+00000180  00 00 01 00 00 02 8b 01  02 04 00 00 27 08 00 00  |............'...|
+00000190  00 00 00 00 00 00 00 00  00 00 00 00 00 0a 00 00  |................|
 000001a0  03 00 00 00 c0 00 40 00  20 00 00 00 00 00 00 00  |......@. .......|
-000001b0  00 00 20 00 80 00 06 18  00 00 00 00 00 00 00 00  |.. ..............|
+000001b0  00 00 20 00 80 00 00 00  00 00 00 00 00 00 00 00  |.. ..............|
 000001c0  00 00 00 00 07 00 00 00  00 00 00 c0 00 10 00 20  |............... |
 000001d0  00 00 00 00 00 00 00 00  00 20 00 04 00 d2 60 00  |......... ....`.|
 000001e0  10 10 00 23 21 10 00 00  00 00 00 07 00 00 02 00  |...#!...........|
```

这个修改过的 VBT 可以通过复制到 `/lib/firmware/modified_vbt` 再传递 `i915.vbt_firmware=modified_vbt` 的内核参数来加载，以及如果有需要，重新生成 initramfs。

参阅

- linux graphics（包含一个有 106 种相关产品的列表）
- Intel® Processor Graphics（包含一个含有处理器系列、正式的代号、发行日期和显示技术的表）

2.21 AMD Catalyst

难度 高级 | **篇幅** 58.1 KB | **所属** 图形

本节将讲到

- 安装
- Xorg 软件库
- Mirrors, if the primary server does not work or is too slow:
- 如果上面那个不行或者太慢的话就换这些镜像站:
- 如果上面那个不行或者太慢的话就换这些镜像站:

**本文已被弃用。**

本文所述教程或工具由于已经过时而被弃用，请使用其他替代方案： ATI、AMDGPU 和 AMDGPU PRO (在Talk:AMD Catalyst讨论)

AMD 的显卡驱动有两种：一是官方私有驱动 (catalystAUR\[损坏的链接：package not found\])，二是开源驱动（较旧的显卡参见 ATI，较新的显卡参见 AMDGPU）。本文主要介绍私有驱动。

警告：官方仓库不再提供 Catalyst。 AMD 已经不再更新这个软件包，而且此软件包与最新的 Xorg 不兼容，需要先安装老的 Xorg 才能使用。

AMD 的 Linux 驱动软件包 *catalyst* 驱动曾被命名为 *fglrx* (**F**ire**GL** and **R**adeon **X**)。现在虽然改变了软件包名称，但内核模块文件名依然为以前的 *fglrx.ko*。因此，下文中任何提及 fglrx 都是指*内核模块*,而**不是指软件包**.

与开源驱动相比，Catalyst 在 2D 上表现比较差，在 3D 上与开源驱动差不多，但缺乏高效的多显支持。Catalyst 支持芯片组 R600 及以上（Radeon HD 2xxx 或者更新）的 ATI/AMD Radeon 显卡。*model* 名称（如 X1900，HD4850）与 *chip* 名称（分别是 R580，RV770）间的对照请参见 Xorg decoder ring 或者这个表格。

安装

共有三种途径安装 Catalyst。第一种是用 Vi0L0（Arch非官方Catalyst维护人员）维护的软件库。此库包涵了所有可用的软件包。第二种方式就是通过AUR，Vi0L0 提供的 PKGBUILD 跟他用于构建他仓库的 PKGBUILD 完全一样。最后，你还可以直接通过 AMD 官方下载 Catalyst。

自 Catalyst 12.4，AMD 已将针对 Radeon HD 5xxx 和 Radeon HD 2xxx，3xxx 与 4xxx 显卡驱动分开开发，因此在你选择何种安装方式之前，应查看你的显卡型号。Radeon HD 2xxx，3xxx 和 4xxx 显卡用 **legacy**驱动，Radeon HD 5xxx（以及更新的）用普通 Catalyst。但无论你需要哪种驱动，都应安装 Catalyst utilities。

注意：你会发现，每种安装方式都会进行的一些相同的操作，无论你采用哪种安装方式，你都应了解一些通用的安装说明。

安装 Catalyst

从非官方软件库

如果你不喜欢通过 AUR 来安装，则使用此方法。此软件库由我们的非官方 Catalyst 维护者 Vi0l0 维护。所有的包都经过签名，所以安全方面无需担心。下文提及的很多其他与 AMD 显卡有关的包也是由 Vi0L0 维护。

Vi0L0 有三个不同的 Catalyst 软件库:

- catalyst\[损坏的链接：无效的章节\]：Radeon HD 5xxx 及更新的显卡使用的普通 Catalyst 驱动。包含了最新的 (稳定版或者 beta 版) Catalyst。
- *catalyst-stable*：Radeon HD 5xxx 及更新的显卡使用的普通 Catalyst 驱动。包含了最新的稳定版 Catalyst。
- catalyst-hd234k\[损坏的链接：无效的章节\]：Radeon HD 2xxx，3xxx 与 4xxx 显卡使用的 legacy Catalyst 驱动。

要启用上述软件库的话，参见 Unofficial user repositories 中所述方法。记得在 `pacman.conf` 中的**其他软件库之前**添加相应软件库。

注意：*catalyst* 和 *catalyst-stable* 软件库的 URL 相同。若要启用 *catalyst-stable* ，步骤与启用 *catalyst* 相同，并在 `pacman.conf` 中将 `[catalyst]` 替换成 `[catalyst-stable]`。如果你需要某个旧版本，这儿也有并且 URL 也相同（比如 *catalyst-stable-13.4*）。

提示：有时 catalyst.wirephire.com 因为超出带宽上限而不能提供下载（曾发生过这个问题），或者你到这个服务器的连接很慢，这时，你可以尝试另外的镜像服务器：\[失效链接 2020-08-02 ⓘ\]（goll 提供，德国）。不过。这些服务器也不保证随时可用。

完成后更新 pacman 数据库并安装这些软件包（更多信息参见#工具）：

- *catalyst-hook*
- *catalyst-utils*
- *catalyst-libgl*
- *opencl-catalyst* - 可选,OpenCL支持
- *lib32-catalyst-utils* - 可选,64-bit系统上32-bit的OpenGL支持
- *lib32-catalyst-libgl* - 可选,64-bit系统上32-bit的OpenGL支持
- *lib32-opencl-catalyst* - 可选,64-bit系统上32-bit的OpenCL支持

注意：如果 pacman 询问是否移除 **libgl**，尽管回答“是”。

警告：软件包 catalyst 已从 Vi0L0 的仓库移除，catalyst-hook 取而代之。

通过 AUR 安装

还可以通过 AUR 安装。如果你需为你的电脑进行定制安装，则用此方法。此方法极为繁琐，因为它需要的工作量最大，而且每次内核更新后你得手动更新 Catalyst。

警告：若通过 AUR 安装 Catalyst，每次内核更新你都得重新编译 Catalyst，否则 X 将不能启动。

在 Vi0L0 的软件库中提到的一切软件包在 AUR 中也可用：

- CatalystAUR\[损坏的链接：package not found\]
- Catalyst-generatorAUR\[损坏的链接：package not found\]
- Catalyst-hookAUR\[损坏的链接：package not found\]
- Catalyst-utilsAUR\[损坏的链接：package not found\]
- Lib32-catalyst-utilsAUR\[损坏的链接：package not found\]

AUR还提供些独家软件包。它含有被称为 *Catalyst-total* 的包和一些 beta 阶段的软件：

- Catalyst-totalAUR\[损坏的链接：package not found\]
- Catalyst-total-hd234kAUR\[损坏的链接：package not found\]
- Catalyst-testAUR\[损坏的链接：package not found\]

catalyst-totalAUR\[损坏的链接：package not found\] 包能让 AUR 用户更为方便。它能构建驱动、内核工具、32位内核工具和 catalyst-hookAUR\[损坏的链接：package not found\]（参见 #工具）。

配置驱动

安装完毕后，要配置 X，让其使用 Catalyst。要确保 fglrx 模块在启动阶段加载，而且要禁用内核级显示模式设置。

配置 X

你需要创建 `xorg.conf` 文件来配置 X。Catalyst 提供了 `aticonfig` 工具来创建和（或）修改此文件。 通过访问 `/etc/ati/amdpcsdb` 文件它几乎能配置显卡的各项参数。了解完整的 `aticonfig` 配置选项可运行:

```
# aticonfig --help | less
```

警告：在将各项配置参数提交到 `/etc/X11/xorg.conf` 之前使用 `--output` 选项，`/etc/X11/xorg.conf.d` 中的所有内容都会被覆盖。(Use the `--output` option before committing to `/etc/X11/` as an `xorg.conf` file will override anything in `/etc/X11/xorg.conf.d/`)

注意：如果坚持使用 `xorg.conf.d` 下的新配置文件，使用 `# aticonfig [...] --output` 来让 `Device` 部分与 `/etc/X11/xorg.conf.d/20-radeon.conf` 相匹配。但这有一个缺点，很多依赖 xorg.conf 的 `aticonfig` 选项都无法使用。

现在来配置 Catalyst。若只有一个显示器，运行：

```
# aticonfig --initial
```

注意，若你使用双显示器则使用下面的命令。 此命令会成两个配置文件，第二个显示器的配置文件在第一个前面：

```
# aticonfig --initial=dual-head --screen-layout=above
```

注意：了解与设置双显有关的更多信息可查看 #双屏显示

虽然目前的版本的Xorg启动时能自动探测大多数选项，但不同Xorg版本的默认参数可能会有所不同，最好明确指定一些参数.

这里是一个示例配置（包含注释），**仅供参考**。标注 # 是必需的，标注 ## 可能会用到：

```
/etc/X11/xorg.conf
Section "ServerLayout"
        Identifier     "Arch"
        Screen      0  "Screen0" 0 0           # 0 这里是必需的
EndSection
Section "Module"
        Load [...]
        [...]
EndSection
Section "Monitor"
        Identifier   "Monitor0"
        ...
EndSection
Section "Device"
        Identifier  "Card0"
        Driver      "fglrx"                          # 必需的
        BusID       "PCI:1:0:0"                      # 推荐，如果自动探测失败的话
        Option      "OpenGLOverlay" "0"              ##
        Option      "XAANoOffscreenPixmaps" "false" ##
EndSection
Section "Screen"
        Identifier "Screen0"
        Device     "Card0"
        Monitor    "Monitor0"
        DefaultDepth    24
        SubSection "Display"
                Viewport   0 0
                Depth     24                         # 不应该改动这里的值 '24'
                Modes "1280x1024" "2048x1536"        ## 第一个值是默认分辨率，第二个是最大分辨率
                Virtual 1664 1200                    ## (x+64, y) to workaround potential OGL rect. artifacts/
        EndSubSection                                ## fixed in Catalyst 9.8
EndSection
Section "DRI"
        Mode 0666                                    # 也许在启用直接渲染方面有用
EndSection
```

注意：一旦升级 Catalyst 就要这样删除 `amdpcsdb` 文件: 关闭 X，删除 `/etc/ati/amdpcsdb` ，启动 X 然后运行 `amdcccle` - 否则 `amdcccle` 将会显示错误的 Catalyst 版本号。

更多信息参考这里.

启动时加载模块

禁用 `radeon` 以防其自动加载。在 `/etc/modprobe.d/modprobe.conf` 里禁用 *radeon*，同时保证它不被 `/etc/modules-load.d/` 里的文件加载。详见 Kernel modules#黑名单。

接下来，使模块 `fglrx` 自动加载。添加 `fglrx` 到 `/etc/modules-load.d/` 下已有的模块文件的新一行，或者创建一个新的模块文件并添加 `fglrx`。

禁用内核级显示模式设置

注意：使用 PowerXpress 技术的用户不要这样做，因为Intel驱动需要它。

禁用内核级显示模式设置很重要，Catalyst 根本不使用内核级显示模式。否则，当系统切换至TTY或在桌面环境下关机时，系统可能会冻结。

添加 `nomodeset` 到你的内核参数。

检查安装是否成功

重启电脑并登录, 运行下列命令可查看`fglrx`是否正确运行:

```
$ lsmod | grep fglrx
```

若有输出, 则证明安装成功。可以尝试用 `$ startx` 或者显示管理器来启动 X（参见 Xorg#Running）。

下面的命令可以输出你的显卡型号信息：

```
$ fglrxinfo
```

运行以下命令检查直接渲染模式是否启用:

```
$ glxinfo | grep "direct rendering"
```

若显示 `"direct rendering: yes"`，恭喜你，到位了! 若无 `$ glxinfo` 命令，安装 mesa-utils包 package。

注意：对于 `glxgears`，你也可以使用：

```
$ fgl_glxgears
```

来测试fglrx。

警告：最近版本的 Xorg 函数库路径发生了变化，因此即使安装了 `libGL.so` 也不一定能被正确加载。请检查图形库是否工作，可阅读 #故障排除段落。

自己编译内核

在手动编译的内核上,你必须构建你自己的 `catalyst-$kernel` 包。

注意：如果你讨厌打包或毫无经验，可先阅读 Arch Build System。

1. 从 Catalyst 上获取 `PKGBUILD` 和 `catalyst.install` 文件。

2. 编辑 PKGBUILD。两个地方需要修改：
3. 将 `pkgname=catalyst` 修改为 `pkgname=catalyst-$kernel_name`，`$kernel_name` 可以随意取（如：custom，mm）。
4. 将 `linux` 的依赖修改为 `$kernel_name`。
5. 构建并安装软件包；运行 `makepkg -i` 和 `makepkg`，接着运行 `# pacman -U pkgname.pkg.tar.gz`

注意：

- 如果在安装有多内核的系统上，你必须为所有内核安装 catalyst-utilsAUR\[损坏的链接：package not found\] 包。这不会引起冲突。
- catalyst-generatorAUR\[损坏的链接：package not found\] 能为你自动构建 `catalyst-{kernver}`，因此这些步骤根本就可省略。请参考 #工具部分。

PowerXpress 支持

PowerXpress 技术允许支持双显卡功能的笔记本电脑从集成显卡 (IGP) 切换到独立显卡，以延长电池寿命或者实现更好的3D渲染效能。

如果要在Arch上使用此功能，你将需要：

- 从 AUR 获取并编译 catalyst-totalAUR\[损坏的链接：package not found\] / catalyst-testAUR\[损坏的链接：package not found\] 软件包，或者
- 从 \[catalyst\] 软件仓库安装 **catalyst-libgl** 和 **catalyst-utils** 软件包。

对于 Intel 集成显卡的切换，你还需要安装 mesa包 软件包和 Intel 的驱动：xf86-video-intel包。

注意：Catalyst 有时不兼容最新的 Intel 驱动程序。如果遇到这种情况，请降级 xf86-video-intel包 与 xorg-server 软件包到 Arch Linux Archive 或者 #Xorg 软件库 上的先前版本。

现在你可以用下面这些命令切换集成显卡和独立显卡：

```
# aticonfig --px-igpu    #for integrated GPU
# aticonfig --px-dgpu    #for discrete GPU
```

请记住 fglrx 需要配置 `/etc/X11/xorg.conf` 为包含 `fglrx`。

你也可以用`pxp_switch_catalyst` 这个切换脚本完成一些其他有用的操作：

- 切换 `xorg.conf` - 这将重命名 `xorg.conf` 为 `xorg.conf.cat`（如果它包含了 fglrx）或者 `xorg.conf.oth`（如果它包含了 intel），并且根据你的选择创建一个软链接 `xorg.conf`。
- 运行 `aticonfig --px-Xgpu`.
- 运行 `switchlibGL`.

- 添加/删除 `fglrx` 到/从 `/etc/modules-load.d/catalyst.conf`.

用法：

```
# pxp_switch_catalyst amd
# pxp_switch_catalyst intel
```

如果你试图在 Intel 驱动上运行 X 时遇到问题，你可以尝试强制开启 "UXA" 加速: 在 `xorg.conf` 中添加 `Option "AccelMethod" "uxa"`，就像这样：

```
/etc/X11/xorg.conf
Section "Device"
        Identifier  "Intel Graphics"
        Driver      "intel"
        #Option      "AccelMethod"  "sna"
        Option      "AccelMethod"  "uxa"
        #Option      "AccelMethod"  "xaa"
EndSection
```

同时运行两个 X server（一个使用 Intel 驱动，一个使用 fglrx）

因为 fglrx 容易崩溃（考虑到 PowerXpress），主要 X server 使用 Intel 驱动，另一个使用需要 3D 加速的 fglrx 驱动是个不错的选择。但是在开启第二个 X server 的时候，简单地从集成显卡 `aticonfig` 或者 `amdcccle` 切换到独立显卡将引发一系列奇怪的问题。

为了同时运行两个 X server（每个用不同的驱动），你首先需要设置出一个可以和 Catalyst 一起正常工作的 X 图形环境，然后把它的 `xorg.conf` 移动到一个临时的地方（比如 `/etc/X11/xorg.conf.fglrx`）。下次 X 图形环境启动时，它将默认使用 Intel 驱动来代替 fglrx。

在开启第二个使用 fglrx 的 X server，只需要在运行 X 之前，把 `xorg.conf` 移回合适的地方 (`/etc/X11/xorg.conf`)。这个方法甚至允许你在两个运行的 X 会话之间来回切换。当你不需要使用 fglrx 时，再把 `xorg.conf` 移动到其他地方。

这种方法唯一的坏处是不能使用 Intel 驱动的 3D 加速。但它的 2D 却能完全发挥作用。除此之外，它还能给我们一个非常稳定的桌面环境。

多显示器的 PowerXpress 笔记本运行于 AMD 模式时 (pxp_switch_catalyst amd) 的问题

当 PowerXpress 笔记本工作于 AMD-only 模式时（比如设置全部渲染工作交给独显），你有可能会遇到显示器伪影/重复的情况。这是一个已知的问题，发生于7xxxM系列显卡。

当旋转或者缩放一个显示器时现象会消失，所以你可以使用 xrandr 来解决这个问题：

```
xrandr --output HDMI1 --left-of LVDS1 --primary --scale 1x1 --output LVDS1 --scale
1.0001x1.0001
```

Xorg 软件库

Catalyst 由于其缓慢的更新而被人大为诟病。因升级 Xorg 而造成两者不兼容是稀松平常的事。也就意味着 Catalyst 用户要么自己编译 Xorg 的包要么使用一个只包含 Xorg 包的回溯软件库，该库中不提供更新版 Xorg 以确保兼容性。Vi0L0 提供好几个这样的库。

要启用上述软件库的话,参见 Unofficial user repositories 中所述方法. 记得在 `pacman.conf` 中的**其他软件库之前（甚至在你的 catalyst 仓库前，如果你有的话）**添加相应软件库（使用和 catalyst\[损坏的链接：无效的章节\] 库相同的 PGP 密匙）。

xorg117

Catalyst 不支持 xorg-server 1.18

```
[xorg117]
Server = http://catalyst.wirephire.com/repo/xorg117/$arch
## Mirrors, if the primary server does not work or is too slow:
#Server = http://mirror.hactar.bz/Vi0L0/xorg117/$arch
```

xorg116

Catalyst < 15.7 不支持 xorg-server 1.17

```
[xorg116]
Server = http://catalyst.wirephire.com/repo/xorg116/$arch
## 如果上面那个不行或者太慢的话就换这些镜像站：
#Server = http://mirror.hactar.bz/Vi0L0/xorg116/$arch
```

xorg115

Catalyst < 14.9 不支持 xorg-server 1.16

```
[xorg115]
Server = http://catalyst.wirephire.com/repo/xorg115/$arch
## 如果上面那个不行或者太慢的话就换这些镜像站：
#Server = http://mirror.hactar.bz/Vi0L0/xorg115/$arch
```

xorg114

Catalyst < 14.1 不支持 xorg-server 1.15.

```
[xorg114]
Server = http://catalyst.wirephire.com/repo/xorg114/$arch
## 如果上面那个不行或者太慢的话就换这些镜像站：
#Server = http://mirror.hactar.bz/Vi0L0/xorg114/$arch
```

xorg113

Catalyst < 13.6 不支持 xorg-server 1.14.

```
[xorg113]
Server = http://catalyst.wirephire.com/repo/xorg113/$arch
## 如果上面那个不行或者太慢的话就换这些镜像站：
#Server = http://mirror.hactar.bz/Vi0L0/xorg113/$arch
```

xorg112

Catalyst < 12.10 和 Catalyst Legacy 不支持 xorg-server 1.13.

```
[xorg112]
Server = http://catalyst.wirephire.com/repo/xorg112/$arch
## 如果上面那个不行或者太慢的话就换这些镜像站：
#Server = http://mirror.hactar.bz/Vi0L0/xorg112/$arch
```

工具

Catalyst-hook

Catalyst-hookAUR\[损坏的链接：package not found\] 是一个 systemd 服务,它在系统关机或重启后重新构建 `fglrx` 模块（如果需要的话，比如内核升级后）。

使用之前请保证 base-devel包组 组和 linux-headers包 包（对应你的内核）已经安装。

只需激活 `catalyst-hook.service` 服务即可：

```
# systemctl enable catalyst-hook
# systemctl start catalyst-hook
```

你也可以用这个软件包来手动构建 `fglrx` 模块。在内核更新后运行 `catalyst_build_module` 脚本即可：

```
# catalyst_build_module all
```

一些技术细节：

The `catalyst-hook.service` is stopping the systemd "river" and is forcing systemd to wait until catalyst-hook finishes its job.

`catalyst-hook.service` 调用 `catalyst_build_module check` 来检查是否有必要重构建fglrx。

`check` 检查 `fglrx` 模块是否存在：

- 不存在，将构建它；
- 存在，它将比较两个参数来确定是否有必要重构建 `fglrx`。

这里的参数是 `/usr/lib/modules/<kernel_version>/build/Module.symvers` 的 md5 值。（因为我（这里指 Vi0L0）发现每一个版本的这个文件都不一样）。第一个参数是现有的 `Module.symvers` 文件 md5，第二个参数是 `fglrx` 模块构建时 `Module.symvers` 文件的 md5。这个参数被 `catalyst_build_module` 脚本编译到 `fglrx` 模块。

如果参数不同，将编译新的 `fglrx` 模块。

check 检查整个 `/usr/lib/modules/` 目录，为安装的所有内核编译 fglrx 模块（如果需要的话）。如果没有必要构建或重构建，进程将很快结束。

Catalyst-generator

catalyst-generatorAUR\[损坏的链接：package not found\] 包能构建并安装 `fglrx` 模块，该模块与 pacman 兼容的 `catalyst-${kernver}` 相适应。与 #Catalyst-hook 的区别是必须手动使用此命令，而 Catalyst-hook 则不需。

通过 makepkg，它能构建 `catalyst-${kernver}` 二进制包并用 pacman 安装。`${kernver}` 是软件包的目标内核版本（例如 catalyst-2.6.35-ARCH 适用于 2.6.35-ARCH 内核）。

非特权用户使用 `catalyst_build_module` 来构建并安装 `catalyst-{kernver}` 包。安装时会提示输入 root 密码。

简单说一下如何使用此包：

1. root 用户: 使用 `catalyst_build_module remove`。此举会删除无用的 `catalyst-{kernver}` 包。
2. 非特权用户: 使用 `catalyst_build_module ${kernver}`，`${kernver}` 是指升级过后的内核版本。也可通过 `catalyst_build_module all` 为所有安装的内核构建 `catalyst-{kernver}`。
3. 若要删除 `catalyst-generator`，在使用 `catalyst_build_module remove_all` 命令删除 catalyst-generator 之前最好切换到 root 用户，**这会删除所有 `catalyst-{kernver}` 包。**

当删除 `Catalyst-generator` 时，`Catalyst-generator` 不能自动删除那些 `catalyst-{kernver}` 包，这是因为 pacman 不允许有一个以上的实例同时运行。若在使用 `# pacman -R catalyst-generator` 前忘记运行 `# catalyst_build_module remove_all`，catalyst-generator 将会询问删除 catalyst-generator 自身后要删除哪个 `catalyst-{kernver}` 包。

Catalyst-generator 是最安全的,最符合KISS原则的,因为:

1. 你可以使用非特权用户来构建包；
2. 它在 fakeroot 环境构建包；
3. 它不乱丢文件，pacman知道文件们在哪；
4. 你需要做的只是，记得去使用它

注意：在构建 `catalyst-{kernver}` 时，如果看到下列警告，这是正常情况，不必惊慌：

```
WARNING: Package contains reference to $srcdir
WARNING: '.pkg' is not a valid archive extension
```

OpenCL 和 OpenGL 开发

这几年 AMD 一直在为 OpenCL 和 OpenGL 的开发做一套工具集。

现在 AMD 在 **"Heterogeneous Computing"** 的旗帜下提供了更多的工具集，幸运的是它们也在 Linux 下可用。

在 AUR 和 \[catalyst\] 软件仓库，你可以找到这些代表了 AMD 最重要工作的工具软件包：

- amdapp-aparapiAUR
- amdapp-sdkAUR
- amdapp-codexlAUR

APP 这个缩写代表 Accelerated Parallel Processing（加速并行处理）。

amdapp-aparapi

AMD 的 Aparapi 是一个用 Java 实现的 API，用于并行地表达图像数据，它同时也是一个能把 Java 字节码翻译成能被 OpenCL 识别的运行时组件，所以它能够被很多种图形处理器执行。如果 Aparapi 在 GPU 上不能执行，那将在Java的线程池中执行。

更多关于 Aparapi 的信息：参见这里\[失效链接 2020-08-02 ⓘ\]。

amdapp-sdk （以前的 amdstream）

AMD APP 软件开发工具套件，是一个由 AMD 制作地一套完整的开发平台。它让你能够快速和容易地使用 AMD APP 技术，使得你的程序得到加速。这个 SDK 提供了代码实例，技术文档和其他资料，让您可以在你的 C\\C++ 程序中使用 OpenCL，Bolt 或 C++ AMP 等技术实现计算加速。

从 2.8 版本开始，amdapp-sdk 提供了 aparapiUtil 和 aparapi 的代码实例。有一个包已经加入到了 \[catalyst\] 软件仓库。它依赖于 amdapp-aparapiAUR。AUR 中的软件包让你选择需不需要 aparapi's additions。

2.8 版本没有提供探查功能 (Profiler functionality)，它已经被移到 CodeXL 中了。

关于 AMD APP SDK 的更多信息：参见这里\[失效链接 2020-08-02 ⓘ\]。

amdapp-codexl

CodeXL 是一个带有静态 OpenCl 内核分析器的 OpenCL and OpenGL 调试器和探查器。它具有 GUI 界面，是在著名的 gdebuggerAUR\[损坏的链接：package not found\] 基础上完成的。它只支持 x86_64 系统。

关于 CodeXL 的更多信息：参见这里\[失效链接 2020-08-02 ⓘ\]。

功能

Tear Free Rendering

在 **Catalyst 11.1** 中，很可能是添加了三重缓存和 v-sync，*Tear Free Desktop* 减少了 2D，3D 视频应用的屏幕撕裂毛病。但这需要额外的 GPU 处理。

要启用 'Tear Free Desktop'，运行 `amdcccle`，然后设置 `Display Options→Tear Free`。

或以 root 身份运行：

```
# aticonfig --set-pcs-u32=DDX,EnableTearFreeDesktop,1
```

若禁用，使用 `amdcccle` 或以 root 身份运行：

```
# aticonfig --del-pcs-key=DDX,EnableTearFreeDesktop
```

视频加速

Video Acceleration API (VA API) 是为基于 Linux/UNIX 操作系统提供利用 GPU 加速视频处理的一个开源函数库和应用程序接口规范。启用视频加速后，通过各种入口 (VLD, IDCT, Motion Compensation, deblocking) 它能加速常用编码标准 (MPEG-2, MPEG-4 ASP/H.263, MPEG-4 AVC/H.264, and VC-1/WMV3) 视频文件的解码过程（俗称硬解）。

VA-API 在 xvba-videoAUR\[损坏的链接：package not found\] 上有一个私有后端（2009年10月），它允许使用 VA-API 的程序通过 XvBA（AMD 设计的 X-Video 比特流加速 API）函数库来充分利用拥有 uvd2（第二代通用视频解码单元）芯片组的视频加速功能。

注意：使用 `catalyst-test` 或者 `catalyst-total` 时不需要安装 xvba-videoAUR\[损坏的链接：package not found\]，因为有已经创建好的符号链接代替。

xvba-video 和支持 XvBA 的软件仍还在开发，**但在大多数情况下它都能很好的工作。**通过 AUR 构建（或通过 Vi0L0 的仓库直接安装）专有 xvba-videoAUR\[损坏的链接：package not found\] 包，若这个版本对你来说有问题，用 libva-xvba-driverAUR\[损坏的链接：package not found\] 取代；并安装 mplayer-vaapiAUR 和 libva包。然后将视频播放器的视频输出设置为 "vaapi:gl":

```
$ mplayer -vo vaapi:gl movie.avi
```

此选项可添加到 mplayer 的配置文件，参考 MPlayer。

针对 **smplayer**：

```
Options → Preferences → General → Video (tab) → Output driver: User Defined : vaapi:gl
Options → Preferences → General → Video (tab) → Double buffering on
Options → Preferences → General → General → Screenshots → Turn screenshots off
```

`Options → Preferences → Performance → Threads for decoding: 1 (to turn off -lavdopts parameter)`
注意：如果启用了 Tear Free Desktop，则按下列步骤：

`Options -> Preferences -> General -> Video (tab) -> Output driver: vaapi`
若视频输出中没有 **vaapi:gl** 选项，可使用 **vaapi**，**vaapi:gl2** 或简单的 **xv(0 - AMD Radeon AVIVO Video)**。

针对 **VLC**：

`Tools → Preferences → Input & Codecs → Use GPU accelerated decoding`
它有助于在 **amdcccle** 中启用 v-sync：

`3D → More Settings → Wait for vertical refresh = Always On`
注意：若使用 **Compiz/KWin**，消除**画面抖动**的唯一方法就是切换至**全屏**并且**关闭 Redirected Fullscreen**。 使用 **compiz**，需在 CCSM 的 General Options 中设置 **Redirected Direct Rendering**。若此举无效，则将其关闭。**KWin** 默认关闭此功能，若出现画面抖动则通过`System Settings → Desktop Effects → Advanced` 将 "Suspend desktop effects for fullscreen windows" 开启或关闭。

显卡/显存频率，温度，风扇转速，超频工具

`$ aticonfig --od-getclocks` 可以获知当前显卡/显存频率。

`$ aticonfig --pplib-cmd "get fanspeed 0"` 可以获知风扇转速（显卡）；

`$ aticonfig --odgt` 可以获知显卡温度。

`$ aticonfig --pplib-cmd "set fanspeed 0 50"` 可以设置风扇转速，其中查询索引 50 代表速度百分比。

若超频或与之相反，则使用图形工具反而相对容易些，如需要 qt 的 **ATi Overclocking Utility**。你可以从这儿找到（不过它可能过时了）。

更复杂的 amdoverdrivectrlAUR 也能达到此目的，它的主页是这里。可通过 amdoverdrivectrlAUR 或 Vi0L0 的非官方软件库构建安装包。

双屏显示

介绍

警告：由于安装方式的不同，并且每种安装方式需要与其相对应进行配置，因此设置双屏显示并没有特定的方法，你必须根据你自己的需要采用相应的步骤。当然可以多尝试几种方法。**所以，在修改之前，请务必将你现在能正常使用的 `/etc/X11/xorg.conf` 备份，以便遇到问题时可以从命令行恢复。**

- 本节讲述如何配置 "BIG Desktop" 效果。主要是不同尺寸的屏幕如何通过两个不同的输出接口 (DVI + HDMI) 共享一个显卡。
- Xinerama 解决办法有些不便，尤其是不能与 XrandR 兼容。因为 XrandR 对于我们所讲的来说是必须的，所以不使用 Xinerama。

- 双头显示能让你有两个不同的会话（一个屏幕一个）。你可以随心所欲地干任何事情，但不能将窗口从一个屏幕移动到另一个屏幕。若只有一个屏幕，你得在 Xorg 会话里为 Server Layout section 的每个会话定义鼠标，具体方法查看：

ATI Documentation

ATI Catalyst Control Center

ATI 的图形工具非常有用，我们将尽可能地使用它。运行下面命令可启用它：

```
$ {kdesu/gksu} amdcccle
```

警告：千万不要直接使用 sudo。 Sudo 虽能给予管理员权限，却使用用户账户的信息（如环境变量）。GNOME 下使用*gksu*，KDE 下使用*kdesu*。

安装

开始之前，确保你的硬件接插正确，电源开启，而且你得你的硬件属性（2D 还是 3D 屏，屏幕尺寸，刷新率等）。通常情况下，在启动阶段两个显示器都会被识别却不需正确区分先后顺序，而是依赖热插拔功能。尤其在不使用 (`/etc/X11/xorg.conf`) 配置时。

首先要让你的桌面环境和 X 认识你的显示器。为此，要为你两个显示器生成基本的 Xorg 配置文件：

```
# aticonfig --initial --desktop-setup=horizontal --overlay-on=1
```

或

```
# aticonfig --initial=dual-head --screen-layout=left
```

{{注意|`overlay` 非常重要，因为它能让两个显示器拥有 1 种（多种）像素。

提示：运行 `aticonfig --help` 了解所有可用的命令。

现在可编辑基本的 Xorg 配置文件（如：添加分辨率）。分辨率一定要正确，尤其是使用不同尺寸的显示器。分辨率在 "Screen" section：

```
 SubSection "Display"
    Depth     24
    Modes     "X-resolution screen 1xY-resolution screen 1" "Xresolution screen 2xY-resolution screen 2"
 EndSubSection
```

这以后就不需手动编辑 `xorg.conf`，而使用 ATI 的图形化工具。重启 X，确保正确支持两个显示器和识别屏幕分辨率（两个屏幕相互独立而不是完全一样）。

设置

现在只需以 root 身份启动 ATI 控制中心，在显示菜单设置你需要的选项（下拉菜单中的小箭头）。设置好后重启 X 就大功告成了！

重启 X 之前，不要忘了核实 `xorg.conf`。主要是核实 "Display" 下的 "Screen" 节，在 "Virtual" 行里，两个显示器的分辨率应该一样。"Server Layout" 节则是剩下的参数。

卸载

你可能会因为 catalyst 不工作或者是想试试开源驱动而要卸载掉 catalyst：移除 `catalyst` 和 `catalyst-utils` 包。当然你也应该移除 catalyst-generatorAUR\[损坏的链接：package not found\]，catalyst-hookAUR\[损坏的链接：package not found\] 和 lib32-catalyst-utilsAUR\[损坏的链接：package not found\] 包（如果你安装了的话）。

警告：

- 你也许需要使用 `# pacman -Rdd` 来移除 catalyst-utilsAUR\[损坏的链接：package not found\]（和/或 lib32-catalyst-utilsAUR\[损坏的链接：package not found\]）因为它（们）包含了*图形库*相关文件，许多包会依赖他们。这些依赖关系将在安装 xf86-video-ati包 时被满足。
- 你也许会需要移除 `/etc/profile.d/ati-flgrx.sh` 和 `/etc/profile.d/lib32-catalyst`（如果他们存在），否则 `r600_dri.so` 将会载入失败，你将得不到3D支持。

注意：你应该从 `/etc/pacman.conf` 移除非官方仓库，然后运行 `# pacman -Syu`，因为那些仓库包含过时的 Xorg 包（为了兼容 `catalyst`），而 xf86-video-ati包 包需要来自官方仓库的最新的 Xorg 包。

按如下步骤：

- 如果你有 `/etc/modprobe.d/blacklist-radeon.conf`，删除文件或者注释掉文件中的 `blacklist radeon`。
- 如果你在 `/etc/modules-load.d` 下有配置文件要在启动时载入 `fglrx` 模块，删除掉或者注释掉 `fglrx` 那一行。
- 记住删除/备份 `/etc/X11/xorg.conf`。
- 如果安装了 catalyst-hookAUR\[损坏的链接：package not found\] 包，记得关掉它的 systemd 服务的自启动。
- 如果你在内核参数中指定了 `nomodeset` 而现在你打算使用 KMS，那么删除 `nomodeset`。
- 安装另一个驱动之前记得重启 .

故障排除

若能启动到命令行，问题很可能出在 `/etc/X11/xorg.conf`

可阅读 `/var/log/Xorg.0.log` 或通过下列命令查找线索：

```
$ grep '(EE)' /var/log/Xorg.0.log
```

```
$ grep '(WW)' /var/log/Xorg.0.log
```

如果不能理解输出的内容，请先搜索论坛，没有结果的话，可将其提交到这里，注意要提交两者显示的信息。

运行 atieventsd.service 失败

从官方仓库安装 acpid包.启用并运行 acpid 守护进程。

如果还不行，编辑 `/usr/lib/systemd/system/atieventsd.service`，把 `acpid.socket` 改成 `acpid.service`。

无法打开 fglrx_dri.so

创建从 `/usr/lib/xorg/modules/dri/fglrx_dri.so`
到 `/usr/X11R6/lib64/modules/dri/fglrx_dri.so` 或其它需要路径的链接：

```
# mkdir -p /usr/X11R6/lib64/modules/dri
# ln -
s /usr/lib/xorg/modules/dri/fglrx_dri.so /usr/X11R6/lib64/modules/dri/fglrx_dri.so
```

启动 GDM 失败

降级 xorg-server包 或者试试其它显示管理器比如 LightDM。

在Wine上3D应用冻结

若在Wine上3D应用挂起，禁用TLS。使用 `aticonfig` 或编辑 `/etc/X11/xorg.conf`。使用 `aticonfig`：

```
# aticonfig --tls=off
```

或以 root 身份打开 `/etc/X11/xorg.conf`，在 ***Device*** 段添加 `Option "UseFastTLS" "off"`。

只需选择其中一种方式，然后重启 X 让其生效。

视频颜色不正常

仍然使用 `vaapi:gl` 来防止画面抖动，但这样不会有视频加速：

- 不使用 `-vo vaapi` 运行 **mplayer**。
- 若是 **smplayer**，在 Options → Preferences → Advanced → Options for MPlayer → Options:中删除 `-vo vaapi`。

此后还可以启动 **smplayer** 的截屏功能。

KWin 与混成

根据你的显卡，在 OpenGL 和 XRender 渲染方式中选择一种更快的。 在某些情况下，XRender 还可以解决一些人为的错误（比如调整命令行大小）。

重启或启动 X 后，黑屏并且一直不退出

检查在启动加载器的内核参数行是否添加了 `nomodeset`（参考 #禁用内核级显示模式设置）。 你如果在使用 legacy 驱动 (`catalyst-hd234k`) 黑屏的话, 尝试降级 xorg-server 到 1.12（使用 #xorg112 库）。

错误的 ACPI 硬件调用

出现此种错误很可能是 fglrx 模块与系统的 ACPI 硬件调用配合不够默契而自身禁用，屏幕也就不会有输出。

如果是那样的话，运行：

```
$ aticonfig --acpi-services=off
```

注销后 KDM 消失

若使用 Catalyst，当注销后你会获取 tty1 这个控制台而不是 KDM 的欢迎界面。每次注销后你必须让 kdm 重启 X 服务器。将标题为 `[X-:*-Core]` 段里下面行前的注释删掉：

```
/usr/share/config/kdm/kdmrc
TerminateServer=True
```

当前注销 KDE 后 KDM 就会出现。

直接渲染无效

警告：在安装或升级 catalyst 后却没重启系统，也有可能出现此种错误，因为系统需要加载 fglrx.ko 模块来让驱动正常工作。

若直接渲染有问题，运行：

```
$ LIBGL_DEBUG=verbose glxinfo > /dev/null
```

从此命令输出的第一行至末尾都与直接渲染无效有关，且非常详细。

常见错误提示和解决办法：

```
libGL error: XF86DRIQueryDirectRenderingCapable returned false
```

- 若使用 AGP 的芯片组，确保加载 fglrx 模块之前加载正确的 agp 模块。要确定哪些 agp 模块，运行 `hwdetect --show-agp`，然后打开 `/etc/modules-load.d/fglrx.conf`，将 agp 模块添加到 fglrx 行之前。

```
libGL error: failed to open DRM: Operation not permitted
libGL error: reverting to (slow) indirect rendering
libGL: OpenDriver: trying /usr/lib/xorg/modules/dri//fglrx_dri.so
libGL error: dlopen /usr/lib/xorg/modules/dri//fglrx_dri.so failed
(/usr/lib/xorg/modules/dri//fglrx_dri.so: cannot open shared object file: No such file
or directory)
libGL error: unable to find driver: fglrx_dri.so
```

- 某些软件未正确安装。在错误提示中，若路径为 `/usr/X11R6/lib/modules/dri/fglrx_dri.so`，彻底注销，然后重

新登录。若使用图形化的登录管理器 (gdm, kdm, xdm)，确保每次登录时 `/etc/profile` 都会被读取。将 `source /etc/profile` 添加到 `~/.xsession` 或 `~/.xinitrc` 通过都达到以上目的（不同的登录管理器修改的文件不一样）。

- 若路径为 `/usr/lib/xorg/modules/dri/fglrx_dri.so`，试着重装 catalystAUR\[损坏的链接：package not found\] 包。

若错误信息为：

```
fglrx: libGL version undetermined - OpenGL module is using glapi fallback
```

可能是因为系统装了多个版本的 `libGL.so`。下面的命令应该得到这样的回显：

```
$ locate libGL.s
/usr/lib/libGL.so
/usr/lib/libGL.so.1
/usr/lib/libGL.so.1.2
```

系统应只有 3 个 libGL.so 文件，若不止（例如 `/usr/X11R6/lib/libGL.so.1.2`），则将其删除。

若使用 X11R7 且系统中有下列文件，系统并不会给出任何错误提示，一定要将他们删除:

```
/usr/X11R6/lib/libGL.so.1.2
/usr/X11R6/lib/libGL.so.1
```

休眠问题

视频播放不能从休眠状态中恢复

若启动了 framebuffer，Catalyst 不能从挂机状态中恢复。在内核参数中添加 `vga=0` 可禁用 framebuffer。

对于其他启动加载器，参考 #禁用内核级显示模式设置。

系统冻结或硬件锁死

- 过去，`radeonfb` 的 framebuffer 驱动很容易导致这个问题。若内核编译时启用对 radeonfb 的支持，应该更换内核看是否能解决此问题。
- 若退出桌面环境（关机、挂机和切换到 tty 等）时系统冻结，很可能是忘记禁用 KMS。（参见 #禁用内核级显示模式设置）

硬件冲突

当和某些版本的 nForce3 芯片组一起使用时，Radeon 显卡没有 3D 加速。目前虽还未找到具体原因，但有资料表明，先启动到 Windows 然后再重启系统就可能可以从 nVIDIA 驱动获得 3D 加速。在 root 控制台使用下列命令可识别此问题（会得到与下列相似（使用基于 nForce3 系统）的输出）：

```
# dmesg | grep agp
     agpgart: Detected AGP bridge 0
     agpgart: Setting up Nforce3 AGP.
     agpgart: aperture base > 4G
```

或者还有下面的命令得到如下的回显：

```
$ tail -n 100 /var/log/Xorg.0.log | grep agp
 (EE) fglrx(0): [agp] unable to acquire AGP, error "xf86_ENODEV"
```

则表明有问题。

有些资料表明在某些情形下降级主板的 BIOS 可能有助于解决问题，但注意此方法可能并不在各种情况下都适用，而且**降级失败很可能让硬件报废，请谨慎。**

参见这个错误报告。

播放视频时系统短时间死机

当用 mplayer，若不定时出现几秒到几分钟的死机。查看日志，若有与下面相似的信息：

```
/var/log/messages.log
Nov 28 18:31:56 pandemonium [<c01c64a6>] ? proc_get_sb+0xc6/0x160
Nov 28 18:31:56 pandemonium [<c01c64a6>] ? proc_get_sb+0xc6/0x160
Nov 28 18:31:56 pandemonium [<f8bc628c>] ? ip_firegl_ioctl+0x1c/0x30 [fglrx]
Nov 28 18:31:56 pandemonium [<c01c64a6>] ? proc_get_sb+0xc6/0x160
Nov 28 18:31:56 pandemonium [<c0197038>] ? vfs_ioctl+0x78/0x90
Nov 28 18:31:56 pandemonium [<c01970b7>] ? do_vfs_ioctl+0x67/0x2f0
Nov 28 18:31:56 pandemonium [<c01973a6>] ? sys_ioctl+0x66/0x70
Nov 28 18:31:56 pandemonium [<c0103ef3>] ? sysenter_do_call+0x12/0x33
Nov 28 18:31:56 pandemonium [<c01c64a6>] ? proc_get_sb+0xc6/0x160
Nov 28 18:31:56 pandemonium ======================
```

给内核参数添加 `nopat` 和/或 `nomodeset` 到内核参数应该能解决问题。

"aticonfig: No supported adaptaters detected"

若得到：

```
# aticonfig --initial
aticonfig: No supported adapters detected
```

可以在 `etc/X11/xorg.conf` 中设置 device，或者复制以前的能工作的 `/etc/ati/control` 文件（推荐 - 这也能解决水印的问题），这可能让 Catalyst 正常工作。

从 AMD 下载先前版本的 fglrx，使用 `--extract driver` 参数运行。文件将提取到 `driver/common/etc/ati/control`，用它覆盖原来的 control 文件后重启 X。你可以试试不同的版本。

设置型号的方法：在 `/etc/X11/xorg.conf` 将 device 段设置为：

```
/etc/X11/xorg.conf
Section "Device"
        Identifier "ATI radeon ****"
        Driver     "fglrx"
EndSection
```

此处 `****` 为设备型号（HD 6870 的显卡为 6870，APU E-350 为 6310，通过网络是很容易查到的）。

Xorg 启动后很可能使用 `amdcccle` 而不是 `aticonfig`。这里会有一个“AMD不支持硬件”水印。

用下面脚本可将此水印删除：

```
#!/bin/sh
DRIVER=/usr/lib/xorg/modules/drivers/fglrx_drv.so
for x in $(objdump -d $DRIVER|awk '/call/&&/EnableLogo/{print "\\x"$2"\\x"$3"\\x"$4"\
\x"$5"\\x"$6}'); do
 sed -i "s/$x/\x90\x90\x90\x90\x90/g" $DRIVER
done
```

然后重启。

让 chromium 支持 WebGL

在 Google 的 Chromium/Chrome 浏览器里，Linux 的 Catalyst 驱动被列入了黑名单。参见 Chromium#WebGL。

用 Adobe 的 flashplugin 观看 flash，画面迟滞或冻结

编辑：

```
/etc/adobe/mms.cfg
#EnableLinuxHWVideoDecode=1
OverrideGPUValidation=true
```

如果你使用 KDE，请确保 系统设置->工作空间外观与行为->桌面效果->高级 里，“为全屏窗口挂起桌面特效”没有勾选。

GNOME3 中移动窗口延迟/很慢

你可以试试这么做，有报告称大多数情况下此方法有效。

将下面行添加到 `~/.profile` 或 `/etc/profile`：

```
export CLUTTER_VBLANK=none
```

重启 X 或操作系统。

在 1920x1080 分辨率下不能全屏（欠扫描，屏幕周围有黑边）

这经常会在使用 HDMI 连接显示器/电视时发生。

这似乎是 AMD/ATI 为适应所有 HDTV 的一项新功能，可以在 amdccle 中调整。

使用 amdcccle（图形界面）你可以选择要修改的显示输出，将欠扫描设置成 0%（aticonfig 默认 15% 欠扫描）。（至少）版本 14.10 有时也会出现显示调整的下面没有欠扫描的滑块的情况。(It is possible as well that the underscan slider will not show under the display's adjustments, as sometimes in (at least) version 14.10.)

这不是永久的配置，重启 X，重启系统，挂起后唤醒甚至切换 tty 之后都会失效。

要使这成为永久配置的话，你需要以 root 身份在 console 下使用 aticonfig 来调整欠扫描设置，或者手动编辑 /etc/ati/amdpcsdb （也是以 root 身份）

用 **aticonfig** 的方法：

```
# aticonfig --set-pcs-u32=MCIL,DigitalHDTVDefaultUnderscan,0
```

改变设置后重启。

新版本的话（比如 12.11），要是 ccc 总不能保存过扫描设置，你可以试试：

```
# aticonfig --set-pcs-u32=MCIL,TVEnableOverscan,0
```

手动编辑 /etc/ati/amdpcsdb 的方法：

在 [AMDPCSROOT/SYSTEM/MCIL] 之下的任意位置添加

```
# DigitalHDTVDefaultUnderscan=V0
```

新版本的话（比如 12.11）， 要是 ccc 总不能保存过扫描设置，你可以试试： 在 [AMDPCSROOT/SYSTEM/MCIL] 之下找到这一行

```
# TVEnableOverscan=V1
```

然后把它变成

```
# TVEnableOverscan=V0
```

改变设置后重启。

注意：也许你会发现，无论你怎么做，登录/注销/重启后，/etc/ati/amdpcsdb 总是被覆写/还原，于是你的修改就丢失了。

所以你需要在 fglrx 没运行的时候改它。

可以试着重启进入某些不需要运行 fglrx 的、直接启动到控制台的"安全模式"。

应对措施： 要是因某些原因你不想动 ATI 的配置文件，你可以设置面板的位置与分辨率来规避问题 以普通用户运行：

```
# aticonfig --set-dispattrib=DISPLAYTYPE,positionX:0
# aticonfig --set-dispattrib=DISPLAYTYPE,positionY:0
# aticonfig --set-dispattrib=DISPLAYTYPE,sizeX:1920
# aticonfig --set-dispattrib=DISPLAYTYPE,sizeY:1080
```

DISPLAYTYPE 代表你想改的显示器，比如 "dfp9"。 用如下命令获得你的 DISPLAYTYPE：

```
# xrandr --current
```

如果没启用 RandR 的话，你可以用这个：

```
# aticonfig --query-monitor
```

也许需要用显示开关或 DISPLAY 变量来设置适当的显示/屏幕（比如 :0.1）。

这将告诉你目前哪一个显示正连接着或断开着，以及它的详情

这个应对措施不是永久配置，不过是个让面板及上边内容看起来合适的方便快捷的方法

注意：新版驱动不应再使用 `aticonfig --set-pcs-val=MCIL,DigitalHDTVDefaultUnderscan,0`，ATI 声明过这已被弃用并即将被移除。

用 `aticonfig --help | grep set-pcs-val` 来阅读 ATI 的说明。

双屏设置：关于加速，OpenGL，合成，效能的一般问题

试着禁用 xinerama 和 xrandr12。如下：

输入：

```
# aticonfig --initial
# aticonfig --set-pcs-str="DDX,EnableRandR12,FALSE"
```

然后重启系统。在 `/etc/X11/xorg.conf` 中检查 xinerama 是不是被禁用了，如果没有，禁用它，然后重启系统。(In /etc/X11/xorg.conf check that xinerama is disabled, if it's not disable it and reboot your system.)

接下来运行 `amdcccle`，在 amdcccle->display manager->multi-display->multidisplay desktop 中选择显示 2。

再次重启，再按你的想法设置显示。

禁用 VariBright 功能

Vari-Bright 动态调整显示面板用电量以节能。 输入以下命令以禁用 VariBright：

```
# aticonfig --set-pcs-u32=MCIL,PP_UserVariBrightEnable,0
```

Hybrid/PowerXpress：关掉独立 GPU

如果你使用支持 PowerXpress 的 Catalyst 软件包，你也许会发现切换到集显时，独显仍在工作，又耗电又使得系统温度很高。

如果集显是 Intel 的话，你可以通过 `vgaswitcheroo` 来关掉独立 GPU。 很不幸的是有些时候不行。

然后这时你可以试试 acpi_call包。MrDeepPurple 准备好了脚本来完成这个任务。脚本在启动和恢复系统的时候通过 systemd 服务运行。 这是他的脚本：

```
#!/bin/sh
libglx=$(/usr/lib/fglrx/switchlibglx query)
modprobe acpi_call
if [ "$libglx" = "intel" ]; then
    echo '_SB.PCI0.PEG0.PEGP._OFF' > /proc/acpi/call
fi
```

从 X 会话切换到 TTY 黑屏/低分辨率 TTY

接受这个“功能”吧，这出现在 Catalyst 13.2 beta，解决方法是使用 `vga=` 内核选项，比如 `vga=792`。 使用

```
$ hwinfo --framebuffer
```

来得到支持的分辨率列表 选择最适合你的分辨率，粘贴到启动引导器的内核行，比如 `vga=0x03d4`

从 X 会话切换到 TTY 黑屏，背光并没有关闭

使用 uvesafb 作为 framebuffer 驱动。而且，`uvesafb` 可以为 TTY 设置任意分辨率。

切换 TTY 后切换回 X 会话时，只有一个带鼠标的黑屏

如果你遇到了这个 bug 的话，试着添加

```
Option      "XAANoOffscreenPixmaps" "true"
```

到你的 xorg.conf 的 'Device' 部分。

同时，确认已安装有 polkit 认证代理并正在运行，因为这种情况可能会在某程序想要请求密码但没有认证代理来显示密码对话框时出现。

30 FPS / 防撕裂 / 垂直同步 bug

这个 Bug 发生于 Catalyst 13.6 beta，到现在 (13.9) 还没修复。

激活 "Tear Free"（防撕裂）功能之后，所有新启动的 OpenGL 程序都出现延迟，常表现为只有 30 FPS，合成桌面环境也是。

M132 找到一个解决方法。在 "AMD Catalyst Control Center" (amdcccle) 里完成这些操作：

```
1. 开启 Tear-Free，这将设置 3D V-Sync 为“总是开启”。
2. 设置 3D V-Sync 为“总是关闭”。
3. 确认 Tear-Free 仍为“开启”。
4. 重启 X/重登录。
```

这在 KDE 4.11.x 下有效。M132 的建议：“试着关闭 "Detect refresh rate"，并为合成器指定显示器刷新率。”

背光调节不起效

如果你的背光调节有问题，你可以试试：

```
# aticonfig --set-pcs-u32=MCIL,PP_PhmUseDummyBackEnd,1
```

一些用户报告说这导致 FPS 降低。要恢复默认的话：

```
# aticonfig --set-pcs-u32=MCIL,PP_PhmUseDummyBackEnd,0
```

更多信息见此

使用 plasma 时 Chromium 出现 glitching

添加 --disable-gpu 参数来启动 chromium，比如，

对 /usr/share/applications/chromium.desktop，改成这样：

```
# cat /usr/share/applications/chromium.desktop | grep -i exec
Exec=chromium %U --disable-gpu
```

Xorg 崩溃

不支持的 Xorg 版本

当 Linux 内核版本受支持而 Xorg 版本不受支持时，Xorg 也许会崩溃，而你可能会得到如此的错误信息：

```
(WW) fglrx: No matching Device section for instance (BusID PCI:0@0:17:0) found
(...)
/usr/bin/Xorg.bin: symbol lookup error: /usr/lib/xorg/modules/drivers/fglrx_drv.so:
undefined symbol: GlxInitVisuals2D
xinit: giving up
xinit: unable to connect to X server: Connection refused
xinit: server error
```

比如，fglrx 15.20.3 不支持 Xorg 1.17。

你应该降级你的 Xorg。这些步骤可能会有用。

参见

- Unofficial Wiki for the ATI Linux Driver
- Unofficial ATI Linux Driver Bugzilla

X 服务器

X 服务器不画窗口边框、也不提供通知，它只把画面送上屏幕、把键鼠事件送进来；本章讲这块地基，以及在它上面搭起来的合成、多屏与桌面小工具。必读《Xorg》，安装、运行、配置、输入设备与显示器设置都在这一节；随后读《多显示器》，把 RandR 与 TwinView、Xinerama 的关系一次理清，否则排屏要反复返工；追求透明与阴影再补《Picom》。《光标主题》《Dunst》《Redshift》《XScreenSaver》装完即见效，按需挑读；《Xprofile》《Xhost》《Idesk》《Xephyr》《Xcompmgr》《FreeNX》场景窄，用到再翻。

最容易踩的坑是把“cannot connect to X server :0.0”当成权限被拒，它多半出在显示号与授权文件；另一个是 Xcompmgr 与 Picom 同时启用，两个合成器抢同一份画面，症状看着像驱动故障。

3.1　Xprofile

难度 入门 | **篇幅** 2.6 KB | **所属** X 服务器

本节将讲到

- 兼容性
- 配置

xprofile 文件，`~/.xprofile` 以及 `/etc/xprofile`, 允许您在刚打开 X 会话时运行命令 - 在窗口管理器运行之前。Xprofile 用于随会话自动运行程序，或从显示管理器启动,尤其是那个会话没有自带自动启动程序功能时 - 比如一个独立的窗口管理器。

xprofile 文件在语法和概念上类似 xinitrc, `~/.xinitrc` 和 `/etc/X11/xinit/xinitrc.d/`.

兼容性

xprofile 和 xinitrc 文件在原生情况下会被以下显示管理器引用:

- GDM - `/etc/gdm/Xsession`
- LightDM - `/etc/lightdm/Xsession`
- LXDM - `/etc/lxdm/Xsession`
- SDDM - `/usr/share/sddm/scripts/Xsession`

在用 init 开启会话之时引用 xprofile

使用以下程序启动会话时能够引用 xprofile 文件:

- `startx`
- `xinit`
- XDM

- 任何使用 `~/.xsession` 或者 `~/.xinitrc` 的显示管理器

以上程序，都会直接或间接地执行，`~/.xinitrc` (如果不存在的话，通常是复制自 `/etc/skel/.xinitrc`) 或 `/etc/X11/xinit/xinitrc`. 这就是我们要从下列文件引用 xprofile 的原因.

```
~/.xinitrc 和 /etc/X11/xinit/xinitrc 和 /etc/skel/.xinitrc
#!/bin/sh

# Make sure this is before the 'exec' command or it won't be sourced.
[ -f /etc/xprofile ] && source /etc/xprofile
[ -f ~/.xprofile ] && source ~/.xprofile

...
```

`xinitrc.d/*` 文件已经引用自默认 xinitrc 文件。

配置

首先，如果文件不存在的话创建 `~/.xprofile`. 然后只需加入你想要随会话一同启动的程序的命令。见以下:

```
~/.xprofile
tint2 &
nm-applet &
```

3.2 Xhost

难度 入门 | **篇幅** 3.1 KB | **所属** X 服务器

本节将讲到

- 安装
- 使用方法
- 提示"无法连接到X服务器:0.0"(cannot connect to X server :0.0)
- 另请参阅

来自Xhost man页面:

: xhost程序用于将主机名或用户名添加和删除到允许连接到X服务器的列表中。就主机而言，其提供了一种基本的隐私控制和安全措施。请注意，**它只适用于工作站（单用户）环境**,尽管它可以限制最严重的滥用行为。需要更复杂措施的环境应该实现基于用户的机制或使用协议中传递其他身份验证数据给服务器的钩子。

参见xhost(1)。

安装

安装软件包 xorg-xhost包 。

使用方法

警告：此命令将授予任何本地用户对你的X屏幕的访问权限。这在单用户环境上是可以被接受的，但是通常情况下不能在多用户环境下使用。如果你想保留root用户对X屏幕的访问权限，建议设置 `$XAUTHORITY` 环境变量。默认情况下这个变量由 *sudo* 和 *su* 保留(但不包括`su -`)。

为了让使用*sudo*或*su*运行的应用程序能够访问图形服务器(X屏幕/电脑屏幕)，在终端中以普通用户身份输入:

```
$ xhost +SI:localuser:username
```

要让一切恢复正常，并控制X屏幕的访问权限，输入以下命令:

```
$ xhost -
```

`xhost +` 将完全禁用X身份验证，除非你知道你在做什么，请不要使用这个命令。

提示"无法连接到X服务器:0.0"(cannot connect to X server :0.0)

警告：此命令将会关闭访问控制，这意味着系统上的任何用户，或者网络上的任何用户(如果你打开了X服务器的网络监听)，都可以在没有任何认证的情况下访问你的`$DISPLAY`。这在你的系统上打开了一个安全漏洞，允许其他用户在你的X服务器上启动应用程序(例如键盘记录器)。

你可以使用`xhost +`暂时消除这个提示，或者(在众多方法中)，你也可以选择关闭访问认证来永久解决这个问题,将:

```
xhost + > /dev/null
```

添加到`~/.bashrc`文件中。这样每次启动终端都将自动执行该命令。如果在home文件夹中没有`.bashrc`文件请创建它。如果不在命令中添加`> /dev/null`,每次启动终端时你将会收到信息:*access control disabled, clients can connect from any host*(这是提示你将在root下运行任何软件)。

另请参阅

- 使用cookie和xhost进行X身份验证(错误:"未指定协议(No protocol specified)")#xhost)

3.3 Idesk

难度 入门 | **篇幅** 3.2 KB | **所属** X 服务器

本节将讲到

- 安装
- 配置

Idesk 是一个可以让您在您的X桌面上添加图标的简单程序。它还可以通过一个类似于 Windows 7 中的内置更换程序来管理你的壁纸。

安装

安装 ideskAUR。然后通过以下方式来复制必要的配置文件到您的主目录下：

```
$ mkdir ~/.idesktop
$ cp /usr/share/idesk/dot.ideskrc ~/.ideskrc
```

也可以：

```
$ cp /usr/share/idesk/default.lnk ~/.idesktop/
```

（这就增加了默认的图标，它用来只是运行 Xdialog 并显示欢迎信息。它可以被用作其他图标的模板。）

配置

ideskAUR 包没有 man 页面，但是有一个 readme 文件：`/usr/share/idesk/README`。在 **SourceForge.net** 上也有一份文档，但大多数配置选项都应是不言自明的。

背景选项

如果你正在使用其他壁纸设置工具，类似于 Feh 或 Nitrogen，Idesk的背景设置不需要进行修改。

如果您使用的是 Idesk 自己的背景设置器，它支持包括 JPEG、PNG、GIF 和 XPM 在内的格式。请使用 `Background.File` 或 `Background.Source` 来指定您希望用作墙纸的图像文件路径。

提示：`Background.Source` 似乎优先于 `Background.File` 执行；然而，如果 `Background.Delay` 被设置为 0

，它将会被忽略。

图标

Idesk 将在 `~/.idesktop` 文件夹中以 `.lnk` 为结尾的文件作为图标。每个文件应定义一个图标，如果您试图定义第二个图标，它将被自动忽略。除了以`.lnk`，文件的名称其实并不重要。

以 Chromium 为例：

```
chromium.lnk
table Icon
  Caption: Chromium
  ToolTip.Caption: 谷歌的开源浏览器
  Icon: /usr/share/icons/hicolor/32x32/apps/chromium.png
  Width: 32
  Height: 32
  X: 977
  Y: 369
  Command: chromium
end
```

宽度 和 高度 应符合图标的实际尺寸。 `X` 与 `Y` 将被 Idesk 修改以反映图标的实际位置。

提示：

- 大多数系统图标可以在 `/usr/share/icons/hicolor`，`/usr/share/icons/gnome` 和 `/usr/share/pixmaps` 中找到。
- 很多图标主题都提供了不同尺寸的图标 - 48x48是桌面图标常用的尺寸。

Idesktool

idesk-extrasAUR 包提供了一个 Idesk 的图形化配置工具。可以通过执行 `idesktool` 命令以启动。用户可以通过 Idesktool 来创建并移除图标，更改设置和重启 Idesk。

3.4 Xephyr

难度 入门 | **篇幅** 3.5 KB | **所属** X 服务器

本节将讲到

- 安装
- 运行
- 提示和技巧

Xephyr 是一个嵌套的 X 服务器，作为 X 应用程序运行。

这对于解决编写不当的应用程序可能很有用。例如，Supermicro 服务器可以通过 java ipmi kvm 查看器应用程序进行控制。当服务器重新启动时，应用程序经常重新创建其窗口。它从当前窗口中窃取焦点。这种情况每分钟发生几次，实际上使您的工作变得不可能。同时，如何制定窗口规则来防止此类应用程序的窗口在创建时获得焦点并不明显，因为您希望它在首次启动时获得焦点。使用 xephyr 可以将这些窗口重新创建保留在单独的窗口中，这不会窃取当前打开的窗口的焦点。

安装

安装 xorg-server-xephyr包 包， xorg-server-xephyr 已收录于官方仓库中，可以用 pacman 进行安装。

运行

如果希望运行一个嵌套的 X 窗口，你需要指定一个新的 **DISPLAY**：`1`。

```
$ Xephyr -br -ac -noreset -screen 800x600 :1
```

这样将会启动一个新的 Xephyr 窗口，这个窗口的 **DISPLAY** 序号是":1"。为了在这个窗口里运行一个应用，你需要明确指定这个 **DISPLAY**：

```
$ DISPLAY=:1 xterm
```

启动窗口管理器

如果想启动其他窗口管理器（例如 spectrwm ），则输入：

```
$ DISPLAY=:1 spectrwm
```

也可以通过 startx 命令让 Xephyr 使用你的 xinitrc ：

```
$ startx -- /usr/bin/Xephyr :1
```

切换用户键鼠焦点

如果想释放 Xephyr 窗口的聚焦，在 Xephyr 窗口中按 `Ctrl+Shift` 可以锁定/解锁鼠标指针和键盘输入。

发送Alt+Tab

如果使用 KDE，请创建一个窗口规则以忽略全局快捷方式。然后你可以在 **Xephyr** 中使用 `Alt+Tab`。 非KDE可以使用屏幕键盘 florenceAUR，也可使用xorg-xkbcomp包工具修改按键映射。

提示和技巧

Xephyr 其他有用的情况示例包括:

1. X 应用程序或功能的测试环境，测试人员希望继续在通常的 X 环境中工作，同时保护其他应用程序免受被测应用程序故障的影响。
2. OpenSSH#Remote emphasize 3 settings in the sshd server configuration file for OpenSSH#X11 forwarding (over ssh). 2 of these, out of 3, are the default settings. When the ssh client can not influence the ssh server administrator to set the 3rd one, `X11Forwarding`, to yes, Forwarding X11 over ssh uses Xephyr as a work around to be installed in the ssh client machine.

3.5 Xcompmgr

难度 入门 | **篇幅** 5.2 KB | **所属** X 服务器

本节将讲到

- 安装
- 配置
- 小提示
- 疑难解答

Xcompmgr是一个简单的混合窗口管理器，可以实现阴影、原生窗口透明（配合`transset`工具）等特效。Xcompmgr设计初衷只是实现混合窗口管理器的概念，所以比起同类混合窗口管理器如 Compiz Fusion，Xcompmgr轻量许多。

Xcompmgr不替代任何窗口管理器，所以对于Openbox和Fluxbox这类缺乏特效的窗口管理器来讲，配合Xcompmgr能得到更华丽的视觉效果。

安装

在安装 Xcompmgr 前，请先确认您已经正确安装并配置了 Xorg。如果你使用了自定义配置，请打开X服务器中的 Composite 拓展。

您可以直接从官方软件仓库中安装xcompmgr包以及透明度设置工具transset-dfAUR。

配置

运行 `xcompmgr`：

```
$ xcompmgr -c
```

如果要在每次 Xorg 启动时运行，添加以下内容到xprofile：

```
xcompmgr -c &
```

用户可以通过命令行参数调整阴影、消隐等效果，以下是一个常用的配置：

```
xcompmgr -c -C -t-5 -l-5 -r4.2 -o.55 &
```

查看命令行帮助：

```
$ xcompmgr --help
```

窗口透明度

`transset-df` 工具用来设置单个窗口的透明度，透明特效可能会降低系统性能。

先启动要设置透明度的程序，然后运行：

```
transset-df n
```

.. 此处 `n` 是0到1之间的数字，0表示完全透明（不可见），1表示不透明。例如：`transset .25` 代表75%的透明度。

运行后，鼠标会变成十字形。点击要设置的窗口，即可应用透明度设置。

小提示

方便开关xcompmgr的脚本

把这个脚本放在合适的应用程序目录，可以方便地启动、关闭xcompmgr。

```
#!/bin/bash
#
# Start a composition manager.
# (xcompmgr in this case)

comphelp() {
  echo "Composition Manager:"
  echo "    (re)start: COMP"
  echo "    stop:      COMP -s"
  echo "    query:     COMP -q"
  echo "               returns 0 if composition manager is running, else 1"
  exit
```

```
}

checkcomp() {
  pgrep xcompmgr &>/dev/null
}

stopcomp() {
  checkcomp && killall xcompmgr
}

startcomp() {
  stopcomp
# Example settings only. Replace with your own.
  xcompmgr -CcfF -I-.015 -O-.03 -D6 -t-1 -l-3 -r4.2 -o.5 &
  exit
}

case "$1" in
    "") startcomp ;;
  "-q") checkcomp ;;
  "-s") stopcomp; exit ;;
     *) comphelp ;;
esac
```

仅用一个快捷键就切换 xcompmgr

利用上面脚本的状态部分，将下面脚本设置为某个快捷键的脚本：

```
#!/bin/bash

if pgrep xcompmgr &>/dev/null; then
       echo "Turning xcompmgr ON"
       xcompmgr -c -C -t-5 -l-5 -r4.2 -o.55 &
else
       echo "Turning xcompmgr OFF"
       pkill xcompmgr &
fi

exit 0
```

疑难解答

Mozilla Firefox 打开 Flash 时崩溃

新建可执行文件`/etc/profile.d/flash.sh`，文件内容如下：

```
export XLIB_SKIP_ARGB_VISUALS=1
```

这将关闭混合特效。

登录后变成浅灰色背景 （如Openbox）

安装hsetroot包，在运行xcompmgr之前，执行`hsetroot -solid "#000000"`（将000000替换成你想要的颜色代码）。

在 awesome 窗口管理器中出现 BadPicture request

如果你在 awesome 得到以下错误信息：

```
 error 163: BadPicture request 149 minor 8 serial 34943
 error 163: BadPicture request 149 minor 8 serial 34988
 error 163: BadPicture request 149 minor 8 serial 35033
```

只需要安装软件包 feh 然后重启 awesome。

Screen not updating in awesome after resolution change

When using an external monitor, you may encounter problems when automatically changing display resolutions: a part of the screen becomes "stuck" and no longer updates itself. This problem occurs because of the initial resolution change (happening before Xcompmgr starts) as well as awesome setting the background via feh.

To fix it, you need to 安装 hsetroot包 and put the following line in `.xinitrc`, just before `xcompmgr`:

```
hsetroot -solid "#000066"
```

(you can replace *#000066* with your color of choice).

3.6 光标主题

难度 进阶 | **篇幅** 11.2 KB | **所属** X 服务器

本节将讲到

- 安装
- 配置
- 疑难解答
- 另见

显示服务器有着一个从各方面帮助GUI界面导航和操作的*光标主题*。显示服务器自带一个光标主题,但依然可以安装并使用其他光标主题。

安装

可以通过软件包安装,也可以下载光标主题并解压到恰当的路径来安装。

通过软件包

下列链接提供可用的光标主题软件包:

- 官方仓库——"xcursor-" search.
- AUR——"cursor" search.

手动安装

在官方仓库或AUR中没有的光标主题可以手动安装。许多网站都有可下载的光标主题。下载之后它们需要被放进*icons*目录(因为光标可以和图标主题绑定在一起)。

一些有光标主题的网站:

- GNOME Look
- Deviant Art
- Open Desktop

如果想要*为指定用户*安装,使用 `~/.local/share/icons/` 或 `~/.icons/` 目录。用以下命令可以解压大多数归档:

```
$ tar xvf foobar-cursor-theme.tar.gz -C ~/.local/share/icons
```

光标主题的目录结构为 `theme-name/cursors`,举个例子: `~/.local/share/icons/*theme*/cursors/`; 请确保解压出文件也是这样的结构。

注意：如果想要*系统范围*安装,使用 `/usr/share/icons` 目录。不推荐直接将文件解压到这个目录, 因为手动添加的文件不会被 pacman 追踪;建议给光标主题创建一个 package 。

已经安装的光标主题可以通过这条命令查看:

```
find /usr/share/icons ~/.local/share/icons ~/.icons -type d -name "cursors"
```

如果软件包包含一个 `index.theme` 文件,检查里面是否有 "Inherits" 这一行。如果有,检查继承的主题是否也存在(有必要时重命名)。

配置

有许多方法设置光标主题。

XDG 规范

这个方法对 X11 和 Wayland 光标主题都有效。

如果想要*为指定用户*配置,创建并编辑 `~/.icons/default/index.theme`;如果想要*系统范围*配置,编辑 `/usr/share/icons/default/index.theme`。

`[icon theme]` 部分中的 `Inherits` 选项必须被设置为X光标主题的目录名 `*cursor_theme_name*`,举个例子 `xcursor-breeze-snow`:

```
~/.icons/default/index.theme
[icon theme]
Inherits=cursor_theme_name
```

然后你应该编辑 `~/.config/gtk-3.0/settings.ini`, 将 `*cursor_theme_name*` 替换为你选择的主题:

```
~/.config/gtk-3.0/settings.ini
[Settings]
gtk-cursor-theme-name=cursor_theme_name
```

重启 X 以应用更改。

如果还是没用,试着创建一个将`~/.icons/default/cursors` (假设*为指定用户安装*)指向`.local/share/icons/*cursor_theme_name*/cursors`的符号链接然后再次重启 X。

LXAppearance

LXAppearance通过创建 `~/.icons/default/index.theme`设置默认光标主题:如果你手动编辑它,LXAppearance 会覆写它。记得还要按照 #XDG 规范手动编辑 `~/.config/gtk-3.0/settings.ini`,因为有些应用,如 Firefox ,使用这个设置文件。

桌面环境

桌面环境使用 X设置协议,通常通过设置守护进程达到效果。虽然这允许即时更改光标,但应用的主题在不同应用中可能不一致, 另见 #XDG 规范来手动更改光标主题.

GNOME

在 GNOME 中更改主题, 使用 gnome-tweaks包 ,或用这条命令直接设置:

```
$ gsettings set org.gnome.desktop.interface cursor-theme cursor_theme_name
```

用这条命令修改尺寸(尺寸有 24,32,48,64,因主题而不同):

```
$ gsettings set org.gnome.desktop.interface cursor-size cursor_theme_size
```

注意：在 Wayland 上, Gnome 应用默认无法显示位于`~/.local/share/icons`的光标主题。作为一种解决方法,你可以 add that path to XCURSOR_PATH.

MATE

在 MATE 中可以使用 mate-control-center 或 gsettings 来修改主题:

```
gsettings set org.mate.peripherals-mouse cursor-theme cursor_theme_name
```

修改尺寸:

```
gsettings set org.mate.peripherals-mouse theme-size
```

XFCE

修改 X光标主题 , 使用:

```
xfconf-query --channel xsettings --property /Gtk/CursorThemeName --set
cursor_theme_name
```

修改尺寸:

```
xfconf-query --channel xsettings --property /Gtk/CursorThemeSize --set
cursor_theme_size
```

X resources

要想在本地命名光标主题,将下面这一行添加到 `~/.Xresources` 文件:

```
Xcursor.theme: cursor-theme
```

要想正确地加载光标主题需要通过窗口管理器完成;如果窗口管理器无法完成,可以在输入 `~/.xinitrc` 或 .xprofile (使用哪个取决于个人配置)中添加以下命令强制在窗口管理器之前加载:

```
$ xrdb ~/.Xresources
```

可选地,如果你的光标主题支持多个尺寸,在 `~/.Xresources` 中添加下面这一行:

```
Xcursor.size: 16
```

提示：32, 48 或 64 也可能是适用的尺寸

如果对支持的光标大小有疑问,在没有这个设置的情况下启动 X 并让它自动选择光标尺寸(查阅你的窗口管理器文档以获得更多细节)。

环境变量

你可以使用一个环境变量来为单个应用暂时设置主题,举个例子:

```
$ XCURSOR_THEME=SomeThemeName xclock
```

XCURSOR_SIZE 是可选的,如果你的光标主题支持多个尺寸就可以使用。

如果你的光标主题安装在 `~/.local/share/icons`,为了避免潜在的问题,将该路径添加到 XCURSOR_PATH 。举个例子:

```
~/.bash_profile
export XCURSOR_PATH=${XCURSOR_PATH}:~/.local/share/icons
```

显示管理器

光标主题通常可以通过显示管理器设置,但注意光标主题可能不会跨用户会话延续。

GDM

请见 GDM#更改光标主题.

疑难解答

创建丢失光标的链接

当缺少一些光标时,应用可能使用默认光标。这可以通过创建丢失光标的链接来纠正。举个例子:

```
$ cd ~/.icons/theme/cursors/
$ ln -s right_ptr arrow
$ ln -s cross crosshair
$ ln -s right_ptr draft_large
$ ln -s right_ptr draft_small
$ ln -s cross plus
$ ln -s left_ptr top_left_arrow
$ ln -s cross tcross
$ ln -s hand hand1
$ ln -s hand hand2
$ ln -s left_side left_tee
$ ln -s left_ptr ul_angle
$ ln -s left_ptr ur_angle
$ ln -s left_ptr_watch 08e8e1c95fe2fc01f976f1e063a24ccd
```

如果以上命令没能解决问题,看看 `/usr/share/icons/whiteglass/cursors` 来获取你的主题丢失的光标,并且给它们也创建链接。

提示：你也可以移除不想要的光标。移除 "watch" 光标的例子:

```
$ cd ~/.icons/theme/cursors/
$ rm watch left_ptr_watch
$ ln -s left_ptr watch
$ ln -s left_ptr left_ptr_watch
```

提供丢失的光标

一些程序设置了它们特制的光标 `~/.Xresources` ,你可能想要覆盖它们。一个常见的例子是 rdesktop,它与 Microsoft Windows 电脑连接并使用远程机器的光标,这个光标因协议限制导致转译质量差,通常很难看清。

这可以通过用相同(或另一个)光标主题替换这些光标来解决。想要达成目的,必须获得图像的**hash**。这可以通过在启动设置这些光标的程序前设置 `XCURSOR_DISCOVER` 环境变量来完成:

```
$ XCURSOR_DISCOVER=1 rdesktop ...
```

在首次(而且只有首次)光标设置时,一些细节会被展示出来,像这样

```
Cursor image name: 240200000028000005280000084810000
...
Cursor image name: 7bf1cc07d310bf080118007e08fc30ff
...
Cursor hash 240200000028000005280000084810000 returns 0x0
```

当X光标寻找丢失的光标时,搜索的路径包含了`~/.icons/default/cursors` 所以这里可以放置用于X光标搜索的图像。首先,如果还没有这个目录就创建它:

```
$ mkdir -p ~/.icons/default/cursors
```

然后将hash链接到目标图像。这里我们使用来自 `Vanilla-DMZ` 光标主题的 `left_ptr` 图像

```
$ ln -s /usr/share/icons/Vanilla-DMZ/cursors/left_ptr
~/.icons/default/cursors/240200000028000005280000084810000
```

应用重启后就可以看到更改效果。不需要特殊的启动应用的方法。

rdesktop

这里有一些 rdesktop 连接运行 Windows 7 的远程机器时常见的 Microsoft Windows 光标。不幸的是,动态光标难以被覆盖因为它们是被逐帧发送的,所以每一帧都需要一个映射!

```
$ ln -s /usr/share/icons/$THEME/cursors/xterm
~/.icons/default/cursors/00000000017e000002fc000000000000
$ ln -s /usr/share/icons/$THEME/cursors/right_ptr
~/.icons/default/cursors/000000930000108600006311000006609
$ ln -s /usr/share/icons/$THEME/cursors/plus
~/.icons/default/cursors/01e00000201c00004038000080300000
$ ln -s /usr/share/icons/$THEME/cursors/left_ptr
~/.icons/default/cursors/240200000028000005280000084810000
$ ln -s /usr/share/icons/$THEME/cursors/left_ptr_watch
~/.icons/default/cursors/6ce0180090108e0005814700a0021400
$ ln -s /usr/share/icons/$THEME/cursors/hand
~/.icons/default/cursors/d2201000a2c622004385440041308800
$ ln -s /usr/share/icons/$THEME/cursors/watch
~/.icons/default/cursors/fc618c00da110f0034fd0e004e082400
```

更改默认 X 形光标

默认的 X 形光标出现在没有设置默认光标为 left_ptr 的窗口管理器或用 XCB 代替 Xlib 的窗口管理器(如 awesome)中。

要想修复只需将下列内容添加到你的 `~/.xinitrc` ,X会话 或 窗口管理器启动配置(如果可以的话,例如 bspwm 的 bspwmrrc)。

```
$ xsetroot -cursor_name left_ptr
```

光标风格列表在 X 协议的 appendix B 中。

.Xdefaults

如果你有冲突的光标,它可能是因为一个不同的光标已经在 `~/.Xdefaults` 文件中被设置。

光标大小在启动时不会改变

如果你正试着通过你的 `~/.xinitrc` 中的 `~/.Xresources` 并且它没用的话,确保 xrandr 在加载 `~/.Xresources` 运行。

确保你的 `~/.xinitrc` 看起来类似于下面这样

```
~/.xinitrc
xrandr
...
xrdb -merge ~/.Xresources
exec wm
```

Plasma (Wayland) 上的光标大小或主题不变

在使用 Wayland 下的 Plasma 修改光标尺寸或主题时,确保在应用更改后重启。 .

另见

- Xcursor(3)——获取有关 X 中光标的更多信息(支持的 目录, 格式, 兼容性等等)。
- 要想创作光标文件, 请见 Xcursorgen

3.7 Dunst

难度 进阶 | **篇幅** 11.5 KB | **所属** X 服务器

本节将讲到

- 安装
- 外观
- 快捷键
- 规则
- 暂停 Dunst

翻译状态：

- 本文（或部分内容）译自 Dunst，最近一次同步于 2026-03-25，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。

- 本文可能与英文原文存在出入。

Dunst 是大多数桌面环境提供的通知守护程序的轻量级替代品。

dunst 可以在 X11 或 Wayland 下运行。

安装

安装 dunst包 软件包。

示例配置文件位于 `/etc/dunst/dunstrc`。将其复制到 `~/.config/dunst/dunstrc` 并进行编辑。

启动 `/usr/bin/dunst`，确保您的窗口管理器或桌面环境在启动/登录时启动 Dunst 。

注意：您似乎无需手动启动 Dunst，因为当应用程序向 D-Bus 发送通知时，dbus-daemon 会自动启动 Dunst。然而，通知服务经常会安装多个守护进程，且无法知晓应当自动启动哪一个。dbus-daemon 的维护者曾明确警告不要依赖自动启动多提供者服务。

外观

可为通知内的文本添加样式如粗体、斜体、删除线或下划线。完整参考见 Pango 标记。若 `markup` 被设为 `none`，Pango 就会从通知中被移除。

通知的格式可被明确规定，选项如下：

```
%a  应用名称
%s  摘要
%b  主体内容
%i  图标名称（包含路径）
%I  图标名称（无需路径）
%p  进度（需要另外设定，[0%] - [100%]）
```

它们可以与 HTML 标签结合起来使用。例如将 `format` 设为 `<b>%s</b>\n%b` 来显示加粗的摘要、换行和无格式的主体内容。

设定图标

在配置文件的 `global` 节 `icon_path` 项中设定图标。需要状态（status）、设备（device）和传统（legacy）图标。默认情况下，Dunst 会查找 gnome-icon-themeAUR 图标。若要使用 adwaita-icon-theme包替换 gnome-icon-theme，请改为：

```
icon_path
= /usr/share/icons/Adwaita/16x16/status/:/usr/share/icons/Adwaita/16x16/devices/:/usr/share/icons/Adwaita/16x16/legacy/
```

除了直接指定图标文件夹的路径，您也可以在 `global` 节中直接指定图标主题。若如此，您需要设置 `enable_recursive_icon_lookup=true` 以在主题主文件夹的子文件夹中进行搜索。

例如：

```
icon_theme = Papirus
enable_recursive_icon_lookup = true
```

快捷键

使用 *dunstctl* 控制 Dunst。您可以更新您的快捷键来使用 *dunstctl*。

例如，关闭所有通知：

```
$ dunstctl close-all
```

显示历史记录：

```
$ dunstctl history-pop
```

规则

你可以在你的 dunstrc 文件中创建匹配某些通知的规则，然后执行某些操作比如运行一个脚本。

匹配通知

要创建新的规则，请在您的配置文件中创建一个自定义命名的节， 您可以使用属性 appname（应用名）、summary（摘要）、body（主体）、icon（图标）、category（类别）、match_transient（瞬态通知）和 msg_urgency（紧急程度）来匹配通知。 支持通配符。请在#脚本查看示例。 用 `-print` 选项启动 Dunst 以在通知中寻找有用的信息来编写合适的规则。关于各属性的详细解释，请参阅 dunst(5) § filtering。

修改格式

当匹配到一条通知时，您可以对其格式进行修改，只需修改 `format` 选项。这在您想完全忽略某些特定的通知时尤其有用。若如此做，您只需将 `format=""` 行添加到您的规则（该规则会把匹配到的通知改成空的）。

另一个有用的特性是您可以完全将某些通知从历史记录中排除，例如您想将 Dunst 用作音量指示器，只需将 `history_ignore=yes` 行添加到相应的规则。

脚本

Dunst 可以根据某些通知内容来运行脚本，以下是一个当某来自 pidginAUR 的人登录时，Dunst 运行脚本的示例：

```
[signed_on]
   appname = Pidgin
   summary = "*signed on*"
   urgency = low
   script = do_something.sh
```

指定的脚本将按以下参数的顺序来匹配：appname（应用名）、summary（摘要）、body（主体内容）、icon（图标）、urgency（紧急程度）。

暂停 Dunst

有两种方法暂停 Dunst：

使用 **`dunstctl`**：您可以执行 `dunstctl set-paused true/false/toggle` 来禁用/重新启用/切换通知状态。执行 `dunstctl is-paused` 来查看 Dunst 是否在运行/暂停。

使用 **`killall`**：执行 `killall -SIGUSR1 dunst` 来禁用，执行 `killall -SIGUSR2 dunst` 来重新启用。

一旦暂停，Dunst 会保留所有通知。再次启用 Dunst 后，将显示所有保留的通知。

Dunstify

Dunstify 是 notify-send 命令的替代品，它完全兼容 notify-send 且可以与其一起使用，但它提供了更多功能。

除了 notify-send 中可用的一些选项，Dunstify 还提供一些其他特性如 ID 和 action（行为）。

替换通知

您可以给 dunstify 加上 `-r ID` 选项来给一条通知分配 ID，其中 `ID` 必须是整数。 如果拥有某 ID 的通知已经存在，那么它将会被具有相同 ID 的新通知替换。 您亦可使用 `dunstify -C ID` 来关闭一条通知。

然而，在大部分情况下，使用标签（tag）比控制 ID 更可取，因为后者有很多陷阱。在调试或面对非常复杂的通知发送者时，请考虑替换 ID。

具有相同标签（tag，本例中的 tag 是 test）的通知会被替换且无视 ID。

```
$ dunstify -h string:x-dunst-stack-tag:test Test -A 'tested,default'
$ dunstify -h string:x-dunst-stack-tag:test Testing
```

行为

您可以从一个或多个 `--action=action,label` 参数中定义可以直接调用的行为。例如：

```
$ dunstify --action="replyAction,reply" "Message received"
```

然后用户可以通过 Dunst 的上下文菜单来做出某些行为，发送到 dunstify 的某些请求会被阻止除非通知消失或一个行为已被选择。在前一种情况下，若通知超时，则 Dunstify 会返回1；若通知被手动关闭，Dunstidy 会返回2。而在后一种情况下，会返回在上下文菜单中选择的行为。

除了通过上下文菜单调用行为，您也可以通过鼠标事件调用行为。这可使 Dunst 更具交互性，这里也建议这样做。当通知只有一个行为或一个行为被命名为 “default” 时，您可以单击鼠标中键来调用这个行为。（Dunst 默认如此，当然您也可以在 `dunstrc` 中定义 `mouse_middle_click = do_action`).

```
reply_action () {}
forward_action () {}
handle_dismiss () {}

ACTION=$(dunstify --action="default,Reply" --action="forwardAction,Forward" "Message
Received")

case "$ACTION" in
"default")
    reply_action
    ;;
"forwardAction")
    forward_action
    ;;
"2")
    handle_dismiss
    ;;
esac
```

提示与技巧

将 dunstify 用作音量/亮度水平指示器

您可以使用替换 ID 功能来实现简单的音量或亮度指示器通知，如图所示 。

要实现音量指标，请将以下脚本放在 `PATH` 中的某个位置。

```
#!/bin/bash
# changeVolume

# Arbitrary but unique message tag
msgTag="myvolume"

# Change the volume using alsa(might differ if you use pulseaudio)
amixer -c 0 set Master "$@" > /dev/null

# Query amixer for the current volume and whether or not the speaker is muted
volume="$(amixer -c 0 get Master | tail -1 | awk '{print $4}' | sed 's/[^0-9]*//g')"
mute="$(amixer -c 0 get Master | tail -1 | awk '{print $6}' | sed 's/[^a-z]*//g')"
if [[ $volume == 0 || "$mute" == "off" ]]; then
    # Show the sound muted notification
    dunstify -a "changeVolume" -u low -i audio-volume-muted -h string:x-dunst-stack-
tag:$msgTag "Volume muted"
else
    # Show the volume notification
    dunstify -a "changeVolume" -u low -i audio-volume-high -h string:x-dunst-stack-tag:
$msgTag \
    -h int:value:"$volume" "Volume: ${volume}%"
fi
```

```
# Play the volume changed sound
canberra-gtk-play -i audio-volume-change -d "changeVolume"
```

现在只需将 `changeVolume 2dB+ unmute` 等绑定到某个热键上即可。您可能还想让 dunst 在历史记录中忽略此类通知，请参阅#修改格式。

覆盖以前的通知

对于某些通知（例如声音或亮度），你可能希望覆盖之前的通知。你可以使用#替换通知中的 Dunst 方法，也可以参考桌面通知程序#替换之前的通知，了解更一般的示例。

疑难解答

Dunst 无法通过 systemd 启动

在无登录管理器的情况下使用 Dunst 时，`DISPLAY` 环境变量可能不会被正确设置。

欲修复，请添加以下行到您的 `.xinitrc` 文件：

```
systemctl --user import-environment DISPLAY
```

提示：这在最近版本的 systemd 上会自动作为 `/etc/X11/xinit/xinitrc.d/50-systemd-user.sh` 的一部分被执行。

字体大小不匹配（表情比字大）

这是因为 fontconfig包 没有重新调节位图字体。您通常会在某些表情符号上注意到这个问题。（例如：noto-fonts-emoji包 ）

要解决这个问题，只需执行：

```
# ln -s /etc/fonts/conf.avail/10-scale-bitmap-fonts.conf /etc/fonts/conf.d/
```

并重启 Dunst。

部分应用程序的通知不遵守超时规则

这个问题具体表现为所有紧急级别的通知超时配置为 30 秒，但是来自特定应用如 Discord 、Mattermost 和 GitLab 的通知消失得很快（典型表现为3秒）。

您可以在上游找到更多细节：issue #276。

这是由于通知被强制关闭而发生的。要解决这个问题，您需要引入一个名为 `ignore_dbusclose` 的特殊参数。您可以通过启用该参数来忽略通过 D-Bus 发送的 `closeNotification` 消息。启用该参数可以确保应用程序通知遵循 Dunst 的配置。默认情况下，该参数是关闭（被设定为 false）的。

参见

- Dunst 官方网站

- Dunst Github 仓库
- Dunst FAQ
- Dunst 文档

3.8 Redshift

难度 进阶 | **篇幅** 11.9 KB | **所属** X 服务器

本节将讲到

- 安装
- 使用
- 配置
- 疑难解答
- 参见

来自 Redshift project web page:

: Redshift) 会根据你的周围环境调节屏幕色温，如果你晚上在屏幕前工作，这可能帮助你减少对眼睛的伤害。 这个程序启发于 f.lux。

注意：Redshift 不支持 Wayland 因为它无法调节色温 。查看 Backlight#Wayland 了解更多。

安装

安装 redshift包 包。或者安装 redshift-minimalAUR 包（最小依赖版本）。

前端

redshift-gtk 命令 来自 redshift包 包 提供托盘图标用于控制 Redshift。 请参阅可选的依赖。

另外，也可以使用这些前端 redshiftgui-binAUR (GTK) 和 redshift-qtAUR, redshiftconfAUR 或者 和 plasma5-applets-redshift-control-gitAUR (Qt)。

使用

Redshift 需要你的位置才能开始运行 (除非使用 `-o` 选项)，即需要你所在位置的经纬度。Redshift 使用一些程序获得你的位置。如果这些程序不工作 (比如这些程序都没有安装), 你需要手动输入你的位置。

快速开始

以基本的设置启动：

```
$ redshift -l LATITUDE:LONGITUDE
```

LATITUDE 为所在位置的维度， *LONGITUDE* 为所在位置的经度。

立即更改屏幕色温：

```
$ redshift -P -O TEMPERATURE
```

TEMPERATURE 为期望的色温值 (介于 `1000` 和 `25000` 之间)。

自动启动

有几个选项可以自动启动Redshif：

- 鼠标右键点击托盘图标选择 *Autostart* 当 *redshift-gtk* 或者 *plasma5-applets-redshift-control* 已经启动的时候。
- 放置 Redshift Desktop entry 在 `~/.config/autostart/` 或通过添加 `redshift` 到你的窗口管理器或者桌面环境的 Autostarting 方法中。
- 使用 Systemd/User。 软件提供了两个 srvices: `redshift.service` 和 `redshift-gtk.service`。激活哪一个服务取决于你是否想要显示系统托盘图表。

注意：

- The Redshift service files contain `Restart=always` so they will restart infinitely. See systemd.service(5).
- When using a systemd user service, Xorg must be started before execution of the service, which is not the case without a Display manager. Otherwise you will get `RANDR Query Version' returned error -1` and `Initialization of randr failed`.

配置

Redshift 会读取配置文件 `~/.config/redshift/redshift.conf` （如果该文件存在的话）。 然而，Redshift 不会自动创建这个文件, 因此你可能需要手动创建它。配置文件示例：redshift.conf.sample。

Automatic location based on GeoClue2

In order to allow access Redshift to use GeoClue2, add the following lines to `/etc/geoclue/geoclue.conf`:

```
/etc/geoclue/geoclue.conf
[redshift]
allowed=true
system=false
users=
```

Restart `redshift.service` and/or any other Redshift instance to apply the changes.

注意：

- This workaround is not needed with Geoclue2 version 2.5.0 and above.
- If using GNOME, also toggle Location Services to "On" in *Settings* > *Privacy*.
- Due possible bugs with geoclue2 and Redshift , it may be required to use the `manual` location-provider instead, e.g. for Paris:

```
~/.config/redshift/redshift.conf
[redshift]
...
; Set the location-provider: 'geoclue2', 'manual'
; type 'redshift -l list' to see possible values.
; The location provider settings are in a different section.
location-provider=manual

...

; Keep in mind that longitudes west of Greenwich (e.g. the Americas)
; are negative numbers.
[manual]
lat=48.853
lon=2.349
```

- If using i3wm or similar, you will also need to enable the geoclue agent on startup. As well as `systemctl --user enable redshift-gtk` or `redshift` user service.

```
~/.i3/config
...
exec --no-startup-id /usr/lib/geoclue-2.0/demos/agent
...
```

Automatic location based on GPS

You can also use gpsd包 to automatically determine your GPS location and use it as an input for Redshift. Create the following script and pass `$lat` and `$lon` to `redshift -l $lat;$lon`:

```
#!/bin/bash
date
#gpsdata=$( gpspipe -w -n 10 |   grep -m 1 lon )
gpsdata=$( gpspipe -w | grep -m 1 TPV )
lat=$( echo "$gpsdata"  | jsawk 'return this.lat' )
lon=$( echo "$gpsdata"  | jsawk 'return this.lon' )
alt=$( echo "$gpsdata"  | jsawk 'return this.alt' )
dt=$( echo "$gpsdata" | jsawk 'return this.time' )
echo "$dt"
echo "You are here: $lat, $lon at $alt"
```

For more information, see this forums thread.

Use real screen brightness

Redshift has a brightness adjustment setting, but it does not work the way most people might expect. In fact it is a fake brightness adjustment obtained by manipulating the gamma ramps, which means that it does not reduce the backlight of the screen.

Changing screen backlight is possible with redshift hooks and xorg-xrandr包 and xorg-xbacklight包 but, please see Backlight#xbacklight as there are some limitations and you may have to find another method of controlling the backlight depending on your hardware.

You need to create a file in `~/.config/redshift/hooks` and make it executable. You can use and edit the following example:

```
$ mkdir -p ~/.config/redshift/hooks
```

Create and adjust the following script:

```
~/.config/redshift/hooks/brightness.sh
#!/bin/sh
```

```
# Set brightness via xbrightness when redshift status changes

# Set brightness values for each status.
# Range from 1 to 100 is valid
brightness_day=100
brightness_transition=50
brightness_night=10
# Set fade time for changes to one minute
fade_time=60000

if [ "$1" = period-changed ]; then
       case $3 in
              night)
                     xbacklight -set $brightness_night -time $fade_time
                     ;;
              transition)
                     xbacklight -set $brightness_transition -time $fade_time
                     ;;
              daytime)
                     xbacklight -set $brightness_day -time $fade_time
                     ;;
       esac
fi
```

Make it executable:

```
$ chmod +x ~/.config/redshift/hooks/brightness.sh
```

Restart the `redshift.service` to apply changes.

Check the service status as it should **not** contain the following message:

```
redshift[..]: No outputs have backlight property
```

疑难解答

Screen 1 could not be found

修改配置文件 "redshift.conf" 将 "screen 1" 改为 "screen 0"。

点击托盘图表不工作

安装 libappindicator-gtk3包。 请看 redshift issue 363 and FS#49971.

Redshift 让屏幕在设置的色温值和默认的色温值之间快速闪烁

确认没有多个Redshift实例在同时运行。

Redshift 使用命令启动时可以工作 但是以 systemd service 启动失败

systemd unit 有一行在 `redshift.service` 文件中，要求必须在`display-manager.service` unit 被一个显示管理器 display manager 启动后才能启动。 如果你不使用显示管理器, 编辑 `redshift.service` 用户服务文件删除 `After=display-manager.service` 行. 运行 `systemctl --user daemon-reload` 后服务应该会正确初始化。

Redshift temporarily resets using some wine apps that reset gamma values

If you notice that using some wine apps, redshift seems to reset temporarily upon launch, or adjusting settings, or etc, then there is a useful registry key that seems to alleviate this. See and . Set or create the string value

```
HKEY_CURRENT_USER\Software\Wine\X11 Driver
UseXVidMode="N"
```

using the registry editor, or import/set it otherwise.

Redshift GDBus.Error:org.freedesktop.DBus.Error.AccessDenied on start

If running `$ redshift` and you are getting:

```
$ redshift
Trying location provider `geoclue2'...
Using provider `geoclue2'.
Unable to start GeoClue client: GDBus.Error:org.freedesktop.DBus.Error.AccessDenied:
'redshift' disallowed, no agent for UID 1000.
Unable to connect to GeoClue.
Unable to get location from provider.
```

or running `$ redshift-gtk` and getting the similar error:

```
$ redshift-gtk
Failed to run Redshift
Trying location provider `geoclue2'...
Unable to start GeoClue client:
GDBus.Error:org.freedesktop.DBus.Error.AccessDenied:
'redshift' disallowed, no agent for UID 1000.
Unable to connect to GeoClue.
Unable to get location from provider.
```

You can create a systemd unit file in `~/.config/systemd/user/geoclue-agent.service` with the following config:

```
~/.config/systemd/user/geoclue-agent.service
[Unit]
Description=redshift needs to get a (geo)clue

[Service]
ExecStart=/usr/lib/geoclue-2.0/demos/agent

[Install]
WantedBy=default.target
```

Start and enable the service with systemctl: `$ systemctl --user enable --now geoclue-agent.service` and try running redshift again.

If you still get the same error, it may be because of geoclue being locked down to a few programs by default. Try adding the following lines to `/etc/geoclue/geoclue.conf` (see redshift issue 158 and FS#40360) and run `$ redshift` again:

```
/etc/geoclue/geoclue.conf
[redshift]
allowed=true
system=false
users=
```

参见

- Redshift website
- Redshift on github
- Wikipedia:Redshift (software))

- Similar software: blugonAUR, xflux-gui-gitAUR

3.9 XScreenSaver

难度 进阶 | **篇幅** 12.6 KB | **所属** X 服务器

本节将讲到

- 安装
- 配置
- 使用
- 提示和技巧
- 问题解决

XScreenSaver 是 X 窗口系统的屏保和锁屏工具。

安装

安装 xscreensaver包 包。

如果安装 xscreensaver-arch-logoAUR 包，可以获得有 Arch Linux 标志的外观。

配置

大多数选项都可以通过运行 *xscreensaver-settings* 来逐个用户配置的。*xscreensaver-settings* 会将所选的配置写入 `~/.xscreensaver`，并丢弃对文件的任何手动修改。全局选项定义在 `/usr/share/X11/app-defaults/XScreenSaver` 中。

至少从 XScreenSaver 5.22 开始，有了另一种方法来编辑 XScreenSaver 的用户配置：使用 Xresources。

主题

从 6.0 版本开始，XScreenSaver 预装了一些主题，选择方式：

- *xscreensaver-settings*
- 修改 `~/.xscreensaver` 配置文件中的 `dialogTheme` 选项 (`dialogTheme: *themename*`
- 使用 X resources: `xscreensaver-auth.dialogTheme: *themename*`).

用 X resources 可以自定义主题，下面的例子修改了颜色和字体。如果使用的是非默认主题，请将 `default` 修改为所选主题的小写名字，使用问号 (`?`) 会影响所有主题:

```
~/.Xresources
! Font for regular texts.

! Font names are case-insensitive.

! You can use a comma-separated list of fonts to set a fallback font.
```

```
xscreensaver-auth.default.Dialog.bodyFont: times new roman 12, dejavu serif 12

! Window background color. You can use color names.

xscreensaver-auth.default.Dialog.background: black

! Main text color. You can also use HEX color codes.

xscreensaver-auth.default.Dialog.foreground: #ffffff
```

在 `/usr/share/X11/app-defaults/XScreenSaver` 中可以查看 X resources. 修改后请记得重新加载所有资源文件。

DPMS 和挂起设置

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因："overrides DPMS" 并不清晰 - *xscreensaver* 做了相当于 `xset s 0 0` 的工作，并使用自己的定时器来处理各种动画，但将 DPMS timeout 设置为 *Display Power Management* 中的值。（在 Talk:XScreenSaver讨论）

XScreenSaver 独立于 X 本身进行显示设备的电源管理（DPMS），会覆盖 X 本身的设置。要配置挂起、待机、关闭显示器等的时间，请使用 *xscreensaver-demo* 或手动编辑配置文件，例如 ~/.xscreensaver。

```
timeout:       1:00:00
cycle:         0:05:00
lock:          False
lockTimeout:   0:00:00
passwdTimeout:       0:00:30
fade:          True
unfade:              False
fadeSeconds:   0:00:03
fadeTicks:     20
dpmsEnabled:   True
dpmsStandby:   2:00:00
dpmsSuspend:   2:00:00
dpmsOff:       4:00:00
```

可通过启动 *xscreensaver-demo Mode* 设置，选择 *Disable Screen Saver* 禁用 DPMS 和屏幕挂起。

注意：如果 *xscreensaver-demo* 中的 *Lock Screen After* 被勾选并设为 0 分钟，则屏幕将在空白后立即锁定。如果不勾选 *Power Manager Enabled*，DPMS 将被禁用；这并不意味着 XScreenSaver 会放弃对 DPMS 设置的控制。

使用

要启动 XScreenSaver，请使用 `-no-splash` 选项。有关完整的选项列表，请参阅 1}。

在 Xfce， LXDE 和 LXQt 环境中, 如果 XScreenSaver 可用，XScreenSaver 就会自动启动 - 且无需进一步操作。对于其他环境，请参阅自动启动。

如果要立即触发 *xscreensaver*，如果它正在运行，并锁定了屏幕，请执行以下命令：

```
$ xscreensaver-command -lock
```

KDE Plasma 中的屏保由 *ksmserver* 控制，这会和 XScreenSaver 产生冲突，要禁用它，需要在 KDE 中启动服务并修改 `plasma-ksmserver.service` 为:

```
~/.config/systemd/user/plasma-ksmserver.service.d/override.conf
[Service]
ExecStart=
ExecStart=/usr/bin/ksmserver --no-lockscreen
```

重新登录， XScreenSaver 就会正常工作，更多信息请参考 xscreensaver(1) § INSTALLING XSCREENSAVER ON KDE。

锁定挂起

XScreenSaver 提供了一个名为 *xscreensaver-systemd* 的工具，可以处理来自 systemd 的 D-Bus 信号 `PrepareForSleep`, 并在挂起和休眠的时候自动锁定屏幕。这个程序会和 *xscreensaver* 一起启动，无需额外设置。详情请参考 xscreensaver-systemd(6)。

这个工具处理不了 `loginctl lock-session`。要处理这种情况，请参阅 Power management#xss-lock. 请使用 `--ignore-sleep` 选项，因为 *xscreensaver-systemd* 已经处理了挂起和休眠事件。

```
$ xss-lock --ignore-sleep -- xscreensaver-command -lock
```

请将 XScreenSaver 的淡出时间设置为 0。

从锁屏画面切换登录用户

警告：当使用 GDM 或 LightDM 这样的显示管理器切换用户时，XScreenSaver 不会锁定原始会话，只需将 TTY 切换到相关会话即可，无需密码即可访问。如果你使用的是 LightDM，请安装light-locker包，然后与 XScreenSaver 一起运行。另外，也可以使用其他的屏幕锁定程序——参见 List of applications/Security#Screen lockers。

By default, XScreenSaver's *New Login* button in the lock screen will call `/usr/bin/gdmflexiserver` to switch users. Display managers other than GDM that support user switching require a different command.

默认情况下，XScreenSaver 锁定屏幕中的 *New Login* 按钮会启动 `/usr/bin/gdmflexiserver` 进行用户切换。除了 GDM 外，其他支持用户切换的显示管理器需要使用不同的命令。

提示：将 *xscreensensaver.newLoginCommand:* 添加到 `~/.Xresources`，而将参数留空将使 *New Login* 按钮消失。

由于 `~/.xscreensensaver` 中的修改被 *xscreensaver-demo* 丢弃，故本节中使用了 ~/.Xresources。

LXDM

使用 LXDM 的切换用户功能：

```
xscreensaver-auth.default.*.newLoginCommand: lxdm -c USER_SWITCH
```

LightDM

使用 LightDM 的切换用户功能：

```
xscreensaver-auth.default.*.newLoginCommand: dm-tool switch-to-greeter
```

注意：如果你用这个切换到已经登录的用户，你可能需要输入两次密码（一次是为LightDM，一次是为你登录的用户的 XScreenSaver 对话框）。

SDDM

SDDM 不支持用户切换 。可以尝试用 using dbus-send 调用 `SwitchToGreeter` 方法，目前可能无法正常工作。

提示和技巧

在媒体播放时禁用

Starting from version 5.45, the *xscreensaver-systemd* utility implements the D-Bus ScreenSaver interface. It is started automatically with *xscreensaver*, so most applications should properly disable the screensaver without additional configuration. However, some applications do not support D-Bus or use another interfaces.

mpv

默认情况下，mpv 会在启动时关闭屏幕保护程序，并在退出时再次开启。当播放器暂停时，屏幕保护程序总是能被重新启用。该选项可以在 *mpv* 的配置文件中控制，位于 `~/.config/mpv/mpv/mpv.conf`：

```
stop-screensaver = "yes"
```

然而这并不是所有的视频输出或平台都会支持。如果你遇到了一些问题，你可以使用 Lua 脚本来手动关闭屏幕保护程序。在 `~/.config/mpv/scripts/xscreensaver.lua` 中创建一个文件，其中包含以下内容：

```
local utils = require 'mp.utils'
mp.add_periodic_timer(30, function()
    utils.subprocess({args={"xscreensaver-command", "-deactivate"}})
end)
```

上面的脚本会每隔30秒调用 `xscreensaver-command -deactivate`。

mplayer

在 `~/.mplayer/config` 中添加以下内容：

```
heartbeat-cmd="xscreensaver-command -deactivate >&- 2>&- &"
```

Kodi

Kodi 不支持禁用 XScreenSaver （它使用自己的屏幕保护程序）。为了实现禁用，请安装 kodi-prevent-xscreensaverAUR 包，或者尝试使用从 https://sourceforge.net/projects/osscreensavermanager/ 下载的 Kodi 扩展。

浏览 HTML5 video/Flash

Firefox supports the D-Bus ScreenSaver interface and should disable the screensaver during HTML5 video playback.

**本文或本章节的事实准确性存在争议。**

原因： The chromium exception was added without reference and disputed. Is it reproducible?（在 Talk:XScreenSaver#loginctl, chromium, resources 中讨论）

Chromium also supports it, but uses the GNOME Session interface when available, so XScreenSaver will not be disabled in some desktop environments such as GNOME and MATE.

If you are experiencing problems, you can try a script named lightsonplus with support for Firefox's Flash plugin, Chromium's Flash plugin, HTML5 fullscreen video in Firefox and Chromium, MPlayer, and VLC.

动态壁纸

可以在后台运行 `xscreensaver`，就像壁纸一样。首先，中断一切控制背景的进程（根窗口）。

然后，找到所需的 XScreenSaver 可执行文件（通常在 `/usr/lib/xscreensaver/` 中），并使用 `-root` 选项运行它，例如：

```
$ /usr/lib/xscreensaver/glslideshow -root &
```

注意：如果 xcompmgr 或 picom 导致问题，请安装 shantz-xwinwrap-bzrAUR 包，然后运行：

```
$ xwinwrap -b -fs -sp -fs -nf -ov -- /usr/lib/xscreensaver/glslideshow -root -window-id WID &
```

禁用欢迎屏幕

When the program is started a screen appears which shows the program version and the author name. The screen is displayed every time the program starts which may be annoying. To disable this behavior run the program with the option --no-splash:

```
$ xscreensaver --no-splash &
```

问题解决

要显示详细的调试信息，启动 *xscreensaver* 时带上 `--verbose` 命令行选项，修改 `~/.xscreensaver` 配置文件，添加 `verbose: True` 选项，则每次启动都打印详细的调试信息。

用 `--log` 选项可以将日志记录到文件，此选项隐含了 verbose 配置。
(`~/.xscreensaver` 或 X resources 中无对应配置)。

参见

- XScreenSaver 的主页

3.10 Picom

难度 进阶 | **篇幅** 14.2 KB | **所属** X 服务器

本节将讲到

- 安装
- 配置
- 使用
- 多显示器
- 灰度

picom 是一个独立的 Xorg 合成器，适合与没有合成功能的窗口管理器一起使用。
picom 是 compton 的一个分支，而 compton 是 xcompmgr 分支 xcompmgr-dana 的一个分支。

安装

安装 picom包 软件包或开发者版本 picom-gitAUR。

配置

默认的配置位于 `/etc/xdg/picom.conf` 中，若要修改，请先将其复制到 `~/.config/picom/picom.conf` 或 `~/.config/picom.conf`。

若要让 picom 使用其他的自定义配置文件，请使用如下的命令：

```
$ picom --config path/to/picom.conf
```

详情请参阅 picom(1) § CONFIGURATION FILES。

为某些窗口禁用阴影效果

可按需使用 `shadow-exclude` 选项以禁用窗口阴影。有关已禁用的窗口，参见这个链接。

若要禁用菜单的阴影效果，请将以下内容添加至 `picom.conf` 中的 `wintypes`：

```
# menu          = { shadow = false; };
dropdown_menu = { shadow = false; };
popup_menu    = { shadow = false; };
utility       = { shadow = false; };
```

EWMH 标准中定义了其他可用的 `WINDOW_TYPE` 值：`unknown`、`desktop`、`dock`、`toolbar`、`menu`、`utility`、`splash`、`dialog`、`normal`、`dropdown_menu`、`popup_menu`、`tooltip`、`notification`、`combo` 以及 `dnd`。

不透明度

若要为聚焦和非聚焦的窗口（如终端模拟器）设置不透明度（实际上是透明度），将以下内容添加至 `picom.conf`：

```
opacity-rule = [
  "90:class_g = 'URxvt' && focused",
  "60:class_g = 'URxvt' && !focused"
];
```

另见 #选项卡式窗口（阴影和透明度）.

使用

可在会话期间的任意时间点手动启用或禁用 picom，或作为会话的一个后台进程自动启动。有一些可选参数用于调整 picom 所提供的合成效果。这些效果包括：

- `-b`：作为会话的一个后台进程运行（例如，随窗口管理器 Openbox 启动）
- `-c`：启用阴影效果
- `--config`：使用指定的配置文件

注意：`-C`（禁用面板和停靠栏的阴影效果）和 `-G`（禁用应用程序窗口和拖放对象的阴影效果）参数已弃用，类似功能请参考 #为某些窗口禁用阴影效果。

更多可用选项，包括设置需要管理的时机和显示效果，菜单、窗口边框和不活跃应用程序的菜单的不透明度，参见 picom(1)。

注意：其他合成管理器应当在 *picom* 运行之前启动。

要在会话期间手动启用默认的合成效果，请使用以下命令：

```
$ picom &
```

要作为会话的后台进程自动启动 picom，可以使用 `-b` 参数（有可能导致显示画面冻结）：

```
$ picom -b
```

以下是使用需要传值的额外参数的例子：

```
$ picom -cfF -o 0.38 -O 200 -I 200 -t 0 -l 0 -r 3 -D2 -m 0.88
```

多显示器

若在没有 xinerama（意味着以多个屏幕启动 X 服务）的情况下使用了多显示器的配置，picom 默认只会在一个屏幕上运行。可以使用 `DISPLAY` 环境变量令其在所有屏幕上运行。例如，在 X 上标识符为 0 的屏幕后台运行 picom：

```
DISPLAY=":0" picom -b
```

上述命令在所有显示器上都应该生效。若无效，请尝试旧方法，指定每一个显示器：

```
seq 0 3 | xargs -l1 -I@ picom -b -d :0.@
```

灰度

可以使用着色器将窗口颜色转换为灰度。

根据 picom(1)，从 picom 的源码中编辑默认的着色器。

```
/path/to/shader/file.glsl
#version 330

in vec2 texcoord;
uniform sampler2D tex;
uniform float opacity;

vec4 default_post_processing(vec4 c);

vec4 window_shader() {
	vec2 texsize = textureSize(tex, 0);
	vec4 color = texture2D(tex, texcoord / texsize, 0);

	color = vec4(vec3(0.2126 * color.r + 0.7152 * color.g + 0.0722 * color.b) * opacity, color.a * opacity);

	return default_post_processing(color);
}
```

将编辑好的文件路径引入着色器后启动 picom。或许还需要 `glx` 后端。

```
$ picom --backend glx --window-shader-fg /path/to/shader/file.glsl
```

故障排除

近期的 picom 版本在 DRI2 加速上存在一些问题，表现为使用 DRI2 时出现严重的画面闪烁（picom bug、mesa bug）。此问题已经过反馈和修复，但仍可能会影响某些用户。DRI3 则不受该问题的影响。

使用合成效果可能会导致一些问题，例如在与其他应用程序一起使用时，错误的配置会引起显示故障。

Conky

若要禁用 Conky 窗口的环绕阴影，请将以下内容添加至 `~/.conkyrc`：

```
own_window_class conky
```

若使用模糊特效时，上述方法无效，请在 `~/.conkyrc` 中尝试如下配置：

```
own_window_type= 'desktop'
```

dwm 和 dmenu

picom 所有能自动排除窗口管理器元素的功能都无法检测到 dwm 的状态栏。无论是 dwm 的状态栏还是 dmenu 都没有固定的窗口 id。若要将其从不活跃窗口透明化中排除，要么在源码中加入一个窗口类，要么使用不那么精确的属性将其排除。下面的示例将 dwm 的状态栏置于顶部，这样就可以实现与位置排除无关的分辨率：

```
$ picom <任意的其他参数> --focus-exclude "x = 0 && y = 0 && override_redirect = true"
```

或者在使用的配置文件中配置如下内容：

```
focus-exclude = "x = 0 && y = 0 && override_redirect = true";
```

对于大多数窗口来说，覆盖重定向属性的值似乎都是 false，在排除规则中使用此属性可以防止在左上角绘制的其他窗口被排除在外（例如，当 dwm 状态栏被隐藏时，x0 y0 将与 dwm 主堆栈（master stack）中的任何内容相匹配）。

Firefox

参见 #为某些窗口禁用阴影效果。

要禁用火狐浏览器元素的阴影效果，将以下内容添加至 `picom.conf` 中的 shadow-exclude：

```
"class_g = 'firefox' && argb",
```

详情请参考这个链接。

slock

使用 slock 时，开启不活跃窗口透明化效果（以 `-i` 参数启动）可能会导致麻烦的问题。一种解决方法是将透明度修改为 `0.2`。例如，以 picom 参数运行命令：

```
$ picom <任意的其他参数> -i 0.2
```

或者在使用的配置文件中配置如下内容：

```
inactive-dim = 0.2;
```

另一种方式是，通过 slock 的窗口 id 将其排除在外，或排除所有无名称的窗口。

注意：某些程序在每次运行新实例时都会改变 id，但 slock 的 id 是固定的。有相关知识的用户会去确认 slock 的 id 是否真的是固定不变的，至此，请谨慎使用排除方法。

使用如下的选项排除所有无名称的窗口：

```
$ picom <其他参数> --focus-exclude "! name~=''"
```

运行以下命令以找出 slock 的窗口 id：

```
$ xwininfo & slock
```

在 slock 退出前快速点击屏幕上的任意位置，然后输入密码解锁。应该可以看到输出中的窗口 id：

```
xwininfo: Window id: 0x1800001 (has no name)
```

提取该 id 并在 picom 中将其排除：

```
$ picom <任意的其他参数> --focus-exclude 'id = 0x1800001'
```

或者在使用的配置文件中配置如下内容：

```
focus-exclude = "id = 0x1800001";
```

画面闪烁（Flicker）

以默认的 `picom.conf` 应用于最大化窗口（无面板的会话中）会引发此问题，可以通过以下选项解决：

```
unredir-if-possible = false;
```

详情请参见这个链接。

全屏撕裂

若仅在全屏播放视频时观察到屏幕撕裂，请参考 #画面闪烁（Flicker）。

使用 xft 字体时显示延迟

若在使用了 Xft 字体的应用程序（例如 xterm（英语：xterm） 和 urxvt）中遇到严重的显示延迟，请尝试以下选项：

```
--xrender-sync --xrender-sync-fence
```

或使用 xrender 后端。

详情请参见这个链接。

选项卡式窗口（阴影和透明度）

以选项卡式显示的透明化窗口，其后方的选项卡式窗口由于透明度的原因依然可见。每个选项卡式窗口都会绘制自己的阴影，这会导致多重阴影。

可将以下内容添加至已有的阴影效果排除列表中以解决多重阴影的问题：

```
"_NET_WM_STATE@:32a *= '_NET_WM_STATE_HIDDEN'"
```

将下面的内容添加至 `picom.conf` 中可禁用绘制后方选项卡式窗口：

```
opacity-rule = [
  "95:class_g = 'URxvt' && !_NET_WM_STATE@:32a",
  "0:_NET_WM_STATE@:32a *= '_NET_WM_STATE_HIDDEN'",
  "0:_NET_WM_STATE@:32a *= '_NET_WM_STATE_HIDDEN'",
  "0:_NET_WM_STATE@:32a *= '_NET_WM_STATE_HIDDEN'",
  "0:_NET_WM_STATE@:32a *= '_NET_WM_STATE_HIDDEN'",
  "0:_NET_WM_STATE@:32a *= '_NET_WM_STATE_HIDDEN'"
];
```

注意，`URxvt` 是终端的 Xorg 类名。若使用不同的终端，请更改该类名。运行命令 `xprop WM_CLASS` 并点击一个窗口以查询该窗口的类名称。

详情请参见这个链接。

警告：使用 i3 并以 kitty 作为终端时，上述操作（截至 2020-08-31）会在重新加载 i3 时冻结所有选项卡式的 kitty 实例（参见这个链接），可能的解决方案可在一个类似的问题中找到。

Xsetroot 无法更改背景颜色

目前，picom 不兼容 `xsetroot` 的 `-solid` 选项，一种解决方法是使用 hsetroot包更改背景颜色：

```
$ hsetroot -solid '#000000'
```

详情请参见这个链接。

使用 NVIDIA 专有驱动时遇屏幕撕裂

请在 `picom.conf` 中尝试以下设置：

```
vsync = true;
```

使用 NVIDIA 专有驱动和 FullCompositionPipeline 时有显示延迟

请尝试以如下参数运行 picom：

```
--backend xrender
```

或在 `picom.conf` 中添加如下的配置项：

```
backend = "xrender";
```

详情请参见这个链接。

另一个降低延迟的方法是在 nvidia 的 OpenGL 设置中禁用“允许翻转”（disable "allow flipping"，参见这个链接）。也可以用命令行实现同样的效果：

```
$ nvidia-settings -a 'AllowFlipping=0'
```

要在重启后加载设置（参见自动启动），请运行：

```
$ nvidia-settings --load-config-only
```

使用 NVIDIA 专有驱动时 Xorg 会泄漏 GPU 的内存

参见 #使用 NVIDIA 专有驱动和 FullCompositionPipeline 时有显示延迟。

设备挂起后 slock 相关问题

若使用 systemd 服务在挂起或休眠时触发 slock，设备屏幕可能在恢复后的几秒中内没有锁定。禁用窗口淡入淡出效果以防止此问题发生：

```
$ picom --no-fading-openclose
```

屏幕共享

将以下内容添加至 `shadow-exclude` 可防止出现屏幕共享上的阴影覆盖和 Zoom Meetings（英语：Zoom Meetings） 弹出窗口的阴影效果：

```
shadow-exclude = [
  "name = 'cpt_frame_xcb_window'",
  "class_g ?= 'zoom'",
];
```

按照如下内容将 Zoom Meetings（英语：Zoom Meetings） 添加至 `blur-background-exclude` 中可禁用屏幕共享时的模糊效果。

```
blur-background-exclude = [
  "class_g ?= 'zoom'",
];
```

对于 Microsoft Teams，分享内容的红色外边框是以一个几乎完全透明的窗口实现的。启用模糊效果将导致该功能无法使用，应按照如下方式禁用模糊效果：

```
shadow-exclude = [
  "name = 'rect-overlay'",
];

blur-background-exclude = [
  "name = 'rect-overlay'",
];
```

切换工作区时禁用窗口的淡入淡出效果

添加 `--fade-in-step=1 --fade-out-step=1 --fade-delta=0` 标识可以在切换至新的工作区时禁用窗口的淡入淡出效果。参见这个链接。

另见

- 在 XFCE 或 LXDE 上使用 Compton 作为无撕裂的窗口合成器

3.11　多显示器

难度 进阶 | **篇幅** 15.4 KB | **所属** X 服务器

本节将讲到

- 历史背景
- 独立屏幕
- RandR
- TwinView
- Xinerama

多头（Multi-head）、**多屏（multi-screen）**、**多显（multi-display）**或**多监视器（multi-monitor）**是指同一台计算机上连接了多个显示设备的配置场景。本文提供多种多头的配置方式，并给出若干设置实例。

注意：为避免混淆，下列名词在本文中有各自专属的含义：

- 监视器（Monitor）:物理显示设备，如液晶平板显示器。
- 屏幕（Screen）:窗口屏幕，是已经连接到一个**显示区（display）**的 **监视器（monitor）**。
- 显示区（Display）: 一组**屏幕（screens）**，各自同时展示同一桌面的某一部分。用户可以在同一个**显示区（display）**的所有**屏幕（screens）**之间拖动窗口。

历史背景

大多数（甚至是全部）提供图形用户界面的Unix/Linux电脑都使用X窗口系统作为底层图形界面。X窗口系统1984年在麻省理工学院被开发出来。历经35年的开发、调校与新功能、新理念的加入，它的强大得到了公认。应该记得，在开发它的那个时代，常见的配置是一个运行着的X为一个分时系统中的X终端（Xterminal）们提供各自独立的视图。如今的标准情形是X为一台台式机或笔记本提供单一一块屏幕。

注意：现在仍然有一种罕见的配置，经常被称作Zaphod display。它允许单台电脑的多个用户每人拥有独立的一组显示、鼠标和键盘，就好像每个人使用的电脑是分开的一样，但每个人的平均成本更低。

以上这些都意味着，要实现同一件事或者满足同一目标，互相之间略有差异的许多件事，有着很多方法。在现代的X版本中，有时候只需要很少量的配置或者不需要配置。最近几年的趋势是X是“自配置”（self configuring）的。当然最好的经验法则是配置越少越好，也就是“只配置出问题的地方”。

独立屏幕

这是为X配置多监视器的原始方法，已经存在了几十年。每个物理监视器都被指派为一块X屏幕，尽管你可以在它们之间移动鼠标，但它们或多或少都是独立的。

一般来说X显示区有一个类似于`:0`的标识符，在`DISPLAY`环境变量中设置。但在这个配置中，每块屏幕都有不同的`$DISPLAY`值。第一块屏幕是`:0.0`，第二块是`:0.1`，依此类推。

这种配置无法实现在屏幕之间移动窗口，除了一些具有多屏幕支持的特殊程序，如GIMP和Emacs。对于大多数程序，需要在启动它们时更改`DISPLAY`环境变量，使程序出现在另一块屏幕上：

```
# Launch a terminal on the second screen
$ DISPLAY=:0.1 urxvt &
```

或者如果在每块屏幕上都有一个终端，启动的程序会继承`DISPLAY`值从而出现在启动它的那块屏幕。但是在屏幕之间移动程序需要关闭它，然后在另一块屏幕上重新打开。

这种工作方式确实有其优点，比如如果你在一块屏幕上工作，另一块屏幕上弹出的窗口不会抢走焦点——每块屏幕都是相当独立的。

RandR

RandR（**R**otate **and** **R**esize，旋转与缩放）是一个X窗口系统扩展，允许客户端动态调整（即缩放、旋转、翻转）屏幕。多数情况下，它可以完全替代旧的Xinerama配置。这篇文章说明了为什么RandR优于Xinerama。

RandR可以通过xrandr工具为当前会话配置，或者通过xorg.conf文件进行持久设置。

注意：同一配置有多种方法实现，可能需要进行一些实验才能找到最佳配置。

用 arandr 配置

arandr 能够图形化地排列监视器、改变分辨率并保存脚本以存储设置。默认状态下点击“另存为”（Save As）会把脚本保存到 `~/.screenlayout/`，之后你就可以把这些文件加入到你的 `~/.profile`。有时候登录之后过早运行 arandr 脚本会引起问题。

编辑 `~/.profile`

```
$ sleep 3
$ /home/<user>/.screenlayout/myLayout.sh
```

用xrandr配置

注意：本节内容假定读者已经阅读过xrandr页面，了解了关于*xrandr*的基本信息。

你既可以对各块屏幕进行相对排列（使用`--right-of`, `--left-of`, `--above`, `--below`选项），也可以使用绝对坐标（使用`--pos`选项；注意这种情况下你通常需要知道监视器们的分辨率）。详情请参阅xrandr(1)。以下对一些常用设置加以说明。

VGA1位于HDMI1左侧，均使用最佳分辨率

```
$ xrandr --output VGA1 --auto --output HDMI1 --auto --right-of VGA1
```

`--right-of`把它前面那块屏幕（`HDMI1`）置于指定屏幕（`VGA1`）的右侧。

VGA1位于HDMI1右侧，均使用固定分辨率

```
$ xrandr --output VGA1 --mode 1024x768 --pos 1920x0 --output HDMI1 --mode 1920x1080 --pos 0x0
```

或者

```
$ xrandr --output VGA1 --mode 1024x768 --output HDMI1 --mode 1920x1080 --left-of VGA1
```

`--left-of`把它前面那块屏幕（`HDMI1`）置于指定屏幕（`VGA1`）的右侧。

将多块屏幕组合为一块虚拟显示区

从randr 1.5版本开始，将多个监视器组合为一块虚拟显示区成为可能。这是Xinerama相应功能的升级版，能够用于开源驱动且不需要重启Xorg。有些桌面环境尚不支持此功能。Openbox经过测试可以支持。

执行`xrandr --listmonitors`获取监视器列表

```
0: +*DisplayPort-4 1920/518x1200/324+1920+0  DisplayPort-4
1: +DisplayPort-3 1920/518x1200/324+0+0  DisplayPort-3
2: +HDMI-A-0 1920/518x1200/324+3840+0  HDMI-A-0
```

创建虚拟显示区`xrandr --setmonitor SomeName auto DisplayPort-4,DisplayPort-3,HDMI-A-0`。`auto`指定了虚拟显示区的尺寸，

设定为auto会自动建立显示阵列的正确尺寸。该命令中监视器的顺序不起作用，在执行该命令之前或之后需要重新正确排列监视器。

更详细的解释见此页。

用xorg.conf配置

此方法与使用*xrandr*类似，每块屏幕都需要各自独立的`Monitor`节。标识符（`Identifier`）使用的值与`xrandr -q`所报告的相同（例如，使用`Identifier "VGA1"`而不是`--output VGA1`）。

例：使用相对坐标的双屏配置

```
/etc/X11/xorg.conf.d/10-monitor.conf
Section "Monitor"
    Identifier  "VGA1"
    Option      "Primary" "true"
EndSection

Section "Monitor"
    Identifier  "HDMI1"
    Option      "LeftOf" "VGA1"
EndSection
```

例：使用相对坐标以及自定义分辨率的双屏配置

可以通过运行`$ xrandr -q`命令查询到各个监视器的ID。应当在`Device`中按照`Monitor-<ID>`的格式指明监视器的ID。

见Xrandr#添加未检测到的分辨率。

```
/etc/X11/xorg.conf.d/10-monitor.conf
Section "Monitor"
  Identifier "DVI"
  Modeline "1680x1050_60.00"  146.25  1680 1784 1960 2240  1050 1053 1059 1089 -hsync
+vsync
  Option "PreferredMode" "1680x1050_60.00"
  Option "LeftOf" "DP"
  Option "DPMS" "true"
EndSection

Section "Monitor"
  Identifier "DP"
  Modeline "1920x1080_60.00"  173.00  1920 2048 2248 2576  1080 1083 1088 1120 -hsync
+vsync
  Option "PreferredMode" "1920x1080_60.00"
  Option "RightOf" "DVI"
  Option "DPMS" "true"
EndSection

Section "Screen"
  Identifier "Screen0"
  Device "Radeon" # e.g. Radeon, Intel, nvidia
  Monitor "DP"
  DefaultDepth 24
  SubSection "Display"
    Depth 24
    Virtual 3600 2130 # 1920 + 1680 (3600), 1080 + 1050 (2130)
  EndSubSection
EndSection
/etc/X11/xorg.conf.d/20-radeon.conf
Section "Device"
```

```
  Identifier "Radeon"
  Driver "radeon"
  Option "Monitor-DVI-0" "DVI" # use DVI-0 as DVI
  Option "Monitor-DisplayPort-0" "DP"
EndSection
```

例：使用绝对坐标的双屏配置

```
/etc/X11/xorg.conf.d/10-monitor.conf
Section "Monitor"
    Identifier  "VGA1"
    Option      "PreferredMode" "1024x768"
    Option      "Position" "1920 312"
EndSection

Section "Monitor"
    Identifier  "HDMI1"
    Option      "PreferredMode" "1920x1080"
    Option      "Position" "0 0"
EndSection
```

此例中不使用相对坐标，整个设置中能够被定位到的最左上角在0,0点。

```
(0,0)-----------------+
|                     |(1920,312)---+
|     1920 x 1080     ||            |
|        HDMI1        || 1024 x 768 |
|                     ||    VGA1    |<br>+---------------------++------------+<br>```

## TwinView

TwinView是nVidia的扩展，可以让系统把一块显卡上连接的两台监视器看做单块屏幕。TwinView提供Xinerama扩展，这样应用程序会知道连接了两台监视器，因此它与Xinerama不兼容。不过如果你有两台连接到同一块nVidia显卡的监视器，TwinView和Xinerama几乎没有差别（尽管这种情况下TwinView可能会提供稍好一些的性能。）

对于两台以上监视器或者监视器接在不同显卡上的情况，需要使用Xinerama而不是TwinView。同样，截至2012年4月，两台监视器的方向必须相同，不能一台水平显示，另一台垂直显示。

以前，要想在能够在屏幕之间拖动窗口的同时获得nVidia显卡的OpenGL加速，TwinView是唯一的办法。不过现在新版本的nVidia闭源驱动即使在使用Xinerama的情况下也能够提供OpenGL加速。

配置示例见[NVIDIA#TwinView](../图形/NVIDIA.md#TwinView)。

## Xinerama

[Xinerama](https://en.wikipedia.org/wiki/Xinerama)是实现真正多屏X的旧方式。Xinerama将所有监视器组合为单块屏幕（`:0`），从而可以做到在屏幕之间拖动窗口。

Xinerama通过自定义的[X配置文件](Xorg.md#配置)进行配置。也有一款叫[WideGuy](https://openapplibrary.org/project/wideguy)的图形化工具可以使得切换Xinerama更容易。注意，要使用WideGuy，你仍然需要包含一个ServerLayout节的Xorg配置。

以下是一些例子：

这是ServerLayout节，用于控制各监视器相互之间的位置关系：

/etc/X11/xorg.conf.d/90-serverlayout.conf

Section "ServerLayout" Identifier "Main" Screen 0 "Primary" Screen 1 "DellPortraitLeft" RightOf "Primary" Screen 2 "Wacom" RightOf "DellPortraitLeft" Screen 3 "U2412" LeftOf "Primary" Option "Xinerama" "1" # enable XINERAMA extension. Default is disabled. EndSection
```

上面这一节中的每块屏幕都在一个单独的文件里定义，比如下面这个：
/etc/X11/xorg.conf.d/30-screen-dell2001.conf

Section "Monitor" Identifier "Dell 2001FP" VertRefresh 60 Option "dpms" "on"

Modeline "1600x1200" 160.96 1600 1704 1880 2160 1200 1201 1204 1242 -HSync +Vsync EndSection

Section "Screen" Identifier "DellPortraitLeft" Device "GeForce 8600GTb" Monitor "Dell 2001FP" DefaultDepth 24 SubSection "Display" Depth 24 Modes "1600x1200" ViewPort 0 0 Virtual 1600 1200 EndSubsection

Option "Rotate" "left" EndSection

对于每个**监视器**都需要创建一个`Device`节，如一块接了两台监视器的显卡应当有两个Device节。下面的例子展示了两块显卡，每块两路输出，总共四台监视器的情况该如何配置。
/etc/X11/xorg.conf.d/20-nvidia.conf

Section "Device" Identifier "GeForce 8600GT" Driver "nvidia"

BusID "PCI:1:0:0"

Screen 0

Option "UseDisplayDevice" "DFP-0"

Option "DynamicTwinView" "FALSE"

Option "AddARGBGLXVisuals" "true" Option "UseEDIDDpi" "false" Option "DPI" "96 x 96" Option "Coolbits" "1" EndSection

Section "Device" Identifier "GeForce 8600GTb" Driver "nvidia" BusID "PCI:1:0:0"

Screen 1

Option "AddARGBGLXVisuals" "true" Option "UseEDIDDpi" "false" Option "DPI" "96 x 96" Option "Coolbits" "1" Option "DynamicTwinView" "FALSE" EndSection

Section "Device" Identifier "G210" Driver "nvidia" BusID "PCI:2:0:0" Screen 0

Option "AddARGBGLXVisuals" "true" Option "UseEDIDDpi" "false" Option "DPI" "96 x 96" Option "Coolbits" "1" Option "DynamicTwinView" "FALSE" EndSection

Section "Device" Identifier "G210b" Driver "nvidia" BusID "PCI:2:0:0" Screen 1 Option "UseDisplayDevice" "DFP-1"

Option "AddARGBGLXVisuals" "true" Option "UseEDIDDpi" "false" Option "DPI" "96 x 96" Option "Coolbits" "1" Option "DynamicTwinView" "FALSE" EndSection

参见

- [论坛帖子：如何使用Nouveau驱动实现双监视器](https://bbs.archlinux.org/viewtopic.php?pid=652861)
- [六显示器，六用户Linux系统](https://linuxgazette.net/124/smith.html)

3.12 FreeNX

难度 进阶 | **篇幅** 15.8 KB | **所属** X 服务器

本节将讲到

- 安装
- 设置
- 运行
- 在已有屏幕上使用 FreeNX
- 设置非 KDE 或非 Gnome 桌面管理器

!](https://wiki.archlinuxcn.org/wzh/File:Tango-preferences-desktop-locale.png)**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： 大量英文段落（在 Talk:FreeNX# 中讨论）

NX 是一项令人激动的远程控制技术。它提供了在高延迟、低带宽环境中**接近本地速度**的应用程序响应链接。NX 的核心库由 NoMachine 在 GPL 授权下提供。**FreeNX** 是一个 NX 服务器和客户端组件的 GPL 实现。

: — FreeNX - 自由的 NX

安装

包分为两部分：

- freenx （服务器）
- nxclient （客户端）

如果你打算用 freenx 来连接无头 PC，记得你得有一个配置好的 X 服务器，这样你应该就能安装任何与 xorg 相关联的包了。

设置

服务器

自由（免费）的服务器在 'freenx' 包中。你必须安装 sshd 守护进程，并且让它正确运行：

```
# systemctl enable sshd
```

要使 freenx 的登录验证工作，sshd 必须正确配置。检查 /etc/ssh/sshd_config 中下列项：

```
RSAAuthentication yes
AllowUsers someuser nx
```

NX 的主配置文件位于:

```
/opt/NX/etc/node.conf
```

如果你的 ssh 守护进程不是运行在默认的 22 端口，你需要取消注释并修改：

```
SSHD_PORT=22
```

注意：As of OpenSSL 1.0.0, it is necessary to set proper md5sum command in `/opt/NX/etc/node.conf`:

```
COMMAND_MD5SUM="md5sum"
```

如果你在用 Gnome 或 KDE 桌面环境，你不需要编辑此文件，应为修改过的 MD5SUM 与默认配置在这种场合下能够工作。如果你在使用其它窗口管理器，比如 Fluxbox/Openbox 或 Xfce，你大概需要稍微修改这个文件（见下方）。

安装完 freenx 包，运行 `/opt/NX/bin/nxsetup --help` for an overview of the install and uninstall procedures.

注意：You should also install "xdialog" on the server or you will not see the "suspend/terminate" dialog when you try to close the window or hit "ctrl-alt-T":

```
pacman -S xdialog
```

密钥

服务器需要用密钥来验证客户端。默认情况下，在安装时会产生一对随机密钥，一个是服务器的，另一个是客户端的。你需要给每一个想要连接的客户端（Windows 也好， linux 也好）复制一份客户端密钥。

客户端密钥能在这儿找到：

```
/opt/NX/home/nx/.ssh/client.id_dsa.key
```

Alternatively you can use the default key that is provided by NoMachine with all clients. In this case you do not need to copy a custom generated key to each client. To get the server to accept the default client keys run:

```
/opt/NX/bin/nxsetup --install --setup-nomachine-key
```

Recreation of random keys:

```
/opt/NX/bin/nxsetup --install
```

Transferring nx keys to another freenx server:

- /opt/NX/home/nx/.ssh contains the key files

```
-rw------- 1 nx root  697  9. Okt 12:55 authorized_keys
-rw------- 1 nx root  668  9. Okt 11:48 client.id_dsa.key
-rw------- 1 nx root  609  9. Okt 12:55 server.id_dsa.pub.key
```

- Save those files.
- Add those files to your new server, they need the same permissions, names, group and directory!

```
# cp authorized_keys client.id_dsa.key server.id_dsa.pub.key /opt/NX/home/nx/.ssh/
# chmod 600 /opt/NX/home/nx/.ssh/*
# chown nx /opt/NX/home/nx/.ssh/*
# chgrp root /opt/NX/home/nx/.ssh/*
```

- Recreate known_hosts file:

```
# echo -n 127.0.0.1 > /opt/NX/home/nx/.ssh/known_hosts
# cat /etc/ssh/ssh_host_rsa_key.pub >> /opt/NX/home/nx/.ssh/known_hosts
```

```
# chmod 633 /opt/NX/home/nx/.ssh/known_hosts
# chown nx /opt/NX/home/nx/.ssh/known_hosts
# chgrp root /opt/NX/home/nx/.ssh/known_hosts
```

启动服务器

Once installed the server is effectively running and ready to go, you do not have to do anything manually The only thing that must be running in order to connect is the sshd daemon. This is because the nxserver is actually started by logging into sshd as the special user 'nx'. This user has been set up to use the nxserver as its shell, much like a normal user has bash as the default shell.

You can specify the local addresses sshd should listen for by editing /etc/ssh/sshd_config and adding them in the following format:

```
ListenAddress host|IPv4_addr|IPv6_addr
```

The original ListenAddress in sshd_config is 0.0.0.0. This listens to all addresses, however adding any address will take precedent to this and and accept connections from the new address only.

Restart the sshd daemon to put any changes to its configuration into effect:

```
# systemctl restart sshd
```

In actual fact, if you check the process list (`ps aux`) you may not see the nxserver running even though it is. This is because the nxserver is actually started by logging into sshd as the special user 'nx'. This user has been set up to use the nxserver as its shell, much like a normal user has bash as the default shell.

客户端

Arch Linux

Get the client from pacman:

```
pacman -S nxclient
```

Windows

Get the client from nomachine's homepage: https://www.nomachine.com

Tip: Nomachine tends to remove old clients from their homepage, If your setup works with a client save it at a safe place ;)

配置

As mentioned above, the client must contain the correct key to connect to the server. If you are using the custom keys generated during install, you need to copy the client key to the following locations:

- Windows: `<yourinstalldironwindows>/share/keys/client.id_dsa.key`
- Arch Linux: `/opt/NX/share/keys/client.id_dsa.key`

After moving the keys you may have use the nxclient GUI to import the new keys. From the configuration dialog press the 'Key...' button and import the new client key.

运行

After installing nxclient on Arch Linux, executables are available in /opt/NX/bin. At the first run of /opt/NX/bin/nxclient, the user will be led through a wizard.

键盘快捷键

```
CTR+ALT+F           Toggles full-screen mode.
CTRL+ALT+T          Shows the terminate, suspend dialog.
CTRL+ALT+M          Maximizes of minimizes the window
CTRL+ALT+Mouse      Drags the viewport, so you can view different portions
                    of the desktop.
CTRL+ALT+Arrows
or                  Moves the viewport by an incremental amount of pixels.
CTRL+ALT+Keypad
CTRL+ALT+S          It will activate "screen-scraping" mode, so all the GetImage
                    originated by the clients will be forwarded to the real
                    display. This should make happy those who love taking
                    screenshots ;-). By pressing the sequence again, nxagent
                    will revert to the usual "fast" mode.
CTRL+ALT+E          lazy image encoding
CTRL+ALT+Shift+ESC Emergency-exit and kill-window
```

退出全屏

There is a magic-pixel in the top right corner of nearly every nx-application in fullscreenmode. Right-click the pixel and application-window gets iconified.

Tips on resume

- Resume is a bit experimental, crashes might appear after session has resumed. You have to find out which apps like resuming and which do not ;) .
- Resuming between Linux and Windows sessions does not work. UPDATE: It appears that version 3.2.0-14 is able to resume Windows-suspended sessions.
- If resume fails let it time out and do not use the cancel button, else sessions will stay open and consume RAM on server. To kill such sessions use the Session Admin program to kill them.

修复 DPI 设置

If you like to have the same font-sizes/dpi sizes on all your client session, set the X resource "Xft.dpi". For example putting the following line into a user's "~/.Xresources" makes her/his "desktop" a 100dpi. Xft.dpi: 100

在已有屏幕上使用 FreeNX

Usually, when connecting to a NX server, a new X session is created. Sometimes it might be useful, to connect to an existing X session, e.g. the root session. This is not possible with NX in default setup, but can be reached, using `tightvnc` and `x11vnc`. Do the following steps on the NX server system.

```
# pacman -S tightvnc x11vnc
```

x11vnc will serve the X session, we have to create a file `$HOME/.x11vncrc` to give x11vnc some options, e.g.:

```
display :0
```

```
shared
forever
localhost
rfbauth /home/USER/.x11vnc/passwd
```

Create the VNC password file:

```
$ mkdir $HOME/.x11vnc
$ x11vnc -storepasswd PASSWORD $HOME/.x11vnc/passwd
$ chmod 600 $HOME/.x11vnc/passwd
```

Create a shell script, which starts the x11vnc service, if not running and starts the vncviewer provided by the package tightvnc.

注意：The variable `$VNC_PORT` in the following script defines the X display, which is configured as `display :0` under `$HOME/.x11vncrc`, `590**0**` is the root session, if you want to use `display :1` use the port `590**1**` and so on

```
#!/bin/sh
VNC_VIEWER=vncviewer
VNC_SERVER=x11vnc
VNC_RESOLUTION=1024x786
VNC_PASSWD=/home/USER/.x11vnc/passwd
VNC_PORT=5900

if [ -z "$(pgrep ${VNC_SERVER})" ]; then
       echo $VNC_SERVER not running, starting...
       exec $VNC_SERVER &
       sleep 5
fi

exec $VNC_VIEWER -geometry $VNC_RESOLUTION -passwd $VNC_PASSWD localhost::$VNC_PORT
```

Save this script with a texteditor of your choice, e.g. under `$HOME/shell/nxvnc.sh`. Make it executable and create a symbolic link, e.g:

```
$ chmod +x $HOME/shell/nxvnc.sh
# ln -s /home/USER/shell/nxvnc.sh /usr/local/bin/nxvnc
```

At this point, you might want to test the current configuration:

```
$ /usr/local/bin/nxvnc
```

If the x11vnc service and a vncviewer session is started, you configuration works well. You are now able to connect to the current X session using your NX client with following options:

```
Login, Password, Host, Port: your default entries
Desktop: Unix -> Custom
 - Settings:
   - Run the following command: /usr/local/bin/nxvnc
   - New virtual desktop
Display:
  - Fullscreen or Custom with you preferred resolution
```

You are able to connect to your current X session via NX client now.

: — FreeNX to existing display (opensuse.org)

设置非 KDE 或非 Gnome 桌面管理器

Before following anything in this part, make sure the server working setup and accepting connections. This section only deals with problems once NXClient has logged on.

It is quite simple (once the server is setup) to connect to Gnome and KDE sessions, however connecting to other window managers (fluxbox, xfce, whatever) is slightly different.

Choosing "custom" and using a command like startx of startfluxbox will either result in a blank screen after the !M logo or the Client to present an error complaining about lack of a X server. A way around this is open a session with the command "startx", and the another with the command to start your window-manager-of-choice.

If you do not want to do this, you can start X by installing a login manager like SLIM or XDM. I would recomend using SLiM because of its small size.

(Authors note: This is how I got fluxbox, xfce and others to work on my arch installation- however, I have now removed slim from inittab and set the run level back to 3, and yet I can still login perfectly with NXClient. Possibly try this if you get your system working this way, if like me you have a low memory machine.)

Alternative fix

A simple fix without resorting to the above seems to involve a simple edit to the config file. This should work for fluxbox/openbox/xfce or any other window manager that uses the **.xinitrc** startup file in a call to **startx**.

Simply edit the config file (as root):

`/opt/NX/etc/node.conf`
and change

`#USER_X_STARTUP_SCRIPT=.Xclients`
to

`USER_X_STARTUP_SCRIPT=.xinitrc`
Remember to remove the # symbol from the start of the line.

Then in the client under configuration settings, choose **Custom** as the desktop, and click on settings:

- In the first group select - **`Run the default X client Script on server`**
- In the second group select - **`New virtual desktop`**

问题

调试问题

Edit the nxserver config file:

```
vi /opt/NX/etc/node.conf
```
Change:

```
#SESSION_LOG_CLEAN=1
```
to

```
SESSION_LOG_CLEAN=0
```

Then you can look/debug the log files in:

```
$HOME/.nx/T-C-<hostname>-<display>-<session-id>
```

For succesfull connections and:

```
$HOME/.nx/F-C-<hostname>-<display>-<session-id>
```

For failed ones.

验证 OK, 但连接失败

If you are trying to startkde

```
vi /opt/NX/etc/node.conf
```

And search for:

```
COMMAND_START_KDE=startkde
```

Replace for:

```
COMMAND_START_KDE=/usr/bin/startkde
```

按键改变

Change the key in GUI setup to new generated key.

Xorg 7

Be aware that you have to remove the /usr/X11R6 directory, else strange things can happen.

密码错误 / 没有可用连接

- If you have changed your ssh daemon to run on an alternate port, be sure to modify SSHD_PORT within /opt/NX/etc/node.conf.
- If you get always wrong password or no connection after authentication was done and you are sure that you typed it correct, check that your server can connect to itself using localhost by ssh.
- If you messed up your key files, create new ones or fix the old ones, it is probably caused by a wrong known_hosts file.
- If you get wrong password or login, put **ENABLE_PASSDB_AUTHENTICATION="1"** in /opt/NX/etc/node.conf and add a user by

```
# /opt/NX/bin/nxserver --adduser [username]
# /opt/NX/bin/nxserver --passwd [username]
```

在启动 session 时 NX 崩溃

If your NX Client shows the NX logo then disappears with a Connection Problem dialog afterwards.

Then it could be due to missing fonts. Mostly applies if you have installed Arch Linux base and then installed freenx after without the whole X11 set.

Solution until FreeNX Dependencies is fixed is to install xorg-fonts-misc on your NX Server (pacman -S xorg-fonts-misc) and your NX should work.

Note: This does not apply to freenx 0.6.1-3 and above, fix has been incorporated in it and following versions.

显示 NX 徽标之后黑屏

If you see the NX logo (!M) then a blank screen.

This problem can be solved by running a login manager- The problem is that X11 is not started, and it appears that "startx" or similar do not work from the freenx client. Follow these instructions to setup a login manager and load it at startup: Display manager

Blind: If this does not resolve your issues, be aware that freenx and bash_completion do not play well together. I only got things to work after removing bash_completion from the .bashrc.

GDM/XDM Session Menu Error with non-KDE or Gnome Desktop Managers (more common with non-Arch Linux users)

Problem: A session menu comes up talking about "chooseSessionListWidget." A window manager never loads.

Fix :

Double check to see if ~/.xinitrc is executable.

```
ls -la ~/ | grep .xinitrc
```

If the file is not executable, simply

```
chmod +x ~/.xinitrc
```

Keep in mind this command should be executed along with pertinent instructions on this page about "Setting up non-KDE or Gnome desktop managers"

Cannot connect because command sessreg not found

If you get the following error while connecting

```
/opt/NX/bin/nxserver: line 941: sessreg: command not found
NX> 280 Exiting on signal: 15
```

then you have to install the package xorg-server-utils.

3.13 Xorg

难度 高级 | **篇幅** 35.4 KB | **所属** X 服务器

本节将讲到

- 安装
- 运行
- 配置
- 输入设备
- 显示器设置

X.Org 服务器——通常简称为 **X**——是 X.Org 基金会实现的 X 窗口系统 (**X11**) 显示服务器，它是 Linux 用户中最流行的显示服务器。它的普及使其成为图形用户界面应用程序的必备条件，因此大多数发行版都广泛采用了它。

关于 2025 年的 fork，参见 XLibre（英语：XLibre）。关于替代品和继任者，参见 Wayland。

安装

Xorg 可以通过安装 xorg-server包 包来安装。

此外，xorg-apps包组 组中的一些软件包对于某些特定的配置任务是必要的。它们会在相关部分中指出。

软件包组 xorg包组 包含了 Xorg 服务器、xorg-apps包组 中的软件包以及字体。

驱动安装

参阅 图形处理器#安装 以确认您使用的硬件，为其选择驱动并安装所需的包。

特定硬件的 Device Dependent X (DDX) 驱动如今可以看作是遗留方案。xorg-server包 中有一个通用的 modesetting(4) DDX 驱动，它使用内核级显示模式设置（KMS）并且在现代硬件上工作得很好。这个 modesetting DDX 驱动使用 Glamor 来提供 2D 加速，而 Glamor 需要使用 OpenGL。

如果您想要安装另一个 DDX 驱动，请注意 Xorg 会自动搜索已安装的驱动：

- 如果无法找到如下列出的特定硬件的驱动，会首先检查是否安装了不支持 2D 和 3D 加速的 *fbdev* (xf86-video-fbdev包).
- 如果依然没有找到，会搜索 *vesa* (xf86-video-vesa包)，这是一个支持大部分显卡的通用驱动，不提供任何 2D 和 3D 加速功能。
- 如果没有找到 *vesa*，Xorg 会回退到 modesetting(4) DDX 驱动.

传统的特定硬件的 DDX 驱动

通常来说推荐使用 modesetting(4) DDX 驱动。但如果它不适合您的硬件，您可以安装以下任意的传统 DDX 驱动：

- xf86-video-amdgpu包 适配 AMDGPU
- xf86-video-ati包 适配 ATI
- xf86-video-intel包 适配 Intel Gen 2 到 Gen 9 的硬件
- xf86-video-nouveau包 适配 Nouveau
- nvidia-utils包 适配 NVIDIA 专有驱动

注意：由于 nvidia-utils包 里包含了一个 `/usr/lib/xorg/modules/drivers/nvidia_drv.so` DDX 驱动，因此 NVIDIA 专有驱动的用户实际上无法避免安装它。

运行

Xorg(1) 命令通常不直接运行。而是使用显示管理器或者xinit来启动 X server。

提示：用户通常需要选择安装窗口管理器或桌面环境以配合使用 X。

配置

注意：Arch 提供了位于 `/usr/share/X11/xorg.conf.d` 的默认配置文件。通常情况下，用户无需进行额外的配置与修改即可正常使用。

Xorg 使用名为 `xorg.conf` 的配置文件和后缀为 `.conf` 的文件作为它的初始设置。这些文件的位置的完整列表可以在xorg.conf(5)中找到，其中还附有全部可用选项的详尽解释。

使用 .conf 文件

`/etc/X11/xorg.conf.d/` 目录保存主机特有设置，你可以创建自己的配置文件，需要以 `XX-` 开头(两个数字和一个连接符)并以 `.conf` 结尾。X 服务器启动时会解析这些文件，将其视为 `xorg.conf` 的一部分进行处理。如果配置之间有冲突，将会使用*最后*被处理的文件。所以通用的设置应该放到前面。最后会解析 `xorg.conf` 文件。

有关配置选项请参考 Fedora wiki。

使用 xorg.conf

可以通过 `/etc/X11/xorg.conf` 或 `/etc/xorg.conf` 配置 Xorg，用下面命令可以生成 `xorg.conf` 模板：

```
# Xorg :0 -configure
```

执行后会在 `/root/` 生成 `xorg.conf.new` 文件，然后你可以将它复制到 `/etc/X11/xorg.conf`。

提示：如果已经运行了 X 服务器，请使用不同的 display，例如 `Xorg :2 -configure`。

或者，显卡的专有驱动可能也提供了自动配置 Xorg 的工具，详情请参考 NVIDIA。

注意：配置文件关键字是大小写敏感的，"_" 字符会被忽略。配置，包括选项名也是大小写敏感的，会自动忽略空白和 "_"。

输入设备

对于输入设备，X 服务器默认使用 libinput 驱动 (xf86-input-libinput包)，但 xf86-input-evdev包 和相关驱动也可作为替代方案。

Udev，作为 systemd 的依赖项被提供，将会检测硬件。这两个驱动程序将作为几乎所有设备的热插拔输入驱动，它们的行为定义在位于 `/usr/share/X11/xorg.conf.d/` 的默认配置文件 `10-quirks.conf` 和 `40-libinput.conf` 中。

在启动 X server 后，日志文件将会为每个设备显示发生了什么热插拔（注意最近的日志名称可能有所不同）：

```
$ grep -e "Using input driver " Xorg.0.log
```

如果两个驱动都不支持您的设备，请从 xorg-drivers包组 组安装需要的驱动程序。如果你想要使用其他驱动，也可以这样做。

想要干预热插拔，请参考 #配置。

更详细的信息，请参考 libinput 文后的链接和 Fedora wiki。

输入映射

参考键盘按键#映射按键码。

鼠标加速

鼠标加速。

扩展鼠标按键

鼠标按键。

触摸板

参考 Libinput 或 Touchpad_Synaptics。

触摸屏

触摸屏。

键盘设置

参考 Xorg/键盘设置（英语：Xorg/Keyboard_configuration）。

显示器设置

手动配置

注意：

- 新的 Xorg 版本会自动配置显示器，无需额外配置。
- 如果 Xorg 无法检测到任何显示器，或者没有自动配置，则可使用配置文件。一个常见的情况是使用无头系统时，系统在没有显示器的情况下启动并自动运行了 Xorg，要么是在登录 时从 虚拟控制台，或者是从一个显示管理器。

无头配置需要 xf86-video-dummy包 驱动； 安装然后创建一个配置文件，例如：

```
/etc/X11/xorg.conf.d/10-headless.conf
Section "Monitor"
        Identifier "dummy_monitor"
        HorizSync 28.0-80.0
        VertRefresh 48.0-75.0
        Modeline "1920x1080" 172.80 1920 2040 2248 2576 1080 1081 1084 1118
EndSection

Section "Device"
        Identifier "dummy_card"
        VideoRam 256000
        Driver "dummy"
EndSection

Section "Screen"
        Identifier "dummy_screen"
        Device "dummy_card"
        Monitor "dummy_monitor"
        SubSection "Display"
        EndSubSection
EndSection
```

多显示器

参考主要文章多显示器了解通用信息。

多于一个显卡

你必须指定正确的驱动，输入你的显卡的 BusID（以十进制表示）。

```
Section "Device"
    Identifier              "Screen0"
    Driver                  "intel"
    BusID                   "PCI:0:2:0"
EndSection

Section "Device"
    Identifier              "Screen1"
    Driver                  "nouveau"
    BusID                   "PCI:1:0:0"
EndSection
```

为了获取 BusID （十六进制）：

```
$ lspci -d ::03xx
00:02.0 VGA compatible controller: Intel Corporation HD Graphics 630 (rev 04)
01:00.0 3D controller: NVIDIA Corporation GP107M [GeForce GTX 1050 Mobile] (rev a1)
```

这个示例的BusID是 `0:2:0` 和 `1:0:0`。

显示大小和 DPI

默认情况下，Xorg 自 2009-01-30 起始终将 DPI 设置为 96。版本 21.1 时有一个自动侦测的修改，但是被回退了。

通过 `-dpi` 命令行选项可以设置 X 服务器的DPI。

在需要精细细节（如字体渲染）的情况下，拥有正确的DPI尤其必要。此前，制造商试图为96 DPI（10.3英寸对角线显示器为800x600，13.2英寸显示器为1024x768）创

建一个标准。如今，屏幕 DPI 各不相同，在水平和垂直方向上可能不相等。例如，1440x900的19英寸宽屏LCD的DPI可能为89x87。

要查看您的显示大小和 DPI 是否正确：

```
$ xdpyinfo | grep -B2 resolution
```

如果明确知道显示器的物理尺寸规格，可以在Xorg配置文件中设置，这样就可以计算出合适的DPI。

```
Section "Monitor"
    Identifier             "Monitor0"
    DisplaySize             286 179    # 单位：毫米
EndSection
```

如果你只想设置显示器的物理尺寸的规格，不用从头到尾创建新的完整 xorg.conf 配置文件的话，可以在 `/etc/X11/xorg.conf.d/90-monitor.conf` 中设置屏幕的规格：

```
Section "Monitor"
    Identifier             "<default monitor>"
    DisplaySize            286 179    # In millimeters
EndSection
```

注意：如果你使用的是专有 NVIDIA 驱动程序，你可能需要把 `Option "UseEdidDpi" "FALSE"` 放在 `Device` 或 `Screen` 部分下才能生效。

如果你没有屏幕的物理宽度和高度的规格（现在大多数规格只按对角线尺寸列出）的话，可以使用显示器的原始分辨率（或长宽比）和对角线长度来计算水平和垂直的物理尺寸。

用勾股定理（毕达哥拉斯定理）计算一个对角线长度为13.3英寸、原始分辨率为1280x800（或长宽比为16:10）的屏幕：

```
$ echo 'scale=5;sqrt(1280^2+800^2)' | bc  # 1509.43698
```

这将给出像素的对角线长度，有了这个值，你就可以获得水平和垂直的物理长度（并将其转换成毫米）：

```
$ echo 'scale=5;(13.3/1509)*1280*25.4' | bc  # 286.43072
$ echo 'scale=5;(13.3/1509)*800*25.4'  | bc  # 179.01920
```

注意：这种计算适用于正方形的显示器。然而，很少有显示器可以压缩纵横比（例如16:10的纵横比分辨率为16:9的显示器）。如果是这种情况，您应该手动测量屏幕大小。

手动设置DPI

注意：虽然你可以设置任何你喜欢的DPI，并且使用Qt和GTK的应用程序将相应地进行缩放，但还是建议将其设置为 **96** (100%, 无缩放), **120** (高出25%), **144** (高出50%), **168** (高出75%), **192** (高出100%) 等，以减少使用位图的GUI的缩放瑕疵。把它降低到96 dpi以下可能不会减少GUI的图形元素的大小，因为通常图标的最低dpi是96。

对于兼容RandR的驱动程序（比如开源的ATI驱动程序），你可以通过以下方式设置。

```
$ xrandr --dpi 144
```

注意：更改并不会即时生效，你需要重新启动它们。

参阅 Autostarting#Xorg 使其永久化保存.

专有的 NVIDIA 驱动程序

您可以通过在`Device`或`Screen`部分下添加选项来手动设置DPI：

```
Option              "DPI" "96 x 96"
```

手动DPI设置的注意事项

GTK 经常通过可选的 X resource `Xft.dpi` 覆盖X服务器的DPI。 可以通过这个来检查当前的DPI：

```
$ xrdb -query | grep dpi
```

对于 3.16 版本之后的 GTK 库，当此变量未明确设置时，GTK 会将其设置为 96。要让 GTK 应用程序遵守X服务器 DPI设置，你可能需要将 Xft.dpi 显式设置为与服务器相同的值。 Xresource `Xft.dpi`是一些桌面环境在个人设置中强制设置DPI到一个特定值的方法。 其中包括 KDE 和 TDE。

DPMS

DPMS (显示器电源管理信号) 是一种技术，可以在计算机不使用时，可以使用显示器的省电行为. 这将允许您的显示器在预定时间段后自动进入待机。

合成

X 的合成扩展使窗口层次结构的整个子树呈现到屏幕外的缓冲区。应用程序可以获取缓冲区的内容并执行它们喜欢的任何操作。屏幕外的缓冲区可以自动合并到父窗口中，也可以被称为合成管理器的外部程序合并。要了解更多信息，参考compositing window manager。

某些窗口管理器（例如 Compiz, Enlightenment, KWin（英语：KWin）, marco包, metacity包, muffin包, mutter包, Xfwm）自己完成这些合成。对于其他窗口管理器，可以使用一个独立的合成管理器。

合成管理器列举

- **Picom** — 支持阴影、高级模糊和渐变的轻量级合成管理器。

: https://github.com/yshui/picom || picom包

- **Xcompmgr** — 复合窗口效果管理器

: https://gitlab.freedesktop.org/xorg/app/xcompmgr/ || xcompmgr包

- **fastcompmgr** — 一个非常轻量级的 X11 合成器，专注于延迟和性能

: https://github.com/tycho-kirchner/fastcompmgr || fastcompmgrAUR

- **Gamescope** — Valve 的微合成器，具有面向游戏的功能，如 FSR 上采样。从 steamos-compositor 分叉而来。

: https://github.com/ValveSoftware/gamescope || gamescope包

- **steamos-compositor-plus** — Valve 的合成器，带有一些额外的调整和修复。

: https://github.com/chimeraos/steamos-compositor-plus || steamos-compositor-plusAUR

技巧和技巧

自动化

这一节列出用于键盘、鼠标输入和窗口操作自动化的实用程序（例如移动、调整大小和层级）。

工具	软件包	手册	keyboard input	窗口操作	注
xautomation	xautomation包	xte(1)	是	否	也包含 screen scraping tools。无法模拟 F13+.
xdo	xdo包	xdo(1)	否	是	用于执行基本窗口操作的轻量级X实用工具
xdotool	xdotool包	xdotool(1)	是	是	充满 Bug 且开发不活跃，比如说CLI解析错误。
xvkbd	xvkbdAUR	xvkbd(1)	是	否	面向 Xorg 的虚拟键盘，在发送字母时也有 `--text` 选项。
AutoKey	autokey-qtAUR autokey-gtkAUR	文档	是	是	强大的宏和脚本工具，有Qt和GTK两种前端。

也可以参考 Clipboard#Tools and an overview of X automation tools.

嵌套 X 会话

在嵌套的 X 的会话中启动其他桌面环境：

```
$ /usr/bin/Xnest :1 -geometry 1024x768+0+0 -ac -name Windowmaker & wmaker -display :1
```

这会在你当前的 X 会话中启动一个1024 × 768大小的 Window Maker 会话。

需要安装xorg-server-xnest包。

Xephyr 是一个更新的方法。

远程启动 GUI 程序

查看主条目：X11转发。

按需禁用和启用输入源

利用*xinput*你可以您可以暂时禁用或启用输入源。这可能很有用，例如，在具有多个鼠标的系统上（如ThinkPad），可以只使用一个鼠标以避免不必要的点击。

从official repositories安装 xorg-xinput包。

找到要禁用的设备的ID：

```
$ xinput
```

例如在Lenovo ThinkPad T500中，输出如下所示：

```
$ xinput
⎡ Virtual core pointer                          id=2    [master pointer  (3)]
⎜   ↳ Virtual core XTEST pointer                id=4    [slave  pointer  (2)]
⎜   ↳ TPPS/2 IBM TrackPoint                     id=11   [slave  pointer  (2)]
⎜   ↳ SynPS/2 Synaptics TouchPad                id=10   [slave  pointer  (2)]
⎣ Virtual core keyboard                         id=3    [master keyboard (2)]
    ↳ Virtual core XTEST keyboard               id=5    [slave  keyboard (3)]
    ↳ Power Button                              id=6    [slave  keyboard (3)]
    ↳ Video Bus                                 id=7    [slave  keyboard (3)]
    ↳ Sleep Button                              id=8    [slave  keyboard (3)]
    ↳ AT Translated Set 2 keyboard              id=9    [slave  keyboard (3)]
    ↳ ThinkPad Extra Buttons                    id=12   [slave  keyboard (3)]
```

使用`xinput --disable *device_id*`禁用设备，*device_id*是你要禁用的设备的ID。在此示例中，我们将禁用ID为10的Synaptics触摸板：

```
$ xinput --disable 10
```

要重新启用该设备，只需发出相反的命令：

```
$ xinput --enable 10
```

持久禁用输入源

你可以使用配置片段来禁用特定的输入源：

```
/etc/X11/xorg.conf.d/30-disable-device.conf
Section "InputClass"
        Identifier   "disable-device"
        Driver       "driver_name"
        MatchProduct "device_name"
        Option       "Ignore" "True"
EndSection
```

`*device*` 是一个任意名称，`*driver_name*` 是输入驱动程序的名称，例如 `libinput`。`*device_name*` 是实际用于匹配正确设备的内容。对于其他方法（例如 libinput 的 `MatchIsTouchscreen`），请参阅输入驱动程序的文档。虽然此示例使用 libinput，但这是一个与驱动程序无关的方法，它只是阻止设备传播到驱动程序。

使用热键结束应用程序

在热键上运行脚本：

```
#!/bin/sh
windowFocus=$(xdotool getwindowfocus);
pid=$(xprop -id "$windowFocus" | grep PID);
kill -9 "$pid"
```

依赖：xorg-xprop包，xdotool包

阻止 TTY 访问

要在 X 会话中阻止 tty 访问，请将以下内容添加到 xorg.conf：

```
Section "ServerFlags"
    Option "DontVTSwitch" "True"
EndSection
```

这可以用来限制不受信任的用户通过命令行访问系统。

如果要仅在锁屏时阻止 TTY 访问，而不是在整个 X 会话中阻止，参阅 vtlockAUR。

防止用户结束 X

要防止用户在运行时被结束，请将以下内容添加到xorg.conf：

```
Section "ServerFlags"
    Option "DontZap"       "True"
EndSection
```

注意：The `Ctrl+Alt+Backspace` shortcut is not directly what triggers killing the X server, but the `Terminate_Server` action from the keyboard map. This is usually not set by default, see Xorg/Keyboard configuration#Terminating Xorg with Ctrl+Alt+Backspace.

可视化结束应用程序

当应用程序行为异常或卡住时，与其从终端使用 `kill` 或 `killall` 并查找进程 ID 或名称，不如使用 xorg-xkill包 点击该应用程序以关闭其与 X 服务器的连接。许多现有应用程序确实会在与 X 服务器的连接关闭时中止，但有些应用程序可能会选择继续运行。

套接字

Xorg 支持监听 TCP 套接字和抽象套接字，这可能会破坏某些沙盒程序。要堵上这个漏洞，可以使用 `-nolisten tcp -nolisten local` 命令行参数来运行 Xorg。要了解背景信息和故障排除方法，请参阅 Tianon 的 write-up.

没有 root 权限的 Xorg

Xorg 可以使用标准用户权限而不是 root 来运行（所谓的“rootless” Xorg）。这对于用 root 权限运行是很大的安全性提升。注意大多数显示管理器都不支持 rootless Xorg。

你可以用 `ps -o user $(pgrep Xorg)` 来验证 Xorg 在以什么用户的身份运行。

也可参考Xorg.wrap(1), systemd-logind(8), Systemd/用户#将 Xorg 作为 systemd 用户服务, 和 FS#41257。

使用 xinitrc

要使用 xinitrc 配置没有 root 权限的 Xorg:

- 将 startx 作为当前登录 shell 的子进程运行；直接运行 `startx`，不要使用 `exec startx`
- 确保 Xorg 使用已设置权限的虚拟终端，即通过 logind 通过 .xserverrc 在 `$XDG_VTNR` 中传递。
- 如果使用某些专有的驱动程序, kernel mode setting 自动检测将失效. 在这种情况下, 必须在 `/etc/X11/Xwrapper.config` 中设置`needs_root_rights = no`。

要注意，不使用 `exec` 直接运行 `startx` 命令会导致在 Xorg 崩溃后 shell 仍然保持打开。由于一些锁屏机制在 Xorg 之中运行，这可能导致任何能物理接触电脑的攻击者都能获得该用户的完整访问权限。

使用 GDM

在使用kernel mode setting时，Xorg将在没有root权限的情况下运行GDM。

重定向Xorg会话日志

当Xorg在没有root权限的情况下运行时，Xorg的日志将保存到`~/.local/share/xorg/Xorg.log`。但是，Xorg会话的stdout和stderr的输出并不会重定向到此日志文件，要重新启动重定向，请使用命令行选项`-keeptty`来启动Xorg,并将stdout和stderr输出重定向到一个文件：

```
startx -- -keeptty >~/.xorg.log 2>&1
```

或者，将`/etc/X11/xinit/xserverrc`复到`~/.xserverrc`并使用命令行选项`-keeptty`来启动Xorg。参阅。

Xorg as Root

如上所述，某些情况下系统会默认启用无root权限的Xorg。若您的配置属于此类情况，且需要以root身份运行Xorg，可通过配置Xorg.wrap(1Xorg.warp(1))强制要求root权限：

警告：在 root下运行 Xorg 可能会引起安全问题。更多讨论参见 #没有 root 权限的 Xorg 。

```
/etc/X11/Xwrapper.config
needs_root_rights = yes
```

Wayback

Wayback 是一个 X11 兼容层，允许使用 Wayland 组件来运行完整的 X11 桌面环境（以及窗口管理器）。它可以从 AUR 下载：wayback-x11AUR。

12to11

12to11 可以让您在 X11 下无缝运行仅支持 Wayland 的应用程序。它可以从 AUR 下载：12to11-gitAUR。使用 EGL 渲染器来运行以获得最佳性能：

```
$ RENDERER=egl 12to11
```

故障和修复

一般问题

如果你在使用Xorg中遇到问题，请查看位于 `/var/log/` 的日志，或者对于从 v1.16 起的没有 `root` 权限的 Xorg，日志文件位于 `~/.local/share/xorg/`。 GDM 用户应当检查 systemd journal.

日志文件的格式为 `Xorg.n.log`， 其中 `n` 表示显示器编号。对于单用户、使用默认配置文件的机器，日志文件常常是 `Xorg.0.log`，其它情形可能会有区别。 想要找到正确的文件，可以考虑检查 X 服务器会话启动的时间戳以及它是从哪个控制台启动的。例如：

```
$ grep -e Log -e tty Xorg.0.log
[    40.623] (==) Log file: "/home/archuser/.local/share/xorg/Xorg.0.log", Time: Thu
Aug 28 12:36:44 2014
[    40.704] (--) controlling tty is VT number 1, auto-enabling KeepTty
```

提示：要使用人类可读的时间戳监控日志，可以将 tail(1) 的输出通过管道传递给 ts(1)（由 moreutils包 包提供）。这将仅在命令运行时为添加到日志中的行提供正确的时间戳。例如：

```
$ tail -f ~/.local/share/xorg/Xorg.0.log | ts
```

- 在日志文件中寻找以 `(EE)` 开头的行，它代表错误，以及 `(WW)`，代表警告（也可能暗示着其他问题）。
- 如果在你的`$HOME`目录下有一个*空的* `.xinitrc` 文件，删除或修改它以使 X 正确启动。如果你不这样做，X 会显示黑屏并不会在 `Xorg.0.log` 中输出任何错误。简单地删除它就可以使 X 以默认配置运行。
- 如果你遇到黑屏，你仍可以尝试切换到不同的虚拟控制台 （例如 `Ctrl+Alt+F6`），然后不假思索地以 root 登录。你可以通过输入 `root` （然后按下 `Enter`） 然后输入密码（同样地，输入后按下 `Enter`）。

: 你也可以尝试使用以下命令杀死 X server： : ```

```
: 如果不行的话，直接重启:
: ```
# reboot
```

- 如果你有关于键盘，鼠标，触摸板等等的问题，参考 Category:Input devices 中具体的文章。
- 最后，在AMDGPU, Intel和NVIDIA等文章中搜索常见问题。

黑屏，没有指定协议..，资源暂时不可用等问题

X在当前用户的主目录中创建配置和临时文件。确保主目录所在的分区上有可用的可用磁盘空间。然而，在这种情况下，X服务器不提供有关磁盘空间不足的更明显信息。

Matrox显卡的DRI功能失效

如果你使用的是Matrox显卡，在升级到Xorg7后它的DRI功能失效，试着在xorg.conf的显卡设备设置段Device section中加入下面一行：

```
Option "OldDmaInit" "On"
```

无法运行在frambuffer模式下

如果X启动失败，日志中有以下信息：

```
 (WW) Falling back to old probe method for fbdev
 (II) Loading sub module "fbdevhw"
 (II) LoadModule: "fbdevhw"
 (II) Loading /usr/lib/xorg/modules/linux//libfbdevhw.so
 (II) Module fbdevhw: vendor="X.Org Foundation"
        compiled for 1.6.1, module version = 0.0.2
        ABI class: X.Org Video Driver, version 5.0
 (II) FBDEV(1): using default device
```

```
 Fatal server error:
 Cannot run in framebuffer mode. Please specify busIDs        for all framebuffer
devices
```

只需要卸载xf86-video-fbdev包就可以了。

无法加载'(null)'字体

- 一些程序无法运行，提示无法加载\`(null)'字体.

这些软件包可能需要一些额外的字体。某些程序只能使用位图字体。 目前有两种主要的位图字体包：xorg-fonts-75dpi包、xorg-fonts-100dpi包。选择其中一个就可以了。通过下面这个命令查看显示设置：

```
$ xdpyinfo | grep resolution
```

根据显示信息选择合适dpi的字体即可(用75 或 100 代替XX)：

```
# pacman -S xorg-fonts-XXdpi
```

修复：在出现GUI登录界面之前，不启动Xorg

如果Xorg设置为自动启动并且出于某些原因你不想让它出现在 登录/显示 管理器之前，有两种办法:

- 将启动目标修改为 rescue.target. 参阅 systemd#Change default target to boot into.
- 如果 X 无法启动，而且 GRUB 超时时间设置成了 0,无法进 Grub 禁止 Xorg boot. 可以使用 Arch CD 启动，然后挂载配置文件所在分区，

: 可以以root用户使用命令

```
# fdisk -l
```

: 来查看你的全部分区。通常你所要的那个是形如 `/dev/sda1` 这样的东东。然后，使用命令

```
# mount /dev/sda1 /mnt
```

挂载该分区至 `/mnt`。 这样你的文件系统就挂载在了 `/mnt` 下。例如，可以删除 `gdm` 来阻止Xorg正常启动，或者做其他一些必需的改动。

无法用"su"以root身份启动X客户端

如果你遇到"Client is not authorized to connect to server"，尝试将以下内容

```
session        optional        pam_xauth.so
```

加入到`/etc/pam.d/su`文件中。`pam_xauth` 就可以正常设置环境变量以及处理 `xauth` 密钥了。

X 启动失败：键盘初始化失败

遇到“X failed to start : Keyboard initialization failed”。 如果您的硬盘已满，startx将失败。 `/var/log/Xorg.0.log` 的末尾会是：

```
(EE) Error compiling keymap (server-0)
(EE) XKB: Could not compile keymap
(EE) XKB: Failed to load keymap. Loading default keymap instead.
(EE) Error compiling keymap (server-0)
(EE) XKB: Could not compile keymap
XKB: Failed to compile keymap
Keyboard initialization failed. This could be a missing or incorrect setup of
xkeyboard-config.
Fatal server error:
Failed to activate core devices.
...
```

在/分区上腾出一些可用空间，X才能启动。

想看视频时屏幕总是绿色

你的颜色深度设置错误。例如它可能需要设置为24位却被设置成了16位。

SocketCreateListener error

如果 X 结束，伴随着错误信息 "SocketCreateListener() failed"，你可能需要删除位于 `/tmp/.X11-unix` 的 socket 文件。这有可能在你先前以 root 身份运行 Xorg （例如想要生成 `xorg.conf`）后发生。

想要以 root 权限运行程序时出现 无效的 MIT-MAGIC-COOKIE-1 密钥

这个错误意味着只有当前用户有权访问 X server。解决办法是将访问权授予 root：

```
$ xhost +si:localuser:root
```

这条命令也可以将 X server 访问权授予其他用户。

另请参阅

- Xplain - X Window 系统的深入讲解
- Xorg(1)
- Prepare for LPIC-1 exam 2 - topic 106.1: X11 - briefly covers architecture, #Configuration, desktop environments, remote usage, Wayland.
- xorg.conf(5)
- Gentoo:Xorg/Guide#Configuration

GNOME

本章解决的是把装好的 Arch 推到能天天用的 GNOME 桌面：会话的安装、导航与配置，登录管理器 GDM 如何接下启动，文件管理与钥匙环、Gedit、Web、Flashback。

先读《GNOME》，导航与配置别跳；再读《GDM》，装完会话不等于登录就位；《GNOME/文件》次之。存 SSH 密钥才读《GNOME/Keyring》，《提示与技巧》备查；Gedit、Web 可略过，Flashback 想回传统面板才装。最常踩的坑是照旧名找组件：多已改名，按旧称检索会扑空，先看“遗留名称”。

4.1　GNOME/Gedit

难度 入门 | **篇幅** 1.2 KB | **所属** GNOME

本节将讲到

- 安装
- 配置
- 另见

gedit是GNOME的通用文本编辑器。

安装

安装gedit包软件包。

若要使用额外功能，请安装gedit-plugins包软件包。

Gedit有许多拼写检查词典可供使用，见Language checking。

配置

不在新的一行结束文档

若想确保gedit不会在新的一行结束文档，请执行以下操作：

```
$ gsettings set org.gnome.gedit.preferences.editor ensure-trailing-newline false
```

为编辑过的文件保存备份

若有需要，gedit可以为编辑过的文件创建备份副本，副本的名称和内容将与编辑前的原始文件的内容相同，但名称后面添加了“~”后缀。例如文件`file1`的副本名称为`file1~`。备份文件默认隐藏。

若要开启此功能，请访问gedit的首选项（GNOME Shell用户可以在gedit的全局菜单中找到）。在首选项中点击“编辑器”标签栏，勾选“在保存前创建备份文件”选项。

另见

- Apps/Gedit - GNOME Wiki!

4.2 GNOME/Web

难度 入门 | **篇幅** 4.3 KB | **所属** GNOME

本节将讲到

- 安装
- 配置
- 参见

Web是GNOME的默认网络浏览器。Web 提供了一个简单而简约的界面来访问互联网。虽然它主要是为 GNOME 开发的，但 Web 在其他桌面环境中也能正常工作。

注意：在3.4版本之前，Web 被称为 Epiphany。该应用程序被赋予新的描述性名称，每种支持的语言都有一个。*Epiphany*这个名字仍然用在许多地方，如可执行文件名、一些软件包名、一些桌面条目和一些 GSettings 纲要。

安装

Web 能以 epiphany包 包安装。如果您想保存登录密码，请安装gnome-keyring包。

配置

广告屏蔽

它是默认启用的，您可以通过解除勾选在*首选项*中的*阻止广告*禁用它。EasyList 是默认的阻止列表。所有列表都会定期刷新。

要获取当前启用的过滤器列表：

```
$ gsettings get org.gnome.Epiphany content-filters
```

可以按照 https://gitlab.com/eyeo/filterlists/contentblockerlists 中的示例使用 JSON 格式的资源修改过滤器：

```
$ gsettings set org.gnome.Epiphany content-filters
"['https://gitlab.com/eyeo/filterlists/contentblockerlists/-/raw/master/easylist_min_content_blocker.json',
'https://gitlab.com/eyeo/filterlists/contentblockerlists/-/raw/master/easylist+easylistchina-minified.json']"
```

提示：dconf-editor包 可用作 GUI 替代品。

跟踪预防

Web 包括“智能跟踪器预防”，可以在“首选项”中启用。

火狐同步

Web 允许使用火狐同步来同步书签、历史记录、密码和打开的标签。它可以在“导入和导出”下拉菜单中进行配置。

Web 应用

Web 可以从网站创建 Web 应用程序并将它们添加到桌面菜单。要配置和删除它们，请在地址栏中输入 `about:applications`。

自定义样式表

Web 支持自定义样式表，您可以在**首选项**中的**外观**下启用。

使用下面的示例根据 Adwaita 深色变体设置新的标签页布局和颜色：

```
~/.config/epiphany/user-stylesheet.css
#overview {
  background-color: #2E3436 !important;
  max-width: 100% !important;
  max-height: 100% !important;
  position: fixed !important;
}

#overview .overview-title {
  color: white !important;
}
```

字体

Web 不会检查 GNOME 字体设置，但会检查 Font configuration。

视频

有关所需的插件安装，请参阅 GStreamer。

要启用硬件加速视频解码，请参阅 GStreamer#Hardware video acceleration 和 #Hardware 加速合成\[损坏的链接：无效的章节\]。

硬件加速合成

默认情况下，仅在需要（按需）显示 3D 变换时才使用硬件加速合成。

强制启用硬件加速合成：

```
$ gsettings set org.gnome.Epiphany.web:/ hardware-acceleration-policy 'always'
```

代理配置

Web 不尊重 socks_proxy，相反，您可以将 http_proxy 设置为 `socks:// URL` ：

```
export http_proxy=socks://127.0.0.1:1080
```

更多信息：代理服务器#环境变量

参见

- 应用程序/网络 - GNOME Wiki!

4.3 GNOME/Flashback

难度 入门 | **篇幅** 5.4 KB | **所属** GNOME

本节将讲到

- 安装
- 启动
- 配置
- 提示与技巧
- 参见

翻译状态：

- 本文（或部分内容）译自 GNOME/Flashback，最近一次同步于 2024-11-16，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面/Flashback_(%E7%AE%80%E4%BD%93%E4%B8%AD%E6%96%87)?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

GNOME Flashback（之前称为“GNOME Fallback”）是一个为 GNOME 3 提供的外壳。桌面布局和底层技术与 GNOME 2 相似。它完全不使用 3D 加速，因此通常比启用了 llvmpipe 的 GNOME Shell 更加快速且对 CPU 的占用更低。

安装

GNOME Flashback 可以从 gnome-flashback包 安装。建议同时安装其可选依赖项，以获得更完整的桌面环境。

您还可以安装以下提供额外 GNOME 面板小程序的包：

- **GNOME Applets** — 小型 GNOME 面板应用程序

: https://wiki.gnome.org/Projects/GnomeApplets || gnome-applets包

- **Pomodoro Applet** — 用于计时番茄工作法的 GNOME 面板小程序

: https://github.com/stump/pomodoro-applet || pomodoro-appletAUR

- **Sensors Applet** — 显示硬件传感器读数（包括 CPU 温度、风扇速度和电压读数）的 GNOME Flashback 面板小程序

: https://sensors-applet.sourceforge.net/ || sensors-applet包

建议安装 gnome包组，其中包含标准 GNOME 体验所需的应用程序。

启动

图形化登录

在所选显示管理器的菜单中选择“GNOME Flashback (Metacity)”。

希望使用 Compiz 的用户应选择“GNOME Flashback (Compiz)”。

手动启动

- 对于 **GNOME Flashback (Metacity)** 会话，在 `~/.xinitrc` 文件中添加以下内容：

```
export XDG_CURRENT_DESKTOP=GNOME-Flashback:GNOME
exec gnome-session --session=gnome-flashback-metacity
```

- 对于 **GNOME Flashback (Compiz)** 会话，在 `~/.xinitrc` 文件中添加以下内容：

```
export XDG_CURRENT_DESKTOP=GNOME-Flashback:GNOME
exec gnome-session --session=gnome-flashback-compiz
```

编辑 `.xinitrc` 后，可以使用 *startx* 启动 GNOME Flashback。详情请参见 xinitrc。

配置

GNOME Flashback 与 GNOME 共享大部分设置。更多详情请参见 GNOME#配置。

自定义 GNOME 面板

- 要配置面板，按住 `Alt` 键，然后在空白区域右键单击面板。
- 要移动面板上的小程序，按住 `Alt` 键，并使用中键拖动它。

注意：如果 `Alt`+右键单击组合无法使用，请尝试使用 `Super+Alt`+右键单击。

替代窗口管理器

您可以通过创建一个自定义的 GNOME 会话来使用替代的窗口管理器，该会话应包含以下组件：

```
RequiredComponents=gnome-flashback-init;gnome-flashback;gnome-panel;窗口管理器;gnome-settings-daemon;nautilus-classic;
```

其中**窗口管理器**是您希望使用的窗口管理器。请参见 GNOME/提示与技巧#自定义 GNOME 会话。

另请参见这篇文章，了解如何在 GNOME 中运行 awesome 窗口管理器。

提示与技巧

面板速度设置

隐藏/显示延迟

要调整启用自动隐藏时面板消失或重新出现的时间，请执行以下命令：

```
$ gsettings set org.gnome.gnome-panel.toplevel:/org/gnome/gnome-panel/layout/toplevels/panel/ hide-delay time
$ gsettings set org.gnome.gnome-panel.toplevel:/org/gnome/gnome-panel/layout/toplevels/panel/ unhide-delay time
```

其中 *panel* 为 *top-panel* 或 *bottom-panel*，*time* 为毫秒值，例如 300。

动画速度

要设置面板动画的速度，请执行以下命令：

```
$ gsettings set org.gnome.gnome-panel.toplevel:/org/gnome/gnome-panel/layout/toplevels/panel/ animation-speed value
```

其中 *panel* 为 *top-panel* 或 *bottom-panel*，*value* 为 `"'fast'"`, `"'medium'"` 或 `"'slow'"`。

替换应用程序菜单图标

注意：此更改将在更新您的图标主题包时被覆盖。

将 `/usr/share/icons/*icon-theme*/16x16/places/start-here.png` 替换为您自己的图标（其中 `*icon-theme*` 为您的图标主题的名称）。

完成更改后，重新启动 GNOME 面板： `gnome-panel --replace`。

参见

- GnomeFlashback - GNOME Wiki

4.4 GNOME/文件

难度 高级 | **篇幅** 20.0 KB | **所属** GNOME

本节将讲到

- 安装
- 配置
- 提示与技巧
- 疑难解答

文件是GNOME的默认文件管理器。文件的目的是提供一种管理文件和应用的简化方法。可扩展以通过SFTP、WebDAV、NFS等不同协议实现网络文件访问。

注意：在3.6版本之前，文件叫做Nautilus，后来则对每种语言做了新的描述性名称。*Nautilus*这个名字仍然用在许多地方，如可执行文件名、软件包名、桌面条目和GSettings Schema等。

安装

安装 nautilus包 软件包，此软件包是 gnome包组 组的一部分。见文件管理器功能#额外功能。

注意："文件"不依赖 gnome-shell包，只需要 gnome-desktop包。

扩展

网络共享与虚拟文件系统

通过安装 gvfs包 及各种 `gvfs-*` 软件包，可以添加对多种基于网络的文件系统（例如 SMB、NFS、WebDAV、Nextcloud）以及部分移动设备（Android MTP、Apple AFC）的支持。

有关更多信息及其他受支持的虚拟文件系统，请参见 文件管理器功能。

其他示例

注意：用 Python 编写的扩展需要 Nautilus 扩展 API 的 Python 绑定（nautilus-python包 包）。

- **Actions For Nautilus** — 允许在文件选择右键菜单中添加任意操作的扩展。（使用 Python 编写）

: https://github.com/bassmanitram/actions-for-nautilus || actions-for-nautilus-gitAUR

- **Folder Color** — 用于选择文件夹颜色的文件浏览器扩展。（使用 Python 编写）

: https://github.com/costales/folder-color || folder-color-nautilusAUR : > 提示：此扩展仅适用于包含额外彩色图标的图标主题，例如：

numix-icon-theme-gitAUR、vibrancy-colorsAUR、humanity-icon-themeAUR、mint-x-iconsAUR。

- **Nautilus Admin** — 在菜单中添加"以管理员身份打开"或"以管理员身份编辑"选项。（使用 Python 编写）

: https://github.com/MacTavishAO/nautilus-admin-gtk4 || nautilus-admin-gtk4AUR

- **Nautilus Annotations** — 为文件和目录添加注释。（使用 C 编写）

: https://gitlab.gnome.org/madmurphy/nautilus-annotations/ || nautilus-annotationsAUR

- **Nautilus Bluetooth** — 在菜单中添加"通过蓝牙发送"选项。（使用 C 编写）

: https://gitlab.gnome.org/madmurphy/nautilus-bluetooth/ || nautilus-bluetoothAUR

- **Nautilus Checksums** — 在 Nautilus 的属性窗口中添加校验和信息。（使用 C 编写）

: https://gitlab.gnome.org/madmurphy/nautilus-checksums/ || nautilus-checksumsAUR

- **Nautilus Compare** — Nautilus 的右键菜单文件差异比较扩展。它在 Nautilus 右键菜单中提供“比较”、“与 ~/foo/bar 比较”和“稍后比较”选项，默认使用 `meld` 进行比较。（使用 Python 编写）

: https://launchpad.net/nautilus-compare || nautilus-compareAUR

- **Nautilus Hide** — 在菜单中添加“隐藏”/“取消隐藏”选项。（使用 C 编写）

: https://gitlab.gnome.org/madmurphy/nautilus-hide/ || nautilus-hideAUR

- **Nautilus Image Converter** — 调整图像大小或旋转图像。（使用 C 编写）

: https://gitlab.gnome.org/coreyberla/nautilus-image-converter || nautilus-image-converter包

- **Nautilus Launch** — 允许通过右键菜单运行可执行文件和启动器的 Nautilus 扩展。（使用 C 编写）

: https://gitlab.gnome.org/madmurphy/nautilus-launch || nautilus-launchAUR

- **Nautilus Open Any Terminal** — 在右键菜单中添加使用其他终端模拟器打开的选项的 Nautilus 扩展。

: https://github.com/Stunkymonkey/nautilus-open-any-terminal || nautilus-open-any-terminalAUR

- **Nautilus Metadata Editor** — 为以下 MIME 类型提供简单元数据编辑器的 Nautilus 扩展：`audio/x-mp3`、`audio/x-flac`、`audio/x-vorbis+ogg`、`audio/x-speex+ogg`、`audio/x-musepack`、`audio/x-wavpack`、`audio/x-tta`、`audio/x-aiff`、`audio/m4a`、`video/mp4`、`video/x-ms-asf`。（使用 C 和 Vala 编写）

: https://gitlab.com/nvlgit/nautilus-metadata-editor-extension || nautilus-metadata-editorAUR

- **Nautilus Share** — 通过 Samba 共享文件夹的 Nautilus 扩展。（使用 C 编写）

: https://gitlab.gnome.org/coreyberla/nautilus-share || nautilus-share包

- **Seahorse Nautilus** — 为 GNOME 文件管理器提供 PGP 加密和签名功能。（使用 C 编写）

: https://gitlab.gnome.org/GNOME/seahorse-nautilus || seahorse-nautilusAUR : > **警告：***seahorse-nautilus* 的上游代码仓库已归档，并且由于存在尚未修复的错误，上游明确警告不要使用该扩展。

- **Sushi** — Nautilus 的快速文件预览工具，属于 gnome包组 软件包组。

: https://gitlab.gnome.org/GNOME/sushi || sushi包

提示：若要编写新的扩展，辅助脚本 nextgenAUR 可以让您轻松地设置 **GNOME**文件的新扩展项目（默认使用C语言）。

集成

有些应用自带与**GNOME/文件**集成的功能。

- **Eiciel** — GNOME file ACL editor — The application includes an "**Eiciel Nautilus**" extension that adds graphical ACL editor into the file properties window (written in C++)

: https://rofi.roger-ferrer.org/eiciel/ || eicielAUR

- **File Roller** — An application for browsing archives — The application includes a "**Nautilus FileRoller**" extension (written in C)

: https://wiki.gnome.org/Apps/FileRoller || file-roller包

- **GNOME Terminal** — The GNOME Terminal Emulator — The application includes a "**Terminal Nautilus**" extension (written in C)

: https://wiki.gnome.org/Apps/Terminal || gnome-terminal包

- **Tilix** — A tiling terminal emulator for GNU/Linux using GTK+ 3 — The application includes an extension for Nautilus which adds an "Open in Tilix" option to the context menu (written in Python)

: https://github.com/gnunn1/tilix || tilixAUR : > **注意：**“Open in Tilix”扩展将 nautilus-python包 软件包标记为可选依赖，但必须安装才能使用。

依赖非自由软件的集成

一些 **GNOME/文件**的集成功能虽然免费/自由，但可能依赖于非自由软件，例如：

- **Code Nautilus** — Nautilus extension to open files and directories in Visual Studio Code (written in Python)

: https://github.com/cra0zy/code-nautilus || code-nautilus-gitAUR

- **JetBrains Nautilus** — Nautilus extension to open files and directories in JetBrains Toolbox installed products (written in Python)

: https://github.com/encounter/jetbrains-nautilus || jetbrains-nautilus-gitAUR

- **Nautilus Code** — Nautilus extension which adds right-click menu items to open current folder in Visual Studio Code or GNOME Builder (written in C)

: https://github.com/realmazharhussain/nautilus-code || nautilus-codeAUR

配置

文件很容易进行图形化配置，但并非所有选项都能在首选项中找到。更多选项可以使用*dconf-editor*在`org.gnome.nautilus`中找到。

注意：如果在GNOME桌面环境之外使用文件，则必须确保`/usr/lib/gsd-xsettings`正在运行，否则dconf设置不会在文件中应用。

改变默认项目视图

设置`default-folder-viewer`变量可以改变项目的默认视图，如设为列表视图：

```
$ gsettings set org.gnome.nautilus.preferences default-folder-viewer 'list-view'
```

按类型排序

若要默认用类型排序所有文件夹中的文件：

```
$ gsettings set org.gnome.nautilus.preferences default-sort-order 'type'
```

始终显示文本文件路径

文件工具栏默认显示用于路径导航的按钮栏界面。若要用**键盘**输入路径，则需按`Ctrl+l`显示文本路径栏。

若要始终以文本路径栏显示，可以使用以下*gsettings*命令：

```
$ gsettings set org.gnome.nautilus.preferences always-use-location-entry true
```

注意：改变此设置后将无法再显示按钮栏，只有把值设为**false**时才能同时使用两种路径栏。

提示与技巧

缩略图

见文件管理器功能#缩略图预览。

注意：在linux-hardened包中，缩略图会生成失败（所有缩略图都在`~/.cache/thumbnails/fail/`目录中），这是由于非特权用户的命名空间因为安全原因在该内核上默认禁用。Nautilus使用`bwrap`(由bubblewrap包提供)对缩略图进行沙盒处理。尽管存在安全隐患，你仍然可以选择启用非特权用户命名空间。有关更多信息，请参见 FS#63316。

有时会无法显示视频缩略图。若要解决此问题（正如No video thumbnails on nautilus），必须安装ffmpegthumbnailer包、gst-libav包、gst-plugins-ugly包并移除`~/.cache/thumbnails/fail/`里的内容。

用右键菜单创建新文档

若要使用此选项，请在您的主文件夹中创建一个`~/Templates/`（模板）文件夹，并通过终端`touch ~/Templates/new`或任何文件管理器在该文件夹中放置一个空文件，然后重新启动文件。

在非英语系统中，模板目录的名称可能不同。可以用`xdg-user-dir TEMPLATES`找到实际目录。

可以在`~/.config/user-dirs.dirs`文件中配置模板目录。

```
XDG_TEMPLATES_DIR="$HOME/some/path"
```

隐藏文件

GNOME文件和大部分文件管理器一样，默认隐藏名称以“.”开头的文件。

GNOME 文件管理器还会隐藏名称列在同一目录下的 `.hidden` 文件中的文件（每行一个文件名）。可以使用 nautilus-hideAUR 扩展，方便在此类 `.hidden` 文件中添加或删除条目。

在Tilix中打开当前路径

如果你使用 tilixAUR 终端，可以通过安装其可选依赖 nautilus-python包，在 GNOME 文件管理器的右键菜单中添加“在 Tilix 中打开”选项。

将文件夹加入书签

若要将文件夹添加到你的书签中，只需在Nautilus中打开该文件夹，并按`Ctrl+d`。请注意，书签列表与其它基于GNOME的图形化文件管理器（如Nemo）共享，因此在其中增减的文件夹也会影响到其它文件管理器中的书签。

自定义脚本

位于`~/.local/share/nautilus/scripts`的脚本可以运行于文件的右键关联菜单。

The context menu can also be organized into subfolders, e.g. `~/.local/share/nautilus/scripts/images` and `~/.local/share/nautilus/scripts/music`.

Scripts have access to the following environment variables:

```
NAUTILUS_SCRIPT_SELECTED_FILE_PATHS
```

```
NAUTILUS_SCRIPT_SELECTED_URIS
NAUTILUS_SCRIPT_CURRENT_URI
NAUTILUS_SCRIPT_WINDOW_GEOMETRY
```

一些脚本示例:

```
~/.local/share/nautilus/scripts/open-terminal-here
#!/bin/sh
gnome-terminal
~/.local/share/nautilus/scripts/remove-extension
#!/bin/sh
echo "$NAUTILUS_SCRIPT_SELECTED_FILE_PATHS" | while read -r filename; do
    mv -n "$filename" "${filename%.*}"
done
```

注意：Make sure the scripts are marked as executable. You may have to restart nautilus with `nautilus -q` for them to show up.

Keybinds

Keybinds to execute scripts can be assigned in the `~/.config/nautilus/scripts-accels` file:

```
; Example Keybinds
; Modifiers: <Control> <Alt> <Shift>
F4 open-terminal-here
<Alt>x remove-extension
```

注意：You cannot overwrite any pre-existing nautilus keybinds.

疑难解答

文件已不再是默认文件管理器

这可能是因为重置了路径的文件关联，安装anjutaAUR或者visual-studio-code-binAUR往往会造成这种情况。

若要解决此问题，打开文件，右键点击一个文件夹，然后选择*使用其它程序打开* > *文件* > *选择*，就会把路径的关联设回文件。

若没有解决此问题，见文件管理器功能#未在文件管理器中打开目录。

每次复制操作后都会卡几秒

如果系统中安装了kdeconnect包，问题可能来自其文件共享模块，停用文件共享即可。

共享文件夹的 Windows 计算机未显示在网络视图中

提示：有关技术细节，请参见 Samba#Windows 1709 及更高版本无法在“网络”视图中发现 Samba 服务器。

要启用 WSD 支持，请安装 gvfs-wsdd包，使 GNOME 文件管理器能够在网络视图中发现较新版本的 Windows 计算机。

挂载选项中缺少 WebDAV

安装 gvfs-dnssd包 并重新启动 Nautilus。

未安装 gnome-console 时“作为程序运行”无法正常工作

右键菜单中的“作为程序运行”选项被硬编码为使用 gnome-console包。如果你使用其他终端模拟器，且系统中未安装默认的 GNOME 终端，则该菜单选项不会执行任何操作。

由于该选项只是直接执行 `/usr/bin/kgx`（gnome-console包 的可执行文件），因此可以创建一个名为 `kgx` 的脚本（如果你的终端支持相同的命令行参数，也可以创建符号链接），并将其放置在 `/usr/bin/` 中，从而让“作为程序运行”右键菜单选项使用你指定的终端。

以下示例展示如何使用 wezterm-gitAUR 作为“作为程序运行”功能的终端：

```
/usr/bin/kgx
#!/bin/sh

wezterm -e $@
```

注意：子程序执行结束后，gnome-console包 会保持窗口打开并显示输出，直到用户手动关闭。而其他终端模拟器（例如 wezterm）会在程序退出后立即关闭窗口。可以在上述脚本末尾添加类似 `read -n 1 -s -r -p "Press any key to continue"` 的命令，使终端等待用户按下任意键后再关闭。

在任意终端中打开目录

可以通过右键菜单中的“打开方式...”选项，在终端模拟器中打开目录。但终端对应的 XDG 桌面条目必须包含适当的 `Exec` 和 `MimeType` 字段值。

以下示例展示如何为 Alacritty 终端模拟器创建额外的桌面条目，使其能够被“打开方式...”菜单识别。

```
~/.local/share/applications/alacritty-directory.desktop
[Desktop Entry]
Type=Application

# `working-directory` 参数需要提供一个值
Exec=alacritty --working-directory %f
# MimeType 包含特殊类型 `inode/directory;`
MimeType=inode/directory;

Icon=Alacritty
Terminal=false
Categories=System;TerminalEmulator;
Name=Open in Alacritty
GenericName=Terminal
StartupNotify=true
StartupWMClass=Alacritty
```

运行 `update-desktop-database ~/.local/share/applications/` 以应用更改。

如果终端应用程序意外成为默认文件管理器，可以使用 `xdg-mime default org.gnome.Nautilus.desktop inode/directory` 命令修复。

请检查你所使用的终端应用程序的命令行参数和桌面条目文件，并据此调整上述示例。

4.5 GNOME/Keyring

难度 高级 | **篇幅** 21.7 KB | **所属** GNOME

本节将讲到

- 安全性
- 安装
- 用图形界面管理
- 使用钥匙环
- SSH 密钥

GNOME Keyring 是“GNOME 中一组用于存储机密、密码、密钥、证书，并将其提供给应用程序使用的组件”。

它提供了 org.freedesktop.secrets，这是一个 API，允许客户端应用借助运行在用户登录会话中的服务来安全地存储机密。

注意：自 GNOME 51 起，默认的密钥存储由 Oo7（英语：Oo7） 提供。

安全性

防御恶意应用

过去曾报告过一个关于 GNOME/Keyring API 行为的安全问题（即 CVE-2018-19358）。只要钥匙环处于解锁状态，任何应用都能轻易读取任何机密。而如果用户已登录，那么登录/默认钥匙环就是解锁的。现有的 D-Bus 防护机制（涉及 busconfig 和 policy XML 元素）默认并未启用，而且就算启用也很容易被绕过。

GNOME 项目不认同这份漏洞报告，因为按照他们声明的安全模型，不应允许不可信的应用与 secret service 通信。

通过行为规范的沙箱（例如未添加额外权限的 Flatpak）运行的应用，对会话总线只有经过过滤的访问权限。

会话锁定时钥匙环并未锁定

会话锁定时，钥匙环并不会自动锁定。 这意味着密码仍留在内存中，从而使系统暴露于 DMA 攻击之下。

安装

gnome-keyring包 是 gnome包组 组的成员，因此在运行 GNOME 的系统上通常已经存在。在其他情况下，也可以单独安装该软件包。还应安装 libsecret包，以便让其他应用访问你的钥匙环。虽然 libgnome-keyringAUR 已被弃用（并由 *libsecret* 取代），但某些应用可能仍然需要它。

gnome-keyring-daemon 会在登录时通过 systemd 用户服务自动启动。它也可以按需通过套接字启动。

与 GNOME Keyring 相关的其他实用工具包括：

- **secret-tool** — 从命令行访问 GNOME Keyring（以及任何其他实现了 DBus Secret Service API 的服务）。

: https://wiki.gnome.org/Projects/Libsecret || libsecret包

- **lssecret** — 使用 *libsecret* 列出所有机密条目（例如 GNOME Keyring）。

: https://gitlab.com/GrantMoyer/lssecret || lssecret-gitAUR

用图形界面管理

你可以使用 Seahorse 管理 GNOME Keyring 的内容；请安装 seahorse包 软件包。

钥匙环的密码（例如默认钥匙环 “Login”）可以修改，甚至可以移除。更多信息见 GNOME 帮助中的 创建新钥匙环 和 更新钥匙环密码。

使用钥匙环

PAM 模块 *pam_gnome_keyring.so* 会部分初始化 GNOME Keyring，并在该过程中解锁 *login* 钥匙环。gnome-keyring-daemon 会通过 systemd 用户服务自动启动。

PAM 步骤

注意：要在**不使用自动登录**的情况下实现自动解锁，用户账户的密码应与 *login* 钥匙环的密码相同。参见 #用户密码变化时自动更改钥匙环密码。

提示：

- 要在自动登录的情况下实现自动解锁，可以把钥匙环的密码设为空。请注意，这种情况下钥匙环的内容将以未加密形式存储。
- 另一种做法是，如果使用 GDM 和 LUKS，当钥匙环密码与 LUKS 密码一致时，GDM 可以解锁你的钥匙环。为此，你需要在 mkinitcpio.conf 中使用 systemd 初始化，并设置相应的内核参数，同时应确保系统的实时时钟设为 UTC 而非本地时间（见 ）。更多细节见 。
- 如果你希望跳过 PAM 这一步，那就必须手动或以其他方式解锁钥匙环。参见#在桌面环境（KDE、GNOME、XFCE……）之外使用 gnome-keyring-daemon和 GnomeKeyring wiki。

使用显示管理器时，多数情况下钥匙环开箱即用。GDM、LightDM、LXDM 和 SDDM 已经带有必要的 PAM 配置。对于不会自动解锁钥匙环的显示管理器，请编辑对应的文件，而不是下文提到的 `/etc/pam.d/login`。

使用基于控制台的登录方式时（即不使用显示管理器），请编辑 `/etc/pam.d/login`：

在 `auth` 部分的末尾添加 `auth optional pam_gnome_keyring.so`，并在 `session` 部分的末尾添加 `session optional pam_gnome_keyring.so auto_start`。

```
/etc/pam.d/login
#%PAM-1.0

auth       required     pam_securetty.so
auth       requisite    pam_nologin.so
auth       include      system-local-login
auth       optional     pam_gnome_keyring.so
account    include      system-local-login
session    include      system-local-login
session    optional     pam_gnome_keyring.so auto_start
```

注意：如果使用 greetd 登录管理器，需要修改的文件是 `/etc/pam.d/greetd`，而不是 `/etc/pam.d/login`。

使用指纹登录时解锁

使用 fprintd 进行指纹认证时不会设置 `PAM_AUTHTOK`：读取器返回的是判定结果，而不是密码。因此 *pam_gnome_keyring.so* 没有可用于解锁 *login* 钥匙环的东西，第一个请求机密的程序便会提示输入密码 。任何不产生账户密码的认证方式都是如此。

可以考虑的做法：

- 在会话的首次提示中输入账户密码。此后该会话余下的时间里钥匙环都保持解锁。
- 按#PAM 步骤中所述，把 *login* 钥匙环的密码设为空。此时钥匙环内容将以未加密形式存储。
- 用一个在 `pam_gnome_keyring.so` 之前运行的 PAM 模块把 `PAM_AUTHTOK` 设为钥匙环密码。该密码存放在哪里、由什么保护，正是这类实现之间的区别所在。

SSH 密钥

GNOME Keyring 可以充当 ssh-agent 的包装层。在这种模式下，每次需要解锁 SSH 密钥时，它都会显示一个图形化密码输入对话框。该对话框带有一个记住所输入密码

的复选框，若勾选，那么只要你的登录钥匙环处于解锁状态，今后就能完全无密码地使用该密钥。

自 1:46 版起，gnome-keyring-daemon 的构建中默认禁用 SSH 功能。该功能已被迁移到 `/usr/lib/gcr-ssh-agent`，它是 gcr-4包 软件包的一部分。

配置 gcr

启用并启动 `gcr-ssh-agent.socket` 这个 systemd 用户单元。

该单元启动后，会创建套接字文件 `$XDG_RUNTIME_DIR/gcr/ssh`。

如果 `gcr-ssh-agent.socket` 单元处于活动状态，则无需手动配置 `SSH_AUTH_SOCK` 环境变量。当用户注销并重新登录时，`SSH_AUTH_SOCK` 环境变量的值会被设为 `$XDG_RUNTIME_DIR/gcr/ssh`。

使用

你可以运行

```
$ ssh-add -L
```

来列出正在运行的 agent 中已加载的 SSH 密钥。这有助于确认你启动了正确的服务，并正确设置了 `SSH_AUTH_SOCK`。

要把口令永久保存到钥匙环中，请使用 seahorse包 软件包提供的 ssh-askpass：

```
$ /usr/lib/seahorse/ssh-askpass my_key
```

提示：在某些情况下（例如你既没有使用登录管理器，也没有使用桌面环境），另一种做法是使用 Seahorse 的 *secret-tool* 来存储口令。

要从其他目录手动添加 SSH 密钥：

```
$ ssh-add ~/.private/id_ed25519
Enter passphrase for ~/.private/id_ed25519:
```

注意：你必须把对应的 *.pub* 文件放在与私钥相同的目录中（示例中为 `~/.ssh/id_ed25519.pub`）。另外，请确保公钥文件名为私钥文件名加上 *.pub*（例如 `my_key.pub`）。

要禁用所有手动添加的密钥：

```
$ ssh-add -D
```

禁用

如果你希望改用其他 SSH agent（例如直接使用 ssh-agent，或使用 gpg-agent），最好禁用 GNOME Keyring 的 ssh-agent 包装层。这样做并非严格必需，因为每个 agent 监听的套接字不同，可以用 `SSH_AUTH_SOCK` 在它们之间选择，但这能让排查问题更容易。请注意，GNOME 的实现不支持许多脚本特性，例如 BatchMode 。

要禁用 gcr-ssh-agent，请确保 `gcr-ssh-agent.socket` 和 `gcr-ssh-agent.service` 在 systemd 中都已禁用并停止。

提示与技巧

与应用程序集成

- Chromium

锁定钥匙环

```
$ dbus-send --session --dest=org.freedesktop.secrets \
   --type=method_call  \
   /org/freedesktop/secrets \
   org.freedesktop.Secret.Service.Lock \
   array:objpath:/org/freedesktop/secrets/collection/login
```

该命令执行一次 D-Bus 方法调用，以锁定 *login* 钥匙环。或者，如果你想使用图形界面，也可以用 Seahorse 锁定钥匙环。

清空口令

```
$ gnome-keyring-daemon -r -d
```

该命令会启动 gnome-keyring-daemon，并关闭此前已经在运行的实例。

Git 集成

当你通过 HTTPS 推送时，GNOME keyring 与 Git 配合使用会很有用。libsecret包 软件包需要安装，该功能才可用。

配置 Git 使用 *libsecret* 助手：

```
$ git config --global credential.helper /usr/lib/git-core/git-credential-libsecret
```

下次运行 `git push` 时，如果你的钥匙环尚未解锁，系统会要求你解锁它。

GnuPG 集成

有几个使用 GnuPG 的应用需要设置 `pinentry-program`。设置以下内容即可使用 GNOME 3 pinentry，让 GNOME Keyring 来管理口令提示。

```
~/.gnupg/gpg-agent.conf
pinentry-program /usr/bin/pinentry-gnome3
```

另一种做法是强制 GPG 使用 loopback，这样应该就能在应用内输入口令。

重命名钥匙环

钥匙环的显示名称（即 Seahorse 和 `file` 命令中显示的名称）可以通过\[更改未加密钥匙环文件中 display-name 的值来修改\]。钥匙环通常存放在 `~/.local/share/keyrings/` 中，扩展名为 *.keyring*。

用户密码变化时自动更改钥匙环密码

注意：这只影响 *login* 钥匙环。

向 `/etc/pam.d/passwd` 中追加 `password optional pam_gnome_keyring.so`：

```
/etc/pam.d/passwd
...
password      optional      pam_gnome_keyring.so
```

在桌面环境（KDE、GNOME、XFCE……）之外使用 gnome-keyring-daemon 启动

本文或本章节的事实准确性存在争议。

原因： 至少 xinit 和 SDDM 都会执行 `/etc/X11/xinit/xinitrc.d/` 中的所有脚本，而 sway 提供了 `/etc/sway/config.d/50-systemd-user.conf`，所以问题并不在于“位于桌面环境之外”。如果 gnome-keyring 需要 XDG Autostart，安装/配置章节里就应当说明这一点。（在 arch:Talk:GNOME/Keyring#Launching gnome-keyring-daemon outside desktop environments (KDE,_GNOME,_XFCE,_...) 中讨论）

如果你使用 sway、i3，或任何不会执行

- `/etc/xdg/autostart/gnome-keyring-*.desktop`
- `/etc/X11/xinit/xinitrc.d/50-systemd-user.sh`

的窗口管理器，那么你的窗口管理器需要在启动时执行以下命令。这些命令不必按特定顺序执行。

```
dbus-update-activation-environment DISPLAY XAUTHORITY WAYLAND_DISPLAY
```

或者

```
dbus-update-activation-environment --all
```

该命令会把环境变量从窗口管理器传递给会话 dbus。若不执行，就无法通过 DBus 触发图形化的提示框。例如，Seahorse 的密码提示就依赖于此。

之所以需要这样做，是因为会话 dbus 在图形环境启动之前就已经启动，因此会话 dbus 并不知道你处在哪个图形环境中。必须有某个东西把描述图形环境的环境变量传给会话 dbus，让它了解当前的图形环境。

```
gnome-keyring-daemon --start --components=secrets
```

登录期间，PAM 会启动 `gnome-keyring-daemon --login`，它负责用登录密码让 gnome-keyring 保持解锁状态。如果 `gnome-keyring-daemon --login` 在几分钟内没有连接到会话 dbus，`gnome-keyring-daemon --login` 就会退出。如果在窗口管理器中针对会话 dbus 启动了 `gnome-keyring-daemon --start ...`，那么 `gnome-keyring-daemon --login` 就会连接到会话 dbus。如果你的登录会话没有在 `gnome-keyring-daemon --login` 退出前启动 `gnome-keyring-daemon --start ...`，你也可以在 `gnome-keyring-daemon --login` 退出之前，先运行任何使用 gnome-keyring 或 secret service API 的程序。

GNOME Keyring XDG 门户

GNOME Keyring 提供一个 XDG 门户后端（例如可供通过 Flatpak 沙箱化的应用使用）。要使其在 GNOME 之外工作，必须配置为使用它。有几种做法。

配置用户级的 Secrets 门户

避免修改系统文件的方法是，在 `$XDG_CONFIG_HOME/xdg-desktop-portal/$XDG_CURRENT_DESKTOP-portals.conf` 中把 `gnome-keyring` 配置为首选的 `Secrets` 门户。例如，要把 sway 配置为使用 `gnome-keyring` 作为 `org.freedesktop.impl.portal.Secret` 的首选门户：

```
~/.config/xdg-desktop-portal/sway-portals.conf
[preferred]
default=wlr;gtk
org.freedesktop.impl.portal.Secret=gnome-keyring
```

修改 gnome-keyring.portal

**本文或本章节的事实准确性存在争议。**

原因： 会修改 `/usr/share` 中的软件包文件，pacman 升级时会被覆盖（在 Talk:GNOME/Keyring 中讨论）

另一种方法是修改 `UseIn` 键，把你的桌面环境加入 `/usr/share/xdg-desktop-portal/portals/gnome-keyring.portal` 配置文件。例如，要加入 sway：

```
/usr/share/xdg-desktop-portal/portals/gnome-keyring.portal
[portal]
DBusName=org.freedesktop.secrets
Interfaces=org.freedesktop.impl.portal.Secret
UseIn=gnome;sway
```

关于 XDG 桌面门户后端的更多信息，见 XDG 桌面门户#后端。

故障排除

密码没有被记住

如果你在登录后被要求输入密码，并且发现密码没有被保存，那么你可能需要创建/设置一个默认钥匙环。要使用 Seahorse（也称“密码和密钥”）完成此操作，见 GNOME 帮助中的 创建新钥匙环 和 更改默认钥匙环。

重置钥匙环

如果你收到以下错误信息：“The password you use to login to your computer no longer matches that of your login keyring”，你需要更改登录钥匙环密码。

或者，你可以从 `~/.local/share/keyrings/` 中删除 `login.keyring` 和 `user.keystore` 文件。请注意，这会永久删除所有已保存的密钥。删除这些文件后，只需注销再重新登录即可。

Unable to locate daemon control file

登录后，日志中可能会出现以下错误：

```
gkr-pam: unable to locate daemon control file
```

如果没有其他相关问题，这条消息“可以安全地忽略”。

No such secret collection at path: /

如果你尝试用 Seahorse 添加新钥匙环，可能会因以下原因收到此错误：

- `~/.local/share/keyrings/` 目录不存在。若缺失请创建它。
- 使用了自定义的 `~/.xinitrc`。添加以下一行可能可以解决

```
~/.xinitrc
source /etc/X11/xinit/xinitrc.d/50-systemd-user.sh
```

终端提示 “discover_other_daemon: 1”

这是因为 *gnome-keyring-daemon* 被第二次启动了。既然该守护进程已经随附了 systemd 服务，就无需再以其他方式启动它。所以请确保从 `.zshenv`、`.bash_profile`、`.xinitrc`、`config.fish` 或类似文件中删除启动命令。或者，你也可以禁用 `gnome-keyring-daemon.service` 和 `gnome-keyring-daemon.socket` 这些用户单元。

钥匙环初始化不正确

这有几种症状：

- SSH 或 Git 之类的程序在等待钥匙环提供密码时挂起，最终超时并报出 “agent refused operation” 之类的错误。
- Seahorse 不显示任何钥匙环，而且手动创建一个名为 “login” 的钥匙环似乎不起作用。
- Seahorse 的输出中出现以下错误信息：

```
    couldn't load all secret collections: No such secret collection at
path: /org/freedesktop/secrets/collection/login
```

要解决此问题，请执行以下操作：

1. 重启 `gnome-keyring-daemon.service` 这个 systemd 用户单元。这应当能正确初始化 *login* 钥匙环。
2. 如果使用的话，重启 `gcr-ssh-agent.service` 这个 systemd 用户单元。其他 agent 可能也需要重启。

如果这样仍无法解决问题，可以考虑重置钥匙环。

SSH agent 不可用

症状：

```
$ ssh-add -L
Error connecting to agent: Connection refused
```

请确保 `gcr-ssh-agent.socket` 用户单元已被启用并启动，并且 `gnome-keyring-daemon` 的套接字和服务用户单元也已启用并启动：

提示：你可以对 `{service,socket}` 使用 shell 的花括号展开。

No Process Capabilities, Insecure Memory Might Get Used

如果看到以下错误 `gnome-keyring-daemon: no process capabilities, insecure memory might get used`，这意味着钥匙环可能会被保存到交换文件/分区中。这是一个安全问题，因为他人可能在没有适当授权的情况下读取并提取这些机密。解决方法是让内核不要这样做，但 `gnome-keyring-daemon` 需要具备相应的权限才行。

运行以下命令，赋予 `gnome-keyring-daemon` 适当的内存锁定权限：

```
# setcap cap_ipc_lock=+ep /usr/bin/gnome-keyring-daemon
```

参见

- https://help.gnome.org/users/seahorse/stable/
- GNOME Wiki 页面（已归档）
- 提供 org.freedesktop.secrets 的软件包

4.6 GNOME/提示与技巧

难度 高级 | **篇幅** 23.8 KB | **所属** GNOME

本节将讲到

- 键盘
- 磁盘
- 隐藏菜单中的应用程序
- 屏幕录像
- 截屏

键盘

登录时启用NumLock

请参阅启动时打开数字锁定键#GNOME

热键替代

许多热键可以通过 GNOME 设置进行更改。例如，要重新启用显示桌面快捷键：

设置 > 键盘 > 自定义快捷键 > 导航 > 隐藏所有正常窗口

然而，某些热键无法通过设置直接更改。为了更改这些热键，请使用 *dconf-editor* 或 *gsettings*。一个特别值得注意的例子是快捷键 `Alt+`（在美国键盘布局中是Tab键上方的键）。在 GNOME Shell 中，它预配置为在应用程序的窗口之间循环切换，但它也是 [Emacs](../../07_开发与工具/Emacs/Emacs.md) 编辑器中常用的快捷键。可以通过使用上述工具修改 org.gnome.desktop.wm.keybindings 中的 *switch-group* 键来进行更改。

XkbOptions 键盘选项

使用 **dconf-editor**, 导航至 xkb-options 键，位于 org.gnome.desktop.input-sources 模式下，并将所需的 XkbOptions（例如 *caps:swapescape*）添加到列表中。

有关所有 XkbOptions，请参阅 /usr/share/X11/xkb/rules/xorg，以及 /usr/share/X11/xkb/symbols/* 的相关描述。

取消绑定 Super 键

默认情况下，Super 键将打开 GNOME Shell 概览模式。你可以通过运行以下命令来取消绑定此键：

```
$ gsettings set org.gnome.mutter overlay-key ''
```

修改 Nautilus 热键

自 3.15 版本起，不再使用 **accel** 文件，但可以通过使用 python-nautilus包 来重新绑定热键。安装该软件包并添加以下文件：

```
~/.local/share/nautilus-python/extensions/modify_keybindings.py
import os, gi
gi.require_version('Nautilus', '3.0')
from gi.repository import GObject, Nautilus, Gtk, Gio, GLib

def rebind():
    app = Gtk.Application.get_default()
    # 在以下链接中查找 open_accels 和 nautilus_application_set_accelerators:
    #   https://github.com/GNOME/nautilus/blob/master/src/nautilus-files-view.c
    app.set_accels_for_action( "win.back", ["<alt>Left", "BackSpace"] )

    # 如果你想弄清楚哪个热键属于哪个操作，可以试试这个:
    # print(f'Alt+Left is: {app.get_actions_for_accel("<alt>Left")}')

class BackspaceBack(GObject.GObject, Nautilus.LocationWidgetProvider):
    def __init__(self):
        pass

    def get_widget(self, uri, window):
        rebind()
        return None
```

重启 Nautilus：

```
$ nautilus -q; nautilus
```

磁盘

GNOME 提供了一款磁盘工具，用于操作存储驱动器的设置。以下是一些功能：

- **启用写入缓存**是大多数硬盘提供的功能。数据会在选定的时间进行缓存和分配，以提高系统性能。你很可能已经默认启用了此功能（不是通过磁盘工具）。可以使用 `hdparm -W /dev/sdX` 检查。

: **设置 > 驱动器设置 > 写入缓存 > *启用* : > 警告：**此性能提升可能会导致在突然断电的情况下丢失数据，请在启用写入缓存之前考虑这一点。

- **用户会话默认设置**启用时，可以使用 `/etc/fstab` 中的自定义选项，禁用时则会自动添加 GPT 驱动器和分区的默认和推荐挂载选项。

: **分区设置 > 编辑挂载选项 > 自动挂载选项 > *启用* : > 警告：**此设置会删除相关的 fstab 条目。

隐藏菜单中的应用程序

提示：

- 桌面条目可以通过编辑 *.desktop* 文件本身来隐藏。请参阅桌面项#隐藏桌面项。
- MenulibreAUR 提供了一个无需 GNOME 依赖的菜单编辑器。

使用**主菜单**应用程序（由 alacarte包 软件包提供）隐藏你不想在菜单中显示的任何应用程序。

屏幕录像

GNOME 提供内建的屏幕录像功能，使用 `Ctrl+Shift+Alt+r` 快捷键组合。在顶部栏右侧的系统状态区旁会显示一个红色圆圈，表示正在录制。录制完成后，名为 `Screencast from %d%u-%c.webm` 的文件会保存在 视频 目录中。

注意：录制的文件名可能会根据你的系统语言进行翻译。

为了使用屏幕录像功能，需安装一些 gst-plugin 软件包。例如，screencast pipeline（屏幕录像管道）依赖于 vp8enc 和 webmmux 元素，来自 gst-plugins-good 包。如果尝试录制时出现“pipewiresrc”模块缺失的错误，安装 gst-plugin-pipewire包。

提示：使用 EasyScreenCast 来便捷地“配置桌面录制的各种设置”（包括自定义上述的 GStreamer 管道）。EasyScreenCast 也可以作为 gnome-shell-extension-easyscreencastAUR 安装。

默认情况下，屏幕录像的最大时长为 30 秒。可以通过以下命令更改：

```
$ gsettings set org.gnome.settings-daemon.plugins.media-keys max-screencast-length length_in_seconds
```

将 *length_in_seconds* 设置为 0 以取消时长限制（根据 max-screencast-length 的描述）。

截屏

默认情况下，gnome-screenshot包 将图像保存到上次保存的目录，你可以查询该目录：

```
$ gsettings get org.gnome.gnome-screenshot last-save-directory
```

你可以设置一个自动保存目录，例如，自动将截图保存到 *user* 的桌面目录：

```
$ gsettings set org.gnome.gnome-screenshot auto-save-directory
file:///home/user/Desktop
```

查看 gnome-screenshot(1) 手册页面了解更多选项。

注销延迟

要消除注销时的默认 60 秒延迟：

```
$ gsettings set org.gnome.SessionManager logout-prompt false
```

更改动画速度

GNOME shell 的动画速度可以通过“减速因子”来配置。大于 1.0 会减慢动画速度，介于 0.0 和 1.0 之间则加速动画。

设置减速因子

要暂时设置，打开 Looking Glass，按 **Alt-F2** 输入 lg，然后运行，例如，若要加速动画：

```
St.Settings.get().slow_down_factor = 0.4
```

或者使用 gnome-shell-extension-impatience-gitAUR

可以通过环境变量 GNOME_SHELL_SLOWDOWN_FACTOR 永久设置减速因子，例如：

```
/etc/environment
GNOME_SHELL_SLOWDOWN_FACTOR=0.4
```

禁用动画

可以通过图形界面禁用动画，在**设置 > 辅助功能 > 视觉 > 减少动画**选项中切换。

Retina (HiDPI) 显示支持

GNOME 在 3.10 版本中引入了 HiDPI 支持。如果显示器未通过 EDID 提供正确的屏幕尺寸，可能导致 UI 元素显示比例不正确。作为解决方法，你可以打开 *dconf-editor*，找到 scaling-factor 键，位于 org.gnome.desktop.interface 下。将其设置为 1 以获得标准比例。

更多信息请参阅 HiDPI。

密码和密钥（PGP 密钥）

你可以使用密码和密钥程序 seahorse包 创建 PGP 密钥，它是 GnuPG 的前端，并将其作为依赖项安装。未来可能会用到这个密钥（例如用于加密文件）。按照以下步骤创建密钥（此过程可能需要约 10 分钟）：

文件 > 新建 > PGP 密钥 > 姓名 > 邮箱 > 默认设置 > 密码短语。

终端

更改默认终端大小

可以在你的配置文件的首选项中调整新终端的默认大小。选择菜单中的*首选项*，然后在*配置文件*下选择你的配置文件，访问设置以更改初始终端大小。

新终端采用当前目录

新终端默认打开在 `$HOME` 目录。你可以通过将 `source /etc/profile.d/vte.sh` 添加到你的 shell 配置文件来配置终端以采用当前工作目录。

填充终端

要为终端添加填充（在窗口边缘和终端内容之间创建一个小的不可见边框），创建以下文件：

```
~/.config/gtk-3.0/gtk.css
vte-terminal,
terminal-window {
    padding: 10px 10px 10px 10px;
    -vte-terminal-inner-border: 10px 10px 10px 10px;
}
```

禁用闪烁光标

在 GNOME 3.8 及以上版本中禁用闪烁光标使用以下命令：

```
$ gsettings set org.gnome.desktop.interface cursor-blink false
```

只在终端中禁用闪烁光标使用：

```
$ gsettings set
org.gnome.Terminal.Legacy.Profile:/org/gnome/terminal/legacy/profiles:/:$(gsettings get
org.gnome.Terminal.ProfilesList default | tr -d \')/ cursor-blink-mode off
```

请注意，`gnome-settings-daemon` 必须正在运行，才能使这些和其他设置更改在 GNOME 应用程序中生效——参见 GNOME#配置。

禁用关闭终端时的确认窗口

当尝试关闭终端窗口并且已登录为 root 用户时，终端总是会显示确认窗口。要避免这种情况，请执行以下命令：

```
$ gsettings set org.gnome.Terminal.Legacy.Settings confirm-close false
```

颜色调色板

终端支持将其颜色调色板更改为你喜欢的样式。只需进入**首选项**，选择你的配置文件，然后编辑颜色调色板。

终端配色方案

安装 gogh-gitAUR 包，它提供了一套为 GNOME 终端制作的自定义配色方案。你可以在选择一个或多个配色方案后运行 `gogh` 并输入你选择的配色方案的编号。

警告：安装配色方案时，切勿一次安装所有配色方案，因为它们会很难删除。请选择单个配色方案进行安装。

安装后，进入终端的**首选项**，点击**颜色**标签，选择你安装的配色方案名称。你会看到名称旁边有一个小箭头，点击它并选择**设置为默认**。

从这里开始，你可以进一步配置。如果某些颜色你不喜欢，可以轻松更改。

要删除一个配色方案，如果该配色方案是你的默认方案，首先将另一个方案设置为默认。然后选择该配色方案的名称并点击**删除**。

鼠标中键

默认情况下，GNOME 3 禁用中键鼠标按钮模拟，无论 Xorg 设置（**Emulate3Buttons**）如何。要启用中键鼠标按钮模拟，请使用以下命令：

```
$ gsettings set org.gnome.settings-daemon.peripherals.mouse middle-button-enabled true
```

启用按钮和菜单图标

从 GTK 3.10 开始，GSettings 键“menus-have-icons”已被弃用。可以通过设置以下覆盖项来启用按钮和菜单中的图标：

```
$ gsettings set org.gnome.settings-daemon.plugins.xsettings overrides
"{'Gtk/ButtonImages': <1>, 'Gtk/MenuImages': <1>}"
```

使用自定义颜色和渐变作为桌面背景

要使用自定义颜色和渐变作为桌面背景，你首先需要设置一个透明的图片或者一个不存在的图片作为桌面背景。例如，以下命令将设置一个不存在的图片作为背景：

```
$ gsettings set org.gnome.desktop.background picture-uri none
```

此时，桌面背景应为纯色——默认的颜色设置为深蓝色。

若要使用其他纯色，只需更改主色设置：

```
$ gsettings set org.gnome.desktop.background primary-color <my color>
```
其中 <my color> 是一个十六进制值（例如 *ffffff* 表示白色）。

对于渐变色，你还需要更改次色设置 `org.gnome.desktop.background secondary-color` 并选择一种着色类型。例如，如果你想要一个水平渐变，执行以下命令：

```
$ gsettings set org.gnome.desktop.background color-shading-type horizontal
```
如果你使用透明的图片作为背景，可以通过以下命令设置其不透明度：

```
$ gsettings set org.gnome.desktop.background picture-opacity <value>
```
其中 value 是一个 1 到 100 之间的数字（100 表示最大不透明度）。

背景过渡

GNOME 可以在特定时间间隔内在不同的壁纸之间过渡。可以通过创建一个 XML 文件，指定要使用的图片和时间间隔来实现。有关如何创建此类文件的更多信息，请参见以下文章。

或者，有许多工具可以自动化此过程：

- **mkwlppr** — 该脚本创建 XML 文件，可以作为 GNOME 的动态壁纸，引用多个壁纸。

: https://pastebin.com/019G2rCy\[失效链接 2023-07-30 ⓘ\] || mkwlppr\[失效链接 2023-07-30 ⓘ\]

自定义 GNOME 会话

可以创建自定义的 GNOME 会话，使用 GNOME 会话管理器但启动不同的组件集（例如使用 Openbox 和 tint2 代替 GNOME Shell）。

自定义 GNOME 会话需要两个文件：一个会话文件，位于 `/usr/share/gnome-session/sessions/`，定义要启动的组件；一个桌面项，位于 `/usr/share/xsessions`，由显示管理器读取。以下是一个会话文件的示例：

```
/usr/share/gnome-session/sessions/gnome-openbox.session
[GNOME Session]
Name=GNOME Openbox
RequiredComponents=openbox;tint2;gnome-settings-daemon;
```
以及一个桌面文件的示例：

```
/usr/share/xsessions/gnome-openbox.desktop
[Desktop Entry]
Name=GNOME Openbox
Exec=gnome-session --session=gnome-openbox
```
注意：GNOME 会话会调用每个要启动组件的 *.desktop* 文件。如果你希望启动的组件没有提供 *.desktop* 文件，你必须在 `/usr/local/share/applications` 等目录中创建一个合适的桌面项。

将特定网址重定向到特定的网页浏览器

这是如何使用 Chromium 来处理某些类型的网址，同时将 Firefox 保持为所有其他任务的默认浏览器。

确保 pcre包 已安装，以使用 *pcregrep*。

设置自定义 *xdg-open*：

```
/usr/local/bin/xdg-open
#!/bin/bash
DOMAIN_LIST_FILE=~/'domains.txt'
OTHER_BROWSER='/usr/bin/chromium-browser'
BROWSER_OPTIONS='' # Optional, for command line options passed to browser
XDG_OPEN='/usr/bin/xdg-open'
DEFAULT_BROWSER='/usr/bin/firefox'

if echo "$1" | pcregrep -q '^https?://'; then
    matching=0
    while read domain; do
       if echo "$1" | pcregrep -q "^https?://${domain}"; then
           matching=1
           break
       fi
    done < "$DOMAIN_LIST_FILE"

    if [[ $matching -eq 1 ]]; then
       "$OTHER_BROWSER" $BROWSER_OPTIONS ${*}
       exit 0
    fi

    "$DEFAULT_BROWSER" ${*}
    exit 0
else
    "$XDG_OPEN" ${*}
fi
```

配置域名以重定向到 *Chromium*：

```
$HOME/domains.txt
stackexchange.com
stackoverflow.com
superuser.com
www.youtube.com
github.com
```

将 *xdg-open web* 设置为桌面应用程序：

```
$HOME/.local/share/applications/xdg-open-web.desktop
[Desktop Entry]
Version=1.0
Name=xdg-open web
GenericName=Web Browser
Exec=xdg-open %u
Terminal=false
Type=Application
MimeType=text/html;text/xml;application/xhtml+xml;application/vnd.mozilla.xul+xml;text/
mml;x-scheme-handler/http;x-scheme-handler/https;
StartupNotify=true
Categories=Network;WebBrowser;
Keywords=web;browser;internet;
Actions=new-window;new-private-window;
$ update-desktop-database $HOME/.local/share/applications/
```

在 GNOME 设置中将 *xdg-open web* 设置为默认 Web 应用程序：转到 **GNOME 设置 > 默认应用程序**并将 **Web** 设置为 *xdg-open web*。

从视频缩略图中移除电影孔/胶片条纹效果（Nautilus）

自 Gnome 3.12 起，Nautilus（文件管理器）在视频缩略图上叠加了电影孔/胶片条纹效果。要移除或覆盖此效果，可以使用环境变量 `G_RESOURCE_OVERLAYS` 引用已编译资源的路径（在此实例中为 `filmholes.png`），并指定相关叠加层的路径。此环境变量自 GLib 2.50 起可用，早于此版本将无效。

从 Nautilus 中提取 `filmholes.png`：

```
$ gresource extract /usr/bin/nautilus /org/gnome/nautilus/icons/filmholes.png > filmholes.png
```

使用您喜欢的编辑器编辑 `filmholes.png`，移除图像中的胶片效果，保留透明度和尺寸不变，然后覆盖提取的图像。

将提取的图像复制或移动到所需位置，例如 `/usr/share/icons/`，并编辑 `~/.profile`，添加以下导出，并根据需要将 `/usr/share/icons/` 更改为您放置文件的位置：

```
export G_RESOURCE_OVERLAYS=/org/gnome/nautilus/icons/filmholes.png=/usr/share/icons/filmholes.png
```

如果已经安装了 ffmpegthumbnailer包 作为其他文件管理器的依赖项，该文件管理器可能会生成缩略图，则应修改 `/usr/share/thumbnailers/ffmpegthumbnailer.thumbnailer` 中的 `Exec` 行，移除 `-f` 标志。

为确保没有已嵌入胶片效果的缩略图，删除缩略图缓存：

```
$ rm -r ~/.cache/thumbnails
```

注销并重新登录到您的会话，您应该不再在 Nautilus 中看到视频缩略图上的胶片孔/胶片条纹效果。

GNOME 软件与 pacman 集成

packagekit包 集成功能以前通过名为 *gnome-software-packagekit-plugin* 的软件包提供，但已被自愿禁用，并且被认为不再受支持。

增加音量至超过 100%

您可以通过运行以下命令来允许音量过度放大：

```
$ gsettings set org.gnome.desktop.sound allow-volume-above-100-percent true
```

或者，安装扩展 Volume Mixer。然后，使用鼠标在顶部面板的音量图标上滚动，以增加超过 100% 的音量。

或者，打开 GNOME 调整工具并切换**常规 > 过度放大**。

按更小步进调整音量

默认情况下，按键盘的音量键会将音量调整 6%。如果需要更小的步进，可以在按音量键时按住 `Shift`，此时音量将以 2% 的步进调整。

此外，从 GNOME 3.36 开始，现在可以通过 dconf 设置直接调整音量步进。例如，要将音量步进设置为 2%，请执行以下命令：

```
$ gsettings set org.gnome.settings-daemon.plugins.media-keys volume-step 2
```

在顶部面板图标旁显示音量百分比

安装扩展 Sound percentage，以在顶部面板的音量图标旁边显示当前输出音量级别。

启动时使用独立 GPU

安装 switcheroo-control包 或 switcheroo-control-gitAUR 并启动/启用 `switcheroo-control.service`。

窗口列表

如果您喜欢在底部显示任务列表，但不喜欢此扩展的默认黑色，可以首先复制其目录：

```
$ cp -r /usr/share/gnome-shell/extensions/window-list@gnome-shell-
extensions.gcampax.github.com/ ~/.local/share/gnome-shell/extensions/
```

然后按您的喜好编辑 CSS。例如，要使窗口列表透明，请将 `stylesheet.css` 编辑如下：

```
.bottom-panel {
  background: transparent;
}
...
.window-button > StWidget,
.window-picker-toggle > StWidget {
  color: #bbb;
  background-color: transparent;
  ...
```

导航

要模仿 Windows 切换窗口的行为，首先禁用默认设置，该设置将窗口切换限制为当前工作区内的窗口：

```
$ gsettings set org.gnome.shell.app-switcher current-workspace-only false
```

然后，将 Alt+Tab 和 Alt+Shift+Tab 绑定为在窗口之间切换，而不是在应用程序之间切换：

```
$ gsettings set org.gnome.desktop.wm.keybindings switch-windows "['<Alt>Tab']"
$ gsettings set org.gnome.desktop.wm.keybindings switch-windows-backward
"['<Shift><Alt>Tab']"
```

此外，您可以重新绑定应用程序之间的切换（此示例使用 Super 而不是默认的 Alt）：

```
$ gsettings set org.gnome.desktop.wm.keybindings switch-applications "['<Super>Tab']"
$ gsettings set org.gnome.desktop.wm.keybindings switch-applications-backward
"['<Shift><Super>Tab']"
```

图像查看器

安装 libheif包 以添加对 HEIC 图像文件格式的支持。图像查看器使用的 gdk-pixbuf2 库将 libheif 列为可选依赖项。

将 dconf（gsettings）数据库存储为文本文件

默认情况下，dconf 将其配置存储在位于 $XDG_CONFIG_HOME/dconf/user 的二进制数据库 blob 中。如果您的主目录存储在 NFS 中，您将 dotfiles 存储在版本控制中，或出于其他原因，可以通过配置 *dconf 配置文件*来覆盖此默认设置。有关创建和使用配置文件的详细信息，请参见 dconf(7) § PROFILES。

在更改系统范围的默认设置之前，先将每个用户现有的 dconf 数据库转储为名为 user.txt 的文本文件。似乎无法使用其他扩展名。假设 XDG_CONFIG_HOME 的默认值，可以使用以下命令完成此操作：

```
$ dconf dump / > ~/.config/dconf/user.txt
```

完成后，以 root 用户身份创建默认 dconf 配置文件。

```
/etc/dconf/profile/user
service-db:keyfile/user
```

注销并重新登录，确认更改 dconf 设置时会更改基于文本的 user.txt 文件，而不是删除二进制 user 文件之前的旧二进制数据库。

此设置应该只会带来一些最小的额外资源使用。Dconf 仍然在临时 XDG_RUNTIME_DIR 目录中使用二进制数据库，但必须在桌面启动时重新创建它。它还必须保持 user.txt 文件的最新状态，并监视文本文件的更改。

4.7 GDM

难度 高级 | **篇幅** 26.8 KB | **所属** GNOME

本节将讲到

- 安装

- 启动
- 配置
- 疑难解答
- 另见

来自GDM - GNOME显示管理器：“GNOME显示管理器（GDM）是一个管理图形显示服务并处理图形用户登录的程序。

显示管理器为X Window System和Wayland提供图形登录提示。

安装

可通过安装gdm包来安装GDM，或作为gnome包组的一部分安装。

启动

若要开机启动GDM，启用`gdm.service`。

自动启动应用

欲使程序于登录后自动启动，请参照 Autostarting#On desktop environment startup 中适用您的桌面环境的指示。

配置

注意：下面列出的大多数配置选项都可以使用 gdm-settingsAUR GUI 应用程序轻松设置。

登录页面背景图片

**本文或本章节的事实准确性存在争议。**

原因： 配置不是持久的，更改在 GNOME Shell 更新后会被撤销。建议：重写以使用“用户主题”GNOME 扩展。（在 Talk:GDM 中讨论）

注意：

- 自GNOME 3.16开始，GNOME Shell主题被存储为二进制文件（gresource）。
- 在 gnome-shell包 的后续升级中，这个变动会被重写。

首先，您需要将现有的 GNOME Shell 主题解压到您的主目录中的一个目录中。 您可以使用以下脚本执行此操作：

```
extractgst.sh
#!/bin/sh
gst=/usr/share/gnome-shell/gnome-shell-theme.gresource
workdir=${HOME}/shell-theme

for r in `gresource list $gst`; do
       r=${r#\/org\/gnome\/shell/}
       if [ ! -d $workdir/${r%/*} ]; then
         mkdir -p $workdir/${r%/*}
       fi
done
```

```
for r in `gresource list $gst`; do
        gresource extract $gst $r >$workdir/${r#\/org\/gnome\/shell/}
done
```

主题文件应当已提取到创建的目录中。现在将您想要的背景图像复制到此目录。

接下来，您需要在目录中创建一个文件，内容如下：

```
gnome-shell-theme.gresource.xml
<?xml version="1.0" encoding="UTF-8"?>
<gresources>
  <gresource prefix="/org/gnome/shell/theme">
    <file>calendar-today.svg</file>
    <file>calendar-today-light.svg</file>
    <file>checkbox.svg</file>
    <file>checkbox-focused.svg</file>
    <file>checkbox-off-focused-light.svg</file>
    <file>checkbox-off-focused.svg</file>
    <file>checkbox-off-light.svg</file>
    <file>checkbox-off.svg</file>
    <file>gnome-shell.css</file>
    <file>gnome-shell-high-contrast.css</file>
    <file>gnome-shell-start.svg</file>
    <file>pad-osd.css</file>
    <file>process-working.svg</file>
    <file>toggle-off.svg</file>
    <file>toggle-off-hc.svg</file>
    <file>toggle-off-light.svg</file>
    <file>toggle-on.svg</file>
    <file>toggle-on-hc.svg</file>
    <file>toggle-on-light.svg</file>
    <file>workspace-placeholder.svg</file>
    <file>filename</file>
  </gresource>
</gresources>
```

将 **filename** 替换为背景图像的文件名，或移除此行后用16位色彩值替代。

现在，打开 `gnome-shell.css` 文件并更改 `#lockDialogGroup` 定义如下：

```
#lockDialogGroup {
  background: url(filename);
  background-size: widthpx heightpx;
  background-repeat: no-repeat;
}
```

将 `background-size` 设置为GDM使用的分辨率, 这可能不一定是图像的分辨率。显示分辨率列表见Display resolution。同样，将 **filename** 替换为背景图像的文件名。

如果您仅想改变背景颜色，请调整 `#lockDialogGroup` 的定义，如下：

```
#lockDialogGroup {
  background-color: #color;
}
```

这里 **color** 是作为背景的十六进制编码的新颜色。

最后，使用以下命令编译主题：

```
$ glib-compile-resources gnome-shell-theme.gresource.xml
```

然后将生成的 `gnome-shell-theme.gresource` 文件复制到 `/usr/share/gnome-shell` 目录下。

然后重启 `gdm.service` (注意：重启而不是退出)，它就会使用您想要的背景图片了。

更多信息见此贴。一个自动执行以上步骤的 shell 脚本在 DimaZirix's github repository 上可用。

dconf配置

一些 GDM 设置存储在 dconf 数据库中。可以通过将 *keyfiles* 添加到 `/etc/dconf/db/gdm.d` 目录，然后以root身份运行 `dconf update` 重新编译GDM数据库来配置它们，或者通过登录系统上的GDM用户并使用"gsettings" 命令直接更改设置。需要一个 GDM 配置文件 - 这必须手动创建，因为它不再向上游发送，见下文：

```
/etc/dconf/profile/gdm
user-db:user
system-db:gdm
file-db:/usr/share/gdm/greeter-dconf-defaults
```

对于后一种方法，您可以用以下命令尝试登入 GDM 用户：

```
# machinectl shell gdm@ /bin/bash
```

登录页面的logo

创建以下密钥文件：

```
/etc/dconf/db/gdm.d/02-logo
[org/gnome/login-screen]
logo='/path/to/logo.png'
```

然后重新编译 GDM 数据库。

或者，作为 GDM 用户执行以下操作来更改 logo：

```
[gdm]$ dbus-launch gsettings set org.gnome.login-screen logo '/path/to/logo.png'
```

要禁用 logo，可以将值设置为空字符串：

```
[gdm]$ dbus-launch gsettings set org.gnome.login-screen logo ''
```

更改光标主题

GDM会忽略 GNOME 光标主题设置，也忽略根据 XDG_规范设置的光标主题。若要更改GDM使用的光标，请创建以下密钥文件：

```
/etc/dconf/db/gdm.d/10-cursor-settings
[org/gnome/desktop/interface]
cursor-theme='theme-name'
```

然后重新编译GDM database，或作为GDM用户执行以下操作：

```
[gdm]$ dbus-launch gsettings set org.gnome.desktop.interface cursor-theme 'theme-name'
```

更改图标主题

可以使用相同的方法来更改图标主题。 创建以下密钥文件：

```
/etc/dconf/db/gdm.d/11-icon-settings
```

```
[org/gnome/desktop/interface]
icon-theme='theme-name'
```

然后，重新编译 GDM 数据库。 或者，执行以下命令临时作为 GDM 用户更改图标主题：

```
[gdm]$ dbus-launch gsettings set org.gnome.desktop.interface icon-theme 'theme-name'
```

在登录页面显示大字体

单击屏幕右上角的辅助功能图标（一个白色圆圈，中间有一个人的轮廓），然后选中*大号文本*选项。

要设置特定的缩放比例，可以创建以下密钥文件（以1.25为例）：

```
/etc/dconf/db/gdm.d/03-scaling
[org/gnome/desktop/interface]
text-scaling-factor='1.25'
```

然后重新编译GDM database，或作为GDM用户执行以下操作：

```
[gdm]$ dbus-launch gsettings set org.gnome.desktop.interface text-scaling-factor '1.25'
```

关闭声音

此调整禁用在登录屏幕上（通过键盘）调整系统音量时听到的声音反馈。

创建以下密钥文件：

```
/etc/dconf/db/gdm.d/04-sound
[org/gnome/desktop/sound]
event-sounds=false
```

然后重新编译GDM database，或作为GDM用户执行以下操作：

```
[gdm]$ dbus-launch gsettings set org.gnome.desktop.sound event-sounds 'false'
```

更改电源按钮行为

注意：

- 对于电源键的设置 logind 设置 已被 GNOME Settings Daemon 覆盖。
- 自从 GDM 3.18起，电源键不能被设置为交互式 (*interactive*).
- 在某些情况下，这条设置会被忽略，而采用硬编码产生的的默认值。

警告：请注意， acpid 守护进程也会控制“电源键”和“休眠键”相关事件。同时运行两套系统可能导致意外的结果。

创建如下的 keyfile:

```
/etc/dconf/db/gdm.d/05-power
[org/gnome/settings-daemon/plugins/power]
power-button-action='action'
```

然后重新编译GDM database，或作为GDM用户并执行以下操作：

```
[gdm]$ dbus-launch gsettings set org.gnome.settings-daemon.plugins.power power-button-action 'action'
```

这里 *action* 可以是 `nothing`, `suspend` 或 `hibernate` 其中之一。

开启触摸板轻触以点击

GDM（和GNOME）中默认关闭轻触以点击，但可以使用dconf设置开启它。

注意：如果想要在X下这么做，必须先正确配置X服务器的访问权限，见 #配置X服务器的访问权限。

若要启用轻触以点击，创建以下keyfile：

```
/etc/dconf/db/gdm.d/06-tap-to-click
[org/gnome/desktop/peripherals/touchpad]
tap-to-click=true
```

然后重新编译GDM database，或作为GDM用户执行以下操作：

```
[gdm]$ dbus-launch gsettings set org.gnome.desktop.peripherals.touchpad tap-to-click
'true'
```

开启或关闭无障碍菜单

若要禁用或启用无障碍菜单，创建以下 keyfile：

```
/etc/dconf/db/gdm.d/07-accessibility
[org/gnome/desktop/interface]
toolkit-accessibility='boolean'
```

然后重新编译GDM database，或作为GDM用户执行以下操作：

```
[gdm]$ dbus-launch gsettings set org.gnome.desktop.interface toolkit-accessibility
'boolean'
```

当key的值为`false`时，无障碍菜单默认关闭；`true`则开启。

在 GDM 上启用夜间模式

欲启用夜间模式，运行

```
[gdm]$ dbus-launch gsettings set org.gnome.settings-daemon.plugins.color night-light-
enabled true
```

键盘布局

GDM 需要 `/etc/vconsole.conf` 中设置 `XKBLAYOUT` 参数；如果没有设置，它将默认使用标准的 `us`（qwerty）布局，即不会遵循 `KEYMAP` 中设置的值。

一种普遍适用的方法是使用 `localectl --no-convert set-x11-keymap [keymap]`：详情参见 Keyboard configuration in Xorg#Setting keyboard layout。

警告：使用 `localectl set-x11-keymap` 而不带 `--no-convert` 也会改变 `KEYMAP=` 的值，如果使用包含重音字符的密码短语加密整个系统，可能会导致系统无法启动。参见 systemd issue #34967。

更改语言

系统语言将应用于 GDM。如果系统有多个用户，则可以为 GDM 设置与系统语言不同的语言。在这种情况下，首先请确保安装了 gnome-control-center包。然后，启动 *gnome-control-center* 并选择“区域和语言”。在标题栏中，选中*登录屏幕*切换按钮。最后，单击*语言*并从列表中选择您的语言。系统将提示您输入 root 密码。请注意，除非系统上存在多个用户，否则标题栏中的“登录屏幕”按钮将不可见 。

提示：添加 2 种不同的输入语言后，注销。然后选择您的默认语言，GDM 将在第二个选项被删除后记住您的选择。

用户与登录

自动登录

警告：不要尝试对由 systemd-homed 管理的用户执行此操作。 这是目前 未解决 的漏洞，并且会使 GDM 崩溃。

将以下内容添加至`/etc/gdm/custom.conf`以开启自动登录（将*username*替换为您的用户名）：

```
/etc/gdm/custom.conf
# Enable automatic login for user
[daemon]
AutomaticLogin=username
AutomaticLoginEnable=True
```

提示：如果在添加这些行后 GDM 失败，请从 TTY 中将它们注释掉。

或延迟自动登录：

```
/etc/gdm/custom.conf
[daemon]
TimedLoginEnable=true
TimedLogin=username
TimedLoginDelay=1
```

您可以设置用于自动登录的会话（将 `gnome-xorg` 替换为所需的会话）：

```
/var/lib/AccountsService/users/username
XSession=gnome-xorg
```

免密登录

如果您想绕过 GDM 中的密码提示，只需在 `/etc/pam.d/gdm-password` 的第一行添加以下行：

```
auth sufficient pam_succeed_if.so user ingroup nopasswdlogin
```

然后，将 `nopasswdlogin` 组加入您的系统。详见用户组以获得组的描述和组管理相关命令 。

现在，把您的用户加入 `nopasswdlogin` 组，然后您只需点击您的用户名以登录。

警告：

- **不要**对**root**账户这么做。
- 您将无法在使用 GDM 登录时更改会话类型。 如果您想更改默认会话类型，您首先需要从 `nopasswdlogin` 组中删除您的用户。

禁用指纹登录

当使用指纹登录时，它不会解锁密钥环，因此您仍然会被提示输入密钥环密码。您可能希望禁用登录并保留指纹以解锁会话。为此，只需为 GDM 用户禁用指纹。

临时以 GDM 用户身份执行以下命令并更改此设置：

```
[gdm]$ dbus-launch gsettings set org.gnome.login-screen enable-fingerprint-
authentication false
```

多个会话的无密码关闭

GDM 使用 polkit 和 logind 来获得关机权限。 当多个用户登录时，您可以通过设置以关闭系统：

```
/etc/polkit-1/localauthority.conf.d/org.freedesktop.logind.policy
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE policyconfig PUBLIC
 "-//freedesktop//DTD PolicyKit Policy Configuration 1.0//EN"
 "http://www.freedesktop.org/software/polkit/policyconfig-1.dtd">

<!--
Policy definitions for logind
 -->

<policyconfig>

  <action id="org.freedesktop.login1.power-off-multiple-sessions">
    <description>Shutdown the system when multiple users are logged in</description>
    <message>System policy prevents shutting down the system when other users are
logged in</message>
    <defaults>
      <allow_inactive>yes</allow_inactive>
      <allow_active>yes</allow_active>
    </defaults>
  </action>

</policyconfig>
```

您可以找到所有可用的 logind 选项（比如，reboot-multiple-sessions，即重启多用户会话）在 org.freedesktop.login1(5) 中。

在GDM中开启root登录

我们并不建议您以 root 登录，但如果必要，请编辑 `/etc/pam.d/gdm-password` 并在行 `auth required pam_deny.so` 前添加以下行：

```
/etc/pam.d/gdm-password
auth            sufficient      pam_succeed_if.so uid eq 0 quiet
```

这个文件应当看起来如下：

```
/etc/pam.d/gdm-password
...
auth            sufficient      pam_succeed_if.so uid eq 0 quiet
auth            sufficient      pam_succeed_if.so uid >= 1000 quiet
auth            required        pam_deny.so
...
```

重启 GDM 后，您可以用 root 身份登录。

在登录列表中隐藏用户

gdm 用户列表的用户由 \[失效链接 2022-09-17 ⓘ\] 收集。 它会自动隐藏系统用户（UID < 1000）。 要从登录列表中隐藏普通用户，请创建或编辑一个以用户命名的文

件，并将其保存在 /var/lib/AccountsService/users/ 中，随之隐藏该文件。文件至少应包含：

```
/var/lib/AccountsService/users/username
[User]
SystemAccount=true
```

通过 RDP 的远程登录

注意：当前只能创建无显示器的远程会话。

注意：一次只能存在一个图形化会话。若另一个图形化会话存在，尝试创建新图形化会话会杀死它。

通过图形化接口

可在 Gnome 设置程序中找到“系统>远程桌面>远程登录”来图形化地配置远程登录。

通过终端接口

注意：要生效，Gnome 远程桌面同时要求 TLS 证书和凭证。除非预先设置过，否则两者都不存在，需要手动设置。

要显示当前状态和凭证，可采用以下命令：

```
# grdctl --system status --show-credentials
```

要设置凭证：

```
# grdctl --system rdp set-credentials rdp_login rdp_password
```

要生成新的 TLS 密钥和证书：

```
# winpr-makecert3 -rdp -path /etc/gnome-remote-desktop -n rdp-tls
```

若省略了 -n rdp-tls 部分，就会使用 *hostname* 作为名字。

要设置新的 TLS 密钥和证书：

```
# grdctl --system rdp set-tls-key /etc/gnome-remote-desktop/rdp-tls.key
# grdctl --system rdp set-tls-cert /etc/gnome-remote-desktop/rdp-tls.crt
```

最终，启用远程登录：

```
# grdctl --system rdp enable
```

设置默认显示器

一些桌面环境将显示设置保存在 ~/.config/monitors.xml。*xrandr* 命令基于该文件内容生成信息。在GDM中类似内容保存在/var/lib/gdm/.config/monitors.xml。

若有显示器设置（例如朝向，缩放，主屏幕等等）信息记录于~/.config/monitors.xml中，并且想要让这些设置用于GDM：

```
# cp ~/.config/monitors.xml /var/lib/gdm/.config/
```

要在每次启动时自动重新配置显示器设置，为 gdm.service 使用 Systemd#附加配置片段：

```
/etc/systemd/system/gdm.service.d/override.conf
```

```
[Service]
ExecStartPre=/bin/cp /home/user/.config/monitors.xml /var/lib/gdm/.config/monitors.xml
```

monitors.xml文件中屏幕旋转和缩放的部分内容如下：

```
<monitors version="2">
  <configuration>
    <logicalmonitor>
      ...
      <scale>2</scale>
      ...
      <transform>
        <rotation>right</rotation>
        <flipped>no</flipped>
      </transform>
      ...
    </logicalmonitor>
  </configuration>
</monitors>
```

必须登出才能使更改生效，因为GDM不遵守xorg.conf。

注意：

- 如果在Wayland下使用GDM，则必须使用在Wayland下创建的monitors.xml。详见GDM bug 224。也可以强制让GDM #使用Xorg后端，并使用在Xorg下创建的monitors.xml。
- 如果您使用 fractional scaling (分数缩放)，您需要为用户 gdm 启用它：

```
[gdm]$ dbus-launch gsettings set org.gnome.mutter experimental-features "['scale-monitor-framebuffer']"
```

配置X服务器的访问权限

您可以使用 xhost 命令来配置 X 服务器的访问权限。

例如，要授予 GDM 访问 X 服务器的权限，请使用以下命令：

```
# xhost +SI:localuser:gdm
```

疑难解答

Wayland 与 NVIDIA 专有驱动

要在 GDM 中使用带有 NVIDIA 驱动程序的 Wayland，您必须满足以下三个条件：

- 启用 DRM KMS；
- 配置 Wayland；
- 遵循 NVIDIA/Tips and tricks#Preserve video memory after suspend。

从 GDM 42 和 NVIDIA 驱动程序 510 开始，GDM 默认为 Wayland。 对于较旧的 NVIDIA 驱动程序（版本 470 和 510 之间），GDM 具有依赖于芯片组的 udev 规则使用 Xorg 而不是 Wayland。 要强制启用 Wayland，不遵循上述三个常规步骤，请通过创建以下符号链接来覆盖这些规则：

```
# ln -s /dev/null /etc/udev/rules.d/61-gdm.rules
```

注意：此命令之所以有效，是因为 `/etc/udev/rules.d/` 中的规则会覆盖 `/usr/lib/udev/rules.d/` 中的规则（参见 Udev#About udev rules）。 建议修改 `/etc/` 中的文件，而非 `/usr/` 中的文件，也是因为 `/etc/` 中的文件由 pacman 管理。

如果出现黑屏而不是 GDM，请尝试在计算机的 BIOS 设置中禁用集成显卡。

一些情况下，GNOME 无法启动，将控制权转回 GDM，最终导致登录页面重复出现。您可以尝试设置以下环境变量，参考BBS#2126478：

```
/etc/environment
MUTTER_DEBUG_KMS_THREAD_TYPE=user
```

注销失败

如果 GDM 在启动时正常启动，但在多次尝试注销后失败，请尝试将此行添加到 `/etc/gdm/custom.conf` 的守护程序部分：

```
GdmXserverTimeout=60
```

无 root 的 Xorg

见Xorg#没有 root 权限的 Xorg。

使用Xorg后端

默认使用 Wayland 后端，只有在 Wayland 后端无法启动时才使用 Xorg 后端。在以下情况下，您可能希望使用 Xorg 后端：

- GDM 崩溃

要默认使用 Xorg 后端，请在 `/etc/gdm/custom.conf` 中将以下行取消注释：

```
#WaylandEnable=false
```

没有完全卸载 GDM

删除 gdm包 后，systemd 可能会报告以下内容：

```
user 'gdm': directory '/var/lib/gdm' does not exist
```

要删除此警告，请以 root 身份登录并删除私有用户 `gdm` 然后删除组 `gdm`：

验证 `gdm` 是否已通过具有 root 权限的 `pwck` 和 `grpck` 成功删除。 解决这个问题，您可能需要仔细检查无主文件是否保留 *gdm*。

GDM自动挂起（GNOME 3.28）

GDM 使用单独的 dconf 数据库来控制电源管理。要应用用户的电源设置，请将它们复制到 GDM 的 dconf 数据库：

```
$ IFS=$'\n'; for x in $(sudo -u username gsettings list-recursively org.gnome.settings-daemon.plugins.power); do eval "sudo -u gdm dbus-launch gsettings set $x"; done; unset IFS
```

这里 `*username*` 是您的用户名。

欲仅禁用 AC 上的自动挂起，运行：

```
[gdm]$ dbus-launch gsettings set org.gnome.settings-daemon.plugins.power sleep-inactive-ac-type 'nothing'
```

（要同时禁用电池自动挂起，请使用 `battery` 而不是 `ac` 运行命令。）

重新启动 GDM 以激活您的更改。

GDM 忽略了 Wayland，默认使用 X.Org

Wayland 需要运行内核模式设置 (KMS) 才能工作，并且在某些机器上 GDM 进程比 KMS 更早启动，导致 GDM 无法看到 Wayland 并且只能使用 X.Org。 这可能会导致您的日志中显示如下消息：

```
gnome-shell: Failed to open gpu '/dev/dri/card0':
GDBus.Error:org.freedesktop.DBus.Error.AccessDenied: Operation not permitted
gnome-shell: Failed to create backend: No GPUs found
systemd: org.gnome.Shell@wayland.service: Failed with result 'protocol'.
systemd: Failed to start GNOME Shell on Wayland.
```

或者，同样的问题可能会导致 GDM 不出现或监视器仅显示 TTY 输出。

您可以通过尽早启动 KMS（缺失中文翻译）来解决这个问题。您可能还希望仅验证在 GDM 配置中启用了 Wayland,（见上文）。

此外，如果您使用 NVIDIA 驱动程序，Wayland 会话可能会被 udev 规则阻止（请参阅 GNOME#Wayland 会话下的注释）。 这可能会导致如下消息：

```
systemd: Condition check resulted in GNOME Shell on Wayland being skipped.
systemd: org.gnome.Shell@wayland.service: Skipped due to 'exec-condition'.
systemd: org.gnome.Shell@wayland.service: Control process exited, code=exited, status=2/INVALIDARGUMENT
```

请参阅 #Wayland与NVIDIA专有驱动作为解决方法。

当存在 NVIDIA (e)GPU 时，AMD 或 Intel GPU 上出现黑屏

起初，在没有 NVIDIA 设备的情况下，GDM 可以在 Wayland 上正常启动和工作。但一旦插入 NVIDIA eGPU（或由于其他原因加载 `nvidia` 模块），GDM 就会停止工作。该问题的一个典型表现是在注销和 GDM 重新启动时出现黑屏，并在 GDM 的日志中显示以下消息（通过以 root 身份运行 `journalctl -u gdm -b` 访问）：

```
Gdm: Child process -<some PID> was already dead.
```

解决方法同上所描述: 防止 `/usr/lib/gdm-disable-wayland` 在 `nvidia` 模块加载时运行。

请注意，一旦 `/usr/lib/gdm-disable-wayland` 运行，Wayland 上的 GDM 将不再工作。这是因为 `WaylandEnable=false` 已写入 `/run/gdm/custom.conf`，它会覆盖 `/etc/gdm/custom.conf`。要在不重新启动系统的情况下解决此问题，请删除 `/run/gdm/custom.conf`，然后重新启动 GDM。

无法启用 GDM

参见 systemd/FAQ#Failure to enable unit due to preexisting symlink。

另见

- GDM 参考手册

4.8 GNOME

难度 高级 | **篇幅** 30.6 KB | **所属** GNOME

本节将讲到

- 安装
- 运行 GNOME
- 导航
- 遗留名称
- 配置

GNOME（读音为/(g)noʊm/）是一个追求简单易用的桌面环境。它由 GNOME项目设计，并且完全由自由开源的软件组成。GNOME默认使用 Wayland 作为显示服务器，并同时提供多个对话可供选择：

- **GNOME**是在Wayland上运行GNOME Shell的默认选项。传统的X应用使用Xwayland运行。
- **GNOME Classic**通过特定的扩展和数值，提供了“传统的桌面体验”（类似GNOME2的界面）。因此，它是GNOME Shell的定制样式，而非真正的不同模式。

安装

有两个软件组可用：

- gnome包组 包含基本的桌面环境和一些集成良好的应用
- gnome-circle包组 包含多种格外应用，极大的拓展了Gnome生态。
- gnome-extra包组 包含部分开发工具，以及其他适合Gnome的应用与游戏。

GNOME 的基础桌面环境由 Mutter) 窗口管理器的插件 GNOME Shell 组成。可以用 gnome-shell包 单独安装它。

注意：*mutter* 是桌面的混成器。它利用硬件图形加速减少屏幕的混乱。GNOME 会话管理器会自动检测显卡驱动是否能够运行 GNOME Shell，如果不行则用 *llvmpipe* 软件渲染。

也可以使用不稳定版本，见官方软件仓库#gnome-unstable。

运行 GNOME

GNOME 可以使用显示管理器以图形方式启动，也可以从控制台手动启动（可能会缺少某些功能）。gnome包组的显示管理器是GDM。

注意：GDM提供了GNOME的锁屏功能（及其它）支持。如果没有使用GDM启动GNOME，则需要使用其它屏幕锁定器。见List of applications/Security#Screen lockers。

图形界面启动

如果安装了gnome包组并希望GNOME在下次启动时自动启动, 请启用 `gdm.service`，随后可以在显示管理器的会话菜单中选择想要使用的会话：*GNOME*、*GNOME Classic* （仅在gnome-shell-extensions包 已安装时显示）。

如果想立即启动 GNOME 以避免重启，可以从一个没有被图形占用的 tty 上启动 `gdm.service`。

手动启动

注意：仍然需要X服务器用于运行尚未移植到Wayland的应用，详见Wayland#XWayland。使用Qt等特定图形库的应用，可以通过强制设置环境变量使用Wayland。详见Wayland#GUI libraries。

会话类型

Gnome会话继承了systemd的会话类型。当会话开始时，systemd的会话类型在会话启动时由`XDG_SESSION_TYPE`环境变量决定，并在启动后只能由该会话上的控制器决定。详见 Github。

因此仅仅在登录后设置`XDG_SESSION_TYPE` 并不起作用。因此应创建一个systemd drop-in 文件给getty设置环境变量：

```
/etc/systemd/system/getty@tty1.service.d/wayland.conf
[Service]
Environment=XDG_SESSION_TYPE=wayland
```

重新加载后检查会话类型：

```
$ loginctl session-status
```

启动会话

在正确设置会话类型后，可以通过以下命令手动启动会话：

```
$ gnome-session
```

由于直接运行`gnome-shell --wayland`缺乏会话管理，所以并不推荐。

请注意，手动调用Gnome不需要`gdm`（因此也不需要附带的`gdm.service`），因此对于安装了Gnome最小安装的用户来说，可以根据个人喜好，选择一些包含在更广泛的`gnome`组中的软件包。

若要在 tty1 登录时启动，将以下内容添加到 `.bash_profile` 中：

```
gnome-session --no-reexec
```

Firefox和Qt应用不遵守`XDG_SESSION_TYPE`，所以最好给它们加上变量：

```
if -z $DISPLAY && $(tty) == /dev/tty1 && $XDG_SESSION_TYPE == wayland ; then
  MOZ_ENABLE_WAYLAND=1 QT_QPA_PLATFORM=wayland exec gnome-session --no-reexec
fi
```

Wayland中的GNOME应用

在使用 *GNOME* 会话时，GNOME 应用将使用 Wayland 运行。出于调试需要, https://docs.gtk.org/gtk3/running.html 和 https://docs.gtk.org/gtk4/running.html 列出的选项和环境变量。

导航

GNOME Shell cheat sheet 中解释了如何高效地使用 GNOME shell，它展示了 GNOME shell 的特色和快捷键，包括切换任务，使用键盘，窗口控制，面板，概览模式等等。以下是部分常用的快捷键：

- `Super+m`：显示消息列表
- `Super+a`：显示应用菜单
- `Alt+Tab`：切换当前使用的应用
- `` ` Alt+` `` （美式键盘Tab`上面的按键）：切换正在前台使用的应用的窗口
- `Alt+F2`，然后输入 `r` 或 `restart`：在图形界面出问题时重启界面（仅用于X/传统模式，不适用于Wayland模式）。

如需改变默认配置，使其更类似Windows，见 GNOME/提示与技巧#导航。

更多快捷键见键盘导航。

遗留名称

注意：一些GNOME应用在文档和对话框中的名称已经更改，但执行文件名称却没有。下面表格列出了一些这样的应用。

提示：在搜索栏中搜索应用的遗留名称将成功找到对应的应用，例如搜索 *nautilus* 会出现 *文件*。

当前	遗留
文件	Nautilus
Web	Epiphany
视频	Totem
主菜单	Alacarte
文档查看器	Evince
磁盘使用情况分析器	Baobab
图像查看器	EoG (Eye of GNOME)
密码和密钥	Seahorse
翻译编辑器	Gtranslator

配置

GNOME系统设置面板（*gnome-control-center*）和GNOME应用使用dconf配置系统存储设置。

您可以使用gsettings(1)命令行工具直接访问 dconf 数据库。这也可以让您修改用户界面不公开的设置。命令行工具 dconf(1) 可以直接修改底层数据库，跳过验证过程。

直到GNOME 3.24，设置由GNOME设置进程应用（位于`/usr/lib/gnome-settings-daemon/gnome-settings-daemon`），其也可以在GNOME会话之外运行。

然而GNOME 3.24用几个相互独立的设置插件（`/usr/lib/gnome-settings-daemon/gsd-*`，后来移动到`/usr/lib/gsd-*`）取代了 GNOME 设置进程。这些插件通过 `/etc/xdg/autostart` (匹配org.gnome.SettingsDaemon.*.desktop) 下的桌面文件进行控制。若要在 GNOME 会话之外运行这些插件，您需要复制或编辑相应的桌面条目到 `~/.config/autostart`。

配置通常是用户特定的，本文将不介绍如何为多个用户创建配置模板。

GNOME 系统设置

色彩

`colord` 守护进程会读取显示器的 EDID 信息并提取出合适的色彩配置内容。大多数情况下，色彩配置都是正确的，不需要额外设置；但是对于某些偏差情况或使用较旧的显示器时，可以把色彩配置文件放在 `~/.local/share/icc/` 下并被指向。

夜间模式

GNOME 内置了类似于 Redshift 的蓝光过滤功能。夜间模式可以在设置面板中启动及自定义启动时间。此外，夜间模式的色温可以使用以下dconf包设置进行调整，5000是一个示例值：

```
$ gsettings set org.gnome.settings-daemon.plugins.color night-light-temperature 5000
```

提示：若要在Wayland中调整白天的色温，请安装Night Light Slider扩展。

注意：在NVIDIA显卡上，夜间颜色只工作在版本545.29.02及之后的驱动上

日期与时间

如果系统已有配置好的网络时间协议 守护进程，它同样会对 GNOME 起作用。如果需要，同步设置可以在菜单内设为手动控制。

GNOME支持自动选择时区，可在系统设置里的*日期和时间*选项中开启。前提是已开启定位服务 (见设置中的*隐私*选项）。

注意：由于Mozilla定位服务的退役，自动选择时区可能不再工作。见 . 解决方案见

若要在顶栏内显示日期，请运行：

```
$ gsettings set org.gnome.desktop.interface clock-show-date true
```

另外，若要在顶栏的日历中显示周数，请运行：

```
$ gsettings set org.gnome.shell.calendar show-weekdate true
```

默认应用程序

首次安装 GNOME 时，您可能会发现某些格式由错误的应用处理，比如视频被*totem*打开而不是以前使用的 VLC。某些格式关联可以使用系统设置中的*默认应用程序*进行调整。

有关其它协议和方法，请参阅默认应用程序进行配置。

鼠标和触摸板

大多数触摸板设置可以使用系统设置中的*鼠标和触摸板*进行调整。

根据您的设备，其它配置可能可用，但不会显示在默认界面内，例如不同的触摸板点击方法：

```
$ gsettings range org.gnome.desktop.peripherals.touchpad click-method
enum
'default'
'none'
'areas'
'fingers'
```

手动设置:

```
$ gsettings set org.gnome.desktop.peripherals.touchpad click-method 'fingers'
```

或使用gnome-tweaks包设置。

注意：GNOME 不支持 synaptics 并默认使用 libinput。参考 这个缺陷报告。

使用鼠标更改窗口大小

默认情况下，您可以使用鼠标移动窗口，方法是按住 Super，单击并按住鼠标左键并拖动鼠标。

此外，您可以通过按住 Super、单击并按住鼠标右键并拖动鼠标来启用鼠标调整窗口大小：

```
$ gsettings set org.gnome.desktop.wm.preferences resize-with-right-button true
```

如果您不喜欢 Super 键，还可以将修饰键更改为其他键，例如 Alt 或 Ctrl：

```
$ gsettings set org.gnome.desktop.wm.preferences mouse-button-modifier "'<Alt>'"
```

网络

NetworkManager 是GNOME项目中控制网络设置的工具。如果尚未安装，则安装networkmanager包软件包并启用`NetworkManager.service`。

虽然可以使用任何其它网络管理器，但 NetworkManager 可以使用整合到桌面环境的网络设置和状态指示器（network-manager-applet包，不必要安装）。

注意：用networkmanager包的*nmtui*设置隐藏的无线网络不会自动连接。您需要在GNOME设置中创建一个新配置文件，以恢复该网络的自动连接功能。

在线帐户

部分在线账户，比如 ownCloud，需要安装 gvfs-goa包 和gvfs-dnssd包以在 GNOME 应用比如 GNOME 文件以及 GNOME 文档中发挥全部功能。

详见在线账户。

搜索

GNOME shell在按下`Super`键并开始输入时会启动搜索。localsearch包作为gnome包组中nautilus包的依赖被默认安装。它提供一个应用和数据的索引数据库。它可以被设置中“搜索”菜单项配置。它在用户登录时自动被gnome-session启动。

localsearch 并不自动递归用户的家目录中所有的目录,所以你可能需要在 搜索 > 搜索位置 中添加自定义的路径。创建一个名为 `.nomedia` 的空文件以将某个目录从索引中排除。

可通过 `localsearch status` 查看当前索引状态。搜索内容可以在命令行中被`localsearch search`查看，被`localsearch tag`编辑，以及被重置。见`localsearch help` 或在线寻求参考。

tinysparql-sql被用作索引数据库。如果需要，数据库可以被直接查询。

设备安全

GNOME 43 在设置中添加了一个新的设备安全面板。该功能依赖fwupd工作。见

高级设置

如上文所述，改变GTK主题或窗口管理器主题等选项，并不会出现在GNOME设置（gnome-control-center）里。想要修改这些设置可以使用GNOME Tweaks （gnome-tweaks包），这是一个展示了许多这类设置的图形化工具。

（存储在 DConf 数据库中的）GNOME设置也可以使用dconf-editor(1)（一个图形化的DConf配置工具）或gsettings命令行工具进行配置。GNOME Tweaks 只能用于改变GUI。注意，你不一定总能看到下文描述的所有设置。

拓展

拓展的目录位于 https://extensions.gnome.org。拓展可通过 archlinux官方仓库 (少数), AUR或者网页安装。

**本文或本章节的事实准确性存在争议。**

原因： 下方的“注意”推荐手动管理用户文件，因为更“简单”，但并没有解释为什么。（在 Talk:GNOME 中讨论）

注意：* 通过浏览器安装拓展仅对当前用户生效，并需要用户手动管理升级。这是比较简单的方法。

- - 此外，如果你想要从浏览器安装拓展，你需要安装 gnome-browser-connector包。如果从官方仓库或者AUR安装，则不需要。
- 从AUR（或者官方仓库）安装拓展是他们对整个系统生效（并且在使用 AUR 助手时自动更新）。

已安装的拓展可以通过 *gnome-extensions-app*在图形界面、在命令行使用gnome-extensions(1)或在浏览器中通过网页配置、启用或禁用。在浏览器中，如需安装和激活拓展，请点击右上角的ON，然后点击弹出窗口的Install（如果该拓展并没有被安装）。已经安装的拓展可在 https://extensions.gnome.org/local/查看并检查更新。

gnome-shell-extensions包 提供了一些GNOME官方维护的非常有用的拓展。

extension-manager包 是一个用于在操作系统或用户范围内安装、拆卸、启用、禁用拓展的图形工具。安装前请仔细确认已知问题列表。

启用一个拓展（默认禁用）：

```
$ gsettings set org.gnome.shell disable-user-extensions false
```

列出当前启用的拓展：

```
$ gsettings get org.gnome.shell enabled-extensions
```

上述命令可能会列出已删除的扩展。要仅列出已启用和安装的扩展，请改用 gnome-extensions：

```
$ gnome-extensions list --enabled
```

关于 GNOME shell拓展的更多信息，详见 https://extensions.gnome.org/about/.

外观

主题

注意：从Gnome 42开始，许多默认的Gnome应用程序使用GTK 4和libadwaita。这些应用程序目前不支持通过gsettings或gnome-tweaks包改变主题，只能通过系统设置的外观选项来调整配置。若要设置Adwaita或Adwaita-dark以外的GTK主题，见GTK#Themes。

GNOME默认使用Adwaita。若要只将Adwaita-dark应用于GTK 2应用，请使用以下符号链接：

```
$ ln -s /usr/share/themes/Adwaita-dark ~/.themes/Adwaita
```

注意：Adwaita-dark主题由gnome-themes-extra包提供，因此GNOME的最小安装可能不会包含此主题。

若要选择新的主题，（将它们移到适当的目录中，并）使用GNOME Tweaks或下面的GSettings命令。

对于GTK主题：

```
$ gsettings set org.gnome.desktop.interface gtk-theme theme-name
```

对于图标主题：

```
$ gsettings set org.gnome.desktop.interface icon-theme theme-name
```

注意：窗口管理器的主题会跟随GTK主题。使用`org.gnome.desktop.wm.preferences theme`的方法已被废弃和忽略。

见GTK#Themes和Icons#Manually.

标题栏按钮排序

设置 GNOME 窗口管理器顺序 (Mutter, Metacity):

```
$ gsettings set org.gnome.desktop.wm.preferences button-layout
':minimize,maximize,close'
```

提示：冒号表示窗口标题栏按钮出现的方向

GNOME Shell主题

GNOME Shell本身的主题是可配置的。首先确认您已安装gnome-shell-extensions包软件包以应用Shell主题。然后通过GNOME Tweaks或通过GNOME Shell Extensions网站启用“User Themes”扩展。Shel主题可以通过使用GNOME Tweaks软件加载并选用。

AUR里有大量可用的GNOME Shell主题。Shell主题也可在gnome-look.org里下载。

AppIndicators/顶部菜单栏图标

AppIndicators 对于监控和控制后台程序很有用，其对应的软件包是 gnome-shell-extension-appindicator包 or gnome-shell-extension-appindicator-gitAUR。安装后重启 GNOME Shell ，然后在 GNOME 扩展程序中启用 AppIndicator 扩展，或者运行 `$`

`gnome-extensions enable $(gnome-extensions list | grep -m 1 appindicatorsupport)`来以命令行方式启用它。

Shell动画速度

GNOME shell 动画可以被提速、降速或禁用。见GNOME/Tips and tricks#Change animation speed.

Shell 模糊效果

Blur my Shell 是一个给概览模式页面、Shell本身和其他部分应用添加模效果的拓展。安装 gnome-shell-extension-blur-my-shellAUR， 或者gnome-shell-extension-blur-my-shell-gitAUR 。该拓展具有高度可自定义性，并且你可以自由选择对哪些应用施加模糊效果。

更好的 Alt-Tab

GNOME中默认的 Alt-Tab功能非常简单，并不展示selected windows的概览。你可以在设置应用中将 Alt-Tab快捷键从 “切换应用” 改为 “切换窗口”以展示窗口概览。

你也可以使用Coverflow Alt-Tab。这是一个拓展Alt-Tab行为的拓展，添加了一些特性使在应用程序之间切换更简单、更好看。 安装gnome-shell-extension-coverflow-alt-tab-gitAUR。你可以根据喜好配置该拓展。

注意：` Super- ``默认提供“切换窗口”功能。

自启动

GNOME提供XDG Autostart。

gnome-tweaks包 程序可以用来管理自启动项。

提示：如果Tweaks中自启动应用选项下加号按钮为灰色不可用，尝试在终端下通过`gnome-tweaks`命令启动Tweaks。见此贴。

桌面

Dash to Dock

To move the dash out of the overview and turn it into a dock to以便于启动和切换应用, 安装 gnome-shell-extension-dash-to-dockAUR.

启动到概览模式

从GNOME 40开始，桌面会直接启动到概览模式，而不是桌面（如以前的版本）。若要模仿经典的行为，可以安装No overview at start-up插件。

如果你在使用gnome-shell-extension-dash-to-dockAUR，你也可以在gsettings中禁用该行为。

```
$ gsettings set org.gnome.shell.extensions.dash-to-dock disable-overview-on-startup true
```

详见此讨论。

剪贴板历史

不像其他桌面环境，GNOME 并没有内建的管理剪贴板历史的工具。你可以使用拓展。安装 gnome-shell-extension-clipboard-indicatorAUR。

天气

以展示选定地点的天气，比如状态、风速、气压等信息，请安装 gnome-shell-extension-openweatherAUR。天气信息实时更新。

音频输入输出选择

**这一章节正在考虑移除。**

原因: 该软件目前最高只支持到Gnome 43。 (在 Talk:GNOME 讨论)

默认情况下，如果你想要选择音频输入输出设备或改变麦克风的音量，你需要打开GNOME控制中心进行配置。你可以安装gnome-shell-extension-sound-output-device-chooserAUR 或者 gnome-shell-extension-sound-output-device-chooser-gitAUR以更方便地控制他们。你可以在安装后进行更多的配置。

字体

提示：如果您把"Scaling factor"调至1.00以上的某值，辅助功能菜单将自动启用

GNOME可以设置窗体标题，界面（应用），文档及等宽字体。查看Tweaks下的字体选项卡以获得相关选项。

对于字体渲染来说，RGBA可能适合更多的显示器类型，如果字体看起来过分拥挤，可以将字体渲染调至“Slight”或“None”。

输入法

GNOME集成了的通过IBus的输入法, 只有ibus包和添加想要的输入法引擎 (例如:ibus-libpinyin包 for Intelligent Pinyin) 需要安装，安装后，输入法引擎可以加入GNOME的区域和语言设置键盘布局。

非主流键盘布局

如果您使用的是像Neo2这样使用多层/修饰键的替代键盘布局，您可能需要转到 GNOME。设置 (*gnome-control-center*)中的 *Keyboard* > *Type Special Characters* 将 *Alternate Characters Key* 从 *右 Alt* 改为其他键，以便它可以作为键盘布局的本地修饰键使用。将其设置为 *Left Alt* 会阻止 *Alt+Tab* 键盘快捷键，因此请小心您所更改的内

容。 如果没有进行这个更改，您的左 *Mod3* 键可能会起作用，但右侧的一个 (*AltGr*) 键可能不会起作用。（截至2021-05-18）

电源

当您使用笔记本时，可能想修改以下设置，包括控制闲置、按下电源按钮和盖子关闭时的行为。

```
$ gsettings set org.gnome.settings-daemon.plugins.power button-power hibernate
$ gsettings set org.gnome.settings-daemon.plugins.power sleep-inactive-ac-timeout 3600
$ gsettings set org.gnome.settings-daemon.plugins.power sleep-inactive-ac-type
hibernate
$ gsettings set org.gnome.settings-daemon.plugins.power sleep-inactive-battery-timeout
1800
$ gsettings set org.gnome.settings-daemon.plugins.power sleep-inactive-battery-type
hibernate
$ gsettings set org.gnome.desktop.lockdown disable-lock-screen true
```

如需在合上盖子后依然保持显示器开启：

```
$ gsettings set org.gnome.settings-daemon.plugins.xrandr default-monitors-setup do-
nothing
```

GNOME 3.24中不建议使用以下设置：

```
org.gnome.settings-daemon.plugins.power button-hibernate
org.gnome.settings-daemon.plugins.power button-power
org.gnome.settings-daemon.plugins.power button-sleep
org.gnome.settings-daemon.plugins.power button-suspend
org.gnome.settings-daemon.plugins.power critical-battery-action
```

笔记本合盖时不挂起

The settings panel of GNOME does not provide an option for the user to change the action triggered when the laptop lid is closed. To change the lid switch action system-wide, edit the systemd settings in `/etc/systemd/logind.conf`. To turn off suspend on lid close, set `HandleLidSwitch=ignore`, as described in Power management#ACPI events.

修改电池电量严重不足时的行为

设置面板不提供对电池电量严重不足行为的设置。这些设置也从dconf中移除。不过它们现在由upower管理。按需编辑`/etc/UPower/Upower.conf`中upower设置。

```
/etc/UPower/UPower.conf
PercentageLow=10
PercentageCritical=3
PercentageAction=2
CriticalPowerAction=HybridSleep
```

电源模式

安装 gnome-control-center包 的可选依赖 power-profiles-daemon 以支持电源配置文件。 由于 *gnome-shell* 和 GNOME 设置都请求在启动时激活 `power-profiles-daemon` 服务，所以不需要特意将其启动/启用。

当服务激活时，可以通过 GNOME 设置的*电源*选项和系统菜单管理电源配置文件。

录屏

内建的截屏工具没有默认录屏选项。安装 gnome-shell包的可选依赖 gst-plugin-pipewire包 以启用录屏功能。

使用不同的窗口管理器

GNOME Shell 不支持更改窗口管理器, 但是 GNOME Flashback 提供使用 Metacity 和 Compiz 的session。此外，可以通过自定义 GNOME sessions 来使用别的组件。

将 GNOME Shell替换为不同的 Wayland compositor 将会导致gnome-control-center 包 （GNOME 设置）的部分设置显示错误。*gnome-control-center* 依然可以正常工作，但因为 mutter包 （GNOME Shell） 无法用于提供这些设置以填充这些区域，他们将不再起效或无法准确反映你的设置。这些设置包括蓝牙，显示以及鼠标/触控板等。

参见

- 官方网站
- Contributing to GNOME, feature requests, bugs, code
- GNOME-shell 扩展
- 主题、图标和壁纸：
- GNOME Art
- GNOME 外观
- GTK/GNOME 程序：
- GNOME 文件管理
- GNOME 项目列表
- 自定义 GNOME Shell
- GNOME 代码和镜像:
- GNOME GitLab
- GNOME Github 镜像

第 5 章　共 8 节　113 KB

Wayland 合成器

把「装个桌面」落到具体的合成器，是这一章要解决的事：安装、启动、配置、排错，八节各有可照做的路径与边界。

读法不必平均。Sway 与 Hyprland 必读，篇幅最长、生态最全，任选其一跟做即可走通主路径，另一节作对照；想看合成器机制，翻 Weston。Wayfire、MangoWM 篇幅短，浏览即可；Hyprlock 只是 Hyprland 的锁屏工具，不是合成器；Niri 与 Dwl 按场景取阅，后者配置思路不同，不宜照搬。最常踩的坑是把一家的配置套给另一家：语法与生态互不通用，先跑通一个再动第二个。

5.1　Wayfire

难度 入门 | **篇幅** 2.1 KB | **所属** Wayland 合成器

本节将讲到

- 安装
- Starting
- 配置
- 参见

Wayfire 是一个 3D wayland 混成器，灵感来自于 Compiz，基于 wlroots 开发。它致力于在不牺牲其外观的前提下创造一个可自定义、可扩展、轻量级的环境。

安装

Wayfire 可以由 wayfire包 安装。安装 wf-shell包 以添加任务栏和墙纸，安装 wcm包 以添加 GUI 配置程序。

为了获得更好的使用体验，下列包也被推荐使用：

- mako包：通知
- kanshi包：多显示器支持
- swayidle包 和 swaylock包：屏幕锁定

Starting

Wayfire 可以由一个显示管理器或以下代码启动：

```
$ wayfire
```

要使用特定的控制文件：

```
$ wayfire -c /path/to/config.ini
```

配置

下载默认配置文件，并保存为 `~/.config/wayfire.ini` 以开始配置。如果你安装了 wcm包，那么一切都可以在 GUI 下配置，否则你将不得不用文本编辑器手动编辑配置文件。

参见

- Wayfire 官方 wiki

5.2 Hyprlock

难度 入门 | **篇幅** 2.2 KB | **所属** Wayland 合成器

本节将讲到

- 安装
- 配置

Hyprlock 是 Hyprland 的锁屏工具，可高度自定义。

安装

安装 hyprlock包

配置

配置可使用在`~/.config/hypr`的 hyprlock.conf 文件。

注意：Hyprlock不会在安装过程中自动生成一个配置文件，如果缺少配置文件，它将不会运行。

组件

Hyprlock 工作在一个基于组件的系统。你可以通过如下格式创建组件：

```
~/.config/hypr/hyprlock.conf
widget {
  property = value
  ...
}
```

注意：多数配置可以放置在hyprlock.conf文件中，如：

```
~/.config/hypr/hyprlock.conf
label {
...
}
label {
...
}
input-field {
...
}
```

示例

设置 Hyprlock 背景：

~/.config/hypr/hyprlock.conf
```
background {
  monitor = # monitor-agnostic
  path = ~/Pictures/mountain.jpg
  blur_passes = 1
  blur_size = 7
}
```
密码输入框：

~/.config/hypr/hyprlock.conf
```
input-field {
  monitor =
  size = 20%, 5%
  font_family = Noto Sans
  dots_size = 0.08
}
```
显示时间：

~/.config/hypr/hyprlock.conf
```
label {
  monitor =
  text = $TIME
  font_size = 100
  position = 0, -80
}
```
在text后的$TIME可被替换为任意文本，$USER，$FAIL和$TIME12属于其他。

5.3 MangoWM

难度 入门 | **篇幅** 2.8 KB | **所属** Wayland 合成器

本节将讲到

- 安装
- 启动
- 配置
- 故障排除
- 参见

Mango 是一个现代、轻量且高性能的平铺式 Wayland 合成器，基于 dwl 构建 — 专为速度、灵活性以及可定制的桌面体验而设计。

安装

安装 mangowm包 软件包。

启动

在显示管理器的菜单中选择 *Mango* 或者在 tty 中运行 mango。

LXQT

参见 LXQt#Wayland 会话

配置

系统级配置文件是 `/etc/mango/config.conf`，用户级配置文件是 `~/.config/mango/config.conf`。

创建配置文件

首先，运行下面命令来创建接下来需要的目录：

```
$ mkdir -p ~/.config/mango/
```

每次启动时，mango 都会先尝试使用 `~/.config/mango/config.conf` 中包含的所有自定义内容，然后才回退到 `/etc/mango/config.conf`。用户级配置文件不会被默认创建，所以要通过下面命令复制模板文件：

```
$ cp /etc/mango/config.conf ~/.config/mango/
```

自动启动

可以在配置文件中添加 `exec_once` 或 `exec` 命令来在启动时运行命令或脚本，例如：

```
exec_once=waybar

exec=bash ~/.config/mango/reload-settings.sh
```

`exec_once` 命令只在 mango 启动时会运行一次，而 `exec` 命令在每一次重加载配置时均会运行。

屏幕共享

安装 xdg-desktop-portal-wlr包 软件包以启动 PipeWire 的屏幕共享 若门户未自动启动，尝试手动执行 `/usr/lib/xdg-desktop-portal-wlr`。

故障排除

修复运行异常的 Java 应用程序

参见 Java#灰色窗口/应用不随窗口管理器调整大小/菜单自动关闭。

参见

- bemenu - dmenu 的一个原生替代方案
- 一个 *mango* 的 介绍
- *Mango* 的 git 仓库

5.4 Niri

难度 进阶 | **篇幅** 7.4 KB | **所属** Wayland 合成器

本节将讲到

- 安装
- 启动
- 配置
- 另见

Niri 是一个可滚动的平铺式 Wayland 合成器。与 Sway 或 Hyprland 不同的地方在于，Niri 将窗口排列在一个无限延伸的水平桌面上，你可以向左或向右滚动（当然也可以实现更复杂的布局）。它类似于 GNOME 的 PaperWM 和 KDE 的 Karousel。

安装

可以通过安装 niri包 来获取 Niri。

此外，为了获得更好的使用体验，你可能还需要安装以下软件：

- **fuzzel**：Niri 的默认应用启动器
- **mako**：通知管理器
- **waybar**：Wayland 状态栏
- **xdg-desktop-portal-gtk**、**xdg-desktop-portal-gnome**：用于实现屏幕共享功能
- **alacritty**：Niri 的默认终端
- **swaybg**：设置桌面背景图片
- **swayidle**、**swaylock**：用于在空闲时锁定屏幕
- **xwayland-satellite**：用于运行 X11 应用程序
- **udiskie**：用于管理和自动挂载 USB 驱动器

启动

Niri 附带了一个可以被显示管理器执行的 桌面项。在显示管理器中选择它将会启动 `niri-session`。`niri-session` 会把 环境变量导入至systemd.

你也可以从 getty 执行:

```
niri-session
```

这可以结合Getty#自动登录到虚拟控制台来实现一个无缝的启动体验。

配置

Niri 从 `~/.config/niri/config.kdl` 读取配置文件。它是一个有多个部分的 KDL 文件。首次启动创建的配置文件通过注释给出了默认的选项设置。然而，随着更新所引入的新选项并不会在用户的配置中所体现，你可能需要查看 Niri 官方文档。

Niri 在配置文件保存时会自动应用更改。无效的配置在自动应用更改时并不会使 Niri 崩溃，而是会保留上一次可工作的状态，并提醒用户配置文件无效。 可以执行 `niri validate` 来在 Niri 会话外检查配置是否有效。

键盘布局

要配置键盘布局，编辑 `input/keyboard/xkb` 部分.

例如，假设你想要有一个 `CapsLock` 当作 `Ctrl` 的 "US Int Alt Gr" 布局：

```
~/.config/niri/config.kdl
input {
    keyboard {
        xkb {
            layout "us"
            variant "altgr-intl"
            options "ctrl:nocaps"
        }
    }
    ...
}
```

显示输出

首先执行 `niri msg outputs` 以获取所有 Niri 识别到的输出。然后你可以给每个显示器应用配置。例如设置 HDMI 显示器为 2560x1440 60Hz 并有 1.2 倍缩放，且关闭笔记本显示器，可以这样设置：

```
~/.config/niri/config.kdl
output "HDMI-A-1" {
    mode "2560x1440@60.000"
    scale 1.2
}

output "eDP-1" {
    off
}
```

使用 kanshi 来动态设置布局

你也可以使用 kanshi 来动态设置布局，例如当你想要在连接至外部显示器时关闭内置显示器。

按键绑定

这一部分允许配置各种 Niri 可用的快捷键。很多已经在初次启动时生成的默认配置中配置好了。它们都可以重新改为其他按键。

请注意 Niri 不 加载任何默认的绑定。若一个绑定没有在配置文件中指定，那么它就不会生效。“默认”只是在自动生成的配置文件中提供的。因此，在移除它们时请小心。建议使用注释来注释掉不需要的绑定。

按键绑定的设置要通过使用修饰键和+号以及在大括号内写上要执行的动作。'spawn' 这个动作会启动指定的程序。例如，你会有这样的按键绑定：`Mod+T` 启动 alacritty 以

及 `Mod+D` 启动 fuzzel包。默认配置下直接运行 Niri 时 `Mod` 一般是 `Super` 键，当它在其他环境下作为一个窗口运行时则是 `Alt` 键。

```
~/.config/niri/config.kdl
binds {
    ...
    Mod+T { spawn "alacritty"; }
    Mod+D { spawn "fuzzel"; }
    ...
}
```

注意使用 `spawn` 时所有用空格分开的参数都需要用引号包裹：

```
~/.config/niri/config.kdl
binds {
    ...
    Mod+Ctrl+semicolon {
        spawn "swaylock" "-c" "121212" "-e" "-f" "-F"
    }
    ...
}
```

WASD-式操作

可以配置 Niri 的工作区、窗口切换快捷键类似于游戏中的 WASD 那样：

```
~/.config/niri/config.kdl
binds {
    ...
    Mod+A { focus-column-left; }
    Mod+S { focus-window-or-workspace-down; }
    Mod+W { focus-window-or-workspace-up; }
    Mod+D { focus-column-right; }
    ...
}
```

注意这可能也需要更改其他的绑定。同时，一些人可能会喜欢在右手侧配置，或是 Vim 式的操作。

自动启动

Niri 可以让某些程序随 Niri 一起启动。 例如上面提到的程序，比如 mako包, waybar 包 和 swayidle包/swaylock包 可以这样配置

```
~/.config/niri/config.kdl
spawn-at-startup "mako"
spawn-at-startup "waybar"
spawn-at-startup "swayidle" "-w" "timeout" "601" "niri msg action power-off-monitors"
"timeout" "600" "swaylock -f" "before-sleep" "swaylock -f"
```

注意这些进程被绑定到了 Niri 的会话，使得当 Niri 退出或挂起时，它们也会被杀死。为使它们能持续运行，你可能需要通过添加 `"&"` 参数来把它们设置成后台任务。

XWayland

Niri 不提供 XWayland 支持来运行 X11 应用。它推荐使用 xwayland-satellite包 这个外部工具来作为替代。这个包被列到了可选依赖中，安装它之后不需要做额外的配置。

另见

- Niri's own wiki

5.5 Dwl

难度 进阶 | **篇幅** 9.4 KB | **所属** Wayland 合成器

本节将讲到

- 安装
- 配置
- 用法
- 技巧和窍门
- 故障排除

翻译状态：

- 本文（或部分内容）译自 dwl，最近一次同步于 2026-07-26，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

dwl 是一个基于 wlroots 的 Wayland 合成器，在设计理念上与 dwm 相似。它精简、快速，并且可以通过社区制作的 补丁 进行扩展。默认配置下仅提供最基本的窗口管理功能，足以日常使用；若需更多特性，可按需选用社区补丁。

安装

dwl 可以通过 dwlAUR 或 dwl-gitAUR 软件包安装。请在**构建和安装**之前完成所需的配置修改，参见 makepkg。

提示：上游构建说明 也可参考，但这种方式安装的文件不会被 pacman 跟踪。

配置

dwl 通过编辑部分源代码文件（主要是 `config.h`）在编译时进行配置。有关这些设置的详细信息，请参阅附带详细注释的 `config.def.h`。

官方网站提供了许多 补丁，可以为 *dwl* 添加额外功能。这些补丁主要修改 `dwl.c` 文件，并在适当的地方修改 `config.h` 文件。有关应用补丁的信息，参见文章 给软件包打补丁。

启用 XWayland

要启用 Xwayland 支持构建 *dwl*，请安装 xorg-xwayland包 并将构建文件中的以下行取消注释：

```
/path/to/config.mk
# 取消注释以构建 XWayland 支持
XWAYLAND = -DXWAYLAND
XLIBS = xcb xcb-icccm
```

用法

dwl 可以从选择的显示管理器中启动，在其提供的菜单中选择对应的桌面项。

要从 getty 启动 *dwl*，执行如下命令：

```
$ dwl
```

使用 `-s` 启动的命令，其标准输入（stdin）会被 *dwl* 持续写入当前的布局、窗口标题、app-id 和标签等信息。如果您的命令不需要读取这些标准输入数据，为避免管道阻塞或产生意外错误，请在该命令后追加 `<&-` 以关闭其标准输入，例如：

```
$ dwl -s 'sh /path/to/autostart.sh <&-'
```

如果使用显示管理器，可以将此附加到桌面项的 `Exec` 行中。

注意：有关 *dwl* 使用的更详细信息，请参阅上游 指南。

技巧和窍门

让某些窗口使用浮动布局

对于某些窗口，例如首选项对话框，平铺并不合适——它们应该自由浮动。例如，要让 Firefox 的首选项对话框浮动，请将以下内容添加到你的 rules 数组中：

```
/path/to/config.def.h
static const Rule rules[] = {
...
{ NULL, "Firefox Preferences", 1 << 8, 1, -1 },
...
};
```

屏幕捕获

下面的脚本会首先创建一个目录 `*/path/to/pics*`，如果成功，它会使用屏幕捕获工具截屏并保存为以当前日期为文件名、扩展名为 `.png` 的文件，存放在该新建的目录中：

```
/path/to/scripts/screenshot.sh
#!/bin/sh
mkdir -p /path/to/pics && grim "/path/to/pics/$(date +'%m-%d-%Y-%H%M%S').png"
```

此外，如果你还想使用选择框截屏而不是截取所有显示器，请创建以下脚本：

```
/path/to/scripts/screenshotsel.sh
#!/bin/sh
mkdir -p /path/to/pics && grim -g "$(slurp)" "/path/to/pics/$(date +'%m-%d-%Y-
%H%M%S').png"
```

如果要截屏并保存到剪贴板而不是文件系统，请创建以下脚本：

```
/path/to/scripts/screenshotselcopy.sh
#!/bin/sh
grim -g "$(slurp)" - | wl-copy
```

最后添加以下内容：

```
/path/to/config.def.h
static const Key keys[] = {
...
{ 0, XKB_KEY_Print, spawn, SHCMD("path/to/scripts/screenshot.sh") },
```

```
{ WLR_MODIFIER_SHIFT, XKB_KEY_Print, spawn,
SHCMD("path/to/scripts/screenshotsel.sh") },
{ WLR_MODIFIER_CTRL, XKB_KEY_Print, spawn,
SHCMD("path/to/scripts/screenshotselcopy.sh") },
...
};
```

这将屏幕捕获映射到 `Print` 键，选择框截屏映射到 `Shift+Print`，而截屏并复制到剪贴板映射到 `Ctrl+Print`。

注意：

- 如果在 *dwl* 构建中打了层动画补丁，可能需要为 slurp 禁用这些动画，因为 *grim* 最终会同时捕获选择器边框。
- 以上所有脚本都需要可执行权限，请确保已设置。此外，它们有一些依赖，请安装 grim包、slurp包 和 wl-clipboard包 才能正常工作。

调整音量

安装 pipewire-pulse包 或 pulseaudio包，然后添加以下音量管理选项：

```
/path/to/config.def.h
static const char *up_vol[]   = { "pactl", "set-sink-volume", "@DEFAULT_SINK@", "+10%",
NULL };
static const char *down_vol[] = { "pactl", "set-sink-volume", "@DEFAULT_SINK@", "-10%",
NULL };
static const char *mute_vol[] = { "pactl", "set-sink-mute",   "@DEFAULT_SINK@",
"toggle", NULL };

static const Key keys[] = {
...
{ 0, XKB_KEY_XF86AudioRaiseVolume, spawn, {.v = up_vol } },
{ 0, XKB_KEY_XF86AudioLowerVolume, spawn, {.v = down_vol } },
{ 0, XKB_KEY_XF86AudioMute, spawn, {.v = mute_vol } },
...
};
```

自启动

有一个 补丁 可用，它允许你自启动指定的应用程序。示例如下：

```
/path/to/config.def.h
/* 自启动 */
static const char *const autostart[] = {
    "wbg", "/path/to/your/image", NULL,
    NULL /* 终止 */
};
```

你可以通过向列表中添加更多条目来指定更多应用程序，每个条目用逗号（,）分隔，后跟 `NULL`。例如，你可以启动状态栏、通知守护进程或任何后台服务。只需确保每个命令及其参数正确引用即可。

更好的鼠标大小调整

默认情况下，调整大小时 *dwl* 会将光标移到右下角，通过一个 补丁 可以改变这一行为。

该补丁允许你配置窗口调整大小行为，使其更灵活，替代默认的跳转到右下角，它提供了多种模式：

- 将光标移动到*选定*的角落。
- 在调整大小时*锁定*光标位置。
- 完全禁用光标跳转。
- 相对角落调整大小，此时光标距离哪个角最近，就使用哪个角进行调整。

该补丁的一个工作配置示例如下：

```
path/to/config.def.h
/* resize_corner:
 * 0: 左上
 * 1: 右上
 * 2: 左下
 * 3: 右下
 * 4: 距离光标最近的角
 */
static const int resize_corner = 4;
static const int warp_cursor = 0;  /* 1: 跳转到角落, 0: 不跳转 */
static const int lock_cursor = 0;  /* 1: 锁定光标, 0: 不锁定 */
```

这样，调整大小时将选择距离光标最近的角落，光标*不会*跳转，并且光标*不会*锁定自身位置。

故障排除

修复 Java 应用程序行为异常

参见 Java#灰色窗口/应用不随窗口管理器调整大小/菜单自动关闭。

修复终端窗口周围的间隙

如果终端窗口外存在桌面空白间隙，很可能是因为终端的字体大小所致。可以调整字体大小直到找到能消除间隙的理想尺寸，或者将 `config.h` 中的 `resizehints` 切换为 *0*。

这会导致 *dwl* 忽略所有客户端窗口的调整大小请求，而不仅仅是终端。此变通方法的缺点是某些终端可能会出现重绘异常，例如显示残影（残留线条）和提前换行等问题。

另外，如果你使用 st 终端模拟器，可以应用 anysize 补丁并重新编译 st。

参见

- bemenu - 一个 Wayland 原生的 dmenu 替代品。
- 一个 *dwl* 的 介绍视频。
- *dwl* 的 git 仓库。

5.6 Weston

难度 进阶 | **篇幅** 12.2 KB | **所属** Wayland 合成器

本节将讲到

- 安装
- 使用
- 配置
- 提示与技巧
- 参考

Weston 是一个开源的、基于Wayland 显示服务器协议的窗口管理器(compositor)，专为正确性、可靠性、可预测性和性能而设计。Weston作为Wayland协议的实现，提供了显示管理器的窗口管理和基本功能 ，包括不限于窗口的创建、移动、缩放、透明度处理、输入事件分发以及多屏幕支持等功能。

安装

安装 weston包 软件包。

使用

要以本机方式（从 TTY）启动 Weston 或在正在运行的 X 会话中运行 Weston：

```
$ weston
```

想了解详细信息和配置，请参阅 weston(1) 。

示例应用

在 Weston 中内置了原生wayland演示应用，可以运行这些示例程序。要启动终端仿真器，请执行以下命令：

```
$ weston-terminal
```

可移动的花

```
$ weston-flower
```

显示图片

```
$ weston-image image1.jpg image2.jpg...
```

快捷键

提示：`Super` (windows按键) 可以自定义, 详细查看 weston.ini

<table class="wikitable"><caption>Weston 键盘快捷键
</caption><tbody><tr><th>组合键</th><th>动作
</th></tr><tr><td><code>Ctrl+Alt+Backspace</code></td><td>退出 Weston 合成器
</td></tr><tr><td><code>Super+Scroll</code> (或 <code>PageUp</code>/

<code>PageDown</code>)</td><td>放大/缩小 窗口
</td></tr><tr><td><code>Super+Tab</code></td><td>切换窗口
</td></tr><tr><td><code>Super+LMB</code></td><td>启动窗口
</td></tr><tr><td><code>Super+MMB</code></td><td>旋转窗口
</td></tr><tr><td><code>Super+RMB</code></td><td>改变窗口大小
</td></tr><tr><td><code>Super+Alt+Scroll</code></td><td>改变窗口透明度
</td></tr><tr><td><code>Super+k</code></td><td>强制关闭当前活动窗口
</td></tr><tr><td><code>Super+Up/Down</code></td><td>切换到 上一个/下一个 工作区</td></tr><tr><td><code>Super+Shift+Up/Down</code></td><td>移动当前活动窗口到其他工作区</td></tr><tr><td><code>Super+Fn</code></td><td>切换到工作区n (例如：F2)</td></tr><tr><td><code>Super+s</code></td><td>截屏</td></tr><tr><td><code>Super+r</code></td><td>录制屏幕
</td></tr></tbody></table>

配置

下面是一个示例配置文件。参见 weston.ini(5) 了解更多信息。

```
~/.config/weston.ini
[core]
# xwayland support
xwayland=true

[libinput]
enable-tap=true

[shell]
#background-image=/usr/share/backgrounds/gnome/Aqua.jpg
background-type=scale-crop
background-color=0xff000000
#background-color=0xff002244
#panel-color=0x90ff0000
panel-color=0x00ffffff
panel-position=bottom
#clock-format=none
#animation=zoom
#startup-animation=none
close-animation=none
focus-animation=dim-layer
#binding-modifier=ctrl
num-workspaces=6
locking=false
cursor-theme=Adwaita
cursor-size=24

# 平板电脑选项
#lockscreen-icon=/usr/share/icons/gnome/256x256/actions/lock.png
#lockscreen=/usr/share/backgrounds/gnome/Garden.jpg
#homescreen=/usr/share/backgrounds/gnome/Blinds.jpg
#animation=fade

# 用于笔记本电脑显示器
[output]
name=LVDS1
mode=preferred
```

```
#mode=1680x1050
#transform=rotate-90

#[output]
#name=VGA1
# 下面使用 modeline 设置模式，您可以使用 cvt 工具获取首选分辨率的模式线
#mode=173.00 1920 2048 2248 2576 1080 1083 1088 1120 -hsync +vsync
#transform=flipped

#[output]
#name=X1
#mode=1024x768
#transform=flipped-rotate-270

# 屏幕键盘输入法
#[input-method]
#path=/usr/lib/weston/weston-keyboard

[keyboard]
keymap_rules=evdev
#keymap_layout=us,de
#keymap_variant=colemak,
#keymap_options=grp:shifts_toggle
#keymap_options=caps:ctrl_modifier,shift:both_capslock_cancel
repeat-rate=30
repeat-delay=300

# keymap_options from /usr/share/X11/xkb/rules/base.lst
#numlock-on=true

[terminal]
font=monospace
font-size=18

[launcher]
icon=/usr/share/weston/icon_flower.png
path=/usr/bin/weston-flower

[launcher]
icon=/usr/share/icons/gnome/32x32/apps/utilities-terminal.png
path=/usr/bin/weston-terminal --shell=/usr/bin/bash

#[launcher]
#icon=/usr/share/icons/gnome/32x32/apps/utilities-terminal.png
#path=/usr/bin/gnome-terminal

[launcher]
icon=/usr/share/icons/hicolor/32x32/apps/firefox.png
path=MOZ_ENABLE_WAYLAND=1 /usr/bin/firefox

#[launcher]
#icon=/usr/share/icons/Adwaita/32x32/apps/multimedia-volume-control.png
#path=/usr/bin/st alsamixer -c0
```

最少 weston.ini 配置:

```
~/.config/weston.ini
[core]
xwayland=true

[keyboard]
keymap_layout=us

[launcher]
icon=/usr/share/weston/terminal.png
path=/usr/bin/weston-terminal
```

显示器

Weston 的输出与 `xorg.conf` 显示器的输出略有不同：

```
$ ls /sys/class/drm
card0
card0-VGA-1
card1
card1-DVI-I-1
card1-HDMI-A-1
card1-VGA-2
```

`card0` 是未使用的显卡(内置视频适配器)。附加适配器 `card1` 通过视频线连接到一个 HDMI 和一个 DVI 显示器，因此输出名称为 `HDMI-A-1` 和 `DVI-I-1`。

Xwayland

请参阅 Wayland#Xwayland 了解可用软件包的详细信息和概述。

设置以下内容以开启 Xwayland 的使用：

```
~/.config/weston.ini
[core]
xwayland=true
```

注意： 如果尚未配置 X，则可能需要配置键盘映射： Xorg中的键盘配置

高 DPI 显示器

对于 Retina 或 HiDPI 的显示器, 使用:

```
~/.config/weston.ini
[output]
name=...
scale=2
```

Weston 字体

Weston 使用默认的无衬线字体作为窗口标题栏、时钟等。有关如何更改此字体的说明，请参见字体配置#设置默认和后备字体。

Systemd启动

weston虽没法作为systemd系统服务启动，但可通过systemd用户服务进行启动，我们先创建 weston 的服务和 socket 单元。

systemd用户服务启动

创建用户级weston套节字(socket) 单元，用于监听图形应用

提示：若您没有 `/etc/systemd/user/` 目录的权限，也可在 `$HOME/.config/systemd/user` 目录下进行创建，两者使用效果相同

```
/etc/systemd/user/weston.socket
[Unit]
Description=Weston, a Wayland compositor
Documentation=man:weston(1) man:weston.ini(5)
Documentation=https://wayland.freedesktop.org/

[Socket]
ListenStream=%t/wayland-0
```

创建用户级weston服务(service)单元

```
/etc/systemd/user/weston.service
[Unit]
Description=Weston, a Wayland compositor, as a user service
Documentation=man:weston(1) man:weston.ini(5)
Documentation=https://wayland.freedesktop.org/

# 使用 systemd 套接字激活
Requires=weston.socket
After=weston.socket

# 由于我们是图形会话的一部分，因此请确保我们之前已启动
Before=graphical-session.target

[Service]
Type=notify
TimeoutStartSec=60
WatchdogSec=20
# 日志默认输出到journal
#StandardOutput=journal
StandardError=journal
# 确保已创建最少配置 ~/.config/weston.ini
ExecStart=/usr/bin/weston --modules=systemd-notify.so

[Install]
WantedBy=graphical-session.target
```

创建完毕后重新加载 `systemd --user daemon-reload`，然后在使用 `systemd --user start weston.service` 启动weston

以上的使用需要我们提前登录到系统，并手动运行，若想实现自动启动请参考下面步骤。

systemd系统会话管理

一些场合下，我们希望在系统引导后自动启动weston，当然也支持ssh远程登录后手动启动weston，如您不想使用第三方登录管理器，也可使用systemd服务单元来完成该操作，下面使用sysemd服务来创建用户会话。

注意：访问`/etc/systemd/system/`要使用root权限,普通用户无法创建

```
/etc/systemd/system/weston-autologin.service
[Unit]
Description=Weston graphical session

# 确保我们在允许登录后启动。
After=systemd-user-sessions.service

# 如果需要，可以将其作为图形会话的一部分。
#Before=graphical.target

# 不是不需，但以防万一。
#ConditionPathExists=/dev/tty7

[Service]
Type=simple
Environment=XDG_SESSION_TYPE=wayland
ExecStart=/usr/bin/systemctl --wait --user start weston-session.target
```

```
# 要运行会话的用户。需要依据实际用户修改!
User=user
Group=user

# 为 Weston 要求的用户设置完整的用户会话。
PAMName=login

# 需要一个虚拟终端。
TTYPath=/dev/tty7
TTYReset=yes
TTYVHangup=yes
TTYVTDisallocate=yes

# 如果不控制 tty，则无法启动。
StandardInput=tty-fail

# 日志输出默认为journal
#StandardOutput=journal
StandardError=journal

# 使用 utmp 记录此用户，让它显示命令 'w' 和 'who'
UtmpIdentifier=tty7
UtmpMode=user

[Install]
WantedBy=graphical.target
```

请确保 `User` 和 `Group` 条目是使用有效的用户。同时还需创一个`weston-session.target`的用户target

```
/etc/systemd/user/weston-session.target
[Unit]
Description=Weston session

BindsTo=weston-session.target
Before=weston-session.target
```

通知系统重新加载配置

```
 $ sudo systemd damon-reload
 $ systemd damon-reload --user
```

最后确保您的默认target使用的是graphical

```
 $ sudo systemctl set-default graphical.target
 $ systemctl --user set-default graphical-session.target
```

提示与技巧

录像录制

Weston 具有内置的截屏和视频录制功能，可以通过按 `Super+r` 组合键来启动和停止。截屏视频保存到 Weston 当前工作目录中的文件 `capture.wcap` 中。WCAP 格式是 Weston 特有的无损视频格式，它只记录帧数的差异。为了能够播放录制的截屏视频，需要将WCAP文件转换为媒体播放器可以理解的格式。首先，将捕获转换为 YUV 像素格式：

```
$ wcap-decode --yuv4mpeg2 capture.wcap > capture.y4m
```

然后，可以使用 FFmpeg 或 x264包 将 YUV 文件转码为其他格式（有关详细信息，请参阅 `x264 -h`）。

窗口切换

要切换带有 `Super+Space` 而不是 `Super+Tab` 的窗口，请在 `desktop-shell/shell.c` 中将 `KEY_TAB` 更改为 `KEY_SPACE`，然后重新编译 weston包。

触摸屏幕映射

触摸显示映射可以通过使用 udev 规则来实现。就像 `WL_SEAT` 或 `ID_SEAT` 设备环境变量将设备绑定到指定的席位一样，`WL_OUTPUT` 变量可用于将设备绑定到相应的输出。此变量应与显示器名称匹配。有关显示器命名约定的说明，请参见 #显示器。

参考

- 官方 Weston 项目Gitlab仓库
- 启动和运行Weston

5.7 Hyprland

难度 高级 | **篇幅** 33.4 KB | **所属** Wayland 合成器

本节将讲到

- 安装
- 配置
- 使用
- Hypr 生态
- 技巧提示

翻译状态：

- 本文（或部分内容）译自 Hyprland，最近一次同步于 2026-07-27，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

Hyprland 是一个独立的且使用 C++ 编写的平铺式 Wayland 合成器。 Hyprland 值得注意的功能包括动态平铺、选项卡窗口、整洁可读的 C++ 代码库，和提供窗口动画的自定义渲染器，窗口圆角，以及透明窗口上的 Dual-Kawase Blur。一般用法和配置在 Hyprland Wiki 中有详细文档介绍。

当你遇到输入法问题时，请查看故障排除。

安装

通过包管理器安装

安装 hyprland包

通过CMake编译安装

首先将 Git 仓库克隆到本地：

```
git clone --recursive https://github.com/hyprwm/Hyprland
```

其次切换到 Git 仓库目录：

```
cd Hyprland
```

最后编译安装：

```
make all && sudo make install
```

自 #6608 起，Hyprland 使用 aquamarine包 作为自己的渲染后端库。在此之前，它捆绑了它自己的 *wlroots* 版本，与wlroot-gitAUR类似。

注意：

- NVIDIA GPU 用户也应该在尝试启动Hyprland之前确保遵守上游的 NVIDIA 页面 Hyprland Wiki。如果不这么做，可能会导致很多错误，包括可能无法登录、窗口闪烁和 CPU 使用率较高。
- Hyprland 不支持使用 NVIDIA GPU，很多用户成功了，但如果一些东西出现了错误，那你只能靠你自己来修复了。
- 由于缺乏`seatd.service`会导致Hyprland无法启动，请确保安装了 Polkit 软件，或者启动并启用 `seatd.service`。
- 对于 Vmware/VirtualBox 用户，可能需要开启 3D 加速才可以正常运行 Hyprland

配置

注意：在上游Hyprland Wiki 中详细解释了下文列出的大部分选项（以及更多选项）。自从0.55版本后，Hyprland 已经更换为使用Lua配置，并且弃用了旧的自定义语法。

配置是通过一个单一的`Lua`文件完成的，它通常在`~/.config/hypr/hyprland.lua`。若该文件不存在，*hyprland* 将会自动生成一个示例配置文件。你也可以将你的配置分为多个文件，并且使用`Lua`语言中的`require()`函数加载这些配置文件。默认生成的配置可见`/usr/share/hypr/hyprland.lua`。

`hyprland.lua` 文件包含配置设备（如键盘、鼠标、触控板、显示器）的指令，以及动画、装饰、布局等设置。你可以设置按键绑定、窗口规则以及在启动时执行命令。

在你保存配置文件时，配置都会自动重新加载。你可以使用 `hyprctl reload` 来重载以获得相同的效果。对于某些设置（尤其是输入设置），你可能需要重启你的 Hyprland 会话。

设置也可以使用 *hyprctl* 动态更改设置，但是这些设置不会保存

移除最上方的警告

请在`~/.config/hypr/hyprland.lua`中移除或使用`#`注释以下内容:

```
~/.config/hypr/hyprland.lua
autogenerated = 1
```

键盘

按键映射

Hyprland 默认使用 `us` 键盘布局，你可以使用`hl.config()` 函数来更改键盘布局：

```
~/.config/hypr/hyprland.lua
-- German Colemak layout
hl.config({
    input = {
        kb_layout = "de",
        kb_variant = "colemak",
    }
})
```

查看 Hyprland Wiki 以查看所有可用选项

注意：Hyprland 将会覆盖 locale 定义，如果你不使用 `us` 布局的键盘则有必要更改按键映射

键入延迟和速率

虽然Xorg用户将习惯于在服务器级别定义此设置，在Wayland上，每个合成器都会自行处理：

```
~/.config/hypr/hyprland.lua
-- Repeat rate and delay
hl.config({
    input = {
        repeat_rate = 25,
        repeat_delay = 600,
    }
})
```

键盘背光

参见背光一文。

你可以在 Hyprland 中使用键盘亮度控制。

你可以尝试在 Hyprland 中控制键盘背光。安装 brightnessctl包 然后添加一个与其有关的按键绑定，随后使用 `SUPER, F*X*` 或 `XF86KbdBrightness` 替换 `*keyboard_brightness_**` ，这取决于你的硬件是如何发出键盘背光的。

```
~/.config/hypr/hyprland.lua
-- Keyboard backlight
hl.bind("keyboard_brightness_up_shortcut", hl.dsp.exec_cmd("brightnessctl -d
*::kbd_backlight set +33%"))
hl.bind("keyboard_brightness_down_shortcut", hl.dsp.exec_cmd("brightnessctl -d
*::kbd_backlight set 33%-"))
```

此外，你还可以设置更改时弹出屏幕通知。

媒体按键

在Hyprland上有可能通过使用 `XF86Audio` 键盘按键和一个外部像 pavucontrol包 或者 pamixer包 和 playerctl 的应用程序使用键盘媒体按键控制媒体（翻译可能不准确，请参照下面英语）

通过 `XF86Audio` 按键在Hyprland中使用键盘的媒体控制键和启动一个像pavucontrol包、pamixer包和playerctl包的应用：

```
~/.config/hypr/hyprland.lua
-- Volume and Media Control
hl.bind("XF86AudioRaiseVolume", hl.dsp.exec_cmd("pamixer -i 5"), { repeating = true })
hl.bind("XF86AudioLowerVolume", hl.dsp.exec_cmd("pamixer -d 5"), { repeating = true })
hl.bind("XF86AudioMicMute", hl.dsp.exec_cmd("pamixer --default-source -m"))
hl.bind("XF86AudioMute", hl.dsp.exec_cmd("pamixer -t"))
hl.bind("XF86AudioPlay", hl.dsp.exec_cmd("playerctl play-pause"))
hl.bind("XF86AudioPause", hl.dsp.exec_cmd("playerctl play-pause"))
hl.bind("XF86AudioNext", hl.dsp.exec_cmd("playerctl next"))
hl.bind("XF86AudioPrev", hl.dsp.exec_cmd("playerctl previous"))
```

当有更改时，也可以触发屏幕通知

鼠标

参见 Hyprland Wiki。

为了应用鼠标设备的配置选项，请像这样添加一个新类别：

```
~/.config/hypr/hyprland.lua
device {
    name = 鼠标设备名称
    # 针对鼠标的选项 ...
}
```

通过检查 `hyprctl devices` 的输出，可以轻松获得设备名称。

鼠标加速度/灵敏度

您可以使用下面所示配置控制鼠标加速度/灵敏度：

```
~/.config/hypr/hyprland.lua
device {
    name = 鼠标设备名称
    sensitivity = -0.5
}
```

将 *\-0.5* 替换为你希望的鼠标加速度/灵敏度。

触摸板手势

作为 Wayland 合成器，Hyprland 完全支持触摸板手势（默认是禁用的）。你可以使用`hl.gesture()`函数启用：

```
~/.config/hypr/hyprland.lua
-- Enable touchpad gestures
hl.gesture({
    fingers = 3,
    direction = "horizontal",
    action = "workspace"
})
```

查看 Hyprland Wiki 来获取所有可用的选项

显示设置

屏幕共享

查看屏幕共享。

作为一个兼容wlroots的合成器，Hyprland 可以利用 xdg-desktop-portal-wlr包 通过 XDG 桌面门户在一系列应用程序中启用屏幕录制。

Hyprland 还维护 xdg-desktop-portal-hyprland包，一个支持屏幕共享（包括选区功能）和窗口共享，全局快捷键，并且有一个图形选择其的工具。关于使用该工具在 Hyprland wiki 中有进一步的文档介绍。

值得注意的是，*xdg-desktop-portal-hyprland* 不包括一个文件选择器，用户可以安装 xdg-desktop-portal-gtk包。

设置屏幕分辨率

Hyprland 将会尝试去自动检测你的屏幕分辨率。不过在某些情况下它将会失败并默认为故障保护，通常是在存在多个屏幕或你有混合笔记本电脑的情况下。如果屏幕下所有的东西都很大，那么你需要设置你默认的显示器和分辨率

首先使用 *hyprctl* 找到你的默认显示器：

```
$ hyprctl monitors
Monitor eDP-1 (ID 0):
        1920x1080@144.003006 at 0x0
        description: Chimei Innolux Corporation 0x153C (eDP-1)
        ...
```

然后使用`hl.monitor()`函数配置你的显示器：

```
~/.config/hypr/hyprland.lua
-- Monitor details
hl.monitor({
    output = "eDP-1",
    mode = "1920x1080@144",
    position = "0x0",
    scale = 1,
})
```

此处的`@144`指的是屏幕的刷新率，`1920x1080`指的是分辨率。请替换为显示器所支持的刷新率和分辨率。

`0x0` 是一个用于多屏幕设置和最终设置的位置偏移，`1` 是屏幕缩放倍数。

更多信息可见Hyprland Wiki

设置GUI

nwg-displays包 是管理多显示器的 GUI 软件，支持 Hyprland. 它是nwg-shell的一部分，但并不依赖 nwg-shell，参阅 nwg-displays github 以获得更多信息。

亮度

安装 brightnessctl包 并添加以下内容：

```
~/.config/hypr/hyprland.lua
-- Screen brightness
hl.bind("XF86MonBrightnessUp", hl.dsp.exec_cmd("brightnessctl s +5%"))
hl.bind("XF86MonBrightnessDown", hl.dsp.exec_cmd("brightnessctl s 5%-"))
```

此外，还可以设置更改时弹出通知。

使用

请注意：确保您已正确安装与您主机相匹配的显卡驱动或 openvmtools 并启用3D加速

启动Hyprland

通用 Wayland 会话管理器（uwsm）

通用Wayland会话管理器 通过systemd 单元文件合成器和与其对应的配置好的软件和守护进程来使你通过*systemctl*控制它们。

当你选择 *hyprland (uwsm-managed)时*，Display manager 会启动一个带有*uwsm*的Hyprland。

注意：其实你选择*Hyprland*也是可以的，只不过pid不同，如果你想为*hyprland(uwsm-managed)*设置变量，请在*uwsm*配置文件夹中创建*env-Hyprland*文件

在命令行解释器#登录 Shell使用以下脚本，你也可以在 getty 通过 uwsm 启动 Hyprland：

```
if uwsm check may-start && uwsm select; then
    exec systemd-cat -t uwsm_start uwsm start default
fi
```

注意：

- `uwsm check may-start`检查是否可以启动 wayland 会话，尤其是在登录 shell运行的情况下。不过，仍应避免在 `.bashrc` 或其他非登录 shell profile 中使用。**请确保这个代码片段放置在你的profile文件最底端**，这是因为执行该代码片段将会屏蔽该文件的剩余部分。
- Hyprland 不再推荐启动一个带有*uswm*的会话，因为这被视为实验性的并且是为明白其含义的高级用户和怪人使用的。

提示：如果你希望绕过合成器选择菜单，并且直接运行Hyprland，你可以在你的登录 shell中运行如下代码：

```
if uwsm check may-start; then
exec uwsm start hyprland.desktop
```

```
fi
```

警告： 如果您选择通过“uwsm”启动 Hyprland，则应相应调整配置。特别是：

- 必须避免使用 `hl.dsp.exit()` 调度器或直接终止 Hyprland 进程，因为这会干扰正常的关机进程。例如，使用 `uwsm stop` 或 `loginctl terminate-user ""` 来终止 Hyprland 并退出用户会话：

```
~/.config/hypr/hyprland.lua
hl.bind("SUPER + M", hl.dsp.exec_cmd("uwsm stop"))
```

- 不要在 `hyprland.lua` 中配置环境变量，而是使用 “uwsm ”文件 `~/.config/uwsm/env` 来配置 uwsm 管理的所有图形会话（GTK、Qt、xcursor、...）的通用变量。...），以及 `~/.config/uwsm/env-hyprland` 用于 Hyprland 专属环境变量（例如 `HYPR*` 和 `AQ_*` 变量）。这些文件的格式为 `export *KEY*=*VALUE*`，每行不加注释。如果使用多个 GPU，强烈建议将环境变量 `AQ_DRM_DEVICES` 置于 `env-hyprland` 内，以避免与其他合成器发生冲突。

请阅读Hyprland Wiki 中的相关页面以了解有关使用 uwsm 配置的更多信息。

终端

使用如下指令在 Getty 中启动 Hyprland：

```
$ start-hyprland
```

显示管理器

虽然从显示管理器中启动不是官方支持的，但是用户们报告成功在 GDM 、 SDDM 和其他的显示管理器中启动。hyprland包 中包含两个桌面项，并且所有 Hyprland AUR 包将会自动生成。

两种方法都提供相同的结果，加上或减去一些环境变量和服务。

自动登录

用户可以通过一个显示管理器或着在登录时采用 Xinit#Autostart X at login 中描述的方法进行自动登录

hyprctl 和 IPC

hyprctl 是一个命令行实用程序，随Hyprland一起安装，用于与显示服务器通信。它允许你向服务器发送命令（相当于配置文件中的命令，但语法略有不同）、设置关键词、发送询问和请求信息。请查看完整文档

Hyprland 还公开了 2 UNIX 套接字，以便通过代码或命令行实用程序控制和获取有关 Hyprland 的信息。这些套接字会广播焦点变化（窗口、工作区、监视器）、窗口/工作区创建等事件。

hyprctl 和 IPC 接口都在脚本中有效使用，对于复杂任务控制 Hyprland。

自动启动

在 Lua 配置文件中，你可以在`hyprland.start`事件监听器中执行命令以实现自动启动软件。

注意：在#配置中提到，Hyprland 会自动解析 `hyprland.lua` 文件**每次的更改**。使用 `hl.on("hyprland.start", ...)`事件确保了软件和守护进程仅仅在引导后启动一次，防止资源竞争情况或者多个实例在重载后同时运行。

提示：如果你通过通用Wayland会话管理器(uwsm)启动Hyprland，并且你想在开机时运行的软件提供一个systemd单元，那么你可以启用它来在Hyprland启动后自动启动该单元。另外，你也可以将那个软件作为参数传递到`uwsm app`中来使它被*uwsm*管理。例如：

```
~/.config/hypr/hyprland.lua
hl.on("hyprland.start", function ()
hl.exec_cmd("uwsm app -- mycommand --arg1 --arg2")
end)
hl.bind("SUPER + E", hl.dsp.exec_cmd("uwsm app -- pcmanfm-qt.desktop"))
```

设置环境变量

你可以使用`hl.env()`函数在`hyprland.lua`中设置环境变量，但是它的语法不同于在Unix shell中使用*env*命令

详见Hyprland Wiki。

如果`XDG_MENU_PREFIX`变量未设置，KDE组件软件（如Dolphin、digiKam）可能会展示一个空的 *使用……打开* 菜单。设置这个来匹配一个已安装在`/etc/xdg/menus/`的菜单文件，如：

```
~/.config/hypr/hyprland.lua
hl.env("XDG_MENU_PREFIX", "gnome-")
```

使用`plasma-`作为`plasma-applications.menu`或者`xfce-`作为`xfce-applications.menu`。然后重构KDE服务缓存：

```
$ kbuildsycoca6 --noincremental
```

Hypr 生态

警告：以下所提及的部分软件仍在开发，有 bug 都是在意料之中的。因此，完整的说明和示例会被略去，直到它足够稳定以成为稳定版。关于使用说明，参见上游 Wiki。

Hyprland 开发团队正在构建一个专门用于 Hyprland 的软件生态，这些工具将包括调度器，允许使用 hyprctl 而不是依赖脚本来控制它们。

目前，软件生态包括以下软件：

Hyprpaper

Hyprpaper 是一个壁纸工具，你可以安装 hyprpaper包 来使用它。

Hyprpicker

Hyprpicker 是一个用于在一个桌面中抓取一个颜色的工具，你可以安装 hyprpicker包 来使用它。该软件不需要其余配置。

Hypridle

Hypridle 是一个idle管理守护程序，你可以安装 hypridle包 来使用它。

Hyprlock

Hyprlock 是一个屏幕锁定管理器，你可以安装 hyprlock包 来使用它。

Hyprcursor

Hyprcursor 是一个处理屏幕光标的新格式，它在传统的方式上提升了很多，你可以安装 hyprcursor包 来使用它。

Hyprcursor 主题

提示：如果你安装了 hyprcursor 并且没有安装一个主题，它将会使用你之前的光标设置

光标主题可以使用 AUR 安装，以下是例子：

- sweet-cursors-hyprcursor-gitAUR
- nordzy-hyprcursorsAUR
- xcursor-pro-hyprcursorAUR
- hyprcursor-dracula-kde-gitAUR

将现有的光标主题移植到 Hyprcursor，你可以参照上游 GitHub 仓库。

关于设置光标样式，你也可以查阅光标主题来设置。

XDG-Desktop-Portal-Hyprland

Hyprland 自己的实现的XDG Desktop Portal，与其他基于wlroots的合成器兼容，但是在使用Hyprland时额外提供一些功能。可安装xdg-desktop-portal-hyprland包

Hyprpolkitagent

Hyprpolkitagent 是一个 polkit 身份验证守护进程。图形用户界面应用程序需要使用它来请求提升权限。安装 hyprpolkitagent包 来使用它。

参见 Hyprland Wiki。

Hyprsunset

Hyprsunset 是提供蓝光滤镜的实用工具，通过 hyprsunset包 安装。

Hyprsysteminfo

Hyprsysteminfo 是一个获取系统信息的软件，与neofetchAUR和fastfetch包类似。你可以安装hyprsysteminfoAUR

技巧提示

注意：

- 对于以下所有部分，通常有不止一种方法可以达到类似的效果，这里提供的只是一些基本示例。
- 有关替代品的完整列表，请参见应用程序列表，Hyprland 的特定列表可在上游 Wiki 找到。

文件管理器

在 Hyprland 环境下，如果需要图形化的文件管理功能，则需要一个兼容 Wayland 的外部应用程序。以 thunar包 为例，我们只需为其分配一个快捷键，操作如下：

```
~/.config/hypr/hyprland.lua
...
hl.bind("SUPER + E", hl.dsp.exec_cmd("thunar"))
...
```

应用程序启动器

在 Hyprland 环境下，如果需要启动应用程序，则需要一个兼容 Wayland 的外部应用程序。以 wofi包 软件包为例，我们只需为其分配一个快捷键，操作如下：

```
~/.config/hypr/hyprland.lua
...
hl.bind("SUPER + F", hl.dsp.exec_cmd("wofi --show drun"))
...
```

Idle

Hyprland 需要一个外部的 Wayland 兼容的 idle 管理守护进程。最普遍安装的是 hypridle包 和 hyprlock包。你可以使用如下的快捷键来手动锁定屏幕：

```
~/.config/hypr/hyprland.lua
...
hl.bind("SUPER + L", hl.dsp.exec_cmd("hyprlock"))
...
```

自动锁定屏幕和挂起

创建下面的文件：

```
~/.config/hypr/hyprland.lua
...
hl.bind("SUPER + L", hl.dsp.exec_cmd("hyprlock"))
...
```

提示：你可以通过修改数值（以秒为单位）来调整无操作时的锁屏超时时间。300 是 5 分钟，600 是 10 分钟等等。

这个文件必须是可执行的，然后运行它：

```
~/.config/hypr/hyprland.lua
-- enable hypridle at start
hl.on("hyprland.start", function()
    ...
    hl.exec_cmd("hypridle")
    ...
end)
```

在超时期限后使用 DPMS 关闭屏幕

Hyprland 有一个内置的调度器来控制 DPMS 请求，不过不建议使用它作为一个直接按键绑定，这样做的结果是你不能打开重新打开你的屏幕，且需要你重新启动。

更改在上文的文件并将其更改为：

```
~/.config/hypr/hypridle.conf
general {
    lock_cmd = pidof hyprlock || hyprlock
}

listener {
    timeout = 300
    on-timeout = loginctl lock-session
}

listener {
    timeout = 600
    on-timeout = hyprctl dispatch dpms off
    on-resume = hyprctl dispatch dpms on
}

listener {
    timeout = 900
    on-timeout = systemctl suspend
}
```

状态栏

在 Hyprland 环境下，如果需要状态栏，则需要一个兼容 Wayland 的外部应用程序。以 waybar包 软件包为例，操作如下：

```
~/.config/hypr/hyprland.lua
-- enable status bar at start
hl.on("hyprland.start", function()
    ...
    hl.exec_cmd("waybar")
    ...
end)
```

工作区总揽

waybar包 内置了适于这个功能的完全可自定义模块，支持Hyprland工作区自然地切换。

查看 waybar Wiki 了解详情。

Polkit 认证

Polkit 认证需要使用外部身份验证代理。Hyprland 推荐使用hyprpolkitagent包 ，你也可以选择其他的Polkit

调用hyprpolkitagent包：

```
~/.config/hypr/hyprland.lua
-- enable polkit authentication agent at start
hl.on("hyprland.start", function()
    ...
    hl.exec_cmd("systemctl --user start hyprpolkitagent")
```

```
    ...
end)
```

桌面壁纸

在 Hyprland 环境下，如果需要管理桌面壁纸，则需要一个兼容 Wayland 的外部应用程序。以 hyprpaper包 软件包为例，操作如下：

```
~/.config/hypr/hyprland.lua
-- enable hyprpaper at start
hl.on("hyprland.start", function()
    ...
    hl.exec_cmd("hyprpaper")
    ...
end)
```

另外，hyprpaper包也需要一个配置文件来启用，创建一个文件，如下：

```
~/.config/hypr/hyprpaper.conf
wallpaper {
    monitor = monitor
    path = /home/me/amongus.png
    fit_mode = cover
```

将*monitor*替换为你想设置壁纸的显示器，你可以通过`hyprctl monitors`来获取一个显示器列表。`monitor`也可以是空的，这样就可以应用规则到所有显示器。

随机壁纸脚本

创建以下脚本并确保其可执行：

```
~/.config/hypr/scripts/hyprpaper-random
#!/usr/bin/env bash

WALLPAPER_DIR="$HOME/.config/hypr/wallpapers/"
CURRENT_WALL=$(hyprctl hyprpaper listloaded)

# Get a random wallpaper that is not the current one
WALLPAPER=$(find "$WALLPAPER_DIR" -type f ! -name "$(basename "$CURRENT_WALL")" | shuf
-n 1)

# Apply the selected wallpaper
hyprctl hyprpaper reload ,"$WALLPAPER"
```

接下来创建一个新目录来存储背景图片，类似于 `~/.config/hypr/wallpapers` 的文件夹即可，并在其中放置任何你想要设置为的图片。

最后，当*指定*按键被按下时调用脚本：

```
~/.config/hypr/hyprland.lua
...
hl.bind("SUPER + R", hl.dsp.exec_cmd("~/.config/hypr/scripts/hyprpaper-random"))
...
```

屏幕通知

屏幕上对亮度和音量调整等操作显示通知的功能可以借助外部的通知服务程序来实现。这里我们以 mako包 为例。

更多屏幕通知软件请见桌面通知程序#独立通知，参阅桌面通知程序以获取媒体键通知的进一步说明。

脚本请见Mako#使用方法#媒体键通知

然后添加以下内容（或编辑任何现有绑定）：

```
~/.config/hypr/hyprland.lua
-- Volume
hl.bind("XF86AudioRaiseVolume", hl.dsp.exec_cmd("~/.config/hypr/scripts/volume --inc"))
hl.bind("XF86AudioLowerVolume", hl.dsp.exec_cmd("~/.config/hypr/scripts/volume --dec"))
hl.bind("XF86AudioMicMute", hl.dsp.exec_cmd("~/.config/hypr/scripts/volume --toggle-
mic"))
hl.bind("XF86AudioMute", hl.dsp.exec_cmd("~/.config/hypr/scripts/volume --toggle"))
```

屏幕背光通知

脚本请见Mako#使用方法#屏幕背光通知

然后添加以下内容（或者编辑任何现有的绑定）：

```
~/.config/hypr/hyprland.lua
-- Screen brightness
hl.bind("XF86MonBrightnessUp", hl.dsp.exec_cmd("~/.config/hypr/scripts/backlight --
inc"))
hl.bind("XF86MonBrightnessDown", hl.dsp.exec_cmd("~/.config/hypr/scripts/backlight --
dec"))
```

键盘（Keyboard）语言通知

要运行此脚本， 你需要一个命令行 JSON 处理器 gojqAUR

脚本请见Mako#使用方法#键盘（Keyboard）语言通知

然后添加以下内容（或编辑任何现有的绑定）：

```
~/.config/hypr/hyprland.lua
hl.device({
  name = "at-translated-set-2-keyboard",
  kb_layout = "us,ru,ua",
  kb_variant = "lang",
  kb_options = "grp:win_space_toggle",
})

-- Language
hl.bind("SUPER + SPACE", hl.dsp.exec_cmd("~/.config/hypr/scripts/lang"))
```

电源控制（Power control）

Hyprland 需要一个与 Wayland 兼容的外部应用程序来进行电源控制。 以 nwg-barAUR 为例，我们只需按如下方式进行绑定：

```
~/.config/hypr/hyprland.lua
...
hl.bind("SUPER + ESCAPE", hl.dsp.exec_cmd("nwg-bar"))
...
```

剪贴板（Clipboard）

Wayland 剪贴板行为会在关闭复制数据的应用程序时删除数据。其他桌面环境通过使用专用的剪贴板管理器来解决这一问题，而 Hyprland 上则有多种兼容的选择。更多信息，请参阅上游 Wiki。

本节将介绍 cliphist包，因为它支持复制图片和文本：

```
~/.config/hypr/hyprland.lua
hl.on("hyprland.start", function()
    ...
    hl.exec_cmd("wl-paste --type text --watch cliphist store")
    hl.exec_cmd("wl-paste --type image --watch cliphist store")
    ...
end)
```

然后创建一个绑定，以便在你选择的应用程序启动器中调用历史记录：

```
~/.config/hypr/hyprland.lua
...
hl.bind("SUPER + V", hl.dsp.exec_cmd("cliphist list | wofi --dmenu | cliphist decode | wl-copy"))
...
```

现在按下 `Super+v` 将打开一个包含剪贴板历史列表的 `wofi` 窗口。

启用/禁用设备

要启用 / 禁用设备（例如触摸板），首先使用：

```
$ hyprctl devices
```

来获取你的设备名称。 将这些代码行放入你的配置文件中（将替换为上面查询到的你的设备名称）以打开 / 关闭设备：

```
~/.config/hypr/hyprland.lua
hl.device({
  name = "<device_name>"
  enabled = {true/false}
})
```

要动态地打开 / 关闭设备，请使用 hyprctl：

```
$ hyprctl eval "hl.device({ name = '<device-name>', enabled = {true/false) })"
```

你也可以创建一个快捷键绑定，例如：

```
~/.config/hypr/hyprland.lua
...
hl.bind("SUPER + T", hl.dsp.exec_cmd([[hyprctl eval "hl.device({ name = 'logitech-g502-hero-gaming-mouse', enabled = false})"]]))

hl.bind("SUPER + SHIFT + T", hl.dsp.exec_cmd([[hyprctl eval "hl.device({ name = 'logitech-g502-hero-gaming-mouse', enabled = true})"]]))
...
```

独立的 dconf 配置文件

您可能不想污染其他基于 GTK 的桌面环境的配置，您可以使用一个独立的 `dconf` 配置文件。例如：

声明新的全局 dconf 配置文件：

```
/etc/dconf/profile/hyprland
user-db:hyprland
~/.config/hypr/hyprland.lua
...
hl.env("DCONF_PROFILE", "hyprland")
...
```

现在您可以使用 *gsettings* 而不影响其他桌面环境。

KDE Plasma 应用不跟随Qt主题

你已经设置了一个Qt主题，但是一些plasma应用可能不会跟随主题。你可以安装qt6ct-kdeAUR，并且选择一个KColorScheme或者通过在kdeglobals中添加如下配置以解决：

```
~/.config/kdeglobals
[UiSettings]
ColorScheme=NameOfTheProfile
```

故障排除

Fcitx5 相关问题

提示：若还未能解决，请重启输入法！

无法输入

当你在任何时候输入中文时出现候选框不在输入框附近（如 QQ 等），而是漂移到 Waybar 的左上角，选择后也无法选词到输入框中，请在 Fcitx5 的配置页面中新建一个分组，选择键盘-英语（美国），然后再设置一个快捷键来轮回切换分组，切换后即可恢复正常

打开 Fcitx5 的配置页面你需要安装 fcitx5-configtool包。

切换英文时出现问题

当你在输入框中切换英文时，在 Waybar 左上角出现了输入法中文状态，且无法在输入框中输入英文，请重新启动 Fcitx5 以临时解决问题。

JetBrains 软件聚焦问题

JetBrains 软件（PyCharm，IntelliJ）会出现一个奇怪的聚焦问题，如：

- 无法将标签页从标签栏拖动来实现分离，或者另一个标签页堆叠时无法更改焦点，亦或者一个标签页只要被你拖动过当前标签栏就会停止拖动状态。
- 自动弹出的窗口直到鼠标移动时才被聚焦。

通过在Hyprland配置文件中写入以下行以缓解该问题：

```
~/.config/hypr/hyprland.lua
hl.window_rule({
     match = { xwayland = true },
     no_initial_focus = true,
 })
```

参见

- Hyprland 网站
- 官方文档

- Hyprland Github 仓库页面
- Community-maintained 工具、插件和扩展程序列表
- SolDoesTech Github 仓库——Hyprland 快速安装脚本（含 Hyprland 配置文件）

5.8 Sway

难度 高级 | **篇幅** 43.2 KB | **所属** Wayland 合成器

本节将讲到

- 安装
- 启动
- 配置
- 提示和技巧
- 疑难解答

Sway（缩写自 **S**irCmpwn's **Way**land compositor）是一款专为 Wayland 设计的合成器，旨在与 i3 完全兼容。根据官网所述：

: Sway 是 Wayland 的合成器，也是 x11 的 i3 窗口管理器的替代品。它可以根据您现有的 i3 配置工作，并支持 i3 的大部分特性以及一些附加功能

若您喜欢视觉效果，也可以试试 swayfxAUR，这是一个带有视觉效果的 Sway 分支。

Sway 的另一个分支 sway-scroll-gitAUR 具有滚动布局（如同 PaperWM 和 niri包）。还支持动画效果、窗口内容缩放和概览模式。

安装

安装 sway包。由于依赖关系紧密，请在更新 *sway* 时始终更新 *wlroots*。

注意：所有专有显卡驱动程序都不受支持，包括 NVIDIA。在 NVIDIA 驱动程序版本 495 之后，需要启用内核级显示模式设置并在运行 sway 时附加 `--unsupported-gpu` 参数。

您还可以安装 swaylock包、 swayidle包 和 swaybg包，用以锁定屏幕、设定空闲管理和设置壁纸。

默认的应用程序启动器为 wmenu包，默认的终端模拟器是 foot。在启动 *sway* 之前，建议安装它们或在配置文件中设置新的启动器和终端。您可在 Sway wiki 的迁移指南上找到一些 i3 实用程序的其他 Wayland 兼容替代品。

启动

Sway 在启动之前需要访问您的硬件设备（如键盘、鼠标和显卡），这些硬件设备的集合称为 seat，如同 sd-login(3) § DEFINITION OF TERMS 的 seat 小节所述。

在 Arch Linux 上，Sway 可以使用以下任一方式访问您的 seat。

- systemd-logind(8) 和 polkit包
- seatd包，会作为 wlroots0.20包的依赖与 Sway 一起安装

若 `polkit` 已安装在您的系统上， Sway 应会自动访问你的 seat。

若 `polkit` 未安装在您的系统上，并且您想使用 `seatd` 来替代，请将您添加到 `seat` 用户组并启用/启动 `seatd.service`，然后重新登录。

您可以选择以下方法之一来启动 Sway。

手动启动

要启动 Sway，只需在 Linux 终端执行 `sway`。

自动启动

与 X 类似，可以通过将以下内容添加到 shell 初始化文件来启动 Sway（请参阅命令行解释器#登录 Shell）：

```
if [ -z "$WAYLAND_DISPLAY" ] && [ -n "$XDG_VTNR" ] && [ "$XDG_VTNR" -eq 1 ] ; then
   exec sway
fi
```

有关更多详细信息，请参阅 Xinit#登录时自动启动 X。

使用显示管理器启动

Sway 会话位于 `/usr/share/wayland-sessions/sway.desktop`。它可以被 GDM 和 SDDM 等现代显示管理器自动识别。

也可以将 systemd 用户服务作为一个显示管理器去运行 sway。

您还可以使用基于文本的显示管理器，请参阅显示管理器#控制台。

配置

若您已在使用 i3，则可以将 i3 配置文件复制到 `~/.config/sway/config`，该文件开箱即用。否则，请将位于 `/etc/sway/config` 的示例配置文件复制到 `~/.config/sway/config`。有关配置的信息参见 sway(5)。

注意：用户配置文件中应当包含 `include /etc/sway/config.d/*` 一行，以便从引置文件（drop-in files）中读取并应用配置。sway包 提供了一个引置文件 `50-systemd-user.conf`，其会将多个环境变量导入 systemd 用户会话和 dbus。一些应用需要这些环境变量以正常运行（如xdg-desktop-portal-wlr包）。

键盘布局

默认情况下，sway 从 US QWERTY 键盘布局开始。配置多个键盘布局：

```
~/.config/sway/config
input * {
    xkb_layout "us,de,ru"
    xkb_variant "colemak,,typewriter"
    xkb_options "grp:win_space_toggle,caps:none"
}

input <identifier> xkb_model "pc101"
```

更多细节请参阅 xkeyboard-config(7) 与 sway-input(5)。

在启动 `sway` 时，也可以使用环境变量（`XKB_DEFAULT_LAYOUT`、`XKB_DEFAULT_VARIANT` 等）配置键盘布局。配置选项优先于环境变量。

打字延迟和速率

要更改打字延迟和速率，您可以在 `input` 节添加以下行：

```
~/.config/sway/config
input <identifier> repeat_delay 300
input <identifier> repeat_rate 30
```

状态栏

Sway 附带了一个默认状态栏，名为 *swaybar* ，可在纯 Wayland 环境中运行。*swaybar* 可以调用 shell 脚本或其他程序来在状态栏中显示信息。有关详细信息，请参阅 sway-bar(5) 与 swaybar-protocol(7)。

提示：*swaybar* 对托盘图标的支持并不完整，请考虑使用 waybar包 等替代方案，在 `bar` 节设置 `swaybar_command waybar` 即可启用。

i3status 是在 Wayland 下获取实用的默认状态栏的一个选项，您只需在 sway 配置的末尾添加以下代码段：

```
~/.config/sway/config
bar {
    status_command i3status
}
```

欲为 i3status 启用色彩输出，请在 i3status 配置中调整以下部分：

```
~/.config/i3status/config
general {
    colors = true
    interval = 5
}
```

Output

`sway` 的 `output` 命令可以为不同的显示输出指定详细配置，包括但不限于壁纸、缩放、位置等。如有需要，您可将不同的 output 命令整合到一行，例如：

```
~/.config/sway/config
output HDMI-A-1 mode 1920x1080 pos 1920 0 bg ~/wallpaper.png stretch
```

output 有多种方式匹配显示输出。可以使用其指定的输出名称，也可以使用 "*" 匹配所有显示输出，或是使用显示器的不同名称（由品牌、型号和序列号组成的字符串），例如：

```
~/.config/sway/config
output "Some Company ABC123 0x00000000" pos 1920 0
```

可以使用以下命令获取输出名称和一些其他信息：

```
$ swaymsg -t get_outputs
```

要深入了解配置和附加选项，请参阅 sway-output(5)。

壁纸

sway 用专用程序处理壁纸，最简单的例子是 swaybg包，其可直接由 `sway` 管理。若要运行 `output ... bg` 命令，请安装 swaybg包。

可以将以下行附加在 sway 配置的任意位置，其会在所有显示器上设置背景图像：

```
~/.config/sway/config
output "*" bg /path/to/image fill
```

当然，`/path/to/image` 应当被替换为存在的图像路径。

如果只需要纯色背景，可以按如下方式设置：

```
output * bg #000000 solid_color
```

请参阅 Sway wiki 以了解更多用于管理壁纸的工具。

HiDPI

注意：Xwayland 也会被缩放，且会降低分辨率。欲以原始分辨率运行 Xwayland 下的程序，则需禁用缩放。您可使用一些工具临时禁用缩放，如使用 swaymsg：

```
swaymsg output "*" scale 1
```

欲恢复到自动设定则设为 `0`。

自动设定

Sway 默认使用整数倍缩放。若满足以下条件：

- 屏幕通过 EDID（ Extended Display Identification Data，外部显示设备标识数据）提供有效的物理显示信息。
- DPI（每英寸像素数）至少为 192。
- 屏幕分辨率的高至少为 1200 像素。

那么 Sway 会使用 2 倍缩放。一些设备（如 Framework Laptop 16）的 DPI 近似（但不是）192，在这种情况下，您可能需要手动配置分数倍缩放。

手动设定

在配置文件里用 `output` 命令设置缩放倍数。可以设置分数倍缩放，不过 HiDPI 屏幕通常为 2。

```
~/.config/sway/config
output <名称> scale <倍数>
```

可以使用以下命令获知显示器名称：

```
$ swaymsg -t get_outputs
```

撕裂

Sway 默认会同步帧更新（vsync，垂直同步）以使输出画面的每一帧都帧完整而不撕裂，但是会带来一些延迟。在一些使用场景（如游戏）下禁用垂直同步可以使输出更为顺畅：

```
~/.config/sway/config
output <name> allow_tearing yes
output * max_render_time off
```

输入设备

可以调整特定输入设备的配置。例如，若要为所有触摸板启用点击以单击和自然滚动，请使用以下配置：

```
~/.config/sway/config
input type:touchpad {
    tap enabled
    natural_scroll enabled
}
```

要为特定触摸板设定配置，请使用 `swaymsg -t get_inputs` 获取设备标识符并用其代替上述配置中的 `type:touchpad`。

注意：命令 `swaymsg -t get_inputs` 的输出可能会使用`\`来转义`/`等符号（如 `"2:14:ETPS\/2_Elantech_Touchpad"`），配置时需要删除`\`。

关于更多文档和选项（如加速度或完全禁用输入），请参阅 sway-input(5)。

若您使用数位板，请参阅 Graphics tablet#Sway（英语：Graphics tablet#Sway）。

触摸屏映射

在多显示器环境中，触摸屏的触摸输入目标可以仅被映射到该触摸屏。

```
~/.config/sway/config
set $display1      "Dell Inc. DELL P2414H VHVTW542165L"
set $display2      "Dell Inc. DELL P2418HT MYDM775F152L"
set $display2-touch "8146:24835:Melfas_LGD_AIT_Touch_Controller"

input $display2-touch map_to_output $display2
```

自定义快捷键

键盘上的特殊按键可用于执行命令，如控制音量、屏幕亮度和媒体播放：

```
~/.config/sway/config
```

```
# 将特定按键绑定到 pactl 以调整 PulseAudio 音量
bindsym --locked XF86AudioRaiseVolume exec pactl set-sink-volume @DEFAULT_SINK@ +5%
bindsym --locked XF86AudioLowerVolume exec pactl set-sink-volume @DEFAULT_SINK@ -5%
bindsym --locked XF86AudioMute exec pactl set-sink-mute @DEFAULT_SINK@ toggle
bindsym --locked XF86AudioMicMute exec pactl set-source-mute @DEFAULT_SOURCE@ toggle

# 将特定按键绑定到 playerctl 以控制媒体
bindsym --locked XF86AudioPlay exec playerctl play-pause
bindsym --locked XF86AudioPause exec playerctl play-pause
bindsym --locked XF86AudioPrev exec playerctl previous
bindsym --locked XF86AudioNext exec playerctl next
bindsym --locked XF86AudioStop exec playerctl stop

# 将特定按键绑定到 brightnessctl 以控制屏幕亮度
bindsym --locked XF86MonBrightnessDown exec brightnessctl set 5%-
bindsym --locked XF86MonBrightnessUp exec brightnessctl set 5%+

# 将特定按键绑定到 grim 以截图
bindsym Print exec grim

# 将特定按键绑定到 wmenu-run 以启动应用程序
bindsym XF86Search exec wmenu-run
```

有关详细信息和替代实用程序，请参阅：

- PulseAudio#键盘控制音量
- WirePlumber#键盘控制音量
- ALSA#键盘控制音量
- 背光#背光实用程序
- MPRIS。

提示：`bindsym` 的 `--locked` 参数用于允许在锁屏下执行快捷键。

提示：wev包 工具提供类似于 xorg-xev包 的功能，但运行于 Wayland。

注意：Systemd 会处理一些特殊按键，如电源键和盖子打开/关闭事件。这可能会干扰在 sway 中配置的快捷键。请参阅 loginctl(1) 与 logind.conf(5) 以获取有关如何在 systemd 中配置它们的详细信息。

图形指示条

在调整某些百分比值设置（如亮度或音量）时，可以显示一个图形指示条。在 Sway 中提供此工具的一个不错的选择是 wob包（或 wob-gitAUR），其是 Wayland 上 layer-shell 协议的实现，提供了流行的 X 工具 xobAUR 的部分功能。有关使用示例，请参阅项目网站。

工作区概览

如果您在工作区开启了很多窗口，导致难以分辨，那么 sovAUR 可以派上用场。它是一个叠加层，可以显示所有工作区的概览，使 sway 导航更容易。其可以显示程序名称、窗口标题，支持多 output 配置。关于详细信息，请参阅项目网站。

空闲管理

Sway 有一个名为 swayidle包 的专用空闲管理守护程序来处理空闲会话。可以用不同的方法来启动和参数化守护程序。最简单的方法是使用 sway 本身的配置。`swayidle` 接受大量参数来配置 `timeout`（超时，又名“空闲”）、`resume`（恢复，超时后从空闲中恢复）、`before-sleep`（睡眠前）等事件，请参阅 swayidle(1) 了解更多细节和事件的进一步解释。然后，可以为每个事件分配一个操作。要为一个事件分配多个操作，只需重复触发器即可。

以下配置可使 `swayidle` 在 30 分钟后锁定屏幕，并在 5 秒后将其关闭：

```
~/.config/sway/config
exec swayidle -w \
       timeout 1800 'swaylock -f' \
       timeout 1805 'swaymsg "output * power off"' \
              resume 'swaymsg "output * power on"'
```

要更快地关闭锁定屏幕，例如在10秒后，请为锁定管理器设置进程列表，并相应地执行 `swaymsg "output * power off"`，如下：

```
timeout 10 'if pgrep -x swaylock; then swaymsg "output * power off"; fi' resume 'if pgrep -x swaylock; then swaymsg "output * power on"; fi'
```

要在屏幕锁定前挂起和暂停播放媒体，请将以下配置添加到 swayidle 命令：

```
before-sleep 'playerctl pause;swaylock -f'
```

注意：Systemd 也处理一些空闲事件，这些事件可能与在 sway 中配置的事件冲突。请参阅 loginctl(1) 和 logind.conf(5) 以获取关于如何配置它们的详细信息。

若您不希望在 Firefox、Chrome 或 VLC 播放视频时触发 swaylock，您可以用 idlehack-gitAUR 监听 dbus 的 screensaver inhibit（阻止屏保）请求并调用 swayidle-inhibit。Firefox、Chrome 和 VLC 等程序会发出该事件以阻止系统空闲。

恢复时屏幕内容短暂显示

使用 `before-sleep swaylock` 可能会使系统从睡眠中恢复时屏幕内容短暂显示。欲缓解该问题，您可使 logind 忽略盖子（lid）事件并改用 Sway 处理盖子和电源按钮事件。

```
/etc/systemd/logind.conf
[Login]
HandlePowerKey=ignore
HandlePowerKeyLongPress=ignore
HandleSuspendKey=ignore
HandleLidSwitch=ignore
HandleLidSwitchExternalPower=ignore
HandleLidSwitchDocked=ignore
~/.config/sway/config
bindswitch --locked lid:toggle exec swaylock --daemonize && systemctl sleep

exec swayidle -w \
         timeout 60  'if pgrep swaylock; then systemctl sleep; fi' \
         timeout 150 'swaylock --daemonize' \
         timeout 155 'systemctl sleep'
```

使用 `--locked` 选项以在屏幕锁定时合盖休眠系统。若要使笔记本在合盖后依然工作，使用 `unbindswitch lid:toggle`。可以使用 `pgrep swaylock` 以使电脑被意外唤醒后再次休眠。

logind 集成 bug 较多且难以维护。另见 。

悬浮窗口和窗口分配

要启用悬浮窗口或窗口分配，请打开应用程序，然后使用 `app_id`、`class`、`instance` 和 `title` 属性来启用悬浮窗口/窗口分配。以下命令可以列出所有打开的窗口的属性：

```
$ swaymsg -t get_tree
```

欲获取所有打开的窗口的 `app_id`，请执行：

```
$ swaymsg -t get_tree | grep "app_id"
```

仅获取当前选中窗口的 `app_id`，请执行：

```
$ swaymsg -t get_tree | jq -r '..|try select(.focused == true)'
```

X11 窗口没有 `app_id` 属性，要匹配它们，您可以使用 `class`、`window_type`、`window_role` 和/或 `instance` 等属性。请在 `swaymsg -t get_tree` 命令的输出中查找相关属性并为窗口创建足够详细的规则。

```
~/.config/sway/config
for_window [app_id="galculator"] floating enable
for_window [window_type="dialog"] floating enable
for_window [window_role="dialog"] floating enable
assign [app_id="firefox"] 3
assign [class="^Urxvt$" instance="^htop$"] 9
```

这类似于在X11中使用 xorg-xprop包 查找 `class` 或 `wm_name` 属性。

使用多个显示器时，悬浮的 scratchpad（暂存器）窗口可能会变得很大，覆盖多个显示器。可以使用以下命令将悬浮窗口居中并调整为当前显示器大小的 80%：

```
$ swaymsg move position center; swaymsg resize set 80ppt 80ppt
```

剪贴板

参见剪贴板。

Xresources

在 Sway 中使用 `~/.Xresources` 需要把它拷贝到`~/.Xdefaults`。

Xwayland

请参阅 Wayland#Xwayland 以了解详细信息和可用软件包的概述。

默认情况下 Xwayland 处于启用状态。

欲完全禁用 Xwayland 并运行“纯”Wayland 会话，请使用以下配置以禁用 Xwayland：

```
~/.config/sway/config
```

```
xwayland disable
```

若您想一眼知晓哪些窗口使用 Xwayland，可以使用以下配置：

```
~/.config/sway/config
for_window [shell="xwayland"] title_format "[XWayland] %title"
```

注意：一些程序需要特殊的环境变量或配置选项才能在原生 Wayland 下运行，而另一些程序（包括大多数专有应用程序）根本不支持 Wayland。目前，建议保持启用 Xwayland，以便使用旧版应用程序。

使用另一个 wlroots 渲染器

要使用另一个渲染器如 Vulkan，请参阅 Wayland#在基于 wlroots 的混成器上使用另一个渲染器。

程序自启动

参见 i3#自启动程序章节，并将配置文件名调整为适用于 Sway 的版本。

提示和技巧

启动时启用 NumLock

Sway 在启动时默认禁用 `NumLock`。欲使其自动启用，请将键盘的 `xkb_numlock` 输入配置设置为 `enable`。例如，欲使所有键盘如此，请将以下行添加到 Sway 配置：

```
~/.config/sway/config
input type:keyboard xkb_numlock enabled
```

任何情况下，都可以通过键盘上的相关按键来切换 `NumLock` 状态。

提示：如需，也可以用 `xkb_capslock` 以相同方式设置 `CapsLock`。

注意：用通配符启用这些可能会导致在重新加载 Sway 配置文件时 Firefox 崩溃，参见 Bugzilla 1652820，请尝试指定特定键盘而非通配符。

当前键盘布局

可以按如下方式通过 jq包 检索当前键盘布局，其中 `*kbd_identifier*` 需要替换为键盘的标识符：

```
$ swaymsg -t get_inputs | jq -r '.[] | select(.identifier == "kbd_identifier")
| .xkb_active_layout_name'
```

组合键

组合键（compose key）和其它按键结合使用可用于输入键盘不支持的字符。例如将 `PrintScreen` 设为组合键：

```
$ swaymsg 'input * xkb_options compose:prsc'
```

可在 Xorg/Keyboard configuration#Configuring compose key（英语：Xorg/Keyboard configuration#Configuring compose key） 查看可用的组合。组合也可以在 XCompose 文件（英语：Xorg/Keyboard configuration#Key combinations）中配置，需要重新启动应用程序才能使更改生效。

背光切换

若要使用某按键（如 `Pause`）关闭（和打开）显示器，请在 Sway`config` 中绑定以下脚本：

```
#!/bin/sh
read lcd < /tmp/lcd
    if [ "$lcd" -eq "0" ]; then
        swaymsg "output * power on"
        echo 1 > /tmp/lcd
    else
        swaymsg "output * power off"
        echo 0 > /tmp/lcd
    fi
```

也可以直接使用 toggle 选项，只是使用多显示器时需要明确指定 output：

```
$ swaymsg "output output_name power toggle"
```

屏幕捕获和屏幕共享

请查看屏幕捕获#Wayland.

色温调节

请查看背光#Wayland.

色彩管理

可以将以下行添加到 Sway 配置以使用色彩管理配置文件：

```
~/.config/sway/config
output * 色彩管理配置文件 icc /path/to/your/color_profile.icc
```

用键盘控制 swaynag

Swaynag 是 sway 附带的默认警告/提示程序，仅支持用户用鼠标与之交互。诸如 swaynagmodeAUR 之类的帮助程序可通过键盘快捷键启用交互。

Swaynagmode 的工作原理是首先启动 swaynag，然后侦听触发操作的信号，例如选择下一步按钮、关闭提示或接受所选按钮。这些信号会被另外的带有控制参数的 swaynagmode 脚本实例（如 `swaynagmode --select right` 和 `swaynagmode --confirm`）发送。

默认情况下，Swaynagmode 在初始化时触发 sway 的 `nag` 模式，在退出时返回 `default`。这样，您就可以在 sway 配置中轻松定义快捷键：

```
~/.config/sway/config
set $nag exec swaynagmode
mode "nag" {
  bindsym {
    Ctrl+d    mode "default"

    Ctrl+c    $nag --exit
    q         $nag --exit
    Escape    $nag --exit

    Return    $nag --confirm
```

```
    Tab       $nag --select prev
    Shift+Tab $nag --select next

    Left      $nag --select next
    Right     $nag --select prev

    Up        $nag --select next
    Down      $nag --select prev
  }
}
```

注意：从 sway 版本 1.2 开始，模式名称区分大小写。

您可将 sway 配置为通过配置命令 `swaynag_command swaynagmode` 使用 swaynagmode。

更改光标主题和大小

要设置光标主题和大小，请使用以下配置：

```
~/.config/sway/config
seat seat0 xcursor_theme 光标主题 光标大小
```

其中*光标主题*可以设置或替换为特定名称（如 `default`、`Adwaita` 或 `Simple-and-Soft`），以及*光标大小*值如 `48`。

要查看二者的值，可以使用 `echo $XCURSOR_SIZE` 和 `echo $XCURSOR_THEME`。

更改配置后需要重新加载 Sway 以使更改生效。

注意：Wayland 光标可由应用程序确定，这意味着应用程序可能不会遵循 `$XCURSOR_SIZE` 和 `$XCURSOR_THEME` 的值。

使用 systemd 管理仅用于 Sway 的守护程序

Systemd 提供 `graphical-session.target` 和 `graphical-session-pre.target` 用户单元。

Services can be made `PartOf=` either unit to let them run as long as or before, respectively, any graphical session is running *and* terminate them cleanly when exiting it, whether Xorg or Wayland.

If Sway is the only window manager on a machine, then `graphical-session.target` may be sufficient even for Sway-specific services (like swayidle包 or kanshi); but otherwise, a more specific `sway-session.target` file can be used (such as the one provided by sway-contrib包).

Additionally, services started by systemd need to be informed of the necessary environment variables; this is taken care of by a file in `/etc/sway/config.d/` provided by sway包, which should be processed if the note at the top of #Configuration has been followed. (But those environment variables should also be

unset when exiting Sway, otherwise it or other services may fail to start after the first exit.)

All this can be done using sway-systemd-gitAUR or uwsm包 (which also take care of #Working with systemd-oomd), or manually.

注意：sway-services-gitAUR has unit startup order issues.

手动配置

将以下行添加到 Sway 的配置文件中（例如，将该行追加到 `~/.config/sway/config`或将新文件添加到 `/etc/sway/config.d/`）：

```
~/.config/sway/config
...
...
...
exec_always systemctl --user start sway-session.target
```

This ensures that `sway-session.target` is activated whenever Sway starts, and *after* it has created the Wayland socket etc. Make sure to place that line after any commands that affect how systemd services would be started, which generally includes the `include /etc/sway/config.d/*` directive.

Then, arrange for the target to be stopped before Sway exits. Note that `systemctl --user stop sway-session.target` will not work, as the command returns before the services are done shutting down; consider creating a unit that `Conflicts=` with that, and starting it instead .

Finally, link the desired services to `sway-session.target` or `graphical-session.target`:

- If an existing service file exists, you can enable it, or (such as if that would not link to the right unit) run `systemctl --user add-wants *sway-session*.target *foo*.service`.
- You can find an example of creating a brand new service file at kanshi#Manage kanshi with systemd.

Working with systemd-oomd

systemd-oomd(8) tracks resource usage by and kills entire Cgroups at once, not processes. This means that, by default, a single memory-hungry service will cause the entire Sway session to be terminated.

The solution is to start each application in its own cgroup. Some of the packages recommended in #Manage Sway-specific daemons with systemd, like uwsm包 or sway-systemd-gitAUR, already take care of this. But it can also be accomplished manually by ensuring that applications are started via *systemd-run* (see systemd-run(1)).

Instead of using a `swaymsg exec -- *foo*` command, create a script:

```
~/.local/share/sway_exec_wrapper
#!/usr/bin/env sh
exec systemd-run --user --ignore-failure --scope --unit "app-$1-$(uuidgen)" -- "$@"
```

and use `swaymsg exec ~/.local/share/exec_wrapper -- *foo*`.

提示：If #Manage Sway-specific daemons with systemd has been followed, add `-p PartOf=graphical-session.target` to let the application exit cleanly when exiting Sway.

The `--ignore-failure` option is used, because a graphical application's unclean exit should not cause the system state to become "degraded"; `--scope` so that the initial process can fork without the entire application being closed *and* for Sway to assign the first window to the correct workspace. The `--unit` parameter is named according to systemd's recommendations.

启动后更改屏幕分辨率/布局

您可使用 GUI 程序 nwg-displays包、wdisplays包、swayrandr-gitAUR 或终端程序 wlr-randr包 来更改屏幕分辨率和方向、排列显示器以及设定缩放。

创建无外设（Headless）输出

创建与物理 video 接口无关的输出 （`HEADLESS-1`、`HEADLESS-2`等）：

```
$ swaymsg create_output
```

显示新输出的说明：

```
$ swaymsg -pt get_outputs | grep -A 10 HEADLESS
```

使用 `output` 命令配置新的输出，例如：

```
~/.config/sway/config
output HEADLESS-1 {
pos 1920,0
mode 1280x720@75Hz
}
```

将修饰键（modifier）修改为 CapsLock 并保留 Super

要将修饰键更改为 `CapsLock` 并在 US 键盘布局上继续使用 `Super`，请创建 `~/.config/xkb/symbols/custom`，具有以下内容：

```
~/.config/xkb/symbols/custom
xkb_symbols "basic" {
    include "us"
    name[Group1]= "English (US Custom)";
    key <CAPS> { [ Hyper_L ] };
    modifier_map Mod4 { Hyper_L };
    key <LWIN> { [ Super_L ] };
    modifier_map Mod5 { Super_L };
};
```

对于其他语言，请编辑第二行和第三行，并在 Sway 配置文件开头附加引用该键盘布局：

```
~/.config/sway/config
input * xkb_layout custom
set $mod Mod4
set $super Mod5
```

通过 VNC 远程访问无外设（Headless） Sway 会话

以下示例教程讲述了通过 SSH 启动并通过 VNC 远程访问无外设 Sway 会话。

wayvnc包 是一个 VNC 服务器，适用于基于 wlroots 的 Wayland 混成器，可以用于承载无外设 Sway 会话。

要启动一个无外设 Sway 会话，请将 `WLR_BACKENDS` 设为 `headless`，可选将 `WLR_LIBINPUT_NO_DEVICES` 设为 `1`（参见 wayvnc FAQ）：

```
$ WLR_BACKENDS=headless WLR_LIBINPUT_NO_DEVICES=1 sway
```

提示：在命令结尾添加 `&` 以使其后台运行。

现在，请运行 `wayvnc` 以启动 VNC 服务器：

```
$ WAYLAND_DISPLAY=wayland-1 wayvnc
```

注意：默认情况下，wayvnc包 仅会监听本机 5900 端口（`localhost:5900`），可以另行配置（参见 `man wayvnc`）。您可能会想按照 OpenSSH#转发其他端口 所述转发端口。请注意 wayvnc包 默认**不加密且不验证身份！**

输出热插拔钩子

若您想在一个输出连接（或断开连接）时搞点事情，可以阅读 kanshi。

疑难解答

应用程序启动器

提示：Sway wiki 包含一个已知应用程序启动器列表。

以下应用程序启动器已知和 Sway 集成良好：

- wmenu包 or bemenu包 are largely drop-in replacements for `dmenu`.
- rofi now supports Wayland natively, but wofi包 is still a good alternative.
- fuzzel包 is an application launcher similar to rofi's *drun* mode.
- Any `dmenu` replacement can be combined with j4-dmenu-desktop包's `--dmenu` option to make an equivalent of i3-dmenu-desktop(1).
- krunner(1) (from plasma-workspace包) can be used stand-alone.
- You can also build your own with a floating terminal and fzf包.

dmenu包 and other non-Wayland launchers also function well in Sway, but run under Xwayland and suffer from an issue where they can become unresponsive if focus is moved to a native Wayland window.

If Xwayland is enabled and bemenu包 does not run under Wayland by default (thus being subject to the above issues), set the `BEMENU_BACKEND=wayland` environment variable.

虚拟化

Sway 可以在 VirtualBox 和 VMware ESXi 中使用。

要使 Sway 在 QEMU 中工作，必须以 `-vga qxl` 参数启动QEMU。另见 QEMU#qxl。

无法从 tty 启动 Sway

对于 ESXi， 您需要在 *Hardware Configuration* > *Video card settings* 下启用 3D 支持。另见 VMware#在 Intel、Optimus 和 AMD 上启用 3D 图形加速。

光标不可见

当使用某些图形驱动程序（例如VMSVGA图形控制器或专有的 NVIDIA 驱动）时，光标可能会变得不可见。可以使用软件光标来解决此问题，具体讨论参见 Issue 3814：

```
$ export WLR_NO_HARDWARE_CURSORS=1
```

未检测到 sway socket

使用 `swaymsg` 时（如 `swaymsg -t get_outputs`）有时会返回以下消息：

```
sway socket not detected.
ERROR: Unable to connect to
```

当在终端多路复用器（如 GNU Screen 或 tmux）内运行时，`swaymsg` 无法连接到 `SWAYSOCK` 中提供的套接字。

要查看当前 `SWAYSOCK` 的值，请执行：

```
$ env | grep -F "SWAYSOCK"
SWAYSOCK=/run/user/1000/sway-ipc.1000.4981.sock
```

欲解决该问题，您可以尝试连接到一个正在运行的 sway 进程的套接字：

```
$ export SWAYSOCK=/run/user/$(id -u)/sway-ipc.$(id -u).$(pgrep -x sway).sock
```

欲避免此错误，请在终端多路复用器之外运行该命令。

无法连接 Wayland 服务器

Tmux 会为每个会话创建环境变量（执行 `tmux show-environment` 以查询），因此，若您想要使用 tmux-resurrect 或 tmux-continuum 重新恢复一个已有的 tmux 会话，或 tmux 服务器在 sway 启动前就在运行，环境变量就会过期。

将以下内容添加到 `.tmux.conf`，之后您可以在恢复会话前使用 `update-environment` 更新环境变量：

```
set-option -g update-environment "DISPLAY WAYLAND_DISPLAY SWAYSOCK SSH_AUTH_SOCK"
```

无法获取 socket 路径

从 `swaymsg -t` 命令请求在tty上的消息可能会返回以下消息：

```
Unable to retrieve socket path
```

`SWAYSOCK` 环境变量在启动Sway后设置，因此解决此错误的一个方法是在Sway内部的终端中请求 `swaymsg -t [message]`。

按键设置和键盘布局

默认情况下，如果您使用多个键盘布局（如 `input * xkb_layout "us,ru"`），在切换到某个次要布局时可能会导致快捷键失效。

得益于 Pull 3058 ，现在您只需在相应的 `bindsym` 行中添加 `--to-code`，如下：

```
bindsym --to-code {
  $mod+$left focus left
  $mod+$down focus down
  $mod+$up focus up
  $mod+$right focus right
}
```

亦可创建一个变量 `set $mybind bindsym --to-code`，然后将所有的 `bindsym` 替换为 `$mybind`，如下：

```
$mybind $mod+w thing
```

Java 应用程序

当打开一些基于Java的应用程序（如 JetBrains的 IntelliJ、CLion 或 PyCharm）时，可能会显示空白画面。欲解决该问题，可以将环境变量 `_JAVA_AWT_WM_NONREPARENTING` 设为 `1` 来启动应用程序。

若您从应用程序启动器（如 rofi 或 dmenu）启动应用程序，则可以修改应用程序的桌面项。

OpenJDK 11 和 Sway 1.5 已经修复了一些 Java 应用程序的问题。然而，对于某些应用程序，需要进行额外的配置才能使用更新版本的 OpenJDK。对于 Android Studio，您需要设置 `STUDIO_JDK=/usr/lib/jvm/java-11-openjdk/`。

JRE 硬编码了一个窗口管理器列表，但该列表不包含 Sway，若您遇到灰色面板、菜单无焦点或窗口尺寸调整错误等问题，可以尝试伪装成其它窗口管理器。

边框上的滚动

如果在应用程序边框上使用鼠标滚轮导致Sway崩溃，您可以为 `app_id`（例如 Firefox）使用`border none` 来禁用边框。

“cannot open display” 错误

如果一个程序启动后出现“cannot open display”（无法打开显示器）并崩溃，那么该应用程序很可能是 X11 程序。您可以使用 Xwayland 兼容层在 Wayland 下运行 X11 程序，详见 #Xwayland。

WINE 中鼠标不工作

运行程序时，WINE 需要设定一个主显示器（primary monitor），但是 Wayland 没有主显示器的概念，这会引发问题（如 “clicks not registering”）。您可以使用 xrandr 为 Xwayland 指定主显示器，只需向您的 Sway 配置添加以下内容：

```
~/.config/sway/config
...
exec_always xrandr --output XWAYLAND0 --primary
...
```

若如此做，因为 Sway 的一个 bug，您需要将屏幕显示偏移量设为 `0,0`。

需要注意 XWAYLAND0（或其它 XWAYLAND 显示名称）可能不代表您的显示器，其后缀数字也可能在不同会话中变化。可以使用以下配置，其可将第一个 XWAYLAND 显示设定为主显示器：

```
exec_always xrandr --output $(xrandr | grep -m 1 XWAYLAND | awk '{print $1;}') --primary
```

您可按需调整此配置，使用不带参数的 `xrandr` 命令可以查看各

注意：Xwayland 会把新连接的显示器识别为全新的显示器，这包括了关闭后又重新打开的显示器。若您设置的“主显示器”也被关闭后又打开，其将不再被视为主显示器。

NVIDIA GPU 上游戏闪烁

即便官方不支持 NVIDIA 之类的专有驱动，但依然可以使用，甚至可以用来玩游戏。若如此做，您可能会看到屏幕上半区闪烁，可以尝试将 wlroots0.19AUR 替换为 wlroots-nvidiaAUR，也可以手动用补丁编译您自己的版本。

System freezes when toggling output power on hybrid NVIDIA setups

On hybrid-GPU laptops with an external monitor connected to the NVIDIA GPU, using `swaymsg "output * power off"` (e.g. via swayidle包) can deadlock `nvidia-modeset`, freezing the whole desktop. Confirm with `journalctl -b -1 -g "blocked for more than"`: a `task sway state:D ... blocked on a semaphore likely last held by task nvidia-modeset` line means this is the cause. Workaround: avoid powering off NVIDIA-attached outputs, e.g. target only the integrated GPU's output or suspend the system instead. See wlroots#2474.

屏幕捕获/WebRTC 不工作

确保您的配置包含以下内容：

```
include /etc/sway/config.d/*
```

然而，AUR 的部分 -git 软件包不提供 `/etc/sway/config.d`，此时您需要添加：

```
exec dbus-update-activation-environment DISPLAY I3SOCK SWAYSOCK WAYLAND_DISPLAY XDG_CURRENT_DESKTOP=sway
```

参见

- Sway 官方网站
- Sway GitHub 项目
- Sway wiki
- Sway 1.0 发布

堆叠式窗口管理器

堆叠式窗口管理器只做一件事：没有桌面环境时，用一个管窗口边框、键绑定和菜单的小程序，把 X 会话搭成顺手的工作台。建议先读《Openbox》，配置、重新配置、键绑定与菜单这条主线最完整；《Fluxbox》可做动手对照，《IceWM》补一条传统桌面习惯的路线。

《Blackbox》《Twm》《2bwm》篇幅短，浏览即可；《Compiz》《FVWM》体量最大，按需再翻。最容易踩的坑是把配置当通用格式：各家语法互不通用，抄来的例子常连启动都撑不住；再就是改完配置就重启会话，而重新配置自有专门做法。

6.1　Blackbox

难度 入门 | **篇幅** 1.8 KB | **所属** 堆叠式窗口管理器

本节将讲到

- 安装
- 启动
- 实用工具
- 参见

Blackbox 是一个轻量级的堆叠式窗口管理器。它的最新更新时间为 2005 年 11 月 3 日，没有未来版本的计划，但是它的设计启发了许多分支和精神继承者。BlackBox Github 上维护着原始 CVS 源的最新版本以及所收集的软件包，并于 2021-05-12 发行了最新版本 0.77。

安装

安装 blackbox包 包。

启动

使用 xinit 运行 `blackbox`。

实用工具

还有一些其他软件包可以增强 Blackbox 的功能（尽管它们也可以与其他极简主义的窗口管理器一起很好地工作）：

- bbrunAUR 用于启动程序的运行对话框
- bbpager包 传呼机
- bbkeysAUR 配置键盘快捷键

参见

- Blackbox Wiki - 官方文档
- HOWTO: A Blackbox Guide - Ubuntu 论坛

6.2 Twm

难度 入门 | **篇幅** 4.1 KB | **所属** 堆叠式窗口管理器

本节将讲到

- 安装
- 启动
- 配置
- 提示与技巧
- 故障排除

twm 是 Xorg 的窗口管理器。这是一个小程序，是针对 Xlib 构建的，而不是使用窗口小部件库构建的，因此，它对系统资源的需求很小。尽管很简单，但是它是高度可配置的。字体，颜色，边框宽度，标题栏按钮等均可由用户设置。

twm 由 Tom LaStrange 编写，这个人对 uwm (Ultrix Window Manager)) 的局限感到沮丧，后者是 X11 首次发布时唯一的窗口管理器。 twm 取代了 uwm 作为 1989 年 X11R4 版本 X11 随附的默认窗口管理器。

twm 代表 *Tom's Window Manager*， *Tab Window Manager* 和最近的 *Timeless Window Manager*。

安装

twm 由 xorg-twm包 提供。

启动

使用 `twm` 运行 xinit。

注意：启动时只有黑屏。尝试移动鼠标并单击鼠标左键以获取 twm 菜单，以确保 twm 确实有用。

您还可以使用显示管理器启动 twm。该 `twm.desktop` 文件不存在，因此我们必须在 `/usr/share/xsessions/` 创建它。在新创建的 `/usr/share/xsessions/twm.desktop` 文件中，复制并粘贴：

```
/usr/share/xsessions/twm.desktop
[Desktop Entry]
Name=twm
Comment=xorg-twm
TryExec=twm
Exec=twm
Type=Application
```

配置

默认情况下，twm 看起来非常过时且不直观。 通过创建 `~/.twmrc` 文件，您可以自定义 twm 以使其更友好。

twm(1) 提供了可以在 `~/.twmrc` 文件中使用的命令的完整详细信息。 许多 `~/.twmrc` 文件已在线发布。xwinman.org 网站上有多个 `~/.twmrc` 文件，这些文件可能带有启发性的屏幕截图。 用 Google 搜索“twmrc”可以找到新的想法。

提示与技巧

修补版本

有一个修补程序版本，不在存储库中，具有更新功能，例如透明度。xorg 邮件列表中提供了描述和构建脚本。可以通过安装 xcompmgr 运行构建脚本，将生成的 `twm` 和 `dot.twmrc` 文件放在一个方便的目录中，然后编辑 `~/.xinitrc` 文件来进行尝试，最后两行是

```
xcompmgr -o 0.3  -c -r 8 -t -10 -l -12 &
/path-to-directory/twm -visual TrueColor -depth 32 -f /path-to-directory/dot.twmrc
```

故障排除

超大的窗口标题和菜单

您可能会发现 TWM 中的标题栏和菜单条目非常大-是人们通常期望的两倍。 这是使用 `UTF-8` 语言环境时发生的 TWM 语言环境问题。将语言环境设置为 `C` 可解决此问题。参见 。

另请参见

- "UWM (computing))", *Wikipedia*. Retrieved October 22, 2009.
- "twm", *Wikipedia*. Retrieved October 22, 2009.
- twm(1) man page
- "Sample twmrc", *custompc.plus.com*. Retrieved August 12, 2013.
- "Window Managers for X: TWM/VTWM", *xwinman.org*. Retrieved October 22, 2009.
- Kask, Eeri. "TWM -- Revised Edition -- Again", *lists.x.org*, January 3, 2010.

6.3 2bwm

难度 入门 | **篇幅** 4.5 KB | **所属** 堆叠式窗口管理器

本节将讲到

- 安装
- 开始
- 使用2bwm

- 提示和技巧
- 参见

2bwm 是一个快速的浮动窗口管理器，其特点是有两个边框，是基于XCB库写的，并派生自Michael Cardell编写的mcwm。在2bwm中，可以通过键盘访问所有功能，但也可以使用指针设备进行移动、调整大小和提升/降低。最近，该软件的名称已从mcwm-beast改为2bwm。

安装

安装 2bwmAUR 软件包。虽然安装过程可以自动进行，但如果直接从AUR构建，烈建议阅读并编辑源目录中的config.h 文件。

开始

使用 xinit 运行 `2bwm`

使用2bwm

在启动2bwm之后，屏幕上只有鼠标光标、背景和终端 (在 .xinitrc 中指定)。 要打开终端，请使用默认配置，按下 `Super+Enter` 键。可以根据需要使用终端，例如用 `*program_name* &` 的方式启动程序，更简单方便的做法是使用菜单来启动程序，比如 dmenu 或 9menuAUR.

通用命令

- `Super+Ctrl+q` – 退出 2bwm
- `Super+Ctrl+r` – 重启 2bwm
- `Super+w` – 启动菜单
- `Super+Enter` – 启动终端
- `Super+Arrows` (`+Shift`) – 移动光标 (使用 `Shift` 键快速移动).

窗口控制

使用 `Super` 键与下面其中一个键结合，在特定的焦点窗口上操作：

- `q` – 关闭窗口.
- `Tab` 或 `Shift+Tab` – 在当前工作区窗口环中切换到下一个窗口.
- `f` – 将窗口固定，使其在所有工作区都可见 (切换).
- `a` – 使一个窗口在按下 `Super+q` 键时无法关闭 (切换).
- `r` – 提升或降低窗口 (切换).
- `i` – 将窗口最小化 (或隐藏).

移动、调整大小和传送窗口

使用 `Super` 键与下面其中一个键结合，在特定的焦点窗口上操作：

- `x` – 最大化 (切换).
- `m` – 垂直最大化 (切换).
- `Shift+m` – 水平最大化 (切换).
- `Shift+h` (`+Ctrl`) – 向左调整大小 (使用 `Ctrl` 键慢速移动).
- `Shift+j` (`+Ctrl`) – 向下调整大小 (使用 `Ctrl` 键慢速移动).
- `Shift+k` (`+Ctrl`) – 向上调整大小 (使用 `Ctrl` 键慢速移动).
- `Shift+l` (`+Ctrl`) – 向右调整大小 (使用 `Ctrl` 键慢速移动).
- `Home` – 保持纵横比增加大小.
- `End` – 保持纵横比缩小大小.
- `h` (`+Ctrl`) – 向左移动 (使用 `Ctrl` 键慢速移动)
- `j` (`+Ctrl`) – 向下移动 (使用 `Ctrl` 键慢速移动)
- `k` (`+Ctrl`) – 向上移动 (使用 `Ctrl` 键慢速移动)
- `l` (`+Ctrl`) – 向右移动 (使用 `Ctrl` 键慢速移动)
- `y` – 将窗口移动到显示器的左上角位置.
- `u` – 将窗口移动到显示器的右上角位置.
- `b` – 将窗口移动到显示器的左下角位置.
- `n` – 将窗口移动到显示器的右下角位置.
- `g` – 将窗口移动到显示器的中心位置.
- `Shift+y`/`Shift+u`/`Shift+b`/`Shift+n` – 移动到左侧/右侧/底部/顶部，同时保持最大化垂直/最大化水平，以及半最大化水平/垂直.

工作区

- `0–9` – 前往工作区 `*n*`, 0–9.
- `Shift+*n*` – 发送到工作区 `*n*`.
- `c` or `v` – 前往下一个/上一个工作区.
- `，` or `．` – 将窗口移动到前一个/后一个显示器.

鼠标控制

通过按下 `Super` 键，鼠标按钮的操作如下所示：

- Button 1 on a window – 移动窗口
- Button 3 on a window – 调整窗口大小
- Button 3 + 在桌面上按下`Ctrl`键 – 启动在 `config.h` 中指定的菜单.

请注意，通过键盘激活的所有功能都会在当前焦点窗口上工作，而不管鼠标光标的位置如何。当然，改变工作区与焦点窗口无关。

您可以从 `config.h` 文件更改键盘映射.

提示和技巧

使用脚本获取当前工作区编号

以下命令可获取当前工作空间编号：

```
$ xprop -root _NET_CURRENT_DESKTOP | sed -e 's/_NET_CURRENT_DESKTOP(CARDINAL) = //'
```

易于记忆的外边框颜色

一个记住外边框颜色含义的简单技巧是将"fixed"设置为蓝色，"unkillable"设置为红色，而"fixed + unkillable"设置为紫色。蓝色和红色的混合将产生紫色！

左上角方块

将 `borders` 设置为负数将使外边框变成一个位于完整边框的左上角的正方形。现在为外边框设置的颜色将固定在这个正方形上。

参见

- 2bwm源代码

6.4 IceWM

难度 进阶 | **篇幅** 7.9 KB | **所属** 堆叠式窗口管理器

本节将讲到

- 安装
- 开始
- 配置
- 提示和技巧
- 问题解决

根据维基百科上的内容:

: IceWM is a window manager for the X Window System graphical infrastructure, written by Marko Maček. It was coded from scratch in C++ and is released under the terms of the GNU Lesser General Public License. It is relatively lightweight in terms of memory and CPU usage, and comes with themes that allow it to imitate the UI of Windows 95, OS/2, Motif, and other graphical user interfaces.

翻译：

: IceWM是一个由Marko Maček写的X窗口系统图形窗口管理器的基础设施。它是用 C++ 编写的，在 GNU Lesser General Public License 协议下发布。在内存和 CPU 使用方面，它相对轻量，且带有模仿 Windows 95、OS/2、Motif 和其他图形化用户界面的主题。

安装

安装 icewm包 包。

安装 icewm-gitAUR 包以获得开发版本。

开始

使用 xinit 来运行 `icewm`， 或者使用 `icewm-session` 来运行 icewmbg 和 icewmtray。

注意：`icewm-session` 需要 `startup` 脚本来执行。

配置

尽管 IceWM 的配置在最初是基于文本的，但是仍有一些图形界面的工具可以使用，特别是 AUR 中的 icewm-utilsAUR。然而这些工具都相对较旧，而且大多数用户都喜欢对配置文件进行简单的编辑。配置文件可以在整个系统的范围内（在 `/etc/icewm/` 之中）或单个用户的范围内（在 `~/.icewm/` 之中）进行更改。

如果要更改您的 icewm 配置， 只需要将默认配置文件从 `/usr/share/icewm/` 复制到 `~/.icewm/`，比如这样：

```
$ cp -r /usr/share/icewm/ ~/.icewm/
```

- `preferences` 是 IceWM 的核心配置文件。
- `menu` 控制 IceWM 的主题菜单包含什么。
- `keys` 允许用户使用定制自己的快捷方式。
- `toolbar` 控制任务栏上的一排启动图标。
- `winoptions` 个别的各应用程序的行为。
- `theme` 主题的路径/名称。
- `startup` 在启动时执行的脚本和命令。
- `shutdown` 在关机时执行的。

自动启动

`startup` 脚本不是 icewm包 提供的 ，所以您需要自己创建它，并使之可执行。

为您希望在 IceWM 会话开始时启动的程序添加命令。

注意：安装系统托盘小程序的启动命令前必须有 `sleep 1 &&`，否侧 IceWM 将会创造一个丑陋的黑色窗口；这种情况下，使用任务栏上的 xkill。

下面是一个 IceWM 启动脚本的例子，它使用 IceWM 会话启动 network-manager-applet包 和 XScreenSaver。

```
~/.icewm/startup
#!/bin/sh

# start network manager
    sleep 1 &&
    nm-applet &

# enable bluetooth applet
    sleep 1 &&
    blueman-applet &!

# enable screensaver
    xscreensaver -nosplash &

# start redshift
    redshift &

# allow notifications
    /usr/lib/notification-daemon-1.0/notification-daemon &

# enable lockscreen
    exec xautolock -detectsleep -time 15 -locker "i3lock -n -
i /home/user/lockscreen.png" -killtime 20 -killer "systemctl suspend" #lock after
inactivity and then sleep
```

生成菜单项

- 官方软件仓库中的 menumaker包 是一个 Python 脚本，它根据系统中安装的内容自动填充您的应用程序菜单。尽管这可能会导致许多无用的的应用程序充满菜单，但它仍然可能比手动编辑菜单配置文件更好。当运行 MenuMaker 时，使用 -f 覆盖现有的菜单文件：

```
$ mmaker -f icewm
```

如果您要避免使用终端的应用程序（比如 *alsamixer*），您可以使用 mmaker 命令运行以下开关： `--no-legacy` 和 `--no-debian`。就像这样：

```
$ mmaker -f --no-legacy --no-debian icewm
```

- 或者，您可以使用 Xdg-menu 生成一个菜单。详见 Xdg-menu#IceWM 部分。

主题

icewm包 包中包含了一小些主题。这些可以通过 icewm-extra-themesAUR 包中提供的主题进行补充。更多的主题可以在 box-look.org 中下载。

桌面图标

像 PCManFM 或者 rox包 这样的文件管理器可以管理壁纸并添加桌面图标。或者，您也可以安装 Idesk，这是一个可以帮您把图标添加到桌面的小程序。

提示和技巧

混成

IceWM 是一个不含混成的窗口管理器。如果您需要在 IceWM 使用混成效果，您可以单独选择一个混成管理器，比如 Xcompmgr 或者 Picom。

问题解决

没有开始菜单图标（Intel 显卡）

如果您的 IceWM 运行在Intel 显卡上，您可能会发现您的任务栏中的开始菜单没有图标。这是 xf86-video-intel包 驱动程序最近的变化造成的，“最近的变化”意味着新但是并不稳定，SNA加速后端是默认开启的。 要修复开始菜单问题（及其他可能的图形问题）您需要切换回就得 UXA 后端。请参考以下文章：Intel graphics#AccelMethod。

当 PCManFM 管理桌面时无法注销

如果您使用 PCManFM管理桌面，您可能会发现 IceWM 的注销按钮不能正常工作。您可以自行定义注销命令并作为一种解决方案。这允许您在 PCManFM 管理桌面时注销。为此，您可以打开 `~/.icewm/preferences`，取消以下这行的注释： `# LogoutCommand=""` 然后输入一个可以用来注销的命令。比如：`LogoutCommand="pkill -u username"` 其中的 username 是您的用户名。

注销菜单中没有关机和重启的选项

- 注销命令已经定义：

如果之前定义了注销命令，则关机和重启命令将被忽略。如果您想要在注销菜单中的关机和重启选项，那么您一定不能定义注销命令。

- 注销命令未被定义：

如果您已经定义了关机和重启命令（比如用 `systemctl poweroff` 和 `systemctl reboot`），并且您还没有定义注销命令，可是您仍然发现注销菜单中没有关机和重启选项。请升级到 `icewm 1.3.8-2`。在 FS#37884 中获取更多信息。

参见

- IceWM 的官方网站
- IceWM - The Cool Window Manager - OSNews 上的详细介绍
- IceWM - A desktop for Windows emigrants - polishlinux.org 上的概述和教程

6.5 Fluxbox

难度 进阶 | **篇幅** 13.4 KB | **所属** 堆叠式窗口管理器

本节将讲到

- 安装
- 启动 Fluxbox
- 配置
- 参阅

Fluxbox 是一个 X11 下的窗口管理器。它基于 Blackbox（现在停止开发） 0.61.1 的代码构建，现在依然在开发中，并且功能显著增强。Fluxbox 占用资源少，速度快，并且提供切换和分组等有趣的窗口管理工具。配置文件简单易懂，便于编辑，有上百种的美化桌面的“主题”。Arch Linux 和 Fluxbox 可以让一台 Pentium 800 和 256M 内存的电脑焕发活力。

安装

安装 fluxbox包。

启动 Fluxbox

图形界面登录

在显示管理器的会话菜单中选择 *fluxbox*。

手动

编辑 `~/.xinitrc` 并添加如下几行:

```
exec startfluxbox
```

查看 xinitrc 来获取详细信息，例如保存登录会话。

配置

系统范围的 Fluxbox 配置文件在 `/usr/share/fluxbox` 中，用户个人配置文件在 `~/.fluxbox` 中，配置文件如下：

- *init* - 主要的 Fluxbox 资源配置文件。参阅编辑init文件。
- *menu* - Fluxbox 菜单配置文件。参阅下文和编辑menu文件。
- *keys* - Fluxbox 键盘快捷键（热键）配置文件。参阅下文和这里。
- *startup* - 编辑开机启动程序，参阅下文中关于 .xinitrc 的部分和这里。
- *overlay* - 重载主题元素的配置文件。参阅这里。
- *apps* - 记忆具体应用程序窗口配置的文件。参阅这里。
- *windowmenu* - 配置窗口菜单自身的文件。看这里。

文件夹中有很多不太重要的配置文件。需要注意的有 init，menu，keys 和 startup。

菜单管理

第一次安装 Fluxbox 时系统会在 `~/.fluxbox/menu` 中创建基本的程序菜单，在桌面上右击鼠标可以访问该菜单。和其他轻量窗口管理器一样，Fluxbox 在安装新程序的时候并不自动升级其程序菜单。因此推荐先在系统上安装需要的应用程序，然后再生成或编辑程序菜单。有四种基本方法来添加/编辑项目和增强菜单：

使用 fluxbox-generate_menu

Fluxbox 提供一个内建的命令：

```
$ fluxbox-generate_menu
```

该命令会根据系统中已安装的程序来自动生成 `~/.fluxbox/menu` 文件。但是生成的菜单综合性不如使用 MenuMaker 生成的菜单。

使用 MenuMaker

MenuMaker 是一个为各种各样的窗口管理器创建基于 XML 菜单的有力工具，支持 Fluxbox。MenuMaker 搜索计算机中所有的可执行程序，基于搜索结果创建程序菜单。如果需要的话，可以配置其排除 Legacy X，GNOME, KDE, 和 Xfce 的应用程序。

安装 menumaker包，运行以下命令生成一个完整菜单并覆盖默认菜单。

```
$ mmaker -f FluxBox
```

可以在运行命令时加入以下开关来避免将基于命令行的程序——例如 *alsamixer*——加入菜单：`--no-legacy` 和 `--no-debian`。例如：

```
$ mmaker -f --no-legacy --no-debian FluxBox
```

要查看更多 MenuMaker 选项：

```
 mmaker --help
```

使用 Xdg-menu

可以使用 Xdg-menu 来生成菜单，参阅 Xdg-menu#FluxBox 章节。

手动创建/编辑菜单

使用编辑器编辑 `~/.fluxbox/menu` 文件，一个菜单项目的基本语法类似：

```
[exec] (name) {command} <path to icon>
```

"name" 是想要在菜单项目上显示的文字，"command" 是程序所在的位置。"<path to icon>"是可选的。例如：

```
[exec] (Firefox Browser) {/usr/bin/firefox} <path to firefox icon>
```

创建子菜单的语法如下：

```
[submenu] (Name)
...
...
```

```
[end]
```

编辑完成后，保存退出，不需要重启 Fluxbox。更多的信息参阅编辑 Fluxbox 菜单。

Init 文件

`~/.fluxbox/init` 文件是 Fluxbox 的主要配置资源文件。可以用来改变 Fluxbox，窗口，工具栏，焦点等基本功能。一些选项也可以从Fluxbox配置菜单中找到。详细内容参阅编辑 init 文件。

热键

Fluxbox 提供基本的热键功能。Fluxbox 热键文件位于 `~/.fluxbox/keys`。 Control 键代表 "Control"。 `Mod1` 对应 Alt 键，`Mod4` 对应 Super 键（Super 键不是一个标准按键，经常被映射到 "Win" 键）。当 Fluxbox 安装完成后第一次运行时，Fluxbox 提供一组几乎完成的非常有用的热键。细读精研 `~/.fluxbox/keys` 文件可以增强 Fluxbox 体验。

例子：快捷的方式来控制扬声器音量：

```
Control Mod1 Up :Exec amixer set Master,0 5%+
Control Mod1 Down :Exec amixer set Master,0 5%-
```

声音

参考 Advanced Linux Sound Architecture#Keyboard volume control 和 PulseAudio#Keyboard volume control。

工作区

Fluxbox 默认提供四个工作区。可以通过 Ctrl+F1-F4 快捷键访问，或者在工具栏上的箭头单击左键。也可以通过使用鼠标中键在桌面上单击，在工作区菜单上选择工作区。

切换和分组

当桌面上有两个以上窗口时，在一个窗口的上窗口标签单击 ctrl+鼠标左键，然后拖动该窗口到另一个窗口，这两个窗口会以标签页的形式集合在一起，现在执行一个窗口操作将影响到整个窗口“集合”。在标签页上单击 ctrl+鼠标左键，然后拖动到桌面空白位置将还原窗口。

壁纸

在 Fluxbox 种设置壁纸一直很复杂，特别是透明度问题。请参阅 Fluxbox wiki 中壁纸设置章节。

检查计算机中是否有壁纸设置程序。

```
$ fbsetbg -i
```

如果没有，安装 feh, esetroot 或者 wmsetbg。然后在 `~/.xinitrc` 文件中 "exec" 行之前添加 "fbsetbg" 行。例如：

```
fbsetbg /path/to/my/image.image
```

如果该命令返回如下信息：

```
$ /usr/bin/fbsetbg: line 153: xmessage: command not found
```

那么需要安装 xorg-xmessage 包。

轻松切换多个壁纸

在 Fluxbox 菜单中添加子菜单如下：

```
[submenu] (Backgrounds)
[wallpapers] (~/.fluxbox/backgrounds) {feh --bg-scale}
[wallpapers] (/usr/share/fluxbox/backgrounds) {feh --bg-scale}
[end]
```

将壁纸放到 `~/.fluxbox/backgrounds` 中或者其他指定文件夹，壁纸将以同样的方式在主题中显示。

在没有 'xinerama' 的系统上显示双屏幕壁纸和上述一样，例如：

```
[submenu] (Backgrounds)
[wallpapers] (/path/to/your/backgrounds) {feh --bg-scale --no-xinerama }
[end]
```

使用 feh

安装 feh包。

以下步骤确保 Fluxbox 下次启动时会加载 feh 背景。

1. 确保 `.fehbg` 可执行:

```
$ chmod 770 ~/.fehbg
```

2. 在 `~/.fluxbox/init` 文件中添加/修改以下行:

```
session.screen0.rootCommand:       ~/.fehbg
```

3. 在 `~/.fluxbox/startup` 文件中添加/修改以下行:

```
~/.fehbg
```

使用 Nitrogen

也可以使用 Nitrogen。使用以下命令来运行图形壁纸选择软件：

```
$ nitrogen
```

添加包含图片的文件夹，选择图片然后应用。当然可以更改图片选项，例如壁纸居中或者放大。

添加如下行：

```
nitrogen --restore &
```

到 `~/.fluxbox/startup` 文件中确保壁纸在登录时使用。

如果重启 Fluxbox 后，当前选择主题的壁纸覆盖了 Nitrogen 设置的壁纸。编辑使用的主题的 `theme.cfg` 文件，注释或者删除如下几行来解决这个问题:

```
background:
background.color:
background.colorTo:
```

主题

要安装 Fluxbox 主题，将主题文件解压缩到主题文件夹，默认的主题文件夹是

- 全局 - `/usr/share/fluxbox/styles`
- 仅用户 - `~/.fluxbox/styles`

Arch Linux AUR 中有一套很漂亮的 Fluxbox 主题——"fluxmod-styles"。从这里获得。正确安装后将会在 Fluxbox 菜单中的 Fluxbox -> Styles section 中显示。

想要创建自己的 Fluxbox 主题，请参阅 fluxbox-style(5)， Fluxbox/Style guide 和主题指南\[失效链接 2022-09-17 ⓘ\]。

如果使用

```
$ mmaker -f FluxBox
```

创建菜单，那么安装完主题后将不会看到主题选择菜单。 将下面几行添加到 `~/.fluxbox/menu` 文件中的重启菜单选项后来修正。

```
[submenu] (System Styles) {Choose a style...}
      [stylesdir] (/usr/share/fluxbox/styles)
        [end]
[submenu] (User Styles) {Choose a style...}
      [stylesdir] (~/.fluxbox/styles)
        [end]
```

Slit

Fluxbox，WindowMaker 和一些轻量的窗口管理器都有 "Slit"，可以是所有程序 "dock" 化。一个 "dock" 化的程序可以锚定在任何工作区上，可以被轻松移动，并且不受任何窗口操作的影响，它基本上是一个小部件，将时钟，系统监视器，天气等 "dock" 化非常有用。参阅 dockapps.net。

自启动程序

Fluxbox 本身提供了自启动程序的功能。`~/.fluxbox/startup` 文件是一个像启动 Fluxbox 一样自启动应用程序的脚本。`#` 标记是注释。

一个简单的例子：

```
fbsetbg -l # sets the last background set, very useful and recommended.
# In the below commands the ampersand symbol (&) is required on all applications that
do not terminate immediately.
# Failure to provide them will cause Fluxbox not to start.
idesk &
xterm &
```

```
# exec is for starting Fluxbox itself, do not put an ampersand (&) after this or
Fluxbox will exit immediately.
exec /usr/bin/fluxbox
# or if you want to keep a log, uncomment the below command and comment out the above
command:
# exec /usr/bin/fluxbox -log ~/.fluxbox/log
```

其他菜单

上面的 "菜单管理" 章节讨论了主要菜单。这些菜单在 Fluxbox 行话中称为"根"菜单。Fluxbox 也为用户提供了一些其他的菜单。

- 工作区菜单：在桌面上点击鼠标中键。
- 配置菜单：位于"根"菜单的 "Fluxbox" 部分。
- 窗口菜单：在任何窗口或者最小化窗口的标题栏右击鼠标。可以编辑窗口菜单。参阅 fluxbox-menu man page。
- 工具栏菜单：在工具栏空白部分右击鼠标。也在配置菜单的子菜单中。
- Slit 菜单：在配置菜单的子菜单中。

桌面效果

想要在 Fluxbox 开启桌面效果——透明或者阴影，需要安装 X compositor，例如 Xcompmgr 或者 Compton。

通知

在 Fluxbox 中开启桌面通知，需要安装通知服务器。参阅 desktop notifications。

没有 xorg.conf 文件后的 Xorg 配置

Xorg 不再需要 *xorg.conf* 文件了，以前在 xorg.conf 文件中所做的关于键盘和省电模式的设定可以通过以下方式调整。

在 Fluxbox 中设定键盘布局

将以下行添加到 `~/.fluxbox/startup` 文件中：

```
setxkbmap us -variant intl & # 在 us 键盘中添加特殊字符支持 (类似 éóíáú)
```

可以添加自己的语言代码来代替 'us'，并且移除 'variant' 选项。参阅 *setxkbmap* 手册页来获得更多选项。

在 `~/.fluxbox/menu` 文件中添加以下行来获得菜单中帮助功能:

```
[submenu] (Keyboard)
      [exec] (normal) {setxkbmap us}
      [exec] (international) {setxkbmap us -variant intl}
[end]
```

参阅

- Fluxbox Homepage
- Fluxbox wiki

- Gentoo Wiki about Fluxbox
- Themes for Fluxbox
- Narada's Fluxbox Guide
- The Fluxbox man pages: fluxbox, fluxbox-menu, fluxbox-style, fluxbox-keys, fluxbox-apps, fluxbox-remote, fbsetroot, fbsetbg, fbrun, startfluxbox.
- Arch Linux Fluxbox screenshots

6.6 Compiz

难度 高级 | **篇幅** 36.6 KB | **所属** 堆叠式窗口管理器

本节将讲到

- 安装
- 开始使用 Compiz
- Using Compiz as a standalone window manager
- 小技巧大放送
- 疑难排解

翻译状态：

- 本文（或部分内容）译自 Compiz，最近一次同步于 2014-08-14，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

**这篇文章或章节的翻译质量不佳。**

原因：Last updated in 2016（在 Talk:Compiz# 中讨论）

来自维基百科的解释:

: Compiz 是第一个由 OpenGL 驱动的运行于 X Window System 上的混合窗口管理器 。Compiz的混合渲染能力使其可以在窗口管理过程中实现多种视觉效果，比如在矩形虚拟桌面上的窗口最小化。

安装

- 2013年5月起， Compiz 在official repositories中就不再可用了。
- 安装0.9和0.8版本的包可以在AUR (简体中文)中找到。
- 这两个版本**不能同时安装**。

安装0.9系列版本

注意：

- 从 Compiz 0.9.8 开始，所有的Compiz组件（包括CCSM、插件、gtk-window-decorator等）都是当作一个单独项目来开发的。也就是说一个安装包就包括了Compiz的全部组件。

- 想要带 *emerald-themes*的*emerald0.9*, 先安装emerald0.9AUR\[损坏的链接：package not found\] 再安装 emerald-themesAUR. 相反的做法会导致依赖关系错误而冲突.

必需的:

- **Compiz** — OpenGL合成管理器与CCSM,插件和GTK窗口装饰器.

: https://launchpad.net/compiz || compizAUR

可选的:

- **Emerald** — Compiz的独立窗口装饰器

: http://www.compiz.org/ || emerald0.9AUR\[损坏的链接：package not found\]

- **Emerald Themes** — Emerald 窗口装饰的额外主题.

: http://www.northfield.ws/projects/compiz/ || emerald-themesAUR

- **Fusion Icon** — Compiz的托盘小程序并且可以切换不同的窗口管理器以及装饰器.

: https://github.com/kozec/fusion-icon-gtk3 || fusion-icon0.9AUR\[损坏的链接：package not found\]

安装0.8系列版本

安装以下任意一个包：compiz-coreAUR、 compiz-gtk-standaloneAUR\[损坏的链接：package not found\] 或 compiz-core-mateAUR\[损坏的链接：package not found\]。

需要Compiz设置中心，请安装 ccsmAUR。

需要插件，请安装 compiz-fusion-plugins-mainAUR、 compiz-fusion-plugins-extraAUR 。 并且可选地，可以安装 compiz-fusion-plugins-unsupportedAUR\[损坏的链接：package not found\]。

Emerald窗口装饰器可以从 emeraldAUR 包中安装。 需要附加的Emerald主题可以安装 emerald-themesAUR 。

窗口装饰器

提示：想知道更多关于选择与管理主题的情况，请访问：Compiz configuration#Window decoration themes。

注意：

- 大多数的Compiz软件包都会默认提供 gtk-window-decorator （一个GTK窗口装饰器组件）。唯一的例外则是 compiz-coreAUR 包中并没有提供窗口装饰器。

- 目前没有任何一个包自动提供 kde-window-decorator （一个KDE窗口装饰器组件）。想要安装它，你需要编辑将要安装的 Compiz 包中的 PKGBUILD 文件，从中打开相关的选项。当你编辑完之后，还需要重新编译并安装 Compiz 包。

窗口装饰器是为程序提供窗口和边框的程序。 不像类似mutter包之类的窗口管理器, 不像Kwin或者Xfwm只提供一个唯一的装饰器， Compiz的用户拥有三个选择：Emerald、gtk-window-decorator以及kde-window-decorator。

其中gtk-window-decorator 和 the kde-window-decorator 就包含在 Compiz 的源代码中，并且已经编译好了（如果你的PKGBUILD文件设置妥当的话）。 而Emerald分离在外，是一个独立的装饰器软件。

设置它可以按照如下步骤：

首先，确定你已经安装了你想要在Compiz中使用的装饰器。然后，打开 CCSM（*Compiz Config Setting Manager ，Compiz设置管理器*）。 你可以找到它的图标或者在终端中运行命令： `ccsm`。在设置界面中，找到“效果”部分，并保证“Window Decoration”（*”窗口装饰“*）已经勾选了。 现在，点击“Window Decoration”（*”窗口装饰“*）按钮，并且在”命令“框中输入跟你需要的装饰器相关的命令：

- `emerald`
- `gtk-window-decorator`
- `kde4-window-decorator`

为了让这些改动立即生效，你可能需要在命令的结尾添加 `--replace` 开关。 另外，注销之后再登录进来也可以让你新设置的装饰器生效。 不过请注意，在CCSM中输入的使用 `--replace` 开关的装饰器命令，会在用户登录的时候使屏幕闪一下。

开始使用 Compiz

启用重要的插件

提示：

- *译者注* 为了大家看得方便，我把插件的名字保留了英文。因为有时候CCSM里面也会显示英文的插件名称。
- *译者注* 插件的中文名称可能翻译得不准确。

在开始使用Compiz之前， 你需要激活一些提供窗口管理基本功能的插件。 要不然你可能连拖拽窗口都困难，更别说缩放和关闭了。

重要的插件:

- Window Decoration（*窗口装饰*）——提供窗口边框（上一节咱已经讨论过了）
- Move Window（*窗口移动*）
- Resize Window（*窗口缩放*）
- Place Windows（*窗口放置*）——设置关于窗口在屏幕上放置的选项
- Application Switcher（*程序切换*）——提供 Alt+Tab 开启的程序切换器（另外也有一些插件可以实现这个功能，而且具有不同的效果，比如 'Shift Switcher,' 'Static Application Switcher' 等等。并不是所有的切换器都是用 Alt+Tab 快捷键）。

想要在不同的视区之间切换，你需要激活以下其中一个插件：

- Desktop Cube（*立体桌面*）和Rotate Cube（*旋转立体*）——提供一个可以浮空的立方体，每个面都是一个视口（*虚拟桌面*）。
- Desktop Wall（*桌面墙壁*）——视口全部并排陈列在一起（非常类似于 Cinnamon 和 GNOME Shell 中的窗口切换效果）。
- Expo——当鼠标移动到屏幕左上角的时候，显示出全部的视口和窗口（*就是Ubuntu Unity里面那种！！*）。这个插件可以单独激活，或者是和前两个一起用不会冲突。

启动 Compiz

注意：曾几何时，出现过一个包错误，导致你需要在运行 `compiz --replace` 命令时使用 `ccp` 开关才能正常导入Compiz插件。不过现在这个错误已经修正了，现在已经**没有任何必要**使用 `ccp` 开关。

你可以使用下面的命令启动Compiz：

```
$ compiz --replace &
```

提示：如果 `compiz --replace &` 命令在进入会话时不管用。 试一试没有a花的命令： `compiz --replace`。

我们来过一下常用的Compiz命令行选项吧：

- `--indirect-rendering`: 使用间接渲染 (AIGLX)
- `--loose-binding`: 可以解决一些性能问题 (NVIDIA?)
- `--replace`: 取代现有的窗口管理器（*window-manager*）
- `--keep-window-hints`: 在可用的视口中保留Gnome窗口管理器的 gconf-settings 设置
- `--sm-disable`: 禁用会话管理功能

小工具 Fusion Icon

Fusion Icon 是一个托盘小工具，功能是在一个会话中切换Compiz和其他不同的窗口管理器以及装饰器。 **它只和Compiz 0.8版本兼容。** 它可以在 fusion-iconAUR 包中安装。

你可以用下面的命令运行 fusion-icon ：

```
$ fusion-icon
```

为了确保Fusion-icon确实启动了Compiz，邮件单击它在通知区域的图标，然后选择“select window manager”， 如果“Compiz”还没被选上的话就选上吧。

下面将要讲到的很多方法会在桌面环境启动时自动运行Compiz。你也可以叫桌面环境自动运行fusion-icon来间接自动启动Compiz。 如果你想这样做的话， 只要把Compiz的命令 （通常都是 `compiz --replace &`）改成 `fusion-icon` 就行了。

让你的桌面环境自动启动Compiz

KDE4

使用系统设置

打开系统设置程序，并找到“系统设置 > 默认程序 > 窗口管理器 > 使用另外的窗口管理器'”然后选择“Compiz”。

如果你需要让Compiz使用你自己设定的选项启动的话，就选择“Compiz custom”，然后新建一个叫做`compiz-kde-launcher` 的脚本，并且在其中添加你希望启动Compiz的命令。 下面是一个例子：

```
/usr/local/bin/compiz-kde-launcher
#!/bin/bash
LIBGL_ALWAYS_INDIRECT=1
compiz --replace &
wait
```

然后还要让他可运行：

```
$ chmod +x /usr/local/bin/compiz-kde-launcher
```

Autostart 自动启动链接

注意：

- 如果你想安装 gtk-window-decorator，就不要新建 `compiz.desktop` 文件，因为这回产生一些问题。
- 如果 `compiz.desktop` 已经有了，那你可能需要在 `Exec` 之后添加 `--replace` 。

在KDE自动启动（*Autostart*）目录中添加一个项目。如果它不存在，就新建一个：

```
~/.kde4/Autostart/compiz.desktop
[Desktop Entry]
Type=Application
Encoding=UTF-8
Name=Compiz
```

```
Exec=/usr/bin/compiz --replace
NoDisplay=true
# name of loadable control center module
X-GNOME-WMSettingsModule=compiz
# autostart phase
X-GNOME-Autostart-Phase=WindowManager
X-GNOME-Provides=windowmanager
# name we put on the WM spec check window
X-GNOME-WMName=Compiz
# back compat only
X-GnomeWMSettingsLibrary=compiz
```

设置 KDEWM 变量

作为root用户你需要搞一个小脚本，这样你就可以在启动Compiz的时候添加命令行选项了。

新建一个文件，里面输入启动Compiz的命令，再加上你希望附加的设置。下面是一个例子：

```
/usr/bin/startcompiz
compiz --replace &
```

注意：你不一定非要把这个脚本叫做 *startcompiz*。只要保证你的脚本的名字不会和系统已有的命令（在`/usr/bin`目录中）弄混就行了。

保证你的脚本可执行：

```
# chmod +x /usr/bin/startcompiz
```

现在你需要新建一个程序来调用你刚刚在 `/usr/bin` 中新建的这个脚本了。

1) 只针对你当前的用户设置:

```
~/.kde4/env/startcompiz.sh
KDEWM="compiz"
```

2) 全系统范围内设置:

```
/etc/kde/env/startcompiz.sh
KDEWM="compiz"
```

注意：

- 如果上面的方法不管用，使用下面的命令可以达到同样的目的：

```
$ export KDEWM="startcompiz"
```

: 把它放到你用户的 `~/.bashrc` 文件中去。

- 如果你使用了 `/usr/local/bin` 目录那可能会不管用。那么你需要在脚本中使用绝对路径：

```
$ export KDEWM="/usr/local/bin/startcompiz"
```

GNOME 桌面环境

GNOME Shell 和 Cinnamon

GNOME Shell (简体中文) 是一个类似 mutter包 的实现插件。 同样， Cinnamon 插件是一个类似于 muffin包 窗口管理器的实现。

也就是说，将GNOME Shell 或 Cinnamon 与 Compiz 或者其他窗口管理器同时使用**是不可能的**（因为他们都是窗口管理器！）。

Unity

确保 'Ubuntu Unity Plugin' （**Ubuntu Unity 插件**）在 CCSM 中启用了，这样 Unity 才能正确运行。

你只有正确安装了Unity之后，这个插件才会显示在CCSM里面。

其它的GNOME会话 (Cairo dock 和 Compiz)

gnome-session-compizAUR\[损坏的链接：package not found\] 包会在一个显示管理器中添加一个新的会话。 只要在你的显示管理器中选择 'gnome-session-compiz' 即可。

请确保 Compiz 以及 Cairo Dock (Taskbar或Panel) 已经正确的设置好了。

在CCSM里面，确保选择了窗口装饰器，已经启用了窗口管理需要的必要插件。详见 #启用重要的插件.

下面是一些建议的Cairo Dock的设置：

- 在Cairo Dock上面添加一个应用程序菜单，并且记住它的快捷键绑定。
- 为了舒服，吧应用程序菜单的快捷键绑定到 `Alt+F1` 和 `Alt+F2`。
- 添加时钟、Wifi、网速等等插件图标到Dock上面。
- 添加一个注销按钮：
- 设置注销命令为 `gnome-session-quit --logout`
- 设置关机命令为 `gnome-session-quit --power-off`
- 添加Notification Area Old (systray) 图标（**通知区域支持**）到 Cairo Dock。

GNOME Flashback

详见：GNOME Flashback#Alternative window manager.

MATE 桌面环境

使用 GSettings

使用下面的 GSettings 命令可以将默认的窗口管理器 marco包 改为 Compiz。

```
$ gsettings set org.mate.session.required-components windowmanager compiz
```

使用 mate-session-properties

注意：当使用这种方法时，Marco会首先启动，然后自动被Compiz替换。

另一种做法是使用mate-session-properties来启用Compiz。 在终端中输入下面的命令：

```
$ mate-session-properties
```

单击“添加”按钮，并在“命令”文本框中输入 `compiz --replace &` 命令。 名称和介绍栏目并不重要，只是用来说明的，不要在意这些细节。 注销再登录，Compiz应该就会顺利启动了。

Xfce

修改默认 (故障保护Failsafe) 会话

想要设置Xfce的默认会话 (也就是故障保护 Failsafe 会话) ，可以使用 `xfconf-query`:

```
xfconf-query -c xfce4-session -p /sessions/Failsafe/Client0_Command -t string -t string -s compiz
```

或者可以编辑下面其中之一：

- 用户：`~/.config/xfce4/xfconf/xfce-perchannel-xml/xfce4-session.xml`
- 系统：`/etc/xdg/xfce4/xfconf/xfce-perchannel-xml/xfce4-session.xml`

注意：如果 xfce4-session.xml 文件不存在于 `~/.config/xfce4/xfconf/xfce-perchannel-xml` ，你需要去编辑 `/etc/xdg/xfce4/xfconf/xfce-perchannel-xml`中的文件。

将 Xfwm 的启动命令：

```
<property name="Client0_Command" type="array">
  <value type="string" value="xfwm4"/>
</property>
```

替换为：

```
<property name="Client0_Command" type="array">
  <value type="string" value="compiz"/>
</property>
```

为了保护默认会话不被覆盖，你可能需要使用：

```
<property name="general" type="empty">
  ...
  ...
  <property name="SaveOnExit" type="bool" value="false"/>
</property>
```

然后移除已经存在的会话：

```
$ rm -r ~/.cache/sessions
```

现在你应该注销。请确保“保存默认会话”标签**没有被勾选**，否则刚才的编辑不会生效。 再次登录，Compiz应该就会运行了。当Compiz正常运行之后，你就可以重新钩上“保存将来的会话”了。

Starting Compiz from a restored Xfce session

注意：If you clear your saved sessions then you will need to run through the steps of this method again.

By default, Xfce saves its sessions on logout which means that running applications will be restored on login. Therefore, to ensure that Compiz is autostarted with each

session, you merely need to ensure that Compiz is running when you logout. To do so, hit `Alt+F2` to start the Xfce run dialog. Then run the command to start Compiz: `compiz --replace &`. Then you just need to logout. When logging out, ensure that the "Save session for future logins" option in the Xfce logout dialog is **ticked**. You should find that Compiz is autostarted in all future sessions.

Using Xfce application autostart

注意：If this method is used, Xfwm will start first and will then be replaced by Compiz.

In the Xfce main menu navigate to 'Settings' and click on 'Session and Startup.' Click on the 'Application Autostart' tab. Click the add button and in the command section enter the `compiz --replace &` command. The name and comment sections are unimportant and are just there to indicate what the entry does. Log out and log in again and Compiz should start.

提示：When using this method, it is advisable to disable saved sessions otherwise Compiz will be started twice at login which could lead to problems such as the `Alt+F4` keybinding not working. To disable saved sessions, open the 'Session and Startup' menu entry, click on the 'Session' tab and click the 'Clear saved sessions' button. Then, untick the 'Save session for future logins' option in the Xfce logout dialog.

LXDE & LXQt

The session files of LXDE and LXQt can be edited to start an alternative window manager (such as Compiz) with the session.

- For LXDE, see the following section: LXDE#Use a different window manager.
- For LXQt, see the following section: LXQt#Use a different window manager.

Using Compiz as a standalone window manager

Starting the session with a display manager

A standalone Compiz session can be started from a display manager. For most display manager's (LightDM, LXDM, GDM etc) all that's required is to create a .desktop file in `/usr/share/xsessions` defining the Compiz session. See the article for your display manager to check if this is the case.

Firstly, create the `/usr/share/xsessions` directory if it does not already exist. Then create the .desktop file. A basic example is provided below:

```
/usr/share/xsessions/compiz.desktop
[Desktop Entry]
Version=1.0
Name=Compiz
Comment=Start a standalone Compiz session
Exec=compiz
Type=Application
```

Then just choose 'Compiz' from your sessions list and log in.

注意：Some display managers are stricter than others regarding the syntax of .desktop files. If a Compiz option does not appear in your display manager's session menu you will need to edit your .desktop file to make it compatible. The example above should work in most cases.

Autostarting programs when using a display manager

One way in which you could start programs with your Compiz session, when it is started from a display manager, is to use an xprofile file. The xprofile file is similar in syntax to xinitrc - it can contain commands for programs you wish to start with your session. Most display managers will parse commands from an xprofile file by default.

Alternatively, you could use Compiz's 'Session Management' plugin. This plugin will save running programs on exit and restore them when the session is next started. Simply enable the 'Session Management' plugin in CCSM.

Starting the session with startx

To start Compiz with the `startx` command, add the following line to your `~/.xinitrc` file:

```
~/.xinitrc
exec compiz
```

You can also use fusion icon as shown below:

```
~/.xinitrc
exec fusion-icon
```

You can autostart additional programs (such as a panel) by adding the relevant command to your `~/.xinitrc` file. Below is an example of a `~/.xinitrc` file which starts Compiz, the tint2 panel and the Cairo dock.

```
~/.xinitrc
tint2 &
cairo-dock &
exec compiz
```

See the xinitrc article for more details.

Add a root menu

To add a root menu similar to that in Openbox and other standalone window managers you can install the compiz-boxmenuAUR package. This program is a fork of compiz-deskmenuAUR. Among the changes that the fork introduces are the addition of some extra features such as a window list and a recent documents list.

After installing the compiz-boxmenuAUR package, copy the config files to your home directory as shown below:

```
# cp -R /etc/xdg/compiz /home/username/.config
# chown -R username:group /home/username/.config/compiz
```

where `username` is your username and `group` is the primary group for your user.

Then, open CCSM, navigate to the 'Commands' plugin and in 'Command line 0' enter the command `compiz-boxmenu`. In the 'Key Bindings' tab, set 'Run command 0' to `Control+Space` (you can use the 'Grab key combination' option to simplify this process.)

Now navigate to the 'Viewport Switcher' plugin and click on the 'Desktop-based Viewport Switching' tab. Change the 'Plugin for initiate action' to `core` and change 'Action name for initiate' to `run_command0_key`.

You should now find that a menu appears when you click `Control+Space`. To launch a graphical menu editor, click on the 'Edit' option or run `compiz-boxmenu-editor` in a terminal. If you would prefer to edit your menu manually, open the following file in your favourite editor: `~/.config/compiz/boxmenu/menu.xml`. For your changes to take effect, you must click the 'Reload' option in your menu.

警告：Whilst `Control+Space` is the default keybinding for `compiz-boxmenu` you can assign the menu to other keybindings or mousebindings as well. Take extreme care if doing so as Compiz bindings will take precedence over keybindings of all other programs. For instance, if you assign `compiz-boxmenu` to `Button3` then you may lose right click functionality in all programs. If the keybinding/mousebinding you are attempting to create has any conflicts, `cssm` will notify you.

Allow users to shutdown/reboot

An unprivileged user should be able to execute commands such as `systemctl poweroff` and `systemctl reboot`. You could assign a keyboard shortcut to one of these commands using the 'Commands' plugin in CCSM. Alternatively, you could create a launcher for one of these commands in compiz-boxmenuAUR - see above. For more detailed information on shutting down see the following article: Allow users to shutdown.

Utilities

Panels & docks

There are a number of panels and docks available in Arch however only a few are compatible with Compiz's viewports. They are listed below:

- xfce4-panel包
- mate-panel包
- perlpanelAUR\[损坏的链接：package not found\]
- gnome-panel包
- cairo-dock包

注意：Other panels and docks can be run with Compiz however their desktop pagers will show only one virtual desktop and all window buttons will be shown in all viewports regardless of which viewport the window happens to be in.

Run dialog

- mate-panel包 provides a run dialog. To enable it, see the following section: Compiz#MATE Panel\[损坏的链接：无效的章节\].
- gnome-panel包 provides a run dialog. To enable it, see the following section: Compiz#GNOME Panel\[损坏的链接：无效的章节\].
- lxpanel包 provides a run dialog. To enable it, see the following section: Compiz#LXDE Panel\[损坏的链接：无效的章节\].

注意：The panels mentioned above must be running in order to use their respective run dialogs.

Alternatively you could install one of the following:

- xfce4-appfinder包 - use the following command to launch a run dialog: `xfce4-appfinder --collapsed`
- bbrunAUR - use the following command to launch a run dialog: `bbrun -w`
- gmrun包 - use the following command to launch a run dialog: `gmrun`
- fbrunAUR\[损坏的链接：package not found\] - use the following command to launch a run dialog: `fbrun`

In each case, simply map the command to `Alt+F2` (or a key combination of your choice) via the 'Commands' plugin in CCSM.

小技巧大放送

还原原生窗口管理器

你可以用下面的命令把Compiz改回你桌面环境原生的窗口管理器：

```
wm_name --replace
```

使用 `kwin`、 `metacity` 或 `xfwm4` 替换掉 `wm_name`.

启用 Alt+F2 运行对话框

GNOME Panel

在CCSM中启用 'Gnome Compatibility' 插件。

MATE Panel

有两种方法在Compiz中启动 MATE Panel 中的运行对话框，你自己选吧：

- 在CCSM中启用 'Gnome Compatibility' 插件
- 使用“命令”（*Commands*）插件，设置 `Alt+F2` 快捷键的绑定如下：

```
mate-panel --run-dialog
```

LXDE Panel

使用“命令”（*Commands*）插件，设置 `Alt+F2` 快捷键的绑定如下：

```
lxpanelctl run
```

Xfce 程序查看器（*Appfinder*）

当在Xfce会话中使用Compiz时，运行对话框（由xfce4-appfinder包提供）无需任何干预应该管用。

但是如果你是在Compiz独立会话中使用程序查看器的话，请查看这一章节：Compiz#Run dialog\[损坏的链接：无效的章节\]。

其他的运行对话框功能

请查看这一章节：Compiz#Run dialog\[损坏的链接：无效的章节\]。

疑难排解

Missing GLX_EXT_texture_from_pixmaps

On ATI cards (first solution)

You may run into the following error when trying to run Compiz on an ATI card:

```
Missing GLX_EXT_texture_from_pixmap
```

This is because Compiz's binary was compiled against Mesa's OpenGL library rather than ATI's OpenGL library.

Firstly, copy the library into a directory to keep it because ATI's drivers will over write it:

```
$ install -Dm644 /usr/lib/libGL.so.1.2 /usr/lib/mesa/libGL.so.1.2
```

Then you can reinstall your fglrx drivers. Now start Compiz as shown below:

```
LD_PRELOAD=/usr/lib/mesa/libGL.so.1.2 compiz --replace &
```

On ATI cards (second solution)

Another possible problem with 'GLX_EXT_texture_from_pixmap' on ATI cards is that the card can only render it indirectly. If so, you have to pass the option to your libgl as shown below:

```
LIBGL_ALWAYS_INDIRECT=1 compiz --replace &
```

(Workaround tested on the following card : ATI Technologies Inc Radeon R250 \[Mobility FireGL 9000\] (rev 02))

On Intel chips

Firstly, check that you are using the intel driver as opposed to i810. Then, run the following command to run Compiz (This must be used every time).

```
LIBGL_ALWAYS_INDIRECT=true compiz --replace --sm-disable &
```

Compiz starts without window borders with NVIDIA binary drivers

Firstly, ensure that your window decorator settings are configured correctly - see the following: Compiz#Starting the window decorator\[损坏的链接：无效的章节\]. If window borders still do not start try adding *Option "AddARGBGLXVisuals" "True"* and *Option "DisableGLXRootClipping" "True"* to your "Screen" section in `/etc/X11/xorg.conf.d/20-nvidia.conf`. If window borders still do not load and you have used other Options elsewhere in `/etc/X11/xorg.conf.d/` try commenting them out and using only the aformentioned ARGBGLXVisuals and GLXRootClipping Options.

Blank screen on resume from suspend-to-ram using the NVIDIA binary drivers

If you receive a blank screen with a responsive cursor upon resume, try disabling sync to vblank. To do so, open CCSM, navigate to the 'OpenGL' plugin and untick the 'Sync to VBlank' option.

Poor performance from capable graphics cards

NVIDIA and Intel chips: If everything is configured correctly but you still have poor performance with some effects, try disabling *CCSM* > *General Options* > *Display Settings* > *Detect Refresh Rate* and instead choose a value manually.

NVIDIA chips only: The inadequate refresh rate with 'Detect Refresh Rate' may be due to an option called 'DynamicTwinView' being enabled by default which plays a factor in accurately reporting the maximum refresh rate that your card and display support. You can disable 'DynamicTwinView' by adding the following line to the "Device" or "Screen" section of your `/etc/X11/xorg.conf.d/20-nvidia.conf`, and then restarting your computer:

```
Option "DynamicTwinView" "False"
```

Screen flicks with NVIDIA card

To fix this behaviour create the file below:

```
/etc/modprobe.d/nvidia.conf
options nvidia NVreg_RegistryDwords="PerfLevelSrc=0x2222"
```

Compiz effects not working (GConf backend)

If you have installed the gtk-window-decorator, check if the GConf schema was correctly installed:

```
$ gconftool-2 -R /apps/compiz/plugins | grep plugins
```

Make sure that all plugins are listed. If they are not, try to install the Compiz schema manually (do **not** run this command as root):

```
$ gconftool-2 --install-schema-file=/usr/share/gconf/schemas/compiz-decorator-gtk.schemas
```

Fusion Icon fails to start

If you get an output like this from the command line:

```
$ fusion-icon
* Detected Session: gnome
* Searching for installed applications...
Traceback (most recent call last):
  File "/usr/bin/fusion-icon", line 57, in <module>
    from FusionIcon.interface import choose_interface
  File "/usr/lib/python2.5/site-packages/FusionIcon/interface.py", line 23, in <module>
    import start
  File "/usr/lib/python2.5/site-packages/FusionIcon/start.py", line 36, in <module>
    config.check()
  File "/usr/lib/python2.5/site-packages/FusionIcon/util.py", line 362, in check
    os.makedirs(self.config_folder)
  File "/usr/lib/python2.5/os.py", line 172, in makedirs
    mkdir(name, mode)
OSError: [Errno 13] Permission denied: '/home/andy/.config/compiz'
```

the problem is with the permission on `~/.config/compiz/`. To fix it, use:

```
# chown -R username /home/username/.config/compiz/
```

Alt+F4 keybinding not working (Xfce)

See the following section: Compiz#Using Xfce application autostart\[损坏的链接：无效的章节\].

Emerald refuses to start (crashes with a segfault)

You may find that Emerald fails to start with your Compiz session and attempting to start it from a terminal gives you the following output (or something similar):

```
Segmentation fault (core dumped)
```

In this case, the solution is to reset the Emerald theme settings:

```
$ rm -rf ~/.emerald/theme
```

Emerald should now start successfully.

No system bell when Compiz is running

You may find that the system bell (such as the drip sound played when pressing backspace at the beginning of a line in GNOME or MATE Terminal) will not sound if Compiz is running. See the following upstream bug report.

For PulseAudio users the following workaround is available:

Append the following lines

```
load-sample-lazy bell /usr/share/sounds/freedesktop/stereo/bell.oga
load-module module-x11-bell sample=bell
```

to your `/etc/pulse/default.pa` file and then restart Pulseaudio.

Compiz crashes when enabling the Gnome Compatibility plugin (GSettings backend)

If you are using the GSettings backend, you may find that Compiz crashes if you try to enable the 'Gnome Compatibility' plugin. In order to enable this plugin whilst using the GSettings backend you need to open CCSM and navigate to 'Preferences.' Under the header 'Integration' untick the box labelled 'Enable integration into the desktop environment.' After unticking this option, you should find it possible to enable the 'Gnome Compatibility' plugin.

Known issues

Xfce panel window buttons are not refreshed when a window changes viewport

You may find that if you right click on a window title and choose an option such as 'Move to Workspace Right' then the window will move but the window button will still be visible in the viewport the window moved from until you switch viewports. See the following upstream bug report.

Compiz crashes when enabling the D-Bus plugin

The D-Bus plugin will cause Compiz to crash if enabled in conjunction with certain other plugins such as the Cube plugin. See the following upstream bug report.

See also

- Compiz in Launchpad
- Compiz Home, including wiki and forum (website and wiki are unmaintained)
- Troubleshooting - Compiz Wiki, (wiki is unmaintained)

6.7 FVWM

难度 高级 | **篇幅** 37.1 KB | **所属** 堆叠式窗口管理器

本节将讲到

- 安装
- 启动
- 配置
- 提示与技巧
- 故障排除

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

FVWM 是一个符合ICCCM标准的X窗口系统的多虚拟桌面窗口管理器。它通过编辑基于文本的配置文件进行配置。虽然使用FVWM不需要任何编程语言的知识，但可以通过 M4， C 和 Perl 预处理来扩展FVWM。FVWM还有一个 Perl library 允许创建 modules。FVWM代表F Virtual Window Manager，优选解释是F并没有特定的含义 .

注意：截至2016年11月，有计划将开发重点转向 FVWM3，这是FVWM2的继任者，将拥有新的功能，但不兼容FVWM2。

安装

Install the fvwmAUR package or fvwm-gitAUR for the development version.

The following packages provide themes and icons for FVWM: fvwm-crystalAUR, fvwm-iconsAUR, fvwm-themesAUR, fvwm-themes-extraAUR. FVWM Crystal provides a separate session for a desktop environment like experience.

启动

Select *FVWM* from the session menu in a display manager of choice. Otherwise, add `exec fvwm` to your user's `.xinitrc`.

For FVWM Crystal, select *FVWM-Crystal* from the session menu or add `exec fvwm-crystal` to your user's `.xinitrc`.

See xinitrc for details, such as preserving the logind session.

Startup functions

注意：It is recommended that you only use the StartFunction and make use of the `Test` command - explained below.

FVWM provides a number of functions to start modules or applications when initialising, restarting or exiting the window manager.

- StartFunction - executed when FVWM is first initialised and on restarts.
- InitFunction - executed only when FVWM is first initialised.
- RestartFunction - executed only when FVWM is restarted.

- ExitFunction - executed when exiting FVWM.

You can add your own actions to any of these functions using the `AddToFunc` command. For example, if one wanted to start *network-manager-applet* on startup (but not for any subsequent restarts of the window manager) one could add the *nm-applet* command to the InitFunction:

```
AddToFunc InitFunction
+ I Exec nm-applet
```

You can also use just StartFunction and prepend your commands with Test commands which check whether the window manager has started or restarted and run the action only if the test is true. Using this method, *nm-applet* could be started in the following manner:

```
AddToFunc StartFunction
+ I Test (Init) Exec nm-applet
```

配置

The following configuration file locations are supported:

- `$HOME/.fvwm/config`
- `/usr/local/share/fvwm/config`

The following configuration locations are supported as of version 2.6.8, but may not be supported in the future:

- `$HOME/.fvwm/.fvwm2rc`
- `$HOME/.fvwm2rc`
- `/usr/local/share/fvwm/.fvwm2rc`
- `/usr/local/share/fvwm/system.fvwm2rc`
- `/etc/system.fvwm2rc`

As of version 2.6.7, FVWM ships with a new default configuration, located in `/usr/share/fvwm/default-config`. As such, the older sample configuration files are no longer provided. However, they can still be viewed on GitHub. The fvwm-themes project also provides ready-made configurations though it should be noted that these have not been updated since 2003 and may require modifications to work correctly with more recent FVWM versions.

提示：You can split up your configuration into multiple files and then source those files from the main configuration file using the `read` command. The syntax is `read /path/to/file`.

The virtual desktop

For its virtual desktop, FVWM implements both workspaces (used by window managers such as Metacity and Openbox) and viewports (used by window managers such as Compiz). See for a description of the differences between workspaces and viewports. FVWM refers to workspaces as desks and viewports as pages.

Pages in FVWM are arranged in a grid. The number of pages used can be defined with the `DesktopSize` command. For instance, adding `DesktopSize 3x3` to your configuration file will give you 9 pages, arranged in a 3x3 grid. Pages can be navigated using the pager module or with the `GoToPage` command which could be mapped to a keyboard shortcut or menu entry. For instance, the command `GoToPage -1p +0p` will move the viewport 1 page to the left of the current page.

The number of desks available in FVWM is very large, the minimum desk number is -2147483648 and the maximum is 2147483647. By default, FVWM starts on desk 0. Desks can be navigated using the pager module (if it is configured to show a number of desks) or with the `GoToDesk` command - `GoToDesk +1` will move to the next desk, relative to the currently used desk.

Keyboard and mouse bindings

Keyboard or bindings can be defined in the configuration file with the `Key` or `Mouse` command. The syntax of the command takes the following form: `Key/Mouse *(window)* *button/key name* *context* *modifiers* *action*`. For instance, the following example will launch an XTerm on an `Alt+F2` keypress: `Key F2 A 1 Exec xterm`. Note that the `*(window)*` argument is optional.

The context value defines where the binding will be applied. The following contexts are valid: **R** (root window), **W** (application window), **D** (desktop manager window - PCManFM desktop manager for instance), **T** (title bar), **S** (window side, top, bottom), **\[\] - _** (left side, right side, top, bottom respectively), **F** (window frame corners), **< > ^ v** (left, right, top, bottom corners respectively), **I** (icon window), **0-9** (titlebar buttons) and **A** (all contexts). Any combination of these letters is also acceptable.

The modifier value can be one of the following: **A** (any), **C** (control), **S** (shift), **M** (meta), **N** (none), or **1-5**, representing the X Modifiers - run `xmodmap` to see which X modifier is which. Multiple modifiers should not be spaced. For instance, to use the modifier `Control+Alt`, you would supply as the modifier argument `C1`.

The action must be an FVWM function to run, such as the *Quit* function or *Menu* function. Execution of external commands, such as *xterm*, can be achieved with the *Exec* function, as shown above.

Window decoration

Titlebar buttons

FVWM can provide up to 10 window buttons. These are numbered 0-9. Even numbers indicate buttons located on the right hand side of the titlebar whilst odd numbers indicate buttons located on the left. The layout is as follows:

```
1 3 5 7 9   0 8 6 4 2
```

Window buttons will remain hidden unless a `Mouse` command is used which specifies one of the titlebar buttons as the context. For instance, to activate the

rightmost titlebar button and make it close the window on a left click, one would use the following command:

```
Mouse 1 2 A Close
```

Mouse is the name of the command, *1* is the mouse button, *2* is the number of the rightmost titlebar button, *A* is the modifier (any) and *Close* is the action to be taken. See #Keyboard and mouse bindings for more information.

Button styles

The style of your titlebar buttons can be configured with the `ButtonStyle` command. This takes the following syntax:

```
ButtonStyle button-number state style -- flag
```

The state can be one of *ActiveUp*, *ActiveDown*, *InactiveUp*, *InactiveDown*. *ActiveUp* and *ActiveDown* refer to the un-pressed and pressed button states of the active window. Likewise for the Inactive states. One can also use just *Active* or *Inactive* which are shortcuts for both the pressed and un-pressed button states.

The style argument can be one of the following:

- Simple - does nothing.
- Default - takes argument of button number for default style to load.
- Solid - fill button a solid color.
- ColorSet - fill button with the ColorSet specified - takes an alpha argument between 0 and 100.
- Vector - draws a line pattern - using the keyword Vector is optional as this is a standard style.
- ?Gradient - fills the button with a gradient - see the Color Gradients section of fvwm(1) for the syntax.
- Pixmap - fills the button with a given pixmap - see also the following variants: AdjustedPixmap, ShrunkPixmap, StretchedPixmap, TiledPixmap.
- MiniIcon - fills the button with the window's mini icon.

A number of vector styles are documented here. You can also create your own vector buttons using this vector buttons viewer. Finally, see this page for some example decoration configurations that use pixmaps, including imitations of Crux (a Sawfish theme), Mac OS and Windows 98.

The flag affects the state for a button. Some examples of flags include *Raised*, *Sunk* and *Flat*. For more information, see fvwm(1) and look for the ButtonStyle section.

注意：To give the buttons the same background as the title, use the `UseTitleStyle` flag. Alternatively, specify your button backgrounds using the `ButtonStyle` command and then follow this with an `AddButtonStyle` command to specify the vector or pixmap image. `AddButtonStyle` takes the same syntax as `ButtonStyle`.

Title and border styles

The window titles and borders can be configured with the `TitleStyle` and `BorderStyle` commands respectively.

TitleStyle can take the following arguments:

```
TitleStyle justify Height height-in-pixels
```

The justify argument can be *LeftJustified*, *RightJustified* or *Centered*. TitleStyle and BorderStyle can take the following arguments:

```
TitleStyle/BorderStyle state style -- flag
```

See #Button styles for the *state*, *style* and *flag* arguments.

Menus

注意：The `AddToMenu` command is cumulative, meaning that if it is used twice on the same menu, the items specified in the second command will be appended to the menu in its current state instead of overwriting the previous menu configuration. For this reason, it is good practice to call `DestroyMenu *menu-name*` before calling `AddToMenu` unless the cumulative behavior is desired.

Menus can be created with the `AddToMenu` command. This takes the following syntax:

```
AddToMenu menu-name menu-title Title
```

Remove the *menu-title* and Title arguments to create a menu with no title. Subsequent entries in the menu take the following syntax:

```
+ entry-name action
```

See the following example:

```
AddToMenu "Web" "Web Browsers" Title
+ "Firefox" Exec firefox
+ "Chromium Exec chromium
+ "Opera"   Exec opera
```

Use the `Popup` command to show other menus. For instance, to include the menu defined above in another menu one could use the following syntax:

```
+ "Web Browsers" Popup "Web"
```

FVWM menus also support icons. To give a menu entry an icon, provide the path to the icon enclosed in `%` signs after the entry name - see below:

```
+ "Chromium %/usr/share/icons/hicolor/16x16/apps/chromium.png%" Exec chromium
```

Use the *xdg_menu* tool, provided by archlinux-xdg-menu包, to automatically generate menus - see Xdg-menu#Fvwm2.

Dynamic menus

Menus in FVWM can be dynamic, meaning that their content is refreshed every time the menu is opened. This can be useful when using a menu generator, such as one that constructs a menu of applications from XDG desktop entries, and you want the content to always be up to date.

Dynamic menus can be created in FVWM by using the *DynamicPopUpAction* and *DynamicPopDownAction* keywords. The former can be used to create/recreate the menu when it is opened whilst the latter can be used to clear the menu when it is closed. It is important to note that for a submenu to be dynamic, the parent menu that it is included in must also be created dynamically even if it does not contain any dynamic content.

Consider a hypothetical menu generator called *my-menu* which generates output of the sort below.

```
DestroyMenu recreate Submenu1
AddToMenu Submenu1 "Submenu 1" Title
+ "Item 1" Nop
+ "Item 2" Nop

DestroyMenu recreate MyMenu
AddToMenu MyMenu "My Menu" Title
+ "Submenu 1" Popup Submenu1
```

注意：Note the use of the `recreate` argument to the `DestroyMenu` command. This must be used for **all** menus which are created dynamically (that includes all menus and submenus generated by a menu generator), otherwise they will not be refreshed properly.

Now suppose that we wished to incorporate *my-menu* into our root menu dynamically. This means that the root menu must also be created dynamically. To accomplish this, we need to define a function that constructs the root menu and a function that calls the dynamic menu generator using the `PipeRead` command. Then in the menu definitions themselves, DynamicPopUpAction can be used to call those functions. See the example below.

```
DestroyMenu RootMenu
AddToMenu RootMenu
+ DynamicPopUpAction Function CreateRootMenu
+ DynamicPopDownAction DestroyMenu recreate RootMenu

DestroyFunc CreateRootMenu
AddToFunc CreateRootMenu
+ I DestroyMenu recreate RootMenu
+ I AddToMenu RootMenu "Root Menu" Title
+ I AddToMenu RootMenu "My Menu" PopUp MyMenu
+ I AddToMenu RootMenu "XTerm" Exec xterm
+ I AddToMenu RootMenu "Firefox" Exec firefox

DestroyMenu MyMenu
AddToMenu MyMenu
+ DynamicPopUpAction Function CreateMyMenu
+ DynamicPopDownAction DestroyMenu recreate MyMenu

DestroyFunc CreateMyMenu
AddToFunc CreateMyMenu
+ I DestroyMenu recreate MyMenu
+ I PipeRead 'my-menu'
```

Style command

注意：Most styles can be negated with a `!`. For instance the style `Title` forces the window manager to give the window a title whilst `!Title` has the opposite effect.

The `Style` command allows one to configure various aspects of the window manager itself and also to set behaviors for certain windows. The syntax is `Style *window-name* *stylename*`. The *window-name* argument can be a window name, class, title name or resource string. Use `*` to match all windows. See fvwm(1) for all available styles - some examples are provided below:

- `Style "*" CascadePlacement` - make the window manager use the cascade placement algorithm for all new windows.

- `Style "Chromium" PositionPlacement center` - ensure that all new Chromium windows are placed in the center of the screen.
- `Style "xterm" StartIconic` - ensure that all new XTerm windows start iconified.
- `Style "*" HilightBack indianred` - set the frame background of any focused window to the color indianred.

Functions

注意：The `AddToFunc` command is cumulative, meaning that if it is used twice on the same fuction, the items specified in the second command will be appended to the function in its current state instead of overwriting the previous function configuration. For this reason, it is good practice to call `DestroyFunc *func-name*` before calling `AddToFunc` unless the cumulative behavior is desired.

FVWM provides many built in functions, examples being `Close` to close a window or `Exec` which allows the execution of an external command. Users can also define their own functions or add to existing functions using the `AddToFunc` command. This uses the following syntax: `AddToFunc *func-name* I|M|C|H|D *action*`. The letter codes stand for the following: **I** - execute immediately, **M** - execute when the user moves the mouse, **C** - execute on mouse click, **H** - execute when the user holds the mouse button, **D** - execute when the user double clicks the mouse button. Below is a trivial example of a function:

```
AddToFunc VolumeFunc
+ I Exec xterm -e alsamixer
```

One can also use conditional commands (see fvwm(1) and look for the Conditional Commands section). For instance, suppose one wanted a function that would close all windows in the current page other than the one which has focus. That function can be defined as below:

```
AddToFunc CloseAllButThis
+ I All (CurrentPage, !Focused) Close
```

The `All` conditional matches all windows that meet the conditions defined in parenthesis. Functions can execute more than one action, just add one line for each action beginning with a plus sign:

```
AddToFunc MyFunc
+ I action1
+ D action2
+ I action3
```

Modules

注意：A number of older FVWM modules have been removed as of version 2.6.7. Those who wish to continue using these modules should use fvwm2-stable which is an LTS release representing the version prior to 2.6.7.

Modules are separate programs, spawned by FVWM that can add extra functionality. Modules can be spawned using the following syntax: `Module *ModuleName* (*identifier*) *ModuleArgs*`.

Sometimes, one might want to spawn multiple instances of the same type of module, each with their own separate configuration. In this case, one should spawn the module with an identifier, for instance:

```
AddToFunc StartFunction
+ I Module FvwmButtons Panel1
+ I Module FvwmButtons Panel2
```

where *Panel1* and *Panel2* are the identifiers.

Most modules will have a number of module commands which can be used to configure the module's appearance or behavior. Use the following syntax:

```
*ModuleName/Identifier: module-command command-args
```

For instance, if one spawned an FvwmPager with the identifier MyPager then one could configure it in the following fashion:

```
*MyPager: Geometry 135x90+0+0
*MyPager: Back midnightblue
```

FvwmPager

The FvwmPager is a module which provides a visual representation of the desks and pages provided by the window manager. Like all modules, it must be spawned by FVWM. To start an FvwmPager, add something similar to the following to your StartFunction:

```
+ I Module FvwmPager
```

With no arguments, the FvwmPager will only display the viewports for desk 0. With the arguments 0 9, FvwmPager will show the 10 desks from 0-9.

```
+ I Module FvwmPager 0 9
```

With the argument * the FvwmPager will show only one desk but it will always be the desk that is currently being used.

```
+ I Module FvwmPager *
```

See FvwmPager(1) for a list of module commands.

FvwmButtons

FvwmButtons is a module which can create a box of buttons which can perform actions when pressed. FvwmButtons can also "swallow" application windows. This might be useful for swallowing a system tray window or a clock window. The size of the panel is automatically determined (the panel will resize to accommodate all elements) however it is also possible to manually set a size using the `Geometry` command. This takes a standard X geometry. The basic commands needed to create an FvwmButtons panel are outlined below.

Set the number of rows and columns:

```
*FvwmButtons: Rows x
*FvwmButtons: Columns x
```

Note that items are added by filling each row from left to right, from the top row to the bottom one.

提示：For debugging purposes, you might wish to set the `BoxSize` option to `fixed`. This ensures that only the specified number of rows and columns are used. Specifying an impossible layout (such as

adding 5 buttons to a panel that has 4 columns and 1 row) will mean that the buttons panel fails to start. More lenient BoxSize algorithms such as `smart` will add extra rows to try and accommodate all elements which could make the configuration error harder to spot.

Create a button (this example creates a button with a title and an icon which launches an XTerm when left clicked):

```
*FvwmButtons: (Title "Xterm", Icon /usr/share/pixmaps/mini.xterm_48x48.xpm, Action (Mouse1) Exec xterm)
```

You can omit any of these arguments if you so choose.

Swallow a window (this example swallows a stalonetray):

```
*FvwmButtons: (Swallow(UseOld, NoClose) "stalonetray" "Nop")
```

The Swallow command takes a number of "hangon" arguments. Here, the *UseOld* and *NoClose* arguments have been used. UseOld means that an existing window will be swallowed if it exists. NoClose means that the window will not be closed if the FvwmButtons process is terminated. The *stalonetray* argument is the class of the window that we want to swallow. Replace as appropriate. The last argument is a command to start the application that is to be swallowed. In this example, it is assumed that the application has already been started so the argument provided is *Nop* which is a function that does nothing. One could replace this with "`Exec stalonetray`" to start the application from the FvwmButtons module instead of assuming that the application has been started elsewhere.

Containers are spaces defined which can span multiple rows and columns or subdivide a row or column into more rows or columns. This can be useful for instance if one wants to allocate a certain percentage of space to an element. Say one wants to swallow an XClock and allocate 100% of the width and 80% of the height to the XClock. This can be defined as such:

```
*FvwmButtons: Rows 5
*FvwmButtons: Columns 1
*FvwmButtons: (1x4, Container)
*FvwmButtons: (Swallow(UseOld, NoClose) "xclock" "Nop")
*FvwmButtons: (End)
```

Note that a container is created by defining a certain number of columns and rows and then using the keyword container. Elements inside the container are defined underneath this line and the container is then closed with the *End* command.

提示：To define containers in terms of pixel dimensions, ensure that the number of rows and columns in the buttons panel is equal to the pixel height and width respectively of the panel. So for a 300x200 panel set 300 columns and 200 rows.

For a full list of options, see FvwmButtons(1).

FvwmEvent

FvwmEvent is a module which can be used to bind a function or an audio file to a window manager event (such as the closing of a window). In the case of an audio file, the audio will be played when the event that it is bound to occurs. FvwmEvent can be spawned from within Fvwm by adding the following to your `StartFunction`:

```
+ I Module FvwmEvent
```
FvwmEvent can also be spawned with an identifier - see #Modules. Once spawned, FvwmEvent will run in the background, waiting for events that it has been configured to recognize. Events can be configured in the following form:

```
* FvwmEvent: windowshade Lower
```
where `windowshade` is the event and `Lower` is the command to be executed when that event occurs. In the case where you wish to execute a function with arguments, that function and its arguments need to be quoted - see below:

```
* FvwmEvent: new_page "Exec xterm"
```
For a full list of events, see FvwmEvent(1).

FvwmIdent

FvwmIdent is a module which can display many items of information about a particular window that it is called on, such as the window's class name, resource name, layer, geometry and more. This information is displayed in a separate window which is created by the module. FvwmIdent can be started using the `Module FvwmIdent` command. This could be bound to a menu entry or a hotkey. When an FvwmIdent is started in the context of a window, that window's information will be displayed. Otherwise, the user will be prompted to select a window manually.

Colors

FVWM color styles can take a number of color code types such as hexcolors (`#ffffff` for instance), rbg colors (`rgb:ff/ff/ff` for instance) as well as the pre-defined X11 colors.

The following styles might be useful:

- `Color` - this takes two arguments, the color of the unfocused window title text and the color of the unfocused window frame separated by a forward slash, *black/lightgrey* for example.
- `HilightBack` - takes a single argument, the color of the focused window frame.
- `HilightFore` - takes a single argument, the color of the focused window title text.

Colorsets

Colorsets in FVWM are a set of four colors (a foreground color, a background color, a shadow color and a highlight color) as well as an optional background pixmap. Any part of FVWM that uses a particular colorset will be affected if that colorset is changed.

All colorsets are identified by a number. Any numbering convention can be used; the fvwm-themes project documents one such convention that uses the first 40 colorsets (0-39). See .

Colorsets can be created with the `ColorSet` command - syntax: `ColorSet *number* *options*`. See fvwm(1) - the Colorsets section - for more information.

提示：Using `Style "*" Colorset *num*` overrides the `Color` style and using `Style "*" HilightColorset *num*` overrides the `HilightFore` and `HilightBack` styles.

Fonts

For styles such as `Font`, use xorg-xfontsel包 to determine the correct font names for X11 fonts - see X Logical Font Description and Font configuration for more information. You can also specify xft fonts, for example: `"xft:DejaVu Sans:size=10"`.

Icons

Windows in FVWM can be iconified (minimized). This means that the window will disappear from view and be replaced by an icon on the desktop, similar to the behavior in Microsoft Windows 3.1.

The program itself will supply the icon. One can also set a default icon to be used, in case a program does not have an icon to supply: `Style "*" Icon /path/to/default/icon.png`.

To disable icons altogether, use the following style: `Style "*" NoIcon`. This means that the window will simply disappear when iconified.

The positioning of icons can be controlled using the `IconBox` style - note that this is not the same thing as the FvwmIconBox module - along with the `IconFill` style. Use `Style "*" IconBox none` to disable the IconBox. This means that icons will be placed at the position of the top left corner of the window. Else, use `Style "*" IconBox l t r b` where *l t r b* stand for *left*, *top*, *right* and *bottom* respectively. These arguments should be the number of pixels away from the screen edge that the IconBox edge should be. Hence, arguments of `+0 +0 -0 -0` means that the IconBox will fill the entire screen.

Multiple IconBox styles can be defined but they must be defined on the same line, for instance:

```
Style "*" IconBox +0 +800 -100 -0, IconBox -200 +0 -0 -120
```

When the first IconBox overflows, icons will then be placed in the second IconBox. Note that if the last IconBox overflows then the default IconBox will be used which covers the entire screen and fills from top to bottom - left to right.

Use the `IconFill` style to control how icons will be filled. A style of `Style "*" IconFill left bottom` means that icons will be filled from left to right - bottom to top (Motif Window Manager behavior). A style of `Style "*" IconFill top left` means that icons will be filled from top to bottom - left to right. Note that the IconFill style should be defined on the same line as the IconBox style, for instance:

```
Style "*" IconBox +0 +0 -0 -0, IconFill left bottom
```

提示与技巧

MWM compatibility options

FVWM provides a number of options that allow it to mimic the appearance and behaviour of MWM (Motif Window Manager).

- `Emulate Mwm` - this commands places the geometry feedback window in the center of the screen.
- `MenuStyle Mwm` - this command gives menus the appearance of Motif menus.
- `MwmButtons` - this style makes the maximize button look pressed in when a window is maximized.
- `MwmBorder` - this style makes the window border bevel more closely match the style of Mwm window borders
- `MwmDecor` - this style makes FVWM attempt to honor MWM hints that some applications might set.
- `MwmFunctions` - this style make FVWM attempt to recognize and respect functions that MWM would prohibit.
- `HintOverride` - this style is similar to MwmFunctions but instead it shades out the prohibited functions but allows the user to perform them anyway.

Force icon size and background

The sizing of icons is not regular as different programs provide icons of differing sizes. Use the `IconSize` style to force a regular size: `Style "*" IconSize 48 48` for instance. Note that icons larger than the given size will be clipped.

By default, icons also have no background. You can use the `IconBackgroundColorset` style to force icons to have a background. See #Colorsets.

Do not use a wireframe when moving or resizing windows

Use the `OpaqueMoveSize unlimited` command to view the window itself when moving.

Use `Style "*" ResizeOpaque` to view the window itself when resizing.

Disable edge scrolling

To disable scrolling to the next viewport when moving the mouse pointer to the screen edge, use the following command: `EdgeScroll 0 0`.

Use ClickToFocus

Ensure that perl-tk包 and perl-x11-protocol包 are installed. The use the following style command: `Style "*" ClickToFocus`. See fvwm(1) for other focus behaviors.

Window tiling

The following functions can tile a window to the left half, right half, top half or bottom half of the screen, or to each corner of the screen, when called and return the window to its original position and size when called again.

```
DestroyFunc TileLeft
AddToFunc TileLeft
+ I ThisWindow (!Shaded, !Iconic) Maximize 50 100
+ I ThisWindow (Maximized, !Shaded, !Iconic) Move +0 +0

DestroyFunc TileRight
AddToFunc TileRight
+ I ThisWindow (!Shaded, !Iconic) Maximize 50 100
+ I ThisWindow (Maximized, !Shaded, !Iconic) Move -0 +0

DestroyFunc TileTop
AddToFunc TileTop
+ I ThisWindow (!Shaded, !Iconic) Maximize 100 50
+ I ThisWindow (Maximized, !Shaded, !Iconic) Move +0 +0

DestroyFunc TileBottom
AddToFunc TileBottom
+ I ThisWindow (!Shaded, !Iconic) Maximize 100 50
+ I ThisWindow (Maximized, !Shaded, !Iconic) Move +0 -0

DestroyFunc TileTopLeft
AddToFunc TileTopLeft
+ I ThisWindow (!Shaded, !Iconic) Maximize 50 50
+ I ThisWindow (Maximized, !Shaded, !Iconic) Move +0 +0

DestroyFunc TileTopRight
AddToFunc TileTopRight
+ I ThisWindow (!Shaded, !Iconic) Maximize 50 50
+ I ThisWindow (Maximized, !Shaded, !Iconic) Move -0 +0

DestroyFunc TileBottomLeft
AddToFunc TileBottomLeft
+ I ThisWindow (!Shaded, !Iconic) Maximize 50 50
+ I ThisWindow (Maximized, !Shaded, !Iconic) Move +0 -0

DestroyFunc TileBottomRight
AddToFunc TileBottomRight
+ I ThisWindow (!Shaded, !Iconic) Maximize 50 50
+ I ThisWindow (Maximized, !Shaded, !Iconic) Move -0 -0
```

Transfer focus on page or desk switch

注意：

- Using FvwmEvent allows for a window in a page to be automatically focused when clicking on that page in the FvwmPager. However, it breaks the right-click and drag panning functionality of the pager. For this reason, you might prefer to call `GoToPage` and `Focus-Previous` from within another function bound to a hotkey instead of using FvwmEvent.
- If you are not using ClickToFocus, you will need to temporarily set the focus method to ClickToFocus before the desk switch is initiated. If you do not do this, any window under the pointer in the new desk will immediately receive focus, even if it is not the previously focused window. Note that this means you cannot use the FvwmEvent approach. Instead, create a function that changes the focus method and then calls `GoToDesk` and `Focus-Previous`.

If you are using ClickToFocus, you might wish to automatically transfer keyboard focus when switching pages or desks to the previously focused window in that page or desk. Otherwise, you will have to click on the window you wish to interact with on

every page or desk switch. This can be accomplished by using the FvwmEvent module to bind a function which focuses the currently or previous focused window to the `new_page` and `new_desk` events.

Below is an example of such a function:

```
DestroyFunc Focus-Previous
AddToFunc Focus-Previous
+ I All ($0, Focused) FlipFocus $1
+ I TestRc (NoMatch) Prev ($0, AcceptsFocus) FlipFocus $1
```

The `$0` argument should be either `CurrentPage` or `CurrentDesk`. The `$1` argument is for supplying the `NoWarp` argument to `FlipFocus` - by default, FlipFocus will initiate a page switch to the page containing the focused window. This behaviour is useful when switching desks but must be disabled when switching pages because windows can span more than one page.

With the function appropriately configured, ensure that FvwmEvent is started and then bind it to the required events as show below:

```
* FvwmEvent: new_page "Focus-Previous CurrentPage NoWarp"
* FvwmEvent: new_desk "Focus-Previous CurrentDesk"
```

Toggle window decorations

Window decorations (borders and titlebars) can be toggled on or off for a selected window using the function defined below.

```
DestroyFunc UndecorateWin
AddToFunc UndecorateWin
+ I ThisWindow (HasHandles) WindowStyle !Title, !Borders
+ I ThisWindow (!HasHandles) WindowStyle Title, Borders
```

提示：If you are looking to make windows fullscreen, this functionality has been added to the *Maximize* command as of FVWM 2.6.8, use the following: `Maximize Fullscreen`

故障排除

Maximized applications do not fully cover screen

Some applications, such as XTerm, supply a maximum size to the window manager that might be smaller than the screen size. This means that if such an application is maximized, it will not cover the whole screen. To force FVWM to ignore these hints, use the following: `Style "*" ResizeHintOverride`.

Stop modifiers from interfering with mouse and key bindings

NumLock, CapsLock and ScrollLock can intefere with ClickToFocus as well as mouse and key bindings. To disable this behavior, use the following command: `IgnoreModifiers L25`.

Window start position changes on launch

You may find that with some programs the starting position of its window changes every time the program is launched. Similarly some windows automatically resize themselves which can also affect their position.

This is typically due to *PPosition* (program position) or *USPosition* (user specified position) hints which applications can set and which FVWM respects by default. For troublesome windows, you can configure FVWM to ignore the hints that are causing the problem. The first step is to get the class name or resource name of the window in question - use the #FvwmIdent module for this. Then try disabling either the PPosition or USPosition hints for the window. For example:

```
Style chromium !UsePPosition
```

For windows whose position is affected by resizing this can typically be fixed by setting the FixedPPosition style on the window which causes FVWM to ignore attempts by the window to change its position. This can be set in conjunction with ignoring USPosition hints if necessary, see below:

```
Style VirtualBox !UseUSPosition, FixedPPosition
```

注意：Windows belonging to an application such as popup dialogs and menus are normally declared to be *Transient*. To ignore PPosition or USPosition hints on transient windows, use the `!UseTransientPPosition` or `!UseTransientUSPosition` styles respectively.

参见

- FVWM on GitHub.
- FVWM beginners guide - still works but is somewhat outdated.
- FVWM Homepage - includes documentation and a FAQ.
- FVWM Wiki.
- FVWM Forums.
- fvwm(1)
- FVWM Themes - Box-Look.org.
- FVWM Configurations - threads in the FVWM forums.
- FVWM Tips - forum post by Thomas Adam - an FVWM developer.

6.8 Openbox

难度 高级 | **篇幅** 37.2 KB | **所属** 堆叠式窗口管理器

本节将讲到

- 安装
- 启动
- 配置
- 重新配置Openbox
- 键绑定

Openbox 是一个轻量、强大且高度可定制的堆叠式窗口管理器，它支持大量扩展标准。它可以单独作为某个桌面环境的一个基础组件独立构建和运行，也可以集成于其它桌面环境如KDE或Xfce中，作为备选窗口管理器。LXDE和LXQt桌面环境则是基于Openbox构建而成。

安装

只需安装openbox包软件包。也可以安装TTF字体，如ttf-dejavu包和ttf-liberation包。

启动

单独运行

通过 xinit 运行 `openbox` 或者 `openbox-session` 。 注意只有 `openbox-session` 支持 #自动启动.

注意：执行openbox-session后,屏幕上只会出现灰色的空白界面。 尝试移动你的鼠标并**右键单击** ，如果显示一个Openbox菜单，说明它真的在工作了。

其它桌面环境

注意：

- 如果你把一个 桌面环境 的原有窗口管理器替换为 Openbox，请记住 Openbox 不会提供任何的混成特效 (比如透明度)。 请参阅 #混成特效.
- Openbox可以与GNOME应用程序一起工作(见GTK#Client-side decorations).

请参阅桌面环境#使用其它窗口管理器。

配置

注意：本地配置文件总是会覆盖全局配置。

四个关键文件构成了 openbox配置的基础，每个文件都有其独特的作用。它们分别是： `rc.xml`, `menu.xml`, `autostart`, 和 `environment`。 尽管这些文件将在下面进行更详细的讨论，但要开始配置Openbox，首先需要基于它们创建一个本地 Openbox 配置文件 (即针对您特定的用户帐户) 。 这可以通过从全局的 `/etc/xdg/openbox` 配置文件中复制它们作为模板来完成 (适用于任何和所有用户):

```
$ mkdir -p ~/.config/openbox
$ cp -a /etc/xdg/openbox/ ~/.config/
```

rc.xml

提示：自定义键盘快捷键 (键绑定) 必须添加到此文件的 `<keyboard>` 部分， 并在 `<!-- Keybindings for running aplications -->` 标题下方添加。

`~/.config/openbox/rc.xml` 是主要的配置文件，负责确定整个会话的行为和设置，其中包括：

- 键盘快捷键 (e.g. starting applications; controlling the volume)
- 主题
- 桌面和虚拟桌面设置

- 应用程序窗口设置

该文件也是预配置的，这意味着只需要修改现有内容就可以根据个人喜好自定义行为。

注意：仅当定义了 x 和 y 位置时，与每个监视器上的应用程序固定放置相关的每个应用程序的特定设置才能生效。

menu.xml

`~/.config/openbox/menu.xml` 定义了桌面菜单的类型和行为，可以通过右键单击背景访问。 尽管提供的默认菜单是一个**静态菜单** (即在安装新应用程序时不会自动更新), 但也有可能使用**动态菜单**，它们将自动更新。

在 #菜单部分下面详细讨论了可用的选项。

自动启动

`openbox-session` 提供了两种自动启动机制： XDG Autostart (仅在安装了 python-pyxdg包 时可用) 和 Openbox's own autostart mechanism.

Openbox 自己的自动启动机制：

- 来源 `/etc/xdg/openbox/environment`
- 来源 `~/.config/openbox/environment`
- 运行 `/etc/xdg/openbox/autostart`
- 运行 `~/.config/openbox/autostart`

解决`~/.config/openbox/autostart` 中命令顺序执行错误（或者被跳过）的问题通常可以通过增加小的延迟来解决。例如：

```
xset -b
(sleep 3s && nm-applet) &
(sleep 3s && conky) &
```

环境

`~/.config/openbox/environment` 可以用于导出和设置相关的环境变量，例如：

- 定义新的路径（例如执行需要将整个路径列出的命令）
- 更改语言设置
- 定义其他要使用的变量（例如，可以在此处列出对于GTK主题的修复）

主题

安装 obconf包 和/或 lxappearance-obconf包 ，用于配置视觉设置和主题的图形界面。

可以在 openbox-themesAUR 软件包或 AUR 中找到大量精选的主题。一些 GTK#Themes 也附带了Openbox主题。 安装时，Openbox特定和兼容的主题将被安装到 `/usr/share/themes` 目录，并且可以立即选择使用。

box-look.org 是一个优秀且历史悠久的主题资源网站。

deviantART.com 是另一个优秀的资源网站。在网上还可以找到更多的资源。

编辑或创建

提示：对于主题，最好将其复制到您的主目录而不是编辑在 `/usr/share/themes/`中找到的主题。这样做可以在出现任何问题时保留原始文件，并确保在更新时不会被覆盖您所做的更改。

在官方的 openbox.org 网站上有详细介绍创建新主题或修改现有主题的过程。 obthemeAUR 是一个用户友好的图形界面工具，用于进行这些操作。

图形界面配置

有几个图形界面应用程序可供选择，可以快速简便地配置您的Openbox桌面环境。

- **ObConf** — 一个基于GTK3的配置工具，用于Openbox窗口管理器。

: http://openbox.org/wiki/ObConf:About || obconf包

- **LXAppearance ObConf** — 这是一个用于配置Openbox的LXAppearance插件。请注意，该插件并不提供所有用于配置Openbox的选项，因此您可能仍然需要安装obconf来完成配置。

: https://lxde.org || lxappearance-obconf包

- **LXInput** — LXDE键盘和鼠标配置。

: https://lxde.org || lxinput包

- **LXRandR** — LXDE显示器配置。

: https://wiki.lxde.org/en/LXRandR\[失效链接 2022-09-22 ⓘ\] || lxrandr包

- **obkey** — 配置Openbox的键盘快捷键。

: https://code.google.com/p/obkey/ || obkeyAUR

- **obapps** — Openbox中用于配置应用程序设置的图形化工具。

: https://sourceforge.net/projects/obapps/ || obappsAUR

在 #菜单部分讨论与配置Openbox桌面菜单相关的程序和应用程序。

重新配置Openbox

提示：如果还没有，为了方便起见，值得将此命令添加到菜单和/或键绑定中。

Openbox并不总是会在会话中自动反映对其配置文件所做的更改。因此，在编辑完配置文件之后，需要手动重新加载这些文件。要执行此操作，请输入以下命令：

```
$ openbox --reconfigure
```

打算将此命令作为键绑定添加到 `~/.config/openbox/rc.xml` ， 将命令列为 `reconfigure` 即可。 以下是一个示例，使用 `Super+F11` 键绑定：

```
<keybind key="W-F11">
  <action name="Reconfigure"/>
</keybind>
```

键绑定

所有键绑定都必须添加到 `~/.config/openbox/rc.xml` 文件中，并放置在 `<!-- Keybindings for running aplications -->` 标题下方。然在这里提供了简要概述，但更详细的键绑定说明可以在openbox.org 网站上找到。

键绑定可以使用以下语法添加到配置文件中：

```
<keybind key="my-key-combination">
  <action name="my-action">
    ...
  </action>
</keybind>
```

执行外部命令的操作名称是Execute。使用以下语法来定义要执行的外部命令：

```
<action name="Execute">
  <command>my-command</command>
</action>
```

请参阅 the Openbox wiki 以获取所有可用操作的列表。

提示：obkeyAUR 实用程序提供了一个图形界面来配置按键绑定。在使用 *obkey* 之前，您应该使用 *obconf* 来创建 `~/.config/openbox/rc.xml` 文件。

对于键位绑定，使用标准的字母数字键是很直观的，但其他类型的键，如 `modifiers`， `multimedia` 和 `navigation`，会分配特殊的名称。

组合键

`Modifier` 键在键绑定中起着重要作用（例如，按住。`shift` 键 或 `CTRL` / `control` 与其他键组合以执行某项操作）。使用修饰键有助于防止冲突的键绑定，即将两个或多个操作链接到同一个键或键组合。使用修饰键与其他键结合的语法是：

```
"<modifier>-<key>"
```

修饰符代码如下：

- `S`: Shift
- `C`: Control / CTRL
- `A`: Alt
- `W`: Super / Windows
- `M`: Meta
- `H`: Hyper (如果它绑定到某个东西上)

多媒体键

在可用的情况下，可以设置适当的 `multimedia` 键来执行其预期的功能，例如控制音量和/或屏幕亮度。 这些通常会集成到 `function` 中，并通过相应的符号进行标识。请参见 Extra keyboard keys 获取更多信息。

音量和亮度的多媒体代码如下（请注意，命令仍然需要分配给它们才能实际发挥作用）：

- `XF86AudioRaiseVolume`: 增加音量
- `XF86AudioLowerVolume`: 减小音量
- `XF86AudioMute`: 静音/取消静音
- `XF86MonBrightnessUp`: 增加屏幕亮度
- `XF86MonBrightnessDown`: 降低屏幕亮度

要查看完整的XF86多媒体键列表，请参阅 LQWiki:XF86 keyboard symbols。

音量控制

控制音量所使用的命令取决于音频播放系统是使用 ALSA，PulseAudio，还是 OSS。

- ALSA: 请参阅 Advanced Linux Sound Architecture#Keyboard volume control.
- PulseAudio: 请参阅 PulseAudio#Keyboard volume control
- OSS: 请参阅 Open Sound System#Keyboard volume control.

导航键

这些是方向/箭头键，通常用于将光标向上、向下、向左或向右移动。以下是（自说明的）导航代码：

- `Up`: 上
- `Down`: 下
- `Left`: 左
- `Right`: 右

菜单

在Openbox中，有三种类型的菜单可供选择使用： `static`, `pipes` (动态) 和 `generators` (静态或动态). 这些菜单可以单独使用，也可以组合使用。

静态菜单

As the name would suggest, this default type of menu does not change in any way, and may be manually edited and/or (re)generated automatically through the use on an appropriate software package.

Fast and efficient, while this type of menu can be used to select applications, it can also be useful to access specific functions and/or perform specific tasks (e.g. desktop configuration), leaving the access of applications to another process (e.g. the synapse包 or xfce4-appfinder包 applications).

The `~/.config/openbox/menu.xml` file will be the sole source of static desktop menu content.

菜单生成器

menumaker包 automatically generates `xml` menus for several window managers, including Openbox, Fluxbox, IceWM and Xfce. It will search for all installed executable programs and consequently create a menu file for them. It is also possible to configure MenuMaker to exclude certain application types (e.g. relating to GNOME or KDE), if desired.

Once installed and executed, it will automatically generate a new `~/.config/openbox/menu.xml` file. To avoid overwriting an existing file, enter:

```
$ mmaker -v OpenBox3
```

Otherwise, to overwrite an existing file, add the `force` argument (`f`):

```
$ mmaker -vf OpenBox3
```

Once a new `~/.config/openbox/menu.xml` file has been generated it may then be manually edited, or configured using a GUI menu editor, such as obmenuAUR.

obmenu

警告： `obm-xdg` - a pipe menu to generate a list of GTK and GNOME applications - is also provided with obmenu. However, it has long-running bugs whereby it may produce an invalid output, or even not function at all. Consequently it has been omitted from discussion.

obmenuAUR is a "user-friendly" GUI application to edit `~/.config/openbox/menu.xml`, without the need to code in `xml`.

xdg-menu

archlinux-xdg-menu包 will automatically generate a menu based on `xdg` files contained within the `/etc/xdg/` directory for numerous Window Managers, including Openbox. Review the Xdg-menu#OpenBox article for further information.

登出菜单选项

提示：The commands provided can also be attached to #键绑定

The `~/.config/openbox/menu.xml` file can be edited in order to provide a sub-menu with the same options as provided by oblogout. The sample script below will provide all of these options, with the exception of the ability to lock the screen:

```
<menu id="exit-menu" label="Exit">
	<item label="Log Out">
		<action name="Execute">
			<command>openbox --exit</command>
		</action>
	</item>
	<item label="Shutdown">
		<action name="Execute">
			<command>systemctl poweroff</command>
		</action>
	</item>
	<item label="Restart">
		<action name="Execute">
			<command>systemctl reboot</command>
		</action>
	</item>
	<item label="Suspend">
		<action name="Execute">
			<command>systemctl suspend</command>
		</action>
	</item>
	<item label="Hibernate">
		<action name="Execute">
			<command>systemctl hibernate</command>
		</action>
	</item>
</menu>
```

Once the entries have been composed, add the following line to present the sub-menu where desired within the main desktop menu (usually as the last entry):

```
<menu id="exit-menu"/>
```

管道菜单

提示：It is entirely feasible for a static menu to contain one or more pipe sub-menus. The functionality of some pipe menus may also rely on the installation of relevant software packages.

This type of menu is in essence a script that provides dynamic, refreshed lists on-the-fly as and when run. These lists may be used for multiple purposes, including to list applications, to provide information, and to provide control functions. Pre-configured pipe menus can be installed, although not from the official repositories. More experienced users can also modify and/or create their own custom scripts. Again, `~/.config/openbox/menu.xml` may and commonly will contain several pipe menus.

范例

- openbox-xdgmenuAUR: 快速将xdg菜单转换为xml管道菜单的转换器。
- obfilebrowserAUR: 应用程序和文件浏览器
- obdevicemenuAUR: 使用 Udisks 管理可移动介质

- wifi pipe menu: 使用 Netctl 进行无线网络连接

Openbox.org 还提供了更多的管道菜单列表。

生成器

This type of menu is akin to those provided by the taskbars of desktop environments such as Xfce or LXDE. Automatically updating on-the-fly, this type of menu can be powerful and very convenient. It may also be possible to add custom categories and menu entries; read the documentation for your intended dynamic menu to determine if and how this can be done.

A menu generator will have to be executed from the `~/.config/openbox/menu.xml` file.

obmenu-generator

提示：icons can still be disabled in obmenu-generatorAUR, even where enabled in `~/.config/openbox/rc.xml`.

obmenu-generatorAUR is highly recommended despite being an unofficial package. With the ability to be used as a static or dynamic menu, it is highly configurable, powerful, and versatile. Menu categories and individual entries may also be easily hidden, customised, and/or added with ease. The official homepage provides further information and screenshots.

Below is an example of how obmenu-generator would be dynamically executed without icons in `~/.config/openbox/menu.xml`:

```
<?xml version="1.0" encoding="utf-8"?>
<openbox_menu>
    <menu id="root-menu" label="OpenBox 3" execute="/usr/bin/obmenu-generator">
    </menu>
</openbox_menu>
```

To automatically iconify entries, the `-i` option would be added:

```
<menu id="root-menu" label="OpenBox 3" execute="/usr/bin/obmenu-generator -i">
```

openbox-menu

提示：If this menu produces an error, it may be solved by enabling icons in `~/.config/openbox/rc.xml`.

openbox-menuAUR uses the LXDE menu-cache to create dynamic menus. The official homepage provides further information and screenshots.

菜单图标

To show icons next to menu entries, it will be necessary to ensure they are enabled in the `<menu>` section of the `~/.config/openbox/rc.xml` file:

```
<showIcons>yes</showIcons>
```

Where using a static menu, it will then be necessary to edit the `~/.config/openbox/menu.xml` file to provide both the `icon =` command, along

with the full path and icon name for each entry. An example of the syntax used to provide an icon for a category is:

```
<menu id="apps-menu" label="[label name]" icon="[pathway to icon]/[icon name]">
```

将桌面菜单作为面板菜单

提示：XDoTool can simulate any keybind for any action, and as such, it may therefore be used for many other purposes...

xdotool is a package that can issue commands to simulate key presses / keybinds, meaning that it is possible to use it to invoke keybind-related actions without having to actually press their assigned keys. As this includes the ability to invoke an assigned keybind for the Openbox desktop menu, it is therefore possible to use XDoTool to turn the Openbox desktop menu into a panel menu. Especially where the desktop menu is heavily customised and feature-rich, this may prove very useful to:

- Replace an existing panel menu
- Implement a panel menu where otherwise not provided or possible (e.g. for Tint2)
- Compensate where losing access to the desktop menu due to the use of an application like xfdesktop包 to manage the desktop.

Once XDoTool has been installed - if not already present - it will be necessary to create a keybind to access the root menu in `~/.config/openbox/rc.xml`, and again below the `<!-- Keybindings for running aplications -->` heading. For example, the following code will bring up the menu by pressing `CTRL` + `m`:

```
<keybind key="C-m">
    <action name="ShowMenu">
       <menu>root-menu</menu>
    </action>
</keybind>
```

Openbox must then be reconfigured. In this instance, XDoTool will be used to simulate the `CTRL` + `m` keypress to access the desktop menu with the following command (note the use of `+` in place of `-`):

```
xdotool key control+m
```

How this command may be used as a panel launcher / icon is largely dependent on the features of panel used. While some panels will allow the above command to be executed directly in the process of creating a new launcher, others may require the use of an executable script. As an example, a custom executable script called `obpanelmenu.sh` will be created in the `~/.config` folder:

```
$ text editor ~/.config/obpanelmenu.sh
```

Once the empty file has been opened, the appropriate XDoTool command must be added to the empty file (i.e. to simulate the `CTRL` + `m` keypress for this example):

```
xdotool key control+m
```

After the file has been saved and closed, it may then be made into an executable script with the following command:

```
$ chmod +x ~/.config/obpanelmenu.sh
```

Executing it will bring up the Openbox desktop menu. Consequently, where using a panel that supports drag-and-drop functionality to add new launchers, simply drag the executable script onto it before changing the icon to suit personal taste.

XDG 兼容菜单

一个符合xdg标准的菜单是基于freedesktop.org标准的。菜单的定义位于/etc/xdg/menus目录中的菜单文件中。新的应用程序将自动出现在菜单中。

范例

- californium\[失效链接 2021-11-15 ⓘ\]: 基于LXQt主菜单的xdg菜单，易于主题化。

提示与技巧

光标与图标主题

详情参阅光标主题和图标。

桌面图标与壁纸

Openbox本身不支持使用桌面图标或壁纸。

请参阅 PCManFM, SpaceFM 和 Idesk.

注意：您可能需要编辑 `~/.conkyrc` 并将 `own_window_type` 设置成 `normal`.

请参阅 List of applications#Wallpaper setters.

混成特效

Openbox对混成不提供原生支持，因此需要外部混成器。

虽然合成不是必需的组件， 但它可以特别避免 oblogout 导致的屏幕失真问题，以及终端窗口透明度引起的视觉故障等问题。请参见 Xorg#Composite 获取常见的选择。

oblogout

请参阅 Oblogout 文章，了解如何使用这个实用的图形化注销脚本的概述。

Openbox多显示器用户

While Openbox provides better than average multihead support on its own, openbox-multihead-gitAUR\[损坏的链接：package not found\] provides a development branch called **Openbox Multihead** that gives multihead users per-monitor desktops. This model is not commonly found in floating window managers, but exists mainly in tiling window managers. It is explained well on the Xmonad web site. Also, please see README.MULTIHEAD for a more comprehensive description of the new features and configuration options found in Openbox Multihead.

Openbox Multihead会在只有一个屏幕时像普通的Openbox一样工作。

热键执行复合命令

如果您需要执行复杂的命令，请使用Shell功能。

特殊字符的替换如下：

- &: &
- <: <
- >: >

这个示例将立即关闭显示器并使用slock包 锁定屏幕。它是从这个帖子中获取的.

```
<keybind key="W-l">
  <action name="Execute">
    <command>sh -c 'slock &amp; (sleep .5 &amp;&amp; xset dpms force off)'</command>
  </action>
</keybind>
```

有时候需要为应用程序指定环境变量：

```
<keybind key="A-F7">
  <action name="Execute">
    <command>sh -c "LC_ALL=C obconf"</command>
  </action>
</keybind>
```

另一个示例将启动应用程序，并将所有stdout和stderr输出保存到文件中：

```
<keybind key="A-f">
  <action name="Execute">
    <command>sh -c sh -c "exec gimp &gt;/tmp/gimp.out 2&gt;&amp;1"</command>
  </action>
</keybind>
```

启用屏幕截图功能：

```
<keybind key="Print">
  <action name="Execute">
    <command>gnome-screenshot -c</command>
  </action>
</keybind>
<keybind key="A-Print">
  <action name="Execute">
    <command>gnome-screenshot -c -w</command>
  </action>
</keybind>
<keybind key="W-Print">
  <action name="Execute">
    <command>gnome-screenshot -i</command>
  </action>
</keybind>
```

应用程序启动器

Given the lack of a desktop environment with a plain Openbox install, it can be useful to install one or more application launchers as supplements to the Openbox menu system and the hotkeys. Lists of such launchers can be found at Category:Application launchers and List of applications/Other#Application launchers; popular examples are Gmrun and dmenu.

鼠标切换桌面

It is possible to switch desktop by moving the mouse cursor to the edges of the screen. First install xdotool包 and add the following two lines to your `~/.xinitrc`:

```
xdotool behave_screen_edge --delay 500 left set_desktop --relative -- -1 &
xdotool behave_screen_edge --delay 500 right set_desktop --relative -- +1 &
```

设置默认应用及文件关联

请参见默认应用程序文章.

Ad-hoc窗口透明

警告：这可能在行动组中定义了其他动作时无法工作。

The program transset-dfAUR can enable window transparency on-the-fly.

For example, using the following code in the `<mouse>` section of the `~/.config/openbox/rc.xml` file will enable control of application window transparency by hovering the mouse-pointer over the title bar and scrolling with the middle button:

```
<context name="Titlebar">
    ...
    <mousebind button="Up" action="Click">
        <action name= "Execute" >
        <execute>transset-df -p .2 --inc  </execute>
        </action>
    </mousebind>
    <mousebind button="Down" action="Click">
        <action name= "Execute" >
        <execute>transset-df -p .2 --dec </execute>
        </action>
    </mousebind>
    ...
</context>
```

使用 obxprop 进行更快的配置

The openbox包 package provides a `obxprop` binary that can parse relevant values for applications settings in `rc.xml`. Officially `obxprop | grep "^_OB_APP"` is recommended for this task. Start the process by running the command shown, then click a window to see its properties in the terminal.

应用程序的 Xprop 值

xorg-xprop包 can be used to relay property values for selected applications. Where frequently using per-application settings, the following Bash Alias may be useful:

```
alias xp='xprop | grep "WM_WINDOW_ROLE\|WM_CLASS" && echo "WM_CLASS(STRING) = \"NAME\", \"CLASS\""'
```

To use Xorg-XProp, run using the alias given `xp`, and click on the active program desired to define with per-application settings. The results displayed will only be the information that Openbox itself requires, namely the `WM_WINDOW_ROLE` and `WM_CLASS` (name and class) values:

```
WM_WINDOW_ROLE(STRING) = "roster"
```

```
WM_CLASS(STRING) = "gajim.py", "Gajim.py"
WM_CLASS(STRING) = "NAME", "CLASS"
```

切换键盘布局

请参考文章在键盘布局之间切换的方法中的说明。

设置虚拟桌面网格

安装 obsetlayoutAUR。例如，要设置一个 2x2 的网格：

```
obsetlayout 0 2 2 0
```

在不带参数的情况下运行它，以了解参数的含义。

启用敏感边角

lead-gitAUR 为openbox和其他轻量级窗口管理器提供了热角功能。在自动启动文件中添加一个条目以启动该应用程序：

```
lead &
```

命令可以在配置文件中进行编辑`~/.config/mlde/lead.conf`:

```
[eDP1]
bottom=
bottomLeft=chromium
bottomRight=thunar
left=
right=
top=
topLeft=mlde.californium toggle
topRight=skippy-xd
```

窗口吸附

Many desktop environments and window managers support *window snapping* (e.g. Windows 7 Aero snap), whereby they will automatically snap into place when moved to the edge of the screen. This effect can also be simulated in Openbox through the use of keybinds on focused windows.

As illustrated in the example below, percentages must be used to determine window sizes (see openbox.org for further information). In this instance, The `super` key is used in conjunction with the `navigation` keys:

```
<keybind key="W-Left">
    <action name="UnmaximizeFull"/>
    <action name="MaximizeVert"/>
    <action name="MoveResizeTo">
        <width>50%</width>
    </action>
    <action name="MoveToEdge"><direction>west</direction></action>
</keybind>
<keybind key="W-Right">
    <action name="UnmaximizeFull"/>
    <action name="MaximizeVert"/>
    <action name="MoveResizeTo">
        <width>50%</width>
    </action>
    <action name="MoveToEdge"><direction>east</direction></action>
</keybind>
```

However, it should be noted that once a window has been 'snapped' to an edge, it will remain vertically maximised unless subsequently maximised and then restored. The solution is to implement additional keybinds - in this instance using the `down` and `up` keys - to do so. This will also make pulling 'snapped' windows from screen edges faster as well:

```
<keybind key="W-Down">
   <action name="Unmaximize"/>
</keybind>
<keybind key="W-Up">
   <action name="Maximize"/>
</keybind>
```

This Ubuntu forum thread provides more information. Applications such as opensnapAUR are also available to automatically simulate window snapping behaviour without the use of keybinds. Another option is to use bunsen-utilities-gitAUR which provides `bl-aerosnap --left` and `bl-aerosnap --right` commands which will snap active window on left or right edge respectively if it's not snapped and restore it to original size and position otherwise. Just bind these commands to the key combination of your choosing.

平滑的显示管理器过渡

注意：这已经确定可以在 LightDM 上工作。

Users of display managers might experience a flickering during the transition between the display manager and the Openbox desktop. The flickering comes from Openbox setting the root window's color during startup. Therefore there is a brief moment when the display flashes in a grey color, between the display manager's background and the desktop's wallpaper.

Setting the root window's background color can be disabled by editing the Openbox startup script found in `/usr/lib/openbox/openbox-autostart`. Simply comment out (or delete) the block starting with `# Set a background color`.

注意：Users who do not specifically set their wallpaper will "inherit" the display manager's background automatically if they disable the root window color adjustment.

窗口渲染

To remove window decorations for all or particular applications, use the *<decor>* option in the *<applications>* section of *rc.xml* (user: *~/.config/openbox/* or system: */etc/xdg/openbox/*). Example for Firefox, including variants like Firefox-Beta and Firefox-Nightly:

```
 <application class="Firefox*">
   <decor>no</decor>
 </application>
```

One could also disable decorations for all applications (using class **"*"**), then enable them (using** yes**) for individual ones. To apply the changes, restart your desktop session, and thus Openbox. Reference: Openbox FAQ

排错

Firefox

基于Mozilla的浏览器可能会忽略应用程序规则 (例如 `<desktop>`) 除非使用 `class="Firefox"` 。请参阅 #应用程序的 Xprop 值.

主题丢失

If for any reason the newly extracted theme cannot be selected, open the theme directory to first ensure that it is compatible with Openbox - there should be an `openbox-3` directory and a `themerc` file within it. An `.obt` (**O**pen**B**ox **T**heme) file may also be present in some instances, which can then be manually loaded in obconf包.

A theme may also be not accessible due to wrong permissions. See File permissions and attributes for more.

连续切换桌面时停顿

By default Openbox switches from the last desktop back to the first desktop on mouse wheel scroll. Use `<wrap>no</wrap>` in the `mousebind` section to disable this behaviour.

```
   <context name="Desktop">
     <mousebind button="Up" action="Click">
       <action name="GoToDesktop">
         <to>previous</to>
         <wrap>no</wrap>
       </action>
     </mousebind>
     <mousebind button="Down" action="Click">
       <action name="GoToDesktop">
         <to>next</to>
         <wrap>no</wrap>
       </action>
     </mousebind>
   </context>
```

新窗口隐藏在活动窗口后面

Some application windows (such as Firefox windows) may load behind the currently active window, causing you to need to switch to the window you just created to focus it. To fix this behavior add this to your `~/.config/openbox/rc.xml` file, inbetween the `<openbox_config>` and `</openbox_config>` tags:

```
<applications>
  <application class="*">
    <focus>yes</focus>
  </application>
</applications>
```

参阅

- Openbox Website - 官方网站
- Box-Look.org - 一个关于主题和相关艺术作品的资源网站

- Openbox Hacks and Configs Thread @ Arch Linux 论坛
- Openbox Screenshots Thread @ Arch Linux 论坛
- An Openbox guide

第 7 章　共 8 节　99 KB

平铺式窗口管理器

本章要解决的是另一套桌面使用方式：窗口不再靠鼠标摆放，而由规则、快捷键与配置文件决定位置和大小。八节各讲一款平铺式窗口管理器，取其一即可。

建议先读《平铺式窗口管理器比较》建立全景，再挑一款深入：《Xmonad》与《Bspwm》篇幅最长，从安装、启动、配置写到排错，可作必读；《Herbstluftwm》《Qtile》《Ratpoison》可作对照与备选；《Notion》极短，《Ion3》五节几乎全是 Lua 配置文件剖析，不用它可直接跳过。最容易踩的坑是把平铺式当成装完即可使用：它们基本都要自己写配置文件、自己绑快捷键，各家启动方式与键位不同，照着别人的配置抄，不如按对应页的顺序走一遍。

7.1　Notion

难度 入门 | **篇幅** 2.3 KB | **所属** 平铺式窗口管理器

本节将讲到

- 安装
- 启动
- 用法
- 配置
- 参见

Notion 是 X 的平铺，选项卡式的窗口管理器。

安装

安装 notion包。

启动

使用 xinit 运行 `notion`。

使用显示管理器

要使用显示管理器启动/选择 Notion，可以在 `/usr/share/xsessions/` 目录中创建一个标准的 .desktop 文件。 以下是一个示例 `notion.desktop` 文件：

```
[Desktop Entry]
Name=Notion
Comment=This session logs you into Notion
Exec=/usr/bin/notion
TryExec=/usr/bin/notion
Icon=
Type=XSession
```

有关更多详细信息，请参见显示管理器文章。

用法

您可以在使用过程中使用 F1 键并按 return 键来查看 Notion 的手册页，这将告诉您 Notion 的默认键绑定。您也可以通过按 F1 键，输入程序名称并按 return 键来访问其他程序的手册页。除了手册之外，还有一个快速入门指南，快速概述了 Notion。

配置

可以使用 Lua 配置 Notion。首先，将 `/etc/notion/cfg_notion.lua` 复制到 `~/.notion`。有关更多信息，请阅读使用 Lua 配置和扩展 Notion。

参见

- https://notionwm.net/ - Notion 网站
- https://raboof.github.io/notion-doc/notionconf/ - 使用 Lua 配置和扩展 Notion
- https://github.com/raboof/notion/ - Notion GitHub

7.2 Herbstluftwm

难度 进阶 | **篇幅** 7.9 KB | **所属** 平铺式窗口管理器

本节将讲到

- 安装
- 启动
- 第一步
- 配置
- 命令

Herbstluftwm是一个使用 Xlib 的手动 X11 窗口管理器。

安装

安装 herbstluftwm包 包或 herbstluftwm-gitAUR （开发版本）。

启动

从 TTY 启动

用 xinit 启动 `herbstluftwm`。

作为桌面环境启动

herbstluftwm包 包含可以启动窗口管理器的 `herbstluftwm.desktop` 作为 Xsession。

从其他窗口管理器启动

如果已经启动了一个窗口管理器会话，可以用 `herbstluftwm --replace` 替换掉原窗口管理器。

第一步

阅读 herbstluftwm(1) 和 herbstclient(1) 手册页，它们包含很多对配置选项和有效值的解释。

配置

复制 `/etc/xdg/herbstluftwm/autostart` 到 `~/.config/herbstluftwm/autostart`。可以按需修改文件。 保证 autostart 文件可执行，否则键绑定极有可能失效！

Herbstluftwm 的配置可以用 `herbstclient reload` 即时更新 （见 #Commands）。每次 reload 配置时 Autostart 脚本都会被调用，所以它一般会先恢复所有配置到默认值。

多显示器支持

Herbstluftwm 支持多个显示器 as a virtual concept；Herbstluftwm 中，显示器信息不必与 xrandr 报告的真实监视器配置相匹配。它带来了很大的灵活性，让用户可以更好地控制显示器的布置。可以使用 `herbstclient detect_monitors` 自动适应物理设置，或者通过手册了解如何添加、移除、调整和移动显示器。多监视器设置中的标记不为某个监视器 "拥有"。这意味着当一个监视器切换到另一个监视器中激活的标记时，两个监视器将交换标记。

命令

Herbstclient 很强大，因为它能让你通过命令行完全控制窗口管理器。

Herbstclient 的参数支持 Tab 补全。用 `herbstclient list_commands` 查看所有参数。

现在如果传给 Herbstclient 的参数有误，Herbstclient 不会打印错误信息，指挥以非零值退出。如果不通过其他方式显示返回值（比如用`$SHELL-prompt`），可能需要用 `echo $?` 打印返回值。

脚本和钩子

The main way to control herbstluftwm is though commands to herbstclient. Since herbstclient can be called from any script, you have great flexibility in controlling

herbstluftwm this way. Furthermore, you can listen to window management events and react to them accordingly.

Herbstluftwm includes a number of example scripts: `/usr/share/doc/herbstluftwm/examples/` or https://github.com/herbstluftwm/herbstluftwm/tree/master/scripts

Script to switch to the next empty tag

The following python script allows you to switch to the (next or previous) (full or empty) tag. Call it with the arguments (+1 or -1) and (full or empty). For example, if you save the script to herbst-move.py, then

```
python3 herbst-move.py +1 full
```

will move you to the next full tag. I use the following key bindings.

```
hc keybind $Mod-Left  spawn herbst-move.py -1 empty
hc keybind $Mod-Right spawn herbst-move.py +1 empty
hc keybind $Mod-Up spawn herbst-move.py -1 full
hc keybind $Mod-Down spawn herbst-move.py +1 full
```

And here is the script.

```
#!/usr/bin/env python3
def run(*cmd):
    from subprocess import Popen, PIPE
    proc = Popen(cmd, shell=False, stderr=PIPE, stdout=PIPE)
    return proc.stdout.read()

import sys
tag_offset, mode = sys.argv[1:]
tag_offset = int(tag_offset)
if mode == 'full':
    ch = {'.'}
elif mode == 'empty':
    ch = {':', '!'}
else:
    raise Exception('Unknown type ' + mode)
tag_list = run('herbstclient', 'tag_status', '0').strip().decode('ascii').split('\t')
tag_curr = int(run('herbstclient', 'attr', 'tags.focus.index').strip())
tag_next = (tag_curr + tag_offset) % len(tag_list)
while (tag_next != tag_curr) and (tag_list[tag_next] in ch):
    tag_next = (tag_next + tag_offset) % len(tag_list)
if tag_next != tag_curr:
    run('herbstclient', 'use_index', str(tag_next))
```

Script to cycle though paddings (or other settings)

Here is a ruby script to cycle through a set of paddings, although you can modify it to cycle though any collection of settings. The script knows the previous layout by looking for the presence of two dummy files in /tmp.

```
#!/usr/bin/ruby

file1 = "/tmp/herbst-padding-1"
file2 = "/tmp/herbst-padding-2"

pad1 = 'pad 0 0 0 0 0'
pad2 = 'pad 0 0 20 0 200'
pad3 = 'pad 0 0 0 0 150'

files = [file1, file2].map{|f| File.exist? File.expand_path(f)}

if files == [false, false]  # 0 files
  system "herbstclient #{pad2}"
```

```
  system "touch #{file1}"
elsif files == [true, false]  # 1 file
  system "herbstclient #{pad1}"
  system "touch #{file2}"
else           # 2 files
  system "herbstclient #{pad3}"
  system "rm #{file1} #{file2}"
end
```

Script to change decoration per-tag

The following Perl script demonstrates how to use hooks to react to window management events. It can be called in autostart (with backgrounding).

```
#!/usr/bin/perl
# This script watches for tag changes and gives visual feedback

## Configuration (fill with your tag names)
my %colors = (
       main => '#DD0000',
       devel => '#13B8E0',
       write => '#96E013',
       admin => '#C713E0'
);

## Apply tag color
# Right now we change the active window's border color to the tag's color.
sub redecorate
{
       my ($foo, $activity) = @_;
       system("herbstclient", "set", "window_border_active_color",
              "$colors{$activity}");
}

## main routine
use v5.20;

# set up a pipe for reading hooks
open HOOKS, "herbstclient -i '(tag_changed|reload)'|"
       or die "can't fork: $!";
# process incoming messages
OUTER:
while (<HOOKS>) {
       chomp;
       for ($_) {
              redecorate(split(/\t/)) when /^tag_changed/;
              last OUTER when /^reload/; # quit on reload
       }
}
close HOOKS or die "unfinished love story: $! $?"; # happens on hlwm crash
```

Troubleshooting

- After installing, `$mod+enter` keybind does not start a terminal: the herbstluftwm autostart configuration file uses the xterm terminal by default. Make sure xterm包 is installed or edit the autostart file to use a different terminal.
- To log out of herbstluftwm, use the command `herbstclient quit` or the default keybind of `$mod+shift-q`

See also

- The herbstluftwm homepage
- Arch Linux BBS thread

- `/usr/share/doc/herbstluftwm/examples/` - various scripts
- `/usr/share/doc/herbstluftwm/BUGS` - bugs
- A herbstluftwm thread on the CrunchBang Forums
- **Screenshots and configuration files:** on Arch Linux Forum, on DotShare.it
- `#herbstluftwm` - IRC channel at irc.libera.chat
- User git repository #1 with autostart written in Perl and a few custom scripts
- User git repository #2 with autostart and panel written in Python
- Github Discussions
- Herbstluftwm Subreddit

7.3 平铺式窗口管理器比较

难度 进阶 | **篇幅** 8.3 KB | **所属** 平铺式窗口管理器

本节将讲到

- Comparison table
- 参见

本文对最流行的*平铺式*窗口管理器（相对于*浮动式*窗口管理器）进行了公正的比较。

Comparison table

The following table lists the most popular tiling window managers alongside notable features, providing readers with a quick overview.

Window Manager	Written in	Configured with	Management style	System tray support	On-the-fly reload	Information bars	Compositing	Default layouts	Pixel usage	External control	Library	Multiple (n) monitor behavior	ICCCM/EWMH compliant	Maintenance
Awesome	C	Lua	Dynamic	Built-in	是	Built-in, images and text	External	max, nh-stack (and invert), nv-stack (and invert), free	Variable borders, optional h-tab titles	dbus (if enabled)	XCB	n-tags (workspaces). Per default 9 are enabled. Example	是	Active
bspwm	C	Anything	Hybrid	None	是	Can write internal state to a FIFO	External	v-split, h-split	Variable borders	via `bspc`	XCB	Monitors hold Desktops	是	Active
	C												否	

Window Manager	Written in	Configured with	Management style	System tray support	On-the-fly reload	Information bars	Compositing	Default layouts	Pixel usage	External control	Library	Multiple (n) monitor behavior	ICCCM/EWMH compliant	Maintenance
dwm		C (recompile)	Dynamic	Optional Patch	Optional	Built-in, reads from root window name	External	v-stack, max		via dwmfifo	Xlib	n regions, 9 workspaces fixed to each region		Active
FrankenWM	C	C (recompile)	Dynamic	None	否	否, outputs information to stdout, which can easily be parsed and displayed by an external monitor or panel (dzen2, conky, etc)	External	v-stack (and invert), h-stack (and invert), dual-v/h-stack, grid, fibonacci (vh-stack), rows, columns, max, free	Variable borders		XCB		否	Active
herbstluftwm	C++	Anything	Manual	None	是		External	vertical, horizontal, grid, max, tabbed	1-pix borders	commands via `herbstclient`	Xlib	n regions, 9 workspaces visible in any region	是	Active

Window Manager	Written in	Configured with	Management style	System tray support	On-the-fly reload	Information bars	Compositing	Default layouts	Pixel usage	External control	Library	Multiple (n) monitor behavior	ICCCM/EWMH compliant	Maintenance
i3	C	Text	Manual	i3bar	是 (Layout is preserved)	text piped to i3bar (`i3status`/`conky` and others can be used)	External	tree, v-split, h-split, stacked, tabbed, max, can be nested infinitely	None, 1-pix or 2-pix, optional title bars, can hide edge borders	commands via ipc (or i3-msg, which uses ipc)	XCB	n regions	是	Active
LeftWM	Rust	RON (user settings) / Anything (themes)	Dynamic	None	是	是, many options through theme system	External	v-stack, columns, rows	Variable based on theme	supports `_NET_ACTIVE_WINDOW` and sending commands to a named pipe	Xlib	Workspaces and monitors are not tied. Many workspaces for monitor or many monitors for workspace	是	Active
Notion	C, Lua	Lua, compatible with Ion3 configs	Manual	trayion, stalonetray	是	configurable	?	h-tab, max	Configurable borders and title bars/tabs	EWMH, arbitrary Lua scripts which have access to the rich internal API	Xlib	n workspaces on each monitor. Supports on-		Active

Window Manager	Written in	Configured with	Management style	System tray support	On-the-fly reload	Information bars	Compositing	Default layouts	Pixel usage	External control	Library	Multiple(n) monitor behavior	ICCCM/EWMH compliant	Maintenance
												the-fly changes in topology		
qtile	Python	Python	Dynamic	是	是	是	External	tree, v-split, h-split, stacked, tabbed, max	否 borders, although customizable	Hooks, Server mode	XCB			Active
Ratpoison	C	Text	Manual	None	是	是	External	max			Xlib		否	Active
Snapwm	C	Reloadable Text	Dynamic	None	是	Built-in, reads from root window name	External	nVertical, Fullscreen, nHorizontal, Grid, CenterStacking	Variable borders, no titles		Xlib	Number of desktops distributed evenly between monitors		Active
Spectrwm	C	Text	Dynamic	None	是	Built-in, reads from user script	否	nv-stack, nh-stack, max	1-pix borders, no titles		XCB	n regions, 10 workspaces visible in any region	否	Active
Stumpwm	Common	Common	Manual	StumpTray	是	是	External	max		SLIME server ("Swank")	CLX (Xlib-		否	Active

Window Manager	Written in	Configured with	Management style	System tray support	On-the-fly reload	Information bars	Compositing	Default layouts	Pixel usage	External control	Library	Multiple (n) monitor behavior	ICCCM/EWMH compliant	Maintenance
	Lisp	Lisp									equivalent)			
xmonad	Haskell	Haskell	Dynamic	None	是	否	是, with xmonad-contrib and an external manager	nv-stack, nh-stack, max	Variable borders, no titles	via XMonad-Hooks-ServerMode	Xlib	n regions, 9 workspaces visible in any region	是 / via XMonad-Hooks-EwmhDesktops	Active
Window Manager	Written in	Configured with	Management style	System tray support	On-the-fly reload	Information bars	Compositing	Default layouts	Pixel usage	External control	Library	Multiple (n) monitor behavior	ICCCM/EWMH compliant	Maintenance

提示：External control can also be achieved by programs like xdotool包 which simulate keystrokes.

Management style

Dynamic management emphasizes automatic management of window layouts for speed and simplicity. Manual management emphasizes manual adjustment of layout and sizing with potentially more precise control, at the cost of more time spent moving and sizing windows.

Layouts

A number of common layout types appear in several tiling WMs, although the terminology varies somewhat.

- max: one window shown fullscreen (with or without a status bar, title and borders). Aka: monocle (dwm, monsterwm).
- h-stack: master area in top half, other windows stack up horizontally in the bottom half. The master area may be resizable. May be inverted top-bottom (wmfs). Aka: bottom stack (dwm), bstack(monsterwm).
- v-stack: master area in left half, other windows stack up vertically in the right half. The master area may be resizable. May be inverted left-right (wmfs). Aka: tile (dwm, monsterwm).
- nh-stack: h-stack allowing >=1 windows in master area. Aka: nbstack (dwm)
- nv-stack: v-stack allowing >=1 windows in master area. Aka: ntile (dwm)
- mirror-h: nh-stack with stacks above and below the master area

- mirror-v: nv-stack with stacks to the left and right of the master area
- h-tab: one window shown fullscreen with all window titles shown horizontally (like browser tabs)
- v-tab: one window shown fullscreen with all window titles shown vertically. Aka: stack (wmii).
- h-split: a keybinding splits the current window horizontally creating space for another
- v-split: a keybinding splits the current window vertically creating space for another
- columns: manual layout style which treats windows as belonging to vertical columns
- rows: manual layout style which treats windows as belonging to horizontal rows
- grid: window positions and sizes based on a regular NxM grid. May be automatic (like wmfs, monsterwm) or manual (like Subtle).

Key bindings

Tiling window managers are usually designed to be used entirely with the keyboard or with keyboard & mouse. This is for speed (reaching for and moving a mouse is slow) and ease of use. Sensible key bindings are crucial to making workflow fast and efficient. Some default sets are better than others, but generally the keys can be rebound as desired by the user.

参见

- 平铺式 Wayland 混成器列表
- 可扩展窗口管理器比较比较了可通过脚本"扩展"的窗口管理器，如 xmonad 和 Sawfish。

7.4 Ratpoison

难度 进阶 | **篇幅** 9.1 KB | **所属** 平铺式窗口管理器

本节将讲到

- 安装
- 配置
- 使用Ratpoison
- 小贴士
- 有用的快捷键

Ratpoison是一个用C语言编写的平铺式窗口管理器，它允许用户不用鼠标就能管理应用程序。它的界面收到GNU Screen的启发。

安装

Ratpoison可以通过ratpoison包 或者是ratpoison-gitAUR\[损坏的链接：package not found\] （开发版本）安装。

配置

为了让Ratpoison成为你的窗口管理器，你必须新建/编辑文件`~/.xinitrc`.

.xinitrc的一个例子:

```
# The black/white grid as background doesn't suit my taste.
xsetroot -solid black &
# Ratpoison is compatible with xcompmgr! now you can have real transparency
xcompmgr -c -f -D 5 &
#fire up ratpoison!
exec /usr/bin/ratpoison
```

使用Ratpoison

在X11启动后你会看到黑色的屏幕背景还有右上角它弹出的一个文本框"Welcome to Ratpoison"。现在按下`Ctrl+t`，再输入`?`来获取快捷键列表。如果你习惯使用GNU screen，你将马上感到它非常熟悉。 你可以自定义快捷键，你甚至可以覆盖`~/.ratpoisonrc`中已存在的快捷键

例如:

```
# Overriding CTRL+t 'c' to start aterm instead of xterm
bind c exec aterm

bind f exec firefox
```

如果使用上述配置， 当你按下`Ctrl+t` 然后`f`后, ratpoison将启动firefox.

这是另一个我正在用的.ratpoisonrc配置:

```
exec xsetroot -cursor_name left_ptr
startup_message off

escape C-z

# Make a screenshot
alias sshot exec import -window root ~/screenshot-$(date +%F).jpg
definekey top M-C-Print sshot

#virtual desks
gnewbg one
gnewbg two

definekey top M-l exec ratpoison -c "select -" -c "gprev" -c "next"
definekey top M-h exec ratpoison -c "select -" -c "gnext" -c "next"

#switch between windows
definekey top M-j next
definekey top M-k prev

#apps
unbind c
```

```
bind c exec urxvt -tr
#bind c exec aterm

bind g exec gftp
bind f exec firefox
```

小贴士

Java Swing应用程序

Java Swing GUI应用程序不认为ratpoison是平铺式窗口管理器, 且不正确地像ratpoison配置的那样全屏显示。不过有一种方法可以让它认为自己在平铺式管理器中并且正确的全屏。

首先安装`wmname` package， 然后将这一行加入你的`.ratpoisonrc`中:

```
~/.ratpoisonrc
exec wmname LG3D
```

啊哈！Java Swing应用现在可以正确全屏了。

多个虚拟桌面

默认情况下，ratpoison只有一个虚拟桌面，但是用一个叫做rpws(默认安装）的脚本，你可以有多个虚拟桌面。

只要更改你的.ratpoisonrc, 然后加入:

```
~/.ratpoisonrc
exec /usr/bin/rpws init 6 -k
```

这样就会创建6个虚拟桌面。默认地，你可以进入通过按下`Alt+F1` 来进入1号虚拟桌面, `Alt+F2` 来进入2号虚拟桌面，以此类推。

你也可以这样添加快捷键:

```
bind C-1 exec rpws 1
bind C-2 exec rpws 2
...
```

这样允许你通过`Ctrl+t Ctrl+1` 进入它们(假定`Ctrl+t` 是你的escape键)

Urxvt和xterm

Urxvt和xter，因为它们被默认安装，发送重调大小的信号给窗口管理器。这个在大多数窗口管理器中起作用，但是在ratpoison中无效。结果就是URxvt/xterm通过更改字号调整自己的大小，而不是将自己调整成全屏，有无法填充空格的几率很高。有两种方法改变这个现状。请看下文。

安装打过补丁的URxvt

如果你使用URxvt,软件包rxvt-unicode-fontspacing-noinc-vteclear-secondarywheelAUR除其他改动外，还不发送重调大小的信号给窗口管理器。如果你安装这个版本的URxvt而不是默认版本，URxvt就会在ratpoison中正常调整大小。

配置边缘

我们可以使用xterm/URxvt默认的边框设置工具将ratpoison的边框设置为0。

```
~/.Xresources
urxvt*internalBorder: 8 #change urxvt to xterm if necessary. Using the font terminus in
urxvt at 14px size, 8 is the correct number here.
~/.ratpoisonrc
set border 0
```

自启动

当ratpoison启动时自启动应用的例子。文件 `~/.ratpoisonrc`中的内容会在ratpoison启动时执行。

```
Launch urxvt with a tmux session
exec urxvt -e bash -c "tmux -q has-session && exec tmux attach-session -d || exec tmux
new-session -n$USER -s$USER@$HOSTNAME"
Launch optimized chromium
exec bash -c 'pidof chromium &>/dev/null || exec /usr/bin/chromium --disk-cache-
dir=~/tmp/cache'
```

壁纸和透明度

通过xcompmgr还有nitrogen设置透明的例子. 首先启动nitrogen然后设置你希望设置成壁纸的壁纸。然后把这个加入到你的.ratpoisonrc中去

```
Wallpaper and transparency
exec xcompmgr -c -f -D 5 &
exec nitrogen --restore
```

有用的快捷键

<table class="wikitable"><tbody><tr><td><code>Ctrl+t</code> <code>!</code> <<i>程序名称</i>> 启动某个程序</td></tr><tr><td><code>Ctrl+t</code> <code>?</code> 显示快捷键设置</td></tr><tr><td><code>Ctrl+t</code> <code>c</code> 启动一个x terminal</td></tr><tr><td><code>Ctrl+t</code> <code>n</code> 切换到下一个窗口</td></tr><tr><td><code>Ctrl+t</code> <code>p</code> 切换到之前的窗口</td></tr><tr><td><code>Ctrl+t</code> <code>1</code>-<code>9</code> 切换到1-9号窗口</td></tr><tr><td><code>Ctrl+t</code> <code>k</code> 关闭当前窗口</td></tr><tr><td><code>Ctrl+t</code> <code>Shift+k</code> 杀死当前窗口</td></tr><tr><td><code>Ctrl+t</code> <code>s</code>,<code>Shift+s</code> 将当前布局改为竖直布局或水平布局</td></tr><tr><td><code>Ctrl+t</code> <code>Tab</code>, <code>←</code>, <code>↑</code>, <code>→</code>, <code>↓</code> 切换到下一个，左边的，顶部的，右边的，顶部的框架.</td></tr><tr><td><code>Ctrl+t</code> <code>Shift+q</code> 将当前框架指定为唯一</td></tr><tr><td><code>Ctrl+t</code> <code>:</code> 执行一个ratpoison命令</td></tr></tbody></table>

焦点跟随鼠标

Arch linux的ratpoison包同时在/usr/share/ratpoison中安装了编译脚本，sloppy.c。它能被用来在ratpoison中启动焦点跟随鼠标。为了启动它，你可以:

```
# cd /usr/share/ratpoison/
# gcc -o sloppy sloppy.c -lX11
# ./sloppy
```

想要自启动，你可以将下列行加入~/.ratpoisonrc

```
exec /usr/share/ratpoison/sloppy
```

Ratpoison和显示管理器

许多display managers (例如 LightDM) 在 `/usr/share/xsessions/`中确认可用的会话，并且大多数窗口管理器和桌面环境会在那建立一个.desktop文件。然而ratpoison却会在`/etc/X11/sessions/`里创建。 为了让显示管理器识别ratpoison，你需要将`/etc/X11/sessions/ratpoison.desktop` 拷贝至`/usr/share/xsessions/ratpoison.desktop`. 如果 `/usr/share/xsessions`不存在，你需要在root下建立

更多链接

- The Ratpoison wiki
- X11 Keys in Ratpoison\[失效链接 2020-08-04 ⓘ\]
- Ratpoison config sample\[失效链接 2021-11-15 ⓘ\]
- Share your Ratpoison (experience) forum thread
- Collection of scripts for the Ratpoison window manager
- Stumpwm - a successor to Ratpoison written in Common lisp

7.5 Qtile

难度 进阶 | **篇幅** 9.8 KB | **所属** 平铺式窗口管理器

本节将讲到

- 安装
- 启动
- 配置
- 调试
- 参阅

摘自 Qtile web site：

: *Qtile 是一个全功能、可轻易修改(骇)的平铺式窗口管理程序。 Qtile 简单、轻巧、扩展性高。 撰写自订的窗口堆叠模式、插件以及指令是轻而易举的事情。程序以及设定*

均是以 Python 写成，意味着：您可以使用语言所提供的所有能力及弹性来满足您对窗口管理的需求。

安装

从下列软件包中选择一个进行安装：

- qtile包 最新正式版
- qtile-gitAUR 最新开发版

为了让 Qtile 作为一个 Wayland 混成器运行，你将需要安装 python-pywlroots包。

启动

Xorg

要让 qtile 作为一个 X11 的窗口管理器启动, 可用 xinit 执行 `qtile start` 来启动.

Wayland

Qtile 同时也可作为一个 Wayland compositor 通过执行 `qtile start -b wayland` 来启动.

关于 Qtile 的 wayland 端开发进度, 可以查看这个讨论.

配置

注意：这个章节只会介绍基本的配置模式，若需更详尽的配置讯息请参见 官方文档 或者 备选文档。

根据配置查询表（或者备选文档）， Qtile 会在路径 `~/.config/qtile/config.py` 提供一个默认的配置文件，它将会在用户没有进行自定义的情况下使用。

默认配置设置了 `Super+Enter` 作为打开*终端*的快捷键,（终端通过 hardcoded list 选择）, 快捷键 `Super+Ctrl+q` 可以退出 Qile.

最新的配置文件可以在 git 仓库 libqtile/resources/default_config.py 下载。

一些更完整的配置文件示例可以在 qtile-examples 仓库找到。

整个 Qtile 的配置均是以python写成：为了快速了解 Python 语言可以阅读这个教程。

在重新启动 Qtile 之前，你可以用以下指令测试 config.py 文件是否有语法错误:

```
$ python -m py_compile ~/.config/qtile/config.py
```

若如果没有其他输出, 此设定文件应该便无重大问题.

组(Groups)

在 Qtile, 工作区(workspaces or views) 被称为**组(Groups)**。他们可透过下列方式定义：

```
from libqtile.config import Group, Match
...
groups = [
    Group("term"),
    Group("irc"),
    Group("web", match=Match(title=["Firefox"])),     #表示标题含此关键字的程序均放置于此组别中
   ]
...
```

组规则(Group Rules)

下面的例子展示了如何根据属性，如 title 和 wm_class，实现自动移动应用到工作区。如果想在 X 上做到这一点，你可能需要用到 **xprop** 。

```
from libqtile.config import Group, Match
...
def is_text_editor(window):
    result = "neovim" in (window.name or "").lower()
    return result

def is_terminal(window):
    result = "kitty" in (window.name or "").lower() and not is_text_editor(window)
    return result
...
groups = [
    Group(name=str(idx), **group)
    for idx, group in enumerate(
        [
            {
                "label": "term",
                # restrict layouts since tiling is handled by kitty
                "layouts": [layout.Max()],
                "matches": [
                    Match(func=is_terminal),
                ],
            },
            {
                "label": "browser",
                "matches": [
                    Match(role="browser"),
                ],
            },
            {
                "label": "music",
                "matches": [
                    Match(title="YouTube Music"),
                ],
            },
            {"label": "text editor", "matches": [Match(func=is_text_editor)]},
            {"label": "other"},
        ],
        start=1,
    )
]
...
```

按键组合(Keys)

你可以配置你的快捷键通过 **Key** 这个类。 底下是一个按键范例, 按下 `Alt+Shift+q` 来退出窗口管理器

```
from libqtile.config import Key
from libqtile.command import lazy
...
keys = [
    Key(
        ["mod1", "shift"], "q",
        lazy.shutdown())
   ]
...
```

可执行 Xmodmap 来得知系统按键与 `modX` 之间的对应关系.

声音控制(Sound)

你可以通过添加快捷键来方便地控制声音以及状态.

将用户添加进 **audio用户组**并且使用 `alsamixer` 命令行界面, 该程序包含在 alsa-utils包 包内.

```
keys= [
    ...
    # Sound
    Key([], "XF86AudioMute", lazy.spawn("amixer -q set Master toggle")),
    Key([], "XF86AudioLowerVolume", lazy.spawn("amixer -c 0 sset Master 1- unmute")),
    Key([], "XF86AudioRaiseVolume", lazy.spawn("amixer -c 0 sset Master 1+ unmute"))
   ]
```

语言(Language)

你可以使用 setxkbmap 来添加快捷键以便轻松切换不同语言的键盘布局：

```
keys= [
    ...
    # Language
        Key([mod], "F1",
            lazy.spawn("setxkbmap us"),
            desc= "Change to US layout"),
        Key([mod],"F2",
            lazy.spawn("setxkbmap gr"),
            desc= "Change to Greek layout"),
       ]
```

显示器(Screens)

为你的每一台显示器配置 **Screen** 类, 而 任务栏(bar) 则配置于 **Screen** 类中.

```
from libqtile.config import Screen
from libqtile import bar, widget
import os.path
...
screens = [
    Screen(                         # 一部显示器
        wallpaper=os.path.join(os.path.expanduser("~"),
"Photos/Wallpapers/arch_fill.png"),
        wallpaper_mode="fill",
        bottom=bar.Bar([            # 将任务栏(bar)放置于屏幕底部
            widget.GroupBox(),      # 插件，用于显示组的状态
            widget.WindowName()     # 插件，用于显示当前窗口名称
            ], 30))
   ]
...
```

任务栏和小部件(Bars and Widgets)

可以从官方文档（或者备选文档）里找到所有内置小部件的列表.

若想新增一个小部件, 只需要像上面任务栏设定那样调用即可 (如 `WindowName` 插件). 例如, 要新增一个显示电池状态的小部件, 我们可使用 `Battery`:

```
from libqtile.config import Screen
from libqtile import bar, widget
...
screens = [
    Screen(top=bar.Bar([
        widget.GroupBox(),     # 插件，用于显示组的状态
        widget.Battery()       # 插件，用于显示电池状态
       ], 30))
   ]
...
```

把Polybar作为主任务栏(Using Polybar as the main bar)

为了用 Polybar 代替默认任务栏，你需要删除 **screen** 类的内容：

```
from libqtile.config import Screen
from libqtile import bar, widget
...
screens = [
    Screen()
]
...
```

为了通过 Qtile 重新启动 Polybar ,在#Keys 类的重启快捷键配置处添加通过 `spawn` 命令的 Polybar 启动代码：

```
...
keys = [
    Key([mod, "control"], "r", lazy.reload_config(),
lazy.spawn("~/.config/polybar/launch.sh")),
]
...
```

启动

你可以使用**钩(hooks)** 来启动程序, 特别是 `startup`.

`startup` 钩是用于 Qtile 初始化事件的钩, 能够在 Qtile 初始化的时连带启动其他的程序(一次性指令或程序).

要查看所有可用的钩, 可参考这个文档

下面是*只启动一次程序*的示例:

```
import os
import subprocess
from libqtile import hook

@hook.subscribe.startup_once
def autostart():
    home = os.path.expanduser('~')
    subprocess.Popen([home + '/.config/qtile/autostart.sh'])
```

调试

有时候会因为插件的参数没有完整, 或者设定之间有冲突情形发生, 模组未 import 等问题出现错误.

需要检查出错位置，可以以如下方式启动一个虚拟的 Xorg 并进行测试:

```
echo "exec qtile start" > /tmp/.start_qtile; xinit /tmp/.start_qtile -- :2
```

Qtile 将会输出日志到 `~/.local/share/qtile/qtile.log` 里.

参阅

- Qtile 官方网站
- 官方文档
- 平铺式窗口管理器比较
- xinitrc
- tilenol - 受 Qtile 启发，以 Python3 写成的类似窗口管理程序

7.6 Bspwm

难度 进阶 | **篇幅** 12.4 KB | **所属** 平铺式窗口管理器

本节将讲到

- 安装
- 启动
- 配置
- 故障排除
- 参照

bspwm 是一个平铺窗口管理器，每一个窗口被表达为满二叉树的叶子。bspwm支持多显示器，可通过messages进行配置和控制。bspwm部分支持EWMH。

安装

安装 bspwm包 或 bspwm-gitAUR (开发版本）。

启动

使用 xinit 来启动 `bspwm`。

配置

示例配置文件位于 `/usr/share/doc/bspwm/examples/`。

复制或安装 `bspwmrc` 和 `sxhkdrc` 分别到 `~/.config/bspwm/` 和 `~/.config/sxhkd/`。

文件 `bspwmrc` 必须是可执行的，因为默认示例是一个shell脚本,此脚本随后通过 `bspc` 命令配置bspwm。

```
$ install -Dm755 /usr/share/doc/bspwm/examples/bspwmrc ~/.config/bspwm/bspwmrc
$ install -Dm644 /usr/share/doc/bspwm/examples/sxhkdrc ~/.config/sxhkd/sxhkdrc
```

这两个文件会分别被用来设置窗口管理器和按键绑定。

查看 bspwm(1) 和 sxhkd(1) 手册获取详细信息。

设置多显示器

以下示例配置将会配置10个桌面在单独的显示器上：

```
bspc monitor -d I II III IV V VI VII VIII IX X
```

如果使用多显示器，则需要为每一个显示器配置：

```
bspc monitor DVI-I-1 -d I II III IV
bspc monitor DVI-I-2 -d V VI VII
bspc monitor DP-1 -d VIII IX X
```

使用 `xrandr -q` 或 `bspc query -M --names` 命令来查看显示器编号。

在上面的示例中，桌面的总数保持在10个，在 sxhkdrc 中，这样每一个桌面仍然可以使用 `super + {1-9,0}` 来定位。

规则（rules）

有两种方法来设置窗口规则 (截至 cd97a32)

第一种是通过使用内置的 rule 命令，如示例 bspwmrc 所示：

```
bspc rule -a Gimp desktop=^8 follow=on state=floating
bspc rule -a Chromium desktop=^2
bspc rule -a mplayer2 state=floating
bspc rule -a Kupfer.py focus=on
bspc rule -a Screenkey manage=off
```

第二种是使用外部的 rule 命令。虽然更加复杂，但是这允许你去构造更复杂的桌面规则。参考 these examples 获取示例 rule 命令。

如果某个特定的窗口似乎没有按照你的规则运行，请检查程序的类名。可以通过运行 `xprop | grep WM_CLASS` 来确保你在使用正确的字符串，这需要 xorg-xprop包 软件包。

面板（Panels）

使用lemonbar

在lemonbar的Github页面提供了一个 lemonbar-gitAUR 的示例面板。你也可从 lemonbar 的维基页面得到了些许启发。面板将通过 bspwmrc 中的 `panel &` 启动。检查在 bspwm包 软件包中的 optdepends，以获取可能需要的依赖。

为了在你的状态栏（status bar）中显示系统信息，你可以使用各种系统调用（system calls）。示例将会展示给你如何编辑你的 `panel` 来使音量状态显示在你的状态栏中：

```
panel_volume()
{
        volStatus=$(amixer get Master | tail -n 1 | cut -d '[' -f 4 | sed 's/].*//g')
        volLevel=$(amixer get Master | tail -n 1 | cut -d '[' -f 2 | sed 's/%.*//g')
```

```
        # is alsa muted or not muted?
        if [ "$volStatus" == "on" ]
        then
                echo "%{Fyellowgreen} $volLevel %{F-}"
        else
                # If it is muted, make the font red
                echo "%{Findianred} $volLevel %{F-}"
        fi
}
```

之后，我们必须确保它已被调用并且重定向至 `$PANEL_FIFO` ：

```
while true; do
echo "S" "$(panel_volume) $(panel_clock)" > "$PANEL_FIFO"
        sleep 1s
done &
```

使用yabar

使用lemonbar示例面板需要你配置你的环境（.profile），并确保面板脚本在你的路径上。yabarAUR 是更易配置的面板，只有一个配置文件。

使用polybar

通过在你的配置文件中添加 `polybar *example* &` 来使用 Polybar，其中的 `*example*` 就是状态栏的名称。

暂存器/便笺（Scratchpad）

使用pid

你可以使用 bspwm 的窗口标签来模仿下拉式终端（例如 i3wm 的便笺功能）。将下面的内容附加到 bspwm 的配置文件末尾（适用于你自己的终端）：

```
bspc rule -a scratchpad sticky=on state=floating hidden=on
# check scratchpad already running
[ "$(ps -x | grep -c 'scratchpad')" -eq "1" ] && st -c scratchpad -e ~/bin/scratch &
```

`sticky` 标签可确保窗口总是出现在当前的桌面上。 `~/bin/scratch` 是：

```
#!/usr/bin/sh
# only add floating scratchpad window node id to /tmp/scratchid
bspc query -N -n .floating | xargs -i sh -c 'bspc query  --node {} -T | grep -q
scratchpad && echo {} > /tmp/scratchid'
exec $SHELL
```

切换 Scratchpad 的热键应该绑定到：

```
id=$(cat /tmp/scratchid);\
bspc node $id --flag hidden;bspc node -f $id
```

使用类名（class name）

在此示例中我们将使用带有自定义类名的 *termite* 作为我们的下拉式终端。当然其类名没有必要必须是 *termite*。

首先在你的路径中创建一个有如下内容的文件并确保其可执行。在此示例中我们命其名为 `scratchpad.sh` ：

```
#!/usr/bin/bash
```

```
if [ -z $1 ]; then
       echo "Usage: $0 <name of hidden scratchpad window>"
       exit 1
fi

pids=$(xdotool search --class ${1})
for pid in $pids; do
       echo "Toggle $pid"
       bspc node $pid --flag hidden -f
done
```

随后添加这个到你的 bspwm 配置中。

```
...
bspc rule -a dropdown sticky=on state=floating hidden=on
termite --class dropdown -e "zsh -i" &
...
```

为了切换窗口，一个在 sxhkd 中的自定义规则是必不可少的。给定自定义类名作为参数。

```
super + u
        scratchpad.sh dropdown
```

其他

对于无需预定义规则就可使用任何窗口类型的Scratchpad，参阅：

有关支持更多开箱即用的终端，并具有用于执行诸如选择性启动tmuxinator / tmux会话，即时将任何窗口变成便笺本以及自动调整便笺本大小以适合当前显示器的标志的更复杂的便笺本脚本，参阅 tdrop-gitAUR 。

不同设备的显示器配置

因为 `bspwmrc` 是一个shell脚本，这允许你做类似如下的操作：

```
#! /bin/sh

 if [[ $(hostname) == 'myhost' ]]; then
     bspc monitor eDP1 -d I II III IV V VI VII VIII IX X
 elif [[ $(hostname) == 'otherhost' ]]; then
     bspc monitor VGA-0 -d I II III IV V
     bspc monitor VGA-1 -d VI VII VIII IX X
 elif [[ $(hostname) == 'yetanotherhost' ]]; then
     bspc monitor DVI-I-3 -d VI VII VIII IX X
     bspc monitor DVI-I-2 -d I II III IV V
 fi
```

注意：使用hostname命令需要 inetutils包 软件包。

设置某一桌面的所有窗口均为浮动模式

下面的示例会将 3 号桌面的所有窗口均设置为浮动窗口。当使用 GIMP 或其他多窗口的应用程序时非常有用。

把这个脚本放到你的 `$PATH` 的某个位置并从 `.xinitrc` 或类似的（最后带有 `&`）调用：

```
#!/bin/bash

 # change the desktop number here
 FLOATING_DESKTOP_ID=$(bspc query -D -d '^3')
```

```
bspc subscribe node_add | while read -a msg ; do
    desk_id=${msg}
    wid=${msg}
    [ "$FLOATING_DESKTOP_ID" = "$desk_id" ] && bspc node "$wid" -t floating
done
```

(源)

键盘

bspwm 不会处理任何键盘输入而是提供 *bspc* 程序作为接口。

你必须为快捷键设置一个守护程序例如 sxhkd包 （开发版本：sxhkd-gitAUR）

故障排除

屏幕空白，按键绑定不可用

- 确保 sxhkd包 已被安装。
- 确保 sxhkd 正在运行。
- 确保 `~/.config/bspwm/bspwmrc` 为可执行文件。

鼠标指针主题没有应用到桌面

参阅 Cursor themes#Change X shaped default cursor

窗口大于实际运行的应用程序

若你正在使用GTK3应用程序并且经常用于对话窗口，那么这种情况就会发生。修复方法是创建或添加以下内容到GTK3主题文件 （`~/.config/gtk-3.0/gtk.css`）。

```
.window-frame, .window-frame:backdrop {
  box-shadow: 0 0 0 black;
  border-style: none;
  margin: 0;
  border-radius: 0;
}

.titlebar {
  border-radius: 0;
}
```

(源: Bspwm forum thread)

Java程序的问题

如果你有例如Java程序窗口无法调整大小，或菜单在点击后立即关闭的问题，参阅 Java#Gray window, applications not resizing with WM, menus immediately closing。

此外，一些基于Java的应用程序显示不出任何窗口内容（例如 Intellij的IDE)。一个解决办法是安装 wmname包 软件包并且添加以下内容到你的 `~/.config/bspwm/bspwmrc` ：

```
wmname LG3D
```

fish shell的按键绑定问题

如果你正在使用 fish ，你会发现你不能切换桌面。这是因为 bspc 的 ^ 字符与 fish 不兼容。你可通过明确告诉 sxhkd 使用 bash 执行命令来解决问题。

```
$ set -U SXHKD_SHELL /usr/bin/bash
```

或者，将 ^ 字符在你的 sxhkdrc 中用反斜线(\\)转义。

使用fish shell的性能问题

sxhkd 使用 SHELL 环境变量中的 shell 来执行命令。由于配置文件很大或配置不当，fish 的加载时间可能很长，因此所有 sxhkd 命令的执行时间可能比其他的shell都长。为了在不改变你的默认 shell 情况下修复这个问题，你可以明确地告诉 sxhkd 使用 bash 或其他更快的 shell 来执行命令 （比如sh）：

```
$ set -U SXHKD_SHELL sh
```

启动时报 "Could not grab key 43 with modfield 68" 错误

要么你尝试绑定同一个键两次，要么你启动了两次 sxhkd ，检查 bspwmrc 和 `~/.profile` 或 `~/.bash_profile`，查看是否在启动 sxhkd 时有多余的命令。

火狐浏览器（Firefox）菜单在右键单击时自动选择第一选项

添加以下内容到 `userChrome.css` Firefox配置文件：

```
#contentAreaContextMenu{ margin: 5px 0 0 5px }
```

文件应位于 `~/.mozilla/firefox/*something*.default/chrome/` （如果没有，就创建一个）。在Firefox中，你必须到 `about:config` 页面开启 `toolkit.legacyUserProfileCustomizations.stylesheets` 选项；否则，Firefox会忽略 userChrome.css 文件。

参照

- Mailing List: bspwm *at* librelist.com.
- `#bspwm` - IRC channel at irc.libera.chat
- https://bbs.archlinux.org/viewtopic.php?id=149444 - Arch BBS thread
- https://github.com/baskerville/bspwm - GitHub project
- https://github.com/windelicato/dotfiles/wiki/bspwm-for-dummies - earsplit's "bspwm for dummies"

7.7 Ion3

难度 进阶 | **篇幅** 19.2 KB | **所属** 平铺式窗口管理器

本节将讲到

- cfg_ion.lua
- cfg_bindings.lua

- cfg_menus.lua
- cfg_ionws.lua
- look.lua

Ion 是一个有着标签(tab)框架的风格的，平铺的窗体管理器。它的配置文件以Lua语言作为内嵌的释译器。它的功能主要使用键盘来控制，同时也支持鼠标。

Ion3现已被移出官方软件仓库，详细信息可以看Arch Linux新闻页面。所以现在只能通过AUR来安装。

安装好之后，把配置文件拷贝到您的主目录下：

```
cp /etc/ion3/* ~/.ion3
```

要启用Ion3，需把下面这一行加到 **~/.xinitrc** 中来：

```
exec ion3
```

现在您可以按下面配置文件所述，来配置Ion。

注意: 相比原有的配置文件，我已经对这些文件基于我自己喜好进行了很大的改动。/xerxes2

cfg_ion.lua

Ion3的主配置文件。

```
-- Ion main configuration file
--
-- Ion主配置文件
--

--
-- Some basic setup
--
-- 一些基本的设置
--

-- Set default modifiers. Alt should usually be mapped to Mod1 on
-- XFree86-based systems. The flying window keys are probably Mod3
-- or Mod4; see the output of 'xmodmap'.
--
-- 设置默认功能键(modifiers)。在基于XFree86的X系统中，Alt键通常被映射为Mod1，
-- Win键常被映射为Mod3或是Mod4，详见 'xmodmap' 命令的输出。
--
MOD1="Mod4+"
MOD2="Mod1+"

ioncore.set{
    -- Maximum delay between clicks in milliseconds to be considered a
    -- double click.
    dblclick_delay=250,

    -- For keyboard resize, time (in milliseconds) to wait after latest
    -- key press before automatically leaving resize mode (and doing
    -- the resize in case of non-opaque move).
    kbresize_delay=1500,

    -- Opaque resize?
    opaque_resize=false,
```

```
    -- Movement commands warp the pointer to frames instead of just
    -- changing focus. Enabled by default.
    warp=true,

    -- Default workspace type.
    default_ws_type="WIonWS",
}

--
-- Load some modules, extensions and other configuration files
--
-- 加载一些模块，扩展功能，以及其它配置文件

-- Load some modules.
dopath("mod_query")
dopath("mod_menu")
dopath("mod_ionws")
dopath("mod_floatws")
dopath("mod_panews")
--dopath("mod_statusbar")
--dopath("mod_dock")
dopath("mod_sp")

-- Load some kludges to make apps behave better.
-- 加载一些补充程序(kludges)，以使程序有更好的外观。
dopath("cfg_kludges")

-- Make some bindings.
-- 完成热键绑定
dopath("cfg_bindings")

-- Define some menus (mod_menu required)
-- 定义一些菜单（需要用到mod_menu）
dopath("cfg_menus")

-- Load additional user configuration. 'true' as second parameter asks
-- Ion not to complain if the file is not found.
-- 加载用户自定义的配置。第二个参数 'ture' 告诉Ion被抱怨找不到该文件。
dopath("cfg_user", true)
```

要使用热键，按住功能键的同时按下热键（Modkey + hotkey）。 **dopath** 命令能配置Ion启动时加载的模块。如果您不想启动某个模块，把它所在的那一行注释即可。

cfg_bindings.lua

这个文件配置键盘上的控制热键。

```
--
-- Ion bindings configuration file. Global bindings and bindings common
-- to screens and all types of frames only. See modules' configuration
-- files for other bindings.
--
--
-- 此配置文件只用于Ion全局、screan，以及各种frame的热键绑定。
-- 想了解配置的细节，参见各模块的配置文件。
--

-- WScreen context bindings
--
-- WScreen 事件的热键绑定
--
-- The bindings in this context are available all the time.
--
-- 此文中的各种热键绑定将一直生效。
```

```
--
-- The variable MOD1 should contain a string of the form 'Mod1+'
-- where Mod1 maybe replaced with the modifier you want to use for most
-- of the bindings. Similarly MOD2 may be redefined to add a
-- modifier to some of the F-key bindings.
--
-- MOD1变量将赋于一个字符串，该字符串以 'Mod1+'的形式表征。
-- 此Mod1可以为任何一个您想使用的modifier，用于绝大多数的热键绑定。
-- 和MOD1相似地，MOD2也可以被重新定义。由此添加一个用于F-key绑定的modifier。
--
defbindings("WScreen", {
    bdoc("Switch to n:th object (workspace, full screen client window) "..
         "within current screen."),
    kpress(MOD1.."1", "WScreen.switch_nth(_, 0)"),
    kpress(MOD1.."2", "WScreen.switch_nth(_, 1)"),
    kpress(MOD1.."3", "WScreen.switch_nth(_, 2)"),
    kpress(MOD1.."4", "WScreen.switch_nth(_, 3)"),
    kpress(MOD1.."5", "WScreen.switch_nth(_, 4)"),
    kpress(MOD1.."6", "WScreen.switch_nth(_, 5)"),
    kpress(MOD1.."7", "WScreen.switch_nth(_, 6)"),
    kpress(MOD1.."8", "WScreen.switch_nth(_, 7)"),
    kpress(MOD1.."9", "WScreen.switch_nth(_, 8)"),
    kpress(MOD1.."0", "WScreen.switch_nth(_, 9)"),

--    bdoc("Switch to next/previous object within current screen."),
--    kpress(MOD2.."comma", "ioncore.goto_previous()"),
--    kpress(MOD2.."period", "ioncore.goto_next()"),

    kpress(MOD1.."Left", "WScreen.switch_prev(_)"),
    kpress(MOD1.."Right", "WScreen.switch_next(_)"),

    submap(MOD1.."K", {
        bdoc("Go to previous active object."),
        kpress("K", "ioncore.goto_previous()"),

        bdoc("Clear all tags."),
        kpress("T", "ioncore.clear_tags()"),
    }),

    bdoc("Go to n:th screen on multihead setup."),
    kpress(MOD1.."Shift+1", "ioncore.goto_nth_screen(0)"),
    kpress(MOD1.."Shift+2", "ioncore.goto_nth_screen(1)"),

    bdoc("Go to next/previous screen on multihead setup."),
    kpress(MOD1.."Shift+Left", "ioncore.goto_next_screen()"),
    kpress(MOD1.."Shift+Right", "ioncore.goto_prev_screen()"),

    bdoc("Create a new workspace of chosen default type."),
    kpress(MOD1.."F9", "ioncore.create_ws(_)"),

    bdoc("Display the main menu."),
    kpress(MOD1.."F12", "mod_menu.bigmenu(_, _sub, 'mainmenu')"),
    mpress("Button3", "mod_menu.pmenu(_, _sub, 'mainmenu')"),

    bdoc("Display the window list menu."),
    mpress("Button2", "mod_menu.pmenu(_, _sub, 'windowlist')"),
})

-- WMPlex context bindings
--
-- WMPlex 事件的热键绑定
--
-- These bindings work in frames and on screens. The innermost of such
-- contexts/objects always gets to handle the key press. Most of these
-- bindings define actions on client windows. (Remember that client windows
-- can be put in fullscreen mode and therefore may not have a frame.)
--
-- 这些热键绑定在各个frame和screan中生效。这些事件/实体会一直在您按下按键时生效。
```

```
-- 它们所定义的，大多是控制窗体的动作。
-- （记得当窗体(client window)处于全屏模式时，它们不会表现为一个个的框(frame)。）
--
-- The "_sub:WClientWin" guards are used to ensure that _sub is a client
-- window in order to stop Ion from executing the callback with an invalid
-- parameter if it is not and then complaining.
-- 守卫进程 "_sub:WClientWin" 用于保证 _sub 为一个窗体(client window)，以便于当Ion
-- 在执行收回函数时接收到一个非法参数而抱怨。（此处可能有翻译不当）

defbindings("WMPlex", {
    bdoc("Close current object."),
    kpress_wait(MOD1.."C", "WRegion.rqclose_propagate(_, _sub)"),

    bdoc("Nudge current client window. This might help with some "..
         "programs' resizing problems."),
    kpress_wait(MOD1.."L",
                "WClientWin.nudge(_sub)", "_sub:WClientWin"),

    bdoc("Toggle fullscreen mode of current client window."),
    kpress_wait(MOD1.."Return",
                "WClientWin.set_fullscreen(_sub, 'toggle')",
                "_sub:WClientWin"),

    submap(MOD1.."K", {
       bdoc("Kill client owning current client window."),
       kpress("C", "WClientWin.kill(_sub)", "_sub:WClientWin"),

       bdoc("Send next key press to current client window. "..
            "Some programs may not allow this by default."),
       kpress("Q", "WClientWin.quote_next(_sub)", "_sub:WClientWin"),
    }),

    bdoc("Query for manual page to be displayed."),
    kpress(MOD2.."F1", "mod_query.query_man(_, ':man')"),

    bdoc("Show the Ion manual page."),
    kpress(MOD1.."F1", "ioncore.exec_on(_, ':man ion3')"),

    bdoc("Run a terminal emulator."),
    kpress(MOD1.."F2", "ioncore.exec_on(_, 'urxvt')"),

    bdoc("Query for command line to execute."),
    kpress(MOD2.."F3", "mod_query.query_exec(_)"),

    bdoc("Query for Lua code to execute."),
    kpress(MOD1.."F3", "mod_query.query_lua(_)"),

    bdoc("Query for host to connect to with SSH."),
    kpress(MOD2.."F4", "mod_query.query_ssh(_, ':ssh')"),

    bdoc("Query for file to edit."),
    kpress(MOD2.."F5",
           "mod_query.query_editfile(_, 'run-mailcap --action=edit')"),

    bdoc("Query for file to view."),
    kpress(MOD2.."F6",
           "mod_query.query_runfile(_, 'run-mailcap --action=view')"),

    bdoc("Query for workspace to go to or create a new one."),
    kpress(MOD2.."F9", "mod_query.query_workspace(_)"),

    bdoc("Query for a client window to go to."),
    kpress(MOD1.."G", "mod_query.query_gotoclient(_)"),
})

-- WFrame context bindings
--
```

```
-- WFrame 事件的热键绑定
--
-- These bindings are common to all types of frames. The rest of frame
-- bindings that differ between frame types are defined in the modules'
-- configuration files.
--
-- 这些热键绑定用于各种框体(frame)。其它框体的配置在模块配置文件中定义。
--

defbindings("WFrame", {
    bdoc("Tag current object within the frame."),
    kpress(MOD1.."T", "WRegion.set_tagged(_sub, 'toggle')", "_sub:non-nil"),

    submap(MOD1.."K", {
        bdoc("Switch to n:th object within the frame."),
        kpress("1", "WFrame.switch_nth(_, 0)"),
        kpress("2", "WFrame.switch_nth(_, 1)"),
        kpress("3", "WFrame.switch_nth(_, 2)"),
        kpress("4", "WFrame.switch_nth(_, 3)"),
        kpress("5", "WFrame.switch_nth(_, 4)"),
        kpress("6", "WFrame.switch_nth(_, 5)"),
        kpress("7", "WFrame.switch_nth(_, 6)"),
        kpress("8", "WFrame.switch_nth(_, 7)"),
        kpress("9", "WFrame.switch_nth(_, 8)"),
        kpress("0", "WFrame.switch_nth(_, 9)"),

        bdoc("Switch to next/previous object within the frame."),
        kpress("N", "WFrame.switch_next(_)"),
        kpress("P", "WFrame.switch_prev(_)"),

        bdoc("Move current object within the frame left/right."),
        kpress("comma", "WFrame.dec_index(_, _sub)", "_sub:non-nil"),
        kpress("period", "WFrame.inc_index(_, _sub)", "_sub:non-nil"),

        bdoc("Maximize the frame horizontally/vertically."),
        kpress("H", "WFrame.maximize_horiz(_)"),
        kpress("V", "WFrame.maximize_vert(_)"),

        bdoc("Attach tagged objects to this frame."),
        kpress("A", "WFrame.attach_tagged(_)"),
    }),

    kpress(MOD1.."comma", "WFrame.switch_prev(_)"),
    kpress(MOD1.."period", "WFrame.switch_next(_)"),

    bdoc("Query for a client window to attach to active frame."),
    kpress(MOD1.."A", "mod_query.query_attachclient(_)"),

    bdoc("Display frame context menu."),
    kpress(MOD1.."M", "mod_menu.menu(_, _sub, 'ctxmenu')"),
    mpress("Button3", "mod_menu.pmenu(_, _sub, 'ctxmenu')"),

    bdoc("Begin move/resize mode."),
    kpress(MOD1.."R", "WFrame.begin_kbresize(_)"),

    bdoc("Switch the frame to display the object indicated by the tab."),
    mclick("Button1@tab", "WFrame.p_switch_tab(_)"),
    mclick("Button2@tab", "WFrame.p_switch_tab(_)"),

    bdoc("Resize the frame."),
    mdrag("Button1@border", "WFrame.p_resize(_)"),
    mdrag(MOD1.."Button3", "WFrame.p_resize(_)"),

    bdoc("Move the frame."),
    mdrag(MOD1.."Button1", "WFrame.p_move(_)"),

    bdoc("Move objects between frames by dragging and dropping the tab."),
    mdrag("Button1@tab", "WFrame.p_tabdrag(_)"),
```

```
    mdrag("Button2@tab", "WFrame.p_tabdrag(_)"),

})

-- WMoveresMode context bindings
--
-- WMoveresMode 事件的绑定
--
-- These bindings are available keyboard move/resize mode. The mode
-- is activated on frames with the command begin_kbresize (bound to
-- MOD1.."R" above by default).
--
-- 这些绑定让您可以使用键盘来移动和改变框体的大小(move/resize mode)。
-- 您可以用begin_kbresize命令来进入这个移动和改变框体大小的状态。
-- （默认是绑定于上面的MOD1.."R"）

defbindings("WMoveresMode", {
    bdoc("Cancel the resize mode."),
    kpress("AnyModifier+Escape","WMoveresMode.cancel(_)"),

    bdoc("End the resize mode."),
    kpress("AnyModifier+Return","WMoveresMode.finish(_)"),

    bdoc("Grow in specified direction."),
    kpress("Left",  "WMoveresMode.resize(_, 1, 0, 0, 0)"),
    kpress("Right", "WMoveresMode.resize(_, 0, 1, 0, 0)"),
    kpress("Up",    "WMoveresMode.resize(_, 0, 0, 1, 0)"),
    kpress("Down",  "WMoveresMode.resize(_, 0, 0, 0, 1)"),
    kpress("F",     "WMoveresMode.resize(_, 1, 0, 0, 0)"),
    kpress("B",     "WMoveresMode.resize(_, 0, 1, 0, 0)"),
    kpress("P",     "WMoveresMode.resize(_, 0, 0, 1, 0)"),
    kpress("N",     "WMoveresMode.resize(_, 0, 0, 0, 1)"),

    bdoc("Shrink in specified direction."),
    kpress("Shift+Left",  "WMoveresMode.resize(_,-1, 0, 0, 0)"),
    kpress("Shift+Right", "WMoveresMode.resize(_, 0,-1, 0, 0)"),
    kpress("Shift+Up",    "WMoveresMode.resize(_, 0, 0,-1, 0)"),
    kpress("Shift+Down",  "WMoveresMode.resize(_, 0, 0, 0,-1)"),
    kpress("Shift+F",     "WMoveresMode.resize(_,-1, 0, 0, 0)"),
    kpress("Shift+B",     "WMoveresMode.resize(_, 0,-1, 0, 0)"),
    kpress("Shift+P",     "WMoveresMode.resize(_, 0, 0,-1, 0)"),
    kpress("Shift+N",     "WMoveresMode.resize(_, 0, 0, 0,-1)"),

    bdoc("Move in specified direction."),
    kpress(MOD1.."Left",  "WMoveresMode.move(_,-1, 0)"),
    kpress(MOD1.."Right", "WMoveresMode.move(_, 1, 0)"),
    kpress(MOD1.."Up",    "WMoveresMode.move(_, 0,-1)"),
    kpress(MOD1.."Down",  "WMoveresMode.move(_, 0, 1)"),
    kpress(MOD1.."F",     "WMoveresMode.move(_,-1, 0)"),
    kpress(MOD1.."B",     "WMoveresMode.move(_, 1, 0)"),
    kpress(MOD1.."P",     "WMoveresMode.move(_, 0,-1)"),
    kpress(MOD1.."N",     "WMoveresMode.move(_, 0, 1)"),
})
```

cfg_menus.lua

cfg_menus.lua配置您的主菜单。

```
--
-- Ion menu definitions
--
-- Ion 菜单定义
--

-- Main menu
-- 主菜单
defmenu("mainmenu", {
```

```
    submenu("Programs",          "appmenu"),
    menuentry("Lock screen",     "ioncore.exec_on(_, 'xlock')"),
    menuentry("Help",            "mod_query.query_man(_)"),
    menuentry("About Ion",       "mod_query.show_about_ion(_)"),
    submenu("Styles",            "stylemenu"),
    submenu("Session",           "sessionmenu"),
})

-- Application menu
-- 程序菜单
defmenu("appmenu", {
    menuentry("Firefox","ioncore.exec_on(_, 'firefox')"),
    menuentry("Thunderbird","ioncore.exec_on(_, 'thunderbird')"),
    menuentry("Editor","ioncore.exec_on(_, 'lazy-edit')"),
    menuentry("Player","ioncore.exec_on(_, 'lazy-player')"),
    menuentry("FTP","ioncore.exec_on(_, 'lazy-ftp')"),
    menuentry("GVim","ioncore.exec_on(_, 'gvim')"),
    menuentry("Run...", "mod_query.query_exec(_)"),
})

-- Session control menu
-- Session 控制菜单
defmenu("sessionmenu", {
    menuentry("Save",            "ioncore.snapshot()"),
    menuentry("Restart",         "ioncore.restart()"),
    menuentry("Restart PWM",     "ioncore.restart_other('pwm')"),
    menuentry("Restart TWM",     "ioncore.restart_other('twm')"),
    menuentry("Exit",            "ioncore.shutdown()"),
})

-- Context menu (frame/client window actions)
-- 事件菜单 （作用于框体/窗体）
defctxmenu("WFrame", {
    menuentry("Close",           "WRegion.rqclose_propagate(_, _sub)"),
    menuentry("Kill",            "WClientWin.kill(_sub)",
                                 "_sub:WClientWin"),
    menuentry("(Un)tag",         "WRegion.set_tagged(_sub, 'toggle')",
                                 "_sub:non-nil"),
    menuentry("Attach tagged",   "WFrame.attach_tagged(_)"),
    menuentry("Clear tags",      "ioncore.clear_tags()"),
    menuentry("Window info",     "mod_query.show_clientwin(_, _sub)",
                                 "_sub:WClientWin"),
})
```

cfg_ionws.lua

cfg_ionws.lua配置工作区。

```
--
-- Ion ionws module configuration file
--
-- Ion的ionws模块配置文件
--

-- Bindings for the tiled workspaces (ionws). These should work on any
-- object on the workspace.
-- 用于叠覆的工作区(ionws)的热键绑定。将用于工作区中的所有实体。

defbindings("WIonWS", {
    bdoc("Split current frame vertically."),
    kpress(MOD1.."V", "WIonWS.split_at(_, _sub, 'bottom', true)"),

    bdoc("Split current frame horizontally."),
    kpress(MOD1.."H", "WIonWS.split_at(_, _sub, 'right', true)"),

    bdoc("Destroy current frame."),
    kpress(MOD1.."Tab", "WIonWS.unsplit_at(_, _sub)"),
```

```
    bdoc("Go to frame above/below/right/left of current frame."),
    kpress(MOD2.."Up", "WIonWS.goto_dir(_, 'above')"),
    kpress(MOD2.."Down", "WIonWS.goto_dir(_, 'below')"),
    kpress(MOD2.."Right", "WIonWS.goto_dir(_, 'right')"),
    kpress(MOD2.."Left", "WIonWS.goto_dir(_, 'left')"),
    submap(MOD1.."K", {
        bdoc("Split current frame horizontally."),
        kpress("S", "WIonWS.split_at(_, _sub, 'right', true)"),

        bdoc("Destroy current frame."),
        kpress("X", "WIonWS.unsplit_at(_, _sub)"),
    }),
})

-- Frame bindings. These work in (Ion/tiled-style) frames. Some bindings
-- that are common to all frame types and multiplexes are defined in
-- ion-bindings.lua.
--
-- 框体的热键绑定。用于（Ion/叠覆风格的）框体。
-- 一些对所有框体皆用的绑定，定义于ion-bindings.lua。
--

--defbindings("WFrame-on-WIonWS", {
--})

-- WFloatFrame context menu extras
-- 额外的 WFloatFrame 事件菜单

if mod_menu then
    defctxmenu("WIonWS", {
        menuentry("Destroy frame",
                  "WIonWS.unsplit_at(_, _sub)"),
        menuentry("Split vertically",
                  "WIonWS.split_at(_, _sub, 'bottom', true)"),
        menuentry("Split horizontally",
                  "WIonWS.split_at(_, _sub, 'right', true)"),
        menuentry("Flip", "WIonWS.flip_at(_, _sub)"),
        menuentry("Transpose", "WIonWS.transpose_at(_, _sub)"),
        submenu("Float split", {
            menuentry("At left",
                      "WIonWS.set_floating_at(_, _sub, 'toggle', 'left')"),
            menuentry("At right",
                      "WIonWS.set_floating_at(_, _sub, 'toggle', 'right')"),
            menuentry("Above",
                      "WIonWS.set_floating_at(_, _sub, 'toggle', 'up')"),
            menuentry("Below",
                      "WIonWS.set_floating_at(_, _sub, 'toggle', 'down')"),
        }),
        submenu("At root", {
            menuentry("Split vertically",
                      "WIonWS.split_top(_, 'bottom')"),
            menuentry("Split horizontally",
                      "WIonWS.split_top(_, 'right')"),
            menuentry("Flip", "WIonWS.flip_at(_)"),
            menuentry("Transpose", "WIonWS.transpose_at(_)"),
        }),
    })
end
```

look.lua

look.lua设置窗体的主题。在文件末端，默认的主题被设置为：

```
dopath("look_xerxes")
```

主题的命名规则为： **look_<主题名>.lua** 。修改主题名，即可得到相应的主题。

7.8 Xmonad

难度 高级 | **篇幅** 30.0 KB | **所属** 平铺式窗口管理器

本节将讲到

- 安装
- 如何启动
- 配置文件
- Exiting xmonad
- Tips and tricks

xmonad 是 X 下的一个平铺窗口管理器。窗口会自动排列、无间隙或重叠地平铺在屏幕上，从而最大限度地利用屏幕。窗口管理器功能可通过键盘访问：鼠标操作是可选的。

Xmonad 使用 Haskell 语言编写、配置并扩展功能的。用户可以将自定义布局算法、按键绑定和其他扩展功能写在配置文件中。

布局是动态应用的，并且在每个工作空间上可以使用不同的布局。 Xmonad 完全支持 Xinerama，后者可允许将窗口平铺在多个物理屏幕上。

安装

安装 xmonad包 软件包，其提供了最基本的配置，最好还安装上 xmonad-contrib包，可获得更有用的桌面配置以及其他平铺算法、配置和脚本等。

此外，可安装开发版本的 xmonad-gitAUR, 其还附带了些额外的依赖；以及同时安装 xmonad-contrib-gitAUR。

注意：记得在升级 xmonad 后运行 `xmonad --recompile` 命令，否则下次启动寻找共享库时可能会遇到问题。

如何启动

用 xinit 运行 `xmonad`.

或者，你也可以在 display manager（SDDM） 的会话菜单中选择 *XMonad* 。

请确保在启动 *XMonad* 之前你已经安装了至少一个默认终端模拟器 Xterm，当然你也可以在这之前修改 *XMonad* 的配置文件 `~/.xmonad/xmonad.hs` 来指定一个默认的终端，这很重要，否则你将无法在新的窗口管理器下进行任何工作。

注意：默认情况下，*XMonad* 之显示一个十字光标， 可以根据这个文章内容来更改默认显示, see Cursor themes#Change X shaped default cursor.

配置文件

创建一个配置目录 `~/.xmonad` 和 配置文件 `.xmonad/xmonad.hs`，然后根据下面的指导编写基础的配置项。

在更改配置文件 `~/.xmonad/xmonad.hs` 后，使用 Mod+q 快捷键来重新启动 *XMonad*，使更改生效。

提示：默认的*XMonad* 的配置在没有 `xmonad.hs` 配置文件的情况下也是十分友好的

因为 xmonad 的配置文件使用的是 *Haskell* 语言编写，一般人需要自定义定制一个自己的配置文件可能是困难且耗费时间的，以下网站资源是一些关于 xmonad 的详细配置示例参考:

- xmonad wiki
- xmonad configuration archive
- xmonad FAQ
- Arch Linux forum thread

把更改和自定义内容写入 `~/.xmonad/xmonad.hs`， XMonad 将会从你的 *def*函数中读取相关参数的设定，

一个编写的 `xmonad.hs` 示例如下:

```
import XMonad

main = xmonad def
    { terminal    = "urxvt"   --默认终端
    , modMask     = mod4Mask --快捷前置键，4是win键，1是alt也是默认的
    , borderWidth = 3         --边框宽度
    }
```

这里只更改了默认终端，快捷前置键和窗口边框的宽度，其他的设置还是 XMonad 的默认值 （默认值从 *Xconfig*继承）。

当配置越来越复杂的时候，可以把参数以赋值的形式传递给 def 函数中的变量，用更容易识别的变量名可以优化配置文件的格式布局，方便自己和其他人查看与维护。 下面是一个简单的例子：

```
import XMonad

myTerminal    = "urxvt"
myModMask     = mod4Mask -- Win key or Super_L
myBorderWidth = 3

main = do
  xmonad $ def
    { terminal    = myTerminal
    , modMask     = myModMask
    , borderWidth = myBorderWidth
    }
```

Also, order at top level (main, myTerminal, myModMask etc.), or within the {} does not matter in Haskell, as long as imports come first.

```
{
  terminal           = myTerminal,
  focusFollowsMouse  = myFocusFollowsMouse,
  borderWidth        = myBorderWidth,
  modMask            = myModMask,
  -- numlockMask deprecated in 0.9.1
  -- numlockMask        = myNumlockMask,
  workspaces         = myWorkspaces,
  normalBorderColor  = myNormalBorderColor,
  focusedBorderColor = myFocusedBorderColor,

  -- key bindings
  keys               = myKeys,
  mouseBindings      = myMouseBindings,

  -- hooks, layouts
  layoutHook         = myLayout,
  manageHook         = myManageHook,
  handleEventHook    = myEventHook,
  logHook            = myLogHook,
  startupHook        = myStartupHook
}
```

The package itself also includes a `xmonad.hs`, which is the latest official example `xmonad.hs` that comes with the **xmonad** Haskell module as an example of how to override everything. This should not be used as a template configuration, but as examples of parts you can pick to use in your own configuration. It is located in an architecture and version dependant directory in `/usr/share/` (e.g. `find /usr/share -name xmonad.hs`).

A base desktop configuration

In xmonad-contrib包 is a better default configuration for average desktop uses. It also helps with problems in some modern programs like Chromium.

It can be added like so:

```
import XMonad
import XMonad.Config.Desktop

main = xmonad desktopConfig
    { terminal    = "urxvt"
    , modMask     = mod4Mask
    }
```

Exiting xmonad

To end the current xmonad session, press `Mod+Shift+Q`. By default, `Mod` is the `Alt` key.

Tips and tricks

X-Selection-Paste

The keyboard-centered operation in xmonad can be further supported with a keyboard shortcut for X-Selection-Paste.

Also, there exists a function "pasteSelection" in XMonad.Util.Paste that can be bound to a key using a line like:

```
xmonad.hs
  import XMonad.Util.Paste -- Remember to include this line

  -- X-selection-paste buffer
  , ((0, xK_Insert), pasteSelection)
```

Pressing the "Insert" key will now paste the mouse buffer in the active window.

Keyboard shortcuts

Default keyboard shortcuts are listed in the man page of xmonad.

**本文或本章节的事实准确性存在争议。**

原因： `Shift+Insert` pastes the PRIMARY buffer.（在 Talk:Xmonad 中讨论）

Users who prefer to work with the keyboard rather than the mouse may benefit from a key binding to the paste operation of the middle mouse button. This is especially useful in a keyboard-centered environment. A workflow example is:

1. In Firefox, select a string you want to web-search for (with the mouse).
2. Hit `Ctrl+k` to enter the "search engine" field.
3. Hit `F9` to paste the buffer, instead of moving the mouse pointer to the field and middle-click to paste.

注意：`Shift+Insert` has a similar yet different functionality, see #Xorg: `Shift+Insert` inserts the clipboard buffer, not the x-selection-paste buffer. In some applications, these two buffers are mirrored.

The method suggested here uses the following three packages::

- xsel包 to give access to the x-selection-buffer content.
- Xbindkeys to bind a key-stroke to an action.
- xvkbdAUR to pass the buffer string to the application by emulating keyboard input.

This example binds the x-selection-paste operation to the `F9` key:

```
.xbindkeysrc
"xvkbd -no-jump-pointer -xsendevent -text "\D1`xsel`" 2>/dev/null"
    F9
```

The `"\D1"` code prefixes a 100 ms pause to inserting the selection buffer (see the xvkbd home page).

注意：Depending on your X configuration, you may need to drop the `-xsendevent` argument to xvkbd.

The key codes for keys other than `F9` can be determined using `xbindkeys -k`.

References:

- Pasting X selection (not clipboard) contents with keyboard
- xvkbd home page

Targeting unbound keys

If you use xmonad as a stand alone window manager, you can edit the xmonad.hs to add unbound keyboard keys. You just need to find the Xf86 name of the key (such as XF86PowerDown) and look it up in `/usr/include/X11/XF86keysym.h`. It will give you a keycode (like 0x1008FF2A) which you can use to add a line like the following in the keybindings section of your `xmonad.hs`:

```
((0,               0x1008FF2A), spawn "sudo pm-suspend")
```

Increase the number of workspaces

By default, xmonad uses a set of 9 workspaces. You can change this by changing the **workspaces** parameter:

```
xmonad.hs
import XMonad
import XMonad.Util.EZConfig (additionalKeys)

main=do
  xmonad $ def
    { ...
    , workspaces = myWorkspaces
    , ...
    } `additionalKeys` myAdditionalKeys

myWorkspaces = ["1","2","3","4","5","6","7","8","9"] ++ (map snd myExtraWorkspaces) --
you can customize the names of the default workspaces by changing the list

myExtraWorkspaces = [(xK_0, "0")] -- list of (key, name)

myAdditionalKeys =
    [ -- ... your other hotkeys ...
    ] ++ [
        ((myModMask, key), (windows $ W.greedyView ws))
        | (key, ws) <- myExtraWorkspaces
    ] ++ [
        ((myModMask .|. shiftMask, key), (windows $ W.shift ws))
        | (key, ws) <- myExtraWorkspaces
    ]
```

Making room for docks/panels/trays (Xmobar, Tint2, Conky, etc)

Wrap your layouts with avoidStruts from XMonad.Hooks.ManageDocks for automatic dock/panel/trayer spacing:

```
import XMonad
import XMonad.Hooks.ManageDocks

main=do
  xmonad $ docks def
    { ...
    , layoutHook=avoidStruts $ layoutHook def
    , manageHook=manageHook def <+> manageDocks
    , ...
    }
```

If you ever want to toggle the gaps, this action can be added to your key bindings:

```
,((modMask x, xK_b     ), sendMessage ToggleStruts)
```

Equally sized gaps between windows

If your goal is to have equally sized gaps between individual windows and the screen, the following code will not work as expected:

```
layoutHook = spacing 10 $ Tall (1 (3/100) (1/2)) ||| Full
```

This makes each window have its own spacing in each direction. If you have two windows side-by-side, the spacing in the middle will be combined, creating a gap that is twice as large as needed.

A workaround is to specify both a screen and a window spacing, but only use the top and left margins for the screen and bottom and right margins for the windows. To do this, change the above code into:

```
 layoutHook = spacingRaw False (Border 10 0 10 0) True (Border 0 10 0 10) True $ Tall (1 (3/100) (1/2)) ||| Full
```

Using xmobar with xmonad



本文或本章节可能需要合并到xmobar。

附注： Make use of the separate article available（在 Talk:Xmonad 中讨论）

xmobar is a light and minimalistic text-based bar, designed to work with xmonad. To use xmobar with xmonad, you will need two packages in addition to the xmonad包 package. These packages are xmonad-contrib包 and xmobar包 from the official repositories, or you can use xmobar-gitAUR from the AUR instead of the official xmobar包 package.

Here we will start xmobar from within xmonad, which reloads xmobar whenever you reload xmonad.

Open `~/.xmonad/xmonad.hs` in your favorite editor, and choose one of the two following options:

Quick, less flexible

注意：There is also dzen2包 which you can substitute for xmobar包 in either case.

Common imports:

```
import XMonad
import XMonad.Hooks.DynamicLog
```

The xmobar action starts xmobar and returns a modified configuration that includes all of the options described in #More configurable.

```
main = xmonad =<< xmobar def { modMask = mod4Mask {- or any other configurations here ... -}}
```

More configurable

As of xmonad(-contrib) 0.9, there is a new statusBar function in XMonad.Hooks.DynamicLog. It allows you to use your own configuration for:

- The command used to execute the bar
- The PP that determines what is being written to the bar
- The key binding to toggle the gap for the bar

The following is an example of how to use it:

```
~/.xmonad/xmonad.hs
-- Imports.
import XMonad
```

```
import XMonad.Hooks.DynamicLog

-- The main function.
main = xmonad =<< statusBar myBar myPP toggleStrutsKey myConfig

-- Command to launch the bar.
myBar = "xmobar"

-- Custom PP, configure it as you like. It determines what is being written to the bar.
myPP = xmobarPP { ppCurrent = xmobarColor "#429942" "" . wrap "<" ">" }

-- Key binding to toggle the gap for the bar.
toggleStrutsKey XConfig {XMonad.modMask = modMask} = (modMask, xK_b)

-- Main configuration, override the defaults to your liking.
myConfig = def { modMask = mod4Mask }
```

Verify XMobar config

The template and default xmobarrc contains this.

At last, open up `~/.xmobarrc` and make sure you have `StdinReader` in the template and run the plugin. E.g.

```
~/.xmobarrc
Config { ...
       , commands = [ Run StdinReader .... ]
         ...
       , template = " %StdinReader% ... "
       }
```

Now, all you should have to do is either to start, or restart, xmonad.

Controlling xmonad with external scripts

Here are a few ways to do it,

- use the following xmonad extension, XMonad.Hooks.ServerMode.
- simulate keypress events using xdotool包 or similar programs. See this Ubuntu forums thread. The following command would simulate the keypress `Super+n`:

```
xdotool key Super+n
```

- wmctrl包 -If you have desktopConfig or EwmhDesktops configured, this is a very easy to use and standard utility.

Launching another window manager within xmonad

If you are using xmonad-gitAUR, as of January of 2011, you can restart to another window manager from within xmonad. You just need to write a small script, and add stuff to your `~/.xmonad/xmonad.hs`. Here is the script.

```
~/bin/obtoxmd
#!/bin/sh
openbox
xmonad
```

And here are the modifications you need to add to your `~/.xmonad/xmonad.hs`:

```
~/.xmonad/xmonad.hs
import XMonad
--You need to add this import
import XMonad.Util.Replace
```

```
main do
    -- And this "replace"
    replace
    xmonad $ def
    {
    --Add the usual here
    }
```

You also need to add the following key binding:

```
~/xmonad/xmonad.hs
--Add a keybinding as follows:
((modm .|. shiftMask, xK_o     ), restart "/home/abijr/bin/obtoxmd" True)
```

Just remember to add a comma before or after and change the path to your actual script path. Now just `Mod+q` (restart xmonad to refresh the configuration), and then hit `Mod+Shift+o` and you should have Openbox running with the same windows open as in xmonad. To return to xmonad you should just exit Openbox. Here is a link to adamvo's `~/.xmonad/xmonad.hs` which uses this setup Adamvo's xmonad.hs

KDE and xmonad

The xmonad wiki has instructions on how to run xmonad inside KDE

It also might be a good idea to set a global keyboard shortcut in KDE to start xmonad in case it is accidentally killed or closed.

Disable plasmashell

You might want to disable plasmashell (the KDE5 thing responsible for desktop background, taskbar, tray, etc.).

```
  cp /etc/xdg/autostart/plasmashell.desktop ~/.config/autostart/
```

Then edit `~/.config/autostart/plasmashell.desktop` and replace `Exec=plasmashell` with `Exec=`. The result will look like this:

```
~/.config/autostart/plasmashell.desktop
[Desktop Entry]
Exec=
Name=Plasma Desktop Workspace
... # more stuff
```

IM Layout for Skype

In orded to create an IM layout for the newer versions of skype, the following code can be used:

```
xmonad.hs
myIMLayout = withIM (1%7) skype Grid
    where
      skype = And (ClassName "Skype") (Role "")
```

Example configurations

Below are some example configurations from fellow xmonad users. Feel free to add links to your own.

- brisbin33 :: simple, useful, readable :: config screenshot
- jelly :: Configuration with prompt, different layouts, twinview with xmobar :: xmonad.hs
- MrElendig :: Simple configuration, with xmobar :: xmonad.hs, .xmobarrc, screenshot.

- thayer :: A minimal mouse-friendly config ideal for netbooks :: configs screenshot
- vicfryzel :: Beautiful and usable xmonad configuration, along with xmobar configuration, xinitrc, dmenu, and other scripts that make xmonad more usable. :: git repository, screenshot.
- vogt :: Check out adamvo's config and many others in the official Xmonad/Config archive
- wulax :: Example of using xmonad inside Xfce. Contains two layouts for GIMP. :: xmonad.hs, screenshot.
- alex-courtis :: Clean xmonad, xmobar, media keys, screenshot, j4/dmenu; fonts rendered at the DPI reported by the monitor :: xmonad.hs, screenshot.

Troubleshooting

GNOME 3 and xmonad

**本文或本章节的事实准确性存在争议。**

原因： This no longer works with GNOME. It might still work with GNOME Flashback.（在 Talk:Xmonad 中讨论）

With the release of GNOME 3 custom GNOME sessions require additional steps to make GNOME play nicely with xmonad.

Add an xmonad session file for use by gnome-session (`/usr/share/gnome-session/sessions/xmonad.session`):

```
[GNOME Session]
Name=Xmonad session
RequiredComponents=gnome-panel;gnome-settings-daemon;
RequiredProviders=windowmanager;notifications;
DefaultProvider-windowmanager=xmonad
DefaultProvider-notifications=notification-daemon
```

Create a desktop file for GDM (`/usr/share/xsessions/xmonad-gnome-session.desktop`):

```
[Desktop Entry]
Name=Xmonad GNOME
Comment=Tiling window manager
TryExec=/usr/bin/gnome-session
Exec=gnome-session --session=xmonad
Type=XSession
```

Create or edit this file (`/usr/share/applications/xmonad.desktop`):

```
[Desktop Entry]
Type=Application
Encoding=UTF-8
Name=Xmonad
Exec=xmonad
NoDisplay=true
X-GNOME-WMName=Xmonad
X-GNOME-Autostart-Phase=WindowManager
X-GNOME-Provides=windowmanager
X-GNOME-Autostart-Notify=false
```

Finally, install xmonad-contrib包 and create or edit `~/.xmonad/xmonad.hs` to have the following

```
import XMonad
import XMonad.Config.Gnome

main = xmonad gnomeConfig
```

xmonad should now appear in the list of GDM sessions and also play nicely with gnome-session itself.

Compositing in GNOME and xmonad

Some applications look better (e.g. GNOME Do) when composition is enabled. This, however, is not the case in the default xmonad window manager. To enable it add an additional .desktop file `/usr/share/xsessions/xmonad-gnome-session-composite.desktop`:

```
[Desktop Entry]
Name=Xmonad GNOME (Composite)
Comment=Tiling window manager
TryExec=/usr/bin/gnome-session
Exec=/usr/sbin/gnome-xmonad-composite
Type=XSession
```

And create `/usr/sbin/gnome-xmonad-composite` and `chmod +x /usr/sbin/gnome-xmonad-composite`:

```
xcompmgr &
gnome-session --session=xmonad
```

Now choose "Xmonad GNOME (Composite)" in the list of sessions during login. Reference xcompmgr(1) for additional "eye candy".

Xfce 4 and xmonad

Use `xfceConfig` instead of `def` after importing `XMonad.Config.Xfce` in `~/.xmonad/xmonad.hs`, e.g. adapting the minimal configuration above:

```
import XMonad
import XMonad.Config.Xfce

main = xmonad xfceConfig
    { terminal    = "urxvt"
    , modMask     = mod4Mask
    }
```

Also add an entry to *Settings* > *Session and Startup* > *Application Autostart* that runs `xmonad --replace`.

Missing xmonad-i386-linux or xmonad-x86_64-linux

xmonad should automatically create the `xmonad-i386-linux` file (in `~/.xmonad/`). If this it not the case, grab a configuration file from the xmonad wiki or create your own. Put the `.hs` and all others files in `~/.xmonad/` and run this command from the folder:

```
xmonad --recompile
```

Now you should see the file.

注意：A reason you may get an error message saying that xmonad-x86_64-linux is missing is that xmonad-contrib包 is not installed.

Problems with Java applications

If you have problems, like Java application Windows not resizing, or menus immediately closing after you click, see Java#Gray window, applications not resizing with WM, menus immediately closing.

Empty space at the bottom of gvim or terminals

See Vim#Empty space at the bottom of gVim windows for a solution which makes the area match the background color.

For rxvt-unicode, you can use rxvt-unicode-patchedAUR.

You can also configure xmonad to respect size hints, but this will leave a gap instead. See on Xmonad.Layout.LayoutHints.

Chromium/Chrome will not go fullscreen

If Chrome fails to go fullscreen when `F11` is pressed, you can use the XMonad.Hooks.EwmhDesktops extension found in the xmonad-contrib包 package. Simply add the `import` statement to your `~/.xmonad/xmonad.hs`:

```
import XMonad.Hooks.EwmhDesktops
```

and then add `handleEventHook = fullscreenEventHook` to the appropriate place; for example:

```
...
        xmonad $ def
            { modMask            = mod4Mask
            , handleEventHook    = fullscreenEventHook
            }
...
```

After a recompile/restart of xmonad, Chromium should now respond to `F11` (fullscreen) as expected.

Multitouch / touchegg

Touchégg polls the window manager for the `_NET_CLIENT_LIST` (in order to fetch a list of windows it should listen for mouse events on.) By default, xmonad does not supply this property. To enable this, use the XMonad.Hooks.EwmhDesktops extension found in the xmonad-contrib包 package.

Keybinding issues with an azerty keyboard layout

Users with a keyboard with azerty layout can run into issues with certain keybindings. Using the XMonad.Config.Azerty module will solve this.

GNOME 3 mod4+p changes display configuration instead of launching dmenu

If you do not need the capability to switch the display-setup in the gnome-control-center, just execute

```
dconf write /org/gnome/settings-daemon/plugins/xrandr/active false
```

as your user, to disable the xrandr plugin which grabs Super+p.

Problems with focused border in VirtualBox

A known issue with Virtualbox (Ticket #6479) can cause problems with the focused window border. A solution can be found by installing a compositing manager like xcompmgr which overrides the incorrect behavior of vboxvideo.

Steam games (Half-Life, Left 4 Dead, …) and xmonad

There seems to be some trouble with xmonad and Source engine based games like Half-Life. If they do not start or get stuck with a black screen, a workaround is to start them in windowed mode. To do so, right click on the game in your Steam library and choose properties, click on launch options and enter :

```
-windowed
```

Another solution is to float the window of the game using the manage hook. For example, the following line can be used for Half-Life:

```
 className =? "hl_linux" --> doFloat
```

This can also be worked around by making xmonad pay attention to EWMH hints and including its fullscreen hook :

```
  main = xmonad $ ewmh def{ handleEventHook =
           handleEventHook def <+> fullscreenEventHook }
```

This has a few other effects and makes it behave more akin to fullscreen apps in other WMs.

LibreOffice - focus flicking between main window and dialog

The LibreOffice UI defaults to the gtk engine outside a desktop environment. This may cause problems with some xmonad configurations resulting in focus rapidly flicking between the LibreOffice main window and any open LibreOffice dialog window. Effectively locking the application. In this case the environment variable SAL_USE_VCLPLUGIN can be set to explicitly force LibreOffice to use another UI theme as outlined in LibreOffice#Theme For instance

```
  export SAL_USE_VCLPLUGIN=gen lowriter
```

to use the general (QT) UI.

Problems with finding shared libraries after update

The xmonad executable is located in `~/.xmonad/`. After upgrading xmonad, an old executable might persist and need in that case be removed for xmonad to compile a new executable. Alternatively use `xmonad --recompile`.

In the case that `xmonad --recompile` cannot find any modules at all (including `XMonad` itself), try regenerating the package database cache

```
sudo ghc-pkg recache
```

Broken/missing XMonad.Prompt and window decorations

XMonad by default uses the font `-misc-fixed-*-*-*-*-10-*-*-*-*-*-*-*`. If this font is missing those windows simply fail to render at all. Easiest fix is to install `(xorg-fonts-misc`.

See also

- xmonad - The official xmonad website
- xmonad.hs - Template xmonad.hs
- xmonad: a guided tour
- dzen - General purpose messaging and notification program
- dmenu - Dynamic X menu for the quick launching of programs
- 平铺式窗口管理器比较 - Arch wiki article providing an overview of mainstream tiling window managers
- Share your xmonad desktop!
- xmonad hacking thread
- xmonad-log-applet - An applet for the GNOME, MATE or xfce panel.

显示管理器

登录界面之前那一层就是显示管理器：接手登录、验证账号、把会话拉起来。本章收七种方案的安装与配置，建议先精读《SDDM》，它篇幅最长，还把故障排除单列；再读《Greetd》，欢迎界面（Greeter）的配置、提示和技巧集中于此，要自己定制登录流程时必看。其余按需查阅：《Ly》《Console TDM》篇幅很短，翻一遍即够，《XDM》《SLiM》《Plasma Login Manager》用到时再翻。

最大的误解是把安装当成终点：程序放进去了，能否正常拉起会话却取决于配置，这正是《SDDM》单列故障排除的原因。另一个误区是把一节的写法套到另一节，各方案的配置并不通用，认准自己装的那一个。

8.1　Ly

难度 入门 | **篇幅** 1.7 KB | **所属** 显示管理器

本节将讲到

- 安装
- 用法
- 配置
- 参见

Ly 是一个轻量级的 TUI (类似ncurses) 显示管理器，适用于 Linux 和 BSD，设计时考虑了可移植性（例如，它不需要 systemd 即可运行）。

安装

安装 ly包。对于 X server 支持，安装 xorg-xauth包。

依赖

为了获得完整的 *Ly* 体验，建议安装 brightnessctl包 和 durdrawAUR，用于管理屏幕亮度设置和设置自定义动画。

用法

要使 *Ly* 在启动时运行，需要执行两个步骤。首先，确保启用 `ly@tty*X*.service`，然后禁用 `getty@tty*X*.service`，其中 *X* 代表 1 到 6 的数字。

配置

系统配置文件在 `/etc/ly/config.ini`，用户配置文件在 `~/.config/ly`。

参见

- Ly on Github
- emptty Display Manager
- Lemurs Display Manager

8.2 Console TDM

难度 入门 | **篇幅** 2.1 KB | **所属** 显示管理器

本节将讲到

- 安装
- 配置

Console TDM是一个用纯bash脚本写出的xorg-xinit扩展组件. 它的灵感来自于CDM,这个软件可以替代诸如GDM这样的显示管理器的工作.

安装

AUR中已经收录了console-tdmAUR.

现在，您需要通过这个命令`systemctl disable`来保证没有其他的显示管理器在开机时被运行.

比如,你正在使用GDM显示管理器,你需要使他在开机时不被运行：

```
# systemctl disable gdm.service
```

or

```
# systemctl disable graphical.target
# systemctl enable multi-user.target
```

当安装Console TDM完成后,你需要修改你的`~/.bash_profile`文件,请加入以下文字

```
source /usr/bin/tdm
```

如果你使用zsh, 将以下的字段加入你的～/.zprofile中

```
bash /usr/bin/tdm
```

或

```
tdm
```

并编辑你的~/.xinitrc配置文件,加入以下字段并注释掉其他以`exec`开头的字段

```
exec tdm --xstart
```

配置

您需要将WM/DE启动器的链接复制到~/.tdm/sessions中, 然后将然后将非X程序连接到~/.tdm/extra. 为了方便,你也可以运行`tdmctl init`来达到同样效果.

tdmctl组件的功能非常像systemctl,并且它也是调整Console TDM设置的强有力工具.

您可以通过编辑~/.tdm/tdminit使Console TDM更具个性化.

8.3 Plasma Login Manager

难度 入门 | **篇幅** 3.5 KB | **所属** 显示管理器

本节将讲到

- 安装
- 配置

Plasma Login Manager 是 SDDM 的一个分支，拥有全新的提供欢迎界面，壁纸插件集成以及系统设置模块（KCM）的前端。

安装

安装 plasma-login-manager包，然后根据显示管理器#加载显示管理器的说明配置 Plasma Login 在系统引导时启动。

提示：如果从 SDDM 迁移过来，请先 禁用 sddm.service 。

迁移完成后，你还可以以 root 用户身份使用命令 userdel -r sddm 清理并删除不再使用的 sddm 用户。

配置

默认的配置文件在 /usr/lib/plasma-login/defaults.conf。要修改配置，请创建 /etc/plasmalogin.conf 或 /etc/plasmalogin.conf.d/。

一切东西都应该开箱即用，自从 Arch Linux 使用 systemd 后，Plasma Login 默认使用 systemd-logind 以进行会话管理。

自动登录

Plasma Login 通过它的配置文件来支持自动登录，例如：

```
/etc/plasmalogin.conf.d/autologin.conf
[Autologin]
User=john
Session=plasma.desktop
```

此配置使得在系统启动后自动以用户 john 开启一个 KDE Plasma 会话。X 的会话类型位于 /usr/share/xsessions/， wayland 的会话类型位于 /usr/share/wayland-sessions/。

要在登录 KDE Plasma 的同时锁定会话，请参阅 KDE#锁屏。

无密码登录

可以配置 Plasma Login 以允许在不需要密码的情况下登录到某些账户。与自动登录不同，用户仍需要选择要登录的账户，并且它与简单地将账户密码设置为空字符串不同，因为它只允许交互式用户登录（而不是，例如，通过 SSH 远程登录的用户）。

Plasma Login 通过 PAM 运行，因此您必须配置 PAM 的 Plasma Login 配置：

```
/etc/pam.d/plasmalogin
#%PAM-1.0
auth        sufficient  pam_succeed_if.so user ingroup nopasswdlogin
auth        include     system-login
...
```

也是为了能在没有密码的情况下解锁 KDE Plasma 锁屏，同样在/etc/pam.d/kde文件的顶部添加相同的行：

```
/etc/pam.d/kde
#%PAM-1.0
auth        sufficient  pam_succeed_if.so user ingroup nopasswdlogin
auth        include     system-login
...
```

然后，只有 `nopasswdlogin` 组的成员，才能在不输入密码的情况下交互式登录：

```
# groupadd -r nopasswdlogin
# gpasswd -a username nopasswdlogin
```

登录后自动解锁 Kwallet

详见 KDE Wallet#登录时自动解锁 Kwallet。

8.4 XDM

难度 入门 | **篇幅** 3.7 KB | **所属** 显示管理器

本节将讲到

- 安装
- 配置

摘自 XDM 手册页:

: *Xdm 能为本地和远程服务器提供一系列图形显示功能。xdm的设计满足图形显示的基本要求并遵循开放组织标准(XDMCPX Display Manager Control Protocol)，即X显示管理协议。Xdm提供的功能与init, getty等以文本登录为主的程序相似:提供登录会话，获取用户名和密码，并将授权给予登录用户并提供工作会话。*

XDM 提供了一个简单而又直观的图形登录界面。

安装

安装软件包 xorg-xdm包 然后启用 `xdm.service` 服务。

要使用 Arch Linux XDM 主题，可以安装软件包 xdm-archlinux包，然后不启用 `xdm.service`，而是启用 `xdm-archlinux.service`。

配置

定义会话

和 \[GDM\]\] 或 LightDM 等大部分显示管理器不同，XDM 不会从 `/usr/share/xsessions` 目录中的 .desktop 文件读取会话。XDM 没有会话菜单。XDM 会执行账号主目录下的 `.xsession` 文件。

例如要启动 xface，`~/.xsession` 应该是：

```
startxfce4
```

请确保 `.xsession` 文件可执行：

```
$ chmod 700 ~/.xsession
```

主题

详情请参考 xdm 手册，默认的配置文件位于 `/etc/X11/xdm/Xresources`,xdm-archlinux包 主题的配置文件位于 `/etc/X11/xdm/archlinux/Xresources`。

壁纸

可以使用 qiv包 设置 XDM 的壁纸:

- 安装 qiv包
- 创建一个文件夹用于存放图片。 (例如 `/root/backgrounds` 或者 `/usr/local/share/backgrounds`)
- 把图片放进文件夹
- 编辑 `/etc/X11/xdm/Xsetup_0`. 将 `xconsole` 修改为：

```
 /usr/bin/qiv -zr /root/backgrounds/*
```

字体

编辑 `/etc/X11/xdm/Xresources`. 添加/替换 下面字段:

```
xlogingreetFont:  -adobe-helvetica-bold-o-normal--20------iso8859-1
xloginfont:       -adobe-helvetica-medium-r-normal--14------iso8859-1
xloginpromptFont: -adobe-helvetica-bold-r-normal--14------iso8859-1
xloginfailFont:   -adobe-helvetica-bold-r-normal--14------iso8859-1
```

登录对话框位置

```
xlogin*frameWidth: 1
xlogin*innerFramesWidth: 1
xlogin*logoPadding: 0
xlogin*geometry:    300x175-0-0
```

删除徽标

注释掉以下字段:

```
#xlogin*logoFileName: /usr/X11R6/lib/X11/xdm/pixmaps/xorg.xpm
#xlogin*logoFileName: /usr/X11R6/lib/X11/xdm/pixmaps/xorg-bw.xpm
```

多 X 会话和登录

启用 XDMCP 后，可以在同一个机器上运行多个 X 会话：

```
# X -query ip_xdmcp_server :2
```

这将启动第二个会话，在窗口中需要 xorg-server-xephyr包

```
# Xephyr -query this_machine_ip :2
```

无密码登录

要启用 XDM 无密码登录，将下面内容加入 `/etc/X11/xdm/Xresources`:

```
xlogin*allowNullPasswd: true
```

8.5 SLiM

难度 进阶 | **篇幅** 11.5 KB | **所属** 显示管理器

本节将讲到

- 安装
- 配置
- 更多的Slim参数
- 更多

**这篇文章或章节的翻译质量不佳。**

原因：Last content update was in 2012（在 Talk:SLiM# 中讨论）

警告：SliM 显示管理器已经停止开发，SliM 的官方网站已经关闭，此外，SliM 与 systemd 也不是特别的兼容，请考虑换一个 显示管理器，或直接使用 startx。

SLiM 是**S**imple **L**ogin **M**anager（简单登录管理器）的缩写。SLiM是简单、轻量级和容易配置的，相对较易在低端和高端的系统中使用。对于那些希望寻找一个不依赖于GNOME或者KDE，可以在Xfce、Openbox、Fluxbox等环境下使用的登录管理器的人来说，SLiM也是非常合适的。

安装

安装位于官方软件仓库的软件包 slim包。

配置

启用SLiM

注意：slim包已经不提供 ConsoleKit 支持，而是使用 systemd-logind，所以系统需要通过 systemd 启动。

启用**slim** 服务. 启用 systemd 之后，已经无法使用 `inittab` 启动 slim。

单用户环境

要将SLiM配置为加载某个特定的环境，只需编辑~/.xinitrc 如下：

```
#!/bin/sh

#
# ~/.xinitrc
#
# Executed by startx (run your window manager from here)
#

exec [session-command]
```

将***\[session-command\]***替换为适当的会话命令。例如：

```
exec awesome
exec dwm
exec startfluxbox
exec fvwm2
exec gnome-session
exec openbox-session
exec startkde
exec startlxde
exec startxfce4
exec enlightenment_start
```

如果你的桌面环境不在上述列表中，请参考你的软件文档。

Remember to make `~/.xinitrc` executable:

```
chmod +x ~/.xinitrc
```

注意：slim包 no longer has ConsoleKit support, but relies on systemd-logind, and the system being booted with systemd.

自动登录

想要使SLiM自动以特定用户身份登录（无须输入密码），需要修改 `/etc/slim.conf` 中的以下几行。

```
# default_user        simone
```

取消该行的注释，然后将“simone”改为需要自动登录的用户名。

```
# auto_login          no
```

取消该行的注释，然后将‘no’改为‘yes’。自动登录功能就被启用了。

Zsh

默认的登录命令不能正确初始化你的环境 \[source\]。 将 login_cmd 一行改为：

```
#login_cmd           exec /bin/sh - ~/.xinitrc %session
login_cmd           exec /bin/zsh -l ~/.xinitrc %session
```

多桌面环境

如果你希望可以加载多个不同的桌面环境，SLiM可以设置为登录到你指定的任何一个桌面环境。

在你的/etc/X11/xinit/xinitrc文件中加入一段类似下面内容的case语句，并且编辑/etc/slim.conf中的sessions变量。 你可以在登录界面上按F1选择会话。请注意这个特性仍处于实验阶段。

```
# The following variable defines the session which is started if the user doesn't
explicitly select a session

DEFAULT_SESSION=twm

case $1 in
kde)
        exec startkde
        ;;
xfce4)
        exec startxfce4
        ;;
icewm)
        icewmbg &
        icewmtray &
        exec icewm
        ;;
wmaker)
        exec wmaker
        ;;
blackbox)
        exec blackbox
        ;;
*)
        exec $DEFAULT_SESSION
        ;;
esac
```

范例源码： http://svn.berlios.de/svnroot/repos/slim/trunk/xinitrc.sample

SLiM的文档： http://slim.berlios.de/manual.php

主题

安装slim-themes包软件包：

```
# pacman -S slim-themes archlinux-themes-slim
```

软件包 archlinux-themes-slim包 包含了多个不同主题。可以在 `/usr/share/slim/themes` 中查看。在to see the themes available. Enter the theme name on the `current_theme` line in `/etc/slim.conf`:

编辑`/etc/slim.conf`中的`current_theme`那行，将"default"改为你想要的主题名：

```
#current_theme       default
current_theme       archlinux-simplyblack
```

要预览一个主题，可以运行

```
$ slim -p /usr/share/slim/themes/<theme name>
```

要退出，只需在登录行输入 "exit" 并回车。

多屏幕设置

定制 `/usr/share/slim/themes/<your-theme>/slim.theme` 中的主题，修改百分比，就可以在多屏幕环境使用。假设宽为 450 高为 250 :

```
input_panel_x            50%
input_panel_y            50%
```

修改为:

```
# 将 "archlinux-simplyblack" 面板放在 1440x900 屏幕的中间
input_panel_x            495
input_panel_y            325
# 将 "archlinux-retro" 面板放在 1680x1050 屏幕的中间
input_panel_x            615
input_panel_y            400
```

如果主题有背景图片，可以设置背景样式('stretch', 'tile', 'center' 或 'color')。更多信息请访问 slim 主题的标准文档。

增强Slim功能

slim登录脚本主要靠你自己写，这对很多人来说比较困难，比如自动加载一些配置文件如~/.xprofile, ~/.xmodmap等。而且slim还无法记住上次登录使用的桌面环境。

AUR里面有个包slim-plus可以解决这些问题。提供了一个Xsession脚本增强slim功能，加载配置文件，记住上次会话等。非常适合xfce等小型桌面环境。而且包含很多实用补丁。如果你想自己编写启动脚本，那里带的Xsession脚本可以做参考。

安装完成后，如果以前使用的就是slim，那么需要编辑slim来使用/etc/X11/Xsession来加载会话，找到如下行：

```
login_cmd           exec /bin/bash -login ~/.xinitrc %session
```

替换为

```
login_cmd           exec /bin/bash -login /etc/X11/Xsession %session
```

只需要修改/etc/slim.conf的Sessions行，添加你需要的会话，slim启动后按F1选择就可以了。会话的名称可以在/usr/share/xsessions/下的*.desktop处得到。

如果你安装的桌面环境没有提供.desktop文件，就需要自行编辑/etc/X11/Xsession文件尾部，不过脚本本身带有提供很多窗口管理器的启动脚本。

Xsession脚本会自动加载常见配置文件。使用ck-launch-session加载桌面，解决一些挂载，无法关机等问题。并且使用~/.dmrc记录上次会话。由于slim功能有限，暂时无法选择语言。你自己可以编辑~/.dmrc来设置语言为中文。

```
Language=zh_CN.UTF-8
```

设置和Splashy一起工作时正常关机

如果你同时使用splashy和slim，有时你无法在gnome，xfce，lxde，等桌面环境中正常关机或者重启。 那么请检查你的/etc/slim.conf 和 /etc/splash.conf, 设置需要为 DEFAULT_TTY=7 或者 xserver_arguments vt07.

Slim的登录信息

By default, Slim fails to log logins to utmp and wtmp which causes who, last, etc.. to misreport login information. 修正这些你需要编辑你的slim.conf:

```
sessionstart_cmd    /usr/bin/sessreg -a -l $DISPLAY %user
sessionstop_cmd     /usr/bin/sessreg -d -l $DISPLAY %user
```

设置默认DPI

如果你设置DPI是通过在/etc/X11/Xinit/Xserverrc中添加参数-dpi 96，那么设置在slim中是不起作用的。你需要在slim.conf中这样设置：

```
xserver_arguments   -nolisten tcp vt07
```

更改为：

```
xserver_arguments   -nolisten tcp vt07 -dpi 96
```

使用随机主题

在slim.conf中的current_theme行，添加多个主题，使用,分隔就可以使用随机主题了。

更多的Slim参数

这个列表显示了所有的参数变量以及其默认值。

注意：welcome_msg 允许2个变量**%host** 与 **%domain** sessionstart_cmd 读取 **%user** *(execd right before login_cmd)* 并且它也会读取 sessionstop_cmd login_cmd allows **%session** and **%theme**

Option Name	Default Value
default_path	`/bin:/usr/bin:/usr/local/bin`
default_xserver	`/usr/bin/X`
xserver_arguments	`vt07 -auth /var/run/slim.auth`
numlock	
daemon	`yes`
xauth_path	`/usr/bin/xauth`
login_cmd	`exec /bin/bash -login ~/.xinitrc %session`
halt_cmd	`/sbin/shutdown -h now`
reboot_cmd	`/sbin/shutdown -r now`
suspend_cmd	
sessionstart_cmd	
sessionstop_cmd	
console_cmd	`/usr/bin/xterm -C -fg white -bg black +sb -g %dx%d+%d+%d -fn %dx%d -T`
screenshot_cmd	`import -window root /slim.png`
welcome_msg	`Welcome to %host`
session_msg	`Session:`
default_user	

Option Name	Default Value
focus_password	no
auto_login	no
current_theme	default
lockfile	/var/run/slim.lock
logfile	/var/log/slim.log
authfile	/var/run/slim.auth
shutdown_msg	The system is halting...
reboot_msg	The system is rebooting...
sessions	wmaker,blackbox,icewm
sessiondir	
hidecursor	false
input_panel_x	50%
input_panel_y	40%
input_name_x	200
input_name_y	154
input_pass_x	-1
input_pass_y	-1
input_font	Verdana:size=11
input_color	#000000
input_cursor_height	20
input_maxlength_name	20
input_maxlength_passwd	20
input_shadow_xoffset	0
input_shadow_yoffset	0
input_shadow_color	#FFFFFF
welcome_font	Verdana:size=14
welcome_color	#FFFFFF
welcome_x	-1
welcome_y	-1
welcome_shadow_xoffset	0
welcome_shadow_yoffset	0
welcome_shadow_color	#FFFFFF
intro_msg	
intro_font	Verdana:size=14
intro_color	#FFFFFF
intro_x	-1
intro_y	-1
background_style	stretch
background_color	#CCCCCC
username_font	Verdana:size=12
username_color	#FFFFFF
username_x	-1
username_y	-1
username_msg	Please enter your username
username_shadow_xoffset	0
username_shadow_yoffset	0
username_shadow_color	#FFFFFF
password_x	-1
password_y	-1
password_msg	Please enter your password
msg_color	#FFFFFF
msg_font	Verdana:size=16:bold
msg_x	40
msg_y	40
msg_shadow_xoffset	0
msg_shadow_yoffset	0
msg_shadow_color	#FFFFFF
session_color	#FFFFFF
session_font	Verdana:size=16:bold
session_x	50%
session_y	90%

Option Name	Default Value
session_shadow_xoffset	0
session_shadow_yoffset	0
session_shadow_color	#FFFFFF

更多

- Display manager
- SLiM 官方网站

8.6 Greetd

难度 进阶 | **篇幅** 14.3 KB | **所属** 显示管理器

本节将讲到

- 安装
- 启动 greetd
- 欢迎界面（Greeter）配置
- 提示和技巧
- 参见

翻译状态：

- 本文（或部分内容）译自 Greetd，最近一次同步于 2025-02-24，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

greetd is a minimal, agnostic and flexible login manager daemon which does not make assumptions about what the user wants to launch, should it be console-based or graphical. Any script or program which can be started from the console may be launched by greetd, which makes it particularly suitable for Wayland compositors. It can also launch a greeter to start user sessions, like any other display manager.

安装

安装 greetd包 或 greetd-gitAUR。

默认配置文件位于 `/etc/greetd/config.toml`。 PAM 相关的配置文件位于 `/etc/pam.d/greetd`。

欢迎界面（Greeters）

Greetd 内置的默认欢迎界面是 greetd-agreety，这是一个最小化的实现。您可以考虑换成其他的欢迎界面：

- **greetd-agreety** — 默认的，基于文本的欢迎界面，类似于 agetty

: https://git.sr.ht/~kennylevinsen/greetd || greetd-agreety包

- **cosmic-greeter** — 一个基于 libcosmic 的欢迎界面

: https://github.com/pop-os/cosmic-greeter || cosmic-greeter包

- **dlm** — 一个基于 fbdev 的欢迎界面

: https://git.sr.ht/~kennylevinsen/dlm || greetd-dlm-gitAUR

- **ddlm** — 一个基于 fbdev 的欢迎界面。`dlm` 的改进/扩展版本

: https://github.com/deathowl/ddlm || greetd-ddlm-gitAUR

- **gtkgreet** — 一个基于 GTK 的欢迎界面

: https://git.sr.ht/~kennylevinsen/gtkgreet || greetd-gtkgreet包，greetd-gtkgreet-gitAUR

- **ReGreet** — 一个基于 GTK 的欢迎界面，支持不同的个性化选项，仅支持 Wayland

: https://github.com/rharish101/ReGreet || greetd-regreet包，greetd-regreet-gitAUR

- **wlgreet** — 一个使用 Wayland 的欢迎界面

: https://git.sr.ht/~kennylevinsen/wlgreet || greetd-wlgreetAUR，greetd-wlgreet-gitAUR

- **tuigreet** — 一个使用控制台 UI 的欢迎界面

: https://github.com/apognu/tuigreet || greetd-tuigreet包

- **qtgreet** — 一个基于 Qt 的欢迎界面

: https://gitlab.com/marcusbritanicus/QtGreet || greetd-qtgreetAUR

- **nwg-hello** — 一个用 Python 写的基于 GTK3 的欢迎界面

: https://github.com/nwg-piotr/nwg-hello || nwg-hello包

启动 greetd

启用 `greetd.service`，以便在启动时启动 greetd。

参见显示管理器#加载显示管理器.

欢迎界面（Greeter）配置

在 `/etc/greetd/config.toml` 的 `default_session` 节中的 `command` 选项配置 greetd 运行的欢迎界面。若没有做出更改，将使用内置的 `agreety` 欢迎界面。另请参阅 #agreety。

欢迎界面默认由 `greeter` 用户运行，您可在配置文件的 `default_session` 节中的 `user` 选项更改用户。将以下片段中的*目标用户*改为您想使用的用户：

```
...
[default_session]
user = "目标用户"
...
```

确保该用户具有 `/etc/greetd` 目录的所有权。

agreety

这是默认的欢迎界面。以下配置文件片段会让 greetd 启动 agreety：

```
...
[default_session]
command = "agreety --cmd $SHELL"
...
```

一旦用户登录，agreety 可以执行任意命令。例如，要启动 Sway，将上例中的 `$SHELL` 改为 `sway`。

gtkgreet

gtkgreet 需要一个混成器才能运行。要获取完整体验需要一个具有 `wlr-layer-shell-unstable` 支持的混成器，当然其他的也能用。例如，推荐使用 sway包，但是其他的一些如 cage包 也能用，下文提供了此二者的示例。

为指定 gtkgreet 可启动的登录环境，请在 `/etc/greetd/environments` 列出它们。 例如：

```
sway
bash
```

您也可以为 gtkgreet 添加 `-c *mycommand*` 参数，将 *mycommand* 替换为所需程序（例如，`gtkgreet -c bash` 或 `gtkgreet -c sway`）。根据所需在以下两个混成器示例中执行该操作。

使用 cage

安装 cage包 并按照以下片段配置 `command` 选项：

```
...
[default_session]
command = "cage gtkgreet"
...
```

使用 sway

安装 sway包。请注意，使用 Sway 时，一旦用户登录，其必须被停止，为此您需要创建一个特定的配置文件。例如位于 `/etc/greetd/sway-config`，具有以下内容：

```
# `-l` activates layer-shell mode. Notice that `swaymsg exit` will run after gtkgreet.
exec "gtkgreet -l; swaymsg exit"

bindsym Mod4+shift+e exec swaynag \
-t warning \
-m 'What do you want to do?' \
-b 'Poweroff' 'systemctl poweroff' \
-b 'Reboot' 'systemctl reboot'

include /etc/sway/config.d/*
```

接下来，greetd 必须被设定为以上述配置文件运行 Sway。按照以下片段设置 `command` 选项：

```
...
[default_session]
command = "sway --config /etc/greetd/sway-config"
...
```

ReGreet

ReGreet 和 gtkgreet 一样需要混成器，和 gtkgreet 一样可以使用 Cage 和 Sway。只需将上述配置中的 *gtkgreet* 命令换成 *regreet*。对 sway 配置应当如此：

```
# Notice that `swaymsg exit` will run after ReGreet.
exec "regreet; swaymsg exit"

bindsym Mod4+shift+e exec swaynag \
-t warning \
-m 'What do you want to do?' \
-b 'Poweroff' 'systemctl poweroff' \
-b 'Reboot' 'systemctl reboot'

include /etc/sway/config.d/*
```

ReGreet 从 `/usr/share/xsession`（X11 会话）和 `/usr/share/wayland-sessions`（Wayland 会话）选取可用的会话，因此无需在`/etc/greetd/environments` 中列出会话。

ReGreet can be configured through a TOML file in `/etc/greetd/regreet.toml`. A sample file is provided in `/usr/share/doc/greetd-regreet/regreet.sample.toml` with all available options. Copy this to `/etc/greetd/regreet.toml` and make the changes you want, commenting out or deleting the lines you do not need. Any invalid options are ignored.

wlgreet

Wlgreet 需要一个具有 `wlr-layer-shell-unstable` 支持的混成器。配置类似于上文为 gtkgreet 设定 Sway，如下：

```
/etc/greetd/sway-config
exec "wlgreet --command sway; swaymsg exit"

bindsym Mod4+shift+e exec swaynag \
-t warning \
-m '君欲何为？' \
-b '关机' 'systemctl poweroff' \
-b '重启' 'systemctl reboot'

include /etc/sway/config.d/*
```

提示：亦如上文，该配置文件可以位于任意位置，只要其被 Greetd 用于启动 Sway。

tuigreet

tuigreet does not require any special setup, just set the `command` option as follows:

```
...
[default_session]
command = "tuigreet --cmd sway"
...
```

`tuigreet --help` will display customization options.

ddlm

ddlm does not require any special setup, just set the `command` option as follows:

```
...
[default_session]
command = "ddlm --target sway"
...
```

qtgreet

In order to use qtgreet, you need a WLR based compositor (e.g. wayfireAUR, sway包).

Using Wayfire

Install wayfireAUR and set the `command` option as follows:

```
...
[default_session]
command = "wayfire --config /etc/qtgreet/wayfire.ini"
...
```

The Wayfire configuration file referred to is included with qtgreet.

Using Hyprland

Set the `command` option as follows:

```
...
[default_session]
command = "Hyprland --config /etc/greetd/hyprland.conf"
...
```

Then create `/etc/greetd/hyprland.conf` as:

```
/etc/greetd/hyprland.conf
exec-once = qtgreet; hyprctl dispatch exit
```

nwg-hello

In order to use nwg-hello, you either need sway包 or hyprland包.

Using Sway

Install sway包 and set the `command` option as follows:

```
...
[default_session]
command = "sway -c /etc/nwg-hello/sway-config"
...
```

The Sway configuration file referred to is included with nwg-hello.

Using Hyprland

Install hyprland包 and set the `command` option as follows:

```
...
[default_session]
command = "Hyprland -c /etc/nwg-hello/hyprland.conf"
...
```

The Hyprland configuration file referred to is included with nwg-hello.

提示和技巧

自动登录

要让某用户自动登录，请在 `/etc/greetd/config.toml` 中定义 `initial_session` 节：

```
...
[initial_session]
command = "sway"
user = "myuser"
...
```

`command` 选项可以是任意可执行文件，在上例中，系统启动后会以 `*myuser*` 用户运行 Sway。

若您不想用 *greetd* 且要使用自动登录，请参阅autologin。

运行本地程序

将您的 PATH 添加到 `~/.profile`，否则 greetd 唤起的桌面环境将无法运行本地程序。Greetd 不会访问 `.bashrc` 和 `.zshrc`，因此不要在这两个文件中设置 PATH。

```
~/.profile
export PATH="$HOME/.local/bin:$PATH"
```

设定环境

Greetd 默认不会设定环境变量如 XDG_SESSION_TYPE 和 XDG_CURRENT_DESKTOP 除非欢迎界面基于您设置的会话设定了环境变量（例如 TUI 会基于会话文件选择的区域设定会话类型）。一种方案是在执行实际命令前用一个脚本设定需要的环境变量，例如要启动 sway：

```
/usr/local/bin/start-sway
#!/bin/sh
export XDG_SESSION_TYPE=wayland
export XDG_SESSION_DESKTOP=sway
export XDG_CURRENT_DESKTOP=sway

# Wayland stuff
export QT_QPA_PLATFORM=wayland
export SDL_VIDEODRIVER=wayland
export _JAVA_AWT_WM_NONREPARENTING=1

exec sway "$@"
```

然后让欢迎界面运行该脚本，例如使用 `gtkgreet` 可以如此：

```
/etc/greetd/config.toml
```

```
...
[default_session]
command = "gtkgreet -c /usr/local/bin/start-sway"
...
```

或将 `start-sway` 置入 `/etc/greetd/environments`。

参阅 How to Set XDG_SESSION_TYPE=wayland。

设定 logind 会话类型

注意：This is necessary for running GNOME on Wayland properly.

The logind session type is set by the XDG_SESSION_TYPE environment variable. However, it must be set **before** the PAM session is opened. Because of this, setting the variable through `~/.profile` or a wrapper script will not work (both happen after session open).

The correct way to achieve this is through the environment variables sent by greeters (these are set before session open). So if your greeter supports it, just make it send the appropriate `XDG_SESSION_TYPE=xxx`.

If your greeter does not support this, it is also possible to use pam_env under the auth group. The drawback is that all the sessions spawned by greetd will use that session type, which may or may not be problematic depending on your use case.

Here is how one could use the pam_env method to have a Wayland session:

```
/etc/greetd/config.toml
[general]
service = "greetd-spawn"
/etc/greetd/greetd-spawn.pam_env.conf
XDG_SESSION_TYPE DEFAULT=wayland OVERRIDE=wayland
/etc/pam.d/greetd-spawn
auth        include      greetd
auth        required     pam_env.so conffile=/etc/greetd/greetd-spawn.pam_env.conf
account     include      greetd
session     include      greetd
```

缺失鼠标指针

若您在一些类似于 `wayfire` 的混成器上使用 `qtgreet`，您通常需要设置某些环境变量以使鼠标指针正常工作，例如 `WLR_NO_HARDWARE_CURSORS=1`。一种方案是创建一个可执行脚本并在 `/etc/greetd/config.toml` 中调用它：

```
/usr/local/bin/greetd-startup.sh
#!/bin/sh
export WLR_NO_HARDWARE_CURSORS=1
exec wayfire --config /etc/qtgreet/wayfire.ini
/etc/greetd/config.toml
...
[default_session]
command = "/usr/local/bin/greetd-startup.sh"
...
```

参见

- greetd 项目页面

- greetd 源代码
- greetd 维基

8.7 SDDM

难度 高级 | **篇幅** 23.9 KB | **所属** 显示管理器

本节将讲到

- 安装
- 配置
- 故障排除

Simple Desktop Display Manager (SDDM) 是一个显示管理器。它是 LXQt 桌面环境推荐的显示管理器。

Wikipedia:Simple Desktop Display Manager 介绍：

: Simple Desktop Display Manager (SDDM) 是用于 X11 和 Wayland 视窗系统的显示管理器（图形登录程序与会话管理器）。SDDM 使用 C++11 从头编写并支持通过 QML 改变主题。

注意：截止至 SDDM 0.20 版本，Wayland 会话能够被列出，并可从 SDDM 启动，但 SDDM 登录界面本身默认仍在 X11 模式下运行，不过可以启用试验性的 Wayland 登录界面。

安装

安装 sddm包。可选安装 sddm-kcm包 或 qt5-declarative包 以分别启用对 KConfig Module 或 Qt5 的支持。

然后根据显示管理器#加载显示管理器的说明配置 SDDM 在系统引导时启动。

配置

SDDM 的默认配置文件为 `/usr/lib/sddm/sddm.conf.d/default.conf`。要修改配置，请在 `/etc/sddm.conf.d/` 目录下创建配置文件。详见 sddm.conf(5) 以获得所有配置选项。

sddm-kcm包 软件包 (包含在 plasma包组 用户组) 提供了一个图形用户界面以在 Plasma 系统设置中配置 SDDM。AUR 中也有基于 Qt 的配置软件 sddm-conf-gitAUR。

一切东西都应该开箱即用，自从 Arch Linux 使用 systemd 后，SDDM 默认使用 `systemd-logind` 以进行会话管理。

自动登录

SDDM 通过它的配置文件来支持自动登录，例如：

```
/etc/sddm.conf.d/autologin.conf
[Autologin]
User=john
Session=plasma
```

此配置使得在系统启动后自动以用户 `john` 开启一个 KDE Plasma 会话。X 的会话类型位于 `/usr/share/xsessions/`， wayland 的会话类型位于 `/usr/share/wayland-sessions/`。

要在登录 KDE Plasma 的同时锁定会话，请参阅 KDE#锁屏。

无密码登录

可以配置 SDDM 以允许在不需要密码的情况下登录到某些账户。与自动登录不同，用户仍需要选择要登录的账户，并且它与简单地将账户密码设置为空字符串不同，因为它只允许交互式用户登录（而不是，例如，通过 SSH 远程登录的用户）。

SDDM 通过 PAM 运行，因此您必须配置 PAM 的 SDDM 配置：

```
/etc/pam.d/sddm
#%PAM-1.0
auth        sufficient  pam_succeed_if.so user ingroup nopasswdlogin
auth        include     system-login
...
```

也是为了能在没有密码的情况下解锁 KDE Plasma 锁屏，同样在`/etc/pam.d/kde`文件的顶部添加相同的行：

```
/etc/pam.d/kde
#%PAM-1.0
auth        sufficient  pam_succeed_if.so user ingroup nopasswdlogin
auth        include     system-login
...
```

然后，只有`nopasswdlogin`组的成员，才能在不输入密码的情况下交互式登录：

```
# groupadd -r nopasswdlogin
# gpasswd -a username nopasswdlogin
```

登录后自动解锁 KDE Wallet

详见 KDE Wallet#在登录时自动解锁 Kwallet。

主题设置

在 `[Theme]` 小节更改主题设置。如果您使用 Plasma 的系统设置，主题可能会显示预览。

设置 `breeze` 以获得 Plasma 默认主题。

AUR 有一些可用的主题，例如 archlinux-themes-sddmAUR。

当前主题

通过 `Current` 的值设置当前主题，例如 `Current=archlinux-simplyblack`。

创建主题

默认 SDDM 主题目录为 `/usr/share/sddm/themes/`。你可以添加你的自制主题到该目录下一个单独的子目录中。注意 SDDM 要求这些子目录的名字要与主题的名字一致。可以通过研究已安装的文件来更改或创建属于你的主题。

自定义主题

要覆盖 `theme.conf` 配置文件中的设置，在相同目录下创建一个自定义的 `theme.conf.user` 文件。例如，要更改主题的背景：

```
/usr/share/sddm/themes/name/theme.conf.user
[General]
background=/path/to/background.png
```

测试（预览）主题

如果需要，你可以预览一个 SDDM 主题。如果你想知道一个主题看起来怎么样，或是想要编辑一个主题后在不必登出的情况下观察改动的效果，那么这将会非常有用。你可以运行下面的命令：

```
$ sddm-greeter-qt6 --test-mode --theme /usr/share/sddm/themes/breeze
```

这应该为所有已连接的显示器打开一个新窗口以显示主题的预览。

注意：这只是一个预览。在这个模式下，一些动作如关机、挂起或登录将不会执行。

鼠标光标

要设置鼠标光标的主题，将 `CursorTheme` 设置成您喜欢的光标主题。

合法的 Plasma 鼠标光标主题有 `breeze_cursors`、`Breeze_Light` 和 `breeze-dark`。

如果 SDDM 在 KDE 设置下默认使用 Adwaita 主题（GNOME 鼠标光标），则应手动将 `/usr/share/icons/default/index.theme` 中的 `Inherits` 变量设置为所需的值。

提示：在 Plasma 上，运行 `grep -i cursortheme ~/.config/kcminputrc` 可以查看当前活动鼠标光标主题的名称。

用户图标（头像）

SDDM 对于每个用户从相应的 `~/.face.icon` 目录下读取 PNG 图片格式的用户图标（即“头像”），或者从 SDDM 配置文件中由 `FacesDir` 为所有用户定义的共有位置。配置设置可以通过修改 `/etc/sddm.conf` 文件被直接替换，更好的方法是创建

一个位于 `/etc/sddm.conf.d/` 下的文件来修改，例如 `/etc/sddm.conf.d/avatar.conf`。

要使用 `FacesDir` 选项来确定头像位置，即在配置文件中 `FacesDir` 所确定的位置为每一个用户放置一个 PNG 图片，命名如 `*username*.face.icon`。`FacesDir` 默认的路径为 `/usr/share/sddm/faces/`。你可以更改默认 `FacesDir` 目录以合乎你的要求。下面是一个例子：

```
/etc/sddm.conf.d/avatar.conf
[Theme]
FacesDir=/var/lib/AccountsService/icons/
```

另一个选项是放置一个名为 `.face.icon` 的 PNG 图片到你的家目录下。在这种情况下，您不用对任何 SDDM 配置文件进行更改。不过，您仍需确定 `sddm` 用户可以读取这些 PNG 图片作为用户图标。

注意：在许多 KDE 的版本中，用户图标图像文件是 `~/.face` 而 `~/.face.icon` 是链接到图像文件的符号链接。如果用户的图标是符号链接，你需要为目标文件设置恰当的文件权限。

为了设置合适权限运行：

```
$ setfacl -m u:sddm:x ~/
$ setfacl -m u:sddm:r ~/.face.icon
```

你可以通过运行下列命令检查权限：

```
$ getfacl ~/
$ getfacl ~/.face.icon
```

详见 SDDM README: No User Icon。

数字锁

如果你想强制启用数字锁，在 `[General]` 小节设置 `Numlock=on`。

如果 SDDM 在 Wayland 下运行，当前 NumLock 设置不起作用。您可能需要更改 KWin 设置以启用它，请参阅此 issue

旋转显示

详见 Xrandr#Configuration。

DPI 设置

有时位于“显示管理器”级别设置正确的显示器 DPI 是很有用的。 你需要在 `ServerArguments` 字符串的末尾加上参数 `-dpi *your_dpi*` 例如：

```
/etc/sddm.conf.d/dpi.conf
[X11]
ServerArguments=-nolisten tcp -dpi 94
```

启用 HiDPI

注意：自 SDDM 0.20.0 起，默认启用了 HiDPI 支持，以下步骤不再必要。

创建 HiDPI 配置文件如下：

```
/etc/sddm.conf.d/hidpi.conf
[Wayland]
EnableHiDPI=true

[X11]
EnableHiDPI=true
```

当使用 Wayland 时，HiDPI 缩放取决于所使用的问候程序。 例如，当使用基于 Qt 的问候程序（如 Breeze）时，请添加以下配置：

```
[General]
GreeterEnvironment=QT_SCREEN_SCALE_FACTORS=2,QT_FONT_DPI=192
```

启用虚拟键盘

安装 qt6-virtualkeyboard包 或你希望的其他虚拟键盘，创建如下文件：

```
/etc/sddm.conf.d/virtualkbd.conf
[General]
InputMethod=您使用的虚拟键盘名称
```

SDDM 在登录屏幕的左下角会显示一个虚拟键盘图标。

使用指纹识别器

提示：自 Plasma 6 发布以来，KDE 的锁屏应该已经支持使用指纹解锁，`kscreenlocker` 已经包含了所需的 `/etc/pam.d/kde-fingerprint`。

然而，该配置不包括登录，只包括解锁现有会话。

因为 KWallet 在登录时需要基于密码的认证（见下文），您可能仍然希望跳过 SDDM 的指纹设置。

注意：在改变设置前要确定你的指纹已经注册完成。指纹支持目前并不是完全工作正常，并且看起来在使用这种方法时仅使用密码登录将不再有效。

SDDM 使用 fprint 以使用指纹识别。在安装了 fprint 和添加指纹签名后，在 `/etc/pam.d/sddm` 的顶部添加：

```
/etc/pam.d/sddm
auth                    sufficient      pam_fprintd.so
```

为了同时使用密码或指纹解锁，您可以在文件顶部添加以下内容：

```
/etc/pam.d/sddm
auth                    [success=1 new_authtok_reqd=1 default=ignore]    pam_unix.so
try_first_pass likeauth nullok
auth                    sufficient      pam_fprintd.so
```

请注意，KWallet不能使用指纹读取器解锁（见 KDE Wallet#在登录时自动解锁 Kwallet），但第一行确保了使用密码登录将自动解锁 KWallet。

如果您现在在空密码字段中按回车，指纹读取器应该开始工作。

无根模式

传统上，Xorg 显示服务器默认以 root 权限运行。这种有根模式允许 Xorg 无限制地访问系统资源，这在直接硬件访问和管理常见的环境中是必要的。然而，随着现代计算

环境中对安全性的日益重视，人们已经努力转向无根模式，这也是 Wayland 默认以无根模式运行的原因。

自 sddm包 0.20.0 以来，支持启动非特权 X11（和 Wayland）会话。

要启用无根模式：在 `/etc/sddm.conf.d/` 下创建一个新的配置文件，给它一个有意义的名字，并添加以下内容，如果需要，将 `x11-user` 替换为 `wayland`。

```
/etc/sddm.conf.d/rootless-x11.conf
[General]
DisplayServer=x11-user
```

要确认您是否正在以无根模式运行，请检查哪个用户拥有compositor 进程的所有权（例如 `kwin_wayland`)

Wayland

注意：

- SDDM 将其对 Wayland 的支持视为实验性的。
- 默认情况下 `DisplayServer=wayland` 尝试使用 weston 混成器，即便它没有安装，要使用不同的 Wayland 混成器，请按照 #KDE Plasma / KWin 中的描述将程序和相关参数输入到 `CompositorCommand` 选项中，也请参考 sddm.conf(5) 了解更多。

KDE Plasma / KWin

将以下行添加到您的配置文件中，将 Wayland 混成器设置为 KWin 并启用 wlr_layer_shell Wayland 协议扩展。使用 Qt5 和 Qt6 的 SDDM 主题分别需要 layer-shell-qt包 和 layer-shell-qt5AUR。

```
/etc/sddm.conf.d/10-wayland.conf
[General]
DisplayServer=wayland
GreeterEnvironment=QT_WAYLAND_SHELL_INTEGRATION=layer-shell

[Wayland]
CompositorCommand=kwin_wayland --drm --no-lockscreen --no-global-shortcuts --locale1
```

虚拟键盘

要启用虚拟键盘支持（例如使用 qt6-virtualkeyboard包 或 plasma-keyboard包），请在 `kwin_wayland` 命令中追加 `--inputmethod` 标志和适当的虚拟键盘，如下所示。不要在 `General` 部分设置 `InputMethod` 选项，因为这将导致虚拟键盘不再显示。

```
/etc/sddm.conf.d/10-wayland.conf
[Wayland]
CompositorCommand=kwin_wayland --drm --no-lockscreen --no-global-shortcuts --locale1 --
inputmethod plasma-keyboard
```

匹配 Plasma 显示配置

在 Plasma Wayland 会话中对显示配置所做的更改（例如监视器布局、分辨率等）将不会保留到 SDDM。要使它们保留，请打开 Plasma 的*系统设置*，导航到**颜色和主题**

>登录屏幕 (SDDM)，然后点击右上**应用 Plasma 设置...**。您需要有执行此操作的权限。

也可以手动完成同样的操作：

要启用 SDDM 中正确的显示和监视器处理（缩放、监视器分辨率、刷新率等），您可以从您的主目录复制或修改适当的配置文件到 SDDM 的目录：

```
# cp ~/.config/kwinoutputconfig.json /var/lib/sddm/.config/
# chown sddm:sddm /var/lib/sddm/.config/kwinoutputconfig.json
```

同时要启用 SDDM 中正确的输入处理（点击触控、触摸屏映射等），您可以从您的主目录复制适当的配置文件到 SDDM 的目录：

```
# cp ~/.config/kcminputrc /var/lib/sddm/.config/
# chown sddm:sddm /var/lib/sddm/.config/kcminputrc
```

界面中文化

按照安装指南#区域和本地化设置设置完locale后，SDDM界面会使用系统默认语言（`localectl`）。若要使用与系统默认不同的语言（比如系统使用`en_US.UTF-8`但是SDDM使用简体中文`zh_CN.UTF-8`），可在`sddm.service`中设置`LANG`环境变量，如

```
# systemctl edit --drop-in=lang sddm.service
```

此时会打开文本编辑器。在第一段空白处填写：

```
[Service]
Environment=LANG=zh_CN.UTF-8
```

然后重启sddm服务即可。

故障排除

只有空白屏幕和光标，但没有欢迎界面

如果没有足够的空间，欢迎界面会崩溃。使用 `df -h` 命令检查您的磁盘空间。

如果磁盘空间足够，那么这个问题可能源于此 bug。切换到另一个 TTY，并尝试 `loginctl unlock-session *session_id*`或重启 SDDM。

登录后挂起

尝试移除 `~/.Xauthority` 文件后不重启再次登入。重启会再次创建该文件即该问题依旧存在。

SDDM 在 tty1 启动而不是 tty7

SDDM 根据 systemd 的规定在 tty1 启动图形会话。

请注意，配置文件仍然有 `MinimumVT` 选项，但自 SDDM 版本 0.20 起已被忽略：sddm.conf(5) § MinimumVT=。

一个或多个用户没有出现在欢迎界面

警告：根据 UID 和 GID，通常不应将 UID 低于 1000 或高于 60513 的用户暴露给显示管理器。

默认情况下，SDDM 配置为仅显示 UID 在 1000 到 60513 范围内的用户。如果所需用户的 UID 在此范围之外，则需要修改该范围。

例如，对于 UID 为 501 的用户，设置 `MinimumUid` 并隐藏那些使用系统用户 shell 的用户：

```
/etc/sddm.conf.d/uid.conf
[Users]
HideShells=/usr/bin/nologin,/sbin/nologin,/bin/false,/usr/bin/git-shell
MinimumUid=500
```

对于具有过大 UID 的用户，将 `MaximumUid` 设置为适当的值。

用户头像未显示在欢迎界面上

如果用户数量超过 `DisableAvatarsThreshold` 参数设定的数量，或者 `EnableAvatars` 参数根本没有启用，那么在欢迎界面上将不会显示用户头像。要解决这个问题，请在您的 SDDM 配置中添加以下几行：

```
/etc/sddm.conf.d/avatars.conf
[Theme]
EnableAvatars=true # enable avatars
DisableAvatarsThreshold=7 # set the threshold for the number of users. Avatars are not shown if this threshold is exceeded.
```

SDDM 只加载 US 键盘布局

SDDM 加载的键盘布局被确定在 `/etc/X11/xorg.conf.d/00-keyboard.conf` 文件中。您可以通过 `localectl set-x11-keymap` 命令以生成此配置文件。详见 Xorg/键盘设置（英语：Xorg/Keyboard_configuration）。

设置仅在 SDDM 中设置而不在后续会话中设置的键盘布局的另一种方法是在 SDDM 的启动脚本（位于 `/usr/share/sddm/scripts/Xsetup`）中调用 *setxkbmap* 命令。有关示例，请参见Xorg/键盘设置#使用 setxkbmap（英语：Xorg/Keyboard configuration#Using setxkbmap）。

SDDM 可能也会错误地显示为 US 布局，但在您开始输入您的密码时立即切换到正确的键盘布局 。此 bug 看起来不是来自 SDDM，而是 Xorg。

屏幕分辨率过低

此问题可能源于显示屏 HiDPI 的使用破坏了 EDID 。如果你启动了 HiDPI，尝试关掉它。

如果上述方法失败了，您可以尝试在 Xorg 配置文件中设置您的显示尺寸：

```
/etc/X11/xorg.conf.d/90-monitor.conf
Section "Monitor"
```

```
        Identifier      "<default monitor>"
        DisplaySize     345 194 # in millimeters
EndSection
```

自动挂载家目录的加载时间过长

SDDM 默认会访问 `~/.face.icon` 文件以尝试显示用户头像。如果您的家目录采用自动挂载的文件系统（autofs），例如如果您使用 dm-crypt，这将会使之等待 60 秒，直到自动挂载的文件系统（autofs）返回此目录不能被挂载。

您可以通过创建以下文件关闭头像功能：

```
/etc/sddm.conf.d/avatar.conf
[Theme]
EnableAvatars=false
```

X authority（也称为 MIT-MAGIC-COOKIE）文件

SDDM 使用一个随机的新鲜 UUID 作为认证文件，详情见 。因此，要找到该文件，可以使用一个脚本来查找：

```
# find /var/run/sddm/ -type f
```

如果需要在没有用户登录时启动 x11vnc，可能需要这个文件。例如：

```
# x11vnc -display :0 -auth "$( find /var/run/sddm/ -type f )"
```

多屏幕设置上的重叠欢迎界面

在多屏幕设置上，X 监视器布局可能没有正确设置，导致欢迎界面重叠。要解决这个问题，添加以下几行来从左到右排序您的 SDDM 问候者布局：

```
/usr/share/sddm/scripts/Xsetup
for next in $(xrandr --listmonitors | grep -E " *[0-9]+:.*" | cut -d" " -f6); do
  [ -z "$current" ] && current=$next && continue
  xrandr --output $current --auto --output $next --auto --right-of $current
  current=$next
done
```

登录会话出现在意外的显示设备上

如果连接了多个显示设备，SDDM 登录会话可能会出现在与您的主显示设备不同的显示设备上。如果次要显示设备旋转了而主显示设备没有，这个问题可能会很烦人。一个简单的解决方法是在登录会话使用 `Xsetup` 脚本之前使用 `xrandr` 配置显示设备。例如，这里 `xrandr` 报告有两个已连接的显示设备，其中次要显示设备（DP-2）位于主显示设备（DP-4）的左侧。

```
# xrandr | grep -w connected
DP-2 connected 2160x3840+0+0 left (normal left inverted right x axis y axis) 597mm x
336mm
DP-4 connected primary 3840x2160+2160+0 (normal left inverted right x axis y axis)
697mm x 392mm
```

注意：xrandr 在 Wayland 会话中会输出不同的名称。确保在 X11 会话中运行上述命令。

以下 `Xsetup` 重新创建了登录窗口的上述设置：

```
/usr/share/sddm/scripts/Xsetup
#!/bin/sh
```

```
# Xsetup - 在登录对话框出现之前作为 root 运行

xrandr --output DP-4 --auto --primary
xrandr --output DP-2 --left-of DP-4 --rotate left --noprimary
```

使用 NVIDIA 显卡注销后黑屏

用户注销后可能会遇到完全黑屏或只有光标/显示设备徽标显示在屏幕上。这是因为 `sddm.service` 比 NVIDIA 驱动程序启动得更快。考虑使用 early KMS。

在 Wayland 上使用混合图形时屏幕不同步

如果您设置 SDDM 使用 `kwin_wayland` 合成器，启动时可能会遇到屏幕不同步的问题。如果返回到 X11 工作正常，并且您正在使用 `mesa-amber` 驱动程序，那么通过用 `mesa` 替换当前的 `mesa-amber` 驱动程序，Wayland 很可能会正常工作。您可以在 KDE Bug 483804 查看有关此问题的更多细节。

部分主题崩溃

部分 SDDM 没有在 `metadata.desktop` 中指定 `QtVersion`， 导致 SDDM 以不兼容的欢迎界面启动（Qt5 而不是 Qt6）。

若您设定了自定义主题，但重启后显示默认主题并报错 `Library import requires a version`，您需要将 `QtVersion=6` 添加到 `/usr/share/sddm/themes/*`主题名称`*/metadata.desktop`。

切换 12/24 小时制时间

SDDM 显示时间所用的小时制可能不符合需要。这种情况更常发生在使用非母语（例如，法语用户使用美式英语）但仍希望单位格式遵循本地习惯的用户身上。

SDDM 设置不能调整时间的小时制，这由 `/etc/locale.conf` 决定，所以请将 `LC_TIME` 设置为所需 locale。详见 Locale。

输入法

桌面要能打中文，得三样对上：输入法框架、引擎，以及各程序的输入模块指向谁。本章按框架讲 Fcitx 5、Fcitx、IBus、Rime 的安装与配置，也保留 SCIM 与 Gcin 两条旧路线。

先读《输入法》理清关系，再挑一个框架落地：《Fcitx 5》是当下该用的，《IBus》是主要替代，精读其一即可，Rime 留到定制方案时再看。《Fcitx》是旧版，配置与 5 不通用，别混着照抄；SCIM 与 Gcin 面向更早的环境，可跳过。最常见的误解是装上框架就完事——"某程序里打不出字"多出在各节的环境适配与故障处理里，先按那里排查，再动配置。

9.1　Gcin

难度 入门 | **篇幅** 2.7 KB | **所属** 输入法

本节将讲到

- 安装
- 配置
- 用法

Gcin 是 Edward Liu 开发的新一代中文输入法服务器。 Gcin 支持各种输入方法，并且可以在大多数类 Unix 操作系统下运行。 它是台湾最受欢迎的中文输入引擎之一。

安装

安装 gcin包 软件包。

安装其他输入法表格

配置

自动启动

使用 xprofile 以自动执行以下命令：

```
export XMODIFIERS=@im=gcin
export LC_CTYPE=zh_TW.UTF-8
gcin &
```

用法

对于 GNOME/GTK 2 应用程序

gcin 提供了一个 gtk 输入模块，因此直接支持所有基于 gtk2 的应用程序，安装后无需进行任何配置（它不是 XIM，gcin 会在需要时自动启动）。

对于其他应用程序

1\. 将环境语言环境设置为使用 UTF-8，例如：

```
export LC_CTYPE=en_US.UTF-8
```

注意：必须设置 LC_CTYPE 语言环境，即使 LC_CTYPE 与 LANG 相同，也，否则gcin可能不会在使用x输入的非gtk2程序中激活。

2\. 设置 XMODIFIERS：

```
export XMODIFIERS=@im=gcin
```

gcin 默认使用名称“gcin”，您可以使用环境变量 GCIN_XIM 进行更改，以便运行多个 gcin 程序，例如：

```
export GCIN_XIM=gcin_zh
export XMODIFIERS=@im=gcin_zh
```

请记住，如果 gtk2 应用程序不存在，它将自动启动 gcin 的一个程序。

3\. 启动 gcin：

```
gcin &
```

4\. 运行您的应用程序！如果在您的应用程序运行时 gcin 被杀死，则很可能导致崩溃或其他问题。

Wine/Crossover Office 的其他说明

1. 如果运行 wine 或 Crossover Office，最好使用 Windows 2000 模拟而不是 Windows 98，并且必须至少使用 LC_CTYPE=zh_TW.utf8 启动 gcin 和 wine/cxoffice，否则 wine 将无法正确显示中文。
2. 在具有 Windows 98 仿真的 wine+IE6 中，如果您想在网页上输入中文，LC_CTYPE 是不够的 - 您必须将 LANG 或 LC_ALL 设置为 zh_TW.utf8，这会大大降低 Wine 的速度。但是，您始终可以在位置栏中或其他位置键入中文，然后将其粘贴。

9.2 Smart Common Input Method

难度 进阶 | **篇幅** 7.8 KB | **所属** 输入法

本节将讲到

- 安装SCIM
- 配置SCIM
- 已知问题
- 外部链接

**这篇文章或章节的翻译质量不佳。**

原因：Last updated in 2012（在 Talk:Smart Common Input Method# 中讨论）

警告：SCIM 已经 "不被开发者维护", 而 IBus 被 Red Hat 积极开发。Arch 用户安装软件时应该考虑用 fcitx、IBus 或 Uim 而不是 SCIM。

Su Zhe (或 James Su)在为TurboLinux工作的时候，于2001年发起了SCIM项目，该项目的目标是：

- 为当前可用的输入法库提供一个统一前端；
- 作为IIIMF输入法框架的语言引擎；
- 尽可能多地提供各国输入引擎；
- 尽可能多地支持输入法协议/接口；
- 尽可能多地支持各种操作系统。

SCIM具有以下特性：

- 使用C++编写，完全面向对象结构；
- 高度模块化
- 体系结构非常灵活，可以作为其它C/S输入法环境的动态链接库；
- 简单的编程接口
- 完全支持i18n UCS4/UTF-8编码
- 具有许多便利的工具可以加速自身开发
- 具有特性丰富的图形化面板
- 统一的配置框架

安装SCIM

```
pacman -S scim
```

安装输入法引擎

目前SCIM包含许多各类的输入法（有些可能需要一些其它的库），覆盖30多种语言，包括中文（简体、繁体）、日文、韩文及许多欧洲语言：

(在这里
中文智能拼音：

```
pacman -S scim-pinyin
```

中文五笔及其它：

```
pacman -S scim-tables
```

日文：

```
pacman -S scim-anthy
```

韩文：

```
pacman -S scim-hangul
```

配置SCIM

方法一： 为了让SCIM在桌面中自动启动并且正常工作，编辑~/.xinitrc，在启动桌面环境/窗口管理器的语句前面加入以下内容（如果使用scim-bridge，把下面的“scim”改为“scim-bridge”）：

```
export LC_CTYPE="zh_CN.UTF-8"  （请改成你在X下使用的locale，如果没有合适的locale，请查询locale-gen相关信息）
export XMODIFIERS=@im=SCIM
export GTK_IM_MODULE="scim"
export QT_IM_MODULE="scim"
scim -d
```

如果使用"scim -f socket -c socket -d"替换"scim -d "，会导致一些qt程序无法使用scim，比如eva。 现在进入X，scim应该已经启动了，你可以在图标上点击右键改变SCIM配置（比如去掉一些不用的输入法）。在任何程序中按Ctrl-Space就可以使用输入法了。

使用kdm/gdm时自动启动scim

创建一个新文件~/.xprofile，加入以下内容（如果使用scim-bridge，把下面的“scim”改为“scim-bridge”）：

```
export LC_CTYPE="zh_CN.UTF-8"
export XMODIFIERS=@im=SCIM
export GTK_IM_MODULE="scim"
export QT_IM_MODULE="scim"
scim -d
```

查看这里获得更多官方信息。

方法二： Gnome用户可以试一下，编辑/etc/gtk-2.0/gtk.immodules，在最后加入以下内容：

```
 "/usr/lib/gtk-2.0/immodules/im-scim.so"
 "scim" "SCIM Input Method" "scim" "/usr/share/locale" "ja:ko:zh"
```

如果LC_CTYPE为"en_US.UTF-8“，需要将"ja:ko:zh"改成"en:ja:ko:zh"。当你输入命令gtk-query-immodules-2.0，你将会发现“en”是关键。随后重启。SCIM托盘可能不显示，但是当你设置好输入法后，按下CTRL+空格，输入条将会显示。KDE或许有类似的方法。警告：加了"en"，PC重启后任务栏可能不显示。按CTRL+ALT+F1进入SHELL，输入sudo reboot或reboot重启。或者用vi或vim编辑/etc/gtk-2.0/gtk.immodules。

环境变量

以下环境变量在运行scim之前必须被export：

```
export XMODIFIERS=@im=SCIM
export GTK_IM_MODULE="scim"
export QT_IM_MODULE="scim"
```

通常这些变量被放在脚本文件中，如 ~/.xinitrc 或者 /etc/profile (全局设置), 又或者 ~/.config/openbox/autostart (如果使用 Openbox 作为 窗口管理器).如果放在 ~/.xinitrc, 必须放在执行桌面环境/窗口管理器之前。

如果你不知道那种方法更好，使用 /etc/profile.

注意：第一个环境变量与一些(少见的)选项如 `XMODIFIERS=urxvt` 冲突.

GNOME, XFCE, LXDE 用户

如果 *export QT_IM_MODULE="scim"* 无法工作，可以使用 AUR 中的 scim-bridgeAUR\[损坏的链接：package not found\]。在 2009.02.15 的更新中(scim 1.4.7-1, AUR 中的 scim-bridge 0.4.15-1)下面配置可以在 GNOME, XFCE, LXDE 中工作：

```
export QT_IM_MODULE="scim-bridge"
```

KDE3

将 QT IMM 设为 **xim** 或安装 AUR 中的 scim-qtimm-cvsAUR\[损坏的链接：package not found\]:

```
export QT_IM_MODULE="xim"
```

GTK

scim 无法在 gtk 应用程序中工作，请检查 GTK_IM_MODULE_FILE 环境变量，应该设置为 /etc/gtk-2.0/gtk.immodules 或包含输入法模块文件的目录。

在上面提到的脚本中加入下面内容，放到执行DE/WM之前:

```
gtk-query-immodules-2.0 > ~/.immodules
export GTK_IM_MODULE_FILE=~/.immodules
```

GTK 会从 ~/.immodules 中查找文件，如果将 GTK_IM_MODULE_FILE 设置为 /etc/gtk-2.0/gtk.immodules，不需要第一行。

Locale 相关的文件

如果键盘 locale 不是 en_US.UTF-8 (或 en_US.utf8)，需要修改 ~/.scim/global (或系统设置 /etc/scim/global)的下面这行：

```
/SupportedUnicodeLocales = en_US.UTF-8,de_CH.UTF-8
```

将 de_CH.UTF-8 修改为所用 locale。

注意：locale 必须已经启用(/etc/locale-gen 中取消注释并以 root 执行 locale-gen) 而且被 SCIM 支持(支持大部分 *.UTF-8 locales).

如下命令可以获得活动的 locale：

```
locale -a
```

(查看 /etc/locale.gen 也可以).

更多关于 locale 的疑难解答

如果安装 scim 和输入法表之后，scim 还无法工作(点击系统图标无反应)，需要在 `/etc/profile` 中将 `LC_CTYPE` 设置为要使用的 locale。

```
LC_CTYPE="zh_CN.UTF-8"                //if you want to type simplified chinese
```

修改/etc/locale.gen 文件，取消 locale 前的注释并用 locale-gen 命令生成 locale：

```
locale-gen
```

执行 SCIM

可以直接执行:

```
scim
```

建议作为守护进程执行：

```
scim -d
```

可以将上面文件放入脚本中自动执行，例如 ~/.xinitrc (环境变量之后，DE/WM之前), /etc/profile (环境变量之后) 或 ~/.config/openbox/autostart (环境变量之后，可以 sleep 几秒)。

GNOME

使用 GNOME 需要执行：

```
 scim -f x11 -c simple -d
```

要自动启动，在 System > Preferences > Session 中创建上面一行命令。

注意：如果使用 scim -f socket -c socket -d，SCIM 的配置文件无法修改。

KDE

KDE 中需要执行：

```
 scim -f socket -c socket -d
```

或者采取：

1. 从 AUR 安装 **skim**.
2. 启动 **SKIM**，系统图标上右键，点击 'Configure'
3. 在 **Frontend > X Window** 中点击"Start skim automatically when KDE starts"
4. 登出并重启 X server (ctrl+alt+del)，重新登录

按键盘 *ctrl+space* 激活输入法。

已知问题

LWJGL (Lightweight Java Game Library) losing keyboard focus

一个解决方法是 kill scim ，应用程序就可以接收键盘输入了。

参见 和 .

外部链接

- See the official news page for more details.

9.3 Rime

难度 进阶 | **篇幅** 10.1 KB | **所属** 输入法

本节将讲到

- 安装
- 配置
- 使用
- 疑难解答
- 技巧

Rime（中州韵输入法引擎）是一款支持多种输入方案的输入法引擎。

Rime 本身没有用于处理用户输入的前端，需要配合输入法框架才能使用，比如 Fcitx5 或 IBus。

注意：Fcitx5 和 Fcitx 是两个不同的软件包。Fcitx 处于维护模式，建议使用 Fcitx5。自 2021 年以来，fcitx-im 的官方页面已指向 Fcitx5。由于官方软件包尚未重命名以反映这一变化，请特别注意。

安装

分别安装提供 Rime 输入引擎的 librime包 和所使用输入法对应的集成：

- 若使用 Fcitx5，安装 fcitx5-rime包；
- 若使用 Fcitx，安装 fcitx-rimeAUR；
- 若使用 IBus，安装 ibus-rime包。

配置

Rime 需要输入方案才能工作，用户可以定制输入方案。默认情况下，一些方案随着 librime-data包 元软件包一起安装，因为它是 librime包 的依赖项。

另外，在软件仓库中有几个可用的官方输入方案：

- rime-luna-pinyin包 - 朙月拼音
- rime-terra-pinyin包 - 地球拼音
- rime-double-pinyin包 - 基于朙月拼音的双拼输入，包含自然码、MSPY、智能ABC、小鹤双拼、拼音加加等
- rime-pinyin-simp包 - 袖珍简化字拼音
- rime-cantonese包 - 粤语拼音
- rime-cangjie包 - 仓颉五代

- rime-quick包 - 速成
- rime-stroke包 - 五笔画（笔画输入法）
- rime-wubi包 - 五笔字型86
- rime-wugniu包 - 上海吴语
- rime-bopomofo包 - 注音
- rime-emoji包 - Emoji 表情符号输入
- rime-flypyAUR - 小鹤音形
- rime-ice-gitAUR 或 rime-ice-gitCNRepo - 雾凇拼音, 自带大词库和其他智能功能

一些软件包提供不止一种输入方案，如 rime-luna-pinyin包 就提供了包括 `luna_pinyin` 和 `luna_pinyin_fluency` 在内的五种输入方案。

使用 Rime 时，默认可以按 `F4` 或 ` Ctrl+`` 切换输入法，详见#选择输入方案。

配置目录

若要自定义 Rime，请先根据使用的输入法创建 Rime 的配置文件夹：

若使用 ibus-rime包：

```
$ mkdir ~/.config/ibus/rime
```

若使用 fcitx-rimeAUR：

```
$ mkdir ~/.config/fcitx/rime/
```

若使用 fcitx5-rime包：

```
$ mkdir ~/.local/share/fcitx5/rime/
```

在文件夹中创建 `default.custom.yaml` 文件，以指定可选的输入法。例如，若要按声调输入拼音，添加下列内容以使用地球拼音：

```
default.custom.yaml
patch:
  schema_list:
    - schema: terra_pinyin
```

添加内容时请注意行首缩进。此文件会覆盖默认的配置文件，若只在文件中添加地球拼音，则只能用地球拼音。

应用配置

请重新部署输入法以使自定义生效。如果使用图形界面的 IBus 或 Fcitx，请点击 ↻（重新部署）按钮。也可以使用命令行重新部署：

若使用 ibus-rime包：

```
$ rm ~/.config/ibus/rime/default.yaml && ibus-daemon -drx
```

若使用 fcitx-rimeAUR：

```
$ rm ~/.config/fcitx/rime/default.yaml && fcitx-remote -r
```

若使用 fcitx5-rime包:

```
$ rm ~/.local/share/fcitx5/rime/default.yaml && fcitx5-remote -r
```

声调过滤

注意：可以选择声调以过滤候选列表，对应按键如下：

```
一声： -
二声： /
三声： <
四声： \
```

例如，若要打出拼音为 `hǎo` 的字，输入 `hao<` 即可。

候选项个数

Rime 默认只会列出 5 个候选项，可以修改 `"menu/page_size"` 的值以改变列出候选项的数量：

```
default.custom.yaml
patch:
    "menu/page_size": 9
```

使用

选择输入方案

使用 Rime 时，默认可以按 `F4` 或 ` Ctrl+ `` 调整基础设置。显示的设置项如下：

```
1.  输入法名称
2. 中文  ―›  西文
3. 全角  ―›  半角
4. 漢字  ―›  汉字
...
```

第一项会显示输入方案的名称，可以在其中切换已启用的不同输入方案。

第二项可以切换中文/英文输入。

第三项可以切换全角/半角标点。

第四项可以切换简体/繁体输入。

其他选项取决于当前使用的输入方案。

自定义输入方案

通过覆盖某一输入方案的默认选项，可以不用每次打开菜单。

这需要创建一个 `custom` 文件（定制文件）来覆盖 `scheme` 文件（方案定义）中的选项。

例子

下文以更改地球拼音的默认选项从「繁体中文」到「简体中文」为例。

该输入方案由 rime-terra-pinyin包 在 `/usr/share/rime-data/build/terra_pinyin.schema.yaml` 提供。

根据输入方案定义里的 `switches` 段：

```
switches:
  - name: ascii_mode
    reset: 0
    states: ["中文", "西文"]
  - name: full_shape
    states: ["半角", "全角"]
  - name: simplification
    states: ["漢字", ]
  - name: ascii_punct
    states: ["。, ", ". , "]
```

这一段是地球拼音所提供的选项菜单（由 `F4` 或者 ` Ctrl+ `` 打开）。对于不同的输入方案，配置文件可能会有所不同。

在 Rime 的 #配置目录中，创建地球拼音的定制文件：

```
terra_pinyin.custom.yaml
patch:
  switches:
    - name: simplification
      reset: 1
```

`name` 项匹配方案定义中的 `switches` 列表的对应选项。 `reset: 1` 指 Rime 总是会重设该项为第二个选项 (下标 1，即 "汉字").

#应用配置以加载定制文件。

Rime 提供了多种通过 YAML 来进行此类自定义的方法，更多案例可在 #进阶内容查看。

中文标点

按下列各键输入不同的符号：

```
[ ->  「  【  〔  [
] ->  」  】  〕  ]
{ ->  『  〖  {
} ->  』  〗  }
< ->  《  〈  «  ‹
> ->  》  〉  »  ›
@ ->  @  @  ☯
/ ->  ／  /  ÷
* ->  ＊  *  ·  ×  ※
% ->  ％  %  °  ℃
$ ->  ¥  $  €  £  ¥
| ->  ·  |  ¦  §  ¦<br>_ -> ——<br>\ ->  、  ＼  \<br>^ ->  ……<br>~ ->  ～  ~  ～  ～<br>```
```

进阶内容

若要查看详细的 Rime 定制指南，见 https://github.com/rime/home/wiki/CustomizationGuide 。

疑难解答

GNOME 环境下，ibus-setup 无法更改候选方向

见 [issue #52](https://github.com/rime/ibus-rime/issues/52)。 创建 `~/.config/ibus/rime/build/ibus_rime.yaml` 并写入以下内容：
style: horizontal: true

技巧

输入希腊字母

若想输入希腊字母，请在 `luna_pinyin.custom.yaml` 或自定义输入方案文件中追加以下内容：
luna_pinyin.custom.yaml

recognizer: patterns:

punct: "^/([0-9]0?|[A-Za-z]+)$" punctuator: symbols:

"/alpha": ["A", "α"] "/beta": ["B", "β"] "/gamma": ["Γ", "γ"] "/delta": ["Δ", "δ"] "/epsilon": ["E", "ε"] "/zeta": ["Z", "ζ"] "/eta": ["H", "η"] "/theta": ["Θ", "θ"] "/iota": ["I", "ι"] "/kappa": ["K", "κ"] "/lambda": ["Λ", "λ"] "/mu": ["M", "μ"] "/nu": ["N", "ν"] "/xi": ["Ξ", "ξ"] "/omicron": ["O", "o"] "/pi": ["Π", "π"] "/rho": ["P","ρ"] "/sigma": ["Σ", "σ", "ς"] "/tau": ["T", "τ"] "/upsilon": ["Y", "υ"] "/phi": ["Φ", "φ"] "/chi": ["X", "χ"] "/psi": ["Ψ", "ψ"] "/omega": ["Ω", "ω"]

在此输入配置下，输入 `/alpha` 即会出现候选词 `α` 。

参见

- [官方网站](https://rime.im/)
- [GitHub wiki](https://github.com/rime/home/wiki)

9.4 IBus

难度 进阶 | **篇幅** 13.5 KB | **所属** 输入法

本节将讲到

- 安装
- 环境适配
- 配置
- 使用提示和技巧
- 疑难解答

**这篇文章或章节的翻译质量不佳。**

原因：Looks nothing like the structure of the English page.（在 Talk:IBus# 中讨论）

IBus ("Intelligent Input Bus") 是一个输入法框架，允许用户便捷地在不同的键盘布局中切换。当IBus框架和特定的输入法结合使用时，你就可以使用你的键盘输入键盘本身所不支持的非拉丁字母。（译者注：即在ArchLinux——其他Linux也类似，输入法需要和一个框架搭配使用，IBus就起到这个框架的作用）

安装

安装 ibus包 软件包。

注意：GNOME自从3.6版本以来，就集成了IBus，并默认使用。所以如果你使用3.6版本以上的GNOME作为你的桌面环境，你可以跳过这一步。

输入法引擎

至少需要一种与所用语言相应的输入法。您可以在输入法#输入法列表查看IBus支持的输入法列表。部分可用的输入法如下，如您发现无法使用，请前往输入法#输入法列表查看并积极反馈以更新该内容

中文引擎

- ibus-libpinyin包：一个强大的智能中文输入引擎，基于 libpinyin，提供了比 ibus-pinyin 更强大的功能，支持动态调整字频、云输入，可以添加用户词典
- ibus-rime包：一个强大的智能中文输入法，支持拼音、注音或者没有音调的拼音、双拼、粤拼、中州韵、仓颉和五笔 86。
- ibus-chewing包：一个智能中文输入法引擎，支持注音符号，基于 libchewing 包。

日文引擎

- ibus-anthy包：日文输入法引擎，基于 anthy包。
- ibus-mozcAUR：谷歌日文输入法的开源版本，基于 Mozc。
- ibus-kkc包：日文输入法引擎，基于 libkkc包。
- ibus-skk包：日文假名转汉字输入法引擎，基于 libskk包。

越南文引擎

- ibus-bambooAUR：越南文输入法引擎，基于 Bamboo 引擎。
- ibus-bogoAUR：越南文输入法引擎，基于 Bogo。（已停止开发）
- ibus-unikey包：用于输入越南字的输入法引擎。（已停止开发）

其他引擎

- ibus-hangul包：一个韩文输入法，基于 libhangul包。
- ibus-table包：一个支持查表型输入法的输入法引擎。

- ibus-m17n包：一个 m17n 输入法引擎，可以用 m17n-db 数据库中的输入法来输入许多语言。

查看所有可用的输入法：

```
$ pacman -Ss ^ibus-*
```

其他软包也供给于 AUR。

环境适配

请按照如下示例设置环境变量

```
/etc/environment
GTK_IM_MODULE=ibus
QT_IM_MODULE=ibus
XMODIFIERS=@im=ibus
```

如果你希望当你登录时，IBus就启动，请使用以下命令创建一个自动启动条目：

```
 ibus-daemon -rxRd
```

这样，下次登录时，IBus就会随着用户会话而自动启动

提示：* ibus-autostartAUR 软件包提供了一个自动启动 条目来自动启动该服务

- ibus-daemonAUR软件包提供了一个服务`ibus@$DISPLAY.service`，可以启用它来达到这个目的
- KDE提供了一个Plasm形式的*输入面板*，可以作为IBus的前端，以提供更好的输入体验
- 欲启动有*输入面板*的IBus，请在*ibus-daemon*命令中加入 `--panel=/usr/lib/kimpanel-ibus-panel`选项。（如果面板没有出现，你还可以尝试用`--desktop=kde`选项。要配置*输入面板*\的宽度，请参见KDE UserBase Wiki和KDE#Widgets
- ibus-autostart-kimpanelAUR软件包提供了一个可以自启动该功能的自动启动条目

译者注：KDE#Widgets这一章节由于KDE翻译的过时，无法链接到最新的中文章节，遂在此链接了原文Wiki的链接。如翻译有所跟进，请随时修改。

配置

你可能需要先设置系统的语言环境，请：

1. 参见Locale来获取关于在系统中安装非拉丁文字体的帮助
2. 参见字体#非拉丁文字获取可用的非拉丁字体的不完全列表

GNOME

GNOME默认使用IBus，因此你只需简单地打开*设置*>*键盘*>*输入源*并添加你所使用的键盘布局即可。

一些非拉丁语言（如简体中文）需要在GNOME Tweaks中启用*显示拓展输入源*选项

其他桌面环境

提示：对于Wayland环境，请参见WaylandDesktop

要启动IBus偏好设置窗口，你可以执行以下任一操作：

- 右键IBus的托盘图标，选择*偏好*；
- 找到并启动IBus偏好设置的图形化应用程序；
- 在终端中运行命令`ibus-setup`

在这个窗口，主要有两个重要的功能：一是*下一个输入法*的快捷键（建议你设置这个选项，来替代你的桌面环境所提供的快捷键）；二是*输入法*页面，你可以在这里添加或者删除不同的键盘布局（建议你在此处操作，而不是在桌面环境所默认提供键盘布局管理器操作）

使用提示和技巧

moji 输入

IBus支持Emoji表情符号的输入。按下快捷键`Super+.`即可看到输入模式变为一个下划线e字符。接下来你可以输入符号或emoji的名称（例如*:)*或*face)*）并敲击Space键（空格键）渲染该符号。如果你对该结果表示满意，你可以按下回车来输入该符号并推出Emoji输入模式。或者你也可以再次按下空格键，来打开一个对话框，以便于你进一步选择你需要的符号。

Unicode输入

IBus支持输入复杂的Unicode字符。使用快捷键`Ctrl+Shift+u`即可看到输入模式变为一个下划线u字符。你可以输入Unicode代码并按`Space` 或 `Enter` 来渲染并输入该字符

托盘图标颜色

IBus默认使用蓝色来显示当前正活跃的键盘布局（如**EN**）这个颜色值存储在gestting schema中，因此如果你希望修改他，请执行如下命令：

```
$ gsettings set org.freedesktop.ibus.panel xkb-icon-rgba 'COLOR'
```

其中，'COLOR'是字符串形式，遵循如下要求：

是一个RGBA值，即：

1. 一个X11中的颜色名称
2. 一个十六进制数值，如'#rrggbb'，r,g,b分别是红、绿、蓝的十六进制数
3. 一个如下格式的RGB数值：'rgb(r,g,b)'
4. 一个RGBA颜色，应该如下格式：'rgba(r,g,b,a)'，其中r,g,b都是\[0,255\]范围内的数值或\[0%,100%\]范围内的百分数，而a是\[0,1\]范围内的浮点数，以表示透明度

提示：你也可以使用dconf-editor包来以一种更友好的方式修改设置模式（gsettings schema）

注意：这不适用于 GNOME 或存在KDE输入法面板的 KDE，因为它们都为 IBus 提供了自己的图标。

疑难解答

rxvt-unicode

如果有 ibus 与 rxvt-unicode包 包的问题，以下的步骤应该能够解决。

将以下的两行代码添加到你的 `~/.Xresources` 文件（可能不需要，先尝试，如果问题出现，再添加代码）：

```
URxvt.inputMethod: ibus
URxvt.preeditType: OnTheSpot,None
```

然后用以下的命令启动Ibus:

```
ibus-daemon --xim
```

如果 ibus-daemon 自动开启（如在 `~/.xinitrc` 或 `~/.xsession` 中)，但是以前执行 `ibus-daemon &` 没有用 `--xim` 选项，确保先结束已打开的进程，再尝试新命令。

GTK 应用程序

有些用户在 GTK 应用程序下使用输入法时会因为无法找到 gtk.immodules 文件而出现问题。在 $HOME/.bashrc 中加入

```
(gtk2) export GTK_IM_MODULE_FILE=/etc/gtk-2.0/gtk.immodules
(gtk3) export GTK_IM_MODULE_FILE=/usr/lib/gtk-3.0/3.0.0/immodules.cache
```

应该会解决问题。

注意：如果你设置为 gtk2，那么你无法使用 gtk3 的应用程序比如gedit, 如果你设置为 gtk3，那么你无法使用 gtk2 的应用程序比如 xfce

中文输入

如果你在输入中文时遇到问题，检查你的 locale 设置。比如在香港，export LANG=zh_HK.utf8。

如需 ibus 随 gnome 启动，把这些加入 `~/.profile` 后重启 gnome。

```
export GTK_IM_MODULE=ibus
export XMODIFIERS=@im=ibus
export QT_IM_MODULE=ibus
ibus-daemon -d -x
```

更详细的解决方案可以查看这里。

在 gnome桌面 wayland 环境下,可能会出现部分软件无法切换中文输入,例如 QQ 等,需要安装Input Method Panel这个 gnome-shell扩展解决.

如果安装的输入法无法调整候选词方向等,可以尝试IBus Tweaker进行调整

LibreOffice

如果 IBus 确实已经启动，但是在 LibreOffice 里没有出现输入窗口，你需要在 ~/.bashrc 里加入这行：

```
export XMODIFIERS=@im=ibus
```

然后你需要用 "--xim -d" 参数来启动 ibus， 你可以在 ~/.xinitrc 中加入这行：

```
ibus-daemon --xim -d
```

但是可怕的是你必须在终端中启动 LibreOffice。

如果你使用 KDE 而上面的方法没用，而你也不介意在 GTK2 模式下运行 LibreOffice，安装 "libreoffice-gnome" 然后在 ~/.xprofile 中添加此行：

```
export OOO_FORCE_DESKTOP="gnome"
```

这会使 IBus 在 LibreOffice 正常使用，你也可以在任何地方启动 LibreOffice -- 而不只是在终端。

修改 Gnome-shell 中 ibus 候选框的字体和字号

通过扩展

Gnome 扩展中心提供了一些扩展来修改 ibus 界面，例如 Customize iBus，用户可以在图形界面中修改 ibus 字体、字号等等

通过自定义主题

很多人对 Gnome-shell 不能独立的设置 ibus 输入法的候选词字体和字号颇有微词，下面，介绍一种修改的办法。 首先，你需要安装一个 Gnome-Shell 主题，且激活它，然后你需要修改主题的 gnome-shell.css 文件。这个文件一般在目录 /usr/share/themes/主题名/gnome-shell/ 下。使用你喜欢的编辑器打开它，搜索 .candidate-popup-content 字段（如果没有就新建一个）：

```
.candidate-popup-content {
}
```

然后根据需要添加以下两行（添加后应该是下框中的样子），通过本设置可以改变输入的字母的字体和字号：

```
.candidate-popup-content {
       /* 设置字体 */
       font-family: "Microsoft YaHei UI", serif,cantarell,sans-serif;
       /* 设置号 */
       font-size: 15px;
}
```

如果需要修改候选框的字体和字号，你需要搜索 .candidate-box 字段（如果没有就新建一个）：

```
.candidate-box {
}
```

然后根据需要添加以下两行（添加后应该是下框中的样子），通过本设置可以改变输入的字母的字体和字号：

```
.candidate-box {
        /* 设置字体 */
        font-family: "Microsoft YaHei UI", serif,cantarell,sans-serif;
        /* 设置号 */
        font-size: 15px;
}
```

开启单行模式

ibus-libpinyin包 可以在其设置界面中的“外观”-“显示风格”这一项选择“紧凑”，以此来开启单行模式。

注：ibus-libpinyin 在输入源和系统托盘中显示为“中文（智能拼音）”。

9.5　输入法

难度 进阶 | **篇幅** 13.9 KB | **所属** 输入法

本节将讲到

- 输入法
- 输入法列表
- 配置
- 参见

输入法

提示：如果以下文本在屏幕上显示为乱码，请确保系统中安装了中文或日文字体。有关可用字体的非详尽列表，请参阅字体#中日韩越文字。

注意：为简单起见，在以下示例中仅提到了键盘，但 IME 实际上可以使用多种输入源，例如使用鼠标或 Wacom tablet 绘制字符。

引自 Wikipedia:Input method：

: **输入法**（或**输入法编辑器**，常缩写为 IME）是一种操作系统组件或程序，它使用户能够通过使用输入设备上原生的字符序列（或鼠标操作）来生成其输入设备上没有的文字。对于拥有比键盘上的按键更多的字位的语言来说，使用一种输入法通常是必要的。

简单来说，IME 是一个允许我们使用拉丁字符输入非拉丁字符的应用程序。

一些 IME 通过一种叫做罗马化的过程来实现这一点，即把非拉丁语的声音音译成与之最相近的拉丁语对应物。例如，日语中“清酒”或“米酒”的写作**酒**，也写成**さけ**，并被罗马化为“sake”。IME 的作用是充当键盘和输入字段之间的中间人，因此，

当我们输入“sake”时，它将拦截键盘的输入，用 **酒** 或 **さけ**（由用户选择）替换 “sake”，并为我们输入本地字符而不是我们按下的键。

也有一些 IME 不使用罗马化。其中最突出的是仓颉，它通过将汉字分解为部首，将这些部首与第二套自己的内部部首相匹配，最后再将这些内部部首与拉丁文字相匹配。举例来说，中文的*酒*由部首 **氵**、**一**、**儿**、**口** 和 **一** 组成。仓颉将这些部首与内部部首**水**、**一**、**金**、**田** 和 **一**匹配，然后将这些部首与拉丁字符 emcw 匹配。这意味着，当我们输入“emcw”时，仓颉会拦截键盘的输入，用 **酒** 替换 "emcw"，并在屏幕上输入该字符。

输入法框架

大多数 IME 是作为**输入法框架**（常缩写为 IMF）的一部分工作的，它是一个允许用户在不同的 IME 之间轻松切换的应用程序。事实上，这正是我们许多人每天在不同的拉丁语键盘布局（如英语、西班牙语、德语等）之间切换时不知不觉地使用的应用程序。

最常见的 IMF 是 IBus（通常用于基于 GTK 的环境，如 GNOME），其次是 Fcitx5（大多用于基于 Qt 的环境，如 KDE），然后是 Scim 和 Fcitx，最后是 Uim。非常不常见的包括 Gcin、Nimf 和 Hime。此外，Emacs是一个非常流行的文本编辑器，它有自己的内部 IMF。

参见 Wikipedia:List of input methods for Unix platforms。

注意：SCIM 目前缺乏维护，因此不推荐使用。

输入法列表

下表显示了 Arch 存储库和 AUR 中当前可用的各种语言的 IME。

注意：在某些情况下，同一个 IME 存在多个软件包。这方面的一个很好的例子是**Mozc**：作为目前最流行的日语 IME，多年来有多个打包者试图创建一个“完美的”Mozc 包。在下面的表格中，每个 IMF 都包括一个 Mozc 软件包；但这并不意味着这些是用户应该考虑安装的唯一 Mozc 软件包。

警告：并非所有 IME/IMF 都是最新的；例如，Anthy 不再维护，通常首选 mozc。一些 IME 也相当不受欢迎，并且不提供太多支持。查看相应的 Localization 页面了解更多信息。

	Fcitx5	Fcitx	IBus	uim	Emacs	Scim	Hime	gcin	Nimf
中文									
Rime	fcitx5-rime包	fcitx-rime包	ibus-rime包	–	–	–	–	–	内置
拼音	fcitx5-chinese-addons包	内置	ibus-pinyin包	–	–	scim-pinyinAUR	–	–	内置
注音	fcitx5-chewing包		ibus-chewing包	–	–	scim-chewing包	–	–	内置

	Fcitx5	Fcitx	IBus	uim	Emacs	Scim	Hime	gcin	Nimf
		fcitx-chewing包							
仓颉 簡易 快速	fcitx5-table-extra包	fcitx-table-extra包	ibus-table-chinese包	–	–	scim-tablesAUR	–	–	–
五笔	内置	内置	ibus-table包	内置	–	scim-tablesAUR	–	–	–
Libpinyin	–	fcitx-libpinyin包	ibus-libpinyin包	–	–	–	–	–	–
SunPinyin	–	fcitx-sunpinyin包	ibus-sunpinyin包	–	–	–	–	–	–
日文									
Mozc	fcitx5-mozc-utAUR	fcitx-mozc-utAUR	ibus-mozcAUR	–	emacs-mozcAUR	–	–	–	–
Anthy	fcitx5-anthy包	fcitx-anthy包	ibus-anthy包	*内置*	–	scim-anthyAUR	*内置*	*内置*	*内置*
SKK	fcitx5-skk包	fcitx-skk包	ibus-skk包	*内置*	–	–	–	–	–
KKC	fcitx5-kkc包	fcitx-kkc包	ibus-kkc包	–	–	–	–	–	–
韩文									
Libhangul	fcitx5-hangul包	fcitx-hangul包	ibus-hangul包	*内置*	–	scim-hangulAUR	–	–	*内置*
越南语									
UniKey	fcitx5-unikey包	fcitx-unikey包	ibus-unikey包	–	–	–	–	–	–
Bamboo	fcitx5-bamboo包	–	ibus-bambooAUR	–	–	–	–	–	–
Indic									
Avro （孟加拉语）	–	–	ibus-avro-gitAUR	–	–	–	–	–	–
Helakuru （僧伽罗语）	–	–	ibus-helakuruAUR	–	–	–	–	–	–
m17n	fcitx5-m17n包	fcitx-m17n包	ibus-m17n包	–	–	scim-m17nAUR	–	–	–
OpenBangla Keyboard （孟加拉语）	fcitx5-openbangla-gitAUR	–	openbangla-keyboardAUR	–	–	–	–	–	–
Sayura （僧伽罗语）	fcitx5-sayura包	fcitx-sayura包	–	–	–	–	–	–	–
Varnam	–	–	libvarnam-ibus-gitAUR	–	–	–	–	–	–

配置

为了让桌面环境正确地将已安装的输入法框架注册为可用并将其用于处理用户输入，必须相应地配置一组环境变量。

注意：如果没有设置这些变量，GTK 和 Qt 都会尝试读取系统的区域设置来决定它们应该使用哪种 IMF，但这个过程依赖于猜测，可能非常容易出错。对于一个正常工作的系统，应该明确地设置这些变量。

提示：如果希望完全禁止桌面环境通过 IMF 处理输入（在 GNOME 中不推荐，因为其与 IBus 紧密集成），那么可以不设置这些变量，或者，对于 GTK 和 Qt，可以替换它们的值：`GTK_IM_MODULE=gtk-im-context-simple` 和 `QT_IM_MODULE=simple`。

Fcitx5

有关详细信息，请参阅 Fcitx5#集成。

Fcitx

有关详细信息，请参阅 Fcitx。

```
GTK_IM_MODULE=fcitx
QT_IM_MODULE=fcitx
XMODIFIERS=@im=fcitx
```

IBus

见 IBus 以获得更多讯息。

```
GTK_IM_MODULE=ibus
QT_IM_MODULE=ibus
XMODIFIERS=@im=ibus
```

Uim

见 Uim 以获得更多讯息。

```
GTK_IM_MODULE=uim
QT_IM_MODULE=uim
XMODIFIERS=@im=uim
```

Emacs

**本文或本章节的事实准确性存在争议。**

原因： 这需要确认（在 Talk:输入法 中讨论）

根据这个 Fcitx wiki 所述, *"（`LC_CTYPE`）仅适用于一些情况下，包括emacs和java。Emacs 有一个积年的Bug，在 en_US.UTF-8 或类似的区域设置下，它将从不使用 XIM（虽然emacs作为一个 gtk 程序，它应使用 XIM）。使用 `LC_CTYPE` 去修复它是唯一的解决方法。*

Scim

见Scim获得更多讯息。

```
GTK_IM_MODULE=scim
QT_IM_MODULE=scim
XMODIFIERS=@im=scim
```

Xim

```
GTK_IM_MODULE=xim
QT_IM_MODULE=xim
```

参见

- Fedora:I18N/InputMethods
- Gentoo:Input methods
- Wikipedia:Input method

9.6 Fcitx

难度 高级 | **篇幅** 26.0 KB | **所属** 输入法

本节将讲到

- 安装
- 使用
- 配置
- 提示与技巧
- 故障排除

Fcitx，即小企鹅输入法，是一个以 GPL 许可发布的输入法平台,可以通过安装引擎支持多种输入法，支持简入繁出，是在 Linux 操作系统中常用的中文输入法。它的优点是：短小精悍、与各种程序的兼容性比较好。

警告：本页面是关于上游仓库已归档、不支持 Wayland 的 Fcitx 3.x-4.x 版本。推荐使用正被积极维护的、支持 Wayland 的 Fcitx5。在社区提问时，也请提前声明您使用的版本。

安装

安装软件包 fcitxAUR。

输入法引擎

Fcitx 内置的输入法支持中文拼音和基于字符表的输入（例如五笔）。根据语言的不同，可以选择其他输入法引擎。

中文

在 Fcitx 支持的拼音输入法中，内置拼音响应速度最快。Fcitx 同样支持流行的第三方拼音输入法以提供更好的整句输入效果。

- fcitx-sunpinyinAUR 在输入速度和输入精度之间有较好的平衡。
- fcitx-libpinyinAUR 算法比 sunpinyin 先进。
- fcitx-rimeAUR, 即著名中文输入法 Rime IME的 Fcitx 版本。但它不支持 Fcitx 本身的 #特殊符号和 #快速输入功能，自定义设置请参见官方,
- fcitx-googlepinyinAUR, Google 拼音输入法 for Android。
- fcitx-sogoupinyinAUR, 搜狗输入法for linux——支持全拼、简拼、模糊音、云输入、皮肤、中英混输入。官方网址

- fcitx-cloudpinyinAUR 可以提供云拼音输入的支持，支持 Fcitx 下的所有拼音输入法，Fcitx-rime 除外。
- fcitx-chewingAUR 为 Fcitx 添加 chewing (繁体中文注音) 输入引擎支持。依赖 libchewing包。
- fcitx-baidupinyinAUR, 百度拼音输入法的fcitx wrapper。
- fcitx-table-extraAUR Fcitx 的一些额外码表支持，包括仓颉 3, 仓颉 5, 粤拼, 速成, 五笔, 郑码等等。

日文

- fcitx-mozcAUR, 为 Fcitx 添加 mozc (日语) 输入引擎支持，mozc 是 Google 日语输入法的开源版本。
- fcitx-kkcAUR, 一款新的基于 libkkc包 的日文假名与汉字输入引擎。
- fcitx-skkAUR, 一款基于 libskk包 的日文假名与汉字输入引擎。
- fcitx-anthyAUR, 为 Fcitx 添加 anthy (日语) 输入引擎支持。

其它语言

- fcitx-hangulAUR, 为 Fcitx 添加 hangul (韩语) 输入引擎支持。
- fcitx-m17nAUR, 为 Fcitx 添加 m17n (多国语言码表) 输入引擎支持。
- fcitx-unikeyAUR, 为 Fcitx 添加 unikey (越南语) 输入引擎支持。
- fcitx-sayuraAUR, 为 Fcitx 添加 sayura （僧伽罗语） 输入引擎支持。

输入法模块

Fcitx 提供对 Qt 提供了输入法模块，请根据需要安装 fcitx-qt4AUR、fcitx-qt5AUR 和 fcitx-qt6AUR。如果没有安装对应的模块，一般还是可以在大部分程序中使用输入法的。不过很可能出现从无法光标跟随、无法显示预编辑字符串、无法输入甚至程序卡死等情况。

某些程序不使用 Qt 的输入法模块，这些程序包括：

- 使用 Tk, motif, 甚至 xlib 的程序
- Emacs、Opera、OpenOffice、LibreOffice、Skype、Wine、Java、Xterm、urxvt、WPS

其它

- fcitx-ui-lightAUR, Fcitx 的轻量 UI。
- fcitx-table-extraAUR adds Cangjie, Zhengma, Boshiamy support.
- fcitx-table-otherAUR, Fcitx 的一些更奇怪的码表支持，包括 Latex, Emoji, 以及一大堆不明字符等等。

- #GUI configuration tools

您还可以在 Arch User Repository 找到更多以上软件包的 Git 版以及其它。

使用

桌面环境下自动启动

如果您用 XDG 兼容的桌面环境，比如 KDE, GNOME, Xfce, LXDE, 那么当您安装好 Fcitx 并重新登录后，Fcitx 应该会自动启动。如果没有的话，可以打开控制台并运行 `fcitx`。为检验 Fcitx 是否正常运行, 打开一个程序，按 `Ctrl+Space` (这是切换输入法的默认快捷键）激活 Fcitx 并试着输入几个字。

如果 Fcitx 没有随桌面环境自动启动，或者您想修改下 Fcitx 启动参数，请用桌面环境提供的自动启动工具autostart配置，或者直接编辑用户目录 `~/.config/autostart/` 下的 `fcitx-autostart.desktop` 文件以确认自动启动是否被禁用。如果用户目录下的文件并不存在，您可以复制自动启动文件 `/etc/xdg/autostart/fcitx-autostart.desktop` 到用户目录。

如果您使用的桌面环境并不自动支持 XDG 自动启动, 请在您使用的启动脚本里面添加 `fcitx` 以实现自动启动。

例如，你使用i3或sway窗口管理器，可以在配置文件中添加`exec --no-startup-id fcitx -d`来自动启动输入法。对于 fcitx5 用户则应添加如下 `exec --no-startup-id fcitx5 -d`。

当 iBus 等其它输入法程序同时启动且开启了 Xim 支持时, 可能导致 Fcitx 无法启动，请确保已禁用了其它输入法程序的自动启动。

同时，请设置以下环境变量，使 GTK/Qt 应用程序优先使用输入法模块（IM module）。

模板错误： 您在尝试使用 = 标志吗？ 有关解决方法，请访问 Help:Template#Escape template-breaking characters。

如果没有设置这些环境变量，应用程序可能会回退到 XIM 协议。Qt5 应用程序例外，因为 Qt5 不支持 XIM 协议，必须使用输入法模块。

设置输入法的环境变量

请按以下方式设置环境变量，如果没有这些环境变量，程序可能默认使用 XIM 协议。qt5 程序不支持 XIM 所以必须配置使用 IM 模块，其它程序也有可能出现问题。

建议通过 `/etc/environment` 设置环境变量，pam-env 模块会在所有登录会话中读取此文件，包括 X11 会话和 Wayland 会话。详情请参考 `man 8 pam_env`。

```
/etc/environment
GTK_IM_MODULE=fcitx
QT_IM_MODULE=fcitx
XMODIFIERS=@im=fcitx
```

注意：pam-env 已不再读取 `~/.pam_environment` 文件。

部分登录管理器（如 LightDM）也支持读取 `~/.xprofile`，可以这样设置：

```
~/.xprofile
export GTK_IM_MODULE=fcitx
export QT_IM_MODULE=fcitx
export XMODIFIERS=@im=fcitx
```

重新登录后让环境变量生效。

如果 *fcitx* 没有自动启动，请将 `fcitx &` 加入 `~/.xinitrc`, 如果 `fcitx &` 不启动，在后面加一个延时 `sleep 2`。

- 请不要在 `.bashrc` 设置这些环境变量。`bashrc`只应用于交互性 bash 会话的初始化，并不应用于非交互性脚本或 X 会话的初始化。
- 如果 Qt 使用 fcitx 有问题，请执行 qtconfig (qtconfig-qt4)，在输入法配置中将 fcitx 设置为 "Default Input Method"。

Xim

要在 Gtk/Qt 程序中用 xim, 请将上面的变量改成如下的值：

```
GTK_IM_MODULE=xim
QT_IM_MODULE=xim
```

如果您在 `~/.xprofile` 或 `~/.xinitrc` 中修改这些变量，请添加 `export`。

警告：使用 XIM 时，很可能会遇到一些 Fcitx 无法解决的问题，例如无法输入、无法跟随光标显示候选框、重启输入法时应用程序卡死等。官方也不支持解决这些问题。这些问题同样存在于其它输入法框架中，因此请尽可能使用 GTK 和 Qt 输入法模块，而不是 XIM。

重新登录后让环境变量生效。

注意：

- Gtk2 从 2.24.20 开始，使用 `/usr/lib/gtk-2.0/2.10.0/immodules.cache` 作为缓存文件。如果设置了 `GTK_IM_MODULE_FILE` 环境变量或在安装后修改了配置，请清掉环境变量并使用 `/usr/bin/gtk-query-immodules-2.0 --update-cache` 更新缓存。
- Qt5 程序不支持 XIM 所以必须配置使用 IM 模块

配置

GUI 配置工具

Fcitx 提供了若干图形界面的配置程序：KDE 中的 kcm-fcitxAUR, 基于 GTK+3 的 fcitx-configtoolAUR。

在终端运行 *fcitx-config-gtk3* 打开已安装的配置工具 fcitx-configtoolAUR。 如果您希望启用其它语言的输入法，请取消勾选 *仅显示当前语言*。

如果要手工编辑 fcitx 的配置文件，请确保系统中并没有在运行 fcitx，否则编辑的配置内容可能丢失。

为了启用拼写检查功能，请先切换到由 `fcitx-keyboard` 提供的输入法，然后按下 `Ctrl+Alt+h`。

输入法配置

您可以在配置界面中添加或移除已启用的输入法。右键点击输入法图标即可打开对应配置。请注意，搜索功能区分大小写。

第一个设置的输入法处于未激活状态，其余所有输入法均处于激活状态。

通常情况下，建议将未激活状态设置为某个 *Keyboard* 选项（例如 *Keyboard - English (US)*）。这些选项仅根据名称中指定的键盘布局进行输入。

在*全局配置*中，*触发输入法*快捷键仅在未激活输入法及最后一个活动输入法之间切换。*切换输入法*快捷键默认仅在不同的活动输入法间切换，但是在高级设置中可以将非激活输入法加入切换选择。*切换输入法*快捷键需要按顺序按下，例如 `ALT_SHIFT` 仅在先按 `alt` 再按 `shift` 后生效。 可以通过将键盘设置为所需的 IME，然后右键点击托盘图标来访问对应 IME 的配置选项。

更改默认界面

Fcitx 支持 kimpanel 协议，以提供更好的桌面集成体验。

- Gnome-Shell：您可以从 extensions.gnome.org 安装 kimpanel，或者安装 gnome-shell-extension-kimpanel-gitAUR，它能够提供类似于 ibus-gjs 的用户体验。

扩充内置拼音词库

用户配置拼音词库在 `~/.config/fcitx/pinyin`, 其中 `pybase.mb` 为拼音单字库，`pyphrase.mb` 为拼音词库。如果这两文件并不存在，直接将您下载的词库放置到 `/usr/share/fcitx/pinyin`. 重启 Fcitx 即可。

皮肤

下载皮肤并解压缩到下面任一目录，如果没有可以新建目录：

- `/usr/share/fcitx/skin` 全局设置
- `~/.config/fcitx/skin` 特定用户设置

云拼音

安装 fcitx-cloudpinyinAUR 后重启 fcitx 即可。在配置程序里却没有看见云拼音，记得勾上“高级”复选框。所选的云拼音输入结果会自动添加到当前输入法的词库中。

建议在fcitx设置里面将“云拼音来源”由Google改为“百度”，Google国内访问不是很顺畅。

启用云拼音后，从云拼音获得的候选词会默认添加到候选词列表中的第二个，显示位置可以通过云拼音的设置配置。如果云拼音的结果和本地输入法给出的结果一致，云拼音后选项会和本地产生的候选项自动合并，不会产生重复的候选项。

注意：不推荐将云拼音候选词设为第一个候选词，因为当网络情况不好，没有及时返回云拼音结果，那么云拼音结果将默认降到第二候选词的位置，于是这个过程可能会涉及到默认候选词的改变。

提示与技巧

快捷键

部分常用默认快捷键：

- `Ctrl+Space` 激活输入法
- 左 `Shift` 临时切换到英文
- `Ctrl+Shift` 输入法间切换
- -/= 向前/向后翻页
- `Shift+Space` 全角、半角切换

注意：您可以在配置界面的全局配置中修改这些快捷键。

Vim

如果您经常在 Vim 下使用 Fcitx, 可以安装 vim-fcitxAUR 插件，或者在 `~/.vimrc` 添加如下代码。以退出插入模式时，自动关闭 Fcitx, 反之则反：

```
"##### auto fcitx  ###########
let g:input_toggle = 1
function! Fcitx2en()
   let s:input_status = system("fcitx-remote")
   if s:input_status == 2
      let g:input_toggle = 1
      let l:a = system("fcitx-remote -c")
   endif
endfunction

function! Fcitx2zh()
   let s:input_status = system("fcitx-remote")
   if s:input_status != 2 && g:input_toggle == 1
      let l:a = system("fcitx-remote -o")
      let g:input_toggle = 0
   endif
endfunction

set ttimeoutlen=150
```

```
"退出插入模式
autocmd InsertLeave * call Fcitx2en()
"进入插入模式
autocmd InsertEnter * call Fcitx2zh()
"##### auto fcitx end ######
```

注意：由于要调用外部程序，这将明显拖慢会反复进出插入模式的映射。建议改写相关映射，用带 Python 支持的 Vim 加以配合 fcitx.vim 亦可改善效率。

剪贴板

小小功能拯救世界！

Fcitx 自带剪贴板，其快捷键为 `Ctrl + ;`，这是一个非常实用的小功能。您可以在剪贴板插件配置页面中修改触发快捷键以及其它选项。

注意：这并不是一个剪贴板管理器。它不会像剪贴板管理器那样保存选择内容或修改剪贴板内容。它只能用于从剪贴板输入内容。

警告：某些客户端不支持多行输入，因此使用 fcitx-clipboard 粘贴多行剪贴板内容时，可能会发现多行内容被粘贴为单行。这可能是正在使用的程序的缺陷或特性，并非 Fcitx 能够解决的问题。

特殊符号

创建 `~/.config/fcitx/data/pySym.mb`, 文件内容示范如下：

```
#第一个字符为"#"的行是注释
#格式: 编码 符号
#编码只能为小写字母，经拼音解析后最长为10(如py为2，pinyin也为2)
#数学符号
sxfh ＋
sxfh －
sxfh ＜
sxfh ＝
sxfh ＞
sxfh ±
sxfh ×
sxfh ÷
sxfh ∈
sxfh ∏
sxfh ∑
sxfh ∕
sxfh √
sxfh ∝
```

直接输入某编码，可以匹配出对应的特殊符号。

注意：编码只能用二十六个小写字母表示；以 v 开头，无效。

快速输入

确保在 `~/.config/fcitx/config` 里把 `SemiColonAction` 修改为 `QuickPhrase`。

创建 `~/.config/fcitx/data/QuickPhrase.mb`, 文件内容示范如下：

```
#第一个字符为"#"的行是注释
```

```
#格式: 编码 符号
dianhua 123456789
youbian 123456
dizhi 中华人民共和国北京市长安街一号
aowu ┗<(=｀O′=)>┛
mobai m<(__ __)>m
baobao <(='▽')爻 (`▽`=)>
baobao <(=*'д`)爻('д`*=)>
qiangbi ▄︻┻┳═一…… ☆<(= ￣□￣ =!)>
xiaoku 😊
canyue ❍
xuehua ❄
```

按 ; 并输入编码，可实现快速输入，自然也能用来当颜文字库和emoji表情库。

注意：编码除了不得有空格，不得以 ; 开头之外，没有其它限制。

fcitx-remote

fcitx-remote 是可以控制 fcitx 状态的命令行工具，位于 fcitxAUR 安装包中。

常见的应用包括用 `fcitx-remote -s *imname*`, 将输入法切换到 `*imname*`。要获得正确的 `*imname*`，执行 *fcitx-diagnose* 并查看 "## Input Methods:" 对应的输出。

通过 Fcitx 输入特殊字符

参见：Fcitx5#通过 Fcitx5 输入特殊字符。该内容同样适用于 Fcitx

故障排除

停用或修改 Extra key for trigger input method \[sic\]

此设置位于 *Global Config* 选项卡中，默认值为 *SHIFT Both*，表示按下任意一个 Shift 键都会立即切换输入法。

虽然该设置本应只在单独按下某个 Shift 键时生效，但在使用标准键盘输入时，它可能会随机干扰大写字母输入、使用键盘选择文本等操作。

此外，该设置可能会在不提示的情况下随时恢复为默认值。若要确保 Fcitx 配置不会被修改，必须将该文件设置为不可变：

```
chattr +i ~/.config/fcitx/config
```

（需要以 root 用户执行。）

首先诊断问题所在

当你遇到任何 fcitx 有关的问题，比如 `Ctrl + Space` 在有的程序中不能工作，首先应该用 `fcitx-diagnose` 命令诊断问题的原因。 `fcitx-diagnose` 会列出所有 fcitx 正常运行所需的前提条件，从输出结果中通常可以找到问题的原因。 在网上（比

如在 irc 或者论坛里）询问别人关于 fcitx 配置的问题时，也请首先提供你的 `fcitx-diagnose` 输出结果（比如贴到 pastebin 服务），这将加速别人帮你找到问题所在。

Emacs

如果您的 `LC_CTYPE` 设置为英文，可能会由于 Emacs 的一个旧 bug 而无法在 Emacs 中使用输入法。您可以在启动 Emacs 前将 `LC_CTYPE` 设置为其它值，例如 `zh_CN.UTF-8`，以避免此问题。

请注意，系统中必须已经生成对应的 locale。

默认字体集合会在 `src/xfns.c` 中使用 `\-\*-\*-\*-r-normal--14-\*-\*-\*-\*-\*-\*-\*'` 作为基础字体。如果系统中没有匹配的字体（例如 `terminus` 或 `75dpi` 字体，可以通过 `\xlsfonts\`` 查看输出），则无法启用 XIM。根据 FAQ 和 字体 页面，可能需要安装 xorg-fonts-misc-otbAUR，因为 xorg-fonts-misc包 已不再提供所需的字体集合。

Emacs Daemon

如果您使用 Emacs daemon/client 模式，则应在启动 daemon 时设置 `LC_CTYPE`。例如：

```
LC_CTYPE=zh_CN.UTF-8 emacs --daemon
```

如果通过 systemd 启动 Emacs daemon，请在 unit 文件中设置：

```
Environment="LC_CTYPE=zh_CN.UTF-8" "XMODIFIERS=@im=fcitx"
```

（这里可能需要显式设置 `XMODIFIERS`，因为 systemd 不会加载 `.xprofile`。可以检查 Emacs 中的 `initial-environment` 变量，以确认这两个变量是否正确设置。）

在 gnome-terminal 中 `Ctrl + Space` 无法调出输入法

这个问题有时会发生，尤其是在 locale 设置为英文时。请确保您的 `GTK_IM_MODULE` 已正确设置。

另请参阅：FAQ

如果您已经将 `*_IM_MODULE` 环境变量设置为 fcitx，但仍然无法激活 Fcitx，请检查是否安装了对应的输入法模块。

某些程序只能使用 xim。如果您正在使用这些程序，请确保 `XMODIFIERS` 已正确设置，并注意可能遇到的问题。这些程序包括所有不使用 GTK 或 Qt 的程序（例如直接使用 tk、motif 或 xlib 的程序），以及 Emacs、Opera、OpenOffice、LibreOffice。

如果您在 Gnome 环境下无法在 *gnome-terminal* 中启用 Fcitx，并且上述方法无效，可以尝试：

```
$ gsettings set org.gnome.settings-daemon.plugins.xsettings overrides "{'Gtk/IMModule':<'fcitx'>}"
```

Ctrl + ；调出 Fcitx 剪贴板

严格来说，这并不是一个 BUG。Fcitx 的 `Ctrl + ；` 快捷键会覆盖许多用户自定义的快捷键，尤其是 Emacs 用户的快捷键。如果有需要，可以在配置界面中禁用剪贴板插件，或者修改其激活快捷键。

fcitx-sogoupinyin 卡死、无法联想

如果您遇到以下问题：

- 输入类似“安装”、“暗影”等以 "a" 开头的词语时，Fcitx 出现卡死。
- 输入并非以拼音 "a" 开头的词语时，却出现“阿拉伯”、“阿里巴巴”等以 "a" 开头的错误联想词。

可以通过删除 `~/.config/fcitx/sogou` 目录下的所有内容来解决。

注意：此操作会清空用户词库。

在某些程序中输入法总是切换到美语键盘

例如，在 XMind 中，当按下 Enter 创建新节点时，输入法会自动切换到美语键盘，需要再次按 `Ctrl + Space` 才能切回中文输入法。

启动 Fcitx 配置工具，在 *Global Config* 选项卡中的 *Share State Among Window* 选项里选择 *PerProgram* 或 *All* 即可解决。

Buildin Chinese Pinyin Default NOT ACTIVE

如果您的 locale 为 `en_US.UTF-8`，Fcitx 默认不会启用内置中文拼音输入法，而只会启用 `fcitx-keyboard-us` 输入法。

您可以通过 `fcitx-diagnose` 命令看到类似如下提示：

```
   ## Input Methods:
       1.  Found 1 enabled input methods:
               fcitx-keyboard-us
       2.  Default input methods:
           **You only have one input method enabled, please add a keyboard input method as the first one and your main input method as the second one.**
```

此时，您应该通过图形配置工具将 `Pinyin` 或 `Shuangpin` 输入法添加到已启用的输入法列表中。

fcitx 和 KDE

由于一些原因，KDE 对键盘布局的处理并不完善。例如，当您从 US（英语）切换到 LT（立陶宛语）时，键盘上的数字键本应输出立陶宛语字母，但实际仍然输出数字。

可以通过以下步骤解决：

1. 如果 Fcitx 正在后台运行，请先关闭它。
2. 禁用 KDE 相关设置：
3. 在 *System Settings* > *Input Devices* > *Layouts（选项卡）* 中，确保取消勾选 *Configure layouts*。
4. 在 *System Settings* > *Input Devices* > *Advanced（选项卡）* 中，确保取消勾选 *Configure keyboard options*。
5. 启动 Fcitx。此时可以关闭终端，Fcitx 仍会继续在后台运行。
6. 设置所需的键盘布局（右键点击系统托盘图标，然后选择 *Configure*）。
7. 右键点击系统托盘图标，然后选择 *Exit*。

完成后，键盘布局应该可以正常工作，KDE 原生的布局切换图标也会出现，并且可以通过鼠标滚轮或点击进行切换。

输入法意外切换到英文

例如，在 XMind 中，当用户按下 `Enter` 创建节点时，输入法总是会切换到英文，需要手动切回中文输入法。

要解决此问题，请打开 Fcitx 图形配置工具（由 fcitx-configtoolAUR 提供），切换到 *Global Config* 选项卡，在 *Share State Among Window* 下拉菜单中选择 *PerProgram* 或 *All*。

xmodmap 设置被覆盖

Fcitx 会控制键盘布局，因此您的 xmodmap 设置可能会被覆盖。

从 4.2.7 版本开始，Fcitx 会尝试加载存在的 `~/.Xmodmap` 文件。

有关如何保存 xmodmap 修改的更多信息，请参阅 FAQ。

Fcitx 和 setxkbmap

如果您使用 *setxkbmap* 设置其它语言的键盘布局，那么根据上述情况，Fcitx 可能会覆盖这些设置，甚至导致某些应用程序（例如 QuteBrowser）无法切换到对应语言。

如果您希望避免这种情况，并独立管理自己的 xmodmap 键盘布局，请按照以下步骤操作：

1. 打开 Fcitx 图形配置工具（由 fcitx-configtoolAUR 提供）。
2. 切换到 *Addon* 选项卡。
3. 选择 *Advanced*。
4. 搜索 *xkb*。
5. 选择 *X Keyboards Integration*。
6. 点击 *Configure*。
7. 取消勾选 *Allow to Override System XKB Settings*。
8. 点击 *OK*。

HiDPI 下 Fcitx 预编辑框过小

参见 HiDPI#Fcitx。

See also

- Fcitx GitLab
- Fcitx Wiki

9.7 Fcitx 5

难度 高级 | **篇幅** 31.5 KB | **所属** 输入法

本节将讲到

- 安装
- 配置
- 故障处理
- 提示和技巧
- 参考来源

注意：本页面**不是**英文页面的翻译。请勿不加选择地将英文页面同步至本页面。

Fcitx5 是一个输入法框架，它具有轻量级内核，并通过 addon 提供各种语言和输入法的支持。Fcitx5 接替 Fcitx。

安装

安装 fcitx5包 软件包。

fcitx5-im包组 包组提供 fcitx5包 本体、#配置工具和#输入法模块。

注意：fcitx5包 仅提供基本框架，且仅支持英文。如果要输入其他语言（例如中文或日文），则需要安装输入法引擎（IME）。

中文

- fcitx5-chinese-addons包 包含与中文相关的 addon，例如拼音、双拼和五笔。

- fcitx5-chewing包 是流行的繁体中文注音输入引擎，它基于 libchewing包。
- fcitx5-rime包 使用 Rime 引擎。
- fcitx5-table-extra包 包含倉頡、嘸蝦米等輸入法。
- fcitx5-mcbopomofo-gitAUR小麥注音的Linux版， McBopomofo 支持。
- rime-flypyAUR 小鹤音形支持。

日文

- fcitx5-anthy包 是流行的日文输入引擎。但是，它已不再受到积极开发。
- fcitx5-kkc包 是日文假名输入引擎，它基于 libkkc包。
- fcitx5-mozc包 基于 Mozc（英语：Mozc）（Google 日文输入法的开源版本）。
- fcitx5-skk包 是日文假名输入引擎，它基于 libskk包。

其他语言

- fcitx5-hangul包 用于输入韩文，基于 libhangul包。
- fcitx5-unikey包 或 fcitx5-bamboo-gitAUR 可用于输入越南语字符。
- fcitx5-m17n包 或 fcitx5-table-other包 可用于各种语言。使用后者输入 IPA（X-SAMPA）。

输入法模块

一些 GUI 工具包提供了输入法模块支持，这些模块在 X11 下能提供更好的体验，但是在 Wayland 下则经常相反。因此 X11 桌面的用户建议安装，而 Wayland 桌面的用户则根据 Wayland 混成器的支持情况按需安装，详见 #Wayland。

- 对于 Qt5/6 程序，安装 fcitx5-qt包。
- 对于 GTK 程序，安装 fcitx5-gtk包。
- 对于 Qt4 程序，安装 fcitx5-qt4-gitAUR（通常不需要）。

配置

X11（以及 Xwayland）和 Wayland 环境的配置方式并不相同。

Fcitx5 支持多种与要使用输入法的应用程序进行通信的方式：

- Wayland 输入法协议
- input-method-v1: 用于 KDE 和 Wayfire 等 Wayland 混成器。
- input-method-v2: 用于 Sway 等 Wayland 混成器。
- GNOME Shell: 用于与 GNOME Wayland 桌面环境。它是通过 D-Bus 进行的。

- 输入法模块。这些通常用于 X11 环境，或者尚未良好支持 Wayland 输入法协议的应用程序。它是通过 D-Bus 进行的。需要额外注意的是，当在原生 Wayland 模式下使用输入法模块时，为了将输入法窗口放置到正确的位置，输入法窗口是由使用输入法的应用程序自己绘制的，因此不同的应用程序的状况不太一样。比如 GTK3 的输入法窗口可能会闪烁，Qt 的输入法窗口可能无法显示部分 GTK 能显示的文字。
- fcitx: 用于 GTK、Qt 的输入法模块。
- fcitx5: 同上，但是不兼容 fcitx4 的输入法模块。不建议使用。
- ibus: 兼容 IBus 的协议。
- XIM: 用于其它 X11 应用程序。有一些缺点，建议尽量不用。

X11

X11 下，输入法可使用 GTK 和 Qt 的输入法模块通过 D-Bus 与输入法通信。某些非 GTK 和 Qt 的应用程序（或者框架）也实现了 fcitx 或者 ibus 的 D-Bus 通信协议。其它程序可以通过 X11 的 XIM 协议进行通信。

为了告诉程序使用 fcitx5 输入法，需要设置相应的环境变量。编辑 `/etc/environment` 并添加以下几行，然后重新登录：

```
GTK_IM_MODULE=fcitx
QT_IM_MODULE=fcitx
XMODIFIERS=@im=fcitx
SDL_IM_MODULE=fcitx
GLFW_IM_MODULE=ibus
```

如果使用 en_US.UTF-8 时，遇到 GTK2 无法激活 fcitx5，可专门为该 GTK2 应用程序设置输入法为 xim，如

```
$ env GTK_IM_MODULE=xim <your_gtk2_application>
```

请勿将 `GTK_IM_MODULE` 全局设置为 xim，因为它也会影响 GTK3 程序。XIM 有各种问题（比如输入法重启之后再无法输入），尽可能不要使用。

注意：

- SDL_IM_MODULE 是为了让一些使用特定版本 SDL2 库的游戏能正常使用输入法。
- GLFW_IM_MODULE 是为了让 kitty 启用输入法支持。此环境变量的值为ibus为了保证所有的kitty均可启用该功能，在基于x11的桌面环境和窗口管理器中此属性设置为fcitx5或fcitx会导致无法启用输入法设置，wayland桌面不受此影响。
- 按 fcitx5 上游推荐，环境变量的值设置为`fcitx`。部分并非由 Arch 从源码编译打包的应用程序因兼容性的需求而需要将之设置为`fcitx5`。

Wayland

不同Wayland混成器的情况有所不同。

共通设置

为了支持运行于Xwayland的GTK软件，在`~/.config/gtk-3.0/settings.ini`中添加：

```
[Settings]
gtk-im-module = fcitx
```

警告：请勿设置`GTK_IM_MODULE`环境变量。

为了支持Qt5和运行于Xwayland的Qt软件，设置以下环境变量：

```
QT_IM_MODULES=wayland;fcitx
QT_IM_MODULE=fcitx
```

为了支持运行于Xwayland 的其他软件，设置环境变量`XMODIFIERS=@im=fcitx`。

KDE Wayland

根据 Fcitx 5 Wiki，Plasma 5.27+ 支持 Wayland text-input-v1、text-input-v2 和 text-input-v3，可被 Chromium、Qt 和 GTK 使用。

要使用 Wayland 输入法协议，首先退出正在运行的 Fcitx 5 进程，前往*系统设置* > *输入与输出* > *键盘* > *虚拟键盘*，选择 *Fcitx 5 Wayland 启动器*。

如果重新登录之后无法使用输入法，尝试运行：

```
qdbus6 org.kde.KWin /VirtualKeyboard org.kde.kwin.VirtualKeyboard.enabled true
```

GNOME Wayland

安装 gnome-shell-extension-kimpanel-gitAUR 并启用 Kimpanel GNOME Shell 拓展。

根据 Fcitx 5 Wiki, GNOME 支持 Wayland text-input-v3 协议，大多数足够新的GUI库和软件应当能够开箱即用。

根据 Fcitx5 Wiki, 在GNOME Wayland 环境中的fcitx，可能出现弹出候选窗口无法在 gnome-shell UI 上显示。一个解决方案是使用Kimpanel扩展，此为扩展连接。

Wayfire

启用 `input-method-v1` 插件。然后运行 Fcitx5 即可。

Wayfire 支持 text-input-v1，但是不会显示输入法窗口。低版本的 Chromium / Electron 在原生 Wayland 模式下会使用这个。

Chromium 和基于 Chromium / Electron 的软件

除了各种基于 Chromium 内核的浏览器之外，这类软件还包括 VS Code、QQ、Discord、Element 等基于 Electron 的软件。请参阅 Chromium#原生 Wayland 上运行。

flatpak

flatpak 沙箱应用启动时不会读到`~/.config/gtk-3.0/settings.ini`，而是`~/.var/app/$APP_ID/config`（受 XDG_CONFIG_HOME 控制）。

你可以使用 Flatseal 为所有 flatpak 应用（global）设定 `GTK_IM_MODULE=fcitx` 环境变量，也可以添加允许读取 `xdg-config/gtk-3.0:ro`，这样就能读到主机配置文件了 。

随桌面环境自动启动

注意：

- 如果您使用的桌面环境是兼容 XDG 的（例如 KDE、GNOME、Xfce、LXDE 等），则无需此步骤。
- 如果使用 i3、awesome 等窗口管理器，需要在其脚本中添加 Fcitx5 以实现自启动。例如，如果您使用 i3 或 sway ,可以在配置文件中添加`exec --no-startup-id fcitx5 -d`
- 如果使用 dwm，则需要添加 autostart 补丁。在 `~/.dwm/autostart.sh` 中添加`fcitx5 -d`
- KDE Wayland 用户请使用「虚拟键盘」设置，请勿使用此方法。

想要 fcitx5包 开机自启，执行

```
$ cp /usr/share/applications/org.fcitx.Fcitx5.desktop ~/.config/autostart/
```

如果使用 xinit 启动，则可以在`~/.xinitrc`中，在`exec`前添加`fcitx5 &`

词库

对于 Fcitx5 的中文输入法, 目前在仓库里提供了数个词库：

- fcitx5-pinyin-zhwiki包：felixonmars 根据中文维基百科创建的词库。适用于**拼音输入法**
- rime-pinyin-zhwiki包：适用于 Rime 输入法的词库
- rime-pinyin-moegirlAUR：适用于 Rime 输入法的词库
- cedict:从 cedict辞典导出的词库。
- fcitx5-pinyin-moegirlAUR： outloudvi 根据萌娘百科创建的词库

注意：

- 词典文件存在的目录为 ~/.local/share/fcitx5/pinyin/dictionaries
- .dict类型的词典文件可以直接手动移动到词典目录，使用搜狗细胞词库下载的.scel文件无法被fcitx识别
- 可在fcitx5-configtool中选择导入来自搜狗细胞词库文件，或使用scel2org5指令手动转换

自定义词库

一般而言,由于 fcitx5包 支持导入搜狗词库，因此很大程度上不需要自定义词库，但是 fcitx5包 依然提供了相关工具。

- 安装 libime包

原始词库文件是一个文本文件，其格式为： 汉字 拼音 频率

在得到原始词库文件后，调用 `libime_pinyindict "词库文件.txt" "词库文件.dict"` 即可。

自定义词库文件放置在 `~/.local/share/fcitx5/pinyin/dictionaries`

注意：以下内容或许能提供帮助：

- 汉字转拼音
- 简繁转换

配置工具

fcitx5包 的配置文件位于 `~/.config/fcitx5`，尽管您可以使用文本编辑器编辑配置文件，但是使用 GUI 配置显然更方便。安装 fcitx5-configtool包 软件包并运行 `fcitx5-configtool`，或者从输入法托盘的菜单里选择「配置」。

KDE 用户请安装 fcitx5-configtool包 后在 KDE 的「设置」程序里找到并设置。

主题和外观

主题

仓库内的主题数量有限，如果需要更多主题，可以去 GitHub 发现更多主题。

- fcitx5-breeze包：提供了与KDE默认的Breeze主题匹配的外观。
- fcitx5-nord包 ：Nord颜色的主题
- fcitx5-material-color包：提供了类似微软拼音的外观。
- fcitx5-solarizedAUR：Solarized颜色主题
- fcitx5-skin-fluentdark-gitAUR：具有模糊效果和阴影的 Fluent-Design 深色主题

然后前往 `Fcitx5设置 -> 配置附加组件 -> 经典用户界面 -> 主题` 设置主题。

注意：如果您在 GNOME 环境下使用了 gnome-shell-extension-kimpanel-gitAUR，那么主题设置对于 Fcitx5 不起作用。

设置单行模式

在拼音输入法（或者 Rime 输入法）的设置中，启用“**在程序中显示预编辑文本**”即可启用单行模式

故障处理

诊断问题

当你遇到任何 Fcitx 5 有关的问题，比如 `Ctrl+Space` 快捷键在有的程序中不能工作，首先应该用 `fcitx5-diagnose` 命令诊断问题的原因。 `fcitx5-diagnose` 会列出所有 Fcitx 5 正常运行所需的前提条件，从输出结果中通常可以找到问题的原因。

部分应用中 Fcitx 5 的单行默认无效

1\. 如果是 Firefox 等 gtk 应用中单行模式不生效，请安装 fcitx5-gtk包

2\. 在 WPS 和 Sublime 中单行模式无效，这是 WPS 和 Sublime 自身的问题，而不是 fcitx5包 的问题。，请参阅 WPS Office#Fcitx5 无法输入中文。

Fcitx5 在 JetBrains IDEs 不正常工作

请确保您的系统的 locale 是正确的并已生成，因为不正确的 locale 会导致Fcitx5 无法在 JetBrains 集成开发环境中正常工作。

emoji 在候选框中显示为空白

首先确保电脑上已经安装了支持 emoji 的字体（例如 noto-fonts-emoji包）。然后对该 emoji 字体禁用反锯齿。编辑 fontconfig 的配置文件并加入：

```
<match target="font">
  <test name="family" qual="first">
    <string>Noto Color Emoji</string>
  </test>
  <edit mode="assign" name="antialias">
    <bool>false</bool>
  </edit>
</match>
```

RStudio 中无法调出输入法

运行以下命令：

```
$ strings /usr/lib/rstudio/lib/libQt5Core.so.5 | grep "Qt 5"
```

找出 Qt 库的版本，使用该版本重新编译 fcitx5-qt包 中的 `libfcitx5platforminputcontextplugin.so`，再放到 `/usr/lib/rstudio/plugins/platforminputcontexts/` 目录中。

如果使用的是 rstudio-desktop-binAUR，可直接安装 rstudio-fcitx5AUR 。

在 Steam 和 Dota2 中无法使用

事实上，Fcitx5 能够在 Steam 大屏模式和 Dota2 中使用。但是需要使用 `Ctrl+Space` 激活输入法而不是 `Ctrl+Shift`，参阅。

如果遇到问题, 请尝试

```
XMODIFIERS=@im=fcitx LANG=zh_CN.UTF-8 steam
```

对于使用「虚拟键盘」启用 Fcitx5 的 KDE Plasma 用户，可能还需要为 Steam 设置 `GTK_IM_MODULE=fcitx` 环境变量。

输入法窗口在Wayland的Chromium中不显示

详见 Chromium#原生 Wayland 上运行.

在 kitty 中 Fcitx5 不可用

详见 kitty#输入法兼容。

Fcitx5 的 右Alt 在 Electron applications 不正常工作

如果应用程序使用的是非系统键盘（如 Discord、Element 等），则应用程序可能会在输入法处理 ISO_Level3_Shift 之前处理 ISO_Level3_Shift。这导致了一些输入法在个别应用各种中会发生错误。一个解决方法是添加另一个输入法组，将系统输入法布局设置成与键盘相对应的布局。例如，要在一个以英语为主要语言系统的QWERTY键盘上输入波兰字母 `ąćęłńóśźż`，你可以用 Fcitx5 的GUI配置工具：

1. 点击*添加组*加号按钮。
2. 在下拉菜单中选择该组，然后添加输入法（键盘，例如“‘键盘 - 波兰语’”）。
3. 使用“‘选择系统键盘布局’”选择与此输入法匹配的键盘布局，然后应用更改。

如果您需要其它解决方案，请参阅 Fcitx5 开发人员的评论。

使用 Wayland 原生协议的软件不显示输入法窗口

Sway

请确保您在使用 Sway 1.10 或更高的版本。

GNOME

安装 gnome-shell-extension-kimpanel-gitAUR 插件。

wayfire

请启用 `input-method-v1` 插件。

由于目前 Qt（6.8.x）默认会使用 text-input-v1（而非 text-input-v3）协议，Qt 应用程序在通过 Wayland 协议使用输入法时看不到输入法窗口。一个绕过的办法是使用 `QT_IM_MODULE=fcitx`。（这样会导致 fcitx 主题不能自适应亮暗色。）另一个办法是设置 `QT_WAYLAND_TEXT_INPUT_PROTOCOL=zwp_text_input_v3` 让 Qt 使用 text-input-v3，但有可能会遇到 bug。

Fcitx5 在一些应用程序中无法使用

如果你的 Fcitx5 在一些应用程序中无法使用（例如：AnkiQt6）检查是否有`~/.xprofile`文件。如果没有就创建一个包含Fcitx5环境变量的`~/.xprofile`文件。然后重启你的桌面会话。

在 Wine 中无法使用输入法

对于运行于Xorg或者Xwayland下的Wine，记得在环境变量中加入

```
XMODIFIERS=@im=fcitx
```

运行于Wayland原生模式下的Wine已经可以使用输入法，但可能不支持显示预编辑字符串。可按`Ctrl-Alt-P`快捷键来关闭预编辑功能，或者使用`FCITX_NO_PREEDIT_APPS`环境变量来对特定的程序禁用预编辑。

提示和技巧

自定义繁简中文转换

一些IME认为默认使用简体中文，就导致了当输出一些繁体中文时字符显示的不正常，例如当输入「為什麼」时被替换成了「爲什麼」。若要修复这个问题，可以自定义简体中文和繁体中文翻译的用法

为了配置转换功能，请将简体中文转换为繁体中文的 `OpenCC` 属性值设置为以下值之一：

- s2t - Simplified to Traditional (OpenCC) （这是默认值，也许不是你正在寻找的属性）
- s2tw - Simplified to Traditional (Taiwan)
- s2twp - Simplified to Traditional (Taiwan) 台湾惯用语
- s2hk - Simplified to Traditional (Hong Kong)

若要设置反转，请将繁体中文转换为简体中文的设置为以下值之一：

- t2s - Traditional (OpenCC) to Simplified (OpenCC) （这是默认值，也许不是你正在寻找的属性）
- tw2s - Traditional (Taiwan) to Simplified (OpenCC)
- tw2sp - Traditional (Taiwan) to Simplified (OpenCC) 大陆习惯用语
- t2hk - Traditional (OpenCC) to Hong Kong variant 香港变体
- t2tw - Traditional (OpenCC) to Taiwan Standard 台湾标准
- tw2t - Traditional (Taiwan) to Traditional (OpenCC)
- hk2s - Traditional (Hong Kong) to Simplified (OpenCC)
- hk2t - Traditional (Hong Kong) to Traditional Chinese (OpenCC)
- t2jp - Traditional (Kyūjitai) to New Japanese Kanji (Shinjitai)

- jp2t - New Japanese Kanji (Shinjitai) to Traditional (Kyūjitai)

最新列表在这：OpenCC。

查看选中字符的 Unicode 编码

- 如果你需要查看文本编辑器中选中文字的 Unicode 编码，那么直接选中文字，然后使用快捷键 `ctrl + alt + shift + u` 可以查看选中文字的编码。
- 如果你需要查看非编辑区域（比如本 wiki）中文字的 Unicode 编码，那么需要首先将该段文字复制到剪贴板，然后点击任意一个可编辑区域（比如搜索框），然后使用快捷键 `ctrl + alt + shift + u` 可以查看剪贴板中文字的编码。

通过 Fcitx5 输入特殊字符

可以使用 Fcitx5 的 Unicode 输入功能来输入特殊字符：

以 ① 为例：

将光标定位到任意一个输入框内，然后按下 `Ctrl + Alt + Shift + U`，然后输入 `circle one` 来搜索字符，你将会看到多种形式的 ①。使用 `Alt+数字` 来选择。如果你知道要输入字符的 Unicode 编码，也可以直接按 `Ctrl + Shift + U` 然后输入编码。这些快捷键可以在设置中更改。

如果你要经常输入某些特殊字符，可以考虑将其加入「快速输入」功能中。

切换半角和全角标点

对于 fcitx5-chinese-addons包，默认情况下使用全角标点，可以使用`Ctrl+.`快捷键在全角和半角标点间切换。

在 vim 中自动切换输入法

推荐使用 fcitx.vim 插件。此插件会为不同缓冲区保持各自的插入模式输入法状态。

有一个简单的方案，可以向 `~/.vimrc` 追加代码：

```
let fcitx5state = system("fcitx5-remote")
" 退出插入模式时禁用输入法，并保存状态
autocmd InsertLeave * :silent let fcitx5state=system("fcitx5-remote") | silent !call system("fcitx5-remote -c")
" 2 表示之前状态打开了输入法，则进入插入模式时启动输入法
autocmd InsertEnter * :silent if fcitx5state == 2 | call system("fcitx5-remote -o") | endif
```

如果你使用的是新的 Vim9 语法，追加的代码应该是：

```
# 仅在使用 vim9script 语法或带有 `vim9script` 的关键字后才会生效
var fcitx5state = system("fcitx5-remote")
autocmd InsertLeave * :silent fcitx5state = system("fcitx5-remote") | silent !fcitx5-remote -c
```

```
autocmd InsertEnter * :silent if fcitx5state == '2' | call system("fcitx5-remote -o") | endif
```

如果使用的是 neovim包 ，则追加上述代码到 `~/.config/nvim/init.vim`。

注意：如果您在 vim.cmd 中添加此代码，可能需要将注释去掉。

拼音输入法

注意：以下功能只对 fcitx5-chinese-addons包 中的拼音输入法有效，其他输入法请自行探索。

导入搜狗词库

- 对于 KDE 用户来说，可以通过 设置 -> 区域设置 -> 输入法 -> 拼音 -> 词典 -> 导入 来导入搜狗词库
- 对于使用 fcitx5-configtool包 的用户来说，需要手动打开 “Fcitx5 配置” 这个软件，并在拼音输入法中手动配置。

既可以导入本地词库也可以**在线浏览词库并自动导入**

云拼音

在拼音输入法的设置页面，你可以启用云拼音。但是如果你需要更改云拼音默认的后端，则需要在 fcitx5包 的全局设置里进行更改。提供的后端有 `Google`, `Baidu`, `GoogleCN`

笔画过滤

在设置的拼音输入法的“笔画过滤”后设置快捷键（默认为 ` ``）然后在输入文字后，按下快捷键，输入法的候选框将会出现**笔画过滤**字样，可以对词语进行笔画过滤，具体规则为：h 横、s 竖、p 撇、n 捺、z 折

默认情况下，笔画过滤是对一个句子的第一个字进行筛选，但是使用**以词定字**可以在一个句子之间的不同字之间进行切换。

例如对词语“中华人民共和国”中的第三个字进行笔画筛选，你可以在启用笔画过滤后连续按两次] 让 fcitx5包 对其进行笔画过滤。

注意：默认情况下，**以词定字**的快捷键为 [和]，该快捷键在**拼音输入法**的设置中可以查看

将输入错误的单词从输入历史中删除

输入过程中有可能因为误操作写入错误单词, 此时可以通过重新输入错误单词拼音 -> 使用忘记词汇快捷键(默认为 `Ctrl + 7`) -> 选择错误单词的对应数字删除.

快捷键可以由`Fcitx 配置 > "附加组件"标签页 > "输入法"部分 > "拼音"齿轮图标 > "忘记词汇" 快捷键`指定.

输入自定义时间

在按照前文安装 fcitx5-lua包之后, 在输入；进入快速输入模式后可使用如下指令:

- ；fh可输入特殊符号
- ；sj可输入当前时间, 格式默认提供3种: \[15:52\]，\[15时52分\]，\[十五时五十三分\]；如果输入；sj后按照时：分的格式输入，可以格式化指定时间。
- ；rq可输入当前日期, 格式默认提供3种: \[2023-07-21\]，\[2023年7月21日\]，\[二〇二三年七月二十一日\]；如果输入；rq后按照年-月-日的格式输入，可以格式化指定日期。

可通过修改pinyin.lua甚至imeapi.lua脚本扩展功能(注意备份).

RIME/中州韵

提示：所有更改皆需重新部署方可生效

导入词库

以导入词库rime-pinyin-zhwiki包和rime-pinyin-moegirlAUR为例.

提示：将自定义词库放入~/.local/share/fcitx5/rime/亦可,文件名(文件名.dict.yaml)应与词库名统一(词库格式)

1.更改~/.local/share/fcitx5/rime/luna_pinyin.custom.yaml文件(以luna_pinyin为例,其余输入方案修改方案名即可)

```
~/.local/share/fcitx5/rime/luna_pinyin.custom.yaml
# 文件中"patch:"应只存在一个,若已存在只需粘贴后面的代码
# 此文件用于修改特定输入方案,把上方的luna_pinyin改为其它输入方案名即可完成对其它输入方案的修改
patch:
    "translator/dictionary": extended #词典名字可自定义,与下方文件名保持一致即可
```

2.新建~/.local/share/fcitx5/rime/extended.dict.yaml文件

提示：导入自定义词库只需将词库名添加到"import_tables:"之后

```
~/.local/share/fcitx5/rime/extended.dict.yaml
# 以下禁用了默认词库同时不启用默认的"八股文"词库及词频系统,如果您不希望候选词中的出现繁体字、方框字的话
---
name: extended
version: "2021.02.19"
sort: by_weight
use_preset_vocabulary: false #是否启用默认的"八股文"词库及词频系统,如需启用请设为 true 。
import_tables:
  # - luna_pinyin #默认词库,如需启用请取消注释
  - zhwiki
  - moegirl
  # - 自定义词库名
...
```

模糊音设置

请根据需要注释(#)或删除不需要的模糊音,若需增加其它模糊音,请参考明月拼音模糊音定制模板

若`luna_pinyin.custom.yaml`文件不存在

```
~/.local/share/fcitx5/rime/luna_pinyin.custom.yaml
patch:
    "speller/algebra":
        - derive/^([zcs])h/$1/ #zh,ch,sh->z,c,s
        - derive/^([zcs])([^h])/$1h$2/ #z,c,s->zh,ch,sh
        - derive/^n/l #n->l
        - derive/^l/n #l->n
        - derive/([ei])n$/$1ng/  # en -> eng, in -> ing
        - derive/([ei])ng$/$1n/ # eng->en, ing -> in
        - abbrev/^([a-z]).+$/$1/ #简拼支持
        - abbrev/^([zcs]h).+$/$1/ #模糊音的简拼支持
        delimiter: " '" #分隔符
```

如果文件存在,则粘贴`patch:`以下的部分到文件末尾(`luna_pinyin.custom.yaml`中有且只有一个`patch:`)

特殊符号

注意：Fcitx5 已经内置了对特殊符号的支持。参阅#通过 Fcitx5 输入特殊字符

导入rime-dict项目中`symbols.dict.yaml`词库即可在拼音中输入希腊字母、部分数学符号及Emoji表情

示例:

希腊字母:输入`alpha`即可输出α

数学符号:输入`jifen`即可输出∫

特殊符号:输入`cha`可输出×,✕

序号:输入`qi`可输出Ⅶ,⑦

Emoji表情:输入`haha`可输出😊,😄

加载 librime-lua 插件

若想加载 librime-lua 插件，须在 fcitx 配置工具的 Rime 输入法设置中添加 `lua` 模块。

参考来源

1. ↑
2. ↑ https://github.com/WayfireWM/wayfire/issues/2315
3. ↑ https://www.zhihu.com/question/341748857/answer/1739052604

4. ↑ https://sur.moe/post/vim%E8%BE%93%E5%85%A5%E6%B3%95%E5%88%87%E6%8D%A2%E4%B8%8D%E4%BD%BF%E7%94%A8%E6%8F%92%E4%BB%B6/
5. ↑ fcitx5 使用自定义 lua 扩展输入当前日期和时间 - tedding's blog

第 10 章　共 6 节　57 KB

Freedesktop.org

换桌面后，文件放哪、由谁打开、登录时谁自启，都不随桌面走。本章讲 freedesktop.org 这一层：用户目录定路径，MIME 应用程序定打开方式，门户替受限程序要权限，另两节管菜单。

先读《XDG 用户目录》，路径不定，后面无处落脚；再读《XDG MIME 应用程序》与《XDG 桌面门户》，排查最细。《Xdg-utils》当速查，《Xdg-menu》只理菜单时用，《XDG Autostart》读目录约定即可。最易误解是把它们当某桌面的附属功能：规范跨桌面通用，换桌面才露馅；门户缺后端不报错，静默失效。

10.1　XDG Autostart

难度 入门 | **篇幅** 2.2 KB | **所属** Freedesktop.org

本节将讲到

- 先决条件
- 目录

XDG Autostart 规范定义了一种通过将其放置在特定#目录中来在桌面环境启动和可移动介质安装中自动启动普通桌面项的方法。

先决条件

您需要使用支持它的桌面环境或专用实现，例如：

- dex包
- dapperAUR
- fbautostartAUR
- sandsmark-xdg-autostart-launcher-gitAUR
- systemd-xdg-autostart-generator

目录

按优先顺序排列的自动启动目录为：

- 用户特定：`$XDG_CONFIG_HOME/autostart`（默认为 `~/.config/autostart`）
- 系统范围：`$XDG_CONFIG_DIRS/autostart`（默认为 `/etc/xdg/autostart`）

系统范围的桌面配置项可以由具有相同文件名的用户特定配置项覆盖。

提示：要禁用系统范围的条目，请创建一个包含 `Hidden=true` 的替代配置项。

有关更多详细信息，请阅读规范。

10.2 XDG 用户目录

难度 入门 | **篇幅** 4.7 KB | **所属** Freedesktop.org

本节将讲到

- 安装
- 创建默认目录
- 创建自定义目录
- 查询配置好的目录

译自 freedesktop.org：

: *xdg-user-dirs* 是一个帮助管理用户常用目录（如桌面文件夹和音乐文件夹）的工具。它还处理这些文件夹名称的本地化（即翻译）。

: 其工作原理是在登录阶段的早期运行 xdg-user-dirs-update(1) 程序。该程序会读取一个配置文件和一组默认目录，然后在用户的主目录中创建这些目录的本地化版本，并在 `$XDG_CONFIG_HOME/user-dirs.dirs`（`XDG_CONFIG_HOME` 默认为 `~/.config`）中设置一个配置文件，供应用程序读取以找到这些目录。

大多数文件管理器会使用特殊图标标识 XDG 用户目录。

安装

安装 xdg-user-dirs包。

注意：某些桌面环境可能已默认安装，例如 GNOME。

创建默认目录

可以用以下命令在 `$HOME` 下创建一整套默认的经本地化的用户目录：

```
$ xdg-user-dirs-update
```

提示：可以使用 `LC_ALL=C.UTF-8 xdg-user-dirs-update --force` 命令强制创建英文命名的目录。

运行后该命令还会自动地：

- 创建一个本地的 `~/.config/user-dirs.dirs` 配置文件：应用程序通过他来查找使用特定帐号指定的用户目录。
- 创建一个本地的 `~/.config/user-dirs.locale` 配置文件：根据使用的 locale 指定语言。

用户服务 `xdg-user-dirs.service` 也将默认安装并启用，以便在每次登录会话开始时运行此命令，保持您的目录是最新的。

创建自定义目录

本地 `~/.config/user-dirs.dirs` 和全局 `/etc/xdg/user-dirs.defaults` 配置文件使用以下环境变量格式指向用户目录：`XDG_DIRNAME_DIR="$HOME/directory_name"`。一个示例配置文件可能如下所示（这些都是默认目录）：

```
~/.config/user-dirs.dirs
XDG_DESKTOP_DIR="$HOME/桌面"
XDG_DOCUMENTS_DIR="$HOME/文档"
XDG_DOWNLOAD_DIR="$HOME/下载"
XDG_MUSIC_DIR="$HOME/音乐"
XDG_PICTURES_DIR="$HOME/图片"
XDG_PROJECTS_DIR="$HOME/项目"
XDG_PUBLICSHARE_DIR="$HOME/公共"
XDG_TEMPLATES_DIR="$HOME/模板"
XDG_VIDEOS_DIR="$HOME/视频"
```

因为 xdg-user-dirs包 会参照本地配置文件来了解正确的用户目录，所以可以自定义。比如若将 `~/.config/user-dirs.dirs` 下 `XDG_DOWNLOAD_DIR` 变量设为了 `$HOME/Internet`，那么任何参照了该变量的程序都会改用这个目录。

注意：和其他的配置文件一样，本地设定覆盖全局设定。另外自定义的目录要自己创建。

或者也可以用命令行修改默认目录。下列命令会产生和上面一样的效果：

```
$ xdg-user-dirs-update --set DOWNLOAD ~/Internet
```

查询配置好的目录

一旦设置完成，任何用户目录都可以通过 xdg-user-dirs包 查看。例如，以下命令将显示 `Templates` 目录的位置，该目录当然对应于本地配置文件中的 `XDG_TEMPLATES_DIR` 变量：

```
$ xdg-user-dir TEMPLATES
```

警告：`xdg-user-dir` 不应从未经检查的输入中获取其参数，因为它会将参数直接传递给 `eval`，而不进行任何有效性检查，代码行如下：

```
eval echo \${XDG_${1}_DIR:-$HOME}
```

这意味着 `xdg-user-dir` 会从未经清理的输入中执行任意代码。除非上游修复了这个不安全的实现，否则 `xdg-user-dir` 应仅在使用硬编码或经过严格审核的参数时使用。

10.3 Xdg-utils

难度 进阶 | **篇幅** 10.5 KB | **所属** Freedesktop.org

本节将讲到

- 安装
- 使用
- 提示与技巧

xdg-utils 提供了官方工具，用于管理 XDG MIME 应用程序。

- xdg-desktop-menu(1) - 安装桌面菜单项
- xdg-desktop-icon(1) - 将桌面项复制到用户的桌面
- xdg-email(1) - 在用户首选的邮件客户端中编写新电子邮件，可能会预填入主题和其他信息
- xdg-icon-resource(1) - 安装图标资源
- xdg-mime(1) - 查询和安装 MIME 类型和关联
- xdg-open(1) - 在用户首选应用程序中打开文件或 URI
- xdg-screensaver(1) - 启用、禁用或暂停屏幕保护程序
- xdg-settings(1) - 获取或设置默认的 Web 浏览器和 URL 处理程序

安装

安装 xdg-utils包 包。

使用

环境变量

注意：正常运行的桌面环境应该会自动设置这些变量。本节仅在理解 *xdg-utils* 的工作原理时需要，即用于故障排除。

xdg-utils 尝试通过调用其提供的专用程序来与您的桌面环境集成，在适用的情况下。当前环境的评估如下：

- 如果设置了标准化的 XDG_CURRENT_DESKTOP 环境变量，并且该值属于已识别的桌面环境，则使用相应的值。
- 如果存在任何经典回退或特定环境变量，例如 KDE 的 `KDE_FULL_SESSION`，则使用相应的值。
- 如果设置了遗留的 DESKTOP_SESSION 环境变量，并且该值属于已识别的桌面环境，则使用相应的值。

在此过程中，如果找到任何匹配项，则 DE 变量会被内部覆盖为检测到的桌面环境的标准化值。因此，`DE` 既是一个遗留的环境变量，也是 *xdg-utils* 的内部状态变量。例如，如果 `XDG_CURRENT_DESKTOP` 是 `KDE`，*xdg-utils* 会将 `DE` 设置为 `kde`。如果没有找到匹配项，则会使用任何已存在的 `DE` 值， 这样 `XDG_CURRENT_DESKTOP=KDE`

就等于 XDG_CURRENT_DESKTOP 未设置并且 DE=kde。此实现细节值得注意，因为它导致**如果桌面环境已被检测到，预设的 DE 会被忽略**。

xdg-utils 识别的环境变量值如下：

桌面环境	XDG_CURRENT_DESKTOP	DE	DESKTOP_SESSION
–	X-Generic[1]	generic	–
Cinnamon	Cinnamon, X-Cinnamon	cinnamon	–
Deepin	Deepin, DEEPIN, deepin[1]	deepin	–
Enlightenment	ENLIGHTENMENT[1]	enlightenment	–
GNOME	GNOME[2]	gnome	gnome
GNOME Flashback	GNOME-Flashback, GNOME-Flashback:GNOME[2]	gnome	gnome
KDE Plasma	KDE	kde	–
LXDE	LXDE	lxde	LXDE
LXQt	LXQt	lxqt	–
MATE	MATE	mate	mate
Xfce	XFCE	xfce	xfce, xfce4, Xfce Session

1. 这不是一个已在 freedesktop.org 上注册的环境。
2. GNOME 变体，包括 *GNOME Classic* 模式，都被 *xdg-utils* 视为相同的。

请注意，这仅仅是 *xdg-utils* 提供的脚本能够*检测*的列表。脚本仍会在以下条件下执行通用的、与环境无关的操作：

- 请求了通用例程，使用 XDG_CURRENT_DESKTOP=X-Generic 或 DE=generic。
- 环境检测失败。所有相关的环境变量都无法识别或未设置，而经典回退也没有揭示任何信息。
- 执行了特定环境的操作，但失败了，例如由于缺少程序。

xdg-mime

xdg-mime(1) 是一个用于直接查询和修改默认 MIME 应用程序的脚本。它在其他脚本中使用，例如 *xdg-open*，并且还是一个有用的故障排除工具。

确定文件的 MIME 类型：

```
$ xdg-mime query filetype photo.jpeg
image/jpeg
```

确定 MIME 类型的默认应用程序：

```
$ xdg-mime query default image/jpeg
gimp.desktop
```

更改 MIME 类型的默认应用程序：

```
$ xdg-mime default feh.desktop image/jpeg
```

将文件管理器设置为默认文件管理器（例如 -Thunar）：

```
$ xdg-mime default thunar.desktop inode/directory
```

调试 MIME 类型的默认应用程序：

```
$ env XDG_UTILS_DEBUG_LEVEL=10  xdg-mime query default text/html
Checking /home/user/.config/mimeapps.list
Checking /home/user/.local/share/applications/defaults.list
and /home/user/.local/share/applications/mimeinfo.cache
Checking /usr/local/share/applications/defaults.list
and /usr/local/share/applications/mimeinfo.cache
Checking /usr/share/applications/defaults.list
and /usr/share/applications/mimeinfo.cache
qutebrowser.desktop
```

当需要确定文件的 MIME 类型时，*xdg-mime* 会尝试使用适合桌面环境的程序：

桌面环境	程序	包
Cinnamon	`gio`	glib2包
GNOME		
GNOME Flashback		
LXDE		
MATE		
Xfce		
Deepin	–	–
Enlightenment		
LXQt		
KDE Plasma	`kmimetypefinder`	kde-cli-tools包

在通用情况下，*xdg-mime* 将：

- 如果存在，委托给 mimetype。需要安装 perl-file-mimeinfo包 包。
- 如果存在，委托给 file包。

xdg-open

xdg-open(1) 是一个资源打开器，被许多应用程序使用，遵循 XDG MIME 应用程序标准，并尽可能与系统的桌面环境集成。

如果检测到桌面环境，将调用其提供的处理程序：

桌面环境	程序	包
Cinnamon	`gio`	glib2包
GNOME		
GNOME Flashback		
MATE		
Deepin	`dde-open`	deepin-api包
Enlightenment	`enlightenment_open`	enlightenment包
KDE Plasma	`kde-open`1	kde-cli-tools包
LXDE	`pcmanfm`	pcmanfm包
LXQt	–	–
Xfce	`exo-open`2	exo包

1. 如果 `KDE_SESSION_VERSION` 未设置，则会使用 `kfmclient` 来自 konqueror包。不过，KDE Plasma 应该始终设置此变量。
2. 还会尝试使用来自 glib2包 的 `gio`。

在通用情况下，*xdg-open* 将：

- 查询 #xdg-mime 以获取与资源关联的默认桌面条目，解析桌面条目，并执行其命令。
- 如果存在，委托给 run-mailcap。需要安装 run-mailcapAUR 包。
- 如果存在，委托给 mimeopen。需要安装 perl-file-mimeinfo包 包。

提示：要查看 *xdg-open* 将使用哪个后端，请设置 `XDG_UTILS_DEBUG_LEVEL=3`。

由于 *xdg-mime* 依赖于 perl-file-mimeinfo包 包来实现 XDG MIME 应用程序标准，如果您没有使用桌面环境，应该安装 perl-file-mimeinfo包，或考虑使用其他资源打开器。

xdg-settings

请参见 xdg-settings(1)。

设置所有 Web MIME 类型使用单个应用程序的快捷方式：

```
$ xdg-settings set default-web-browser firefox.desktop
```

设置 URL 方案的默认应用程序的快捷方式：

```
$ xdg-settings set default-url-scheme-handler irc xchat.desktop
```

提示与技巧

URL 方案处理程序

要设置 URL 方案的默认应用程序，您可能还需要更改 `x-scheme-handler/*` MIME 类型的默认应用程序：

```
$ xdg-mime default firefox.desktop x-scheme-handler/https x-scheme-handler/http
```

10.4 Xdg-menu

难度 进阶 | **篇幅** 11.7 KB | **所属** Freedesktop.org

本节将讲到

- 安装
- 菜单层级结构
- 配置
- 使用
- 示例

xdg-menu是一个用于为以下窗口管理器生成XDG桌面菜单的工具：

- awesome
- Blackbox
- Fluxbox
- FVWM
- IceWM

- Openbox
- twm
- Window Maker

KDE, Gnome, Xfce，Enlightenment已经兼容XDG。

安装

安装archlinux-xdg-menu包包。

菜单层级结构

- 应用
- 辅助功能
- 附件
- 开发
- 教育
- 游戏
- 图形
- 互联网
- 多媒体
- 办公
- 其他
- 科学
- 设置
- 系统

配置

Xdg_menu依赖三类信息来生成菜单：一个根菜单（即通常通过命令行传入的 XML 菜单模板）、上次运行时缓存的信息，以及一系列配置文件。

- 你可以在`/etc/xdg/menus`中找到一些XML菜单模板.
- 如果更改 xdg_menu 的代码以更改菜单布局，请确保删除`~/.xdg_menu_cache`中的所有内容，否则你可能将花费数小时试图弄明白为什么对Perl脚本所做的修改没有生效。
- 各个应用程序的配置文件位于`/usr/share/applications`。

其他配置文件目录可以在`/usr/share`下找到。在大多数情况下，你无需改动这些文件。不过，如果你希望调整菜单的布局，可以通过修改菜单模板以进行小幅调整。若需大幅改动，则需要直接修改Xdg_menu的Perl脚本本身。

如果你发现某些应用程序未显示在菜单中，或者其名称显示异常，那么你需要检查 `/usr/share/applications` 目录下的相应 `.desktop` 文件，并参考桌面入口规范。

从其他目录添加桌面入口

默认情况下，Xdg-menu会加载那些将桌面入口（desktop entry）安装到`/usr/share/applications`的应用程序。若要将桌面入口安装在用户目录（例如`~/.local/share/applications`）中的应用程序也加入菜单的菜单中，需编辑`/etc/xdg/menus/arch applications.menu`文件，并为相关目录添加`<AppDir>`标签，如下所示：

```
/etc/xdg/menus/arch-applications.menu
<Menu>

  <Name>Applications</Name>
  <Directory>Arch-Applications.directory</Directory>
  <DefaultAppDirs/>
  <AppDir>/home/username/.local/share/applications</AppDir>
  <DefaultDirectoryDirs/>
  <DefaultMergeDirs/>
  ...
```

使用

xdg_menu

```
xdg_menu [--format <format>] [--desktop <desktop>]
         [--charset <charset>] [--language <language>]
        [--root-menu <root-menu>] [--die-on-error]
        [--fullmenu] [--help]

       format - output format
                possible formats: twm, WindowMaker, fvwm2, icewm, ion3
                                  blackbox, fluxbox, openbox,
                                xfce4, openbox3, openbox3-pipe, awesome
                                readable
              default: WindowMaker

       fullmenu  - output a full menu and not only a submenu

       desktop - desktop name for NotShowIn and OnlyShowIn
              default: the same as format

       charset - output charset
              default: <locale>

       language - output language
              default: <locale>

       root-menu - location of root menu file
              default: /opt/gnome/etc/xdg/menus/applications.menu

       die-on-error - abort execution on any error,
              default: try to continue

       verbose - print debugging information

       help - print this text
```

update-menus

update-menus 可根据 XDG 数据更新窗口管理器（WM）的菜单，并可配置为自动执行此操作。

该脚本是xdg_menu的一个封装，依赖于 `/etc/update-menus.conf`配置文件

要使用它，你需要安装 archlinux-xdg-menu包 (其中包含xdg_menu)

在`/etc/update-menus.conf` 文件中，你需要从支持的窗口管理器列表中选择需要为其生成菜单的WM。配置文件中允许使用#添加注释。

所有生成的菜单都会存放在`/var/cache/xdg-menu/`目录下。有关具体窗口管理器的配置示例，请参阅本页的“WM特定示例”部分。

示例

Awesome窗口管理器

使用 xdg_menu ：

```
$ xdg_menu --format awesome > ~/.config/awesome/xdg_menu.lua
```

编辑 rc.lua ，按如下方式更改

- 添加一行require语句，以加载你生成的xdg_menu.lua文件
- 在你的awful.menu对象中添加一个菜单项，调用xgd_menu生成的应用程序菜单

```
...
xdg_menu = require("xdg_menu")
...

...
mymainmenu = awful.menu({ items = { { "awesome", myawesomemenu,
beautiful.awesome_icon },
                                    { "Applications", xdg_menu },
                                    { "open terminal", terminal }
                                  }
                        })
...
```

IceWM

使用 xdg_menu

```
$ xdg_menu --format icewm --fullmenu --root-menu /etc/xdg/menus/arch-applications.menu
>>~/.icewm/programs
```

使用 update-menus

- 在 /etc/update-menus.conf 取消对icewm的注释
- 使用root用户运行 update-menus 命令
- 将 ~/.icewm/programs 符号链接到 /var/cache/xdg-menu/icewm/programs

Ion3

使用 xdg_menu

```
$ xdg_menu --format ion3  --root-menu /etc/xdg/menus/arch-applications.menu
>~/.ion3/default-session--0/_xdg-menu.lua
```

之后，将 cfg_menus.lua 更改为包含 _xdg-menu.lua 文件，并将菜单添加到主菜单中。例如：

```
...

dopath("_xdg-menu")

-- Main menu
defmenu("mainmenu", {
    submenu("XDG Menu",          "<NAME-OF-FIRST-MENU-IN-_xdg-menu.lua-FILE>"),
    submenu("Programs",          "appmenu"),
    menuentry("Lock screen",     "ioncore.exec_on(_, 'xlock')"),
    menuentry("Help",            "mod_query.query_man(_)"),
    menuentry("About Ion",       "mod_query.show_about_ion(_)"),
    submenu("Styles",            "stylemenu"),
    submenu("Session",           "sessionmenu"),
})

...
```

使用update-menus

- 在 /etc/update-menus.conf 取消对ion3的注释
- 使用root用户运行 update-menus 命令
- 将 cfg_menus.lua 更改为包含 xdg-menu.lua 文件，并将菜单添加到主菜单中

例如:

```
...

dopath("/var/cache/xdg-menu/ion3/xdg-menu.lua")

-- Main menu
defmenu("mainmenu", {
    submenu("XDG Menu",          "<NAME-OF-FIRST-MENU-IN-xdg-menu.lua-FILE>"),
    submenu("Programs",          "appmenu"),
    menuentry("Lock screen",     "ioncore.exec_on(_, 'xlock')"),
    menuentry("Help",            "mod_query.query_man(_)"),
    menuentry("About Ion",       "mod_query.show_about_ion(_)"),
    submenu("Styles",            "stylemenu"),
    submenu("Session",           "sessionmenu"),
})

...
```

FluxBox

使用 xdg_menu

```
$ xdg_menu --format fluxbox  --root-menu /etc/xdg/menus/arch-applications.menu
>~/.fluxbox/my-menu
```

将菜单文件更改为包含生成的菜单。

例如添加行:

```
[include] (my-menu)
```

使用 update-menus

- 在 /etc/update-menu.conf 中取消对fluxbox的注释
- 使用root用户运行 update-menus
- 将菜单文件更改为包含生成的菜单

例如添加行:

```
[include] (/var/cache/xdg-menu/fluxbox/boxrc)
```

OpenBox

使用 xdg_menu

生成菜单:

```
$ xdg_menu --format openbox3 --root-menu /etc/xdg/menus/arch-applications.menu > xdg-menu.xml
```

然后手动将其添加到 menu.xml 中。例如，将 xdg-menu.xml 放入 menu.xml 并添加：

```
<menu id="Applications" />
```

进入根菜单。

作为管道菜单

使用 xdg_open 作为管道菜单，您可以获得额外的好处，即在安装新应用程序时，菜单会自动更新。

在 menu.xml 中的根菜单标记之间添加以下内容:

```
<menu id="applications" label="Applications" execute="xdg_menu --format openbox3-pipe --root-menu /etc/xdg/menus/arch-applications.menu" />
```

一个非常基本的例子:

```
<?xml version="1.0" encoding="UTF-8"?>

<openbox_menu xmlns="http://openbox.org/3.4/menu">

<menu id="root-menu" label="Openbox 3">
  <menu id="applications" label="Applications" execute="xdg_menu --format openbox3-pipe --root-menu /etc/xdg/menus/arch-applications.menu" />
  <separator />
  <item label="Log Out">
    <action name="Exit">
      <prompt>yes</prompt>
    </action>
  </item>
</menu>

</openbox_menu>
```

使用 update-menus

- 在 /etc/update-menu.conf 中取消对openbox的注释
- 以root用户运行 update-menus

- 将 menu.xml 文件更改为包含生成的菜单

例如，在根菜单中添加以下内容:

```
<menu id="xdg-menu" label="XDG Menu" execute="cat /var/cache/xdg-
menu/openbox/menu.xml"/>
```

Twm

使用 xdg_menu

使用

```
$ xdg_menu --format twm --root-menu /etc/xdg/menus/arch-applications.menu >my-twm-menu
```

并手动添加到 twmrc 中。对于具有m4预处理的twm衍生物，如vtwm或ctwm，可通过添加

```
sinclude(`/PATH/TO/my-twm-menu')
```

到 *twmrc.

使用 update-menus

- 在 /etc/update-menu.conf 中取消对twm的注释
- 添加到 /etc/X11/twm/system.twmrc 文件应用程序菜单中 (添加以下这行:

```
 "apps"          f.menu "Applications"
```

)到菜单中

- 使用root用户运行 update-menus
- 运行 twm -f /var/cache/xdg-menu/twm/twmrc

(您还需要将其他自定义项添加到 /etc/X11/twm/system.twmrc)

WindowMaker

使用 xdg_menu

使用

```
$ xdg_menu --format WindowMaker --root-menu /etc/xdg/menus/arch-applications.menu >my-
wm-menu
```

然后添加

```
#include "my-wm-menu"
```

到你的 WindowMaker 菜单文件。

您还可以使用WPrefs“应用程序菜单定义”，并将xdg命令作为参数添加到“生成的子菜单”对象中。

使用 update-menus

- 在 /etc/update-menus.conf 取消对 WindowMaker 的注释
- 用root用户运行 update-menus

- 添加

```
#include "/var/cache/xdg-menu/WindowMaker/wmrc"
```

到你的菜单文件

Fvwm2

使用 xdg_menu

生成菜单

```
$ xdg_menu --format fvwm2 --root-menu /etc/xdg/menus/arch-applications.menu >fvwm2-menu
```

然后把菜单添加到根菜单

```
read fvwm2-menu

AddToMenu MenuFvwmRoot  "Root Menu"              Title
+ "&0. XDG Menu"          Popup xdg_menu
```

使用 update-menus

- 在 /etc/update-menus.conf 取消对 fvwm2 的注释
- 使用root用户运行 update-menus
- 修改你的 .fvwm2rc 以使其包含创建的菜单，示例:

```
AddToMenu MenuFvwmRoot  "Root Menu"              Title
+ "&0. XDG Menu"          Popup xdg_menu
read /var/cache/xdg-menu/fvwm2/fvwm2rc
```

BlackBox

使用 xdg_menu

```
$ xdg_menu --format blackbox  --root-menu /etc/xdg/menus/arch-applications.menu >my-
menu
```

修改你的菜单文件以使其包含创建的菜单

例如添加:

```
[include] (my-menu)
```

使用 update-menus

- 在 /etc/update-menus.conf 取消 blackbox 的注释
- 使用root用户运行 update-menus
- 更改你的菜单文件以使其包含创建的菜单

例如添加:

```
[include] (/var/cache/xdg-menu/blackbox/boxrc)
```

另请参见

- Sawfish#Menus
- https://github.com/gapan/xdgmenumaker

10.5 XDG 桌面门户

难度 进阶 | **篇幅** 13.8 KB | **所属** Freedesktop.org

本节将讲到

- 安装
- 后端
- 配置
- 疑难解答
- 参见

引自 Flatpak 文档：

: 门户是用于安全访问沙箱外资源的框架。沙箱内程序可以使用的特性包括：检测网络状态；使用文件选择器来打开文件；打开 URI；阻止设备睡眠、休眠或关机；打印；发送邮件；显示通知；截取屏幕或录屏［……］

门户（portals）系统最初专为 Flatpak 应用程序设计，但任何应用程序均可借助门户访问跨桌面环境和 UI 框架的统一功能。这一功能得到了广泛使用，例如通过 PipeWire 在 Wayland 上实现屏幕共享，或让 Firefox 采用与当前桌面环境相同 UI 框架的文件打开/保存对话框。

安装

安装 xdg-desktop-portal包 以及至少一个 #后端，该软件包包含一个 systemd/用户服务，通过 D-Bus 自动触发启动。

后端

当一个软件通过 XDG 桌面门户发起请求时，xdg-desktop-portal包 会将这个请求转发至可用的后端实现，由后端实现适合桌面环境的 UI，并访问特定于桌面环境的 API 处理诸如打开 URI、录制屏幕之类的请求。后端可以安装多个，例如使用 Sway 可以安装 xdg-desktop-portal-wlr包 支持屏幕共享，并同时安装 xdg-desktop-portal-gtk包 作为后备，处理 xdg-desktop-portal-wlr包 未实现的接口。

门户后端配置储存在 `/usr/share/xdg-desktop-portal/portals/*.portal` 中。门户后端配置文件包含其支持的接口和桌面环境。

所有可用后端及其接口支持情况

后端	支持的桌面环境	使用的UI框架	接口支持情况																		
对话框	账户信息	应用程序选择器	后台活动	剪贴板	桌面启动项	电子邮件	文件选择器	全局快捷键	用户会话管理	输入设备捕获	系统通知	打印	远程桌面	屏幕录制	截屏	密钥	系统设置	壁纸			
xdg-desktop-portal-cosmic包	COSMIC	iced	是	否	否	否	否	否	否	是	否	否	否	否	否	否	是	是	否	是	否
xdg-desktop-portal-dde包	深度桌面环境	Qt 5	是	是	是	是	否	否	否	是	是	是	否	是	否	否	否	是	是	是	是
xdg-desktop-portal-gnome包	GNOME	GTK 4	是	是	是	是	是	是	否	是	否	否	是	是	是	是	是	是	否	是	是
xdg-desktop-portal-gtk包	（通用）	GTK 3	是	是	是	否	否	是	是	是	否	是	否	是	是	否	否	否	否	是	否
xdg-desktop-portal-hyprland包	Hyprland 1	Qt 6	否	否	否	否	否	否	否	否	是	否	是	否	否	否	是	是	否	否	否
xdg-desktop-portal-kde包	KDE Plasma	Qt 6	是	是	是	是	是	是	是	是	是	是	是	是	是	是	是	是	否	是	否
xdg-desktop-portal-liri-gitAUR	Liri	Qt 5	是	是	是	是	否	否	是	是	否	是	否	是	是	否	是	是	否	是	是
xdg-desktop-portal-lxqt包	LXQt	Qt 6	否	否	否	否	否	否	否	是	否	否	否	否	否	否	否	否	否	否	否
xdg-desktop-	theDesk	Qt 6	是	是	否	否	否	否	否	是	否	否	否	是	否	否	否	是	否	是	否

后端	支持的桌面环境	使用的UI框架	接口支持情况																	
portal-tdAUR																				
xdg-desktop-portal-wlr包	wlroots	–	否	否	否	否	否	否	否	否	否	否	否	否	否	否	是	是	否	否	否
xdg-desktop-portal-xapp包	Cinnamon2	–	否	否	否	是	否	否	否	否	否	是	否	否	否	否	否	是	否	是	是

1. 适用于所有基于 wlroots 的混成器，但提供针对 Hyprland 的额外功能，例如可在截屏或屏幕录制时共享单个窗口。
2. 其也为 MATE 和 Xfce 提供部分支持。

除此以外，以下后端跨桌面环境实现了特定接口：

- darkman包 实现了系统设置接口（仅颜色方案设置）。
- gnome-keyring包 实现了密钥接口。
- kwallet包 实现了密钥接口。
- pikeruAUR 使用自己的 GUI 实现了文件选择器接口。
- xdg-desktop-portal-shanaAUR 通过将请求重定向至 GNOME/GTK/KDE/LXQt 后端实现了文件选择器接口。
- xdg-desktop-portal-termfilechooser-gitAUR 使用终端文件选择器实现了文件选择器接口。

配置

当收到请求时，`xdg-desktop-portal` 将会使用位于 `/usr/share/xdg-desktop-portal/*桌面环境*-portals.conf` 的配置文件，其中*桌面环境*由环境变量 `XDG_CURRENT_DESKTOP` 决定。配置文件由桌面环境提供，决定应当使用的后端。

如果需要自定义配置，或者使用的桌面环境并没有提供默认配置，可以创建门户配置文件 `$XDG_CONFIG_HOME/xdg-desktop-portal/portals.conf` 以决定全局或者特定接口使用的后端。如果使用多个桌面环境，也可以创建并配置特定桌面环境的配置文件 `$XDG_CONFIG_HOME/xdg-desktop-portal/*桌面环境*-portals.conf`。

例如，如果正在使用的桌面环境没有门户后端，希望使用xdg-desktop-portal-gtk包 作为通用后备后端，同时通过 xdg-desktop-portal-lxqt包 使用 LXQt 文件选择器，可以使用以下配置：

```
~/.config/xdg-desktop-portal/portals.conf
[preferred]
default=gtk
org.freedesktop.impl.portal.FileChooser=lxqt
```

参见 portals.conf(5)。

桌面环境伪装

某些情况下（例如独立使用窗口管理器时），可能需要让 `xdg-desktop-portal` 以为正在使用特定的桌面环境。可以在 `xdg-desktop-portal.service` 用户单元中使用附加配置片段添加环境变量 XDG_CURRENT_DESKTOP。例如，可以指定使用 KDE 配置的后端：

```
~/.config/systemd/user/xdg-desktop-portal.service.d/override.conf
[Service]
Environment="XDG_CURRENT_DESKTOP=KDE"
```

疑难解答

门户无法启动

要使 xdg-desktop-portal-wlr包 和 xdg-desktop-portal-hyprland包 正常工作，必须在 systemd 用户会话中设置 `XDG_CURRENT_DESKTOP` 和 `WAYLAND_DISPLAY` 环境变量。

`XDG_CURRENT_DESKTOP` 需要设置为混成器名称，例如 `XDG_CURRENT_DESKTOP=sway`。`WAYLAND_DISPLAY` 则由混成器自动设置。这两个变量通常会在混成器启动时自动设置，但如果 systemd 用户会话先于混成器启动，混成器启动时需要导入这两个变量。

注意：Hyprland 不会自动设置 `XDG_CURRENT_DESKTOP` 环境变量，需要手动配置。请参阅 Hyprland Wiki。

使用 `systemctl --user show-environment` 检查这些变量是否已设置。如果未设置，请在启动混成器时运行以下命令（例如，将其写入混成器的配置文件），将这些环境变量导入到 systemd 用户会话和 D-Bus 中：

```
$ systemctl --user import-environment WAYLAND_DISPLAY XDG_CURRENT_DESKTOP
$ dbus-update-activation-environment --systemd WAYLAND_DISPLAY XDG_CURRENT_DESKTOP=混成器名称
```

注意：这些环境变量需要在混成器启动时设置，通常通过混成器自身的配置完成，请参见混成器的文档说明。

提示：Sway 提供了一个可自动完成此操作的插入文件，请参阅 Sway#配置。

更多详情请参阅 和 。

在多显示器环境下使用 xdg-desktop-portal-wlr

`xdg-desktop-portal-wlr` 需要一个外部选择器来选择要共享的显示器。默认按顺序查找 slurp包、wofi包 和 bemenu包。若使用 slurp，收到屏幕共享请求后，鼠标指针会变成十字形，需要点击要共享的屏幕。若使用 wofi 或 bemenu，会出现一个可用显示器的菜单供选择。如果没有任何选择器可用，`xdg-desktop-portal-wlr` 会回退到找到的第一个显示器。详见 xdg-desktop-portal-wlr(5) § SCREENCAST OPTIONS。

KDE Plasma 下 GTK 应用程序字体渲染不佳

部分 GTK 应用程序在 Plasma 环境中需要安装 xdg-desktop-portal-gtk包 才能正确渲染字体。请安装该软件包，然后运行：

```
$ /usr/lib/xdg-desktop-portal --replace
```

GTK（可能还有其它的）文件选择器无法工作

如果应用程序通过 Xwayland 运行（最简单的检查方法是运行 `xeyes`，然后将鼠标指针移至应用程序窗口内，若程序运行在 Xwayland 下，其中眼睛会看向鼠标指针），那么 `xdg-desktop-portal-gtk` 会按需启动，但在选择文件后可能没有任何反应。这种情况下，为 `xdg-desktop-portal-gtk` 的环境添加 `DISPLAY=:0` 可能会解决问题。要实现此操作，可以按照前文（#门户无法启动）关于 `import-environment` 的说明进行设置，或者直接编辑 `xdg-desktop-portal-gtk` 的 systemd 用户单元文件。此外，也可以尝试通过配置开启应用程序的原生 Wayland 支持（例如对于 Electron 程序）。

参见

- 项目主页
- 门户文档：列出了应用程序和后端可使用的所有接口。

10.6 XDG MIME 应用程序

难度 进阶 | **篇幅** 13.9 KB | **所属** Freedesktop.org

本节将讲到

- 共享 MIME 数据库
- mimeapps.list
- 工具
- 故障排除

翻译状态：

- 本文（或部分内容）译自 XDG MIME Applications，最近一次同步于 2026-07-20，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

XDG MIME 应用程序规范基于#共享 MIME 数据库和桌面项，提供默认应用程序。

1. 应用程序通过桌面项描述它们能够处理哪些 MIME 类型。
2. desktop-file-utils包 注册了一个 pacman 钩子，用于构建由桌面项处理的 MIME 类型的缓存数据库，详见 update-desktop-database(1)。
3. 应用程序可以通过将 XML 文件放置在 `/usr/share/mime/packages/` 中来安装新的 MIME 类型。
4. shared-mime-info包 注册了一个 pacman 钩子，用于构建共享 MIME 信息数据库缓存，详见 update-mime-database(1)。
5. 桌面环境和用户可以通过 mimeapps.list 文件更改默认应用程序并添加或删除 MIME 类型与应用程序的关联。

共享 MIME 数据库

XDG 共享 MIME 信息数据库规范促进了桌面环境之间的共享 MIME 数据库，并允许应用程序轻松地在系统范围内注册新的 MIME 类型。

该数据库是由软件包在 `/usr/share/mime/packages/` 中安装的 XML 文件构建的，使用来自 shared-mime-info包 的工具。

`/usr/share/mime/` 中的文件不应直接编辑，但可以在 `~/.local/share/mime/` 树中为每个用户维护单独的数据库。

“URI 方案处理 \[...\] 通过处理 `x-scheme-handler/foo` MIME 类型的应用程序来进行，其中 foo 是相关的 URI 方案。”

新的 MIME 类型

此示例定义了一个新的 MIME 类型 `application/x-foobar`，并将其分配给任何以 *.foo* 结尾的文件。只需创建以下文件：

```
~/.local/share/mime/packages/application-x-foobar.xml
<?xml version="1.0" encoding="UTF-8"?>
<mime-info xmlns="http://www.freedesktop.org/standards/shared-mime-info">
    <mime-type type="application/x-foobar">
        <comment>foo 文件</comment>
        <icon name="application-x-foobar"/>
        <glob-deleteall/>
        <glob pattern="*.foo"/>
    </mime-type>
</mime-info>
```

然后更新 MIME 数据库：

```
$ update-mime-database ~/.local/share/mime
```

当然，如果没有桌面条目与 MIME 类型关联，这不会产生任何效果。您可能需要创建新的桌面项或修改 mimeapps.list。

mimeapps.list

XDG 标准是配置桌面环境最常用的方式。每个 MIME 类型的默认应用程序存储在 `mimeapps.list` 文件中，该文件可以存储在多个位置。它们按照以下顺序搜索，前面的关联优先于后面的关联：

路径	用途
`~/.config/mimeapps.list`	用户覆盖
`/etc/xdg/mimeapps.list`	系统范围的覆盖
`~/.local/share/applications/mimeapps.list`	（**已废弃**）用户覆盖
`/usr/local/share/applications/mimeapps.list` `/usr/share/applications/mimeapps.list`	分发提供的默认值

此外，还可以在名为 `*桌面环境*-mimeapps.list` 的文件中定义桌面环境特定的默认应用程序，其中 `*桌面环境*` 是桌面环境的名称（来自 `XDG_CURRENT_DESKTOP` 环境变量）。例如，`/etc/xdg/xfce-mimeapps.list` 定义了 Xfce 的系统范围默认应用程序覆盖。这些特定桌面的覆盖优先于相应的非特定桌面文件。例如，`/etc/xdg/xfce-mimeapps.list` 优先于 `/etc/xdg/mimeapps.list`，但仍会被 `~/.config/mimeapps.list` 覆盖。

提示：虽然已废弃，但许多应用程序仍然读写 `~/.local/share/applications/mimeapps.list`。为了简化维护，可以将其符号链接到 `~/.config/mimeapps.list`：

```
$ ln -s ~/.config/mimeapps.list ~/.local/share/applications/mimeapps.list
```

注意：您还可能会在这些位置找到名为 `defaults.list` 的文件。此文件类似于 `mimeapps.list`，但仅列出默认应用程序（而不是已添加/删除的关联）。它现在已废弃，应手动与 `mimeapps.list` 合并。

要发现所有被扫描的文件，可以通过设置环境变量 `XDG_UTILS_DEBUG_LEVEL=2` 启用调试模式：例如，`xdg-mime query default *类型*` 命令将打印它搜索 MIME 信息的每个配置文件。

格式

考虑以下示例：

```
mimeapps.list
[Added Associations]
image/jpeg=bar.desktop;baz.desktop
video/H264=bar.desktop
[Removed Associations]
video/H264=baz.desktop
[Default Applications]
image/jpeg=foo.desktop
```

每个部分将一个或多个桌面条目分配给 MIME 类型。

- **Added Associations** 表示应用程序支持打开该 MIME 类型。例如，`bar.desktop` 和 `baz.desktop` 可以打开 JPEG 图像。这可能会影响您在文件浏览器中右键点击文件时看到的应用程序列表。
- **Removed Associations** 表示应用程序不支持该 MIME 类型。例如，`baz.desktop` 无法打开 H.264 视频。
- **Default Applications** 表示该应用程序应为打开该 MIME 类型的默认选择。例如，JPEG 图像应使用 `foo.desktop` 打开。这隐式地在应用程序和 MIME 类型之间添加了一个关联。如果有多个应用程序，它们会按顺序尝试。

每个部分都是可选的，如果不需要可以省略。

工具

虽然可以通过直接编辑 mimeapps.list 和#共享 MIME 数据库来配置默认应用程序和 MIME 类型，但有许多工具可以简化这一过程。这些工具也很重要，因为应用程序可能将文件的打开委托给这些工具，而不是自己尝试实现 MIME 类型标准。

如果您使用的是桌面环境，应首先检查它是否提供了自己的工具。应该优先使用这些工具，而不是其他替代方案。

官方的 xdg-utils 包含用于根据 XDG 标准管理 MIME 类型和默认应用程序的工具（xdg-mime）。最重要的是，它提供了 xdg-open，许多应用程序用它来用默认应用程序打开文件。

lsdesktopf

lsdesktopfAUR 提供了几种搜索 MIME 数据库和桌面 MIME 条目的方法。

例如，要查看系统中所有与 MIME 类型 `video` 相关联的 *.desktop* 文件的 MIME 扩展名，可以使用 `lsdesktopf --gm -gx video`，或要在 XML 数据库文件中搜索，请使用 `lsdesktopf --gdx -gx video`。要快速了解有多少个且哪些 *.desktop* 文件可以与某个 MIME 类型关联，请使用 `lsdesktopf --gen-mimeapps`。要查看 XML 数据库文件中的所有文件名扩展名，可以使用 `lsdesktopf --gdx -gfx`。

selectdefaultapplication

selectdefaultapplication-gitAUR 是一个 GUI 应用程序，列出了所有支持各种 MIME 类型的应用程序，并允许您快速将其设置为所有或一些它支持的 MIME 类型的默认应用程序（通过修改 `mimeapps.list`）。

它还显示了“可读”的名称和文件扩展名，因此您不需要记住 MIME 类型的名称。

mime-tui

mime-tuiAUR 是一个 TUI（文本用户界面）应用程序，它允许用户通过两种方式之一来编辑 mimetype 关联（通过修改 `mimeapps.list`）：一种是按应用程序编辑（此应用程序应支持哪些 mimetype 类型？），另一种是按 mimetype 编辑（此 mimetype 类型应与哪些应用程序关联？）。此外，它还能标记与已卸载应用程序关联的 mimetype 类型。

故障排除

如果文件未通过您期望的默认应用程序打开，可能有多个原因。您可能需要检查每种情况。

缺少桌面条目

需要一个桌面项来将应用程序与 MIME 类型关联。确保存在这样的条目，并且可以用它来（手动）打开文件。

缺少关联

如果应用程序的桌面条目没有在其 `MimeType` 键下指定 MIME 类型，它在需要打开该类型时将不会被考虑。编辑 mimeapps.list，将桌面文件与 MIME 类型关联起来。

非默认应用程序

如果桌面条目已与 MIME 类型关联，可能只是没有设置为默认值。编辑 mimeapps.list，将其设置为默认关联。

非标准关联

应用程序可以忽略或仅部分实现 XDG 标准。检查是否使用了已废弃的文件，如 `~/.local/share/applications/mimeapps.list` 和 `~/.local/share/applications/defaults.list`。如果您尝试从其他应用程序（如 Web 浏览器或文件管理器）打开文件，请检查该应用程序是否有自己的方法来选择默认应用程序。

.desktop 文件中影响应用程序启动的变量

支持该规范的桌面环境和文件管理器根据 *.desktop* 文件中的定义启动程序。参见桌面项#应用程序项。

通常不需要配置打包的 *.desktop* 文件，但它可能并非没有缺陷。即使一个包含必要 MIME 类型描述的应用程序在 *.desktop* 文件中的 `MimeType` 变量被用于关联，它也可能无法正确启动，根本无法启动，或者启动时未能打开文件。

例如，这可能发生在 `Exec` 变量缺少打开文件所需的内部选项，或缺少如何在菜单中显示应用程序的配置。`Exec` 变量通常以 `%` 开头；有关当前支持的选项，请参见 exec-variables。

以下表格列出了影响应用程序启动的 *.desktop* 文件中主要的变量条目，如果它与 MIME 类型关联。

变量名称	示例 1 内容	示例 2 内容	描述
DBusActivatable	DBusActivatable=true	DBusActivatable=false	应用程序与 D-Bus 交互。 另见配置：D-Bus。
MimeType	MimeType=application/vnd.oasis.opendocument.text	MimeType=application/vnd.sun.xml.math	应用程序支持的

变量名称	示例 1 内容	示例 2 内容	描述
			MIME 类型列表
StartupWMClass	StartupWMClass=google-chrome	StartupWMClass=xpad	将窗口与拥有该窗口的应用程序关联
Terminal	Terminal=true	Terminal=false	在默认终端中启动

空的 MIME 关联 / 在 KDE 中打开菜单

**此页面或章节适合移动到 KDE#疑难解答。**

附注： KDE 特有的故障排除。（在 Talk:XDG MIME 应用程序 讨论）

安装 archlinux-xdg-menu 并运行 `XDG_MENU_PREFIX=arch- kbuildsycoca6`，或将 `export XDG_MENU_PREFIX=plasma-` 添加到 `.xinitrc`。

第 11 章　共 6 节　140 KB

字体

字体在终端、浏览器、Java 程序与中文界面里的表现常常对不上，这一章解决的就是它。《字体》与《字体配置》两节必读，前者讲字体格式、安装、字体包、后备字体顺序与别名，后者讲字体路径、Fontconfig 写法、不支持 Fontconfig 的应用程序与疑难问题；《字体配置/中文》也建议一并读，对标 Android 与 Windows 的显示效果、抗锯齿参考配置和禁用中文斜体都在其中。《字体配置/示例》可当配方库按需取用，《微软字体》按需再翻，不用 Java 的话《Java 运行环境字体配置》整节可跳。

最常见的坑是以为装上字体就会生效：字形最终选哪个由 Fontconfig 规则决定，装完毫无变化，多半是规则顺序或路径不对。另一个误解是把中文斜体、粗体当成现成字形，它们通常要另行处理。

11.1　Java 运行环境字体配置

难度 入门 | **篇幅** 4.8 KB | **所属** 字体

本节将讲到

- 抗锯齿
- 选择字体

可能一部分人认为Java应用程序中的默认字体和字体的显示模式不大理想。下面有几种方法可以改进Oracle Java Runtime Environment (JRE)中的字体显示。这些方法可以单独使用，但是经过许多用户实践发现将它们组合使用可以获得更好的效果。

Java对于TrueType格式字体的支持似乎是最好的。

抗锯齿

基础设置

Linux上的Oracle Java 1.6和OpenJDK提供了字体的抗锯齿功能。使用这个功能，请将以下内容添加到`/etc/environment`:

```
_JAVA_OPTIONS='-Dawt.useSystemAAFontSettings=setting'
```

`*setting*` 是下面内容的其中一项:

设置值	描述
`off`, `false`, `default`	不开启抗锯齿
`on`	全效抗锯齿
`gasp`	使用字体文件自带的配置信息
`lcd`, `lcd_hrgb`	为流行的显示器调整过的抗锯齿
`lcd_hbgr`, `lcd_vrgb`, `lcd_vbgr`	替代显示器的设置

`gasp` 和`lcd` 设置在大部分情况下表现良好。

选择使用GTK的显示风格，请将下面的内容添加到.bashrc：

```
_JAVA_OPTIONS='-Dswing.defaultlaf=com.sun.java.swing.plaf.gtk.GTKLookAndFeel'
```
注意：

- 所描述的Java选项仅适用于使用Java绘制GUI的应用程序，如Jdownloader，而不适用于仅使用Java作为后端的应用程序，如Openoffice.org和Matlab。
- “TrueType” 字体包含一个网格显示拟合和扫描转换过程的表(Grid-fitting And Scan-conversion Procedure “GASP”)，其中包含了字体作者对不同大小字体显示的建议。一些字号可以完全抗锯齿，一些只有部分提示，还有一些显示为位图。对于一些字号，上面的方法会组合使用。

在运行之前，在命令行中指定其他的变量，可以尝试别的配置:

```
_JAVA_OPTIONS=options executable
```
你需要重新登录使配置生效。

OpenJDK 补丁

即使通过Java选项强制执行了抗锯齿，得到的抗锯齿效果也可能不如本机应用程序。可以通过OpenJDK的一个补丁来弥补，AUR提供了这个补丁:

- 修补后的 **OpenJDK7** 可用 jre7-openjdk-infinalityAUR\[损坏的链接：package not found\] (> **模板错误：** 您在尝试使用 = 标志吗？ 有关解决方法，请访问 [Help:Template#Escape template-breaking characters](https://wiki.archlinuxcn.org/wzh/Help:Template#Escape_template-breaking_characters)。)
- 修补后的 **OpenJDK8** 可用 jre8-openjdk-infinalityAUR\[损坏的链接：package not found\]

修补后的版本从fontconfig获得FreeType类型字体的渲染和加载标志，而不是使用OpenJDK的方法。虽然这是一个Infinality包，但是补丁本身实际上并不依赖于fontconfig-InfinalityAUR\[损坏的链接：package not found\]，因为只使用了普通的fontconfig包 api。

选择字体

TrueType 字体

使一些应用程序知道所需字体的目录路径，那么这些Java应用程序就会使用特定的TrueType字体。TrueType字体安装在`/usr/share/fonts/TTF`目录中。将以下内容添加到`/etc/environment`以启用这些字体。

```
JAVA_FONTS=/usr/share/fonts/TTF
```
你需要重新登录使配置生效。

修复乱码 (For JRE8)

将字体文件放在下面的目录下。如果目录不存在，则创建该目录。

```
/usr/lib/jvm/java-8-openjdk/jre/lib/fonts/fallback/
```

11.2 微软字体

难度 进阶 | **篇幅** 8.8 KB | **所属** 字体

本节将讲到

- 安装
- 适用于微软字体的 Fontconfig 规则
- 已知问题

本文介绍如何安装微软 TrueType 字体并模拟 Windows 的渲染设置。

提示：参见 公制兼容字体 以获取替代方案。

安装

自动安装

注意：这些包将在构建时自动获取 Windows Enterprise 90 天评估版 ISO，并从中提取所需的字体。

安装字体的最简单方法是安装以下包中的一个：

- win11fonts-autoAUR — Windows 11 字体
- win10fonts-autoAUR — Windows 10 字体
- ttf-vista-fontsAUR — Vista 和 Office 2007 字体

现有软件包

注意：这些包需要用户预先从 Windows 分区或者 Windows 和/或 Office 2007 ISO 中手动提取字体。参阅相关的 PKGBUILD 获取更多信息。

- ttf-ms-win11AUR / win11fonts-autoAUR — Windows 11 字体
- ttf-ms-win10AUR / win10fonts-autoAUR — Windows 10 字体
- ttf-ms-win8AUR — Windows 8.1 字体
- ttf-win7-fontsAUR — Windows 7 字体
- ttf-ms-office365AUR — Office 365 字体
- ttf-office-2007-fontsAUR — Office 2007 字体

使用 Windows 分区中的字体

如果有一个 Windows 分区已被挂载，可以直接使用其中的字体。有以下几种方法（假设 Windows 的 `C:\` 盘被挂载在 `/windows`）：

- 使用符号链接：

```
# ln -s /windows/Windows/Fonts /usr/local/share/fonts/WindowsFonts
```

注意：google-chromeAUR 的用户不应该选择这种方法，因为链接的字体会导致 Chrome 崩溃。

- 将字体文件复制到 `/usr/local/share/fonts/`：

```
# mkdir -p /usr/local/share/fonts/WindowsFonts
# cp -r /windows/Windows/Fonts/* /usr/local/share/fonts/WindowsFonts/
# chmod -R 644 /usr/local/share/fonts/WindowsFonts/*
```

- 使用绑定挂载：

```
# mount --bind /windows/Windows/Fonts /usr/local/share/fonts/WindowsFonts
```

要使其永久生效，像这样修改 fstab 文件：

```
/etc/fstab
/windows/Windows/Fonts    /usr/local/share/fonts/WindowsFonts    none    bind    0    0
```

更多详细信息参见 mount(8) § Bind mount operation。

请注意，若不能读取字体文件，可能需要对系统压缩的文件采取权变措施。

在选择以上任一方法后，重新生成字体缓存：

```
# fc-cache --force
# fc-cache-32 --force
```

从 Windows ISO 中提取字体

在 Windows ISO 文件中也可以找到字体。若 ISO 是在网络上下载的，则包含字体的镜像文件格式为 WIM (*Windows Imaging Format*)；若是使用 Windows 媒体创建工具创建的，则为 ESD (*Windows Electronic Software Download*)。从 ISO 文件中提取 `sources/install.esd` 或 `sources/install.wim` 文件并从中找到 `Windows/Fonts` 目录。它可以用 7z 或 *wimextract* (wimlib包) 提取。以下是一个使用 ***7z*** 的示例：

```
$ 7z e WinXY_YYMM_Chinese_Simplified_x64.iso sources/install.wim
$ 7z e install.wim 1/Windows/{Fonts/"*".
{ttf,ttc},System32/Licenses/neutral/"*"/"*"/license.rtf} -ofonts/
```

字体和许可证将放在 `fonts` 目录。

旧软件包

注意：由这些包提供的字体已过时，缺少现代微调指令和完整的字符集，但好处是不需要用户提供字体文件。建议使用#现有软件包所述的软件包。

ttf-ms-fontsAUR 包含：

- Andalé Mono
- Arial
- Arial Black
- Comic Sans
- Courier New
- Georgia)
- Impact
- Lucida Sans
- Lucida Console

- Microsoft Sans Serif
- Times New Roman
- Trebuchet
- Verdana
- Webdings

警告：根据原《微软最终用户许可协议》，使用上述字体时存在一些法律限制。

ttf-tahomaAUR 包含 Tahoma)。

ttf-vista-fontsAUR包括：

- Calibri
- Cambria
- Candara
- Consolas
- Constantia)
- Corbel)

适用于微软字体的 Fontconfig 规则

相似字体的映射规则

网站通常使用字体的通用名称 (helvetica, courier, times 或 times new roman)，Fontconfig 的一个规则文件 `/etc/fonts/conf.d/30-metric-aliases.conf` 将其替换为免费字体（Liberation、Google CrOS、GUST TeX Gyre……）。

要使用微软字体，需要将上述通用名称映射到微软的字体：

```
<?xml version="1.0"?>
<!DOCTYPE fontconfig SYSTEM "fonts.dtd">
<fontconfig>
<!-- Map generics to MS specifics -->
       <!-- PostScript -->
       <alias binding="same">
         <family>Helvetica</family>
         <accept>
         <family>Arial</family>
         </accept>
       </alias>
       <alias binding="same">
         <family>Times</family>
         <accept>
         <family>Times New Roman</family>
         </accept>
       </alias>
       <alias binding="same">
         <family>Courier</family>
         <accept>
         <family>Courier New</family>
         </accept>
       </alias>
</fontconfig>
```

建议在浏览器中，将 serif、sans-serif 和 monospace 字体也映射到微软字体。

禁用字体内嵌位图

一些微软的 TTF 字体，如 Calibri 和 Cambria，包含了特定大小的内嵌位图字体，这些字体不支持抗锯齿。如果启用了字体内嵌位图，在这些特定的尺寸下，字体不会被进行抗锯齿处理。通过字体配置可以禁用内嵌位图字体。

已知问题

基于 Poppler 的 PDF 阅读器无法正确显示符号

微软的 TrueType 符号字体（Symbol)）在 Poppler 上有问题，数学符号可能无法在基于 Poppler 的 PDF 阅读器中正确显示。

11.3　字体配置/中文

难度 进阶 | **篇幅** 18.5 KB | **所属** 字体

本节将讲到

- Android 显示效果的字体参考配置
- Windows 显示效果的字体参考配置
- 反锯齿效果的字体参考配置
- 禁用中文斜体

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：Article consists mainly of config dumps.（在Talk:字体配置/中文讨论）

全局修改字体配置，修改 `/etc/fonts/local.conf`。配置用户字体，修改 `$XDG_CONFIG_HOME/fontconfig/fonts.conf`。

Android 显示效果的字体参考配置

安装字体

安装 ttf-roboto包 noto-fonts包 noto-fonts-cjk包 adobe-source-han-sans-cn-fonts包 adobe-source-han-serif-cn-fonts包 ttf-dejavu包。

```
pacman -S ttf-roboto noto-fonts noto-fonts-cjk adobe-source-han-sans-cn-fonts adobe-source-han-serif-cn-fonts ttf-dejavu
```

详见 archlinux wiki 。

添加配置文件

`~/.config/fontconfig/fonts.conf` 或 `/etc/fonts/local.conf`：

```
<?xml version="1.0"?>
<!DOCTYPE fontconfig SYSTEM "fonts.dtd">
<fontconfig>
```

```
  <its:rules xmlns:its="http://www.w3.org/2005/11/its" version="1.0">
    <its:translateRule
      translate="no"
      selector="/fontconfig/*[not(self::description)]"
    />
  </its:rules>

  <description>Android Font Config</description>

  <!-- Font directory list -->

  <dir>/usr/share/fonts</dir>
  <dir>/usr/local/share/fonts</dir>
  <dir prefix="xdg">fonts</dir>
  <!-- the following element will be removed in the future -->
  <dir>~/.fonts</dir>

  <!-- 关闭内嵌点阵字体 -->
  <match target="font">
    <edit name="embeddedbitmap" mode="assign">
      <bool>false</bool>
    </edit>
  </match>

  <!-- 英文默认字体使用 Roboto 和 Noto Serif ,终端使用 DejaVu Sans Mono. -->
  <match>
    <test qual="any" name="family">
      <string>serif</string>
    </test>
    <edit name="family" mode="prepend" binding="strong">
      <string>Noto Serif</string>
    </edit>
  </match>
  <match target="pattern">
    <test qual="any" name="family">
      <string>sans-serif</string>
    </test>
    <edit name="family" mode="prepend" binding="strong">
      <string>Roboto</string>
    </edit>
  </match>
  <match target="pattern">
    <test qual="any" name="family">
      <string>monospace</string>
    </test>
    <edit name="family" mode="prepend" binding="strong">
      <string>DejaVu Sans Mono</string>
    </edit>
  </match>

  <!-- 中文默认字体使用思源黑体和思源宋体,不使用 Noto Sans CJK SC 是因为这个字体会在特定情况下显示片假字. -->
  <match>
    <test name="lang" compare="contains">
      <string>zh</string>
    </test>
    <test name="family">
      <string>serif</string>
    </test>
    <edit name="family" mode="prepend">
      <string>Source Han Serif CN</string>
    </edit>
  </match>
  <match>
    <test name="lang" compare="contains">
      <string>zh</string>
    </test>
    <test name="family">
      <string>sans-serif</string>
```

```
    </test>
    <edit name="family" mode="prepend">
      <string>Source Han Sans CN</string>
    </edit>
  </match>
  <match>
    <test name="lang" compare="contains">
      <string>zh</string>
    </test>
    <test name="family">
      <string>monospace</string>
    </test>
    <edit name="family" mode="prepend">
      <string>Noto Sans Mono CJK SC</string>
    </edit>
  </match>

  <!-- Windows & Linux Chinese fonts. -->
  <!-- 把所有常见的中文字体映射到思源黑体和思源宋体，这样当这些字体未安装时会使用思源黑体和思源宋体.
解决特定程序指定使用某字体，并且在字体不存在情况下不会使用fallback字体导致中文显示不正常的情况. -->
  <match target="pattern">
    <test qual="any" name="family">
      <string>WenQuanYi Zen Hei</string>
    </test>
    <edit name="family" mode="assign" binding="same">
      <string>Source Han Sans CN</string>
    </edit>
  </match>
  <match target="pattern">
    <test qual="any" name="family">
      <string>WenQuanYi Micro Hei</string>
    </test>
    <edit name="family" mode="assign" binding="same">
      <string>Source Han Sans CN</string>
    </edit>
  </match>
  <match target="pattern">
    <test qual="any" name="family">
      <string>WenQuanYi Micro Hei Light</string>
    </test>
    <edit name="family" mode="assign" binding="same">
      <string>Source Han Sans CN</string>
    </edit>
  </match>
  <match target="pattern">
    <test qual="any" name="family">
      <string>Microsoft YaHei</string>
    </test>
    <edit name="family" mode="assign" binding="same">
      <string>Source Han Sans CN</string>
    </edit>
  </match>
  <match target="pattern">
    <test qual="any" name="family">
      <string>SimHei</string>
    </test>
    <edit name="family" mode="assign" binding="same">
      <string>Source Han Sans CN</string>
    </edit>
  </match>
  <match target="pattern">
    <test qual="any" name="family">
      <string>SimSun</string>
    </test>
    <edit name="family" mode="assign" binding="same">
      <string>Source Han Serif CN</string>
    </edit>
  </match>
  <match target="pattern">
    <test qual="any" name="family">
```

```
      <string>SimSun-18030</string>
    </test>
    <edit name="family" mode="assign" binding="same">
      <string>Source Han Serif CN</string>
    </edit>
  </match>

  <!-- Load local system customization file -->
  <include ignore_missing="yes">conf.d</include>

  <!-- Font cache directory list -->

  <cachedir>/var/cache/fontconfig</cachedir>
  <cachedir prefix="xdg">fontconfig</cachedir>
  <!-- the following element will be removed in the future -->
  <cachedir>~/.fontconfig</cachedir>

  <config>
    <!-- Rescan configuration every 30 seconds when FcFontSetList is called -->
    <rescan>
      <int>30</int>
    </rescan>
  </config>
</fontconfig>
```

Windows 显示效果的字体参考配置

安装 MS core fonts

参见微软字体。

加入配置

```
<match target="font">
  <edit name="autohint">
    <bool>false</bool>
  </edit>
  <edit name="hinting">
    <bool>true</bool>
  </edit>
  <edit name="antialias">
    <bool>true</bool>
  </edit>
</match>

<!--
    Synthetic emboldening for fonts that do not have bold face available
 -->

<match target="font">
  <!-- check to see if the font is just regular -->
  <test name="weight" compare="less_eq">
    <const>medium</const>
  </test>
  <!-- check to see if the pattern requests bold -->
  <test target="pattern" name="weight" compare="more">
    <const>medium</const>
  </test>
  <!--
      set the embolden flag
      needed for applications using cairo, e.g. gucharmap, gedit, ...
  -->
  <edit name="embolden" mode="assign">
    <bool>true</bool>
  </edit>
  <!--
      set weight to bold
      needed for applications using Xft directly, e.g. Firefox, ...
```

```
  -->
  <edit name="weight" mode="assign">
    <const>bold</const>
  </edit>
</match>

<match target="font">
  <test target="pattern" name="lang" compare="contains">
    <string>zh</string>
    <string>ja</string>
    <string>ko</string>
  </test>
  <edit name="spacing">
    <const>proportional</const>
  </edit>
  <edit name="globaladvance">
    <bool>false</bool>
  </edit>
</match>

<match target="pattern">
  <test name="family">
    <string>SimSun</string>
    <string>SimSun-18030</string>
    <string>AR PL ShanHeiSun Uni</string>
    <string>AR PL New Sung</string>
    <string>MingLiU</string>
    <string>PMingLiU</string>
  </test>
  <edit binding="strong" mode="prepend" name="family">
    <string>Tahoma</string>
    <string>Arial</string>
    <string>Verdana</string>
    <string>DejaVu Sans</string>
    <string>Bitstream Vera Sans</string>
  </edit>
</match>

<match target="font">
  <test name="family" qual="any">
    <string>AR PL ShanHeiSun Uni</string>
    <string>AR PL New Sung</string>
    <string>SimSun</string>
    <string>NSimSun</string>
    <string>MingLiu</string>
    <string>PMingLiu</string>
  </test>
  <test name="pixelsize" compare="less_eq">
    <double>12</double>
  </test>
  <edit name="pixelsize" mode="assign">
    <double>12</double>
  </edit>
</match>

<match target="font">
  <test compare="eq" name="family" qual="any">
    <string>宋体</string>
    <string>新宋体</string>
    <string>SimSun</string>
    <string>NSimSun</string>
    <string>宋体-18030</string>
    <string>新宋体-18030</string>
    <string>SimSun-18030</string>
    <string>NSimSun-18030</string>
    <string>AR PL ShanHeiSun Uni</string>
    <string>AR PL New Sung</string>
    <string>MingLiU</string>
    <string>PMingLiU</string>
```

```
  </test>
  <test compare="less_eq" name="pixelsize">
    <double>16</double>
  </test>
  <edit mode="assign" name="hinting">
    <bool>true</bool>
  </edit>
  <edit mode="assign" name="autohint">
    <bool>false</bool>
  </edit>
  <edit name="antialias">
    <bool>false</bool>
  </edit>
  <edit mode="assign" name="hintstyle">
    <const>hintslight</const>
  </edit>
</match>

<match target="font">
  <test name="family">
    <string>Andale Mono</string>
    <string>Arial</string>
    <string>Comic Sans MS</string>
    <string>Georgia</string>
    <string>Impact</string>
    <string>Trebuchet MS</string>
    <string>Verdana</string>
    <string>Courier New</string>
    <string>Times New Roman</string>
    <string>Tahoma</string>
    <string>Webdings</string>
    <string>Albany AMT</string>
    <string>Thorndale AMT</string>
    <string>Cumberland AMT</string>
    <string>Andale Sans</string>
    <string>Andy MT</string>
    <string>Bell MT</string>
    <string>Monotype Sorts</string>
  </test>
  <test name="pixelsize" compare="less_eq">
    <double>16</double>
  </test>
  <edit name="autohint">
    <bool>false</bool>
  </edit>
  <edit name="antialias">
    <bool>false</bool>
  </edit>
</match>

<alias>
  <family>serif</family>
  <prefer>
    <family>Times New Roman</family>
    <family>Thorndale AMT</family>
    <family>Nimbus Roman No9 L</family>
    <family>DejaVu Serif</family>
    <family>Bitstream Vera Serif</family>
    <family>Luxi Serif</family>
    <family>Likhan</family>
    <family>FreeSerif</family>
    <family>Times</family>
    <family>SimSun</family>
    <family>SimSun-18030</family>
    <family>MingLiU</family>
    <family>WenQuanYi Bitmap Song</family>
    <family>AR PL ShanHeiSun Uni</family>
    <family>AR PL New Sung</family>
    <family>FZSongTi</family>
    <family>FZMingTiB</family>
    <family>AR PL SungtiL GB</family>
```

```
    <family>AR PL Mingti2L Big5</family>
    <family>Kochi Mincho</family>
    <family>UnBatang</family>
    <family>Baekmuk Batang</family>
    <family>HanyiSong</family>
    <family>ZYSong18030</family>
  </prefer>
</alias>
<alias>
  <family>sans-serif</family>
  <prefer>
    <family>Arial</family>
    <family>Albany AMT</family>
    <family>Nimbus Sans L</family>
    <family>Verdana</family>
    <family>DejaVu Sans</family>
    <family>Bitstream Vera Sans</family>
    <family>Luxi Sans</family>
    <family>FreeSans</family>
    <family>Helvetica</family>
    <family>SimSun</family>
    <family>SimSun-18030</family>
    <family>MingLiU</family>
    <family>WenQuanYi Bitmap Song</family>
    <family>AR PL ShanHeiSun Uni</family>
    <family>AR PL New Sung</family>
    <family>FZSongTi</family>
    <family>FZMingTiB</family>
    <family>AR PL SungtiL GB</family>
    <family>AR PL Mingti2L Big5</family>
    <family>Kochi Gothic</family>
    <family>UnDotum</family>
    <family>Baekmuk Gulim</family>
    <family>Baekmuk Dotum</family>
  </prefer>
</alias>
<alias>
  <family>monospace</family>
  <prefer>
    <family>Courier New</family>
    <family>Cumberland AMT</family>
    <family>Nimbus Mono L</family>
    <family>Andale Mono</family>
    <family>DejaVu Sans Mono</family>
    <family>Bitstream Vera Sans Mono</family>
    <family>Luxi Mono</family>
    <family>FreeMono</family>
    <family>NSimSun</family>
    <family>NSimSun-18030</family>
    <family>PMingLiU</family>
    <family>WenQuanYi Bitmap Song</family>
    <family>AR PL ShanHeiSun Uni</family>
    <family>AR PL New Sung</family>
    <family>FZSongTi</family>
    <family>FZMingTiB</family>
    <family>AR PL SungtiL GB</family>
    <family>AR PL Mingti2L Big5</family>
    <family>Kochi Gothic</family>
    <family>UnDotum</family>
    <family>Baekmuk Gulim</family>
    <family>Baekmuk Dotum</family>
    <family>HanyiSong</family>
    <family>ZYSong18030</family>
  </prefer>
</alias>
```

反锯齿效果的字体参考配置

```
<match target="font">
  <edit name="autohint">
    <bool>true</bool>
```

```
  </edit>
  <edit name="hintstyle">
    <const>hintfull</const>
  </edit>
  <edit name="antialias">
    <bool>true</bool>
  </edit>
</match>

<match target="font">
  <!-- check to see if the font is just regular -->
  <test name="weight" compare="less_eq">
    <const>medium</const>
  </test>
  <!-- check to see if the pattern requests bold -->
  <test target="pattern" name="weight" compare="more">
    <const>medium</const>
  </test>
  <!--
      set the embolden flag
      needed for applications using cairo, e.g. gucharmap, gedit, ...
  -->
  <edit name="embolden" mode="assign">
    <bool>true</bool>
  </edit>
  <!--
      set weight to bold
      needed for applications using Xft directly, e.g. Firefox, ...
  -->
  <edit name="weight" mode="assign">
    <const>bold</const>
  </edit>
</match>

<match target="font">
  <test target="pattern" name="lang" compare="contains">
    <string>zh</string>
    <string>ja</string>
    <string>ko</string>
  </test>
  <edit name="spacing">
    <const>proportional</const>
  </edit>
  <edit name="globaladvance">
    <bool>false</bool>
  </edit>
</match>

<match target="pattern">
  <test name="family">
    <string>SimSun</string>
    <string>SimSun-18030</string>
    <string>AR PL ShanHeiSun Uni</string>
    <string>AR PL New Sung</string>
    <string>MingLiU</string>
    <string>PMingLiU</string>
  </test>
  <edit binding="strong" mode="prepend" name="family">
    <string>Tahoma</string>
    <string>Arial</string>
    <string>Verdana</string>
    <string>DejaVu Sans</string>
    <string>Bitstream Vera Sans</string>
  </edit>
</match>

<match target="font">
  <test name="family" qual="any">
    <string>AR PL ShanHeiSun Uni</string>
    <string>AR PL New Sung</string>
```

```
    <string>SimSun</string>
    <string>NSimSun</string>
    <string>MingLiu</string>
    <string>PMingLiu</string>
  </test>
  <test name="pixelsize" compare="less_eq">
    <double>12</double>
  </test>
  <edit name="pixelsize" mode="assign">
    <double>12</double>
  </edit>
</match>

<match target="font">
  <test compare="eq" name="family" qual="any">
    <string>宋体</string>
    <string>新宋体</string>
    <string>SimSun</string>
    <string>NSimSun</string>
    <string>宋体-18030</string>
    <string>新宋体-18030</string>
    <string>SimSun-18030</string>
    <string>NSimSun-18030</string>
    <string>AR PL ShanHeiSun Uni</string>
    <string>AR PL New Sung</string>
    <string>MingLiU</string>
    <string>PMingLiU</string>
  </test>
  <test compare="less_eq" name="pixelsize">
    <double>16</double>
  </test>
  <edit mode="assign" name="hinting">
    <bool>true</bool>
  </edit>
  <edit mode="assign" name="autohint">
    <bool>false</bool>
  </edit>
  <edit name="antialias">
    <bool>false</bool>
  </edit>
  <edit mode="assign" name="hintstyle">
    <const>hintslight</const>
  </edit>
</match>

<alias>
  <family>serif</family>
  <prefer>
    <family>Nimbus Roman No9 L</family>
    <family>Thorndale AMT</family>
    <family>DejaVu Serif</family>
    <family>Bitstream Vera Serif</family>
    <family>Times New Roman</family>
    <family>Luxi Serif</family>
    <family>Likhan</family>
    <family>FreeSerif</family>
    <family>Times</family>
    <family>SimSun</family>
    <family>SimSun-18030</family>
    <family>MingLiU</family>
    <family>WenQuanYi Bitmap Song</family>
    <family>AR PL ShanHeiSun Uni</family>
    <family>AR PL New Sung</family>
    <family>FZSongTi</family>
    <family>FZMingTiB</family>
    <family>AR PL SungtiL GB</family>
    <family>AR PL Mingti2L Big5</family>
    <family>Kochi Mincho</family>
    <family>UnBatang</family>
    <family>Baekmuk Batang</family>
```

```
    <family>HanyiSong</family>
    <family>ZYSong18030</family>
  </prefer>
</alias>
<alias>
  <family>sans-serif</family>
  <prefer>
    <family>DejaVu Sans</family>
    <family>Bitstream Vera Sans</family>
    <family>Luxi Sans</family>
    <family>Arial</family>
    <family>Verdana</family>
    <family>Albany AMT</family>
    <family>Nimbus Sans L</family>
    <family>FreeSans</family>
    <family>Helvetica</family>
    <family>SimSun</family>
    <family>SimSun-18030</family>
    <family>MingLiU</family>
    <family>WenQuanYi Bitmap Song</family>
    <family>AR PL ShanHeiSun Uni</family>
    <family>AR PL New Sung</family>
    <family>FZSongTi</family>
    <family>FZMingTiB</family>
    <family>AR PL SungtiL GB</family>
    <family>AR PL Mingti2L Big5</family>
    <family>Kochi Gothic</family>
    <family>UnDotum</family>
    <family>Baekmuk Gulim</family>
    <family>Baekmuk Dotum</family>
  </prefer>
</alias>
<alias>
  <family>monospace</family>
  <prefer>
    <family>DejaVu Sans Mono</family>
    <family>Bitstream Vera Sans Mono</family>
    <family>Luxi Mono</family>
    <family>Courier New</family>
    <family>Cumberland AMT</family>
    <family>Nimbus Mono L</family>
    <family>Andale Mono</family>
    <family>FreeMono</family>
    <family>NSimSun</family>
    <family>NSimSun-18030</family>
    <family>PMingLiU</family>
    <family>WenQuanYi Bitmap Song</family>
    <family>AR PL ShanHeiSun Uni</family>
    <family>AR PL New Sung</family>
    <family>FZSongTi</family>
    <family>FZMingTiB</family>
    <family>AR PL SungtiL GB</family>
    <family>AR PL Mingti2L Big5</family>
    <family>Kochi Gothic</family>
    <family>UnDotum</family>
    <family>Baekmuk Gulim</family>
    <family>Baekmuk Dotum</family>
    <family>HanyiSong</family>
    <family>ZYSong18030</family>
  </prefer>
</alias>
```

禁用中文斜体

针对中文字体添加「斜体」属性，即可让此字体的「斜体」不再倾斜（注意替换你所使用的中文字体名）。更改此项配置之后需要执行 `fc-cache -f`。

```
  <match target="scan">
    <test qual="any" name="family">
```

```
      <string>Noto Sans CJK SC</string>
    </test>
    <edit name="slant" binding="same" mode="append">
      <const>Italic</const>
    </edit>
  </match>
```

11.4 字体配置

难度 高级 | **篇幅** 28.2 KB | **所属** 字体

本节将讲到

- 字体路径
- Fontconfig 配置
- 不支持 Fontconfig 的应用程序
- 疑难问题
- 参见

Fontconfig 是一个用于向应用程序提供可用字体列表，以及字体渲染配置的库。

FreeType 库依此配置渲染字体。freetype2包 字体渲染包启用了字节码解释器（bytecode interpreter，BCI）以优化字体渲染，尤其是在 LCD 屏幕上。参见 #Fontconfig 配置和字体配置/示例。

尽管Fontconfig在现代类Unix系统中广泛使用，但部分应用程序仍在使用原始的X Logical Font Description（XLFD）作为字体选择与显示方法。

字体路径

为了使字体能够被应用程序发现，必须对它们进行编目，以便快速访问。

Fontconfig 的默认字体路径是 `/usr/share/fonts/` 和 `~/.local/share/fonts`（以及过时的 `~/.fonts/`）。Fontconfig 将在这些路径下递归查找字体。为了便于安装与管理，在安装字体时，建议使用这些路径。

以下命令列出 Fontconfig 已知的字体文件：

```
$ fc-list ':' file
```

fc-list(1) 中有更多的输出格式。

Xorg 已知的字体路径可通过其日志了解：

```
$ grep /fonts ~/.local/share/xorg/Xorg.0.log
```

提示：

- 也可使用 `xset q` 命令查看 Xorg 的已知字体。
- 若 Xorg 以根用户权限运行，应查看 `/var/log/Xorg.0.log`。

注意：

Xorg 不像 Fontconfig 一样递归查找 `/usr/local/share/fonts`。添加路径时，必须像这样添加绝对路径：

```
Section "Files"
FontPath     "/usr/share/fonts/local/"
EndSection
```

有关 Xorg 读取配置的更多细节，请阅读Xorg#配置。如果要为用户单独配置字体路径，可以像这样修改对应的 `~/.xinitrc` 来添加或移除默认的字体路径：

```
xset +fp /usr/share/fonts/local/           # 将一个自定义字体路径添加到 Xorg 的已知字体路径列表开头
xset -fp /usr/share/fonts/sucky_fonts/     # 从 Xorg 的已知字体路径列表中移除指定路径
```

要查看 Xorg 已知的字体列表，可使用 xorg-xlsfonts包 软件包中的 `xlsfonts` 命令。

Fontconfig 配置

Fontconfig 的配置文档见 fonts-conf(5)。

用户配置文件位于`$XDG_CONFIG_HOME/fontconfig/fonts.conf`（通常是 `$HOME/.config/fontconfig/fonts.conf`）。全局配置文件位于`/etc/fonts/local.conf`。 用户配置优先级高于全局配置。两个文件使用相同的语法。

注意：

- 需要启用 `50-user.conf` 预设才能让用户配置生效，启用 `51-local.conf` 预设才能让全局配置生效。阅读#预设一节了解如何启用这些预设。
- `~/.fonts.conf/`、`~/.fonts.conf.d/` 和 `~/.fontconfig/*.cache-*` 已过时，以后的包将不再读取这些路径和文件。对应的新路径分别是 `$XDG_CONFIG_HOME/fontconfig/fonts.conf`、`$XDG_CONFIG_HOME/fontconfig/conf.d/NN-name.conf` 和 `$XDG_CACHE_HOME/fontconfig/*.cache-*`。如果使用第二个路径，确保文件命名正确（`NN` 是两位数，如 `00`、`10` 和 `99`）。

Fontconfig 把所有配置集中到一个单独的中心文件（`/etc/fonts/fonts.conf`）中。Fontconfig 升级时会替换这个文件，因此不应当直接修改它。支持 Fontconfig 的程序查看这个文件以获取可用的字体及如何渲染之；重启这类程序即可载入新的配置。该文件是全局配置（`/etc/fonts/local.conf`）、预设配置（`/etc/fonts/conf.d/`）、用户配置（`$XDG_CONFIG_HOME/fontconfig/fonts.conf`）的组合。使用 `fc-cache` 命令重建 Fontconfig 的配置，但只有新运行的程序会使用新的配置。

注意：某些桌面环境（如 GNOME 和 KDE Plasma）的字体设置界面会自动创建或覆盖用户的字体配置文件。对于此类桌面环境，最好让设置界面中的字体设置和配置文件相符合。同时请确定桌面的*区域设置*或 locale 受配置好的字体支持，否则字体配置可能被覆盖。

Fontconfig 配置文件使用 XML 格式，其需要的头部如下：

```
<?xml version="1.0"?>
<!DOCTYPE fontconfig SYSTEM "urn:fontconfig:fonts.dtd">
<fontconfig>

  <!-- settings go here -->

</fontconfig>
```

文中的配置示例省略了这些标签。

预设

预设存放在 `/usr/share/fontconfig/conf.avail`。可以通过创建符号链接到此处来启用，无论是单用户配置还是全局配置。这些预设会覆盖各自配置文件中的对应设置。

例如，以下命令启用全局次像素 RGB 渲染：

```
# ln -s /usr/share/fontconfig/conf.avail/10-sub-pixel-rgb.conf /etc/fonts/conf.d/
```

以下命令启用用户级次像素 RGB 渲染：

```
$ mkdir $XDG_CONFIG_HOME/fontconfig/conf.d
$ ln -s /usr/share/fontconfig/conf.avail/10-sub-pixel-rgb.conf
$XDG_CONFIG_HOME/fontconfig/conf.d
```

抗锯齿（Anti aliasing）

字体光栅化将字体从向量数据（比如一个TrueType字体）转换为光栅或者位图数据。由于混叠现象，转换后的字体可能会出现锯齿状边缘。抗锯齿技术可以使边缘看起来更平滑。

Fontconfig**默认启用**抗锯齿，如要禁用：

```
  <match target="font">
    <edit name="antialias" mode="assign">
      <bool>false</bool>
    </edit>
  </match>
```

注意：某些程序可能会覆盖默认的抗锯齿设置。

字体微调（hinting）

字体微调使用数学指令来调整轮廓字体的显示，使其与光栅化网格（如显示器上的像素网格）对齐。它的预期效果是使字体看起来更清晰，从而使它们更容易阅读。当显示器拥有 300 左右的 DPI 时，字体即使不经微调也能正确排列。

字节码解释器（BCI）

TrueType 字体的微调指令由 FreeType 的解释器使用 BCI 微调渲染。通常来说，微调指令完备的字体使用 BCI 微调效果很好。

微调是**默认启用**的，如要禁用：

```
  <match target="font">
    <edit name="hinting" mode="assign">
      <bool>false</bool>
    </edit>
  </match>
```

注意：具体的 BCI 实现可以在脚本 `/etc/profile.d/freetype2.sh` 中修改，通过传递参数 `truetype:interpreter-version=*NN*` 设置解释器。*NN* 对应所选择的版本，传递给脚本中的 `FREETYPE_PROPERTIES` 变量。一些流行的选项如下：

- 35 - 经典模式，模拟 Windows 98
- **40** - 最小模式（精简版的无穷模式，默认值）

次像素渲染应该使用次像素 BCI。详见 。

自动微调（autohint）

自动微调依自己的算法尝试微调，无视字体内所有的微调指令。以前这是默认选项，因为 TrueType2 字体受专利保护，但现在专利已过期，基本不需要使用它。如果字体的微调指令很少或没有，它的效果会更好，否则效果是不如 BCI 的。常见字体都有完备的微调指令，因此自动微调用处不大，现在是**默认关闭**的。如要启用：

```
  <match target="font">
    <edit name="autohint" mode="assign">
      <bool>true</bool>
    </edit>
  </match>
```

微调风格（hint style）

微调风格是指对字体进行调整显示、使其与光栅化网格对齐的调整量。支持的值有 `hintnone`、`hintslight`、`hintmedium` 和 `hintfull`，微调程度依次加深，即字形变得更扭曲、更清晰——`hintslight` 会使字体更模糊，无法与网格对齐，但在保留字体形状方面会更好（参见 。），而 `hintfull` 会使字体更清晰，能很好地与像素网格对齐，但会让字体严重变形。对于非 CFF（.otf）字体，`hintslight` 设置会在竖直方向上隐式使用自动微调，以支持这类字体的原生字体信息。

默认选项是 `hintslight`，如要更改：

```
  <match target="font">
    <edit name="hintstyle" mode="assign">
      <const>hintnone</const>
    </edit>
  </match>
```

注意：有些程序会覆盖默认的微调设置。

次像素排布

Fontconfig 需要知道你的次像素排布才能正确显示字体。使用次像素排布显示器测试（The Lagom LCD 显示器测试页面的一部分）来找出你的次像素排列。

次像素几何通过 `rgba` 属性配置，可以是：

- `rgb`——*红、绿、蓝*——最常见，大多数现代显示器使用
- `bgr`
- `vrgb`——纵向
- `vbgr`
- `none`——无次像素渲染

```
  <match target="font">
    <edit name="rgba" mode="assign">
      <const>rgb</const>
    </edit>
  </match>
```

注意：如果不开启#次像素渲染，freetype 将只关心子像素方向，不关心像素排布。例如 **RGB** 和 **BGR** 没有区别。

不支持 "Pentile" 和 "RGBY" 等不常见的次像素排布，这些排布偶尔出现在电视、OLED 和移动屏幕上。对于这些设备，最好放弃次像素渲染，使用 `none` 选项依赖灰度。

次像素渲染

次像素渲染（英文维基百科页面）通过单独控制一个像素的三个分色，在一个方向上获得三倍分辨率，以使得字体看看起来更锐利。在 Windows 上，这项技术叫做 zhwp:ClearType。

FreeType2 提供两种次像素渲染方式（`FT_CONFIG_OPTION_SUBPIXEL_RENDERING`），Harmony 和 ClearType 。从 FreeType 2.10.3 起，Arch 默认启用 ClearType 次像素渲染 。

启用 ClearType 次像素渲染时建议使用 LCD 滤镜。Harmony 次像素渲染不需要设置 LCD 滤镜，在默认 LCD 几何下，它等同于 ClearType 使用 `lcdlight` 滤镜 。参见下一节了解如何启用 LCD 滤镜及其好处。

LCD 滤镜（LCD filter）

LCD 滤镜是用来减轻 ClearType 次像素渲染的色边现象的。这是 FreeType 2 API 文档对其的描述，其中 FT_LcdFilter 一节解释了各个选项。这个网页展示了各个 LCD 滤镜的测试效果。

大多数用户可使用 `lcddefault`。其他选项包括：适合太粗或模糊的字体使用的滤镜 `lcdlight`、原始的 Cairo 滤镜 `lcdlegacy` 、完全关闭 LCD 滤镜的选项 `lcdnone`。

```
  <match target="font">
    <edit name="lcdfilter" mode="assign">
      <const>lcddefault</const>
    </edit>
  </match>
```

高级 LCD 滤镜选项

如果对内置的 LCD 过滤器不满意，可以通过构建一个自定义的 freetype2 软件包并修改硬编码的过滤器的方式，来非常具体地调整字体的渲染。Arch 构建系统可用于从源代码构建和安装软件包。

查看 freetype2包 的 PKGBUILD 并下载/解压构建文件。Arch 构建系统#通过 Git 获取 PKGBUILD 源码列出了一些方法。

编辑文件 `src/freetype-VERSION/include/freetype/config/ftoption.h` 并取消对 `FT_CONFIG_OPTION_SUBPIXEL_RENDERING` 宏的注释，启用 ClearType 次像素渲染。

然后，编辑文件 `src/freetype-VERSION/src/base/ftlcdfil.c` 并查找常量 `default_weights` 的定义：

```
 static const FT_LcdFiveTapFilter  default_weights =
    { 0x10, 0x40, 0x70, 0x40, 0x10 };
```

这个常量定义了一个应用于渲染的字形的低通滤波器。根据需要对其进行修改。(参考 freetype 邮件列表的讨论)

最后，保存文件，构建并安装自定义软件包：

```
$ makepkg --noextract
# pacman --remove --nodeps freetype2
# pacman --upgrade freetype2-VERSION-ARCH.pkg.tar.zstd
```

重启 X。现在，lcddefault 过滤器应该会以不同方式渲染字体。

某些字体/字形的单独设置

一些字体也许在 BCI 自动微调下外观不佳。可以为这些字体禁用微调：

```
...
<match target="font">
    <test name="family" qual="any">
        <string>My Font</string>
    </test>
    <edit name="hinting" mode="assign">
        <bool>false</bool>
    </edit>
</match>
...
```

设置默认和后备字体

匹配测试

设置默认或后备字体的一个可靠方法是添加一个 XML 片段来进行匹配测试。例如，通过设置 "binding" 属性，下面的设置将使 *Georgia* 回退到 *Gentium*。

```
...
<match target="pattern">
  <test qual="any" name="family" compare="eq"><string>Georgia</string></test>
```

```
  <edit name="family" mode="assign" binding="same"><string>Gentium Book
Plus</string></edit>
</match>
...
```

在上面的例子中，"compare" 属性可以是 "eq"（即完全等于 *Georgia*）、"contains"（例如，匹配 *Georgia* 或 *Georgia Pro*），或其他值。参见 fonts-conf(5)。

别名

另一种方法是使用 *<alias>* 来设置“首选”字体。*<family>* 元素指定的字体在匹配到实际字体前，会被修改为 *<prefer>* 字体列表中的一个字体，顺序由上至下。下面的例子使 *Helvetica* 回退到 *Bitstream Vera Sans*：

```
...
<alias>
    <family>Helvetica</family>
    <prefer>
        <family>Bitstream Vera Sans</family>
    </prefer>
</alias>
...
```

<alias> 也可以用来指定当缺少某些字形时使用的后备字体。例如，一个用户将 *Noto Sans* 设置为默认的无衬线字体后，安装了 *Helvetica Neue*，想用它书写拉丁文字。由于很多版本的 *Helvetica Neue* 并不包括希腊字符，这位用户希望使用 *FreeSans* 来书写希腊文字，因为它与 *Helvetica* 很像。但事实上，希腊文字将会回退到用户设置的默认字体 **Noto Sans**，而使用下面的方法则可以实现：

```
...
<alias>
    <family>Helvetica Neue LT Pro</family>
    <prefer>
        <family>Helvetica Neue LT Pro</family>
        <family>FreeSans</family>
    </prefer>
</alias>
...
```

如果用户只是想在使用的字体缺少相应字形时，回退到默认字体，则不需要上述设置。

字体白名单/黑名单

禁用位图字体

位图字体有时会作为缺失字体的后备字体，可能导致文本渲染为像素化或过大。使用 `70-no-bitmaps-except-emoji.conf` #预设来禁用此行为。

要禁用所有字体的嵌入式位图：

```
...
 <match target="font">
   <edit name="embeddedbitmap" mode="assign">
     <bool>false</bool>
   </edit>
```

```
 </match>
...
```

如果要禁用所有字体的嵌入式位图，仍可以为特定字体启用它们，以防该字体在没有嵌入式位图时无法正常工作。例如，对于 Noto Color Emoji：

```
<?xml version="1.0"?>
<!DOCTYPE fontconfig SYSTEM "urn:fontconfig:fonts.dtd">
<fontconfig>
  <match target="font">
    <edit name="embeddedbitmap" mode="assign">
      <bool>false</bool>
    </edit>
  </match>
  <match target="font">
    <test name="family" qual="any">
      <string>Noto Color Emoji</string>
    </test>
    <edit name="embeddedbitmap">
      <bool>true</bool>
    </edit>
  </match>
</fontconfig>
```

禁用位图字体缩放

要禁用位图字体的缩放（这通常会使它们模糊），删除 `/etc/fonts/conf.d/10-scale-bitmap-fonts.conf`。 注意这可能会破坏 Segoe UI Emoji 等表情符号字体的缩放，使它们变得巨大。

要重新启用位图字体的缩放，重新创建符号链接：

```
# ln -s /usr/share/fontconfig/conf.avail/10-scale-bitmap-fonts.conf /etc/fonts/conf.d/
```

自动生成粗体和斜体

规则优先级

Fontconfig 按数字顺序处理 `/etc/fonts/conf.d` 中的文件。因此，`01-aaa.conf` 和 `02-bbb.conf` 的规则将与单个 `01-aaabbb.conf` 文件具有相同的效果——首先包含 `01-aaa` 的规则，然后是 `02-bbb` 的规则。

通常来说，这意味着前缀较小文件将具有较高的优先级。但是，Fontconfig 的语法很灵活，允许新规则优先于现有规则。因此，建议#查看当前设置来测试规则相互作用的结果。

**本文或本章节的事实准确性存在争议。**

原因： 实际行为似乎与下面描述的不同。（在 Talk:字体配置#规则优先级 中讨论）

注意：在 `$XDG_CONFIG_HOME/fontconfig/fonts.conf` 和目录 `$XDG_CONFIG_HOME/fontconfig/conf.d` 中定义的用户规则是通过文件 `/etc/fonts/conf.d/50-user.conf` 载入的，并且通常优先于以较大数字开头的文件中定义的规则。

查看当前设置

要查看当前生效的设置，可使用 `fc-match --verbose`。例如：

```
$ fc-match --verbose Sans
family: "DejaVu Sans"(s)
hintstyle: 3(i)(s)
hinting: True(s)
...
```

可在 font-config(5) 中查询数字的意义。例如，*hintstyle: 3* 代表 *hintfull*。

不支持 Fontconfig 的应用程序

注意： 至少在使用 xcb 后端的 Qt 6.8 中，X11 资源优先级高于 fontconfig，因此这些设置将成为 Qt 6 客户端的权威配置，尽管它们完全支持 fontconfig。

某些应用程序如 URxvt 会忽略 Fontconfig 设置。你可以通过使用 `~/.Xresources` 来解决这个问题，但它不如 Fontconfig 灵活。示例（选项说明见 #Fontconfig 配置）：

```
~/.Xresources
Xft.autohint: 0
Xft.lcdfilter: lcddefault
Xft.hintstyle: hintslight
Xft.hinting: 1
Xft.antialias: 1
Xft.rgba: rgb
```

确保 X 启动时使用 `xrdb -q` 正确加载这些设置（更多信息请参阅 X 资源）。

疑难问题

字体模糊

系统 DPI 设置与显示器的硬件 DPI 不匹配会导致字体显示模糊不清；关于处理高 DPI 或混合 DPI 显示器的说明，请参见HiDPI。

一些字体在小尺寸下显示不正确（如 Calibri, Cambria, Monaco 等）

一些矢量字体内嵌有位图版本，这些位图字体主要用于在较小的尺寸下渲染。在这种情况下，使用公制兼容字体作为替代可以改善渲染效果。

你也可以通过禁用字体的嵌入式位图来强制在所有尺寸下使用矢量字体，但会牺牲一些渲染质量。

应用程序无视 Fontconfig 的微调或抗锯齿设置

一些应用程序或桌面环境可能会覆盖默认的 Fontconfig 微调和抗锯齿设置。这种情况可能发生在 GNOME 上，例如，当你使用 vlc包 或 smplayer包 等 Qt 应用程序时。在这种情况下，请使用该应用程序特定的配置程序。对于 GNOME，请尝试使用 gnome-tweaks包。

应用程序无视 GNOME 的微调设置

在 GNOME 下，有时会发生这样的情况：即使在 GNOME 设置中将微调设置为"无"，Firefox 之类的应用也会使用"完全"微调设置渲染字体，导致字体变得宽而锐利。在这种情况下，你必须在你的 `fonts.conf` 文件中添加微调设置：

```
<?xml version="1.0"?>
<!DOCTYPE fontconfig SYSTEM "urn:fontconfig:fonts.dtd">
<fontconfig>
 <match target="font">
  <edit mode="assign" name="hinting">
   <bool>false</bool>
  </edit>
 </match>
</fontconfig>
```

在上面的例子中，微调级别被设置为了 `grayscale`。

GTK 应用程序的字体微调不正确

在某些桌面环境（尤其是 GNOME 和 Plasma 之外的）中，一些 GTK 应用程序无法正确读取字体配置。为了解决这个问题，请安装 xsettingsd包 或 xsettingsd-gitAUR 并在每次系统启动时运行它。更多信息请参见 Xsettingsd 和xsettingsd wiki。

对于此问题，可以使用以下配置：

```
~/.xsettingsd
Xft/Hinting 1
Xft/HintStyle "hintslight"
Xft/Antialias 1
Xft/RGBA "rgb"
```

如果这在一些应用程序中依然不起作用，可以安装 xorg-xrdb包 并使用以下配置：

```
~/.Xresources
Xft.hinting: 1
Xft.hintstyle: hintslight
Xft.antialias: 1
Xft.rgba: rgb
```

然后在每次系统启动时执行脚本 `xrdb -merge ~/.Xresources` 来应用这些设置。参见 X resources 和 #不支持 Fontconfig 的程序。

此外，使用 GTK4 和 libadwaita 的程序会忽略字体微调设置。可以创建或修改以下配置来修正：

```
~/.config/gtk-4.0/settings.ini
[Settings]
gtk-hint-font-metrics=true
```

生成的 PDF 文件中的 Helvetica 字体模糊

如果执行如下命令

```
 fc-match helvetica
```

返回

```
 helvR12-ISO8859-1.pcf.gz: "Helvetica" "Regular"
```

那么由 xorg-fonts-75dpi包 提供的位图字体就有可能被嵌入到各种应用程序的"打印到文件"或"导出"功能所生成的 PDF 文件中。该位图字体可能是在安装整个 xorg包组 包组的时候安装的（通常不推荐这样做）。要解决这个问题，可以卸载该软件包，并安装gsfonts包（Type 1）或tex-gyre-fonts包（OpenType）字体，以获得 Helvetica（以及其他 PostScript/PDF 基本字体）的自由替代品。

FreeType 破坏了位图字体

一些用户报告说，在将 freetype2包 升级到 2.7.1 版本后，各个位图字体的名称发生了变化，导致终端模拟器和其他一些程序（如 dwm 或 dmenu）的字体混乱，因为它们会回退到另一种不同的字体进行显示。该问题是由 PCF 字体族格式的改变造成的，这在他们的*发布说明*中有所描述。从旧格式过渡到新格式的用户可以通过创建*字体别名*来解决这个问题，就像中描述的解决方案一样；该方案同时在下面给出：

假设我们想为terminus-font包创建一个别名，在之前提到的 freetype2包 更新中，它从 { {ic|Terminus}} 被改名为 `xos4 Terminus`，那么在 `/etc/fonts/conf.d/` 中创建一个文件，用于设置字体别名：

```
/etc/fonts/conf.d/99-TerminusPCFFont.conf
<?xml version="1.0"?>
 <!DOCTYPE fontconfig SYSTEM "urn:fontconfig:fonts.dtd">
 <fontconfig>
     <alias>
         <family>Terminus</family>
         <prefer><family>xos4 Terminus</family></prefer>
         <default><family>fixed</family></default>
     </alias>
 </fontconfig>
```

现在，一切都应该能够像之前一样正常工作，但务必重新加载 `.Xresources` 或重启显示服务器，以便使受影响的程序能够使用字体别名。

DejaVu 的等宽字体不能显示下划线

自 Pango 1.44 起，在以某些字号使用 DejaVu Sans Mono 字体时，下划线会消失。一种解决办法是使用 Liberation Mono 作为等宽字体，见#设置默认或后备字体。

调试 FreeType 字体

freetype2-demos包 提供用于调试 FreeType 字体配置的工具。`ftview` 是它的一个 GUI，你可以在其中实时预览字体的渲染设置。比如：

```
$ ftview -e unic -d 1024x768x24 -r 96 10 /usr/share/fonts/noto/NotoSans-Regular.ttf
```

文字模糊

某些应用程序（如 Chromium/Electron）没有正确应用伽马校正，有些在特定场景（灰度）下禁用了伽马校正，这会导致深色背景上的小文字模糊不清，在 <=1080p 的屏幕上出现难以阅读的文字。这是 Chromium/Electron 的一个长期存在的问题，一个

解决方法是使用 `FREETYPE_PROPERTIES="cff:no-stem-darkening=0 autofitter:no-stem-darkening=0"` 环境变量来启用 stem darkerning。

font-config-info

你可以使用 font-config-info-gitAUR 收集有效的字体配置信息。它会从多个 GTK 源、X resources、XSETTINGS 协议和 fontconfig包 查询信息。

参见

- Xft 教程 by Keith Packard
- X11 中的字体
- FreeType 2
- FreeType 2 API 参考：控制 FreeType 模块
- Gentoo 字体渲染讨论
- 关于轻微提示

11.5 字体配置/示例

难度 高级 | **篇幅** 33.6 KB | **所属** 字体

本节将讲到

- 微调字体
- 对斜体和粗体禁用微调
- 仅对较大的字体启用抗锯齿
- 禁用粗体
- 禁用等宽字体的连字

阅读主文章：字体配置。

配置可能一定程度上有所不同。请发布 Fontconfig 配置并解释原因。

中文用户可能更希望查看字体配置/中文。

微调字体

```
~/.config/fontconfig/fonts.conf
<?xml version='1.0'?>
<!DOCTYPE fontconfig SYSTEM "urn:fontconfig:fonts.dtd">
<fontconfig>
  <match target="font">
    <edit mode="assign" name="antialias">
      <bool>true</bool>
    </edit>
    <edit mode="assign" name="embeddedbitmap">
      <bool>false</bool>
    </edit>
    <edit mode="assign" name="hinting">
      <bool>true</bool>
    </edit>
    <edit mode="assign" name="hintstyle">
```

```
      <const>hintslight</const>
    </edit>
    <edit mode="assign" name="lcdfilter">
      <const>lcddefault</const>
    </edit>
    <edit mode="assign" name="rgba">
      <const>rgb</const>
    </edit>
  </match>
</fontconfig>
```

对斜体和粗体禁用微调

```
<?xml version='1.0'?>
<!DOCTYPE fontconfig SYSTEM "urn:fontconfig:fonts.dtd">
<fontconfig>
  <match target="font" >
    <edit mode="assign" name="autohint">  <bool>true</bool></edit>
    <edit mode="assign" name="hinting">     <bool>false</bool></edit>
    <edit mode="assign" name="lcdfilter"> <const>lcddefault</const></edit>
    <edit mode="assign" name="hintstyle"> <const>hintslight</const></edit>
    <edit mode="assign" name="antialias"> <bool>true</bool></edit>
    <edit mode="assign" name="rgba">      <const>rgb</const></edit>
  </match>

  <match target="font">
    <test name="pixelsize" qual="any" compare="more"><double>15</double></test>
    <edit mode="assign" name="lcdfilter"><const>lcdlight</const></edit>
    <edit mode="assign" name="hintstyle"><const>hintnone</const></edit>
  </match>

  <match target="font">
    <test name="weight" compare="more"><const>medium</const></test>
    <edit mode="assign" name="hintstyle"><const>hintnone</const></edit>
    <edit mode="assign" name="lcdfilter"><const>lcdlight</const></edit>
  </match>

  <match target="font">
    <test name="slant"  compare="not_eq"><double>0</double></test>
    <edit mode="assign" name="hintstyle"><const>hintnone</const></edit>
    <edit mode="assign" name="lcdfilter"><const>lcdlight</const></edit>
  </match>

</fontconfig>
```

仅对较大的字体启用抗锯齿

一些用户更喜欢抗锯齿所不能提供的，更清晰的渲染：

```
<?xml version="1.0"?>
<!DOCTYPE fontconfig SYSTEM "urn:fontconfig:fonts.dtd">
<fontconfig>
  <match target="font">
    <edit name="antialias" mode="assign">
      <bool>false</bool>
    </edit>
  </match>

  <match target="font" >
    <test name="size" qual="any" compare="more">
      <double>12</double>
    </test>
    <edit name="antialias" mode="assign">
      <bool>true</bool>
    </edit>
  </match>

  <match target="font" >
```

```
    <test name="pixelsize" qual="any" compare="more">
      <double>16</double>
    </test>
    <edit name="antialias" mode="assign">
      <bool>true</bool>
    </edit>
  </match>
</fontconfig>
```

禁用粗体

当某个字体不能很好地以粗体字形显示，并且不能在应用程序中禁用粗体字时（例如 st）有用。

```
...
<match target="pattern">
    <test qual="any" name="family">
        <string>Envy Code R</string>
    </test>
    <test name="weight" compare="more">
        <const>medium</const>
    </test>
    <edit name="weight" mode="assign" binding="same">
        <const>medium</const>
    </edit>
</match>
...
```

禁用等宽字体的连字

这可以防止像 "ffi" 这样的字母组合在某些等宽字体中被压缩成单个宽度的字符。需要复制整个 `<match>` 块以包含额外的字体。

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE fontconfig SYSTEM "urn:fontconfig:fonts.dtd">
<fontconfig>
  <description>Disable ligatures for monospaced fonts to avoid ff, fi, ffi, etc.
becoming only one character wide</description>

  <match target="font">
    <test name="family" compare="eq" ignore-blanks="true">
      <string>Nimbus Mono PS</string>
    </test>
    <edit name="fontfeatures" mode="append">
      <string>liga off</string>
      <string>dlig off</string>
    </edit>
  </match>
</fontconfig>
```

一些其他字体也可能需要禁用如 `calt` 和/或 `clig` 等排版特征。

您可以使用以下命令测试其有效性：

```
$ echo -e "| worksheet |\n| buffering |\n| difficult |\n| finishing |\n| different |\n|
efficient |" | pango-view --font="Nimbus Mono PS" /dev/stdin
```

有些程序（比如 Firefox）不支持 `fontfeatures` 标签，所以这对于那些用另一种字体替换的程序来说是唯一的选择。有关详细信息，参见字体配置#设置默认和后备字体。

默认字体

为了保证字体的一致性，所有应用程序都应该设置为使用衬线（serif）、无衬线（sans-serif）和等宽（monospace）别名，这些别名通过 Fontconfig 映射到特定的字体。有关选项和示例，请参见 Metric-compatible fonts。

阿拉伯语

以下示例 fonts.conf 为阿拉伯语指定了单独的默认字体，并为拉丁字母保留西式字体。您需要 ttf-arabeyes-fontsAUR 或 noto-fonts包 才能使以下各项工作。您还可以选择安装任何其他阿拉伯语字体，并根据您的喜好相应地更改下面的字体名称。

```
<?xml version='1.0'?>
<!DOCTYPE fontconfig SYSTEM "urn:fontconfig:fonts.dtd">
<fontconfig>

<!-- Default font for the Arabic language (no fc-match pattern) -->
 <match>
  <test compare="contains" name="lang">
   <string>ar</string>
  </test>
  <edit mode="prepend" name="family">
   <string>Tholoth</string>
  </edit>
 </match>
</fontconfig>
```

上述方法应该对大多数应用程序有用，但有些应用程序，如 Chromium 在语言匹配测试中不起作用。如果你发现一些应用程序不使用你选择的字体，你可以使用下面这些似乎有效的的 *alias* 和 *prefer* 标签。

```
<?xml version='1.0'?>
<!DOCTYPE fontconfig SYSTEM "urn:fontconfig:fonts.dtd">
<fontconfig>

<!-- Fallback fonts preference order -->
 <alias>
  <family>sans-serif</family>
  <prefer>
   <family>Noto Sans</family>
   <family>Open Sans</family>
   <family>Droid Sans</family>
   <family>Roboto</family>
   <family>Tholoth</family>
   <family>Noto Sans Arabic</family>
  </prefer>
 </alias>
 <alias>
  <family>serif</family>
  <prefer>
   <family>Noto Serif</family>
   <family>Droid Serif</family>
   <family>Roboto Slab</family>
   <family>Tholoth</family>
   <family>Noto Sans Arabic</family>
  </prefer>
 </alias>
 <alias>
  <family>monospace</family>
  <prefer>
   <family>Noto Sans Mono</family>
   <family>Inconsolata</family>
```

```
   <family>Droid Sans Mono</family>
   <family>Roboto Mono</family>
  </prefer>
 </alias>

</fontconfig>
```

日语

以下示例 fonts.conf 同样为日语区域设置（ja_JP）指定默认字体，并为拉丁字母保留西式字体。

```
<?xml version='1.0'?>
<!DOCTYPE fontconfig SYSTEM "urn:fontconfig:fonts.dtd">
<fontconfig>

<!-- Default font (no fc-match pattern) -->
 <match>
  <edit mode="prepend" name="family">
   <string>Noto Sans</string>
  </edit>
 </match>

<!-- Default font for the ja_JP locale (no fc-match pattern) -->
 <match>
  <test compare="contains" name="lang">
   <string>ja</string>
  </test>
  <edit mode="prepend" name="family">
   <string>Noto Sans CJK JP</string>
  </edit>
 </match>

<!-- Default sans-serif font -->
 <match target="pattern">
   <test qual="any" name="family"><string>sans-serif</string></test>
   <!--<test qual="any" name="lang"><string>ja</string></test>-->
   <edit name="family" mode="prepend" binding="same"><string>Noto Sans</string>
</edit>
 </match>

<!-- Default serif fonts -->
 <match target="pattern">
   <test qual="any" name="family"><string>serif</string></test>
   <edit name="family" mode="prepend" binding="same"><string>Noto Serif</string>
</edit>
   <edit name="family" mode="append" binding="same"><string>IPAPMincho</string>
</edit>
   <edit name="family" mode="append" binding="same"><string>HanaMinA</string>  </edit>
 </match>

<!-- Default monospace fonts -->
 <match target="pattern">
   <test qual="any" name="family"><string>monospace</string></test>
   <edit name="family" mode="prepend" binding="same"><string>Noto Sans
Mono</string></edit>
   <edit name="family" mode="append"
binding="same"><string>Inconsolatazi4</string></edit>
   <edit name="family" mode="append" binding="same"><string>IPAGothic</string></edit>
 </match>

<!-- Fallback fonts preference order -->
 <alias>
  <family>sans-serif</family>
  <prefer>
   <family>Noto Sans</family>
   <family>Open Sans</family>
   <family>Droid Sans</family>
```

```
   <family>Ubuntu</family>
   <family>Roboto</family>
   <family>NotoSansCJK</family>
   <family>Source Han Sans JP</family>
   <family>IPAPGothic</family>
   <family>VL PGothic</family>
   <family>Koruri</family>
  </prefer>
 </alias>
 <alias>
  <family>serif</family>
  <prefer>
   <family>Noto Serif</family>
   <family>Droid Serif</family>
   <family>Roboto Slab</family>
   <family>IPAPMincho</family>
  </prefer>
 </alias>
 <alias>
  <family>monospace</family>
  <prefer>
   <family>Noto Sans Mono</family>
   <family>Inconsolatazi4</family>
   <family>Ubuntu Mono</family>
   <family>Droid Sans Mono</family>
   <family>Roboto Mono</family>
   <family>IPAGothic</family>
  </prefer>
 </alias>

</fontconfig>
```

中文

```
~/.config/fontconfig/fonts.conf
or
/etc/fonts/local.conf
<?xml version="1.0"?>
<!DOCTYPE fontconfig SYSTEM "urn:fontconfig:fonts.dtd">
<fontconfig>

    <match target="font">
        <edit name="embeddedbitmap" mode="assign">
            <bool>false</bool>
        </edit>
    </match>

    <match>
        <test qual="any" name="family">
            <string>serif</string>
        </test>
        <edit name="family" mode="prepend" binding="strong">
            <string>Noto Serif</string>
        </edit>
    </match>
    <match target="pattern">
        <test qual="any" name="family">
            <string>sans-serif</string>
        </test>
        <edit name="family" mode="prepend" binding="strong">
            <string>Roboto</string>
        </edit>
    </match>
    <match target="pattern">
        <test qual="any" name="family">
            <string>monospace</string>
        </test>
        <edit name="family" mode="prepend" binding="strong">
            <string>DejaVu Sans Mono</string>
        </edit>
    </match>
```

```
    <match>
        <test name="lang" compare="contains">
            <string>zh</string>
        </test>
        <test name="family">
            <string>serif</string>
        </test>
        <edit name="family" mode="prepend">
            <string>Source Han Serif CN</string>
        </edit>
    </match>
    <match>
        <test name="lang" compare="contains">
            <string>zh</string>
        </test>
        <test name="family">
            <string>sans-serif</string>
        </test>
        <edit name="family" mode="prepend">
            <string>Source Han Sans CN</string>
        </edit>
    </match>
    <match>
        <test name="lang" compare="contains">
            <string>zh</string>
        </test>
        <test name="family">
            <string>monospace</string>
        </test>
        <edit name="family" mode="prepend">
            <string>Noto Sans Mono CJK SC</string>
        </edit>
    </match>

<!--Windows & Linux Chinese fonts. -->
    <match target="pattern">
        <test qual="any" name="family">
            <string>WenQuanYi Zen Hei</string>
        </test>
        <edit name="family" mode="assign" binding="same">
            <string>Source Han Sans CN</string>
        </edit>
    </match>
    <match target="pattern">
                <test qual="any" name="family">
                        <string>WenQuanYi Micro Hei</string>
                </test>
                <edit name="family" mode="assign" binding="same">
                        <string>Source Han Sans CN</string>
                </edit>
    </match>
        <match target="pattern">
                <test qual="any" name="family">
                        <string>WenQuanYi Micro Hei Light</string>
                </test>
                <edit name="family" mode="assign" binding="same">
                        <string>Source Han Sans CN</string>
                </edit>
        </match>
    <match target="pattern">
                <test qual="any" name="family">
                        <string>Microsoft YaHei</string>
                </test>
                <edit name="family" mode="assign" binding="same">
                        <string>Source Han Sans CN</string>
                </edit>
        </match>
        <match target="pattern">
                <test qual="any" name="family">
                        <string>SimHei</string>
```

```
                </test>
                <edit name="family" mode="assign" binding="same">
                        <string>Source Han Sans CN</string>
                </edit>
        </match>
        <match target="pattern">
                <test qual="any" name="family">
                        <string>SimSun</string>
                </test>
                <edit name="family" mode="assign" binding="same">
                        <string>Source Han Serif CN</string>
                </edit>
        </match>
        <match target="pattern">
                <test qual="any" name="family">
                        <string>SimSun-18030</string>
                </test>
                <edit name="family" mode="assign" binding="same">
                        <string>Source Han Serif CN</string>
                </edit>
        </match>
</fontconfig>
```

Noto Fonts 中文字体

使用 Noto Fonts，同时用文泉驿微米黑替换 Microsoft 字体：

```
~/.config/fontconfig/fonts.conf
or
/etc/fonts/local.conf
<?xml version="1.0"?>
<!DOCTYPE fontconfig SYSTEM "urn:fontconfig:fonts.dtd">
<fontconfig>
    <match target="font">
        <edit name="embeddedbitmap" mode="assign">
            <bool>false</bool>
        </edit>
    </match>
    <match>
        <test qual="any" name="family">
            <string>serif</string>
        </test>
        <edit name="family" mode="prepend" binding="strong">
            <string>Noto Serif</string>
        </edit>
    </match>
    <match target="pattern">
        <test qual="any" name="family">
            <string>sans-serif</string>
        </test>
        <edit name="family" mode="prepend" binding="strong">
            <string>Noto Sans</string>
        </edit>
    </match>
    <match target="pattern">
        <test qual="any" name="family">
            <string>monospace</string>
        </test>
        <edit name="family" mode="prepend" binding="strong">
            <string>Noto Sans Mono</string>
        </edit>
    </match>
    <match>
        <test name="lang" compare="contains">
            <string>zh</string>
        </test>
        <test name="family">
            <string>serif</string>
        </test>
        <edit name="family" mode="prepend">
            <string>Noto Serif CJK SC</string>
```

```
        </edit>
    </match>
    <match>
        <test name="lang" compare="contains">
            <string>zh</string>
        </test>
        <test name="family">
            <string>sans-serif</string>
        </test>
        <edit name="family" mode="prepend">
            <string>Noto Sans CJK SC</string>
        </edit>
    </match>
    <match>
        <test name="lang" compare="contains">
            <string>zh</string>
        </test>
        <test name="family">
            <string>monospace</string>
        </test>
        <edit name="family" mode="prepend">
            <string>Noto Sans Mono CJK SC</string>
        </edit>
    </match>
```

```
    <!--WenQuanYi Zen Hei -> WenQuanYi Micro Hei -->
    <match target="pattern">
        <test qual="any" name="family">
            <string>WenQuanYi Zen Hei</string>
        </test>
        <edit name="family" mode="assign" binding="same">
            <string>WenQuanYi Micro Hei</string>
        </edit>
    </match>
    <match target="pattern">
        <test qual="any" name="family">
            <string>WenQuanYi Zen Hei Lite</string>
        </test>
        <edit name="family" mode="assign" binding="same">
            <string>WenQuanYi Micro Hei Lite</string>
        </edit>
    </match>
    <match target="pattern">
        <test qual="any" name="family">
            <string>WenQuanYi Zen Hei Mono</string>
        </test>
        <edit name="family" mode="assign" binding="same">
            <string>WenQuanYi Micro Hei Mono</string>
        </edit>
    </match>
```

```
    <!--Microsoft YaHei, SimHei, SimSun -> WenQuanYi Micro Hei -->
    <match target="pattern">
        <test qual="any" name="family">
            <string>Microsoft YaHei</string>
        </test>
        <edit name="family" mode="assign" binding="same">
            <string>WenQuanYi Micro Hei</string>
        </edit>
    </match>
    <match target="pattern">
        <test qual="any" name="family">
            <string>SimHei</string>
        </test>
        <edit name="family" mode="assign" binding="same">
            <string>WenQuanYi Micro Hei</string>
        </edit>
    </match>
    <match target="pattern">
        <test qual="any" name="family">
            <string>SimSun</string>
```

```
        </test>
        <edit name="family" mode="assign" binding="same">
            <string>WenQuanYi Micro Hei</string>
        </edit>
    </match>
    <match target="pattern">
        <test qual="any" name="family">
            <string>SimSun-18030</string>
        </test>
        <edit name="family" mode="assign" binding="same">
            <string>WenQuanYi Micro Hei</string>
        </edit>
    </match>
</fontconfig>
```

首选拉丁字体的同时使用 CJK 字体

需要 noto-fonts-cjk包。

你可以将 `PT Serif`/`Roboto`/`Cascadia Code PL` 替换为您喜欢的衬线/无衬线/等宽字体。

```
~/.config/fontconfig/fonts.conf
<?xml version="1.0"?>
<!DOCTYPE fontconfig SYSTEM "urn:fontconfig:fonts.dtd">
<fontconfig>
  <!-- Default serif font -->
  <alias binding="strong">
    <family>serif</family>
    <prefer>
      <family>PT Serif</family>
    </prefer>
  </alias>

  <!-- Default sans-serif font -->
  <alias binding="strong">
    <family>sans-serif</family>
    <prefer>
      <family>Roboto</family>
    </prefer>
  </alias>

  <!-- Default monospace font -->
  <alias binding="strong">
    <family>monospace</family>
    <prefer>
      <family>Cascadia Code PL</family>
    </prefer>
  </alias>

  <!-- Default system-ui font -->
  <alias binding="strong">
    <family>system-ui</family>
    <prefer>
      <family>Roboto</family>
    </prefer>
  </alias>

  <!-- Serif CJK -->

  <!-- Default serif when the "lang" attribute is not given -->
  <!-- You can change this font to the language variant you want -->
  <match target="pattern">
    <test name="family">
      <string>serif</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Serif CJK SC</string>
```

```
    </edit>
  </match>

  <!-- Japanese -->
  <!-- "lang=ja" or "lang=ja-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>ja</string>
    </test>
    <test name="family">
      <string>serif</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Serif CJK JP</string>
    </edit>
  </match>

  <!-- Korean -->
  <!-- "lang=ko" or "lang=ko-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>ko</string>
    </test>
    <test name="family">
      <string>serif</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Serif CJK KR</string>
    </edit>
  </match>

  <!-- Chinese -->
  <!-- "lang=zh" or "lang=zh-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>zh</string>
    </test>
    <test name="family">
      <string>serif</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Serif CJK SC</string>
    </edit>
  </match>
  <!-- "lang=zh-hans" or "lang=zh-hans-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>zh-hans</string>
    </test>
    <test name="family">
      <string>serif</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Serif CJK SC</string>
    </edit>
  </match>
  <!-- "lang=zh-hant" or "lang=zh-hant-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>zh-hant</string>
    </test>
    <test name="family">
      <string>serif</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Serif CJK TC</string>
    </edit>
  </match>
  <!-- Compatible -->
  <!-- "lang=zh-cn" or "lang=zh-cn-*" -->
  <match target="pattern">
```

```
    <test name="lang" compare="contains">
      <string>zh-cn</string>
    </test>
    <test name="family">
      <string>serif</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Serif CJK SC</string>
    </edit>
  </match>
  <!-- "lang=zh-tw" or "lang=zh-tw-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>zh-tw</string>
    </test>
    <test name="family">
      <string>serif</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Serif CJK TC</string>
    </edit>
  </match>

  <!-- Sans CJK -->

  <!-- Default sans-serif when the "lang" attribute is not given -->
  <!-- You can change this font to the language variant you want -->
  <match target="pattern">
    <test name="family">
      <string>sans-serif</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans CJK SC</string>
    </edit>
  </match>

  <!-- Japanese -->
  <!-- "lang=ja" or "lang=ja-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>ja</string>
    </test>
    <test name="family">
      <string>sans-serif</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans CJK JP</string>
    </edit>
  </match>

  <!-- Korean -->
  <!-- "lang=ko" or "lang=ko-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>ko</string>
    </test>
    <test name="family">
      <string>sans-serif</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans CJK KR</string>
    </edit>
  </match>

  <!-- Chinese -->
  <!-- "lang=zh" or "lang=zh-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>zh</string>
    </test>
```

```
    <test name="family">
      <string>sans-serif</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans CJK SC</string>
    </edit>
  </match>
  <!-- "lang=zh-hans" or "lang=zh-hans-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>zh-hans</string>
    </test>
    <test name="family">
      <string>sans-serif</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans CJK SC</string>
    </edit>
  </match>
  <!-- "lang=zh-hant" or "lang=zh-hant-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>zh-hant</string>
    </test>
    <test name="family">
      <string>sans-serif</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans CJK TC</string>
    </edit>
  </match>
  <!-- "lang=zh-hant-hk" or "lang=zh-hant-hk-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>zh-hant-hk</string>
    </test>
    <test name="family">
      <string>sans-serif</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans CJK HK</string>
    </edit>
  </match>
  <!-- Compatible -->
  <!-- "lang=zh-cn" or "lang=zh-cn-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>zh-cn</string>
    </test>
    <test name="family">
      <string>sans-serif</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans CJK SC</string>
    </edit>
  </match>
  <!-- "lang=zh-tw" or "lang=zh-tw-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>zh-tw</string>
    </test>
    <test name="family">
      <string>sans-serif</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans CJK TC</string>
    </edit>
  </match>
  <!-- "lang=zh-hk" or "lang=zh-hk-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
```

```
      <string>zh-hk</string>
    </test>
    <test name="family">
      <string>sans-serif</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans CJK HK</string>
    </edit>
  </match>

  <!-- Mono CJK -->

  <!-- Default monospace when the "lang" attribute is not given -->
  <!-- You can change this font to the language variant you want -->
  <match target="pattern">
    <test name="family">
      <string>monospace</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans Mono CJK SC</string>
    </edit>
  </match>

  <!-- Japanese -->
  <!-- "lang=ja" or "lang=ja-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>ja</string>
    </test>
    <test name="family">
      <string>monospace</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans Mono CJK JP</string>
    </edit>
  </match>

  <!-- Korean -->
  <!-- "lang=ko" or "lang=ko-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>ko</string>
    </test>
    <test name="family">
      <string>monospace</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans Mono CJK KR</string>
    </edit>
  </match>

  <!-- Chinese -->
  <!-- "lang=zh" or "lang=zh-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>zh</string>
    </test>
    <test name="family">
      <string>monospace</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans Mono CJK SC</string>
    </edit>
  </match>
  <!-- "lang=zh-hans" or "lang=zh-hans-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>zh-hans</string>
    </test>
    <test name="family">
```

```
      <string>monospace</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans Mono CJK SC</string>
    </edit>
  </match>
  <!-- "lang=zh-hant" or "lang=zh-hant-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>zh-hant</string>
    </test>
    <test name="family">
      <string>monospace</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans Mono CJK TC</string>
    </edit>
  </match>
  <!-- "lang=zh-hant-hk" or "lang=zh-hant-hk-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>zh-hant-hk</string>
    </test>
    <test name="family">
      <string>monospace</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans Mono CJK HK</string>
    </edit>
  </match>
  <!-- Compatible -->
  <!-- "lang=zh-cn" or "lang=zh-cn-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>zh-cn</string>
    </test>
    <test name="family">
      <string>monospace</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans Mono CJK SC</string>
    </edit>
  </match>
  <!-- "lang=zh-tw" or "lang=zh-tw-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>zh-tw</string>
    </test>
    <test name="family">
      <string>monospace</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans Mono CJK TC</string>
    </edit>
  </match>
  <!-- "lang=zh-hk" or "lang=zh-hk-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>zh-hk</string>
    </test>
    <test name="family">
      <string>monospace</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans Mono CJK HK</string>
    </edit>
  </match>

  <!-- System UI CJK -->

  <!-- Default system-ui when the "lang" attribute is not given -->
```

```
  <!-- You can change this font to the language variant you want -->
  <match target="pattern">
    <test name="family">
      <string>system-ui</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans CJK SC</string>
    </edit>
  </match>

  <!-- Japanese -->
  <!-- "lang=ja" or "lang=ja-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>ja</string>
    </test>
    <test name="family">
      <string>system-ui</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans CJK JP</string>
    </edit>
  </match>

  <!-- Korean -->
  <!-- "lang=ko" or "lang=ko-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>ko</string>
    </test>
    <test name="family">
      <string>system-ui</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans CJK KR</string>
    </edit>
  </match>

  <!-- Chinese -->
  <!-- "lang=zh" or "lang=zh-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>zh</string>
    </test>
    <test name="family">
      <string>system-ui</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans CJK SC</string>
    </edit>
  </match>
  <!-- "lang=zh-hans" or "lang=zh-hans-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>zh-hans</string>
    </test>
    <test name="family">
      <string>system-ui</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans CJK SC</string>
    </edit>
  </match>
  <!-- "lang=zh-hant" or "lang=zh-hant-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>zh-hant</string>
    </test>
    <test name="family">
      <string>system-ui</string>
    </test>
```

```
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans CJK TC</string>
    </edit>
  </match>
  <!-- "lang=zh-hant-hk" or "lang=zh-hant-hk-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>zh-hant-hk</string>
    </test>
    <test name="family">
      <string>system-ui</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans CJK HK</string>
    </edit>
  </match>
  <!-- Compatible -->
  <!-- "lang=zh-cn" or "lang=zh-cn-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>zh-cn</string>
    </test>
    <test name="family">
      <string>system-ui</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans CJK SC</string>
    </edit>
  </match>
  <!-- "lang=zh-tw" or "lang=zh-tw-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>zh-tw</string>
    </test>
    <test name="family">
      <string>system-ui</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans CJK TC</string>
    </edit>
  </match>
  <!-- "lang=zh-hk" or "lang=zh-hk-*" -->
  <match target="pattern">
    <test name="lang" compare="contains">
      <string>zh-hk</string>
    </test>
    <test name="family">
      <string>system-ui</string>
    </test>
    <edit name="family" mode="append" binding="strong">
      <string>Noto Sans CJK HK</string>
    </edit>
  </match>
</fontconfig>
```

字体的备用样式集

某些字体通过 OpenType 功能为字符提供了可选的样式集。

通常，这些样式集被命名为 `ss0x`，包含针对单个字符的小更改。

以下示例演示如何将等宽版本 ttf-ibm-plex包 默认的虚线零更改为斜线零。

```
~/.config/fontconfig/fonts.conf
<?xml version="1.0"?>
<!DOCTYPE fontconfig SYSTEM "urn:fontconfig:fonts.dtd">
<fontconfig>
    <match target='font'>
```

```
        <test name='fontformat' compare='not_eq'>
            <string/>
        </test>
        <test name='family'>
            <string>IBM Plex Mono</string>
        </test>
        <edit name='fontfeatures' mode='assign_replace'>
            <string>ss03</string>
        </edit>
    </match>
</fontconfig>
```

查看 What are "Stylistic Sets?" 了解关于此的更多信息。

另请参阅

- Gentoo 论坛
- Ubuntu Wiki

11.6 字体

难度 高级 | **篇幅** 46.1 KB | **所属** 字体

本节将讲到

- 字体格式
- 安装
- 字体包
- 后备字体顺序
- 字体别名

翻译状态：

- 本文（或部分内容）译自 Fonts，最近一次同步于 2022-07-18，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

引自维基百科：“计算机字体或电脑字体，简称字体，是包含有一套字形与字符的电子数据文件。”

注意：部分字体许可协议可能会有一些法律限制。

字体格式

现在使用的计算机字体绝大多数都属于**点阵**或**轮廓**数据格式。

点阵字体：用点或者像素组成的矩阵代表每种字形的每种形式和尺寸的图像。 **轮廓字体，或称矢量字体**：使用贝塞尔曲线、绘图指令和数学公式描述每种字形，使得字符轮廓可以适应各种尺寸。

点阵格式

- Adobe 的位图字体分布格式（Glyph Bitmap Distribution Format，BDF）
- Xorg 的便携式编译格式（Portable Compiled Format，PCF）
- Linux 内核的控制台使用的个人电脑屏幕字体（PC Screen Font，PSF），不受 Xorg 支持（Unicode PSF 文件的扩展名是 `psfu`）

这些格式也可以被 gzip 压缩。可用的点阵字体请参见#点阵。

轮廓格式

- Adobe 的 PostScript 字体，有多种格式，如打印机 ASCII 字体（Printer Font ASCII，PFA）和打印机二进制字体（Printer Font Binary，PFB）
- Apple 和 Microsoft 的 TrueType（扩展名为 `ttf`）
- Microsoft 的 OpenType，基于 TrueType（扩展名为 `otf` 或 `ttf`）

在大多数情况下都可以忽略 TrueType 和 OpenType 之间的技术差异。

其它格式

排版程序 TeX 和配套的字体软件 *Metafont* 使用它们自己的传统方法渲染字体。部分用于这两个程序的字体文件扩展名有 `*pk`、`*gf`、`mf` 与 `vf`。更新的版本也支持 TrueType 和 OpenType 字体。

FontForge（fontforge包）是一个字体编辑程序，可以以自己基于文本的原生格式 `sfd`（*s*pline *f*ont *d*atabase）储存字体。

SVG 格式也有自己的字体描述方法。

安装

可以使用多种方法安装字体。

Pacman

可以使用 pacman 安装已启用的软件源中的字体和字体集。

可用的字体可以通过查询包查找（比如以 `font` 或 `ttf` 作为关键字查找）。

创建字体包

字体应该由 pacman 管理，这可以通过创建 Arch 包完成。这些包也可以在 AUR 中与社区分享。用于安装字体的包都非常相似；请参见字体打包指南。

可以用 `fc-query` 查看字体的字体族，例如：`fc-query -f '%{family}\n' /`文件/路径。FcPatternFormat(3) 中描述了输出的格式。

手动安装

要安装不在源中的字体，推荐的方法是#创建字体包。这使得 pacman 在以后能够更新或者移除它们。

或者，也可以手动安装字体：

- 要为单个用户安装，则安装到 `~/.local/share/fonts/`。
- 多数时候这就足够了，除非以其他用户的身份运行图形程序。
- 之前曾使用过 `~/.fonts/`，但现在已被弃用。
- 要为系统（所有用户）安装，则安装到 `/usr/local/share/fonts/`。
- 可能需要先创建该目录：`mkdir -p /usr/local/share/fonts`。
- `/usr/share/fonts/` 由包管理器管理，请勿手动修改。

是否要创建子目录由用户决定，各发行版也有不同。为清晰起见，可将每个字体族放在单独的目录里。Fontconfig 将递归搜索默认路径，确保可以发现子目录下的文件。

以下是一个目录结构示例：

```
/usr/local/share/fonts/
├── otf
│   └── SourceCodeVariable
│       ├── SourceCodeVariable-Italic.otf
│       └── SourceCodeVariable-Roman.otf
└── ttf
    ├── AnonymousPro
    │   ├── Anonymous-Pro-B.ttf
    │   ├── Anonymous-Pro-I.ttf
    │   └── Anonymous-Pro.ttf
    └── CascadiaCode
        ├── CascadiaCode-Bold.ttf
        ├── CascadiaCode-Light.ttf
        └── CascadiaCode-Regular.ttf
```

请确保所有用户都有读取字体文件的权限，即至少将文件 chmod 为 `444`，目录为 `555`。

要让 Xserver 直接载入字体（而不是使用某些*字体服务*），就需要将新增字体的目录加入到 FontPath 中。它位于 Xorg 配置文件（例如 `/etc/X11/xorg.conf` 或 `/etc/xorg.conf`) 中。更多详细内容请参见#较旧的应用程序。

最后，更新 Fontconfig 的缓存（通常不需要，因为使用 Fontconfig 库的软件会这样做）：

```
$ fc-cache
```

较旧的应用程序

对于不支持 Fontconfig 的旧应用程序（例如 GTK 1.x 应用程序和 `xfontsel`），需要在字体目录中创建索引：

```
$ mkfontscale
```

```
$ mkfontdir
```

或在一条命令中包含多个目录：

```
$ for dir in /字体/目录1/ /字体/目录2/; do xset +fp $dir; done && xset fp rehash
```

或者，如果字体被安装在不同的子文件夹中，比如在 `/usr/share/fonts` 下：

```
$ for dir in * ; do if [  -d  "$dir"  ]; then cd "$dir";xset +fp "$PWD" ;mkfontscale;
mkfontdir;cd .. ;fi; done && xset fp rehash
```

有时 X 服务器可能无法加载字体目录，这时需要重新扫描所有 `fonts.dir` 文件：

```
# xset +fp /usr/share/fonts/misc # 将新目录告诉 X 服务器
# xset fp rehash                 # 强制重新扫描
```

要查询字体是否已经生效：

```
$ xlsfonts | grep fontname
```

注意：许多软件包在安装时会自动配置 Xorg。这种情况下无需执行此步骤。

这也可以在 `/etc/X11/xorg.conf` 或 `/etc/X11/xorg.conf.d` 中全局设置。

这个例子演示了必须加入到 `/etc/X11/xorg.conf` 中的一节内容。请根据实际需要添加或删除路径。

```
# 让 X.Org 知道自定义字体目录
Section "Files"
    FontPath    "/usr/share/fonts/100dpi"
    FontPath    "/usr/share/fonts/75dpi"
    FontPath    "/usr/share/fonts/cantarell"
    FontPath    "/usr/share/fonts/cyrillic"
    FontPath    "/usr/share/fonts/encodings"
    FontPath    "/usr/share/fonts/misc"
    FontPath    "/usr/share/fonts/truetype"
    FontPath    "/usr/share/fonts/TTF"
    FontPath    "/usr/share/fonts/util"
EndSection
```

Pango 警告

如果系统使用 Pango，那它就会从 Fontconfig 读取字体来源。

```
(process:5741): Pango-WARNING **: failed to choose a font, expect ugly output. engine-
type='PangoRenderFc', script='common'
(process:5741): Pango-WARNING **: failed to choose a font, expect ugly output. engine-
type='PangoRenderFc', script='latin'
```

如果看到类似上面的错误，或在应用程序中看到方块而不是文字，就需要添加字体并更新字体缓存。以下示例以 ttf-liberation包 字体为例讲解（成功安装软件包后）如何解决这个问题，并以 root 权限运行使字体在系统范围内生效。

```
# fc-cache
/usr/share/fonts: caching, new cache contents: 0 fonts, 3 dirs
/usr/share/fonts/TTF: caching, new cache contents: 16 fonts, 0 dirs
/usr/share/fonts/encodings: caching, new cache contents: 0 fonts, 1 dirs
/usr/share/fonts/encodings/large: caching, new cache contents: 0 fonts, 0 dirs
/usr/share/fonts/util: caching, new cache contents: 0 fonts, 0 dirs
/var/cache/fontconfig: cleaning cache directory
fc-cache: succeeded
```

可以这样测试设置的默认字体：

```
$ fc-match
LiberationMono-Regular.ttf: "Liberation Mono" "Regular"
```

字体包

以下是一个有选择的列表，其中包括 AUR 以及官方存储库中的许多字体包。标记为“Unicode”的字体具有广泛的 Unicode 支持。

提示：Archfonts 是一个可用于生成官方存储库和 AUR 中所有 TTF 字体的概述的 Python 脚本。

点阵

注意：pango包 1.44 放弃支持 FreeType，转而支持 HarfBuzz，所以不再支持传统 BDF/PCF 点阵字体，因此一些应用程序（例如 gnome-terminal包）将不再能够使用此类字体，结果显示方块而不是字形。参见 FS#63297、Pango issue #386 和 HarfBuzz issue #1897。

- 默认的 8x16 大小
- Berry（pcf-spectrum-berryAUR）——8px
- Dina（dina-font包）——6pt、8pt、9pt、10pt；等宽；基于 Proggy
- Efont（efont-unicode-bdfAUR——10px、12px、14px、16px、24px；普通体、粗体和斜体
- Gohu（gohufontAUR）——11px、14px；普通体和粗体
- Lime（artwiz-fontsAUR）
- ProFont（ttf-profont-iixAUR）——10px、11px、12px、15px、17px、22px、29px；普通体
- Proggy（proggyfontsAUR）——有不同的变体
- Tamsyn （tamsyn-font包）
- Terminus（terminus-font包）
- Tewi（bdf-tewi-gitAUR）
- Unifont（覆盖最广泛，支持 Unicode 范围的所有字体）（bdf-unifontAUR）
- 适用于 Pango 1.44：
- ProFont（profont-otbAUR）——ProFont的 OpenType 位图（OpenType Bitmap，OTB）变体
- Misc Fixed（xorg-fonts-misc-otbAUR）
- Gohufont（gohufont-otbAUR）
- Cozette（cozette-otbAUR）
- Terminus（terminus-font包）
- AUR 中的其他 OTB 字体

拉丁文字

字体族

提供基本字体集的包:

- Bitstream Vera（ttf-bitstream-vera包）——包含无衬线、衬线和等宽字体。Bitstream Vera Sans 的公制与 Verdana 兼容。
- Croscore 字体（ttf-croscore包）——与 Helvetica、Times 和 Courier 公制兼容的字体，随 Chrome OS 提供。
- DejaVu 字体（ttf-dejavu包）——Bitstream Vera modified for greater Unicode coverage
- Droid（ttf-droid包）——Default font for older Android versions with wide Unicode coverage including CJK but not symbols and emojis
- GNU FreeFont（gnu-free-fonts包）——Includes three fonts that are open-source clones of Helvetica, Times, and Courier, but are metrically slightly different. Most Latin characters are from URW fonts (e.g., Nimbus Roman, Nimbus Sans), good Unicode coverage but does not include CJK
- IBM Plex（ttf-ibm-plex包）——Serif, sans-serif, condensed sans-serif and monospace with true italics
- Liberation 字体（ttf-liberation包）——Metric-compatible fonts for Helvetica, Times, and Courier, but are visually different
- Linux Libertine（ttf-linux-libertine包）——Serif (Libertine) and sans serif (Biolinum) fonts with large Unicode coverage
- 微软字体——Windows 和 Office 中的字体。
- Noto（noto-fonts包）——Google font family with full Unicode coverage if installed with its emoji and CJK optional dependencies

Packages not providing a base font set:

- B612 (ttf-b612AUR)——由 Airbus 赞助的开源字体系列（无衬线和等宽），专为在飞机驾驶舱屏幕上阅读而设计
- Luxi fonts (font-bh-ttfAUR)——类似 Lucida 的 X.Org 字体族
- Roboto (ttf-roboto包) – Default font for newer Android versions where it is complemented by Noto fonts for languages not supported like CJK
- TeX Gyre fonts (tex-gyre-fonts包) – Metric-compatible fonts for Helvetica, Times, and Courier by the Polish GUST association of TeX users. They have their own math companion fonts, see #数学.
- Ubuntu 字体族 (ttf-ubuntu-font-family包)

旧版 Microsoft 字体包：

- Microsoft 字体 (ttf-ms-fontsAUR) – Andalé Mono, Courier New, Arial, Arial Black, Comic Sans, Impact, Lucida Sans, Microsoft Sans Serif, Trebuchet, Verdana, Georgia, Times New Roman

- Vista 字体 (ttf-vista-fontsAUR) – Consolas, Calibri, Candara, Corbel, Cambria, Constantia

等宽

支持编程连字的字体标有 → 符号。更多等宽字体参见#点阵和#字体族。

- Anonymous Pro (ttf-anonymous-pro包, included in ttf-google-fonts-gitAUR)
- Cascadia Code (ttf-cascadia-code包) → – Designed to enhance the look of the Windows Terminal, with programming ligatures, released by Microsoft under the Open Font License.
- Courier Prime (ttf-courier-primeAUR) – Courier alternative which has been supplemented by a sans serif font and a version optimized for programming, released under the Open Font License.
- Envy Code R (ttf-envy-code-rAUR) – Font designed for programmers
- Fantasque Sans Mono (ttf-fantasque-sans-mono包, otf-fantasque-sans-mono包)
- Fira Mono) (ttf-fira-mono包, otf-fira-mono包) – Font optimized for small screens and adopted by Mozilla for the Firefox OS
- Fira Code#Fira_Code) (ttf-fira-code包) → – Extension of Fira Mono with programming ligatures for common programming multi-character combinations
- Hack (ttf-hack包) - Open source monospaced font, used as the default in KDE Plasma
- Hasklig (otf-haskligAUR) - 具有等宽连字的代码字体
- Hermit (otf-hermit包) - 由程序员为程序员设计的字体
- Inconsolata（ttf-inconsolata包，包含在 ttf-google-fonts-gitAUR 中）——为显示源代码设计，灵感来自 Consolas 和 Letter Gothic
- Inconsolata-g（ttf-inconsolata-gAUR）——添加一些对程序员友好的修改
- Iosevka (ttc-iosevka包) → – Slender sans-serif and slab-serif typeface inspired by Pragmata Pro, M+ and PF DIN Mono, designed to be the ideal font for programming; it supports programming ligatures and over 2000 latin, greek, cyrillic, phonetic and PowerLine glyphs
- JetBrains Mono (ttf-jetbrains-mono包) → – JetBrains 开发的免费开源字体
- Lucida Typewriter（包含在 jreAUR 软件包中）
- Menlo) (ttf-mesloAUR) – Customized version of Apple's Menlo Regular font for OS X with larger vertical gap spacing
- Monaco) (ttf-monacoAUR) – Apple 为 OS X 设计的专有字体
- Monofur (ttf-monofur包)
- Mononoki（ttf-mononokiAUR）——用于编程和 code review 的字体

- Source Code Pro（adobe-source-code-pro-fonts包，包含在 ttf-google-fonts-gitAUR 中）

相关网站：

- Trevor Lowing 的字体清单
- Slant：最好的编程字体是什么?
- Stack Overflow：推荐编程字体
- 编程字体 - 试驾
- 编程字体比较
- Coding Font by Typogram

无衬线

- Andika (ttf-andikaAUR)
- Cantarell) (cantarell-fonts包) – Default font supplied with GNOME, it is required by the GNOME and GTK 3 related packages
- DMCA Sans Serif (ttf-dmcasansserifAUR) – General purpose sans serif font metric-compatible with Microsoft Consolas
- Fira Sans) (ttf-fira-sans包, otf-fira-sans包) – Sans serif font designed by Erik Spiekermann for Mozilla and the Firefox OS. Fira Mono and Fira Code are monospaced companions of Fira Sans (see #等宽)
- FreeSans (gnu-free-fonts包) – Visually similar to Helvetica but metrically different, see #字体族
- Inter (inter-font包) – A geometric neo-grotesque font designed for user interfaces
- Jost* (otf-jostAUR) – An open-source typeface based on Futura)
- Liberation Sans (ttf-liberation包) – Metric-compatible with Helvetica but visually distinct, see #字体族
- Montserrat (montserrat-otfAUR) – An open source font that shares similarities with Gotham) and Proxima Nova
- Nunito (ttf-nunitoAUR) – An open source font with rounded terminal, hence shares similarities with Gotham Rounded#Variations) and Proxima Soft
- Open Sans (ttf-opensans包) – Sans serif font commissioned by Google, based on Droid sans but slightly wider.
- PT Sans (ttf-google-fonts-gitAUR) – 3 major variations: normal, narrow, and caption - Unicode: Latin, Cyrillic
- Source Sans (adobe-source-sans-fonts包) – Open source sans serif font from Adobe with a design based on News Gothic and Franklin Gothic

- Tahoma (Wine Replacement) (ttf-tahomaAUR) – Open source substitute for Tahoma) developed by the Wine project. It was created because many Windows applications expected Tahoma to be available

衬线

- Bitstream Charter (ttf-bitstream-charterAUR, otf-bitstream-charterAUR) – Originally a commercial font designed by Matthew Carter. A version was released under a free license and later converted to modern formats (provided as the aforementioned packages).
- Bodoni* (otf-bodoniAUR) – An open-source Bodoni revival
- Crimson (otf-crimson包) – An open-source font that shares similarities with Minion)
- EB Garamond (ebgaramond-otfAUR) – An open-source Garamond revival, the aforementioned package is the version developed by Octavio Pardo
- FreeSerif (gnu-free-fonts包) – Visually similar to Times New Roman but metrically different, see #字体族
- Gentium (gentium-plus-font包) – Unicode: Latin, Greek, Cyrillic, Phonetic Alphabet
- Heuristica#Derived_typefaces) (ttf-heuristicaAUR) – Based on a version of Utopia) that was released under a free license
- Liberation Serif (ttf-liberation包) – Metric-compatible with Times New Roman but visually distinct, see #字体族
- Libre Baskerville (ttf-librebaskervilleAUR) – An open-source Baskerville revival designed by Impallari Type
- Libre Caslon (otf-libre-caslonAUR) – An open-source Caslon revival designed by Impallari Type
- Linux Libertine (ttf-linux-libertine包) – Developed as a substitute of Times New Roman, but different both visually and metrically (the metric differences are more notable for italic and bold fonts)
- TeX Gyre Termes (tex-gyre-fonts包) – Visually similar to Times New Roman (but there are some minor metric differences), see #字体族
- Tinos (ttf-croscore包) – Metric-compatible with Times New Roman but visually distinct (and looks similar to Liberation Serif), see #字体族

非拉丁文字

古代文字

- ttf-ancient-fontsAUR——包含 Aegean、Egyptian、Cuneiform、Anatolian、Maya 和 Analecta 文字的 Unicode 符号的字体

阿拉伯文

- ttf-amiriAUR——一个典型的阿拉伯文誊抄体 (Naskh) 字体，一开始由 Amiria Press 采用。其拉丁字符基于 Crimson (otf-crimson包)
- ttf-arabeyes-fontsAUR——自由的阿拉伯文字体集合
- ttf-qurancomplex-fontsAUR——位于麦地那的 King Fahd Glorious Quran Printing Complex 制作的字体
- ttf-sil-lateefAUR——来自 SIL 的 Unicode 阿拉伯文字体
- ttf-sil-scheherazadeAUR——来自 SIL 的 Unicode 阿拉伯文字体（传统阿拉伯文字体的替代品）

孟加拉文

详见 Localization/Bengali#Fonts。

盲文

- ttf-ubraille AUR——包含 Unicode *盲文*符号的字体。

中日韩越文字

泛 CJK

- 思源字体——大量字体集合，全面支持简体中文、繁体中文、日文和韩文，具有一致的设计和外观。
- adobe-source-han-sans-otc-fonts包——思源黑体，无衬线字体
- adobe-source-han-serif-otc-fonts包——思源宋体，衬线字体
- noto-fonts-cjk包——大量字体集合，全面支持简体中文、繁体中文、日文和韩文，具有一致的设计和外观。它目前是 adobe-source-han-sans-otc-fonts包 重新贴标的版本。

提示：中文用户在安装思源cjk全集或者noto-cjk全集后，部分文字显示为日文异形字体，可参看Arch Linux Localization#修正简体中文显示为异体（日文）字形 进行解决。

中文

参见 Localization/Chinese#Fonts。

日文

参见 Localization/Japanese#Fonts。

韩文

参见 Localization/Korean#Fonts。

越南语

- ttf-hannom包 – 用于喃字的越南语 TrueType 字体

西里尔文字

另请参阅#拉丁文字。

- ttf-paratypeAUR – Font family by ParaType: sans, serif, mono, extended cyrillic and latin, OFL 许可证
- otf-russkopisAUR – A free OpenType cursive font for Cyrillic script

希腊文字

几乎所有 Unicode 字体都包含希腊代码集 (也包含多调变音符号)。某些额外字体包虽然不包含完整的 Unicode 集，但拥有高质量的希腊字字形 (也包含拉丁字)：

- otf-gfsAUR - 由 Greek Font Society 选用的 OpenType 字体
- ttf-mgopenAUR - 来自 Magenta 的专业 TrueType 字体

希伯来文字

- opensiddur-hebrew-fontsAUR – Large collection of Open-source licensed Hebrew fonts. There are also few Latin, Greek, Cyrillic, Arabic, and Amharic.
- culmusAUR——不错的自由的希伯来文字体集合。
- alefbetAUR – 2 Hebrew fonts (at the moment): the commonly used "David Libre", and the handwriting font "Gveret Levin".
- ttf-ms-fontsAUR——包含 Arial 和其他字体。

印地文字

另请参阅 Localization/Indic#Fonts。

高棉文字

- ttf-khmer包 - 涵盖高棉语 (Khmer) 文字的字体
- Hanuman (ttf-google-fonts-gitAUR)

蒙古语和通古斯语

- ttf-abkaiAUR – Fonts for Sibe, Manchu and Daur scripts (incomplete, currently in development)

波斯文字

- persian-fontsAUR – 用于安装 AUR 中所有波斯字体的元包。
- borna-fontsAUR – Borna Rayaneh Co. Persian B font series.
- iran-nastaliq-fontsAUR – 自由的 Unicode 书法波斯字体。
- iranian-fontsAUR – Iranian-Sans and Iranian-Serif Persian font family.

- ir-standard-fontsAUR – 伊朗信息与通信技术高级理事会（SCICT）的标准波斯字体。
- persian-hm-ftx-fontsAUR – A Persian font series derived from X Series 2, Metafont and FarsiTeX fonts with Kashida feature.
- persian-hm-xs2-fontsAUR – A Persian font series derived from X Series 2 fonts with Kashida feature.
- gandom-fontsAUR, parastoo-fontsAUR, sahel-fontsAUR, samim-fontsAUR, shabnam-fontsAUR, tanha-fontsAUR, vazirmatn-fontsAUR, vazir-code-fontsAUR – Beautiful Persian fonts made by Saber RastiKerdar.
- ttf-yasAUR – The Yas Persian font series (with **hollow zero**).
- ttf-x2AUR – Free fonts with support for Persian, Arabic, Urdu, Pashto, Dari, Uzbek, Kurdish, Uighur, old Turkish (Ottoman) and modern Turkish (Roman).

壮侗语系

- fonts-tlwgAUR – Collection of scalable Thai fonts
- ttf-laoAUR – Lao TTF font (Phetsarath_OT)

藏缅语族

- ttf-tibetan-machine包——藏文 (Tibetan) Machine TTFont
- ttf-sil-padaukAUR——支持多种使用缅文的语言的 Unicode 字体

Emoji 和符号

Unicode 标准的一部分用于称为“emoji”的象形字符。

Emoji 字体有不同的格式：CBDT/CBLC（Google）、SBIX（Apple）、COLR/CPAL（Microsoft）和 SVG（Mozilla/Adobe）。

只要安装了一种支持 Emoji 的字体，它就应当开箱即用。根据标准，Emoji 字体的回退需要额外的代码来处理。

有关 Emoji 的发现和输入，参见应用程序列表/工具#文字输入。

软件	CBDT/CBLC	SBIX	COLR/CPAL	SVG	Emoji font fallback
Freetype	是	是	是	否	–
Pango	Freetype	是			
WebKitGTK	Freetype	是			
Qt	Freetype	否			
Chromium	Freetype	是			
Firefox	Freetype	是	否，参阅 Firefox#字体故障排除以获得解决方案。		

注意：Qt 只能同时使用字体列表中的前 255 个。请确保你的首选后备字体中有 Emoji 字体。

CBDT/CBLC：

- noto-fonts-emoji包——Google 的开源 Emoji 14.0。
- ttf-joypixelsAUR——EmojiOne 创作者的专有 Emoji。
- ttf-twemojiAUR——Twitter 的开源 Emoji 13.0。

SVG：

- otf-openmojiAUR——施瓦本格明德德国设计大学的开源 Emoji 13.0。
- ttf-twemoji-colorAUR——Twitter 的开源 Emoji 13.0。

仅轮廓字体:

- ttf-symbolaAUR——提供许多 Unicode 符号，包括 Emoji。

颜文字，有时被称为“日式表情符号”，由来自各种字符集的字符组成，包括 CJK 和印度字符。下面的一组包涵盖了大部分现有颜文字中所使用的字符：gnu-free-fonts包、ttf-arphic-uming包 和 ttf-indic-otf包。

数学

- Computer Modern (ttf-cm-unicodeAUR, otf-cm-unicodeAUR)
- Computer Modern (otf-latin-modern包, otf-latinmodern-math包) – LaTeX 中使用的改进版本
- STIX fonts (otf-stixAUR) – Released under a royalty-free license and designed to be a math compansion of Times New Roman. The current version is called STIX Two Math.
- TeX Gyre math fonts (tex-gyre-math-fontsAUR) – Math compansions of TeX Gyre fonts (see #字体族). Notably, TeX Gyre Termes Math is a math compansion of Times New Roman.
- XITS fonts (otf-xitsAUR) – A fork of STIX and is therefore a math compansion of Times New Roman. XITS and STIX Two Math have similar levels of completeness for mathematical symbols and alphabets , but do have some visual differences.

此外，texlive-core包 和 texlive-fontsextra包 中包含许多数学字体，例如 Latin Modern Math 和 STIX 字体。有关配置参见 TeX Live#Making fonts available to Fontconfig。

其他操作系统字体

- ttf-mac-fontsAUR - Apple MacOS TrueType 字体

后备字体顺序

Fontconfig 会自动选择一个满足当前显示需求的字体。例如，如果浏览一个既有英文又有中文的窗口，而默认的字体不支持中文，它会自动用另外的字体显示中文。

Fontconfig 允许每个用户通过 `$XDG_CONFIG_HOME/fontconfig/fonts.conf` 调整字体的选择顺序。 如果要优先使用衬线字体，遇到中文再使用某个特定的中文字体，配置文件会是这样：

```
<?xml version="1.0"?>
<!DOCTYPE fontconfig SYSTEM "fonts.dtd">
<fontconfig>
<alias>
   <family>serif</family>
   <prefer>
     <family>喜欢的拉丁衬线字体名称</family>
     <family>中文字体名称</family>
   </prefer>
 </alias>
</fontconfig>
```

提示：

- 如果系统的 locale 是中文，请将 `FC_LANG` 设置为 `und`或`en-US:zh-CN`，否则中英文都会使用中文字体渲染。
- 修改配置后可以用 `fc-match -a monospace | head` 检查后备字体设置是否正确。

无衬线和等宽字体也可以增加一样的设置，更多信息请参考 fontconfig 手册。

另请参阅字体配置#设置默认和后备字体。

字体别名

有几种字体别名实际上代表着别的字体，以让应用程序可以使用类似的字体。最常见的别名是：`serif` 代表衬线字体（如 DejaVu Serif）；`sans-serif` 代表无衬线字体（如 DejaVu Sans）；以及 `monospace` 代表等宽字体（如 DejaVu Sans Mono）。然而，这些别名所代表的字体有可能会变化，但 KDE 和其他桌面环境的字体管理工具通常不会显示对应关系。

如果想通过别名反向查找是展现了哪种字体，请运行：

```
$ fc-match monospace
DejaVuSansMono.ttf: "DejaVu Sans Mono" "Book"
```

在这种情况下 monospace 别名展现的是 `DejaVuSansMono.ttf` 字体。

提示与技巧

列出所有已安装的字体

可以使用以下命令列出系统上所有已安装且可用的 Fontconfig 字体。

```
$ fc-list
```

列出特定语言的已安装字体

应用程序和浏览器根据 fontconfig 首选项和 Unicode 文本的可用字体字形选择和显示字体。要列出特定语言的已安装字体，请使用命令 `fc-list :lang="*双字母语言代码*"`。例如，要列出已安装的阿拉伯字体或支持阿拉伯字形的字体：

```
$ fc-list -f '%{file}\n' :lang=ar
/usr/share/fonts/TTF/FreeMono.ttf
/usr/share/fonts/TTF/DejaVuSansCondensed.ttf
/usr/share/fonts/truetype/custom/DroidKufi-Bold.ttf
/usr/share/fonts/TTF/DejaVuSansMono.ttf
/usr/share/fonts/TTF/FreeSerif.ttf
```

列出特定 Unicode 字符的已安装字体

要搜索支持特定 Unicode 码点的等宽字体：

```
$ fc-match -s monospace:charset=1F4A9
```

Set terminal font on-the-fly

For terminal emulators that use X resources, e.g. xterm or rxvt-unicode, fonts can be set by using escape sequences. Specifically, `echo -e "\033]710;$font\007"` to change the normal font (`*font` in `~/.Xresources`), and replace `710` with `711`, `712`, and `713` to change the `*boldFont`, `*italicFont`, and `*boldItalicFont`, respectively.

`$font` uses the same syntax as in `~/.Xresources` and can be anything the terminal emulator will support. (Example: `xft:dejavu sans mono:size=9`)

应用程序专用的字体缓存

Matplotlib (python-matplotlib包) 使用自己的字体缓存，因此更新字体后记得删除 `$HOME/.matplotlib/fontList.cache`, `$HOME/.cache/matplotlib/fontList.cache`, `$HOME/.sage/matplotlib-1.2.1/fontList.cache` 等文件，这样它才会重新生成缓存并找到新字体 。

终端中的双向文本

要在终端中正确显示阿拉伯语和希伯来语，请运行 BiCon（bicon-gitAUR）。

参见

- State of Text Rendering
- Font Library - Fonts under Free licenses
- Fonts on screenshots.debian.net

第 12 章　共 3 节　93 KB

KDE

本章解决 KDE 桌面在 Arch 上的落地问题：把它装起来、启动 Plasma、配置到顺手，并在出故障时能自己查出来。建议以《KDE》为必读主干，安装、启动、配置与疑难解答都集中在这一节，读完再动手；《KDE Wallet》相对独立，通读一遍了解密码库的安装与配置即可，用到时再回看提示与技巧；《Amarok》是音乐播放器专题，与桌面本身关系不大，不做音频用途可以直接跳过。

最容易踩的坑是把 Plasma 当作装完即用的成品，跳过配置直接启动，结果往往只得到空白或反复崩溃的桌面。

12.1　Amarok

难度 入门 | **篇幅** 5.1 KB | **所属** KDE

本节将讲到

- 安装
- 定制
- SHOUTcast
- Ampache/MP3 流式传输
- 收藏集数据库

Amarok 是一个有着直观的 Qt 界面且适用于 Linux 的音乐播放器和管理工具,它与 KDE 集成得很好。

安装

安装 amarokAUR 软件包。

Amarok 现在依赖于 Phonon ,所以你必须为其选择一个工作后端。请见 KDE#Phonon 。你也许还需要为选择的后端安装一些编解码器。

定制

与 GNOME 集成

请见统一 Qt 和 GTK 应用的外观来获得主要 GUI 的外观集成。

脚本与系统托盘

新的脚本和系统托盘可以直接在 Amarok (*工具* > *脚本管理* > *获取更多脚本*) 中找到或在 store.kde.org 找到。

Moodbar

Moodbar 是一个能将基本的进度条转换为取决于音轨旋律被划分成多个色块的进度条的功能。

安装 moodbarAUR.

然后前往*设置* > *配置 Amarok* 勾选 "在进度中显示 moodbar"。

因为 Amarok 2 不会生成 moodfile ,你可以根据这篇教程自己创造这些文件。

SHOUTcast

要想获取 SHOUTcast 使用 "SHOUTcast 服务" 脚本。启动 Amarok,前往*工具* > *脚本管理* > *获取更多脚本*,搜索 *SHOUTcast* 安装 *Shoutcast 服务*,重启 Amarok。然后你可以在 "互联网" 菜单中找到它。

另见: 如何使用 Amarok 流式传输到我自己的广播电台?,这篇文章推荐使用 Internet DJ Console,在AUR(idjcAUR)中可用。

Ampache/MP3 流式传输

如果你正在直接或使用 Ampache 插件流式传输 MP3,并且你正用着 GStreamer 后端,你就无法在查看音轨。安装需要的包:phonon-qt4-gstreamerAUR phonon-qt5-gstreamer包 gst-libav包。 然后前往 Amarok 的*设置* > *配置 Amarok* > *回放* > *配置 Phonon*> 的*后端*标签页。将 Gstreamer 设置为首选后端。

收藏集数据库

Amarok 2.x 可以使用 Sqlite (默认) 或 MySQL 储存收藏集数据库。有着大量收藏集的用户和有着更高的性能表现要求的用户可能更想使用 MySQL。

MySQL

对于基础的 MySQL 配置,请参考 MariaDB 页面。

当 Amarok 使用 MySQL 时,你需要创建一个可以访问数据库的 MySQL 用户。要想实现,输入下列命令:

```
# mysql -p -u root
# CREATE DATABASE amarokdb;
# USE amarokdb;
# GRANT ALL ON amarokdb.* TO amarokuser@localhost IDENTIFIED BY 'password-user';
# FLUSH PRIVILEGES;
# quit
```

这会创建一个名为 'amarokdb' 的数据库和一个名为 'amarokuser' ,有着密码 'password-user',可以从本地主机访问所述数据库的用户。如果你想从另一台电脑连接到你的数据库电脑,将命令更改为:

```
# GRANT ALL ON amarok.* TO amarokuser@'%' IDENTIFIED BY  'password-user';
```

要想将 Amarok 配置为使用 MySQL,进入 Amarok 配置界面,选择数据库并勾选 "使用外部 MySQL 数据库"。输入服务器(如果是本机数据库通常为 "localhost",否则输入远

程主机的名字),用户名(在这个例子中是 "amarokuser")和你选择的密码用户。不要忘记选择你的音乐收藏集的路径。

音频 CD 回放

如果你没有使用 KDE 桌面环境, Amarok 可能缺少需要的工具来播放音频 CD 。安装 audiocd-kio包 来获取这个功能。

Firefly/Daap 共享

要想让 Daap 共享在 Amarok 中可见,在 Amarok 设置中启用 "DAAP 收藏集" 插件。

安装 nss-mdns包 并且在 `/etc` 像这样填写主机名:

```
hosts: files mdns4_minimal [NOTFOUND=return] nis dns mdns4
```

启动 `avahi-daemon.service`.

另见

应用列表#音频播放器

12.2 KDE Wallet

难度 进阶 | **篇幅** 14.1 KB | **所属** KDE

本节将讲到

- 安装
- 配置
- 提示与技巧
- 另见

KDE Wallet Manager 是一个用于管理 KDE Plasma 系统上密码的工具。使用 KWallet 子系统，用户既可以保存自己的机密信息，也可以访问所有与 KWallet 集成的应用所保存的密码。

钱包（wallet，在 KDE 的术语中有时也称 vault 或 keyring）是一个由用户自定义密码保护的加密卷，用户和/或软件可以在其中保存机密信息（通常是用户在应用中勾选 *“记住账户”* 后的凭据）。这些加密卷既可以由用户手动创建和使用，也可以由某些集成了钱包子系统的软件（*例如*邮件应用或游戏）在后台自动创建和使用。加密卷通常会在用户登录时通过 PAM 模块自动解密（见下文）。

提示：

- 如果你只是想让使用钱包的应用能有一个可用的钱包，建议使用默认名称（*即* `kdewallet`），并让钱包密码与用户密码相同（方便 PAM 使用）。

- 钱包默认以 `.kwl` 扩展名的加密文件形式存放在 `~/.local/share/kwalletd` 目录中。

注意：自 KDE Frameworks 5.97.0 起，KDE Wallet 支持 org.freedesktop.secrets DBus API，libsecret 现在可以通过 Secret Service API 使用它来存储和获取密码及其他机密信息。

安装

KDE Wallet 通常随 KDE Plasma 桌面环境一起提供。钱包子系统也可以通过 kwallet 包 软件包手动安装。

可选安装 kwalletmanager包 软件包以获得钱包管理工具。该工具可用于以图形化方式创建和管理 KDE 钱包。

配置

在 Plasma 6（KF6）上配置 PAM

**本文或本章节的事实准确性存在争议。**

原因： 配置 PAM 似乎并非使 KWallet 正常工作所必需。此外也未提及 Plasma Login Manager（SDDM 已不再是首选的显示管理器）。（在 英文讨论页 中讨论）

Plasma 6 使用 `ksecretd` 作为 `kwallet` 的 secret service API。因此 PAM 必须显式指向该守护进程。

在相关的 PAM 文件中添加以下内容（例如视具体配置选择 `system-login`、`sddm` 或 `login`）：

```
auth            optional        pam_kwallet5.so
session         optional        pam_kwallet5.so auto_start kwalletd=/usr/bin/ksecretd
```

如果不指定 `kwalletd=` 参数，`pam_kwallet5.so` 会回退到默认值，尝试使用 `kwallet5`。

根据你的情况编辑 PAM 配置：

- 对于 SDDM：无需进一步修改，因为 `/etc/pam.d/sddm` 中已经包含这些行。
- 对于 LightDM：无需进一步修改，因为 `/etc/pam.d/lightdm` 和 `/etc/pam.d/lightdm-autologin` 中已经包含这些行。
- 对于 GDM：相应地编辑 `/etc/pam.d/gdm-password`。
- 对于 greetd：相应地编辑 `/etc/pam.d/greetd`。
- 若要在 tty 登录时解锁（没有显示管理器，或使用 greetd-tuigreet包 之类），请相应地编辑 `/etc/pam.d/login`。此时需要指定 **force_run** 参数。

`/etc/pam.d/login`

```
auth            optional        pam_kwallet5.so
session         optional        pam_kwallet5.so auto_start force_run
kwalletd=/usr/bin/ksecretd
/etc/pam.d/greetd
#%PAM-1.0

auth       required     pam_securetty.so
auth       requisite    pam_nologin.so
auth       include      system-local-login
auth       optional     pam_kwallet5.so
account    include      system-local-login
session    include      system-local-login
session    optional     pam_kwallet5.so auto_start force_run kwalletd=/usr/bin/ksecretd
```

Plasma 5（KF5）——旧版

较旧的 Plasma 版本使用 `kwalletd5`，不需要指定守护进程。

```
/etc/pam.d/login
auth     optional  pam_kwallet5.so
session  optional  pam_kwallet5.so auto_start
```

登录时自动解锁 KDE Wallet

要随登录自动解锁 KDE Wallet，请安装 kwallet-pam包 以获得兼容 PAM 的模块。所选 KWallet 密码必须与当前用户密码相同。

注意：

- kwallet-pam包 不兼容 GnuPG 密钥，KDE Wallet 必须使用标准的 `blowfish` 加密方式。
- 使用自动登录时，只有自动登录方式保存了密码，钱包才能被解锁。例如 pam_autologin 就会保存。
- 使用指纹阅读器登录时无法解锁钱包。
- 钱包必须命名为 `kdewallet`（默认名称）。它不会解锁任何其他钱包。
- 如果使用 KDE，建议在 KDE Wallet 设置中关闭 *Close when last application stops using it*，以防钱包在每次使用后（如解锁 Wi-Fi 密码等）被关闭。
- 可能需要先删除默认创建的钱包，这样会一并删除所有已保存的条目。
- 如果 kwallet Migration Assistant 在每次登录后都要求输入密码，请重命名或删除 `~/.kde4/share/apps/kwallet` 文件夹。

提示：另一种做法是使用 KWalletManager 并把 KWallet 密码设为空，这样解锁钱包时就无需输入密码。只需在 *Change Password..* 中两个字段都留空即可。不过这可能导致他人对用户钱包的非预期（读/写）访问。强烈建议在 *Access Control* 中启用 *Prompt when an application accesses a wallet*，以防止对钱包的非预期访问。

使用 Plasma Login Manager 自动登录与 LUKS 加密

把 `/usr/lib/pam.d/plasmalogin-autologin` 复制为 `/etc/pam.d/plasmalogin-autologin`，然后在 pam_kwallet5.so 之前添加 pam_systemd_loadkey(8)：

```
/etc/pam.d/plasmalogin-autologin
-auth       optional    pam_systemd_loadkey.so
-auth       optional    pam_gnome_keyring.so
-auth       optional    pam_kwallet5.so
```

然后向 plasmalogin.service 添加以下内容：

```
/etc/systemd/system/plasmalogin.service.d/keyringmode.conf
[Service]
KeyringMode=inherit
```

使用 SDDM 自动登录与 LUKS 加密

**本文或本章节可能需要合并到#使用 Plasma Login Manager 自动登录与 LUKS 加密。**

附注： 前提条件和配置相同，具体取决于桌面环境使用哪个登录管理器。（在 Talk:KDE Wallet 中讨论）

当系统使用 dm-crypt 加密时，可以使用解密磁盘的口令来自动解锁 KDE Wallet。配置 SDDM#自动登录 后，钱包依然可以自动解锁。编辑 /etc/pam.d/sddm-autologin，加入 pam_systemd_loadkey(8)：

```
/etc/pam.d/sddm-autologin
-auth       optional    pam_systemd_loadkey.so
-auth       optional    pam_gnome_keyring.so
-auth       optional    pam_kwallet5.so
-session    optional    pam_gnome_keyring.so auto_start
-session    optional    pam_kwallet5.so auto_start kwalletd=/usr/bin/ksecretd
```

然后编辑 sddm.service，添加：

```
/etc/systemd/system/sddm.service.d/keyringmode.conf
[Service]
KeyringMode=inherit
```

提示与技巧

使用 KDE Wallet 存储 SSH 密钥口令

安装 ksshaskpass包 软件包。

将 SSH_ASKPASS 环境变量设为 ksshaskpass，并将 SSH_ASKPASS_REQUIRE 设为 prefer（优先使用 askpass 程序而不是 TTY）。要在每次登录时自动设置，请创建以下 environment.d(5) 文件：

```
~/.config/environment.d/ssh_askpass.conf
SSH_ASKPASS=/usr/bin/ksshaskpass
SSH_ASKPASS_REQUIRE=prefer
```

重启会话（即重新登录）以使环境变量生效。

首次尝试使用某个 SSH 密钥时，会提示输入其口令。请务必勾选 *Remember password* 复选框。下次口令将从 KDE Wallet 读取。

使用 KDE Wallet 存储 Git 凭据

Git 可以把凭据处理委托给凭据助手。通过把 ksshaskpass包 用作凭据助手，HTTP/HTTPS 和 SMTP 密码可以安全地保存在 KDE Wallet 中。

安装 ksshaskpass包 软件包。

通过设置 `GIT_ASKPASS` 环境变量来配置 Git：

```
~/.config/environment.d/git_askpass.conf
GIT_ASKPASS=/usr/bin/ksshaskpass
```

提示：如果 `SSH_ASKPASS` 环境变量已设为 ksshaskpass，则无需再额外设置 `GIT_ASKPASS`。

其他替代方案和更多细节见 gitcredentials(7)。

存储 GPG 密钥口令

可以使用原生 KDE 窗口提示输入 GPG 密钥口令，并将其保存到 KDE Wallet 中。

配置 `gpg-agent` 使用 `/usr/bin/pinentry-qt`。

启用 Secret Service 接口。有两种方式：

- 打开 *系统设置* > *KDE Wallet*，启用 *Use KWallet for the Secret Service interface*。
- 编辑 KDE Wallet 配置文件：

```
~/.config/kwalletrc
[org.freedesktop.secrets]
apiEnabled=true
```

关闭钱包再重新打开，这些更改才会生效。你可以使用 kwalletmanager包 完成，也可以直接向 Qt D-Bus 发送命令：

```
$ qdbus org.kde.kwalletd6 /modules/kwalletd6 closeAllWallets
$ qdbus org.kde.kwalletd6 /modules/kwalletd6 open kdewallet 0 $0
```

在 Chromium 和 VSCode 中使用 KDE Wallet

基于 Chromium 的浏览器内置了钱包集成。基于 VSCode 的 IDE 依赖同样的 Chromium oscrypt 模块来检测密钥环。

要启用该功能，请以 `--password-store=kwallet5`、`--password-store=kwallet6` 或 `--password-store=detect` 参数运行程序。

要永久保留该改动，见 Chromium#使 flag 持久化（设置 CHROMIUM_USER_FLAGS 不会生效）。

从终端查询密码

与其把密码保存在纯文本文件中，你可以在钱包中手动添加新条目，然后用 *kwallet-query* 取回。

例如，你想用 Podman 登录 Docker Hub 仓库，而它支持通过 `--password-stdin` 标志从标准输入读取密码，那么可以用下面的命令登录：

```
$ kwallet-query -r folder_entry wallet_name -f folder_name | podman login docker.io -u dockerhub_username --password-stdin
```

这样，你的密码既不会保存在任何文本文件中，也不会进入终端历史记录文件。

如果要在图形会话之外运行 `kwallet-query`（例如作为无人值守备份脚本的一部分），请设置 `QT_QPA_PLATFORM=offscreen` 环境变量：

```
$ QT_QPA_PLATFORM=offscreen kwallet-query -r folder_entry wallet_name -f folder_name
```

在窗口管理器中自动解锁 KWallet

要解锁由登录密码保护的 KWallet，除了配置 PAM 之外，还必须在窗口管理器配置文件的自启动部分启动 `/usr/lib/pam_kwallet_init`。

禁用 KWallet

如果你希望永久禁用 kwallet：

```
~/.config/kwalletrc
[Wallet]
Enabled=false
```

注意：可以把 KWallet 用作其他钱包管理器的代理，这样 plasma-nm包 之类的应用就不会出问题。参见此讨论。

自动 D-Bus 激活

大多数应用使用 `org.freedesktop.secrets.service` 这个 D-Bus 服务。KWallet 并未开箱即用地提供对应的服务文件。

你可以创建这样的服务文件来实现自动激活：

```
~/.local/share/dbus-1/services/org.freedesktop.secrets.service
[D-BUS Service]
Name=org.freedesktop.secrets
Exec=/usr/bin/kwalletd6
```

另见

- Wikipedia:KWallet
- dennogumi.org: 使用 PAM 解锁 KWallet
- org.freedesktop.secrets DBus API 初始支持

12.3 KDE

难度 高级 | **篇幅** 73.3 KB | **所属** KDE

本节将讲到

- 安装
- 启动 Plasma
- 配置
- 应用程序
- 提示和技巧

KDE 是一套由桌面环境(KDE Plasma)、应用程序（KDE Applications）以及 Qt 附加库（KDE Frameworks）构成的软件项目。

安装

Plasma 桌面

安装 plasma-meta包 元软件包或者 plasma包组 组。关于 plasma-meta包 和 plasma包组 两者的不同见元软件包与软件包组。若要使用 Plasma 的最小安装，请安装 plasma-desktop包 包。上游 KDE 有软件包和安装推荐以获得功能齐全的 Plasma 会话。

如果你是 NVIDIA 用户，请确保已启用 DRM内核级显示模式设置。

Plasma Mobile

安装 plasma-mobileAUR。

KDE 应用

若要安装 KDE 的全套应用，请安装 kde-applications-meta包 元软件包或 kde-applications包组 组。若只需要安装特定类别的 KDE 应用（如游戏或教育），请只安装 kde-applications-meta包 中与此相关的依赖。请注意只安装kde应用不会安装任何版本的 Plasma 桌面。

不稳定版

见 Official repositories#kde-unstable。

启动 Plasma

从 Plasma 6.4 开始，Wayland 会话已足够成熟，成为默认且首选的会话：X11 会话需要单独安装 plasma-x11-session包 软件包才能使用 。Xorg 会话目前仍受支持，但将在 Plasma 6.8 中被移除。更多信息请参阅 Wayland 已知重要问题 和 X11 已知重要问题 文档。

Plasma 既可以通过显示管理器启动，也可以从控制台启动。

使用显示管理器

提示：推荐使用 Plasma Login Manager 显示管理器。

- 选择 *Plasma (Wayland)* 以在 Wayland 下开启新会话。
- 选择 *Plasma (X11)* 以在 Xorg 下开启新会话。
- 选择 *Plasma mobile (Wayland)* 以在 Wayland下开启新的 Plasma mobile 会话。

从控制台启动

- 要从控制台启动 Plasma on Wayland 会话，请运行 [`/usr/lib/plasma-dbus-run-session-if-needed /usr/bin/startplasma-wayland`\[2\]](https://invent.kde.org/plasma/plasma-workspace/-/merge_requests/128)。
- 若要使用 "xinit/startx" 启动 Plasma 桌面，请在 `.xinitrc` 文件中添加 `export DESKTOP_SESSION=plasma` 和 `exec startplasma-x11`，或者直接在控制台运行`startx /usr/bin/startplasma-x11`。若要在登录时开启 Xorg，见登录时启动 X。

配置

KDE应用的大部分配置存储于 `~/.config` 目录下。KDE 主要使用**"系统设置"**调整配置，也可以在终端执行 `systemsettings` 启动它。

个性化

Plasma 桌面

主题

有不同类型的 KDE 主题，根据它们修改的范围而有所不同：

- 全局主题，全面的包，可以包括 Plasma 主题、应用程序样式、颜色、字体、图标、光标、启动屏幕、SDDM 主题和 Konsole 配色方案。要应用全局主题，可用 `lookandfeeltool` 命令行工具。
- Plasma 视觉风格， 修改 Plasma 面板和小部件的外观。这些通常有一个推荐的 Kvantum 或 Aurorae 主题来完成外观。
- 应用程序风格，修改程序的外观。
- 使用主题引擎的应用程序样式，例如 Kvantum、QtCurve 、QSvgStyle 以及 Aurorae。
- #图标主题，为应用程序、文件和操作提供图标。

为了便于系统范围内的安装和更新，一些主题在官方仓库和 AUR 都可用。

全局主题亦可通过*系统设置* > *颜色和主题* > *全局主题* > *获取新全局主题...* 安装。

警告：终端用户提供的全局主题通常没有经过监管。你应该对下载并应用这些全局主题保持极度警惕。这些全局主题可能运行恶意代码并造成用户数据丢失

GTK 应用的外观

提示：若要使 Qt 和 GTK 应用程序主题一致，请阅读统一 Qt 和 GTK 应用程序的外观。

在 GTK 中推荐使用 breeze-gtk包 主题，这款 GTK 主题模仿了 Plasma 的 Breeze 主题。 安装 kde-gtk-config包（该软件是 plasma包组 的一部分），在*系统设置* > *颜色和主题* > *应用风格* > *配置 GNOME/GTK 应用风格*中选择 `Breeze` 并重新登陆即可。

在某些主题下，GTK应用的提示条在白色背景下会显示难以阅读的白色字体内容。要改变GTK2应用的颜色，找到 `.gtkrc-2.0` 文件中的tooltips部分并更改。对于GTK3应用，需要更改 `gtk.css` 和 `settings.ini` 两个文件。

有些GTK2程序如 vuescan-binAUR 在Breeze或Adwaita皮肤的Plasma会话下因"消失的勾选框"问题仍然很难使用。要解决这个问题，安装并在*系统设置* > *颜色和主题* > *应用风格* > *配置 GNOME/GTK 应用风格* > *GTK 主题*中选择如 numix-frost-themesAUR 提供的Numix-Frost-Light皮肤，该皮肤风格与Breeze类似。

头像

Plasma和 SDDM 都会使用存在 `/var/lib/AccountsService/icons/` 的图像作为用户头像。要使用图形界面配置，你可以使用*系统设置* > *用户*。移除与用户名对应的文件可以恢复默认头像。

小部件

Plasmoids 是 Plasma 桌面 Shell 的小部件，旨在增强桌面功能，可查看 AUR。

Plasmoid 脚本也可以通过在面板或桌面上点击右键，选择*进入编辑模式* > *添加挂件...* > *获取新挂件...* > *下载新 Plasma 挂件*来安装。这将呈现一个 https://store.kde.org/ 的前端，您只需点击就可以安装、卸载或更新第三方 Plasmoid 脚本。

系统托盘中的声音小程序

安装 plasma-pa包 或 kmix包 (从程序启动器启动 Kmix)。前者已自动安装，无需另外设置。

注意：若要调整 音量单次增减的长度，请将例如 `VolumePercentageStep=1` （后面的数字代表长度）的一行文字添加到 `~/.kde4/share/config/kmixrc` 的 `[Global]` 部分中。

系统托盘中的网络管理器

如果您使用 networkmanager包，那么请安装 plasma-nm包。

禁用面板阴影

因为 Plasma 的面板在其他窗口之上，所以其阴影也会渲染在其他窗口之上。 若要在不影响其他阴影的情况下禁用此行为，安装 xorg-xprop包 并运行:

```
$ xprop -remove _KDE_NET_WM_SHADOW
```

然后用增大的光标选择面板。 若要自动化此操作，请安装 xorg-xwininfo包 并创建以下脚本：

```
/usr/local/bin/kde-no-shadow
#!/bin/bash
for WID in $(xwininfo -root -tree | sed '/"plasmashell": ("plasmashell"
"plasmashell")/!d; s/^  *\([^ ]*\) .*/\1/g'); do
   xprop -id $WID -remove _KDE_NET_WM_SHADOW
done
```

并添加此脚本的可执行权限。

**本文或本章节的事实准确性存在争议。**

原因： 由于脚本启动过早，自动启动可能无法生效。（`sleep 5` 可能有用，但不可靠）（在 Talk:KDE 中讨论）

在*自动启动*的*添加脚本*中添加此脚本，可以使其在登录时启动：

```
$ kcmshell6 autostart
```

显示缩放/高DPI显示

见 HiDPI#KDE。

Plasma Mobile

plasma-phone-settings仓库包含了一些能够用于全局(`/etc/xdg`)和/或用户(`~/.config`) 的推荐设置。

锁屏

`/etc/xdg/kscreenlockerrc`（或 `~/.config/kscreenlockerrc`）可配置登录后立即锁屏。这在使用 SDDM#自动登录功能时会很有用。

```
/etc/xdg/kscreenlockerrc
[Daemon]

LockOnStart=true
```

虚拟键盘

如果你的设备具备实体键盘，但你想使用虚拟键盘，在环境变量里添加 `KWIN_IM_SHOW_ALWAYS=1` 到你的 Wayland 会话中。

若在 Wayland 会话中使用虚拟键盘，请安装 maliit-keyboard包并确保*系统设置* > *键盘* > *虚拟键盘*已经启用

若在 X11 会话中使用虚拟键盘，应在应用程序列表/工具#屏幕键盘中选择合适的包，再手动运行它。

窗口装饰

窗口装饰可以在 AUR 中找到。

可在*系统设置* > *颜色和主题* > *窗口装饰*中设置窗口装饰，也可以直接下载更多主题并一键安装。

图标主题

可在*系统设置* > *颜色和主题* > *图标*中安装或改变图标主题。

注意：虽然所有现代的Linux发行版都共享统一的图标主题格式，但像 GNOME这样的桌面使用更少的图标（特别是在工具栏和菜单中）。为这些桌面开发的主题一般都缺少 Plasma 和 KDE 应用需要的图标。建议安装与 Plasma 兼容的主题。

提示：部分图标主题不从默认图标主题中引入图标，这将导致部分图标缺失。要想从默认图标主题中导入缺失图标，把 `breeze` 添加到 `/usr/share/icon/*theme-name*/index.theme` 的 `Inherits=` 中，比如 `Inherits=breeze,hicolor`。每次更新图标主题时都需要重新使用这个补丁，尝试使用 Pacman hooks 来自动完成这项任务。

空间效率

Plasma Netbook shell（上网本交互界面）已从 Plasma 5 中移除，请阅此 KDE 论坛帖子。但您仍然可以通过编辑 `~/.config/kwinrc`，在 `[Windows]` 部分加上 `BorderlessMaximizedWindows=true` 来实现类似的操作。

缩略图生成

若要在桌面和 Dolphin 内为媒体或文档文件生成缩略图，安装 kdegraphics-thumbnailers包 和 ffmpegthumbs包。

要启用或配置桌面的缩略图功能，右键桌面背景，选择*配置桌面和壁纸* > *图标* > *配置预览插件*进行设置。

在 Dolphin 中，点击窗口右上角的*打开菜单*按钮（或按下 F10），选择*配置* > *配置 Dolphin(C)...* > *界面* > *预览图*进行设置。

夜间颜色

Plasma 提供了一种类似 Redshift 的特性（在 Xorg 和 Wayland 上都可以使用），称为夜间颜色。它使屏幕上的颜色呈现更暖的色调，以在指定的时间里减少眼睛疲劳。您可以在*系统设置* > *颜色和主题* > *夜间颜色*中启动该功能。

打印

提示：使用 CUPS 的 Web 接口进行快速配置。这种方式配置的打印机可以被 KDE 应用使用。

您也可以在*系统设置* > *打印机*中配置打印机。要使用这种配置方式，必须首先安装 system-config-printer包， print-manager包 和 cups包 软件包。请阅 CUPS配置。

Samba/Windows 支持

Dophin 的共享服务需要 kdenetwork-filesharing包 软件包以及 usershares。关于如何配置 usershares（在不启动 `smb.conf` 的情况下），见建立 Usershare 路径。重新启动 Samba 后，应无需进一步配置 Dolphin 的共享。

从 Dolphin 访问 Windows 共享是无需额外设置的。使用 `smb://*服务器名或地址*/*共享目录*路径访问共享文件。`

提示：在 Dolphin 命令行中，无授权认证访问 Windows 共享时，用户名以及密码将使用 `*`（星号）表示。

不像 GTK 文件浏览器利用 GVfs 启动程序，如果在 Dolphin 中通过 KIO 打开 Samba 共享的文件，大多数程序中 Plasma 会先把整个文件复制到本地系统（除了 VLC）。使用基于 GTK 的文件浏览器可以解决此问题，如 thunar包 和 gvfs-smb包 可以更有效地访问SMB共享（同时要安装 gnome-keyring包 以保存登录凭证）。

另一种方法则是使用 cifs-utils包 挂载 Samba 共享从而让 Plasma 把 SMB 共享当成一个普通的本地文件夹从而正常访问。见手动挂载和自动挂载。

另一种简单的GUI解决方法则是使用 samba-mounter-gitAUR。它在*系统设置* > *网络驱动*中提供了基本相同的功能。要注意，在新版 KDE Plasma 中此应用可能会崩溃。

KDE 桌面活动

KDE 桌面活动是一种类似于"虚拟桌面"的 Plasma 组件，您可以独立设置特定的活动。 只有在您正在使用这个活动时，这些设定才会生效。

电源管理

安装 powerdevil包 以使用 KDE 内置的 "**PowerDevil 电源管理**"的节能服务，它可以调整系统的节能配置、屏幕亮度（如果支持）并提供详细的电源报告。

提示：电源配置文件的集成需要 power-profiles-daemon 可选依赖。

**本文或本章节的事实准确性存在争议。**

原因： 根据下面的注意内容，问题也可能是因为 logind 中 *LidSwitchIgnoreInhibited* 的默认值是 *yes*。见（在 Talk:KDE 中讨论）

注意：Powerdevil 可能无法覆盖所有的 logind 设置(例如笔记本合盖动作). 如果遇到这样的问题，需要修改logind的设置，请参考 电源管理#ACPI 事件。

自动启动

Plasma 可以在启动和关闭时自动启动应用程序并运行shell脚本。若要自动启动应用程序，请进入*系统设置* > *自动启动*并添加您想要的程序或shell脚本。若选择程序，则

会创建一个 `.desktop` 文件；选择脚本，则会创建一个用来启动脚本的 `.desktop` 文件。

注意：

- 程序只能在登录时自动启动，而shell脚本也可以在关机和 Plasma 启动前启动。
- Shell 脚本需要可执行权限才能运行。
- 之前位于 `~/.config/autostart-scripts/` 的 Shell 脚本会被自动迁移到 .desktop 文件。
- 将桌面配置项（即 *.desktop* 文件）放入适当的 XDG Autostart 目录。
- 在以下目录之一放置 Shell 脚本或其符号链接：
- `~/.config/plasma-workspace/env`: 在 Plasma 启动前启动脚本。
- `~/.config/plasma-workspace/shutdown`: 在关机时启动脚本。

见官方文档。

Phonon

摘自维基百科)：

: Phonon 是 KDE 的多媒体 API, 提供了多个多媒体框架的抽象，为 KDE 和一些 QT 程序提供多媒体流处理功能。

: Phonon 最初的目的，是让 KDE 和 Qt 软件独立于其他多媒体框架（例如GStreamer或xine），并为其提供一个稳定的 API。

KDE 广泛地将 Phonon 用于音频（例如系统通知或者 KDE 声音应用）和视频（例如 Dolphin 中的视频缩略图）中。它可以使用以下后端：

- VLC: phonon-qt6-vlc包
- GStreamer: phonon-qt6-gstreamer-gitAUR 参见GStreamer#安装以获取其他编码支持
- mpv: phonon-qt6-mpv包

KDE 仅推荐VLC后端，因为 GStreamer 不再维护。

注意：

- 可以同时安装多个后端，并在 *系统设置* > *多媒体* > *后端* 中进行优先级设定。
- 根据 KDE 这个帖子, VLC 后端不支持 ReplayGain。
- 如果使用 VLC 后端，可能会遇到崩溃问题，如在每次 Plasma 发送语音警告时崩溃等等，详见 。重建 VLC 插件缓存也许能解决问题：

```
# /usr/lib/vlc/vlc-cache-gen /usr/lib/vlc/plugins
```

备份与恢复

Plasma 将个性化配置以配置文件的形式储存在 XDG_CONFIG_HOME 中。参照 KDE 的配置文件并选择恰当的备份恢复方案。

systemd 启动

自Plasam5.25起，默认启动方式更改为使用 systemd 用户服务实例来启动或管理所有 Plasma 服务。可以关闭该方法并使用基于脚本的启动，使用下列命令（此方法在未来版本中可能会停用）：

```
$ kwriteconfig6 --file startkderc --group General --key systemdBoot false
```

更多关于该方法的实现细节可以参考 "Edmundson 的博客：Plasma 和 systemd 启动"。

拼写检查

KDE 使用 sonnet包 提供拼写检查功能。查看它的可选依赖以获取支持的拼写检查。

可在*系统设置->拼写检查*配置拼写检查。

使用 NVIDIA 显卡运行 KWin Wayland

见 https://community.kde.org/Plasma/Wayland/Nvidia 。

应用程序

KDE项目提供了一套与Plasma桌面集成的应用程序。有关可用应用程序的完整列表，详见 kde-applications包组 软件包。另见 KDE 相关应用页面。

除了 KDE 应用程序包提供的程序之外，还有许多其他可用于补充 Plasma 的应用程序。其中一些将在下面讨论。

系统管理

KDE 系统设置中配置终止 Xorg-server

浏览到子菜单： *系统设置* > *键盘* > *高级（标签）*中选中“杀死 X 服务器的按键序列”复选框。

KCM

KCM 意为 KDE 配置模块（**KC**onfig **M**odule）。这些模块在系统设置中提供了界面从而帮助您配置系统，或通过命令行（*kcmshell6*）。

- **sddm-kcm** — SDDM 的 KDE 配置模块

: https://invent.kde.org/plasma/sddm-kcm || sddm-kcm包

- **kde-gtk-config** — GTK2 和 GTK3 的 KDE 配置器。

: https://invent.kde.org/plasma/kde-gtk-config || kde-gtk-config包

- **wacom tablet** — KDE Wacom 驱动的图形界面。

: https://www.linux-apps.com/p/1127862/ || kcm-wacomtablet包

可在 linux-apps.com 找到更多的 KCM 。

桌面搜索

KDE 使用 Baloo 索引与查找文件。

Web 浏览器

下列浏览器可以与 Plasma 集成：

- **Konqueror** — KDE 项目的一部分, 支持两种渲染引擎 – KHTML 和基于 Chromium的 Qt Web引擎。

: https://konqueror.org/ || konqueror包

- **Falkon** — 一个具有 Plasma 集成特性的 Qt 浏览器，前身是 Qupzilla，使用 Qt WebEngine。

: https://userbase.kde.org/Falkon/ || falkon包

- **Chromium** — Chromium 及它的专有版本 Google Chrome 具有有限的 Plasma 集成。它们可以使用 KWallet 以及 KDE 窗口 打开/保存。

: https://www.chromium.org/ || chromium包

- **Firefox** — Firefox 可以通过配置以和 Plasma 更好地集成。参考 Firefox KDE 整合。

: https://mozilla.org/firefox || firefox包

提示：从 Plasma 5.13 起，你可以通过 Plasma 与 Firefox 或 Chromium 的集成功能，从Plasma托盘控制媒体播放、显示下载状态以及在KRunner中找到打开选项。通过安装plasma-browser-integration包和对应浏览器的插件完成。Chrome/Chromium应该已经随包支持，Firefox插件参见Firefox KDE整合。

个人信息管理 (PIM)

KDE 提供了自有的个人信息管理 (PIM) 存储，包括电子邮件，联系人，日历等。可以使用 kde-pim包组 安装包组或 kde-pim-meta包 元软件包安装所有 PIM 软件包。

Akonadi

Akonadi 是系统中本地缓存各种来源的 PIM 数据的一种方法，接着这些数据可以被其它的应用使用。这包含了用户的邮件、联系人、日历、事件、刊物、闹钟、笔记等。Akonadi 自身并不存储任何数据：存储格式依赖于数据的性质（例如，联系人可能以 vcard 格式存储）。

安装 akonadi包. 若需其他插件，安装 kdepim-addons包。

注意：

- 若要使用除 MariaDB/MySQL 以外的数据库引擎，请在安装 akonadi包 包时使用以下命令从而跳过 mariadb包 依赖项的安装:

```
# pacman -S akonadi --assume-installed mariadb
```

另见 FS#32878。

- 如果 Akonadi 第一次启动找不到 `/usr/bin/mysqld`，它会尝试使用 SQLite 作为后备。

MySQL

默认情况下 Akonadi 将使用`/usr/bin/mysqld`（MariaDB 默认, 参考 MySQL 来寻找替代方案）去运行一个被管理的 MySQL 实例并且将数据存储在 `~/.local/share/akonadi/db_data/`.

系统级 MySQL 实例

Akonadi 支持将系统范围的 MySQL 用于其数据库。

```
~/.config/akonadi/akonadiserverrc
[%General]
Driver=QMYSQL

[QMYSQL]
Host=
Name=akonadi_username
Options="UNIX_SOCKET=/run/mysqld/mysqld.sock"
StartServer=false
```

PostgreSQL

Akonadi 支持使用现有的系统范围 PostgreSQL 实例，即 `postgresql.service`，或运行具有用户权限的 PostgreSQL 实例和 `~/.local/share/` 中的数据库 `akonadi/db_data/`。

每用户 PostgreSQL 实例

安装 postgresql包 和 postgresql-old-upgrade包。

编辑 Akonadi 配置文件，使其具有以下内容：

```
~/.config/akonadi/akonadiserverrc
[%General]
Driver=QPSQL
```

注意：

- 当 Akonadi 启动时，它将创建 `[QPSQL]` 部分并在其中设置适当的变量。
- 数据库将被存储在 `~/.local/share/akonadi/db_data/`。

使用 `akonadictl start` 启动 Akonadi，然后使用 `akonadictl status` 检查其状态。

注意：

- 从 akonadi包 19.08.0-1 开始，当检测到主要的 PostgreSQL 版本升级时，`~/.local/share/akonadi/db_data/` 中的 PostgreSQL 数据库集群将自动升级。
- 对于以前的 akonadi包 版本，主要的 PostgreSQL 版本升级将需要手动数据库升级。遵循 KDE UserBase Wiki 上的更新说明。确保将 PostgreSQL 二进制文件的路径调整为 postgresql包 和 postgresql-old-upgrade包 使用的路径，请参阅 PostgreSQL#Upgrading PostgreSQL。

系统范围的 PostgreSQL 实例

这需要一个已经配置并正在运行的 PostgreSQL。

为您的用户创建一个 PostgreSQL 用户帐户：

```
[postgres]$ createuser username
```

为 Akonadi 创建一个数据库：

```
[postgres]$ createdb -O username -E UTF8 --locale=C -T template0 akonadi-username
```

对 Akonadi 配置文件做如下编辑：

```
~/.config/akonadi/akonadiserverrc
[%General]
Driver=QPSQL

[QPSQL]
Host=/run/postgresql
Name=akonadi-username
StartServer=false
```

注意：可以使用 `[QPSQL]` 部分中的选项 `Port=`、`User=`、`Password=` 指定自定义端口、用户名和密码。

使用 `akonadictl start` 启动 Akonadi，并检查其状态：`akonadictl status`。

SQLite

编辑 Akonadi 配置文件以匹配以下配置：

```
~/.config/akonadi/akonadiserverrc
[%General]
Driver=QSQLITE
```

注意：

- 当 Akonadi 启动时，它将创建 `[QSQLITE]` 部分并在其中设置适当的变量。
- 数据库存储在 `~/.local/share/akonadi/akonadi.db`。

禁用 Akonadi

想要禁用 Akonadi 的用户不需要启动任何依赖它的 KDE 应用程序。有关详细信息，请参阅此在 KDE 用户库中的部分。

KDE Connect

KDE Connect 提供了一些功能以将 Android 或 iOS 手机与Linux桌面连接：

- 从任何应用向 KDE 共享文件和 URL 或从 KDE 向任何应用共享，无需连线。
- 触摸板模拟：将手机屏幕用作计算机的触摸板。
- 通知同步（4.3+）：从桌面读取您的安卓通知。
- 共享剪贴板：在手机和电脑之间复制粘贴。
- 多媒体远程控制：将手机用作 Linux 媒体播放器的遥控器。
- Wi-Fi 连接：不需要 usb 和蓝牙。
- RSA加密：保证您的信息安全。

您需要同时在电脑和安卓上安装 KDE Connect。PC端上安装 kdeconnect包 软件包，而安卓端可以在 Google Play 或 F-Droid 上安装 KDE Connect。如果你想浏览手机上的文件系统，你需要去安装 sshfs包 同时在手机app上设置文件系统可被访问。对于 iOS，从 App Store 安装 KDE Connect。并非 Android 版本的所有功能都可用于 iOS 版本。

要在Plasma Wayland会话上使用远程输入功能，需要安装 xdg-desktop-portal包。

即使不使用 Plasma 桌面，您也可以使用 KDE Connect。对于 GNOME 用户，可以通过安装 gnome-shell-extension-gsconnectAUR 获得更好的集成，而不是安装 kdeconnect包。如果要手动启动 KDE Connect 进程, 运行 `/usr/bin/kdeconnectd`。

如果你使用 firewall, 你需要打开 UDP 和 TCP 端口 `1714` 到 `1764`。

有些时候, KDE Connect 不会检测到你的手机。 你可以通过杀死KDE Connect进程 `killall kdeconnectd` 之后在系统设置里打开 kdeconnect 或者运行 `kdeconnect-cli --refresh` 后运行 `kdeconnect-cli -l`。您亦可在 KDE Connect for Android 上使用*配对新设备* > *以 IP 添加设备*（仅大意）。

提示和技巧

使用不同的窗口管理器

可以使用其它窗口管理器替换 Plasma 自带的 KWin 窗口管理器。这使你能够将 KDE 桌面的功能与平铺窗口管理器的特性结合起来，这样做可能会比KWin的平铺脚本的效果更丰富。

Plasma 中的组件选择设置不再允许更改窗口管理器，但仍然可以通过其它方法替换 KWin 服务。

注意：当您用不包含混成器的窗口管理器（例如 Openbox）替换 Kwin 时，任何桌面特殊效果都会失效（例如窗口透明度）。在这种情况下，请安装并运行其他独立的混成器，如 Xcompmgr 或 picom 。

替换KWin服务

自 KDE 5.25，Plasma默认使用基于 systemd 的启动。

要在此启动方式中替换 KWin，首先为当前用户禁用 (mask) `plasma-kwin_x11.service`服务避免 KWin 启动。

接下来创建 (create) 一个新的 systemd 用户单元 (user unit) 来启动你选择的窗口管理器：

```
~/.config/systemd/user/plasma-custom-wm.service
[Install]
WantedBy=plasma-workspace.target

[Unit]
Description=Plasma Custom Window Manager
Before=plasma-workspace.target

[Service]
ExecStart=/path/to/other/wm
Slice=session.slice
Restart=on-failure
```

最后重新扫描 (daemon-reload) 用户单元 (user unit)，确保 Kwin 服务 `plasma-kwin_x11.service` 已经禁用 (mask)，然后启用 (enable) 新的`plasma-custom-wm.service`窗口管理器服务。

提示：当使用 i3 窗口管理器时，需要手动设置对话窗以浮动模式打开以确保显示正确。更多信息见 i3#Correct handling of floating dialogs。

使用基于脚本的启动和 KDEWM 环境变量

关闭 #systemd 启动使 Plasma 以基于脚本的方式启动。如果是这种情况你可以在 Plasma 启动前设置 `KDEWM` 环境变量 (environment variable) 来切换窗口管理器。

系统全局

如果有 root 权限，您也可以为所有用户在登录界面添加 XSession 选项。

首先，创建如下脚本并赋予执行权限：

```
/usr/local/bin/plasma-i3.sh
#!/bin/sh
```

```
export KDEWM=/usr/bin/i3
/usr/bin/startplasma-x11
```

若使用别的窗口管理器，则替换示例中的 `/usr/bin/i3`，务必保证路径正确。若窗口管理器无法正常启动，用户将退回登录界面。

然后，在 `/usr/share/xsessions/` 下创建如下文件：

```
/usr/share/xsessions/plasma-i3.desktop
[Desktop Entry]
Type=XSession
Exec=/usr/local/bin/plasma-i3.sh
DesktopNames=KDE
Name=Plasma (i3)
Comment=KDE Plasma with i3 as the WM
```

KDE/Openbox 会话

软件包 openbox包 为在plasma下使用 Openbox 提供了会话. 要使用这个会话，请禁用 KDE#systemd启动，并在 display manager 菜单中选择 *KDE/Openbox* .

若要手动启动会话，请将下面这行添加到您的 xinit 配置中：

```
~/.xinitrc
exec openbox-kde-session
```

KWin 平铺窗口脚本

下列 KWin 扩展能够让 KDE 变得更像一个平铺窗口管理器：

- **Polonium** — 一个 Bismuth 的（非官方）继任者。

: https://github.com/zeroxoneafour/polonium Polonium || kwin-poloniumAUR

- **Kröhnkite** — 受dwm启发的一个动态平铺扩展组件。

: https://github.com/anametologin/krohnkite || kwin-scripts-krohnkiteAUR

- **KZones** — 一个模仿微软 PowerToys 和 Windows 11 的 snap layouts 功能的 KWin 脚本。

: https://github.com/gerritdevriese/kzones KZones || kwin-scripts-kzonesAUR

配置显示器分辨率 / 多显示器

若要在 Plasma 中启用分辨率和多显示器管理, 请安装 kscreen包. 它在*系统设置* > *显示和监视器*中添加了更多选项.

配置 ICC 特性文件

在 X11 上，ICC 特性文件由 colord包 管理，要在 Plasma 中配置这些文件，可安装 colord-kde包。在新增的*系统设置* > *色彩管理*选项中，点击*导入特性文件*可导入 ICC 特性文件.

HDR

HDR功能目前是实验性的且仅在Wayland会话中支持。相关设置位于*系统设置* > *显示和监视器* > *高动态范围 (HDR)* > *启用 HDR*。

关于更多 HDR 显示内容信息参见HDR 监视器支持。Development details about HDR in Plasma can be found on Xaver Hugl's blog post.

禁用使用 Super 键（Windows 键）打开应用程序启动器

若要禁用此功能，需要编辑`kwinrc`配置文件并设置`ModifierOnlyShortcuts`下的`Meta`为空：

```
$XDG_CONFIG_HOME/kwinrc
[ModifierOnlyShortcuts]
Meta=
```

也可以运行以下命令：

```
$ kwriteconfig6 --file kwinrc --group ModifierOnlyShortcuts --key Meta ""
```

在应用程序菜单中禁用书签

安装了Plasma Browser集成后，KDE将在应用程序启动器中显示书签。

要禁用此功能，进入*系统设置* > *搜索* > *Plasma 搜索*，取消勾选*书签*

IBus 集成

IBus是一个输入法框架，可以集成到KDE中。有关详细信息，请参阅IBus#Integration。

在 Wayland 上使用 KDE 以提供重音字符和死键支持时，可能需要使用 IBus 。

在 plasma-nm 中启用热点

参见 NetworkManager#通过Wi-Fi共享网络连接。

恢复以前保存的会话

如果您选择了*系统设置* > *会话* > *桌面会话* > *会话恢复：上次注销时正在打开的应用程序*（默认），ksmserver（KDE 的会话管理器）将在注销时自动将所有打开的应用程序保存/加载到/从 `~/.config/ksmserverrc` 注销/登录。

注意：目前，无法恢复原生 Wayland 窗口。有关当前开发状态，请参阅 Wayland Showstoppers。

在 KMail 中接收本地邮件

如果您使用 Maildir 格式的 mail server 设置了本地邮件传递，您可能希望在 KMail 中接收此邮件。为此，您可以重新使用 KMail 的默认接收帐户“本地文件夹”，该帐户将邮件存储在 `~/.local/share/local-mail/` 中。

将 ~/Maildir 目录（通常发送 Maildir 格式邮件的地方）符号链接到本地文件夹的收件箱：

```
$ ln -s .local/share/local-mail/inbox ~/Maildir
```

或者，添加一个类型为“Maildir”的新接收帐户并将 ~/Maildir 设置为其目录。

为所有用户配置 Plasma

编辑 /usr/share/plasma 中的 config/main.xml 文件。例如，要为所有用户配置应用程序启动器，请编辑 /usr/share/plasma/plasmoids/org.kde.plasma.kickoff/contents/config/main.xml。要防止文件被包更新覆盖，请将文件添加到 Pacman's NoUpgrade

禁用休眠

**本文或本章节可能需要合并到Power management。**

附注： 此不限于 KDE。可将本节作为一个存根链接，指向该节。（在 Talk:KDE 中讨论）

使用 Polkit 策略规则正确禁用休眠功能并将其从菜单中隐藏。

```
/etc/polkit-1/rules.d/99-disable-hibernate.rules
// Disable hibernate for all users
polkit.addRule(function(action, subject) {
   if ((action.id == "org.freedesktop.login1.hibernate")) {
      return polkit.Result.NO;
   }
});
polkit.addRule(function(action, subject) {
   if ((action.id == "org.freedesktop.login1.hibernate-multiple-sessions")) {
      return polkit.Result.NO;
   }
});
```

或者，添加以下行到在 /etc/systemd/sleep.conf.d/ 中的一个文件：

```
/etc/systemd/sleep.conf.d/00-disable-hibernation.conf
[Sleep]

AllowHibernation=no

AllowSuspendThenHibernate=no

AllowHybridSleep=no
```

窗口规则

Kwin 能够为特定的窗口/应用程序指定规则。例如，即使应用程序开发人员没有启用窗口标题栏，您也可以强制启用。 您可以设置特定的位置，大小，最小化状态，保持最前/后以及其他规则。

要创建规则，您可以在特定窗口处于焦点时按 Alt+F3。 然后，在"更多操作>配置特殊应用程序设置/窗口设置"中，您可以设置所需的属性。 创建的规则列表可从"系统设置>窗口管理>窗口规则"中获取。

在固定位置挂载网络共享

KDE的挂载管理器(kio-fuse包)默认会将网络共享挂载到 ${XDG_RUNTIME_DIR}/kio-fuse-*长度为6个字符的随机字符串*。

在你的家目录创建一个目录，例如mnt_kio：

```
$ mkdir ~/mnt_kio
```

使用一个#附加配置片段覆盖默认值：

```
~/.config/systemd/user/kio-fuse.service.d/mountpoint.conf
[Service]
ExecStart=
ExecStart=/usr/lib/kio-fuse -f %h/mnt_kio
```

修改后如果你通过dbus或在Dolphin里打开一些网络共享的文件：

```
$ dbus-send --session --print-reply --type=method_call \
          --dest=org.kde.KIOFuse \
                 /org/kde/KIOFuse \
                 org.kde.KIOFuse.VFS.mountUrl "smb://etcetc"
```

这些目录会挂载到~/mnt_kio。

本地集成菜单

如果要将菜单栏集成到标题栏, 请从AUR安装 material-kwin-decoration-gitAUR, 然后再 系统设置 > 窗口装饰, 选择 'Material' ， 然后添加应用菜单按钮到标题栏 (最好放到从左数第二个的位置)。

在 Wayland 上预授权远程控制

Xdg-desktop-portal-kde 支持来自远程桌面会话、虚拟 KVM 切换器、kde-connect、通过 steam-input 等模拟的设备（如游戏手柄）的远程输入、等等。此授权在应用程序或 desktop-portal 重启后会失效，导致“请求远程控制”窗口每次弹出，使得无人值守访问无法实现。

自 Plasma 6.3 版本起，引入了一套权限系统，允许对应用程序进行预授权。目前，该权限 API 仅可通过 flatpak 命令行工具使用，但应用程序无需以 Flatpak 形式运行即可获得预授权。

根据 the upstream docs 和 flatpak-permission-set 手册页, 你需要确认要授权的应用程序是否设置了应用程序 ID, 如果通过 KRunner 这种启动器启动, 该 ID 通常由 Plasma 设置， 并且一般是在 /usr/share/applications 下的 .desktop 文件。

例如，要为类似 lan-mouse 这样的虚拟 KVM 切换器进行预授权，可以执行：

```
$ flatpak permission-set kde-authorized remote-desktop de.feschbar.LanMouse yes
```

如果将其作为 systemd 用户单元中的守护进程启动，则应使用该单元的名称：

```
$ flatpak permission-set kde-authorized remote-desktop lan-mouse yes
```

如果你的应用程序未设置 ID，可以将该字段留空：

```
$ flatpak permission-set kde-authorized remote-desktop "" yes
```

疑难解答

升级到 KDE 6 后 KDE 应用在 GNOME 中启动失败

KDE 6应用默认使用Wayland，在GNOME Wayland（也可能在其他桌面环境/窗口管理器中） 启动失败。通过设置环境变量`QT_QPA_PLATFORM=xcb`来解决该问题。

这是一个KDE bugs的解决方法，问题不是来源于Wayland本身。

升级到 KDE 6 后图标消失

升级到KDE 6后可能会出现所有KDE图标不显示的问题，但新创建的用户正常显示。

该问题的原因是升级过程主题丢失，因此需要手动重新指定。在*系统设置* > *颜色和主题* > *图标*中再次选择你要使用的图标即可解决。

字体过大或比例异常

尝试在*系统设置* > *文字和字体*中强制字体 DPI 为 `**96**` 。

若无效，请尝试按照 Xorg#手动设置 DPI 中的说明直接在 Xorg 配置文件中设置 DPI。

有关配置的问题

KDE 中许多问题都跟配置相关。

Plasma 桌面行为异常

Plasma 故障通常是由不稳定的 **plasma 小部件**（plasmoids）或者 **plasma 主题**引起的。首先寻找最近安装的 plasmoid 或者 Plasma 主题并禁用或者卸载它。

因此，如果您的桌面突然被“锁定”了，很可能是由于安装了有问题的组件造成的。如果您不记得故障发生前您安装了什么小部件（有时它可能是一个不寻常的问题），请通过逐个移除小部件直到问题不再出现。然后您可以卸载这个小部件并提交一份缺陷报告，**若是官方小部件时**到KDE 缺陷跟踪页提交一份缺陷报告。如果它不是，您可以在 https://store.kde.org/ 上寻找它的条目并告知小部件的开发者您所碰到的问题（以及再现它的详细步骤等）。

如果您找不到问题，也不想丢失*所有*设置，浏览到`~/.config`：

```
$ for j in plasma*; do mv -- "$j" "${j%}.bak"; done
```

这个命令会将所有用户中跟 Plasma 有关的设置重命名为 *.bak （例如 `plasmarc.bak`），*并且当重新登录 Plasma 时，将会恢复**默认**设置。若要撤销该操作，请删除.bak文件扩展名。若已有* *.bak 文件，请先重命名，移动或删除它们。强烈建议您经常备份。 有关可能的方案列表，见同步和备份程序（英文）。

清理缓存以解决升级故障

此故障可能由旧的缓存导致。有时，升级后旧缓存可能会产生奇怪的、难以调试的行为，例如关不掉的 shell、改变设置时失去响应、以及像 ark 不能解压 rar/zip 文件又或者 amarok 不能识别音乐等各种其它问题。这个办法也能解决 KDE 和 Qt 程序在升级后变得难看的问题。

用以下命令来重建缓存：

```
$ rm ~/.config/Trolltech.conf
$ kbuildsycoca6 --noincremental
```

或者，清空 `~/.cache/` 文件夹内容，但是，这也会清除其他应用程序的缓存：

```
$ rm -rf ~/.cache/*
```

有些情况下清空 `~/.cache/` 文件夹不能解决问题，如果你遇到了类似下文所示的错误：

```
kf.service.sycoca: The menu spec file ( "" ) contains a Layout or DefaultLayout tag without the mandatory Merge tag inside. Please fix it.
```

这可能和一些过期配置文件有关，这种情况移走`~/.config/menus/`文件夹可能会修复问题。另外尝试逐一移走`~/.config/menus/`内的配置文件来确认哪一个文件导致问题出现也是一个好的解决办法。

Plasma 桌面不遵从区域设置/语言设置

Plasma 桌面可能使用与您在 KDE 系统设置面板或 `locale.conf` 中设置的设置不同的设置（根据 Locale#Variables）。首先要做的是注销并在删除 `~/.config/plasma-localerc` 后登录，如果这不能解决问题，请尝试手动编辑文件。例如，将 `LANG` 变量设置为 `zh_CN.UTF-8` 并将 `LC_MESSAGES` 变量设置为 `en_US.UTF-8`：

```
~/.config/plasma-localerc
[Formats]
LANG=zh_CN.UTF-8

[Translations]
LANGUAGE=zh_CN:en_US
```

不能在系统设置中更改主题、图标、字体、颜色；大多数图标不显示

确保 `QT_QPA_PLATFORMTHEME` 在环境变量未设置，命令 `printenv QT_QPA_PLATFORMTHEME` 输出应该为空。否则，如果您设置了环境（很可能是

qt5ct 或 qt6ct），该变量将在 Qt 应用程序上强制设置 qt5ct/qt6ct，命令 `export QT_QPA_PLATFORMTHEME=` 应该取消设置环境。

一个更简单（更可靠）的解决方案是完全卸载 qt5ct 和 qt6ct。

音量控制、通知或多媒体键不起作用

隐藏系统托盘设置中的某些项(如音频音量、媒体播放器或通知)也会禁用相关功能。隐藏“音频音量”会禁用音量控制键，“媒体播放器”会禁用多媒体键（倒带、停止、暂停），隐藏“通知”会禁用显示通知。

登录界面不会将光标设置同步到 SDDM

登录界面从 `~/.config/kcminputrc` 读取您的光标设置，如果没有此文件，则不会同步任何设置。生成此文件的最简单方法是在*系统设置* > *颜色和主题* > *光标*中更改光标主题，然后将其更改回首选的光标主题。

缺少面板/小部件

崩溃或硬件更改可以修改屏幕编号，即使在单个显示器设置上也是如此。此类事件后面板/小部件可能会丢失，这可以通过在 `~/.config/plasma-org.kde.plasma.desktop-appletsrc` 文件中修复 `lastScreen` 值解决。

图形问题

请先确保您已安装了适合您 GPU 的驱动程序。详见 Xorg#Driver installation。如果您的显卡较旧，您可以尝试 #禁用桌面特效或 #禁用混成。

在混合显卡的系统上强制使用独立显卡

Hybrid graphics 是笔记本电脑常用的一种电源管理策略，它使独立显卡（dGPU）在不需要时保持非活动状态，默认使用集成显卡（iGPU）进行基本的桌面渲染，以节省电池电量。

虽然这种方式有助于节能，但可能导致桌面性能不佳，包括动画帧率低下和潜在的图形异常，即使在配备性能足够的独立显卡的系统上也是如此。

强制 KDE Plasma 使用独立显卡，可以显著改善桌面响应速度和视觉质量。

方法 1: DRI_PRIME (开源驱动程序)

对于使用开源图形驱动程序 (Intel + AMDGPU, Intel + Nouveau) 的系统, 可通过 globally set 设置 `DRI_PRIME` 环境变量来指定使用独立显卡：

```
DRI_PRIME=1
```

索引值（0 或 1）取决于你的系统配置。通过运行以下命令来验证哪个索引对应于你的独立显卡：

```
DRI_PRIME=1 glxinfo
```

注意：此方法不适用于 NVIDIA 专有驱动程序。对于 NVIDIA，请用 PRIME render offload 或下面的 Kwin方法

方法 2: KWIN_DRM_DEVICES (KWin专用)

为了直接控制 KWin 的 GPU 选择，创建一个设置 DRM 设备优先级的启动脚本：

```
~/.config/plasma-workspace/env/gpu.sh
#!/bin/bash

export KWIN_DRM_DEVICES=/dev/dri/card1:/dev/dri/card0
```

要识别你的 DRM 设备卡及其对应的 GPU：

```
for i in /sys/class/drm/card*/device; do

    echo "Card: $(basename $(dirname $i))"

    if [ -f "$i/vendor" ] && [ -f "$i/device" ]; then

        echo "GPU: $(cat $i/vendor) $(cat $i/device)"

    fi

    echo "---"

done
```

在 `KWIN_DRM_DEVICES` 变量中将独立显卡的设备列在前面，以使其获得渲染优先权。

获取 KWin 的当前状态以用于获得帮助和调试

该命令打印出 KWin 当前状态的摘要，包括使用的选项，使用的合成后端以及相关的 OpenGL 驱动程序功能。详见Martin's blog。

```
$ qdbus6 org.kde.KWin /KWin org.kde.KWin.supportInformation
```

禁用桌面特效

Plasma 默认启用了桌面特效，并且不是所有的游戏都会自动禁用它们。您可以通过 *系统设置* > *窗口管理* > *桌面特效*禁用桌面特效。您也可以使用 `Alt+Shift+F12` 切换桌面效果。

另外，您也可以在*系统设置* > *窗口管理* > *窗口规则*下创建自定义 KWin 规则，以在某个应用程序/窗口启动时自动禁用/启用某些项。

启用透明功能

如果使用透明背景而不启用混成器，则会收到以下消息：

```
配色方案使用的透明背景在您的桌面上似乎不受支持
```

在*系统设置* > *显示和监视器* > *混成器*中，选中*启动时开启混成器*并重启 Plasma。

禁用混成

在*系统设置* > *显示和监视器* > *混成器*中，取消选中*启动时开启混成器*并重启 Plasma。

启用混成时全屏状态会闪烁

在*系统设置* > *显示和监视器* > *混成器*中，取消选中*允许应用程序阻止混成*，这可能会影响性能。

桌面网格等特效卡顿

为 KWIN 设置环境变量 `QSG_USE_SIMPLE_ANIMATION_DRIVER` 可以减少部分基于 Quick Scene Graphics 的特效的卡顿现象， 只需为运行 KWIN 的服务创建一个置入式配置片段即可：

```
/etc/systemd/user/plasma-kwin_x11.service.d/10-kwin_QSG_SAD.conf
[Service]

Environment="QSG_USE_SIMPLE_ANIMATION_DRIVER=1"
```

（对于 Wayland 会话，目录名应使用 plasma-kwin_wayland.service.d）

然后重启会话。

另一种方法是设置 `QSG_NO_VSYNC` 而非 `QSG_USE_SIMPLE_ANIMATION_DRIVER`.

Plasma 光标有时显示不正确

创建 `~/.local/share/icons/default/`（或 `~/.icons/default`）目录并在其内创建如下文件：

```
~/.local/share/icons/default/index.theme
[Icon Theme]
Inherits=breeze_cursors
```

如有需要，将 `breeze_cursors` 替换为自定义的光标主题。（光标主题可在 `/usr/share/icons/` 中找到，如`Breeze_Light`）

注意：你必须重新登录以确保设置生效。

Wayland 下，需要安装xdg-desktop-portal-gtk包 以保证 GTK/GNOME 应用的光标主题正确。

Firefox 和 Thunderbird 没有使用光标主题

Wayland 下，Firefox 和 Thunderbird 会根据 GSettings 来确定光标如何显示。

要将 KDE 的设置应用到GTK应用，安装 kde-gtk-config包。

如果不想安装额外的包，也可以手动设置光标主题：

```
$ gsettings set org.gnome.desktop.interface cursor-theme cursor-theme-name
```

（例如将鼠标悬停在超链接上时）光标抖动/闪烁

尝试为系统和窗口管理器安装相应的 2D 加速驱动程序。

屏幕分辨率设置不可用

您的 kscreen 本地配置设置可以覆盖 `xorg.conf` 中设置的配置。在 `~/.local/share/kscreen/` 中查找 kscreen 配置文件，并检查分辨率是否设置为显示器不支持的分辨率。

系统托盘上的图标模糊

为了在托盘中添加图标，应用程序经常使用 appindicator 库。如果图标模糊，请检查已安装的 libappindicator 版本。如果只安装了 libappindicator-gtk2包，那么您可以安装libappindicator-gtk3包，以尝试获得清晰的图标。

在虚拟机中无法更改屏幕分辨率

当在 VMware、VirtualBox 或 QEMU 虚拟机上运行 Plasma 时，kscreen 可能不允许将 Guest 的屏幕分辨率更改为高于 800x600 的分辨率。

解决方法是在 xorg.conf.d(5) 中设置 `PreferredMode` 选项，或者尝试在 VM 中使用其他图形适配器。例如，对于VirtualBox，使用 VBoxSVGA 代替 VMSVGA；对于 QEMU，用 Virtio 代替 QXL。详细信息请参见KDE Bug 407058。

Dolphin、Kate 等软件启动时卡住很长时间

检查您的用户文件夹(`Documents` 和 `Downloads` 等)是否设为了只读。

Spectacle 使用旧的屏幕状态截图

在*系统设置* > *显示和监视器* > *混成器*，更改*保持窗口缩略图*从*只对显示的窗口*到*从不*。如果你正使用 Intel 显卡， 请确认 xf86-video-intel包 没有安装。

GTK 应用里，字体渲染很差

参见 XDG 桌面门户#GTK应用在KDE Plasma中字体渲染很差.

窗口大小调整不当

您可能会发现某些应用程序的窗口不能正确调整大小，而是调整后的部分是透明的，鼠标点击会被发送到底层窗口。 要纠正这种行为，请将 KDE 的 GTK3 主题改为 oxygen-gtk 以外的其他主题。

在使用 modesetting 或 nouveau 驱动时，老旧 NVIDIA 显卡发生随机性死机

参阅 Nouveau#Random lockups with kernel error messages.

声音问题

注意：首先保证您已经安装了 alsa-utils包。

挂起后没有声音

如果挂起后没有声音并且 Kmix 没有显示应该显示的音频设备，可以尝试重新启动 plasmeshell 和 pulseaudio：

```
$ killall plasmashell
$ systemctl --user restart pulseaudio.service
$ plasmashell
```

某些应用程序也可能需要重新启动才能再次播放声音。

使用 GStreamer Phonon 后端时不能播放 MP3 文件

安装 GStreamer libav 插件（软件包gst-libav包）可以解决问题。如果仍然碰到，您可以尝试换一个软件包，例如phonon-qt6-vlc包。

然后确保通过*phononsettings*设置了正确的后端。

系统托盘没有音量控制图标并且无法使用快捷键控制音量

检查是否安装plasma-pa包

短时间后声音消失

如果`journalctl -p4 -t pulseaudio`包含`Failed to create sink input: sink is suspended`的信息，尝试注释`/etc/pulse/default.pa`内的行：

```
#load-module module-suspend-on-idle
```

如果问题仍然存在，plasma-meta包 或 plasma包组可能同时安装了pulseaudio包 和 wireplumber包 。要修复这个问题，将pulseaudio包 替换为 pipewire-pulse包 。 如果想使用 pulseaudio包 ， 将 wireplumber包 替换为 pipewire-media-session包 。参考 PipeWire#PulseAudio_客户端和这个贴子。

电源管理

没有挂起/休眠选项

如果您的系统可以使用 systemd 挂起/休眠，但 KDE 中没有这些选项，请确保 powerdevil包 已被安装。

没有电源配置方案选项

确保安装了powerdevil包 和 power-profiles-daemon包。 运行*powerprofilesctl*并检查驱动。如果是`intel_pstate` 或 `amd_pstate`，那问题应该已经解决，否则参考CPU_调频#调频驱动以启用相关驱动。

KMail

清理 akonadi 配置来修复 kmail

详见 。

若要备份配置文件，请复制以下文件夹：

```
$ cp -a ~/.local/share/akonadi ~/.local/share/akonadi-old
$ cp -a ~/.config/akonadi ~/.config/akonadi-old
```

KMail 的 IMAP 收件箱是空的

对于某些 IMAP 账户，kmail将把收件箱当作一个包含此帐户所有其他文件夹的容器显示。Kmail 不会在收件箱容器中显示消息，而是在所有其他子文件夹中显示消息，见 。若要解决此问题，只需在kmail帐户设置中禁用服务器端订阅即可。

KMail中EWS帐户的授权错误

在KMail中设置EWS帐户时，您可能会不断收到关于授权失败的错误，即使是使用有效的凭证。这可能是由于KWallet和KMail之间的通讯中断造成的。要解决此问题，请通过qdbus设置密码：

```
$ qdbus6 org.freedesktop.Akonadi.Resource.akonadi_ews_resource_0 /Settings
org.kde.Akonadi.Ews.Wallet.setPassword "XXX"
```

Aggressive QXcbConnection / kscreen.xcb.helper journal logging

见 Qt#Disable/Change Qt journal logging behaviour。

KF5/Qt 5应用程序在i3/FVWM/awesome中不显示图标

见 Qt#Configuration of Qt 5/6 applications under environments other than KDE Plasma。

保存凭据和持续显示 KWallet 对话框的问题

不建议在用户设置中关闭 KWallet 密码保存系统，因为需要它为每个用户保存加密凭证（如Wi-Fi密码）。关闭它可能会导致 KWallet 对话框持续出现。

如果您嫌每当应用程序想要访问 Kwallet 时需要解锁烦，您可以让登录管理器 SDDM 和 LightDM 在登录时自动解锁 KWallet，见 KDE Wallet。第一个钱包需要由 KWallet 生成（而不是"用户生成"），以便用于系统程序凭据。

如果您不希望让钱包凭据在内存内为每个应用打开，可以通过 kwalletmanager包 在KWallet设置中限制应用程序访问它。

如果您不关心凭证加密，您可以在创建钱包，KWallet 要求输入密码时，将密码留空。这样，应用程序将可以在不解锁钱包的情况下访问密码。

Discover不显示任何程序

根据你的Plasma/Qt版本，请安装packagekit-qt6包 或 packagekit-qt5包 以解决问题。

警告：包管理者在GitHub comment里做出了声明："通过packagekit管理系统软件包，从根本上不兼容我们有着大量维护的滚动式发行版，如果用户在重启前没有关注pacman的输出日志或合并pacnew文件，任何更新都有可能导致系统无法启动或处于一个不稳定的状态。"

Discover无法从Arch仓库获取更新

Discover有时不能释放PackageKit alpm锁，手动删除`/var/lib/PackageKit/alpm/db.lck`文件后更新Discover可解决问题。

NVIDIA驱动程序的kscreenlocker_greet的CPU使用率很高

如 KDE 错误 347772 中所述，NVIDIA OpenGL 驱动程序和 QML 可能无法与 Qt 5 很好地配合使用。这可能导致 `kscreenlocker_greet`解锁会话后 CPU 使用率高。要解决此问题，请将 `QSG_RENDERER_LOOP` 环境变量设置为 `basic`。

然后用 `killall kscreenlocker_greet` 杀死之前的 greeter 实例。

在 ZFS 上运行 Akonadi 时出现操作系统错误 22

如果您的主目录位于 ZFS 池中，请创建一个 `~/.config/akonadi/mysql-local.conf` 文件，其内容如下：

```
[mysqld]
innodb_use_native_aio = 0
```

请参阅 MariaDB#OS error 22 when running on ZFS.

某些程序的窗口处于非活动状态时无法滚动

这是由GTK3处理鼠标滚动事件处理有问题造成的。一个解决方法是设置environment variable `GDK_CORE_DEVICE_EVENTS=1`。但是，这种方法也会破坏触摸板的平滑滚动和触摸屏滚动。

TeamViewer 很卡

在使用TeamViewer时，如果您使用平滑动画(比如最小化窗口)，它可能会运行得很慢。参见 #禁用混成作为一种解决方案。

Kmail、Kontact 和 Wayland

Kmail可能会失去响应，显示一个黑色的消息视图或者类似的，通常是在最小化和恢复之后。一个解决方案可能是设置 environment variable `QT_QPA_PLATFORM="xcb;wayland"`。

解锁小部件 (Plasma ≥ 5.18)

如果您之前锁定了小部件，您可能会发现自己无法再次解锁它们。 您只需运行此命令即可：

```
$ qdbus6 org.kde.plasmashell /PlasmaShell evaluateScript "lockCorona(false)"
```

新的 `Customize Layout` 不需要将它们锁定，但如果想这样做：

```
$ qdbus6 org.kde.plasmashell /PlasmaShell evaluateScript "lockCorona(true)"
```

小部件无法运行或出现bug

可安装`plasmoidviewer`包查看输出信息，做出相应处理：

```
yay -S plasmoidviewer-debug
plasmoidviewer --applet <小部件名称>
```

KIO以错误的程序打开URL连接

检查HTML, PHP等的文件关联，将其设置为浏览器。KIO缓存文件位于`$HOME/.cache/kioexec`。另参见xdg-utils#URL scheme handlers。

在暂停和休眠之前锁定屏幕

在系统设置应用程序中，KDE 提供了从睡眠中醒来后自动锁定屏幕的设置。恢复后，some users报告说屏幕在锁定前短暂显示。要防止这种行为并让 KDE 在挂起之前锁定屏幕，请在 systemd(1) 中以 root 用户创建以下文件来创建挂钩：

```
/usr/lib/systemd/system-sleep/lock_before_suspend.sh
#!/bin/bash

case $1/$2 in
    pre/*)
        case $2 in
            suspend|hibernate)
                loginctl lock-session
                sleep 1
                ;;
            esac
        ;;
esac
```

为了在设备挂起之前完成`loginctl lock-session`命令，必须使用`sleep`。较低的值可能不足以完成这个命令。

创建文件后，将其设为可执行的。

最后，通过转到*系统设置* > *锁屏*并选中*从休眠恢复时锁定屏幕*复选框，确保启用了 该 KDE 设置。

X11快捷键在Wayland上冲突

自KDE 5.27起，一些X11软件例如freerdp包会捕获键盘输入，其它软件像VMware无法正确捕获。

在Xserver或在混成器强制捕获快捷键并不合适。 下面的方法解决问题更优雅一些：

- 右键窗口标题栏（例如VMware或Citrix）；
- *More Actions* > *Configure Special Window Settings...*
- 单击 *Add Property...*并选择*Ignore global shortcuts*。
- 选择 *force* 和 *yes*，应用设置。

改变系统设置后没有生效

这可能是因为系统设置无法访问/修改位于家目录的 .config 文件夹。

要解决该问题，你需要改变文件夹的所有者：

```
# chown user:user /home/user/.config
```

`*user*` 代表你在KDE Plasma登陆的用户。如果你的家目录和你登陆的账户不同，请在相应位置更改。

如果上面的办法无效，你可能还需要更改文件夹的权限：

```
# chmod 755 /home/user/.config
```

某些应用的 Plasma 6 全局菜单不能使用

即使安装了appmenu-gtk-module包和libdbusmenu-glib包，"全局菜单"小部件在一些应用上仍然不能使用。要解决这个问题，安装plasma5-integration包并重启你的会话。

参见

- KDE 主页
- KDE news
- KDE Blogs
- KDE 论坛
- KDE Wikis
- KDE 缺陷跟踪页
- Martin Graesslin's blog
- KDE Matrix 房间

LXDE

本章把轻量桌面栈从登录界面推到日常可用：LXDM 管登录与 greeter、GTK 两种主题，LXDE 管自身的安装与启动，PCManFM 除文件管理外还管桌面。按 LXDM、LXDE、PCManFM 的顺序读：《LXDE》与《PCManFM》必读，前者是地基，后者不读容易把桌面当成可有可无；LXDM 只在需要图形登录界面时精读。

最常见的坑是关掉 PCManFM：桌面由它提供，进程一退就没了，得用守护进程模式配开机自启让它常驻。另一误解是 greeter 与 GTK 主题共用一套配置，其实改错位置就不会生效。

13.1　LXDM

难度 进阶 | **篇幅** 8.2 KB | **所属** LXDE

本节将讲到

- 安装
- 配置
- 提示和技巧
- the theme of greeter
- GTK theme

LXDM 是轻量级的 LXDE 桌面环境使用的显示管理器。

LXDM 不支持 XDMCP 协议，要使用 XDMCP，请使用 LightDM.

安装

安装 GTK+ 2版本的lxdm包 软件包 或 GTK+ 3版本的lxdm-gtk3包软件包 。

启用 systemd 服务 `lxdm.service`。

配置

警告：lxdm.conf 中必须包含语言选择控制，请设置 **lang=1** 否则 LXDM 会不停循环启动，无法载入会话。

LXDM 的配置文件都位于 `/etc/lxdm`。主配置文件是 `lxdm.conf`，注释非常详细。`Xsession` 是系统 X 会话配置文件，一般不需要修改。目录中的其他文件都是 shell 脚本，在 LXDM 发生相应事件时运行：

1. `LoginReady`: 在 LXDM 准备显示登录窗口时以 root 权限运行。
2. `PreLogin`: 用户登录前以 root 权限运行。
3. `PostLogin`: 用户登录后以登录的用户运行。
4. `PostLogout`: 用户注销后以用户权限运行。

5. `PreReboot`: 通过 LXDM 重启时以 root 运行。
6. `PreShutdown`: 通过 LXDM关机时以 root 运行。

默认会话

默认会话可以全局配置，也可以在用户级别配置。用户设置优先于全局设置。

全局设置

要修改 LXDM 的默认会话或桌面环境，请编辑 `/etc/lxdm/lxdm.conf` 把会话行改为如下：

```
session=/usr/bin/startlxde
```

例如 Xfce:

```
session=/usr/bin/startxfce4
```

例如 Openbox:

```
session=/usr/bin/openbox-session
```

例如 GNOME:

```
session=/usr/bin/gnome-session
```

例如 LXQt:

```
session=/usr/bin/xfwm4
```

在使用的主题无会话选择框或者自动登录有问题时，这个设置很有用。

各个用户设置

要定义独立用户的会话，请编辑 `~/.dmrc` 并定义会话。

例如：用户1要用 xfce4，用户2要用cinnamon,用户3要用GNOME:

用户1的设置：

```
[Desktop]
Session=xfce
```

用户2的设置：

```
[Desktop]
Session=cinnamon
```

用户3的设置：

```
[Desktop]
Session=gnome
```

可用如下命令显示已安装的会话列表

```
$ ls /usr/share/xsessions/
```

自动登录

如果要免密自动登录一个用户，找到 `/etc/lxdm/lxdm.conf` 中的:

```
#autologin=dgod
```

取消前面的注释，并将dgod改为想要免密登录用户名。

上次的登录选项

以前使用的 LXDM 选项可以在以下位置找到：

```
/var/lib/lxdm/lxdm.conf
[base]
last_session=/usr/share/xsessions/LXDE.desktop
last_lang=sv_SE.UTF-8
last_langs=sv_SE.UTF-8 fa_IR.UTF-8 en_US.UTF-8
```

注意：卸载 LXDM 时，此文件不会自动删除。如果要删除 LXDM 选项的所有痕迹，则必须手动删除它。

提示和技巧

添加表情图标

一个96x96像素的图像（jpg或png格式）可以选择性地在每个用户上替代默认图标进行显示。只需将目标图像复制或使用符号链接到 `$HOME/.face`。gnome-control-center包 包提供了一些适用于lxdm屏幕的默认图标。 在安装该软件包后，在 `/usr/share/pixmaps/faces` 目录下查找。

注意：

- 用户不需要保留 gnome-control-center包 包来使用这些图像。只需安装它，将它们复制到其他地方，然后卸载它即可。
- 用户的目录应该对其他人具有r-x权限，而.face文件对其他人应该有r--权限。但显然，这会带来安全和访问方面的影响，因为现在任何人都可以浏览你的主目录。
- 一个名为 `lxdm-config` 的图形化工具随lxdm一起提供，可以用于在主目录中放置 `.face` 文件，以及其他配置文件。

同时登录和切换用户

LXDM 可以让多个用户同时登录到不同 ttys，下面的命令可以登录另一个用户，而不需要注销当前用户会话：

```
$ lxdm -c USER_SWITCH
```

注意：用USER_SWITCH登录新用户时，使用的是tty7的下一个tty。使用USER_SWITCH 命令登录用户1后，用户2正常登录，用户2会在tty7,，用户1将会位于 tty1。

主题

LXDM 主题位于 `/usr/share/lxdm/themes`.

LXDM 仅提供了一个主题 Industrial. 要显示主题背景文件 `wave.svg`，请安装软件包 librsvg包.

lxdm-themesAUR 提供了 6 个额外的主题：Archlinux, ArchlinuxFull, ArchlinuxTop, Arch-Dark, Arch-Stripes 和 IndustrialArch. lxdm-gitAUR 也提供了 ArchStripes 和 ArchDark(名字改了一下以避免冲突).

主题文件通过 /etc/lxdm/lxdm.conf 配置:

```
## the theme of greeter
theme=theme_name
```

要让 LXDM 使用 GTK 主题(位于 /usr/share/themes)，在配置文件中设置：

```
## GTK theme
gtk_theme=gtk_theme_name
```

高级会话配置

用户登录后，LXDM 会按下面顺序引用全部文件：

1. /etc/profile
2. ~/.profile
3. /etc/xprofile
4. ~/.xprofile

这些文件可以设置会话的环境变量，启动必须的服务例如 ssh-agent。详情请参考 Xprofile。

LXDM 不会引用 ~/.xinitrc，所以如果需要从使用这些文件的显示管理器迁移到 LXDM，需要将设置移动到其它文件，例如 ~/.xprofile. LXDM 也不会引用 ~/.bash_profile。

如果还想使用 ~/.xinitrc，可以在 /etc/lxdm/PostLogin 中加入：

```
source ~/.xinitrc
```

LXDM 也会使用 .Xresource, .Xkbmap, 和 .Xmodmap。LXDM 系统配置和用户配置的详细状况可以参考 /etc/lxdm/Xsession。

问题处理

白闪

当使用默认的LXDM主题 theme=Industrial 和一个暗色背景图片（例如 bg=/usr/share/backgrounds/img.png）时，LXDM启动之前可能会出现短暂的明亮闪光。 这是由于所选 GTK+ 主题的 bg_color: 属性引起的。 为了避免这种情况，请将 gtk_theme=Adwaita 更改为 gtk_theme=Adwaita-dark 或其他暗色主题。

注销问题

如果您在使用 lxdm 时无法注销（例如卡住、显示冻结等），请尝试取消 /etc/lxdm/lxdm.conf 中的 reset=1 选项的注释，以便在每次注销时刷新 xserver

13.2 LXDE

难度 进阶 | **篇幅** 10.7 KB | **所属** LXDE

本节将讲到

- 安装
- 运行 LXDE
- 小提示
- 故障排除
- 相关资源

摘自 LXDE.org | LXDE官方主页:

: *"Lightweight X11 Desktop Environment" 是一种极快且节能的桌面环境。由一个国际开发者社区维护，它拥有漂亮的界面、多语言支持、标准键盘快捷键，以及诸如选项卡式文件浏览等额外功能。LXDE比其他环境使用更少的CPU和RAM。它专为具有低硬件规格的云计算机而设计，例如网书、移动设备（如MIDs）或老旧计算机。*

安装

使用LXDE最少需要安装 lxde-common包, lxsession包, openbox包(或者其他窗口管理器）。lxde包组包含完整的桌面。

GTK+ 3 版本

你可以安装 lxde-gtk3包组 软件包组来试用这个尚在实验阶段的版本.

它几乎可以正常工作，但是可能会有一些问题，包括： gpicview, lxappearance-obconf, lxlauncher 和 lxpanel.

运行 LXDE

显示管理器

LXDM是LXDE的默认显示管理器，并作为lxde包组组的一部分。另请参见Display manager。

命令行

使用 "startx"， 你需要在你的xinitrc文件最后添加

```
~/.xinitrc
exec startlxde
```

另见Start X at login

小提示

应用程序配置项

应用程序菜单是通过解析放在`/usr/share/applications/`和`~/.local/share/applications/`中的`.desktop`文件实现的。要添加或编辑菜单中的项，请见desktop entries。可在AUR中找到第三方的菜单编辑器（如lxmedAUR）。同时也有官方的菜单编辑器，如alacarte包（gnome）和mozo包（mate）等。

程序自动开启

应用程序可以用以下方式设置自动打开：

- 使用`.desktop`文件

LXDE 工具 XDG Autostart

- 使用LXsession

`~/.config/lxsession/LXDE/autostart`中的每一行表示要执行的命令。如果一行以`@`开头，并且其后的命令崩溃，则该命令将自动重新执行。例如：

```
~/.config/lxsession/LXDE/autostart
@lxterminal
@leafpad
```

注意：这些命令*不*以"&"结尾。

也有一个全局自启动文件，位于`/etc/xdg/lxsession/LXDE/autostart`。

注意：如果两个文件都存在，LXsession只执行v0.4.9之后的本地文件

鼠标键盘动作

鼠标和键绑定（即键盘快捷键）是使用Openbox实现的。LXDE用户应该参考Openbox 维基来编辑 `~/.config/openbox/lxde-rc.xml`.

obkeyAUR 包提供了一个可选的图形用户界面，用于编辑键位绑定。默认情况下，它编辑 `rc.xml`，但您可以按照以下方式将其定向到LXDE配置文件中：

```
$ obkey ~/.config/openbox/lxde-rc.xml
```

请参见 来获取更多信息

光标

lxappearance包 包是一个图形化工具，用于设置 GTK 的外观和感觉，包括光标主题。使用LXAppearance配置的设置将被写入到 `~/.gtkrc-2.0`,

`~/.config/gtk-3.0/settings.ini` 和 `~/.icons/default/index.theme` 文件中。请参见 Cursor themes。

任务栏时间

您可以右键单击面板上的数字时钟小插件，并使用strftime格式设置它显示当前时间的方式。请参见 strftime(3)。

字体设置

lxappearance-obconf包 用于配置 Openbox 设置。此外，还可以参考 Font configuration。

键盘布局

lxpanel包 包括一个键盘布局小程序。 请参阅 Keyboard configuration in Xorg 以获得通用指令，以及 #程序自动开启以在LXDE中自动启动 *setxkbmap*。

锁屏

LXDE本身不提供屏幕锁的功能。请参见 List of applications/Security#Screen lockers 和 #程序自动开启相关说明来启动它们。

Screen Lock 图标执行的脚本（位于 `/usr/bin/lxlock`）会搜索一些众所周知的屏幕锁定程序，并使用找到的第一个程序锁定屏幕。请参见 GitHub 上的 lxlock。

`/etc/xdg/lxsession/LXDE/autostart` 文件（位于 lxde-common包 包中）列出了 XScreenSaver，它将自动启动。

请参阅 DPMS，了解如何在没有外部程序的情况下控制屏幕保护。

LXPanel中的图标

LXpanel 使用的默认图标存储在 `/usr/share/pixmaps/` 中，任何自定义图标也应该保存在这里。

要更改应用程序的默认图标，请参考 Desktop entries#Icons。

LXPanel中的菜单

面板的菜单可以在 `/etc/xdg/menus/lxde-applications.menu` 中进行配置，遵循 xdg-menu 格式，以便与其他会话（特别是 MATE）中的应用程序配合使用，以增加 LXDE 缺乏的某些功能。

使用不同的窗口管理器

LXsession 使用 `~/.config/lxsession/LXDE/desktop.conf` 文件中定义的 window manager （默认为 Openbox）。 如果该文件不存在，则会在 `/etc/xdg/lxsession/LXDE/desktop.conf` 中进行搜索。

将文件中的 `openbox-lxde` 替换为您选择的窗口管理器：

对于 metacity：

```
window_manager=metacity
```

对于 compiz：

```
window_manager=compiz
```

对于 EXWM：

```
window_manager=emacs
```

或者按照 #程序自动开启部分中定义的方法使用 `*WM* --replace`，其中 *WM* 是要启动的窗口管理器可执行文件的名称。 这意味着每次登录时都会首先启动 *openbox* ，然后立即被替换。 请注意，Openbox和LXDE不共享相同的 `rc.xml` 此键盘快捷键可能会有所不同。请参见 xbindkeys。

使用多个窗口管理器

LXDE默认情况下不启用混合，这会导致屏幕撕裂问题。如果您对此不满意，您可以牺牲更多的资源来启用窗口混合，要么安装混合管理器，要么将lxde替换为其他支持混合的窗口管理器。

故障排除

一些程序不能运行

最近LXDE转向gtk3引入了一些错误，可能导致分段错误并阻止应用程序启动（尤其是自定义外观设置，也就是lxappearance）。在此问题完全修复之前，您可以尝试使用JWM来绕过gtk3的分段错误：

- 安装 JWM

```
$ sudo pacman -S jwm
```

- 将LXDE替换为jwm, 打开终端并运行：

```
$ jwm
```

- 这可能会导致一些小问题，但现在您应该能够正常打开segfault应用程序了
- 完成后，请重新登录以刷新LXDE桌面

带有中文字符的NTFS

对于具有NTFS文件系统的存储设备，您需要安装 NTFS-3G 软件包。通常情况下，PCManFM与NTFS文件系统兼容良好，但是，对于使用NTFS的用户来说，存在一个问题，即如果您在NTFS文件系统上有包含非拉丁字符 （例如中文字符）的文件或目录， 当打开（或自动挂载）NTFS卷时，这些文件或目录的名称可能会消失。这是因为 lxsession mount-helper 未能正确解析策略和区域设置选项。对此有一个解决方法：

创建一个新的 Bash 脚本，将其命名为 `/usr/local/bin/mount.ntfs-3g` 并包含以下内容：

```
#!/bin/bash
/usr/bin/ntfs-3g $1 $2 -o locale=en_US.UTF-8
```

然后将其设置为可执行：

```
# chmod +x /usr/local/bin/mount.ntfs-3g
```

LXPanel崩溃问题解决方法

使用某些 GTK 主题时，启动 *lxpanel* 可能会导致以下错误：

```
lxpanel: cairo-scaled-font.c:459: _cairo_scaled_glyph_page_destroy: Assertion `!
scaled_font->cache_frozen' failed.
```

在这种情况下，安装 ttf-dejavu包。

如果在浏览特定的Unicode网页时，lxpanel崩溃，请安装 ttf-droid包。

LXPanel任务栏图标大小设置

正在运行的应用程序的图标与 *Panel Settings* > *Geometry* 设置的 *Icon size* 不匹配，而是小4px， 这会导致其中一些图标模糊。为了在任务栏中显示清晰的32px图标，需要将 *Icon size* 设置为36px，但这样做会使其他活动面板小工具的图标变模糊。为了解决这个问题，可以创建额外的面板，并通过在 *Panel Settings* > *Geometry* 中调整对齐和边距，使它们共同形成一个连续的面板。

在LXTerminal中实现虚拟透明效果

最新版本的 VTE terminal widget library 要求使用一个合成窗口管理器才能实现背景透明效果。而未维护的、遗留的 GTK 2 版本的 VTE 则采用伪透明技术，终端将显示桌面背景图像。如果你更喜欢伪透明效果，可以使用 lxterminal-gtk2AUR 软件包安装 GTK 2 版本的 LXTerminal。

光标太小

高分辨率屏显示光标过小，可以在Gnome-look里找有36px或48px的光标主题文件并下载，之后在自定义外观里的光标下安装并选择使用。

相关资源

- LXDE项目
- 最新组件
- PCMan文件管理器

13.3 PCManFM

难度 进阶 | **篇幅** 14.5 KB | **所属** LXDE

本节将讲到

- 安装
- 桌面管理
- 守护进程模式
- 开机自启
- 其他特性和功能

PCManFM 是一个非常快速和轻量级的文件管理器，也是 LXDE 的标准文件管理器。它使用 GTK 作为其 UI，并使用 GVFS （在 GNOME 的 GIO) 库）提供虚拟文件系统功能，例如文件回收站功能和挂载远程文件系统的功能。

PCManFM-Qt是使用Qt的版本，是\[LXQt\]\] 的标准文件管理器。尽管使用 Qt作为UI工具包，但PCManFM-Qt保留了GVFS，而非使用KDE的 KIO 。从本质上讲，这两个文件管理器都与桌面环境无关。

安装

安装下列软件包之一：

- GTK 2: pcmanfm包
- GTK 3: pcmanfm-gtk3包
- Qt6: pcmanfm-qt包

可选组件：

- gvfs包：提供回收站功能
- udisks：远程文件系统的挂载支持
- gvfs-smb包 支持SMB/CIFS

开发版本

AUR中提供开发版本:

- GTK 2: pcmanfm-gitAUR
- Qt6: pcmanfm-qt-gitAUR

桌面管理

如果要用PCManFM进行桌面管理，比如设置壁纸和显示桌面图标，使用这个命令：

```
$ pcmanfm --desktop
```

窗口管理器提供的桌面菜单会被PCManFM提供的桌面菜单所替换。如果要还原，在桌面首选项的高级选项卡中勾选点击桌面时显示窗口管理器提供的菜单选项。

桌面首选项

如果使用窗口管理器提供的桌面菜单，请输入以下命令以随时设置或修改桌面首选项：

```
$ pcmanfm --desktop-pref
```

可以考虑给这条命令绑定快捷键或添加到原生桌面菜单中以方便使用。

新建图标

文件可以直接拖放到桌面上。至于应用程序快捷方式，需要把它们的`.desktop`文件**复制**到`~/Desktop`文件夹；不能拖放`.desktop`文件，否则就会是移动而不是复制，这会导致这个应用从应用启动器中消失。如果用命令行就应该是这样：

```
$ cp /usr/share/applications/name-of-application.desktop ~/Desktop
```

例如，下面的命令为 lxterminal包 创建了一个桌面快捷方式：

```
$ cp /usr/share/applications/lxterminal.desktop ~/Desktop
```

使用 XDG user directories 程序能创建自己的目录，无需再配置`$HOME` 环境变量。

守护进程模式

如果你想在后台运行PCManFM (比如说要自动挂载移动硬盘等可移动介质)，使用：

```
$ pcmanfm --daemon-mode
```

一次只能有一个 PCManFM 实例作为守护进程运行。

如果自动挂载失败，请参见 udisks.

开机自启

PCManFM 可以被自启为一个守护进程或用于管理桌面。

其他特性和功能

经验不足的用户应该意识到，单独的文件管理器 ，尤其是当安装一个独立窗口管理器（如 Openbox）将无法提供完整桌面环境（如 Xfce和KDE ）的用户所习惯的特性和功能。有关详细信息，请查看文件管理器功能一文。

提示与技巧

获取其他文件类型缩略图

查看文件管理功能# PCManFM获取其他文件类型缩略图.

设置终端模拟器

在 *Edit* > *Preferences* > *Advanced* 里面的 *Tools* > *Open Current Folder in Terminal*，你可以配置 PCManFM 调用的终端模拟器。

集成压缩包管理器

可以在 *Edit* > *Preferences* > *Advanced* 中设置集成的压缩包管理器。目前 PCManFM 和PCManFM-Qt都支持 file-roller包, xarchiver包 (或者 xarchiver-gtk2包), engrampa 包, ark包 . PCManFM-Qt 还支持 lxqt-archiver包，这是 LXQt 的默认选择。

将自定义项目添加到上下文菜单

PCManFM 支持桌面文件规范扩展 (DES-EMA)，它允许您将任意项目添加到文件和目录的上下文菜单中。要添加你自己的项目，请创建（如果不存在）`~/.local/share/file-manager/actions/`文件夹并向其中添加`.desktop` 文件：

```
~/.local/share/file-manager/actions/action.desktop
[Desktop Entry]
Type=Action
Profiles=profile_id
Name=Action name in English
Name[cc]=Action name in Language
Icon=Icon name
# Example: Icon=text-editor

[X-Action-Profile profile_id]
MimeTypes=MIME-types (semicolon separated)
Exec=command and arguments
# Example: Exec=gedit %f
```

你可以通过列出其id（以分号分隔），将一个或多个配置文件绑定到单个操作。配置文件允许你指定要为哪些文件类型执行哪些命令，因此同一操作可以根据所选文件类型运行不同的命令。除了特定的 MIME 类型（例如：`text/plain`文本文件），你可以使用以下常规类型：

- `all/allfiles` --文件；
- `inode/directory` -- 文件夹
- `all/all` -- 文件和文件夹。

注意：可以使用 pkexec 以 root 身份运行命令，但直接调用它不起作用，您需要创建一个中间脚本，如 LXDE 的 wiki中所述。

“创建新的...”模板

模板文件保存在 `~/Templates` ，点击*文件*>*新建...*可以选择相应的模板。默认的模板是“创建文件夹”和“创建空白文件”。

缩略图

**本文或本章节的事实准确性存在争议。**

原因： 使用现代 SSD 时，PCManFM 可以毫无问题地呈现包含数千个媒体文件的目录的缩略图。（在 Talk:PCManFM 中讨论）

和其他一些文件管理器（例如 Nautilus）一样，PCManFM 将加载文件夹中所有图像的预览。为了不滥用 HDD，请将文件夹中的图像数量保持在 100 个之内。

故障排除

“打开方式”对话框窗口为空

如果您在打开方式对话框中没有看到任何可供选择的应用程序，那么你可以试着卸载 gnome-menus包 然后安装 lxmenu-data包。此外，设置如下环境变量：

- `XDG_MENU_PREFIX=lxde-`
- `XDG_CURRENT_DESKTOP=LXDE`

列表中没有 "应用"

删掉 `$HOME/.cache/menus` 文件夹里的东西，然后重新运行 PCManFM。

PCManFM 需要设置`XDG_MENU_PREFIX`环境变量。变量的值应与`/etc/xdg/menus/`文件夹中文件的开头部分匹配。

参考 #“打开方式”对话框窗口为空.

参考获取更多信息 以及Linux Mint论坛的（特别推荐）

无图标

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：当建议编辑配置文件时使用Template:hc（在Talk:PCManFM讨论）

**本文或本章节的事实准确性存在争议。**

原因： 我们是否应该建议编辑文件管理器的全局 gtk 配置？（在 Talk:PCManFM 中讨论）

如果你用的是窗口管理器而不是桌面环境，而文件夹和文件没有图标，你需要指定 GTK 图标主题。

例如，你安装了 oxygen-icons包，在 `~/.gtkrc-2.0` **或者** `/etc/gtk-2.0/gtkrc` 里添加这一行：

```
gtk-icon-theme-name = "oxygen"
```

注意：重启 PCManFM 才能生效。

或者，用一个不同的主题（gnome，hicolor和 locolor 三个主题除外）。用下面这个命令查看安装了的图标主题：

```
$ ls ~/.icons/ /usr/share/icons/
```

如果看着都不爽，那就用这个命令查看所有可以安装的图标主题，选一个来安装：

```
$ pacman -Ss icon-theme
```

提示：如果想要有个图形界面，安装 lxappearance包 并用它来设置图标主题。

鼠标按钮不能触发 "上一/下一 文件夹" 功能

用 Xbindkeys 来修复这个功能。

安装 xbindkeys包、xvkbdAUR，在 `~/.xbindkeysrc` 里添加以下内容：

```
~/.xbindkeysrc
# Sample .xbindkeysrc for a G9x mouse.
"/usr/bin/xvkbd -text '\[Alt_L]\[Left]'"
 b:8
"/usr/bin/xvkbd -text '\[Alt_L]\[Right]'"
 b:9
```

按键代码可以通过 xorg-xev包 获取。

最后在 `~/.xinitrc` 里添加以下内容来在登录时触发 *xbindkeys*。

```
xbindkeys &
```

\--desktop 参数不生效或使X-server崩溃

确保你有 `~/.config/pcmanfm` 文件夹的所有权和写权限。

通过使用 `--desktop-pref` 参数或者修改 `~/.config/pcmanfm/default/pcmanfm.config` 来设置桌面壁纸来解决问题。

终端模拟器的高级配置没有保存

**本文或本章节的事实准确性存在争议。**

原因： 为什么 libfm 的配置文件默认没有权限（在 Talk:PCManFM 中讨论）

请设置 libfm 配置文件的权限：

```
$ chmod -R 755 ~/.config/libfm
$ chmod 644 ~/.config/libfm/libfm.conf
```

记住文件排序设置

在 *View* > *Sort Files* 里可以设置文件排序，但是如果要让 PCManFM 记住这个设置，需要打开 *Edit* > *Preferences* 然后再关掉，这样会让当前的sort_type 和 sort_by 的值写入 `~/.config/pcmanfm/LXDE/pcmanfm.conf` 文件。

挂载设备时候提醒 "Not authorized"

在 `/etc/polkit-1/rules.d/00-mount-internal.rules` 文件里添加这个 polkit 规则：

```
/etc/polkit-1/rules.d/00-mount-internal.rules
polkit.addRule(function(action, subject) {
    if ((action.id == "org.freedesktop.udisks2.filesystem-mount-system" &&
       subject.local && subject.active && subject.isInGroup("storage")))
       {
          return polkit.Result.YES;
       }
 });
```

并且把你的用户添加到 `storage` 用户组里：

```
# usermod -aG storage username
```

不支持操作

如果您忘记安装可选的依赖项 gvfs包，请先检查，否则请参阅会话权限一文。

系统重新启动时忘记密码

如果合适，安装一个 keyring 应用程序，比如 GNOME/Keyring， KDE Wallet 或者 lxqt_walletAUR 用于 网络共享或SSH agent。

Suckless

本章处理桌面里最容易被套件包办的三件事：锁屏、菜单、终端。Slock、Dmenu、St 同属 suckless，体积小、依赖少，代价是默认状态近乎空白，能拿到多少功能取决于配置读到什么程度。建议先读 St：篇幅最长，除配置外还专列故障排除，问题多集中于此；再读 Dmenu，安装与设置过一遍，延伸与其他资源按需取用；Slock 最短，扫一遍用法与提示即可，其余可跳。最容易踩的坑是把三者当成装完即用的成品，遇到异常应先回到安装与配置两节逐项核对，而不是先去别处找现成模板。

14.1　Slock

难度 入门 | **篇幅** 2.9 KB | **所属** Suckless

本节将讲到

- 安装
- 配置
- 用法
- 提示与窍门
- 参见

Slock，或 "Simple X display locker"，是用于 X 的显示锁定器，其目标是最小、快速和轻量。。

安装

安装 the slock包 或者 slock-gitAUR 软件包。

配置

配置可通过编辑`config.h`头文件，然后重新编译软件包来完成。配置完成后，您应该创建软件包。

用法

只需运行 `slock` 即可锁定屏幕。您还可以提供一个参数，以便在屏幕锁定后运行：

```
$ slock cmd [arg ...]
```

要解锁屏幕，只需输入密码即可。

提示与窍门

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： 您可以参与翻译本节。（在 Talk:Slock# 中讨论）

挂起时锁定

创建以下服务，关闭显示器并锁定屏幕。

```
/etc/systemd/system/slock@.service
[Unit]
Description=Lock X session using slock for user %i
Before=sleep.target

[Service]
User=%i
Environment=DISPLAY=:0
ExecStartPre=/usr/bin/xset dpms force suspend
ExecStart=/usr/bin/slock

[Install]
WantedBy=sleep.target
```

启用`slock@*user*.service`systemd 单元，使其对用户名 *user* 生效。

Block VT switching and prevent killing X

slock recommends blocking VT switching so that the screen lock cannot be bypassed. For the same reason, *slock* recommends preventing users from killing the X server. See Xorg#Block TTY access and Xorg#Prevent a user from killing X.

参见

- 官方主页

14.2 Dmenu

难度 入门 | **篇幅** 3.0 KB | **所属** Suckless

本节将讲到

- 安装
- 设置
- 延伸
- 其他资源

dmenu是一个X下的快速、轻量级的软件启动器，它从stdin读取任意文本，并创建一个菜单，每一行都有一个菜单项。 然后，用户可以通过方向键或键入名称的一部分来选择一个项目，该行就会被输出到stdout。 dmenu_run是 dmenu 发行版附带的包装器，可将其用作应用程序启动器。

安装

安装 dmenu包，或者是开发者版本的 dmenu-gitAUR。

通过以下命令来运行 *dmenu*：

```
$ dmenu_run
```

设置

现在你可以将此命令关联到一个快捷键，很多窗口管理器和桌面环境都有设置工具可以做到，用 xbindkeys 也可以做到，想得到更详细的信息参见 Hotkeys 条目。

显示自定义项目

自定义项目可以通过用换行符(*n*)来分隔并输入到 *dmenu* 来让它们显示出来，比如：

```
$ echo -e "第一个\n第二个\n第三个" | dmenu
```

手动添加项目

dmenu 会在你的 `$PATH` 路径下的目录里查找可执行文件并生成菜单项。如果要修改 `$PATH`，参见 environment variables。

字体

dmenu 4.6 起 XFT 字体渲染默认启用（4.6 发行注记）。fontconfig包的font.conf 语法会被使用。

shell 别名支持

dmenu 不支持 shell 别名. 为了让 *dmenu* 能够识别别名，安装 *dmenu-recent-aliases*（已从AUR移除） 包然后运行 `dmenu_run_aliases`。 别名必须要在 `~/.bash_aliases` 或者 `~/.zsh_aliases` 文件里面来让 *dmenu_run_aliases* 识别出来。

延伸

```
$ pacman -Ql dmenu | grep bin
dmenu /usr/bin/dmenu
dmenu /usr/bin/dmenu_path
dmenu /usr/bin/dmenu_run
```

可见/usr/bin/下有三个文件，其中**dmenu_path**和**dmenu_run**是两个shell脚本，真正的执行/显示部分都由**dmenu**完成，其中**dmenu_path**用于列出$PATH里的可执行文件，每个文件名一行，然后通过“|”传递给**dmenu**，具体语法可从**dmenu_run**里找到。 你也可以在终端里执行：

```
$ echo | dmenu
```

然后输入任意字串，这个字串就会被显示在终端里，这其实才是dmenu的核心功能。

由此配合其他工具可以完成其他任务，比如运行：

```
$ notify-send "`exec $(echo | dmenu)`"
```

你可以试着在运行后输入date，之后系统就会借助notify-send弹出日期提示。

其他资源

- dmenu - dmenu官方网站

- Yeganesh - dmenu的一个前段处理器，可以按照使用频率进行排序
- LinuxTOY - dmenu运行后的效果图

14.3 St

难度 进阶 | **篇幅** 7.6 KB | **所属** Suckless

本节将讲到

- 安装
- 配置
- 故障排除
- 请见

st 是一个简单的由 suckless 制作的用于 Xorg 上的终端实现。它被用作 xterm 或 urxvt 的轻量替代。它当前支持 256 颜色,真颜色,大多数 VT10X 转义序列,UTF-8,X11 复制/粘贴,抗锯齿字体,备用字体,调整大小,快捷键和线描。

安装

安装 stAUR 软件包 或 开发版本的 st-gitAUR 软件包。

- 在 Wayland 上, *st* 使用 Xwayland;如果你想要一个类似的终端来避免 Xwayland 的内存占用,请考虑使用 wterm-gitAUR。

配置

st 是通过它在编译时被复制的 `config.h` 文件来配置的。默认配置保存在源代码中的 `config.def.h`。考虑制作你自己的 `config.h` 和 PKGBUILD。

Shell

要想修改 *st* 的默认 shell ,编辑这一行:

```
static char *shell = "/bin/sh";
```

或将你想要的 shell 作为最后一个参数启动 *st* :

```
$ st options fish
```

终端

要想修改终端的类型, 编辑这一行:

```
static char *termname = "st-256color";
```

st 会将 `TERM` 变量的值设置为 `termname`。

字体

根据你的喜好编辑下面这一行:

```
static char *font = "Liberation Mono:pixelsize=12:antialias=false:autohint=false";
```

你也可以在命令行中传递使用的字体:

```
$ st -f "Liberation Mono:size=12"
$ st -f 'Liberation Mono-12'
```

字体的名字可以用 `fc-list` 命令显示。

光标

默认的光标指针为难以找到的 `XC_xterm`。要想改为你的光标主题的普通光标,编辑下面这一行:

```
static unsigned int mouseshape = XC_left_ptr;
```

颜色

编辑下列行来设置前景,背景和光标颜色:

```
unsigned int defaultfg = 7;
unsigned int defaultbg = 0;
static unsigned int defaultcs = 256;
```

这些值引用配置文件中的 `*colorname[]` 数组。你可以使用默认的颜色或在 `#rrggbb` 中添加你自己的颜色:

```
static const char *colorname[] = {
       /* 8 normal colors */
       "black",
       "red3",
       "green3",
       "yellow3",
       "blue2",
       "magenta3",
       "cyan3",
       "gray90",

       /* 8 bright colors */
       "gray50",
       "red",
       "green",
       "yellow",
       "#5c5cff",
       "magenta",
       "cyan",
       "white",

        = 0,

       /* more colors can be added after 255 to use with DefaultXX */
       "#cccccc",
       "#eeeeee",
       "#111111",
 };

/*
 * Default colors (colorname index)
 * foreground, background, cursor
 */
unsigned int defaultfg = 257;
unsigned int defaultbg = 258;
static unsigned int defaultcs = 256;
```

有便于创建调色板的工具。举个例子 terminal.sexy 有一套预制的颜色并且可以直接导出为 *st* 的格式(请见 issue 22 的评论)。

有 Solarized 颜色主题的补丁。请见 https://st.suckless.org/patches/solarized/ 来安装。

补丁

suckless 网站有许多补丁。要想使用补丁,下载 diff 并且使用 `patch -i patch.diff` 命令来安装补丁。这会修改默认的配置文件 `config.def.h`;如果你正在制作你自己的 `config.h`,将你的配置从 `config.h` 复制到一份 `config.def.h` 的副本中并且将它重命名为 `config.h`,然后执行 `make clean install` 命令。

Desktop 条目

要想用在桌面环境优美的字体(例如 adobe-source-code-pro-fonts包)简化 *st*的启动,你可以创建一个 Desktop 条目:

```
~/.local/share/applications/st.desktop
[Desktop Entry]
Name=Simple Terminal
GenericName=Terminal
Comment=Suckless terminal emulator for X
Exec=st -t "Simple Terminal" -f "Source Code Pro:style=Semibold:size=12"
Terminal=false
Type=Application
Encoding=UTF-8
Icon=utilities-terminal
Categories=System;TerminalEmulator;
Keywords=shell;prompt;command;commandline;cmd;
```

菜单条目会在*系统工具*应用列表中以*Simple Terminal*出现。

故障排除

键盘

将下列内容添加到 `~/.inputrc` 或 `/etc/inputrc` 如果 {{ic}Delete}} 不能在某些应用里正常工作。

```
set enable-keypad on
```

如果以上操作后还不能在某些使用 bash 的应用,例如 IPython ,中正常工作,相反,关闭 `enable-keypad` 并且将下列内容加入到你的 `~/.bashrc`,就如在st FAQ提到的做:

```
printf '\033[?1h\033=' >/dev/tty
```

Vim

vim 中非字母的文字的背景颜色不会被填充

试着在你的 `config.h` 设置 `termname` 的值为 `st-256color` 然后重新编译。并且不要在你的 shell 中设置 `TERM` 变量,至少不要设置成 `st-256color` ,因为似乎会造成问题。

另一个也许更好的解决方法,是在 `.vimrc` 文件中加入下列内容:

```
if &term =~ '256color'
```

```
    " disable Background Color Erase (BCE) so that color schemes
    " render properly when inside 256-color tmux and GNU screen.
    " see also https://sunaku.github.io/vim-256color-bce.html
    set t_ut=
endif
```

256颜色和真颜色在 tmux 或其他终端复用器中不工作

首先,确保你在你的 `~/.bashrc` 没有设置且没有导出 `TERM` 的值,正如在这篇帖子中提到的。

注意：请不要显式设置 `TERM` 变量。通过在你的 `tmux.conf` 设置 `default-terminal` 来正确地设置。

接着,确保你使用的 `vim` 版本为 **`>=7.4.1799`**, 该版本加入了 `termguicolors`。

最后,在 `~/.vimrc` 中添加下列内容:

```
set t_8f=^[[38;2;%lu;%lu;%lum         " set foreground color
set t_8b=^[[48;2;%lu;%lu;%lum         " set background color
colorscheme Tomorrow-Night-Eighties
set t_Co=256                          " Enable 256 colors
set termguicolors                     " Enable GUI colors for the terminal to get
truecolor
```

注意：`^[` 是 (`Esc`) 的转义字符,它为每个 `t_8f` 和 `t_8b` 的值添加前缀。它是一个单独的字符,可以在 `vim` 中打出来。在 **INSERT** 模式中,按下 `Ctrl+v` 然后按 `Esc`。你还会保持在 **INSERT** 模式;再次按下 `Esc` 来回到 **NORMAL** 模式。

提示：建议在设置 `colorscheme`, `t_Co` 和 `termguicolors` 之前设置 `t_8f` 和 `t_8b`。

要想获得更多信息,请见 `vim` 中的 `:help: xterm-true-color`, `t_8f`, `t_8b`。

请见

- 主页
- 常见问题
- 官方 git 仓库

Wayland

图形会话最终要落到协议栈上。Wayland 一节篇幅最长，从系统需求、混成器、显示管理器讲到 Xwayland、图形库与输入法，讲的是协议怎么支撑起整个图形会话，应当先读，读到能分清混成器与显示管理器的分工为止。Kanshi 与通用Wayland会话管理器各只几页，前者管多显示器布局的声明式配置，后者管会话启动与进程管理，放在后面当速查即可；时间紧时只扫安装与配置，疑难解答留到动手时翻。最容易踩的坑是把混成器当显示管理器来配，或 Xwayland 缺位时先怀疑驱动——多半是配置层级错位。

15.1　Kanshi

难度 入门 | **篇幅** 4.9 KB | **所属** Wayland

本节将讲到

- 安装
- 配置
- 使用
- 提示和技巧
- 疑难解答

kanshi 可以在热插拔时自动启用和禁用显示输出的配置文件。例如，当有扩展屏幕时，kanshi 可以自动关闭笔记本内置屏幕。

这是 autorandr 等工具在 Wayland 上的等效替代品。kanshi 可在支持 wlr-output-management 协议的 Wayland 混成器上使用

安装

安装正式版本 kanshi包 或开发版本 kanshi-gitAUR。

配置

基础配置

创建 kanshi 配置文件：

```
~/.config/kanshi/config
profile {
        output LVDS-1 disable
        output "Some Company ASDF 4242" mode 1600x900 position 0,0
}

profile {
        output LVDS-1 enable scale 2
}
```

每个配置（profile）都由括号分隔，其包含几个输出指令（语法类似 sway-output(5)）。当一个配置所列的所有输出（output）都已连接时，该配置就会启用。

提示：可以使用 `swaymsg -t get_outputs` 命令获取输出名称列表和其它信息。

进阶配置

为了易于管理多种配置（*如“仅笔记本内置屏幕”、“在家连接一个扩展屏”、“工作时连接两个扩展屏”等配置*），您可定义配置里使用的输出的默认行为并为其分配别名。默认行为会应用到所有提到该输出的配置中。更多信息参见 kanshi(5)。

```
~/.config/kanshi/config
output "Dell Inc. DELL S2721DGF G52TR83" {
  mode 2560x1440@165.08
  position 1280,0
  scale 2
  alias $HOME_1
}

output "LG Display 0x058B Unknown" {
  mode 2560x1440@59.99800
  position 0,0
  scale 2
  alias $INTERNAL
}

profile home_1 {
  output $INTERNAL disable
  output $HOME_1 enable
}
```

注意 *$INTERNAL* 输出的名称的第三段是 *UNKNOWN*。这是因为 kanshi 需要三个字段（制造商，型号，序列号）都被填充，如果缺少一个则需要使用 *UNKNOWN* 代替。

您可使用 hyprctl monitors 等命令查询这些字段，其会输出这三个字段：

```
hyprctl monitors
make: LG Display
model: 0x058B
serial:
```

您还可以让您的工作流与 kanshi 进一步集成，使用 *exec* 以在配置启用时执行命令：

```
~/.config/kanshi/config
profile home_1 {
  output $INTERNAL disable
  output $HOME_1 enable
  exec uwsm app -- $HOME/.config/hypr/scripts/move-workspaces.sh "LG Display 0x058B"
}
```

使用

执行命令：

```
$ kanshi
```

自动用法参见#使用 systemd 管理 kanshi。

提示和技巧

使用 systemd 管理 kanshi

按照 Sway#使用 systemd 管理仅用于 Sway 的守护程序创建并启动 `~/.config/systemd/user/sway-session.target`（如果尚未创建）。

创建 `kanshi.service` 文件：

```
~/.config/systemd/user/kanshi.service
[Unit]
Description=Dynamic output configuration for Wayland compositors
Documentation=https://gitlab.freedesktop.org/emersion/kanshi
BindsTo=sway-session.target

[Service]
Type=simple
ExecStart=/usr/bin/kanshi

[Install]
WantedBy=sway-session.target
```

启用该用户单元。其仅会在 Sway 运行时激活并在 Sway 停止时停用。

疑难解答

唤醒后外置屏幕工作区编号从 2 开始

若您参照上文配置，当第二块屏幕连接到计算机时，第一块屏幕被禁用，第二块屏幕从 1 开始编号工作区，您首次接入屏幕或开机后应当是这种状况。但从睡眠唤醒后，这个数字可能会变成 2。欲解决，只需将第一个工作区移动到第二块屏幕，向 kanshi 配置文件中添加配置命令：

```
~/.config/kanshi/config
profile {
        output LVDS-1 disable
        output "Some Company ASDF 4242" mode 1600x900 position 0,0
        # 以下这条就是添加的命令（别忘了更新输出名称）：
        exec swaymsg workspace 1, move workspace to HDMI-A-1
}

profile {
        output LVDS-1 enable scale 2
}
```

另请参阅

- 源码和文档

15.2 通用Wayland会话管理器

难度 进阶 | **篇幅** 6.7 KB | **所属** Wayland

本节将讲到

- 安装
- 配置

- 使用
- 技巧提示
- 另见

通用Wayland会话管理器 (**uwsm**) 运行时会将独立 Wayland混成器变成一组systemd单元。这提供一个包括环境的稳定的会话管理，并且支持XDG Autostart，与登录会话双向连接，并且可以完全关闭。

注意：强烈建议使用dbus-broker作为D-Bus守护进程的实现。因为它会重复利用systemd激活环境，而不是启动一个独立的环境。这个简化的环境管理可以有恰当的清理，也支持reference implementation，但是不允许取消设置一个变量，所以更高效的清理方式是将其设置为空字符。唯一可能的清理办法是使用 `loginctl terminate-user ""`来将D-Bus守护进程引用的环境分离

安装

安装 uwsm包（或者你也可以安装uwsm-gitAUR的Git版本以获取`master`分支上的最新提交）

配置

服务启动通知和混成器设置的变量

注意：如果被管理的混成器已经将 `WAYLAND_DISPLAY`（和其他有用的环境变量）添加到 systemd 激活的环境中，那么你可以跳过这部分，并且无需使用 `uwsm finalize`。

为了找到当前的混成器，一个Wayland软件作为systemd服务运行需要 `WAYLAND_DISPLAY` 环境变量（如果它们预期是在Xwayland中运行则是 `DISPLAY`）。因此，一旦混成器设置了这个和其他有用的环境变量，都应该被放进 systemd/dbus 激活环境中。

`uwsm finalize` 命令会根据 `UWSM_FINALIZE_VARNAMES` 列表（使用空白字符 (white-space) 作为分隔符），将 `WAYLAND_DISPLAY`、`DISPLAY` 和其他环境变量加入到激活环境。建议在混成器就绪后执行该命令。

如果在激活环境中需要其他由混成器设置的变量，可以将变量名称作为参数传递给 `uwsm finalize` 或将其放入 `UWSM_FINALIZE_VARNAMES` 中的使用空白字符分隔的列表中。例如：

```
exec uwsm finalize VAR1 VAR2 ...
export UWSM_FINALIZE_VARNAMES=VAR1 VAR2 ...
```

环境变量

所有来源于*uwsm*的环境变量都设置在`${XDG_CONFIG_HOME}/uwsm/env`并且可用于在这个会话中所有的被管理混成器和图形化软件。

如果你想仅仅为一个指定的*混成器*（以及对应图形会话中的图形应用程序）设置一些环境变量，那么请将变量放在`${XDG_CONFIG_HOME}/uwsm/env-*compositor*`

这个文件的模板如下：

```
~/.config/uwsm/env
export KEY1=VAR1
export KEY2=VAR2
export KEY3=VAR3
...
```

使用

启动

注意：如果在`uwsm start`中的环境变量被成功使用，那么环境预加载器不再生效POSIX shell配置。

uwsm 可以从TTY或者显示管理器启动。

从TTY启动

将如下代码添加进`~/.profile`：

```
if uwsm check may-start && uwsm select; then
  exec uwsm start default
fi
```

如果你想始终启动指定*混成器*，将上述代码替换为：

```
if uwsm check may-start; then
  exec uwsm start compositor.desktop
fi
```

从显示管理器中启动

你可以创建一个自定义桌面项以通过*uwsm*启动你的混成器：

```
/usr/share/wayland-sessions/my-compositor-uwsm.desktop
[Desktop Entry]
Name=My compositor (with UWSM)
Comment=My cool compositor, UWSM session

# either full command line with metadata and executable
Exec=uwsm start -N "My compositor" -D mycompositor:mylib -C "My cool compositor" -- my-compositor

# or a reference to another entry
Exec=uwsm start -- my-compositor.desktop

DesktopNames=mycompositor;mylib
Type=Application
```

终止会话

如果你想终止当前*uwsm*会话，那么你可以使用`loginctl terminate-user ""`（关闭整个用户会话）或`uwsm stop`（如果uwsm取代了登录shell，那么它会在`uwsm start`后启动代码或终止用户会话）

注意：请不要使用混成器的原生退出机制或者直接杀死其进程。这将会使混成器从所有客户端之下强制抽除，并干扰有序的单元停用流程。

技巧提示

软件与自动启动

默认情况下，uwsm 通过在`session.slice`中的一个自定义systemd服务来启动混成器。很多Wayland混成器允许你在混成器服务中启动其他软件，然而这将会消耗混成器资源，甚至会干扰通知sockets

要将应用程序作为独立的 systemd 范围单元 (scope units) 启动，你可以使用 `uwsm app`，它既可以启动可执行文件：

```
uwsm app -- /my/program/path
```

也可以启动桌面项：

```
uwsm app -- myprogram.desktop
```

uwsm 默认会将启动的范围单元放置在 `app-graphical.slice`。如果你想要将它们放置在 `background-graphical.slice` 或 `session-graphical.slice`，应该分别使用 `-s b` 和 `-s s`：

```
uwsm app -s b -- background-app.desktop
```

替代

你可以使用以下方案作为替代，比起`uwsm app`（运行一个Python脚本）会更快：

- uwsm 的 `uwsm-app` 脚本，与 uwsm 的应用守护进程进行通信。
- app2unit-gitAUR，一个shell脚本。你可以通过设置`APP2UNIT_SLICES`环境变量为如下值来让它作为`uwsm app`的替代方案：

```
    APP2UNIT_SLICES='a=app-graphical.slice b=background-graphical.slice s=session-graphical.slice'
```

- runappAUR，由C++编写，但是少了些许功能。

另见

- uwsm(1)

15.3 Wayland

难度 高级 | **篇幅** 38.2 KB | **所属** Wayland

本节将讲到

- 系统需求
- 混成器
- 显示管理器
- Xwayland
- 图形库

Wayland 是一个显示服务器协议，已普遍作为 X 窗口系统的继任者 。可参见 Wikipedia 上的 Wayland 与 Xorg 的对比#Differences_between_Wayland_and_X)。

由于使用 Wayland 协议的显示服务器也可作为混成窗口管理器，因此被称为**混成器**（compositor）。你可以在#混成器中找到相关信息。

为了兼容原生 X11 应用程序，使其在 Wayland 中无缝运行，可以使用 #Xwayland，它在 Wayland 中提供了 X 服务器。

系统需求

和 Xorg 不同的是，Wayland 本身只是一个协议，没有一个通用的“显示服务器”可供安装。需要安装的是兼容的 GPU 驱动（本节）和混成器（下一节）或将混成器内置其中的桌面环境（例如 GNOME 或 Plasma）。大多数 Wayland 混成器只能在使用内核级显示模式设置的系统上工作。

GPU 驱动与 Wayland 混成器必须支持相同的缓冲区 API (Buffer API) 才能够互相兼容。现在主要有两种 API：GBM 和 EGLStreams。

缓冲区 API	GPU 驱动支持	Wayland 混成器支持
GBM	除版本号低于 495 的 NVIDIA 驱动，均支持	全部
EGLStreams	NVIDIA	GNOME

: * 版本 ≥ 495 的 NVIDIA 驱动同时支持 EGLStreams 和 GBM。

自 NVIDIA 在版本 495 引入 GBM 支持以来，许多混成器 (包括 Mutter 和 KWin) 都开始默认使用 GBM 。通常认为 GBM 更好，有更为广泛的支持，而以前仅支持 EGLStreams 是因为之前无法在 Wayland 下通过专有驱动程序来使用 NVIDIA GPU。此外，在 NVIDIA 支持 GBM 后，KWin 放弃了对 EGLStreams 的支持

如果您使用的是流行的桌面环境/混成器，GPU 也受 NVIDIA 支持，那么很可能已经在使用 GBM 后端了。要检查是否使用 GBM 后端，请执行 `journalctl -b 0 --grep "renderer for"`。要强制使用 GBM 后端，请设置以下环境变量：

```
GBM_BACKEND=nvidia-drm
__GLX_VENDOR_LIBRARY_NAME=nvidia
```

混成器

堆叠式、**平铺式**和**动态**的区别参见窗口管理器#类型。

堆叠式

- **hikari** — 基于 wlroots 并受 cwm 启发开发，在 FreeBSD 上开发很活跃，但也支持 Linux。

: https://hikari.acmelabs.space/ || hikariAUR

- **KDE KWin** — 请参阅 KDE#启动 Plasma。

: https://userbase.kde.org/KWin || kwin包

- **labwc（英语：labwc）** — 基于 wlroots 并受 Openbox 启发的混成器。

: https://github.com/labwc/labwc || labwc包

- **Mutter)** — 请参阅 GNOME#运行 GNOME。

: https://gitlab.gnome.org/GNOME/mutter || mutter包

- **waybox** — 一个 *box 风格（极简主义）的 Wayland 混成器，主要基于 Openbox 设计。

: https://github.com/wizbright/waybox || wayboxAUR

- **wayfire** — 3D 混成器，受 Compiz 启发并基于 wlroots 开发。

: https://wayfire.org/ || wayfireAUR

- **Weston** — Wayland 混成器，设计注重正确性、可靠性、可预测性和性能。

: https://gitlab.freedesktop.org/wayland/weston || weston包

- **wio** — 基于 wlroots 的混成器，致力于复刻 Plan 9 的 Rio 桌面的外观和质感。

: https://gitlab.com/Rubo/wio || wio-wlAUR

- **wlmaker** — 基于 wlroots 的混成器，受 Window Maker（英语：Window Maker） 启发。

: https://phkaeser.github.io/wlmaker/ || wlmakerAUR

- **woodland** — 一个极简轻量级的基于 wlroots 的堆叠式混成器，适用于 Wayland，受 wayfire 和 TinyWl 启发。

: https://github.com/DiogenesN/woodland || woodlandAUR

平铺式

- **Cagebreak（英语：Cagebreak）** — 基于 Cage，受 Ratpoison 启发。

: https://github.com/project-repo/cagebreak || cagebreakAUR

- **jay** — 一个使用 Rust 编写的 Wayland 混成器，具有类似于 i3 的默认外观。Jay 可以使用一个声明式的 TOML 文件或一个注入到混成器中的共享数据库配置。

: https://github.com/mahkoh/jay || jayAUR

- **IonWL** — 手动平铺混成器。使用全面的 Python API 开发，遵循 Ion3 的设计。

: https://codeberg.org/ideasman42/IonWL || ionwl-gitAUR

- **miracle-wm** — 基于 Mir 的 Wayland 混成器，风格类似 i3 和 sway，旨在比两者更炫目且功能更丰富，如 swayfx。

: https://github.com/miracle-wm-org/miracle-wm || miracle-wmAUR

- **niri** — 可滚动的平铺式 Wayland 混成器。

: https://github.com/YaLTeR/niri/ || niri包

- **Qtile** — 功能齐全、可定制性强的平铺式 Xorg 窗口管理器和 Wayland 混成器，使用 Python 开发、配置。

: https://github.com/qtile/qtile || qtile包

- **Sway** — 基于 wlroots、与 i3 兼容的 Wayland 混成器。

: https://github.com/swaywm/sway || sway包

- **SwayFx** — Sway，但带有视觉效果！

: https://github.com/WillPower3309/swayfx || swayfxAUR

- **Velox** — 基于 swc 的简单窗口管理器，受 dwm 和 xmonad 启发。

: https://github.com/michaelforney/velox || velox-gitAUR

动态

- **cosmic-comp** — COSMIC 桌面环境的混成器。

: https://github.com/pop-os/cosmic-comp || cosmic-comp包

- **cwc（英语：cwc）** — 基于 wlroots、类似awesome的 Wayland 混成器

: https://cudiph.github.io/cwc/apidoc/ || cwcAUR

- **dwl** — 基于 wlroots、类似 dwm 的 Wayland 混成器。

: https://codeberg.org/dwl/dwl || dwlAUR

- **Hyprland** — 不以牺牲外观为代价的动态平铺式 Wayland 混成器。

: https://hypr.land || hyprland包

- **japokwm** — 基于 wlroots 创建布局的动态平铺式 Wayland 混成器。

: https://github.com/werererer/japokwm || japokwm-gitAUR

- **Mangowc（英语：Mangowc）** — 基于 dwl 的混成器，提供标准配置文件、可选的滚动布局以及视觉特效支持。

: https://github.com/DreamMaoMao/mangowc || mangowcAUR

- **River Classic（英语：River Classic）** — 受 dwm 和 bspwm 启发的动态平铺式 Wayland 混成器。

: https://codeberg.org/river/river-classic || river-classic包

- **pinnacle-comp** — A Smithay-based Wayland compositor, inspired by AwesomeWM and configured in Lua or Rust

: https://github.com/pinnacle-comp/pinnacle || pinnacle-compAUR

其它

- **Cage** — 像自助终端那样，全屏显示单个应用程序。

: https://www.hjdskes.nl/projects/cage/ || cage包

- **GNOME Kiosk** — 基于 Mutter 的混成器，适用于固定用途或单应用部署，如墙面显示器和销售终端（kiosk）。

: https://gitlab.gnome.org/GNOME/gnome-kiosk || gnome-kioskAUR

- **phoc** — 适用于移动设备的基于 wlroots 的轻量混成器。

: https://gitlab.gnome.org/World/Phosh/phoc || phoc包

- **Wayback** — X11 兼容层，允许使用 Wayland 组件运行完整的 X11 桌面环境。实验性，开发处于早期阶段。

: https://wayback.freedesktop.org/ || wayback-x11AUR

- **River（英语：River）** — 一个非整体的 Wayland 混成器。和其他的 Wayland 混成器不同，它不把混成器和窗口管理器结合在一个程序中。相反，用户可以选择使用任何遵循 river-window-management-v1 协议的窗口管理器。

: https://isaacfreund.com/software/river/ || river包

上述某些混成器支持从显示管理器启动。请查看 `/usr/share/wayland-sessions/*混成器*.desktop` 文件来了解如何启动显示上述混成器。

显示管理器

下面列出的显示管理器均支持启动 Wayland 混成器。

名称	运行于	描述
atriumAUR	Wayland	具有第一类多 seat 支持的显示管理器。
emptty包	tty	简单的 TTY CLI 显示管理器。
plasma-login-manager-gitAUR	Wayland	KDE目前正在活跃开发的显示管理器。
GDM	Wayland	GNOME 的显示管理器。
greetd	Wayland/Xorg/tty 参考 Greetd#Greeters。	

Xwayland

Xwayland(1) 是一个在 Wayland 下运行的 X 服务器，以兼容尚未支持 Wayland 的原生 X11 应用程序。要使用它，请安装 xorg-xwayland包。

Xwayland 通过混成器启动，因此请参见所使用混成器的文档了解其兼容性和启动方式。

默认情况下，Xwayland 使用 rootless 模式运行 X 服务器。

注意：

- 安全性：Xwayland 是一个 X 服务器，因此不具备 Wayland 的安全特性。
- 性能：大部分情况下，Xwayland 的性能与 X11 几乎相同。
- 兼容性：Xwayland 并不完全向后兼容 X11。某些应用程序在 Xwayland 下可能无法正常工作。

Wayback

Wayback（wayback-x11AUR、wayback-x11-gitAUR）是一个 X11 兼容层，允许使用 Wayland 组件运行完整的 X11 桌面环境，旨在最终取代 Xorg，从而减轻 X11 应用程序的维护负担。

NVIDIA 驱动

注意：470 版本之前的 NVIDIA 驱动（例如 nvidia-390xx-dkmsAUR）不支持硬件加速的 Xwayland，导致非 Wayland 原生应用程序在 Wayland 会话中性能不佳。

需要启用 DRM KMS。有关您的显示管理器（例如 GDM）的更多信息，请参阅官方文档。

检测应用程序是否通过 Xwayland 运行

使用 xorg-xeyes包 的 `xeyes` 可以确定应用程序是否通过 Xwayland 运行，如果鼠标指针在 Xwayland 程序上，xeyes 窗口中的眼珠会转向鼠标指针。

或使用 xorg-xwininfo包 的 `xwininfo`。如果鼠标指针在 Xwayland 程序上，鼠标指针会变成“+”。此时点击窗口，它会输出窗口信息并终止。

或使用 xorg-xlsclients包 的 `xlsclients`。运行 `xlsclients -l` 可列出所有正运行在 Xwayland 的应用程序。

或者也可以使用 extramausAUR。如果鼠标指针在 Xwayland 程序上，红色指针会随之移动。

提示：对于 KDE Plasma，还可以使用 KWin 调试控制台检查窗口。

图形库

GTK

gtk3包 和 gtk4包 包已经提供了 Wayland 支持。GTK 默认使用 Wayland 后端，但是可以通过修改环境变量为 `GDK_BACKEND=x11` 覆盖默认设置切换到 Xwayland。

关于主题问题，请参阅 GTK#Wayland_后端。

Qt

要在 Qt 5 中启用 Wayland 支持，请安装 qt5-wayland包。Qt 5 应用程序就会在 Wayland 会话下运行。

虽然通常不需要，但若要显式使用 Wayland 插件运行 Qt 应用程序 ，请使用 `-platform wayland` 或 `QT_QPA_PLATFORM=wayland` 环境变量。

要在 Wayland 会话中强制使用 X11，请使用 `QT_QPA_PLATFORM=xcb`。

这对于某些不使用系统 Qt 实现的应用可能是必需的。`QT_QPA_PLATFORM="wayland;xcb"` 允许 Qt 在 Wayland 不可用时使用 xcb (X11) 插件 。

在某些混成器，例如sway，Qt 应用原生运行可能会有功能缺失。例如， KeepassXC 将无法最小化到托盘。在运行前安装qt5ct并设置`QT_QPA_PLATFORMTHEME=qt5ct` 即可解决此问题。

由于 Qt WebEngine 在 Wayland 上使用分数缩放时出现错误尺寸和文本渲染问题的 Qt WebEngine 错误，使用 Qt WebEngine 的应用程序（例如 Calibre）可能会显示锯

齿字体。一个解决方法是启动应用程序时设置 `QT_SCALE_FACTOR_ROUNDING_POLICY=RoundPreferFloor`，这可以防止应用程序窗口进行分数缩放。

Clutter

Clutter 工具包有 Wayland 后端支持，可以作为 Wayland 客户端运行。此后端支持已在 clutterAUR 中启用。

要在 Wayland 上运行 Clutter 应用，请设置 `CLUTTER_BACKEND=wayland` 环境变量。

SDL

在 SDL3 中，如果混成器支持 fifo-v1 协议，则默认使用 Wayland 。否则，会先尝试 X11。可以将 `SDL_VIDEO_DRIVER` 环境变量设置为 `x11` 或 `wayland` 来强制使用其一（或者，使用优先级较低的下述 SDL2 环境变量）。

sdl2-compat包 遵循上述 SDL3 规则，但有特定应用程序的例外。至于 SDL2 本身（例如 sdl2AUR），请设置 `SDL_VIDEODRIVER=wayland`。`SDL_VIDEODRIVER="wayland,x11"` 允许 SDL2 在 Wayland 不可用时使用 X11 。可能还需要安装 libdecor包 才能启用窗口装饰（例如在 GNOME 上）。

参见官方说明。

GLFW

glfw包 支持 Wayland，如果环境变量 `XDG_SESSION_TYPE` 设置为 `wayland`，并且应用程序开发人员没有设置特定的后端，则使用 Wayland 后端。

查看源代码以获取更多信息。

GLEW

如果 glew-wayland-gitAUR 包无法与所需的 GLEW 应用程序一起使用，则可以使用 glew包 和 Xwayland。参见 FS#62713。

EFL

Enlightenment 已完全支持 Wayland。

要在 Wayland 上运行基于 EFL 的应用程序，请设置 `ELM_DISPLAY=wl`。

winit

winit 是 Rust 语言中的窗口处理库。默认使用 Wayland 后端，但可以通过修改环境变量将其切换到 Xwayland：

- 在 0.29.2 版本之前，设置 `WINIT_UNIX_BACKEND=x11`
- 对于 0.29.2 及更高版本，取消设置 `WAYLAND_DISPLAY`，这将强制回退到使用 `DISPLAY` 变量的 X11

Electron

注意：在 Plasma 中，某些 Electron 应用程序可能在窗口上使用错误的图标（默认的 Wayland 图标），而在任务栏上使用正确的图标。使用特殊应用程序/窗口规则强制指定“桌面文件名”（即 `.desktop` 文件的文件名）可以绕过此问题。

使用命令行标志或环境变量可启用 Electron 的 Wayland 支持。

命令行标志

注意：某些应用程序不会将标志转发给 Electron，需要应用程序开发者提供解决方案。

关于启用 Electron Wayland 支持的命令行标志，请参见 Chromium#原生 Wayland 上运行。但需要注意，命令行标志 `--ozone-platform-hint=auto` 自 Electron 38 起已不再有效。

这些标志可以手动传递或#通过配置文件添加。通过编辑 .desktop 文件可以每次通过桌面项启动时传递。

**本文或本章节的事实准确性存在争议。**

原因：旧版本的 Electron 需要 `--enable-features=WebRTCPipeWireCapturer`，但它从哪个版本开始默认启用？此外，非自由软件捆绑的 Electron 的默认行为可能是错误的。（在 en:Talk:Wayland 中讨论）

Electron 默认启用通过 PipeWire 进行的 WebRTC 屏幕捕获。该捕获基于 xdg-desktop-portal包。

顶部栏缺失的问题可以通过使用 `--enable-features=WaylandWindowDecorations` 来解决。这在 GNOME 下通常是必需的（自 electron17 起支持）。

通过配置文件添加

Electron 读取 `~/.config/electron*XX*-flags.conf`（*XX* 为 Electron 版本号）文件添加到命令行参数（一行一个），如果版本特定的文件不存在，则回退到全版本的 `~/.config/electron-flags.conf`。

例如，将上述参数写入共享配置文件：

```
~/.config/electron-flags.conf
--enable-features=WaylandWindowDecorations
--ozone-platform-hint=auto
```

注意：这些配置文件仅对官方仓库的 Electron 软件包和使用这些 Electron 包的应用程序有效。对于自带 Electron 的软件包（如 slack-desktopAUR）来说，这些配置文件是无效的。不过在某些情况下也有替代品，如 slack-electronAUR。

环境变量

使用 Electron 28 至 37 版本的应用程序可以使用环境变量 `ELECTRON_OZONE_PLATFORM_HINT`，将其设置为 `auto` 或 `wayland`。

如果同时指定，命令行标志优先。

Java

Java 平台的开源实现 OpenJDK 尚未原生支持 Wayland。在 Wakefield 项目（旨在在 OpenJDK 中实现 Wayland）可用之前，可以使用 Xwayland。

参见 Debian:Wayland#Java Programs (supported since OpenJDK 16?)：

: 从 OpenJDK 16 开始，JRE 可以动态加载 GTK3（支持 Wayland），根据此讨论，这似乎是支持的。 : 可以设置 `_JAVA_AWT_WM_NONREPARENTING` 环境变量为 "1" 以修复应用程序启动时出现空白屏幕的问题。

由于 Xwayland 没有与 Wayland 完全的功能对等，WLToolkit 可以在 Wakefield 未准备好时填补空白。可以通过 `-Dawt.toolkit.name=WLToolkit` 激活。一些程序支持它。

输入法

Fcitx 5

参见 Fcitx 5#Wayland。

IBus

参见 。

提示与技巧

自动化

- ydotool (ydotool包) - 通用的命令行自动化工具 (不限于 Wayland)。启用/启动 `ydotool.service` 用户单元。参见 ydotoold(8)，ydotool(1)。
- wtype (wtype包) - 适用 Wayland 的 xdotool type。参见 wtype(1)。
- keyboard - 可在 Windows 和 Linux 上稳定工作、实验性支持 OS X 的 Python 库。另请参见鼠标 (mouse) 库。
- wlrctl (wlrctlAUR) - 用于各种 wlroots 扩展的命令行工具（支持 foreign-toplevel-management、virtual-keyboard、virtual-pointer）。

重新映射键盘或鼠标按键

参见输入重映射工具。

录制

参见屏幕捕获#屏幕录制和屏幕捕获#通过X11应用程序录制Wayland窗口。

应用程序关闭后保留剪贴板内容

**本文或本章节可能需要合并到剪贴板。**

附注： 这是 Xorg 上的标准行为。还有许多其他剪贴板管理器。（在 Talk:Wayland 中讨论）

由于 Wayland 的设计理念，剪贴板数据存储在源客户端的内存中。当客户端关闭时，剪贴板数据将丢失。您可以使用 wl-clip-persist包 来解决这个问题，它在后台运行以读取剪贴板数据并将其存储在自己的内存中，与源客户端分离。

将混成器作为 systemd 服务自动启动

提示：通用Wayland会话管理器可自动为混成器生成 systemd 单元，此外还可用于将图形应用程序与 systemd 集成。

如果不想使用显示管理器或 shell，可以通过 systemd 自动启动 Wayland 混成器。调整 `ExecStart` 行启动指定混成器。以下是 KDE Plasma 的示例:

```
/etc/systemd/system/wayland-compositor.service
[Unit]
After=graphical.target systemd-user-sessions.service modprobe@drm.service
Conflicts=getty@tty1.service
[Service]
User=username
WorkingDirectory=~
PAMName=login
TTYPath=/dev/tty1
UnsetEnvironment=TERM
StandardOutput=journal
ExecStart=/usr/lib/plasma-dbus-run-session-if-needed /usr/bin/startplasma-wayland
[Install]
WantedBy=graphical.target
```

在基于 wlroots 的混成器上使用另一个渲染器

您可以通过为基于 wlroots 的混成器设定 `WLR_RENDERER` 环境变量以使用另一个 wlroots 渲染器如 vulkan。请参阅 wlroots 文档以查找可用的渲染器。

故障排除

颜色校正

请参阅背光#色彩校正。

慢动作、图形显示故障和崩溃

Gnome-shell 用户从 X 切换到 Wayland 时可能会遇到此问题。根本原因之一是可能用户自己为基于 Xorg 的 gnome-shell 设置了 `CLUTTER_PAINT=disable-clipped-redraws:disable-culling` 变量。只需尝试将变量从 `/etc/environment` 或其它 rc 文件中移除，即可查看是否一切恢复正常。

远程显示

- sway 使用的wlroots0.18包 和wlroots0.19包 从 0.10 版本开始通过 wayvnc包 提供了一个 VNC 后端。RDP 后端则已被移除。
- 目前 mutter包 在编译时就启用了远程桌面功能，详情请参阅 和 gnome-remote-desktop包。
- krfb包 为 kwin包 提供了一个 VNC 服务器。`krfb-virtualmonitor` 可用于将另一台设备设置为额外的显示器。
- 在 2013 年 Weston 合并了对 FreeRDP 的支持，可通过编译标志 (compile flag) 启用。weston包 自 6.0.0 版本开始启用了 FreeRDP。
- waypipe包 (或 waypipe-gitAUR) 是适用于 Wayland 应用的透明代理，可通过封装的命令在 SSH 上运行。
- 以下是一个在 Plasma 下启动远程 KDE kcalc 的示例：

: : ``` $ waypipe ssh example.local env QT_QPA_PLATFORM=wayland QT_QPA_PLATFORMTHEME=KDE dbus-launch kcalc

游戏、远程桌面和虚拟机窗口中的输入捕获问题

与 Xorg 不同的是，Wayland 并不允许独占输入设备捕获 （也被称为主动捕获或显式捕获，比如[键盘](https://tronche.com/gui/x/xlib/input/XGrabKeyboard.html)、[鼠标](https://tronche.com/gui/x/xlib/input/XGrabPointer.html)等设备）。Wayland 依赖混成器传递键盘快捷键，并将指针设备限制在应用窗口中。

输入捕获方式的变化破坏了当前应用程序的行为，意味着：

- 热键组合和修饰符输入会被混成器捕获，并且不会发送到远程桌面和虚拟机窗口中。
- 鼠标指针将不会被限制在应用程序的窗口中，这可能会导致视差效应，即虚拟机或远程桌面的窗口内鼠标指针的位置与主机的鼠标指针发生偏差。

Wayland 通过为 Wayland 和 Xwayland 添加协议扩展来解决此问题。为此 Wayland 混成器需要添加对这些扩展的支持。如果是本地 Wayland 客户端，其使用的部件工具集 （widget toolkits，比如 GTK，Qt） 需要支持这些插件；如果没有使用部件工具集，则需要支持应用程序本身。如果是 Xorg 应用程序，则不需要改变应用程序或者部件工具集，因为 Xwayland 的支持就足够了。

[wayland-protocols](https://archlinux.org/packages/?name=wayland-protocols)包 中已经包含了这些扩展，并由 [xorg-xwayland](https://archlinux.org/packages/?name=xorg-xwayland)包 支持。

相关的扩展有：

- [Xwayland 键盘捕获协议](https://gitlab.freedesktop.org/wayland/wayland-protocols/-/blob/main/unstable/xwayland-keyboard-grab/xwayland-keyboard-grab-unstable-v1.xml)
- [混成器快捷键禁止协议](https://gitlab.freedesktop.org/wayland/wayland-protocols/-/blob/main/unstable/keyboard-shortcuts-inhibit/keyboard-shortcuts-inhibit-unstable-v1.xml)
- [相对指针协议](https://gitlab.freedesktop.org/wayland/wayland-protocols/-/blob/main/unstable/relative-pointer/relative-pointer-unstable-v1.xml)
- [指针约束协议](https://gitlab.freedesktop.org/wayland/wayland-protocols/-/blob/main/unstable/pointer-constraints/pointer-constraints-unstable-v1.xml)

支持的 Wayland 混成器有：

- Mutter, [GNOME](../GNOME/GNOME.md) 的混成器 (窗口管理器)，[从 3.28 版本](https://bugzilla.gnome.org/show_bug.cgi?id=783342)开始支持
- wlroots 支持相对指针协议和指针约束协议
- Kwin
    - [KDE#X11快捷键在Wayland上冲突](../KDE/KDE.md#X11快捷键在Wayland上冲突)
    - [键盘快捷键禁止](https://invent.kde.org/plasma/kwin/-/blob/master/src/wayland/keyboard_shortcuts_inhibit_v1_interface.cpp)

支持的部件工具集有：

- GTK, 从 3.22.18 版本开始支持。

GTK 主题没有生效

请查看 https://github.com/swaywm/sway/wiki/GTK-3-settings-on-Wayland.

避免加载 NVIDIA 模块

在 Wayland 混成器（如 [sway](../Wayland%20合成器/Sway.md)）启动之前，将 `__EGL_VENDOR_LIBRARY_FILENAMES=/usr/share/glvnd/egl_vendor.d/50_mesa.json` 添加到系统[环境变量](../../02_系统管理与内核/系统管理/环境变量.md)中。

放大/表面缩放

屏幕放大问题尚未解决，2022 年中合并了一个拉取请求[提供了 wp-surface-scale 协议](https://gitlab.freedesktop.org/wayland/wayland-protocols/-/merge_requests/145)。

Wayland 卡顿/延迟（自内核 6.11.2，AMD）

在此问题在未来的内核版本中修复之前，一个临时解决方案是在 cmdline 中添加 `amdgpu.dcdebugmask=0x400`。

参见：https://community.frame.work/t/wayland-lag-stuttering-since-kernel-6-11-2/59422

切换工作区/虚拟桌面时游戏/应用程序被挂起

当切换工作区/虚拟桌面或使用 `Alt+Tab` 时，游戏（以及可能的其他图形应用程序）会被挂起，进入某种奇怪的状态，并（部分）停止运行。这包括 VRR 应用程序和开启了 VSync 的应用程序，但可能不仅限于此。症状包括音频（部分）中断、游戏无法进行、ping 大幅上升或掉线，但仅在游戏窗口未处于焦点时才会出现。

某些游戏可能可以通过切换到窗口模式来规避此问题，但有些则不行。这在需要大量使用网页浏览、文档和第三方工具的更复杂游戏中极其烦人，或者当游戏因某种原因被中断时也是如此。

可能的解决方法包括设置环境变量 `MESA_VK_WSI_PRESENT_MODE=immediate` 和/或 `vk_xwayland_wait_ready=false`，但设置这些会破坏所有 VSync 或 VRR 实现。

另见

- [Wayland 在线文档](https://wayland.freedesktop.org/docs/html/)（英文）
- [官方仓库](https://gitlab.freedesktop.org/wayland)（英文）
- [Fedora：如何排查 Wayland 问题](https://fedoraproject.org/wiki/How_to_debug_Wayland_problems)（英文）
- [我们现在是 Wayland 了！](https://wearewaylandnow.com/)（英文）- "Wayland 可用了吗？"的更新版本
- [Wayland 项目汇总](https://awesomeopensource.com/projects/wayland)（英文）
- [光标主题](../X%20服务器/光标主题.md)
- [Arch Linux 论坛讨论](https://bbs.archlinux.org/viewtopic.php?id=107499)（英文）（页面将持续关注 Wayland 信息，如有兴趣请留意）
- [i3 迁移指南 - i3 上常用 X11 应用的 Wayland 替代品](https://github.com/swaywm/sway/wiki/i3-Migration-Guide#common-x11-apps-used-on-i3-with-wayland-alternatives)（英文）
- [Wayland Explorer - 阅读 Wayland 文档的更好方法](https://wayland.app/protocols/)（英文）
- [如何判断应用程序是否在使用 Xwayland](https://askubuntu.com/questions/1393618/how-can-i-tell-if-an-application-is-using-xwayland)（英文）

Xfce

Xfce 不是一整块，而是窗口管理器、文件管理器，以及把它们组织成会话的那一层。建议先读《Xfce》把桌面跑起来，这一节必读，配置和排错也集中在这里；再读《Thunar》，日常改动最多的就是文件管理器，自定义动作与卷管理器值得看清；《Xfwm》篇幅最短，只在调窗口行为、快捷键时翻，其余可跳。

最容易踩的坑是把「装好了」当成「配好了」：窗口行为归 Xfwm，文件管理器的右键菜单和挂载归 Thunar，两边配置不在同一处，从别处整段抄来的往往只生效一半。启动或显示不对时，先按各节末尾的故障排除顺一遍。

16.1　Xfwm

难度 入门 | **篇幅** 5.5 KB | **所属** Xfce

本节将讲到

- 安装
- 启动
- 配置
- 提示与技巧
- 故障排除

Xfwm 是用于 Xfce 桌面环境的窗口管理器。

安装

安装 xfwm4包 软件包。

启动

使用 xinit 启动 `xfwm4`。

配置

Xfwm 的大多数窗口表现和快捷键设置可以通过 `xfwm4-settings` 修改,高级设置和窗口合成可以通过 `xfwm4-tweaks-settings` 修改,工作区数量和名字可以通过 `xfwm4-workspace-settings` 修改。

合成管理器

注意：

- 这个窗口合成器可能导致应用中出现画面撕裂,请见 #画面撕裂。
- 窗口合成器自从 Xfwm 4.12 开始默认启用。

要启用或禁用 Xfwm 窗口合成器和修改它的设置,前往*窗口管理器微调*:

```
$ xfwm4-tweaks-settings
```

或者,使用带有 `--compositor` 参数的 *xfconf* 来启用它。例如:

```
~/.xinitrc
exec xfwm4 --compositor=on
$ xfconf-query -c xfwm4 -p /general/use_compositing -s true
```

窗口卷起

双击标题栏或在窗口菜单中点击*卷起窗口*会让窗口内容消失,只留下标题栏。要使用 `xfconf` 禁用这个功能,运行命令:

```
$ xfconf-query -c xfwm4 -p /general/mousewheel_rollup -s false
```

窗口平铺

Xfwm 可以在将窗口移动到屏幕边缘时会自动将其平铺。它通过调整窗口大小以适合屏幕的上半部分来实现自动平铺。要使用 `xfconf` 启用或禁用这个操作,运行命令:

```
$ xfconf-query -c xfwm4 -p /general/tile_on_move -s false
$ xfconf-query -c xfwm4 -p /general/tile_on_move -s true
```

或者,勾选(取消勾选) *Window Manager Tweaks > Accessibility > Automatically tile windows when moving toward the screen edge*。

Xfce 设置管理器提供的额外设置

安装 官方软件仓库的 xfce4-settings包。

注意：安装 xfce4-settings包 可能会修改一些任务的默认应用。请见 xdg-open 以设置你想要的默认应用。

附加主题

安装 官方软件仓库的 xfwm4-themes包。

安装的主题会在 `xfwm4-settings` 窗口中显示出来。

提示与技巧

在窗口最大化时隐藏标题栏

前往 `Accessibility` 并勾选 `Hide title of windows when maximized`。

注意：如果你想把当前最大化窗口的标题栏放在面板上,安装 xfce4-windowck-pluginAUR。

故障排除

在浏览器中下载的项目没有图标

这可以通过安装官方软件仓库的 xfce4-settings包 来修复。

工作区数量发生意外更改

请注意 *Xfwm* 给添加和移除工作区分配了快捷键。默认情况分别是 `Alt+Delete` 和 `Alt+Insert`。

如果在登录时工作区的数量会重置,请在启动 Xfwm 之后修改工作区数量。可以通过 `sleep` 命令来保证启动顺序。

```
~/.xinitrc
(sleep 3 && xfconf-query -v -c xfwm4 -p /general/workspace_count -s number) &
exec xfwm4
```

或者,通过 xfce4-session包:

```
~/.config/autostart/workspace.desktop
[Desktop Entry]
Exec=sh -c "sleep 3 && xfconf-query -v -c xfwm4 -p /general/workspace_count -s number"
```

另见: 注销会改变工作区

画面撕裂

如果有画面撕裂现象,你可以试着修改 Xfwm 的 `--vblank` 模式选项(*glx*,*xpresent* 或 *off*),先使用以下命令尝试一下:

```
$ xfwm4 --replace --vblank=glx &
```

要想保存这个设置:

```
$ xfconf-query -c xfwm4 -p /general/vblank_mode -s glx
```

如果你使用 Intel 显卡并且如 Intel graphics#Tearing 中所述在 Xorg 中启用了 *TearFree* 选项,请禁用 *Synchronize drawing to the vertical blank* 选项。

如果这不能修复撕裂现象,请考虑关闭 Xfwm 的窗口合成器并使用替代的合成管理器。

Dock 窗口上方的水平线

Xfwm 可能会错误地渲染一些 Dock 窗口的阴影(例如Plank)。这可能会导致屏幕上有一条水平线。一个解决方案是禁用*设置* > *窗口管理器微调* > *窗口合成*中的*显示 Dock 窗口下的阴影*。

另见

- Xfwm - 介绍

16.2 Thunar

难度 进阶 | **篇幅** 16.2 KB | **所属** Xfce

本节将讲到

- 安装
- 配置
- Thunar 卷管理器

- 自定义动作
- 提示与技巧

来自项目官网:

: Thunar 是一个用于 Xfce 桌面环境的现代化的文件管理器。Thunar 的设计初衷就是快速且易于使用。其界面简洁直观,默认不包含令人误解或无用的选项。Thunar 快速且响应迅速，具有良好的启动时间和文件夹加载时间。

安装

安装 thunar包 软件包。Thunar xfce4包组 软件包组的一部分,也是 Xfce 桌面环境的默认文件管理器。

插件与附件

- **Gnome 虚拟文件系统** — 安装后显示垃圾桶，可移动介质，和远程文件系统（`mtp`/`smb`）。详情请见 File manager functionality#Mounting。

: https://wiki.gnome.org/Projects/gvfs || gvfs包

- **Thuanr 归档插件** — 让你能够使用上下文菜单创建和解压归档文件的插件。它并不会直接创建或解压归档,而是作为其他程序的前端,例如 File Roller (file-roller包), Ark (ark包), Xarchiver (xarchiver包) 或 Engrampa (engrampa包)。是软件包组 xfce4-goodies包组 的一部分。

: https://goodies.xfce.org/projects/thunar-plugins/thunar-archive-plugin || thunar-archive-plugin包

- **Thunar 媒体标签插件** — 让你能够预览媒体文件详细信息的插件。它还带有批量重命名和编辑媒体标签的功能。它支持 ID3 (MP3 文件格式系统) 和 Ogg/Vorbis 标签。是软件包组 xfce4-goodies包组 的一部分。

: https://goodies.xfce.org/projects/thunar-plugins/thunar-media-tags-plugin || thunar-media-tags-plugin包

- **Thunar 共享插件** — 让你能够无需 root 权限,通过 Thunar ,使用 Samba 快速分享文件夹的插件。另见如何配置方向。

: https://goodies.xfce.org/projects/thunar-plugins/thunar-shares-plugin || thunar-shares-plugin包

- **Thunar 卷管理器** — 在 Thunar 中自动管理可移动设备。是软件包组 xfce4-goodies包组 的一部分。

: https://goodies.xfce.org/projects/thunar-plugins/thunar-volman || thunar-volman包

- **Tumbler** — 生成缩略图的外部程序。还可以安装 ffmpegthumbnailer包 来启用视频缩略图。

: https://gitlab.xfce.org/xfce/tumbler || tumbler包

- **libgsf** — GNOME 结构化文件库是一个用于读写结构化文件格式的实用程序库。如果你需要 odf 缩略图的支持就安装。

: https://directory.fsf.org/wiki/Libgsf || libgsf包

- **RAW thumbnailer** — 一个轻量快速的原始图像缩略图生成器,用于显示原始图像缩略图。

: https://code.google.com/archive/p/raw-thumbnailer/ || raw-thumbnailerAUR

- **Extra thumbnailers** — 为 Tumbler 定制的缩略图生成器。

: https://docs.xfce.org/xfce/tumbler/available_plugins/ || tumbler-extra-thumbnailersAUR

- **Folder thumbnailer** — 通过 Tumbler 支持自定义文件夹缩略图。

: https://github.com/j-james/thunar-folder-thumbnails || tumbler-folder-thumbnailerAUR

- **STL thumbnailer** — 为 Tumbler 生成 STL 缩略图的支持。

: https://github.com/j-james/thunar-stl-thumbnails || tumbler-stl-thumbnailerAUR

- **webp-pixbuf-loader** — gdk-pixbuf2包 的可选依赖，为默认的缩略图生成器提供了生成 webp 缩略图的支持。

: https://github.com/aruiz/webp-pixbuf-loader || webp-pixbuf-loader包

配置

要配置按键绑定，请编辑 `~/.config/Thunar/accels.scm` 文件。要配置 Thunar 的隐藏变量，请使用 `xfconf-query -c thunar -l -v`。

Thunar 卷管理器

若已安装 gvfs包 和 thunar-volman包，可配置 Thunar 在介质连接时自动执行命令。对于可移动设备,通常遵循 MTP,需要额外安装 gvfs-mtp包 软件包。

提示：要想让 Thunar 处理自动挂载,必须在守护进程模式中启动。

配置

它还可以配置为在连接相机和音频播放器时执行某些操作。 在安装插件之后:

1. 启动 Thunar 然后前往 *Edit* > *Preference*
2. 在 'Advanced' 标签页下,勾选 'Enable Volume Management'
3. 点击配置并勾选下列项目:
4. Mount removable drives when hot-plugged.
5. Mount removable media when inserted.
6. 再做出你想要的设置(请见下面的例子)

这是一个自动用 Amarok 播放音频 CD 的设置例子。

```
Multimedia - Audio CDs: amarok --cdplay %d
```

自定义动作

本节介绍有用的自定义操作,可以通过 `Edit -> Configure custom actions` 访问,存储在 `~/.config/Thunar/uca.xml`。thunar wiki 中列出了更多例子。此外,这篇博客文章提供了全面的自定义操作集合。

在此处打开终端

“在此处打开终端” 是安装时的唯一操作。为 exo包 配置要使用的终端:

```
~/.config/xfce4/helpers.rc
TerminalEmulator=terminal
```

Name	Command	File patterns	Appears if selection contains
Open Terminal Here	`exo-open --working-directory %f --launch TerminalEmulator`	*	Directories

搜索文件的目录

要想使用这个动作,你需要安装 catfish包。对于想要有预构建的索引数据库的用户, 可以安装plocate包 和 zeitgeist包 可选依赖。

Name	Command	File patterns	Appears if selection contains
Search	`catfish --path=%f`	*	Directories

扫描病毒

要想使用这个动作,你需要安装 clamav包 和 clamtk包。

Name	Command	File patterns	Appears if selection contains
Scan for virus	`clamtk %F`	*	Select all

链接到 Dropbox

Name	Command	File patterns	Appears if selection contains
Link to Dropbox		*	Directories, other files

Name	Command	File patterns	Appears if selection contains
	`ln -s %f /path/to/DropboxFolder`		

请注意当使用许多自定义操作将文件和文件夹符号链接到特定位置时,将它们放到上下文菜单中的 发送到 目录可以避免上下文菜单变得臃肿。这很容易实现,需要在 `~/.local/share/Thunar/sendto` 中为每个动作创建一个 .desktop 文件。假设我们要将上述 Dropbox 符号链接操作放入 "发送到",创建一个带有下列内容的 `dropbox_folder.desktop` 。新应用的动作会在 Thunar 重启后激活。

```
[Desktop Entry]
Type=Application
Version=1.0
Encoding=UTF-8
Exec=ln -s %f /path/to/DropboxFolder
Icon=/usr/share/icons/dropbox.png
Name=Dropbox
```

提示与技巧

使用 Thunar 浏览远程位置

因为 Xfce 4.8 (Thunar 1.2) 可以直接在 Thunar 浏览远程位置 (例如 FTP 服务器或 Samba 共享)。要想启用这个功能，确保 gvfs包 和 sshfs包（若需要 SMB/CIFS 支持，还需要 gvfs-smb包）软件包已安装。Thunar 侧边栏可以看到 '网络' 项目,可以在位置对话框中使用以下 URI 方案打开远程位置 (用 `Ctrl+l` 打开):smb://, ftp://, ssh://, sftp://, davs:// 后跟服务器主机名或 IP 地址。

没有用于 NFS 共享的 URI 方案,但是如果你正确设置你的 fstab ,Thunar 可以发出 `mount` 命令。

```
/etc/fstab
# nas1 server
nas1:/c/home          /media/nas1/home      nfs     noauto,user,_netdev,bg   0 0
```

这里要注意的是 `noauto` 选项,它会在你点击挂载前阻止共享被自动挂载,`user` 选项是被允许挂载和卸载共享的用户,`_netdev` 选项使网络连接成为先决条件,最后 `bg` 选项使挂载操作在后台进行,如果你的服务器需要一些启动时间,这么做将使你在它工作前不必处理超时消息并重新单击。

提示：

- 如果你想要持久存储远程文件系统位置的密码,你必须安装 GNOME Keyring。
- 为了让 Thunar 显示任意非 root 用户的新设备,可能需要将挂载点设置为 `/media` 的子目录。

以守护进程模式启动

Thunar 可以以守护进程模式运行。这有许多优势,包括加快 Thunar 的启动速度,Thunar 在后台运行而且在需要时只打开一个窗口(例如当闪存驱动器插入时),和让 Thunar 处理可移动介质的自动挂载。

确保命令 `thunar --daemon` 在登录时自动运行。详情请见 Xfce 和自动运行。

缓慢的冷启动问题的解决方案

一些人还有 Thunar 在首次启动时耗时过长的问题。这是因为 gvfs 检查网络,阻止 Thunar 启动直到它完成该操作。要想改善此行为,编辑 `/usr/share/gvfs/mounts/network.mount` ,将 `AutoMount=true` 修改为 `AutoMount=false`。

在侧窗格中隐藏快捷方式

在侧窗格中有一个隐藏菜单用于隐藏快捷方式。

在侧窗格中没有快捷方式的地方右键,例如在 DEVICES 标签上。然后会弹出一个菜单,你可以将不想显示出的项目取消勾选。

在 Thunar 中分配键盘快捷键

请见 GTK#键盘快捷键。

显示在 fstab 中定义的分区

默认情况下 Thunar 不会在设备中显示 `/etc/fstab` 中定义的任何分区,除了 root 分区。

我们可以通过向 fstab 中为想要显示出的分区添加 `x-gvfs-show` 选项。

故障排除

大型外部驱动器的自动挂载

如果安装了 thunar-volman 和 gvfs,但 Thunar 拒绝挂载大型可移动介质(大小 > 1TB),试着安装一个不同的自动挂载器例如 udevilAUR 或 udiskie包。更推荐后者因为它使用 udisks2 而且它与 gvfs 兼容。要想启动带有 udisks2 支持的 udiskie,将下列内容添加到你的自动启动文件:

```
udiskie -2 &
```

Tumblerd 挂起,占用过多 CPU

Tumblerd 在监视文件系统并在需要制作缩略图时通知系统的服务时可能会陷入循环,请见 bug 报告。下面的脚本是一个阻止这种情况发生的临时解决方案。复制,将它粘贴

到一个 *.sh* 文件,将它保存的你的家目录的某个位置,将文件标记为可运行,然后在将它设置为在系统启动时自动运行。

```
#!/bin/bash
period=20
tumblerpath="/usr/lib/*/tumbler-1/tumblerd" # The * here should find the right one,
whether 32 and 64-bit
cpu_threshold=50
mem_threshold=20
max_strikes=2                                  # max number of above cpu/mem-threshold's
in a row
log="/tmp/tumblerd-watcher.log"

if [[ -n "${log}" ]]; then
    cat /dev/null > "${log}"
    exec >"${log}" 2>&1
fi

strikes=0
while sleep "${period}"; do
    while read pid; do
       cpu_usage=$(ps --no-headers -o pcpu -f "${pid}"|cut -f1 -d.)
       mem_usage=$(ps --no-headers -o pmem -f "${pid}"|cut -f1 -d.)

       if [[ $cpu_usage -gt $cpu_threshold ]] || [[ $mem_usage -gt $mem_threshold ]];
then
           echo "$(date +"%F %T") PID: $pid CPU: $cpu_usage/$cpu_threshold %cpu MEM:
$mem_usage/$mem_threshold STRIKES: ${strikes} NPROCS: $(pgrep -c -f ${tumblerpath})"
           (( strikes++ ))
           if [[ ${strikes} -ge ${max_strikes} ]]; then
              kill "${pid}"
              echo "$(date +"%F %T") PID: $pid KILLED; NPROCS: $(pgrep -c -f
${tumblerpath})"
              strikes=0
           fi
       else
           strikes=0
       fi
    done < <(pgrep -f ${tumblerpath})
done
```

垃圾桶/网络 图标随机消失

确保所有 Thunar 示例在 *gvfs* 之后启动。对于 `thunar --daemon`, 你可以将它包装成等待直到 GVFS 启动:

注意：在 `$PATH` 中 `/usr/local/bin` 应该在 `/usr/bin` 的前面。

```
/usr/local/bin/Thunar
#!/bin/bash
if [[ $1 == --daemon ]]; then
  until pgrep gvfs >/dev/null; do
    sleep 1
  done
  exec /usr/bin/Thunar "$@"
else
  exec /usr/bin/Thunar "$@"
fi
```

未经身份验证挂载文件系统

请见 File manager functionality#Troubleshooting。

Thunar 新窗口或标签打开速度太慢

可能是因为你在设置为 `XDG_TEMPLATES_DIR` 的目录中有太多文件。请见 XDG 用户路径。

解决方案是将 `XDG_TEMPLATES_DIR` 中文件移到其他地方,或者将 `XDG_TEMPLATES_DIR` 设置为其他目录。

另见

- Thunar 项目页面
- Thunar 卷管理器项目页面
- 这个插件列表
- 一些设置,比如在标题中显示完整路径,可以通过 `xfconf-query` 启用。详情请见这个列表。

16.3 Xfce

难度 高级 | **篇幅** 40.9 KB | **所属** Xfce

本节将讲到

- 安装
- 启动
- 配置
- 提示和小技巧
- 故障排除

Xfce 是一个基于 GTK 3 的轻量级模块化的桌面环境。为了提供完整的用户体验，它包含窗口管理器、文件管理器、桌面和面板。

安装

安装 xfce4包组 包组。如果需要的话，还可以安装 xfce4-goodies包组 包组。此包组提供了一些额外的插件和一些有用的工具，如 mousepad包 编辑器。 Xfce 默认使用 Xfwm 作为窗口管理器。

启动

从显示管理器中选择*Xfce Session*，或者添加 `exec startxfce4` 到 Xinitrc 中。

注意：不要直接调用 `xfce4-session`可执行文件，`startxfce4` 是正确的命令，它会在恰当的时间调用前述可执行文件。

配置

Xfce 把配置的选项保存到 Xfconf。有几个方式来修改这些选项：

- 在主菜单中，选择设置和要自定义的类别。类别是通常位于 `/usr/bin/xfce4-*` 和 `/usr/bin/xfdesktop-settings` 中的程序。
- `xfce4-settings-editors` 可以查看和修改所有设置。此处修改的选项会立即生效。使用`xfconf-query`从命令行更改设置；文档中有更多的细节。
- 设置保存在 XML 文件中。此文件位于 `~/.config/xfce4/xfconf/xfce-perchannel-xml/`，可以手动修改。但是，此处的修改不会立即生效。

菜单

关于使用 Free Desktop 菜单系统的信息，参见 Xdg-menu。

Whisker 菜单

xfce4-whiskermenu-plugin包（包含在 xfce4-goodies包组 中）是默认的应用启动器的可选替代品。它可以显示收藏夹列表，通过类别按钮浏览所有已安装的应用程序，并支持模糊搜索。安装完成后，就可以替换掉面板1的第一个项目“应用程序菜单”了（在 *设置* > *面板* > *项目*中添加*Whisker 菜单*）。

为 Whisker 菜单设置快捷键

要想设置启动 Whisker 菜单的键盘快捷键,前往 *设置* > *键盘*的*应用快捷键*页面。点击 *添加*,将命令设置为 `xfce4-popup-whiskermenu` 并分配你想要的键盘快捷键。

编辑菜单

许多图形工具可以用来实现此项需求：

- **MenuLibre** — 一个高级的菜单编辑器，提供了一个清晰、易用的界面。

: https://launchpad.net/menulibre || menulibreAUR

- **Alacarte** — GNOME的菜单编辑器。

: https://www.gnome.org/ || alacarte包

- **XAME (XFCE Applications Menu Editor)** — 使用 Gambas 编写，专门用于编辑Xfce菜单项的图形工具，在其他环境中没有效果。（已停止开发）

: http://redsquirrel87.altervista.org/doku.php/xfce-applications-menu-editor || xameAUR

或者也可以手动创建 `~/.config/menus/xfce-applications.menu`。下面给出一个示例的配置：

```
<!DOCTYPE Menu PUBLIC "-//freedesktop//DTD Menu 1.0//EN"
  "http://www.freedesktop.org/standards/menu-spec/1.0/menu.dtd">
```

```
<Menu>
    <Name>Xfce</Name>
    <MergeFile type="parent">/etc/xdg/menus/xfce-applications.menu</MergeFile>

    <Exclude>
        <Filename>xfce4-run.desktop</Filename>
        <Filename>exo-terminal-emulator.desktop</Filename>
        <Filename>exo-file-manager.desktop</Filename>
        <Filename>exo-mail-reader.desktop</Filename>
        <Filename>exo-web-browser.desktop</Filename>
        <Filename>xfce4-about.desktop</Filename>
        <Filename>xfhelp4.desktop</Filename>
    </Exclude>

    <Layout>
        <Merge type="all"/>
        <Separator/>
        <Menuname>Settings</Menuname>
        <Separator/>
        <Filename>xfce4-session-logout.desktop</Filename>
    </Layout>
</Menu>
```

`<MergeFile>` 标签包含了默认的Xfce菜单。

`<Exclude>` 标签剔除了你不想在菜单中出现的应用程序。尽管此处我们只剔除了一些Xfce的默认快捷方式，但是你也可以剔除 `firefox.desktop` 或其他任何的应用程序。

`<Layout>` 标签定义了菜单的布局。应用程序可以分组放在文件夹中，或以任何我们想要的方式进行组织。在 Xfce wiki 有更多的详细信息。

你可以通过编辑 `.desktop` 本身来改变Xfce的菜单。关于如何隐藏项目，参见 Desktop entries#Hide desktop entries。你可以通过改变 `.desktop` 中的 `Categories=` 来编辑应用程序类别。参见 Desktop entries#File example。

设置首选应用程序

要想修改打开某种文件的默认应用,使用 exo-preferred-applications 。这会修改 `exo-open` 的行为,它由 resource opener 调用,例如 xdg-open。

桌面

图标文字的透明背景

默认桌面图标的文字是白色背景，可以创建或者修改 `~/.gtkrc-2.0` 来得到不一样的效果：

```
style "xfdesktop-icon-view" {
    XfdesktopIconView::label-alpha = 10
    base[NORMAL] = "#000000"
    base[SELECTED] = "#71B9FF"
    base[ACTIVE] = "#71B9FF"
    fg[NORMAL] = "#fcfcfc"
    fg[SELECTED] = "#ffffff"
    fg[ACTIVE] = "#ffffff"
}
```

```
widget_class "*XfdesktopIconView*" style "xfdesktop-icon-view"
```

删除桌面图标

使用如下的命令：

```
$ xfconf-query -c xfce4-desktop -v --create -p /desktop-icons/style -t int -s 0
```

要想恢复桌面上的图标,使用相同的命令,但将数值改为 2。

多显示器连续壁纸

打开 `xfce4-settings-editor` 创建如下的属性：

```
Property: /backdrop/screen0/xinerama-stretch
Type: Boolean
Value: TRUE|1|Enabled
```

关闭窗口的快捷键

Xfce没有关闭窗口的快捷键，当程序假死时，我们可能需要这样的快捷键。

使用 xorg-xkill包，`xkill` 可以交互关闭窗口。对于当下的激活窗口，使用包 xdotool 包：

```
$ xdotool getwindowfocus windowkill
```

也可以：

```
$ sh -c "xkill -id $(xprop -root -notype | sed -n '/^_NET_ACTIVE_WINDOW/ s/^.*# *\| \,.*$//g p')"
```

添加快捷键，使用**设置 > 键盘**或者使用应用程序，如 xbindkeys包。

会话

自动运行

可以在**程序 > 设置 > 设置管理器 > 会话和自启动**中点击**应用程序自启动**，设置与Xfce一起启动的自启动程序。 此处列出了所有自启动的程序。点击**添加**按钮后可以添加自定义的自启动任务，需指定可执行文件的路径。

自启动应用会在 `~/.config/autostart/` 中以 `*name*.desktop` 存储。

或者也可以将要执行的命令（包括设置环境变量）加入 xinitrc。如果使用显示管理器，则加入 xprofile 。

提示：有时**延迟应用的启动**是很有用的。注意在 *Application > Autostart* 中的设定的命令例如 `sleep 3 && *command*` 不会工作;一个解决方案是使用 `sh -c "sleep 3 && *command*"`的语法

延迟某个应用程序启动有时可能很有用。在**应用程序自启动**中指定类似 `sleep 3 && command` 的命令不会起作用。作为一个解决办法，可以使用如下命令：

```
sh -c "sleep 3 && command"
```

锁定屏幕

xflock4 是用于锁定 Xfce 会话的参考 Bash 脚本。

它试图使用 xfce4-screensaver包 (是软件包组 xfce4-goodies包组 的一部分),xscreensaver包,gnome-screensaverAUR,slock包,xlockmore包 中的一个来锁定屏幕。它连续查找相应的二进制文件，如果找不到其中任何一个，则返回代码 1 退出。

应用程序列表/安全#锁屏包含了对这些锁屏软件和其他流行应用的简短描述。这里列出一些替代锁屏应用,light-locker包 与 xfce4-power-manager包 集成得特别好。安装之后,Xfce 电源管理的设置会有一个额外的*安全*标签页用于配置 *light-locker*,已存在的*系统睡眠时锁定屏幕*设置也会重定位到此页面下。在这个新的图形界面可以设置是否应在屏幕保护程序活动或系统进入睡眠状态时锁定会话。

要想让 *xflock4* 运行 *light-locker* 或上述五个以外的任意自定义的会话锁定软件,必须在会话的 xfconf 频道中将 `LockCommand` 设置为使用的命令(以下示例中引号内的命令可以针对其他屏幕锁定软件进行相应调整):

```
$ xfconf-query --create -c xfce4-session -p /general/LockCommand -t string -s "light-locker-command --lock"
```

面板的*动作按钮*界面中的锁定按钮只是运行 `/usr/bin/xflock4`。只要 *xflock4* 有用,那么它应该能如预期工作,例如安装原生锁屏软件中的一个,或如上述将自定义锁屏软件配置为与它集成。

睡眠

当请求睡眠时,Xfce 运行带有 `suspend` 选项的 xfce4-session-logout(1) 命令:

```
$ xfce4-session-logout --suspend
```

会话是否系统地锁定为*睡眠*可以通过 xfconf 属性或图形界面进行配置。

要想使用命令行控制此状态:有两个设置,分别位于会话和电源管理的 xfconf 频道的 `LockScreen` 和 `lock-screen-suspend-hibernate`。

如果不想在睡眠时锁定,将它们设置为 `false`:

```
$ xfconf-query -c xfce4-session -p /shutdown/LockScreen -s false
$ xfconf-query -c xfce4-power-manager -p /xfce4-power-manager/lock-screen-suspend-hibernate -s false
```

类似地,将它们设置为 `true` 将会在睡眠时锁定。

这些设置也可以通过图形界面修改:打开*会话与启动*应用,打开或关闭*Advanced > Lock screen before sleep*。

无论何时按下睡眠键,它都会由 Xfce 的电源管理器或 *systemd-logind* 处理。要提高 logind 的优先级,下列 xfconf 设置必须设置为 `true`:

```
$ xfconf-query --create -c xfce4-power-manager -p /xfce4-power-manager/logind-handle-suspend-key -t bool -s true
```

注意：要想查看 *systemd-logind* 在优先于 Xfce 时如何处理事件,查看 logind.conf(5)

禁用保存的会话

可以通过下面命令禁用某个用户已保存的会话：

```
$ xfconf-query -c xfce4-session -p /general/SaveOnExit -s false
```

*然后进入应用程序 -> 设置 -> 会话和启动 -> 会话*并点击*清除已保存的会话*按钮来清除所有之前保存的会话。

提示：如果上面命令无法持久生效，可以用下面命令：`xfconf-query -c xfce4-session -p /general/SaveOnExit -n -t bool -s false`

Xfce kiosk 模式可以用来彻底禁用对话的保存。要禁用对话，创建或者编辑 `/etc/xdg/xfce4/kiosk/kioskrc` 并加入如下内容：

```
[xfce4-session]
SaveSession=NONE
```

如果kiosk模式不起作用，用户可以给会话目录设置只读权限：

```
$ rm ~/.cache/sessions/* && chmod 500 ~/.cache/sessions
```

此操作会防止Xfce保存所有的会话，即使任何别的设置允许保存会话。

使用不同的窗口管理器

注意： 要使更改生效，需要清除保存的会话，并确保在首次注销时禁用会话保存。 一旦选择的窗口管理器正在运行，可以再次启用会话保存。

窗口管理器的设定保存在：

- /etc/xdg/xfce4/xfconf/xfce-perchannel-xml/xfce4-session.xml - 系统设置
- ~/.config/xfce4/xfconf/xfce-perchannel-xml/xfce4-session.xml - 用户设置

单个用户的默认窗口管理器可以用*xfconf-query*命令设置：

```
$ xfconf-query -c xfce4-session -p /sessions/Failsafe/Client0_Command -t string -sa xfsettingsd
$ xfconf-query -c xfce4-session -p /sessions/Failsafe/Client1_Command -t string -sa wm_name
```

如果要使用命令行选项启动窗口管理器，请使用以下命令:

```
$ xfconf-query -c xfce4-session -p /sessions/Failsafe/Client0_Command -t string -sa xfsettingsd
 $ xfconf-query -c xfce4-session -p /sessions/Failsafe/Client1_Command -t string -s wm_name -t string -s --wm-option
```

如需更多命令行选项，只需向命令中添加更多 `-t string` 和 `-s **--wm-option**` 参数。

如需更改整个系统的默认窗口管理器，手动编辑上面指定的文件，将*xfwm4*更改为首选窗口管理器，并添加更多 `<value type="string" value="**--wm-option**"/>` 选项（如果需要）。

注意：如果将 `Client0_Command` 设置为 `wm_name` 并将 `Client1_Command` 设置为 `xfsettingsd`,就可以阻止 `xfce4-session-logout` 恢复到默认主题。但还是需要手动设置光标主题。

如需在系统范围内修改默认的窗口管理器,手动编辑上述的文件,将 *xfwm4* 改为你想要的窗口管理器,对于额外的命令行参数,添加更多的 `<valuetype="string" value="*--wm-option*"/>`。

还可以通过使用自启动工具自动启动 `*wm_name* --replace` 或通过在终端中运行 `*wm_name* --replace &` 并确保注销时会话已被保存来修改窗口管理器。请注意,这种方法并没有真正修改默认窗口管理器,而是在登录时替换窗口管理器。请注意如果使用了自启动工具,你应该禁用已保存的会话,因为这可能会导致早默认的窗口管理器启动后,新的窗口管理器还启动两次。

更换主题

在 xfce-look.org 上有不少 XFCE 的主题。 *Xfwm* 的主题保存在 `/usr/share/themes/*theme_name*/xfwm4`, 在*设置* > *窗口管理器*中可以更改主题。 GTK 主题保存在 `/usr/share/themes/*theme_name*/gtk-2.0` 和 `/usr/share/themes/*theme_name*/gtk-3.0`,可以在*设置* > *外观*中修改 GTK 主题。

如果想要使所有的应用能有一个统一的外观, 参见 Uniform look for Qt and GTK applications 获得更多的信息。

另参见光标主题, Icons, 和 Font configuration。

统一 SSD 和 CSD 窗口的外观

对于多数窗口,Xfce 当前使用服务器端装饰(SSD)(请见窗口装饰 #Window_decoration)),大多数情况下由 Xfwm 提供主题,而客户端装饰(CSD) 以 Xfce 设置、打印、保存和其他对话框的相应程序为主题。

Xfwm SSD 窗口风格可以通过一些方法转变为符合 CSD 窗口的主题,比如手动调整,或在 `/usr/share/themes/*theme_name*/xfwm4` 中手动创建主题,或使用例如 Xfwm4 Theme Generator 这样"从客户端装饰创建 xfwm4 主题"的工具。

恢复客户端装饰

要想去除 Xfce 的大多数 CSD,安装 libxfce4ui-nocsdAUR 并进行设置:

```
$ xfconf-query -c xsettings -p /Gtk/DialogsUseHeader -s false
```

这会将保存按钮移到窗口的保存对话框的底部,并对打印对话框也做出相同的改动。Xfce 设置对话框将会恢复成 SSD。但是像 Catfish 这样的程序还是会有 CSD.

要在所有窗口上强制启用 SSD,试试 gtk3-nocsd-gitAUR。请务必阅读项目页面上的所有说明。

声音

声音主题

XFCE4支持 freedesktop system sounds， 但需要配置。

启用声音主题：

1. 安装 libcanberra包 以提供 PulseAudio 支持。
2. "canberra-gtk-module" 应该在 GTK_MODULES 环境变量 (需要重新登录);
3. 在 设置管理器 → 外观 → 设置 选项卡 勾选 "启用事件声音";
4. 在 设置编辑器（Settings Editor） 设置 "xsettings/Net/SoundThemeName" 为一个在 `/usr/share/sounds/` 下的声音主题;
5. 在混声器里（比如pavucontrol）打开系统声音（System Sounds）。

sound-theme-freedesktop包 提供兼容的声音主题，缺少许多需要的事件。另一个更好的选择是 sound-theme-smoothAUR (上面的 SoundThemeName 设置为 "Smooth")。

键盘音量键

xfce4-pulseaudio-plugin包 提供了一个面板小程序，它支持键盘音量控制和音量提示。或者，可以用不提供面板图标的 xfce4-volumed-pulseAUR，它还提供键绑定和通知控制。当同时使用 pasystray包 进行更细微的控制时会很方便。

或者使用 xfce4-mixerAUR，它同样提供面板小程序和键盘快捷键，并支持Alsa。然而，请注意，它是基于已在1.0中放弃的GStreamer 0.10的功能。

在安装面板之后,必须将它添加到任务栏否则键盘快捷键不会工作。

对于不需要特定桌面环境的替代方案,请见 List of applications#Volume control。

快捷键

如不使用控制音量键的小程序或守护程序，则可以使用Xfce的键盘设置手动将音量控制命令映射到音量键。对于您正在使用的音响系统，请参阅以下链接到相应命令的部分。

- ALSA: Advanced Linux Sound Architecture#Keyboard volume control.
- PulseAudio: PulseAudio#键盘音量控制
- OSS: OSS#Keyboard volume control.

键盘快捷键

键盘快捷键在两个地方设置：*设置* > *窗口管理器* > *键盘*和*设置* > *键盘* > *快捷键*。

Polkit 验证代理

在安装 xfce4-session包 时，会同时安装 polkit-gnome包，并随系统自动启动;无需用户干预。更多信息请参见 Polkit#Authentication agents。

Xfce 可用的第三方 Polkit 身份认证代理，参见 xfce-polkit-gitAUR 和 xfce-polkitAUR。

熄屏

一些程序通常和 Xfce 一起使用来控制显示器消隐和 DPMS (显示器节能)设置。接下来将讨论它们。

Xfce 电源管理器

*Xfce 电源管理器*控制熄屏和 DPMS 设置。这些设置可以在*电源管理器*的图形界面的*显示*标签页中配置。

请注意当*显示电源管理*被关闭时,DPMS 会被完全禁用,这并不意味着*电源管理器*会停止控制 DPMS。它也不会禁用熄屏。要想将熄屏和 DPMS 都禁用,右键电源管理器的系统托盘图标或左键面板小程序并确保选中标记为"演示模式"的选项。

XScreenSaver

如果安装了 xscreensaver包 并且和 Xfce 电源管理器一起运行,控制熄屏和 DPMS 的应用可能不明确,因为它们会抢占同一个设置的控制权。因此在显示器不需要熄屏的情况下(例如看视频时),建议关闭两个应用的熄屏和 DPMS 设置。关于 *XScreenSaver* 设置的更多细节,请见 XScreenSaver#DPMS and blanking settings。

xset

如果上述两个应用的没有运行,熄屏和 DPMS 设置可以使用 *xset* 命令控制,请见 DPMS#用xset修改DPMS和屏保设定。

注意：在某些配置中,存在与熄屏和从熄屏恢复相关的一些问题。请见 。

提示和小技巧

Thunar 和 Xfdesktop 的挂载支持

如果在 Thunar 和桌面中插入的外部驱动器没有显示,而且安装分区显示为已挂载的设备,请安装 gvfs包。请见 Udisks#隐藏选中的分区和 Thunar#大型外部驱动器的自动挂载以获取高级配置选项。

屏幕截图

Xfce 有自己的截图工具 xfce4-screenshooter包。它是 xfce4-goodies包组 包组的一部分。

到*应用程序* > *设置* > *键盘*, *应用程序快捷方式*. 添加 `xfce4-screenshooter -f` (或 `-w` 为活动窗口)命令用 `Print` 键截屏。 其他可选参数参见 xfce4-screenshooter(1)。

此外，也可用其他独立的截图程序如 scrot。

禁用终端 F1 和 F11 快捷方式

XFCE 终端下 F1 和 F11 分别被绑定给了帮助和全屏，给一些程序造成了冲突。要禁用这些快捷方式，创建或修改下面的配置文件然后注销重新登录。F10 可以在设置里更改。

```
~/.config/xfce4/terminal/accels.scm
(gtk_accel_path "<Actions>/terminal-window/fullscreen" "")
(gtk_accel_path "<Actions>/terminal-window/contents" "")
```

终端的颜色主题和调色板

可以在首选项的外观标签下修改终端主题颜色和调色板。这些色彩可用于多大数控制台程序如Emacs, Vi 等。 它们的设置单独存储在每个用户的 `~/.config/xfce4/terminal/terminalrc` 文件中。 还有更多主题可供选择。论坛下 Terminal Colour Scheme Screenshots 有数百的更多终端配色方案。

修改默认颜色主题

XFCE 的 `extra/terminal` 包使用了较暗的颜色使得文字在默认的黑色背景下很难阅读并会使人感到不适，请把以下文字写入到 terminalrc 文件中来使用一个较明亮的颜色主题, 它会在一直在较暗的终端背景下可见。

```
~/.config/xfce4/terminal/terminalrc
ColorPalette5=#38d0fcaaf3a9
ColorPalette4=#e013a0a1612f
ColorPalette2=#d456a81b7b42
ColorPalette6=#ffff7062ffff
ColorPalette3=#7ffff7bd7fff
ColorPalette13=#82108210ffff
```

终端 Tango 颜色主题

用你喜欢的编辑器打开 `~/.config/xfce4/terminal/terminalrc` 并加入下列内容(用下列内容替换原内容):

```
ColorForeground=White
ColorBackground=#323232323232
ColorPalette1=#2e2e34343636
ColorPalette2=#cccc00000000
ColorPalette3=#4e4e9a9a0606
ColorPalette4=#c4c4a0a00000
ColorPalette5=#34346565a4a4
ColorPalette6=#757550507b7b
ColorPalette7=#060698989a9a
ColorPalette8=#d3d3d7d7cfcf
ColorPalette9=#555557575353
ColorPalette10=#efef29292929
ColorPalette11=#8a8ae2e23434
ColorPalette12=#fcfce9e94f4f
ColorPalette13=#72729f9fcfcf
ColorPalette14=#adad7f7fa8a8
ColorPalette15=#3434e2e2e2e2
ColorPalette16=#eeeeeeeeecec
```

终端下用鼠标中键打开 URL

升级到 0.8 后鼠标中键的默认行为改成了粘贴到光标。 要改回元行为，修改 `${XDG_CONFIG_HOME}/xfce4/terminal/terminalrc`（默认 `XDG_CONFIG_HOME=${HOME}/.config`）

```
${XDG_CONFIG_HOME}/xfce4/terminal/terminalrc
[Configuration]
MiscMiddleClickOpensUri=TRUE
```

终端中的 env-modules 自动补全

env-modulesAUR 和 env-modules-tclAUR 软件包提供了为 login shell 提供了 shell 自动补全。但是 `xfce4-terminal` 默认并不被当作 *login* 会话。要启用环境变量模块的自动补全,在*首选项*中勾选相应的复选框或者在 `~/.config/xfce4/terminal/terminalrc` 中将 `CommandLoginShell` 改为 `TRUE`。

颜色管理

Xfce 本身没有颜色管理的功能支持。 查看 ICC profiles 寻找替代。

多显示器

Xfce 有着多显示器的支持。可以在*应用* > *设置* > *显示*对话框中配置。

在*高级*标签页中可以保存用于不同显示器的配置并且当连接的显示器更改时自动应用相应的配置。

详情请见官方文档的 display 文章。

或者使用 arandr包 来简单地用 xrandr 格式的命令管理显示器配置,而且可以分配 XFCE 键盘快捷键。

SSH 代理

默认 Xfce 4.10 会在会话启动时试着按顺序打开 gpg-agent 或 ssh-agent。要禁用的话，运行如下命令：

```
xfconf-query -c xfce4-session -p /startup/ssh-agent/enabled -n -t bool -s false
```

若 gpg-agent 安装了也要启动 ssh-agent 的话运行：

```
xfconf-query -c xfce4-session -p /startup/ssh-agent/type -n -t string -s ssh-agent
```

要使用 GNOME Keyring，在*设置*里的*会话和启动*的*高级*页选中*桌面启动时启动 GNOME 服务*。这还会禁止 gpg-agent 和 ssh-agent 的启动。

参见：https://docs.xfce.org/xfce/xfce4-session/advanced

滚动时不获得焦点

在*设置* > *窗口管理器微调* > *辅助功能*下取消*按下任意鼠标按钮时提升窗口*。

修改窗口管理器 modifier

默认的 modifier 是 `Alt`。可以用 *xfconf-query*更改。比如说下面的命令会将其改为 `Super`：

```
$ xfconf-query -c xfwm4 -p /general/easy_click -n -t string -s "Super"
```

严格地说，并不支持多 modifier。可是实际可以用 >< 把多个键分隔起来。比如下面的命令会把 modifier 改为 `Ctrl+Alt`：

```
$ xfconf-query -c xfwm4 -p /general/easy_click -n -t string -s "Ctrl><Alt"
```

设置触摸板两指单击为鼠标中键

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：更简单的方法 Touchpad Synaptics（在Talk:Xfce讨论）

如果你想让触摸板两指单击识别为鼠标中键，创建或更改如下文件：

```
~/.config/xfce4/xfconf/xfce-perchannel-xml/pointers.xml
<channel name="pointers" version="1.0">
  <property name="SynPS2_Synaptics_TouchPad" type="empty">
    <property name="Properties" type="empty">
      <property name="Synaptics_Tap_Action" type="array">
        <value type="int" value="0"/>
        <value type="int" value="0"/>
        <value type="int" value="0"/>
        <value type="int" value="0"/>
        <value type="int" value="1"/>
        <value type="int" value="2"/>
        <value type="int" value="3"/>
      </property>
    </property>
  </property>
```

```
</channel>
```
数组中的2就是鼠标中键。

限制亮度划块的最小亮度

在一些显示器下亮度等级设为0后背光会完全关掉。`xfce4-power-manager 1.3.2` 有一个新的隐藏选项可以调节最小亮度。用 xfconf4 添加一个名为 `brightness-slider-min-level` 的整数键，将其改为合适的最小亮度值。

添加个人资料图片

要为每个用户添加显示在 whisker 菜单中的资料图片,将一张 96x96 的 PNG 图片放在每个用户的家目录中,并命名为 `.face`。举个例子,PNG 文件 `/home/*bob*/.face` 是用于用户 *bob* 的资料图片。

像 GIMP 这样的图像编辑软件可以将你喜欢的图像转换到 96x96 分辨率。

电源管理器插件标签

Xfconf 的 `int` 类设置 `show-panel-label` 控制电源管理器的标签,它可以被配置成不同的标签格式:设置成 0 (无标签), 1 (百分比), 2 (剩余时间) 或 3 (百分比和剩余时间)。

它也可以通过电源管理器的图形界面中的 *Properties* > *Show label* 来配置。

在快捷键中使用 Windows (Super) 键

Super 键被当作一个附加键,就像 Ctrl 和 Alt,而不是产生按键。将动作分配给它会阻止你在其他快捷键中使用它,因为它在触发你分配给它的其他动作的同时还会触发该动作。

要解决这个问题,并让它在快捷键中更有用,安装应用 xcape包。这个应用让你可以将附加键配置成在按下和松开时充当其他键。

接下来,前往 设置 > 键盘 > 应用快捷键 并将一个为使用的按键组合,例如 Alt-F1,分配给应用菜单 (或是任何你希望按下 Super 键时执行的动作)。测试一下是否工作。 接下来,使用 xcape包 将 Alt-F1 分配给 Super 键:

```
$ xcape -e 'Super_L=Alt-L|F1'
```
检查 Super 键是否执行你分配给 Alt-F1 的动作。

如果一切正常,将它设置成一个自启动动作;前往 设置 > 会话与启动 > 应用自启动,按下添加按钮并输入上述命令,让它在每次启动 Xfce 都运行(如果 xcape 已经安装了,检查有没有相同的条目被注册)。

现在,你可以在快捷键中随意使用 Super 键。 举个例子:在 窗口管理器 > 键盘 中,你可以使用 Super 和上下键来提升或降低窗口。

故障排除

桌面图标顺序被打乱

在一些情况下（比如打开面板设置对话框时）桌面图标的顺序会被改变。这是因为其顺序是由在 `~/.config/xfce4/desktop/` 下的文件所决定的，而每次改变桌面（添加删除图标或改变位置）就会生成一个新文件，导致了可能的冲突。

要解决这个问题，打开那个目录然后只留下一个正确的配置文件。可以通过其内容来判别到底是哪个文件。里面行数定义为 `row 0`，列数定义为 `col 0`。因而如下的文件内容：

```
[Firefox]
row=3
col=0
```

意为火狐在最左边第四行。

GTK 主题在多显示器下不正常

一些配置工具会损坏 displays.xml 从而导致*应用程序* > *设置* > *外观*无法工作。要解决问题，删除 `~/.config/xfce4/xfconf/xfce-perchannel-xml/displays.xml` 然后重新设置。

右键菜单没有图标

注意：GConf 已被不建议使用，但这个方法还有效。

有时一些程序，包括用 Qt 写的程序的右键菜单没有图标。这个问题只发生在 Xfce 下。运行如下命令:

```
$ gconftool-2 --type boolean --set /desktop/gnome/interface/buttons_have_icons true
$ gconftool-2 --type boolean --set /desktop/gnome/interface/menus_have_icons true
```

修改设置不生效

如果你正在运行一个单独的 Xsettings 守护进程,可能会有一些配置不生效。通过移除或注释对应行禁用它然后重启 Xorg。

NVIDIA 和 xfce4-sensors-plugin

要探测 NVIDIA gpu 的温度，需要安装 libxnvctrl包 并且用 ABS 重新编译 xfce4-sensors-plugin包 软件包。或者改安装 xfce4-sensors-plugin-nvidiaAUR。

NVIDIA 显卡和多显示器在启动时黑屏

使用 NVIDIA,多显示器和 NVIDIA/Troubleshooting#Avoid screen tearing 可能会导致在启动 Xfce 时黑屏。屏幕的位置与文件 `/etc/X11/xorg.conf` 和 `~/.config/xfce4/xfconf/xfce-perchannel-xml/displays.xml` 冲突。删除 `displays.xml` 文件可以修复这个问题。

```
$ rm ~/.config/xfce4/xfconf/xfce-perchannel-xml/displays.xml
```

面板小程序挤在左边

添加一个分割符并选中”扩展”属性。

首选应用程序没有效果

**本文或本章节的事实准确性存在争议。**

原因： xorg-xprop包 should only be needed to activate the classic fallback, a "last effort" attempt at detecting Xfce. This should be fixable by setting `XDG_CURRENT_DESKTOP=XFCE` \[sic\], if modern Xfce does not already do so.（在 Talk:Xfce 中讨论）

大多数程序依赖 xdg-open 来用首选应用程序打开想要的文件和 URL。

要让 xdg-open 和 xdg-settings 与 Xfce 桌面环境检测和整合，需要安装 xorg-xprop包 包。

如果不这么做的话，在 exo-preferred-applications 设置的首选应用程序就没有效果。安装后 *xdg-open* 会检测到你正在运行 Xfce，从而把调用全转交给 *exo-open*。它会正常地使用你的首选应用程序设置。

要确认 xdg-open 是否正常工作，询问 *xdg-settings* 默认浏览器的返回结果：

```
# xdg-settings get default-web-browser
```

如果输出的是：

```
xdg-settings: unknown desktop environment
```

这说明 xdg-open 没有检测出你的桌面环境。原因很可能在没有安装 xorg-xprop包 包。

恢复默认设置

如果出于某些愿意需要恢复默认设置，重命名 `~/.config/xfce4-session/` 和 `~/.config/xfce4/`

```
$ mv ~/.config/xfce4-session/ ~/.config/xfce4-session-bak
$ mv ~/.config/xfce4/ ~/.config/xfce4-bak
```

重新登录后就会起效果。若登录时出现 `Unable to load a failsafe session`，见 #会话失败一节。

会话失败

包括以下症状：

- 鼠标变成了叉号甚至没有鼠标
- 没有标题栏，无法关闭窗口
- (`xfwm4-settings`) 不起动，报 `These settings cannot work with your current window manager (unknown)`
- 显示管理器报错，例如 `No window manager registered on screen 0`。
- `Unable to load a failsafe session`

```
Unable to load a failsafe session.
Unable to determine failsafe session name.  Possible causes: xfconfd is not running (D-Bus setup problem); environment variable $XDG_CONFIG_DIRS is set incorrectly (must include "/etc"), or xfce4-session is installed incorrectly.
```

重启可能会解决问题，但原因也可能在于错误的会话。删除会话目录：

```
$ rm -r ~/.cache/sessions/
```

还有就是保证 `$HOME` 的对应目录是被启动 `xfce4` 的用户所拥有的。见 Chown。

标题栏字体使 xfce4-title 崩溃

安装 ttf-droid包 和 ttf-dejavu包。参见 FS#44382。

笔记本盖设置没有效果

你可能会发现 Xfce4 电源管理器的合盖设置没有效果，不论什么设置合盖后总是挂起。这是因为默认 logind 而非电源管理器接管了合盖的事件。要更改该行为，运行命令：

```
$ xfconf-query -c xfce4-power-manager -p /xfce4-power-manager/logind-handle-lid-switch -s false
```

注意：有些情况下当你更改合盖动作和挂起时锁定的设置时 `logind-handle-lid-switch` 设置会又变成 true，详见 。你需要再手动把它设成 `logind-handle-lid-switch` false。

切换用户动作按钮不可用

*切换用户*动作按钮假设*gdmflexiserver*程序(由 GDM 提供)存在。因此如果没有使用 GDM,按钮就不可用,请见 upstream bug report 。

可用的解决方案是在 `/usr/bin` 中创造一个名为 *gdmflexiserver* 的可执行的脚本,该脚本调用显示管理器提供的欢迎程序切换命令。

- 对于 LXDM - LXDM#自动用户和切换用户。
- 对于 LightDM - LightDM#User switching。

.Xresources 中的宏不起作用

Xfce 使用 `xrdb` 加载 `$HOME/.Xresources` 文件,但是带有 {{ic}-nocpp}} 选项来跳过预处理。对于正常工作的宏,将 `/etc/xdg/xfce4/xinitrc` 复制到 `$HOME/.config/xfce4` 目录,并在产生的文件中移除 `xrdb` 的 `-nocpp` 选项。请见相关论坛话题。

光标主题在登录时不会改变

确保系统范围内的 XDG 光标设置成你想要的光标主题--请见光标主题#XDG 规范。

Mousepad 编辑器菜单栏不可见

运行下面这条命令使它可见:

```
$ gsettings set org.xfce.mousepad.preferences.windowmenubar-visible true
```

垃圾桶图标不可见而且垃圾桶小程序不工作

垃圾桶需要可选依赖 gvfs 才能工作。安装 gvfs包 并重启系统。

桌面变成灰色而且所有桌面图标消失

删除 `~/.cache/sessions`:

```
$ rm -rf ~/.cache/sessions
```

之后重启 Xfce。

thunar无法解压文件

安装 xarchiver包

Parole无法播放视频

安装 gst-libav包

thunar界面不为中文

通常发生在用户使用“为图形界面配置中文 locale”本地化方案出现，在 `~/.xprofile`、`~/.xinitrc` 文件追加

```
dbus-update-activation-environment --systemd LANG
```

另见

- Xfce - 文档
- Xfce - Wiki
- Xfce - Tour
- Wikipedia:Xfce
- Xfce-Look - 主题，壁纸等
- Xfce Wikia

动态窗口管理器

本章解决的是选定并调好一个动态窗口管理器：dwm、Awesome、i3 三节各自给出安装、启动、配置与日常用法，并给出状态条、键盘映射的改法与出问题时的排查入口。

i3 一节最完整，使用、键盘映射与配置建议逐段读完；Awesome 的安装、启动与配置也应细读，小技巧与额外链接可先跳过；dwm 篇幅最短，基本用法可速览，状态条与扩展使用按需查阅。最容易踩的坑是以为装上就是一个完整桌面——三者都要自己启动、配置状态条与键位；改 Awesome 配置后起不来也常见，故障排除一节正为此准备。

17.1　Dwm

难度 进阶 | **篇幅** 12.3 KB | **所属** 动态窗口管理器

本节将讲到

- 安装
- 启动dwm
- 配置状态条
- 基本用法
- 扩展使用

dwm是X下的一个动态窗口管理器。它用平铺的、栈式的和全屏的布局方式，借助一些可选的补丁还可以实现其他的布局。布局可以动态得改变，为程序提供最优的环境和性能。dwm特别轻量快速，用C语言编写，被设计的目标是控制在2000行以下的代码。在xrandr和Xinerama支持下可实现multi-head。

安装

dwm可以通过dwmAUR或dwm-gitAUR安装，可在编译安装前配置所需的更改，参阅makepkg。

注意：也可遵循上游说明，但pacman将无法对其进行跟踪。

配置

dwm通过在编译时修改它的配置文件的一部分源代码来进行配置，特别是 `config.h`。有关这些配置的详细信息，请参阅包含的、注释良好的 `config.def.h` 以及网站上的自定义部分。

官方网站上有一些补丁可以为dwm添加额外功能。这些补丁主要对 `dwm.c`文件进行更改，但也会在适当的情况下对 `config.h`文件进行更改。有关应用修补程序的信息，请参阅Patching packages一文。

启动dwm

从所选display manager的菜单中选择“Dwm”。或者，用startx将 `exec dwm`附加到`~/.xinitrc`并预先编写其他程序来执行它们，例如：

```
redshift -O3500; xset r rate 300 50; exec dwm
```

配置状态条

dwm使用根窗口名称来显示状态条信息，可以使用`xsetroot -name`来改变。

基本状态条

这个例子显示ISO 8601格式的日期。把下边的内容添加到`~/.xinitrc`或者`~/.Xclients`，或者可以从GDM-3的讨论页中获得更多细节：

```
while true; do
   xsetroot -name "$( date +"%F %R" )"
   sleep 1m    # Update time every minute
done &
exec dwm
```

这是个笔记本使用的，依赖acpi包包来显示电池信息：

```
while true ; do
    xsetroot -name "$( acpi -b | awk '{ print $3, $4 }' | tr -d ',' )"
    sleep 1m
done &
exec dwm
```

这个脚本显示电池剩余电量和充电状态，用awk命令过滤acpi输出的无用信息，然后用tr去除逗号。

另一个方式是根据电池的状态选择性地输出信息：

```
while true; do
        batt=$(LC_ALL=C acpi -b)

        case $batt in
        *Discharging*)
                batt="${batt#* * * }"
                batt="${batt%%, *} "
                ;;
        *)
                batt=""
                ;;
        esac

        xsetroot -name "$batt$(date +%R)"

        sleep 60
done &

exec dwm
```

最后，要确保在`~/.xinitrc`或者`~/.Xclients`里只有一个dwm实例, 所以把整合起来应该像这样：

```
~/.setbg
autocutsel &
termirssi &
```

```
urxvt &

while true; do
   xsetroot -name "$(date +"%F %R")"
   sleep 1m    # Update time every minute
done &
exec dwm
```

这是另一个显示alsa音量和电池状态的例子。它一直显示到系统退出为止。

```
#set statusbar
while true
do
   if acpi -a | grep off-line > /dev/null; then
       xsetroot -name "Bat. $( acpi -b | awk '{ print $4 " " $5 }' | tr -d ',' ) | Vol.
$(amixer get Master | tail -1 | awk '{ print $5}' | tr -d '[]') | $(date +"%a, %b %d
%R")"
   else
       xsetroot -name "Vol. $(amixer get Master | tail -1 | awk '{ print $5}' | tr -d
'[]') | $(date +"%a, %b %d %R")"
   fi
   sleep 1s
done &
```

Conky状态条

Conky可以使用`xsetroot -name`来往状态条里输出信息:

```
conky | while read -r; do xsetroot -name "$REPLY"; done &
exec dwm
```

要想这样做，conky需要只往终端里输出文本。这是个应用于dual core CPU的简单conkyrc，显示几个状态信息：

```
out_to_console yes
out_to_x no
background no
update_interval 2
total_run_times 0
use_spacer none

TEXT
$mpd_smart :: ${cpu cpu1}% / ${cpu cpu2}%  ${loadavg 1} ${loadavg 2 3} :: ${acpitemp}
c :: $memperc% ($mem) :: ${downspeed eth0}K/s ${upspeed eth0}K/s :: ${time %a %b %d %I:
%M%P}
```

状态条颜色

可以在`config.def.h`中更改，然后复制到`config.h`中。

```
static const char col_gray1[]       = "#000000"; #状态条底色
static const char col_gray2[]       = "#FFFFFF"; #当static const unsigned int borderpx不
为0时，非活动窗口外边框颜色
static const char col_gray3[]       = "#39C5BB"; #当前非活动的title字体颜色
static const char col_gray4[]       = "#7FFF00"; #当前活动的title字体颜色
static const char col_cyan[]        = "#696969"; #title底色
```

static const unsigned int borderpx也是`config.def.h`的选项，不为0时，会在窗口边缘产生边框

基本用法

使用dmenu

Dmenu是dwm的一个有用的扩展。它不是一个单独的列表式菜单，而是生成一个可执行文件的列表并根据输入进行自动补全。比起许多程序启动器，它能与dwm更好地整合。

可以按 `Mod1 + P` 来启动Dmenu（`Mod1` 缺省是 `Alt` ）。当然你也可以按自己的喜好改变它。然后只需要在上边出现的工具条中输入你想运行的程序的前几个字母，也可以按左右箭头在进行选择，按回车键完成。

更详细的信息请查阅 dmenu.

控制窗口

将窗口移动到另一个tag

将当前的活动窗口移到其他的标签页：`Shift + Mod1 + x`, 其中的x是其他的标签页的序号，如果x是5，则表示将当前的活动窗口移到第5号标签页上去。`Mod1` 缺省是 `Alt` 键，这个键值可以在config.h中定义。

关闭窗口

关闭当前的活动窗口： `Shift + Mod1 + C`.

窗口布局

dwm缺省工作在平铺模式。当新窗口不断出现在同一个标签页时，窗口会越来越小。所有窗口会占满整个屏幕（除了目录条）。然而还有其他两种模式：浮动和单页模式。浮动模式对非平铺窗口管理器用户来说更熟悉，它允许用户重新按自己需要摆放窗口。单页模式会让一个窗口在最上边。

要切换到浮动模式，只需要按`Mod1 + F`。`Mod1`缺省是 `Alt`。如果你看到标签页右上角有X>这样的标志，就进行了浮动模式。

切换到单页模式，按 `Mod1 + M`。检查上否在单页模式，你会看到\[M\]标志（如果当前标签页无窗口），或者\[n\]（n是打开窗口的编号）。

回来平铺模式，按`Mod1 + T`，你会看到 \[\]= 这样的标志。

退出dwm

退出dwm（登出）： `Shift + Mod1 + Q`.

官方关于默认的快捷键的说明: dwm tutorial.

再次说明，快捷键可以根据自己的喜好自由定义，本维基中的快捷键都是指官方默认的快捷键。

扩展使用

补丁和增加窗口布局（tiling modes）

DWM官网上有很多可以给DWM添加功能的补丁， 用户可以通过补丁定制DWM。

bottomstack补丁能够使DWM由原来的垂直布局变为水平布局，而gapless grid补丁则可使窗口变为网格状布局.

实现为每个标签定制布局

dwm的缺省行为是将当前选定的布局应用到所有标签上。如果想为每个标签定制布局可以打pertag这个补丁。

解决模拟终端窗口缝隙问题

如果你发现模拟终端（例如xterm，Urxvt）的窗口占不满屏幕，这是因为其窗口的大小和字体有关。你可以试图调整字体大小直到恰好合适为止（这也许是很困难的），或者只需要把`config.h` 文件中的 `resizehints` 设置为 False：

```
static Bool resizehints = False; /* False means respect size hints in tiled resizals */
```

这样dwm会忽略所有窗口的改变大小的请求，不只是模拟终端的。这样的缺点是一些模拟终端可能会刷新异常，像显示一些错误的内容。

在不登出和退出程序程序的情况下重启dwm

如果你要在在线重启dwm（不关闭它以及其他应用程序），修改启动脚本，并让dwm在一个while循环中运行，像这样：

```
while true; do
    # Log stderror to a file
    dwm 2> ~/.dwm.log
    # No error logging
    #dwm >/dev/null 2>&1
done
```

这样就可以用Mod-Shift-Q快捷键来实现重启。

把上边的内容写到一个其他的文件（例如`~/bin/startdwm`）是一个好主意。然后在 `~/.xinitrc` 文件里添加（exec startdwm）。这样我们可以使用 `killall startdwm` 来真正地退出X会话，或者绑定到一个方便的快捷键上。

把右Alt键作为Mod4（Win键）使用

当把Mod4（Win键）作为 `MODKEY` 时，你可能希望右Alt键也可以作为Mod4，这样就两个手都可以按到了

首先，找到右Alt键对应的键码：

```
xmodmap -pke | grep Alt_R
```

然后只需要在启动脚本（比如 `~/.xinitrc`）添加如下内容, 把 *113* 换成之前 `xmodmap` 的结果：

```
xmodmap -e "keycode 113 = Super_L"  # reassign Alt_R to Super_L
xmodmap -e "remove mod1 = Super_L"  # make sure X keeps it out of the mod1 group
```

现在，右Alt键就和Mod4键一样了。

防止鼠标下的程序自动获取焦点

为了防止鼠标下的程序自动获取焦点，可以注释掉dwm.c中的如下代码：

```
[EnterNotify] = enternotify,
```

添加自定义的快捷键绑定

`config.h` 文件中的两个地方涉及到创建修改快捷键，一个是"/\ \/" 注释中，并一个是"static Key keys\[\] = {"语句中。

```
static const char *<keybindname>[]    = { "<command>", "<flags>", "<arguments>", NULL };
```

<keybindname> 可以是任何东西， <command> <-flags> and <arguments> 也是，但要用双引号括起来。

```
{ MODKEY,             XK_<key>,       spawn,          {.v = <keybindname> } },
```

这会绑定 Mod+<key> 来执行之前定义的命令。

```
{ MODKEY|ShiftMask,  XK_<key>,       spawn,          {.v = <keybindname> } },
```

这会绑定 Mod+Shift+<key> ，使用要用Ctrl键是 ControlMask 。

单个按键像Fn或者多媒体键必须要用16进制数来表示，可以用xev程序来获得。或者查看 /usr/include/X11/XF86keysym.h 中的定义。

```
{ 0,                  0xff00,     spawn,        {.v = <keybindname> } },
```

这会把0xff00键绑定到<keybindname>。

解决Java程序不正常的问题

对于使用JRE 6u20的Java程序在dwm中可能表现异常，因为dwm不是Java的一个已知的窗口管理器，这会导致一些像鼠标释放使菜单消失之类的小问题。可以先安装里的 \[community\] 里的wmname：

```
# pacman -S wmname
```

然后你只需要用wmname来设置一个Java能识别的WM名称：

```
$ wmname LG3D
```

这不是永久的，所以你可以把它写进.xinitrc。

资源

- dwm's official website

- dmenu - Simple application launcher from the developers of dwm
- The dwm thread on the forums
- Hacking dwm thread
- Check out the forums' wallpaper thread for a selection of dwm wallpapers
- HowTo by Snake
- Moved to dwm

17.2 Awesome

难度 高级 | **篇幅** 30.8 KB | **所属** 动态窗口管理器

本节将讲到

- 安装
- 启动
- 配置
- 小技巧
- 故障排除

来自awesome)网站：

Awesome 是 Xorg 下可高度定制的新一代窗口管理器，运行快捷、扩展性强。
Awesome主要面向高级用户、开发者和那些希望完美控制自己电脑的图形界面的人。

安装

安装位于官方软件仓库的软件包 awesome包。

注意：自2019年起，awesome 就没有新的正式版本发布了。如果你遇到了任何问题或者想要缺失的特性，安装 awesome-gitAUR。

启动

使用 xinit 运行 `awesome`。要使用其包括的 xsession 文件，参考显示管理器页面。

与GNOME一同使用

你可以设置 GNOME 使用 awesome 作为其窗口管理器，但必须让 GNOME 在后台运行。详见 awesome-gnomeAUR。

XFCE

详见 XFCE#使用不同的窗口管理器。

配置

配置文件是一个lua脚本：`~/.config/awesome/rc.lua`。

创建配置文件

创建配置文件所在的文件夹：

```
$ mkdir -p ~/.config/awesome/
```

编译后，awesome会自动使用~/.config/awesome/rc.lua里的所有配置。这个文件并不会自动创建，所以我们先要从模板复制一个过来：

```
$ cp /etc/xdg/awesome/rc.lua ~/.config/awesome
```

配置文件的语法会随着Awesome的版本升级而变化，所以当你升级了之后遇到问题时，重复上面的步骤，或者你得手动修改配置文件。

要获得关于配置Awesome的更多信息，请看Awesome wiki的配置部分。

例子

注意：Awesome配置文件语法有时会变化，所以你可能得动手修改下载的配置文件。

- Awesome screenshot thread
- Setkeh 的awesome 配置
- 支持不同主题的用户配置，包括一个状态栏
- 有两个现代主题的 awesome 配置

扩展

下列扩展对 awesome 来说是可用的：

E	功能	版本
 - Revelation		

- Shifty
- Naughty
- Vicious
- Obvious
- Bashets

自动启动

要实现 XDG 自动启动规范，安装 xorg-xrdb包和 dex包，并将以下行添加到 `~/.config/awesome/rc.lua`：

```
~/.config/awesome/rc.lua
awful.spawn.with_shell(
    'if (xrdb -query | grep -q "^awesome\\.started:\\s*true$"); then exit; fi;' ..
    'xrdb -merge <<< "awesome.started:true";' ..
    'dex --environment Awesome --autostart --search-paths "${XDG_CONFIG_HOME:-
$HOME/.config}/autostart:${XDG_CONFIG_DIRS:-/etc/xdg}/autostart";'
    )
```

尤其是对于没有托盘图标的守护进程，使用自动启动目标的 systemd 用户单元是一个有趣的替代方案，因为它们可以被配置为保持运行状态，即被监控并在意外停止后自动重启。

最后, 也可以创建 autorun.sh 然后插入下列文字：

```
~/.config/awesome/autorun.sh
#!/bin/sh

run() {
  if ! pgrep -f "$1" ;
  then
    "$@"&
  fi
```

然后，让它可执行。

要添加自启动程序, 只需要向 autorun.sh 添加 run "program [some arguments]"。 run 函数检查是否已经有一个带着一些参数的 program 实例，并且只在没有该实例时运行 program。你可以通过运行你的 autorun.sh 来检查它：

```
$ ~/.config/awesome/autorun.sh
```

如果配置成功， 将以下行添加到 rc.lua：

```
~/.config/awesome/rc.lua
awful.spawn.with_shell("~/.config/awesome/autorun.sh")
```

或者，添加你希望运行的命令（包括设置环境变量）到 xinitrc（或 xprofile，当使用显示管理器时）。

改变键盘布局

有许多方法配置键盘布局。

setxkbmap 配置方法

Awesome的默认配置已经激活了键盘布局 widget。

要暂时设置多键盘布局，运行

```
$ setxkbmap -layout "us,de"
```

点击 widget 将改变键盘布局。如果你想要一个组合键来改变键盘布局，你可用加上 -option "grp:alt_shift_toggle"。这是一个通过按 Shift+Alt 改变键盘布局的例子，完整的命令如下：

```
$ setxkbmap -layout "us,de" -option "grp:alt_shift_toggle"
```

设置带有音标的英语和俄语键盘布局：

```
$ setxkbmap -model pc105 -option "grp:shifts_toggle,compose:sclk"
"us,ru(phonetic_YAZHERTY)"
```

其中

- Compose key is 设为 "Scroll Lock"，语言切换键设为左 Shift + 右 Shift。
- 键盘上有105个键。

如果使用 LightDM 开启会话，添加上述命令到 .xprofile 文件。

rc.lua 配置方法 (自 v.4 版本开始可用)

你可以使用 awesome 自身来切换键盘布局 (自 v.4 版本开始可用)。要通过按 `Shift+Alt` 改变键盘布局， 添加这两行到 `globalkeys`:

```
~/.config/awesome/rc.lua
-- Key bindings
globalkeys = gears.table.join(
    -- ...
    awful.key({ "Shift" }, "Alt_L", function () mykeyboardlayout.next_layout(); end),
    awful.key({ "Mod1" }, "Shift_L", function () mykeyboardlayout.next_layout(); end)
)
```

这需要你提前通过 setxkbmap 命令或 X 配置文件设置你想要切换的键盘布局。

当你找到了设置你的键盘布局的适合方法，添加到 #自启动。

Xorg 配置方法

详见 Xorg/键盘配置（英语：Xorg/Keyboard configuration）。

主题

Beautiful 是一个允许你使用外部文件给 awesome 配置主题的 lua 库，可以让你非常容易地动态地改变背景图片和颜色主题，而不需要改变 `rc.lua`。

默认的主题文件在 `/usr/share/awesome/themes/default`。把它复制到 `~/.config/awesome/themes/default`（可选复制其他所有主题）然后修改一下 `rc.lua` 中的 `theme_path`。

```
 -- beautiful.init(gears.filesystem.get_configuration_dir() ..
"/themes/default/theme.lua")
 local theme_path = string.format("%s/.config/awesome/themes/%s/theme.lua",
os.getenv("HOME"), "default")
 beautiful.init(theme_path)
```

如果你同时也复制了其他主题，你可以把 "default" 替换为 "sky"，"gtk" 或 "zenburn" 来容易地切换主题，并且本地主题文件可以被学习、修改和用于测试。额外的主题配置选项参考这里。例如要添加一个 useless gap， 添加

```
beautiful.useless_gap = 5
```

到你的 `rc.lua` 的主题配置部分。

注意：Awesome 不支持点阵字体，如果你的系统上只安装了点阵字体，字体渲染将损坏。

设置墙纸

Beautiful 可以设置墙纸，所以你就不用在 `.xinitrc` 或者 `.xsession` 中自己设置了。这允许你给每个主题配一个墙纸。

有两种方式设置墙纸：

- 在 `theme.lua` 中：

```
    -- 绝对路径
    theme.wallpaper = "/path/to/wallpaper.png"

    -- 相对于主题目录的相对路径
    theme.wallpaper = theme_path .. "path/to/wallpaper.png"
```

- 或在 `rc.lua` 中，位于 `beautiful.init` 和 `local function set_wallpaper` 之间：

```
    -- 绝对路径
    beautiful.wallpaper = "/path/to/wallpaper.png"

    -- 相对于主题目录的相对路径
    beautiful.wallpaper = awful.util.get_configuration_dir() .. "path/to/wallpaper.png"
```

要设置随机墙纸，添加这个到 `rc.lua`（v3.5+）。要从一个特点目录自动获取墙纸，使用这个。

小技巧

显示/隐藏wibox

从 awesome 4.0 开始：

```
awful.key({ modkey }, "b",
          function ()
              myscreen = awful.screen.focused()
              myscreen.mywibox.visible = not myscreen.mywibox.visible
          end,
          {description = "toggle statusbar"}
),
```

在 awesome 中，只在按下 Modkey 时显示 wibox（或执行其他操作）是不可能的，但这里有一个 Python 脚本实现了这个功能：autohidewibox。

截图

设置键盘输入，确保 `PrtSc` 键被正确绑定了。然后安装一个截图工具，例如 scrot。

在配置文件中的 `globalkeys` 中输入并保存：

```
 awful.key({ }, "Print", function () awful.util.spawn("scrot -e 'mv $f ~/screenshots/ 2>/dev/null'") end),
```

这里的 `~/screenshots/` 可以改成你截图想要保存的地方。

移除窗口间隔

自 awesome 3.4 起，移除创建之间的小间隔是可能的。在 *awful.rules.rules* 表格中的 *properties* 章节添加：

```
 size_hints_honor = false
```

透明度

参见合成器。

在 awesome 3.5 中， 窗口透明度可以用信号动态设置。例如，`rc.lua` 可以包含下列内容：

```
client.add_signal("focus", function(c)
                               c.border_color = beautiful.border_focus
                               c.opacity = 1
                           end)
client.add_signal("unfocus", function(c)
                                 c.border_color = beautiful.border_normal
                                 c.opacity = 0.7
                             end)
```

Conky

**本文或本章节可能需要合并到Conky。**

当使用 conky 时，你必须设置它让它创建自己的窗口，而不是使用桌面。为此，编辑 `~/.conkyrc`以包含

```
own_window yes
own_window_transparent yes
own_window_type desktop
```

否则你可能会看到奇怪的表现，比如所有的窗口都变得完全透明。注意由于 conky 将会在你的桌面上创建一个透明窗口，所有定义于 `rc.lua` 的动作在 conky 所在的位置都不会工作。

wiboxes

Wiboxes具有内置的伪透明功能。要启用它，在主题文件中的颜色后附加2位十六进制数字（例如`~/.config/awesome/themes/default/theme.lua`, 通常为 `/usr/share/awesome/themes/default/theme.lua` 的复制品）， 如下所示:

```
theme.bg_normal = "#000000AA"
```

其中 "AA" 为透明度。

To change transparency for the actual selected window by pressing `Modkey + PgUp/PgDown` you can also use transset-dfAUR and the following modification to your `rc.lua`:

```
globalkeys = gears.table.join(
    -- your keybindings
    [...]
    awful.key({ modkey }, "Next", function (c)
        awful.util.spawn("transset-df --actual --inc 0.1")
    end),
    awful.key({ modkey }, "Prior", function (c)
        awful.util.spawn("transset-df --actual --dec 0.1")
    end),
    -- Your other key bindings
    [...]
)
```

Widget 间隔

默认 `rc.lua` 在一个 wibox 里放置包括键盘布局和时钟等所有 widgets，widgets 之间只有很小的间隔。使用 `spacing` 属性在 widgets 之间增加额外的间隔是可能的：

```
        { -- Right widgets
        layout = wibox.layout.fixed.horizontal,
        spacing = 10,
        mykeyboardlayout,
        ...
```

ImageMagick

**本文或本章节可能需要合并到Xorg#合成。**

如果你用 ImageMagick 的 *display* 命令来设置你的墙纸，可能会遇到 xcompmgr 效果不好的问题。请注意 awsetbg 可能会用 *display* 如果它没有其他选项。安装 habak，feh，hsetroot 或者其他的包应该会解决这个问题。（*grep -A 1 wpsetters /usr/bin/awsetbg* 来看你有哪些选项）

使用 awesome-client 给文本插件传递信息

只需要创建一个新的插件，就可以很容易的传递信息。

```
mywidget = widget({ type = "textbox", name = "mywidget" })
 mywidget.text = "initial text"
```

使用 awesome-client 从外部更新“initial text":

```
echo -e 'mywidget.text = "new text"' | awesome-client
```

不要忘记把插件增加到 wibox。

使用其他任务栏

如果你喜欢 awesome 既轻量又强大的功能，但又不喜欢默认那个任务栏的外观，你可以安装其他的，比如 xfce4-panel包。

然后要把它添加到配置文件 `rc.lua` 的自动启动部分，你可以注释掉配置文件中给每个桌面创建 wiboxes 的那部分（开头是`mywibox[s] = awful.wibox({ position = "top", screen = s })`），但不是必要的。检查配置文件没有错误之后就可以执行命令生效:

```
$ awesome -k rc.lua
```

另外你还需要改变 `modkey+R` 的快捷键绑定，来用一些其他的应用启动器替代awesome自带的启动器。别忘了添加

```
properties = { floating = true } },
    { rule = { instance = "$yourapplicationlauncher" },
```

到你的 `rc.lua`。

菜单栏中的应用程序目录

Awesome 软件包含有 menubar（默认情况下，按下 `Mod+p` 会在屏幕上方打开一个类似于 dmenu 的应用程序菜单）。它搜索位于`$XDG_DATA_DIRS/applications` 及 `$XDG_DATA_HOME/applications/` 目录下的 .desktop 文件。

通过修改 `menubar.menu_gen.all_menu_dirs`，你可以扩充或取代这些目录：

```
~/.config/awesome/rc.lua
table.insert(menubar.menu_gen.all_menu_dirs, "/path/to/custom/applications/")
```

注意每次 awesome 启动都会重新读取 `.desktop` 文件，因此文件过多会拖慢 Awesome 的启动速度。如果你更喜欢使用其他方式来运行程序，可以通过在 `rc.lua` 移除 `local menubar = require("menubar")` 及其它涉及到 `menubar` 的变量来禁用菜单栏。

应用菜单

自 awesome 3 以来，默认提供了一个简单的菜单，简化了自定义菜单。如果你想要一个freedesktop.org的菜单，可以查看awesome-freedesktop。

如果想要在点击Awesome图标，或者在桌面空白处右键点击时，看到较传统模式的应用菜单，可以参照Xdg-menu#Awesome的说明。但应用菜单不会在安装或卸载程序时进行更新。所以，确保运行类似如下的命令，更新应用菜单：

```
 xdg_menu --format awesome --root-menu /etc/xdg/menus/arch-applications.menu
>~/.config/awesome/archmenu.lua
```

标题栏

你可以很容易地在配置文件中rule把 `titlebars_enabled` 设置为 true 来启用标题栏。

```
   { rule_any = {type = { "normal", "dialog" }
     }, properties = { titlebars_enabled = true }
   },
```

如果想要切换标题栏的显示与否，可以把以下代码加入配置文件中键位绑定部份的`clientkeys`部份中间位置（因为这段代码是以逗号结尾，请不要放在 `clientkeys` 的最后），然后通过按 `modkey + Ctrl + t` 来切换。

```
   -- working toggle titlebar
   awful.key({ modkey, "Control" }, "t", function (c) awful.titlebar.toggle(c)
end,
              {description = "Show/Hide Titlebars", group="client"}),
```

如果想默认情况下隐藏标题栏，仅需要在配置文件中标题栏创建后（即在 `"manage"` 信号处理器内）加入以下代码：

```
awful.titlebar.hide(c)
```

电池通知

参考这篇博文以向 `rc.lua` 添加一个简单的电池通知。注意它需要 *naughty* 用来通知（3.5 版本起默认安装）。Awesome wiki 上也有其他的参考案例。

媒体控制

使用 amixer(1)（通过 alsa-utils包 安装）和 playerctl包 的组合控制音量和媒体播放是可能的。下列代码可以被添加到你的 `rc.lua` 配置文件中与快捷键相关的章节：

```
    -- Volume Keys
    awful.key({}, "XF86AudioLowerVolume", function ()
      awful.util.spawn("amixer -q -D pulse sset Master 5%-", false) end),
    awful.key({}, "XF86AudioRaiseVolume", function ()
      awful.util.spawn("amixer -q -D pulse sset Master 5%+", false) end),
    awful.key({}, "XF86AudioMute", function ()
      awful.util.spawn("amixer -D pulse set Master 1+ toggle", false) end),
    -- Media Keys
    awful.key({}, "XF86AudioPlay", function()
      awful.util.spawn("playerctl play-pause", false) end),
    awful.key({}, "XF86AudioNext", function()
      awful.util.spawn("playerctl next", false) end),
    awful.key({}, "XF86AudioPrev", function()
      awful.util.spawn("playerctl previous", false) end),
```

Steam 键盘

可通过 Steam 控制器激活的屏幕Steam键盘在尝试输入一个字符后似乎会冻结。这是因为本应接收输入的客户端必须处于焦点状态才能接收输入，而键盘会等待输入成功发送。手动将焦点切换到另一个客户端会将输入发送到此客户端，并再次解冻键盘，直到输入下一个字符。

让键盘正常工作的诀窍在于防止它获得焦点。请将以下信号添加到您的配置中（或与现有的客户端焦点信号合并）：

```
client.connect_signal("focus", function(c)
    if awful.rules.match(c, { name = "^Steam Keyboard$" }) then
        awful.client.focus.history.previous()
    end
end)
```

每当键盘获得焦点时，焦点将返回至上一个客户端。由于键盘输入是由 Steam 客户端处理的，因此不需要焦点，现在输入文本将正常工作。

故障排除

调试 rc.lua

Xephyr 允许你在一个 X 客户端窗口内运行另一个 X。用这种方式可以在不破坏现有桌面的情况下对 `rc.lua` 进行测试。首先把 `rc.lua` 复制到一个新文件（例如 `rc.lua.new`），接着进行修改。然后在 Xephyr 中运行新的 awesome 实例，使用 `rc.lua.new` 作为配置文件，就像这样：

```
$ Xephyr -ac -br -noreset -screen 1152x720 :1 &
$ DISPLAY=:1.0 awesome -c ~/.config/awesome/rc.lua.new
```

这种方式的巨大优势在于如果你弄坏了 `rc.lua.new`，你不至于把现有的 awesome 桌面弄得一团糟，并且很可能还会把 XWindow 弄崩溃了，没保存的工作全部丢失。一旦你觉得新的配置文件不错，就用 `rc.lua.new` 代替 `rc.lua`，然后重启 awesome。

自动重载

作为上述方法的补充，你可以使用 inotify-tools包 以在更新 `~/.config/awesome` 目录下的配置文件时，在 Xephyr 内自动重载 awesome 实例：

```
#!/usr/bin/env bash

Xephyr :1 -ac -br -noreset -screen 1920x1080 -dpi 96 &
sleep 1
DISPLAY=:1.0 awesome -c ~/.config/awesome/rc.lua.new &
instance=$!

while inotifywait -r -e close_write ~/.config/awesome; do
       kill -s SIGHUP $instance
done
```

使用 awmtt

awmttAUR (Awesome WM Testing Tool) 是一个基于 Xephyr 的易于使用的脚本。默认情况下，它会测试 `~/.config/awesome/rc.lua.test` 。如果该文件不存在，它会测试当前使用的 rc.lua 。也可以指定要测试的配置文件所在路径：

```
$ awmtt start -C ~/.config/awesome/rc.lua.new
```

当测试完成后，使用以下命令关闭窗口:

```
$ awmtt stop
```

或者通过以下命令立即查看变化:

```
$ awmtt restart
```

aawmtt

aawmttAUR (Another Awesome WM Testing Tool) 是 awmttAUR 的另一种实现，默认包含Live-Reload功能。它与 awmtt-ng 类似，但针对 Xorg 显示检测问题做了一些修复，因为 awmtt-ng 在某些机器上无法正常检测 Xorg 显示。它与 awmtt 的不同之处在于，它不会先尝试运行测试文件，而是直接运行默认配置。

要打开一个已加载 awesome 的 Xephyr 窗口，请运行：

```
$ aawmtt
```

现在，awesome 的输出将打印到您的终端，并且当您更改配置文件夹中的任何文件时，awesome 将重新加载。

如果你想修改被监视以检测文件更改的目录，或者配置文件的位置，只需运行：

```
$ aawmtt --config "location_of_config_file" --watch "directory_to_watch_for_changes"
```

默认情况下，会被监视以检测更改的目录是配置文件的父目录，因此默认情况下，该目录将是“~/.config/awesome”。

日志文件

如果你正在使用 LightDM, awesome 会将日志记录到 \$HOME/.xsession-errors\。如果你使用 .xinitrc 启动 awesome， awesome FAQ 中的 "Where are logs, error messages or something?" 也许是有用的资源。

Mod4 键

**本文或本章节可能需要合并到Xorg/键盘配置。**

Awesome建议重新映射 mod4 键，该键默认应为超级键或“Windows”键。如果由于某种原因它未被映射到 mod4，请使用 xmodmap 来找出其映射情况。要更改映射，请使用 xev 来查找要映射的键的键码和名称。然后在 ~/.xinitrc 中添加类似以下的内容

```
xmodmap -e "keycode 115 = Super_L" -e "add mod4 = Super_L"
exec awesome
```

此案例中的问题是，某些 Xorg 安装能够识别键码115，但错误地将其识别为“选择”键。上述命令明确地将键码115重新映射到正确的“Super_L”键。

要使用 setxkbmap 重新映射 mod4（与 xmodmap 冲突）参见：

```
tail -50 /usr/share/X11/xkb/rules/evdev
```

要将 caps lock 键设为 mod4，添加下列内容到 ~/.xinitrc：

```
setxkbmap -option caps:hyper
```

修复 Java (GUI 只显示灰色)

参见 Java#灰色窗口/应用不随窗口管理器调整大小/菜单自动关闭和这篇文档。

Eclipse: 不能调整或移动主窗口

如果你遇到卡顿，无法移动或调整主窗口大小（使用 mod4 键 + 鼠标左/右键），请编辑 workbench.xml 文件，将 fullscreen/maximized 设置为 false（如果已设置），并将宽度和高度减小到小于你的单屏桌面区域的值。

注意：workbench.xml 可以在这个位置找到：<eclipse_workspace>/.metadata/.plugins/org.eclipse.ui.workbench/ 需要编辑的行是 <window height="xx" maximized="true" width="xx" x="xx" y="xx">.

Netbeans: 代码预测出现在错误的屏幕上

如果你有两个显示器，同时使用 Netbeans 的代码预测（Ctrl + Space），代码预测可能会出现在错误的屏幕。下列代码能修复问题：

```
.config/awesome/rc.lua
awful.rules.rules = {
        ...
       {
              rule_matches = { -- Fix Netbeans
                     class = {
                            "sun-awt-X11-XWindowPeer", "NetBeans IDE 8.2"
                     },
                     name = {
                            "win1"
                     }
              }, properties = { screen = 1 } -- even with screen 1 here, this still
works on the seccond screen, too (do not know why).
       },
        ...
```

IntelliJ: 菜单出现在错误的位置，一些窗口打不开

参见 GitHub issue #2204。下列代码能解决问题：

```
.config/awesome/rc.lua
clientbuttons_jetbrains = gears.table.join(
    awful.button({ modkey }, 1, awful.mouse.client.move),
    awful.button({ modkey }, 3, awful.mouse.client.resize)
)

...

awful.rules.rules = {
        ...
       {
            rule = {
                class = "jetbrains-.*",
            }, properties = { focus = true, buttons = clientbuttons_jetbrains }
        },
        {
            rule = {
                class = "jetbrains-.*",
                name = "win.*"
            }, properties = { titlebars_enabled = false, focusable = false, focus =
true, floating = true, placement = awful.placement.restore }
        },
        ...
}
```

scrot: 无法使用键盘快捷键进行鼠标选区的截图

使用 scrot 时，在为鼠标选择选项分配键盘快捷键时，你可能会遇到问题（形式上是 `scrot -s`）。要修复它，向你的 `rc.lua` 添加下列内容：

```
awful.key( { modkey,  }, <shortcut>, nil, function () awful.spawn("scrot -s") end)
```

注意 `nil` 被传送到 `awful.key` 的 `press` 参数。相反，回调函数作为第四个参数传递，该参数名为 `release`。.

YouTube: 全屏出现在背景

参见此处。如果在全屏模式下，YouTube 视频出现在你的浏览器下层，添加这些到你的 `rc.lua`：

```
   { rule = { instance = "plugin-container" },
     properties = { floating = true } },
```

如果你使用 Chromium 浏览器，添加：

```
   { rule = { instance = "exe" },
```

```
    properties = { floating = true } },
```

或：

```
{ rule = { role = "_NET_WM_STATE_FULLSCREEN" },
  properties = { floating = true } },
```

参见此处。

防止鼠标滚轮更改标签

在你的 `rc.lua` 中，修改鼠标绑定为下列内容：

```
-- {{{ Mouse bindings
root.buttons(gears.table.join(
    awful.button({ }, 3, function () mymainmenu:toggle() end)
))
-- }}}
```

Xdg-menu 生成重复菜单项

如果你将 /usr/share/applications 中的桌面文件复制到 ~/.local/share/applications，Xdg-menu将会生成重复条目，尽管直接覆盖原始文件可能更可取，例如为特定应用程序使用不同的主题。解决这个问题的一种方法是使用awk过滤生成的输出，以删除与前一条目名称相同的条目。

```
 xdg_menu --format awesome --root-menu /etc/xdg/menus/arch-applications.menu | awk -F,
'{if (a!=$1) print $a; a=$1}' >~/.config/awesome/archmenu.lua
```

Xfce4 中部分快捷键因按键重叠而失效

检查你的

```
$ xfce4-keyboard-settings
```

关于像“Super L”这样的重叠键或应由 awesome 运行的组合键。

内存泄漏

有些用户即使没有进行任何操作也会遇到内存泄漏问题。当使用大量小部件时，内存泄漏的速度可能高达每分钟5 MB。为了缓解这一问题，您可以通过在 `~/.config/awesome/rc.lua` 文件中添加以下内容来强制执行更频繁的垃圾收集：

```
-- Run garbage collector regularly to prevent memory leaks
gears.timer {
       timeout = 30,
       autostart = true,
       callback = function() collectgarbage() end
}
```

额外链接

- https://awesomewm.org/apidoc/documentation/90-FAQ.md.html - FAQ
- https://www.lua.org/pil/ - Lua编程指南（第一版）
- https://awesomewm.org/ - awesome 官方网站
- https://bbs.archlinux.org/viewtopic.php?id=88926 - 分享你的awesome！

- https://web.archive.org/web/20111222025635/http://www.penguinsightings.org/desktop/awesome/ - 一条评论
- https://web.archive.org/web/20120427051008/http://compsoc.tardis.ed.ac.uk/wiki/AwesomeWM_guide - Awesome 指南

17.3 I3

难度 高级 | **篇幅** 33.3 KB | **所属** 动态窗口管理器

本节将讲到

- 安装
- 启动
- 使用
- 键盘映射
- 配置

i3 是一种动态的平铺式窗口管理器，其灵感来自于面向开发者与资深用户的 wmii。

i3 的既定目标包括清晰可读的文档，完善的多显示器支持，基于树形结构的窗口管理，提供 vim 式的多种操作模式。

安装

通过安装i3-wm包软件包来安装*i3*.

也可以安装i3包组软件包组。它包括了窗口管理器, 一个锁屏器和两个生成状态栏并通过stdout与i3bar交互的程序。

注意：i3-gaps(是*i3*的一个分支，具有间隙和其他功能）最近被合并到i3-wm包中，因此不再可用。请改为安装i3-wm包

启动

从tty启动

使用 xinit 来启动`i3`

从显示管理器启动

i3-wm包 软件包包括了 `i3.desktop`，可以支持将 i3 作为 Xsession 启动。`i3-with-shmlog.desktop` 这个入口则附加了日志功能（对于调试很有用）。i3-gnomeAUR软件包则把 `i3-wm` 集成到了 GNOME 中去。

使用

通过官方文档获取全面的信息，基本使用方法见i3 User's Guide。

键盘映射

在 *i3* 里，一切命令均以「修饰键」开头，即 `$mod`. 默认上来说是 Alt 键 `alt`(Mod1), 但开始键 `Super`(Mod4) 也更为广泛接受。Super 键往往带有 Windows 图标，在苹果键盘上则是 Command 键。

见 i3 偏好和 Using i3 以获取默认映射表。见 Keyboard 绑定以获取添加新快捷键的说明。

非Qwerty键盘布局的用户可能会希望跳过以下所说的“配置助手”。

如果在多个窗口管理器之间切换，可以考虑使用sxhkd或者其它environment agnostic程序来管理键盘映射。更多信息可在Keyboard shortcuts#Xorg中找到。

容器和布局

i3 以树形结构的方式管理窗口，容器是最小的单位。这种结构可以被水平或竖式分割。容器默认是平铺的，但也可以设置为标签或堆叠式布局。它同样也能设置为浮动式（适用于对话窗口）。浮动的窗口总是显示在顶部。

参见 i3 Tree 和 Containers and the tree data structure 以了解更多信息。

程序启动器

i3 使用 dmenu 作为首席程序启动器，键绑定默认为 `$Mod+d`。在使用它之前必须先安装可选依赖 dmenu包

i3-wm包 包含了 i3-dmenu-desktop——一个 "dmenu" 的Perl 包装器，它通过Desktop entries创建一个已安装应用的列表。 也可以使用j4-dmenu-desktop-gitAUR 作为替代。

rofi 是一个流行的dmenu替代品，它能列出更多的选项。

在KDE Plasma/i3中使用KRunner作为应用启动器

在KDE Plasma中使用i3也是可行的，参见 KDE#使用不同的窗口管理器.

当通过 `KDEWM=/usr/bin/i3`运行Plasma时, 你可以添加以下内容到i3的config配置中将`$mod+d`绑定到KRunner上来替代dmenu作为程序启动器：

```
~/.config/i3/config
set $menu --no-startup-id qdbus org.kde.krunner /App display
bindsym $mod+d exec $menu
```

配置

见配置i3 以获取更多细节。本文余下部分假设 i3 的配置文件位于 `~/.config/` 目录。

配置助手和可选键盘布局

当 i3 首次启动时，它会启动配置助手 *i3-config-wizard*。此工具会通过重写位于 `/etc/i3/config.keycodes` 的模板配置文件来创建 `~/.config/i3/config`。他会对默认模板造成两次修改。

1. 它会询问用户以选择默认的修饰建。它会在模板文件中添加一行，类似于 `set $mod Mod1`; 然后
2. 它会用用户设置的键盘布局相应的 *bindsyn* 行替换所有 *bindcode* 行。

第二步是设计用于确保 Qwerty 键盘上的四个导航键 `j`， `k`， `l` 和 `;` on a Qwerty keyboard会被映射在拥有相同位置的按键上，举例说，Dvorak 键盘上的 `h`， `t`， `n`， `s`。这个小戏法的副作用是 最多十五个按键会被以一种破话位置记忆的方式被映射 - 所以，对于 Dvorak 用户， “重启”被绑定于 `$mod1+p` 而不是 `$mod1+r`，“竖直分割”被绑定于 `$mod1+d` 而不是 `$mod1+h`,类似的还有更多。

因此，其他键盘布局的用户若是想要直截了当的，符合教程中给出的按键绑定的键盘绑定，可能更倾向于不去使用 "config wizard" 。这可以拷贝 `/etc/i3/config` 到 `~/.config/i3/config` （或 `~/.i3/config`），然后编辑此文件。

注意，用户也可以建立一份以键码为基础的配置。例如，对于那些经常切换键盘布局，但是想要 *i3* 的键盘绑定保持相同的用户。

自启动程序

添加一行 `exec` 命令到 `~/.config/i3/config` 文件中。

例如：

```
exec terminator
```

你也可以使用 XDG Autostart来设置自启动程序。

i3bar

除了显示工作区信息外，i3bar 也可以作为 i3status 或其替代品的输入。下一章节会对此进行详细描述。示例：

```
~/.config/i3/config
bar {
    output            LVDS1
    status_command    i3status
    position          top
```

```
    mode                hide
    workspace_buttons yes
    tray_output         none

    font -misc-fixed-medium-r-normal--13-120-75-75-C-70-iso10646-1

    colors {
        background #000000
        statusline #ffffff

        focused_workspace  #ffffff #285577
        active_workspace   #ffffff #333333
        inactive_workspace #888888 #222222
        urgent_workspace   #ffffff #900000
    }
}
```

更多细节，见设置i3bar 。

i3bar可选方案

一些用户可能更偏好于类似于常规桌面环境提供的面板。这可以通过在 *i3* 启动时运行面板程序达成。

例如, 要使用 Xfce 面板 (xfce4-panel包), 把下面这行内容放到 `~/.config/i3/config`中的任何位置：

```
exec --no-startup-id xfce4-panel --disable-wm-check
```

i3bar 可以通过注释掉 `~/.config/i3/config` 中的 `bar{ }` 段落禁用，或者定义一个按键以切换 i3bar 显示状态：

```
~/.config/i3/config
# bar toggle, hide or show
bindsym $mod+m bar mode toggle
```

独立于桌面环境的替代品有：

- **polybar** — 一个快速并且易用的创建状态栏的工具

: https://github.com/jaagr/polybar || polybar包

- **excalibar** — 用C写成的轻量可定制的状态栏

: https://github.com/cylgom/excalibar || excalibar-gitAUR

i3status

拷贝默认的配置文件到家目录：

```
$ cp /etc/i3status.conf ~/.config/i3status/config
```

不是所有的插件都被在默认配置文件中被定义，一些配置值对于系统也可能是无效的，所以需要相应的更新。具体见 i3status(1) 。

i3status 可选方案

- **conky** — 高度可扩展系统监视器。配合 i3 的使用方法见 这个教程

: https://github.com/brndnmtthws/conky || conky包

- **i3blocks** — 通过 shell 脚本扩展。 它可以处理点击事件，中断，和定义或更在块的基础上更新间隔。

: https://github.com/vivien/i3blocks || i3blocks包

- **i3pystatus** — 默认带有许多插件和选项的可扩展 Python 3 状态栏

: https://github.com/enkore/i3pystatus || i3pystatus-gitAUR

- **j4status** — 提供了状态栏，可以通过插件扩展，且是用 C 写成的。额外的插件来自j4status-plugins-gitAUR.

: https://sardemff7.github.io/j4status/ || j4status-gitAUR

- **goi3bar** — Go语言 写的 i3status 替代品。配置文件同时还有各种插件，并发选项和丰富的插件支持。

: https://github.com/denbeigh2000/goi3bar/ || goi3bar-gitAUR

- **goblocks** — Go语言 写的轻便快速的 i3status 替代品。

: https://github.com/davidscholberg/goblocks || goblocksAUR

- **bumblebee-status** — 多主题的Python状态栏生成器。

: https://github.com/tobi-wan-kenobi/bumblebee-status/ || bumblebee-statusAUR

- **ty3status** — Typescript编写的 i3status 替代品，

: https://github.com/mrkmg/ty3status || ty3status-gitAUR

- **i3status-rust** — 使用Rust编写的高效并且特性丰富的i3status替代品。能够处理push更新, 单独设置更新间隔, 支持主题和点击事件

: https://github.com/greshake/i3status-rust || i3status-rust包

i3status 包装器

- **i3cat** — 基于go语言做的包装器，它可以链接来自多个资源的输入，也可以处理鼠标操作来转发用户特定的信号到子程序。

: https://vincent-petithory.github.io/i3cat/ || i3cat-gitAUR

- **py3status** — 一个可扩展的基于Python的 i3status 包装器。

: https://github.com/ultrabug/py3status || py3status包

- **YaGoStatus** — Go编写的另一个i3status替代品。.

: https://github.com/burik666/yagostatus || yagostatus-gitAUR

状态栏中的图标字体

安装 i3-wm-iconpatchAUR来让状态栏支持图标。你也可以使用下面的图标字体。

- **ttf-font-awesome** — 可以通过 CSS定制的可缩放矩阵字体。显示了每个图像的 Unicode 值。

: https://fontawesome.com/ || ttf-font-awesome包

- **ttf-font-icons** — 也提供了全面的图像字符，包括彼此毫无重叠的 Awesome 和 Ionicons, 也很好地避免了 Awesome 与 DejaVu Sans 的微秒重叠。

: http://kageurufu.net/icons.pdf || ttf-font-iconsAUR.

- **ttf-ionicons** — Ionic Framework的独特图标

: https://ionicframework.com/docs/ionicons/ || ttf-ionicons包.

要配合这些字体使用，在配置文件中一种字体属性的后缀用,分割字体。示例：

```
~/.config/i3/config
bar {
    ...
    font pango:DejaVu Sans Mono, Icons 8
    ...
 }
```

依照 pango语法, 在逗号分割的多个字体的后面，字体大小只被设置一次，对每个字体都设置大小将会造成除了最后一个字体以外，其他的字体都被忽略。

在`~/.config/i3status/config` 中使用unicode数字添加图标。输入法将会在文本处理器间有所区分。 例如，插入心型图标(unicode 数字 f004):

- 在多个图形化的文本处理器(如 gedit, Leafpad) 和终端模拟器 (如 GNOME Terminal, xfce4-terminal)中: `Ctrl+Shift+u`, `f004`, `Enter`
- Emacs: `C-x`, `8`, `RET`, `f004`, `RET`
- Vim (在插入模式): `Ctrl+v`, `uf004`
- urxvt: 按住 `Ctrl+Shift`, 键入`f004`

终端模拟器

默认配置下按 `$mod+Enter` 会运行 `i3-sensible-terminal`这个查找终端模拟器程序的脚本.参见i3-sensible-terminal(1)了解它查找终端模拟器的顺序。

要想立即启动自定义的终端模拟器, 编辑`~/.config/i3/config`中的这一行:

```
bindsym $mod+Return exec i3-sensible-terminal
```

也可以通过设置 `$TERMINAL` environment variable达到同样效果.

禁用标题栏点击动作

添加 `bindsym button1 nop` 来禁止i3选中你点击的标题栏对应的窗口。当你的默认布局是标签式并且经常不小心误点到窗口标签时比较有用。

技巧与提示

跳转到打开的窗口

- **quickswitch-i3** — 一个可在 i3 的窗口之间快速跳转，定位的 Python 实现

: https://github.com/OliverUv/quickswitch-for-i3/ || quickswitch-i3AUR

- **i3-wm-scripts** — 用正则表达式在窗口之间进行搜索并跳转

: https://github.com/yiuin/i3-wm-scripts || 未被打包？在 AUR 里搜索

- **rofi** — Search and jump to open and scratchpad window

: https://github.com/davatorium/rofi || rofi包

- **i3-easyfocus** — Focus and select windows in *i3*

: https://github.com/cornerman/i3-easyfocus || i3-easyfocus-gitAUR

- **wmfocus** — Focus and select windows in *i3* and other window managers

: https://github.com/svenstaro/wmfocus || wmfocus包

- **i3-cycle-focus** — Provides an Alt-Tab functionality for *i3*

: https://github.com/acrisci/i3ipc-python/blob/master/examples/i3-cycle-focus.py || 未被打包？在 AUR 里搜索

- **i3-winmotion** — Focus and select visible windows in *i3*

: https://github.com/iiKoe/i3-winmotion || 未被打包？在 AUR 里搜索

跳转到急需处理的窗口

添加这行到 `~/.config/i3/config`:

```
bindsym $mod+x [urgent=latest] focus
```

保存和恢复窗口布局

从4.8版本起, *i3*能够保存和恢复工作区布局。 要实现这个需要Official repositories中的这些包： perl-anyevent-i3包 和 perl-json-xs包

注意： 这一节内容只提供如何保存和恢复一个单独工作区当前窗口布局的快速指导，参见 官方文档 以获取更多信息

保存一个工作区的当前窗口布局

要保存当前的窗口布局，按这几步做：

1. 首先, 在一个你喜欢的工作区内执行命令打开一些窗口，并根据需要调整它们的大小。 记下打开每个窗口所执行的命令
2. 现在, 切换到一个新的工作区, 打开终端然后执行:

```
    $ i3-save-tree --workspace N > ~/.config/i3/workspace_N.json
```

N是上一步的工作区号码。 这将会保存N号工作区的布局到这个文件中 `~/.config/i3/workspace_N.json`.

1. 这个新创建的文件需要被编辑, 但是你可以用这个命令来完成:

```
    $ sed -i 's|^\(\s*\)// "|\1"|g; /^\s*\/\//d' ~/.config/i3/workspace_N.json
```

恢复工作区的窗口布局

有两种方法恢复工作区的布局: 写一个脚本, 或者编辑 `~/.config/i3/config` 来自动载入布局. 这里只讨论第一种方法, 参考官方文档了解第二种方法怎么用。

要恢复在上一小节中保存的工作区布局, 写一个叫 `load_layout.sh`的文件， 它包含以下内容:

- The starting lines:

```
~/load_layout.sh
#!/bin/bash
i3-msg "workspace M; append_layout ~/.config/i3/workspace_N.json"
```

M是你想要载入布局的工作区编号，N是之前保存布局的那个工作区的编号

- 还有在之前你所使用的打开那些窗口的命令, 但它们必须被封闭在()内，并且在命令末尾加上&符号来让它后台运行

例如， 如果保存的布局包括3个 `uxterm` 窗口:

```
~/load_layout.sh
#!/bin/bash

# First we append the saved layout of workspace N to workspace M
i3-msg "workspace M; append_layout ~/.config/i3/workspace_N.json"

# And finally we fill the containers with the programs they had
(uxterm &)
(uxterm &)
(uxterm &)
```

然后把文件设为可执行的：

```
chmod u+x ~/load_layout.sh
```

最后，运行下面的命令将N号工作区的布局载入到M号工作区：

`~/load_layout.sh`

提示：添加 `bindsym $mod+g exec ~/load_layout.sh` 到 `~/.config/i3/config` 并重启*i3*将绑定Mod+g 到上面的脚本

注意：如果上面的脚本没有正常工作, 参考 官方文档. `~/.config/i3/workspace_N.json` 的 *swallows* 部分需要被手动编辑

Scratchpad容器

默认地, scratchpads 只包括一个窗口. 但是，容器也能变成一个scratchpad

创建一个新的容器(例如, `Mod+Enter`), 分割它 (`Mod+v`) 然后创建另外一个容器。 聚焦到父窗口 (`Mod+a`), 于另一方向上分割(`Mod+h`), 然后重新创建。

聚焦到第一个容器 (并且聚焦到父窗口), 把窗口变为浮动式 (`Mod+Shift+Space`),然后移动它到scratchpad (`Mod+Shift+-`).现在你可以根据喜好拆分容器了

注意：容器不能在浮动窗口模式下重新设定大小，应该在设置为浮动式之前设定好大小

提示：当只使用终端应用时, 考虑使用 tmux 之类的复用器。

屏保和电源管理

通过 Power management#xss-lock 你可以为你的*i3*会话注册一个屏保。使用 *xautolock* 的`-time` 选项在给定时间后锁定屏幕：

```
xautolock -time 10 -locker "i3lock -i 'background_image.png'" &
```

一个 systemd 服务文件能在系统将要进入睡眠或休眠状态时用来锁屏。参见 Power management#Suspend/resume service files，注意i3lock要求服务类型为 `forking`.

另请参见 DPMS.

另一个方案是使用 xidlehookAUR 并配合 betterlockscreenAUR 或其它锁屏器. *xidlehook* 是一个用Rust写的 *xautolock* 替代品 , 带有一些额外的功能。 它包括了在播放音频或者全屏时禁用锁屏的选项。 `--timer` 参数的单位是秒:

```
xidlehook --not-when-audio --not-when-fullscreen --timer 360 "betterlockscreen -l dim"
"" &
```

关机，重启和锁屏

关机，重启和锁屏的键位绑定可以添加到 `~/.config/i3/config`。接下来的例子假定你已经安装了 polkit包 来允许非特权用户执行电源管理命令。

```
set $Locker i3lock && sleep 1

set $mode_system System (l) lock, (e) logout, (s) suspend, (h) hibernate, (r) reboot,
(Shift+s) shutdown
mode "$mode_system" {
    bindsym l exec --no-startup-id $Locker, mode "default"
    bindsym e exec --no-startup-id i3-msg exit, mode "default"
    bindsym s exec --no-startup-id $Locker && systemctl suspend, mode "default"
    bindsym h exec --no-startup-id $Locker && systemctl hibernate, mode "default"
```

```
    bindsym r exec --no-startup-id systemctl reboot, mode "default"
    bindsym Shift+s exec --no-startup-id systemctl poweroff -i, mode "default"

    # back to normal: Enter or Escape
    bindsym Return mode "default"
    bindsym Escape mode "default"
}

bindsym $mod+Pause mode "$mode_system"
```

一旦配置好, 在任何时候按下 `$mod+pause`将会看到一个提示。要实现更复杂的行为表现，使用一个独立的脚本, 并在mode中引用它。

注意：

- `sleep 1` 添加一个小的延迟来组织可能的休眠竞争状况
- `systemctl poweroff`的`-i`参数会导致其它用户已登录的情况下关机(这需要 polkit包)或者当 *logind* (错误地)这样认为.

对于其它可用于替代的锁屏器，参见 List of applications/Security#Screen lockers.

“吞噬”终端窗口

和dwm相似, i3 能够用新启动的GUI窗口"吞噬"当前的终端窗口。这可以通过使用 i3-swallowAUR\[损坏的链接：package not found\] 或 i3-swallow-gitAUR 软件包来实现。

例如, 要让 mpv的窗口吞噬原始的终端：

```
$ i3-swallow mpv video.mp4
```

多显示器手动管理

感谢 xrandr的存在，有很多方法轻松管理系统显示。 下面的例子在*i3*配置文件中集成了它, 和之前电源管理章节里的做法差不多。

下面的例子是一个有VGA和HDMI输出的笔记本使用一个菜单来切换显示输出的开关：

```
## Manual management of external displays
# Set the shortcuts and what they do
set $mode_display Ext Screen (v) VGA ON, (h) HDMI ON, (x) VGA OFF, (y) HDMI OFF
mode "$mode_display" {
    bindsym v exec --no-startup-id xrandr --output VGA1 --auto --right-of LVDS1, mode
"default"
    bindsym h exec --no-startup-id xrandr --output HDMI1 --auto --right-of LVDS1, mode
"default"
    bindsym x exec --no-startup-id xrandr --output VGA1 --auto --off, mode "default"
    bindsym y exec --no-startup-id xrandr --output HDMI1 --auto --off, mode "default"

    # back to normal: Enter or Escape
    bindsym Return mode "default"
    bindsym Escape mode "default"
}
# Declare here the shortcut to bring the display selection menu
bindsym $mod+x mode "$mode_display"
```

在一个关闭了的显示器上打开的任何窗口将会自动回到活跃的显示器上。

得到显示器设备名最简单的方法是连接上它们，然后执行：

```
$ xrandr --query
```

这将输出可用的、已识别的设备及其系统内名称，可以帮助你正确地设置配置文件。

参考 xrandr 页面或者man page来获取完整可使用的参数列表, 访问i3 userguide 和/或 i3 FAQ on reddit了解更多信息.

标签式或层叠式的网络浏览器

有些网络浏览器并没有实现 Tab, 因为原则上对 Tabs 的管理是由窗口管理器负责，而不是浏览器。

为了让 i3 管理您的无 Tab 网络浏览器，以 uzbl 为例，在 `~/.i3/config` 添加以下：

```
for_window [class="Uzbl-core"] focus child, layout stacking, focus
```

这是层叠式浏览，即所有窗口为垂直地显示。好处是任何 Tab 的标题都可见，哪怕开了很多浏览器窗口。

如果您更偏好标签式浏览，即窗口在水平方向上显示，用：

```
for_window [class="Uzbl-core"] focus child, layout tabbed, focus
```

工作区名称变量

由于工作区在*i3*配置文件中定义了多次, 自定义工作区名称变量也许有帮助，例如：

```
set $WS1 term
set $WS2 web
set $WS3 misc
set $WS4 media
set $WS5 code
```

然后使用对应的名称变量替代工作区名称：

```
bindsym $mod+1          workspace $WS1
...
bindsym $mod+Shift+1    move container to workspace $WS1
```

参见Changing named workspaces获取更多信息。

正确处理浮动对话框

即使是默认情况下就该在浮动模式中打开的对话框 ,也可能仍在平铺模式中打开。要改变这个行为,用xorg-xprop包检查对话框的 `WM_WINDOW_ROLE`，然后添加正确的规则到 `~/.i3/config` (使用 pcre 语法):

```
for_window [window_role="pop-up"] floating enable
for_window [window_role="task_dialog"] floating enable
```

还可以使用标题规则和正则表达式：

```
for_window [title="Preferences$"] floating enable
```

或者 `WM_CLASS`:

```
for_window [class="(?i)mplayer"] floating enable
```

状态栏网速显示

你可以试试上游的脚本， 要使用它：

- 根据你的系统重命名两个网卡 (use `ip addr`)
- 在`/sys/devices`找到它们，然后合适地替换掉:

```
$ find /sys/devices -name network_interface
```

提示：使用 `/sys/class/net/*interface*/statistics/`来不依靠pci位置查找

现在，保存脚本到一个合适的位置 (例如 `~/.config/i3`) ，然后让你的状态栏程序配合它工作

自动切换水平/垂直窗口分割方向

这个 autotilingAUR软件包能够用来自动切换水平/垂直窗口分割方向以达到和bspwm螺旋平铺相似的效果，安装完成之后，把下面内容添加到`~/.config/i3/config`, 然后重启i3。

```
exec_always --no-startup-id autotiling
```

疑难解答

通用

很多时候，你遇到的 bug 在开发版中被修复了。可以从 AUR 上的 i3-gitAUR 与 i3status-gitAUR 编译安装最新的开发版。上游（即 i3 项目开发者）希望你能用开发版再试试看能否重现问题 延伸阅读：调试/获取跟踪数据#Compilation options.

不让焦点一直跟随鼠标

在`~/.config/i3/config`中添加：

```
focus_follows_mouse no
```

i3 信息栏上的某些按钮不能用

举例：`i3-nagbar` 上的“Edit Config”按钮是要调用 `i3-sensible-terminal` 命令的。这就需要你的终端模拟器能够被*i3*识别。

平铺的终端窗口中出现折行错乱

从 v4.3 版本起，*i3* 会无视平铺窗口的扩大提示（size increment hints）。这就带来了折行错乱等问题。平铺之前尝试先把窗口改成浮动状态，可以绕过这个问题。

鼠标指针总处于忙碌状态

当启动了某些并不支持启动提醒的脚本或程序时，鼠标指针会逗留在忙碌状态六十秒左右。

为排除此现象，凡是 `exec` 命令都加 `--no-startup-id` 后缀，比如：

```
exec --no-startup-id ~/script
bindsym $mod+d exec --no-startup-id dmenu_run
```

为了在全局禁用这个鼠标动画，查看：Cursor themes#Create links to missing cursors。

绑定快捷键不起作用

像 scrot 这类工具，用普通的按键绑定方式（按下按键后，未松开按键前就立即执行）可能不会正常工作。可以试试绑定时加上 `--release` 参数，使命令在松开按键之后执行：

```
bindsym --release Print exec --no-startup-id scrot
bindsym --release Shift+Print exec --no-startup-id scrot -s
```

画面撕裂现象

因为 *i3* 的双倍缓冲实现得不到位，会出现画面撕裂或闪烁。安装并配置好 picom 可以解决这一问题。

看不到系统托盘图标

为了让 `tray_output primary` 这一行配置生效，有时需要用 *xrandr* 来指定一个主显示输出。你可以试试按照 tray_output 显式地指定一个具体的输出，或者干脆删掉这行配置。 关于显示输出的细节可以参看 Xrandr 页面. 自从 i3 4.12 版本起，由 i3-config-wizard 生成的默认配置已经不再包含这行配置了。

Spotify的默认工作区

要为spotify窗口分配默认工作区，不能使用标准的`assign` 方法 ，而应该使用 `for_window` 命令, 例如

```
~/.config/i3/config
...
for_window [class="Spotify"] move container to workspace $ws10
```

为了确保 `for_window` 不移动已经在 `$ws10`的窗口, 可以使用 `move --no-auto-back-and-forth`.

参见

- 官方网站
- Funtoo:I3 Tiling Window Manager
- i3源代码
- i3-extras - 脚本和补丁合集
- i3ipc-glib - 一个*i3*拓展插件库
- i3ipc-ruby - An improved library for *i3* extensions in Ruby
- j4tools - 为*i3*设计的非官方工具

Arch Linux Forums

- The i3 thread - *i3*的通用讨论
- i3桌面截图和配置分享

截屏与视频

- i3窗口管理器v4.1截屏与视频
- i3窗口管理器v4.1X截屏与视频

应用程序启动器

启动程序和切回已开窗口，往往比敲几个字符慢得多。本章把这步交给键盘：唤出，键入，回车。三节可按需取用：Rofi 内容最全，安装、配置、自定义主题与提示技巧齐备，属必读；KRunner 适合已习惯其所在桌面者，重点看 Meta 键唤醒与切换被激活窗口；Gmrun 篇幅最短，只讲安装、使用、配置和键绑定，暂时跳过不影响后文。

最容易踩的坑是装完就以为生效：它们都靠一个唤醒键触发，而该组合键常已被桌面或输入法占用，按了没反应；其次是给 Rofi 换主题时配置文件放错位置，看不出效果。

18.1　KRunner

难度 入门 | **篇幅** 3.6 KB | **所属** 应用程序启动器

本节将讲到

- 安装
- 使用方法
- 使用Meta键唤醒KRunner
- 更改被激活的窗口
- 参见

KRunner 是 Plasma 5 中内置的搜索应用, 能够快速的启动应用程序以及运行命令, 并且具有一个"runner"系统, 用户可以自定义可用的功能.

安装

安装 krunner包 软件包.

使用方法

需要在Plasma桌面中运行KRunner, 你可以右键单击桌面然后从菜单中选择"run command"(中文环境下为"显示KRunner"), 也可以使用键盘快捷键 `Alt+Space` 或者 `Alt+F2` 来启动KRunner. 在某些工作区状态下(例如一个空白的桌面), 直接在键盘上输入就会自动唤醒KRunner.

使用Meta键唤醒KRunner

为了使用Meta键作为启动KRunner的快捷键, 请运行命令

```
kwriteconfig5 --file kwinrc --group ModifierOnlyShortcuts --key Meta "org.kde.krunner,/App,,display"
```

要将更改应用于当前会话，使用qdbus向KWin发送特定的DBus信号：

```
qdbus org.kde.KWin /KWin reconfigure
```

如果不使用DBus那么需要重启KWin才能够生效：

```
kwin_x11 --replace # For X11
kwin_wayland --replace # For wayland
```

更改被激活的窗口

Plasma 5 桌面不包含默认的方式来指定krunner仅由活动窗口标题来进行搜索. 以下方法可用于解决此问题.

使用生成的完整窗口列表来搜索窗口标题

此方法要求系统中安装了 xdotool包 软件包.

1. 打开*系统设置* > *工作区* > *快捷键* > *自定义快捷键*.
2. 右键列表新建一个全局快捷键, 并选择 命令/URL: 项
3. 勾选名称右边的复选框.
4. 在触发器选项卡中点击按钮来设置所需的快捷键组合.
5. 在动作选项卡中输入 `/usr/local/bin/krunner-search-by-windows.sh`
6. 创建文件 `/usr/local/bin/krunner-search-by-windows.sh` 并输入以下内容:

```
    #!/bin/bash
    qdbus org.kde.krunner /App querySingleRunner windows ""
    sleep 0.2
    xdotool type 'window '
    xdotool key "shift+BackSpace"
```

1. 给文件赋予运行权限:

```
    chmod a+x /usr/local/bin/krunner-search-by-windows.sh
```

输入上面的内容时要注意 `window` 后面的空格.

现在你可以通过你指定的快捷键来获得已打开窗口的列表, 并在键入时根据该列表进行搜索;

在没有完整窗口列表的情况下搜索窗口标题

这种方法局限性会大一些, 但没有上面那种方法那么丑.

1. 打开*系统设置* > *工作区* > *快捷键* > *自定义快捷键*.
2. 右键新建一个全局快捷键, 并选择 D-bus 命令
3. 勾选名称右边的复选框.
4. 在触发器选项卡中点击按钮来设置所需的快捷键组合.
5. 在 动作 选项卡中填写下面这些信息:

```
- 远程应用程序          : org.kde.krunner
- 远程对象              : /App
```

```
  - 函数                : querySingleRunner
  - 参数                : windows ""
```

参见

- KRunner on KDE UserBase Wiki

18.2 Gmrun

难度 入门 | **篇幅** 5.2 KB | **所属** 应用程序启动器

本节将讲到

- 安装
- 使用
- 配置
- 键绑定

Gmrun (Gnome Completion-Run) 是一个和 GNOME Run, Application Finder, KRunner 等相似的轻量应用程序启动器。

安装

可用 gmrun包 安装 Gmrun。

使用

- 键入并按 `Enter` 运行 GUI 应用程序。键入并按 `Ctrl+Enter` 运行终端程序。不键入按 `Ctrl+Enter` 则打开终端仿真器。
- Gmrun tab 补全功能： 按 Tab 键会出现一个可滚动的可能匹配列表。
- 键入 Gmrun 的 Web 地址会自动启动 Web 浏览器以打开。
- Email 地址同理：使用 `mailto:` 前缀，例如 `mailto:foo@bar.com`, 会启动您的 Email 客户端。
- 可添加更多快捷方式到 `~/.gmrunrc`。（见下方 #配置），或您可修改系统范围文件： `/etc/gmrunc`。
- 您可键入 ！作为第一字符以启用搜索模式，标题将会由 *Run program* 变为 *Search*，你键入后，Gmrun 将自动根据你的命令历史提出建议。
- `Ctrl+r` 将允许您在历史记录中向后搜索。
- `Ctrl+s` 将在历史记录中向前搜索。
- `Ctrl+g` 将取消搜索。
- 如果在配置文件中定义了扩展名，那么只需键入文件名，文件就可以被正确的程序启动。
- 按下 `Esc` 以关闭 Gmrun，不运行任何东西。

配置

配置文件位于 /etc/gmrunrc 但您可创建每个用户的配置（推荐）于 ~/.gmrunrc。请注意，%u 将被扩展为输入的完整命令。 %s 是快捷方式后的最后部分。（例如，如果您键入 URL https://archlinux.org，%u 将保留 https://archlinux.org 而 %s 将缩减为 //archlinux.org。）

下面是一个示例配置文件。

```
~/.gmrunrc
# gmrun 配置文件
# gmrun 由 Mihai Bazon <mishoo@infoiasi.ro> 授权
# 适用 GPL v2.0

# 设置终端
Terminal = urxvt
TermExec = ${Terminal} -e
AlwaysInTerm = ssh telnet ftp lynx mc vi vim pine centericq perldoc man

# 设置窗口的几何形状（除了高度）。
Width = 400
Top = 300
Left = 450

# 历史记录大小
History = 256

# 是否显示调用时选择的最后一个历史记录行
ShowLast = 1

# 显示以“.”开头的文件
# 默认值为 0（关闭），如果您希望显示“隐藏”文件，请将其设置为 1
# 在完成窗口中
ShowDotFiles = 0

# 超时（以毫秒为单位），gmrun 将模拟 TAB 按下
# 如果不喜欢此功能，请将其设置为 NULL。
TabTimeout = 0

# URL 处理程序
# 如果输入的文本是"http://www.baidu.com"，则：
#   - %u 被整个 URL 替换 ("http://www.baidu.com")
#   - %s 被替换为"//www.baidu.com"。 这对多 URL 很有用
#     例如 "man:printf" --> %s 会被替换为 "printf"
# 译者注：原文为谷歌搜索，如有需要，参见原版界面。
URL_http = firefox %u
URL_mailto = firefox -remote "mailto(%s)"
URL_man = ${TermExec} 'man %s'
URL_info = ${TermExec} 'info %s'
URL_pd = ${TermExec} 'perldoc %s'
URL_file = pcmanfm %s
URL_readme = ${TermExec} 'less /usr/doc/%s/README'
URL_info = ${TermExec} 'info %s'
URL_sh = sh -c '%s'
URL_paci = ${TermExec} 'pacman -S %s'
URL_pacs = ${TermExec} 'pacman -Ss %s'

# 扩展名处理程序
EXT:doc,rtf = AbiWord %s
```

```
EXT:txt,cc,cpp,h,java,html,htm,epl,tex,latex,js,css,xml,xsl,am,php,css,js,py,rb = gedit
%s
EXT:mpeg,mpg,avi,mkv,flv = vlc %s
EXT:mp3,ogg,m4a,wmv,wma = deadbeef %s
EXT:ps = gv %s
EXT:pdf = epdfview %s
```

ShowDotFiles（1 或 0）：是否显示'.'开头文件（例如 `.gmrunrc`）。 TabTimeout（1 或 0）：当输入时，是否自动显示 tab 补全目录。 Terminal：当按下 `Ctrl+Enter` 且无输入时运行的命令。 TermExec：想要在终端运行一个特定的命令时运行的命令 （例如 `ls ~`）。 URL_http：当输入网址时运行的命令。 URL_mailto：当输入 E-mail 地址时运行的命令。 EXT:extension：指定打开特定扩展名文件的程序。 Top and Left：Gmrun 窗口的位置（以相应边缘的像素为单位）。 Width：窗口的宽度（以像素为单位）。 History：存储的命令历史记录的长度。

添加自定义快捷方式

可轻松添加快捷方式。例如，使用 **b** 作为百度搜索的快捷方式，则添加：

```
URL_b = firefox 'https://www.baidu.com/s?wd=%s'
```

这样使用：

```
b:Arch
```

译者注：谷歌搜索，请见原版 Gmrun#Adding custom shortcuts。

键绑定

您可使用桌面环境或窗口管理器的键绑定设置来为 Gmrun 设置。

18.3 Rofi

难度 进阶 | **篇幅** 7.9 KB | **所属** 应用程序启动器

本节将讲到

- 安装
- 配置
- 自定义主题
- 提示与技巧

Rofi 是一个窗口切换器、运行对话框、ssh 启动器以及 dmenu 的替代品，是 simpleswitcher 的复制，经 Sean Pringle 编写并在随后由 Dave Davenport 扩展。

安装

安装 rofi包 软件包，rofi 需要在 Xorg 显示服务下使用。

主分支中对 Wayland 支持的合并已无限期停滞。若需 Wayland 支持，安装 wofi包 或 rofi-wayland包 作为替代。注意，这些软件包需要一个支持 Layer Shell 协议的

Wayland 混成器。包括了基于 wlroots 的混成器，如 Sway，以及一些基于 Mir 的混成器（但如 Gnome 的混成器 mutter 不包括在内）。

更多关于 rofi 的信息和用法，请参阅 rofi(1) 手册页。

配置

设置配置选项有如下方式：

- 本地配置。通常依赖于 XDG，位于 `~/.config/rofi/config.rasi` 内。
- 命令行选项：

```
$ rofi -combi-modi window,drun,ssh -theme solarized -font "hack 10" -show combi
```

也可在配置文件中列出：

```
configuration {
 modi: "window,drun,ssh,combi";
 font: "hack 10";
 combi-modi: "window,drun,ssh";
 }
@theme "solarized"
```

运行 `rofi -dump-config` 以获取 `config.rasi` 可用选项的完整列表。也可执行 `rofi -dump-config > ~/.config/rofi/config.rasi` 将输出直接写入配置文件。

注意：i3 用户需注意，*i3* 配置文件中的逗号会引起一些问题。要设置绑定按键启动 *rofi*，要么使用配置文件，要么将逗号替换为 `#`，例如：`rofi -combi-modi window#drun#ssh`。

图标

可以让图标与其对应条目一同显示。假设已安装 papirus-icon-theme包，并使用 `-show-icons` 与定义图标主题的选项 `-icon-theme` 令 *rofi* 显示图标，命令如下所示：

```
$ rofi -combi-modi window,drun,ssh -theme solarized -font "hack 10" -show combi -icon-theme "Papirus" -show-icons
```

自定义主题

使用如下命令预览并为 *rofi* 应用主题：

```
$ rofi-theme-selector
```

rofi-theme(5) 手册页包含了大量使用一种类似 CSS 的自定义语言为 rofi 设置主题的介绍。该页面还包含了 rofi 所使用的布局系统的额外信息，如 rofi 主题所使用的准确语法和绝大多数的主题属性。Rofi 的主题系统很简单（默认的 dmenu 主题为 30 行左右的 CSS）但足够灵活，可创建出多种可交互、类似小部件的组件和菜单，如 rofi-advanced。

社区主题

`/usr/share/rofi/themes` 目录下存放着一些 Rofi 自带的官方主题，更多用户主题可于 rofi 主题仓库中找到。

如下所示，使用命令行加载一个官方主题，或是一个下载的 *.rasi* 格式的用户主题（应置于 `~/.config/rofi/example.rasi` 内）：

```
$ rofi options -theme example
```

也可写在 `configuration { }` 代码块外部：

```
@theme "example"
```

提示与技巧

Rofi 用作 dmenu 的替代品

被以 dmenu 的形式调用时（通过符号链接），*rofi* 类似于 *dmenu*。通过脚本调用 *dmenu* 的程序（如 pass 的 *passmenu*）将使用 *rofi* 而非 *dmenu*。`rofi` 在 *dmenu* 模式下的具体行为请参阅 rofi-dmenu(5)。

若要使 rofi 贴近 dmenu 的外观，将 rofi包 软件包默认的 `/usr/share/rofi/themes/dmenu.rasi` 文件复制到 rofi 的配置文件目录下（通常是 `~/.config/rofi`）并将 `@theme dmenu` 添加至 `config.rasi` 的末尾或将 `-theme dmenu` 选项附加至命令行命令以启用该主题。

在 rofi 中执行 shell 命令

若要在 rofi 中直接运行 shell 命令或用户脚本并查看输出内容，需确保：

定义 `-run-shell-command '{terminal} -e *SHELL* -ic "{cmd} && read"`，其中的 `*SHELL*` 是用户所使用的 shell（例如 bash、zsh）。之后可在输入框中输入命令并按下 `Shift+Enter` 执行。若无后续按键操作，终端将保持开启状态。

以下是一个包含 *i3* 所推荐的转义序列的使用示例：

```
bindsym $mod+d exec --no-startup-id rofi -show drun -run-shell-command '{terminal} -e zsh -ic "{cmd} && read"'
```

集成 Unicode 选择功能

安装 rofimoji包 以令 *rofi* 集成 Unicode emoji 与字符选择器的功能。参见项目说明以了解使用方式和配置。

Emoji 选择菜单

安装 rofi-emoji包 以令 *rofi* 集成 emoji 选择器的功能。参见项目说明以了解使用方式。

若遇 emoji 相关渲染问题（例如，渲染为方块），应考虑安装 ttf-symbolaAUR、noto-fonts-cjk包 和noto-fonts-emoji包。

Rofi 用作电源管理菜单

Rofi 可与 systemd 一同用于电源管理。安装 rofi-power-menuAUR。该 AUR 软件包默认会将 rofi-power-menu 脚本的路径添加至 $PATH，因此只需按如下方式使用：

```
$ rofi -show p -modi p:rofi-power-menu
```

若不打算使用 AUR 软件包而是将其手动复制到系统中，则需要为 rofi 指定可执行文件 *rofi-power-menu* 的路径。例如，假设可执行文件位于 `$HOME/.rofi-power-menu`：

```
$ rofi -show p -modi p:$HOME/.rofi-power-menu
```

也可将可执行文件置于 PATH 中，使用时仅需要传递该文件的名称即可。

若要显示默认的符号，需要安装 ttf-nerd-fonts-symbols包。

更多信息请参见项目仓库中的 README 文件。

Rofi 用作剪切板管理器

Rofi 可与 Greenclip（英语：Greenclip） 集成并用作 X11 的剪切板管理器。安装 rofi-greenclipAUR 软件包。

Rofi 用作计算器

使用 rofi-calc包 软件包可将 rofi 用作计算器，用于自然语言输入、单位转换和货币转换。

Xorg 命令

本章处理图形会话的两个入口：怎样把 X 服务器连同窗口管理器或桌面拉起来，以及会话起来后怎样查看、测试并调整分辨率与布局。两节独立，可依次通读：《Xinit》讲安装、配置与用法，是必读主干，技巧和窍门可后翻；《Xrandr》同样必读，测试配置与添加未被检测到的有效分辨率最实用，脚本与在 VNC 上使用 xrandr 按需取用，日常单屏环境可跳过。

最容易踩的坑是只做临时改动：xrandr 调通的设置重启即失效，必须照使设置永久生效那一节写进配置；“未被检测到”不等于可以随意编造，能补进去的只有本身有效的分辨率。

19.1　Xinit

难度 进阶 | **篇幅** 10.0 KB | **所属** Xorg 命令

本节将讲到

- 安装
- 配置
- 用法
- 技巧和窍门

摘自 Wikipedia:

: *xinit* 程序允许用户手动启动 Xorg 显示服务器。startx(1) 脚本是 xinit(1) 的一个前端。

xinit 通常用于启动窗口管理器或桌面环境。虽然您也可以用 *xinit* 在没有窗口管理器的情况下运行 GUI 应用程序，但许多图形应用程序期待一个 EWMH 兼容的窗口管理器。显示管理器一般都依据 xprofile 为您启动 Xorg。

安装

安装 xorg-xinit包 包。

配置

xinit 和 *startx* 需要一个可选的客户端程序参数，见 #覆盖 xinitrc 。 如果您没有提供参数，它们会寻找 `~/.xinitrc`，作为一个shell脚本运行，以启动客户程序。

xinitrc

`~/.xinitrc` 可以方便地在X服务器启动时运行依赖于X的程序并设置环境变量。如果它存在于用户的主目录中，*startx* 和 *xinit* 会执行它。否则 *startx* 将运行默认的`/etc/X11/xinit/xinitrc`。

注意：*xinit* 有自己的默认行为，而不是执行该文件。详见 xinit(1)。

这个默认的xinitrc将启动一个包含 Twm、xorg-xclock包 和 Xterm 的基本环境（假设安装了必要的软件包）。因此，要启动不同的窗口管理器或桌面环境，首先要在主目录下创建一个默认的 `xinitrc` 副本。

```
$ cp /etc/X11/xinit/xinitrc ~/.xinitrc
```

然后编辑#添加、创建、编辑文件)该文件，用想要的命令替换默认程序。记住，在使用 `exec` 的命令之后的行会被忽略。例如，要在后台启动 `xscreensaver`，然后启动 openbox，请使用以下命令。

```
~/.xinitrc
...
xscreensaver &
exec openbox-session
```

注意：至少要确保 `/etc/X11/xinit/xinitrc` 中的最后一个 `if` 块出现在您的 `~/.xinitrc` 文件中，以确保 `/etc/X11/xinit/xinitrc.d` 中的脚本有源可循。

在窗口管理器之前启动的长期运行的程序，如屏保和墙纸程序，必须自己分叉或通过附加 `&` 符号在后台运行。否则，在执行窗口管理器或桌面环境之前，脚本会停止运行并等待每个程序退出。请注意，有些程序反而不应该被分叉，以避免出现竞争性错误，如 xrdb 的情况。在 `exec` 前缀会将脚本进程替换为窗口管理器进程，这样，即使该进程分叉到后台，X也不会退出。

xserverrc

`xserverrc` 文件是一个负责启动X服务器的shell脚本。startx "和 "xinit "都会执行 `~/.xserverrc`，如果它存在的话，"startx "会使用 `/etc/X11/xinit/xserverrc`。

为了保持 `logind`的会话权限，并防止通过切换终端绕过屏幕锁定器，Xorg 必须在发生登录的同一虚拟终端上启动 。因此建议在 `vt$XDG_VTNR`文件中指定 `~/.xserverrc`。

```
~/.xserverrc
}文件：
#!/bin/sh

exec /usr/bin/Xorg -nolisten tcp "$@" vt$XDG_VTNR
```

参见 Xserver(1) 中的所有命令行选项列表。

提示：`-nolisten local` 可以加在 `-nolisten tcp` 之后，以禁用X11的抽象套接字，帮助隔离。 关于这对X11安全的潜在影响，有一个 简单背景。

另外，如果您想让X显示在一个独立的控制台，而不是调用服务器的控制台，您可以通过使用 `/usr/lib/systemd/systemd-multi-seat-x` 提供的X服务器包装器实

现。为方便起见，可以通过修改 `~/.xserverrc` 设置 "xinit "和 "startx "使用该包装器。

`-keeptty`选项，重新启用将X会话的输出重定向至Xorg日志文件的功能。详见 Xorg#重定向Xorg会话日志。}}

用法

要以普通用户身份运行 Xorg，请输入：

```
$ startx
```

或者如果配置了 #xserverrc。

```
$ xinit -- :1
```

注意：*xinit* 不能处理已启动的另一个X服务器的多个显示。为此，您必须通过附加 `--:*display_number*` 来指定显示，其中 `*display_number*` 为 `1` 或以上。

您选择的窗口管理器（或桌面环境）现在应该可以正确启动了。

要退出X，请运行您的窗口管理器的退出功能（假设它有的话）。如果它缺乏这样的功能，请运行。

```
$ pkill -15 Xorg
```

注意：*pkill* 将杀死所有正在运行的X实例。要指定杀死当前虚拟终端上的窗口管理器，请运行：

```
$ pkill -15 -t tty"$XDG_VTNR" Xorg
```

另见 signal(7)。

技巧和窍门

覆盖 xinitrc

如果您有一个正常工作的 `~/.xinitrc`，但只是想试试其他窗口管理器或桌面环境，您可以通过发出*startx*，并在后面加上窗口管理器的路径来运行它，例如。

```
$ startx /usr/bin/i3
```

如果二进制文件需要参数，它们需要加引号，以便被识别为*startx*的第一个参数的一部分。

```
$ startx "/usr/bin/application --key value "
```

注意，完整的路径是**必须的**。您也可以为 #xserverrc 脚本指定自定义选项，在双破折号 `--` 后加上这些选项。

```
$ startx /usr/bin/enlightenment -- -br +bs -dpi 96
```

另请参见 startx(1)。

注意：由于 `/etc/X11/xinit/xinitrc.d/` 下的脚本被跳过，环境变量 `DISPLAY` 可能需要被设置。您可以通过执行`DISPLAY=:*display_number* startx /usr/bin/i3`，在所需的显示器上试用 *i3*。

提示：这可用于启动常规GUI程序，但没有任何基本的窗口管理器功能。参见 #在没有窗口管理器的情况下启动应用程序 和 在独立的X显示屏中运行程序。

登录时自动启动 X

确保 *startx* 是正确的配置。

在您的 login shell 初始化文件（例如，Bash 的 `~/.bash_profile` 或 Zsh 的 `~/.zprofile`）中放置以下内容。

```
if [ -z "${DISPLAY}" ] && [ "${XDG_VTNR}" -eq 1 ]; then
  exec startx
fi
```

您可以用 `-eq` 替换 `-le 3` 这样的比较（用于 vt1 到 vt3），如果您想在一个以上的虚拟终端上使用图形化登录。

检测虚拟终端的其他条件包括 `"$(tty)" = "/dev/tty1"`，它不允许与 `-le` 比较，以及 `"$(fgconsole 2>/dev/null || echo -1)" -eq 1`，它在串行控制台不起作用。

`exec` 命令确保在X服务器退出、崩溃或被攻击者杀死时，用户被注销。如果您想承担风险，在X会话结束时保持登录状态，请删除 `exec`。

参见 Fish_(简体中文)#登录_fish_时自动起动_X 和 Systemd_(简体中文)/User_(简体中文)#Automatic_login_into_Xorg_without_display_manager。

提示：此方法可与 自动登录到虚拟控制台 相结合。

在桌面环境/窗口管理器之间进行切换

如果您经常在不同的桌面环境或窗口管理器之间切换，使用显示管理器或扩展 `~/.xinitrc` 来实现切换是很方便的。

下面的例子展示了如何用一个参数来启动一个特定的桌面环境或窗口管理器。

```
~/.xinitrc
...

# 这里 Xfce 保持默认
session=${1:-xfce}

case $session in
    i3|i3wm           ) exec i3;;
    kde               ) exec startplasma-x11;;
    xfce|xfce4        ) exec startxfce4;;
    # No known session, try to run it as command
    * ) exec $1;;
esac
```

要传递参数 *session*：

```
$ xinit session
```

或

```
$ startx ~/.xinitrc session
```

在没有窗口管理器的情况下启动应用程序

可以在没有窗口管理器的情况下只启动特定的应用程序，尽管这很可能只对全屏模式下的单个应用程序有用。比如说：

```
~/.xinitrc
...

exec chromium
```

另外，也可以像 #覆盖 xinitrc 中描述的那样，直接从命令提示符中调用二进制。

使用这种方法，您需要通过自己的配置文件设置每个应用程序的窗口几何形状（如果可能的话）。

提示：这对启动图形游戏很有用，排除合成器的开销有助于提高游戏的性能。

另请参见显示管理器#没有窗口管理启动应用程序。

使用 startx 进行输出重定向

参见Xorg#重定向Xorg会话日志了解详情。

19.2 Xrandr

难度 进阶 | **篇幅** 17.8 KB | **所属** Xorg 命令

本节将讲到

- 安装
- 测试配置
- 添加未被检测到的有效分辨率
- 使xrandr所更改的分辨率设置永久生效
- 脚本

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： 原文已有大量更新，请持续更新译文（在 Talk:Xrandr# 中讨论）

"xrandr" 是一款官方的 RandR (*Resize and Rotate*)维基百科：X 窗口系统扩展配置工具。它可以设置屏幕显示的大小、方向、镜像等。对多显示器的情况，请参考 Multihead 页面。

安装

安装软件包 xorg-xrandr包。

图形化操作程序

- **ARandR** — 提供了一个简单的图形化程序给 XrandR。显示器的位置会图形化地展示出来并可以使用拖拽的方式调整。

: https://christian.amsuess.com/tools/arandr/ || arandr包

- **LXRandR** — 是 LXDE 中的屏幕分辨率和外置显示器管理的工具.

: https://wiki.lxde.org/en/LXRandR\[失效链接 2022-09-23 ⓘ\] || GTK 2: lxrandr包, GTK 3: lxrandr-gtk3包

命令行前端

- **xlayoutdisplay** — 检测和排列显示器。可处理：笔记本电脑开合状态，最高有效刷新率，计算并设定当前的 DPI。最适合用在.xinitrc 中，以便于在接入或断开外置显示器或闭合笔记本盖子时引用这些参数

: https://github.com/alex-courtis/xlayoutdisplay || xlayoutdisplayAUR

测试配置

当没有添加任何选项直接运行时，*xrandr* 列出该系统可用的显示输出设备 (`VGA-1`, `HDMI-1` 等等) 和每一台设备可设置的分辨率，当前分辨率后面带有一个*号和一个+号:

```
xrandr
Screen 0: minimum 320 x 200, current 3200 x 1080, maximum 8192 x 8192
VGA-1 disconnected (normal left inverted right x axis y axis)
HDMI-1 connected primary 1920x1080+0+0 (normal left inverted right x axis y axis) 531mm
x 299mm
   1920x1080     59.93 +  60.00*   50.00    59.94
   1920x1080i    60.00    50.00    59.94
   1680x1050     59.88
…
```

注意：如果你的分辨率没有出现在上方， 请看 #添加未被检测到的有效分辨率

你可以使用 *xrandr* 设置不同的分辨率（必须是出现在上面输出列表中的分辨率）：

```
$ xrandr --output HDMI-1 --mode 1920x1080
```

当列表中出现多个刷新率，可以通过 `--rate` 选项改变，一次性设置或者分开设置，例如：

```
$ xrandr --output HDMI-1 --mode 1920x1080 --rate 60
```

如果输出设备已经连接但被禁用，`--auto` 选项会以系统偏好的分辨率（最大分辨率）开启特定的输出设备：

```
$ xrandr --output HDMI-1 --auto
```

还可以用一条命令设置多个输出设备，例如，使用系统偏好的选项关闭 `HDMI-1` 并打开 `HDMI-2`：

```
$ xrandr --output HDMI-1 --off --output HDMI-2 --auto
```
注意：

- 通过 *xrandr* 作出的改变只在本次会话中有效
- *xrandr* 有很多功能 - 详情请参考 xrandr(1)。

添加未被检测到的有效分辨率

由于出错的硬件或驱动，xrandr 可能并不能检测出您的显示器所有的有效分辨率。不过，我们可以在xrandr里添加所需要的分辨率。 Also, this same procedure can be used to add refresh rates you know are supported, but not enabled by your driver。

首先，运行`gtf`或者`cvt`，查询某分辨率的有效扫描频率。

```
 $ cvt 1280 1024
```

```
 # 1280x1024 59.89 Hz (CVT 1.31M4) hsync: 63.67 kHz; pclk: 109.00 MHz
 Modeline "1280x1024_60.00"  109.00  1280 1368 1496 1712  1024 1027 1034 1063 -hsync
+vsync
```
然后通过--newmode参数新建一种xrandr模式，输入上面所得到的查询结果，其中Modeline关键词自然需要被省略。

```
   xrandr --newmode "1280x1024_60.00"  109.00  1280 1368 1496 1712  1024 1027 1034 1063
-hsync +vsync
```
新建模式后，我们需要把这模式添加到当前的输出设备（假定为VGA1）上。由于一些参数已经事先设置，只需输入模式名称即可，即1280x1024_60.00。

```
   xrandr --addmode VGA1 1280x1024_60.00
```
最后，再把VGA1的分辨率指定为刚刚添加的新模式。

```
   xrandr --output VGA1 --mode 1280x1024_60.00
```
注意，以上设置同样地只能在当前会话暂时生效。

如果您对所要添加的某分辨率感到不放心，您可以追加新命令“sleep 5”以及一条切换到已有有效分辨率的命令，以保证不会被困在实际无效的分辨率，示例：

```
   xrandr --output VGA1 --mode 1280x1024_60.00 && sleep 5 && xrandr --newmode
"1024x768-safe" 65.00 1024 1048 1184 1344 768 771 777 806 -HSync -VSync && xrandr --
addmode VGA1 1024x768-safe && xrandr --output VGA1 --mode 1024x768-safe
```
其他输出设备如法炮制：VGA1或DVI-I……

使xrandr所更改的分辨率设置永久生效

使xrandr定制永久生效的方案有：

- `xorg.conf`（推荐）
- `.xprofile`
- kdm/gdm

在xorg.conf设置分辨率（推荐）

示例：

```
/etc/X11/xorg.conf
Section "Monitor"
    Identifier      "External DVI"
    Modeline        "1280x1024_60.00"  108.88  1280 1360 1496 1712  1024 1025 1028 1060
-HSync +Vsync
    Option          "PreferredMode" "1280x1024_60.00"
EndSection
Section "Device"
    Identifier      "ATI Technologies, Inc. M22 [Radeon Mobility M300]"
    Driver          "ati"
    Option          "Monitor-DVI-0" "External DVI"
EndSection
Section "Screen"
    Identifier      "Primary Screen"
    Device          "ATI Technologies, Inc. M22 [Radeon Mobility M300]"
    DefaultDepth    24
    SubSection "Display"
        Depth           24
        Modes   "1280x1024" "1024x768" "640x480"
    EndSubSection
EndSection

Section "ServerLayout"
        Identifier      "Default Layout"
        Screen          "Primary Screen"
EndSection
```

关于更多的配置细节，请阅读Xorg或xorg.conf(5)。

在xprofile设定xrandr命令

请阅读xprofile.

这方案具有缺点：如果您使用Display manager的话，那么在启动进程之后很大程度上就会执行失败，最终无法顺利修改分辨率。

在KDM/GDM的启动脚本设定xrandr命令

KDM和GDM都具备在X初始化时，会被自动执行的启动脚本。GDM的启动脚本放在`/etc/gdm/`， KDM的则是`/usr/share/config/kdm/Xsetup`，SDDM 的则是在 `/usr/share/sddm/scripts/Xsetup`。您可以把相关的xrandr命令添加到这些启动脚本里。

这些脚本需要root权限及其他系统配置的配合，不过在启动进程里会比xprofile更早生效。

脚本

控制第二显示器的开关状态，默认显示器保持开启：

```
~/bin/xdisplay
#!/bin/bash
#
# This script toggles the extended monitor outputs if something is connected
#
```

```
# all available outputs
OUTPUTS=$(xrandr |awk '$2 ~ /connected/ {print $1}')

# your notebook LVDS monitor
DEFAULT_OUTPUT=$(sed -ne 's/.*(LVDS[^ ]*).*/1/p' <<<$OUTPUTS)

# get info from xrandr
XRANDR=`xrandr`

EXECUTE=""

for CURRENT in $OUTPUTS
do
        if [[ $XRANDR == *$CURRENT\ connected*  ]] # is connected
        then
                if [[ $XRANDR == *$CURRENT\ connected\ \(* ]] # is disabled
                then
                        EXECUTE+="--output $CURRENT --auto --above $DEFAULT_OUTPUT "
                else
                        EXECUTE+="--output $CURRENT --off "
                fi
        else # make sure disconnected outputs are off
                EXECUTE+="--output $CURRENT --off "
        fi
done

xrandr --output $DEFAULT_OUTPUT --auto $EXECUTE
```

在显示器之间切换，且只开启其中一个。

```
/usr/local/bin/toggle-display
#!/bin/bash
#
# toggle-display.sh
#
# Iterates through connected monitors in xrander and switched to the next one
# each time it is run.
#

# get info from xrandr
xStatus=`xrandr`
connectedOutputs=$(echo "$xStatus" | grep " connected" | sed -e "s/\([A-Z0-9]\+\)
connected.*/\1/")
activeOutput=$(echo "$xStatus" | grep -e " connected [^(]" | sed -e "s/\([A-Z0-9]\+\)
connected.*/\1/")
connected=$(echo $connectedOutputs | wc -w)

# initialize variables
execute="xrandr "
default="xrandr "
i=1
switch=0

for display in $connectedOutputs
do

       # build default configuration
       if [ $i -eq 1 ]
       then
              default=$default"--output $display --auto "
       else
              default=$default"--output $display --off "
       fi

       # build "switching" configuration
       if [ $switch -eq 1 ]
       then
              execute=$execute"--output $display --auto "
```

```
                switch=0
        else
                execute=$execute"--output $display --off "
        fi

        # check whether the next output should be switched on
        if [ $display = $activeOutput ]
        then
                switch=1
        fi

        i=$(( $i + 1 ))

done

# check if the default setup needs to be executed then run it
echo "Resulting Configuration:"
if [ -z "$(echo $execute | grep "auto")" ]
then
        echo "Command: $default"
        `$default`
else
        echo "Command: $execute"
        `$execute`
fi
echo -e "\n$(xrandr)"
```

您也可以使用xrr-events (from AUR)，一种负责监听XrandR事件的daemon服务。当某台显示器接通状态发生变动时，就会执行相关脚本。可在man页面进一步查询具体信息。

在VNC上使用xrandr

如果您在使用某台支持xrandr的VNC服务器，您可以通过"xrandr -s <width>x<height>"命令实时修改VNC的分辨率。tigervnc就是一种支持xrandr的VNC客户端。

示例：

```
xrandr -s 1920x1200
```

登录VNC之后，如果您在控制台上输入"xrandr"，您将得到列出当前已配置模式的清单。每个模式均可通过xrandr -s选项激活。不过，若您所需要的模式并不在清单中，您可以按照以下来添加它。

示例：不妨想添加的是1024x600（上网本的一种常见分辨率）

首先执行CVT，得到理想分辨率所对应的正确刷新频率。

```
$ cvt 1024 600
```

您会得到如下类似的输出：

```
# 1024x600 59.85 Hz (CVT) hsync: 37.35 kHz; pclk: 49.00 MHz
Modeline "1024x600_60.00"   49.00  1024 1072 1168 1312  600 603 613 624 -hsync +vsync
```

在以下命令使用那些关于刷新频率的输出部分。

```
xrandr --newmode "1024x600"   49.00  1024 1072 1168 1312  600 603 613 624 -hsync +vsync
xrandr --addmode default "1024x600"
```

通过以上流程，输入xrandr -s 1024x600，就可以设置当前分辨率为1024x600，但是这设置只在当前的X会话暂时生效。为确保其模式永久可用，在~/.vnc/xstartup添加以下：

```
xrandr --newmode "1024x600"   49.00  1024 1072 1168 1312  600 603 613 624 -hsync +vsync
xrandr --addmode default "1024x600"r
```

疑难排除

添加未检测到的分辨率

由于硬件以及驱动程序可能的缺陷，例如，请求的EDID数据块不正确，导致 xrandr 可能不能准确侦测到显示器的分辨率。不过我们可以手工添加期望的分辨率。

首先，运行 `gtf` 或 `cvt` 以获取所需分辨率的**模式行（Modeline）**：

```
$ cvt 1280 1024
# 1280x1024 59.89 Hz (CVT 1.31M4) hsync: 63.67 kHz; pclk: 109.00 MHz
Modeline "1280x1024_60.00"  109.00  1280 1368 1496 1712  1024 1027 1034 1063 -hsync +vsync
```

提示：For some LCD screens (e.g. Samsung 2343NW, Acer XB280HK), the command `cvt -r` (= with reduced blanking) is to be used.

注意：如果使用了 Intel 的显示驱动程序 xf86-video-intel包，期望的分辨率会在 `/var/log/Xorg.0.log` 中与其他特征值一并报告——如果该值不同于 `gtf` or `cvt` 的输出应首选该值。这里给出一个log文件的报告值与 xrandr 使用值的实例：

```
[    45.063] (II) intel(0): clock: 241.5 MHz    Image Size:  597 x 336 mm
[    45.063] (II) intel(0): h_active: 2560  h_sync: 2600  h_sync_end 2632 h_blank_end 2720 h_border: 0
[    45.063] (II) intel(0): v_active: 1440  v_sync: 1443  v_sync_end 1448 v_blanking: 1481 v_border: 0
xrandr --newmode "2560x1440" 241.50 2560 2600 2632 2720 1440 1443 1448 1481 -hsync +vsync
```

然后创建一个新的 xandr 模式。注意模式行中那些被忽略的关键字。

```
$ xrandr --newmode "1280x1024_60.00"  109.00  1280 1368 1496 1712  1024 1027 1034 1063 -hsync +vsync
```

创建完毕还需要一个操作步骤，把这个新模式追加到当前显示输出端口（VGA1）。我们只需引用模式的名称，因为参数已经在前面设置好了。

```
$ xrandr --addmode VGA1 1280x1024_60.00
```

现在，我们可以把屏幕分辨率切换为刚刚追加的值：

```
$ xrandr --output VGA1 --mode 1280x1024_60.00
```

注意，上述这些设置仅在本次会话中有效。

如果你不能确保将要测试的分辨率可用，可以在后面附加一个延迟和一个安全分辨率命令行，就像这样：

```
$ xrandr --output VGA1 --mode 1280x1024_60.00 && sleep 5 && xrandr --newmode "1024x768-safe" 65.00 1024 1048 1184 1344 768 771 777 806 -HSync -VSync && xrandr --addmode VGA1 1024x768-safe && xrandr --output VGA1 --mode 1024x768-safe
```

还有，要把 `VGA1` 改为正确的输出端口名。

EDID 校验和无效

如果前述方法导致引导期间发生 `*ERROR* EDID checksum is invalid` 错误，参阅这里和这里.

也许 `xrandr --addmode` 会返回错误 `X Error of failed request: BadMatch`。NVIDIA 用户请参阅 NVIDIA/Troubleshooting#xrandr BadMatch。`BadMatch` 能指示出无效的 EDID 校验和。要验证确实是这种情况，请以 verbose mode 运行 X 服务（例如：`startx -- -logverbose 6`）然后查阅 Xorg 日志中有关 EDID 错误的信息。

纠正电视机分辨率过扫

With a flat panel TV, w:overscan looks like the picture is "zoomed in" so the edges are cut off.

Check your TV if there is a parameter to change. If not, apply an `underscan` and change border values. The required `underscan vborder` and `underscan hborder` values can be different for you, just check it and change it by more or less.

```
$ xrandr --output HDMI-0 --set underscan on --set "underscan vborder" 25 --set "underscan hborder" 40
```

通过 --transform 纠正电视机过扫

如果underscan不可用另一个解决方案是使用 `xrandr --transform *a,b,c,d,e,f,g,h,i*`，它在输出上应用了一个变换矩阵，参考 xrandr(1) § RandR_version_1.3_options手册页面来了解transform的解释.

举个例子，将横坐标缩放0.8，纵坐标缩放1.04，并将屏幕向右移动35像素，向下移动19像素，参考如下指令。

```
$ xrandr --output HDMI1 --transform 0.80,0,-35,0,1.04,-19,0,0,1
```

HDMI中的Full RGB支持

当Intel驱动没有正确设置HDMI输出时可能出现该种问题。这会限制颜色范围到(16-235)通过Broadcast RGB property，这导致黑色看起来不够黑，它会变成灰色。

要验证你是不是这种情况:

```
$ xrandr --output HDMI1 --set "Broadcast RGB" "Full"
```

屏幕分辨率在屏幕闪一下之后恢复原状

如果你使用GNOME而且你的显示器没有EDID, 上述#添加未检测到的分辨率可能不会工作，当你使用xrandr --output时，屏幕只会闪一下。

你可以试试~/.config/monitors.xml,或者把这个文件删除，然后重启计算机。

更好的解释可以参考这个文章。

参见

- DualScreen Arch wiki page. How to get dual screens with Xrandr
- https://wiki.ubuntu.com/X/Config/Resolution
- Debian Wiki - RandR 1.2 tutorial
- https://bbs.archlinux.org/viewtopic.php?pid=652861
- https://nouveau.freedesktop.org/Randr12.html
- https://wiki.debian.org/XStrikeForce/HowToRandR12
- man xrandr

第 20 章　共 2 节　23 KB

图形用户界面

本章解决的是从字符终端到图形会话那一步：系统装好之后，怎么变成能登录的桌面。《显示管理器》从选型、加载讲到 systemd-logind 与会话配置，并附 LUKS 自动登录解锁 GNOME Keyring、KDE KWallet；《Gamescope》另成一套，讲安装、使用与常见问题。

两节按序读。《显示管理器》必读，进不去桌面多半出在加载与会话配置上；《Gamescope》可跳，不玩游戏时略过即可。最常踩的坑是以为装好就会自动接管启动——它得显式加载并开机自启；它也不是桌面本身，会话没配好，登录后照样空白。

20.1　显示管理器

难度 进阶 | **篇幅** 10.5 KB | **所属** 图形用户界面

本节将讲到

- 显示管理器列表
- 加载显示管理器
- 使用 systemd-logind
- 会话配置
- 提示与技巧

显示管理器)（或登录管理器）通常是一个在启动过程结束时代替默认 shell 显示的图形用户界面。与窗口管理器和桌面环境类似，显示管理器也有很多实现。每个显示管理器通常都能进行一定程度的定制。

显示管理器列表

控制台

- **CDM** — 用 Bash 编写的显示管理器。

: https://github.com/evertiro/cdm || cdmAUR

- **Console TDM** — 用 Bash 编写的 *xinit* 扩展。

: https://github.com/dopsi/console-tdm || console-tdmAUR

- **nodm** — 用于自动登录的显示管理器（自 2017 年以来不再维护）。

: https://github.com/spanezz/nodm || nodm包

- **Ly** — 用于 Linux 和 BSD 的 TUI（类似 ncurses）显示管理器。支持 X 和 Wayland 会话。

: https://github.com/nullgemm/ly || ly包

- **tbsm** — Bash 会话或应用程序启动器。支持 X 和 Wayland 会话。

: https://loh-tar.github.io/tbsm/ || tbsmAUR

- **emptty** — TTY 上的简单 CLI 显示管理器，支持 X 和 Wayland，用Go编写。

: https://github.com/tvrzna/emptty/ || emptty包

- **loginx** — 带有友好 curses 用户界面的 getty/login/xinit 组合。

: https://sourceforge.net/projects/loginx/ || loginxAUR

- **Lemurs** — TUI显示管理器，用Rust编写。支持 X、Wayland 和 TTY shell 会话。

: https://github.com/coastalwhite/lemurs || lemurs包

- **lidm** — 完全彩色且可定制的 TUI 显示管理器，由 C 语言制作。

: https://github.com/javalsai/lidm || lidmAUR

图形界面

- **Entrance** — Enlightenment 显示管理器。

: https://github.com/Obsidian-StudiosInc/entrance || entrance-gitAUR

- **GDM** — GNOME 显示管理器。

: https://wiki.gnome.org/Projects/GDM || gdm包

- **LightDM** — 跨桌面显示管理器，可以使用任何工具包编写的各种前端。

: https://github.com/CanonicalLtd/lightdm/ || lightdm包

- **LXDM** — LXDE 显示管理器。可以独立于 LXDE 桌面环境使用。

: https://sourceforge.net/projects/lxdm/ || lxdm包

- **Plasma 登录管理器** — Plasma 登录管理器；设计为SDDM的继任者。

: https://invent.kde.org/plasma/plasma-login-manager/ || plasma-login-manager包

- **SDDM** — 基于 QML 的显示管理器，KDM 的继任者；推荐用于 Plasma 和 LXQt。

: https://github.com/sddm/sddm || sddm包

- **XDM** — 支持 XDMCP 的 X 显示管理器，适合服务器主机。

: xdm(8) || xorg-xdm包

登录守护进程

- **greetd** — 支持控制台和图形欢迎程序的登录守护程序。

: https://git.sr.ht/~kennylevinsen/greetd || greetd包

加载显示管理器

要在启动时启用图形登录，请启用相应的 systemd 服务：

- GDM — `gdm.service`
- LightDM — `lightdm.service`
- LXDM — `lxdm.service`
- Plasma 登录管理器 — `plasmalogin.service`
- SDDM — `sddm.service`
- SLiM — `slim.service`
- XDM — `xdm.service`

注意：如果安装了 xdm-archlinux，请启用 xdm-archlinux.service

警告：只应启用一个显示管理器服务。

如果默认启动目标是`graphical.target`，应能开箱即用。如果没有，可能需要重置自定义`default.target`符号链接，指向默认`graphical.target`。参见 systemd#更改开机默认启动目标。

启用显示管理器服务后，应在 `/etc/systemd/system/` 中设置符号链接`display-manager.service`。你可能需要用`--force`来覆盖旧的符号链接。

```
$ /etc/systemd/system/display-manager.service
/etc/systemd/system/display-manager.service: symbolic link
to /usr/lib/systemd/system/display-manager.service
```

使用 systemd-logind

为了检查用户会话的状态，你可以使用 *loginctl*。所有 Polkit 动作，比如休眠系统或挂载外部驱动器，开箱即用。

```
$ loginctl show-session $XDG_SESSION_ID
```

会话配置

许多显示管理器会读取 `/usr/share/xsessions/` 目录来获取可用的会话列表。此目录中包含各个桌面环境或窗口管理器的标准桌面文件。一些显示管理器使用单独的 `/usr/share/wayland-sessions/` 来列出特定于 Wayland 的会话。

要向显示管理器的会话列表添加/删除条目，请根据需要在 `/usr/share/xsessions/` 中创建/删除 *.desktop* 文件。典型的 *.desktop* 文件如下所示：

```
[Desktop Entry]
Name=Openbox
Comment=Log in using the Openbox window manager (without a session manager)
Exec=/usr/bin/openbox-session
TryExec=/usr/bin/openbox-session
Icon=openbox.png
Type=Application
```

运行 ~/.xinitrc 会话

安装 xinit-xsessionAUR 后会在显示管理器中提供一个运行 xinitrc 会话的选项。请确保 `~/.xinitrc` 可执行，然后在显示管理器中选择 `xinitrc` 作为会话。

没有窗口管理启动应用程序

也可以在没有窗口修饰、桌面或窗口管理的情况下启动应用程序。例如，要启动 google-chromeAUR，在 `/usr/share/xsessions/` 中创建 `web-browser.desktop`：

```
[Desktop Entry]
Name=Web Browser
Comment=Use a web browser as your session
Exec=/usr/bin/google-chrome --auto-launch-at-startup
TryExec=/usr/bin/google-chrome --auto-launch-at-startup
Icon=google-chrome
Type=Application
```

在这种情况下，登录后，`Exec` 中设置的应用程序将立即启动。关闭应用程序后，会和注销桌面环境或窗口管理器一样回到登录管理器。

需要注意的是大部分图形程序都不支持此环境，因此可能需要手动调整或忍受限制（没有窗口管理器，所以不要期望能够移动或调整*任何*窗口，包括对话框；不过，也许可以在应用程序的配置文件中设置窗口几何形状）。

请参阅 Xinit#在没有窗口管理器的情况下启动应用程序。

提示与技巧

自动启动

大多数显示管理器都会提供 `/etc/xprofile`、`~/.xprofile` 和 `/etc/X11/xinit/xinitrc.d/`。有关详细信息，请参阅 xprofile。

为用户会话设置语言

使用 AccountsService 的显示管理器可以通过以下编辑设置用户会话的区域：

```
/var/lib/AccountsService/users/$USER
[User]
Language=your_locale
```

其中*your_locale*的值类似 `zh_CN.UTF-8`。

也可以使用 D-Bus： `busctl call org.freedesktop.Accounts /org/freedesktop/Accounts/User$UID org.freedesktop.Accounts.User SetLanguage s your_locale`

GNOME和KDE用户也可以在GNOME或KDE设置中设置语言。

注销然后重新登录以使更改生效。

用LUKS自动登录时自动解锁GNOME Keyring/KDE KWallet

自 systemd 版本 255 起，pam_systemd_loadkey 可用于解锁 GNOME/Kwallet Keyring，前提是用户密码与系统的 LUKS 密码短语相符。要实现这一点，你需要在显示管理器中启用自动登录。

- 登录管理器：入门介绍——讨论登录管理器的实现

20.2 Gamescope

难度 进阶 | **篇幅** 12.6 KB | **所属** 图形用户界面

本节将讲到

- 安装
- 使用
- 常见问题
- 另请参阅

Gamescope 是来自 Valve 并用于 Steam Deck 的微混成器. 其目标是提供一种针对游戏量身定制的独立合成器，并支持许多以游戏为中心的功能，例如:

- 分辨率伪装.
- 使用 AMD FidelityFX™ Super Resolution 或 NVIDIA Image Scaling 进行图像上采样.
- 限制帧率.

作为一个微混成器，它被设计为在现有桌面环境之上作为嵌套会话运行，尽管也可以将其用作嵌入式混成器。

安装

Gamescope 可以通过 gamescope包包或gamescope-gitAUR包安装。另外，还有 gamescope-plusAUR包提供了包含不在主线版本中的一些额外补丁的版本。

安装要求

- AMD: Mesa 20.3 或更高
- Intel: Mesa 21.2 或更高
- NVIDIA: 专有驱动 515.43.04 或更高，以及 `nvidia_drm.modeset=1` 内核参数

使用

Gamescope 有很多选项。可以通过在终端执行 `gamescope --help` 命令获取完整选项列表。

从显示管理器运行

参见steam#无窗口管理器的大屏幕模式

从桌面环境运行

下面的命令会用 Gamescope 运行 supertuxkart，强制使用 1920x1080 60 FPS分辨率。

```
$ gamescope -W 1920 -H 1080 -r 60 -- supertuxkart
```

运行 Steam 上的游戏

您可以使用 Gamescope 从 Steam 运行游戏，只需将以下内容添加到游戏启动选项即可：

```
$ gamescope -e -- steam
```

注意：标志 `-e` 告诉 Gamescope 启用 Steam Integration.

你也可以通过添加以下游戏启动参数，用 Gamescope 运行 Steam 上的游戏

```
gamescope -- %command%
```

注意：从Steam启动时，仍然需要设置分辨率、FPS等标志，否则Gamescope将以不正确的分辨率启动。例如，您可以通过与从终端相同的方式执行此操作

```
gamescope -W 1920 -H 1080 -r 60 -- %command%
```

运行Wine

当使用 Gamescope 通过 Wine 运行程序时，简单地用 *wine* 加上可执行程序名称即可。

```
$ gamescope -W 1920 -H 1080 -r 60 -- wine supertuxkart
```

几乎全部流行的 Wine 管理器都支持 Gamescope，例如 Lutris，Bottles，和 PlayOnLinux。使用它们就像安装所需的 Gamescope 软件包并检查“使用 Gamescope”（或类似）选项一样简单。

注意：Wine 管理器还倾向于使用GUI界面来配置基本的Gamescope选项，这使它们成为使用Gamescope的最简单方法。

运行 Flatpak

您也可以将 Gamescope 与 Flatpak 或 Steam 提供的 Wine 管理器一起使用，方法与安装软件包相同。但是，它要求您首先使用以下命令从 Flathub 安装 Gamescope：

```
$ flatpak install gamescope
```

上采样

可以用 `-F fsr` 和 `-F nis` 标志对游戏分别使用 AMD FidelityFX™ Super Resolution 1.0 (FSR) 或 NVIDIA Image Scaling v1.0.3 (NIS) 进行上采样。你也可以用 `-S integer` 进行整数上采样，或者用 `-S stretch` 拉伸画面.

使用 FSR 将 720p 游戏上采样至 1440p：

```
$ gamescope -h 720 -H 1440 -F fsr -- supertuxkart
```

使用 NIS 在内部以 1080p 分辨率运行游戏但是以 4K 显示:

```
$ gamescope -w 1920 -h 1080 -W 3840 -H 2160 -F nis -- supertuxkart
```

低分辨率的游戏通常默认在全屏上使用线性过滤，有时会被拉伸。这在经典的JRPG中尤为明显。要获得像素化外观并保持纵横比：

```
$ gamescope -F nearest -S fit -- tecnoballz
```

在游戏运行时调整使用的滤镜：

- `Super+n` 切换最近邻采样。
- `Super+u` 切换FSR上采样。
- `Super+y` 切换NIS上采样。
- `Super+o` FSR锐度减1。
- `Super+i` FSR锐度加1。

HDR 支持

运行游戏时，Gamescope 是 HDR10 的必要条件 ，要使用此功能， 请给 Gamescope 启动参数加上 `--hdr-enabled` 标志.

注意：Linux上的HDR支持仍处于起步阶段，有许多需要注意的地方。有关详细信息，请参阅HDR monitor support。

Mangoapp

不支持将传统 MangoHud 和 Gamescope 结合使用。相反，应当使用 gamescope 的 `--mangoapp` 标志。这允许 MangoHud 在 gamescope ，而不是在底层应用程序上运行。某些 MangoHud 配置，如显示 FSR 或 HDR 状态，需要使用带有 gamescope 的 mangoapp 才能工作。某些 MangoHud 配置，比如显示 FSR 或 HDR 状态，需要使用 gamescope 才能工作。

可变刷新率

如果显示器支持它，通过传递 `--adaptive-sync` 标志以启用可变刷新率。

Wayland 支持

Gamescope 默认不支持 Wayland 客户端。要启用对 Wayland 客户端的支持，请给 Gamescope 启动参数加上 `--expose-wayland` 标志。

SDR 增益范围

Since SteamOS 3.5.5, Valve has changed the default color rendering for the Steam Deck LCD. The effect is achieved through Gamescope by changing the "wideness" of the gamut for SDR content, which can result in a warmer and more vibrant color appearance depending on the adjustment.

In a Steam game's launch options, simply add `--sdr-gamut-wideness` followed by a value that's equal or between 0-1:

```
gamescope --sdr-gamut-wideness 1 -- %command%
```

常见问题

光标表现不正确

如果光标没有被应用程序捕捉到，例如限制相机的移动或者没有正确地从菜单中消失，那么使用 `--force-grab-cursor` 标志。一些 proton 或 wine 游戏需要这种变通方法。

切换至全屏模式后性能降低

当使用Gamescopes全屏热键 `Meta+f` 时，这是一个已知的错误，如果你遇到这个问题，可以在启动游戏时使用全屏标志 `-f` 来解决。

设置 Gamescopes 优先级

另一个已知的低性能或卡顿的原因是没有正确设置Gamescope的优先级。如果你在Gamescope运行时在终端中看到这样的错误，说明你遇到了现在所说的这种情况：

```
No CAP_SYS_NICE, falling back to regular-priority compute and threads.
Performance will be affected.
```

用下面的命令可以修复：

```
# setcap 'CAP_SYS_NICE=eip' $(which gamescope)
```

警告：使用此命令将导致忽略一些 Vulkan 环境变量： 例如，如果通过设置 `MESA_VK_DEVICE_SELECT` 为 Gamescope 指定要使用的 GPU。看 这里。您可以使用ananicy-gitAUR或类似的方法来绕过这个限制。

在 NVIDIA 环境运行 Flatpak 应用没有窗口

这是因为 Flatpak Gamescope 访问 NVIDIA DRM's GBM 后端失败. 这可以简单地通过下面的命令设置一个环境变量解决：

```
$ flatpak override --env=GBM_BACKENDS_PATH=/usr/lib/x86_64-linux-gnu/GL/nvidia-XXX-YY-ZZ/extra/gbm packageid
```

其中 `packageid` 是 Gamescope 或您要使用Gamescope的应用程序的Flatpak包标识符，如Bottles。将 `nvidia-*XXX*-*YY*-*ZZ*` 替换为当前安装的nvidia驱动程序版本；在Flatpak内部，可以使用以下命令进行查询：

```
$ flatpak run --command=ls packageid /usr/lib/x86_64-linux-gnu/GL
```

其中 `*packageid*` 是任何一个 Flatpak 包标识符，请注意该目录只存在于 Flatpak 内。

注意：

使用 Intel 显卡出现图像损坏

If gamescope outputs corrupted image colors on Intel graphics disabling lossless color compression can be a work-around at the cost of increased memory bandwidth utilization. To disable it pass `INTEL_DEBUG=noccs` environment variable.

启用 HDR 时 VRR 卡顿

If VRR and HDR work independently, but the framerate is unstable when they're both enabled, then you may be hitting issues with long HDR compositing times. See https://github.com/ValveSoftware/gamescope/issues/1006. This only applies to using Gamescope in embedded mode, and not when using gamescope within an existing wayland or X session.

使用AMD显卡时可以使用实验性的 AMD 颜色管理解决这个问题，这会使用硬件 planes 完成最终图像的合成。使用以下两个步骤之一：

Steam Deck 内核

- The Steam Deck Linux kernel linux-neptune-65AUR or a kernel built with the Steam Deck color management patch
- gamescope包 or gamescope-gitAUR

启用实验性AMD颜色管理的 Linux 内核

- Linux 6.8 及以上，编译时 `KCFLAGS` 包含 `-DAMD_PRIVATE_COLOR`，例如 linux-amd-colorAUR
- gamescope patched to work with upstream kernel, available in AUR as gamescope-amd-colorAUR\[损坏的链接：package not found\]

Wayland下在较老的AMD显卡上崩溃

Gamescope 3.14.3 引入了一个Wayland后端，以提升Wayland下的性能。虽然，尽管如此，它目前无法在较旧的AMD GPU上运行。追踪上游问题#1218。

Gamescope在启动时崩溃，例如这样：

```
gamescope: types/wlr_linux_dmabuf_v1.c:526: feedback_compile: Assertion `table_len > 0' failed.
```

您可以通过强制回退旧的 SDL2 后端，添加 `--backend` 标志来解决这个问题：

```
$ gamescope --backend sdl  -- game
```

Swapchain 错误

Swapchain错误的一个常见原因是不正确地尝试使用 mangohud 而不是 mangoapp。请参阅上文。

从 Steam 启动游戏，在大约24分钟后卡顿

如果从steam启动gamescope后，您在大约24分钟后遇到严重的卡顿，那么您可以添加Steam overlay 的 `-e` 标志 或用不同的值覆盖环境变量 `LD_PRELOAD` 来解决这个问题。例如：

```
$ LD_PRELOAD="" gamescope -- %command%
```

参见https://github.com/ValveSoftware/gamescope/issues/163

高回报率的鼠标导致卡顿

在游戏窗口中移动一个高回报率(observed with 4000Hz)的鼠标时，可能导致卡顿或暂时冻结 。设置一个更低的比如 1000Hz 的回报率应该可以绕过这个问题。

另请参阅

- Gamescope Github Page
- Gamescope at Steamtinkerlaunch Github Wiki

Xorg commands

Xcalc 是 Xorg 自带的最小图形客户端，这一章只做三件事：装上它、按习惯配好、说清它能验证什么。它解决的问题很小：图形栈刚搭好、心里没底时，用一个依赖极少、行为好判断的程序，看显示连接与窗口能否起来。整章只有《Xcalc》一节，无可跳过之处，按安装、配置、参见的顺序读即可；只想确认图形环境跑得通，读完安装就够了。

最容易踩的坑是当成能干活的计算器——它只管演示与验证，功能精度都不值得依赖。其次是配置改完看不到变化：设置要等客户端重启才读取，不重开等于没改。还有显示环境没连上时它启动即失败，报错很像没装好，先查图形环境再怀疑安装。

21.1　Xcalc

难度 入门 | **篇幅** 4.7 KB | **所属** Xorg commands

本节将讲到

- 安装
- 配置
- 参见

xcalc 是 X Window System 的科学计算器。

安装

安装 xorg-xcalc包 软件包。

配置

你可以在 X resources 文件中设置几个选项，让 xcalc 看起来比默认的白底黑字更好看。每个按钮的位置、标签、功能和颜色都可以单独更改。更多信息，请参见 xcalc(1) § CUSTOMIZATION。

主题

xcalc 有一个内置主题，模仿 Texas Instruments TI-30 的外观。如何启用该主题，请参见 xcalc(1) § COLORS。

如果你只是想要一些更漂亮的颜色，可以试试这个主题。

```
~/.Xresources
XCalc.geometry:                          200x275
XCalc.stipple:                           true
XCalc.ti.Command.background:             #777777
XCalc.ti.Command.foreground:             White
XCalc.ti.bevel.background:               #111111
XCalc.ti.bevel.screen.DEC.background:    #000000
XCalc.ti.bevel.screen.DEC.foreground:    LightSeaGreen
XCalc.ti.bevel.screen.DEG.background:    #000000
```

```
XCalc.ti.bevel.screen.DEG.foreground:  LightSeaGreen
XCalc.ti.bevel.screen.GRAD.background: #000000
XCalc.ti.bevel.screen.GRAD.foreground: LightSeaGreen
XCalc.ti.bevel.screen.HEX.background:  #000000
XCalc.ti.bevel.screen.HEX.foreground:  LightSeaGreen
XCalc.ti.bevel.screen.INV.background:  #000000
XCalc.ti.bevel.screen.INV.foreground:  Red
XCalc.ti.bevel.screen.LCD.background:  #000000
XCalc.ti.bevel.screen.LCD.foreground:  LightSeaGreen
XCalc.ti.bevel.screen.LCD.shadowWidth: 0
XCalc.ti.bevel.screen.M.background:    #000000
XCalc.ti.bevel.screen.M.foreground:    LightSeaGreen
XCalc.ti.bevel.screen.OCT.background:  #000000
XCalc.ti.bevel.screen.OCT.foreground:  LightSeaGreen
XCalc.ti.bevel.screen.P.background:    #000000
XCalc.ti.bevel.screen.P.foreground:    Yellow
XCalc.ti.bevel.screen.RAD.background:  #000000
XCalc.ti.bevel.screen.RAD.foreground:  LightSeaGreen
XCalc.ti.bevel.screen.background:      #000000
XCalc.ti.button4.background:           Orange3
XCalc.ti.button5.background:           Red4
XCalc.ti.button23.background:          #611161
XCalc.ti.button24.background:          #611161
XCalc.ti.button35.background:          #611111
XCalc.ti.button37.background:          #222262
XCalc.ti.button38.background:          #222262
XCalc.ti.button39.background:          #222262
XCalc.ti.button40.background:          #722222
XCalc.ti.button42.background:          #333373
XCalc.ti.button43.background:          #333373
XCalc.ti.button44.background:          #333373
XCalc.ti.button45.background:          #833333
XCalc.ti.button47.background:          #444484
XCalc.ti.button48.background:          #444484
XCalc.ti.button49.background:          #444484
XCalc.ti.button50.background:          #944444
XCalc.ti.button52.background:          #555595
XCalc.ti.button53.background:          #555595
XCalc.ti.button54.background:          #555595
XCalc.ti.button55.background:          #a55555
```

更改分部标签

如果您愿意，可以将分隔符改为正斜线。

```
XCalc.ti.button35.label: /
```

桌面项

xcalc 不会显示在应用程序菜单中，因为它没有桌面项。要让它出现在菜单中，可以添加一个桌面项。

```
~/.local/share/applications/xcalc.desktop
[Desktop Entry]
Name=XCalc
GenericName=Scientific Calculator
Comment=Scientific calculator for X
Exec=xcalc
Icon=accessories-calculator
Terminal=false
Type=Application
StartupNotify=true
Keywords=calculator;scientific;
Categories=Utility;Calculator;
```

在 *Utility* > *Calculator*应用程序列表中，菜单项将显示为 *XCalc*。

参见

- 更改 xcalc 上的颜色
- Xcalc 主题的神奇之处
- xcalc 打扮得花枝招展！

无障碍

无障碍功能决定的是系统能不能装上去，而不是装好之后在桌面上调整的一项偏好。本章只有一节，按安装前、安装、配置系统、重新启动依次展开，是一条完整流程，需要无障碍支持的读者必须照这个顺序走完，某一环没做对，后面的步骤都无从补救；如果安装介质本来就能正常操作，用不上这些功能，可以跳过去读别的章节。最容易踩的坑是把无障碍当成装完再配的事：它要在安装前就启用，从启动安装介质那一步起就生效，等系统装好再想打开，前面的界面早已无法操作。

22.1　使用无障碍功能安装 Arch Linux

难度 入门 | **篇幅** 3.1 KB | **所属** 无障碍

本节将讲到

- 安装前
- 安装
- 配置系统
- 重新启动

Arch Linux的官方安装媒介提供了多种辅助功能（Accessibility）：

- 朗读功能由espeakup包提供
- 盲文终端支持由brltty包提供

本文档用于说明如何使用上述辅助功能安装Arch Linux。

注意：本页面仅说明与官方Installation guide不同的步骤。

安装前

启动安装环境

注意：安装媒介的引导加载程序在启动默认菜单项前有15秒的等待时间。

当安装媒介启动后，按下，然后按回车以使用朗读功能。

USB盲文显示设备会通过udev被自动检测到。

多个音频输出设备

如果电脑拥有多个音频输出设备，将会听到如下信息：*Please select your sound card for speech output*（请选择朗读功能使用的音频输出设备）。

在希望使用的音频输出设备听到“嘀”声时，按回车选择该音频输出设备。

安装

安装必要的软件包

在新安装的系统使用朗读功能，需安装espeakup包和alsa-utils包。如果使用盲文显示设备，需要安装brltty包。

在使用pacstrap(8)安装系统时，使用以下命令添加所需的软件包：

```
# pacstrap /mnt base linux linux-firmware espeakup alsa-utils
```

配置系统

音频输出设备

如果检测到 #多个音频输出设备，需要使用以下命令将安装媒介生成的`/etc/asound.conf`复制到新安装的系统：

```
# cp /etc/asound.conf /mnt/etc/
```

激活开机启动

为了在进入新安装的系统时使用朗读功能，需要将`espeakup.service`设置为开机自动启动（方法参照systemd#使用单元）。

重新启动

在启动至新安装的系统后，朗读功能应该会自动开始工作。

深度

本章只解决一件事：在已经备好的图形栈之上，用 deepin-wine 让一批常见的 Windows 应用在 Arch 里跑起来。全章只有《Deepin-wine》一节，安装与故障排除两步必读——前者决定环境是否就绪，后者几乎总会翻到；应用程序列表当作可查的索引，不必通读，参考条目留到核对版本与出处时再翻。

最容易踩的坑是把"能装上"当成"能用"：deepin-wine 提供的是兼容层加一套打包约定，安装走通只说明环境就绪，并不保证应用一定正常。另一类误解是一遇故障就回头重装，其实多数现象在故障排除里已有对应条目，按现象对号入座比推翻重来省事。

23.1　Deepin-wine

难度 入门 | **篇幅** 5.1 KB | **所属** 深度

本节将讲到

- 安装
- Deepin-wine应用程序列表
- 故障排除
- 参考

Deepin-wine是由武汉深之度科技有限公司开发的基于Wine的兼容层, 它附属于 deepin Linux.

Deepin-wine 的应用程序几乎是开箱即用的，漏洞（相较于wine）会更少。

安装

安装以下包之一:

- **Deepin-wine5** — Deepin-wine5.

: https://www.deepin.org || deepin-wine5AUR

- **Deepin-wine5 stable** — Deepin-wine5 stable.

: https://www.deepin.org || deepin-wine5-stableAUR

- **Deepin-wine6 stable** — Deepin-wine6 stable.

: https://www.deepin.org || deepin-wine6-stableAUR

- **Deepin-wine8 stable** — Deepin-wine8 stable.

: https://www.deepin.org || deepin-wine8-stableAUR

Deepin-wine应用程序列表

- **Deepin-wine DingTalk** — DingTalk Client on Deepin-wine.

: https://www.dingtalk.com/ || deepin.com.dingtalk.comAUR

- **Deepin-wine QQ Light** — QQ Light on Deepin-wine.

: https://im.qq.com/ || deepin.com.qq.im.lightAUR

- **Deepin-wine TIM** — TIM on Deepin-wine.

: https://office.qq.com/, https://tim.qq.com/ || com.qq.office.deepinAUR, deepin-wine-timAUR, or from Spark Store: com.qq.tim.sparkAUR

- **Deepin-wine WeChat** — WeChat on Deepin-wine.

: https://pc.weixin.qq.com/ || com.qq.weixin.deepinAUR, or from Spark Store: com.qq.weixin.sparkAUR

- **Deepin-wine Weixin Work** — Weixin Work on Deepin-wine.

: https://work.weixin.qq.com/ || com.qq.weixin.work.deepinAUR

故障排除

Deepin-wine应用程序启动失败

在不基于GNOME的桌面环境中，例如KDE，Deepin-wine应用程序将不会启动。

要想解决这个问题，你需要安装 xsettingsd包, 然后设置 `/usr/bin/xsettingsd` 到自启动项 , 参考自动启动#桌面环境.

重新登录后即可解决。

参考

- deepin Wiki\[失效链接 2022-09-17 ⓘ\]
- 此项目用于将 Deepin-wine 移植到 Ubuntu/Debian

第 24 章　共 1 节　32 KB

窗口管理器

本章解决的是“窗口管理器在图形界面里到底管什么，以及该选哪一个”这两个互相牵连的问题：概述划清职责边界，窗口管理器列表给出可选项与各自的定位，参见留出继续深挖的线索。

读法上，概述必读，它是后面所有选择的判断依据；窗口管理器列表属查阅型内容，不必从头读到尾，先想清楚日常需要什么样的操作方式，再按图索骥；参见只是延伸线索，时间紧可整段跳过。最容易踩的坑有两个：一是把窗口管理器当成桌面环境的同义词，选完才发现还缺一整套配套组件；二是把列表当成推荐榜单，逐条试装、反复更换，却没先确定自己的需求。先定需求，再查列表，比反过来省力得多。

24.1　窗口管理器

难度 高级 | **篇幅** 31.5 KB | **所属** 窗口管理器

本节将讲到

- 概述
- 窗口管理器列表
- 参见

翻译状态：

- 本文（或部分内容）译自 Window manager，最近一次同步于 2026-07-22，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

关于 窗口管理器:

: 窗口管理器是在一个使用图形化界面的窗口系统中控制窗口)的放置与配色的系统软件。

这可以是桌面环境的一部分，也可以被单独使用。

注意：窗口管理器是 Xorg 特有的。Wayland 下的窗口管理器的类似物被称为混成器，因为它们同时也发挥混成窗口管理器的作用。

概述

窗口管理器是控制绘制各种图形应用程序的框架（“窗口”）的外观和行为的 X 客户端。它们决定窗口的边框、标题栏、大小和调整大小的能力，并经常提供其他功能，如为粘附像 Window Maker 这样的 dockapps 保留区域，或像 Fluxbox 那样为窗口做标签。一些窗口管理器甚至包含了一些简单的工具，如启动程序的菜单或配置窗口管理器本身的工具。

Extended Window Manager Hints 规范用于允许窗口管理器以标准方式与服务器和其他客户端进行交互。

一些窗口管理器是作为更全面的桌面环境的一部分而开发的，通常允许提供的其他应用程序更好地相互交互，从而给用户带来更一致的体验，通过桌面图标、字体、工具栏、墙纸或桌面小部件等功能完善用户体验。

其他窗口管理器则被设计成*独立*使用，让用户完全自由地选择要使用的其他应用程序。这使得用户可以根据自己的具体需求，创建一个更加轻量级和个性化的环境。如果需要的话，像桌面图标、工具栏、墙纸或桌面小部件等“额外的东西”，将需要依赖于额外的专用应用程序。

一些独立的窗口管理器也可以用来替换桌面环境的默认窗口管理器，就像一些面向桌面环境的窗口管理器也可以独立使用一样。

安装窗口管理器需要能够正常运行的 X 服务器。详细信息请参阅 Xorg。

类型

- 堆叠（又名悬浮) 窗口管理器提供在 Windows 和 OS X 等商业操作系统中使用的传统桌面隐喻。不同窗口就像桌子上的白纸一样，且可以相互重叠。有关可用的 Arch Wiki 页面，请参见 分类:堆叠式窗口管理器。
- 平铺窗口管理器“平铺”窗口，以便没有重叠。他们通常非常广泛地使用键绑定，并且对鼠标的依赖较少（或不依赖）。平铺窗口管理器可以是手动的，也可以提供预定义的布局，甚至两者兼而有之。有关可用的 Arch Wiki 页面请参见 分类:平铺式窗口管理器。
- 动态窗口管理器可以在平铺或堆叠窗口布局之间动态切换。有关可用的 Arch Wiki 页面请参见 分类:动态窗口管理器。

有关窗口管理器的比较，请参见平铺式窗口管理器比较和 Wikipedia:窗口管理器对比。

窗口管理器列表

堆叠窗口管理器

- **2bwm** — 快速浮动窗口管理器，其特点是具有两个边框，使用 XCB 库编写，是基于 Michael Cardell 编写的 mcwm 的衍生版本。 在 2bwm 中，所有操作均可通过键盘完成，但也可以使用指示设备来移动、调整大小和提升/降低窗口。最近，该管理器的名称已从 mcwm-beast 更改为 2bwm。

: https://github.com/venam/2bwm || 2bwmAUR

- **9wm** — 基于 Plan 9 的 rio 窗口管理器的 X11 窗口管理器。

: https://github.com/9wm/9wm || 9wmAUR

- **aewm++** — Minimal window manager based on aewm. It keeps aewm's minimalist look and feel but is written in C++, and adds some modern features like virtual desktops.

: https://github.com/frankhale/aewmpp || aewm++AUR

- **AfterStep** — AfterStep 是适用于 Unix X 窗口系统的窗口管理器。最初基于 NeXTStep 界面的外观和感觉，它为终端用户提供了一致、清洁和优雅的桌面环境。AfterStep 的目标是提供灵活的桌面配置、改善美观度和高效地利用系统资源。

: http://www.afterstep.org/ || afterstep-gitAUR

- **berry** — Berry 是适用于 Unix 系统的健康、紧凑的窗口管理器，用 C 语言编写。它通过命令行客户端进行控制，允许用户通过热键守护进程，例如 sxhkd 来控制窗口，或通过 Shell 脚本拓展功能。它提供了可扩展的主题选项，包括双边框、标题栏和窗口文本。Berry 会直观地将新窗口放置在空闲空间中，并支持虚拟桌面。

: https://berrywm.org/ || berry-gitAUR

- **Blackbox** — Blackbox 是 X 窗口系统的快速、轻量级的窗口管理器，没有烦人的库依赖。Blackbox 使用 C++ 构建，并包含完全原创的代码（尽管其图形实现与 WindowMaker 类似）。

: http://blackboxwm.sourceforge.net/ || blackbox包

- **Compiz** — 这是一个使用 GLX_EXT_texture_from_pixmap 将重定向的顶层窗口绑定到纹理对象的 OpenGL 合成管理器。它拥有灵活的插件系统，并且设计用于在大多数图形硬件上良好运行。

: https://launchpad.net/compiz || compizAUR, compiz-coreAUR

- **CTWM** — Claude's Tab Window Manager, based on twm. It provides titlebars, shaped windows, virtual screens (workspaces), several forms of icon management, user-defined macro functions, click-to-type and pointer-driven keyboard focus, and user-specified key and pointer button bindings.

: https://www.ctwm.org/ || ctwm包

- **cwm** — 最初是从 evilwm 演变而来，但后来完全重新编写。cwm 的目标是简单，并提供了一些有用的功能，如搜索窗口。

: https://github.com/chneukirchen/cwm || cwmAUR

- **E16** — Predecessor of Enlightenment, a fast, flexible and extensible window manager.

: https://www.enlightenment.org/e16 || e16包

- **eggwm** — 一个轻量级的 QT4/QT5 窗口管理器。

: eggwm-qt5AUR || eggwmAUR

- **EMWM** — Fork of the Motif Window Manager with fixes and enhancements. Supports Xinerama, Xrandr, UTF-8/Xft, and relevant EWMH.

: https://fastestcode.org/emwm.html || emwmAUR

- **Enlightenment** — Enlightenment 不仅是 Linux/X11 及其他平台上的窗口管理器，还是一整套库，可帮助您创建美观的用户界面，比传统方式更轻松，无需与传统工具包和窗口管理器进行斗争。

: https://www.enlightenment.org/ || enlightenment包

- **evilwm** — 这是一款适用于X 窗口系统的极简主义窗口管理器。这里的"极简主义"并不意味着它过于简陋无法使用，而是指它省略了许多使其他窗口管理器*难以使用的功能*。

: https://www.6809.org.uk/evilwm/ || evilwmAUR

- **Fluxbox** — 这是一个基于 Blackbox 0.61.1 代码的 X 窗口管理器。它对资源要求非常低，并且易于操作，但却具备许多功能，可以提供简单且极快的桌面体验。它使用 C++ 构建，并且根据 MIT 许可证进行许可。

: https://github.com/fluxbox/fluxbox || fluxbox包

- **Flwm** — 试图将多个窗口管理器的最佳思想结合在一起。主要影响和代码基础来自 Chris Cannam 的 wm2。

: http://flwm.sourceforge.net/ || flwmAUR

- **FVWM** — 这是一个功能非常强大的、符合 ICCCM 标准的多虚拟桌面窗口管理器，适用于 X 窗口系统。开发活跃，支持也非常出色。

: https://www.fvwm.org/ || fvwmAUR

- **Gala** — 来自 elementaryos 的漂亮窗口管理器，是 Pantheon 的一部分。也是个基于libmutter的混成管理器。

: https://launchpad.net/gala || gala包

- **Goomwwm** — X11 窗口管理器是用 C 语言实现的，是一个净室软件项目。它以最小的浮动布局管理窗口，同时为窗口的切换、调整大小、移动、标记和平铺提供了灵活的键盘驱动控制。它还具有快速、轻量级、无模式、Xinerama 感知和尽可能兼容 EWMH 的特点。

: https://github.com/seanpringle/goomwwm || goomwwmAUR

- **IceWM** — Xorg 窗口管理器。 IceWM 的目标是速度、简单性，并且不会妨碍用户

: https://ice-wm.org/ || icewm包

- **jbwm** — 基于 evilwm，最小配置大小约为 16kb，专注于小二进制大小和可用性，结合可选的标题栏和 XFT 标题栏字体渲染作为编译时选项，jbwm 还具有比 evilwm 更易于使用键绑定的特征。

: https://github.com/jefbed/jbwm || jbwmAUR

- **JWM** — X11 窗口系统的窗口管理器。JWM 由 C 语言编写，只使用 Xlib。

: https://joewing.net/projects/jwm/ || jwm包

- **Karmen** — X 的窗口管理器，由 Johan Veenhuizen 编写。它的设计宗旨是"只需工作"。它没有配置文件，也不依赖除了 Xlib 之外的其他库。输入焦点模型是点击到焦点。Karmen 的目标是符合 ICCCM 和 EWMH 标准。

: http://karmen.sourceforge.net/ || karmenAUR

- **KWin** — 自 KDE 4.0 以来的标准 KDE 窗口管理器，它也是一个合成管理器。这使得 KWin 能够提供类似于 Compiz 的高级图形效果，同时还提供以前 KDE 版本的所有功能（例如与 KDE 其余部分的非常好的集成、高级可配置性、防止焦点窃取、经过充分测试的窗口管理器、对行为异常的应用程序/工具包的稳健处理等）。还用作 Wayland的合成器。

: https://techbase.kde.org/Projects/KWin || kwin包

- **lwm** — Xorg的窗口管理器，它试图避开你的脸。没有图标，没有按钮栏，没有图标停靠，没有根菜单，什么都没有：如果你想要这一切，那么其他程序可

以提供它。也没有可配置性：如果你想要，你需要一个不同的窗口管理器;它以磁盘空间的邪恶占用和物理内存吞并的方式帮助你操作系统。

: http://www.jfc.org.uk/software/lwm.html || lwm包

- **Marco** — TMATE 窗口管理器，Metacity 的分支。

: https://github.com/mate-desktop/marco || marco包

- **Metacity** — 这个窗口管理器力求安静、小巧、稳定、继续工作，并远离您的注意。它被传统的 GNOME 2 和 GNOME 闪回会话使用，并被 Mutter 取代。

: https://blogs.gnome.org/metacity/ || metacity包

- **Muffin#Muffin)** — Cinnamon 的窗口管理器和合成器。基于Clutter，使用 OpenGL，是Mutter分支。它不能在 Cinnamon 之外使用。

: https://github.com/linuxmint/muffin/ || muffin包

- **Mutter)** — 基于Clutter的 GNOME窗口和合成管理器，使用 OpenGL。还提供 Wayland 合成器。

: https://gitlab.gnome.org/GNOME/mutter/ || mutter包

- **MWM** — Motif 窗口管理器 （MWM） 是基于 Motif 工具包的 X 窗口管理器。

: https://sourceforge.net/projects/motif/ || openmotif包

- **Openbox** — 高度可配置的窗口管理器，具有广泛的标准支持。*\box 视觉样式以其简约的外观而闻名。Openbox 使用* \box 视觉样式，相比比旧版实现，它为主题开发者提供了更多的选项。自 2010 年以来一直是“功能完整”，但继续维护。.

: http://openbox.org/ || openbox包

- **PekWM** — 曾经基于aewm++，现已不再类似于aewm++的窗口管理器。它具有大大扩展的功能集，包括窗口分组（类似于 Ion、PWM 或 Fluxbox）、自动属性、Xinerama、keygrabber that supports keychains，等等。

: https://www.pekwm.org/ || pekwm包

- **progman** — Simple X11 window manager modeled after the Windows 3 era.

: https://github.com/jcs/progman || progmanAUR

- **Sawfish** — 可扩展的窗口管理器，使用基于 Lisp 的脚本语言。与大多数窗口管理器相比，它的策略非常简单。它的目标只是以最灵活、最有吸引力的方式管理窗口。所有高级窗口管理器功能都是用 Lisp 实现的，以便将来进行扩展或重新定义。

: https://sawfish.wikia.com/wiki/Main_Page || sawfishAUR

- **sowm** — 自用的简单窗口管理器，提供全屏切换、窗口居中和混合工作流程（即鼠标和键盘）。

: https://github.com/dylanaraps/sowm || sowmAUR

- **TinyWM** — 作为极简主义的练习而创建的 Tiny Window Manager。它可能有助于学习创建窗口管理器的一些非常基本的知识。它由大约 50 行 C 组成。还有一个使用 python-xlib 的 Python 版本。

: http://incise.org/tinywm.html || tinywmAUR

- **twm** — X 的简单窗口管理器，Xorg 自 1989 年以来一直使用的默认/回退。它提供标题栏、形状窗口、多种形式的图标管理、用户定义的宏函数、单击键入和指针驱动的键盘焦点以及用户指定的键和指针按钮绑定。

: https://gitlab.freedesktop.org/xorg/app/twm/ || xorg-twm包

- **UWM** — TUDE 的终极窗口管理器。

: http://udeproject.sourceforge.net/ || udeAUR

- **VTWM** — One of many TWM descendants, implements a Virtual Desktop, meaning that what is currently on screen is just a portion of a larger workspace. What portion of the virtual desktop that is displayed, and whatever windows might be visible within it, are simple point-and-click operations within a scaled representation of the workspace.

: http://www.vtwm.org/ || vtwmAUR

- **WindowLab** — 这是一个小巧简单、设计新颖的窗口管理器。它采用了点击即焦点但不自动提升的策略，窗口大小可通过一次操作改变一个或多个边界，并且创新地将菜单栏与任务栏共享屏幕上的同一部分。通过限制鼠标指针的移动，防止窗口标题栏超出屏幕边缘。而且，在合适的时候，鼠标指针也被限制在任务栏/菜单栏上，以便更容易点击目标菜单项。

: https://github.com/nickgravgaard/windowlab || windowlabAUR

- **Window Maker** — 最初设计为提供GNUstep桌面环境的整合支持的X11窗口管理器。它在各个方面都尽可能地复制了NEXTSTEP用户界面的优雅外观和感觉。它快速、功能丰富、易于配置和使用。而且它是自由软件，来自世界各地的程序员做出了贡献。

: https://windowmaker.org/ || windowmakerAUR

- **WM2** — wm2是一个X的窗口管理器。它提供了独特的窗口装饰风格，并只提供作者认为在窗口管理器中感到舒适的最少功能。wm2不可配置，除非通过编辑源代码并重新编译来实现，它真正适用于那些不希望他们的窗口管理器过于友好的用户。

: https://www.all-day-breakfast.com/wm2/ || wm2AUR

- **worm** — Worm是一个基于标签的X11动态窗口管理器。它支持浮动布局和主栈平铺布局，并采用开放式开发方式。它还使用Nim编程语言编写。

: https://github.com/codic12/worm || worm-gitAUR

- **Xfwm** — Xfce 窗口管理器负责管理应用程序窗口在屏幕上的放置，提供美丽的窗口装饰，管理工作区或虚拟桌面，并原生支持多屏幕模式。它提供了自己的合成管理器（来源于 X.Org 合成扩展），以实现真正的透明度和阴影效果。Xfce 窗口管理器还包括一个键盘快捷键编辑器，用于用户特定命令和基本窗口操作，并提供了一个高级调整的首选项对话框。

: https://docs.xfce.org/xfce/xfwm4/start || xfwm4包

平铺窗口管理器

- **Bspwm** — bspwm 是一个平铺式窗口管理器，将窗口表示为完全二叉树的叶子节点。它支持EWMH和多个监视器，并通过消息进行配置和控制。

: https://github.com/baskerville/bspwm || bspwm包

- **EXWM** — EXWM（Emacs X Window Manager）是一个完整功能的平铺式X窗口管理器，构建在 XELB 之上，专为Emacs设计。它具有完全基于键盘操作的特点，混合布局模式（平铺和堆叠），动态工作区支持，ICCCM/EWMH 兼容，RandR（多显示器）支持，并且内置了系统托盘。

: https://github.com/ch11ng/exwm || emacs-exwm-gitAUR

- **Herbstluftwm** — 这是一个使用Xlib和Glib的手动平铺窗口管理器，适用于X11。其布局是基于将框架分割成子框架，这些子框架可以再次分割或填充窗

口（类似于i3/musca）。标签（工作区或虚拟桌面等）可以在运行时添加/删除。每个标签都包含自己的布局。每个显示器只显示一个标签。标签与显示器无关（类似于 xmonad）。它通过从herbstclient发出的IPC调用在运行时进行配置。因此，配置文件只是在启动时运行的脚本（类似于wmii/musca）。

: https://herbstluftwm.org || herbstluftwm包

- **i3** — i3 是一个平铺窗口管理器，完全从头开始编写。i3 的创建是因为当时开发人员最喜欢的窗口管理器 wmii 没有提供一些我们想要的功能（例如正确的多监视器支持），存在一些错误，并且在相当长的时间内没有进展，而且不容易进行修改（源码注释/文档完全缺失）。值得注意的区别在于多监视器支持和树形概念。为了速度，i3 没有实现wmii的Plan 9接口。

: https://i3wm.org/ || i3-wm包

- **Larswm** — 一个基于9wm的平铺窗口管理器。

: http://porneia.free.fr/larswm/larswm.html || larswmAUR

- **LeftWM** — 一个用 Rust 编写的平铺窗口管理器。

: https://leftwm.org || leftwmAUR

- **Notion** — Tiling是一种为X窗口系统设计的平铺式窗口管理器，它利用“平铺”和“选项卡”窗口。
- 平铺：您将屏幕分割成不重叠的“瓷砖”。每个窗口占用一个瓷砖，并最大化到该瓷砖。
- 标签页：一个瓷砖可以包含多个窗口 - 它们将被“标签页化”。
- 静态模式：大多数平铺窗口管理器都是“动态”的，这意味着它们在窗口出现和消失时自动调整大小和移动瓷砖。而Notion则不会自动更改平铺方式，它采用静态方式进行。

: Notion是Ion3的一个分支。 : https://notionwm.net/ || notion包

- **Ratpoison** — Ratpoison 是一个简单的窗口管理器，没有庞大的库依赖，没有花哨的图形界面，没有窗口装饰，也不依赖鼠标。它在很大程度上模仿了在虚拟终端市场上表现出色的 GNU Screen。Ratpoison 通过一个简单的文本文件进行配置。Ratpoison的信息栏有点不同，它只在需要时显示。它既可以作为应用程序启动器，也可以作为一个通知栏。Ratpoison 不包括系统托盘。

: https://www.nongnu.org/ratpoison/ || ratpoison包

- **Stumpwm** — Stumpwm 是一个完全使用 Common Lisp 编写的平铺式、键盘驱动的X11窗口管理器。Stumpwm 试图在可自定义的同时保持视觉上的最简化。它提供了多种钩子来添加个性化定制，并且可以在运行时重新配置和重新加载。它没有窗口装饰、图标、按钮和系统托盘。它的信息栏可以设置为始终显示或只在需要时显示。

: https://stumpwm.github.io/ || stumpwm包

- **ZWM** — A minimalistic tiling window manager written in C, it uses the XCB api to communicate with Xorg server. It supports multi-monitors, supports a subset of EWMH specs, and offers extensive configurability.

: https://github.com/Yazeed1s/zwm || zwm-gitAUR

动态窗口管理器

- **awesome** — Awesome 是一个高度可配置的、面向下一代的X框架窗口管理器。它非常快速、可扩展，并且 在GNU GPLv2 许可下发布。使用 Lua 进行配置，它内置了系统托盘、信息栏和启动器。还可以使用Lua编写扩展。与 Xlib 相比，Awesome 使用 XCB，可能会提高速度。Awesome 还具有其他功能，例如用于通知 -daemon 的早期替代品、类似 *box 窗口管理器的右键菜单等。

: https://awesomewm.org/ || awesome包

- **dk** — A list based tiling window manager in the vein of dwm, bspwm, and xmonad.

: https://bitbucket.org/natemaia/dk/src/master/ || dkAUR

- **dwm** — X 的动态窗口管理器。它以平铺、单占据和浮动布局管理窗口。所有布局都可以动态应用，优化正在使用的应用程序和执行的任务的环境。它不包括系统托盘或自动启动器，尽管 dmenu 与其集成得很好，因为它们来自同一作者。它没有文本配置文件。配置必须通过修改 C 源代码完成，每次更改都必须重新编译和重新启动。

: https://dwm.suckless.org/ || dwmAUR

- **echinus** — echinus 是一个简单且轻量级的 X11 平铺和浮动窗口管理器。它最初是从 dwm 分支出来的，但配置更加简单。随着发展，echinus 成为了一个功能齐全的重新父窗口管理器，并支持 EWMH 协议。它还配备了一个兼容 EWMH 的面板/任务栏，称为 ouricoAUR。

: http://plhk.ru || echinusAUR

- **FrankenWM** — 基本上是 Monsterwm 的浮动布局改进版。在基本的窗口管理器上添加了以下功能：更多的布局选项（斐波那契布局、等分布局、双栈布局），可以动态调整间隔（以及边框），最小化/最大化单个窗口，隐藏/显示所有窗口，可单独调整主窗口和副窗口的大小，反转堆叠次序。

: https://github.com/sulami/FrankenWM || frankenwmAUR

- **Qtile** — Qtile是一个功能齐全且可定制的平铺窗口管理器，用Python编写。Qtile 简单、轻巧且可扩展。你可以轻松编写自己的布局、小部件和内置命令。它完全使用 Python 编写和配置，这意味着你可以利用语言的全部能力和灵活性，使其适应你的需求。

: https://github.com/qtile/qtile || qtile包

- **spectrwm** — 这是一个基于X11的小型动态平铺窗口管理器，主要受到 xmonad 和 dwm 的启发。它试图保持低调，以便宝贵的屏幕空间可以用于更重要的事情。它具有合理的默认设置，并通过文本文件进行配置。它是由黑客编写给黑客的，努力保持小巧、紧凑和快速。它还具有内置状态栏，通过用户定义的脚本进行数据传递。

: https://github.com/conformal/spectrwm/ || spectrwm包

- **Wingo** — Wingo 是一个功能齐全的混合窗口管理器，完全支持多显示器工作区，并且既不是浮动模式也不是平铺模式的思考后想法。这使得你可以在一个工作区使用平铺模式，而在另一个工作区使用浮动模式。Wingo 可以使用自己的命令语言进行脚本编写，具有完全可定制的主题，并支持用户定义的钩子。Wingo 是用 Go 语言编写的，没有运行时依赖。它提供了高度的灵活性和可扩展性，使得用户能够根据自己的需求来定制和扩展窗口管理器的功能。

: https://github.com/BurntSushi/wingo || wingo-gitAUR

- **xmonad** — XMonad 是一个动态平铺的 X11 窗口管理器，它使用 Haskell 编写和配置。在普通的窗口管理器中，你要花费大部分时间来调整和搜索窗口的位置。XMonad 通过自动化这些过程来简化工作。XMonad 的配置是使用 Haskell 语言完成的。对于所有的配置更改，必须重新编译 xmonad，因此需要安装 Haskell 编译器（超过100MB）。一个名为 xmonad-contrib包 的大型库提供许多额外的功能和扩展。

: https://xmonad.org/ || xmonad包

参见

- https://www.gilesorr.com/wm/
- https://www.slant.co/topics/390/~best-window-managers-for-linux
- https://l3net.wordpress.com/2013/03/17/a-memory-comparison-of-light-linux-desktops/
- http://www.xwinman.org/others.php

通知

桌面通知常被当成顺手的一环：程序要提醒用户什么，弹一个窗口就行。真正落地时它牵涉两端——应用一侧通过 Libnotify 把请求发出去，会话一侧必须有通知服务器接住并负责绘制。缺了任何一半，屏幕上都不会有反应，也很难判断问题出在哪一端。本章只处理这条链路。

建议先读 Libnotify 与通知服务器，把两端职责分清，这是后面所有讨论的前提；在编程中的使用面向需要调用它的读者，只用桌面的读者可以跳过；提示和技巧与故障排除不必通读，动手之后按需回查更有用。最容易踩的坑，是把 Libnotify 当成通知本身，以为装上它通知就必然出现，却忽略了真正显示通知的是通知服务器。

25.1　桌面通知程序

难度 高级 | **篇幅** 36.7 KB | **所属** 通知

本节将讲到

- Libnotify
- 通知服务器
- 在编程中的使用
- 提示和技巧
- 故障排除

翻译状态：

- 本文（或部分内容）译自 Desktop notifications，最近一次同步于 2024-10-20，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： 本文的翻译不完全。（在 Talk:桌面通知程序# 中讨论）

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：中文与英文之间没有空格，部分标点符号使用不正确，影响阅读体验。（在Talk:桌面通知程序讨论）

桌面通知是种很小的被动弹出对话框，以异步方式通知用户特定事件。

Libnotify

Libnotify 是桌面通知规范的实现，它为 GTK 和 Qt 应用程序提供支持，并且独立于桌面：它已经被许多开源应用程序（如 Evolution 和 Pidgin）使用。可以通过 libnotify包安装。

要使用 libnotify，您必须安装通知服务器。

通知服务器

内置通知

Cinnamon, Deepin, Enlightenment, GNOME, GNOME Flashback 和 KDE Plasma 使用自己的实现来显示通知，并且无法替换。他们的通知服务器在登录时自动启动，以通过 DBus 接收来自应用程序的通知。

独立通知

在其他桌面环境中，需要使用自动启动选项启动通知服务器。 或者，通过将通知服务器作为 D-Bus的服务，通知服务器可以在第一次调用时自动启动。大多数通知服务器已经在`/usr/share/dbus-1/services`目录中。对于某些实现，例如 notification-daemon包包，需要在用户 D-Bus 服务目录（`$XDG_DATA_HOME/dbus-1/services`） 中手动创建一个服务：

```
org.freedesktop.Notifications.service
[D-BUS Service]
Name=org.freedesktop.Notifications
Exec=/usr/lib/notification-daemon-1.0/notification-daemon
```

每当应用程序通过向 `org.freedesktop.Notifications`发送信号来发送通知时，如果他尚未激活，则 D-Bus 将激活`/usr/lib/notification-daemon-1.0/notification-daemon`。

您还可以选择以下实现之一：

- **Deadd Notification Center** — 通知中心受Dunst启发的一个通知守护进程。

: https://github.com/phuhl/linux_notification_center || deadd-notification-centerAUR

- **Dunst** — 适用于 Linux 的极简通知守护进程，旨在很好地适应 dwm等极简窗口管理器。自 1.6 n版本起包括对 Wayland 的支持。

: https://dunst-project.org/ || dunst包

- **fnott** — 键盘驱动的轻量级 Wayland 通知守护程序，适用于基于 wlroots 的合成器。

: https://codeberg.org/dnkl/fnott || fnott包

- **LXQt Notification Daemon** — LXQt的通知服务。

: https://github.com/lxde/lxqt-notificationd || lxqt-notificationd包

- **mako** — 适用于Wayland的轻量级通知守护程序;它目前在Sway和 river包上运行。

: https://github.com/emersion/mako || mako包

- **MATE Notification Daemon** — MATE的通知服务。

: https://github.com/mate-desktop/mate-notification-daemon/ || mate-notification-daemon包

- **Notification Daemon** — 最初的通知服务。

: https://gitlab.gnome.org/Archive/notification-daemon || notification-daemon包 : 你可以通过`/usr/lib/notification-daemon-1.0/notification-daemon`手动运行它

- **Notify OSD** — Unity的通知服务.

: https://launchpad.net/notify-osd || notify-osd包

- **sandsmark-notificationd** — 可以静音的最小化通知守护程序

: https://github.com/sandsmark/sandsmark-notificationd || sandsmark-notificationd-gitAUR\[损坏的链接：package not found\]

- **statnot** — 小巧轻便的通知守护进程，可以将通知输出到根窗口的标题、stdout 或 FIFO 管道，使其与平铺窗口管理器完美集成。

: https://github.com/halhen/statnot || statnotAUR

- **swaync** — 用于Sway的简单基于 GTK 的通知守护程序。

: https://github.com/ErikReider/SwayNotificationCenter || swaync包

- **twmn** — 用于平铺窗口管理器的通知系统。

: https://github.com/sboli/twmn || twmn-gitAUR

- **wired** — 轻量级通知守护进程，具有高度可定制的布局块，用 Rust 编写。

: https://github.com/Toqozz/wired-notify || wiredAUR

- **Xfce Notification Daemon** — Xfce的通知服务

: https://docs.xfce.org/apps/notifyd/start || xfce4-notifyd包 : 你可以通过 {`/usr/lib/xfce4/notifyd/xfce4-notifyd`手动运行它 : > 提示：要配置 xfce4-notifyd，请运行以下命令：`xfce4-notifyd-config`.

在编程中的使用

**这一章节正在考虑移除。**

原因: API reference pages are out of scope. (在 ArchWiki talk:Maintenance Team#Dealing with API reference pages 讨论)

您可以通过GObject-Introspection或绑定方式，轻松地使用多种编程语言来编写自己的 libnotify 显示消息，或者您可以简单地使用 bash。

以下示例显示一个简单的 “Hello world” 通知。

Bash

- 依赖: libnotify包包

```
hello_world.sh
#!/bin/bash
notify-send 'Hello world!' 'This is an example notification.' --icon=dialog-information
```

提示：

- An overview on the available icons can be found in the specification.
- To send desktop notification to another user, e.g. from a background script running as root:

```
# systemd-run --machine=user@.host --user notify-send 'Hello world!' 'This is an example notification.'
```

- Dependency: glib2包

```
hello_world.sh
#!/bin/bash
gdbus call --session \
    --dest=org.freedesktop.Notifications \
    --object-path=/org/freedesktop/Notifications \
    --method=org.freedesktop.Notifications.Notify \
    "" 0 "" 'Hello world!' 'This is an example notification.' \
    '[]' '{"urgency": <1>}' 5000
```

提示：

- See org.freedesktop.Notifications.Notify for more details about the D-Bus interface.
- `dbus-send`, though being similar to `gdbus`, does not work because one of the arguments in the method `org.freedesktop.Notifications.Notify` requires data type `variant`, which is not supported by `dbus-send`.

C

- Dependency: glib2包
- Build with: `` `gcc -o hello_world pkg-config --cflags --libs gio-2.0 hello_world.c` ``

```
hello_world.c
#include <gio/gio.h>
```

```
int main() {
	GApplication *application = g_application_new ("hello.world",
G_APPLICATION_FLAGS_NONE);
	g_application_register (application, NULL, NULL);
	GNotification *notification = g_notification_new ("Hello world!");
	g_notification_set_body (notification, "This is an example notification.");
	GIcon *icon = g_themed_icon_new ("dialog-information");
	g_notification_set_icon (notification, icon);
	g_application_send_notification (application, NULL, notification);
	g_object_unref (icon);
	g_object_unref (notification);
	g_object_unref (application);
	return 0;
}
```

- Dependency: libnotify包
- Build with: `gcc -o hello_world pkg-config --cflags --libs libnotify hello_world.c`

```
hello_world.c
#include <libnotify/notify.h>
int main() {
	notify_init ("Hello world!");
	NotifyNotification * Hello = notify_notification_new ("Hello world", "This is an
example notification.", "dialog-information");
	notify_notification_show (Hello, NULL);
	g_object_unref(G_OBJECT(Hello));
	notify_uninit();
	return 0;
}
```

- Dependency: dbus包
- Build with: `gcc -o hello_world $(pkg-config --cflags --libs dbus-1) hello_world.c`

```
hello_world.c
#include <dbus-1.0/dbus/dbus.h>
int main()
{
	DBusConnection *connection = dbus_bus_get(DBUS_BUS_SESSION, 0);
	DBusMessage *message = dbus_message_new_method_call(
		"org.freedesktop.Notifications",
		"/org/freedesktop/Notifications",
		"org.freedesktop.Notifications",
		"Notify");
	DBusMessageIter iter;
	dbus_message_iter_init_append(message, iter);
	char *application = "hello.world";
	dbus_message_iter_append_basic(iter, 's', &application);
	unsigned id = 0;
	dbus_message_iter_append_basic(iter, 'u', &id);
	char *icon = "dialog-information";
	dbus_message_iter_append_basic(iter, 's', &icon);
	char *summary = "Hello world!";
	dbus_message_iter_append_basic(iter, 's', &summary);
	char *body = "This is an example notification.";
	dbus_message_iter_append_basic(iter, 's', &body);
	dbus_message_iter_open_container(iter, 'a', "s", iter + 1);
	dbus_message_iter_close_container(iter, iter + 1);
	dbus_message_iter_open_container(iter, 'a', "{sv}", iter + 1);
	dbus_message_iter_open_container(iter + 1, 'e', 0, iter + 2);
	char *urgency = "urgency";
	dbus_message_iter_append_basic(iter + 2, 's', &urgency);
	dbus_message_iter_open_container(iter + 2, 'v', "y", iter + 3);
	enum {LOW, NORMAL, CRITICAL};
	unsigned char level = LOW;
	dbus_message_iter_append_basic(iter + 3, 'y', &level);
	dbus_message_iter_close_container(iter + 2, iter + 3);
	dbus_message_iter_close_container(iter + 1, iter + 2);
```

```
        dbus_message_iter_close_container(iter, iter + 1);
        int timeout = 5 * 1000;
        dbus_message_iter_append_basic(iter, 'i', &timeout);
        dbus_connection_send(connection, message, 0);
        dbus_connection_flush(connection);
        dbus_message_unref(message);
        dbus_connection_unref(connection);
}
```

C++

- Dependency: glibmm包
- Build with: `g++ -o hello_world pkg-config --cflags --libs giomm-2.4 hello_world.cc`

```
hello_world.cc
#include <giomm-2.4/giomm.h>
int main(int argc, char *argv[]) {
        auto Application = Gio::Application::create("hello.world",
Gio::APPLICATION_FLAGS_NONE);
        Application->register_application();
        auto Notification = Gio::Notification::create("Hello world");
        Notification->set_body("This is an example notification.");
        auto Icon = Gio::ThemedIcon::create("dialog-information");
        Notification->set_icon (Icon);
        Application->send_notification(Notification);
        return 0;
}
```

- Dependency: libnotifymmAUR\[损坏的链接：package not found\]
- Build with: `g++ -o hello_world pkg-config --cflags --libs libnotifymm-1.0 hello_world.cc`

```
hello_world.cc
#include <libnotifymm.h>
int main(int argc, char *argv[]) {
        Notify::init("Hello world!");
        Notify::Notification Hello("Hello world", "This is an example notification.",
"dialog-information");
        Hello.show();
        return 0;
}
```

C#

- Dependency: notify-sharp-3AUR
- Build with: `mcs -pkg:notify-sharp-3.0 hello_world.cs`
- Run with: `mono hello_world.exe`

```
hello_world.cs
using Notifications;
public class HelloWorld {
        static void Main() {
                var Hello = new Notification();
                Hello.Summary  = "Hello world!";
                Hello.Body     = "This is an example notification.";
                Hello.IconName = "dialog-information";
                Hello.Show();
        }
}
```

Common Lisp

- Dependency: dbus包, death/dbus (from quicklisp)

```
hello_world.lisp
(defun hello ()
```

```
  "send notification with dbus"
  (let ((app_name "hello.world")
        (replaces_id 0)
        (app_icon "dialog-information")
        (summary "hello.world")
        (body "Hello world!")
        (expire_timeout -1))
    (dbus:with-open-bus (bus (dbus:session-server-addresses))
      (dbus:with-introspected-object
          (notif bus "/org/freedesktop/Notifications" "org.freedesktop.Notifications")
        (notif "org.freedesktop.Notifications" "Notify"
               app_name replaces_id app_icon summary body '() '() expire_timeout)))))

(hello)
```

Crystal

- Dependency: woodruffw/notify.cr (from shards)
- Build with: shards build

```
hello_world.fs
require "notify"
notifier = Notify.new
notifier.notify "Hello", body: "World!"
```

F#

- Dependency: notify-sharp-3AUR
- Makedependency: dotnet-sdk包
- Build with: `fsharpc -r:notify-sharp.dll -I:/usr/lib/mono/notify-sharp-3.0/ -I:/usr/lib/mono/gtk-sharp-3.0/ hello_world.fs`
- Run with: `mono hello_world.exe`

```
hello_world.fs
open Notifications
let Hello = new Notification()
Hello.Summary  <- "Hello world!"
Hello.Body     <- "This is an example notification."
Hello.IconName <- "dialog-information"
Hello.Show()
```

Genie

- Dependency: glib2包
- Makedependency: vala包
- Build with: `valac --pkg gio-2.0 hello_world.gs`

```
hello_world.gs
uses
       GLib

init
       var Application = new GLib.Application ("hello.world",
GLib.ApplicationFlags.FLAGS_NONE);
       Application.register ();
       var Notification = new GLib.Notification ("Hello world");
       Notification.set_body ("This is an example notification.");
       var Icon = new GLib.ThemedIcon ("dialog-information");
       Notification.set_icon (Icon);
       Application.send_notification (null, Notification);
```

- Dependency: libnotify包
- Makedependency: vala包

- Build with: `valac --pkg libnotify hello_world.gs`

```
hello_world.gs
uses
	Notify

init
	Notify.init ("Hello world")
	var Hello=new Notify.Notification ("Hello world!","This is an example
notification.","dialog-information")
	Hello.show ()
```

Go

- Dependency: libnotify包
- Makedependency: go-notify-gitAUR\[损坏的链接：package not found\]
- Build with: `go build hello_world.go`
- (Or run with: `go run hello_world.go`)

```
hello_world.go
package main
import ("github.com/mqu/go-notify")

func main() {
	notify.Init("Hello world")
	hello := notify.NotificationNew("Hello World!", "This is an example
notification.","dialog-information")
	hello.Show()
}
```

Groovy

- Dependencies: groovy包, java-gnomeAUR
- Build with: `groovyc -cp /usr/share/java/gtk.jar HelloWorld.groovy && jar cfe HelloWorld.jar HelloWorld HelloWorld.class`
- Run with: `java -cp /usr/share/groovy/embeddable/groovy-all.jar:/usr/share/java/gtk.jar:HelloWorld.jar HelloWorld` or `groovy -cp /usr/share/java/gtk.jar HelloWorld.groovy`

```
HelloWorld.groovy
import org.gnome.gtk.*
import org.gnome.notify.*

Gtk.init()
Notify.init("Hello world")
def Hello = new Notification("Hello world!", "This is an example notification.",
"dialog-information")
Hello.show()
```

Haskell

- Makedependency: haskell-fdo-notify包
- Build with: `ghc -dynamic hello_world`

```
hello_world.hs
import DBus.Notify
main = do
	client <- connectSession
	let hello = blankNote { summary="Hello world!",
	                        body=(Just $ Text "This is an example notification."),
	                        appImage=(Just $ Icon "dialog-information") }
```

```
        notification <- notify client hello
        return 0
```

IronPython

- Dependencies: notify-sharp-3AUR, ironpythonAUR
- Run with: `ipy hello_world.py`

```
hello_world.py
import clr
clr.AddReference('notify-sharp')
import Notifications
Hello = Notifications.Notification()
Hello.Summary  = "Hello world!"
Hello.Body     = "This is an example notification."
Hello.IconName = "dialog-information"
Hello.Show()
```

Java

- Dependency: java-gnomeAUR
- Makedependency: java-environment
- Build with: `javac -cp /usr/share/java/gtk.jar HelloWorld.java && jar cfe HelloWorld.jar HelloWorld HelloWorld.class`
- Run with: `java -cp /usr/share/java/gtk.jar:HelloWorld.jar HelloWorld`

```
HelloWorld.java
import org.gnome.gtk.Gtk;
import org.gnome.notify.Notify;
import org.gnome.notify.Notification;

public class HelloWorld
{
    public static void main(String[] args) {
        Gtk.init(args);
        Notify.init("Hello world");
        Notification Hello = new Notification("Hello world!", "This is an example
notification.", "dialog-information");
        Hello.show();
    }
}
```

JavaScript

- Dependency: gjs包

```
hello_world.js
#!/usr/bin/gjs
const Gio = imports.gi.Gio;
var Application = new Gio.Application ({application_id: "hello.world"});
Application.register (null);
var Notification = new Gio.Notification ();
Notification.set_title ("Hello world");
Notification.set_body ("This is an example notification.");
var Icon = new Gio.ThemedIcon ({name: "dialog-information"});
Notification.set_icon (Icon);
Application.send_notification (null, Notification);
```

- Dependencies: libnotify包, gjs包

```
hello_world.js
#!/usr/bin/gjs
const Notify = imports.gi.Notify;
Notify.init ("Hello world");
var Hello=new Notify.Notification ({summary: "Hello world!",
                                    body: "This is an example notification.",
```

```
                                    "icon-name": "dialog-information"});
Hello.show ();
```

- Dependencies: nodejs包, node-dbus-notifier

```
hello_world.js
#!/usr/bin/env node
const notifier = require("node-dbus-notifier");

const notify = new notifier.Notify({
  appName: "JavaScript",
  appIcon: "archlinux-logo",
  summary: "Hello world!",
  hints: {
    urgency: 2,
  },
  body: "This is an example notification.",
  timeout: 5000,
});

notify.addAction("Click Me", () => {
  console.log("Click Click");
});

notify.show()
  .then(() => {
    console.log("notify is closed");
  });
```

JRuby

- Dependencies: java-gnomeAUR, jruby包
- Build with: `jrubyc hello_world.rb && jar cfe hello_world.jar hello_world hello_world.class`
- Run with: `java -cp /opt/jruby/lib/jruby.jar:hello_world.jar hello_world` or `jruby hello_world.rb`

```
hello_world.rb
require '/usr/share/java/gtk.jar'
import Java::OrgGnomeGtk::Gtk
import Java::OrgGnomeNotify::Notify
import Java::OrgGnomeNotify::Notification

Gtk.init(nil)
Notify.init("Hello world")
Hello = Notification.new("Hello world!", "This is an example notification.", "dialog-
information")
Hello.show
```

Jython

- Dependencies: java-gnomeAUR, jython包
- Run with: `jython -Dpython.path=/usr/share/java/gtk.jar hello_world.py`

```
hello_world.py
from org.gnome.gtk import Gtk
from org.gnome.notify import Notify, Notification
Gtk.init(None)
Notify.init("Hello world")
Hello=Notification("Hello world!", "This is an example notification.", "dialog-
information")
Hello.show()
```

Lua

- Dependency: lua-lgi-gitAUR

```
hello_world.lua
#!/usr/bin/lua
lgi = require 'lgi'
Gio = lgi.require('Gio')
Application = Gio.Application.new("hello.world",Gio.ApplicationFlags.FLAGS_NONE);
Application:register();
Notification = Gio.Notification.new("Hello world");
Notification:set_body("This is an example notification.");
Icon = Gio.ThemedIcon.new("dialog-information");
Notification:set_icon(Icon);
Application:send_notification(nil, Notification);
```

- Dependencies: libnotify包, lua-lgi-gitAUR

```
hello_world.lua
#!/usr/bin/lua
lgi = require 'lgi'
Notify = lgi.require('Notify')
Notify.init("Hello world")
Hello=Notify.Notification.new("Hello world","This is an example notification.","dialog-
information")
Hello:show()
```

Nemerle

- Dependency: notify-sharp-3AUR
- Makedependency: nemerleAUR
- Build with: `ncc -pkg:notify-sharp-3.0 -out:hello_world.exe hello_world.n`
- Run with: `mono hello_world.exe`

```
hello_world.n
using Notifications;
public class HelloWorld {
	static Main() : void {
		def Hello = Notification();
		Hello.Summary  = "Hello world!";
		Hello.Body     = "This is an example notification.";
		Hello.IconName = "dialog-information";
		Hello.Show();
	}
}
```

Pascal

- Dependency: libnotify包
- Makedependency: fpc包, libnotify binding
- Build with: `fpc hello_world`

```
hello_world.pas
program	hello_world;
uses	libnotify;
var	hello : PNotifyNotification;
begin
	notify_init(argv);
	hello := notify_notification_new ('Hello world', 'This is an example
notification.', 'dialog-information');
	notify_notification_show (hello, nil);
end.
```

Perl

Using libnotify

- Dependencies: libnotify包, perl-glib-object-introspection包

```
hello_world.pl
#!/usr/bin/perl
use Glib::Object::Introspection;
Glib::Object::Introspection->setup(basename => 'Notify',
                                   version => '0.7',
                                   package => 'Notify');
Notify->init;
my $hello = Notify::Notification->new('Hello world!',
                                      'This is an example notification.',
                                      'dialog-information');
$hello->show;
```

Using direct D-Bus calls

- Dependencies: perl-net-dbus包

```
hello_world.pl
#!/usr/bin/perl
use Net::DBus;
my $bus = Net::DBus->session;
my $svc = $bus->get_service('org.freedesktop.Notifications');
my $obj = $svc->get_object('/org/freedesktop/Notifications');
my $id = $obj->Notify('myapp', 0,
                      'dialog-information',
                      'Hello world!',
                      'This is an example notification.',
                      [], {}, 0);
```

Python

- Dependency: python-gobject包

```
hello_world.py
#!/usr/bin/python3
import gi
gi.require_version('Gio', '2.0')
from gi.repository import Gio
Application = Gio.Application.new("hello.world", Gio.ApplicationFlags.FLAGS_NONE)
Application.register()
Notification = Gio.Notification.new("Hello world")
Notification.set_body("This is an example notification.")
Icon = Gio.ThemedIcon.new("dialog-information")
Notification.set_icon(Icon)
Application.send_notification(None, Notification)
```

- Dependencies: libnotify包, python-gobject包

```
hello_world.py
#!/usr/bin/python3
import gi
gi.require_version('Notify', '0.7')
from gi.repository import Notify
Notify.init("Hello world")
Hello = Notify.Notification.new("Hello world", "This is an example notification.",
"dialog-information")
Hello.show()
```

- Dependency: python-dbus包

```
hello_world.py
#!/usr/bin/env python3
import dbus
```

```
obj = dbus.SessionBus().get_object("org.freedesktop.Notifications",
"/org/freedesktop/Notifications")
obj = dbus.Interface(obj, "org.freedesktop.Notifications")
obj.Notify("", 0, "", "Hello world", "This is an example notification.", [],
{"urgency": 1}, 10000)
```

For the arguments in the method `Notify`, please refer to the section of org.freedesktop.Notifications.Notify at Desktop Notifications Specification.

Ruby

- Dependencies: libnotify包, ruby-gir_ffiAUR\[损坏的链接：package not found\]

```
hello_world.rb
#!/usr/bin/ruby
require 'gir_ffi'
GirFFI.setup :Notify
Notify.init("Hello world")
Hello = Notify::Notification.new("Hello world!", "This is an example notification.",
"dialog-information")
Hello.show
```

Rust

Using notify-rust.

- Makedependency: rust包
- Build with: `cargo build`
- Run with: `target/debug/hello_world` or `cargo run`

```
Cargo.toml
[package]
name = "hello_world"
version = "0.1.0"

[dependencies]
notify-rust = "^3"
src/main.rs
extern crate notify_rust;
use notify_rust::Notification;
fn main(){
    Notification::new()
        .summary("Hello world")
        .body("This is an example notification.")
        .icon("dialog-information")
        .show().unwrap();
}
```

Scala

- Dependency: java-gnomeAUR (and scalaAUR)
- Makedependency: scalaAUR
- Build with: `scalac -cp /usr/share/java/gtk.jar -d HelloWorld.jar HelloWorld.scala`
- Run with: `java -cp /usr/share/java/gtk.jar:HelloWorld.jar HelloWorld` (or `scala -cp /usr/share/java/gtk.jar HelloWorld.scala`)

```
HelloWorld.scala
import org.gnome.gtk._
import org.gnome.notify._

object HelloWorld {
  def main(args: Array[String]) {
```

```
    Gtk.init(args)
    Notify.init("Hello world")
    var Hello = new Notification("Hello world!", "This is an example notification.",
"dialog-information")
    Hello.show()
  }
}
```

Vala

- Dependency: glib2包
- Makedependency: vala包
- Build with: `valac --pkg gio-2.0 hello_world.vala`

```
hello_world.vala
using GLib;
public class HelloWorld {
	static void main () {
		var Application = new GLib.Application ("hello.world",
GLib.ApplicationFlags.FLAGS_NONE);
		Application.register ();
		var Notification = new GLib.Notification ("Hello world");
		Notification.set_body ("This is an example notification.");
		var Icon = new GLib.ThemedIcon ("dialog-information");
		Notification.set_icon (Icon);
		Application.send_notification (null, Notification);
	}
}
```

- Dependency: libnotify包
- Makedependency: vala包
- Build with: `valac --pkg libnotify hello_world.vala`

```
hello_world.vala
using Notify;
public class HelloWorld {
	static void main () {
		Notify.init ("Hello world");
		var Hello = new Notify.Notification("Hello world!", "This is an example
notification.", "dialog-information");
		Hello.show ();
	}
}
```

Visual Basic .NET

- Dependency: notify-sharp-3AUR
- Makedependency: mono-basicAUR
- Build with: `vbnc -r:/usr/lib/mono/notify-sharp-3.0/notify-sharp.dll hello_world.vb`
- Run with: `mono hello_world.exe`

```
hello_world.vb
Imports Notifications
Public Class Hello
	Public Shared Sub Main
		Dim Hello As New Notification
		Hello.Summary  = "Hello world!"
		Hello.Body     = "This is an example notification."
		Hello.IconName = "dialog-information"
		Hello.Show
	End Sub
End Class
```

提示和技巧

替换之前的通知

如果通知的ID已知，则可以替换通知；如果新的通知请求指定了相同的 ID，它将始终替换旧通知。(上面显示的 libnotify 绑定会自动处理此问题。)遗憾的是，notify-send 不会报告此 ID，因此需要替代工具才能在 CLI 上执行此操作。一个功能强大的 CLI 工具是 notify-send.py脚本，它提供 notify-send 语法以及额外的 ID 报告和替换功能。

但是，对于某些通知服务器（例如 Notify-OSD），您可以把`string:x-canonical-private-synchronous:`的提示与 notify-send结合使用，以达到相同的结果。

例如，要获取通知的显示时间：

```
while true; do
  date=$(date)
  notify-send "$date" -h string:x-canonical-private-synchronous:my-notification
  sleep 1
done
```

包含按钮或侦听通知的关闭/单击事件

使用notify-send.py脚本，有显示按钮操作或监听通知的默认操作（通常是在用户单击通知时）和关闭操作。当 action-icons 提示设置为 true 并且通知守护程序支持此功能时，按钮将显示图标而不是文本。当相应的事件发生时，脚本会打印操作标识符或命令行的 “close”要监听默认操作 （on-click），必须使用 action-identfier “default”。

按钮上有图标的示例：

```
notify-send.py "Buttons" "Do you like em?" --hint boolean:action-icons:true --action
yes:face-cool no:face-sick
```

具有D-Bus服务的多个通知服务器

如#独立通知部分所述，用户可以创建D-Bus服务，以便可以自动启动一个通知服务器。一些实现已经包含D-Bus服务文件。但是，当安装了多个通知服务器并且其中一些附带了D-Bus服务文件时，会导致问题。例如，同时安装了dunst包和 mako包，在没有明确指定服务器时，D-Bus会为用户选择一个，此选择不受用户控制。为避免这种情况，您可以通过创建`org.freedesktop.Notifications.service`文件来覆盖服务 （请参阅 #独立通知）并指向要使用的服务，然后重新启动会话。

Systemd services失败的通知

要获得失败服务的通知，请在目标用户上安装并运行 systembus-notify包，并按照 systemd#通知失败的单元中给出的说明进行操作，但将`failure-notification@`模板单元替换为下面的这个：

```
/etc/systemd/system/failure-notification@.service
```

```
[Unit]
Description=Send a notification about a failed systemd unit
After=Graphical.target

[Service]
Type=simple
ExecStart=/usr/bin/dbus-send --system / net.nuetzlich.SystemNotifications.Notify
'string:%i' 'string:Unit failed'
```

遗憾的是，此解决方案尚不支持设置通知的紧急性或超时，请参阅此问题。

故障排除

应用程序挂起整整一分钟

如果应用程序在尝试显示通知时挂起，可能是因为D-Bus服务错误地发布了通知服务的其可用性。 例如，假设一个用户最近安装了一个需要plasma-workspace包的 KDE 组件，但该用户仍在运行XFCE。在这种情况下，KDE通知器将被优先考虑，但用户没有运行它。应用程序将在等待服务时挂起，只有在超时后才会回退到xfce4-notifyd包。

最明显的挂起可能是来自音量指示器滚动调整。

如果您处于这种情况，您应该有两个通知处理程序：

```
$ find /usr/share/dbus-1/services/ -name '*Notif*'
org.kde.plasma.Notifications.service
org.xfce.xfce4-notifyd.Notifications.service
```

在这两个应用程序中，一个在 1 分钟超时后经常失败，如journal日志中所示：

```
# journalctl -g notif
Jul 01 09:40:49 laptop dbus-daemon: [session uid=1000 pid=866] Activating service
name='org.freedesktop.Notifications' requested by ':1.193' (uid=1000 pid=5432
comm="/usr/lib/xfce4/panel/wrapper-2.0 /usr/lib/xfce4/pa")
Jul 01 09:41:49 laptop plasma_waitforname: org.kde.knotifications: WaitForName: Service
was not registered within timeout
Jul 01 09:41:49 laptop dbus-daemon: [session uid=1000 pid=866] Activated service
'org.freedesktop.Notifications' failed: Process org.freedesktop.Notifications exited
with status 1
```

按照 #具有D-Bus服务的多个通知服务器中所述选择要使用的服务来解决问题。

另请参阅

- Libnotify Reference Manual\[失效链接 2024-07-30 ⓘ\]
- C example (archived version)
- Python notification examples
- Python notify example (french article)